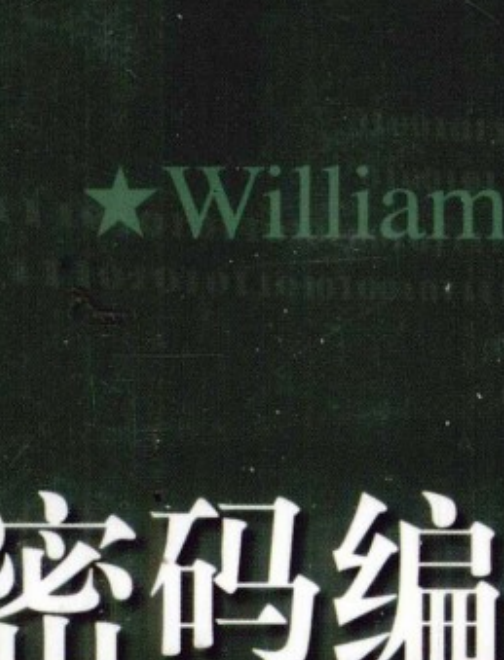
国 外 计 算 机 科 学 教 材 系 列

PEARSON

★William Stallings

密码编码学与网络安全
——原理与实践（第五版）

Cryptography and Network Security
Principles and Practice, Fifth Edition



[美] William Stallings 著

王张宜 杨 敏 杜瑞颖 等译

张焕国 审校

電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY

http://www.phei.com.cn

权威经典

密码编码学与网络安全
——原理与实践（第五版）

Cryptography and Network Security
Principles and Practice, Fifth Edition

在全球实现了电子化连接，充满病毒、黑客、电子窃听、电子欺诈的年代，安全是一个极其重要的主题。本书针对密码编码学和网络安全，从原理和实践两方面提供了实用的知识。

本书适合用做密码编码学、计算机安全、网络安全专业的本科生或研究生一学期课程的教材。在讲解密码编码学和网络安全的实用知识时，本书还为教师和学生提供了大量的支持材料。

第五版的特点

- 涵盖最新的主题，扩充了分组密码模型的内容，包含认证加密
- 优化并扩展了AES的讲解
- 扩展了伪随机数发生器的内容
- 新增联合身份（federated identity）、HTTPS、SSH以及无线网络安全的内容
- 全面重写并更新了IPsec
- 新增关于法律和伦理的一章
- 使用Sage计算机代数系统演示密码编码学算法
- 关于密码编码学算法的全面比较
- 对认证和数字签名的完整比较
- 统一、全面地论述了相互信任的主题，比如密钥管理和用户认证
- 针对电子邮件安全同时介绍了PGP和S/MIME

访问本书的配套网站http://williamstallings.com/Crypto/Crypto5e.html，可获得学生和教师资源，包括测试库（Testbank）、PowerPoint教案、教师用习题解答手册、教师用项目手册、书中的图和表、Java样本程序、实验室练习模板、附加的PowerPoint教案、勘误表、安全与密码编码学论坛、密码编码学演示、各章的链接、在线内容（包括在线章节和附录、重要的页面、支持文档、Sage代码例子等）。

为使本书的中文版读者能读到原书的完整内容，特地翻译了原书的在线内容〔第20章至第23章，以及附录C至附录Q。这些内容的中文版已上载至华信教育资源网(http: //www.hxedu.com.cn)，有兴趣的读者可免费下载〕，从而为中文读者提供了一本完整的中文图书。

PEARSON

PEARSON
www.pearson.com

ISBN 978-7-121-15250-4
9 787121 152504 >

定价：63.00 元

策划编辑：谭海平
责任编辑：李秦华
责任美编：孙焱津

华信 欢迎登录 免费 获取本书教学资源
http://www.hxedu.com.cn

本书贴有激光防伪标志，凡没有防伪标志者，属盗版图书。

国外计算机科学教材系列

密码编码学与网络安全

——原理与实践(第五版)

Cryptography and Network Security

Principles and Practice, Fifth Edition

[美] William Stallings 著

王张宜 杨 敏 杜瑞颖 等译

张焕国 审校

電子工業出版社

Publishing House of Electronics Industry

北京 · BEIJING

内 容 简 介

本书系统介绍了密码编码学与网络安全的基本原理和应用技术。全书主要包括以下七个部分：对称密码部分讨论了对称加密的算法和设计原则；公钥密码部分讨论了公钥密码的算法和设计原则；密码学中的数据完整性算法部分讨论了密码学 Hash 函数、消息验证码和数字签名；相互信任部分讨论了密钥管理和认证技术；网络与因特网安全部分讨论了应用密码算法和安全协议为网络和 Internet 提供安全；法律与道德问题部分讨论了与计算机和网络安全相关的法律与道德问题。本书的第五版与第四版相比，书中的内容和组织结构都做了较大的调整，增加了许多新内容，并首次采用了在线内容和使用 Sage 计算机代数系统。

本书可作为研究生和高年级本科生的教材，也可以从事信息安全、计算机、通信、电子工程等领域的科技人员参考。

Authorized translation from the English language edition, entitled Cryptography and Network Security: Principles and Practice, Fifth Edition, 9780136097044 by William Stallings, published by Pearson Education, Inc., publishing as Prentice Hall, Copyright©2011 by Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD., and PUBLISHING HOUSE OF ELECTRONICS INDUSTRY Copyright©2012.

本书中文简体字版专有出版权由 Pearson Education（培生教育出版集团）授予电子工业出版社。未经出版者预先书面许可，不得以任何方式复制或抄袭本书的任何部分。

本书贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

版权贸易合同登记号　图字：01-2011-6684

图书在版编目（CIP）数据

密码编码学与网络安全：原理与实践：第 5 版/（美）斯托林斯（Stallings，W.）著；王张宜等译.
北京：电子工业出版社，2012.1
国外计算机科学教材系列
书名原文：Cryptography and Network Security：Principles and Practice，Fifth Edition
ISBN 978-7-121-15250-4

I. ①密… II. ①斯… ②王… III. ①电子计算机-密码术-高等学校-教材 ②计算机网络-安全技术-高等学校-教材 IV. ①TP309.7 ②TP393.08

中国版本图书馆 CIP 数据核字（2011）第 241303 号

策划编辑：谭海平
责任编辑：李秦华
印　　刷：涿州市京南印刷厂
装　　订：涿州市桃园装订有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路 173 信箱　邮编　100036
开　　本：787×1092　1/16　印张：33.75　字数：1045 千字
印　　次：2012 年 1 月第 1 次印刷
定　　价：63.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

服务热线：（010）88258888。

出 版 说 明

21世纪初的5至10年是我国国民经济和社会发展的重要时期，也是信息产业快速发展的关键时期。在我国加入WTO后的今天，培养一支适应国际化竞争的一流IT人才队伍是我国高等教育的重要任务之一。信息科学和技术方面人才的优劣与多寡，是我国面对国际竞争时成败的关键因素。

当前，正值我国高等教育特别是信息科学领域的教育调整、变革的重大时期，为使我国教育体制与国际化接轨，有条件的高等院校正在为某些信息学科和技术课程使用国外优秀教材和优秀原版教材，以使我国在计算机教学上尽快赶上国际先进水平。

电子工业出版社秉承多年来引进国外优秀图书的经验，翻译出版了“国外计算机科学教材系列”丛书，这套教材覆盖学科范围广、领域宽、层次多，既有本科专业课程教材，也有研究生课程教材，以适应不同院系、不同专业、不同层次的师生对教材的需求，广大师生可自由选择和自由组合使用。这些教材涉及的学科方向包括网络与通信、操作系统、计算机组织与结构、算法与数据结构、数据库与信息处理、编程语言、图形图像与多媒体、软件工程等。同时，我们也适当引进了一些优秀英文原版教材，本着翻译版本和英文原版并重的原则，对重点图书既提供英文原版又提供相应的翻译版本。

在图书选题上，我们大都选择国外著名出版公司出版的高校教材，如Pearson Education培生教育出版集团、麦格劳－希尔教育出版集团、麻省理工学院出版社、剑桥大学出版社等。撰写教材的许多作者都是蜚声世界的教授、学者，如道格拉斯·科默（Douglas E. Comer）、威廉·斯托林斯（William Stallings）、哈维·戴特尔（Harvey M. Deitel）、尤利斯·布莱克（Uyless Black）等。

为确保教材的选题质量和翻译质量，我们约请了清华大学、北京大学、北京航空航天大学、复旦大学、上海交通大学、南京大学、浙江大学、哈尔滨工业大学、华中科技大学、西安交通大学、国防科学技术大学、解放军理工大学等著名高校的教授和骨干教师参与了本系列教材的选题、翻译和审校工作。他们中既有讲授同类教材的骨干教师、博士，也有积累了几十年教学经验的老教授和博士生导师。

在该系列教材的选题、翻译和编辑加工过程中，为提高教材质量，我们做了大量细致的工作，包括对所选教材进行全面论证；选择编辑时力求达到专业对口；对排版、印制质量进行严格把关。对于英文教材中出现的错误，我们通过与作者联络和网上下载勘误表等方式，逐一进行了修订。

此外，我们还将与国外著名出版公司合作，提供一些教材的教学支持资料，希望能为授课老师提供帮助。今后，我们将继续加强与各高校教师的密切联系，为广大师生引进更多的国外优秀教材和参考书，为我国计算机科学教学体系与国际教学体系的接轨做出努力。

电子工业出版社

教材出版委员会

主　任　杨芙清　北京大学教授
中国科学院院士
北京大学信息与工程学部主任
北京大学软件工程研究所所长

委　员　王　珊　中国人民大学信息学院院长、教授

胡道元　清华大学计算机科学与技术系教授
国际信息处理联合会通信系统中国代表

钟玉琢　清华大学计算机科学与技术系教授、博士生导师
清华大学深圳研究生院信息学部主任

谢希仁　中国人民解放军理工大学教授
全军网络技术研究中心主任、博士生导师

尤晋元　上海交通大学计算机科学与工程系教授
上海分布计算技术中心主任

施伯乐　上海国际数据库研究中心主任、复旦大学教授
中国计算机学会常务理事、上海市计算机学会理事长

邹　鹏　国防科学技术大学计算机学院教授、博士生导师
教育部计算机基础课程教学指导委员会副主任委员

张昆藏　青岛大学信息工程学院教授

译 者 序

随着信息科学技术的高速发展和广泛应用，社会逐步信息化。在信息化社会中，通信、计算机和消费电子的结合，产生了 Internet、信息高速公路或全球信息基础设施(GII)，构成了人类生存的信息环境，即信息空间(Cyberspace)。在信息空间中，计算机和网络在军事、金融、工业、商业、人们的生活和工作等方面的应用越来越广泛，社会对计算机和网络的依赖越来越大，如果计算机和网络系统的信息安全受到破坏将导致社会的混乱并造成巨大损失。

我们应当清楚，人类社会中的安全可信与信息空间中的安全可信是休戚相关的。对于人类生存来说，只有同时解决了人类社会和信息空间的安全可信，才能保证人类社会的安全、和谐、繁荣和进步。

因此，确保信息空间、计算机和网络系统的信息安全成为世人关注的社会问题，并成为信息科学技术领域中的研究热点。

发展我国信息安全技术与产业的关键是人才，而培养人才的关键是教育。目前，我国许多大专院校都开设了信息安全专业或开设了信息安全课程，迫切需要一本合适的教科书。为此，电子工业出版社组织我们于 2006 年翻译出版了《密码编码学与网络安全——原理与实践(第四版)》这本优秀的教科书。这本书翻译出版后得到了广大读者的厚爱，许多著名大学都采用它作为教材，为我国信息安全人才培养和传播信息安全知识发挥了重要作用。

2010 年原书作者又出版了该书的第五版。在第五版中，作者对原书的内容和组织结构都做了较大的调整和更新。

1. 在书的组织结构方面做了如下调整：
   ① 在密码学方面增加了第三部分：密码学数据完整性算法。专门讨论密码算法中涉及数据完整性的内容，包括密码学 Hash 函数，消息认证码和数字签名。
   ② 增加了第四部分：相互信任。集中讨论了信息系统中的实体相互信任问题，包括密钥管理和用户认证。
   ③ 首次采用了在线内容，包括在线章和在线附录。将第六部分：系统安全、第七部分：法律与道德、附录 C 至附录 Q 放到网站上，读者可以上网阅读学习①。这样可以节约书的篇幅，降低书的成本。
   ④ 将伪随机数产生与序列密码集成为独立的一章。类似单独列章的还有传输层安全等。在以前的版本中，这些相关内容分散在各章中。这样将相关联的内容集成为一章，便于读者学习掌握。
2. 在内容方面进行了许多修改，并增加了一些新内容。如对于欧几里得算法、AES、分组密码工作模式、伪随机数、Hash 函数和消息认证码、密钥管理和分配、远程用户认证、IPsec 等内容进行了修改，并新增了 ElGamal 加密和数字签名、SHA-3、认证加密、联合身份认证、HTTPS、安全框架 SSH、域密钥身份认证邮件 DKIM、无线网络安全、法律与道德、在线附录、Sage 示例与问题等方面的新内容。

---

① 为使本书的中文版读者能读到原书的完整内容，特地翻译了原书的在线内容[第 20 章至第 23 章，以及附录 C 至附录 Q。这些内容的中文版已上载至华信教育资源网(http://www.hxedu.com.cn)，有兴趣的读者可免费下载]，从而为中文读者提供了一本完整的中文图书——编者注。

3. 作为对本书内容的补充，增加了 15 个附录。这些附录为感兴趣的读者提供了更深入、更广泛的补充材料。这是以前的版本中所没有的。

4. 首次使用开源免费的 Sage 计算机代数系统，使学生们能够亲手进行各种密码算法的实验。

为了使广大读者能够读到新版书，电子工业出版社又组织我们翻译出版了本书的第五版。

《密码编码学与网络安全——原理与实践》一书的作者 William Stallings 先后获得了 Notre Dame 电气工程学士学位和 MIT 计算机科学博士学位。他累计编写出版了 48 本计算机网络和计算机体系结构领域的书籍，在计算机网络和计算机体系结构的学术交流和教育方面做出了卓越的贡献。其中《密码编码学与网络安全——原理与实践》就是其中最成功的一本书籍。William Stallings 的著作不仅学术造诣很高，而且十分实用，先后 11 次获得美国教材和著作家协会（Textbook and Academic Authors Assiciation）颁发的优秀计算机科学教材奖。

本书系统地介绍了密码学与网络安全的基本原理和应用技术。全书主要包含以下七个部分。第一部分：对称密码，介绍了古典和现代对称密码算法，重点介绍数据加密标准（DES）和高级加密标准（AES）。此外，还讨论了伪随机数和流密码。第二部分：非对称密码，给出了数论基础、RSA 密码、椭圆曲线密码和其他公钥密码。第三部分：密码学中的数据完整性算法，介绍了密码学 Hash 函数、消息认证码和数字签名。第四部分：相互信任，介绍了密钥管理和密钥分配，以及用户认证协议。第五部分：网络与因特网安全，讨论了传输层安全、无线网络安全、E-mail 安全和 IP 安全等内容。第六部分：系统安全，讨论了非法入侵、恶意软件和防火墙技术。第七部分：法律与道德，讨论了与计算机和网络安全相关的法律和道德问题。

《密码编码学与网络安全——原理与实践（第五版）》一书内容丰富，讲述深入浅出，便于理解，尤其适合于课堂教学和自学，是一本难得的好书。本书可作为研究生和高年级本科生的教材，也可供从事信息安全、计算机、通信、电子工程等领域的科技人员参考。

本书的第一部分由杨敏和孟庆树翻译，第二部分由王后珍翻译，前言和第三部分由王张宜翻译，第四部分由陈晶翻译，第五部分由杜瑞颖翻译，第六部分和第七部分由彭国军翻译。

本书的附录 C、E、F、G、H、I 由杨敏翻译，附录 A、B、D、K、N、M 由王张宜翻译，附录 J 由王后珍翻译，附录 L 由陈晶翻译，附录 O、P、Q 由杜瑞颖翻译。全书由张焕国统稿和审校。研究生陈新姣、王丹、梁玉、郑美凤及叶青晟等参与了翻译书稿的整理工作。

由于译者的专业知识和外语水平有限，书中错误在所难免，敬请读者指正，译者在此先致感谢之意。

译者于武汉大学珞珈山

2011 年 8 月

符　　号

符　　号	表　达　式	意　　义
D,K	D(K,Y)	用密钥 K 和对称算法解密密文 Y
D,PR_a	D(PR_a,Y)	用 A 的私钥 PR_a 和非对称算法解密密文 Y
D,PU_a	D(PU_a,Y)	用 A 的公钥 PU_a 和非对称算法解密密文 Y
E,K	E(K,X)	用密钥 K 和对称算法加密明文 X
E,PR_a	E(PR_a,X)	用 A 的私钥 PR_a 和非对称算法加密明文 X
E,PU_a	E(PU_a,X)	用 A 的公钥 PU_a 和非对称算法加密明文 X
K		密钥
PR_a		用户 A 的私钥
PU_a		用户 A 的公钥
MAC,K	MAC(K,X)	消息 X 的消息认证码,密钥为 K
GF(p)		阶为 p 的有限域,p 为素数。域定义为 Z_p 及其上的模 p 算术运算
GF(2^n)		阶为 2^n 的有限域
Z_n		小于 n 的非负整数集合
gcd	gcd(i,j)	最大公因子,整除 i 和 j 的最大正整数
mod	$a \bmod m$	a 除以 m 的余数
mod,≡	$a \equiv b \pmod m$	$a \bmod m = b \bmod m$
mod,≠	$a \neq b \pmod m$	$a \bmod m \neq b \bmod m$
dlog	$\mathrm{dlog}_{a,p}(b)$	以 a 为底 b 的对数,模 p 运算
ϕ	$\phi(n)$	欧拉函数,小于 n 且和 n 互素的正整数个数
Σ	$\sum_{i=1}^{n} a_i$	$a_1 + a_2 + \cdots + a_n$
Π	$\prod_{i=1}^{n} a_i$	$a_1 \times a_2 \times \cdots \times a_n$
|	$i \mid j$	i 整除得尽 j,即 i 除 j 的余数为零
|,|	$\lvert a \rvert$	a 的绝对值
‖	$x \parallel y$	级联 x 和 y
≈	$x \approx y$	x 约等于 y
⊕	$x \oplus y$	单位变量时是异或运算,多位变量时是按位异或
⌊,⌋	$\lfloor x \rfloor$	小于等于 x 的最大整数
∈	$x \in S$	元素 x 包含于集合 S
↔	$A \leftrightarrow (a_1, a_2, \cdots, a_k)$	整数 A 和整数序列(a_1,a_2,…,a_k)对应

前　言

在当前全球电子互联互通的时代，由于病毒、黑客、电子窃听和电子欺诈，使得安全性在任何时候都十分重要。第一，由于计算机系统的大量增加以及计算机系统通过网络互连，使得组织和个人越来越依赖于这些系统所存储和传输的信息。这也导致人们对如下的需求加深了认识：保护数据和资源不被泄露，保证数据和消息的真实性，保护系统不受基于网络的攻击。第二，密码和网络安全学科已经成熟，这样可开发出方便实用的应用软件来加强网络安全。由于这两种发展趋势，本书所讨论的内容就显得十分重要。

本书的目标

本书的目标是概述密码学与网络安全的原理和应用。本书的前两部分讨论密码学和网络安全技术，阐述网络安全的基本内容。其他部分讨论网络安全的实际应用，包括已经实现或正用于提供网络安全的实用应用软件。

因此本书涉及多个学科。特别地，要想理解本书讨论的某些技术的精髓，必须要有数论的基本知识和概率论中的某些结果。然而本书试图自成体系，不仅给出了必需的数论知识，而且让读者对这些知识有直观的理解。采用的方法是，在需要的时候才引入这些背景知识。这样有助于读者理解讨论这些知识的动机，作者认为这种方法比把所有的数学知识一次性全部放在本书开头要好。

本书适用对象

本书适合于学术和专业人员使用。作为教科书，本书可作为计算机科学、计算机工程、电气工程专业本科生密码编码学与网络安全方面课程的教材，学时为一学期。本书的内容包括了 IAS2 安全机制、NET4 安全和 IT311 里的材料（IAS2，NET4 是信息技术知识体系的两个核心领域，而 IT311 是密码学的高级教程）。这些材料都是 ACM/IEEE 计算机方向 2005 课程的讲授内容。

本书也可作为参考用书或作为自学教材。

本书的组织

本书由七个部分组成（概览全貌请见第 0 章）：

- 对称密码
- 公钥密码
- 数据完整性算法
- 相互信任
- 网络与因特网安全
- 系统安全
- 法律与道德

本书还针对教学的需要，提供了计算机代数系统 Sage 和大量图表使得表达更加清晰。每一章中都有关键术语表、课后习题、思考题、推荐读物和网站。本书还给出了术语表，常用的首字母缩略词表和参考文献。另外，对于教师还提供了试题库。

提供给学生的在线文档

对于本书的新版，我们提供了大量在线原始支持材料，包括如下类别：

- **在线章节**：为了减少本书（英文版）的篇幅和成本，书中的 4 个章节提供了 PDF 格式的电子文档，其中包括关于计算机安全的三章以及关于法律和道德的一章。在本书的目录中列出了这些章。
- **在线附录**：支持材料包含了本书正文中涉及的大量有趣的话题，但在本书（英文版）纸质印刷版中没有提供。我们为对此感兴趣的学生们提供了包含了这些话题的总计 15 个在线附录。在本书的目录中列出了这些附录。
- **[illegible]**：为了帮助学生更好地学习和理解本书内容，我们提供了[illegible]系列[illegible]庭作业和答案。这些能使学生们测试自己对于教材的掌握程度。
- **[illegible]**：从专业文献中选择了 24 篇论文，其中许多是很难找到的。提供给读者进一步地阅读。
- **支持文档**：本书引用的其他各种类型的有用文档同时在线提供。
- **Sage 代码**：附录 B 中给出了示例的 Sage 源代码。如果学生们想要实现这些示例，可以以此作为参考。

可提供读者 6 个月的在线材料访问权限。详见本书（英文版）扉页中的访问卡。[illegible]购买本书（英文版）

教学支持文档

对于教师，我们提供了下列材料：

- **答案手册**：对于每章末尾的思考题和习题的答案。
- **项目手册**：对于下面列出的所有项目的建议的任务分配方案。
- **PPT 幻灯片**：包含所有章节内容的幻灯片，适于讲课中使用。
- **PDF 文件**：本书中所有图和表的副本文档。
- **习题集**：按章的习题集。

在教师资源中心（IRC）中提供了所有的这些支持文档，可以通过 personhighered. com/stallings 或点击本书网站 WilliamStallings. com/Crypto/Crypto5e. html 中的“Book Info and More Instructor Resources”按钮访问 IRC。要想获取访问 IRC 的权限，请通过 pearsonhighered. com/educator/replocator/requestSalesRep. page 或者 Prentice Hall 的客服电话 1-800-526-0485 联系当地的 Prentice Hall 的经销商[①]。

教师和学生的 Internet 服务

本书的网站可给学生和教师提供支持，该网站链接一些相关的站点，并以 PDF（Adobe Acro-

① 为获取本书的教学支持文档，可参阅书后所附的《教辅申请表》——编者注。

bat)格式存储的本书中出现的图片、表格、幻灯片。该网站地址是 WilliamStallings. com/ Crypto/ Crypto5e. html。更多的信息请参阅本书第 0 章。

本书第五版在网上新提供了一系列的家庭作业和答案。学生们能够对这些题目进行求解并检查答案是否正确,这些题目能够加深学生们对于所学内容的理解。

我们已经建立了一个邮件列表,方便使用本书的各位教师以及与本书作者之间互相交换信息、建议和问题。若发现印刷或其他错误,则在 WilliamStallings. com 可找到本书的勘误表。另外计算机科学专业学生资源网(WilliamStallings. com/StudentSupport. html)为计算机专业学生和专业人员提供了有关的文档、信息和链接。

项目和其他学生练习

对许多教师来说,密码学或信息安全课程的一个重要组成部分就是制定一个或一系列项目使得学生有机会亲手实践,以加深对本书中所学知识的理解。本书在很大程度上对该课程的讲授计划提供支持,包含了课程中一整套的项目组件。教师资源中心不仅包含如何布置和安排项目,而且还包括一系列涵盖本书内容的推荐教学项目:

- **Sage 项目**:下一节中详细介绍。
- **黑客项目**:本项目的目的是阐明入侵检测和预防的关键问题。
- **分组密码项目**:本实验对 AES 加密算法的操作过程进行跟踪,手工进行一轮的计算,并使用不同的分组密码工作模式进行计算。实验也包括 DES 算法。每种情况下都由在线(或离线下载)Java 小程序来实现 AES 或 DES 的运算。
- **实验室练习**:就本书中的概念进行编程和做实验的系列项目。
- **研究项目**:一系列指导学生研究 Internet 有关课题以及撰写研究报告的课外研究课题。
- **编程项目**:一系列涵盖大部分课程内容且可在任何平台上用任何适当的语言实现的程序设计项目。
- **安全评估实践**:用于检验已有组织机构的现有架构和实现的一系列活动。
- **书面作业**:每一章里推荐了一些书面作业。
- **课外阅读/报告作业**:每一章在参考文献中都包含有论文列表,可让学生阅读并写出简短报告。具体细节请参见附录 A。

Sage 计算机代数系统

本书第五版新增的一项最重要的内容就是使用 Sage 实现密码算法示例和作业。Sage 是开源、跨平台支持的免费软件,能够在数学和计算机代数系统的学习过程中提供强大、灵活和易学的软件包。与 Mathematica, Maple 和 MATLAB 等系统不同,Sage 没有专利保护和使用费的限制。因此,Sage 可以在学校的计算机和网络上使用,学生也可以分别将其下载到他们自己的个人计算机上在家里使用。使用 Sage 的另外一个好处是学生可以掌握一个非常强大、灵活的工具来帮助计算几乎所有的数学问题,而不仅限于密码学。

在对密码算法数学基础的教学过程中,使用 Sage 能够显著增强教学效果。本书的附录 B 中提供了涵盖各种密码学概念的大量 Sage 示例。

附录 C 按照密码学概念的分类给出了习题集,使学生能够通过练习手把手地理解密码算法。

本书的教师资源中心 IRC 为教师提供了该附录。附录 C 中专门有一节介绍如何下载和使用 Sage，另一节介绍 Sage 编程基础，除此以外还包括为学生准备的以下分类习题：

- **第 2 章——古典密码**：仿射密码和 Hill 密码。
- **第 3 章——分组密码和数据加密标准 DES**：基于 SDES 的练习。
- **第 4 章——数论和有限域基本概念**：欧几里得算法和扩展欧几里得算法，多项式算法，有限域 GF(2^n)。
- **第 5 章——高级加密标准 AES**：基于 Sage 的练习。
- **第 6 章——伪随机数发生器和流密码**：BBS，线性同余和 ANSI X9.17 伪随机数发生器。
- **第 8 章——数论**：欧拉函数，Miller-Rabin 测试，因子分解，模幂运算，离散对数，以及中国剩余定理。
- **第 9 章——公钥密码和 RSA**：RSA 加密/解密以及签名。
- **第 10 章——其他公钥密码算法**：Diffie-Hellman 密钥交换，椭圆曲线密码。
- **第 11 章——密码学 Hash 函数**：基于数论的 Hash 函数。
- **第 13 章——数字签名**：DSA。

第五版新增内容

与本书的前几次再版相比，《密码编码学与网络安全——原理与实践》的本次新版的修改更加广泛和充实。

本书第四版出版后的三年中，该领域仍处于不断地变革之中。该新版中，我试图在继续广泛涵盖本领域内容的同时，增加这些新的变化。进行本次修订之初，本书第四版由许多讲授该领域课程的教授仔细审阅过。而且，许多研究该领域的专业人员也审阅过某些章节。这使得许多地方的叙述变得清晰、紧凑，对插图也进行了改进，而且增加了许多新的“现场测试”习题。

本书的主要修改之一是，对整体结构重新进行了组织，这使得相关问题的描述更加清晰。新增了第三部分，其中涵盖了密码算法中所有涉及数据完整性的内容，包括密码学 Hash 函数，消息认证码，以及数字签名。关于密钥管理和密钥交换的内容，在本书早期版本中散布在各章中，新版中组成了独立的一章。对于用户认证的内容也有类似调整。

除了这些为改进教学法和方便用户所做的修改以外，还有一些实质性的变化贯穿本书，最主要包括下列几个方面：

- **欧几里得算法和扩展的欧几里得算法（修订）**：这些算法对于许多密码函数和算法都非常重要。我们对于整数和多项式的欧几里得算法和扩展的欧几里得算法的内容进行了彻底地重新编写，给出了更清晰和系统的描述。
- **高级加密标准 AES（修订）**：AES 目前已经成为最通用的对称加密算法，在各种应用中被广泛使用。相应地，本版大幅扩展了学习和理解 AES 标准的资源。关于 AES 算法的章节被重新修订并扩充，为了清晰描述该算法而添加了更多的图和细节示例。同时还添加了使用 Sage 的示例和习题。本书目前包括了 AES 密码实验库，能够给学生提供手把手的实验环境，来学习 AES 密码的内容结构和工作模式。本书的网站中提供了该库使用的 AES 计算小程序，能够使用 AES 分组密码对测试数据进行加密或解密。
- **分组密码工作模式（修订）**：我们对第 6 章中关于工作模式的内容进行了扩充，并重新绘制了更清晰的图示。

- **伪随机数发生器和伪随机函数(修订)**:我们对该重要主题的处理方法的介绍进行了扩充,增加了使用对称加密算法和密码学 Hash 函数来构造伪随机函数的新内容。
- **ElGamal 加密和数字签名(新增)**:新增加了一节对于该流行的公钥算法进行介绍。
- **密码学 Hash 函数和消息认证码(修订)**:关于 Hash 函数和 MAC 的内容被重新修订和重新组织,使得描述更清晰和系统。
- **SHA-3(新增)**:尽管 SHA-3 算法目前还在征集选择过程中,对于学生们掌握该即将问世的密码标准的设计准则是非常重要的。
- **认证加密(新增)**:本书增加了重要的新算法 CCM 和 GCM,这两个算法能够同时提供机密性和数据完整性保护。
- **密钥管理和分配(修订)**:在第四版中该部分内容被穿插在三个章节中介绍,第五版中该部分内容被重新修改合并为独立的一章,使得描述统一和系统。
- **远程用户认证(修订)**:在第四版中该部分内容被分为两章介绍,第五版中该部分内容被重新修改合并为独立的一章,使得描述统一和系统。
- **联合身份识别(新增)**:新增一节介绍这种通用的身份管理方案,该方案在许多企业和无数应用中使用,服务于成百上千甚至于百万的用户。
- **HTTPS(新增)**:新增一节介绍 HTTPS 协议,该协议为 Web 浏览器和 Web 服务器之间的安全通信提供保护。
- **安全内核(新增)**:新增一节介绍 SSH,SSH 是加密技术最广泛的应用之一。
- **域密钥身份识别邮件 DKIM(新增)**:新增一节介绍 DKIM,DKIM 是应对垃圾邮件的邮件认证的标准。
- **无线网络安全(新增)**:新增一章专门讨论该网络安全中的重要领域。本章介绍无线局域网中使用的 IEEE 802.11(WiFi)安全标准;移动 Web 浏览器与 Web 服务器之间通信的安全标准无线应用协议(WAP)。
- **IPsec(修订)**:我们对介绍 IPsec 的一章几乎完全重新进行了编写。新版包括了 IPsecv3 和 IKEv2。此外,我们对表述也进行了修改使得更加清晰,并增加了内容的广度。
- **法律与道德(新增)**:新增了在线章节讨论该重要话题。
- **在线附录(新增)**:本书的网站上有 15 个附录,为感兴趣的同学们提供更深更广的各种材料。
- **Sage 示例和问题(新增)**:如上所述,本次新版使用开源、免费的 Sage 计算机代数系统,使学生能够亲手进行各种密码算法的实验。

对于每次新版的修订,在对全书保持合理的页数和增加新的内容之间进行取舍是非常困难的。在前几版中对篇幅的控制是通过将过时的内容删除以及尽量精简描述来实现的。对于本版,一些次重要的章节和附录转移到了独立的在线 PDF 文档中。这使得我们能够尽可能地扩充本书的内容而不增加本书的页数和价格。

致谢

本次修改得益于许多人的审阅,他们花费了大量的时间和精力。下列这些人员审阅了所有或大部分手稿:Marius Zimand(Towson State University),Shambhu Upadhyaya(University of Buffalo),Nan Zhang(George Washington University),Dongwan Shin(New Mexico Tech),Michael Kain(Drexel

University), William Bard(University of Texas), David Arnold(Baylor University), Edward Allen(Wake Forest University), Michael Goodrich(UC-Irvine), Xunhua Wang(James Madison University), Xianyang Li(Illinois Institute of Technology), 以及 Paul Jenkins(Brigham Young University)。

我还要感谢那些详细审阅其中某一章或数章的人员: Martin Bealby, Martin Hlavac(Department of Algebra, Charles University in Prague, Czech Republic), Martin Rublik(BSP Consulting and University of Economics in Bratislava), Rafael Lara(President of Venezuela's Association for Information Security and Cryptography Research), Amitabh Saxena, 以及 Michael Spratte(Hewlett-Packard Company)。我还要特别感谢 Nikhil Bhargava(IIT Delhi)对本书许多章节的细致审阅。

Joan Daemen 审阅了关于 AES 的章节。Vincent Rijmen 审阅了有关 Whirlpool 的内容。而 Edward F. Schaefer 审阅了有关简化 AES 的内容。

Nikhil Bhargava(IIT Delhi)开发了一系列的在线家庭作业和解答。Microsoft 和 University of Washington 的 Dan Shumow 开发了附录 B 和附录 C 中所有的 Sage 示例和作业。Dakota State University 的 Sreekanth Malladi 教授开发了黑客练习。Australian Defence Force Academy 的 Lawrie Brown 提供了 AES/DES 分组密码项目和安全评估练习。

Purdue University 的 Sanja Rao 和 Ruben Torres 为教师资源中心 IRC 里的实验室练习做了很多工作。下列人员为教师资源中心中的课程计划方面做了工作: Henning Schulzrinne(Columbia University), Cetin Kaya Koc(Oregon State University)和 David Balenson(Trusted Information Systems and George Washington University)。Kim McLaughlin 提供了习题库。

最后,我要感谢负责本书出版的工作人员,他们都做得很优秀。包括编辑 Tracy Dunkelberger,以及她的助理 Melinda Hagerty,产品经理 Rose Kernan。还要感谢 Warde 出版社的 Jake Warde 组织了整个审稿。

在这么多帮助面前,我几乎没有什么可以居功自傲的。但我可以自豪地说,没有这些帮助,我也会选择所有这些内容。

作者介绍

William Stallings 在计算机安全、计算机网络和计算机体系结构等技术的发展方面成就卓著。他编写出版了 17 部著作,经修订再版累计共 42 本上述相关领域的书籍。他的著作无数次出现在 ACM 和 IEEE 出版物中,包括 Proceedings of the IEEE 和 ACM Computing Reviews。

他 11 次获得美国"教材和著作家协会"(Text and Academic Authors Association)颁发的"年度最佳计算机科学教材"奖。

在过去的 30 年中,他曾在该领域的数个高科技企业中担任技术骨干、技术管理者和技术执行领导。他设计和实现了适用于从微型机到大型机的各种类型的计算机和操作系统的基于 TCP/IP 和基于 OSI 的协议。目前,他作为独立顾问为政府机构、计算机硬件制造商、软件开发商以及广大用户提供包括设计、选择和使用网络软件和产品的咨询服务。

他建设并维护计算机科学专业学生资源网 WilliamStallings.com/StudentSupport.html。该网站提供为计算机科学专业的学生(和专业人员)提供各种文档和链接。他是 *Cryptologia* 杂志的编委,该杂志是密码学的学术期刊。

William Stallings 博士先后获得了 Notre Dame 电气工程学士学位和 MIT 计算机科学博士学位。

目　录

第0章　读者导引 …… 1
0.1　本书概况 …… 1
0.2　读者和教师导读 …… 1
0.3　Internet 和 Web 资源 …… 2
0.4　标准 …… 4

第1章　概述 …… 5
1.1　计算机安全概念 …… 6
1.2　OSI 安全框架 …… 9
1.3　安全攻击 …… 10
1.4　安全服务 …… 12
1.5　安全机制 …… 14
1.6　网络安全模型 …… 15
1.7　推荐读物和网站 …… 17
1.8　关键术语、思考题和习题 …… 18

第一部分　对 称 密 码

第2章　传统加密技术 …… 22
2.1　对称密码模型 …… 22
2.2　代替技术 …… 26
2.3　置换技术 …… 37
2.4　转轮机 …… 38
2.5　隐写术 …… 39
2.6　推荐读物和网站 …… 40
2.7　关键术语、思考题和习题 …… 41

第3章　分组密码和数据加密标准 …… 46
3.1　分组密码原理 …… 47
3.2　数据加密标准 …… 53
3.3　DES 的一个例子 …… 59
3.4　DES 的强度 …… 61
3.5　差分分析和线性分析 …… 62
3.6　分组密码的设计原理 …… 64
3.7　推荐读物和网站 …… 66
3.8　关键术语、思考题和习题 …… 67

第4章　数论和有限域的基本概念 …… 71
4.1　整除性和除法 …… 72
4.2　Euclid 算法 …… 73
4.3　模运算 …… 75

4.4 群、环和域 …… 81
4.5 有限域 GF(p) …… 84
4.6 多项式运算 …… 86
4.7 有限域 GF(2^n) …… 91
4.8 推荐读物和网站 …… 99
4.9 关键术语、思考题和习题 …… 100
附录 4A mod 的含义 …… 102

第 5 章 高级加密标准 …… 104
5.1 有限域算术 …… 105
5.2 AES 的结构 …… 106
5.3 AES 的变换函数 …… 110
5.4 AES 的密钥扩展 …… 117
5.5 一个 AES 例子 …… 119
5.6 AES 的实现 …… 123
5.7 推荐读物和网站 …… 126
5.8 关键术语、思考题和习题 …… 127
附录 5A 系数在 GF(2^8) 中的多项式 …… 128
附录 5B 简化 AES …… 131

第 6 章 分组密码的工作模式 …… 138
6.1 多重加密与三重 DES 算法 …… 138
6.2 电码本模式 …… 142
6.3 密文分组链接模式 …… 143
6.4 密文反馈模式 …… 144
6.5 输出反馈模式 …… 146
6.6 计数器模式 …… 147
6.7 用于面向分组的存储设备的 XTS-AES 模式 …… 149
6.8 推荐读物和网站 …… 153
6.9 关键术语、思考题和习题 …… 153

第 7 章 伪随机数的产生和流密码 …… 156
7.1 随机数产生的原则 …… 156
7.2 伪随机数发生器 …… 160
7.3 使用分组密码的伪随机数产生 …… 162
7.4 流密码 …… 164
7.5 RC4 算法 …… 166
7.6 真随机数发生器 …… 168
7.7 推荐读物和网站 …… 168
7.8 关键术语、思考题和习题 …… 170

第二部分 公 钥 密 码

第 8 章 数论入门 …… 174
8.1 素数 …… 175

8.2 费马定理和欧拉定理 …… 177
8.3 素性测试 …… 179
8.4 中国剩余定理 …… 181
8.5 离散对数 …… 183
8.6 推荐读物和网站 …… 187
8.7 关键术语、思考题和习题 …… 188

第 9 章 公钥密码学与 RSA …… 190
9.1 公钥密码体制的基本原理 …… 191
9.2 RSA 算法 …… 197
9.3 推荐读物和网站 …… 207
9.4 关键术语、思考题和习题 …… 208
附录 9A RSA 算法的证明 …… 211
附录 9B 算法复杂性 …… 212

第 10 章 密钥管理和其他公钥密码体制 …… 215
10.1 Diffie-Hellman 密钥交换 …… 215
10.2 ElGamal 密码体系 …… 218
10.3 椭圆曲线算术 …… 221
10.4 椭圆曲线密码学 …… 227
10.5 基于非对称密码的伪随机数生成器 …… 229
10.6 推荐读物和网站 …… 231
10.7 关键术语、思考题和习题 …… 232

第三部分 密码学数据完整性算法

第 11 章 密码学 Hash 函数 …… 236
11.1 密码学 Hash 函数的应用 …… 237
11.2 两个简单的 Hash 函数 …… 239
11.3 需求和安全性 …… 241
11.4 基于分组密码链接的 Hash 函数 …… 245
11.5 安全 Hash 算法(SHA) …… 246
11.6 SHA-3 …… 253
11.7 推荐读物和网站 …… 254
11.8 关键术语、思考题和习题 …… 254
附录 11A 生日攻击的数学基础 …… 257

第 12 章 消息认证码 …… 261
12.1 对消息认证的要求 …… 262
12.2 消息认证函数 …… 262
12.3 对消息认证码的要求 …… 267
12.4 MAC 的安全性 …… 269
12.5 基于 Hash 函数的 MAC:HMAC …… 270
12.6 基于分组密码的 MAC:DAA 和 CMAC …… 272
12.7 认证加密:CCM 和 GCM …… 274

12.8 使用 Hash 函数和 MAC 产生伪随机数 …… 279
12.9 推荐读物和网站 …… 281
12.10 关键术语、思考题和习题 …… 281
第 13 章 数字签名 …… 283
13.1 数字签名 …… 283
13.2 ElGamal 数字签名方案 …… 286
13.3 Schnorr 数字签名方案 …… 287
13.4 数字签名标准 …… 288
13.5 推荐读物和网站 …… 290
13.6 关键术语、思考题和习题 …… 291

第四部分 相 互 信 任

第 14 章 密钥管理和分发 …… 296
14.1 对称加密的对称密钥分发 …… 297
14.2 非对称加密的对称密钥分发 …… 302
14.3 公钥分发 …… 304
14.4 X.509 认证服务 …… 307
14.5 公钥基础设施 …… 312
14.6 推荐读物和网站 …… 314
14.7 关键术语、思考题和习题 …… 315
第 15 章 用户认证 …… 318
15.1 远程用户认证原理 …… 318
15.2 基于对称加密的远程用户认证 …… 320
15.3 Kerberos …… 322
15.4 基于非对称加密的远程用户认证 …… 334
15.5 联合身份管理 …… 336
15.6 推荐读物和网站 …… 340
15.7 关键术语、思考题和习题 …… 340
附录 15A Kerberos 加密技术 …… 342

第五部分 网络与因特网安全

第 16 章 传输层安全 …… 346
16.1 Web 安全性思考 …… 347
16.2 安全套接层和传输层安全 …… 348
16.3 传输层安全 …… 357
16.4 HTTPS …… 360
16.5 SSH …… 361
16.6 推荐读物和网站 …… 369
16.7 关键术语、思考题和习题 …… 370

第 17 章　无线网络安全 …… 371
17.1　IEEE 802.11 无线网络概述 …… 372
17.2　IEEE 802.11i 无线局域网安全 …… 376
17.3　无线应用通信协议概述 …… 386
17.4　无线传输层安全 …… 391
17.5　WAP 端到端安全 …… 398
17.6　推荐读物和网站 …… 400
17.7　关键术语、思考题和习题 …… 401
第 18 章　电子邮件安全 …… 404
18.1　PGP …… 404
18.2　S/MIME …… 417
18.3　DKIM …… 429
18.4　推荐读物和网站 …… 433
18.5　关键术语、思考题和习题 …… 434
附录 18A　基数 64 转换 …… 435
第 19 章　IP 安全性 …… 437
19.1　IP 安全性概述 …… 438
19.2　IP 安全性策略 …… 441
19.3　封装安全性有效载荷 …… 444
19.4　组合安全性关联 …… 449
19.5　因特网密钥交换 …… 451
19.6　密码学套件 …… 456
19.7　推荐读物和网站 …… 457
19.8　关键术语、思考题和习题 …… 458
附录 A　用于密码学和网络安全教学的项目 …… 460
附录 B　Sage 示例 …… 464
参考文献 …… 499
索引 …… 510

在 线 部 分

第六部分　系 统 安 全

第 20 章　入侵者
20.1　入侵者
20.2　入侵检测
20.3　口令管理
20.4　推荐读物和网站
20.5　关键术语、思考题和习题
附录 20A　基于比率的错误

第 21 章　恶意软件
21.1　恶意软件的类型

21.2 病毒
21.3 计算机病毒的防治策略
21.4 蠕虫
21.5 分布式拒绝服务攻击
21.6 推荐读物和网站
21.7 关键术语、思考题和习题

第 22 章 防火墙
22.1 防火墙的必要性
22.2 防火墙的特性
22.3 防火墙的分类
22.4 防火墙基础
22.5 防火墙的位置与配置
22.6 推荐读物和网站
22.7 关键术语、思考题和习题

第七部分 法律与道德

第 23 章 法律与道德
23.1 网络犯罪和计算机犯罪
23.2 知识产权
23.3 隐私
23.4 道德问题
23.5 推荐读物和网站
23.6 关键术语、思考题和习题

在线附录

附录 C Sage 习题
附录 D 标准和标准化组织
附录 E 线性代数的基本概念
附录 F 保密和安全的测度
附录 G 简化 DES
附录 H AES 的评估准则
附录 I 简化 AES 的补充
附录 J 背包公钥算法
附录 K 数字签名算法的证明
附录 L TCP/IP 和 OSI
附录 M Java 密码函数 API
附录 N Whirlpool 算法
附录 O 使用 ZIP 的数据压缩
附录 P PGP 随机数产生
附录 Q 国际参考字母表
术语表

第0章 读者导引

0.1 本书概况

0.2 读者和教师导读

0.2.1 主题

0.2.2 主要内容的顺序

0.3 Internet 和 Web 资源

0.3.1 支持本书的网站

0.3.2 其他网站

0.3.3 新闻组和论坛

0.4 标准

The art of war teaches us to rely not on the likelihood of the enemy's not coming, but on our own readiness to receive him; not on the chance of his not attacking, but rather on the fact that we have made our position unassailable.

—*The Art of War*, Sun Tzu

本书及其配套网站涵盖了很多的内容。下面为读者介绍一下概况。

0.1 本书概况

第 1 章是一个全书的介绍章,在第 1 章之后本书由如下七部分组成:

第一部分:对称密码:提供了有关对称加密算法的综述,包括传统和现代加密算法。重点放在两个最为重要的算法上,即数据加密标准(DES)和高级加密标准(AES)。这部分还讨论了最重要的流密码算法 RC4,以及关于伪随机数发生器这一重要话题。

第二部分:非对称密码:给出了公钥密码算法的综述,包括 RSA(Rivest-Shamir-Adelman)密码和椭圆曲线密码。

第三部分:密码学中的数据完整性算法:首先给出了密码学 Hash 函数的综述。然后分为两个方面介绍基于密码学 Hash 函数的数据完整性方案,以及消息验证码和数字签名。

第四部分:相互信任:包括密钥管理和密钥分配,以及用户认证技术。

第五部分:网络与因特网安全:讨论了应用密码算法和安全协议为网络和 Internet 提供安全。涉及的主题包括传输层安全、无线网络安全、E-mail 安全和 IP 安全。

第六部分:系统安全:探讨了用来保护计算机系统免受各种安全性威胁的设施。这些威胁包括非法入侵、病毒和蠕虫。该部分还介绍了防火墙技术。

第七部分:法律和道德问题:讨论与计算机和网络安全相关的法律和道德问题。

本书网站中的许多在线附录包括了与本书相关的额外内容。

0.2 读者和教师导读

0.2.1 主题

本书的内容分为如下 4 大类:

- **密码学**:研究确保信息的秘密性、真实性的技术。密码学的三个主要的分支是:(1)对称密码;(2)非对称密码;(3)密码学 Hash 函数,以及消息认证码和数字签名等内容。

- **相互信任**:包括对于两个主要范畴提供相互信任的技术和算法的研究。这两个范畴是:第一,通信双方基于加密密钥建立信任的密钥管理和密钥分配的问题;第二,在通信方基于身份建立信任的用户认证问题。
- **网络安全**:这一领域讨论如何将密码算法用于网络协议和网络应用。
- **计算机安全**:本书里我们用这个术语来表示防止入侵(如黑客)和恶意软件(如病毒)的计算机安全。典型地,需要保护的计算机常连于网络,而大量的威胁也来自于网络。

本书的前两部分讨论两种不同的加密方法:对称加密算法和公钥(或称为非对称)加密算法。在对称算法中,双方使用单个共享的密钥。公钥算法使用两个密钥:私钥只被自己掌握,而公钥可被其他任何通信方掌握。

0.2.2 主要内容的顺序

本书包含了很多的内容,对于希望简短地阅读本书的教师和读者,有很多的方式可以这么做。

若想全面了解前三部分的内容,则应该逐章依次序阅读。除了高级加密标准(AES)外,第一部分的内容都不需要任何特别的数学背景知识。为了理解 AES,有必要对有限域有所了解。而理解有限域则需要素数和模算术的一些基本背景知识。相应地,在第 5 章使用它们之前,我们在第 4 章里包含了这些数学预备知识。因此,如果准备跳过第 5 章,那么跳过第 4 章也没有问题。

第 2 章中介绍了这部分其他章里都有用的一些概念。然而,对于那些只对现代密码学感兴趣的读者来说,这一章可以很快跳过。第 3 章和第 5 章分别讨论了两个最重要的对称密码算法:DES 和 AES。

第 6 章讨论使用分组对称密码的技术,即工作模式。第 7 章包括流密码和随机数发生器。在初次阅读的时候这两章也可以跳过,但该部分内容在本书的后续部分将被引用到。

对于第二部分,唯一需要补充的数学背景知识是数论,这将在第 8 章讨论。那些跳过第 4 章和第 5 章的读者应该首先复习一下 4.1 节至 4.3 节的内容。

两个使用得最广的通用公钥密码算法是 RSA 和椭圆曲线密码,RSA 更为人们所接受。读者也许会希望跳过第 10 章中有关椭圆曲线密码的内容,至少是初次阅读时会如此。

第三部分中 12.6 节和 12.7 节内容的重要性稍微低一些。

第四部分、第五部分和第六部分彼此相互独立,阅读顺序可以随意。这三部分都需要对第一部分、第二部分和第三部分的基本内容有所了解。第五部分中的 4 个章节讨论关于网络与因特网安全,这 4 章相互间彼此独立,阅读顺序可以随意。

0.3 Internet 和 Web 资源

Internet 和 Web 上有许多的资源都支持本书,以便读者可以跟踪该领域的发展。

0.3.1 支持本书的站点

为了支持本书,作者制作了一个专用网站:WilliamStallings. com/Crypto/Crypto5e. html。该网站包含如下内容:

- **有用的 Web 站点**:按章的顺序给出了通往其他站点的链接,这些站点包括全书已列出的站点。
- **勘误表**:我们会维护和按需更新勘误表。请读者将发现的错误通过 E-mail 告诉我们。书中已有的勘误表位于 WilliamStallings. com。

- **图**:以 PDF 格式给出了本书中的所有图形。
- **表**:以 PDF 格式给出了本书中的所有表格。
- **幻灯片**:按章组织的一套幻灯片。
- **密码编码学与网络安全教程**:含有指向本书课程主页的链接,这些主页有助于教师组织课程。

作者还维护了一个计算机科学专业学生资源网站:WilliamStallings. com/StudentSupport. html。该网站的目的是为计算机科学的学生和专业人士提供文档,信息和链接。链接和文档分为如下 6 类:

- **数学**:包括基本的数学复习,排队论分析初步读物,数字系统初步读物以及很多通向其他数学网站的链接。
- **如何做的问题**:给出了一些有关如何做家庭作业、写技术报告、准备技术汇报的建议和指导。
- **研究资源**:有关重要论文集,技术报告以及参考文献的链接。
- **杂项**:大量其他各类有用文档和链接。
- **计算机科学相关职业**:关于计算机科学的相关职业的有用文档和链接。
- **娱乐和消遣**:读者可以在学习间歇稍作休息。

0.3.2 其他网站

有很多的网站提供了与本书主题相关的信息。在后续各章中,在"推荐读物和网站"小节中有指向特定网站的链接。由于网站的地址往往会变,所以本书中未包含 URL。本书中列出的所有网站的合适链接可以在本书的网站中找到。而本书中未提到的其他链接将会随时被添加到本书的网站上。

0.3.3 新闻组与论坛

大量的 USENET 新闻组致力于密码编码学与网络安全的各个方面。事实上,所有的 USENET 新闻组都有很高的噪信比,然而仍然很值得去浏览一下,看看是否有东西是你需要的。最为相关的一些新闻组如下:

- sci. crypt. research:最好的新闻组。这是一个有管理的新闻组,处理一些研究类主题;张贴的内容必须与密码学的技术方面有关系。
- sci. crypt:一般性地讨论密码学和相关主题。
- sci. crypt. random-numbers:讨论具有密码学强度的随机数发生器。
- alt. security:一般性地讨论安全主题。
- comp. security. misc:一般性地讨论计算机安全主题。
- comp. security. firewalls:讨论防火墙产品和技术。
- comp. security. announce:来自 CERT 的新闻,公告等。
- comp. risks:讨论计算机和用户关于公众的威胁。
- comp. virus:有管理的,对计算机病毒的讨论。

此外,互联网上还有许多论坛讨论与密码学相关的话题。其中最值得关注的有:

- Security and Cryptography forum:由 DevShed 主办。讨论内容包括编码、服务器应用程序、网络保护、数据保护、防火墙、密码及相关话题。

- Cryptography forum:网址在 Topix,集中于相关技术问题的讨论。
- Security forum:网址在 WindowsSecurity. com,论坛的内容广泛,包括密码学理论,密码软件,防火墙和恶意软件。

本书网站中提供了这些论坛的链接。

0.4 标准

本书介绍了众多安全技术和应用,其中许多都已经成为标准。并且这些标准进一步发展成为涵盖实践管理和整个安全机制和设备的架构。在本书中,我们主要介绍正在使用或正在制定中的一些关于密码学与网络安全中最重要的标准。一大批组织机构参与了这些标准的制定或推广。其中最重要的组织包括:

- **美国标准与技术研究所 NIST**:NIST 是专门处理为美国政府使用及向公众推广的关于科学、标准和技术创新的美国政府官方机构。尽管 NIST 标准是为美国政府的使用而开发的,NIST 发布的联邦信息处理标准(FIPS)以及特殊文献(SP)对全世界都有广泛影响。
- **国际互联网协会 ISOC**:ISOC 是由全世界范围的组织和个人共同构成的行业性国际组织。ISOC 领导处理 Internet 未来发展的问题,同时 ISOC 也是负责 Internet 结构性标准部分的上级机构,下属机构包括互联网工程任务组(IETF)和互联网架构委员会(IAB)。这些机构负责制定 Internet 标准和相关规范,其发布的所有文档称为 RFC。
- ITU-T:国际电信联盟(ITU)是联合国的一个专门机构,被作为世界范围内联系各国政府和私营部门的纽带。国际电信联盟远程通信标准化组织(ITU-T)是 ITU 的三个部门之一。ITU-T 的使命是制定远程通信相关国际标准。ITU-T 标准通常被称为建议(Recommendations)。
- ISO:国际标准化组织(ISO)是由超过 140 个国家组成的专门制定国际标准的机构①。ISO 是非政府部门,旨在便于商品和服务的国际交换,涉及智力、科学、技术和经济领域中的标准和相关合作。ISO 通过国际认可的结果被制定为国际标准。

关于上述机构更详细的描述请见附录 D。

① ISO 并不是首字母缩写(国际标准化组织 International Organization for Standardization 的缩写应该是 IOS),而是希腊语中的“相等”。

第1章 概 述

1.1 计算机安全概念
- 1.1.1 计算机安全的定义
- 1.1.2 例子
- 1.1.3 计算机安全的挑战

1.2 OSI 安全框架

1.3 安全攻击
- 1.3.1 被动攻击
- 1.3.2 主动攻击

1.4 安全服务
- 1.4.1 认证
- 1.4.2 访问控制
- 1.4.3 数据保密性
- 1.4.4 数据完整性
- 1.4.5 不可否认性
- 1.4.6 可用性服务

1.5 安全机制

1.6 网络安全模型

1.7 推荐读物和网站

1.8 关键术语、思考题和习题

The combination of space, time, and strength that must be considered as the basic elements of this theory of defense makes this a fairly complicated matter. Consequently, it is not easy to find a fixed point of departure.

—*On War*, Carl Von Clausewitz

要 点

- 开放系统互连(Open Systems Interconnection, OSI)安全框架,提供了定义安全攻击、机制和服务的系统框架。
- 安全攻击分为被动攻击和主动攻击。被动攻击包括非授权阅读消息、文件以及流量分析。主动攻击包括对消息或文件的篡改以及拒绝服务等。
- 安全机制是一种处理过程(或实现该处理过程的设备),用来检测、阻止攻击或者从攻击状态恢复为正常状态。安全机制的例子有加密算法、数字签名和认证协议。
- 安全服务包括认证、访问控制、数据保密性、数据完整性、非否认性以及可用性。

本书集中讨论两大领域:一是密码算法和协议,它们有着广泛的应用;二是网络和 Internet 安全,它们大量地依赖密码技术。

密码算法和协议又可以分为 4 个主要领域:

- **对称加密**:用于加密任意大小的数据块或数据流的内容,包括消息、文件、加密密钥和口令。
- **非对称加密**:用于加密小的数据块,如加密密钥或者数字签名中使用的 Hash 函数值。
- **数据完整性算法**:用于保护数据块(例如一条消息)的内容免于被修改。
- **认证协议**:有许多基于密码算法的认证方案,用来认证实体的真实性。

网络和 Internet 安全领域涉及阻止、防止、检测和纠正信息传输中出现的安全违规行为的措施。它所包含的内容相当广泛。为使读者对本书中讨论的领域有所了解,我们先举几个有关安全违规行为的例子:

(1) 用户 A 向用户 B 传送文件,该文件包含不能泄密的敏感信息(如工资单),用户 C 无权读取该文件,但他能够监视传输过程并截获该文件。

(2) 网络管理员 D 向其管辖的计算机 E 传输一条消息,命令计算机 E 更新权限文件以允许一批新用户可以访问 E。用户 F 截获并修改该消息,如增加或删除一些用户,然后再将消息转发给 E,而 E 误以为是管理员 D 发来的消息并按照消息的内容更新权限文件。

(3) 上例中,用户 F 也可以不截获消息,而是按自己的意愿构造消息并发送给 E,同样 E 误以为是管理员 D 发来的消息并更新权限文件。

(4) 一位雇员事先未得到警告就被解雇。人事经理向服务器系统发送消息以注销该雇员的账号,账号注销后,服务器将发送通知到该雇员的文件中以确认注销。该雇员可以截获并延时发送人事经理发的消息,直至他有足够的时间访问服务器来获取敏感信息,然后再转发这条消息,以注销账号。雇员的这些活动在相当长时间内不会被察觉。

(5) 顾客向股票经纪人发送消息,请求完成各种交易,后来这些投资失败而顾客否认发送过该消息。

尽管上述举例不能穷尽所有可能的安全威胁类型,但它们说明了网络安全所关注的范围。

1.1 计算机安全概念

1.1.1 计算机安全的定义

NIST 的《计算机安全手册》[NIST95]针对计算机安全这一术语的定义如下:

计算机安全

对于一个自动化的信息系统,采取保护措施确保信息系统资源(包括硬件、软件、固件、信息/数据和通信)的完整性、可用性和保密性。

这个定义引进了处于计算机安全核心地位的三个关键目标。

- **保密性(Confidentiality)**:这个术语包含了两个相关的概念:
  - 数据[①]保密性:确保隐私或者秘密信息不向非授权者泄露,也不被非授权者所使用。
  - 隐私性:确保个人能够控制或确定与其自身相关的哪些信息是可以被收集、被保存的,这些信息可以由谁来公开以及向谁公开。
- **完整性(Integrity)**:这个术语包含两个相关的概念:
  - 数据完整性:确保信息和程序只能以特定和授权的方式进行改变。
  - 系统完整性:确保系统以一种正常的方式来执行预定的功能,免于有意或者无意的非授权操纵。
- **可用性(Availability)**:确保系统能工作迅速,对授权用户不能拒绝服务。

这三个概念形成了被经常称道的 CIA 三元组(如图 1.1 所示)。这三个概念体现了数据、信息和计算服务的基本安全目标。例如,NIST 标准 FIPS 199(《联邦信息和信息系统安全分类标准》)

① RFC 2828 定义信息为"事实和想法,可以用各种形式的数据进行表示",而定义数据为"用特定物理表示的信息,通常是一串有意义的符号序列;特别地,是可以由计算机处理和产生的信息表示"。安全文献一般不做这种区分,本书也不做区分。

将保密性、完整性和可用性作为信息和信息系统的三个安全目标。FIPS 199 从安全需求和安全缺失的角度对这三个目标进行了刻画。

- **保密性**:对信息的访问和公开进行授权限制,包括保护个人隐私和秘密信息。保密性缺失的定义是信息的非授权泄露。
- **完整性**:防止对信息的不恰当修改或破坏,包括确保信息的不可否认性和真实性。完整性缺失的定义是对信息的非授权修改和毁坏。
- **可用性**:确保对信息的及时和可靠的访问与使用。可用性的缺失是对信息和信息系统访问与使用的中断。

尽管 CIA 三元组足以定义安全目标,但是许多从事安全领域研究的人认为还需要其他概念来定义才更全面。下面是其中两个被提及较多的概念。

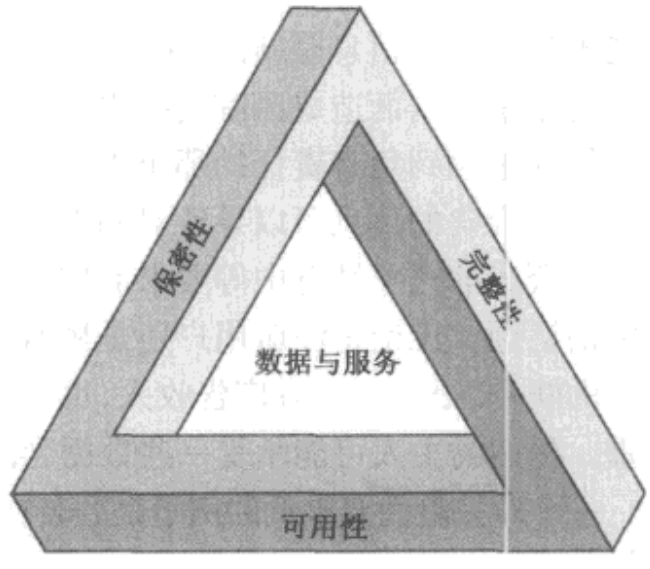


图 1.1 安全需求三元组

- **真实性(Authenticity)**:一个实体是真实性的、是可被验证的和可被信任的特性;对传输信息来说,信息和信息的来源是正确的。也就是说,能够验证那个用户是否是他声称的那个人,以及系统的每个输入是否均来自可信任的信源。
- **可追溯性(Accountability)**:这一安全目标要求实体的行为可以唯一追溯到该实体。这一属性支持不可否认性、阻止、故障隔离、入侵检测和预防、事后恢复,以及法律诉讼。因为无法得到真正安全的系统,我们必须能够把安全泄露追查到负有责任的一方。系统必须保留他们活动的记录,以允许事后的审计分析来跟踪安全事件或者解决争执。

1.1.2 例子

下面我们提供一些应用的例子来展示刚才列举的一些要求①。对于这些例子,如果发生了安全泄露事件(保密性、完整性或可用性的缺失),我们使用三个层次说明对组织和个人的影响。这些层次定义在 FIPS PUB 199 中。

- **低**:这种损失对组织的运行、组织的资产或者个人的负面影响有限。有限的负面影响是指,例如,保密性、完整性或可用性的缺失可能:(1)导致执行使命的能力在一定程度上和时期内的降级,这期间仍能完成主要的功能,但功能的效果会有所降低;(2)导致资产的较小损失;(3)导致很小的经济损失;(4)导致对个人的很小伤害。
- **中**:这种损失对组织的运行、组织的资产和个人有严重的负面影响。严重的负面影响是指,例如,这种损失可以:(1)导致执行使命的能力在一定程度上和时期内的显著降级,这期间仍能够完成主要的功能,但功能的效果会显著降低;(2)导致资产的显著损失;(3)导致显著的经济损失;(4)导致对个人的显著伤害,但不包括丧命或者严重威胁生命安全的伤害。
- **高**:这种损失对组织的运行、资产和个人有严重的或者灾难性的负面影响。严重的或灾难性的负面影响是指,例如,这种损失可以:(1)导致执行使命的能力在一定程度上和时期内的严重降级,

① 这些例子来自普度大学信息技术安全与隐私办公室发表的安全政策文献。

这期间不能完成主要的一项或多项功能;(2)导致大部分资产的损失;(3)导致大部分经济的损失;(4)导致对个人的严重或灾难性的伤害,包括丧命或者严重威胁生命安全的伤害。

保密性:学生的分数信息是一种资产,它的保密性被学生们认为是非常重要的。在美国,这种信息的发布受家庭教育权和隐私权法案(FERPA)管理。学生的分数仅可以由学生自己、他们的父母,以及需要这些信息来完成工作的学校雇员得到。学生的注册信息有中等程度的保密等级。尽管注册信息仍然受 FERPA 管理,但这些信息可以以天为单位被更多人看到,它比起分数信息更少受到攻击,即使受到攻击,损失也比较小。目录信息,如学生、老师、院系名单可列为低保密等级或者无须保密。这些信息对公众自由开放,可以在学校网页上发布。

完整性:存储在医院数据库内的病人的过敏信息的例子可以说明完整性的几个方面。医生应该能够信任这些信息是新的、正确的。现在假设一个有权查看和更新这些信息的雇员(比如护士)有意篡改了数据而造成医院的损失。这个数据库需要快速恢复到可以信任的状态,而且应该能把这些错误追溯到负有责任的那个人。这个例子说明病人的过敏信息是对完整性要求很高的一种资产。不准确的信息可以导致对病人的伤害甚至是造成病人死亡,从而使医院担负重大的责任。

对资产的完整性有中等要求的例子是 Web 站点,这些站点提供论坛供用户注册来讨论一些特定的话题。无论是注册用户还是黑客都不能篡改某些项或者丑化网站。如果网站仅仅是为了用户的娱乐,很少或没有广告收入,也不是用于如科研等重要的事情,那么潜在的危害就不是那么严重。网站的主人可能承受一些数据、经济和时间上的损失。

一个对完整性要求低的例子是匿名在线民意调查。许多 Web 站点,如新闻机构,为他们的用户提供几乎没有监管的这类民意调查。然而,这类民意调查的不准确性和非科学性早已为大家所理解。

可用性:一个部件或服务越关键,可用性的要求就越高。考虑一个为关键系统、应用和设备提供认证服务的系统,服务的瘫痪将导致顾客不能访问计算资源,员工不能访问他们执行重要任务所需要的资源。由于员工生产率的损失和顾客潜在的损失,使得服务的缺失转换为大量的经济损失。

对资产的可用性要求中等的一个例子是大学的公共网站;这样的网站为现有的和潜在的学生和捐助人提供信息。这样的网站不能算是大学信息系统的关键部分,但它的不可用仍然会给大学造成窘境。

在线电话目录查询应用可以划分为可用性要求低的例子。尽管服务的临时缺失是一件恼人的事情,但有其他办法获得这些信息,如纸质的电话号码簿或者接线员。

1.1.3 计算机安全的挑战

计算机和网络安全很吸引人也很复杂。原因如下:

(1) 安全对初学者而言并没有想象的那么简单。安全的要求看上去很直观;的确,对安全服务的大部分要求都可以用其含义不言自明的单词给出,如保密性、认证、不可否认或完整性。但是满足这些要求的机制却非常复杂,理解它们需要缜密的推理。

(2) 当设计一个特别的安全机制或算法时,一定要考虑各种各样潜在的攻击。以与设计完全不同的方式看问题往往可以使攻击成功,这样做通常是利用了设计的机制中没有预料到的弱点。

(3) 根据第二点,设计安全机制的过程通常采用逆向思维。安全机制是复杂的,从要求的陈述中并不能明显地看出需要精心的设计,只有当威胁的各个方面被考虑到时,对精心设计的复杂机制的需求就变得易于理解了。

(4) 设计好各种安全机制后,接下来是决定在哪里使用这些安全机制。包括物理位置(例如,

网络的什么地方需要某一安全机制)和逻辑位置(例如,像 TCP/IP 这样的网络协议的哪一层或哪几层)。

(5) 安全机制所使用的算法或协议通常不止一个。这些算法和协议需要参与者使用一些秘密信息(如加密密钥),这就带来对秘密信息的产生、分发和保护等问题。它们所依赖的通信协议可能会使开发安全机制的过程变得复杂。例如,安全机制要正常行使功能需要对消息的传输时间设定限制,而任何协议和网络都存在不确定且不可预测的延迟,从而使得那个时间限制变得毫无意义。

(6) 计算机和网络安全本质上是一场入侵者和设计者(或管理员)之间的智力战争。入侵者努力要找到漏洞,而设计者或管理员努力要封堵漏洞。入侵者的优势在于他或她仅仅需要找到一个弱点,而设计者必须找到并根除所有的弱点来获得完全的安全。

(7) 用户和系统管理员有一种倾向,直到发生了安全事件,才意识到安全投资可以带来收益。

(8) 安全需要经常的甚至是不断的监管,而这在当今短期、负荷过重的环境中是难以做到的。

(9) 绝大多数情况下,安全仍然是一种事后措施,当系统设计完成以后,再来把安全机制增加到系统中,而不是作为整个系统设计过程的一个主要部分。

(10) 许多用户,甚至是安全管理员认为强的安全不利于信息系统高效工作、有碍于用户友好操作,或不利于对信息的使用。

本书中,随着我们对各种安全威胁和机制的考察,我们会以多种方式遇到上面列举的各种困难。

1.2 OSI 安全框架

为了有效评价一个机构的安全需求,以及对各种安全产品和政策进行评价和选择,负责安全的管理员需要某种系统的方法来定义对安全的要求并刻画满足这些要求的措施。在集中式数据处理环境下做到这一点已经非常困难。随着局域网和广域网的使用,这一问题变得更加复杂。

为此,ITU-T① 推荐方案 X.800,即 OSI 安全框架,给出了一种系统化的定义方法②。对安全人员来说,OSI 安全框架是提供安全的一种组织方法。而且,因为这个框架是作为国际标准而开发的,所以许多计算机和通信的服务商已经开发了与 OSI 安全框架的安全特性相适应的产品和服务。

对于我们来说,OSI 安全框架实际上是对本书将要涉及的许多概念做了一个尽管抽象但非常有用的综述。OSI 安全框架主要关注安全攻击、安全机制和安全服务。可以简短地定义如下:

- **安全攻击**:任何危及信息系统安全的行为。
- **安全机制**:用来检测、阻止攻击或者从攻击状态恢复到正常状态的过程(或实现该过程的设备)。
- **安全服务**:加强数据处理系统和信息传输的安全性的一种处理过程或通信服务。其目的在于利用一种或多种安全机制进行反攻击。

在某些文献中,术语威胁和攻击差不多是用来指相同的事情的。表 1.1 列出了 RFC 2828(互联网安全术语表)给出的定义。

---

① 国际电信联盟(ITU)电信标准化组(ITU-T)是一个联合国资助的机构,主要职责是开发与长途通信、开放系统互连(OSI)有关的各种标准,这些标准也称为建议。

② OSI 安全框架是在 OSI 协议框架范围内开发的,协议框架在附录 L 中给出。然而,本章的目的不需要理解 OSI 协议框架。

表 1.1 威胁和攻击(RFC 2828)

威胁 破坏安全的潜在可能,在环境、能力、行为或事件允许的情况下,它们会破坏安全,造成危害。也就是说,威胁是脆弱性被利用而可能带来的危险
攻击 对系统安全的攻击,它来源于一种具有智能的威胁,也就是说,有意违反安全服务和侵犯系统安全策略的(特别是在方法或技巧方面的)智能行为

1.3 安全攻击

X.800 和 RFC 2828 都使用了一种有效的方式来对安全攻击进行分类,即被动攻击和主动攻击。被动攻击试图了解或利用系统的信息但不影响系统资源。主动攻击试图改变系统资源或影响系统运作。

1.3.1 被动攻击

被动攻击的特性是对传输进行窃听和监测。攻击者的目标是获得传输的信息。信息内容的泄漏和流量分析就是两种被动攻击。

信息内容泄漏攻击很容易理解[参见图 1.2(a)]。电话、电子邮件消息和传输的文件都可能含有敏感或秘密的信息。我们希望能阻止攻击者获得传输的内容。

第二种被动攻击是流量分析[参见图 1.2(b)],这种分析有些微妙。设想我们已有一种方法来隐藏消息内容或其他信息流量,使得攻击者即使捕获了消息也不能从消息中获得信息。加密是隐藏内容的常用技巧。即使我们恰当地进行了加密保护,攻击者仍可能获得这些消息模式。攻击者可以确定通信主机的身份和位置,可以观察传输消息的频率和长度。这些信息可以用于判断通信的性质。

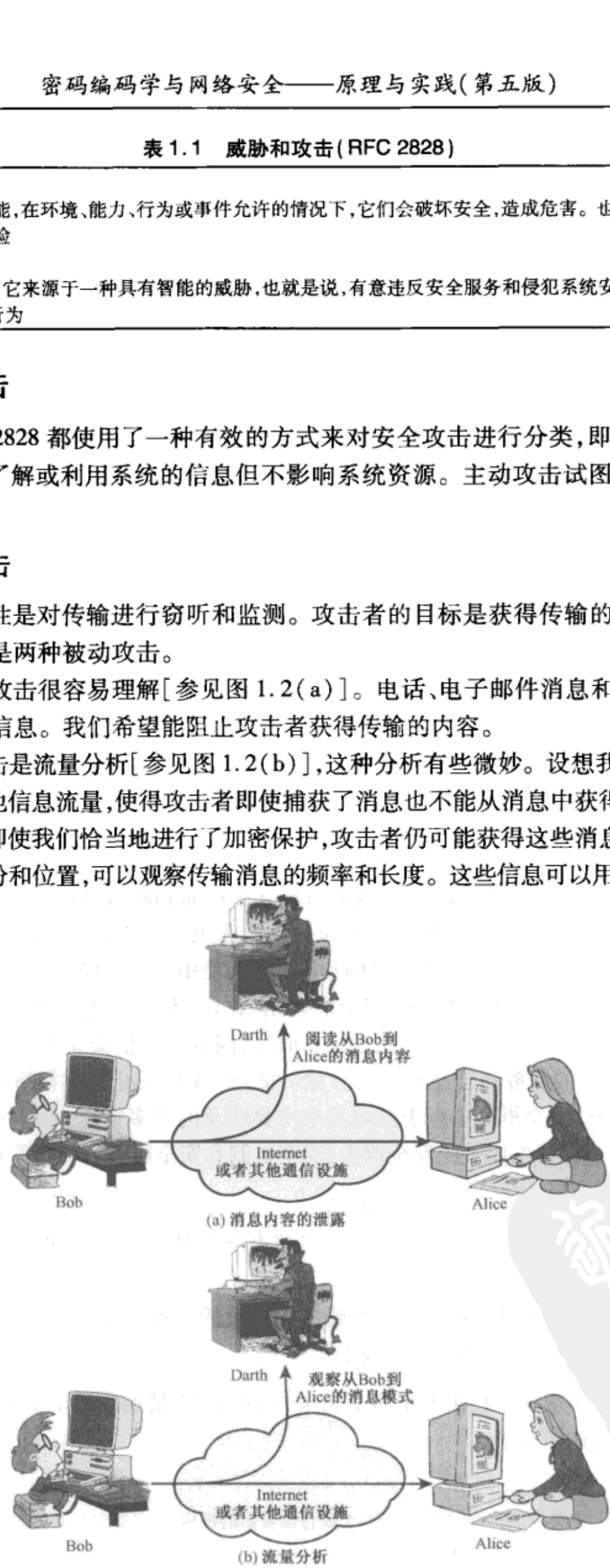


图 1.2 被动攻击

被动攻击由于不涉及对数据的更改，所以很难觉察。典型的情况是，信息流表面上以一种常规的方式在收发，收发双方谁也不知道有第三方已经读了信息或者观察了流量模式。然而，通过加密的手段阻止这种攻击却是可行的。因此处理被动攻击的重点是预防，而不是检测。

1.3.2 主动攻击

主动攻击包括对数据流进行修改或伪造数据流，可分为四类：伪装、重播、消息修改和拒绝服务。

伪装是指某实体假装别的实体[参见图1.3(a)]。伪装攻击通常还包含其他形式的主动攻击。例如：捕获认证信息，并在真的认证信息之后进行重播，这样，没有权限的实体就通过冒充有权限的实体获得了额外的权限。

重播是指将获得的信息再次发送以产生非授权的效果[参见图1.3(b)]。

消息修改指修改合法消息的一部分或延迟消息的传输或改变消息的顺序以获得非授权的效果[参见图1.3(c)]。例如，将消息“Allow John Smith to read confidential file accounts”修改为“Allow Fred Brown to read confidential file accounts”。

拒绝服务阻止或禁止对通信设施的正常使用或管理[参见图1.3(d)]。这种攻击可能有具体的目标。比如，某实体可能会查禁所有发向某目的地(如安全审计服务)的消息。拒绝服务的另一种形式是破坏整个网络，它或者是使网络失效，或者是使其过载以降低其性能。

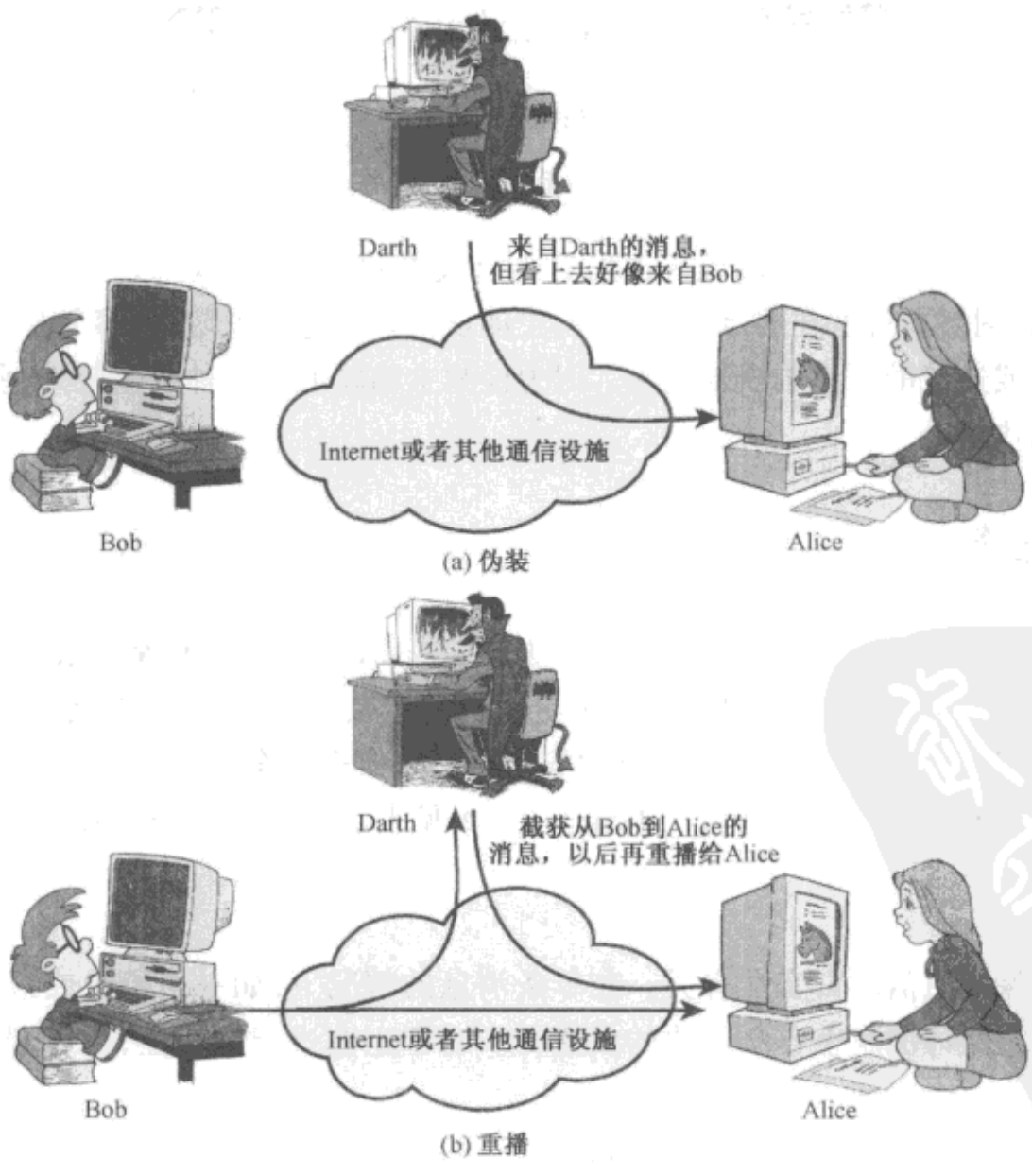


图1.3 主动攻击

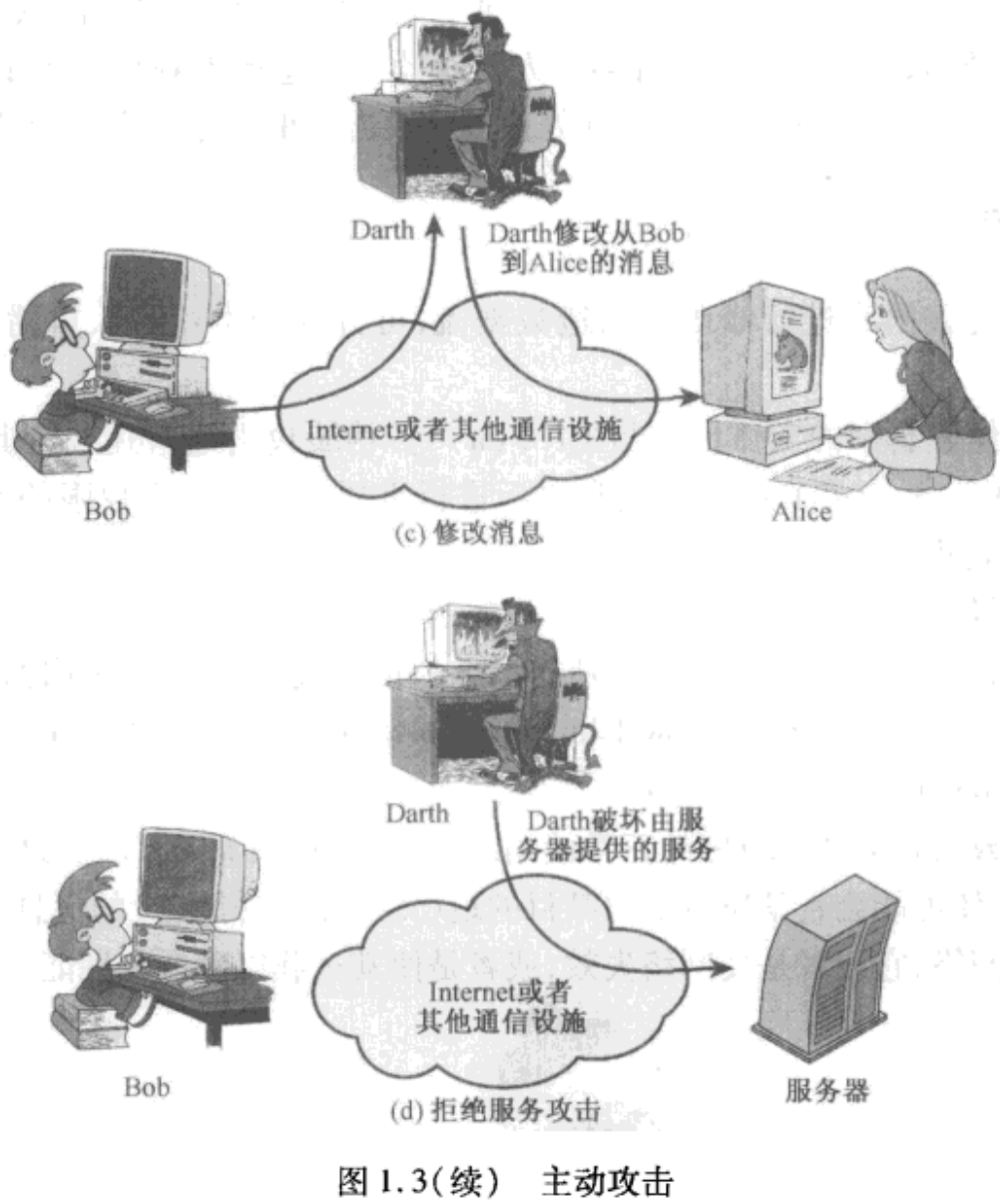


图 1.3(续) 主动攻击

主动攻击与被动攻击相反。被动攻击虽然难以被检测到,但可以防止。另一方面,因为物理通信设施、软件和网络本身潜在弱点的多样性,主动攻击难以绝对预防,但容易检测。所以重点在于检测并从破坏或造成的延迟中恢复过来。因为检测主动攻击有一种威慑效果,所以也可在某种程度上阻止主动攻击。

1.4 安全服务

X.800 将安全服务定义为通信开放系统的协议层提供的服务,从而保证系统或数据传输有足够的安全性。也许在 RFC 2828 中可找到一种更清楚的定义:是一种由系统提供的对系统资源进行特殊保护的处理或通信服务;安全服务通过安全机制来实现安全策略。

X.800 将这些服务分为 5 类共 14 个特定服务(参见表 1.2)。我们下面逐类进行讨论[①]。

1.4.1 认证

认证服务与保证通信的真实性有关。在单条消息的情况下,如一条警告或报警信号,认证服务功能是向接收方保证消息来自所声称的发送方。对于正在进行的交互,如终端和主机连接,就

① 信息安全文献中使用的许多术语尚未达成广泛的一致。例如,完整性有时是指信息安全性的多个方面,认证有时既用来指身份验证,又用来指本章中列出的各种数据完整性功能。我们这里使用的术语与 X.800 和 RFC 2828 中的术语是相一致的。

涉及两个方面。首先,在连接的初始化阶段,认证服务保证两个实体是可信的,也就是说,每个实体都是他们所声称的实体。其次,认证服务必须保证该连接不受第三方的干扰:这种干扰是指,第三方能够伪装成两个合法实体中的一个进行非授权传输或接收。

X.800 还定义了两个特殊的认证服务:

- **同等实体认证**:为连接中的同等实体提供身份确认。如果处于不同系统中的两个实体实行相同的协议,则考虑他们为对等的,例如位于两个通信系统中的两个 TCP 模块。对等实体认证用于连接的建立或数据传输阶段。该服务想提供这样的保证:一个实体没有试图进行伪装或对以前的连接进行非授权重播。
- **数据源认证**:为数据的来源提供确认。但对数据的复制或修改并不提供保护。这种服务支持电子邮件这样的应用,在这种应用的背景下,通信实体间没有预先的交互。

表 1.2　安全服务(X.800)

认证	**数据完整性**
保证通信的实体是它所声称的实体	保证收到的数据的确是授权实体所发出的数据(即没有修改、插入、删除或重播)
同等实体认证	**具有恢复功能的连接完整性**
用于逻辑连接时为连接的实体的身份提供可信性	提供一次连接中所有用户数据的完整性。检测整个数据序列内存在的修改、插入、删除或重播,且试图恢复
数据源认证	**无恢复的连接完整性**
连接传输时保证收到的信息来源是声称的来源	同上,但仅提供检测,无恢复
访问控制	**选择域连接完整性**
阻止对资源的非授权使用(即这项服务控制谁能访问资源,在什么条件下可以访问,这些访问的资源可用于做什么)	提供一次连接中传输的单个数据块内用户数据的指定部分的完整性,并判断指定部分是否有修改、插入、删除或重播
数据保密性	**无连接完整性**
保护数据免于非授权泄露	为单个无连接数据块提供完整性保护,并检测是否有数据修改。另外,提供有限的重播检测
连接保密性	**选择域无连接完整性**
保护一次连接中所有的用户数据	为单个无连接数据块内的指定域提供完整性保护;判断指定域是否被修改
无连接保密性	**不可否认性**
保护单个数据块里的所有用户数据	防止整个或部分通信过程中,任一通信实体进行否认的行为
选择域保密性	**源不可否认**
对一次连接或单个数据块中指定的数据部分提供保密性	证明消息是由特定方发出的
流量保密性	**宿不可否认性**
保护那些可以通过观察流量而获得的信息	证明消息被特定方收到

1.4.2　访问控制

在网络安全中,访问控制是一种限制和控制那些通过通信连接对主机和应用进行访问的能力。为此,每个试图获得访问控制的实体必须被识别或认证后才能获取其相应的访问权限。

1.4.3　数据保密性

保密性的目的是防止传输的数据遭到被动攻击。关于数据传输,可以有几层保护。最广泛的服务在一段时间内为两个用户间所传输的所有用户数据提供保护。例如,如果两个系统间建立了 TCP 连接,则这种广泛的保护将防止在 TCP 连接上传输的任何用户数据的泄露。也可以定义一种较窄的保密性服务,可以是对单条消息或对单条消息内某个特定的范围提供保护。这种细化比起广泛的方法用处要少,而且实现起来更复杂昂贵。

保密性的另一个方面是防止流量分析。这要求攻击者不能观察到消息的源和宿、频率、长度,或通信设施上的其他流量特征。

1.4.4 数据完整性

与保密性一样,完整性可应用于消息流、单条消息或消息的指定部分。同样,最有用也最直接的方法是对整个数据流提供保护。

用于处理消息流、面向连接的完整性服务保证收到的消息和发出的消息一致,没有复制、插入、修改、更改顺序或重播。该服务也涉及对数据的破坏。因此,面向连接的完整性服务处理消息流的修改和拒绝服务两个问题。另一方面,无连接的完整性服务,仅仅处理单条消息,而不管大量的上下文信息,其通常仅仅防止对单条消息的修改。

我们可以区分有恢复和无恢复的服务。因为完整性服务和主动攻击有关,我们更关心检测而不是阻止攻击。如果检测到完整性遭破坏,那么服务可以简单地报告这种破坏,并通过软件的其他部分或人工干预来恢复被破坏部分。另外,我们下面会看到,有些机制可用来恢复数据完整性。通常,自动恢复机制是一种更具吸引力的选择。

1.4.5 不可否认性

不可否认性防止发送方或接收方否认传输或接收过某条消息。因此,当消息发出后,接收方能证明消息是由声称的发送方发出的。同样,当消息接收后,发送方能证明消息确实已声称的接收方收到。

1.4.6 可用性服务

X.800 和 RFC 2828 都定义可用性为:根据系统的性能说明,能够按被授权系统实体的要求访问或使用系统和系统资源的性质(即当用户请求服务时,若系统能够提供符合系统设计的这些服务,则系统是可用的)。许多攻击可导致可用性的损失或减少。一些自动防御措施,如认证、加密,可对付某些攻击。而其他的一些攻击需要一些物理措施来阻止或恢复分布式系统中要素可用性的损失。

X.800 将可用性视为和各种安全服务相关的性质。但是,单独说明可用性服务是颇有意义的。可用性服务确保系统的可用性。这种服务处理由拒绝服务攻击引起的安全问题。它依赖于对系统资源的恰当管理和控制,因此依赖于访问控制服务和其他安全服务。

1.5 安全机制

表 1.3 列出了 X.800 中定义的安全机制。由表可知,这些安全机制可分成两类:一类在特定的协议层实现,如 TCP 或应用层协议;另一类不属于任何的协议层或安全服务。本书将会在适当的时候讨论这些机制,在此除了讨论加密的定义外,我们不详论之。X.800 区分可逆和不可逆加密机制。可逆加密机制只是一种加密算法,数据可以加密和解密。不可逆加密机制包括 Hash 算法和消息认证码,用于数字签名和消息认证应用。

基于 X.800 中的定义,表 1.4 给出了安全服务和安全机制的关系。

表 1.3 安全机制(X.800)

特定安全机制	普遍的安全机制
可以并入适当的协议层以提供一些 OSI 安全服务	不局限于任何特定的 OSI 安全服务或协议层的机制
加密 运用数学算法将数据转换成不可知的形式。数据的变换和还原依赖于算法和零个或多个加密密钥	**可信功能** 据某些标准被认为是正确的功能(例如,根据安全策略所建立的标准)
数字签名 附加于数据单元之后的一种数据,它是对数据单元的密码变换,以使得(如接收方)可证明数据源和完整性,并防止伪造	**安全标签** 资源(可能是数据单元)的标志,命名或指定该资源的安全属性
访问控制 对资源行使访问控制的各种机制	**事件检测** 检测与安全相关的事件
数据完整性 用于保证数据单元或数据单元流的完整性的各种机制	**安全审计跟踪** 收集可用于安全审计的数据,它是对系统记录和行为的独立回顾和核查
认证交换 通过信息交换来保证实体身份的各种机制	**安全恢复** 处理来自安全机制的请求,如事件处理、管理功能和采取恢复行为
流量填充 在数据流空隙中插入若干位以阻止流量分析	
路由控制 能够为某些数据选择特殊的物理上安全的路线并允许路由变化(尤其是在怀疑有侵犯安全的行为时)	
公证 利用可信的第三方来保证数据交换的某些性质	

表 1.4 安全服务与安全机制间的关系

服务	加密	数字签名	访问控制	数据完整性	认证交换	流量填充	路由控制	公证
同等实体认证	Y	Y			Y			
数据源认证	Y	Y						
访问控制			Y					
保密性	Y						Y	
流量保密性	Y					Y	Y	
数据完整性	Y	Y		Y				
不可否认性		Y		Y				Y
可用性				Y	Y			

1.6 网络安全模型

我们要讨论的大多数通信模型如图 1.4 所示,通信一方要通过 Internet 将消息传送给另一方,那么通信双方,称为交互的主体,必须协调努力共同完成消息交换,我们可以通过定义 Internet 上从源到宿的路由以及通信主体共同使用的通信协议(如 TCP/IP)来建立逻辑信息通道。

在需要保护信息传输以防攻击者威胁消息的保密性、真实性等的时候,就会涉及信息安全,任何用来保证安全的方法都包含两个方面:

- 与待发送信息安全相关的变换。如对消息加密,它打乱消息使得攻击者不能读懂消息,或者将基于消息的编码附于消息后,用于验证发送方的身份。
- 双方共享某些秘密信息,并希望这些信息不为攻击者所知。如加密密钥,它配合加密算法在消息传输之前将消息加密,而在接收端将消息解密①。

① 第二部分讨论的公钥密码中,只需发送方或者接收方拥有秘密信息。

为了实现安全传输,可能需要有可信的第三方。例如,第三方负责将秘密信息分配给通信双方,而对攻击者保密,或者当通信双方关于信息传输的真实性发生争执时,由第三方来仲裁。

上述模型说明,设计安全服务应包含下列4个方面的内容:

(1) 设计一个算法,它执行与安全相关的变换。该算法应是攻击者无法攻破的。
(2) 产生算法所使用的秘密信息。
(3) 设计分配和共享秘密信息的方法。
(4) 指明通信双方使用的协议,该协议利用安全算法和秘密信息实现安全服务。

本书的第一部分至第五部分讨论的是安全机制和服务,它们遵循图1.4所示的模型。但是,还有其他与安全有关的情形不完全符合该模型,本书也会讨论这些内容,它们的一般模型如图1.5所示,该模型希望保护信息系统不受有害的访问,大多数读者都熟悉黑客引起的问题,黑客试图渗入到通过网络可访问的系统,他可能没有恶意,只是对闯入或进入计算机系统感到满足。入侵者可能是一个不如意的雇员,想进行破坏,或者是一个罪犯,想利用计算机获利(如获取信用卡号或者进行非法的资金转账)。

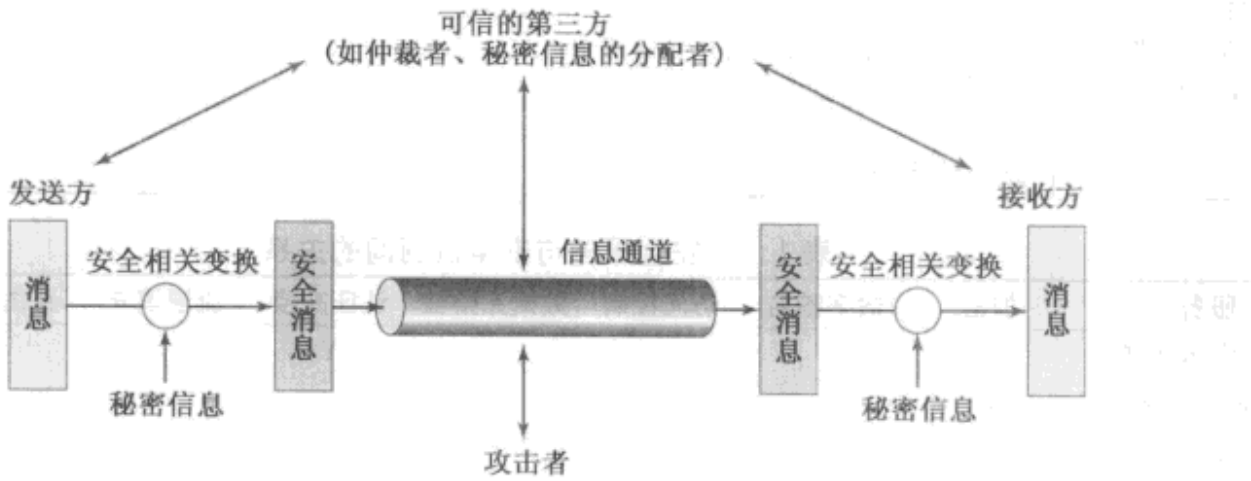


图1.4　网络安全模型

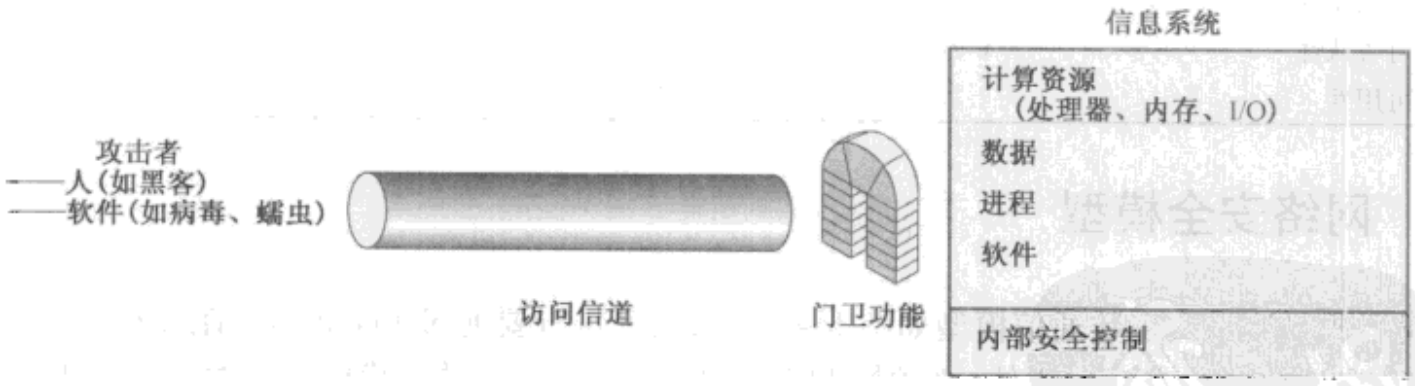


图1.5　网络访问安全模型

另一种类型的有害访问是在计算机系统中加入程序,它利用系统的弱点来影响应用程序和实用功能程序,如编辑程序和编译程序。对程序的威胁有两种:

- **信息访问威胁**。以非授权用户的名义截获或修改数据。
- **服务威胁**。利用计算机中的服务缺陷禁止合法用户使用这些服务。

病毒和蠕虫是两种软件攻击,如果磁盘上的应用软件中隐藏着有害程序,那么这些攻击可通过磁盘进入系统。这些攻击也可以通过网络进入系统。网络安全更关心的是通过网络进入系统的攻击。

对付有害访问所需的安全机制分为两大类(参见图 1.5)。第一类称为看门人功能,它包括基于口令的登录过程,该过程只允许授权用户的访问,还包括监控程序,该程序负责检测和割断蠕虫、病毒以及其他类似的攻击。一旦非法用户或软件获得了访问权,那么由各种内部控制程序组成的第二道防线就监视其活动,分析存储的信息,以便检测非法入侵者。这些问题将会在第六部分讨论。

1.7　推荐读物和网站

[STAL02]提供了计算机和网络安全方面的广泛介绍。[SCHN00]对从事计算机和网络安全的实践者来说是很有价值的读物。该书讨论了技术,特别是密码学,在提供安全方面的局限性。该书认为还需要考虑硬件和软件实现、网络及从事安全与攻击的人员等因素。

读一些有关计算机安全的引论性的论文是有用的。它们提供了一种从历史的观点来理解现在的工作和想法。可以读的论文有[WARE79]、[BROW72]、[SALT75]、[SHAN77]及[SUMM84]。两篇稍近一些的论文[ANDR04]、[LAMP04]则简短地介绍了计算机安全。[NIST95](p. 290)则是该主题大全式的文章。另一篇较好的论文是[NRC91]。[FRAS97]也很有用。

ANDR04 Andrews, M., and Whittaker, J. "Computer Security." *IEEE Security and Privacy* September/October 2004.

BROW72 Browne, P. "Computer Security—A Survey." *ACM SIGMIS Database*, Fall 1972.

FRAS97 Fraser, B. *Site Security Handbook*. RFC 2196, September 1997.

LAMP04 Lampson, B. "Computer Security in the Real World," *Computer*, June 2004.

NIST95 National Institute of Standards and Technology. *An Introduction to Computer Security: The NIST Handbook*. Special Publication 800 – 12, October 1995.

NRC91 National Research Council. *Computers at Risk: Safe Computing in the Information Age*. Washington, D. C.: National Academy Press, 1991.

SALT75 Saltzer, J., and Schroeder, M. "The Protection of Information in Computer Systems." *Proceedings of the IEEE*, September 1975.

SCHN00 Schneier, B. *Secrets and Lies: Digital Security in a Networked World*. New York: Wiley, 2000.

SHAN77 Shanker, K. "The Total Computer Security Problem: An Overview." *Computer*, June 1977.

STAL08 Stallings, W., and Brown, L. *Computer Security*. Upper Saddle River, NJ: Prentice Hall, 2008.

SUMM84 Summers, R. "An Overview of Computer Security." *IBM Systems Journal*, Vol. 23, No. 4, 1984.

WARE79 Ware, W., ed. *Security Controls for Computer Systems*. RAND Report 609 – 1. October 1979. http://www.rand.org/pubs/reports/R609-1/R609.1.html

推荐网站

以下是与密码编码学和网络安全相关的有用站点[①]:

① 因为 URL 时常会变,所以没有包含进来。此处和以后各章中给出的所有 Web 站点都可以在 williamstallings.com/Crypto/Crypto5e.html 上找到合适的链接。

- IETF 安全域(IETF Security Area):与 Internet 安全标准化努力相关的材料。
- 密码学常见问题(Cryptography FAQ):覆盖密码编码学所有领域的较长且较有价值的常见问题。
- Tom Dunigan 的安全主页(Tom Dunigan's Security page):指向密码编码学和网络安全站点的列表。
- Peter Gutmann 的个人主页(Peter Gutmann's home page):收集了很多密码学资料。
- Helgar Lipma 的密码学链接(Helgar Lipma's Cryptology Pointers):又一个关于密码编码学和网络安全站点的链接表。
- 密码学电子出版文档(Cryptology ePrint archive):可以快速访问密码学的近期进展;由大量未经评审的文章组成。
- IEEE 安全保密技术委员会(IEEE Technical Committee on Security and Privacy):含有其新闻和与 IEEE 有关活动信息的副本。
- 计算机安全资源中心(Computer Security Resource Center):由美国国家标准技术局维护;包含大量有关安全威胁、技术和标准的信息。
- 计算机和网络安全参考索引(Computer and Network Security Reference Index):它给出了销售商、商业产品、常见问题、新闻组文档、论文和其他站点的索引。
- 安全聚焦(Security Focus):各类安全信息,重点是零售产品和终端用户关心的东西。
- SANS 协会(SANS Institute):类似于安全聚焦。大量广泛的白皮书。
- 风险摘要(Risks Diegest):面向公众的论坛,有关计算机和相关系统的风险。
- 安全和公开方法论研究机构(Institute for Security and Open Methodologies):一个开发和合作的安全研究社区。有很多有意思的信息。
- Internet 安全中心(Center for Internet Security):就操作系统、网络设备以及应用提供免费的用于安全评估的标准和评分工具。包含个案研究和技术论文。

1.8 关键术语、思考题和习题

关键术语

访问控制	拒绝服务	被动威胁
主动威胁	加密	重播
认证	完整性	安全攻击
真实性	入侵者	安全机制
可用性	伪装	安全服务
数据保密性	非否认性	流量分析
数据完整性	OSI 安全框架	

思考题

1.1 OSI 安全框架是什么?

1.2 被动和主动安全威胁的区别是什么?

1.3 列出并简短地定义被动和主动安全攻击的种类。

1.4 列出并简短地定义安全服务的种类。

1.5 列出并简短地定义安全机制的种类。

习题

1.1　考虑一个自动柜员机(ATM),用户为其提供个人身份码(PIN)和账户访问的卡。给出有关这个系统的安全性、完整性和可用性要求的例子,就每一个例子说明要求的重要性程度。

1.2　以电话交换系统为例重做习题 1.1。电话交换系统根据呼叫用户呼叫的电话号码通过交换网络路由呼叫。

1.3　考虑为各种组织产生文档的桌面发布系统。

(a)给出一种发布类型的例子,此时对存储数据的安全性是最重要的。

(b)给出一种发布类型的例子,此时对存储数据的完整性是最重要的。

(c)给一个例子,其对系统可用性的要求是最重要的。

1.4　对于下面的资产,当发生保密性、可用性和完整性损失时,分别为其产生的影响划分低、中和高等级,并陈述理由。

(a)在自己的网络服务器上管理公开信息的组织。

(b)法律执行组织管理极端敏感的调查信息。

(c)金融机构管理例行的行政信息(非隐私信息)。

(d)合同签订机构的大单收购信息系统,包含敏感、预投标阶段的合同信息和例行的行政信息。请分别评价这两份信息资产的影响情况以及整个信息系统的影响情况。

(e)电厂包含有 SCADA(监管控制和数据获取)系统,该系统负责为军事设施提供电能分发。SCADA 系统包含有实时传感器数据和例行的行政信息。请分别评价这两份信息资产的影响情况以及整个信息系统的影响情况。

1.5　类似于表 1.4,画一个矩阵来表明安全服务和安全攻击的关系。

1.6　类似于表 1.4,画一个矩阵来表明安全机制和安全攻击的关系。

PDG

第一部分　对称密码

第2章　传统加密技术
第3章　分组密码和数据加密标准
第4章　数论和有限域的基本概念
第5章　高级加密标准
第6章　分组密码的工作模式
第7章　伪随机数的产生和流密码

第2章　传统加密技术

2.1　对称密码模型
　2.1.1　密码编码学
　2.1.2　密码分析学和穷举攻击
2.2　代替技术
　2.2.1　Caesar 密码
　2.2.2　单表代替密码
　2.2.3　Playfair 密码
　2.2.4　Hill 密码
　2.2.5　多表代替加密
　2.2.6　一次一密
2.3　置换技术
2.4　转轮机
2.5　隐写术
2.6　推荐读物和网站
2.7　关键术语、思考题和习题

"I am fairly familiar with all the forms of secret writings, and am myself the author of a trifling monograph upon the subject, in which I analyze one hundred and sixty separate ciphers," said Holmes.

—The Adventure of the Dancing Men, Sir Arthur Conan Doyle

要　点

- 对称密码是一种加解密使用相同密钥的密码体制，也称为传统密码。
- 对称密码利用密钥和加密算法将明文变为密文。运用相同的密钥和解密算法，可以从密文恢复出明文。
- 对密码的两种攻击方法分别是基于密码算法性质的密码分析和基于穷举密钥的穷举攻击。
- 传统的对称密码（计算机出现前）使用代替和/或置换技术。代替技术将明文元素（字符、位）映射为密文元素，置换技术将明文元素的位置进行系统的置换。
- 转轮机密码系统是计算机出现前使用代替技术的复杂硬件设备。
- 隐写术是一种将秘密信息隐藏于其他更大信息中的一种技术，使得其他人无法识别它的存在或隐藏信息的内容。

对称加密，也称传统加密或单钥加密，是20世纪70年代公钥密码产生之前唯一的加密类型。迄今为止，它仍是两种加密类型中使用最为广泛的加密类型。第一部分讨论了许多对称密码。本章中，我们首先介绍对称加密过程的一般模型，了解传统加密算法的使用环境。然后，我们讨论计算机出现之前的许多算法。最后，简要地介绍隐写术。第3章和第5章则探讨当今使用最广泛的两种加密算法：DES 和 AES。

首先，我们来定义一些术语。原始的消息为明文，而加密后的消息为密文。从明文到密文的变换过程被称为加密；从密文到明文的变换过程被称为解密。研究各种加密方案的领域被称为密码编码学。这样的加密方案被称为密码体制或密码。不知道任何加密细节的条件下解密消息的技术属于密码分析学的范畴。密码分析学即外行所说的"破译"。密码编码学和密码分析学统称密码学。

2.1　对称密码模型

对称加密方案有5个基本成分（参见图2.1）：

- **明文**：原始可理解的消息或数据，是算法的输入。

- **加密算法**:加密算法对明文进行各种代替和变换。
- **密钥**:密钥也是加密算法的输入。密钥独立于明文和算法。算法根据所用的特定密钥而产生不同的输出。算法所用的确切代替和变换也依靠密钥。
- **密文**:作为算法的输出,看起来完全随机而杂乱的消息,依赖于明文和密钥。对于给定的消息,不同的密钥产生不同的密文,密文看上去是随机的数据流,并且其意义是不可理解的。
- **解密算法**:本质上是加密算法的逆运算。输入密文和密钥,输出原始明文。

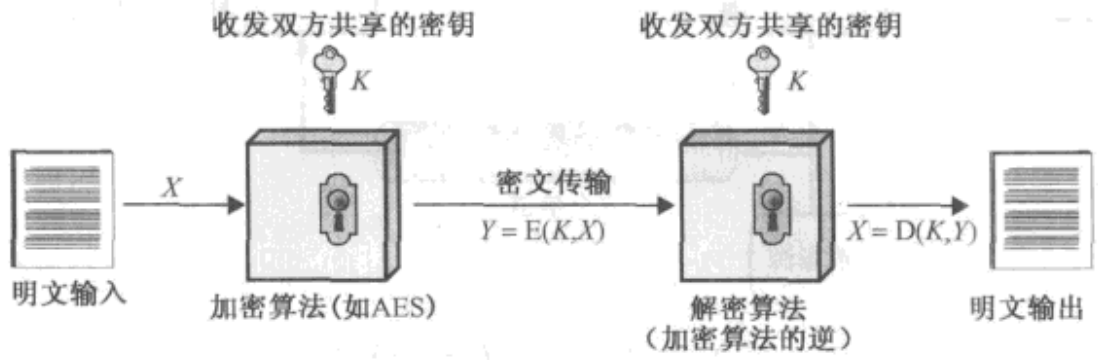


图 2.1　传统密码的简化模型

传统密码的安全使用要满足如下两个要求:

(1) 加密算法必须是足够强的。至少,我们希望这个算法在攻击者知道它并且能够得到一个或者多个密文时,也不能破译密文或计算出密钥。这个要求通常用一种更强的形式表述为:即使攻击者拥有一定数量的密文和产生这些密文的明文,他/她也不能破译密文或发现密钥。

(2) 发送者和接收者必须在某种安全的形式下获得密钥并且必须保证密钥安全。如果有人发现该密钥,而且知道相应的算法,那么就能解读使用该密钥加密的所有通信。

我们假设基于已知密文和已知加密/解密算法而破译消息是不实际的。换句话说,我们并不需要算法保密,仅需要密钥保密。对称密码的这些特点使其能够广泛应用。算法不需要保密这一事实,使得制造商可以开发出低成本的芯片以实现数据加密算法。这些芯片能够广泛地使用,许多产品中都有这种芯片。采用对称密码,首要的安全问题是密钥的保密性。

我们从图 2.2 中可以更清楚地理解对称加密方案的基本成分。发送方产生明文消息 $X=[X_1,X_2,\cdots,X_M]$,X 的 M 个元素是某个字母表中的字母。传统上,字母表由 26 个大写字母组成。而现在,最常用的是二进制字母表{0,1}。加密的时候,先产生一个形如 $K=[K_1,K_2,\cdots,K_J]$ 的密钥。如果密钥是由信息的发送方产生的,那么它要通过某种安全信道发送到接收方;另一种方法是由第三方生成密钥后再安全地分发给发送方和接收方。

加密算法 E 根据输入信息 X 和密钥 K 生成密文 $Y=[Y_1,Y_2,\cdots,Y_N]$,即

$$Y = \mathrm{E}(K,X)$$

该式表明密文 Y 是明文 X 的函数,而具体的函数由密钥 K 值决定。

拥有密钥 K 的预定接收者,可以进行变换

$$X = \mathrm{D}(K,Y)$$

而得到明文。

假设某攻击者窃得 Y 但是并不知道 K 或 X,而企图得到 K 或 X,或 K 和 X。假设他知道加密算法 E 和解密算法 D,如果他只是对这个特定信息感兴趣,那么他将注意力集中在计算明文的估计值 $\hat{X}$ 来恢复 X 上;不过,攻击者往往对进一步的信息同样有兴趣,这种情况下,他企图通过计算密钥的估计值 $\hat{K}$ 来恢复 K。

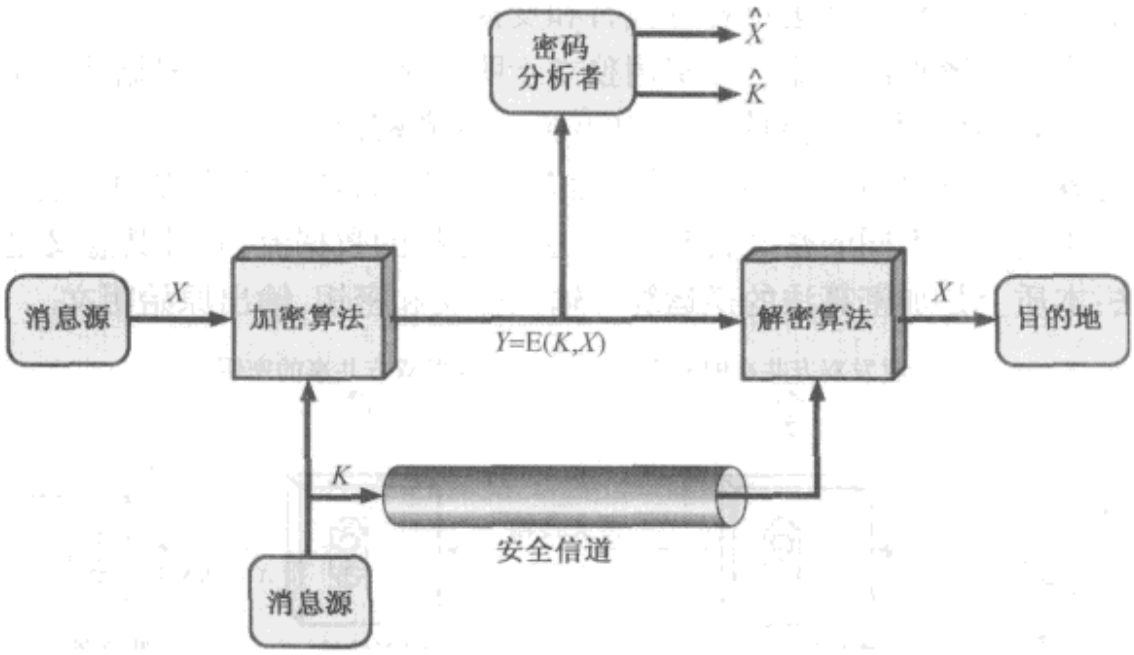


图 2.2 对称密码体制的模型

2.1.1 密码编码学

密码编码学系统具有以下三个独立的特征:

(1) **转换明文为密文的运算类型。**所有的加密算法都基于两个原理:代替和置换。代替是将明文中的每个元素(如位、字母、位组或字母组等)映射成另一个元素;置换是将明文中的元素重新排列。上述运算的基本要求是不允许有信息丢失(即所有的运算是可逆的)。大多数密码体制,也称为乘积密码系统,都使用了多层代替和置换。

(2) **所用的密钥数。**如果发送方和接收方使用相同的密钥,这种密码就称为对称密码、单密钥密码、秘密钥密码或传统密码。如果发收双方使用不同的密钥,这种密码就称为非对称密码、双钥或公钥密码。

(3) **处理明文的方法。**分组密码每次处理输入的一组元素,相应地输出一组元素。流密码则是连续地处理输入元素,每次输出一个元素。

2.1.2 密码分析学和穷举攻击

攻击密码系统的典型目标是恢复使用的密钥,而不是仅仅恢复出单个密文对应的明文。攻击传统的密码体制有两种通用的方法:

- **密码分析学:**密码分析学攻击依赖于算法的性质、明文的一般特征或某些明密文对。这种形式的攻击企图利用算法的特征来推导出特定的明文或使用的密钥。
- **穷举攻击:**攻击者对一条密文尝试所有可能的密钥,直到把它转化为可读的有意义的明文。平均而言,获得成功至少要尝试所有可能密钥的一半。

如果上述任意一种攻击能成功地推导出密钥,那么影响将是灾难性的:将会危及所有未来和过去使用该密钥加密消息的安全。

我们首先考虑密码分析学,然后讨论穷举攻击。

基于密码分析者知道信息的多少,表 2.1 概括了密码攻击的几种类型。表中唯密文攻击难度最大。有些情况下,攻击者甚至不知道加密算法,但是我们通常假设攻击者知道。这种情况下,一种可能的攻击是试遍所有可能密钥的穷举攻击。如果密钥空间非常大,这种方法就不太实际。因此攻击者必须依赖于对密文本身的分析,这一般要运用各种统计方法。使用这种方法,攻击者对

隐含的明文类型必须有所了解，比如说，明文是英文文本、法文文本，或可执行 EXE 文件、Java 源代码清单文件、会计文件等。

表 2.1　对加密信息的攻击类型

攻击类型	密码分析者已知的信息
唯密文攻击	● 加密算法 ● 密文
已知明文攻击	● 加密算法 ● 密文 ● 用（与待解的密文）同一密钥加密的一个或多个明密文对
选择明文攻击	● 加密算法 ● 密文 ● 分析者选择的明文，以及对应的密文（与待解的密文使用同一密钥加密）
选择密文攻击	● 加密算法 ● 密文 ● 分析者选择的一些密文，以及对应的明文（与待解的密文使用同一密钥解密）
选择文本攻击	● 加密算法 ● 密文 ● 分析者选择的明文，以及对应的密文（与待解的密文使用同一密钥加密） ● 分析者选择的一些密文，以及对应的明文（与待解的密文使用同一密钥解密）

唯密文攻击是最容易防范的，因为攻击者拥有的信息量最少。不过在很多情况下，分析者可以得到更多的信息。分析者可以捕获到一段或更多的明文信息及相应密文，也可能知道某段明文信息的格式等。比如，按照 Postscript 格式加密的文件总是以相同的格式开头，还有，电子金融消息往往有标准化的文件头或者标志。类似的例子还有很多。这些都是已知明文攻击的例子。拥有这些知识的分析者就可以从转换明文的方法入手来推导出密钥。

与已知明文攻击紧密相关的是可能词攻击。如果攻击者处理的是一般散文信息，他可能对信息的内容一无所知，但是如果他处理的是一些特定的信息，他就可能知道其中的部分内容。比如说，对于一个完整的会计文件，攻击者可能知道放在文件最前面的是某些关键词。又比如，某公司开发的程序源代码可能含有该公司的版权信息，并且放在某个标准位置。

如果分析者能够通过某种方式获得信源系统，让发送方在发送的信息中插入一段由他选择的信息，那么选择明文攻击就有可能实现。一个例子是差分密码分析，这在第 3 章中将会讲到。一般说来，如果分析者有办法选择明文加密，那么他将特意选取那些最有可能恢复出密钥的数据。

表 2.1 还列举了另外两种类型的攻击方法：选择密文攻击和选择文本攻击。它们在密码分析技术中很少用到，但是仍然是两种可能的攻击方法。

只有相对较弱的算法才抵挡不住唯密文攻击。一般地，加密算法起码要能经受得住已知明文攻击才行。

此外，还有两个概念值得注意。如果一个密码体制满足条件：无论有多少可使用的密文，都不足以唯一地确定密文所对应的明文，则称该加密体制是无条件安全的。也就是说，无论花多少时间，攻击者都无法将密文解密，这仅仅因为他（或她）所需的信息不在密文中。除了一次一密（在以后的章节中将会讲到）之外，所有的加密算法都不是无条件安全的。因此，加密算法的使用者应挑选尽量满足以下标准的算法：

- 破译密码的代价超出密文信息的价值。
- 破译密码的时间超出密文信息的有效生命期。

如果加密体制满足了上述两条标准中的任意一条，则它在计算上是安全的。然而，估计攻击者成功破译密文所需的工作量是非常困难的。

对称密码体制的所有分析方法都利用了这样一个事实:明文的结构和模式在加密之后仍然保存了下来,并能在密文中找到一些蛛丝马迹。本章中,随着对各种对称密码体制讨论的深入,这一点将会变得很显然。在第二部分,我们将会看到对公钥密码体制的分析依据的是一个完全不同的假设,即密钥对的数学性质使得从一个密钥推出另一个密钥成为可能。

试遍所有密钥直到有一个合法的密钥能够把密文还原成明文,这就是穷举攻击。我们可以从这种方法入手,考虑其所需的时间代价。平均地讲,要获得成功,必须尝试所有可能密钥的一半。表 2.2 给出了对于不同密钥空间所耗用的时间,显示了 4 个密钥长度(二进制表示)的结果。DES(数据加密标准)算法使用的是 56 位密钥。3DES 使用的是 168 位密钥。AES(高级加密标准)的最小密钥长度是 128 位。表中最后一行还列出了采用 26 个字母的排列作为密钥的代替密码的一些结果。假设执行一次解密需要 1 μs 的话(这是今天普通计算机的速度),表中数据说明了对于不同长度密钥执行穷尽搜索所需的时间。随着大规模并行计算机的应用,处理速度可能会高出若干数量级。表 2.2 最后一列列举了每微秒能处理一百万个密钥的系统所需的时间。正如你所见,对于这种性能的计算机,DES 算法不再是计算上安全的。

表 2.2 穷尽密钥空间所需的平均时间

密钥大小(位)	密钥个数	每微秒执行一次解密所需的时间	每微秒执行一百万次解密所需的时间
32	$2^{32}=4.3\times10^{9}$	$2^{31}\mu s=35.8$ min	2.15 ms
56	$2^{56}=7.2\times10^{16}$	$2^{55}\mu s=1142$ 年	10.01 小时
128	$2^{128}=3.4\times10^{38}$	$2^{127}\mu s=5.4\times10^{24}$ 年	5.4×10^{8} 年
168	$2^{168}=3.7\times10^{50}$	$2^{167}\mu s=5.9\times10^{36}$ 年	5.9×10^{30} 年
26 个字母的排列组合	$26!=4\times10^{26}$	$2\times10^{26}\mu s=6.4\times10^{12}$ 年	6.4×10^{5} 年

2.2 代替技术

在本节和下一节中,我们将举例探讨古典加密方法。研究这些方法可以使我们弄清楚今天所用的对称密码的一些基本方法,以及随之而来的密码攻击的类型等。

首先我们讨论一下所有加密技术都要用到的两个基本模块:代替和置换,我们将在接下来的两节中讨论这些内容。最后我们将讨论将两种技巧综合应用的密码系统。

代替技术是将明文字母替换成其他字母、数字或符号的方法①。如果把明文视为二进制序列的话,那么代替就是用密文位串来代替明文位串。

2.2.1 Caesar 密码

已知最早的代替密码是由 Julius Caesar 发明的 Caesar。它非常简单,就是对字母表中的每个字母,用它之后的第三个字母来代替。例如:

明文:meet me after the toga party

密文:PHHW PH DIWHV WKH WRJD SDVWB

注意到字母表是循环的,即认为紧随 Z 后的是字母 A。我们通过列出所有的字母来定义如下变换:

明文:a b c d e f g h i j k l m n o p q r s t u v w x y z

密文:D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

① 当涉及字母时,本书约定:用小写字母表示明文,大写字母表示密文,斜体小写字母表示密钥。

如果我们让每个字母等价于一个数值：

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

那么加密算法可以如下表达。对每个明文字母 p，代替成密文字母 C[①]：

$$C = \mathrm{E}(3, p) = (p + 3) \bmod 26$$

移位可以是任意整数 k，这样就得到了一般的 Caesar 算法：

$$C = \mathrm{E}(k, p) = (p + k) \bmod 26 \tag{2.1}$$

这里 k 的取值范围是从 1 到 25。解密算法是

$$p = \mathrm{D}(k, C) = (C - k) \bmod 26 \tag{2.2}$$

如果已知某给定的密文是 Caesar 密码，那么穷举攻击是很容易实现的：只要简单地测试所有的 25 种可能的密钥。图 2.3 给出了应用这种策略解密的结果。对于这个例子，显然明文出现在第三行。

```
       PHHW PH DIWHU WKH WRJD SDUWB
KEY
 1     oggv og chvgt vjg vqic rctva
 2     nffu nf bgufs uif uphb qbsuz
 3     meet me after the toga party
 4     ldds ld zesdq sgd snfz ozqsx
 5     kccr kc ydrcp rfc rmey nyprw
 6     jbbq jb xcqbo qeb qldx mxoqv
 7     iaap ia wbpan pda pkcw lwnpu
 8     hzzo hz vaozm ocz ojbv kvmot
 9     gyyn gy uznyl nby niau julns
10     fxxm fx tymxk max mhzt itkmr
11     ewwl ew sxlwj lzw lgys hsjlq
12     dvvk dv rwkvi kyv kfxr grikp
13     cuuj cu qvjuh jxu jewq fqhjo
14     btti bt puitg iwt idvp epgin
15     assh as othsf hvs hcuo dofhm
16     zrrg zr nsgre gur gbtn cnegl
17     yqqf yq mrfqd ftq fasm bmdfk
18     xppe xp lqepc esp ezrl alcej
19     wood wo kpdob dro dyqk zkbdi
20     vnnc vn jocna cqn cxpj yjach
21     ummb um inbmz bpm bwoi xizbg
22     tlla tl hmaly aol avnh whyaf
23     skkz sk glzkx znk zumg vgxze
24     rjjy rj fkyjw ymj ytlf ufwyd
25     qiix qi ejxiv xli xske tevxc
```

图 2.3　对 Caesar 密码的穷举密码学分析

① 定义 $a \bmod n$ 为 a 除 n 的余数。例如，$11 \bmod 7 = 4$。关于模算术的更多讨论，请参阅第 4 章。

Caesar 密码的三个重要特征使我们可以采用穷举攻击分析方法:

(1) 已知加密和解密算法。

(2) 需测试的密钥只有 25 个。

(3) 明文所用的语言是已知的,且其意义易于识别。

在大多数网络情况下,假设密码算法是已知的。一般说来,密钥空间很大的算法使得穷举攻击分析方法不太可能。例如第 6 章将介绍的 3DES 算法,它的密钥长度是 168 位,其密钥空间是 2^{168},或者说有大于 3.7×10^{50}种可能的密钥。

上述的第三个特征也是非常重要的,如果明文所用的语言不为我们所知,那么明文输出就不可识别。而且,输入可能按某种方式经过缩写或压缩,也就更不可识别了。例如,图 2.4 给出的是经过 ZIP 压缩之后的部分文本文件。如果这个文件用一种简单的代替密码来加密(将字母集合扩充为不只包含 26 个英文字母),那么即使用穷举攻击进行密码分析,恢复出来的明文也是不可识别的。

```
˜+Wμ"— Ω–O)≤4{∞‡ ë~Ω%ràu·¯Í ◊¯Z-
Ú≠2Ò#Åæð œ«q7 ΩnÒ®3N◊Ú Œz'Y-ƒ∞Í[±Û_ èΩ,<NO¬±«˘xã Åäfèü3Å
x}ö§k°Â
_yÍ ^ΔÉ] ¤ J/˙iTê&ı 'c<uΩ-
 ÄD(G WÄC~y_ïõÄW PÔı«ÎÜ†ç],¤¡˘Ì^üÑπ˘≈˘L˘9OgflO˘&Œ≤ ¬≤ ØÔ§˝:
 ˘Œ!SGqèvo^ ú\¸S>h<-*6ø‡%x´˝|fiÓ#≈˜my‰˘≥ñP<,fi Áj Å◊¿˝Zù-
Ω¨Õ¯6Œÿ{% „ΩÊó ¸ï π÷ÁÎ˚úO2çSŸ´O-
2ÄflßiÍ /@^˝∏K°•PŒπ¸úé^´3∑˜ö˘ÔZÌ"Y¬ŸΩœY> Ω+eô/˙<K£¿*÷˜"≤û˜
B ZøK˜Qßÿüƒ¸!ÒflÎzsS/]>ÈQ ü
```

图 2.4 经过压缩的文本样本

2.2.2 单表代替密码

Caesar 密码仅有 25 种可能的密钥,是远不够安全的。通过允许任意代替,密钥空间将会急剧增大。在继续之前,我们先定义术语置换。有限元素的集合 S 的置换是 S 的所有元素的有序排列,且每个元素只出现一次。例如,如果 $S = \{a, b, c\}$,则 S 有 6 个置换:

abc, acb, bac, bca, cab, cba

一般,具有 n 个元素的集合有 $n!$ 个置换,因为第一个元素有 n 种选择方式,第二个元素有 $n-1$ 种方式,第三个元素有 $n-2$ 种方式,以此类推。

回忆 Caesar 密码的对应:

明文:abcdefghijklmnopqrstuvwxyz

密文:DEFGHIJKLMNOPQRSTUVWXYZABC

如果密文行是 26 个字母的任意置换,那么就有 26! 或大于 4×10^{26}种可能的密钥,这比 DES 的密钥空间要大 10 个数量级,好像可以抵挡穷举攻击了。这种方法被称为单表代替密码,这是因为每条消息用一个字母表(给出从明文字母到密文字母的映射)加密。

不过,攻击办法仍然存在。如果密码分析者知道明文(例如,未经压缩的英文文本)的属性,他就可以利用语言的一些规律进行攻击。为了说明分析过程,我们在这里给出一段从参考文献[SINK66]中摘选出来的例子。需要解密的密文是

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
```

首先把字母使用的相对频率统计出来,与英文字母的使用频率分布进行比较,参见图 2.5(来

自参考文献[LEWAOO])。如果已知消息足够长的话,只用这种方法就已经足够了,但是如果这段消息相对较短,我们就不能得到准确的字母匹配。密文中字母的相对频率(按百分比)如下:

P	13.33	H	5.00	F	3.33	B	1.67	C	0.00
Z	11.67	D	5.83	W	3.33	G	1.67	K	0.00
S	8.33	E	5.00	Q	2.50	Y	1.67	L	0.00
U	8.33	V	4.17	T	2.50	I	0.83	N	0.00
O	7.55	X	4.17	A	1.67	J	0.83	R	0.00
M	6.67								

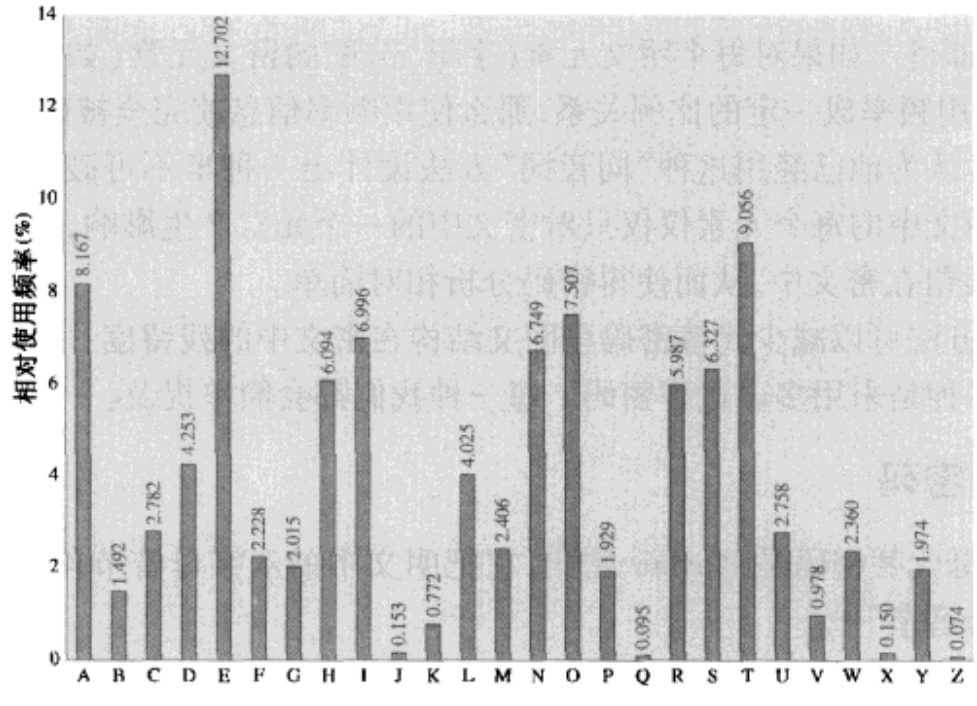


图 2.5　英文字母的相对频率

将这种统计规律与图 2.5 比较,可以得出结论:密文字母中的 P 和 Z 可能相当于明文中的 e 和 t,但是并不能确定是 P 对应 e(Z 对应 t)还是 Z 对应 e(P 对应 t)。密文中的 S、U、O、M 和 H 的相对频率都比较高,可能与明文中的字母{a,h,i,n,o,r,s}中的某一个相对应。相对频率较低的字母(即 A、B、G、Y、I、J)可能对应着明文字母集{b,j,k,q,v,x,z}中的某个元素。

此时我们可以从以下几种方法入手。我们可以尝试着做一些代替,填入明文,看看是否像一个消息的轮廓。更系统一点的方法是寻找其他的规律。例如,明文中有某些词可能是已知的,或者寻找密文字母中的重复序列,推导它们的等价明文。

统计双字母组合的频率是一个很有效的工具。由此可以得到一个类似于图 2.5 的双字母组合的相对频率图。最常用的一个字母组合是 th。而在我们的密文中,用得最多的双字母组合是 ZW,它出现了三次。所以我们可以估计 Z 对应明文 t,而 W 对应明文 h。根据先前的假设,我们可以认为 P 对应 e。现在我们意识到密文中的 ZWP 很可能就是 the,这是英语中最常用的三字母组合,这表明我们的思路是正确的。

接下来注意到第一行中的序列 ZWSZ。我们并不知道它是否为一个完整的单词,若是这样的话,它应该翻译成 th_t 的形式,如果是这样,则 S 很可能是明文 a。

至此我们有以下结果:

```
UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZ
 t a        e  e te  a that e e a       a
VUEPHZHMDZSHZOWSFPAPPDTSVPQUZWYMXUZUHSX
   e t   ta t ha e ee  a e  th    t  a
EPYEPOPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ
 e  e e tat  e   the   t
```

虽然只确定了4个字母,但是我们已经有眉目了。继续进行类似的分析、测试,很快就可以得出完整的明文,加上空格后如下:

```
it was disclosed yesterday that several informal but
direct contacts have been made with political
representatives of the viet cong in moscow
```

单表代替密码较容易被攻破,因为它带有原始字母使用频率的一些统计学特征。一种对策是对每个字母提供多种代替,称为同音词(就像一个读音可以代表多个单词的同音词一样),一个明文单元也可以变换成不同的密文单元。比如字母 e 可以替换成 16,74,35 和 21,循环或随机地选取其中一个同音词即可。如果对每个明文元素(字母)分配的密文元素(如数字等)的个数与此明文元素(字母)的使用频率成一定的比例关系,那么使用频率信息就完全被破坏了。伟大的数学家 Carl Friedrich Gauss 认为他已经用这种"同音词"方法设计出一种牢不可破的密码。然而,即使采用了同音词方法,明文中的每个元素仅仅只对密文中的一个元素产生影响,多字母语法模式(比如双字母音节)仍然残留在密文中,从而使得密码分析相对简单。

有两种主要的方法可以减少代替密码里明文结构在密文中的残留度:一种是对明文中的多个字母一起加密,另一种是采用多表代替密码。每一种我们都会简单提及。

2.2.3 Playfair 密码

最著名的多字母代替密码是 Playfair 密码,它把明文中的双字母音节作为一个单元并将其转换成密文的"双字母音节"①。

Playfair 算法基于一个由密钥词构成的 5×5 字母矩阵。下面的例子由 Lord Peter Wimsey 在 Dorothy Sayers 所著 *Have His Carcase*② 一书中给出:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

本例使用的密钥词是 monarchy。填充矩阵的方法是:首先将密钥词(去掉重复字母)从左至右、从上至下填在矩阵格子中,再将剩余的字母按字母表的顺序从左至右、从上至下填在矩阵剩下的格子里。字母 I 和 J 暂且当做一个字母。对明文按如下规则一次加密两个字母:

(1) 如果该字母对的两个字母是相同的,那么在它们之间加一个填充字母,比如 x。例如先把 balloon 变成 ba lx lo on 这样的 4 个字母对。

(2) 落在矩阵同一行的明文字母对中的字母,由其右边的字母来代替,每行中最右边的一个字母就用该列中最左边的第一个字母来代替,比如 ar 变成 RM。右移

(3) 落在矩阵同一列的明文字母对中的字母,由其下面的字母来代替,每行中最下面的一个字母就用该列中最上面的第一个字母来代替,比如 mu 变成 CM。下移

(4) 其他的每组明文字母对中的字母按如下方式代替:该字母所在行为密文所在行,另一字母所在列为密文所在列。比如 hs 变成 BP,ea 变成 IM(或 JM)。对角法

---

① 这个密码实际上是由英国科学家 Charles Wheatstone 于 1854 年发明的,但是却挂着他的朋友 Baron Playfair 的名字。在英国的外事机构中,Baron Playfair 在密码方面的成就是首屈一指的。

② 本书对可能词攻击的描述引人入胜。

Playfair 密码相对于简单的单表密码是一个巨大的进步。首先，尽管只有 26 个字母，但有 26 ×26 =676 个字母对，因此识别出单个字母对要困难得多。而且，单个字母的相对频率比字母对的相对频率变化的幅度大，在统计规律上要好，这样利用频率分析字母对就更困难一些。因为这些原因，Playfair 密码在很长一段时间内被认为是牢不可破的。第一次世界大战中，英军就使用它作为陆军的战时加密体制，并且在第二次世界大战中，美军及其他一些盟国军队仍在大量使用。

尽管 Playfair 密码被认为是较安全的，它仍然是相对容易攻破的，因为它的密文仍然完好地保留了明文语言的大部分结构特征。几百个字母的密文就足够我们分析出规律了。

图 2.6 显示了 Playfair 密码和其他一些密码加密的有效性（来自参考文献[SIMM93]）。标有“明文”的曲线画出了超过 70 000 个字母的频率分布①，这些字母来自《大不列颠百科全书》中关于密码学的文章。这也是任意单表代替密码的频率分布，因为单字母的频率值是相同的，不同的是对原始字母用了不同的字母进行代替。曲线代表这样的含义：对文章中出现的每个字母计数，计数结果除以使用频率最高的字母 e 的出现次数。设 e 出现的频率为 1，那么 t 就大约为 0.76，等等。水平轴上的点对应着按使用频率递降的字母。

图 2.6 也表明了使用 Playfair 密码加密后文本的字母频率分布情况。为了便于比较，密文中的每个字母的使用频率仍然除以明文中 e 的使用频率。因此，（加密后的）曲线体现了加密后字母频率分布被掩盖的程度，而按字母的频率分布来分析代替密码是一种简单的方法。如果频率分布的信息完全被加密过程给隐藏了，那么密文的频率曲线应该是一条水平的线，唯密文密码分析由此下手将一无所获。图中所示表明 Playfair 密码虽然有比明文稍平坦的频率分布曲线，但是仍然透露了大量的信息给密码分析者。

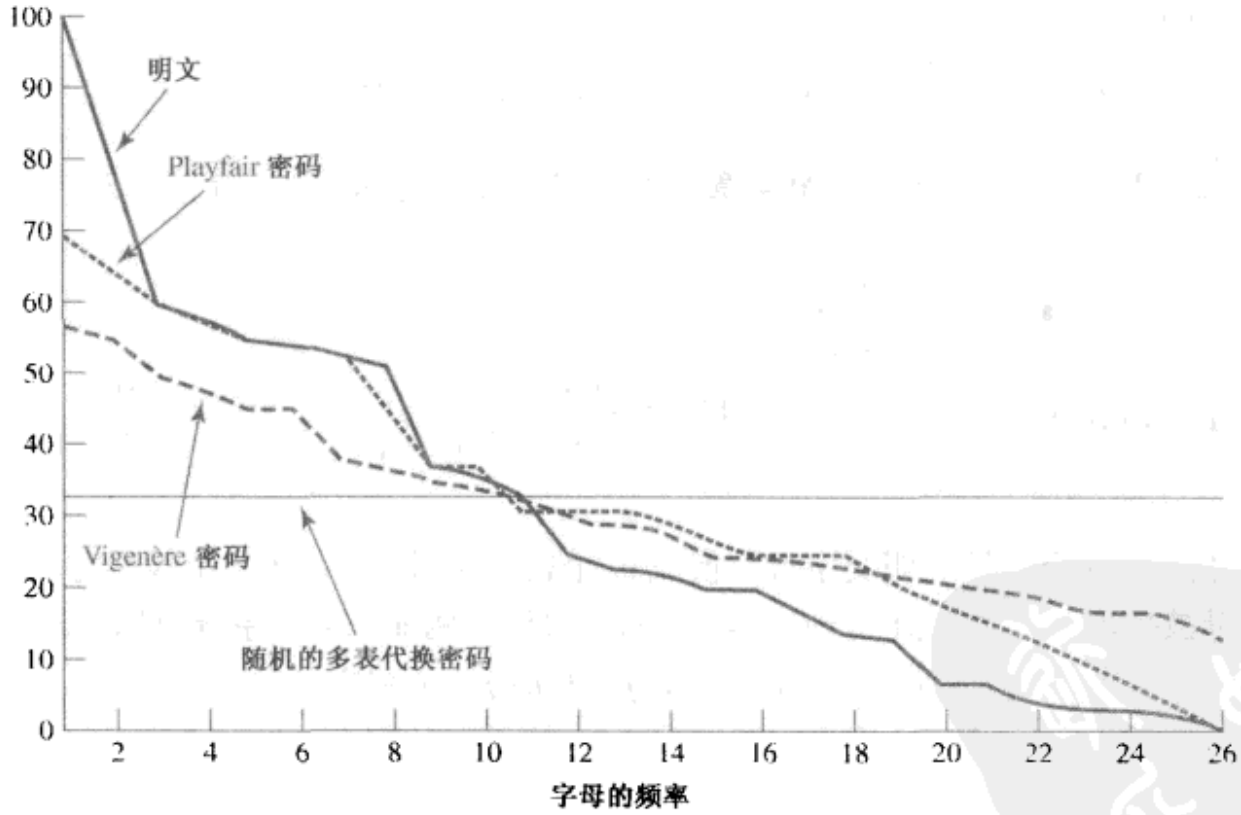


图 2.6　字母出现的相对频率

2.2.4 Hill 密码②

另一个有趣的多表代替密码是 Hill 密码，由数学家 Lester Hill 在 1929 年发明。

① 我很感激 Gustavus Simmons 提供这些曲线并解释它们的构造方法。

② 该密码理解起来比本章的其他密码稍微困难一些，但它阐明了密码分析学里有用的一些要点。初学者可以跳过这一节。

线性代数中的一些概念

在描述 Hill 密码前,让我们先简短地复习一下线性代数中的一些术语。现在我们关心模 26 的矩阵算术。对于需要补习矩阵乘法和逆运算的读者,请参考附录 E。

我们定义方阵 $\boldsymbol{M}$ 的逆矩阵 $\boldsymbol{M}^{-1}$ 应满足 $\boldsymbol{M}(\boldsymbol{M}^{-1}) = (\boldsymbol{M}^{-1})\boldsymbol{M} = \boldsymbol{I}$,其中 $\boldsymbol{I}$ 为单位矩阵。$\boldsymbol{I}$ 矩阵是除了主对角线上的元素为 1 外其余全为 0 的矩阵。一个矩阵的逆矩阵并不总是存在,但当存在的时候,它满足前述的方程。例如

$$\boldsymbol{A} = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} \qquad \boldsymbol{A}^{-1} \bmod 26 = \begin{bmatrix} 9 & 2 \\ 1 & 15 \end{bmatrix}$$

$$\boldsymbol{A}\boldsymbol{A}^{-1} = \begin{bmatrix} (5 \times 9) + (8 \times 1) & (5 \times 2) + (8 \times 15) \\ (17 \times 9) + (3 \times 1) & (17 \times 2) + (3 \times 15) \end{bmatrix}$$

$$= \begin{bmatrix} 53 & 130 \\ 156 & 79 \end{bmatrix} \bmod 26 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

为了解释如何计算矩阵的逆矩阵,我们先介绍行列式的概念。对于任意一个方阵 $m \times m$,其行列式的值等于不同行、不同列上的元素的乘积的代数和,部分项的前面有负号。对于 2×2 的矩阵 $\begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$,行列式是 $k_{11}k_{22} - k_{12}k_{21}$。对于 3×3 的矩阵,行列式的值为 $k_{11}k_{22}k_{33} + k_{21}k_{32}k_{13} + k_{31}k_{12}k_{23} - k_{31}k_{22}k_{13} - k_{21}k_{12}k_{33} - k_{11}k_{32}k_{23}$。如果矩阵 $\boldsymbol{A}$ 有非零的行列式,则这个矩阵的逆如下计算:$[\boldsymbol{A}^{-1}]_{ij} = (\det \boldsymbol{A})^{-1}(-1)^{i+j}(D_{ji})$,其中 (D_{ji}) 是将矩阵 $\boldsymbol{A}$ 去掉第 j 行和第 i 列后的子行列式的值,$\det(\boldsymbol{A})$ 是 $\boldsymbol{A}$ 的行列式,而 $(\det \boldsymbol{A})^{-1}$ 是 $\det(\boldsymbol{A}) \bmod 26$ 的逆。

继续我们刚才的例子:

$$\det \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix} = (5 \times 3) - (8 \times 17) = -121 \bmod 26 = 9$$

因为 $9 \times 3 = 27 \bmod 26 = 1$(参考第 4 章或附录 E),我们得到 $9^{-1} \bmod 26 = 3$。因此,我们如下计算 $\boldsymbol{A}$ 的逆:

$$\boldsymbol{A} = \begin{bmatrix} 5 & 8 \\ 17 & 3 \end{bmatrix}$$

$$\boldsymbol{A}^{-1} \bmod 26 = 3 \times \begin{bmatrix} 3 & -8 \\ -17 & 5 \end{bmatrix} = 3 \times \begin{bmatrix} 3 & 18 \\ 9 & 5 \end{bmatrix} = \begin{bmatrix} 9 & 54 \\ 27 & 15 \end{bmatrix} = \begin{bmatrix} 9 & 2 \\ 1 & 15 \end{bmatrix}$$

Hill 算法

该加密算法将 m 个连续的明文字母替换成 m 个密文字母,这是由 m 个线性方程决定的,在方程中每个字母被指定为一个数值($a = 0, b = 1, \cdots, z = 25$)。例如 $m = 3$,系统可以描述为

$$c_1 = (k_{11}p_1 + k_{12}p_2 + k_{13}p_3) \bmod 26$$
$$c_2 = (k_{21}p_1 + k_{22}p_2 + k_{23}p_3) \bmod 26$$
$$c_3 = (k_{31}p_1 + k_{32}p_2 + k_{33}p_3) \bmod 26$$

用行向量和矩阵表示如下①:

$$[c_1\ c_2\ c_3] = [p\ p_2\ p_3] \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix} \bmod 26$$

① 一些密码书将明文和密文表达为列向量,以使得列向量放在矩阵的后面而不是像行向量放在矩阵的前面。Sage 使用了行向量,所以我们也采用这个习惯。

或

$$\boldsymbol{C} = \boldsymbol{PK} \bmod 26$$

这里 $\boldsymbol{C}$ 和 $\boldsymbol{P}$ 是长度为 3 的行向量，分别代表密文和明文，$\boldsymbol{K}$ 是一个 3 × 3 矩阵，代表加密密钥。运算按模 26 执行。

例如，明文为“paymoremoney”，加密密钥为

$$\boldsymbol{K} = \begin{bmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{bmatrix}$$

明文的前面 3 个字母用向量(15　0　24)表示，那么 (15　0　24)$\boldsymbol{K}$ = (303　303　531) mod 26 = (17　17　11) = RRL 照此方式转换余下字母，可得整段明文对应的密文是 RRLMWBKASPDH。

解密则需要用到矩阵 $\boldsymbol{K}$ 的逆。我们可以计算 det $\boldsymbol{K}$ = 23，所以(det $\boldsymbol{K}$)$^{-1}$ mod 26 = 17。计算逆矩阵为

$$\boldsymbol{K}^{-1} = \begin{bmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{bmatrix}$$

验证如下：

$$\begin{bmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{bmatrix} \begin{bmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{bmatrix} = \begin{bmatrix} 443 & 442 & 442 \\ 858 & 495 & 780 \\ 494 & 52 & 365 \end{bmatrix} \bmod 26 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

很显然，如果把逆矩阵 $\boldsymbol{K}^{-1}$ 应用到密文上，则可以恢复明文。

用一般术语，Hill 密码系统可以表示如下：

$$\boldsymbol{C} = \mathrm{E}(\boldsymbol{K}, \boldsymbol{P}) = \boldsymbol{PK} \bmod 26$$
$$\boldsymbol{P} = \mathrm{D}(\boldsymbol{K}, \boldsymbol{C}) = \boldsymbol{CK}^{-1} \bmod 26 = \boldsymbol{PKK}^{-1} = \boldsymbol{P}$$

同 Playfair 密码相比，Hill 密码的优点是完全隐蔽了单字母频率特性。实际上，Hill 用的矩阵越大，所隐藏的频率信息就越多。因此，一个 3 × 3 的 Hill 密码不仅隐藏了单字母的频率特性，还隐藏了双字母的频率特性。

尽管 Hill 密码足以抗唯密文攻击，但是它较易被已知明文攻击破解。对于一个 $m \times m$ 的 Hill 密码，假如我们有 m 个明密文对，每个长度都是 m，定义 $\boldsymbol{P}_j = (p_{1j}\ \ p_{2j} \cdots\ p_{mj})$ 和 $\boldsymbol{C}_j = (c_{1j} c_{2j} \cdots\ c_{mj})$，使得对每个 $\boldsymbol{C}_j$ 和 $\boldsymbol{P}_j (1 \leqslant j \leqslant m)$ 都有 $\boldsymbol{C}_j = \boldsymbol{KP}_j$，其中 $\boldsymbol{K}$ 是未知的矩阵形密钥。现在定义两个 $m \times m$ 的矩阵 $\boldsymbol{X} = (p_{ij})$ 和 $\boldsymbol{Y} = (c_{ij})$。那么我们可以得出矩阵等式 $\boldsymbol{Y} = \boldsymbol{XK}$，若 $\boldsymbol{X}$ 可逆，则可得 $\boldsymbol{K} = \boldsymbol{X}^{-1}\boldsymbol{Y}$。若 $\boldsymbol{X}$ 不可逆，那么我们可以另找 $\boldsymbol{X}$ 和对应的 $\boldsymbol{Y}$，直至得到一个可逆的 $\boldsymbol{X}$。

考虑如下例子。假设明文“hillcipher”经过一个 2 × 2 的 Hill 密码加密生成密文 HCRZSSXNSP。因此我们知道(7　8) $\boldsymbol{K}$ mod 26 = (7　2)，(11　11) $\boldsymbol{K}$ mod 26 = (17　25)，以此类推。那么由前两个明、密文对，可得

$$\begin{bmatrix} 7 & 2 \\ 17 & 25 \end{bmatrix} = \begin{bmatrix} 7 & 8 \\ 11 & 11 \end{bmatrix} \boldsymbol{K} \bmod 26$$

$\boldsymbol{X}$ 的逆是

$$\begin{bmatrix} 7 & 8 \\ 11 & 11 \end{bmatrix}^{-1} = \begin{bmatrix} 25 & 22 \\ 1 & 23 \end{bmatrix}$$

因此

$$\boldsymbol{K} = \begin{bmatrix} 25 & 22 \\ 1 & 23 \end{bmatrix} \begin{bmatrix} 7 & 2 \\ 17 & 25 \end{bmatrix} = \begin{bmatrix} 549 & 600 \\ 398 & 577 \end{bmatrix} \bmod 26 = \begin{bmatrix} 3 & 2 \\ 8 & 5 \end{bmatrix}$$

该结果可以由剩下的明密文对来验证。

2.2.5 多表代替加密

对简单单表代替的改进方法是在明文消息中采用不同的单表代替。这种方法一般称之为多表代替密码。所有这些方法都有以下的共同特征：

(1) 采用相关的单表代替规则集。

(2) 密钥决定给定变换的具体规则。

transformation

Vigenère 密码 Virginia each letter have a shif rule / consecutive linear

多表代替密码中最著名和最简单的是 Vigenère 密码。它的代替规则集由 26 个 Caesar 密码的代替表组成,其中每一个代替表是对明文字母表移位 0 到 25 次后得到的代替单表。每个密码由一个密钥字母来表示,这个密钥字母用来代替明文字母 a,故移位 3 次的 Caesar 密码由密钥值 d 来代表。

我们可用如下方式来表述 Vigenère 密码。假设明文序列为 $P = p_0, p_1, p_2, \cdots, p_{n-1}$,密钥由序列 $K = k_0, k_1, k_2, \cdots, k_{m-1}$ 构成,其中典型的是 $m < n$。密码序列 $C = C_0, C_1, C_2, \cdots, C_{n-1}$ 计算如下:

$$\begin{aligned} C = C_0, C_1, C_2, \cdots, C_{n-1} &= \mathrm{E}(K, P) = \mathrm{E}[(k_0, k_1, k_2, \cdots, k_{m-1}), (p_0, p_1, p_2, \cdots, p_{n-1})] \\ &= (p_0 + k_0) \bmod 26, (p_1 + k_1) \bmod 26, \cdots, (p_{m-1} + k_{m-1}) \bmod 26, \\ &\quad (p_m + k_0) \bmod 26, (p_{m+1} + k_1) \bmod 26, \cdots, (p_{2m-1} + k_{m-1}) \bmod 26, \cdots \end{aligned}$$

因此,密钥的第一个字母模 26 加到了明文的第一个字母,接着是第二个字母,以此类推,直到前 m 个明文处理完毕。对于第二组的 m 个明文,重复使用密钥字母。继续该过程,直到所有的明文序列都被加密完。加密过程的一般方程是

$$C_i = (p_i + k_{i \bmod m}) \bmod 26 \tag{2.3}$$

该式和 Caesar 密码的方程式(2.1)类似。本质上,每一个明文字母根据相应的密钥字母用不同的 Caesar 密码加密。类似地,解密是方程(2.2)的推广:

$$p_i = (C_i - k_{i \bmod m}) \bmod 26 \tag{2.4}$$

加密一条消息需要与消息一样长的密钥。通常,密钥是一个密钥词的重复,比如密钥词是 deceptive,那么消息“we are discovered save yourself”将被这样加密:

密钥:*deceptivedeceptivedeceptive*

明文:wearediscoveredsaveyourself

密文:ZICVTWQNGRZGVTWAVZHCQYGLMGJ

用数字来表述,我们有如下结果。

密码	3	4	2	4	15	19	8	21	4	3	4	2	4	15
明文	22	4	0	17	4	3	8	18	2	14	21	4	17	4
密文	25	8	2	21	19	22	16	13	6	17	25	6	21	19

密钥	19	8	21	4	3	4	2	4	15	19	8	21	4
明文	3	18	0	21	4	24	14	20	17	18	4	11	5
密文	22	0	21	25	7	2	16	24	6	11	12	6	9

这种密码的强度在于每个明文字母对应着多个密文字母,且每个密文字母使用唯一的密钥字母,因此字母出现的频率信息被隐蔽了,不过并非所有的明文结构信息都隐蔽了。例如,图 2.6 给出

了 9 个密钥词的 Vigenère 密码频率分布特征。尽管它对于 Playfair 密码是一个较大的改进，却依然保留了许多频率信息。

略述这种密码的攻击方法很有益，因为它体现了密码分析学中的一些数学原理。

首先，假设敌手认为密文是用单表代替或 Vigenère 密码来加密的。可以用一个简单的测试来进行区分。如果用单表代替，那么密文的统计特性应该与明文语言的统计特性相同，因此参照图 2.5，应该有一个密文字母的出现频率大约是 12.7%，一个大约是 9.06%，等等。如果只有一条消息可用于密码分析，那么我们并不期望它所体现出来的统计规律与明文的统计规律刚好吻合。然而当它们的统计规律很接近时，我们就认为是用单表代替加密的。

另一方面，如果认为是用 Vigenère 密码加密的，破译能否取得进展将取决于能否判定密钥词的长度，我们很快就会意识到这一点。现在集中精力寻找密钥词长度。能够导致答案的重要观察是：如果两个相同的明文序列之间的距离是密钥词长度的整数倍，那么产生的密文序列也是相同的。在前面的例子中，red 的两次出现相隔 9 个字母，在两种情况下，r 都是用 e 加密，e 都是用 p 加密，d 都是用 t 加密，因此得到了两个相同的密文序列 VTW。我们对相关的密文字母做了下画线处理，对相关的密文数字用阴影处理，以指示上述说明。

分析者只需发现重复序列 VTW，而重复的 VTW 之间相隔 9 个字符，那么他（她）推断密钥词的长度是 3 或 9。VTW 的两次出现也可能是偶然的，而不一定是用相同密钥加密相同明文序列导致的。然而如果信息长度足够大，就会有大量重复的密文序列出现。通过计算重复密文序列间距的公因子，分析者就很有可能猜出密钥词的长度。

破解密码也依靠另一个重要的观察。如果密钥词的长度是 m，那么密码实际上包含了 m 个单表代替。例如，以 DECEPTIVE 作为密钥词，那么处在位置 1，10，19，…的字母的加密实际上是单表密码。因此，我们可以用明文语言的频率特征对这样的单表密码分别进行攻击。

密钥词的周期性可以用与明文信息一样长的不重复密钥词来消除。Vigenère 提出用一个所谓的密钥自动生成系统，它将密钥词和明文自身连接来生成不重复的密钥词。请看下面的例子：

密钥：*deceptivewearediscoveredsav*
明文：wearediscoveredsaveyourself
密文：ZICVTWQNGKZEIIGASXSTSLVVWLA

即使采用这个方案，它也是易受攻击的。因为密钥和明文具有相同频率的分布特征，所以我们可以应用统计学的方法。例如，用 e 加密 e，由图 2.5 可以估计出其发生的概率为 $(0.127)^2 \approx 0.016$，而用 t 加密 t 发生的概率大概只有它的一半，等等。密码分析者利用这些规律能够成功地进行分析[①]。

Vernam 密码

最终的反破译措施也许只有选择一个与明文毫无统计关系，且与它一样长的密钥。1918 年 AT&T 公司的工程师 Gilbert Vernam 首先引入了这种体制，其运算基于二进制数据而非字母。该体制（参见图 2.7）可以简明地表述为

$$c_i = p_i \oplus k_i$$

其中，p_i 是明文的第 i 个二进制位；k_i 是密钥的第 i 个二进制位；c_i 是密文的第 i 个二进制位；⊕是异或运算符。

① 虽然破译 Vigenère 密码的技术并不复杂，但是 1917 年的一期《科学美国人》杂志上却称之为不可破的。当对现代密码算法做出类似的论断时，这是值得汲取的教训。

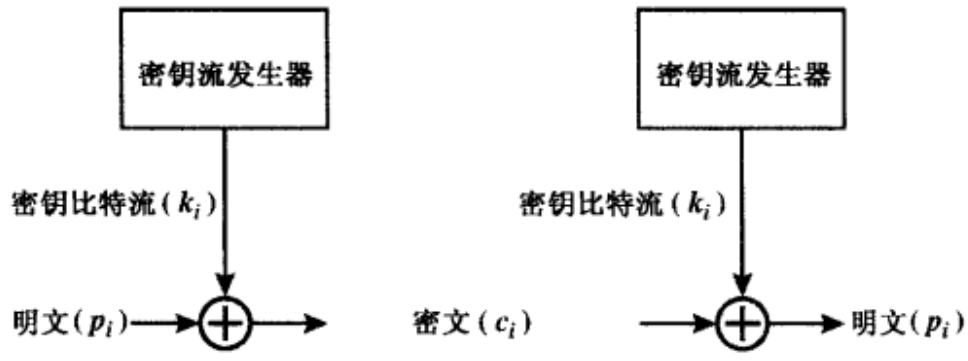


图 2.7 Vernam 密码

该式和 Vigenère 密码的方程式(2.3)类似。

因此,密文是通过对明文和密钥的逐位异或而成的。根据异或运算的性质,解密过程为

$$p_i = c_i \oplus k_i$$

该方程和方程式(2.4)类似。

这种技术的本质在于构造密钥的方式。Vernam 提出使用连续的磁带,其最终也将循环。所以事实上该体制是使用周期很大的循环密钥。尽管周期很长对于密码分析增添了相当大的难度,但是如果有足够的密文,使用已知或可能的明文序列,或者联合使用二者,该方案是可以被破解的。

2.2.6 一次一密

陆军情报军官 Joseph Mauborgne 提出了一种对 Vernam 密码的改进方案,从而达到了最完善的安全性。Mauborgne 建议使用与消息一样长且无重复的随机密钥来加密消息,另外,密钥只对一个消息进行加解密,之后丢弃不用。每一条新消息都需要一个与其等长的新密钥。这就是著名的一次一密,它是不可攻破的。它产生的随机输出与明文没有任何统计关系。因为密文不包含明文的任何信息,所以无法可破。

下面的例子能够说明我们的观点。假设我们使用的是 27 个字符(第 27 个字符是空格)的 Vigenère 密码,但是这里使用的一次性密钥和消息一样长。请看下面的密文:

ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

现在我们用两种不同的密钥解密:

密文:ANKYODKYUREPFJBYOJDSPLREYIUDNOFDOIUERFPLUYTS
密钥:*pxlmvmsydoftyrvzwc tnlebnecvgdupahfzzlmnyih*
明文:mr mustard with the candlestick in the hall
密文:ANKYODKYUREPFJBYOJDSPLREYIUDNOFDOIUERFPLUYTS
密钥:*mfugpmiydgaxgoufhkllmhsqdqogtewbqfgyovuhwt*
明文:miss scarlet with the knife in the library

假设密码分析者已设法找到了这两个密钥,于是就产生了两个似是而非的明文。分析者如何确定正确的解密(即正确的密钥)呢? 如果密钥是在真正随机的方式下产生的,那么分析者就不能说哪一种密钥更有可能。因此没有办法确定正确的密钥,也就是说没有办法确定正确的明文。

事实上,给出任何长度与密文一样的明文,都存在着一个密钥产生这个明文。因此,如果你用穷举法搜索所有可能的密钥,就会得到大量可读、清楚的明文,但是没有办法确定哪一个才是真正所需的,因而这种密码是不可破的。

一次一密的安全性完全取决于密钥的随机性。如果构成密钥的字符流是真正随机的,那么构成密文的字符流也是真正随机的,因此分析者没有任何攻击密文的模式和规则可用。

理论上,我们不再需要寻找密码了,一次一密提供了完全的安全性,但是在实际中,一次一密存在两个基本难点:

(1) 产生大规模随机密钥有实际困难。任何经常使用的系统都需要建立在某个规则基础上的几百万个随机字符,提供这样规模的真正随机字符是相当艰巨的任务。
(2) 更令人担忧的是密钥的分配和保护。对每一条发送的消息,需要提供给发送方和接收方等长度的密钥。因此,存在庞大的密钥分配问题。

因为上面这些困难,一次一密实际很少使用,主要用于安全性要求很高的低带宽信道。
一次一密是唯一的具有完善保密的密码体制。附录 F 探讨了这一概念。

2.3 置换技术

到现在为止,我们所讨论的都是将明文字母代替为密文字母。与之极不相同的一种加密方法是对明文进行置换,这种密码称为置换密码。

最简单的例子是栅栏技术,按照对角线的顺序写出明文,而按行的顺序读出作为密文。例如,用深度为 2 的栅栏技术加密信息“meet me after the toga party”,可写为

```
m e m a t r h t g p r y
e t e f e t e o a a t
```

加密后的信息是

```
MEMATRHTGPRYETEFETEOAAT
```

这种技巧对密码分析者来说实在微不足道。一个更复杂的方案是把消息一行一行地写成矩形块,然后按列读出,但是把列的次序打乱。列的次序就是算法的密钥。例如:

```
密钥:4  3  1  2  5  6  7
明文:a  t  t  a  c  k  p
     o  s  t  p  o  n  e
     d  u  n  t  i  l  t
     w  o  a  m  x  y  z
密文:TTNAAPTMTSUOAODWCOIXKNLYPETZ
```

因此在本例中,密钥是 4312567。为了加密,从标号为 1 的那列开始,本例中为第 3 列。写下那列的所有字母,接着是标号为 2 的列,即第 4 列,然后是第 2 列、第 1 列,再后是第 5、6、7 列。

单纯的置换密码因为有着与原始明文相同的字母频率特征而易被识别。对于上述所示的列变换类型密码,密码分析很直观,可以从将密文排列成矩阵入手,再来处理列的位置。双字母音节和三字母音节频率表分析可以派上用场。

多次置换密码相对来讲要安全得多。这种复杂的置换是不容易重构的。前面那条消息用相同算法再加密一次:

```
密钥:4  3  1  2  5  6  7
明文:t  t  n  a  a  p  t
     m  t  s  u  o  a  o
     d  w  c  o  i  x  k
     n  l  y  p  e  t  z
密文:NSCYAUOPTTWLTMDNAOIEPAXTTOKZ
```

为了更清晰地看出经过双重置换后的结果,我们用字母所在位置的序号来代替原始明文信

息。于是,共28个字母的原始消息序列是

```
01 02 03 04 05 06 07 08 09 10 11 12 13 14
15 16 17 18 19 20 21 22 23 24 25 26 27 28
```

经过第一次置换后变成了

```
03 10 17 24 04 11 18 25 02 09 16 23 01 08
15 22 05 12 19 26 06 13 20 27 07 14 21 28
```

这还是多少有些规律,但是经过第二次置换,变成

```
17 09 05 27 24 16 12 07 10 02 22 20 03 25
15 13 04 23 19 14 11 01 26 21 18 08 06 28
```

之后,排列结构已经没有什么规律了,分析者攻击它要困难得多。

2.4 转轮机

刚才的例子表明用多次置换得到的算法对密码分析来说有很大的难度。这对代替密码也适用。在介绍DES之前,我们先来介绍多层加密原理的一个重要应用例子:轮转机密码系统[①]。

转轮机的基本原理如图2.8所示。转轮机包括一组相互独立的旋转圆筒,电脉冲可以通过它。每个圆筒有26个输入引脚和26个输出引脚。内部连线使每一个输入仅同唯一一个输出连接,为简明起见,图中只画出了三条内部连接。

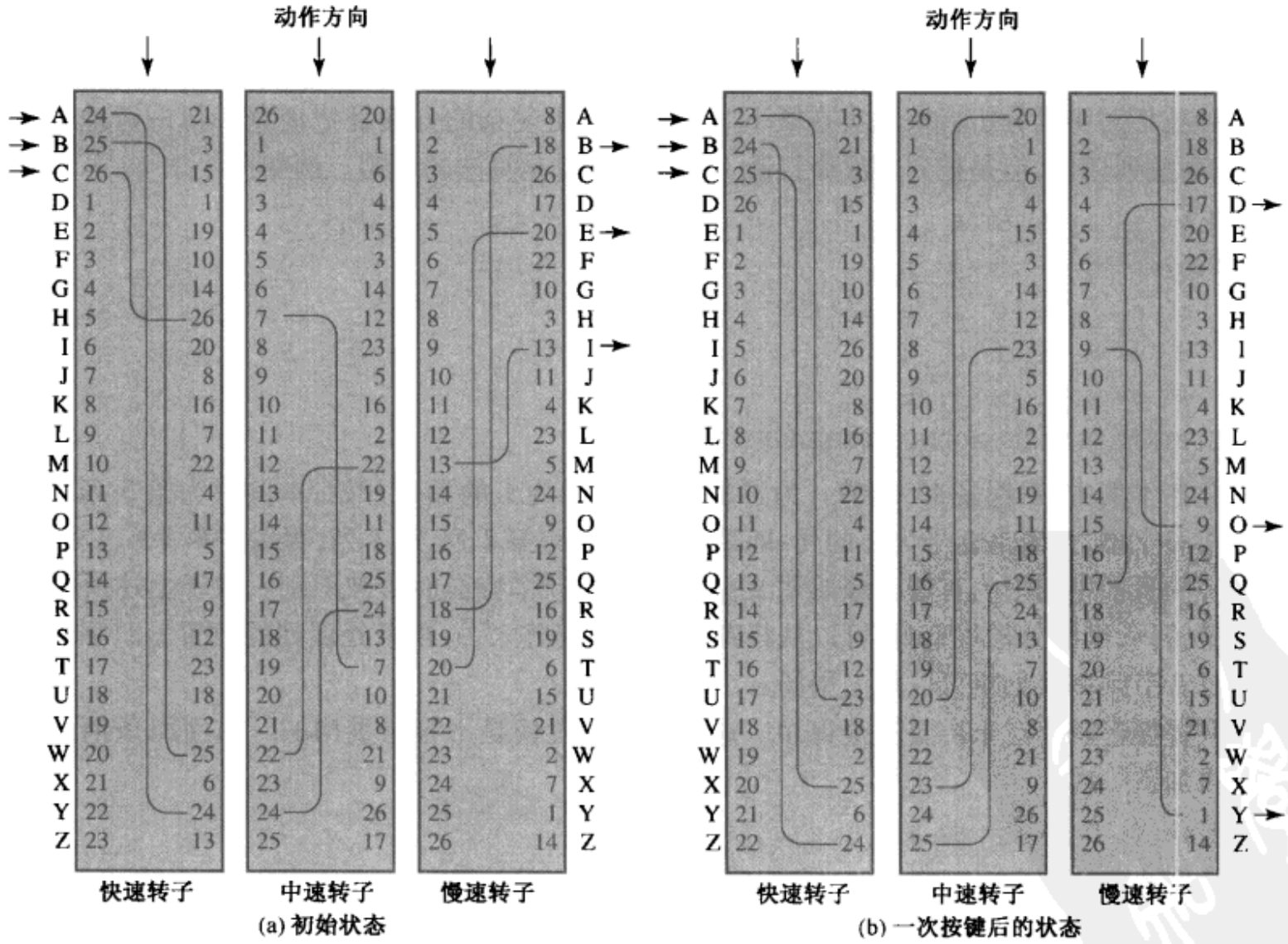


图2.8　用编号表示内部引线的三转子机

① 在第二次世界大战中,德国(Enigma密码机)和日本(Purple密码机)都使用了基于转轮原理的密码机。盟军破译了这两种密码,对"二战"的结局产生了重要的影响。

如果我们把每个输入和输出引脚当做字母表中的一个字母,那么一个圆筒就定义了一个单表替换。如图 2.8 所示,如果操作员按下代表字母 A 的键,那么电信号就加在了第一个圆筒的第一个输入引脚上,经过内部连线被传送到第 25 个输出引脚。

考虑只有一个圆筒的转轮机。每次按下一个输入键,圆筒就旋转一个位置,所以内部连线也就相应改变了。因此就定义了不同的单表代替密码。经过 26 个明文字母后,圆筒回到初始位置。于是我们可以得到一个周期为 26 的多表代替算法。

单筒系统还是比较简单的,容易对付。转轮机的威力在于它使用了多个圆筒,每个圆筒的输出引脚连接到下一个圆筒的输入引脚。图 2.8 所示是一个三筒系统。图中左半部分,操作员从第一个引脚输入(明文字母 a),经过三个圆筒之后,电信号出现在第三个圆筒的第二个引脚(密文字母 B)。

多筒系统中,操作员每按一次输入键,第一个圆筒就旋转一个引脚的位置。图 2.8 的右半部分表明了经过一次按键后的情况。内层的(第一个)圆筒旋转完一圈之后,中间的(第二个)圆筒就转一个引脚的位置。最后,当中间的圆筒转完一圈后,外层的圆筒旋转一个引脚的位置。它的原理就像里程表。所以整个系统重复使用 $26 \times 26 \times 26 = 17\ 576$ 个不同的替换字母表,如果用 4 个或 5 个转轮,则周期数将分别为 456 976 和 11 881 376。David Kahn 在参考文献[KAHN96]的第 413 页对 5 筒转轮机做了详细的论述,其中一段是:

> 足够大的周期可以挫败任何想直接通过字母的频率特征来解密的实际可能。这种普通攻击方法对每个密文表大约需要 50 个字母,这意味着所有 5 个转轮将必须经历 50 次循环。这需要有多长的密文呢?密文长度相当于连续三届国会在美国参议院和众议院发表演讲的长度。没有任何密码分析者可能在一生中戴上这样的桂冠,即使是像政客那样善于辞令的外交家也不可能这么饶舌。

今天,转轮机的意义在于它曾经给最为广泛使用的密码——数据加密标准(DES)指明了方向。我们将在第 3 章中对 DES 进行讨论。

2.5　隐写术

我们讨论一种技术来结束这一章,严格说来,它并不是加密,而是隐写术。

有两种办法可用来隐藏明文信息。隐写术,它可以隐藏信息的存在;而密码学则是通过对文本信息的不同转换而实现信息的对外不可读①。

一种简单却很耗时的隐写术可由一段明显无伤大雅的文本字词重新排列而成。例如,在一整段文本中用每个单词的第一个字母连起来就可以拼出隐藏的消息。图 2.9 中的例子可以利用一整段文本信息中单词的某个子集来表达隐藏信息。这个并不难,看看你能否解密它。

历史上还用过其他一些技术;下面给出一些例子(参见参考文献[MYER91]):

- **字符标记**:选择一些印刷字母或打字机打出的文本,用铅笔在其上书写一遍。这些标记需要做得在一般场合下辨认不出,除非将纸张按某个角度对着亮光看。
- **不可见墨水**:有些物质用来书写后不留下可见痕迹,除非加热或加之以某种化学物质。
- **针刺**:在某些字母上刺上小的针孔,这一般是分辨不出来的,除非对着光线看。
- **打字机的色带校正**:用黑色的色带在行之间打印。用这种色带打印后的东西只在强光下可见。

① 隐写术是一个过时的说法,现在 David Kahn 将其修正并且给定了其含义,参见参考文献[KAHN96]。

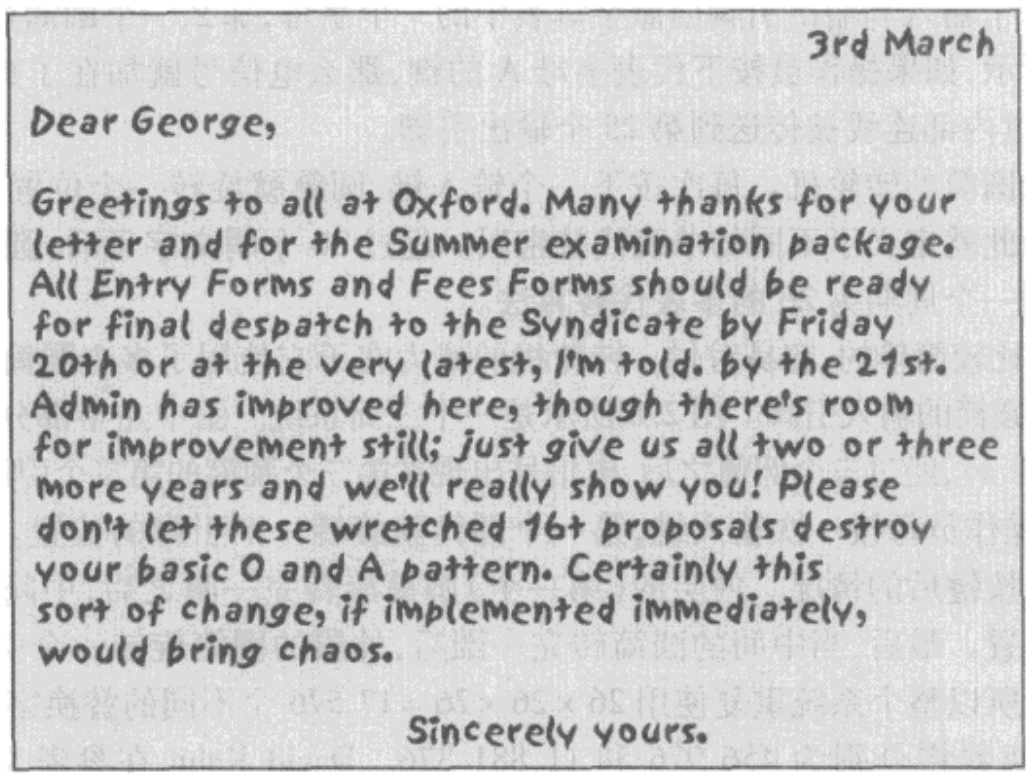


图 2.9 Morse 侦探的疑惑(取自于 Colin Dexter 的《Nicholas Quinn 的沉默世界》)

尽管以上这些方法显得古老,但是却具有时代的意义。参考文献[WAYN93]陈述了如何用 CD 每一帧的最后一位来隐藏信息。例如,柯达 CD 格式的最大分辨率是 2048 × 3072,其中每一个像素包含有 24 位的 RGB 颜色信息。改动这 24 位像素的最低一位对整个画质影响不大。结果是你可以在每一张数字快照中隐藏 2.3 MB 的信息。有很多软件包就是采用诸如此类的办法来进行信息隐蔽的。

同加密相比,隐写术有一些缺点。它需要许多额外的付出来隐藏相对较少的信息。尽管采用一些诸如上述方案也许很有效;但是一旦被破解,整个方案就毫无价值了。不过这个问题也可以克服,比如具体的加入方法由密钥来决定(参见习题 2.20)。另外一种方法是,将消息先加密再将其隐写。

隐写术的优点是可以应用于如下的情况:通信双方宁愿内容丢失,也不愿他们进行秘密通信的事实被人发现。加密标志信息也是重要和秘密的,通过它可以找出想进行消息隐藏的发送方或接收方。

2.6 推荐读物和网站

对密码编码学和密码破译学的历史感兴趣的读者可参阅参考文献[KAHN96]。尽管这本书更注重于密码学的影响而非密码技术的发展,但是它的确是一本介绍密码学的优秀读物,读来饶有趣味。另一本介绍密码学发展历史的读物是[SING99]。

[GARD72]一书则简洁地介绍了本章和其他章节所涉及的技术。涉及古典密码技术并更侧重于技术的文献有很多:[SINK66]是最好的文献之一;[KORN96]读来让人感到很愉快,它用很长篇幅的一节讲述传统技巧;[GARR01]和[NICH99]这两本密码书包含大量传统技术的材料;两卷本的[NICH96]详细地介绍了大量的传统密码,提供了许多密文及其解,对于真正有兴趣的读者有较大的参考价值。

另一本参考文献[KUMA97]对转轮机进行了很好的讨论,包括对它们的密码分析。

参考文献[KATZ00]全面地介绍了隐写术。[WAYN96]是另一个很好的资源。

GARD72 Gardner, M. *Codes, Ciphers, and Secret Writing*. New York: Dover, 1972.

GARR01 Garrett, P. *Making, Breaking Codes: An Introduction to Cryptology*. Upper Saddle River, NJ: Prentice Hall, 2001.

KAHN96 Kahn, D. *The Codebreakers: The Story of Secret Writing*. New York: Scribner, 1996.

KATZ00 Katzenbeisser, S., ed. *Information Hiding Techniques for Steganography and Digital Watermarking*. Boston: Artech House, 2000.

KORN96 Korner, T. *The Pleasures of Counting*. Cambridge, England: Cambridge University Press, 1996.

KUMA97 Kumar, I. *Cryptology*. Laguna Hills, CA: Aegean Park Press, 1997.

NICH96 Nichols, R. *Classical Cryptography Course*. Laguna Hills, CA: Aegean Park Press, 1996.

NICH99 Nichols, R., ed. *ICSA Guide to Cryptography*. New York: McGraw-Hill, 1999.

SING99 Singh, S. *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography*. New York: Anchor Books, 1999.

SINK66 Sinkov, A. *Elementary Cryptanalysis: A Mathematical Approach*. Washington, D. C.: The Mathematical Association of America, 1966.

WAYN96 Wayner, P. *Disappearing Cryptography*. Boston: AP Professional Books, 1996.

推荐网站

- **美国密码电文协会(American Cryptogram Association)**:一个业余密码爱好者的协会。该网上有传统密码学的信息,并可以链接到有相关信息的网站。
- **密码角(Crypto Corner)**:Simon Singh 的站点,有许多有用的信息,还有一些学习密码学的交互工具。
- **隐写术(Steganography)**:有大量相关链接和文档。

2.7　关键术语、思考题和习题

关键术语

分组密码	密码学	Playfair 密码
穷举攻击	解密过程	多表代替密码
Caesar 密码	解密	栅栏密码
密码	双字母	单钥加密
密文	加密过程	隐写术
计算安全	加密	流密码
传统加密	Hill 密码	对称加密
密码分析学	单表代替密码	置换密码
密码编码系统	一次一密	无条件安全
密码编码学	明文	Vigenère 密码

思考题

2.1 什么是对称密码的本质成分?
2.2 密码算法中两个基本函数是什么?
2.3 用密码进行通信的两个人需要多少密钥?
2.4 分组密码和流密码的区别是什么?
2.5 攻击密码的两种一般方法是什么?
2.6 列出并简要地定义基于攻击者所知道信息的密码分析攻击类型。
2.7 无条件安全密码和计算上安全密码的区别是什么?
2.8 简要地定义 Caesar 密码。
2.9 简要地定义单表代替密码。
2.10 简要地定义 Playfair 密码。
2.11 单表代替密码和多表代替密码的区别是什么?
2.12 一次一密的两个问题是什么?
2.13 什么是置换密码?
2.14 什么是隐写术?

习题

2.1 仿射 Caesar 密码,即 Caesar 密码的一种推广,具有如下定义:对于每一个明文 p,用密文 C 代替,其中

$$C = \mathrm{E}([a,b],p) = (ap + b) \bmod 26$$

对加密算法的一个基本要求是算法是单射,即如果 $p \neq q$,则 $\mathrm{E}(k,p) \neq \mathrm{E}(k,q)$。否则,就会因为有很多的明文映射成相同的密文而不能解密。仿射 Caesar 密码并不是对所有的 a 都是一对一的映射。例如,当 $a=2, b=3$ 时,有 $\mathrm{E}([a,b],0) = \mathrm{E}([a,b],13) = 3$。

(a) 对 b 的取值是否有限制? 解释原因。

(b) 判定 a 不能取哪些值?

(c) 分析 a 可以取哪些值,不可以取哪些值,并给出理由。

2.2 有多少种仿射 Caesar 密码?

2.3 用仿射 Caesar 密码加密得到一份密文。频率最高的字母为 B,次高的字母为 U,请破译该密码。

2.4 已知下面的密文由简单的代替算法产生:

```
53‡‡†305))6*;4826)4‡.)4‡);806*;48†8¶60))85;;]8*;:‡*8†83
(88)5*†;46(;88*96*?;8)*‡(;485);5*†2:*‡(;4956*2(5*—4)8¶8*
;4069285);)6†8)4‡‡;1(‡9;48081;8:8‡1;48†85;4)485†528806*81
(‡9;48;(88;4(‡?34;48)4‡;161;:188;‡?;
```

请将它破译。

提示:

(1) 正如你所知,英文中最常见的字母是 e。因此,密文第一个或第二个(或许第三个)出现频率最高的字符应该代表 e。此外,e 经常成对出现(如 meet、fleet、speed、seen、been、agree 等)。找出代表 e 的字符,并首先将它译出来。

(2) 英文中最常见的单词是 the。利用这个事实猜出什么字符代表字母 t 和 h。

(3) 根据已经得到的结果破译其他部分。

注意:最终得到的英文明文第一眼看起来可能不太好懂。

2.5 解决密钥分配问题的一个办法是,使用收发双方都有的一本书中的某行文字。至少在某些侦探小

说中经常把一本书的第一句话作为密钥。这里就从一本涉及编码的富于悬念的侦探小说——Ruth Rendell 的《与陌生人的谈话》中找到了一个例子。请不要找到这本书之后再来做这道题!

给定下列消息:

SIDKHKDM AF HCRKIABIE SHIMC KD LFEAILA

这段密文是用《沉默的背后》(一本有关侦探 Kim Philby 的小说)一书的第一句话和单表代替方法产生的,这句话是:

The snow lay thick on the steps and the snowflakes driven by the wind looked black in the headlights of the cars.

使用的是简单的代替密码。

(a) 加密算法是什么样的?

(b) 它的安全性怎么样?

(c) 为了使密钥分配问题简单化,通信双方都同意使用一本书的第一句话或最后一句话作为密钥。要想改变密钥,他们只需更换一本书就行了。使用第一句话比使用最后一句话要好,为什么?

2.6 在 Sherlock Holmes 的案件中,有一案例是这样的。有一段消息:

534 C2 13 127 36 31 4 17 21 41

DOUGLAS 109 293 5 37 BIRLSTONE

26 BIRLSTONE 9 127 171

尽管 Watson 被难住了,但 Holmes 一下子就推出了所用密码的类型。你能吗?

2.7 这是真实世界的一个例子,来自以前的美国特种兵手册(公开部分)。可从本书的网址下载副本。

(a) 用记忆词 *cryptographic* 和 *network security* 作为两个密钥来加密如下消息:

Be at the third pillar from the left outside the lyceum theatre tonight at seven. If you are distrustful bring two friends.

如何处理记忆词中的冗余字母、多余的字母?如何处理空格和标点符号?请做出合理的假设。

注:这条消息摘自 Sherlock Holmes 的小说 *The Sign of Four*。

(b) 解密密文,说明你是如何做的。

(c) 评议一下何时采用这种技术?有何优势?

2.8 普通的单表代替密码的一个缺点是,收发双方都必须将打乱的密码序列记住。为了避免这一点,经常使用这种方法:采用可推出整个密码序列的一个密钥。例如,使用密钥词 CIPHER,然后将字母表中没有使用的字母按正常的顺序排在 CIPHER 的后面:

明文:abcdefghijklmnopqrstuvwxyz

密文:CIPHERABDFGJKLMNOQSTUVWXYZ

如果感觉这样做不能产生足够的混淆,那么可以将字母表中没有使用的字母按密钥词排成几行,再按列读出:

C I P H E R

A B D F G J

K L M N O Q

S T U V W X

Y Z

这就得到了序列:

C A K S Y I B L T Z P D M U H F N V E G O W R J Q X

在 2.2 节的例子(该例子以"it was disclosed yesterday"开头)中使用了这种方法。试确定密钥词。

2.9 当海军上尉 John F. Kennedy 管理的美国巡逻船 PT-109 被日本毁灭者击沉时,位于澳大利亚的一个无线站截获了一条用 Playfair 密码加密的消息:

KXJEY UREBE ZWEHE WRYTU HEYFS
KREHE GOYFI WTTTU OLKSY CAJPO
BOTEI ZONTX BYBWT GONEY CUZWR
GDSON SXBOU YWRHE BAAHY USEDQ

密钥为 *royal new zealand navy*。请解密这条消息,将 TT 换为 tt。

2.10 (a) 用密钥 *largest* 构造一个 Playfair 矩阵。

(b) 用密钥 *occurrence* 构造一个 Playfair 矩阵。对密钥中的冗余字母的处理方法做出合理的假设。

2.11 (a) 使用 Playfair 矩阵

M	F	H	I/J	K
U	N	O	P	Q
Z	V	W	X	Y
E	L	A	R	G
D	S	T	B	C

加密消息:

Must see you over Cadogan West. Coming at once.

注:该消息摘自 Sherlock Holmes 的故事 *The Adventure of the Bruce-Partington Plans*。

(b) 用习题 2.10(a)中的 Playfair 矩阵重做习题 2.11(a)。

(c) 对这个习题的结果你如何解释?所得结论能做一般性推广吗?

2.12 (a) Playfair 密码有多少可能的密钥?那些会产生相同加密结果的密钥也计算在内。将结果用 2 的幂形式表示,取最佳逼近。

(b) 除去那些会产生相同加密结果的密钥,那么 Playfair 密码有多少有效的唯一密钥?

2.13 在代替密码中,使用 25 × 1 的 Playfair 矩阵,结果会如何?

2.14 (a) 用 Hill 密码加密消息"meet me at the usual place at ten rather than eight oclock",密钥为 $\begin{bmatrix} 9 & 4 \\ 5 & 7 \end{bmatrix}$。要求写出计算过程和结果。

(b) 写出从密文恢复明文所做的解密计算。

2.15 我们知道只要有足够多的明密文对,Hill 密码是不能抗已知明文攻击的。如果可使用选择明文攻击,Hill 就更脆弱了。请描述这种攻击方案。

2.16 已经证明了 Hill 密码中的矩阵 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 需要满足$(ad - bc)$与 26 互素。也就是说,$(ad - bc)$与 26 的唯一正整数因子是 1。因此,如果$(ad - bc)$为 13 或偶数,那么这样的矩阵不合格。运用下面的方法而不是逐个去数,给出 2 × 2 Hill 密码密钥的个数。

(a) 判定如下矩阵的个数:判别式为偶数,且其一行或两行都是偶的(称一行为偶的是指该行的所有元素都为偶数)。

(b) 判定如下矩阵的个数:判别式为偶数,且其一列或两列都是偶数。

(c) 判定如下矩阵的个数:判别式为偶数,且其所有元素为奇数。

(d) 考虑重叠情况,找出判别式为偶数的矩阵总数。

(e) 判定如下矩阵的个数:判别式为 13 的倍数,且其第一列为 13 的倍数。

(f) 判定如下矩阵的个数:判别式为 13 的倍数,且其第一列不为 13 的倍数,但第二列在模 13 的意义下是第一列的倍数。

(g) 计算判别式为 13 的倍数的所有矩阵个数。

(h) 判定如下矩阵的个数:判别式为 26 的倍数,且满足情况(a)和(e),(b)和(e),(c)和(e),(a)和(f),等等。

(i) 计算出判别式既不是 2 的倍数也不是 13 的倍数的矩阵总数。

2.17　用 Vigenère 密码加密单词 explanation，密钥为 *leg*。

2.18　本题可显示出 Vigenère 密码的一次一密版本的用途。该方案中，密钥是位于 0 ~ 26 的随机数字。例如，如果密钥是 3 19 5…，则明文的第一个字母使用 3 个字母的移位加密，第二个字母使用 19 个字母的移位加密，第三个字母使用 5 个字母的移位加密，以此类推。

(a) 使用密钥流 9 0 1 7 23 15 21 14 11 11 2 8 9 加密明文 sendmoremoney。

(b) 使用(a)中产生的密文找到一个密钥，以便该密文解密为 cashnotneeded。

2.19　隐藏在图 2.9 中的消息是什么？

2.20　在多罗西的怪诞小说中，有一个故事是这样的：地主彼得遇到了图 2.10 所示的消息，他找到了密钥，是一段整数：

78765654343211234345656787887876565 4

34321123434565678788787656544332112 34

(a) 破译这段消息。提示：最大的整数是什么？

(b) 如果只知道算法而不知道密钥，这种加密方案的安全性怎么样？

(c) 如果只知道密钥而不知道算法，这种加密方案的安全性又怎么样？

I thought to see the fairies in the fields, but I saw only the evil elephants with their black backs. Woe! how that sight awed me! The elves danced all around and about while I heard voices calling clearly. Ah! how I tried to see-throw off the ugly cloud-but no blind eye of a mortal was permitted to spy them. So then came minstrels, having gold trumpets, harps and drums. These played very loudly beside me, breaking that spell. So the dream vanished, whereat I thanked Heaven. I shed many tears before the thin moon rose up, frail and faint as a sickle of straw. Now though the Enchanter gnash his teeth vainly, yet shall he return as the Spring returns. Oh, wretched man! Hell gapes, Erebus now lies open. The mouths of Death wait on thy end.

图 2.10　地主彼得的迷惑

编程题

2.21　编写一个加密解密程序实现广义 Caesar 密码，该密码也称为加法密码。

2.22　编写一个加密解密程序实现习题 2.1 中的仿射密码。

2.23　编写一个程序，无须人工干预，对加法密码实现字母频率攻击。软件按可能性大小的顺序给出可能的明文。如果用户界面允许用户定义“给出前 10 个可能明文”，则更好。

2.24　编写一个程序，无须人工干预，对单表代替密码实现字母频率攻击。软件按可能性大小的顺序给出可能的明文。如果用户界面允许用户定义“给出前 10 个可能明文”，则更好。

2.25　编写一个程序实现 2 × 2 Hill 密码的加解密算法。

2.26　编写一个程序实现对 m 维 Hill 密码的已知明文攻击。算法的效率如何？用 m 的函数表示。

第3章　分组密码和数据加密标准

3.1　分组密码原理
- 3.1.1　流密码与分组密码
- 3.1.2　Feistel 密码结构的设计动机
- 3.1.3　Feistel 密码

3.2　数据加密标准
- 3.2.1　DES 加密
- 3.2.2　DES 解密

3.3　DES 的一个例子
- 3.3.1　结果
- 3.3.2　雪崩效应

3.4　DES 的强度
- 3.4.1　56 位密钥的使用
- 3.4.2　DES 算法的性质
- 3.4.3　计时攻击

3.5　差分分析和线性分析
- 3.5.1　差分密码分析
- 3.5.2　线性密码分析

3.6　分组密码的设计原理
- 3.6.1　DES 的设计标准
- 3.6.2　迭代轮数
- 3.6.3　函数 F 的设计
- 3.6.4　密钥扩展算法

3.7　推荐读物和网站

3.8　关键术语、思考题和习题

All the afternoon Mungo had been working on Stern's code, principally with the aid of the latest messages which he had copied down at the Nevin Square drop.

Stern was very confident. He must be well aware London Central knew about that drop. It was obvious that they didn't care how often Mungo read their messages, so confident were they in the impenetrability of the code.

—*Talking to Strange Men*, Ruth Rendell

a ciphertext block of the same length.

要　点

- 分组密码是一种加/解密方案，它将输入的明文分组当做一个整体处理，输出一个等长的密文分组。It proceus the input plaintext block as a whole, and outputs
- 许多分组密码都采用 Feistel 结构，这样的结构由许多相同的轮函数组成。每一轮中，对输入数据的一半进行代替，接着用一个置换来交换数据的两个等分部分。扩展初始的密钥使得每一轮中使用不同的子密钥。
- 直到最近，DES 一直是应用最为广泛的加密算法，它体现了经典的 Feistel 结构。DES 使用 64 位的分组和 56 位的密钥。
- 差分分析和线性分析是两种重要的密码分析方法。DES 对这两种攻击具有很强的免疫性。

本章的目的在于阐明现代对称密码的基本原理。因此，我们主要探讨使用最广泛的对称密码：数据加密标准（Data Encryption Standard，DES）。尽管 DES 之后涌现了大量的对称密码，尽管 DES 注定要被高级加密标准（Advanced Encryption Standard，AES）所取代，但它依然是一个极其重要的算法。而且，详细讨论 DES 算法，也可以帮助我们深刻地理解其他对称密码所涉及的原理。

首先，本章将研究对称分组密码的一般原理，本书所说的对称密码主要是指对称分组密码（除第 7 章的流密码 RC4 外）。接下来我们研究完整的 DES。讨论过这个具体的算法后，我们又回到对分组密码设计问题的更一般性讨论上来。

与 RSA 这样的公钥密码相比，DES 以及大多数传统加密算法的结构都非常复杂，不能像 RSA 或类似的算法给予简洁的描述。因此，读者可能会希望以简化的 DES 开始，简化 DES 将在附录 G 中给出。对于这个简化版本，读者可以手工加解密，从而对算法的详细工作过程有较好的理解。课堂教学经验表明，对简化版本的研究能够提高对 DES 的理解①。

3.1　分组密码原理

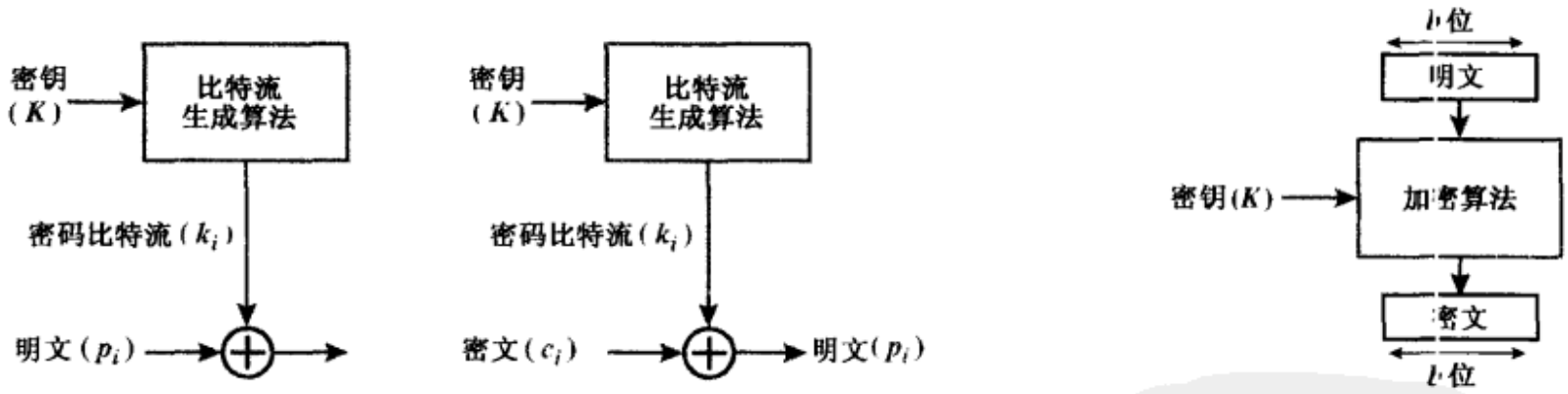
事实上，现在使用的大多数对称分组加密算法都是基于 Feistel 分组密码结构的[FEIS73]。因为这个原因，研究 Feistel 密码的设计原理就显得很重要。我们先比较流密码和分组密码，然后讨论 Feistel 结构分组密码的设计动机，最后来讨论它的一些含义。

3.1.1　流密码与分组密码

流密码每次加密数据流的一位或一个字节。古典流密码的例子有密钥自动产生的 Vigenère 密码和 Vernam 密码。理想情况下，可以使用一次一密版本的 Vernam 密码，其中密钥流（k_i）和明文位流（p_i）一样长（参见图 2.7）。如果密钥流是随机的，除非是获得了密钥流，否则这个密码是不可破的。然而，密钥流必须提前以某种独立、安全的信道提供给双方。如果待传递的数据流量很大，这就带来了一个不可逾越的障碍。

相应地，出于实用的原因，位流必须以算法程序的方式实现，从而双方都可以生产具有密码学意义的位流。这种方法中[参见图 3.1(a)]，位流发生器是一个由密钥控制的算法，它必须产生在密码学意义上讲是强壮的位流。现在，两个用户只需要共享生成密钥，则各自可以生产密钥流。

分组密码是将一个明文分组作为整体加密并且通常得到的是与明文等长的密文分组。典型的分组大小是 64 位或 128 位。同流密码一样，两个用户要共享一个对称加密密钥[参见图 3.1(b)]。第 6 章中还要讲到，使用某些工作模式，分组密码可以获得与流密码相同的效果。



(a) 使用算法比特流发生器的流密码　　(b) 分组密码

图 3.1　流密码与分组密码

人们已经对分组密码进行了大量的研究。一般来说，分组密码的应用范围比流密码要广泛。绝大部分基于网络的对称密码应用使用的是分组密码。因此，本章及本书我们所讨论的对称密码将集中在分组密码的讨论上。

3.1.2　Feistel 密码结构的设计动机

分组密码作用在 n 位明文分组上，而产生 n 位密文分组。共有 2^n 个不同的明文分组，且由于

① 然而，你也可以跳过附录 G，至少初次阅读时可以这样做。如果因为陷入 DES 的技术细节而迷惘，你可以回过头来从简化的 DES 开始。

加密是可逆的(即可以解密),每一个明文分组将唯一对应一个密文分组。这样的变换称为可逆变换,或非奇异变换。下面这个例子解释了 $n=2$ 时的非奇异变换和奇异变换。

可逆映射	
明 文	密 文
00	11
01	10
10	00
11	01

不可逆映射	
明 文	密 文
00	11
01	10
10	01
11	01

在后面这个变换中,密文 01 可由两个明文分组中的任意一个产生。所以,如果我们限定在可逆映射上,不同变换的总数是 $2^n!$ 个①。

图 3.2 给出了 $n=4$ 时的一个普通代替密码的结构。4 位的输入有 16 种可能的输入状态,每一种被代替密码映射成 16 种可能输出状态中的唯一一种,每一种表示 4 位的密文输出。加密和解密映射可如表 3.1 一样用表来定义。这是分组密码的最一般形式,能用来定义明密文之间的任意可逆变换。Feistel 称这种密码为理想分组密码,因为它允许生成最大数量的加密映射来映射明文分组[FEIS75]。

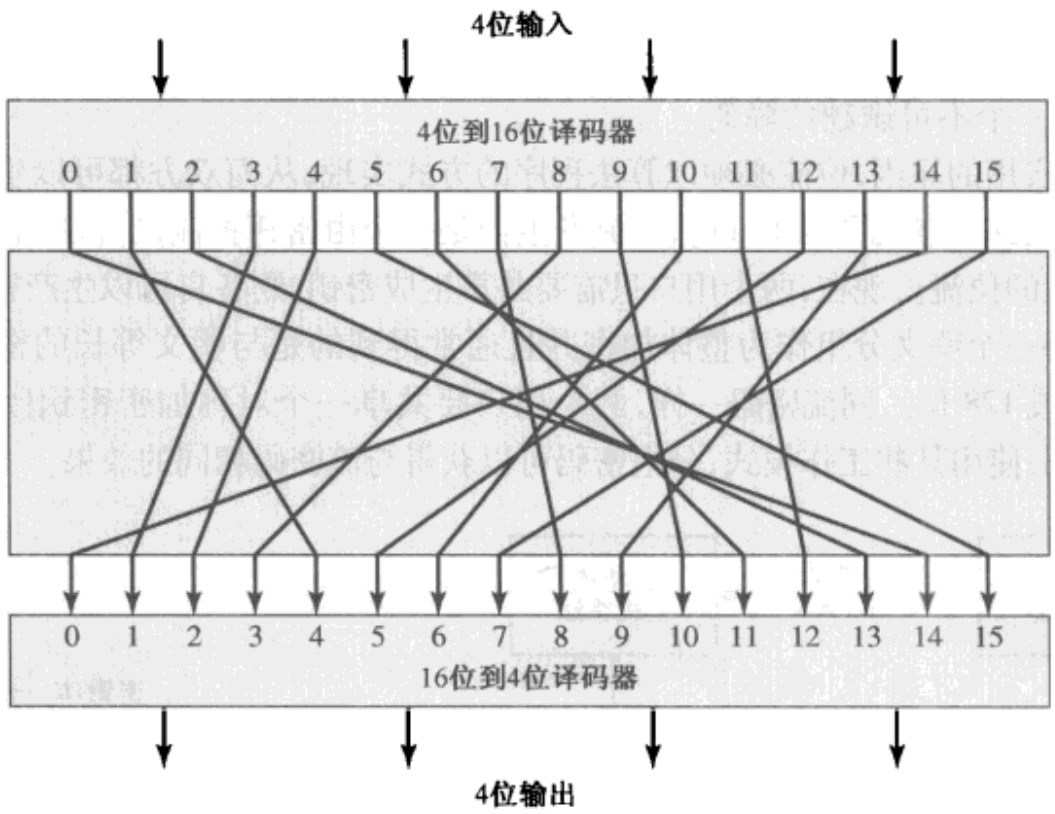


图 3.2 $n=4$ 时的一个 n 位到 $-n$ 位分组密码

但是应用这种方法有着实际上的困难。对于像 $n=4$ 这样的较小分组,密码系统等价于传统代替密码。正如我们所见,用明文的统计分析方法攻击它是轻而易举的。这种脆弱性并非来自于代替密码,而是因为使用的分组规模太小。如果 n 充分大,并且允许明密文之间采用任意的可逆变换,那么明文的统计特征将被掩盖,从而不能用统计方法来攻击这种体制。

然而,从实现和运行的角度来看,采用大规模分组的任意可逆代替密码(即理想分组密码)是不可行的。因为对于这样的变换,映射本身就是密钥。再看一下表 3.1,它定义了 $n=4$ 时的某个可逆映射。这个映射可以直接由表中的第二列来定义,它给出了每个明文对应的密文。本质上,它就是决定所有可能映射中某一个映射的密钥。在这种情况下,运用这种直接的方法定义密钥,密钥长度为

① 原因如下:对于第一个明文,我们可以选择 2^n 个密文分组中的任意一个。对于第二个明文,我们可以从剩下的 2^n-1 个中选择一个,以此类推。

(4 位) × (16 行) =64 位。一般地,对于 n 位的代替分组密码,密钥的规模是 $n \times 2^n$ 位。一个64 位的分组密码,若分组有抗统计攻击的理想长度,其密钥大小将要 $64 \times 2^{64} = 2^{70} \approx 10^{21}$ 位。

表 3.1　图 3.2 所示代替密码的加/解密表

明　文	密　文	明　文	密　文
0000	1110	0000	1110
0001	0100	0001	0011
0010	1101	0010	0100
0011	0001	0011	1000
0100	0010	0100	0001
0101	1111	0101	1100
0110	1011	0110	1010
0111	1000	0111	1111
1000	0011	1000	0111
1001	1010	1001	1101
1010	0110	1010	1001
1011	1100	1011	0110
1100	0101	1100	1011
1101	1001	1101	0010
1110	0000	1110	0000
1111	0111	1111	0101

考虑到这些困难,Feistel 指出我们所需要的是对理想分组密码体制(分组长度 n 较大)的一种近似体制而已,它可以在容易实现部件的基础上逐步建立起来[FEIS75]。在讨论 Feistel 的方法之前,我们来进行其他的讨论。我们可以使用一般的分组代替密码,但为了实现起来方便,我们只讨论 $2^n!$ 个不同可逆映射的一个子集。例如,假设我们按照线性方程的集合来定义映射。在 $n=4$ 时,则有

$$
\begin{aligned}
y_1 &= k_{11}x_1 + k_{12}x_2 + k_{13}x_3 + k_{14}x_4 \\
y_2 &= k_{21}x_1 + k_{22}x_2 + k_{23}x_3 + k_{24}x_4 \\
y_3 &= k_{31}x_1 + k_{32}x_2 + k_{33}x_3 + k_{34}x_4 \\
y_4 &= k_{41}x_1 + k_{42}x_2 + k_{43}x_3 + k_{44}x_4
\end{aligned}
$$

其中 x_i 是明文分组中的 4 位二进制数,y_i 是密文分组中的四位二进制数。k_{ij} 是二进制系数,所有运算都是模 2 运算。密钥的大小只是 n^2,这里为 16 位。如果算法为攻击者所知,这种公式表示的密码在密码分析攻击下将是很危险的。本例中,我们只是将第 2 章所讨论的 Hill 密码的知识运用到二进制数据而不是字符。正如第 2 章所见,这样一个简单的线性系统是较易攻破的。

3.1.3　Feistel 密码

Feistel 建议[FEIS73]使用乘积密码的概念来逼近理想分组密码。乘积密码是指依次使用两个或两个以上的基本密码,所得结果的密码强度将强于所有单个密码的强度。这种方法的本质是开发一个分组密码,密钥长为 k 位,分组长为 n 位,采用 2^k 个变换,而不是理想分组密码的 $2^n!$ 个可用变换。

特别地,Feistel 建议使用这样的密码:该种密码交替地使用代替和置换。代替和置换的定义如下:

- **代替**:每个明文元素或元素组被唯一地替换为相应的密文元素或元素组。
- **置换**:明文元素的序列被替换为该序列的一个置换。也就是说,序列中没有元素被添加、删除或替换,但序列中元素出现的顺序改变了。

实际上,这是 Claude Shannon 提出的交替使用混淆和扩散的乘积密码的实际应用[SHAN49]①。我们接下来看一看混淆和扩散的概念,然后研究 Feistel 密码。但是,首先请注意这样一个不平凡的事实:基于 1945 年 Shannon 理论的 Feistel 密码结构,仍是当前使用的大多数重要对称分组密码的基本结构。

混淆和扩散

Shannon 引进混淆和扩散这两个术语来刻画任何密码系统的两个基本构件[SHAN49]。他关注的是如何挫败基于统计方法的密码分析。理由是这样的:假设攻击者拥有明文统计特征的知识。例如某种人类语言的可读信息,其不同字母的频率分布是已知的,或者已知信息中极有可能出现某些单词或短语。如果这些特征以任何形式体现在密文中,密码分析者就有可能推导出密钥或其一部分,至少是包含确切密钥的一个密钥集。在 Shannon 所指的强理想密码中,密文所有的统计特征都是独立于所用密钥的。我们前面所讲的任意代替密码(参见图 3.2)就是这样一种密码,不过正如我们所见,它是不可能获得实际应用的②。

舍弃对理想系统的追求,Shannon 建议了两种对付统计分析的方法:扩散和混淆。扩散就是指使明文的统计特征消散在密文中,这可以通过让每个明文数字尽可能地影响多个密文数字获得,等价于说每个密文数字被许多明文数字影响。一个扩散的例子是用如下方法加密一段消息 $M = m_1, m_2, m_3, \cdots$,其中 m_i 为字母:

$$y_n = \left(\sum_{i=1}^{k} m_{n+i}\right) \bmod 26$$

将 k 个连续字母相加得到密文字母 y_n。明文的统计结构因而消失了。故密文中各字母的频率比明文更趋于一致;双字母频率也是如此;等等。在二进制分组密码中,对明文进行置换后再用某个函数作用,重复多次就可获得较好的扩散效果;这来自于原始明文中不同位置的多个位对密文的某一位产生的影响③。

每一个分组密码都是明文分组到密文分组的变换,而这个变换又是依赖于密钥的。扩散的方法是尽可能地使明文和密文间的统计关系变得复杂,以挫败推导出密钥的企图。另一方面,混淆则是尽可能使密文和加密密钥间的统计关系更加复杂,以阻止攻击者发现密钥。因此,即使攻击者拥有一些密文的统计特征信息,利用密钥产生密文的方法的复杂性也会使得推导密钥极其困难。这可以用一些复杂的代替算法来实现,简单的线性代替函数几乎增加不了混淆。

正如参考文献[ROBS95b]指出的那样,扩散和混淆正是抓住了设计分组密码的本质而成为现代分组密码设计的里程碑。

Feistel 密码结构

图 3.3 描述了 Feistel 提出的结构。加密算法的输入是长为 $2w$ 位的明文分组和密钥 K。明文分组被分为等长的两部分:L_0 和 R_0。这两半数据经过 n 轮迭代后组合成密文分组。第 i 轮迭代的

---

① 可以在本书的网站上找到该论文。这是 Shannon 于 1949 年发表的论文,它最初就是 1945 年给出的一个保密报告。Shannon 在计算机和信息科学史上占有杰出的地位。他不仅奠定了现代密码学的基本思想,而且他还是信息论的开创者。基于信息论的工作,他提出了数据通信信道的容量计算公式,至今仍在使用。此外,他还创立了另一门学科,即把布尔代数用于数字电路的研究,这是他在自己的硕士论文中提出来的。

② 附录 F 扩展了 Shannon 关于密码算法保密和安全的度量概念。

③ 某些密码学的书把置换和扩散等同起来,这是不正确的。置换本身并不改变明文在单个字符或置换块上的统计特性。例如,在 DES 中用置换交换两个 32 位数据块,因而 32 位数据块内的统计特性将保持不变。

输入 L_{i-1}和 R_{i-1}来自于上轮迭代的输出；而输入的子密钥 K_i是由整个密钥 K 推导出的。一般地，K_i不同于 K，也互不相同。尽管可以使用任意轮数，但图 3.3 使用了 16 轮。

每轮迭代都有相同的结构。代替作用在数据的左半部分。它通过用轮函数 F 作用于数据的右半部分后，与左半部分数据进行异或来完成。每轮迭代的轮函数是相同的，但是输入的子密钥 K_i不同。换一种说法，F 是 w 位长的右半分组以及 y 位长的子密钥的函数，输出 w 位的值：$F(RE_i, K_{i+1})$。代替之后，交换数据的左右两半完成置换①。这种结构是 Shannon 提出的代替置换网络（Substitution-Permutation Network，SPN）的一种特别形式。

Feistel 结构的具体实现依赖于以下参数和特征：

- **分组长度**：分组越长意味着安全性越高（其他数据不变），但是会降低加/解密的速度。这种安全性的增加来自于更好的扩散性。传统上，64 位的分组长度比较合理，在分组密码设计中很常用。然而，高级加密标准使用的是 128 位的分组长度。
- **密钥长度**：密钥较长同样意味着安全性较高，但会降低加/解密速度。这种安全性的增加来自于更好的抗穷尽攻击能力和更好的混淆性。现在一般认为 64 位的密钥还不够。通常使用的密钥长度是 128 位。
- **迭代轮数**：Feistel 密码的本质在于单轮不能提供足够的安全性，而多轮加密可取得很高的安全性。迭代轮数的典型值是 16。
- **子密钥产生算法**：子密钥产生越复杂，密码分析就越困难。
- **轮函数 F**：同样，轮函数越复杂，抗攻击的能力就越强。

设计 Feistel 密码还有两个其他方面的考虑：

- **快速软件加/解密**：许多情况下，加密算法被嵌入到应用程序中，以避免硬件实现的麻烦。因此，算法执行的速度很重要。
- **简化分析难度**：尽管我们喜欢把算法设计得使密码分析尽可能困难，然而将算法设计得容易分析也有它的好处。也就是说，如果算法描述起来简洁清楚，那么分析其脆弱性也就容易一些，因而可以开发出更强的算法。不过 DES 并没有容易的分析办法。

Feistel 解密算法

Feistel 密码的解密过程本质上与加密过程一致。其规则如下：将密文作为算法的输入，但是逆序使用子密钥 K_i。也就是说。第一轮使用 K_n，第二轮使用 K_{n-1}，直到最后一轮使用 K_1。这是一个很好的特点，因为我们不需要分别实现加密和解密两个算法。

图 3.3 表示了用逆序密钥和相同算法解密的过程。加密过程在图的左边，自上而下，而解密过程在图的右边，自下而上进行了 16 轮算法（无论多少轮结果都是相同的）。为清楚起见，我们用 LE_i和 RE_i表示加密过程的中间数据，而用 LD_i和 RD_i表示解密过程的中间数据。图中表明，每轮的解密过程中间值与加密过程中间值左右互换的结果是相同的。换句话说，第 i 轮加密的输出是 $LE_i \| RE_i$，解密的第 $(16-i)$ 轮的相应输入则是 $RE_i \| LE_i$，或 $LD_{16-i} \| RD_{16-i}$。

我们按照图 3.3 通过计算来证明上面说法的正确性。加密过程的最后一轮迭代后，输出数据的左右两部分互换，所以密文是 $RE_{16} \| LE_{16}$。现在将它作为同一个算法的输入。第一轮的输入就是 $RE_{16} \| LE_{16}$，它应等于加密过程第 32 轮输出左右部分互换的值。

---

① 在最后一轮迭代后有一个交换，以抵消在最后一轮迭代中的那个交换。你可以从图 3.3 中去掉这两个交换，但这样会使问题的表述不一致。我们将会看到，无论如何，去掉最后一轮中的交换将使解密更简单。

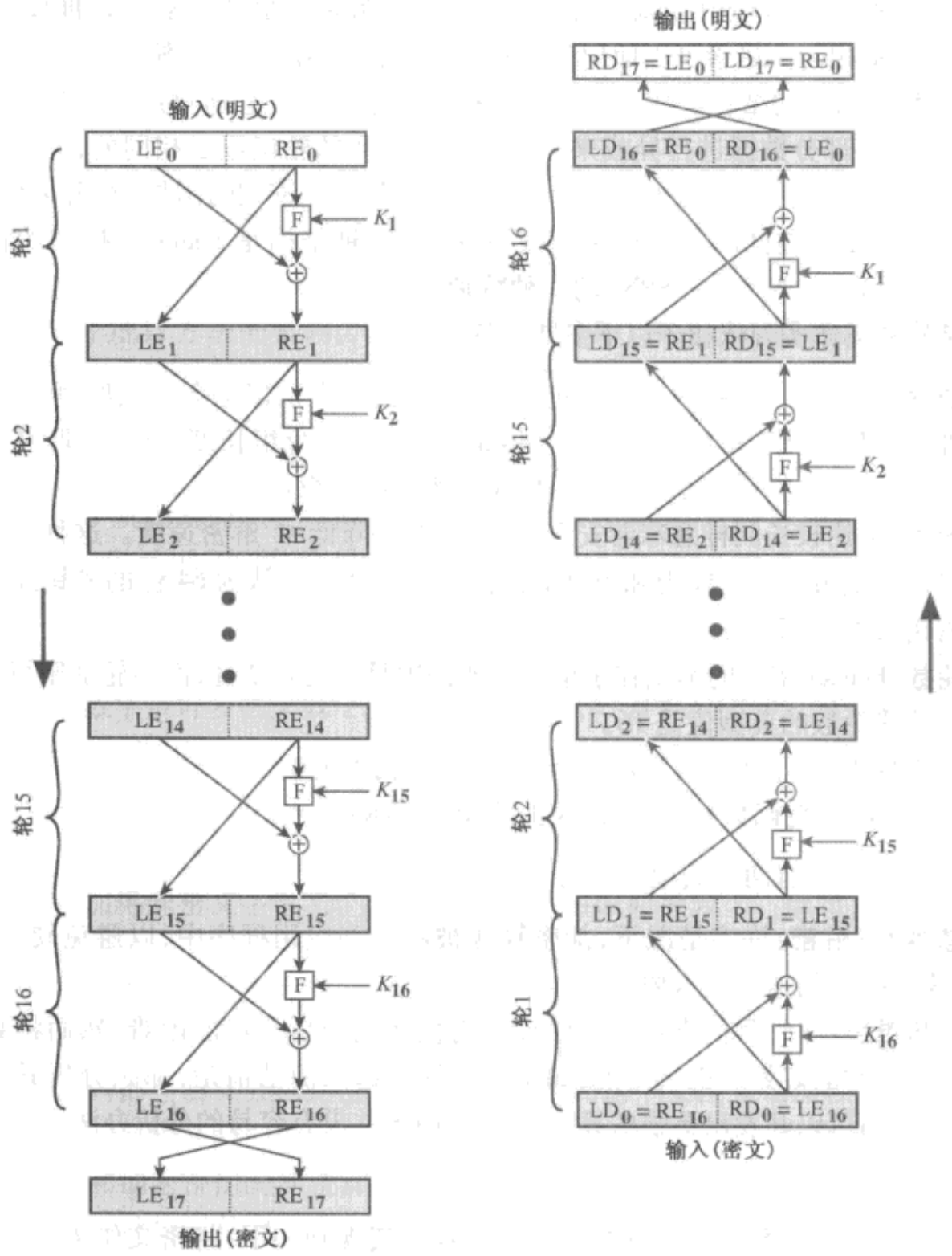


图 3.3 Feistel 加密和解密

现在我们来证明解密过程第一轮的输入等于加密过程第 32 轮输出左右部分互换的值。首先,对于加密过程有

$$\begin{aligned} LE_{16} &= RE_{15} \\ RE_{16} &= LE_{15} \oplus F(RE_{15}, K_{16}) \end{aligned}$$

对于解密则有

$$\begin{aligned} LD_1 &= RD_0 = LE_{16} = RE_{15} \\ RD_1 &= LD_0 \oplus F(RD_0, K_{16}) \\ &= RE_{16} \oplus F(RE_{15}, K_{16}) \\ &= [LE_{15} \oplus F(RE_{15}, K_{16})] \oplus F(RE_{15}, K_{16}) \end{aligned}$$

XOR 运算有以下性质:

$$[A \oplus B] \oplus C = A \oplus [B \oplus C]$$
$$D \oplus D = 0$$
$$E \oplus 0 = E$$

因此我们有 $LD_1 = RE_{15}$ 及 $RD_1 = LE_{15}$，所以解密过程的第一轮输出为 $RE_{15} \| LE_{15}$，正是加密过程第 16轮输入左右部分互换的值。对于其他各轮也是如此。我们把它表示成一般形式。对于第 i 轮加密算法：

$$LE_i = RE_{i-1}$$
$$RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$$

又可写为

$$RE_{i-1} = LE_i$$
$$LE_{i-1} = RE_i \oplus F(RE_{i-1}, K_i) = RE_i \oplus F(LE_i, K_i)$$

因此，我们已经描述了第 i 轮的输入是输出的函数，这些等式证实了图 3.3 右半部分的正确性。

最后，我们注意到解密过程最后一轮的输出是 $RE_0 \| LE_0$，左右互换的结果正是原始明文，说明 Feistel 密码的解密过程是正确的。

注意，我们在推导过程并没有要求函数 F 是可逆的，比如取一种极端情况，假设它的输出为一个常数(例如全为 1)，等式依然成立。

为了帮助说明前面的概念，我们来看一个特别的例子(参见图 3.4)并且关注加密的第 15 轮，对应于解密的第二轮。假设每一阶段的分组长度为 32 位(分为两个 16 位)，密钥长度为24 位。假设第 14 轮加密后的中间分组是 DE7F03A6。那么 LE_{14} = DE7F，RE_{14} = 03A6。同时假设 K_{15} = 12DE52。15 轮后，我们有 LE_{15} = 03A6，RE_{15} = F(03A6,12DE52)⊕DE7F。

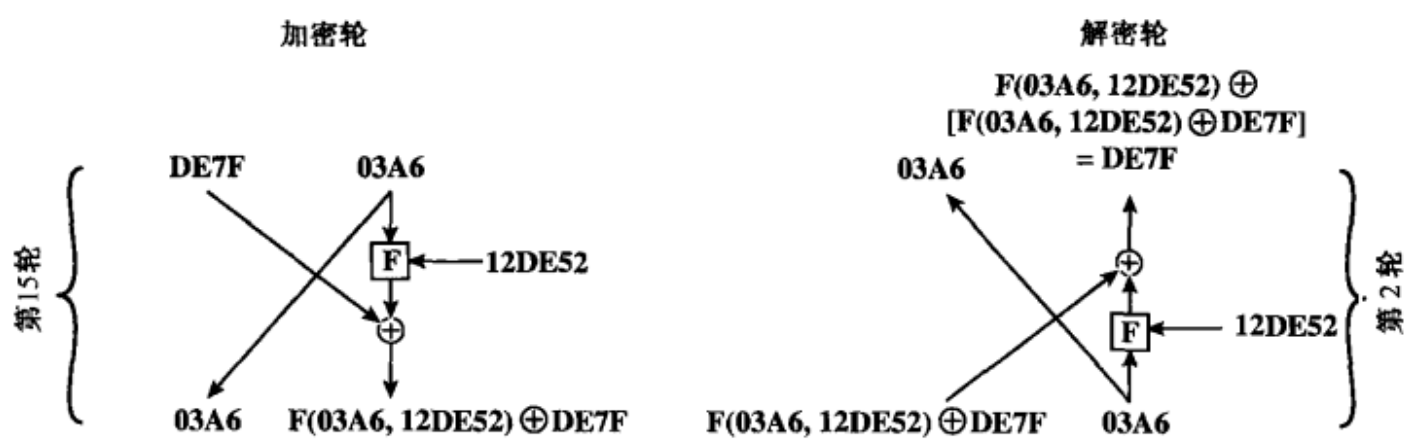


图 3.4　Feistel 示例

现在我们来看解密过程。如图 3.3 所示，假设 $LD_1 = RE_{15}$，$RD_1 = LE_{15}$，我们想证明 $LD_2 = RE_{14}$，$RD_2 = LE_{14}$。所以从 LD_1 = F(03A6,12DE52)⊕DE7F，RD_1 = 03A6 出发，接着从图 3.3有 LD_2 = 03A6 = RE_{14}，RD_2 = F(03A6,12DE52) ⊕[F(03A6,12DE52)⊕DE7F] = DE7F = LE_{14}。

3.2　数据加密标准

使用最广泛的加密体制是数据加密标准(Data Encryption Standard，DES)，它于 1977 年被美国国家标准局(National Bureau of Standard，NBS)，即现在的国家标准和技术协会(National Institute of Standards and Technology，NIST)采纳为联邦信息处理标准 46(FIPS PUB 46)。这个算法本身指的

是数据加密算法(Data Encryption Algorithm,DEA)①。DES 采用了 64 位的分组长度和 56 位的密钥长度。它将 64 位的输入经过一系列变换得到 64 位的输出。解密则使用了相同的步骤和相同的密钥。

DES 使用得很广泛。DES 的安全强度究竟如何一直是争论的话题。为了理解争论的原因,我们先快速回顾一下 DES 的历史。

在 20 世纪 60 年代后期,IBM 公司成立了一个由 Horst Feistel 负责的计算机密码学研究项目。1971 年设计出算法 LUCIFER[FEIS73]后,该项目宣告结束。LUCIFER 被卖给了伦敦的 Lloyd 公司,用在同样由 IBM 公司开发的现金发放系统上。LUCIFER 是分组长度为 64 位、密钥长度为128 位、具有 Feistel 结构的分组密码。因为 LUCIFER 非常成功,IBM 决定开发一个适合于芯片实现的商业密码产品。这一次由 Walter Tuchman 和 Carl Meyer 牵头,参与者不仅有 IBM 公司的研究人员,而且还有美国国家安全局(NSA)的技术顾问。这次努力的结果是给出了 LUCIFER 的一个修订版,它的抗密码分析能力更强,而且密钥长度减小为 56 位,因而很适合于在单片机环境下使用。

1973 年,美国国家标准局(NBS)征求美国国家密码标准方案时,IBM 将 Tuchman-Meyer 方案提交给 NBS,它是所有应征方案中最好的一个,所以 1977 年 NBS 将它采纳为数据加密标准,即 DES。

DES 在被采纳为标准之前,曾受过激烈的批评,其实直到今天也未平息。批评主要集中在两个方面:第一,IBM 公司最开始的 LUCIFER 算法的密钥长度是 128 位,而 DES 却只使用了 56 位的密钥,整整减少了 72 位。批评者一直担心密钥太短而不能抗穷举攻击。第二是 DES 的内部结构,S 盒的设计标准被列入官方机密。所以,用户不能确信 DES 的内部结构是没有弱点的,NSA 有可能利用这些弱点在没有密钥的情况下来解密。后来的事件,特别是近年来差分分析方面的工作表明,DES 的内部结构是强健的。而且按照 IBM 的参与者所说,原始算法中唯一做了改动的是 S 盒,这是由 NSA 建议的,它去掉了测试过程中所发现的算法的一些脆弱性。

不管这件事的价值如何,DES 已经风行全世界了,特别是在金融领域的应用。1994 年 NIST 重新决定将 DES 在美联邦机构使用期延长 5 年。NIST 推荐在一般商业应用中使用 DES,而不用 DES 来保护官方机密。1999 年 NIST 颁布了标准的新版本(FIPS PUB 46-3),新版本规定了 DES 只能用于(历史)遗留系统以及 3DES 的使用。我们将会在第 6 章研究 3DES。因为 DES 与 3DES 的加/解密算法是相同的,所以理解 DES 算法仍然有很重要的意义。

3.2.1 DES 加密

图 3.5 表明了 DES 加密的整个机制。对于任意加密方案,总有两个输入:明文和密钥。DES 的明文长为 64 位,密钥长为 56 位②。

从图中左半部分,可见明文的处理经过了三个阶段。首先,64 位的明文经过初始置换(IP)而被重新排列。然后进行 16 轮相同函数的作用,每轮作用都有置换和代替。最后一轮迭代的输出有 64 位,它是输入明文和密钥的函数。其左半部分和右半部分互换产生预输出。最后预输出再被与初始置换(IP)互逆的置换(IP^{-1})产生 64 位的密文。除了初始和末尾的置换,DES 的结构与图 3.3 所示的 Feistel 密码结构完全相同。

图 3.5 的右半部分给出了使用 56 位密钥的过程。首先,密钥经过一个置换后,再经过循环左

---

① 术语有点混乱,直到现在还经常将 DES 和 DEA 互换使用。然而,最新版的 DES 的文件中包含了此处描述的 DEA,还有第 6 章的三重 DEA。它们都是数据加密标准的一部分。三重 DEA 算法一直被称为三重 DES,并写为 3DES,直到最近,官方术语采用了 TDEA。为了方便,我们采用 3DES。

② 实际上这个密码函数希望采用 64 位的密钥。然而却仅仅采用了 56 位,其余 8 位可以用做奇偶校验或随意设置。

移和一个置换分别得到各轮的子密钥 K_i用于各轮的迭代。每轮的置换函数都一样,但是由于密钥的循环移位使得各轮子密钥互不相同。

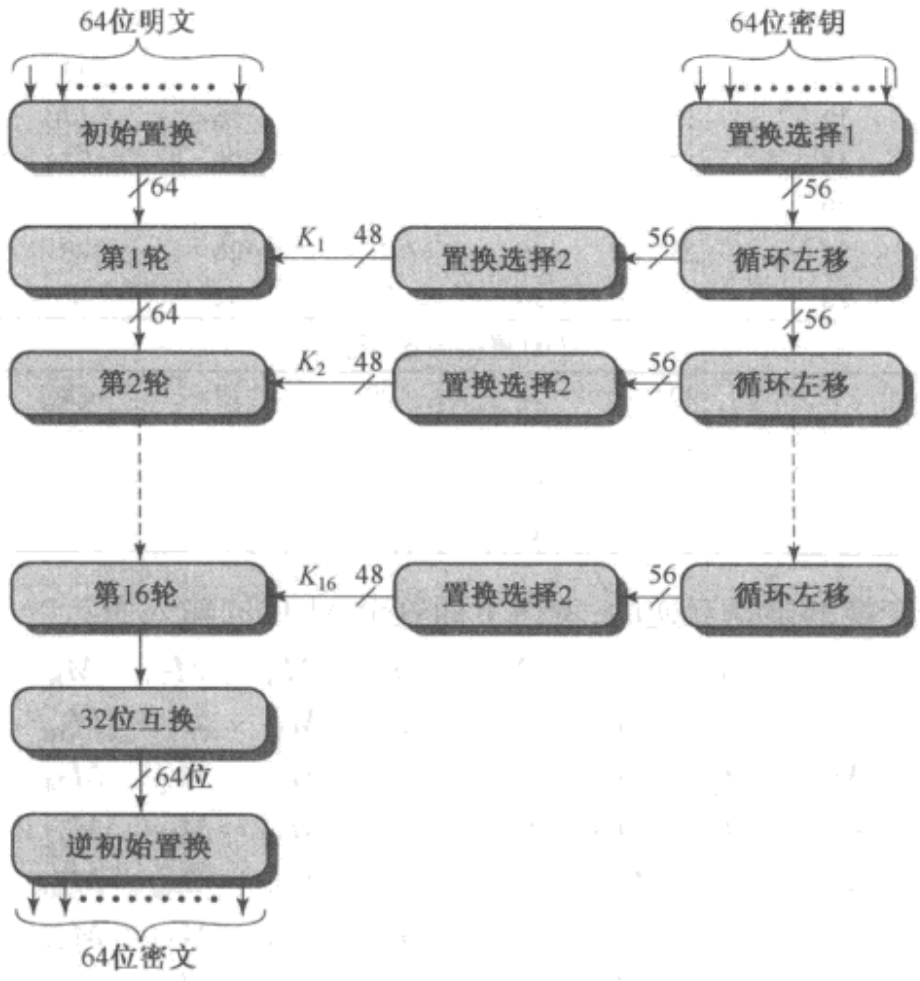


图 3.5　DES 加密算法的总体描述

初始置换

表 3.2(a)和表 3.2(b)分别定义了初始置换及其逆置换,其解释如下。表的输入标记为从 1 到 64,共 64 位。置换表中 64 个元素代表从 1 到 64 这些数的一个置换。置换表中的每个元素表明了某个输入位在 64 位输出中的位置。

表 3.2　DES 的置换表

(a) 初始置换(IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) 逆初始置换(IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
49	17	57	25	[illegible]	[illegible]	33	[illegible]

(续表)

(c)扩展置换(E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(d)置换函数(P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

为了说明这两个变换的确是互逆的,考虑下面这个64位的输入 M:

$$\begin{array}{cccccccc} M_1 & M_2 & M_3 & M_4 & M_5 & M_6 & M_7 & M_8 \\ M_9 & M_{10} & M_{11} & M_{12} & M_{13} & M_{14} & M_{15} & M_{16} \\ M_{17} & M_{18} & M_{19} & M_{20} & M_{21} & M_{22} & M_{23} & M_{24} \\ M_{25} & M_{26} & M_{27} & M_{28} & M_{29} & M_{30} & M_{31} & M_{32} \\ M_{33} & M_{34} & M_{35} & M_{36} & M_{37} & M_{38} & M_{39} & M_{40} \\ M_{41} & M_{42} & M_{43} & M_{44} & M_{45} & M_{46} & M_{47} & M_{48} \\ M_{49} & M_{50} & M_{51} & M_{52} & M_{53} & M_{54} & M_{55} & M_{56} \\ M_{57} & M_{58} & M_{59} & M_{60} & M_{61} & M_{62} & M_{63} & M_{64} \end{array}$$

这里 M_i都是二进制数。经过置换 $X = \mathrm{IP}(M)$之后,得到的 X 是:

$$\begin{array}{cccccccc} M_{58} & M_{50} & M_{42} & M_{34} & M_{26} & M_{18} & M_{10} & M_2 \\ M_{60} & M_{52} & M_{44} & M_{36} & M_{28} & M_{20} & M_{12} & M_4 \\ M_{62} & M_{54} & M_{46} & M_{38} & M_{30} & M_{22} & M_{14} & M_6 \\ M_{64} & M_{56} & M_{48} & M_{40} & M_{32} & M_{24} & M_{16} & M_8 \\ M_{57} & M_{49} & M_{41} & M_{33} & M_{25} & M_{17} & M_9 & M_1 \\ M_{59} & M_{51} & M_{43} & M_{35} & M_{27} & M_{19} & M_{11} & M_3 \\ M_{61} & M_{53} & M_{45} & M_{37} & M_{29} & M_{21} & M_{13} & M_5 \\ M_{63} & M_{55} & M_{47} & M_{39} & M_{31} & M_{23} & M_{15} & M_7 \end{array}$$

如果我们再对它作用逆置换 $Y = \mathrm{IP}^{-1}(X) = \mathrm{IP}^{-1}(\mathrm{IP}(M))$,就会恢复出 M。

每轮变换的详细过程

图3.6给出了一轮变换的内部结构。我们同样先看图的左半部分。64位中间数据的左右两部分作为独立的32位数据,分别记为 L 和 R。在经典的 Feistel 密码中,每轮变换的整个过程可以写为下面的公式:

$$L_i = R_{i-1}$$
$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

轮密钥 K_i长48位,R 是32位。首先将 R 用表3.2(c)定义的置换扩展为48位,其中有16位是重复的。这48位与 K_i异或,所得结果再用一个代替函数作用产生32位的输出,再用表3.2(d)定义的置换进行作用后输出。

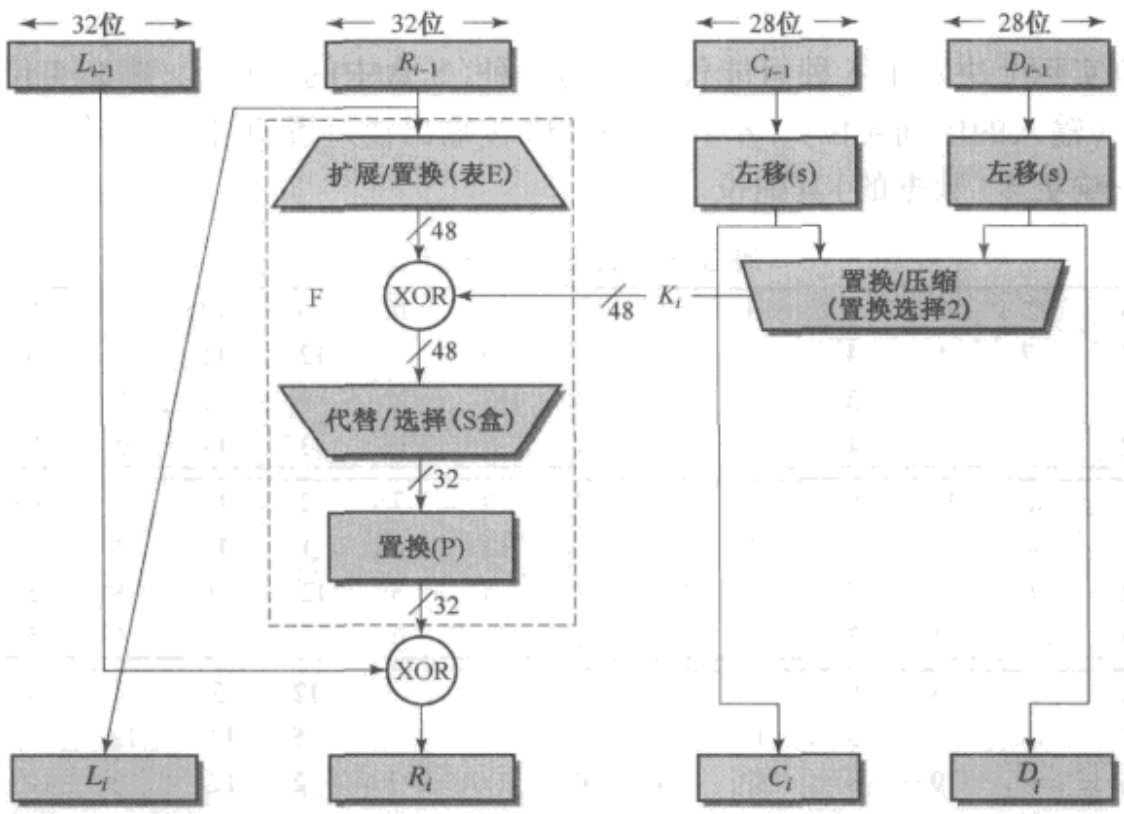


图 3.6　DES 算法一轮迭代的过程

图 3.7 解释了 S 盒在函数 F 中的作用。代替函数由 8 个 S 盒来组成，每个 S 盒都输入 6 位，输出 4 位。这些变换参见表 3.3，其解释如下：盒 S_i 输入的第一位和最后一位组成一个 2 位的二进制数，用来选择 S 盒 4 行代替值中的一行，中间 4 位用来选择 16 列中的某一列。行列交叉处的十进制值转换为二进制之后可得到输出的 4 位二进制数。例如，在 S_1 中，若输入为 011001，则行是 1(01)，列是 12(1100)，该处的值是 9，所以输出为 1001。

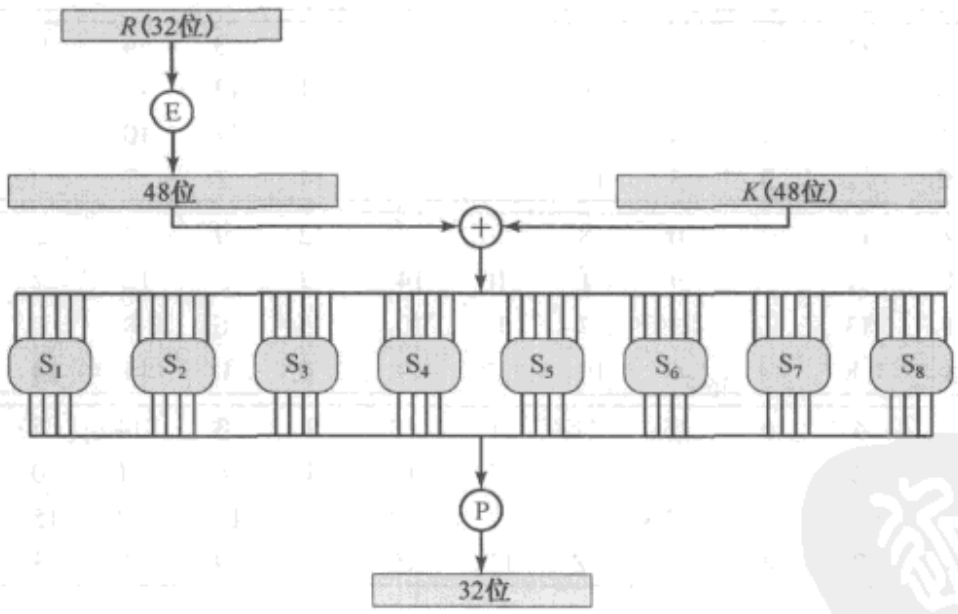


图 3.7　F(*R*,*K*)的计算

S 盒的每行都定义了一个普通的可逆代替。图 3.2 对于理解这里面的映射关系可能有所帮助。图中显示了盒 S_1 第 0 行的代替关系。

S 盒的操作值应进一步讨论。暂且不管子密钥 K_i 的作用。如果仔细研究扩散表，就会发现 32 位输入被 4 位 4 位地分成了 8 组，然后每组再从前后相邻的两组中取得位置靠近的一位，而变成 6 位。例如，如果输入字的部分为

```
... efgh ijkl mnop ...
```

它将变为

```
... defghi hijklm lmnopq ...
```

每组的外面两位实际上决定了4种可能代替中的一种(S盒中的一行)。那么4位输出代替了输入中特殊的4位(输入的中间4位)。8个S盒的32位输出经过置换,使得每个S盒的输出在下一轮中尽可能地影响更多(数据的)其他位。

表3.3 DES的S盒定义

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

密钥产生

回到图3.5和图3.6中,我们看到算法输入了64位的密钥,密钥各位分别标记为1到64。如表3.4(a)中没有阴影的部分,也就是每行第8个位被忽略。首先用标记为置换选择1的表[参见表3.4(b)]作用。所得56位密钥分为两个28位数据C_0和D_0。每轮迭代中,C_{i-1}和D_{i-1}分别循环左移(或旋转)一位或两位,具体左移位数参见表3.4(d)。移位后的值作为下一轮的输入。它们同时也作为置换选择2[参见表3.4(c)]的输入,将产生一个48位的输出作为函数$F(K_{i-1},K_i)$的输入。

表 3.4　DES 密钥的使用

(a) 输入密钥

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

(b) 置换选择 1(PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) 置换选择 2(PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) 左移次数的确定

迭代轮数	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
移位次数	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

3.2.2　DES 解密

Feistel 密码的解密算法与加密算法是相同的，只是子密钥的使用次序相反。

3.3　DES 的一个例子

我们现在跟踪一个例子并考虑它的含义。尽管不要求你去手工重复该例子，但是你将发现研究一下例子中十六进制模式的数据从某一步到下一步的变化是非常有用的。

本例中，明文是一个十六进制的回文，明文、密钥和所得密文如下：

明文:02468aceeca86420

密钥:0f1571c947d9e859

密文:da02ce3a89ecac3b

3.3.1　结果

表 3.5 显示了算法的过程。第一行是经过初始置换后数据的左右两边的 32 位值。下面的 16 行显示的是各轮运行后的结果。表中也显示了各轮的 48 位子密钥。注意 $L_i = R_{i-1}$。最后一行显示的是经过逆初始置换后左边和右边的值。这两个值组合起来形成密文。

表 3.5 DES 的例子

轮 数	K_i	L_i	R_i
IP		5a005a00	3cf03cf0f
1	1e030f03080d2930	3cf03cf0f	bad22845
2	0a31293432242318	bad22845	99e9b723
3	23072318201d0c1d	99e9b723	0bae3b9e
4	05261d3824311a20	0bae3b9e	42415649
5	3325340136002c25	42415649	18b3fa41
6	123a2d0d04262a1c	18b3fa41	9616fe23
7	021f120b1c130611	9616fe23	67117cf2
8	1c10372a2832002b	67117cf2	c11bfc09
9	04292a380c341f03	c11bfc09	887fbc6c
10	2703212607280403	887fbc6c	600f7e8b
11	2826390c31261504	600f7e8b	f596506e
12	12071c241a0a0f08	f596506e	738538b8
13	300935393c0d100b	738538b8	c6a62c4e
14	311e09231321182a	c6a62c4e	56b0bd75
15	283d3e0227072528	56b0bd75	75e8fd8f
16	2921080b13143025	75e8fd8f	25896490
IP^{-1}		da02ce3a	89ecac3b

3.3.2 雪崩效应

明文或密钥的微小改变将对密文产生很大的影响,是任何加密算法需要的一个好性质。特别地,明文或密钥的某一位发生变化会导致密文的很多位发生变化。这被称为雪崩效应。如果相应的改变很小,可能会给分析者提供缩小搜索密钥或明文空间的渠道。

运用表3.5 的例子,表3.6 显示了当明文的第4 位改变时的结果,此时明文是12468aceeca86420。表的第二列显示的是这两个明文在每轮结束后得到的64 位值。第三列显示的是这两个值所差的位数量。该表表明,仅仅经过 3 轮,两组数据差别达到了 18 位,当 16 轮全部结束时,两组密文差32 位。

表 3.6 DES 的雪崩效应:改变明文

轮 数		δ	轮 数		δ
	02468aceeca8642012468aceeca86420	1	9	c11bfc09887fbc6c99f911532eed7d94	32
1	3cf03c0fbad228453cf03c0fbad32845	1	10	887fbc6c600f7e8b2eed7d94d0f23094	34
2	bad2284599e9b723bad3284539a9b7a3	5	11	600f7e8bf596506ed0f23094455da9c4	37
3	99e9b7230bae3b9e39a9b7a3171cb8b3	18	12	f596506e738538b8455da9c47f6e3cf3	31
4	0bae3b9e42415649171cb8b3ccaca55e	34	13	738538b8c6a62c4e7f6e3cf34bc1a8d9	29
5	4241564918b3fa41ccaca55ed16c3653	37	14	c6a62c4e56b0bd754bc1a8d91e07d409	33
6	18b3fa419616fe23d16c3653cf402c68	33	15	56b0bd7575e8fd8f1e07d4091ce2e6dc	31
7	9616fe2367117cf2cf402c682b2cefbc	32	16	75e8fd8f258964901ce2e6dc365ef59	32
8	67117cf2c11bfc092b2cefbc99f91153	33	IP^{-1}	da02ce3a89ecac3b057cde97d7683f2a	32

表 3.7 显示的是一个类似的测试,仍然使用原明文,使用的两个密钥仅在第 4 位有差异:初始密钥为 0f1571c947d9e859,改变后的密钥为 1f1571c947d9e859。同样,结果表明密文中大约有一半的位发生改变,且仅仅经过几轮的变换就出现了雪崩效应。

表 3.7　DES 的雪崩效应:改变密钥

轮	数	δ	轮	数	δ
	02468aceeca8642002468aceeca86420	0	9	c11bfc09887fbc6c548f1de471f64dfd	34
1	3cf03c0fbad228453cf03c0f9ad628c5	3	10	887fbc6c600f7e8b71f64dfd4279876c	36
2	bad2284599e9b7239ad628c59939136b	11	11	600f7e8bf596506e4279876c399fd3d	32
3	99e9b7230bae3b9e9939136b768067b7	25	12	f596506e738538b8399fdc0d6d208dbb	28
4	0bae3b9e42415649768067b75a8807c5	29	13	738538b8c6a62c4e6d208dbbb9bdeaaa	33
5	4241564918b3fa415a8807c5488dbe94	26	14	c6a62c4e56b0bd75b9bdeeaad2c3a56f	30
6	18b3fa419616fe23488dbe94aba7fe53	26	15	56b0bd7575e8fd8fd2c3a56f2765c1fb	33
7	9616fe2367117cf2aba7fe53177d21e4	27	16	75e8fd8f258964902765c1fb01263dc4	30
8	67117cf2c11bfc09177d21e4548f1de4	32	IP^{-1}	da02ce3a89ecac3bEe92b50606b62b0b	30

3.4　DES 的强度

自从 DES 被采纳为美联邦标准,对它的安全性就一直争论不休,焦点主要集中于密钥长度和算法本身的性质上。

3.4.1　56 位密钥的使用

56 位的密钥共有 2^{56} 种可能,这个数字大约为 7.2×10^{16}。所以穷举攻击明显是不太实际的。平均而言,要搜索一半的密钥空间。一台每毫秒执行一次 DES 加密的计算机需要用一千年才能破译出密文(参见表 2.2)。

然而,每毫秒执行一次加密运算的假设过于保守。早在 1977 年 Diffie 和 Hellman 就指出,若用现有的技术去制造一台并行机,它带有 100 万个加密器,每一个加密器都可以在 1 ms 内执行一次加密运算[DIFF77],那么平均的穷举时间将减少为大约 10 个小时。他们估计这台计算机的造价在当时可能为 2000 万美元。

1998 年 7 月,当电子前哨基金会(Electronic Frontier Foundation,EFF)宣布一台造价不到 25 万美元的特殊设计的机器"DES 破译机"破译了 DES 时,DES 终于被清楚地证明是不安全的。这次攻击所花的时间不到三天。EFF 还公布了这台机器的细节,使其他人也能建造自己的破译机[EFF98]。当然,随着速度的提高,硬件造价的下降,最终会导致 DES 毫无价值。

不过,要进行真正的穷举攻击,仅仅靠简单地将所有可能的密钥代入到程序中去执行是不够的。如果不提供已知明文,分析者必须能够从不同密钥解密的所谓明文中辨认出真正的明文。如果消息是用英文写的,即使要求达到自动辨认,也是很好办的。但是如果文本信息在加密之前进行了压缩,识别就太困难了。或者原始信息是更一般的数据类型,比如数据文件,还进行了压缩,那么就几乎不可能实现自动化识别。因此要进行穷举攻击,需要事先知道一些有关期望明文的知识,并且需要将正确的明文从可能的明文堆里辨认出来的自动化方法。EFF 考虑了这个问题,而且还引进了在很多环境很有效的自动化技术。

幸运的是,有大量 DES 的替代算法,最重要的有 AES 和 3DES,我们将在第 5 章、第 6 章分别加以讨论。

3.4.2　DES 算法的性质

人们关心的另外一件事是,密码分析者有没有利用 DES 算法本身的特征来攻击它的可能性。问题集中在每轮迭代所用的 8 个代替表,即 S 盒身上。因为这些 S 盒的设计标准,实际上包括整个算法的设计标准是不公开的,因此人们怀疑密码分析者若是知道 S 盒的构造方法,就可能知道

S 盒的弱点,进而攻击 DES。这种说法很诱人,多年来人们的确发现了 S 盒的许多规律和一些缺点。尽管如此,仍然没有人发现 S 盒存在致命的弱点①。

3.4.3 计时攻击

由于计时攻击与公钥算法有关,我们将在第二部分详细讨论。然而它也与对称密码有些关系。本质上计时攻击是通过观察算法的一个既定实现对多种密文解密所需的时间,来获得关于密钥或明文的信息。计时攻击所利用的事实是加密或解密算法对于不同的输入所花的时间有着细微的差别。参考文献[HEVI99]给出了利用计时攻击获得密钥汉明权重(二进制串中 1 的位数)的一种方法。这离知道实际密钥还很遥远,但这的确是很让人感兴趣的一步。作者给出的结论是 DES 似乎能够很好地抵抗计时攻击,但是也给出了一些可能的研究建议。尽管这是很有趣的攻击路线,但是到目前为止,它还不可能成功地攻击 DES,更不能攻击更强的对称密码,如 3DES 和 AES。

3.5 差分分析和线性分析

自从 DES 出现后,人们就很关心它究竟能否经受住穷举攻击。毕竟,它的密钥长度只有 56 位。不过,用密码分析的方法来攻击 DES 也是很有趣的事情。随着包括 3DES 在内的长密钥分组密码的逐渐增多,穷举攻击自然是越来越不实际。因此用密码分析的方法来攻击 DES 及其他分组密码就越来越受人们重视了。本节中,我们对其中两种最有希望的强有力的分析方法做一个概述,它们是差分密码分析和线性密码分析。

3.5.1 差分密码分析

近年来密码分析领域最重要的进展之一就是差分分析。本节中我们将讨论该技术以及该技术对 DES 的适用性。

历史

直到 1990 年,差分密码分析才公开发表。已知最早的研究是 Murphy 用它来分析分组密码 FEAL[MURP90]。接着 Biham 和 Shamir 发表了数篇文章,对多种加密算法和 Hash 函数进行了此类攻击,结果汇集在[BIHA93]中。

有关这种方法发表的大多数结果在分析 DES 上有应用。差分分析是第一个公开的能对 DES 在小于 2^{55} 复杂度情况下攻击成功的方法。[BIHA93]中有整个攻击方案。它表明,若有 2^{47} 个选择明文,用差分分析就可以在 2^{47} 次加密运算内成功攻击 DES。尽管 2^{47} 比 2^{55} 小得多,但是要拥有 2^{47} 个选择明文的条件使得这种方法只具有理论上的意义。

尽管差分分析是一个强有力的密码攻击方法,但它对 DES 并不十分奏效。根据设计 DES 的 IBM 小组一个成员所述,原因是该小组早在 1974 年就知道了差分分析。为了提高 DES 抗差分攻击的能力,在设计 S 盒和置换 P 的时候做了充分考虑。参考文献[BIHA93]对这些做了相应的论述,可以证明知道差分分析之后对设计 DES 产生了很大的影响。用差分分析对 8 层迭代的 LUCIFER 算法只需 256 个选择明文,而对 8 层迭代的 DES 则需 2^{14} 个选择明文。

差分密码攻击

差分密码攻击非常复杂,参考文献[BIHA93]对此有完整的描述。差分密码分析的合理性在

① 至少没有人公开宣布这样的发现。

于,它关注了一对明文分组在加密过程中通过轮函数的演变情况,而不是观察单个明文分组的演变。这里,我们只对它进行简单的描述,只要能看出它的特点就行了。

下面讨论的 DES 算法中符号有所改变。设原始明文分组 m 被分成两部分 m_0、m_1。DES 的每一轮迭代都将其右侧的输入映射为左侧的输出,而将左侧的输入和该轮子密钥经运算后作为右手的输出。所以在每一轮迭代中,仅产生 32 位新数据。如果我们将其记为 $m_i(2 \leqslant i \leqslant 17)$,可得以下关系式:

$$m_{i+1} = m_{i-1} \oplus \mathrm{f}(m_i, K_i), \qquad i = 1, 2, \cdots, 16$$

在差分密码分析中,我们从两条已知异或 XOR 差分 $\Delta m = m \oplus m'$ 的明文 m 和 m' 开始,考虑中间数据的差分 $\Delta m_i = m_i \oplus m_i'$。我们有

$$\begin{aligned}\Delta m_{i+1} &= m_{i+1} \oplus m'_{i+1} \\ &= [m_{i-1} \oplus \mathrm{f}(m_i, K_i)] \oplus [m'_{i-1} \oplus \mathrm{f}(m'_i, K_i)] \\ &= \Delta m_{i-1} \oplus [\mathrm{f}(m_i, K_i) \oplus \mathrm{f}(m'_i, K_i)]\end{aligned}$$

现在,假设函数 f 有许多对具有相同差分的输入,当使用相同的子密钥时产生相同差分的输出。更精确地说,若差分为 X 的所有输入,产生差分为 Y 的输出占所有输出的百分比为 p,我们就称由差分 X 导致差分 Y 的概率为 p。假设有许多 X 都会以很高的概率产生某些特定的差分。因此,若以很高的概率知道 Δm_{i-1} 和 Δm_i,就能以很高的概率知道 Δm_{i+1}。而且,如果知道很多那样的差分数据,那么确定函数 f 所使用的子密钥就是可行的。

单轮的差分密码分析策略就基于上述考虑。步骤是这样的:首先,两段具有给定差分的明文 m 和 m',跟踪它们每一轮迭代后产生的差分概率模式,形成密文的概率差分。实际上输出的两个 32 位数据($\Delta m_{17} \parallel \Delta m_{16}$)有两个差分概率模式。然后我们给出 m 和 m' 在未知密钥下加密的实际差分,将结果与概率差分比较。如果它们是匹配的,即

$$\mathrm{E}(K, m) \oplus \mathrm{E}(K, m') = (\Delta m_{17} \| \Delta m_{16})$$

那么我们可猜测所有中间各轮的所有概率模型是正确的。根据这个假设,可以对密钥的某些位进行推导。这个过程可能要重复很多次,才能确定整个密钥。

图 3.8 基于参考文献[BIHA93]的一幅图形,它表明了 DES 经过三轮迭代后差分的扩散情况。图右边的概率指定给作为输入差分函数的中间差分出现的概率。经过三轮迭代后,如图给出的输出差分的概率为 $0.25 \times 1 \times 0.25 = 0.0625$。

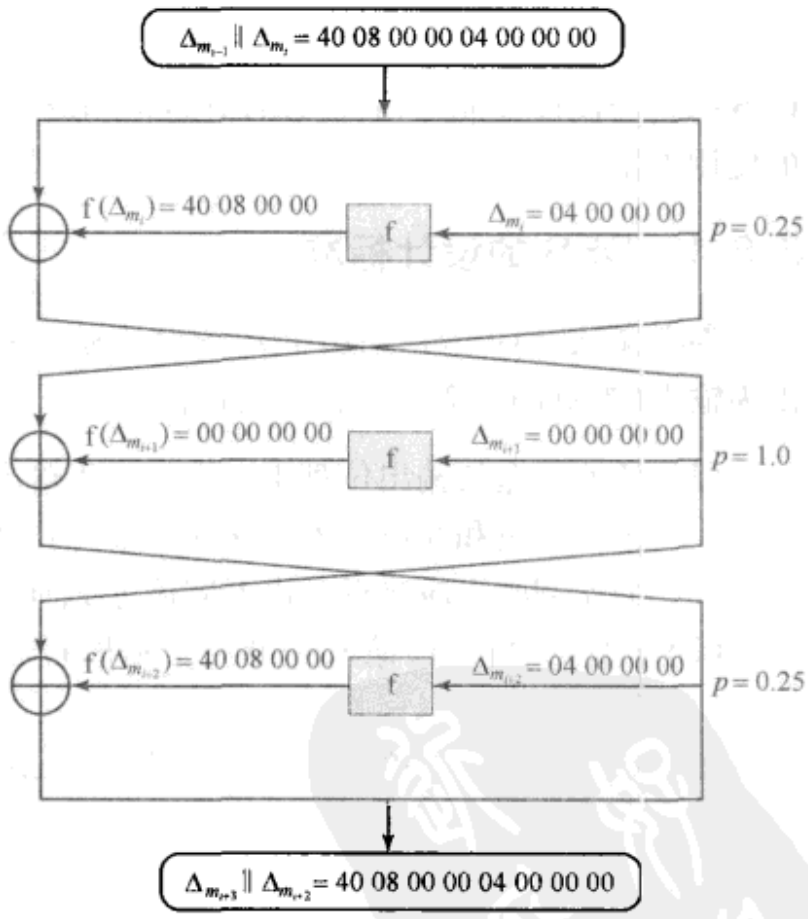


图 3.8　经过三轮迭代的差分传播
(图中数据为十六进制)

3.5.2　线性密码分析

最近的一种密码分析方法是[MATS93]中所描述的线性密码分析。这种方法通过寻找 DES 变换的线性近似来进行攻击。这种方法只需知道 2^{43} 个明文就可以找出 DES 的密钥,而用差分分析则需知道 2^{47} 个选择明文。尽管获取明文比获取选择明文容易得多,但是这只是一个小的改进,

线性密码分析对于攻击 DES 还是不可行的。到现在为止,几乎没有其他研究组做工作来验证线性分析方法。

先简要叙述一下线性密码分析的基本原理。对于 n 位的明文分组和密文分组,m 位密钥的密码,记明文分组为 $P[1],\cdots,P[n]$,密文分组为 $C[1],\cdots,C[n]$,密钥为 $K[1],\cdots,K[m]$。定义

$$A[i,j,\cdots,k] = A[i] \oplus A[j] \oplus \cdots \oplus A[k]$$

线性密码分析的目标是找到一个下列形式的有效线性等式:

$$P[\alpha_1,\alpha_2,\cdots,\alpha_a] \oplus C[\beta_1,\beta_2,\cdots,\beta_b] = K[\gamma_1,\gamma_2,\cdots,\gamma_c]$$

(其中 $x=0$ 或 1;$1 \leqslant a,b \leqslant n$;$c \leqslant m$;$\alpha,\beta$ 和 γ 代表固定且唯一的位位置)要求上式成立的概率为 $p \neq 0.5$。p 离 0.5 越远,等式就越有效。一旦确定了这种关系,实际过程就是对大量明、密文对计算上述等式的左半部分。若多数情况下结果为 0,则假设 $K[\gamma_1,\gamma_2,\cdots,\gamma_c]=0$,若多数情况下结果为 1,则假设 $K[\gamma_1,\gamma_2,\cdots,\gamma_c]=1$。这就给出了密钥位的一个线性等式。尽量获得较多的上述等式以便我们从中解出密钥。因为我们处理的是线性等式,所以可以一次处理一轮迭代,把这些结果组合起来就能解决我们的问题。

3.6 分组密码的设计原理

尽管在设计具有强密码学意义的分组密码方面有了很大的进展,但是自 20 世纪 70 年代早期 Feistel 和 DES 设计小组所做的工作以来,基本设计原理并没有很大的改变。讨论之前我们先看看 DES 的公开设计标准,然后再看看分组密码设计的三个主要问题:加密轮数、函数 F 的设计和密钥的使用方案。

3.6.1 DES 的设计标准

参考文献[COPP94]给出了 DES 设计标准,主要针对 S 盒及 P 函数的设计,P 函数以 S 盒的输出为输入(参见图 3.7)。S 盒的设计标准如下:

(1) 任何 S 盒的输出位都不应太接近于输入位的线性函数。特别地,如果我们选择了任何一个输出位和 6 位输入的任何子集,输入位的异或值等于这个输出位的输入个数占总输入的比例既不应接近 0 也不应接近 1,而应该接近于 0.5。

(2) S 盒的每行(由输入的最左位和最右位决定的固定值)应包含所有的 16 种可能输出位组合。

(3) 对 S 盒的两个输入,若仅有 1 位不同,输出至少有 2 位不同。

(4) 对 S 盒的两个输入,若中间 2 位不同,输出至少有 2 位不同。

(5) 对 S 盒的两个输入,若前面 2 位不同而后 2 位相同,输出不应相同。

(6) 对任意非零 6 位输入差分,具有该输入差分的 32 对输入中,至多有 8 对输入有着相同的输出差分。

(7) 这条标准与前一条类似,但是对三个 S 盒的情形而言的。

Coppersmith 指出,上述第一条标准的必要性在于 S 盒是 DES 中唯一的非线性部分。如果 S 盒是线性的(即 S 盒的输出是输入的线性组合),那么整个算法就是线性的,很容易攻破。这个现象我们在讨论线性的 Hill 密码时已经看到了。其余的标准主要用来抗差分分析以及提供好的混淆性质。

置换 P 的标准如下:

(1) 在第 i 轮的每个 S 盒的 4 位输出中，有 2 位影响第 $i+1$ 轮的中间 2 位，另 2 位则影响两端的位。S 盒的 6 位输入中的中间 2 位不与相邻 S 盒共用，而左右各 2 位则与相邻 S 盒共用。

(2) S 盒的 4 位输出影响下一轮的 6 个不同 S 盒，且没有 2 位能影响相同的 S 盒。

(3) 对 S 盒 j 和 k，若 S_j 的某位输出影响了下一轮 S_k 的中间某位，那么 S_k 的某位输出不能影响 S_j 的中间某位。这隐含着对于 $j=k$，S_j 的输出某位不影响它的中间某位。

这些标准的目的在于增加算法的扩散程度。

3.6.2 迭代轮数

Feistel 密码的强度来自于三个方面：迭代轮数、函数 F、密钥使用算法。首先来看轮数的选择。

迭代轮数越多，密码分析就越困难，即使是函数 F 相对较弱也同样适用。一般说来，迭代轮数的选择标准是使密码分析的难度大于简单穷举攻击的难度。这个标准当然也用到了 DES 的设计中。Schneier(参见[SCHN96])观察到，对 16 层迭代的 DES，差分密码分析比穷举攻击的效率要差一点：差分密码分析需要 $2^{55.1}$ 次操作①，而穷举攻击平均则需要 2^{55} 次操作。如果 DES 只有 15 层迭代或更少，差分密码分析比穷举攻击的效率就要高一些。

这个标准是很有吸引力的，因为它使得判别算法强度和比较算法优劣变得容易。如果在密码分析方面没有突破，则任何满足这个标准的算法强度仅需要从密钥长度判断。

3.6.3 函数 F 的设计

Feistel 密码的核心是函数 F。正如我们在 DES 中所见，这个函数依赖于 S 盒的使用。大多数分组对称密码皆是如此。然而我们可以先对函数 F 的设计标准做一般性评价，然后再专门讨论S 盒的设计。

F 的设计标准

函数 F 给 Feistel 密码注入了混淆的成分。因此难以破解由 F 实现的这个代替。F 的明显标准是非线性，这一点在前面我们已经讨论过。F 的非线性成分越多，任何形式的分析就会越困难。非线性有多种度量方法，但这个话题超出了本书的主题，在此不多赘述。粗略说来，越难将 F 近似表示为某些线性等式，F 的非线性度就越高。

设计 F 时还应考虑其他几个标准。我们希望算法有较好的雪崩效应，即输入的一位变化应该引起输出的很多位变化。一个更严格的定义是**严格雪崩效应准则**(Strict Avalanche Criterion，SAC)，参见参考文献[WEBS86]，即对于所有的 i 和 j，它要求若 S 盒的输入的任意一位 i 发生变化，输出的任意一位 j 发生变化的可能性为 1/2。尽管 SAC 是对 S 盒而言的，作为一个标准它同样适用于整个 F。当 F 中不含 S 盒时，这条准则很重要。

参考文献[WEBS86]中建议的另一项标准是 **BIC 标准**(Bit Independent Criterion)，即对任意的 i,j,k，当输入中的一位 i 发生变换时，输出中的位 j 和位 k 的变化应是彼此无关的。SAC 和 BIC 明显是为了加强混淆的有效性。

S 盒的设计

对称分组密码的研究中，S 盒的设计是最热门的领域之一，有关方面的文献亦是浩如烟海②。

---

① 前面讲过，对 DES 进行密码分析需要 2^{47} 个选择明文。如果仅有已知明文，就需要大量的已知明密文对，并从中找到一个有用的。这样就使工作量上升到 $2^{55.1}$。

② [SCHN96]含有很多关于 S 盒设计得很好的总结。

这里我们叙述几个一般原理。本质上,我们希望输入向量的任何变化导致 S 盒的输出产生近乎随机的变化,这种关系应是非线性的并且难以用线性函数逼近。

S 盒的明显特征是其大小。$n \times m$ 的 S 盒有 n 位输入,m 位输出。DES 的 S 盒是 6 × 4,而 Blowfish 算法采用 8 × 32 的 S 盒。一般说来,S 盒越大,抗差分分析和线性分析的能力就越强[SCHN96]。而另一方面,n 越大,要查的表(指数式)也就越大。所以,出于实际应用考虑,n 通常在 8 到 10 之间。另一个实际考虑是,S 盒越大,设计起来也就越困难。

S 盒的组织通常与 DES S 盒的方式不同,一个 $n \times m$ 的 S 盒一般包括 2^n 行,每行有 m 位,输入的 n 位可用来选择行数,而该行的 m 位则是输出。例如,一个 8 × 32 的 S 盒,输入为 00001001,输出则是第 9 行的 32 位数。

Mister 和 Adams[MIST96]提出了一系列 S 盒的设计标准,包括 S 盒应该满足 SAC 和 BIC。Mister 和 Adams 还建议 S 盒各列的线性组合应该是 bent 的。bent 函数是一种特殊的布尔函数,根据一定的数学衡量标准,它具有很高的非线性度[ADAM90]。用 bent 函数来设计和分析 S 盒已引起了人们很大的兴趣。

参考文献[HEYS95]提出并分析了 S 盒的一个相关设计标准。作者如下定义**保证雪崩(GA)**标准:S 盒满足阶为 γ 的 GA 是指,1 位输入的变化,至少引起 γ 位输出的变化,作者的结论是:若算法能够提供阶为 2 至 5 之间的 GA,则它具有强扩散特征。

对于像 8 × 32 这样的较大 S 盒,如何找到最好方法来选择 S 盒符合上述标准,是很重要的问题。Nyberg 写了许多有关 S 盒设计的理论和实现的文章,提出了下列方法[ROBS95b]:

- **随机方法**:使用伪随机数发生器或者随机数表来产生 S 盒的元素。在 S 盒较小时这可能导致一些不太想要的特征,比如 6 × 4,但是在 S 盒较大时(如 8 × 32)还是可以接受的。
- **随机加测试方法**:随机选择 S 盒的元素,按照各种标准进行测试,然后舍弃那些不能通过的元素。
- **人工构造方法**:利用简单的数学知识,再或多或少地用些手工方法。DES 的 S 盒的设计明显就是这种方法。在 S 盒较大时用这种方法很困难。
- **数学构造方法**:根据数学原理生成 S 盒,用这种办法生成的 S 盒安全性很高,可以抵抗差分分析和线性分析,且有很好的扩散效果。

上述第一种方法可以做一些改动,即 S 盒是随机且与密钥相关的,Blowfish 算法便是如此,它的 S 盒最初是由伪随机数填充的,根据密钥而改变内容。与密钥相关的 S 盒的最大优点是,因为它们不固定,所以预先分析 S 盒以找出其弱点不可行。

3.6.4 密钥扩展算法

分组密码设计的最后一个方面是密钥扩展算法,它没有受到像 S 盒那样的关注。所有的 Feistel 分组密码,密钥在每轮迭代都要产生一个子密钥。一般说来,子密钥的选择应该加大推导子密钥及密钥种子的难度。目前还没有这方面的一般原理见诸报道。

Hall 指出密钥扩展算法至少应保证密钥和密文符合严格雪崩标准和位独立标准[ADAM94]。

3.7 推荐读物和网站

有关对称加密算法的文献浩如烟海,这里列出其中较有价值的一部分。基本参考书有[SCHN96],这部巨著描述了它出版之前几乎所有发表的密码算法和协议。作者搜集了期刊、会议

论文集、政府出版物和标准文件等，并将它们做了综合整理，全面且容易理解。另外一本有价值的较详细的文献是[MENE97]。参考文献[STIN06]则侧重严密的数学推理。

前面的参考文献也覆盖了公钥密码学。

或许对 DES 描述最详细的文献是[SIMO95]，这本书也广泛讨论了 DES 的差分分析和线性分析。参考文献[BARK91]给出了针对 DES 结构的可读性强而有趣的分析，也给出了它的潜在密码学分析方法。[EFF98]详细介绍了 DES 最有效的穷举攻击。参考文献[COPP94]考察了 DES 的内在强度和抗密码学分析的能力。读者还会发现 J. Orlin Grable 的文章 *The DES Algorithm Illustrated* 也值得一读，本书的网站可以找到该文章。[HEYS02]给出了线性分析和差分分析的极好描述。

BARK91 Barker, W. *Introduction to the Analysis of the Data Encryption Standard (DES)*. Laguna Hills, CA: Aegean Park Press, 1991.

COPP94 Coppersmith, D. "The Data Encryption Standard (DES) and Its Strength Against Attacks." *IBM Journal of Research and Development*, May 1994.

EFF98 Electronic Frontier Foudation. *Cracking DES: Secrets of Encryption Research, Wiretap Politics, and Chip Design*. Sebastopol, CA: O'Reilly, 1998.

HEYS02 Heys, H, "A Tutorial on Linear and Differential Cryptanalysis." *Cryptologia*, July 2002.

MENE97 Menezes, A.; van Ooorshcot, P.; and Vanstone, S. *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1997.

SCHN96 Schneier, B. *Applied Cryptography*. New York: Wiley, 1996.

SIMO95 Simovits, M. *The DES: An Extensive Documentation and Evaluation*. Laguna Hills, CA: Aegean Park Press, 1995.

STIN06 Stinson, D. *Cryptography: Theory and Practice*. Boca Raton, FL: Chapman&, 2006.

推荐网站

- Block Cipher Hospital：含有许多论文和分组密码分析材料的链接。

3.8　关键术语、思考题和习题

关键术语

雪崩效应	扩散	乘积密码
分组密码	Feistel 密码	可逆映射
混淆	不可逆映射	轮
数据加密标准(DES)	密钥	轮函数
差分密码学分析	线性密码学分析	子密钥
置换	代替	

思考题

3.1　为什么说研究 Feistel 密码很重要？

3.2　分组密码和流密码的差别是什么？

3.3　为什么使用表 3.1 所示的任意可逆代替密码不实际？

3.4 什么是乘积密码?

3.5 混淆与扩散的差别是什么?

3.6 哪些参数与设计选择决定了实际的 Feistel 密码算法?

3.7 设计 DES S 盒的目的是什么?

3.8 解释什么是雪崩效应。

3.9 差分分析与线性分析的区别是什么?

习题

3.1 (a) 在 3.1 节的名为"Feistel 密码结构设计动机"的小节中说到,对于 n 位的分组长度,理想分组密码的不同可逆映射个数为 $2^n!$ 个。请证明。

(b) 在同样的小节里说到,对于理想分组密码,若允许所有可逆映射,则密钥的长度为 $n \times 2^n$ 位。但是,如果有 $2^n!$ 个映射,则应该需要 $\log_2 2^n!$ 个位就可以区分这 $2^n!$ 个映射。所以密钥长度应为 $\log_2 2^n!$。然而,$\log_2 2^n! < n \times 2^n$。请解释这种差别。

3.2 考虑分组长度为 128 位、密钥长度为 128 位的 16 轮 Feistel 密码。假设对于给定的 k,前 3 个轮密钥 $k_1, \cdots, k_8$ 由密钥扩展算法决定,而设定 $k_9 = k_8, k_{10} = k_7, \cdots, k_{16} = k_1$。假设你有密文 c,请解释你如何只向加密 oracle 做一次提问,而解密 c 获得明文 m? 这表明上述的密码易于被选择明文攻击(可以认为加密 oracle 就是一种装置,给定一个明文,返回相应的密文。装置的内部结构是未知的。当然你也不能打开装置来查看,你所能做的就是对其进行提问而观察相应的输出)。

3.3 考虑分组长度为 n 的分组加密算法,设 $N = 2^n$。比如说我们有 t 个明密文对 $P_i, C_i = \mathrm{E}(K, P_i)$,这里我们假设密钥 K 选择了 $N!$ 种映射中的一种。如果我们希望用穷举的方法找出 K。可以产生密钥 K',并对这 t 个明密文对测试 $C_i = \mathrm{E}(K', P_i)(1 \leqslant i \leqslant t)$ 是否成立。若成立,我们就有理由说 $K = K'$。然而也有这样的情况,恰好 $\mathrm{E}(K, \cdot)$ 和 $\mathrm{E}(K', \cdot)$ 对这 t 个明密文对的结果一样,但其他明密文对不相同。

(a) $\mathrm{E}(K, \cdot)$ 和 $\mathrm{E}(K', \cdot)$ 确实表示不同映射的概率有多大?

(b) $\mathrm{E}(K, \cdot)$ 和 $\mathrm{E}(K', \cdot)$ 对另外 $t'(1 \leqslant t' \leqslant N - t)$ 个明密文对正好都成立的概率有多大?

3.4 设 π 表示整数 $0, 1, \cdots, 2^n - 1$ 的一个置换。$\pi(m)$ 表示 m 的置换值,其中 $0 \leqslant m < 2^n$。换句话说,π 将 n 位的整数集合映射到其自身,并且没有两个整数映射到相同的整数。DES 就是 64 位整数的一个置换。若存在 m 满足 $\pi(m) = m$,我们称 π 有一个不动点。也就是说,如果 π 是一个加密映射,则不动点意味着一段消息加密后仍为它自己。我们感兴趣的是 π 没有不动点的可能性有多大。证明如下有点意外的结论:多于 60% 的映射将至少有一个不动点。

3.5 考虑由表 3.3 中 S 盒 S_1 的第一行定义的代替。给出对应这个代替的类似于图 3.2 的分组图。

3.6 假设密文和密钥的各位为 1,计算 DES 解密时第一轮输出中的 1 16 33 48 位。

3.7 假设无论输入密钥 K 为何值,DES 的 F 函数总是将每一个 32 位的输入 R 映射为:

(a) 32 位的全 1 串

(b) R 的按位补

问:1. DES 算法起什么作用?

2. 解密看上去如何?

提示:运用异或运算的如下性质:

$$(A \oplus B) \oplus C = A \oplus (B \oplus C)$$

$$A \oplus A = \mathbf{0}$$

$$A \oplus 0 = A$$

$$A \oplus \mathbf{1} = A \text{ 的按位取反}$$

其中 A, B, C 是长为 n 位的字符串,$\mathbf{0}$ 是长为 n 位的全 0 串,$\mathbf{1}$ 是长为 n 位的全 1 串。

3.8　这个问题给出了用一轮 DES 加密的具体数值的例子。我们假设明文和密钥 K 有相同的位模式，即

用十六进制表示：0 1 2 3 4 5 6 7 8 9 A B C D E F

用二进制表示：　0000 0001 0010 0011 0100 0101 0110 0111

1000 1001 1010 1011 1100 1101 1110 1111

(a) 推导第一轮的子密钥 K_1。

(b) 推导 L_0，R_0。

(c) 扩展 R_0 得到 $E[R_0]$，其中 $E[.]$ 是表 3.2 的扩展函数。

(d) 计算 $A = E[R_0] \oplus K_1$。

(e) 把(d)的 48 位结果分成 6 位(数据)一组的集合并求对应 S 盒代替的值。

(f) 将(e)的结果连接起来获得一个 32 位的结果 B。

(g) 应用置换获得 $P(B)$。

(h) 计算 $R_1 = P(B) \oplus L_0$。

(i) 写出密文。

3.9　证明 DES 解密算法实际上是 DES 加密算法的逆。

3.10　为了使加密算法可逆，16 轮后 DES 算法需要交换 32 位。这样，仅仅将密文沿着原算法逆回去并将密钥逆序即可解密。习题 3.7 可说明这一点。然而，为什么需要这 32 位的互换，你可能并不是非常清楚，所以看看下面的练习。首先给出一些记号：

$A \| B =$　　将串 A 和串 B 连接起来

$T_i(R \| L) =$　　加密过程第 i 轮迭代所定义的变换($1 \leqslant i \leqslant 16$)

$TD_i(R \| L) =$　　解密过程第 i 轮迭代所定义的变换($1 \leqslant i \leqslant 16$)

$T_{17}(R \| L) =$　　$L \| R$，加密过程第 16 轮迭代之后的变换

(a) 证明 $TD_1(IP(IP^{-1}(T_{17}(T_{16}(L_{15} \| R_{15}))))) = R_{15} \| L_{15}$。

$$TD_1(IP(IP^{-1}(T_{17}(T_{16}(L_{15} \| R_{15}))))) = R_{15} \| L_{15}$$

(b) 假设我们去掉了加密算法最后的 32 位的交换，请判断下式是否成立：

$$TD_1(IP(IP^{-1}(T_{16}(L_{15} \| R_{15})))) = L_{15} \| R_{15}$$

3.11　比较初始置换表[参见表 3.2(a)]和置换选择表一[参见表 3.4(b)]，它们的结构是否相似？如果是，请说明它们的相似之处。通过分析你可以得出什么结论？

3.12　16 个密钥($K_1, K_2, \cdots, K_{16}$)在 DES 解密过程中是逆序使用的。因此，图 3.5 的右半部分对解密而言不再正确。请模仿表 3.4(d)为解密过程设计一个合适的密钥移位扩展方案。

3.13　(a) 设 X' 是对 X 按位取反的结果。证明对于 DES 加密算法，如果明文和密钥都取反，则密文也取反。即

如果 $Y = E(K, X)$

那么 $Y' = E(K', X')$

提示：首先证明对任意两个相同长度的串 A 和 B，有 $(A \oplus B)' = A' \oplus B$。

(b) 据说对 DES 的穷举攻击需要搜索 2^{56} 个密钥的密钥空间。(a)中的结论对此是否有影响？

3.14　证明 DES 中每个子密钥的前 24 位均来自于初始密钥的同一个子集，该子集有 28 位，而后 24 位来自于初始秘密钥的另外 28 位。

3.15　对于任意的分组密码，它的非线性对安全是至关重要的。为了证明这一点，假设我们有一个线性分组密码 EL，加密 128 位的明文分组为 128 位的密文分组。令 $EL(k, m)$ 是 128 位明文 m 在密钥为 k 时的加密结果。则对任意的 128 位的 m_1 和 m_2，有

$$EL(k, [m_1 \oplus m_2]) = EL(k, m_1) \oplus EL(k, m_2)$$

请说明给定 128 个选择密文，在不知密钥 k 的情况下，对手如何解密任何密文(选择密文即对手可以选择密文，并能得到解密结果。此处，你有 128 个明密文对，且你可以选择密文的值)？

注意:下面的问题请参考附录 G 中的简化 DES。

3.16 图 G.2 描述了 S-DES 密钥生成,参考该图回答:

(a) 初始 P10 置换函数有什么重要性?

(b) 两个 LS-1 移位函数有什么重要作用?

3.17 对 S-DES 的分析一节中,定义了一个关于变量 q 和 r 的等式。请给出 s 和 t 的等式。

3.18 使用 S-DES,用密钥(0111111101)手工解密二进制串(10100010)。给出执行 IP、F_K、SW、F_K、IP^{-1}后的中间值。把明文串的前 4 位译码为某个字母,把后 4 位译码为另一个字母(这里,字母 A,B,…,P 分别对应于 0000,0001,…,1111)。**提示**:执行 SW 后,串的形式为(00010011)。

编程题

3.19 编写一个软件,实现一般代替分组密码的加解密。

3.20 编写一个软件,实现 S-DES 分组密码的加解密。测试数据:请使用习题 3.18 中的明文、密文和密钥。

第4章　数论和有限域的基本概念

4.1　整除性和除法
4.2　Euclid 算法
4.3　模运算
4.4　群、环和域
4.5　有限域 GF(p)
4.6　多项式运算
4.7　有限域 GF(2^n)
4.8　推荐读物和网站
4.9　关键术语、思考题和习题
附录4A　Mod 的含义

The next morning at daybreak, Star flew indoors, seemingly keen for a lesson. I said, "Tap eight." She did a brilliant exhibition, first tapping it in 4,4, then giving me a hasty glance and doing it in 2,2,2,2, before coming for her nut.

It is astonishing that Star learned to count up to 8 with no difficulty, and of her own accord discovered that each number could be given with various different divisions, this leaving no doubt that she was consciously thinking each number. In fact, she did mental arithmetic, although unable, like humans, to name the numbers. But she learned to recognize their spoken names almost immediately and was able to remember the sounds of the names. Star is unique as a wild bird, who of her own free will pursued the science of numbers with keen interest and astonishing intelligence.

—*Living with Birds*, Len Howard

要　点

- 模算术是一种整数算术，它将所有整数约减为一个固定的集合[0,1,…,$n-1$]，其中 n 为某个整数。任何这个集合外的整数通过除以 n 取余数的方式约减到这个范围内。
- 两个整数的最大公因子是可以整除这两个整数的最大正整数。
- 域是一些元素的集合，其上定义了两个算术运算（加法和乘法），它也具有一些常规的算术性质，如封闭性，结合律，交换性，分配律、加法和乘法逆等。
- 有限域在密码的若干领域都有重要应用。一个有限域就是有有限个元素的域。可以证明有限域的阶（元素的个数）一定可以写为素数的幂形式 p^n，n 为一个整数，p 为素数。
- 阶为 p 的有限域可以由模 p 算术来定义。
- 阶为 p^n，$n>1$ 的有限域可由多项式算术定义。

有限域在密码学中的地位越来越重要。许多密码算法都对有限域的性质有很大的依赖性，特别是高级加密标准（AES）和椭圆曲线加密算法。其他例子包括消息认证码 CMAC 及认证加密方案 GMC。

本章为读者提供了足够有关有限域概念的背景知识，从而能够理解 AES 以及其他使用有限域知识的密码算法的设计。本章的前三节介绍了来自数论的一些基本概念，本章的其他地方会用到。这些概念包括整除性、欧几里得（Euclid）算法及模算术。接着对群、环和域的概念做一个简要介绍。本节有一些抽象。读者初次阅读可以选择快速浏览的方式。我们将讨论形如 GF(p) 的有

限域,其中 p 为素数。随后我们了解有关多项式运算的知识。本章最后讨论形如 GF(2^n)的有限域,其中 n 是一个正整数。

数论中的概念和方法都非常抽象,如果没有具体实例,想要直观地掌握它们是比较困难的。因此,在本章和第 8 章我们都安排了大量的例子,而且每个例子都用阴影框加以突出。

4.1 整除性和除法

4.1.1 整除性

设 a,b,m 是整数,如果 $a=mb$,我们说非零整数 b 整除 a。也就是说,如果在除的时候没有余数,则称 b 整除 a。用 $b|a$ 表示 b 整除 a。同样,如果 $b|a$,则称 b 是 a 的因子。

24 的正因子有 1,2,3,4,6,8,12 和 24。
13|182; −5|30; 17|289; −3|33; 17|0

接下来我们需要一些简单的整数整除的性质:

- 如果 $a|1$,则 $a=\pm 1$。
- 如果 $a|b$ 且 $b|a$,则 $a=\pm b$。
- 任意非零数 b 整除 0。
- 如果 $a|b$ 且 $b|c$,则 $a|c$。

11|66 和 66|198 = 11|198

- 如果 $b|g$ 且 $b|h$,则对任意的整数 m 和 n,有 $b|(mg+nh)$。

为了说明刚才的一点,注意:

- 如果 $b|g$,则存在某个整数 g_1,使得 g 可写为 $g=bg_1$。
- 如果 $b|h$,则存在某个整数 h_1,使得 h 可写为 $h=bh_1$。

所以有

$$mg+nh=mbg_1+nbh_1=b\times(mg_1+nh_1)$$

因此,b 整除 $mg+nh$。

$b=7$; $g=14$; $h=63$; $m=3$; $n=2$; 7|14 且 7|63。
为了证明 7|(3×14+2×63),我们有(3×14+2×63) =7(3×2+2×9),显然 7|(7(3×2+2×9))。

4.1.2 除法 带余数的除法

给定任意的正整数 n 和任意的非负整数 a,如果用 n 除 a,我们得到一个整数商 q 和一个整数余数 r,它们服从下面的关系:

$$a=qn+r \qquad 0\leqslant r<n; q=\lfloor a/n\rfloor \tag{4.1}$$

其中 $\lfloor x\rfloor$ 是小于等于 x 的最大整数。式(4.1)被称为除法①。

① 式(4.1)表达的是一个定理而不是一个算法,但传统上这被称为除法。

图 4.1(a)显示对于给定的 a 和正数 n,总是可以找到 q 和 r 满足前述的关系式。将整数用线上的点表示;a 将落在该线的某个地方(图中显示的是正数,对于负数的情况可类似显示)。从 0 开始,依次为 n,$2n$,直到 qn,使得 $qn \leqslant a$ 并且 $(q+1)n > a$,从 qn 到 a 的距离是 r,此时我们已经找到了唯一的值 q 和 r。余数 r 经常被称为剩余数。

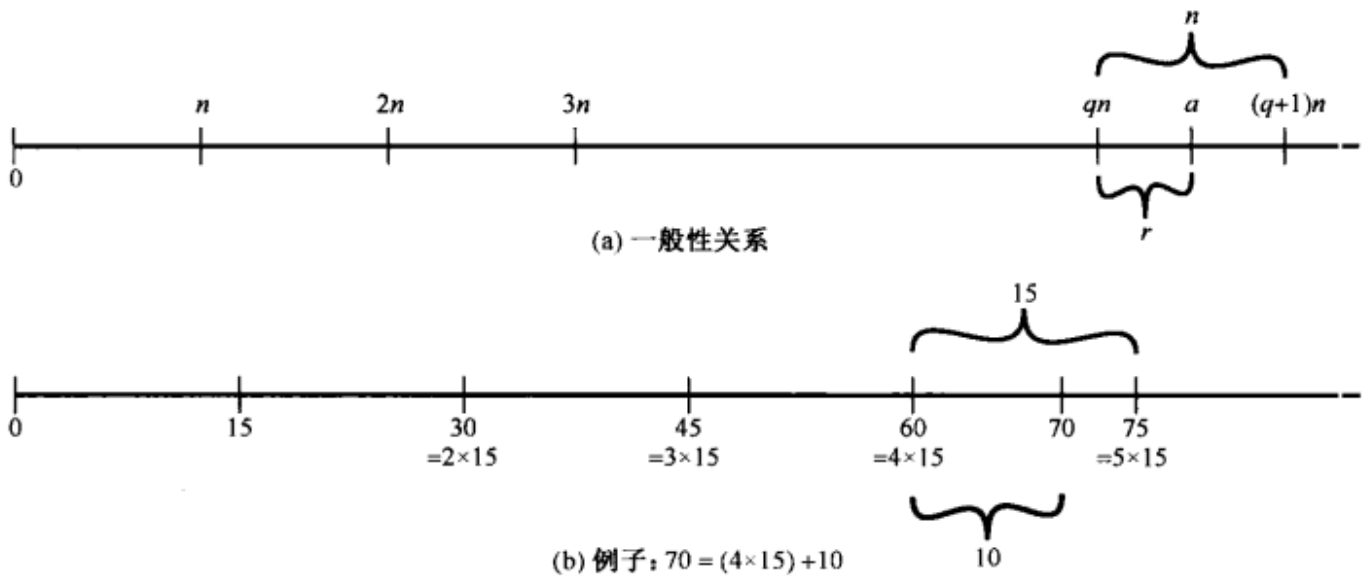


图 4.1　关系 $a = qn + r$,　$0 \leqslant r < n$

$a = 11$; $n = 7$; $11 = 1 \times 7 + 4$;　$r = 4$　$q = 1$
$a = -11$; $n = 7$; $-11 = (-2) \times 7 + 3$;　$r = 3$　$q = -2$
图 4.1(b)提供了另外一个例子。

4.2　Euclid 算法

数论的一个最基本技巧是 Euclid 算法,它是求两个正整数最大公因子的简单过程。首先我们需要一个简单的定义:两个整数称为互素的,如果它们唯一的正整数公因子是 1。

4.2.1　最大公因子

前面讲过,对于整数 a,b,m,如果满足 $a = mb$,则称非 0 整数 b 是 a 的一个因子。我们用 $\gcd(a,b)$ 表示 a 和 b 的最大公因子。a 和 b 的最大公因子是能够同时整除 a 和 b 的最大整数。我们还定义 $\gcd(0,0) = 0$。

更正式的描述是,称 c 为 a 和 b 的最大公因子,如果

(1) c 是 a 和 b 的因子。

(2) a 和 b 的任何因子都是 c 的因子。

以下是一个等价的定义:

$$\gcd(a,b) = \max[k, \text{其中 } k \mid a \text{ 且 } k \mid b]$$

因为我们所求的最大公因子是正数,所以 $\gcd(a,b) = \gcd(a,-b) = \gcd(-a,b) = \gcd(-a,-b)$。

一般来说,$\gcd(a,b) = \gcd(|a|,|b|)$。

$$\gcd(60,24) = \gcd(60,-24) = 12$$

同样地,因为 0 可被所有非零整数整除,所以 $\gcd(a,0) = |a|$。

我们称整数 a 和 b 是互素的,如果 a 和 b 只有唯一的正公因子 1。也就是说,如果 $\gcd(a,b) = 1$,那么 a 和 b 互素。

8 和 15 互素,因为 8 的正因子是 1,2,4 和 8。15 的正因子是 1,3,5 和 15,所以 1 是二者唯一的公因子。

4.2.2 求最大公因子

现在我们描述一个由 Euclid 发明的算法来容易地求两个整数的最大公因子。这个算法在本章的后续部分有重要意义。假设我们有整数 a,b 使得 $d=\gcd(a,b)$。因为 $\gcd(|a|,|b|)=\gcd(a,b)$,所以可以假设 $a\geqslant b>0$。现在用 b 除 a,由除法可得到

$$a = q_1 b + r_1 \qquad 0 \leqslant r_1 < b \tag{4.2}$$

如果恰巧 $r_1=0$,则 $b|a$ 且 $d=\gcd(a,b)=b$。但是,如果 $r_1\neq 0$,我们说 $d|r_1$。这基于整除性的基本性质:由 $d|a$ 和 $d|b$ 可以推出 $d|(a-q_1 b)$,即 $d|r_1$。在开始 Euclid 算法之前,我们需要回答下面的问题:$\gcd(b,r_1)$ 是什么?我们知道 $d|b$ 和 $d|r_1$。现在假设有任意的整数 c 整除 b 和 r_1。因此 $c|(q_1 b+r_1)=a$。因为 c 同时整除 a 和 b,必须有 $c\leqslant d$,而 d 是 a 和 b 的最大公因子。因此 $d=\gcd(b,r_1)$。

我们再来看式(4.2),且假设 $r_1\neq 0$。因为 $b>r_1$,可以用 r_1 除 b,应用除法有

$$b = q_2 r_1 + r_2 \qquad 0 \leqslant r_2 < r_1$$

如前所述,如果 $r_2=0$,则 $d=r_1$。如果 $r_2\neq 0$,则 $d=\gcd(r_1,r_2)$。继续除法过程直到余数为 0,比如说在第 $(n+1)$ 阶段有 r_n 整除 r_{n-1}。结果为如下的方程系统:

$$\left.\begin{aligned}
a &= q_1 b + r_1 && 0 < r_1 < b \\
b &= q_2 r_1 + r_2 && 0 < r_2 < r_1 \\
r_1 &= q_3 r_2 + r_3 && 0 < r_3 < r_2 \\
&\vdots && \vdots \\
r_{n-2} &= q_n r_{n-1} + r_n && 0 < r_n < r_{n-1} \\
r_{n-1} &= q_{n+1} r_n + 0 \\
d &= \gcd(a,b) = r_n
\end{aligned}\right\} \tag{4.3}$$

在每一次循环,我们都有 $d=\gcd(r_i,r_{i+1})$,直到最后得到 $d=\gcd(r_n,0)=r_n$。因此,通过重复运用除法运算,我们求得了两个整数的最大公因子。这个算法称为 Euclid 算法。

本质上,我们已经利用自顶向下的讨论方法在最后求得了 $\gcd(a,b)$。也可以自下而上讨论。第一步是证明 r_n 同时整除 a 和 b。从式(4.3)的最后一个除式推出 r_n 整除 r_{n-1}。倒数第二个式子表明 r_n 整除 r_{n-2},这是因为 r_n 整除式子右边的两项。顺序地可以证明 r_n 整除所有的 r_i,最后是 a 和 b。还需要证明 r_n 是同时整除 a 和 b 的最大的因子。如果我们取一个能同时整除 a 和 b 的任意整数 c,如前解释的那样,它也可以整除 r_1。按照式(4.3)的向下顺序可以证明 c 整除所有的 r_i。因此 c 必须整除 r_n,所以有 $r_n=\gcd(a,b)$。

下面例子的数值比较大,便于观察算法的能力:

求 $d=\gcd(a,b)=\gcd(1160718174,316258250)$		
$a=q_1 b+r_1$	$1160718174=3\times 316258250+211943424$	$d=\gcd(316258250,211943424)$
$b=q_2 r_1+r_2$	$316258250=1\times 211943424+104314826$	$d=\gcd(211943424,104314826)$
$r_1=q_3 r_2+r_3$	$211943424=2\times 104314826+3313772$	$d=\gcd(104314826,3313772)$
$r_2=q_4 r_3+r_4$	$104314826=31\times 3313772+1587894$	$d=\gcd(3313772,1587894)$
$r_3=q_5 r_4+r_5$	$3313772=2\times 1587894+137984$	$d=\gcd(1587894,137984)$
$r_4=q_6 r_5+r_6$	$1587894=11\times 137984+70070$	$d=\gcd(137984,70070)$

$r_5 = q_7 r_6 + r_7$	$137984 = 1 \times 70070 + 67914$	$d = \gcd(70070, 67914)$
$r_6 = q_8 r_7 + r_8$	$70070 = 1 \times 67914 + 2156$	$d = \gcd(67914, 2156)$
$r_7 = q_9 r_8 + r_9$	$67914 = 31 \times 2156 + 1078$	$d = \gcd(2156, 1078)$
$r_8 = q_{10} r_9 + r_{10}$	$2156 = 2 \times 1078 + 0$	$d = \gcd(1078, 0) = 1078$
因此，$d = \gcd(1160718174, 316258250) = 1078$		

本例中，用 316258250 除 1160718174 作为开始，这一步给出了 3 和余数 211943424。我们取出 316258250，并用 211943424 去除它。继续该过程直到余数为 0，此时得到结果 1078。

将上述的计算过程重新排列为表格的形式比较有用。对于循环的每一步，我们有 $r_{i-2} = q_i r_{i-1} + r_i$，其中 r_{i-2} 称为被除数，r_{i-1} 是除数，q_i 是商，r_i 是余数。表 4.1 总结了结果。

表 4.1　Euclid 算法例子

被除数	除数	商	余数
$a = 1160718174$	$b = 316258250$	$q_1 = 3$	$r_1 = 211943424$
$b = 316258250$	$r_1 = 211943424$	$q_2 = 1$	$r_2 = 104314826$
$r_1 = 211943424$	$r_2 = 104314826$	$q_3 = 2$	$r_3 = 3313772$
$r_2 = 104314826$	$r_3 = 3313772$	$q_4 = 31$	$r_4 = 1587894$
$r_3 = 3313772$	$r_4 = 1587894$	$q_5 = 2$	$r_5 = 137984$
$r_4 = 1587894$	$r_5 = 137984$	$q_6 = 11$	$r_6 = 70070$
$r_5 = 137984$	$r_6 = 70070$	$q_7 = 1$	$r_7 = 67914$
$r_6 = 70070$	$r_7 = 67914$	$q_8 = 1$	$r_8 = 2156$
$r_7 = 67914$	$r_8 = 2156$	$q_9 = 31$	$r_9 = 1078$
$r_8 = 2156$	$r_9 = 1078$	$q_{10} = 2$	$r_{10} = 0$

4.3　模运算

4.3.1　模数

如果 a 是整数，n 是正整数，则我们定义 a 模 n 是 a 除以 n 所得的余数。整数 n 称为模数。因此，对于任意整数 a，我们总可以写出

$$a = qn + r \qquad 0 \leqslant r < n;\ q = \lfloor a/n \rfloor$$

$$a = \lfloor a/n \rfloor \times n + (a \bmod n)$$

$$11 \bmod 7 = 4;\ -11 \bmod 7 = 3$$

如果 $(a \bmod n) = (b \bmod n)$，则我们称整数 a 和 b 是模 n 同余的。可以表示为 $a \equiv b \pmod n$[①]。

$$73 \equiv 4 \pmod{23};\ 21 \equiv -9 \pmod{10}$$

注意，如果 $a \equiv 0 \pmod n$，则 $n \mid a$。

4.3.2　同余的性质

模运算有如下性质：

(1) 如果 $n \mid (a - b)$，那么 $a \equiv b \pmod n$。

① 我们以两种方式使用了运算符 mod：首先是作为产生余数的二元操作符，如 $a \bmod b$；第二种，作为一种同余关系，表明两个整数的等价关系，如 $a \equiv b \pmod n$。附录 4A 中有进一步的讨论。

(2) $a \equiv b \pmod n$ 隐含 $b \equiv a \pmod n$。

(3) $a \equiv b \pmod n$, $b \equiv c \pmod n$ 隐含 $a \equiv c \pmod n$。

先证明第一条。如果 $n|(a-b)$,那么存在某个 k 使得 $(a-b)=kn$。于是我们知道 $a=b+kn$。因此 $(a \bmod n)=(b+kn$ 除以 n 的余数$)=(b$ 除以 n 的余数$)=(b \bmod n)$。

$23 \equiv 8 \pmod 5$ 因为 $23-8=15=5\times 3$

$-11 \equiv 5 \pmod 8$ 因为 $-11-5=-16=8\times(-2)$

$81 \equiv 0 \pmod{27}$ 因为 $81-0=81=27\times 3$

剩下的容易证明。

4.3.3 模算术运算

注意,由定义(参见图 4.1)可知,运算 $\pmod n$ 将所有的整数映射到集合 $\{0,1,\cdots,(n-1)\}$。于是出现了这样的一个问题:能否限制在这个集合上进行算术运算?这是可以的,这种技术称为模算术。

模算术有如下性质:

(1) $[(a \bmod n)+(b \bmod n)] \bmod n=(a+b) \bmod n$

(2) $[(a \bmod n)-(b \bmod n)] \bmod n=(a-b) \bmod n$

(3) $[(a \bmod n)\times(b \bmod n)] \bmod n=(a\times b) \bmod n$

我们来证明第一个性质。定义 $(a \bmod n)=r_a$, $(b \bmod n)=r_b$,于是存在整数 j,k 使得 $a=r_a+jn$, $b=r_b+kn$。那么

$$\begin{aligned}(a+b) \bmod n &= (r_a+jn+r_b+kn) \bmod n \\ &= (r_a+r_b+(k+j)n) \bmod n \\ &= (r_a+r_b) \bmod n \\ &= [(a \bmod n)+(b \bmod n)] \bmod n\end{aligned}$$

剩下的性质容易证明。下面是关于这三个性质的一些例子。

$11 \bmod 8=3$; $15 \bmod 8=7$

$[(11 \bmod 8)+(15 \bmod 8)] \bmod 8=10 \bmod 8=2$

$(11+15) \bmod 8=26 \bmod 8=2$

$[(11 \bmod 8)-(15 \bmod 8)] \bmod 8=-4 \bmod 8=4$

$(11-15) \bmod 8=-4 \bmod 8=4$

$[(11 \bmod 8)\times(15 \bmod 8)] \bmod 8=21 \bmod 8=5$

$(11\times 15) \bmod 8=165 \bmod 8=5$

和普通算术中的一样,幂运算可以通过重复乘法实现(我们将在第 8 章中进一步讨论幂运算)。

求 $11^7 \bmod 13$,我们可以如下进行:

$11^2=121 \equiv 4 \pmod{13}$

$11^4 \equiv (11^2)^2 \equiv 4^2 \equiv 3 \pmod{13}$

$11^7 \equiv 11\times 4\times 3 \equiv 132 \equiv 2 \pmod{13}$

因此,普通算术的加、减、乘运算规则可以平移到模算术中。

表 4.2 给出了模 8 的加法和乘法的例子。请看模 8 的加法例子，结果非常简单并且矩阵有一定的规律。两个矩阵都关于主对角线对称，这是加法和乘法可交换的结果。就像在普通的加法运算中一样，模运算中对于每个整数也存在加法逆元素，或称为负数。在模运算中，整数 x 的负数 y 是使得 $(x+y) \bmod 8 = 0$ 成立的值。可以这样求出左边列中整数的加法逆元素：浏览矩阵的对应行，并找出值 0；0 所在列顶部的整数就是所求的加法逆元素；例如 $(2+6) \bmod 8 = 0$。同样地，在乘法表中元素的意义更明确。在普通运算中，每个整数都有其乘法逆元素，或称为倒数。在模 8 乘法运算中，整数 x 的乘法逆元 y 是使得 $(x \times y) \bmod 8 = 1 \bmod 8$ 成立的值。下面我们在乘法表中求一个整数的乘法逆元：浏览矩阵中整数所在的行并寻找值 1，1 所在列顶部的整数就是所求的乘法逆元素；例如 $(3 \times 3) \bmod 8 = 1$。需要指出的是，并不是所有的整数模 8 都有乘法逆元；后面我们将会经常提到。

表 4.2　模 8 运 算

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

(a)模 8 加法

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

(b)模 8 乘法

w	$-w$	w^{-1}
0	0	—
1	7	1
2	6	—
3	5	3
4	4	—
5	3	5
6	2	—
7	1	7

(c)模 8 加法逆元和乘法逆元

4.3.4　模运算的性质

定义比 n 小的非负整数集合为 Z_n：

$$Z_n = \{0, 1, \cdots, (n-1)\}$$

这个集合被称为剩余类集，或模 n 的剩余类。更准确地说，Z_n 中每一个整数都代表一个剩余类，我们可以将模 n 的剩余类表示为 $[0], [1], [2], \cdots, [n-1]$，其中

$$[r] = \{a : a \text{ 是一个整数}, a \equiv r \pmod{n}\}$$

模 4 剩余类为

$[0] = \{\cdots, -16, -12, -8, -4, 0, 4, 8, 12, 16, \cdots\}$

$[1] = \{\cdots, -15, -11, -7, -3, 1, 5, 9, 13, 17, \cdots\}$

$[2] = \{\cdots, -14, -10, -6, -2, 2, 6, 10, 14, 18, \cdots\}$

$[3] = \{\cdots, -13, -9, -5, -1, 3, 7, 11, 15, 19, \cdots\}$

在剩余类的所有整数中,我们通常用最小非负整数来代表这个剩余类。寻找与 k 是模 n 同余的最小非负整数的过程,称为模 n 的 k 约化。

如果我们在 Z_n 中进行模运算,表 4.3 中所列的性质对于 Z_n 中的整数适用。因此,Z_n 是有乘法单位元的交换环。

模运算有一个特性使它区分于普通运算。首先,如普通算术里的运算一样,我们有

$$\text{如果}(a+b)\equiv(a+c)\pmod n,\text{那么}\ b\equiv c\pmod n \tag{4.4}$$

$$(5+23)\equiv(5+7)\pmod 8;\ 23\equiv 7\pmod 8$$

式(4.4)与加法逆元的存在性是一致的。将 a 的加法逆元加在式(4.4)的两边,我们得到

$$((-a)+a+b)\equiv((-a)+a+c)(\bmod n)$$
$$b\equiv c(\bmod n)$$

然而,下面的陈述只在有附加条件时才是成立的:

$$\text{令}\ a\ \text{与}\ n\ \text{互素,如果}(a\times b)\equiv(a\times c)\pmod n,\text{那么}\ b\equiv c\pmod n \tag{4.5}$$

其中互素的概念这样定义:如果两个整数的最大公因子是 1,则称它们是互素的。同式(4.4)的情况类似,我们也可以说式(4.5)同乘法逆元的存在性是一致的。将 a 的乘法逆元素施加在式(4.5)的两边,我们得到

$$((a^{-1})ab)\equiv((a^{-1})ac)(\bmod n)$$
$$b\equiv c(\bmod n)$$

表 4.3 Z_n 中整数模运算的性质

性质	表达式
交换律	$(w+x)\bmod n=(x+w)\bmod n$ $(w\times x)\bmod n=(x\times w)\bmod n$
结合律	$[(w+x)+y]\bmod n=[w+(x+y)]\bmod n$ $[(w\times x)\times y]\bmod n=[w\times(x\times y)]\bmod n$
分配律	$[w\times(x+y)]\bmod n=[(w\times x)+(w\times y)]\bmod n$
单位元	$(0+w)\bmod n=w\bmod n$ $(1\times w)\bmod n=w\bmod n$
加法逆元($-w$)	对于 Z_n 中的任意 w,存在一个 z 使得 $w+z\equiv 0\bmod n$

要明白这点,我们来看一个使式(4.5)的条件不成立的例子。6 和 8 不是互素的,因为它们有公因子 2。我们有以下式子:

$$6\times 3=18\equiv 2\pmod 8$$
$$6\times 7=42\equiv 2\pmod 8$$

然而 $3\neq 7\pmod 8$。

得到这个奇怪结果的原因是:对于任何一般的模数 n,如果 a 和 n 有任何公因子的话,用乘数 a 依次作用于 0 到 $n-1$ 的所有整数将不会产生一个完整剩余类集。

如果 $a=6$ 且 $n=8$,则有

Z_8	0	1	2	3	4	5	6	7
乘以 6	0	6	12	18	24	30	36	42
剩余类	0	6	4	2	0	6	4	2

因为乘以 6 时，我们得不到一个完整的剩余类集，Z_8 中有多个整数映射到相同的剩余类。特别地，$6 \times 0 \bmod 8 = 6 \times 4 \bmod 8$；$6 \times 1 \bmod 8 = 6 \times 5 \bmod 8$；以此类推。因为这是一个多对一的映射，所以乘法运算并没有唯一的逆运算。

然而，如果我们令 $a = 5$ 且 $n = 8$，二者唯一的公因子是 1。

Z_8	0	1	2	3	4	5	6	7
乘以 5	0	5	10	15	20	25	30	35
剩余类	0	5	2	7	4	1	6	3

剩余类这一行包含了 Z_8 所在行的所有整数，只是顺序不同。

一般来说，如果一个整数与 n 互素，那么它在 Z_n 中有一个乘法逆元。从表 4.2(c) 可以看出，整数 1,3,5,7 在 Z_8 中有一个乘法逆元，而 2,4 和 6 则没有。

4.3.5 修改的 Euclid 算法

Euclid 算法基于以下的定理：对于任意非负整数 a 和任意正整数 b

$$\gcd(a, b) = \gcd(b, a \bmod b) \tag{4.6}$$

$$\gcd(55,22) = \gcd(22,55 \bmod 22) = \gcd(22,11) = 11$$

为保证式(4.6)成立，使 d 等于 $\gcd(a,b)$，根据 gcd 的定义，有 $d|a$ 和 $d|b$ 成立。对于任意正整数 b，a 可以表示为如下形式：

$$a = kb + r \equiv r \pmod b$$
$$a \bmod b = r$$

其中 k，r 为整数。因此，对某个整数 k，有 $(a \bmod b) = a - kb$。因为 $d|b$，所以有 $d|kb$；又因为 $d|a$，所以有 $d|(a \bmod b)$。这说明 d 是 b 和 $(a \bmod b)$ 的公因子。反之，如果 d 是 b 和 $(a \bmod b)$ 的公因子，那么 $d|kb$ 并且由此可知 $d|[kb + (a \bmod b)]$，即 $d|a$。因此，a 和 b 的公因子的集合与 b 和 $(a \bmod b)$ 的公因子的集合相等。因此，两对数的 gcd 相同，定理得证。

为了求出最大公因子，可多次重复使用式(4.6)。

$$\gcd(18,12) = \gcd(12,6) = \gcd(6,0) = 6$$
$$\gcd(11,10) = \gcd(10,1) = \gcd(1,0) = 1$$

这和式(4.3)所示的方案相同，可重新改写如下：

Euclid 算法

计　算	满　足
$r_1 = a \bmod b$	$a = q_1 b + r_1$
$r_2 = b \bmod r_1$	$b = q_2 r_1 + r_2$
$r_3 = r_1 \bmod r_2$	$r_1 = q_3 r_2 + r_3$
⋮	⋮
$r_n = r_{n-2} \bmod r_{n-1}$	$r_{n-2} = q_n r_{n-1} + r_n$
$r_{n+1} = r_{n-1} \bmod r_n = 0$	$r_{n-1} = q_{n+1} r_n + 0$
	$d = \gcd(a,b) = r_n$

我们可以精确地将 Euclid 算法定义为如下的递归函数。

```
Euclid(a,b)
    if (b=0) then return a;
    else return Euclid(b, a mod b);
```

4.3.6 扩展的 Euclid 算法

我们将看一下 Euclid 算法的扩展算法,这对于有限域中的以及 RSA 等密码算法中的计算非常重要。对于给定的整数 a 和 b,扩展的 Euclid 算法不仅计算出最大公因子 d,而且还有另外两个整数 x 和 y,它们满足如下方程:

$$ax + by = d = \gcd(a, b) \tag{4.7}$$

应该明白 x 和 y 具有相反的正负号。在研究算法之前,让我们看一下当 $a = 42, b = 30$ 时,x 和 y 的一些值。注意 $\gcd(42,30) = 6$。下面是 $42x + 30y$ 的部分值[①]。

y \ x	-3	-2	-1	0	1	2	3
-3	-216	-174	-132	-90	-48	-6	36
-2	-186	-144	-102	-60	-18	24	66
-1	-156	-114	-72	-30	12	54	96
0	-126	-84	-42	0	42	84	126
1	-96	-54	-12	30	72	114	156
2	-66	-24	18	60	102	144	186
3	-36	6	48	90	132	174	216

观察到所有的项都可以被 6 整除。这并不奇怪,因为 42 和 30 都可以被 6 整除,所以每一个形如 $42x + 30y = 6(7x + 5y)$ 的数都是 6 的倍数。还注意到 $\gcd(42,30) = 6$ 也出现在表中。一般地,可以证明对于两个给定的整数 a 和 b,$ax + by$ 的最小正整数等于 $\gcd(a,b)$。

现在我们展示对于给定的 a 和 b,如何扩展 Euclid 算法去计算 (x, y, d)。我们再次沿着式(4.3)所示的除法顺序进行,并且假设在每一步骤 i 都可以找到 x_i 和 y_i 满足 $r_i = ax_i - by_i$。结果可得到如下式子。

$$\begin{aligned}
a &= q_1 b + r_1 & r_1 &= ax_1 + by_1 \\
b &= q_2 r_1 + r_2 & r_2 &= ax_2 + by_2 \\
r_1 &= q_3 r_2 + r_3 & r_3 &= ax_3 + by_3 \\
&\vdots & &\vdots \\
r_{n-2} &= q_n r_{n-1} + r_n & r_n &= ax_n + by_n \\
r_{n-1} &= q_{n+1} r_n + 0
\end{aligned}$$

现在,可以通过移项得到

$$r_i = r_{i-2} - r_{i-1} q_i \tag{4.8}$$

同样,从 $i-1$ 和 $i-2$ 行,可得到值

$$r_{i-2} = ax_{i-2} + by_{i-2} \qquad r_{i-1} = ax_{i-1} + by_{i-1}$$

① 该例子取自[SILV06]。

代入式(4.8)有

$$\begin{aligned} r_i &= (ax_{i-2} + by_{i-2}) - (ax_{i-1} + by_{i-1})q_i \\ &= a(x_{i-2} - q_i x_{i-1}) + b(y_{i-2} - q_i y_{i-1}) \end{aligned}$$

然而,我们已经假设 $r_i = ax_i + by_i$,因此

$$x_i = x_{i-2} - q_i x_{i-1} \qquad y_i = y_{i-2} - q_i y_{i-1}$$

总结计算过程如下:

扩展的 Euclid 算法

计　算	满　足	计　算	满　足
$r_{-1} = a$		$x_{-1} = 1;\ y_{-1} = 0$	$a = ax_{-1} + by_{-1}$
$r_0 = b$		$x_0 = 0;\ y_0 = 1$	$b = ax_0 + by_0$
$r_1 = a \bmod b$	$a = q_1 b + r_1$	$x_1 = x_{-1} - q_1 x_0 = 1$	$r_1 = ax_1 + by_1$
$q = \lfloor a/b \rfloor$		$y_1 = y_{-1} - q_1 y_0 = -q_1$	
$r_2 = b \bmod r_1$	$b = q_2 r_1 + r_2$	$x_2 = x_0 - q_2 x_1$	$r_2 = ax_2 + by_2$
$q_2 = \lfloor b/r_1 \rfloor$		$y_2 = y_0 - q_2 y_1$	
$r_3 = r_1 \bmod r_2 \quad q_3 = \lfloor r_1/r_2 \rfloor$	$r_1 = q_3 r_2 + r_3$	$x_3 = x_1 - q_3 x_2$	$r_3 = ax_3 + by_3$
		$y_3 = y_1 - q_3 y_2$	
⋮	⋮	⋮	⋮
$r_n = r_{n-2} \bmod r_{n-1}$	$r_{n-2} = q_n r_{n-1} + r_n$	$x_n = x_{n-2} - q_n x_{n-1}$	$r_n = ax_n + by_n$
$q_n = \lfloor r_{n-2}/r_{n-3} \rfloor$		$y_n = y_{n-2} - q_n y_{n-1}$	
$r_{n+1} = r_{n-1} \bmod r_n = 0$	$r_{n-1} = q_{n+1} r_n + 0$		$d = \gcd(a,b) = r_n$
$q_{n+1} = \lfloor r_{n-1}/r_{n-2} \rfloor$			$x = x_n;\ y = y_n$

我们还需要多做一些注解。每一行里,基于前两行的余数 r_{i-1} 和 r_{i-2},我们计算一个新的余数 r_i。为开始程序,需要 r_0 和 r_{-1} 有值,它们分别是 a 和 b。而决定 x_{-1}, y_{-1}, y_0 的值是直观的。

从原始的 Euclid 算法知该过程当余数为 0 时结束,求得 a 和 b 的最大公因子为 $d = \gcd(a, b) = r_n$。但是我们也决定了 $d = r_n = ax_n + by_n$。因此,在式(4.7)中,$x = x_n, y = y_n$。

作为一个例子,令 $a = 1759, b = 550$,解等式 $1759x + 550y = \gcd(1759, 550)$。结果如表 4.4 所示。因此,我们有 $1759 \times (-111) + 550 \times 355 = -195249 + 195250 = 1$。

表 4.4　扩展 Euclid 算法例子

i	r_i	q_i	x_i	y_i
-1	1759		1	0
0	550		0	1
1	109	3	1	-3
2	5	5	-5	16
3	4	21	106	-339
4	1	1	-111	355
5	0	4		

结果:$d = 1;\ x = -111;\ y = 355$。

4.4　群、环和域

群、环和域都是数学理论中的一个分支,即抽象代数或称为近世代数的基本元素。在抽象代数中,我们关心的是其元素能进行代数运算的集合,也就是说,我们可以通过很多种方法,使集合上的两个元素组合得到集合中的第三个元素。这些运算方法都遵守特殊的规则,而这些规则又能确定集合的性质。根据约定,集合上元素的两种主要运算符号与普通数字的加法和乘法所使用的

符号是相同的。然而,我们必须指出,在抽象代数中,我们并不仅仅限于普通的算术操作。这一点通过后面章节的介绍很容易看明白。

4.4.1 群

群 G,有时记为$\{G,\cdot\}$,是定义了一个二元运算的集合,这个二元运算可表示为 $\cdot$,G 中每一个序偶(a,b)通过运算生成 G 中的元素$(a\cdot b)$,并满足以下公理[①]:

(A1)封闭性:如果 a 和 b 都属于 G,则 $a\cdot b$ 也属于 G。

(A2)结合律:对于 G 中任意元素 a,b,c,都有 $a\cdot(b\cdot c)=(a\cdot b)\cdot c$ 成立。

(A3)单位元:G 中存在一个元素 e,对于 G 中任意元素 a,都有 $a\cdot e=e\cdot a=a$ 成立。

(A4)逆元:对于 G 中任意元素 a,G 中都存在一个元素 a',使得下式成立:

$$a\cdot a'=a'\cdot a=e$$

我们用 N_n 表示 n 个不同符号的集合,为方便起见,我们将其表示成$\{1,2,\cdots,n\}$。n 个不同符号的一个置换是一个 N_n 到 N_n 的一一映射[②]。定义 S_n 为 n 个不同符号所形成的所有置换组成的集合。S_n 中的每一个元素都代表集合$\{1,2,\cdots,n\}$的一个置换 π。可以很容易地验证 S_n 是一个群:

A1:如果$(\pi,\rho\in S_n)$,则合成映射 $\pi\cdot\rho$ 根据置换 π 来改变 ρ 中元素的次序而形成。例如,$\{3,2,1\}\cdot\{1,3,2\}=\{2,3,1\}$。显然,$\pi\cdot\rho\in S_n$。

A2:映射的合成显而易见满足结合律。

A3:恒等映射就是不改变 n 个元素位置的置换。对于 S_n,单位元就是$\{1,2,\cdots,n\}$。

A4:对于任意 $\pi\in S_n$,抵消由 π 所定义的置换的映射就是 π 的逆元。这个逆元总是存在的。例如,$\{2,3,1\}\cdot\{3,1,2\}=\{1,2,3\}$。

如果一个群的元素是有限的,则该群称为**有限群**,并且群的阶就等于群中元素的个数。否则,称该群为**无限群**。

一个群如果还满足以下条件,则称为**交换群**:

(A5)交换律:对于 G 中任意的元素 a,b,都有 $a\cdot b=b\cdot a$ 成立。

加法运算下的整数集合(包括正整数、负整数和零)是一个交换群。乘法运算下的非零实数集合是一个交换群。前面例子中的集合 S_n 是一个群,但对于 $n>2$,它不是交换群。

当群中的运算符是加法时,其单位元是 0;a 的逆元是 $-a$;并且将减法用以下规则定义:$a-b=a+(-b)$。

循环群

我们在群中定义求幂运算为重复运用群中的运算,如 $a^3=a\cdot a\cdot a$。而且,我们定义 $a^0=e$ 作为单位元;并且 $a^{-n}=(a')^n$,其中 a' 是 a 在群内的逆元素。如果群 G 中的每一个元素都是一个固定元素 $a(a\in G)$的幂 $a^k(k$ 为整数),则称群 G 是循环群。我们认为元素 a 生成了群 G,或者说 a 是群 G 的生成元。循环群总是交换群,它可能是有限群或无限群。

① 操作符 $\cdot$ 具有一般性;可以指加法、乘法,或某些其他的数学运算。

② 这和第 2 章中置换的定义是等价的,那里说有限个元素的集合 S 的置换是 S 中元素的有序序列,其中每个元素只出现一次。

整数的加法群是一个无限循环群，它由 1 生成。在这种情况下，幂被解释为用加法合成的，因此 n 是 1 的 n 次幂。

4.4.2　环

环 R，有时记为 $\{R, +, \times\}$，是一个有两个二元运算的集合，这两个二元运算分别称为加法和乘法[①]，且对于 R 中的任意元素 a, b, c 满足以下公理。

(A1 ~ A5) R 关于加法是一个交换群；也就是说，R 满足从 A1 到 A5 的所有原则。对于此种情况下的加法群，我们用 0 表示其单位元，$-a$ 表示 a 的逆元。

(M1) 乘法的封闭性：如果 a 和 b 都属于 R，则 ab 也属于 R。

(M2) 乘法的结合律：对于 R 中的任意元素 a, b, c，有 $a(bc) = (ab)c$ 成立。

(M3) 分配律：对于 R 中的任意元素 a, b, c，下面两个式子总成立：乘法对加法的分配律

$$a(b + c) = ab + ac$$

$$(a + b)c = ac + bc$$

本质上说，环就是一个集合，我们可以在其上进行加法、减法 $[a - b = a + (-b)]$ 和乘法而不脱离该集合。

实数上所有 n 阶方阵的集合关于加法和乘法够成一个环。

环如果还满足以下条件，则称为交换环。

(M4) 乘法的交换律：对于 R 中任意元素 a, b，有 $ab = ba$ 成立。

偶整数集合（包括正数、负数和 0）记为 S，在普通加法和乘法运算下是交换环。前面一个例子中定义的所有 n 阶方阵的集合就不是交换环。

整数 $\{0, 1, \cdots, n-1\}$ 的集合 Z_n 加上模 n 的算术运算构成一个交换环（参见表 4.3）。

下面，我们定义整环，它是满足以下公理的交换环：

(M5) 乘法单位元：在 R 中存在元素 1，使得对于 R 中的任意元素 a，有 $a1 = 1a = a$ 成立。

(M6) 无零因子：如果有 R 中元素 a, b，且 $ab = 0$，则必有 $a = 0$ 或 $b = 0$。

将普通加法和乘法运算下的整数集合（包括正数、负数和 0）记为 S，则 S 是一个整环。

4.4.3　域

域 F，有时记为 $\{F, +, \times\}$，是有两个二元运算的集合，这两个二元运算分别称为加法和乘法，且对于 F 中的任意元素 a, b, c 满足以下公理：

(A1 ~ M6) F 是一个整环；也就是说 F 满足从 A1 到 A5 以及从 M1 到 M6 的所有原则。

(M7) 乘法逆元：对于 F 中的任意元素 a（除 0 以外），F 中都存在一个元素 a^{-1} 使得下式成立：

$$a\,a^{-1} = (a^{-1})a = 1$$

本质上说，域就是一个集合，我们可以在其上进行加法、减法、乘法和除法而不脱离该集合。除法又按以下规则来定义：$a/b = a(b^{-1})$。

---

① 一般地，我们并不用 × 表示乘法，而是用两个元素的连接表示乘法。

有理数集合、实数集合以及复数集合都是我们所熟悉的域的例子。需要指出的是,所有整数的集合并不是一个域,因为并不是集合中所有的元素都有乘法逆元;实际上,整数集合中只有元素 1 和 -1 有乘法逆元。

图 4.2 总结了定义群、环和域的公理。

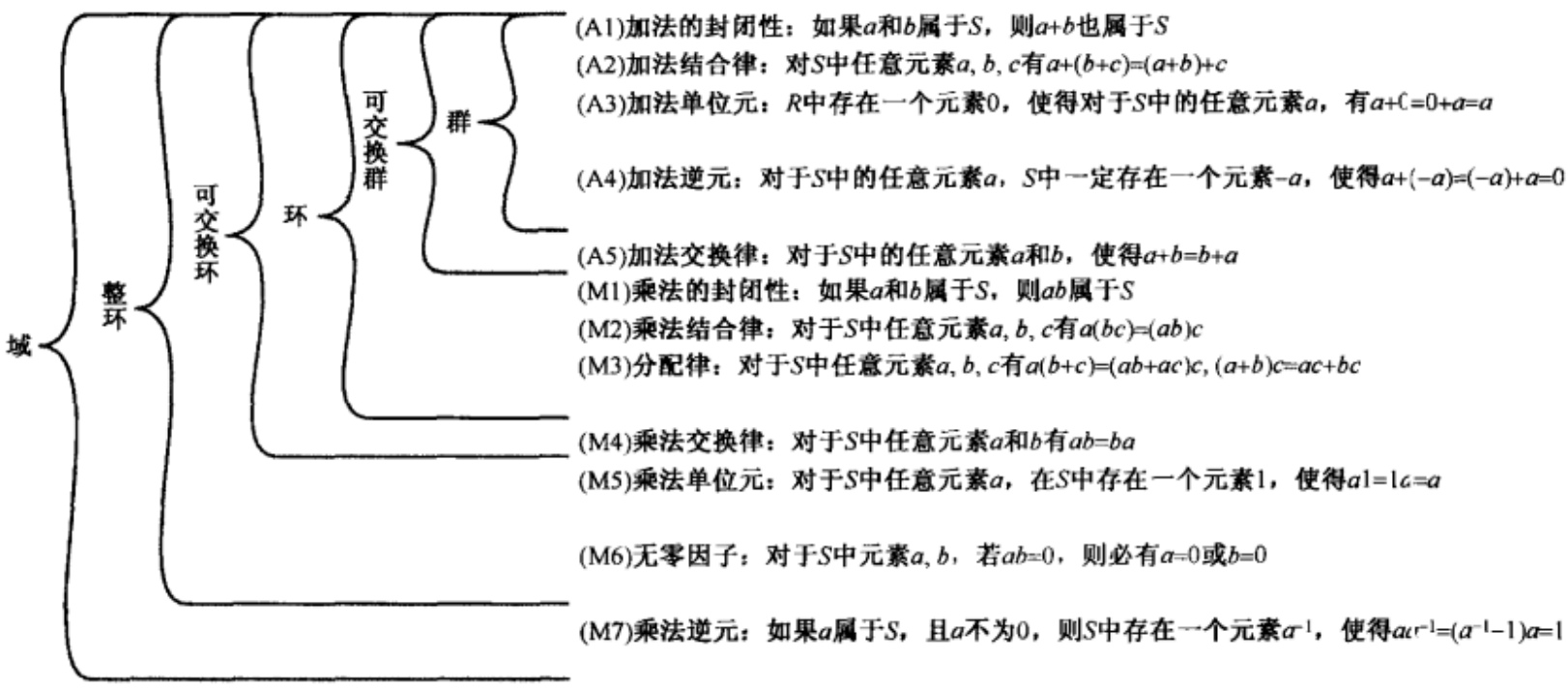


图 4.2 群、环和域

4.5 有限域 GF(p)

在 4.4 节中,我们把域定义为一个满足图4.2中所有公理的集合,并给出了一些无限域的例子。无限域在密码学中没有特别的意义,然而,有限域却在许多密码学算法中扮演着重要的角色。可以看出,有限域的阶(元素的个数)必须是一个素数的幂 p^n,n 为正整数。我们将在第 8 章详细讨论素数。在这里,我们只需要知道素数是只有两个正整数因子(即 1 和它本身)的整数。

阶为 p^n 的有限域一般记为 $GF(p^n)$,GF 代表 Galois 域,以第一位研究有限域的数学家的名字命名。我们在此要关注两种特殊的情形:$n=1$ 时的有限域 $GF(p)$,它和 $n>1$ 的有限域相比有着不同的结构,我们将在本节对它进行研究,在 4.7 节将研究 $GF(2^n)$ 形式的有限域。

4.5.1 阶为 p 的有限域

给定一个素数 p,元素个数为 p 的有限域 $GF(p)$ 被定义为整数 $\{0,1,\cdots,p-1\}$ 的集合 Z_p,其运算为模 p 的算术运算。

我们曾在 4.3 节讲到,整数 $\{0,1,\cdots,n-1\}$ 的集合 Z_n,在模 n 的算术运算下,构成一个交换环(参见表 4.3)。我们进一步发现,Z_n 中的任一整数有乘法逆元当且仅当该整数与 n 互素[参见式(4.5)的讨论]①。若 n 为素数,Z_n 中所有的非零整数都与 n 互素,因此 Z_n 中所有非零整数都有乘法逆元。因此,我们可以在表 4.3 中列出的 Z_p 性质加上下面一条:

① 讨论式(4.5)时说过,如果两个整数的唯一正整数公因子是 1,则称它们互素。

乘法逆元(w^{-1})	任意 $w \in Z_p$，$w \neq 0$，存在 $z \in Z_p$ 使得 $w \times z \equiv 1 \pmod p$

因为 w 和 p 互素，如果我们用 w 乘以 Z_p 的所有元素，得出的剩余类是 Z_p 中所有元素的另一种排列。因此，恰好只有一个剩余类值为 1。因而，Z_p 中有这样的整数，当它乘以 w，得余数 1。这个整数就是 w 的乘法逆元，记为 w^{-1}，所以 Z_p 其实是一个有限域。而且，式(4.5)和乘法逆元的存在是一致的，不需要附加条件就可以被改写如下：

$$\text{如果}(a \times b) \equiv (a \times c) \pmod p, \text{那么} b \equiv c \pmod p \tag{4.9}$$

将式(4.9)的两边同乘以 a 的乘法逆元，可得

$$((a^{-1}) \times a \times b) \equiv ((a^{-1}) \times a \times c) \pmod p$$
$$b \equiv c \pmod p$$

最简单的有限域是 GF(2)。它的算术运算可以简单地描述如下：

+	0	1
0	0	1
1	1	0

加

×	0	1
0	0	0
1	0	1

乘

w	$-w$	w^{-1}
0	0	–
1	1	1

求逆

在此，加等价于异或运算，乘等价于逻辑与运算。

表 4.5 描述了 GF(7)的算术运算。这是一个阶为 7 的域，采用模 7 运算。将会看到它满足图 4.2 中所示域的所有性质，与表 4.2 比较，表 4.2 中的集合 Z_8 用模 8 算术运算，不是域。本章的后面，我们将展示在 Z_8 上定义合适的加法和乘法使之成为一个域。

表 4.5　GF(7)的算术运算

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

(a)模 7 加法

×	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

(b)模 7 乘法

w	$-w$	w^{-1}
0	0	—
1	6	1
2	5	4
3	4	5
4	3	2
5	2	3
6	1	6

(c)模 7 加法逆元和乘法逆元

4.5.2 在 GF(p)中求乘法逆元

当 p 值较小时,求 GF(p)中元素的乘法逆元很容易。你只需构造一个乘法表,如表 4.5(b)所示,所要的结果可以从中直接读出。但是,当 p 值比较大时,这种方法是不切实际的。

如果 a 和 b 互素,则 b 有模 a 的乘法逆元。也就是说,如果 $\gcd(a,b)=1$,那么 b 有模 a 的乘法逆元。即对于正整数 $b<a$,存在 $b^{-1}<a$ 使 $bb^{-1}=1 \bmod a$。如果 a 是素数并且 $b<a$,则显然 a 和 b 互素,且其最大公因子为 1。运用扩展的Euclid 算法容易计算 b^{-1}。

再次重复式(4.7),我们已经证明过该式可以用扩展 Euclid 算法来解:

$$ax + by = d = \gcd(a, b)$$

现在,如果 $\gcd(a,b)=1$,则有 $ax+by=1$。运用 4.3 节定义的模算术的基本等式,我们有

$$[(ax \bmod a) + (by \bmod a)] \bmod a = 1 \bmod a$$

$$0 + (by \bmod a) = 1$$

然而,如果 $by \bmod a = 1$,则 $y = b^{-1}$。因此,如果 $\gcd(a,b)=1$,则通过对式(4.7)运用扩展 Euclid 算法可以获得 b 的乘法逆元。考虑表 4.4 中的那个例子。此时我们有 $a=1759$,这是一个素数,且 $b=550$。从方程 $1759x+550y=d$ 的解得到 $y=355$。因此 $b^{-1}=355$。容易验证,$550 \times 355 \bmod 1759 = 195\,250 \bmod 1759 = 1$。

更一般地,对于任意的 n,扩展 Euclid 算法可以用于求取 Z_n 内的乘法逆元。如果运用扩展 Euclid 算法于方程 $nx+by=d$,并且得到 $d=1$,则在 Z_n 内有 $y=b^{-1}$。

4.5.3 小结

在本节中,我们说明了如何构造阶为 p 的有限域,其中 p 为素数。特别地,我们定义的GF(p)具有如下性质。

(1) GF(p)由 p 个元素组成。

(2) 集合上定义了二元运算符 + 和 ×。加、减、乘、除可以在集合内进行。除了 0 外,集合内的其他元素都有乘法逆。

我们说明了 GF(p)的元素是整数$\{0,1,\cdots,p-1\}$,算术运算是模 p 的加法和乘法。

4.6 多项式运算

在继续讨论有限域之前,我们需要介绍一个有趣的问题——多项式算术。我们只讨论单变元多项式,且可把多项式运算分为三种。

- 使用代数基本规则的普通多项式运算。
- 系数运算是模 p 运算的多项式运算,即系数在 GF(p)中。
- 系数在 GF(p)中,且多项式被定义为模一个 n 次多项式 $m(x)$ 的多项式运算。

本节讨论前两种多项式运算,下一节再讨论最后一种。

4.6.1 普通多项式运算

一个 n 次多项式($n \geq 0$)的表达形式如下:

$$f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0 = \sum_{i=0}^{n} a_i x^i$$

其中 a_i 是某个指定数集 S 中的元素，该数集称为系数集，且 $a_n \neq 0$。我们称 $f(x)$ 是定义在系数集 S 上的多项式。

零次多项式称为常数多项式，仅仅是系数集里的一个元素。如果 $a_n = 1$，那么对应的 n 次多项式就被称为首 1 多项式。

在抽象代数中，我们一般不给多项式中的 x 赋一个特定值[例如 $f(7)$]。为了强调这一点，变元 x 有时被称为不定元。

多项式运算包含加法、减法和乘法，这些运算是把变量 x 当做集合 S 中的一个元素来定义的。除法也以类似的方式定义，但这时要求 S 是域。这些域包括实数域、有理数域和素数 p 的域 Z_p。注意整数集并不是域，也不支持多项式除法运算。

加法、减法的运算规则是把相应的系数相加减。因此，如果

$$f(x) = \sum_{i=0}^{n} a_i x^i; \qquad g(x) = \sum_{i=0}^{m} b_i x^i; \qquad n \geq m$$

那么加法运算定义为

$$f(x) + g(x) = \sum_{i=0}^{m} (a_i + b_i)x^i + \sum_{i=m+1}^{n} a_i x^i$$

乘法运算定义为

$$f(x) \times g(x) = \sum_{i=0}^{n+m} c_i x^i$$

其中

$$c_k = a_0 b_k + a_1 b_{k-1} + \cdots + a_{k-1} b_1 + a_k b_0$$

在最后一个公式中，当 $i > n$ 时，我们令 $a_i = 0$，当 $i > m$ 时，令 $b_i = 0$。注意结果的次数等于两多项式次数之和。

例如，令 $f(x) = x^3 + x^2 + 2$，$g(x) = x^2 - x + 1$，其中 S 是整数集，则

$$f(x) + g(x) = x^3 + 2x^2 - x + 3$$
$$f(x) - g(x) = x^3 + x + 1$$
$$f(x) \times g(x) = x^5 + 3x^2 - 2x + 2$$

手工计算过程如图 4.3(a)至图 4.3(c)所示。除法将在后面讨论。

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ +\ (x^2 - x + 1) \\ \hline x^3 + 2x^2 - x + 3 \end{array}$$

(a) 加法

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ -\ (x^2 - x + 1) \\ \hline x^3 \quad + x + 1 \end{array}$$

(b) 减法

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ \times\ (x^2 - x + 1) \\ \hline x^3 + x^2 \quad + 2 \\ -x^4 - x^3 \quad - 2x \\ x^5 + x^4 \quad + 2x^2 \\ \hline x^5 \quad + 3x^2 - 2x + 2 \end{array}$$

(c) 乘法

$$\begin{array}{r|l} & x + 2 \\ \hline x^2 - x + 1 & x^3 + x^2 \quad + 2 \\ & x^3 - x^2 + x \\ \hline & 2x^2 - x + 2 \\ & 2x^2 - 2x + 2 \\ \hline & x \end{array}$$

(d) 除法

图 4.3　多项式运算的例子

4.6.2 系数在 Z_p 中的多项式运算

现在我们考虑这样的多项式,它的系数是域 F 的元素。我们称其为域 F 上的多项式。这种情况下,容易看出这样的多项式集合是一个环,称为多项式环。也就是说,如果我们把每个不同的多项式视为集合中的元素,这个集合是一个环①。

对域上的多项式进行多项式运算时,除法运算是可能的。注意这并不说能进行整除。我们来阐明两者的区别,在一个域中,给定两个元素 a 和 b,a 除以 b 的商也是这个域中的一个元素。然而,在非域的环 R 中,普通除法将得到一个商式和余式,这并不是整除。

考虑集合 S 中的除法运算 5/3。如果 S 是有理数集合(是域),那么结果可以简单地表示成 5/3,这是 S 中的一个元素。现在假设 S 是域 Z_7,在这种情况下,用表 4.5(c)计算:

$$5/3 = (5 \times 3 - 1) \bmod 7 = (5 \times 5) \bmod 7 = 4$$

这是一个整除。最后假设 S 是整数集(是环但不是域),那么 5/3 的结果是商为 1 余数为 2:

$$5/3 = 1 + 2/3$$

$$5 = 1 \times 3 + 2$$

因此,除法在整数集上并不是整除。

现在,如果试图在非域系数集上进行多项式除法,我们会发现除法运算并不总是有定义的。

如果系数集是整数集,那么$(5x^2)/(3x)$没有结果,因为需要一个值为 5/3 的系数,这在系数集中是没有的。同一个多项式,如果在 Z_7 上进行除法运算,有$(5x^2)/(3x) = 4x$,这在 Z_7 中是一个合法的多项式。

然而,正如我们现在所见,即使系数集是一个域,多项式除法也不一定是整除。一般说来,除法会产生一个商式和一个余式。对于域上的多项式,式(4.1)的除法可以重述如下:给定 n 次多项式 $f(x)$ 和 $m(m \leqslant n)$ 次多项式 $g(x)$,如果用 $g(x)$ 除 $f(x)$,我们得到一个商 $q(x)$ 和一个余数 $r(x)$,满足如下关系式:

$$f(x) = q(x)g(x) + r(x) \tag{4.10}$$

各多项式的次数为:

Degree $f(x) = n$

Degree $g(x) = m$

Degree $q(x) = n - m$

Degree $r(x) \leqslant m - 1$

如果允许余数,我们说域上多项式除法是可以的。

与整数运算相似,我们可以在式(4.10)中把余式 $r(x)$ 写为 $f(x) \bmod g(x)$。即 $r(x) = f(x) \bmod g(x)$。如果这里没有余式[即 $r(x) = 0$],那么可以说 $g(x)$ 整除 $f(x)$,写为 $g(x) \mid f(x)$;等价地,可以说 $g(x)$ 是 $f(x)$ 的一个因式或除式。

对于上个例子[$f(x) = x^3 + x^2 + 2$ 且 $g(x) = x^3 - x + 1$],$f(x)/g(x)$ 产生一个商式 $q(x) = x + 2$ 和一个余式 $r(x) = x$,如图 4.3(d)所示。这很容易证实,只要注意到

$$q(x)g(x) + r(x) = (x+2)(x^2 - x + 1) + x = (x^3 + x^2 - x + 2) + x = x^3 + x^2 + 2 = f(x)$$

① 事实上,系数在交换环中的多项式集合构成多项式环,但是这个结论在这里并无多大的意义。

对我们而言，GF(2)上的多项式最有意义的。4.5 节中提过，在 GF(2)中，加法等价于 XOR 运算，乘法等价于逻辑与运算。而且，模 2 的加法和减法是等价的：$1+1=1-1=0$；$1+0=1-0=1$；$0+1=0-1=1$。

图 4.4 给出了 GF(2)上多项式运算的一个例子。对于 $f(x)=(x^7+x^5+x^4+x^3+x+1)$ 和 $g(x)=(x^3+x+1)$，图中显示了 $f(x)+g(x)$，$f(x)-g(x)$，$f(x)\times g(x)$ 和 $f(x)/g(x)$。注意，$g(x)|f(x)$。

域 F 上的多项式 $f(x)$ 被称为不可约的（既约的）当且仅当 $f(x)$ 不能表示为两个多项式的积[两个多项式都在 F 上，次数都低于 $f(x)$ 的次数]。与整数相似，一个不可约多项式也被称为素多项式。

$$
\begin{array}{rrrrrrrr}
 & x^7 & +x^5 & +x^4 & +x^3 & +x & +1 & \\
 & & & & +(x^3 & +x & +1) & \\
\hline
 & x^7 & +x^5 & +x^4 & & & &
\end{array}
$$

(a) 加法

$$
\begin{array}{rrrrrrrr}
 & x^7 & +x^5 & +x^4 & +x^3 & +x & +1 & \\
 & & & & -(x^3 & +x & +1) & \\
\hline
 & x^7 & +x^5 & +x^4 & & & &
\end{array}
$$

(b) 减法

$$
\begin{array}{rrrrrrrrrrrr}
 & & & x^7 & +x^5 & +x^4 & +x^3 & & +x & +1 & \\
 & & & & & & \times(x^3 & & +x & +1) & \\
\hline
 & & & x^7 & +x^5 & +x^4 & +x^3 & & +x & +1 & \\
 & & x^8 & & +x^6 & +x^5 & +x^4 & +x^2 & +x & & \\
 & x^{10} & +x^8 & +x^7 & +x^6 & & +x^4 & +x^3 & & & \\
\hline
 & x^{10} & & & & & +x^4 & +x^2 & & +1 &
\end{array}
$$

(c) 乘法

$$
\begin{array}{r}
x^4+1 \\
x^3+x+1 \;\big/\; x^7+x^5+x^4+x^3+x+1 \\
\quad x^7+x^5+x^4 \\
\hline
\qquad\qquad x^3+x+1 \\
\qquad\qquad x^3+x+1 \\
\hline
\qquad\qquad 0
\end{array}
$$

(d) 除法

图 4.4　GF(2)上的多项式运算的例子

GF(2)上的多项式[①] $f(x)=x^4+1$ 是可约的，因为 $x^4+1=(x+1)(x^3+x^2+x+1)$。

考虑多项式 $f(x)=x^3+x+1$，容易观察到 x 不是 $f(x)$ 的一个因式，很容易得出 $x+1$ 也不是 $f(x)$ 的一个因式：

$$
\begin{array}{r}
x^2+x \\
x+1 \;\big/\; x^3+x+1 \\
\quad x^3+x^2 \\
\hline
\qquad x^2+x+1 \\
\qquad x^2+x \\
\hline
\qquad\qquad 1
\end{array}
$$

因此 $f(x)$ 没有一次因式。但是很容易观察到，如果 $f(x)$ 是可约的，它一定有一个二次因式和一个一次因式。因此，$f(x)$ 是不可约的。

① 如无标明，本章其余部分的多项式均是 GF(2)上的多项式。

4.6.3 求最大公因式

我们可以通过定义最大公因式来扩展域上的多项式运算和整数运算之间的相似性。如果

(1) $c(x)$能同时整除 $a(x)$和 $b(x)$。

(2) $a(x)$和 $b(x)$的任何因式都是 $c(x)$的因式。

就称多项式 $c(x)$为 $a(x)$和 $b(x)$的最大公因式。

下面是一个等价的定义:$\gcd[a(x),b(x)]$是能同时整除 $a(x)$和 $b(x)$的多项式中次数最高的一个。

我们可以改写 Euclid 算法来计算两个多项式的最大公因式。式(4.6)中的等式可以改写为如下的定理:

$$\gcd[a(x), b(x)] = \gcd[b(x), a(x) \bmod b(x)] \tag{4.11}$$

式(4.11)可以重复使用来决定最大公因子。将下列的方案和用于整数的 Euclid 算法的定义进行比较。

多项式 Euclid 算法

计　　算	其　满　足
$r_1(x) = a(x) \bmod b(x)$	$a(x) = q_1(x)b(x) + r_1(x)$
$r_2(x) = b(x) \bmod r_1(x)$	$b(x) = q_2(x)r_1(x) + r_2(x)$
$r_3(x) = r_1(x) \bmod r_2(x)$	$r_1(x) = q_3(x)r_2(x) + r_3(x)$
⋮	⋮
$r_n(x) = r_{n-2}(x) \bmod r_{n-1}(x)$	$r_{n-2}(x) = q_n(x)r_{n-1}(x) + r_n(x)$
$r_{n+1}(x) = r_{n-1}(x) \bmod r_n(x) = 0$	$r_{n-1}(x) = q_{n+1}(x)r_n(x) + 0$ $d(x) = \gcd(a(x), b(x)) = r_n(x)$

每一循环里我们都有 $d(x) = \gcd(r_{i+1}(x), r_i(x))$,直至最后 $d(x) = \gcd(r_n(x), 0) = r_n(x)$。因此,通过重复应用除法,我们得到了两个多项式[①]的最大公因子。这就是用于多项式的 Euclid 算法。算法假设 $a(x)$的次数大于 $b(x)$的次数。

求 $\gcd[a(x),b(x)]$,其中 $a(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$, $b(x) = x^4 + x^2 + x + 1$。首先用$b(x)$除 $a(x)$:

$$\begin{array}{r} x^2 + x \\ x^4 + x^2 + x + 1 \overline{)\, x^6 + x^5 + x^4 + x^3 + x^2 + x + 1} \\ \underline{x^6 \quad + x^4 + x^3 + x^2} \\ x^5 \quad + x + 1 \\ \underline{x^5 \quad + x^3 + x^2 + x} \\ x^3 + x^2 \quad + 1 \end{array}$$

从而获得 $r_1(x) = x^3 + x^2 + 1$ 和 $q_1(x) = x^2 + x$。

接着用 $r_1(x)$除 $b(x)$。

① 原文为整数,有误——译者注。

$$
\begin{array}{r}
x+1 \\
x^3+x^2+1 \overline{)\,x^4 \quad +x^2+x+1} \\
\underline{x^4+x^3 \quad +x} \\
x^3+x^2 \quad +1 \\
\underline{x^3+x^2 \quad +1} \\
\end{array}
$$

从而获得 $r_2(x)=0, q_2(x)=x+1$。

因此，$\gcd[a(x),b(x)]=r_1(x)=x^3+x^2+1$。

4.6.4　小结

首先，我们讨论了一般多项式的算术。在一般多项式算术里，变量不予赋值，即我们不给多项式的变量赋值。相反，运用代数的一般规则对多项式进行算术操作（加、减、乘、除）。除非系数是域的元素，否则多项式除法是不行的。

接着，我们讨论了域 GF(p) 上的多项式算术：加、减、乘、除。然而，除法不是整除，即通常除法的结果是商和余数。

最后，我们说明了 Euclid 算法可以扩展用于求域上两个多项式的最大公因子。

本节论述的知识为下一节提供了一个基础。下一节中，我们将用多项式来构造阶为 p^n 的有限域。

4.7　有限域 GF(2^n)

在这一章的前几节中，我们提到过有限域的元素个数必须为 p^n，其中 p 为素数，n 为正整数。在 4.5 节中，我们讨论了元素个数为 p 的有限域这一特殊情况。我们发现在使用 Z_p 上的模运算时，它满足域的所有条件（参见图 4.2）。当 $n>1$，p^n 上的多项式在模 p^n 运算时并不能产生一个域。在本节中，我们将看到在一个具有 p^n 个元素的集合中，什么样的结构满足域的所有条件，并集中讨论 GF(2^n)。

4.7.1　动机

实际上所有的加密算法（包括对称密钥和公开密钥算法）都涉及整数集上的算术运算。如果某种算法使用的运算之一是除法，那么我们就必须使用定义在域上的运算。为了方便使用和提高效率，我们希望这个整数集中的数与给定的二进制位数所能表达的信息一一对应而不出现浪费。也就是说，我们希望这个整数集的范围是从 0 到 2^n-1，这样正好对应一个 n 位的字。

假设我们要构造一个传统加密算法，该算法一次处理 8 位的数据并且要使用除法。我们可以用 8 个二进制位来表示 0 ~ 255 之间的整数。但是由于 256 不是素数，如果使用 Z_{256} 上的模运算（以 256 为模的运算），这个集合就不是一个域了。小于 256 的素数中最大的是 251，所以在使用以 251 为模的运算时，集合 Z_{251} 是一个域。然而在这种情况下，在 8 个二进制位所能表示的全部整数中，251 ~ 255 不能使用，造成了存储空间的浪费。

上面的例子指出，如果我们既要使用所有的运算，又要使用 n 个二进制位所能表示的全部整

数,那么以2^n为模的运算是不行的。也就是说,当$n>1$时,以2^n为模的整数集不是一个域。退一步说,即使加密算法只使用加法和乘法而不使用除法,以Z_{2^n}为模的整数集仍然存在问题,如下例所示。

假设我们在加密算法中要使用3位的信息块,并且只使用加法和乘法。如表4.2所示,以8为模的运算是定义合理的。但是请注意,在乘法运算表中,各非零元素出现的次数是不同的。比如3只出现了4次,而4却出现了12次。另一方面,还可以使用本节提到的形式为GF(2^n)的有限域,即具有$2^3=8$个元素的特定的有限域。这个域上的运算如表4.6所示。这样的话,各非零元素的出现次数就是一致的。两种情况对比如下:

整数	1	2	3	4	5	6	7
在Z_8中的出现次数	4	8	4	12	4	8	4
在GF(2^3)中的出现次数	7	7	7	7	7	7	7

目前,我们暂时不管表4.6中的矩阵是如何构造的,我们先做如下观察:

(1) 加法和乘法表关于主对角线对称,这对应于加法和乘法的交换性。这个性质在表4.2中讲过,表4.2是模8算术。

(2) 表4.6中的非零元素都有乘法逆,而表4.2不是。

(3) 表4.6定义的集合满足有限域的所有要求,因此,我们可以称该域为GF(2^3)。

(4) 为方便,我们给出了GF(2^3)元素的3位表示。

表4.6 GF(2^3)上的运算

+	000	001	010	011	100	101	110	111
	0	1	2	3	4	5	6	7
000 0	0	1	2	3	4	5	6	7
001 1	1	0	3	2	5	4	7	6
010 2	2	3	0	1	6	7	4	5
011 3	3	2	1	0	7	6	5	4
100 4	4	5	6	7	0	1	2	3
101 5	5	4	7	6	1	0	3	2
110 6	6	7	4	5	2	3	0	1
111 7	7	6	5	4	3	2	1	0

(a)加法

×	000	001	010	011	100	101	110	111
	0	1	2	3	4	5	6	7
000 0	0	0	0	0	0	0	0	0
001 1	0	1	2	3	4	5	6	7
010 2	0	2	4	6	3	1	7	5
011 3	0	3	6	5	7	4	1	2
100 4	0	4	3	7	6	2	5	1
101 5	0	5	1	4	2	7	3	6
110 6	0	6	7	1	5	3	2	4
111 7	0	7	5	2	1	6	4	3

(b)乘法

w	$-w$	w^{-1}
0	0	—
1	1	1
2	2	5
3	3	6
4	4	7
5	5	2
6	6	3
7	7	4

(c)加法和乘法的逆元

直觉告诉我们，一个将整数集不均匀地映射到自身的算法用于加密时，可能要弱于一个提供均匀映射的算法。正因为如此，有限域 GF(2^n)对加密算法是很有吸引力的。

总之，我们要寻找一个包含 2^n 个元素的集合，其上定义了加法和乘法使之成为一个域。我们给集合的每一个元素赋值为 0 ~ $2^n - 1$ 之间的唯一整数。请记住我们不会用模算术，因为那样不能构成域。我们将会用多项式算术来构造我们需要的域。

4.7.2　多项式模运算

设集合 S 由域 Z_p 上次数小于等于 $n-1$ 的所有多项式组成。每一个多项式具有如下形式：

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1x + a_0 = \sum_{i=0}^{n-1} a_i x^i$$

其中，a_i 在集合 $\{0,1,\cdots,p-1\}$ 上取值。S 中共有 p^n 个不同的多项式。

当 $p=3, n=2$ 时，集合中共有 $3^2=9$ 个多项式，分别是

$$\begin{array}{ccc} 0 & x & 2x \\ 1 & x+1 & 2x+1 \\ 2 & x+2 & 2x+2 \end{array}$$

当 $p=2, n=3$ 时，集合中共有 $2^3=8$ 个多项式，分别是

$$\begin{array}{ccc} 0 & x+1 & x^2+x \\ 1 & x^2 & x^2+x+1 \\ x & x^2+1 & \end{array}$$

如果定义了合适的运算，那么每一个这样的集合 S 都是一个有限域。定义由如下几条组成：

(1) 该运算遵循代数基本规则中的普通多项式运算规则及如下两条限制。

(2) 系数运算以 p 为模，即遵循有限域 Z_p 上的运算规则。

(3) 如果乘法运算的结果是次数大于 $n-1$ 的多项式，那么必须将其除以某个次数为 n 的既约多项式 $m(x)$ 并取余式。对于多项式 $f(x)$，这个余数可表示为 $r(x) = f(x) \bmod m(x)$。

高级加密标准(AES)使用有限域 GF(2^8)上的运算，其中既约多项式 $m(x) = x^8 + x^4 + x^3 + x + 1$。

考虑多项式 $f(x) = x^6 + x^4 + x^2 + x + 1$，$g(x) = x^7 + x + 1$，则

$$\begin{aligned} f(x) + g(x) &= (x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1) \\ &= x^7 + x^6 + x^4 + x^2 \end{aligned}$$

$$\begin{aligned} f(x) \times g(x) &= (x^{13} + x^{11} + x^9 + x^8 + x^7) + (x^7 + x^5 + x^3 + x^2 + x) + (x^6 + x^4 + x^2 + x + 1) \\ &= x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 \end{aligned}$$

$$\begin{array}{r} x^5 + x^3 \\ x^8 + x^4 + x^3 + x + 1 \;\overline{)\; x^{13} + x^{11} + x^9 + x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + 1} \\ \underline{x^{13} \quad + x^9 + x^8 \quad\quad x^6 + x^5} \\ x^{11} \quad\quad\quad\quad\quad + x^4 + x^3 \\ \underline{x^{11} \quad\quad + x^7 + x^6 \quad\quad + x^4 + x^3} \\ x^7 + x^6 \quad\quad\quad\quad\quad\quad + 1 \end{array}$$

所以，$f(x) \times g(x) \bmod m(x) = x^7 + x^6 + 1$。

和简单模运算类似,多项式模运算中也有剩余类集合的概念。设 $m(x)$ 为 n 次多项式,则模 $m(x)$ 剩余类集合有 p^n 个元素,其中每个元素都可以表示成一个 m 次多项式($m<n$)。

以 $m(x)$ 为模的剩余类 $[x+1]$ 由所有满足 $a(x) \equiv (x+1) \pmod{m(x)}$ 的多项式 $a(x)$ 组成。也就是说,剩余类 $[x+1]$ 中的所有多项式 $a(x)$ 满足等式 $a(x) \bmod m(x) = x+1$。

由此可见,以 n 次既约多项式 $m(x)$ 为模的所有多项式组成的集合满足图 4.2 的所有公理,于是可以形成一个有限域。还可以进一步看到,所有具有相同阶的有限域都是同构的,即任意两个具有相同阶的有限域具有相同的结构,但是元素的表示和标记可能不同。

为了构造有限域 $GF(2^3)$,我们需要选择一个 3 次既约多项式。只有两个满足条件的多项式:x^3+x^2+1 和 x^3+x+1。若选择后者,则 $GF(2^3)$ 上的加法和乘法表如表 4.7 所示。由于该表和表 4.6 有相同的结构,所以我们已成功找到了定义阶为 2^3 的有限域的方法。

从表容易读出加法和乘法。例如,考虑 $100+010=110$。这等价于 x^2+x。再考虑 $100 \times 010 = 011$,这等价于 $x^2 \times x = x^3$,约减后为 $x+1$。

表 4.7 以 x^3+x+1 为模的多项式运算

(a)加法

+		000 0	001 1	010 x	011 $x+1$	100 x^2	101 x^2+1	110 x^2+x	111 x^2+x+1
000	0	0	1	x	$x+1$	x^2	x^2+1	x^2+1	x^2+x+1
001	1	1	0	$x+1$	x	x^2+1	x^2	x^2+x+1	x^2+x
010	x	x	$x+1$	0	1	x^2+x	x^2+x+1	x^2	x^2+1
011	$x+1$	$x+1$	x	1	0	x^2+x+1	x^2+x	x^2+1	x^2
100	x^2	x^2	x^2+1	x^2+x	x^2+x+1	0	1	x	$x+1$
101	x^2+1	x^2+1	x^2	x^2+x+1	x^2+x	1	0	$x+1$	x
110	x^2+x	x^2+x	x^2+x+1	x^2	x^2+1	x	$x+1$	0	1
111	x^2+x+1	x^2+x+1	x^2+x	x^2+1	x^2	$x+1$	x	1	0

(b)乘法

×		000 0	001 1	010 x	011 $x+1$	100 x^2	101 x^2+1	110 x^2+x	111 x^2+x+1
000	0	0	0	0	0	0	0	0	0
001	1	0	1	x	$x+1$	x^2	x^2+1	x^2+x	x^2+x+1
010	x	0	x	x^2	x^2+x	$x+1$	1	x^2+x+1	x^2+1
011	$x+1$	0	$x+1$	x^2+x	x^2+1	x^2+x+1	x^2	1	x
100	x^2	0	x^2	$x+1$	x^2+x+1	x^2+x	x	x^2+1	1
101	x^2+1	0	x^2+1	1	x^2	x	x^2+x+1	$x+1$	x^2+x
110	x^2+x	0	x^2+x	x^2+x+1	1	x^2+1	$x+1$	x	x^2
111	x^2+x+1	0	x^2+x+1	x^2+1	x	1	x^2+1	x^2	$x+1$

4.7.3 求乘法逆元

正如 Euclid 算法可以用来求两个多项式的最大公因式,扩展 Euclid 算法则可以用来求一个多项式的乘法逆元。如果多项式 $b(x)$ 的次数小于 $a(x)$ 的次数且 $\gcd[a(x),b(x)]=1$,那么该算法能求出 $b(x)$ 以 $a(x)$ 为模的乘法逆元。若 $a(x)$ 为既约多项式,即除了本身与 1 之外没有其他因

式,则始终有 $\gcd[a(x),b(x)]=1$。算法的描述方式和整数情形的扩展 Euclid 算法一样，给定两个多项式 $a(x)$和 $b(x)$,其中 $a(x)$的次数大于 $b(x)$的次数。我们希望解如下方程获得 $v(x)$,$w(x)$以及 $d(x)$,其中 $d(x)=\gcd[a(x),b(x)]$:

$$a(x)v(x)+b(x)w(x)=d(x)$$

如果 $d(x)=1$,则 $w(x)$是 $b(x)$模 $a(x)$的乘法逆。计算过程如下:

用于多项式的扩展 Euclid 算法

计　算	其 满 足	计　算	其 满 足
$r_{-1}(x)=a(x)$		$v_{-1}(x)=1;w_{-1}(x)=0$	$a(x)=a(x)v_{-1}(x)+bw_{-1}(x)$
$r_0(x)=b(x)$		$v_0(x)=0;w_0(x)=1$	$b(x)=a(x)v_0(x)+b(x)w_0(x)$
$r_1(x)=a(x) \bmod b(x)$ $q_1(x)=a(x)\mid b(x)$的商	$a(x)=q_1(x)b(x)+r_1(x)$	$v_1(x)=v_{-1}(x)-q_1(x)v_0(x)=1$ $w_1(x)=w_{-1}(x)-q_1(x)w_0(x)=-q_1(x)$	$r_1(x)=a(x)v_1(x)+b(x)w_1(x)$
$r_2(x)=b(x) \bmod r_1(x)$ $q_2(x)=b(x)\mid r_1(x)$的商	$b(x)=q_2(x)r_1(x)+r_2(x)$	$v_2(x)=v_0(x)-q_2(x)v_1(x)$ $w_2(x)=w_0(x)-q_2(x)w_1(x)$	$r_2(x)=a(x)v_2(x)+b(x)w_2(x)$
$r_3(x)=r_1(x) \bmod r_2(x)$ $q_3(x)=r_1(x)\mid r_2(x)$的商	$r_1(x)=q_3(x)r_2(x)+r_3(x)$	$v_3(x)=v_1(x)-q_3(x)v_2(x)$ $w_3(x)=w_1(x)-q_3(x)w_2(x)$	$r_3(x)=a(x)v_3(x)+b(x)w_3(x)$
⋮	⋮	⋮	⋮
$r_n(x)=r_{n-2}(x) \bmod r_{n-1}(x)$ $q_n(x)=r_{n-2}(x)\mid r_{n-3}(x)$的商	$r_{n-2}(x)=q_n(x)r_{n-1}(x)+r_n(x)$	$v_n(x)=v_{n-2}(x)-q_n(x)v_{n-1}(x)$ $w_n(x)=w_{n-2}(x)-q_n(x)w_{n-1}(x)$	$r_n(x)=a(x)v_n(x)+b(x)w_n(x)$
$r_{n+1}(x)=r_{n-1}(x) \bmod r_n(x)$ $q_{n+1}(x)=r_{n-1}(x)\mid r_{n-2}(x)$的商	$r_{n-1}(x)=q_{n+1}(x)r_n(x)+0$		$d(x)=\gcd(a(x),b(x))=r_n(x)$ $v(x)=v_n(x);w(x)=w_n(x)$

表4.8 给出了计算$(x^7+x+1) \bmod (x^8+x^4+x^3+x+1)$的乘法逆元的过程。结果是$(x^7+x+1)^{-1}=(x^7)$,即$(x^7+x+1)(x^7)\equiv 1 \pmod{(x^8+x^4+x^3+x+1)}$。

表 4.8　扩展 Euclid $[(x^8+x^4+x^3+x+1),(x^7+x+1)]$

初始化	$a(x)=x^8+x^4+x^3+x+1;\ v_{-1}(x)=1;\ w_{-1}(x)=0$
	$b(x)=x^7+x+1;\ v_0(x)=0;\ w_0(x)=1$
循环 1	$q_1(x)=x;\ r_1(x)=x^4+x^3+x^2+1$
	$v_1(x)=1;\ w_1(x)=x$
循环 2	$q_2(x)=x^3+x^2+1;\ r_2(x)=x$
	$v_2(x)=x^3+x^2+1;\ w_2(x)=x^4+x^3+x+1$
循环 3	$q_3(x)=x^3+x^2+x;\ r_3(x)=1$
	$v_3(x)=x^6+x^2+x+1;\ w_3(x)=x^7$
循环 4	$q_4(x)=x;\ r_4(x)=0$
	$v_4(x)=x^7+x+1;\ w_4(x)=x^8+x^4+x^3+x+1$
结果	$d(x)=r_3(x)=\gcd(a(x),b(x))=1$
	$w(x)=w_3(x)=(x^7+x+1)^{-1} \bmod (x^8+x^4+x^3+x+1)=x^7$

4.7.4 计算上的考虑

GF(2^n)中的多项式

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1x + a_0 = \sum_{i=0}^{n-1} a_i x_i$$

可以由它的 n 个二进制系数($a_{n-1}a_{n-2}\cdots a_0$)唯一地表示。因此,GF(2^n)中的每个多项式都可以表示成一个 n 位的二进制整数。

表4.6和表4.7给出了以 $m(x) = x^3 + x + 1$ 为模的域 GF(2^3)的加法和乘法表。其中,表4.6用二进制整数表示,而表4.7用多项式表示。

加法

我们发现这里的多项式加法是将相应的系数分别相加,而对于 Z_2 上的多项式,加法其实就是异或(XOR)运算。所以,GF(2^n)中的两个多项式加法等同于按位异或运算。

考虑前面例子中 GF(2^8)上的两个多项式 $f(x) = x^6 + x^4 + x^2 + x + 1$ 和 $g(x) = x^7 + x + 1$。

$(x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1)$	$= x^7 + x^6 + x^4 + x^2$	(多项式表示)
$01010111 \oplus 10000011$	$= 11010100$	(二进制表示)
$\{57\} \oplus \{83\}$	$= \{D4\}$	(十六进制表示)

乘法

简单的异或运算不能完成 GF(2^n)上的乘法。但是可以使用一种相当直观且容易实现的技巧。以 $m(x) = x^8 + x^4 + x^3 + x + 1$ 为多项式的有限域 GF(2^8)是 AES 中用到的有限域,我们将参照该域来讨论该技巧。这个技巧容易推广到域 GF(2^n)。

这个技巧基于下面的等式:

$$x^8 \bmod m(x) = [m(x) - x^8] = (x^4 + x^3 + x + 1) \tag{4.12}$$

稍做思考就不难证明式(4.12)是正确的。如果不确信,可以除一下。一般地,在 GF(2^n)上对于 n 次多项式 $p(x)$,有 $x^n \bmod p(x) = p(x) - x^n$。

现在考虑 GF(2^8)上的多项式 $f(x) = b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$,将它乘以 x,可得

$$\begin{aligned} x \times f(x) = (&b_7x^8 + b_6x^7 + b_5x^6 + b_4x^5 + b_3x^4 \\ &+ b_2x^3 + b_1x^2 + b_0x) \bmod m(x) \end{aligned} \tag{4.13}$$

如果 $b_7 = 0$,那么结果就是一个次数小于8的多项式,已经是约化后的形式,不需要进一步计算。如果 $b_7 = 1$,那么可以通过式(4.12)进行模 $m(x)$ 的约化:

$$\begin{aligned} x \times f(x) = &(b_6x^7 + b_5x^6 + b_4x^5 + b_3x^4 + b_2x^3 + b_1x^2 + b_0x) \\ &+ (x^4 + x^3 + x + 1) \end{aligned}$$

这表明乘以 x(即 00000010)的运算可以通过左移一位后再根据条件按位异或(00011011)(代表 $x^4 + x^3 + x + 1$)来实现。总结如下:

$$x \times f(x) = \begin{cases} (b_6b_5b_4b_3b_2b_1b_00), & b_7 = 0 \\ (b_6b_5b_4b_3b_2b_1b_00) \oplus (00011011), & b_7 = 1 \end{cases} \tag{4.14}$$

乘以 x 的更高次幂可以通过重复使用式(4.14)来实现。这样一来，GF(2^8)上的乘法可以用多个中间结果相加的方法实现。

在前面的例子中，我们给出了当$f(x)=x^6+x^4+x^2+x+1$，$g(x)=x^7+x+1$，$m(x)=x^8+x^4+x^3+x+1$时，$f(x)\times g(x) \bmod m(x)=x^7+x^6+1$的计算过程。现在我们用二进制运算的方法重做一遍，即计算(01010111)×(10000011)。首先要求出 x 幂乘 01010111 的中间结果：

$$(01010111)\times(00000010)=(10101110)$$
$$(01010111)\times(00000100)=(01011100)\oplus(00011011)=(01000111)$$
$$(01010111)\times(00001000)=(10001110)$$
$$(01010111)\times(00010000)=(00011100)\oplus(00011011)=(00000111)$$
$$(01010111)\times(00100000)=(00001110)$$
$$(01010111)\times(01000000)=(00011100)$$
$$(01010111)\times(10000000)=(00111000)$$

所以，$(01010111)\times(10000011)=[(01010111)\times(00000001)\oplus(00000010)\oplus(10000000)]$
$=(01010111)\oplus(10101110)\oplus(00111000)=(11000001)$

(11000001)等价于 x^7+x^6+1。

4.7.5 使用生成元

定义有限域 GF(2^n)的另一种等价方式有时更方便，它使用相同的不可约多项式。首先，我们需要两个定义：阶为 q 的有限域 F 的生成元是一个元素，记为 g，该元素的前 $q-1$ 个幂构成了 F 的所有非零元素，即域 F 的元素为 $0, g^0, \cdots, g^{q-2}$。考虑由多项式 $f(x)$ 定义的域 F，如果 F 内的一个元素 b 满足 $f(b)=0$，则称 b 为多项式的 $f(x)$ 的根。最后，可以证明一个不可约多项式的根 g 是这个不可约多项式定义的有限域的生成元。

我们考虑前面讨论的由不可约多项式 x^3+x+1 定义的有限域 GF(2^3)。设生成元为 g，则 g 满足 $f(g)=g^3+g+1=0$。如前所述，请记住我们不要这个方程的数值解。我们只处理多项式算术(系数运算是模 2 的)。因此，方程的解满足 $g^3=-g-1=g+1$。可以证明事实上 g 产生了所有次数小于 3 的多项式，我们有：

$$g^4=g(g^3)=g(g+1)=g^2+g$$
$$g^5=g(g^4)=g(g^2+g)=g^3+g^2=g^2+g+1$$
$$g^6=g(g^5)=g(g^2+g+1)=g^3+g^2+g=g^2+g+g+1=g^2+1$$
$$g^7=g(g^6)=g(g^2+1)=g^3+g=g+g+1=1=g^0$$

即 g 的幂产生了 GF(2^3)的所有非零多项式。同样，应清楚，对所有 k，有 $g^k=g^{k \bmod 7}$。表 4.9 列出了幂表示、多项式表示及二进制表示。

幂表示使乘法运算很容易。用幂表示，做乘法运算，只需对指数进行模 7 加，如 $g^4+g^6=g^{10 \bmod 7}=g^3=g^{+1}$。使用多项式算术也有相同的结果：$g^4=g^2+g$，$g^6=g^2+1$，则$(g^2+g)\times(g^2+1)=g^4+g^3+g^2+1$。接着，用除法计算$(g^4+g^3+g^2+1) \bmod (g^3+g+1)$：

$$
\begin{array}{r}
g+1 \\
g^3+g+1 \overline{)\, g^4+g^3+g^2+g} \\
\underline{g^4+\quad g^2+g} \\
g^3 \\
\underline{g^3+\quad g+1} \\
g+1
\end{array}
$$

得到结果 g+1。这和用幂表示计算的结果一致。

表 4.10 给出了用幂表示时,域 GF(2^3)的加法和乘法。请注意除了一些行或列做了交换以外,这些结果和用多项式表示的所得结果一致(参见表 4.7)。

表 4.9 模 x^3+x+1 的域 GF(2^3)的生成元

幂表示	多项式表示	二进制表示	十(十六)进制表示
0	0	000	0
g^0 ($=g^7$)	1	001	1
g^1	g	010	2
g^2	g^2	100	4
g^3	$g+1$	011	3
g^4	g^2+g	110	6
g^5	g^2+g+1	111	7
g^6	g^2+1	101	5

表 4.10 使用生成元的 GF(2^3)的算术,对应多项式为(x^3+x+1)

(a)加法

	+	000 0	001 1	010 G	100 g^2	011 g^3	110 g^4	111 g^5	101 g^6
000	0	0	1	G	g^2	$g+1$	g^2+g	g^2+g+1	g^2+1
001	1	1	0	$g+1$	g^2+1	g	g^2+g+1	g^2+g	g^2
010	g	g	$g+1$	0	g^2+g	1	g^2	g^2+1	g^2+g+1
100	g^2	g^2	g^2+1	g^2+g	0	g^2+g+1	g	$g+1$	1
011	g^3	$g+1$	g	1	g^2+g+1	0	g^2+1	g^2	g^2+g
110	g^4	g^2+g	g^2+g+1	g^2	g	g^2+1	0	1	$g+1$
111	g^5	g^2+g+1	g^2+g	g^2+1	$g+1$	g^2	1	0	g
101	g^6	g^2+1	g^2	g^2+g+1	1	g^2+g	$g+1$	g	0

(b)乘法

	×	000 0	001 1	010 G	100 g^2	011 g^3	110 g^4	111 g^5	101 g^6
000	0	0	0	0	0	0	0	0	0
001	1	0	1	G	g^2	$g+1$	g^2+g	g^2+g+1	g^2+1
010	g	0	g	g^2	$g+1$	g^2+g	g^2+g+1	g^2+1	1
100	g^2	0	g^2	$g+1$	g^2+g	g^2+g+1	g^2+1	1	g
011	g^3	0	$g+1$	g^2+g	g^2+g+1	g^2+1	1	g	g^2
110	g^4	0	g^2+g	g^2+g+1	g^2+1	1	g	g^2	$g+1$
111	g^5	0	g^2+g+1	g^2+1	1	g	g^2	$g+1$	g^2+g
101	g^6	0	g^2+1	1	g	g^2	$g+1$	g^2+g	g^2+g+1

通常，对于由不可约多项式 $f(x)$ 生成的域 GF(2^n)，有 $g^n = f(g) - g^n$。计算 g^{n+1} 到 g^{2^n-2} 的值。域的元素对应 g 的方幂：g^0 到 g^{2^n-2}，另外再加上 0 元素。域元素的乘法用等式 $g^k = g^{k \bmod (2^n-1)}$ 进行，k 为任意整数。

4.7.6　小结

本节中，我们说明了如何构造阶为 2^n 的有限域。特别地，我们定义了具有如下性质的域 GF(2^n)：

(1) GF(2^n)由 2^n 个元素组成。

(2) 集合上定义了二元运算 + 和 ×。加减乘除可以在集合内进行。除 0 元素外的其他元素都有乘法逆元。

我们说明了 GF(2^n)的元素可由二元域上所有次数不大于 $n-1$ 的多项式集合定义。每一个多项式可由唯一的 n 位数来表示。其上的算术是模某个次数为 n 的不可约多项式的多项式算术。我们还介绍了 GF(2^n)的一种等价定义。该定义利用了生成元，其运算用生成元的幂来进行。

4.8　推荐读物和网站

[HERS75]是抽象代数的经典参考资料，严谨精密，可读性强。[DESK92]是另一个很好的参考资料来源。[KNUT98]是关于多项式运算比较全面的参考资料。

[BERL84]是本章内容的最佳参考资料之一，仍在继续出版。[GARR01]是一本涵盖内容广泛的参考资料。关于有限域的全面而严谨的参考资料有[LIDL94]。[MURP00]也是一本严谨的读物。[HORO71]对本章所讨论的若干主题有很好的述论。

BERL84 Berlekamp, E. *Algebraic Coding Theory*. Laguna Hills, CA: Aegean Peak Press, 1984.

DESK92 Deskins, W. *Abstract Algebra*. New York: Dover, 1992.

GARR01 Garrett, P. *Making, Breaking Codes: An Introduction to Cryptology*. Upper Saddle River, NJ: Prentice Hall, 2001.

HERS75 Herstein, I. *Topics in Algebra*. New York: Wiley, 1975.

KNUT71 Horowita, E. "Modular Arithmetic and Finite Field Theory: A Tutorial." *Proceedings of the Second ACM Symposium and Symbolic and Algebraic Manipulation*, March 1971.

KNUT98 Knuth, D. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Reading, MA: Addison - Wesley, 1998.

LIDL94 Lidl, R. and Niederreiter, H. *Introduction to Finite Fields and Their Applications*. Cambridge: Cambridge University Press, 1994.

MURP00 Murphy, T. *Finite Field*. University of Dublin, Trinity College, School of Mathematics, 2000 (本书的网站可以找到文档)。

推荐网站

- PascGalois Project：包含一系列巧妙的举例和演示，能帮助学生形象化地理解抽象代数的重要概念。

4.9 关键术语、思考题和习题

关键术语

交换群	生成元	模运算
结合律	最大公因子	模数
系数集合	群	首 1 多项式
交换律	单位元	阶
交换环	无限群	多项式
循环群	无限环	多项式算术
因子	无限域	多项式环
Euclid 算法	整环	素数
域	逆元素	素多项式
有限群	不可约多项式	互素
有限环	模算术	余数
有限域	模多项式算术	环

思考题

4.1 简短地定义一个群。

4.2 简短地定义一个环。

4.3 简短地定义一个域。

4.4 解释“b 是 a 的因子”的含义。

4.5 模运算与普通运算的区别是什么?

4.6 列举三类多项式运算。

习题

4.1 设 S_n 是 n 个不同符号的所有置换组成的群。

(a) S_n 中有多少个元素?

(b) 说明当 $n>2$ 时,S_n 不是交换群。

4.2 判断对于下列运算,模 3 剩余类组成的集合是否能构成一个群。

(a) 模加

(b) 模乘法

4.3 集合 $S=\{a,b\}$ 上的加法和乘法定义如下:

+	a	b
a	a	b
b	b	a

×	a	b
a	a	a
b	a	b

判断 S 是否构成环,并证明你的结论。

4.4 去掉 a 为非负整数的限制,即 a 可以为任何整数,重新叙述式(4.1)。

4.5 对于 $a<0$,画一个与图 4.1 类似的图。

4.6 对于如下的每个方程,求相应的 x:

(a) $5x\equiv 4 \pmod 3$

(b) $7x \equiv 6 \pmod 5$

(c) $9x \equiv 8 \pmod 7$

4.7　本书我们假设模数为正整数。但是如果模数 n 为负数，表达式 $a \bmod n$ 也是完全有意义的。求如下表达式的值：

(a) $5 \bmod 3$

(b) $5 \bmod -3$

(c) $-5 \bmod 3$

(d) $-5 \bmod -3$

4.8　模数为 0 并不适合上述定义，但习惯上如下定义：$a \bmod 0 = a$。按此定义，式子 $a \equiv b \pmod 0$ 有何含义？

4.9　在 4.3 节中我们定义同余关系如下：整数 a 和 b 模 n 同余，当且仅当 $(a \bmod n) = (b \bmod n)$。然后证明了若 $n \mid (a-b)$，则 $a \equiv b \pmod n$。有些教材将后者作为同余关系的定义：如果 $n \mid (a-b)$，则整数 a 和 b 称为模 n 同余。以后一个定义为出发点，证明如果 $(a \bmod n) = (b \bmod n)$，那么 n 整除 $(a-b)$。

4.10　对于 $1 \leqslant k \leqslant 6$，刚好有 k 个因子的最小正整数是多少？

4.11　证明以下论断：

(a) 由 $a \equiv b \pmod n$ 可推出 $b \equiv a \pmod n$

(b) 由 $a \equiv b \pmod n$ 且 $b \equiv c \pmod n$ 可推出 $a \equiv c \pmod n$

4.12　证明以下等式：

(a) $[(a \bmod n) - (b \bmod n)] \bmod n = (a-b) \bmod n$

(b) $[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$

4.13　求 Z_5 中各非零元素的乘法逆元。

4.14　证明任意十进制整数 N 和它的各位数字之和模 9 同余。例如，$475 \equiv 4+7+5 \equiv 16 \equiv 1+6 \equiv 7 \pmod 9$。

4.15　(a) 求 gcd(24140,16762)

(b) 求 gcd(4655,12075)

4.16　本题的目的在于给 Euclid 算法的迭代轮数设置一个上限。

(a) 设 $m = qn + r$，其中 $q > 0, 0 \leqslant r < n$，证明 $m/2 > r$。

(b) 设 A_i 是 Euclid 算法进行 i 次迭代后 A 的值，证明 $A_{i+2} < A_i/2$。

(c) 证明如果整数 m, n, N 满足 $1 \leqslant m, n \leqslant 2^N$，那么 Euclid 算法至多进行 $2N$ 次迭代就可求出 $\gcd(m,n)$。

4.17　Euclid 算法已经有 2000 多年的历史，并且一直都被数论学者们公认为最佳算法。经过这么多年以后，J. Stein 于 1961 年发明了一种有潜力与 Euclid 算法竞争的算法。该算法求 $\gcd(A,B)$ $(A,B \geqslant 1)$ 的过程如下：

第 1 步　令 $A_1 = A, B_1 = B, C_1 = 1$

第 n 步　(1) 若 $A_n = B_n$，则返回 $\gcd(A,B) = A_n C_n$

(2) 若 A_n 和 B_n 均为偶数，则令 $A_{n+1} = A_n/2, B_{n+1} = B_n/2, C_{n+1} = 2C_n$

(3) 若 A_n 是偶数且 B_n 是奇数，则令 $A_{n+1} = A_n/2, B_{n+1} = B_n, C_{n+1} = C_n$

(4) 若 A_n 是奇数且 B_n 是偶数，则令 $A_{n+1} = A_n, B_{n+1} = B_n/2, C_{n+1} = C_n$

(5) 若 A_n 和 B_n 均为奇数，则令 $A_{n+1} = |A_n - B_n|, B_{n+1} = \min(A_n, B_n), C_{n+1} = C_n$

继续第 $n+1$ 步。

(a) 为了获得一些感性认识，请分别用 Euclid 算法和 Stein 算法计算 gcd(2152,764)。

(b) Stein 算法相对于 Euclid 算法的一个明显优势是什么？

4.18　(a) 证明若 Stein 算法在第 n 步前没有终止，则 $C_{n+1} \times \gcd(A_{n+1}, B_{n+1}) = C_n \times \gcd(A_n, B_n)$。

(b) 证明若 Stein 算法在第 $n-1$ 步前没有终止，则 $A_{n+2} B_{n+2} \leqslant A_n B_n / 2$。

(c) 证明若 $1 \leqslant A, B \leqslant 2^N$,则 Stein 算法至多需要 $4N$ 步就能求出 gcd(m,n)。因此,两个算法所需的计算步数大体相当。

(d) 证明 Stein 算法的返回值确实是 gcd(A,B)。

4.19 用扩展 Euclid 算法求下列乘法逆元。

(a) 1234 mod 4321 (b) 24140 mod 40902 (c) 550 mod 1769

4.20 类似于表 4.5,给出对应于 GF(5)的相应表。

4.21 证明以域元素为系数的多项式组成的集合是一个环。

4.22 判断下列关于域上多项式的说法是否正确,并加以证明。

(a) 首 1 多项式的乘积是首 1 多项式。

(b) 次数分别为 m 和 n 的两个多项式的乘积的次数为 $m+n$。

(c) 次数分别为 m 和 n 的两个多项式的和的次数为 $\max[m,n]$。

4.23 对于系数在 Z_{10}上取值的多项式运算,分别计算

(a) $(7x+2)-(x^2+5)$

(b) $(6x^2+x+3)\times(5x^2+2)$

4.24 判断下列多项式在 GF(2)上是否可约。

(a) x^3+1 (b) x^3+x^2+1 (c) x^4+1(仔细考虑)

4.25 求下列各对多项式的最大公因式。

(a) x^3+x+1 和 x^2+x+1 在 GF(2)上。

(b) x^3-x+1 和 x^2+1 在 GF(3)上。

(c) $x^5+x^4+x^3-x^2-x+1$ 和 x^3+x^2+x+1 在 GF(3)上。

(d) $x^5+88x^4+73x^3+83x^2+51x+67$ 和 $x^3+97x^2+40x+38$ 在 GF(101)上。

4.26 类似于表 4.7,给出以 $m(x)=x^2+x+1$ 为模的域 GF(4)的对应表。

4.27 求 x^3+x+1 在 GF(2^4)里的乘法逆元,模 $m(x)=x^4+x+1$。

4.28 类似于表 4.9,给出以 $m(x)=x^4+x+1$ 为模的域 GF(2^4)的对应表。

编程题

4.29 编写一个简单的程序,完成域 GF(2^4)的 4 种运算功能。可以用查表的方法来求乘法逆。

4.30 编写一个简单的程序,完成域 GF(2^8)的 4 种运算功能。求乘法逆应该一步完成。

附录 4A mod 的含义

本书和文献里使用的 mod 运算符有两种使用方式:作为一个二元操作符以及作为同余关系。本附录解释这种区别,并精确地定义本书中关于括号的标志。这种标志很常见,然而却不是通用的。

附录 4A.1 二元操作符 mod

如果 a 是整数,n 是非零整数,我们定义 $a \bmod n$ 是 a 被 n 除所得的余数。称整数 n 为模数,称余数为剩余。因此,对于任意整数 a,我们总有

$$a = \lfloor a/n \rfloor \times n + (a \bmod n)$$

正式地,mod 操作符定义为

$$a \bmod n = a - \lfloor a/n \rfloor \times n, \qquad n \neq 0$$

作为一个二元运算符,mod 输入两个整数变量,返回一个余数。例如,$7 \bmod 3 = 1$。变量可以

是整数、整数型变量或者整数型变量表达式。例如，如下例子都是正确的，具有明显的意义：

$$7 \bmod 3$$
$$7 \bmod m$$
$$x \bmod 3$$
$$x \bmod m$$
$$(x^2 + y + 1) \bmod (2m + n)$$

其中所有的变量都是整数。对每一个例子，左侧被右侧除，所得结果是余数。注意左侧或右侧变量是表达式，表达式需要用括号括起来。mod 运算符不在括号内。

事实上，如果两个变量是任意的实数，mod 运算符也有效，而不是仅仅对整数。本书中，我们仅关心整数运算。

附录 4A.2　同余关系 mod

作为一种同余关系，mod 表示两个变量关于一个给定的模数有相同的余数。例如，$7 \equiv 4 \pmod 3$ 表示 7 和 4 在被 3 除的情况下余数都为 1。如下两个表达式是等价的：

$$a \equiv b \pmod m \qquad \Leftrightarrow \qquad a \bmod m = b \bmod m$$

上式的另一种表述是“说表达式 $a \equiv b \pmod m$”和“说 $a - b$ 是 m 的整数倍”是一样的。同样，所有的变量可以是整数、整型变量或整型变量表达式。例如，如下例子都是正确的，具有明显意义：

$$7 \equiv 4 \pmod 3$$
$$x \equiv y \pmod m$$
$$(x^2 + y + 1) \equiv (a + 1) \pmod{[m + n]}$$

其中所有的变量是整数。此处有两个习惯。同余符号是 ≡，同余关系的模数放在括号内，在 mod 运算符之后。

同余关系用于定义剩余类。被 m 除后具有相同余数 r 的所有数形成一个 $(\bmod m)$ 的剩余类。有 m 个 $(\bmod m)$ 的剩余类。对于一个给定的余数 r，r 所在的剩余类由如下数字组成：

$$r, r \pm m, r \pm 2m, \cdots$$

根据我们的定义，同余

$$a \equiv b \pmod m$$

表明 a 和 b 相差 m 的整数倍。结果就是，同余式也可以表示为 a 和 b 属于相同的 $(\bmod m)$ 剩余类。

第5章　高级加密标准

5.1　有限域算术
5.2　AES的结构
　5.2.1　总体结构
　5.2.2　详细结构
5.3　AES变换函数
　5.3.1　字节代替变换
　5.3.2　行移位变换
　5.3.3　列混淆变换
　5.3.4　轮密钥加变换
5.4　AES的密钥扩展
　5.4.1　密钥扩展算法
　5.4.2　基本原理
5.5　一个AES例子
　5.5.1　结果
　5.5.2　雪崩效应
5.6　AES的实现
　5.6.1　等价的逆密码
　5.6.2　实现方面
5.7　推荐读物和网站
5.8　关键术语、思考题和习题
附录5A　系数在GF(2^8)内的多项式
附录5B　简化AES

"It seems very simple."

"It is very simple. But if you don't know what the key is it's virtually indecipherable."

—Talking to Strange Men, Ruth Rendell

要　点

- AES是一种分组密码，用以取代DES的商业应用。其分组长度为128位，密钥长度为128位、192位或256位。
- AES未采用Feistel结构。每轮由4个单独的运算组成：字节代替，置换，有限域上的算术运算，以及与密钥的异或运算。

美国国家标准技术研究所(NIST)在2001年发布了高级加密标准(AES)。AES是一个对称分组密码算法，旨在取代DES成为广泛使用的标准。与公钥密码如RSA相比，AES以及大多数对称密码的结构都很复杂，描述起来不能4像RSA及许多其他密码学算法那样容易。因此，读者也许希望从简化版的AES开始，附录5B为读者介绍了简化版本的AES。读者能使用该版本手工实现加密和解密，从而对于算法工作的具体细节有更深的理解。教学实践也表明对于简化版本AES的学习有助于更深入地理解AES①。一种可能的方法是先读本章，然后仔细阅读附录5B，最后再重新阅读本章的主体部分。

附录H介绍了NIST用来挑选AES算法的评估标准，以及挑选Rijndael作为胜出算法的合理性分析。这份材料不但对理解AES的设计有帮助，而且对理解评估其他对称加密算法的标准也有帮助。

① 读者在第一次阅读时可以跳过附录5B。如果读者对于AES的细节不能充分理解的话，则可以回过头从简化版本的AES开始。

5.1　有限域算术

AES 中的所有运算都是在 8 位的字节上进行的。特别地，加、减、乘、除算术都是在有限域 GF(2^8)上进行的。4.7 节比较详细地讨论了这些运算。对于没有读过第 4 章的读者，本节总结了一些重要的概念。对于读过第 4 章的读者，本节可以作为一个快速回顾。

本质上，一个域就是一个集合，在该集合内，我们可以进行加、减、乘、除运算，结果不离开集合。除法遵循规则 $a/b = a(b^{-1})$。一个有限域(具有有限个元素)的例子是集合 $Z_p = \{0,1,\cdots,p-1\}$，其中 p 是一个素数，且其内的运算是模 p 进行的。

实际上所有的加密算法，包括对称的和非对称的，都用到了整数的算术运算。如果算法用到了除法，我们需要用到定义在域上的算术。这是因为除法需要每一个非零元素有一个乘法逆元。出于习惯和实现的效率考虑，给定整数的位数，我们希望使用具有该位长度的所有整数，而不浪费一些位模式。即我们希望使用 0 到 $2^n - 1$ 内的所有整数，其刚好是一个 n 位的字。遗憾的是，这些整数的集合 Z_{2^n} 在模算术下并不是一个域。例如，整数 2 在 Z_{2^n} 内没有乘法逆元。也就是说，不存在 b 使得 $2b \bmod 2^n = 1$。

有一种方法可以定义含有 2^n 个元素的有限域，该域被称为 GF(2^n)。考虑所有次数小于等于 $n-1$，系数为 0、1 的多项式集合 S。因此，每一个多项式具有如下形式：

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_1x + a_0 = \sum_{i=0}^{n-1} a_i x^i$$

其中每个 a_i 取值 0 或 1，集合 S 内共有 2^n 个元素。对于 $n=3$，集合内的 8 个多项式为

$$0 \quad x \quad x^2 \quad x^2+x$$
$$1 \quad x+1 \quad x^2+1 \quad x^2+x+1$$

定义合适的算术运算，则每一个这样的集合 S 都是一个有限域。定义由如下各项组成。

(1) 该运算遵循代数基本规则中的普通多项式运算规则及如下两条限制。

(2) 系数运算以 2 为模，这和 XOR 运算是一样的。

(3) 如果乘法运算的结果是次数大于 $n-1$ 的多项式，那么必须用某个次数为 n 的不可约多项式 $m(x)$ 进行约化，即用 $m(x)$ 去除并取余式。对于多项式 $f(x)$，这个余数可表示为 $r(x) = f(x) \bmod m(x)$。一个多项式 $m(x)$ 称为不可约多项式，当且仅当该多项式不能表示为次数小于 $m(x)$ 的次数的两个多项式的乘积。

例如，为了构造 GF(2^3)，我们需要选择次数为 3 的不可约多项式。仅仅有两个这样的多项式：(x^3+x^2+1) 和 (x^3+x+1)。加法等价于各项的对位异或，因此 $(x+1)+x=1$。

GF(2^n)内的一个多项式可以由它的二元系数 $(a_{n-1}a_{n-2}\cdots a_0)$ 唯一表示。因此，GF(2^n)内的每个多项式可以由 n 位的数来表示。加法由两个 n 位的数进行对位异或来实现。GF(2^n)内的乘法没有简单的异或操作来实现，然而却有一种相当直观且容易实现的技巧。本质上，GF(2^n)内的数乘以 2 可以先左移，然后根据条件异或上一个常数。乘上一个大数可以重复运用该规则。

例如，AES 使用有限域 GF(2^8)内的算术，其中不可约多项式为 $m(x) = x^8 + x^4 + x^3 + x + 1$。考虑两个元素 $A = (a_7a_6\cdots a_1a_0)$ 和 $B = (b_7b_6\cdots b_1b_0)$。$A + B = (c_7c_6\cdots c_1c_0)$，其中 $c_i = a_i \oplus b_i$。当 $a_7 = 0$ 时乘积 $\{02\} \cdot A$ 等于 $(a_6\cdots a_1a_0 0)$，当 $a_7 = 1$ 时，乘积 $\{02\} \cdot A$ 等于 $(a_6\cdots a_1a_0 0) \oplus (00011011)$。

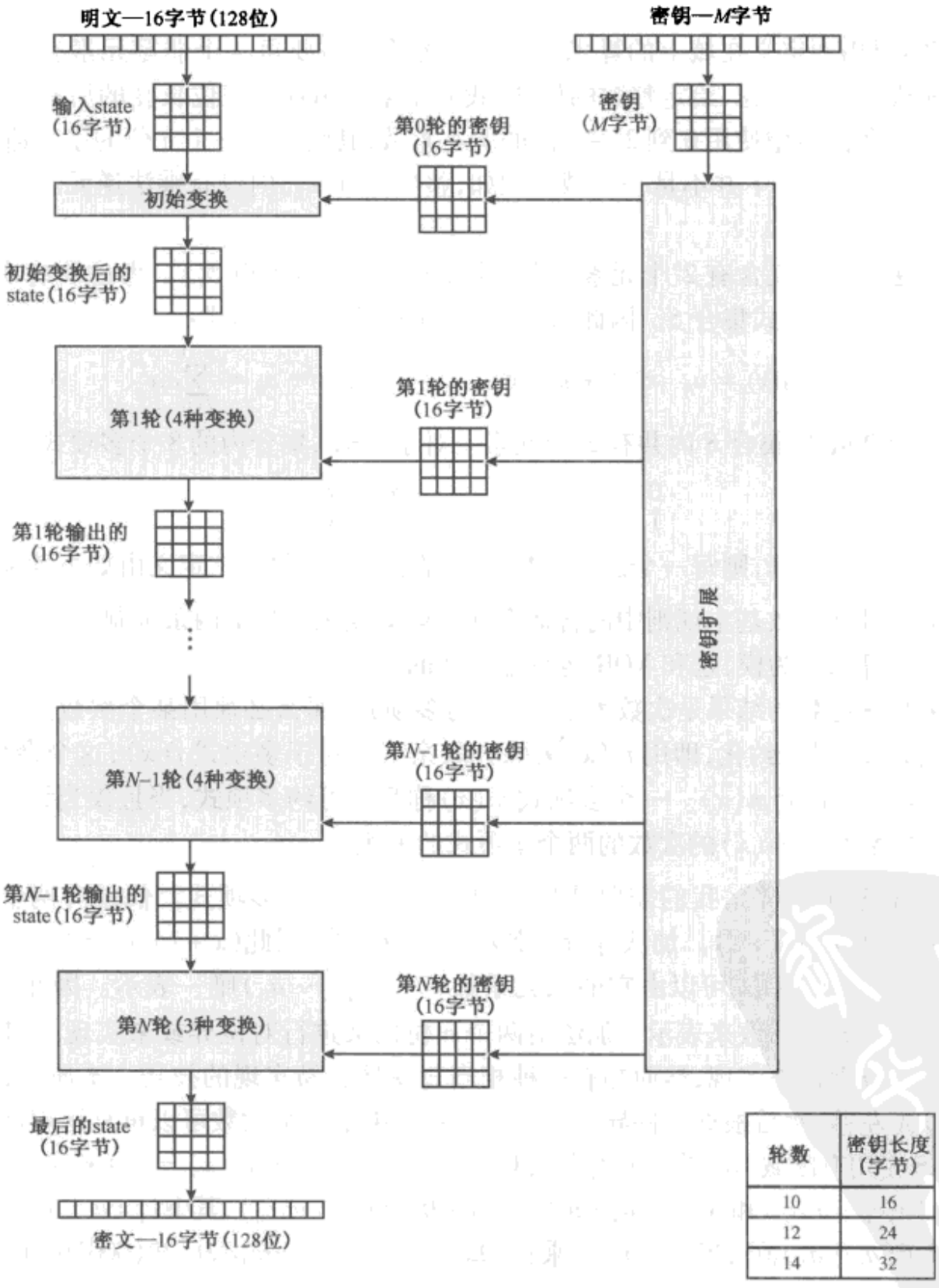
总结一下，AES 在 8 位的字节上运算。两个字节的加定义为对位异或操作。两个字节的乘定

义为有限域 GF(2^8)内的乘法,其中不可约多项式①为 $m(x) = x^8 + x^4 + x^3 + x + 1$。Rijndael 的开发者在谈到选择该多项式的动机时说,这个多项式是在[LIDL94]里 30 个不可约多项式列表的第一个多项式,所以他们就选择了该多项式。

5.2 AES 的结构

5.2.1 总体结构

图 5.1 展示了 AES 加密过程的总体结构。明文分组的长度为 128 位即 16 字节,密钥长度可以为 16,24 或 32 字节(128,192 或 256 位)。根据密钥的长度,算法被称为 AES-128、AES-192 或 AES-256。

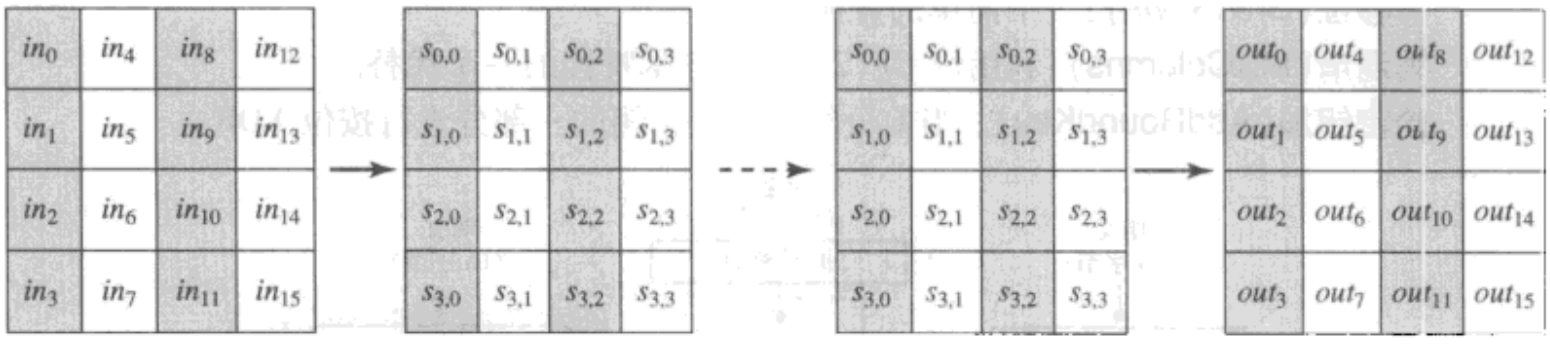


轮数	密钥长度(字节)
10	16
12	24
14	32

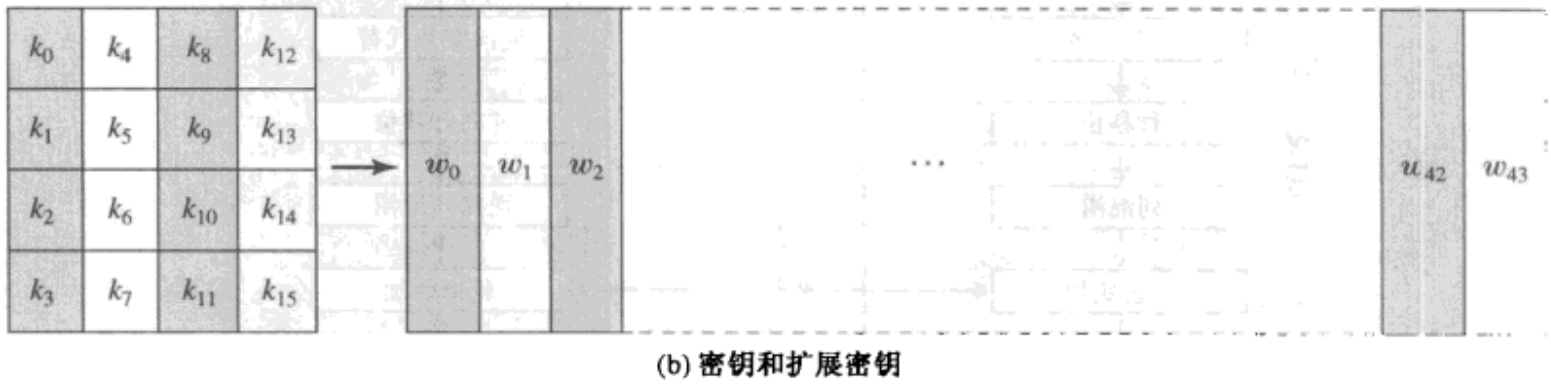
图 5.1 AES 的加密过程

① 本次讨论的其余部分,用到的域 GF(2^8)是指用这个多项式定义的有限域。

加密和解密算法的输入是一个 128 位分组。在 FIPS PUB 197 中,这个分组被描述为 4 ×4 的字节方阵。这个分组被复制到 state 数组,并在加密或解密的各个阶段被修改。图 5.2(a)描述了这些操作。同样地,密钥也被描述为字节的方阵。这个密钥接着被扩展为密钥字阵列。图 5.2(b)展示了 128 位密钥的扩展。每个字是 4 个字节,128 位的密钥最终扩展为 44 字的序列。注意在矩阵中字节是按照列进行排序的。所以,加密算法的 128 位明文分组输入的前 4 个字节被按顺序放在了 in 矩阵的第一列,接着的 4 个字节放在了第二列,等等。相似地,扩展密钥的前四个字节(形成一个字)被放在 w 矩阵的第一列。



(a) 输入、state数组和输出



(b) 密钥和扩展密钥

图 5.2　AES 的数据结构

密码由 N 轮组成,其中轮数依赖于密钥长度:16 字节密钥是 10 轮,24 字节密钥对应 12 轮,32 字节密钥对应 14 轮(参见表 5.1)。前 $N-1$ 轮由 4 个不同的变换组成:字节代替、行移位、列混淆和轮密钥加,后面将逐一介绍。最后一轮仅包含三个变换,而在第一轮的前面有一个起始的单变换(轮密钥加),可以视为 0 轮。每一个变换输入一个或多个 4 ×4 的矩阵,并输出一个 4 ×4 的矩阵。图 5.1 说明每轮的输出是一个 4 ×4 的矩阵,最后一轮的输出为密文。同样,密钥扩展函数产生 $N+1$ 轮密钥,它们是互不相同的 4 ×4 矩阵。每一个轮密钥作为每轮的轮密钥加变换的一种输入。

表 5.1　AES 的参数

密钥长度(字/字节/位)	4/16/128	6/24/192	8/32/256
明文分组长度(字/字节/位)	4/16/128	4/16/128	4/16/128
轮数	10	12	14
每轮的密钥长度(字/字节/位)	4/16/128	4/16/128	4/16/128
扩展密钥长度(字/字节)	44/176	52/208	60/240

5.2.2　详细结构

图 5.3 更加详细地展示了 AES 密码,指明了每轮的变换顺序,并展示了相应的解密函数。同第 3 章的做法一样,图中加密的过程是沿着页面向下,而解密是沿着页面向上。

在讨论 AES 的全部细节之前,我们对 AES 的总体结构做一些讨论。

(1) AES 结构的一个显著特征是它不是 Feistel 结构。回想一下经典的 Feistel 结构,数据分组中的一半被用来修改数据分组中的另一半,然后交换这两部分。AES 算法未使用 Feistel 结构,而是在每一轮都使用代替和混淆将整个数据分组作为一个单一的矩阵处理。

(2) 输入的密钥被扩展成由 44 个 32 位字所组成的数组 $w[i]$。从图 5.3 中可以看出,每轮有四个不同的字(128 位)作为该轮的轮密钥。

(3) 由四个不同的阶段组成,包括一个置换和三个代替:

字节代替(SubBytes):用一个 S 盒完成分组的字节到字节的代替。

行移位(ShiftRows):一个简单的置换。

列混淆(MixColumns):利用域 GF(2^8)上的算术特性的一个代替。

轮密钥加(AddRoundKey):当前分组和扩展密钥的一部分进行按位 XOR。

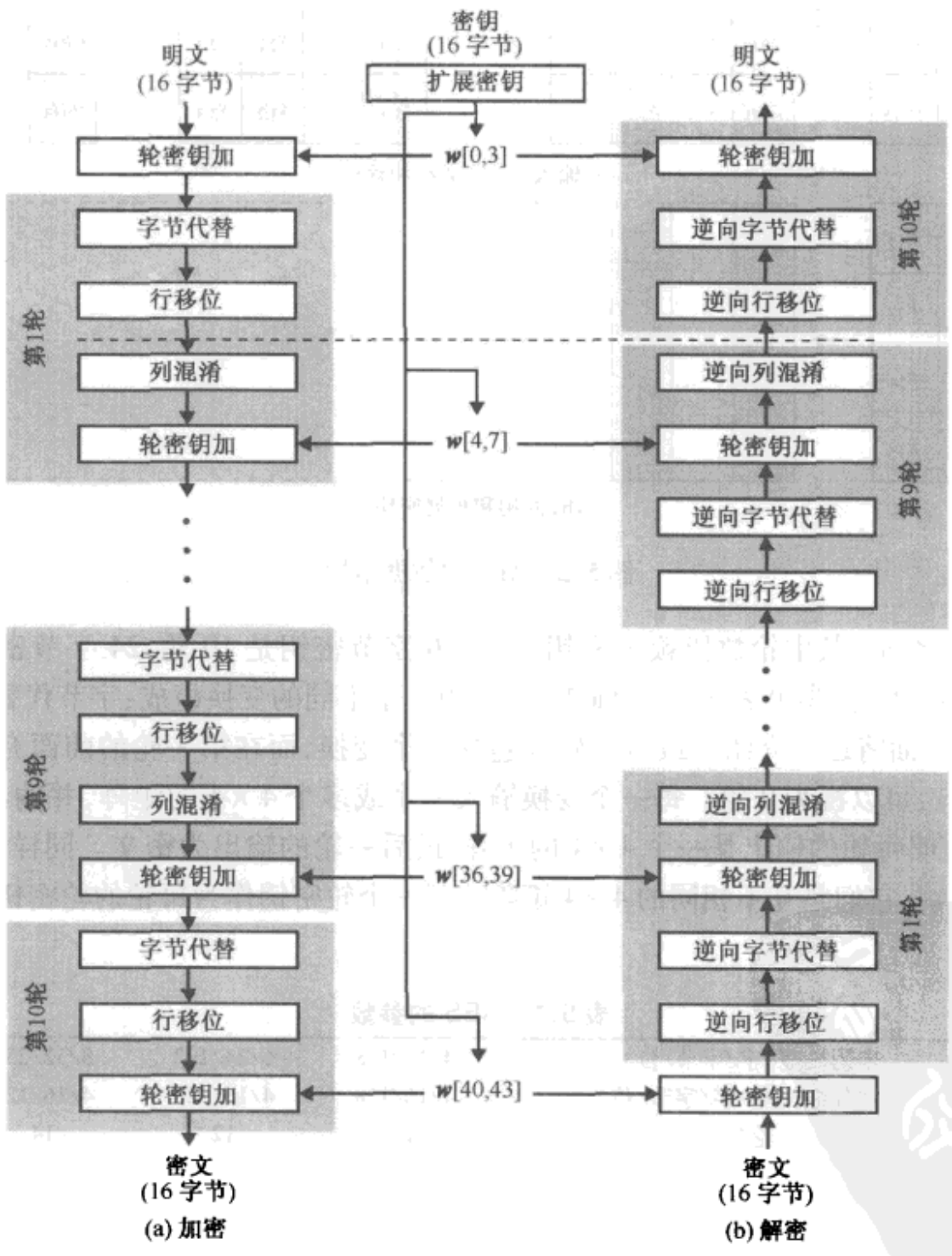


图 5.3 AES 加密和解密

(4) 算法结构非常简单。对加密和解密操作,算法由轮密钥加开始,接着执行 9 轮迭代运算,每轮都包含所有 4 个阶段的代替,接着是第 10 轮的三个阶段。图 5.4 描述了包含全部加密轮的结构。

(5) 仅仅在轮密钥加阶段中使用密钥。由于这个原因，该算法以轮密钥加开始，以轮密钥加结束。如果将其他不需要密钥的运算用于算法开始或结束的阶段，在不知道密钥的情况下就能计算其逆，故不能增加算法的安全性。

(6) 轮密钥加实质上是一种 Vernam 密码形式，就其本身是不难破译的。而另外三个阶段一起提供了混淆、扩散以及非线性功能。因这些阶段没有涉及密钥，故就它们自身而言，并未提供算法的安全性。我们可把该算法视为一个分组的 XOR 加密(轮密钥加)，接着对这个分组混淆(其他的三个阶段)，再接着又是 XOR 加密，如此交替执行。这种方式非常有效且非常安全。

(7) 每个阶段均可逆。对字节代替、行移位和列混淆，在解密算法中用与它们相对应的逆函数。轮密钥加的逆就是用同样的轮密钥和分组相异或，其原理就是 $A \oplus B \oplus B = A$。

(8) 同大多数分组密码一样，解密算法按逆序方式利用了扩展密钥。然而，AES 的解密算法和加密算法并不一样。这是由 AES 的特定结构所决定的。

(9) 一旦将所有的四个阶段求逆，很容易证明解密函数的确可以恢复原来的明文。图 5.3 中加密和解密流程在纵向上是相反的。在每个水平点上(如图中由破折号组成的线)，state 数组在加密和解密函数中是一样的。

(10) 加密和解密过程的最后一轮均只包含三个阶段。这是由 AES 的特定结构所决定的，而且也是密码算法可逆性所要求的。

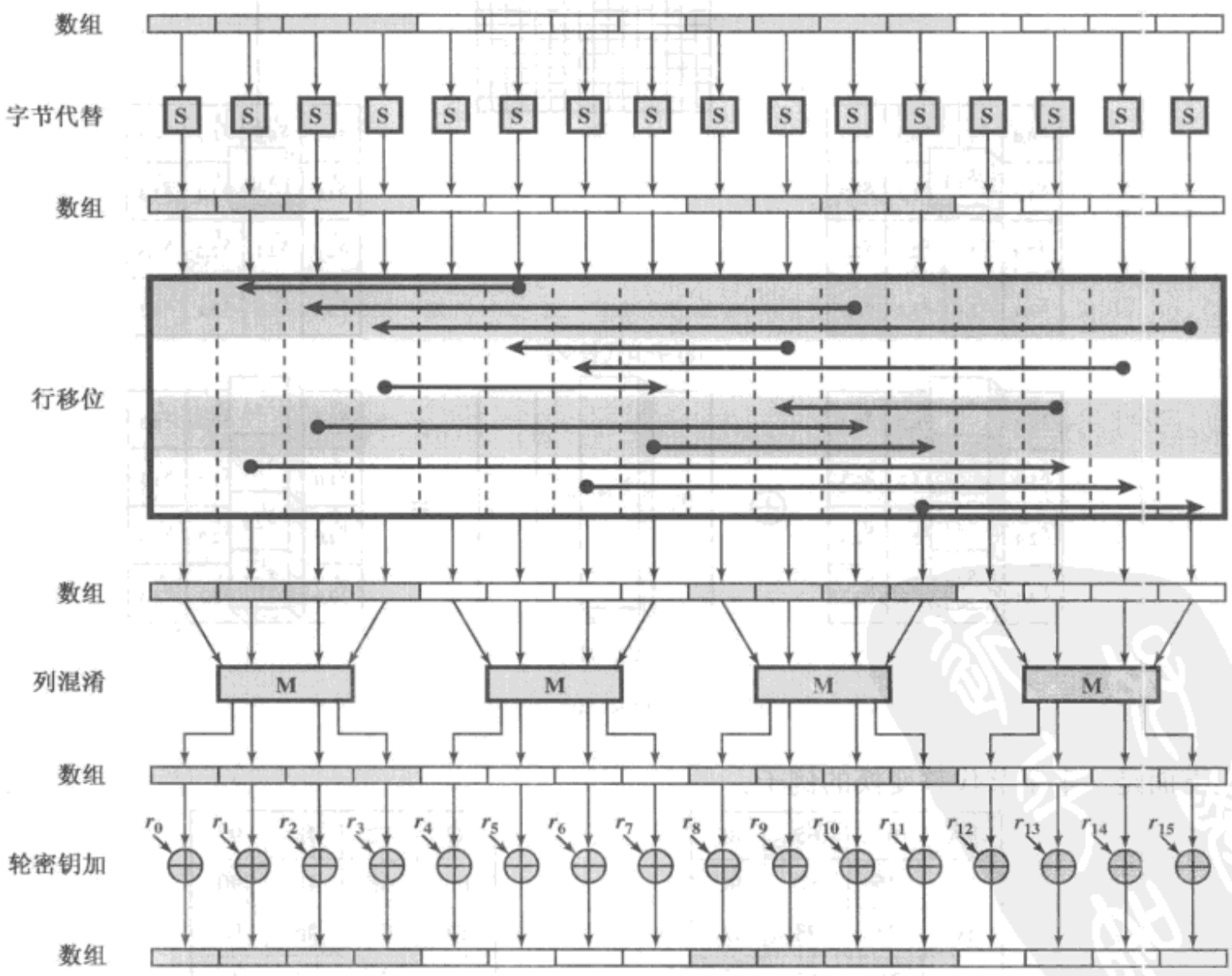


图 5.4　AES 的一轮加密过程

5.3 AES 的变换函数

我们现在开始分别讨论 AES 中使用的四个阶段。对每个阶段首先描述正向算法(加密算法)、逆向算法(解密算法),接着讨论该阶段的基本原理。

5.3.1 字节代替变换

正向和逆向变换。被称为字节代替的正向字节代替变换是一个简单的查表操作[参见图 5.5(a)]。AES 定义了一个 S 盒[参见图 5.2(a)],它是由 16 × 16 个字节组成的矩阵,包含了 8 位所能表示的 256 个数的一个置换。state 中每个字节按照如下的方式映射为一个新的字节:把该字节的高 4 位作为行值,低 4 位作为列值,以这些行列值作为索引从 S 盒的对应位置取出元素作为输出。例如,十六进制数{95}①所对应的 S 盒的行值是 9,列值是 5,S 盒中在此位置的值是{2A}。相应地,{95}被映射为{2A}。

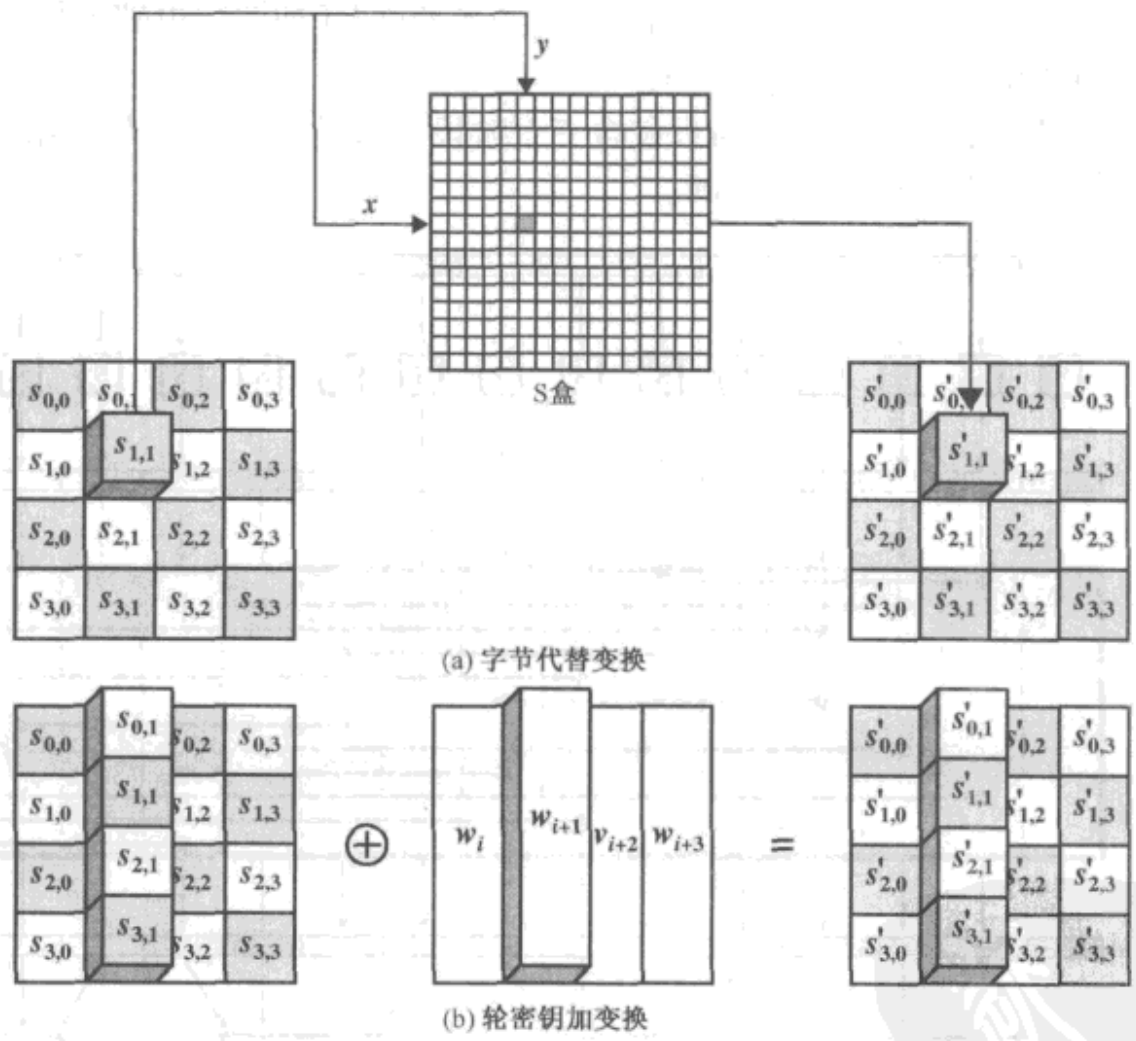


图 5.5 AES 的字节层操作

下面是一个字节代替变换的例子:

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

① 在 FIPS PUB 197 中,一个十六进制数用花括号来表示,本章我们也采用这种表示方法。

S 盒按如下的方式构造[参见图 5.6(a)]：

(1) 按字节值的升序逐行初始化 S 盒。第一行是{00}，{01}，{02}，…，{0F}；第二行是{10}，{11}，等。因此，在行 y 列 x 的字节值是{yx}。

(2) 把 S 盒中的每个字节映射为它在有限域 GF(2^8)中的逆；{00}被映射为它自身{00}。

(3) 把 S 盒中的每个字节的 8 个构成位记为($b_7, b_6, b_5, b_4, b_3, b_2, b_1, b_0$)。对 S 盒的每个字节的每个位做如下的变换：

$$b_i' = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i \qquad (5.1)$$

这里的 c_i 是指值为{63}的字节 c 的第 i 位，即($c_7\ c_6\ c_5\ c_4\ c_3\ c_2\ c_1\ c_0$) = (01100011)。符号(′)表示变量的值要被等式右边的值更新。AES 标准用矩阵形式描述了这个变换：

$$\begin{bmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \\ b_4' \\ b_5' \\ b_6' \\ b_7' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \qquad (5.2)$$

式(5.2)的解释需要仔细。在通常的矩阵乘法中[①]，乘积矩阵中的每个元素是一行和一列的对应元素乘积的和。在这个例子中，乘积矩阵中每个元素是一行和一列的对应元素乘积按位异或的值。进一步而言，式(5.2)中最终的加法是按位异或的。回忆 4.7 节知对位异或是 GF(2^8)内的加法。

例如，我们考虑输入值为{95}的情况。在 GF(2^8)中{95}$^{-1}$的乘法逆为{8A}，用二进制表示就是 10001010。用式(5.2)表示，就是

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

得到的结果是{2A}，这是 S 盒中行号为{09}、列号为{05}所对应的元素。这可以从表 5.2(a)中得到证实。

逆字节代替变换利用了表 5.2(b)中所示的逆 S 盒。例如，输入{2A}到逆 S 盒中，输出为{95}；输入{95}到 S 盒中，输出为{2A}。逆 S 盒的构造方法[参见图 5.6(b)]是利用式(5.1)的逆变换，然后再求其在 GF(2^8)内的乘法逆。该逆变换是

$$b_i' = b_{(i+2) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus d_i$$

在这里，字节 d = {05}，或者 00000101。我们可以用如下的方式来描述这个转换：

① 对于矩阵和向量乘法规则的简要介绍，请参考附录 E。

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

表5.2 AES的S盒

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a)S盒

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b)逆S盒

为了理解逆字节代替变换是字节代替变换的逆,令字节代替变换和逆字节代替变换中的矩阵

分别为 X 和 Y[①]，常量 c、d 的向量表示分别为 C 和 D。对某个 8 位的向量 B，式(5.2)变成了 $B' = XB \oplus C$。我们需要证明 $Y(XB \oplus C) \oplus D = B$。将括号里的内容乘出来，即我们应该证明 $YXB \oplus YC \oplus D = B$。这变成了

$$\begin{bmatrix} 0&0&1&0&0&1&0&1 \\ 1&0&0&1&0&0&1&0 \\ 0&1&0&0&1&0&0&1 \\ 1&0&1&0&0&1&0&0 \\ 0&1&0&1&0&0&1&0 \\ 0&0&1&0&1&0&0&1 \\ 1&0&0&1&0&1&0&0 \\ 0&1&0&0&1&0&1&0 \end{bmatrix} \begin{bmatrix} 1&0&0&0&1&1&1&1 \\ 1&1&0&0&0&1&1&1 \\ 1&1&1&0&0&0&1&1 \\ 1&1&1&1&0&0&0&1 \\ 1&1&1&1&1&0&0&0 \\ 0&1&1&1&1&1&0&0 \\ 0&0&1&1&1&1&1&0 \\ 0&0&0&1&1&1&1&1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \oplus$$

$$\begin{bmatrix} 0&0&1&0&0&1&0&1 \\ 1&0&0&1&0&0&1&0 \\ 0&1&0&0&1&0&0&1 \\ 1&0&1&0&0&1&0&0 \\ 0&1&0&1&0&0&1&0 \\ 0&0&1&0&1&0&0&1 \\ 1&0&0&1&0&1&0&0 \\ 0&1&0&0&1&0&1&0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1&0&0&0&0&0&0&0 \\ 0&1&0&0&0&0&0&0 \\ 0&0&1&0&0&0&0&0 \\ 0&0&0&1&0&0&0&0 \\ 0&0&0&0&1&0&0&0 \\ 0&0&0&0&0&1&0&0 \\ 0&0&0&0&0&0&1&0 \\ 0&0&0&0&0&0&0&1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix}$$

我们验证了 YX 等于单元矩阵，$YC = D$，于是 $YC \oplus D$ 等于零向量。

基本原理

S 盒被设计成能防止已有的各种密码分析攻击。Rijndael 的开发者特别寻求在输入位和输出位之间相关性很低的设计且输出值不能是输入的线性数学函数[DAEM01]。非线性度的产生是由于使用了乘法逆。另外，在式(5.1)中所选择的常量使得在 S 盒中没有不动点[S-box(a) = a]，也没有“反不动点”[S-box(a) = $\bar{a}$]，其中 $\bar{a}$ 表示 a 的逐位取反。

当然，S 盒必须是可逆的，即逆 S-box[S-box(a)] = a。然而，因 S-box(a) = 逆 S-box(a)不成立，在这个意义上 S 盒不是自逆的。例如，S-box({95}) = {2A}，但逆 S-box({95}) = {AD}。

5.3.2　行移位变换

正向和逆向变换

图 5.7(a)描述了正向行移位变换。stage 的第一行保持不变。把 state 的第二行循环左移一

① 原文为 B，有误——译者注。

个字节,state 的第三行循环左移两个字节,state 的第四行循环左移三个字节。行移位变换的一个例子如下:

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

⟶

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

逆向行移位变换将 state 中的后三行执行相反方向的移位操作,如第二行向右循环移一个字节,其他行类似。

基本原理

行移位变换要比它看起来有用得多。这是因为 state 和密码算法的输入和输出数据一样,是一个由 4 列字节组成的数组,其中每列由 4 个字节组成。因此在加密过程中,明文的前 4 个字节直接被复制到 state 的第一列中,接着的 4 个字节被复制到 state 的第二列中,等等。进一步而言,如下面将要看到的那样,轮密钥也是逐列地应用到 state 上的。因此,行移位就是将某个字节从一列移到另一列中,它的线性距离是 4 字节的倍数。同时请注意这个转换确保了某列中的 4 字节被扩展到了 4 个不同的列,图 5.4 说明了这个效果。

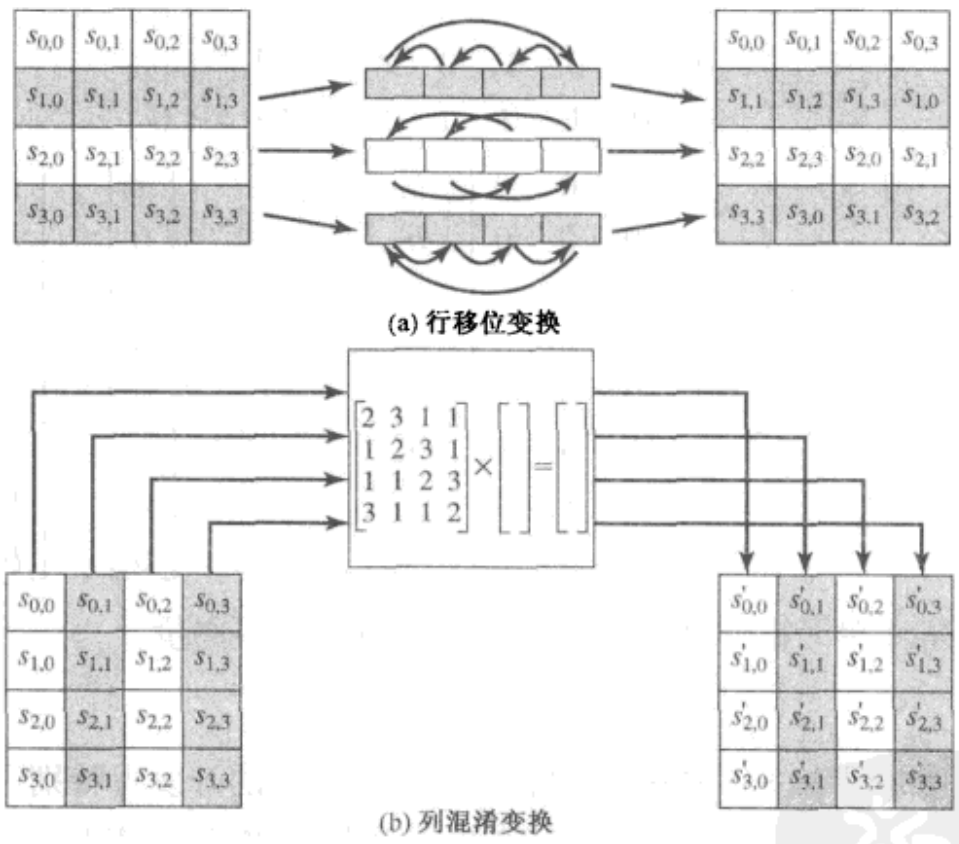


图 5.7　AES 的行与列操作

5.3.3　列混淆变换

正向和逆向变换

列混淆变换的正向列混淆变换对每列独立地进行操作。每列中的每个字节被映射为一个新值,此值由该列中的 4 个字节通过函数变换得到。这个变换可由下面基于 state 的矩阵乘法表示[参见图 5.7(b)]:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} \tag{5.3}$$

乘积矩阵中的每个元素均是一行和一列中的对应元素的乘积之和。在这里的乘法和加法[①]都是定义在 GF(2^8)上的。状态中单列的列混淆变换表示为

$$\begin{aligned} s'_{0,j} &= (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\ s'_{1,j} &= s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j}) \oplus s_{3,j} \\ s'_{2,j} &= s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j}) \\ s'_{3,j} &= (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j}) \end{aligned} \tag{5.4}$$

列混淆变换的一个例子如下：

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

⟶

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

让我们来验证一下该例子的第一列。我们在 4.7 节中提到在 GF(2^8)中，加法就是按位 XOR 操作，乘法的操作是按照式(4.14)所建立的规则进行的。特别是将某值乘上 x(即{02})其结果就是将该值向左移一位，如果该值的最左边的位为 1，那么在移位后还要异或(0001 1011)。所以，为了验证第一列的列混淆变换，我们需要证明

$$\begin{aligned} (\{02\} \cdot \{87\}) \oplus (\{03\} \cdot \{6E\}) \oplus \{46\} \oplus \{A6\} &= \{47\} \\ \{87\} \oplus (\{02\} \cdot \{6E\}) \oplus (\{03\} \cdot \{46\}) \oplus \{A6\} &= \{37\} \\ \{87\} \oplus \{6E\} \oplus (\{02\} \cdot \{46\}) \oplus (\{03\} \cdot \{A6\}) &= \{94\} \\ (\{03\} \cdot \{87\}) \oplus \{6E\} \oplus \{46\} \oplus (\{02\} \cdot \{A6\}) &= \{ED\} \end{aligned}$$

对第一个方程，我们有 $\{02\} \cdot \{87\} = (0000\ 1110) \oplus (0001\ 1011) = (0001\ 0101)$；$\{03\} \cdot \{6E\} = \{6E\} \oplus (\{02\} \cdot \{6E\}) = (0110\ 1110) \oplus (1101\ 1100) = (1011\ 0010)$。于是

$$\begin{array}{lll} \{02\} \cdot \{87\} & = & 0001\ 0101 \\ \{03\} \cdot \{6E\} & = & 1011\ 0010 \\ \{46\} & = & 0100\ 0110 \\ \{A6\} & = & \underline{1010\ 0110} \\ & & 0100\ 0111 = \{47\} \end{array}$$

其他的方程也可以通过类似的方式得以验证。

逆向列混淆变换可由如下的矩阵乘法定义：

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} \tag{5.5}$$

从直观上，不容易看出式(5.5)是式(5.3)的逆。我们需要进行如下运算：

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

① 我们遵从 FIPS PUB 197 的协定，使用符号 · 表示有限域 GF(2^8)上的乘法；使用⊕表示按位 XOR，这对应于 GF(2^8)中的加法。

这等价于

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.6}$$

即逆变换矩阵乘以正向变换矩阵结果为单位矩阵,为了验证式(5.6)的第一列,我们需要进行如下运算:

$$\begin{aligned} (\{0E\} \cdot \{02\}) \oplus \{0B\} \oplus \{0D\} \oplus (\{09\} \cdot \{03\}) &= \{01\} \\ (\{09\} \cdot \{02\}) \oplus \{0E\} \oplus \{0B\} \oplus (\{0D\} \cdot \{03\}) &= \{00\} \\ (\{0D\} \cdot \{02\}) \oplus \{09\} \oplus \{0E\} \oplus (\{0B\} \cdot \{03\}) &= \{00\} \\ (\{0B\} \cdot \{02\}) \oplus \{0D\} \oplus \{09\} \oplus (\{0E\} \cdot \{03\}) &= \{00\} \end{aligned}$$

对第一个等式,我们有 $\{0E\} \cdot \{02\} = 00011100$;而且 $\{09\} \cdot \{03\} = \{09\} \oplus (\{09\} \cdot \{02\}) = 00001001 \oplus 00010010 = 00011011$。于是

$$\begin{aligned} \{0E\} \cdot \{02\} &= 00011100 \\ \{0B\} &= 00001011 \\ \{0D\} &= 00001101 \\ \{09\} \cdot \{03\} &= \underline{00011011} \\ & \quad 00000001 \end{aligned}$$

利用类似的方法可以验证其他的方程。

AES 文档描述了另一种刻画列混淆变换的方式,即利用数学上的多项式来表示。在标准中,列混淆变换可以通过将 state 的每列考虑为一个系数在 $GF(2^8)$ 上的 4 项多项式来定义。每列乘上一个固定的多项式 $a(x)$,然后模 (x^4+1),$a(x)$ 的定义如下:

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\} \tag{5.7}$$

附录5A 论述了将 state 的每列乘上 $a(x)$ 能写成式(5.3)中的矩阵乘法。相似地,能把式(5.5)中的变换的每列视为一个 4 项多项式且把该列乘上 $b(x)$。$b(x)$ 的定义如下:

$$b(x) = \{0B\}x^3 + \{0D\}x^2 + \{09\}x + \{0E\} \tag{5.8}$$

显而易见 $b(x) = a^{-1}(x) \bmod (x^4+1)$。

基本原理

在式(5.3)中矩阵的系数是基于码字间有最大距离的线性编码,这使得在每列的所有字节中有良好的混淆性。列混淆变换和行移位变换使得在经过几轮变换后,所有的输出位均与所有的输入位相关。关于此方面的详细讨论参见[DAEM99]。

此外,列混淆变换的系数,即{01}、{02}、{03}是基于算法实现角度考虑的。正如上文所述,这些系数的乘法涉及至多一次移位和一次 XOR。逆向列混淆变换中的系数更加难以实现。然而加密被视为比解密更重要,原因如下:

(1) 对于 CFB 和 OFB 密码模式(第 6 章中的图 6.5 和图 6.6),仅使用加密算法。

(2) 和任何其他的分组密码一样,AES 能用于构造消息验证码(参见第 12 章),这仅仅用到了加密过程。

5.3.4 轮密钥加变换

正向和逆向变换

在轮密钥加变换中,128 位的 state 按位与 128 位的轮密钥 XOR。如图 5.5(b)所示,该操作可

以视为 state 的一列中的四个字节与轮密钥的一个字进行列间的操作；我们也能将其视为字节级别的操作。下面是轮密钥加的一个例子：

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$\oplus$

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

=

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

例子中第一个矩阵是 state，第二个矩阵是轮密钥。

逆向轮密钥加变换是和正向轮密钥加变换一样的，因为异或操作是其本身的逆。

基本原理

轮密钥加变换非常简单，却能影响 state 中的每一位。密钥扩展的复杂性和 AES 的其他阶段运算的复杂性，确保了该算法的安全性。

图 5.8 是描述单轮 AES 的另一种视角，强调各变换的机制和输入。

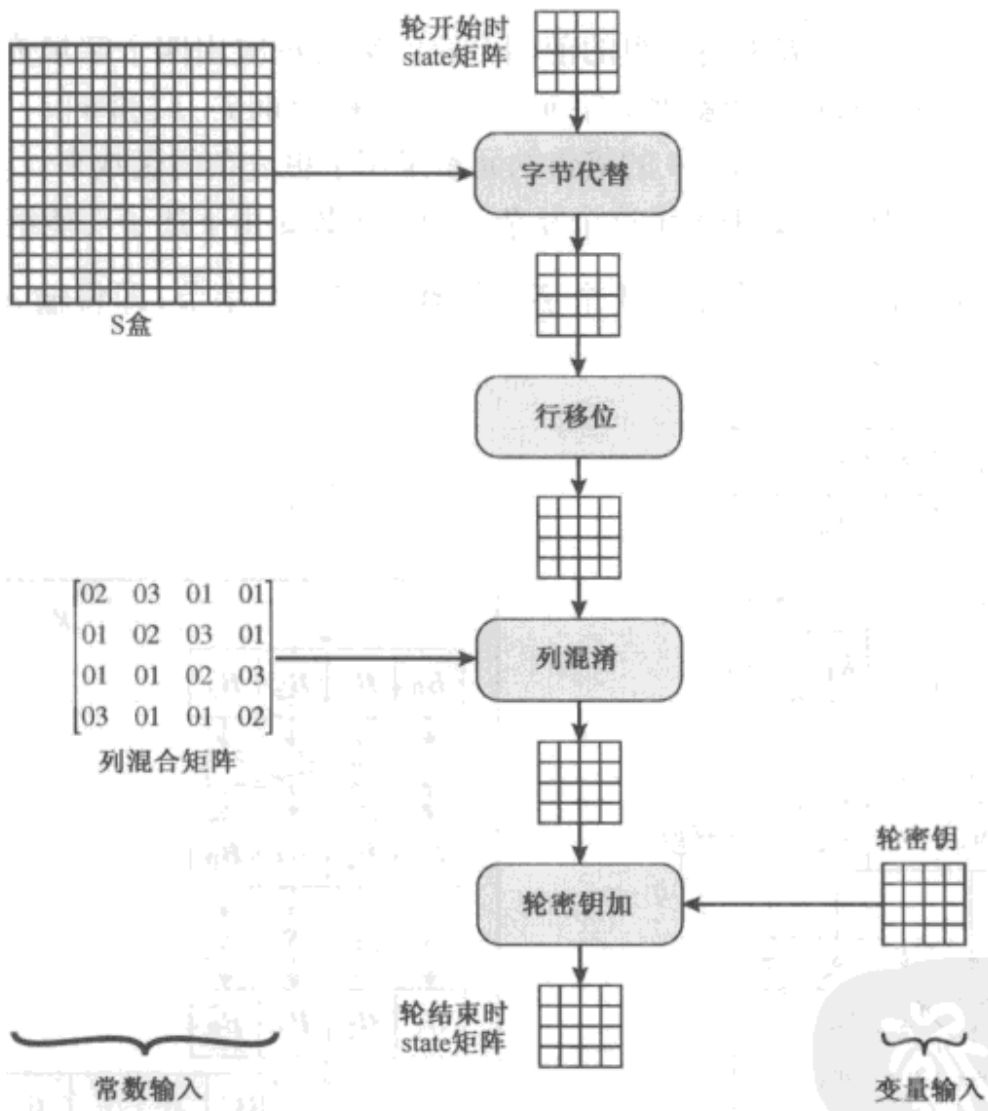


图 5.8　单轮 AES 的输入

5.4　AES 的密钥扩展

5.4.1　密钥扩展算法

AES 密钥扩展算法的输入值是 4 个字(16 字节)，输出值是一个由 44 个字组成(176 字节)的一维线性数组。这足以为初始轮密钥加阶段和算法中其他 10 轮中的每一轮提供 4 字的轮密钥。下面用伪码描述了这个扩展：

```
KeyExpansion (byte key[16], word w[44])
{
    word temp
    for (i = 0; i < 4; i++)    w[i] = (key[4*i], key[4*i+1],
                                       key[4*i+2],
                                       key[4*i+3]);

    for (i = 4; i < 44; i++)
    {
     temp = w[i - 1];
     if (i mod 4 = 0)    temp = SubWord (RotWord (temp))
                                ⊕ Rcon[i/4];

     w[i] = w[i-4] ⊕ temp
    }
}
```

输入密钥直接被复制到扩展密钥数组的前四个字。然后每次用四个字填充扩展密钥数组余下的部分。在扩展密钥数组中,每一个新增的字 $w[i]$ 的值依赖于 $w[i-1]$ 和 $w[i-4]$。在4个情形中,三个使用了异或。对 w 数组中下标为4的倍数的元素采用了更复杂的函数来计算。图5.9阐明了如何计算扩展密钥,其中使用符号 g 来表示这个复杂函数。函数 g 由下述子功能组成:

(1) 字循环的功能是使一个字中的4个字节循环左移一个字节,即将输入字 $[B_0, B_1, B_2, B_3]$ 变换成 $[B_1, B_2, B_3, B_0]$。

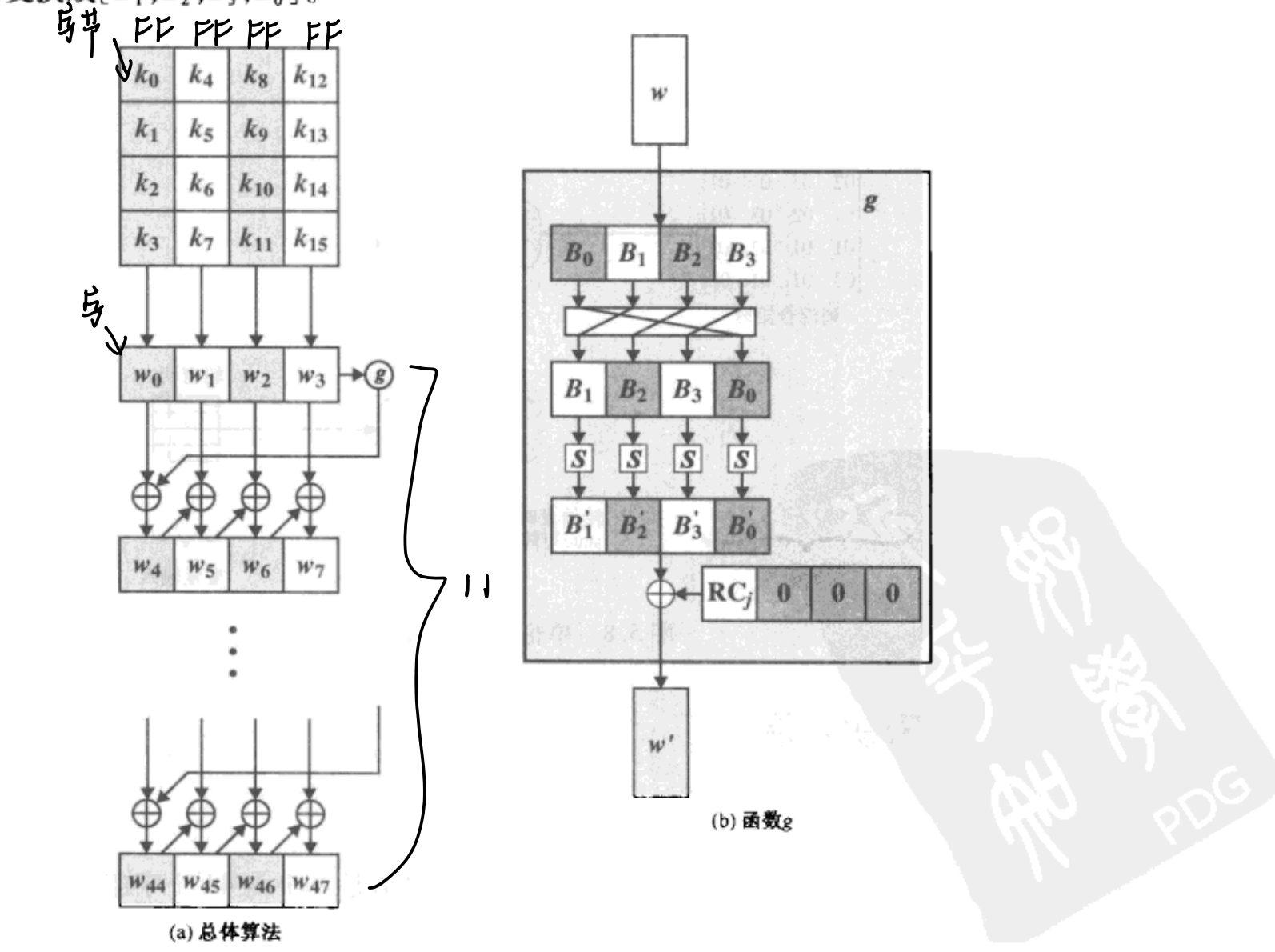


图 5.9 AES 的密钥扩展

（2）字代替利用 S 盒对输入字中的每个字节进行字节代替[参见表 5.2(a)]。

（3）步骤 1 和步骤 2 的结果再与轮常量 Rcon[j]相异或。

轮常量是一个字,这个字最右边三个字节总为0。因此字与 Rcon 相异或,其结果只是与该字最左的那个字节相异或。每轮的轮常量均不同,其定义为 Rcon[j] = (RC[j],0,0,0),其中RC[1] =1,RC[j] =2 · RC[j −1][乘法是定义在域 GF(2^8)上的]。RC[j]的值按十六进制表示为

j	1	2	3	4	5	6	7	8	9	10
RC[j]	01	02	04	08	10	20	40	80	1B	36

例如,假设第 8 轮的轮密钥为

EA　D2　73　21　B5　8D　BA　D2　31　2B　F5　60　7F　8D　29　2F

那么第 9 轮的轮密钥的前 4 个字节(第一列)能按如下的方式计算:

i(十进制)	temp	字循环后	字代替后	Rcon(9)	与 Rcon 进行 XOR 后	$w[i-4]$	$w[i] = temp \oplus w[i-4]$
36	7F8D292F	8D292F7F	5DA515D2	1B000000	46A515D2	EAD27321	AC7766F3

5.4.2 基本原理

Rijndael 的开发者设计了密钥扩展算法来防止已有的密码分析攻击。使用与轮相关的轮常量是为了防止不同轮的轮密钥产生方式上的对称性或相似性。[DAEM99]中使用的标准如下:

（1）知道密钥或轮密钥的部分位不能计算出轮密钥的其他位。

（2）它是一个可逆的变换[即知道扩展密钥中任何连续的 Nk 个字能够重新产生整个扩展密钥(Nk 是构成密钥所需的字数)]。

（3）能够在各种处理器上有效地执行。

（4）使用轮常量来消除对称性。

（5）将密钥的差异性扩散到轮密钥中的能力;即密钥的每个位能影响到轮密钥的许多位。

（6）足够的非线性以防止轮密钥的差异完全由密钥的差异所决定。

（7）易于描述。

作者并未量化上述列表的第一点,但指出了如果你知道密钥或轮密钥中少于 Nk 个连续字,那么你将难于构造出其余的未知位。知道的密钥的位数量越少,就越难于重构出或推测出密钥扩展中的其他位。

5.5 一个 AES 例子

现在我们研究一个例子并考虑它的含义。尽管不期待你手工来重做该例子,你还是会发现研究每一步间的十六进制模式将会给你带来很多的知识。

本例中,明文是十六进制表示的回文(顺读和倒读都一样)。明文、密钥和密文如下:

明文	0123456789abcdeffedcba9876543210
密钥	0f1571c947d9e8590cb7add6af7f6798
密文	ff0b844a0853bf7c6934ab4364148fb9

5.5.1 结果

表 5.3 展示了 16 字节密钥扩展为 10 轮的密钥。如前所述,这个过程是逐字进行的,每个 4 字节规模的字占用轮密钥字矩阵的一列。左侧的列展示的是为每轮产生的 4 个轮密钥字。右侧的

列展示的是密钥扩展中用于产生辅助字的步骤。我们从密钥自身开始,其作为0轮的轮密钥。

接着,表5.4展示的是AES加密过程中state的变换情况。第一列展示的是轮开始时state的值。第一行里,state仅仅是明文的重排矩阵。第二、第三和第四列分别展示的是该轮state在经过字节代替、行移位以及列混淆变换后的值。第五列展示的是轮密钥。你可以验证这些轮密钥和表5.3里的值相等。第一列展示的是前轮列混淆后的state和前轮的轮密钥进行逐位异或后的state值。

表5.3 AES例子的密钥扩展

密钥字	辅助函数
w0 = 0f 15 71 c9	字循环 Rotword(w3) = 7f 67 98 af = x1
w1 = 47 d9 e8 59	字代替 SubWord(x1) = d2 85 46 79 = y1
w2 = 0c b7 ad d6	轮常数 Rcon(1) = 01 00 00 00
w3 = af 7f 67 98	y1 ⊕ Rcon(1) = d3 85 46 79 = z1
w4 = w0 ⊕ z1 = dc 90 37 b0	字循环 Rotword(w7) = 81 15 a7 38 = x2
w5 = w4 ⊕ w1 = 9b 49 df e9	字代替 Subword(x4) = 0c 59 5c 07 = y2
w6 = w5 ⊕ w2 = 97 fe 72 3f	轮常数 Rcon(2) = 02 00 00 00
w7 = w6 ⊕ w3 = 38 81 15 a7	y2 ⊕ Rcon(2) = 0e 59 5c 07 = z2
w8 = w4 ⊕ z2 = d2 c9 6b b7	字循环 Rotword(w11) = ff d3 c6 e6 = x3
w9 = w8 ⊕ w5 = 49 80 b4 5e	字代替 SubWord(x2) = 16 66 b4 83 = y3
w10 = w9 ⊕ w6 = de 7e c6 61	轮常数 Rcon(3) = 04 00 00 00
w11 = w10 ⊕ w7 = e6 ff d3 c6	y3 ⊕ Rcon(3) = 12 66 b4 8e = z3
w12 = w8 ⊕ z3 = c0 af df 39	字循环 Rotword(w15) = ae 7e c0 b1 = x4
w13 = w12 ⊕ w9 = 89 2f 6b 67	字代替 SubWord(x3) = e4 f3 ba c8 = y4
w14 = w13 ⊕ w10 = 57 51 ad 06	轮常数 Rcon(4) = 08 00 00 00
w15 = w14 ⊕ w11 = b1 ae 7e c0	y4 ⊕ Rcon(4) = ec f3 ba c8 = 4
w16 = w12 ⊕ z4 = 2c 5c 65 f1	字循环 Rotword(w19) = 8c dd 50 43 = x5
w17 = w16 ⊕ w13 = a5 73 0e 96	字代替 SubWord(x4) = 64 c1 53 1a = y5
w18 = w17 ⊕ w14 = f2 22 a3 90	轮常数 Rcon(5) = 10 00 00 00
w19 = w18 ⊕ w15 = 43 8c dd 50	y5 ⊕ Rcon(5) = 74 c1 53 1a = z5
w20 = w16 ⊕ z5 = 58 9d 36 eb	字循环 Rotword(w23) = 40 46 bd 4c = x6
w21 = w20 ⊕ w17 = fd ee 38 7d	字代替 SubWord(x5) = 09 5a 7a 29 = y6
w22 = w21 ⊕ w18 = 0f cc 9b ed	轮常数 Rcon(6) = 20 00 00 00
w23 = w22 ⊕ w19 = 4c 40 46 bd	y6 ⊕ Rcon(6) = 29 5a 7a 29 = z6
w24 = w20 ⊕ z6 = 71 c7 4c c2	字循环 Rotword(w27) = a5 a9 ef cf = x7
w25 = w24 ⊕ w21 = 8c 29 74 bf	字代替 SubWord(x6) = 06 d3 bf 8a = y7
w26 = w25 ⊕ w22 = 83 e5 ef 52	轮常数 Rcon(7) = 40 00 00 00
w27 = w26 ⊕ w23 = cf a5 a9 ef	y7 ⊕ Rcon(7) = 46 d3 bf 8a = z7
w28 = w24 ⊕ z7 = 37 14 93 48	字循环 Rotword(w31) = 7d a1 4a f7 = x8
w29 = w28 ⊕ w25 = bb 3d e7 f7	字代替 SubWord(x7) = ff 32 d6 68 = y8
w30 = w29 ⊕ w26 = 38 d8 08 a5	轮常数 Rcon(8) = 80 00 00 00
w31 = w30 ⊕ w27 = f7 7d a1 4a	y8 ⊕ Rcon(8) = 7f 32 d6 68 = z8
w32 = w28 ⊕ z8 = 48 26 45 20	字循环 Rotword(w35) = be 0b 38 3c = x9
w33 = w32 ⊕ w29 = f3 1b a2 d7	字代替 SubWord(x8) = ae 2b 07 eb = y9
w34 = w33 ⊕ w30 = cb c3 aa 72	轮常数 Rcon(9) = 1B 00 00 00
w35 = w34 ⊕ w31 = 3c be 0b 3	y9 ⊕ Rcon(9) = b5 2b 07 eb = z9
w36 = w32 ⊕ z9 = fd 0d 42 cb	字循环 Rotword(w39) = 6b 41 56 f9 = x10
w37 = w36 ⊕ w33 = 0e 16 e0 1c	字代替 SubWord(x9) = 7f 83 b1 99 = y10
w38 = w37 ⊕ w34 = c5 d5 4a 6e	轮常数 Rcon(10) = 36 00 00 00
w39 = w38 ⊕ w35 = f9 6b 41 56	y10 ⊕ Rcon(10) = 49 83 b1 99 = z10
w40 = w36 ⊕ z10 = b4 8e f3 52	
w41 = w40 ⊕ w37 = ba 98 13 4e	
w42 = w41 ⊕ w38 = 7f 4d 59 20	
w43 = w42 ⊕ w39 = 86 26 18 76	

5.5.2 雪崩效应

如果密钥或明文的小变化对密文的影响也是小变化，这有可能被用来有效减少明文（或密钥）的搜索空间。而我们期待的是雪崩效应，即对明文或密钥的小变化导致密文的大变化。

用表 5.4 的例子，表 5.5 展示的是当改变明文的第 8 位时的结果。表的第 2 列显示的是两个明文在每轮结束时 state 矩阵的值。注意到仅经过一轮后，state 向量里有 20 位发生了改变。两轮后，接近一半的位发生了改变。这种规模的差别一直扩散到后面的各轮里。1 位的差别导致了密文大约一半的位置变了，这是很好的结果①。显然，如果几乎所有的位都改变了，这在逻辑上等价于几乎没有位发生改变。换一种说法，如果我们随机地选择两个明文，我们期望明文约有一半的位的位置不同，密文也约有一半位的位置不同。

当明文相同，但加密的两个密钥在第 8 位不同时，表 5.6 显示了此时的 state 矩阵的变化。也就是说，对于第二种情况，密钥是 0e1571c947d9e8590cb7add6af7f6798。同样，一轮就产生了很大的变化，后面各轮结束后的变化规模也大约是一半的位。因此，基于该例子，AES 展示了很强的雪崩效应。

在明文发生 1 位的变化或密钥发生 1 位变化的同等情况下，注意 AES 的这种雪崩效应比起 DES（参见表 3.5）来要强一些，DES 需要经过三轮后才达到大约 1 半位发生改变的效果。

表 5.4　AES 例子

轮　开　始	字节代替后	行 移 位 后	列 混 淆 后	轮　密　钥
01 89 fe 76				0f 47)c af
23 ab dc 54				15 d9 ɔ7 7f
45 cd ba 32				71 e8 ad 67
67 ef 98 10				c9 59 d6 98
0e ce f2 d9	ab 8b 89 35	ab 8b 89 35	b9 94 57 75	dc 9b 97 38
36 72 6b 2b	05 40 7f f1	40 7f f1 05	e4 8e 16 51	90 49 fe 81
34 25 17 55	18 3f f0 fc	f0 fc 18 3f	47 20 9a 3f	37 df 72 15
ae b6 4e 88	e4 4e 2f c4	c4 e4 4e 2f	c5 d6 f5 3b	b0 e9 3f a7
65 0f c0 4d	4d 76 ba e3	4d 76 ba e3	8e 22 db 12	d2 49 de e6
74 c7 e8 d0	92 c6 9b 70	c6 9b 70 92	b2 f2 dc 92	c9 80 7e ff
70 ff e8 2a	51 16 9b e5	9b e5 51 16	df 80 f7 c1	6b b4 c6 d3
75 3f ca 9c	9d 75 74 de	de 9d 75 74	2d c5 1e 52	b7 5e 61 c6
5c 6b 05 f4	4a 7f 6b bf	4a 7f 6b bf	b1 c1 0b cc	c0 89 57 b1
7b 72 a2 6d	21 40 3a 3c	40 3a 3c 21	ba f3 8b 07	af 2f 51 ae
b4 34 31 12	8d 18 c7 c9	c7 c9 8d 18	f9 1f 6a c3	df 6b ad 7e
9a 9b 7f 94	b8 14 d2 22	22 b8 14 d2	1d 19 24 5c	39 67 06 c0
71 48 5c 7d	a3 52 4a ff	a3 52 4a ff	d4 11 fe 0f	2c a5 f2 43
15 dc da a9	59 86 57 d3	86 57 d3 59	3b 44 06 73	5c 73 22 8c
26 74 c7 bd	f7 92 c6 7a	c6 7a f7 92	cb ab 62 37	65 0e a3 dd
24 7e 22 9c	36 f3 93 de	de 36 f3 93	19 b7 07 ec	f1 96 90 50
f8 b4 0c 4c	41 8d fe 29	41 8d fe 29	2a 47 c4 48	58 fd 0f 4c
67 37 24 ff	85 9a 36 16	9a 36 16 85	83 e8 18 ba	9d ee cc 40
ae a5 c1 ea	e4 06 78 87	78 87 e4 06	84 18 27 23	36 38 9b 46
e8 21 97 bc	9b fd 88 65	65 9b fd 88	eb 10 0a f3	eb 7d ed bd
72 ba cb 04	40 f4 1f f2	40 f4 1f f2	7b 05 42 4a	71 8c 83 cf
1e 06 d4 fa	72 6f 48 2d	6f 48 2d 72	1e d0 20 40	c7 29 e5 a5
b2 20 bc 65	37 b7 65 4d	65 4d 37 b7	94 83 18 52	4c 74 ef a9
00 6d e7 4e	63 3c 94 2f	2f 63 3c 94	94 c4 43 fb	c2 bf 52 ef

① 原文有误，推断该如此——译者注。

(续表)

轮 开 始	字节代替后	行移位后	列混淆后	轮 密 钥
0a 89 c1 85	67 a7 78 97	67 a7 78 97	ec 1a c0 80	37 bb 38 f7
d9 f9 c5 e5	35 99 a6 d9	99 a6 d9 35	0c 50 53 c7	14 3d c8 7d
d8 f7 f7 fb	61 68 68 0f	68 0f 61 68	3b d7 00 ef	93 e7 08 a1
56 7b 11 14	b1 21 82 fa	fa b1 21 82	b7 22 72 e0	48 f7 a5 4a
db a1 f8 77	b9 32 41 f5	b9 32 41 f5	b1 1a 44 17	48 f3 cb 3c
18 6d 8b ba	ad 3c 3d f4	3c 3d f4 ad	3d 2f ec b6	26 1b c3 be
a8 30 08 4e	c2 04 30 2f	30 2f c2 04	0a 6b 2f 42	45 a2 aa 0b
ff d5 d7 aa	16 03 0e ac	ac 16 03 0e	9f 68 f3 b1	20 d7 72 38
f9 e9 8f 2b	99 1e 73 f1	99 1e 73 f1	31 30 3a c2	fd 0e c5 f9
1b 34 2f 08	af 18 15 30	18 15 30 af	ac 71 8c c4	0d 16 c5 6b
4f c9 85 49	84 dd 97 3b	97 3b 84 dd	46 65 48 eb	42 e0 4a 41
bf bf 81 89	08 08 0c a7	a7 08 08 0c	6a 1c 31 62	cb 1c 6e 56
cc 3e ff 3b	4b b2 16 e2	4b b2 16 e2	4b 86 8a 36	b4 8e f3 52
a1 67 59 af	32 85 cb 79	85 cb 79 32	b1 cb 27 5a	ba 98 13 4e
04 85 02 aa	f2 97 77 ac	77 ac f2 97	fb f2 f2 af	7f 4d 59 20
a1 00 5f 34	32 63 cf 18	18 32 63 cf	cc 5a 5b cf	86 26 18 76
ff 08 69 64				
0b 53 34 14				
84 bf ab 8f				
4a 7c 43 b9				

表5.5 AES的雪崩效应:明文的改变

轮 数		不同的位数目
	0123456789abcdeffedcba9876543210 0023456789abcdeffedcba9876543210	1
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
1	657470750fc7ff3fc0e8e8ca4dd02a9c c4a9ad090fc7ff3fc0e8e8ca4dd02a9c	20
2	5c7bb49a6b72349b05a2317ff46d1294 fe2ae569f7ee8bb8c1f5a2bb37ef53d5	58
3	7115262448dc747e5cdac7227da9bd9c ec093dfb7c45343d689017507d485e62	59
4	f867aee8b437a5210c24c1974cffeabc 43efdb697244df808e8d9364ee0ae6f5	61
5	721eb200ba06206dcbd4bce704fa654e 7b28a5d5ed643287e006c099bb375302	68
6	0ad9d85689f9f77bc1c5f71185e5fb14 3bc2d8b6798d8ac4fe36a1d891ac181a	64
7	db18a8ffa16d30d5f88b08d777ba4eaa 9fb8b5452023c70280e5c4bb9e555a4b	67
8	f91b4fbfe934c9bf8f2f85812b084989 20264e1126b219aef7feb3f9b2d6de40	65
9	cca104a13e678500ff59025f3bafaa34 b56a0341b2290ba7dfdfbddcd8578205	61
10	ff0b844a0853bf7c6934ab4364148fb9 612b89398d0600cde116227ce72433f0	58

表 5.6　AES 的雪崩效应:密钥的改变

轮		不同的位数
	0123456789abcdeffedbca9876543210 0123456789abcdeffedcba9876543210	0
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
1	657470750fc7ff3fc0e8e8ca4dd02a9c c5a9ad090ec7ff3fc1e8c8ea4cd02a9c	22
2	5c7bb49a6b72349b05a2317ff46d1294 90905fa9563356d15f3760f3b8259985	58
3	7115262448dc747e5cdac7227da9bd9c 18aeb7aa794b3b66629448d575c7cebf	67
4	f867aee8b437a5210c24c1974cffeabc f81015f993c978a876ae017cb49e7eec	63
5	721eb200ba06206dcbd4bce704fa654e 5955c91b4e769f3cb4a94768e98d5267	81
6	0ad9d85689f9f77bc1c5f71185e5fb14 dc60a24d137662181e45b8d3726b2920	70
7	db18a8ffa16d30d5f88b08d777ba4eaa fe8343b8f88bef66cab7e977d005a03c	74
8	f91b4fbfe934c9bf8f2f85812b084989 da7dad581d1725c5b72fa0f9d9d1366a	67
9	cca104a13e678500ff59025f3bafaa34 0ccb4c66bbfd912f4b511d72996345e0	59
10	ff0b844a0853bf7c6934ab4364148fb9 fc8923ee501a7d207ab670686839996b	53

5.6　AES 的实现

5.6.1　等价的逆算法

如上所述,AES 的解密算法和加密算法不同(参见图 5.3)。尽管在加密和解密中密钥扩展的形式一样,但在解密中变换的顺序与加密中变换的顺序不同。其缺点在于对同时需要加密和解密的应用而言,需要两个不同的软件或固件模块。然而,解密算法的一个等价版本与加密算法有同样的结构。这个版本与加密算法的变换顺序相同(用逆变换取代正向变换)。为了达到这个目标,需要对密钥扩展进行改进。

两处改进使解密算法的结构与加密算法的结构一致。如图 5.3 所示,在加密过程中,其轮结构为字节代替、行移位、列混淆、轮密钥加。在标准的解密过程中,其轮结构为逆向行移位、逆向字节代替、轮密钥加、逆向列混淆。因此,在解密轮中的前两个阶段应交换,后两个阶段也需要交换。

交换逆向行移位和逆向字节代替

逆向移行影响在 state 中字节的顺序,但并不更改字节的内容,同时也不依赖字节的内容来进行它的变换。逆向字节代替影响 state 中字节的内容,但不更改字节的顺序同时也不依赖字节的顺序来进行它的变换。因此,这两个操作可以交换。如对一个给定的状态 S_i:

$$\text{逆向移行}[\text{逆向字节代替}(S_i)] = \text{逆向字节代替}[\text{逆向移行}(S_i)]$$

交换轮密钥加和逆向列混淆

轮密钥加和逆向列混淆并不更改 state 中字节的顺序。如果我们将密钥视为字的序列,那么

轮密钥加和逆向列混淆每次都对 state 的一列进行操作。这两个操作对列输入是线性的。即对给定的状态 S_i 和给定的轮密钥 w_j:

$$\text{逆向列混淆}(S_i \oplus w_j) = [\text{逆向列混淆}(S_i)] \oplus [\text{逆向列混淆}(w_j)]$$

为了说明这一点,假定状态 S_i 的第一列是序列 (y_0, y_1, y_2, y_3),轮密钥 w_j 的第一列为 (k_0, k_1, k_2, k_3)。然后我们有

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} y_0 \oplus k_0 \\ y_1 \oplus k_1 \\ y_2 \oplus k_2 \\ y_3 \oplus k_3 \end{bmatrix} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} \oplus \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} k_0 \\ k_1 \\ k_2 \\ k_3 \end{bmatrix}$$

我们来论证第一列,则有

$$\begin{aligned} &[\{0E\} \cdot (y_0 \oplus k_0)] \oplus [\{0B\} \cdot (y_1 \oplus k_1)] \oplus [\{0D\} \cdot (y_2 \oplus k_2)] \oplus [\{09\} \cdot (y_3 \oplus k_3)] \\ &= [\{0E\} \cdot y_0] \oplus [\{0B\} \cdot y_1] \oplus [\{0D\} \cdot y_2] \oplus [\{09\} \cdot y_3] \oplus \\ &\quad [\{0E\} \cdot k_0] \oplus [\{0B\} \cdot k_1] \oplus [\{0D\} \cdot k_2] \oplus [\{09\} \cdot k_3] \end{aligned}$$

我们可以看到这个方程是正确的。因此,假使先对轮密钥应用逆向列混淆,可以交换轮密钥加和逆向列混淆。注意无须对第一次或最后一次轮密钥加变换的输入进行逆向列混淆操作(第 10 轮),因为这两个轮密钥加变换不能与逆向列混淆交换来产生等价的解密算法。

图 5.10 描述了等价的解密算法。

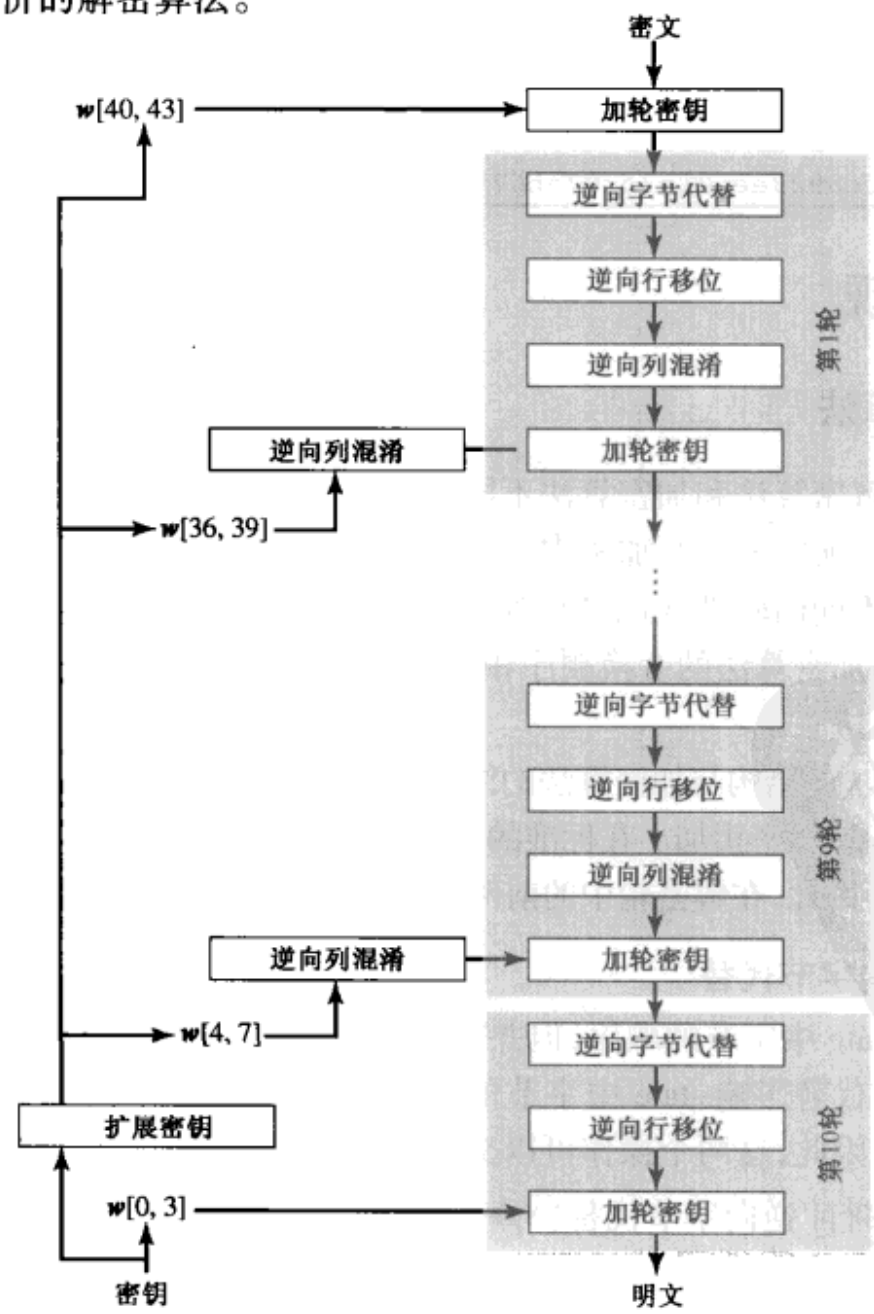


图 5.10 等价逆密码

5.6.2 实现方面

Rijndael 提案[DAEM99]针对如何用像智能卡等 8 位处理器、像个人计算机等 32 位处理器有效实现 Rijndael 算法提出了一些建议。

8 位处理器

AES 能在 8 位处理器上非常有效地实现。轮密钥加是按字节异或操作。行移位是简单的移字节操作。字节代替是在字节级别上进行操作的，而且只要求一个 256 字节的表。

列混淆变换要求是在域 GF(2^8)上的乘法，即所有的操作都是基于字节的。列混淆仅要求乘以{02}和{03}，正如我们所看到的一样，这涉及简单的移位、条件异或和异或。若不用移位或条件异或操作，算法的执行将更有效。方程组(5.4)是说明在单列上进行列混淆变换的等式。利用性质$\{03\}\cdot x = (\{02\}\cdot x)\oplus x$，我们能用如下的方式重写方程组(5.4)：

$$\begin{aligned}
\text{Tmp} &= s_{0,j}\oplus s_{1,j}\oplus s_{2,j}\oplus s_{3,j}\\
s'_{0,j} &= s_{0,j}\oplus \text{Tmp}\oplus [2\cdot(s_{0,j}\oplus s_{1,j})]\\
s'_{1,j} &= s_{1,j}\oplus \text{Tmp}\oplus [2\cdot(s_{1,j}\oplus s_{2,j})]\\
s'_{2,j} &= s_{2,j}\oplus \text{Tmp}\oplus [2\cdot(s_{2,j}\oplus s_{3,j})]\\
s'_{3,j} &= s_{3,j}\oplus \text{Tmp}\oplus [2\cdot(s_{3,j}\oplus s_{0,j})]
\end{aligned} \tag{5.9}$$

方程组(5.9)能通过展开和约去项的方式加以验证。

乘以{02}包含一次移位和一次条件异或操作。这种实现方式可能易于受到 3.4 节中所描述的计时攻击。为了防止这种攻击并提高执行的效率，可以使用查表的方式取代乘法操作，其代价是需要花费一定的存储空间。我们定义一个包含 256 字节的表 $X2$，使得 $X2[i] = \{02\}\cdot i$。方程组(5.9)被描述为

$$\begin{aligned}
\text{Tmp} &= s_{0,j}\oplus s_{1,j}\oplus s_{2,j}\oplus s_{3,j}\\
s'_{0,j} &= s_{0,j}\oplus \text{Tmp}\oplus X2[s_{0,j}\oplus s_{1,j}]\\
s'_{1,c} &= s_{1,j}\oplus \text{Tmp}\oplus X2[s_{1,j}\oplus s_{2,j}]\\
s'_{2,c} &= s_{2,j}\oplus \text{Tmp}\oplus X2[s_{2,j}\oplus s_{3,j}]\\
s'_{3,j} &= s_{3,j}\oplus \text{Tmp}\oplus X2[s_{3,j}\oplus s_{0,j}]
\end{aligned}$$

32 位处理器

上一节所描述的实现方式仅是基于在 8 位处理器上的操作。对 32 位处理器，若将操作定义在 32 位的字上，就能执行更有效的操作。为了论述这一点，我们首先利用代数形式定义一轮的 4 个变换。假如我们利用 $a_{i,j}$ 表示 state 矩阵，利用 $k_{i,j}$ 表示轮密钥矩阵，那么能用如下的方式来描述这些变换：

字节代替	$b_{i,j} = \text{S}[a_{i,j}]$
行移位	$\begin{bmatrix} c_{0,j}\\ c_{1,j}\\ c_{2,j}\\ c_{3,j}\end{bmatrix} = \begin{bmatrix} b_{0,j}\\ b_{1,j-1}\\ b_{2,j-2}\\ b_{3,j-3}\end{bmatrix}$
列混淆	$\begin{bmatrix} d_{0,j}\\ d_{1,j}\\ d_{2,j}\\ d_{3,j}\end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01\\ 01 & 02 & 03 & 01\\ 01 & 01 & 02 & 03\\ 03 & 01 & 01 & 02\end{bmatrix} \begin{bmatrix} c_{0,j}\\ c_{1,j}\\ c_{2,j}\\ c_{3,j}\end{bmatrix}$

轮密钥加	$\begin{bmatrix} e_{0,j} \\ e_{1,j} \\ e_{2,j} \\ e_{3,j} \end{bmatrix} = \begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$

在行移位等式中,列下标需要模4。我们能把所有的这些表达式表示成一个等式:

$$\begin{bmatrix} e_{0,j} \\ e_{1,j} \\ e_{2,j} \\ e_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} \boldsymbol{S}[a_{0,j}] \\ \boldsymbol{S}[a_{1,j-1}] \\ \boldsymbol{S}[a_{2,j-2}] \\ \boldsymbol{S}[a_{3,j-3}] \end{bmatrix} \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$$

$$= \left(\begin{bmatrix} 02 \\ 01 \\ 01 \\ 03 \end{bmatrix} \bullet \boldsymbol{S}[a_{0,j}] \right) \oplus \left(\begin{bmatrix} 03 \\ 02 \\ 01 \\ 01 \end{bmatrix} \bullet \boldsymbol{S}[a_{1,j-1}] \right) \oplus \left(\begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} \bullet \boldsymbol{S}[a_{2,j-2}] \right)$$

$$\oplus \left(\begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} \bullet \boldsymbol{S}[a_{3,j-3}] \right) \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$$

在第二个方程中,用向量的线性组合来表示矩阵的乘法。我们定义4个256字(1024字节)的表如下:

$T_0[x] = \left(\begin{bmatrix} 02 \\ 01 \\ 01 \\ 03 \end{bmatrix} \bullet \boldsymbol{S}[x] \right)$	$T_1[x] = \left(\begin{bmatrix} 03 \\ 02 \\ 01 \\ 01 \end{bmatrix} \bullet \boldsymbol{S}[x] \right)$	$T_2[x] = \left(\begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} \bullet \boldsymbol{S}[x] \right)$	$T_3[x] = \left(\begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} \bullet \boldsymbol{S}[x] \right)$

因此,每个表接受一个字节的输入,同时输出一个列向量(32位字),这可以视为字节为输入的S盒的一个函数。这些表能被事先计算。

用如下的方式定义轮函数对一列的操作:

$$\begin{bmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{bmatrix} = T_0[s_{0,j}] \oplus T_1[s_{1,j-1}] \oplus T_2[s_{2,j-2}] \oplus T_3[s_{3,j-3}] \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$$

结果,基于上述等式的实现仅需要4张表,加上每轮每列的4次异或以及存储这些表所需的4 KB存储空间。Rijndael的开发者们认为这种紧凑、有效的实现方式可能是选择Rijndael作为高级加密标准的最重要因素之一。

5.7 推荐读物和网站

到目前为止可获取的AES最完整版本[DAEM02]是由AES的开发者提供的。作者也在[DAEM01]中给出了简单的描述和设计原理。[LAND04]就AES及其密码分析提供了严谨的数学分析。

另一个完成的 AES 运算的例子在本书的网站可以得到，作者为新西兰 Massey 大学的老师。

DAEM01 Daemen, J., and Rijmen, V. "Rijndael: The Advanced Encryption Standard." *Dr. Dobb's Journal*, March 2001.

$$
\begin{array}{r|l}
 & x^2+1 \\
\hline
x^6+x^4+x+1 & x^8+x^4+x^3+x+1 \\
 & x^8+x^6+x^3+x^2 \\
\hline
 & x^6+x^4+x^2+x+1 \\
 & x^6+x^4+x+1 \\
\hline
 & x^2
\end{array}
$$

$$
\begin{array}{r|l}
 & x^4+x^2 \\
\hline
x^2 & x^6+x^4+x+1 \\
 & x^6 \\
\hline
 & x^4+x+1 \\
 & x^4 \\
\hline
 & x+1
\end{array}
$$

$$
\begin{array}{r|l}
 & x+1 \\
\hline
x+1 & x^2 \\
 & x^2+x \\
\hline
 & x \\
 & x+1 \\
\hline
 & 1
\end{array}
$$

5.1 5.2 5.3

5.2 (a) 在 GF(2^8)上{01}的逆是什么?

(b) 验证{01}在 S 盒中的项。

5.3 当 128 位的密钥是全 0 时,给出密钥扩展数组的前 8 个字。

5.4 若明文是{00010203040506070809 0A0B0C0D0E0F},密钥是{0101010101010101010 10101010101},

(a) 用 4 ×4 的矩阵来描述 state 的最初内容。

(b) 给出初始化轮密钥加后 state 的值。

(c) 给出字节代替后 state 的值。

(d) 给出行移位后 state 的值。

(e) 给出列混淆后 state 的值。

5.5 验证式(5.11),即验证 $x^i \bmod (x^4+1) = x^{i \bmod 4}$。

5.6 比较 AES 和 DES。对如下所述 DES 中的元素,指出 AES 与之相对应的元素或解释 AES 中为什么不需要该元素:

(a) f 函数的输入与子密钥相异或。

(b) f 函数的输出与分组左边的部分相异或。

(c) f 函数。

(d) 置换 P。

(e) 交换一个分组的两半部分。

5.7 在关于实现的小节中,描述了利用表的方式来防止计时攻击。请提出另一种可防止计时攻击的技术。

5.8 在关于实现小节中,提出了仅用一个代数方程描述加密算法的一个典型轮的 4 个阶段。请给出与第 10 轮等价的方程。

5.9 计算由输入字节序列"67 89 AB CD"经过列混淆后的输出结果,并对输出结果使用逆向列混淆变换来验证计算结果。将输入的第一个字节从'67'改为'77',对新的输入重新执行列混淆变换,并判断输出有多少位发生了变化。**注意**:你可以通过手工计算或者编程来执行上述运算。如果是编程,请独立编写所有代码,不要使用任何库或者公开的源代码。

5.10 使用密钥 1010 0111 0011 1011 加密表示为 ASCII 码的明文"ok",即 0110 1111 0110 1011。S-AES 的设计者得出密文是 0000 0111 0011 1000,你的呢?

5.11 如下矩阵是 GF(2^4)上的矩阵,说明该矩阵是 S-AES 的列混淆使用的矩阵的逆。

$$\begin{bmatrix} x^3+1 & x \\ x & x^3+1 \end{bmatrix}$$

5.12 对于 S-AES 算法用密钥 1010 0111 0011 1011 解密密文 0000 0111 0011 1000。得出的明文应该与习题 5.10 所给明文相同。注意 S 盒的逆运算可以通过逆向查表来实现,而列混淆的逆运算可以通过习题 5.11 给出的矩阵来实现。

5.13 证明式(5.9)和式(5.4)等价。

编程题

5.14 开发使用 S-AES 实现加密和解密的软件。测试数据:使用密钥 1010 0111 0011 1011 加密二进制明文 0110 1111 0110 1011,得出二进制密文 0000 0111 0011 1000。解密过程同上。

5.15 实现对于 1 轮 S-AES 的差分攻击。

附录 5A 系数在 GF(2^8)中的多项式

在 4.5 节中,我们讨论了系数定义在 Z_p 上的多项式算术和模 $M(x)$ 的多项式,其中 $M(x)$ 的次数为 n。在这里,多项式系数的加法和乘法都是定义在域 Z_p 上的,即加法和乘法都要模 p。

AES 文档描述的多项式算术中的多项式的系数在 GF(2^8)上,且次数不大于 3。其中使用了如下的规则:

(1) 加法操作就是两个多项式的系数在 GF(2^8)上相加。正如 4.5 节中所指出的,如果我们把 GF(2^8)中的元素视为 8 位的串,那么加法就等价于异或操作。于是有

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0 \tag{5.10}$$

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0 \tag{5.11}$$

那么

$$a(x) + b(x) = (a_3 \oplus b_3)x^3 + (a_2 \oplus b_2)x^2 + (a_1 \oplus b_1)x + (a_0 \oplus b_0)$$

(2) 乘法操作和一般多项式乘法差不多,但有两个限制:

(a) 系数是在 GF(2^8)上相乘。

(b) 结果多项式需要模多项式($x^4 + 1$)。

应对所讨论的多项式有清醒的认识。回顾 4.6 节,GF(2^8)中的每个元素可以表示为系数为{0}或{1},次数不大于 7 的多项式。多项式乘积的结果需要模一个次数为 8 的多项式。相应地,GF(2^8)中的每个元素都能被视为一个 8 位的字节,该字节中每一位都与相对应多项式的系数对应。对本节所定义的集合,我们定义了一个多项式环,该环中的每个多项式的系数在 GF(2^8)中,次数不大于 3。多项式乘积的结果需要模一个次数为 4 的多项式。相应地,这个环中的每个元素均可被视为一个四字节的字,该字中的每一个字节是 GF(2^8)内的元素,其值都与该元素所对应多项式的 8 位系数相对应。

定义 $a(x)$和 $b(x)$的模乘积为 $a(x) \otimes b(x)$。为计算 $d(x) = a(x) \otimes b(x)$,首先执行没有模操作的乘法并将具有相同次数的项相合并。我们把这个过程称为 $c(x) = a(x) \times b(x)$。然后

$$c(x) = c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0 \tag{5.12}$$

其中

$$c_0 = a_0 \cdot b_0$$
$$c_1 = (a_1 \cdot b_0) \oplus (a_0 \cdot b_1)$$
$$c_2 = (a_2 \cdot b_0) \oplus (a_1 \cdot b_1) \oplus (a_0 \cdot b_2)$$
$$c_3 = (a_3 \cdot b_0) \oplus (a_2 \cdot b_1) \oplus (a_1 \cdot b_2) \oplus (a_0 \cdot b_3)$$
$$c_4 = (a_3 \cdot b_1) \oplus (a_2 \cdot b_2) \oplus (a_1 \cdot b_3)$$
$$c_5 = (a_3 \cdot b_2) \oplus (a_2 \cdot b_3)$$
$$c_6 = a_3 \cdot b_3$$

最后一步是执行模操作:

$$d(x) = c(x) \bmod (x^4 + 1)$$

即 $d(x)$必须满足方程

$$c(x) = [(x^4 + 1) \times q(x)] \oplus d(x)$$

以使 $d(x)$的次数不大于 3。

在多项式环上进行乘法的一种实际技巧基于如下观察:

$$x^i \bmod (x^4 + 1) = x^{i \bmod 4} \tag{5.13}$$

如果联合式(5.12)和式(5.13),可得到

$$\begin{aligned} d(x) &= c(x) \bmod (x^4 + 1) \\ &= [c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0] \bmod (x^4 + 1) \\ &= c_3x^3 + (c_2 \oplus c_6)x^2 + (c_1 \oplus c_5)x + (c_0 \oplus c_4) \end{aligned}$$

将系数 c_i 进行扩展,将得到关于 $d(x)$ 系数的如下方程:

$$\begin{aligned}
d_0 &= (a_0 \bullet b_0) \oplus (a_3 \bullet b_1) \oplus (a_2 \bullet b_2) \oplus (a_1 \bullet b_3) \\
d_1 &= (a_1 \bullet b_0) \oplus (a_0 \bullet b_1) \oplus (a_3 \bullet b_2) \oplus (a_2 \bullet b_3) \\
d_2 &= (a_2 \bullet b_0) \oplus (a_1 \bullet b_1) \oplus (a_0 \bullet b_2) \oplus (a_3 \bullet b_3) \\
d_3 &= (a_3 \bullet b_0) \oplus (a_2 \bullet b_1) \oplus (a_1 \bullet b_2) \oplus (a_0 \bullet b_3)
\end{aligned}$$

其矩阵表达方式为

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \tag{5.14}$$

附录 5A.1 列混淆变换

在讨论列混淆变换时,说明了有两种等价的方式来定义这个变换。第一个是式(5.3)所示的矩阵乘法,即

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

第二种方式是将 state 中的每列视为一个系数在 GF(2^8)中的 4 次多项式。每列乘上固定的多项式 $a(x)$,然后模(x^4+1),其中 $a(x)$ 是

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

根据式(5.10),有 $a_3 = \{03\}$;$a_2 = \{01\}$;$a_1 = \{01\}$;$a_0 = \{02\}$。对 state 中的第 j 列,有列多项式 $\text{col}_j(x) = s_{3,j}x^3 + s_{2,j}x^2 + s_{1,j}x + s_{0,j}$。代入式(5.14),可将 $d(x) = a(x) \times \text{col}_j(x)$ 表示为

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}$$

这与式(5.3)等价。

附录 5A.2 乘以 x

考虑环中一个多项式乘上 x 的情况:$c(x) = x \otimes b(x)$。有

$$\begin{aligned}
c(x) = x \otimes b(x) &= [x \times (b_3x^3 + b_2x^2 + b_1x + b_0] \bmod (x^4 + 1) \\
&= (b_3x^4 + b_2x^3 + b_1x^2 + b_0x) \bmod (x^4 + 1) \\
&= b_2x^3 + b_1x^2 + b_0x + b_3
\end{aligned}$$

因此,多项式乘上 x 就相当于将这个多项式所对应的由 4 字节所组成的字循环左移一个字节。如果把这个多项式作为一个 4 字节的列向量,那么就有

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 00 & 00 & 00 & 01 \\ 01 & 00 & 00 & 00 \\ 00 & 01 & 00 & 00 \\ 00 & 00 & 01 & 00 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

附录 5B　简化 AES

简化 AES(S-AES)是 Santa Clara 大学的 Edward Schaefer 教授以及他的几个学生开发出来的[MUSA03]。S-AES 与其说是一个加密算法,不如说是一个教学示例。S-AES 与 AES 在特征和结构上非常相似,只是参数规模更小。读者会发现按照本附录手工计算对算法的理解非常有用。而对于 S-AES 的充分理解会使得学生对于 AES 的结构和工作原理有更深入的理解。

附录 5B.1　整体结构

图 5.11 给出了 S-AES 的整体结构。加密算法以 16 位分组的明文作为输入,使用 16 位的密钥产生 16 位分组的密文。S-AES 解密算法以 16 位分组的密文作为输入,使用同样的 16 位密钥产生 16 位分组的原始明文。

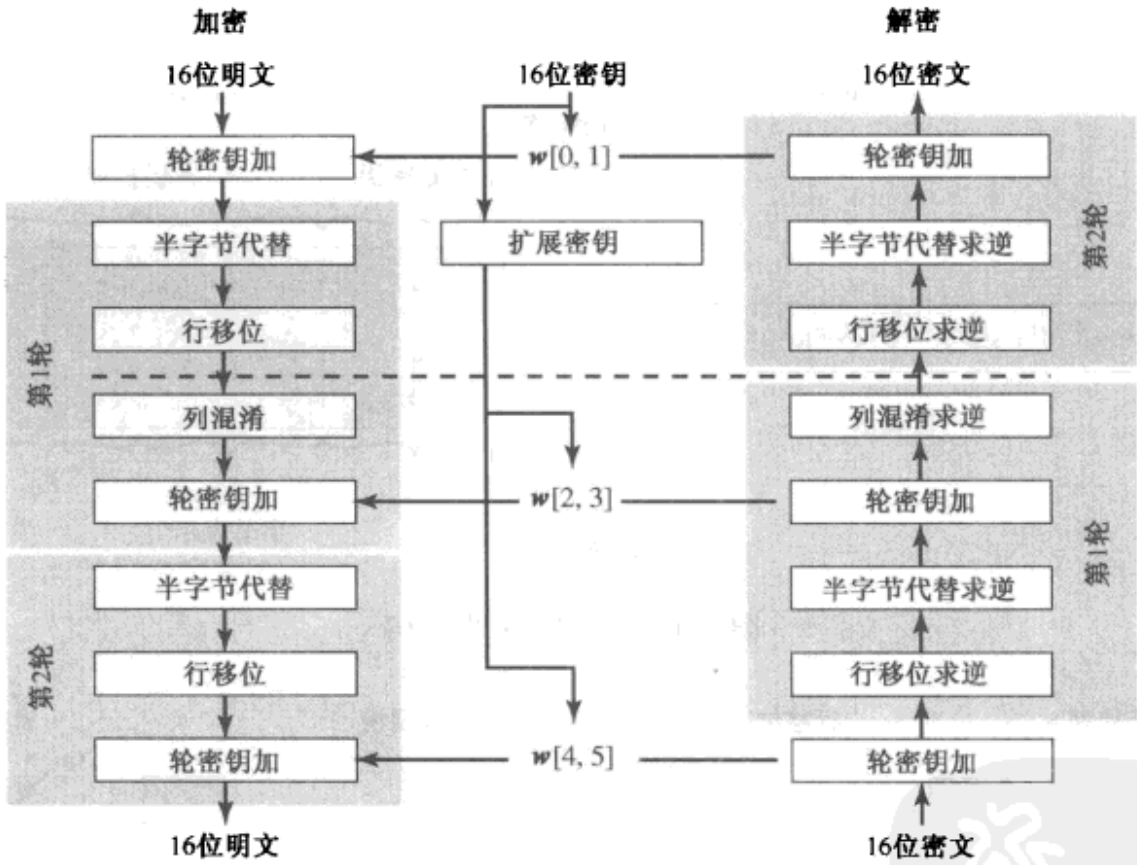


图 5.11　S-AES 加密和解密

接下来我们将依次解释加密算法用到的 4 个不同的函数或变换:密钥加(A_K),半字节代替(NS),行移位(SR),列混淆(MC)。

可用函数的复合①把加密过程简单地表示为

$$A_{K_2} \circ \mathrm{SR} \circ \mathrm{NS} \circ A_{K_1} \circ \mathrm{MC} \circ \mathrm{SR} \circ \mathrm{NS} \circ A_{K_0}$$

即首先执行 A_{K_0}。

① 定义:如果 f 和 g 是两个函数,而 $y = F(x) = g[f(x)]$,则称函数 F 是 f 和 g 的复合函数,并记为 $F = g \circ f$。

加密算法包括三轮。第一轮仅进行密钥加;第二轮是包括 4 个函数变换的完整轮;第三轮仅包括 3 个函数。每一轮都包括使用 16 位密钥的密钥加变换。初始的 16 位密钥被扩展成 48 位子密钥,因此每一轮分别使用不同的 16 位子密钥。

每个函数都进行 16 位运算,可将其视为 2 × 2 的半字节 state 矩阵,每半字节对应 4 位。state 矩阵的初始值为 16 位明文;在加密过程中每个函数变换依次改变 state 矩阵,最后一个函数输出 16 位密文。如图 5.12(a)所示,半字节在矩阵中按列顺序排列。例如加密过程中 16 位明文的前 8 位存在矩阵的第一列,后 8 位存在矩阵的第二列。16 位密钥的存储方法也是如此,但将密钥视为 2 字节比视为 4 个半字节有时更方便[参见图 5.12(b)]。48 位的扩展密钥成为 3 轮的轮密钥,其按位分别是 $K_0 = k_0 \cdots k_{15}$;$K_1 = k_{16} \cdots k_{31}$;$K_2 = k_{32} \cdots k_{47}$。

图 5.13 详细展示了 S-AES 的完整一轮的基本元素。

如图 5.11 所示,解密过程本质上是加密的逆过程:

$$A_{K_0} \circ \text{INS} \circ \text{ISR} \circ \text{IMC} \circ A_{K_1} \circ \text{INS} \circ \text{ISR} \circ A_{K_2}$$

其中的三个函数有对应的逆函数变换:半字节代替求逆(INS),行移位求逆(ISR),列混淆求逆(IMC)。

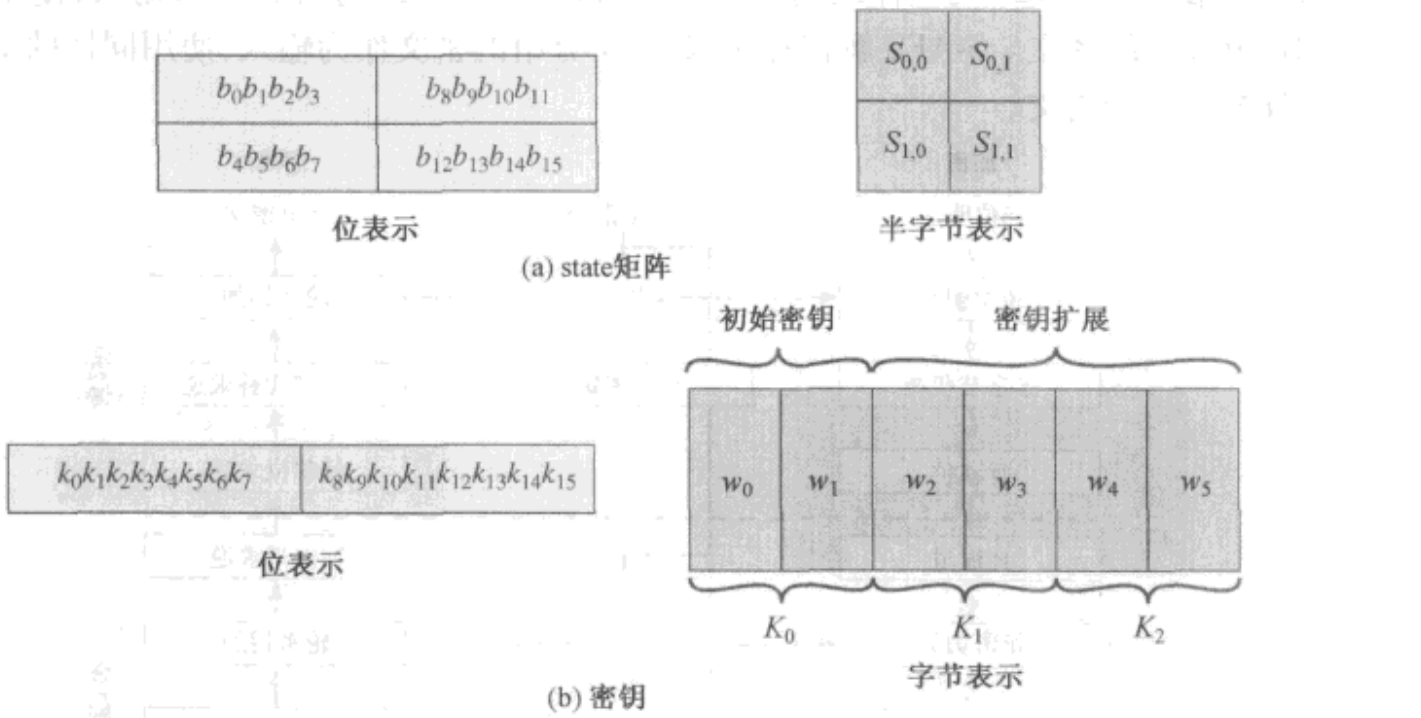


图 5.12　S-AES 数据结构

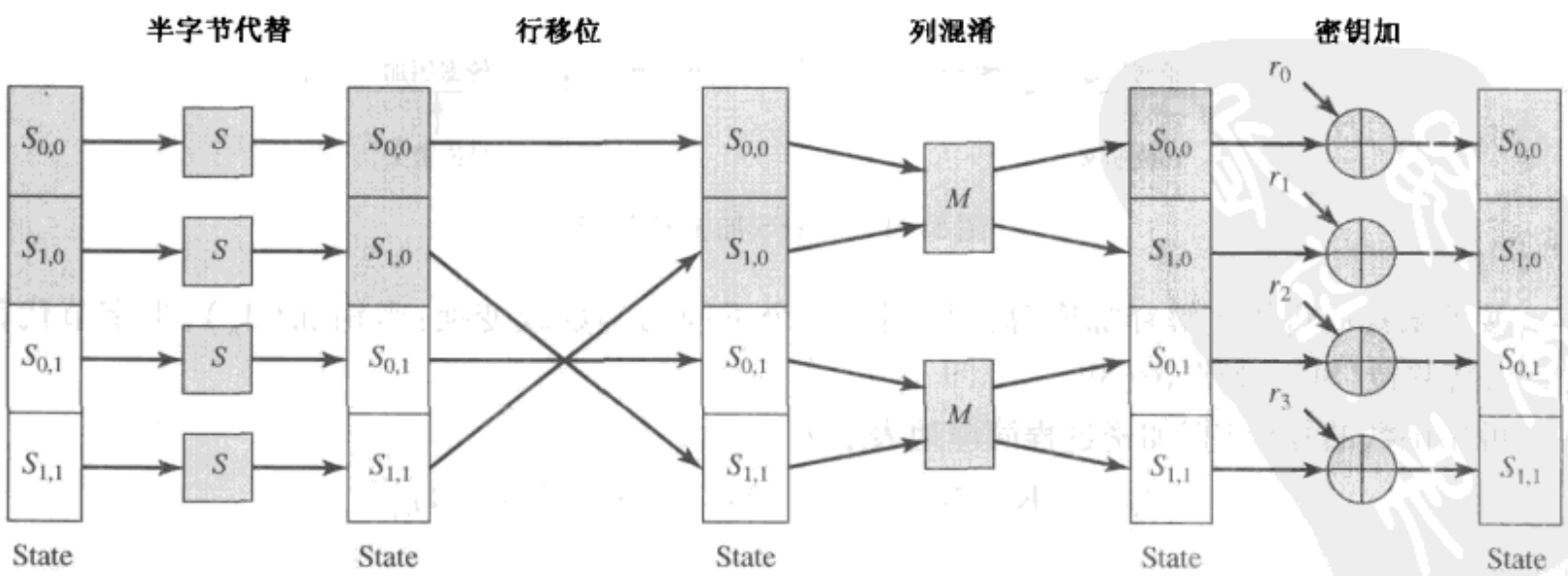


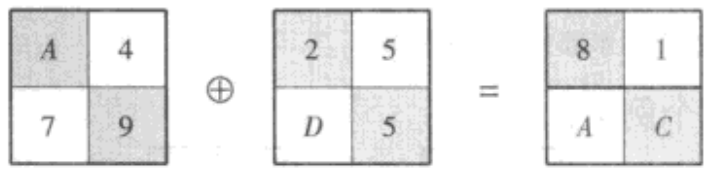
图 5.13　S-AES 加密轮函数

附录 5B.2　S-AES 的加密和解密

我们现在具体看加密算法的每个部分。

密钥加

密钥加函数是对于 16 位 state 矩阵和 16 位轮密钥的按位 XOR。图 5.14 按列操作进行描述，同样地，我们可以把其视为按半字节操作或者按位操作。例如



密钥加函数的逆与密钥加函数完全相同，因为 XOR 操作是其自身的逆。

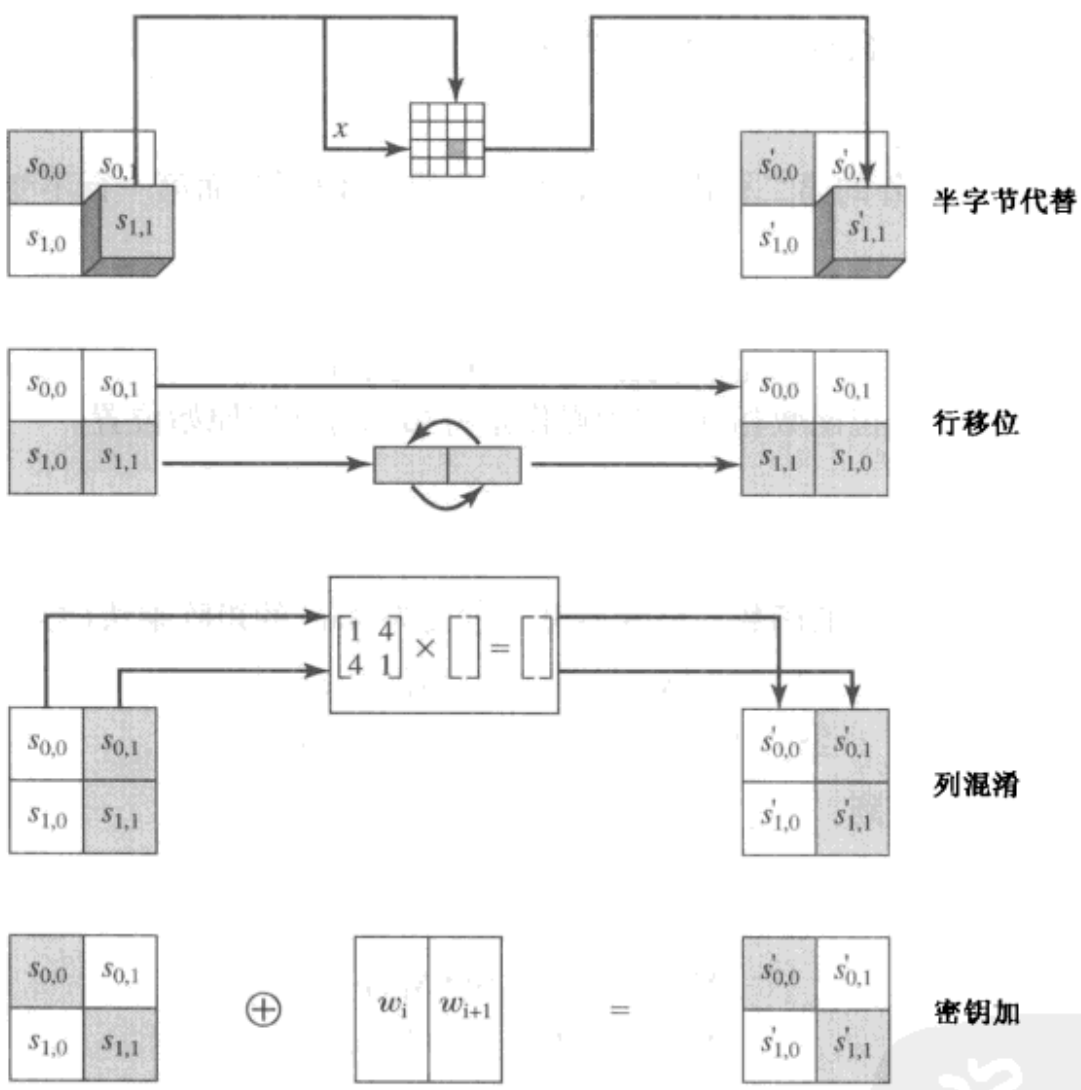


图 5.14　S-AES 的各种函数变换

半字节代替

半字节代替函数实际上是简单的查表(参见图 5.14)。S-AES 定义的 4 ×4 半字节矩阵称为 S 盒[参见表5.7(a)]，包含所有4 位数的一个置换。state 矩阵中每个单独的半字节由如下方法映射为新的半字节：半字节的左边 2 位作为行值，右边 2 位作为列值。行值和列值作为索引在 S 盒中选择唯一的 4 位输出值。例如十六进制 *A* 对应S 盒的第二行、第二列，即 0。于是值 *A* 就被映射为 0。

表 5.7 S-AES S 盒

		j			
		00	01	10	11
i	00	9	4	A	B
	01	D	1	8	5
	10	6	2	0	3
	11	C	E	F	7

(a)S 盒

		j			
		00	01	10	11
i	00	A	5	9	B
	01	1	7	8	F
	10	6	0	2	3
	11	C	4	D	E

(b)逆 S 盒

注:盒中阴影部分为十六进制数;无阴影部分为二进制数。

下面是半字节代替的一个例子:

8	1
A	C

→

6	4
0	C

半字节代替求逆函数使用表 5.7(b)中的逆 S 盒。例如在半字节代替求逆中输入 0 产生输出 A,而在半字节代替中使用 S 盒输入 A 产生输出 0。

行移位

行移位函数对 state 矩阵的第二行进行一个半字节的循环移位;而第一行不变(参见图 5.14)。下面是行移位的一个例子:

6	4
0	C

→

6	4
C	0

行移位求逆函数与行移位函数相同,因为操作是将第二行移回原始位置。

列混淆

列混淆函数对每个列单独操作。一列中的每个半字节通过该列中的函数被映射为一个新值,该新值是那一列中两个半字节的函数。该变换可以定义为 state 的矩阵乘法(参见图 5.14):

$$\begin{bmatrix}1 & 4\\ 4 & 1\end{bmatrix}\begin{bmatrix}s_{0,0} & s_{0,1}\\ s_{1,0} & s_{1,1}\end{bmatrix}=\begin{bmatrix}s'_{0,0} & s'_{0,1}\\ s'_{1,0} & s'_{1,1}\end{bmatrix}$$

执行矩阵乘法后我们得到

$$\begin{aligned}S'_{0,0} &= S_{0,0} \oplus (4 \bullet S_{1,0})\\ S'_{1,0} &= (4 \bullet S_{0,0}) \oplus S_{1,0}\\ S'_{0,1} &= S_{0,1} \oplus (4 \bullet S_{1,1})\\ S'_{1,1} &= (4 \bullet S_{0,1}) \oplus S_{1,1}\end{aligned}$$

这里的运算是在 $GF(2^4)$ 上,符号 · 表示在 $GF(2^4)$ 上的乘法。附录 I 提供了加法和乘法表。下面是其中的一个例子:

$$\begin{bmatrix}1 & 4\\ 4 & 1\end{bmatrix}\begin{bmatrix}6 & 4\\ C & 0\end{bmatrix}=\begin{bmatrix}3 & 4\\ 7 & 3\end{bmatrix}$$

列混淆求逆函数定义如下:

$$\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\begin{bmatrix}s_{0,0} & s_{0,1}\\ s_{1,0} & s_{1,1}\end{bmatrix}=\begin{bmatrix}s'_{0,0} & s'_{0,1}\\ s'_{1,0} & s'_{1,1}\end{bmatrix}$$

可用如下方式证明我们确实定义了列混淆求逆函数:

$$\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\begin{bmatrix}1 & 4\\ 4 & 1\end{bmatrix}\begin{bmatrix}s_{0,0} & s_{0,1}\\ s_{1,0} & s_{1,1}\end{bmatrix}=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}s_{0,0} & s_{0,1}\\ s_{1,0} & s_{1,1}\end{bmatrix}=\begin{bmatrix}s_{0,0} & s_{0,1}\\ s_{1,0} & s_{1,1}\end{bmatrix}$$

前述矩阵乘法用到了 GF(2^4)上的如下结果：$9 + (2 \cdot 4) = 9 + 8 = 1$ 和 $(9 \cdot 4) + 2 = 2 + 2 = 0$。这些操作可以由附录 I 的运算表或者多项式运算来验证。

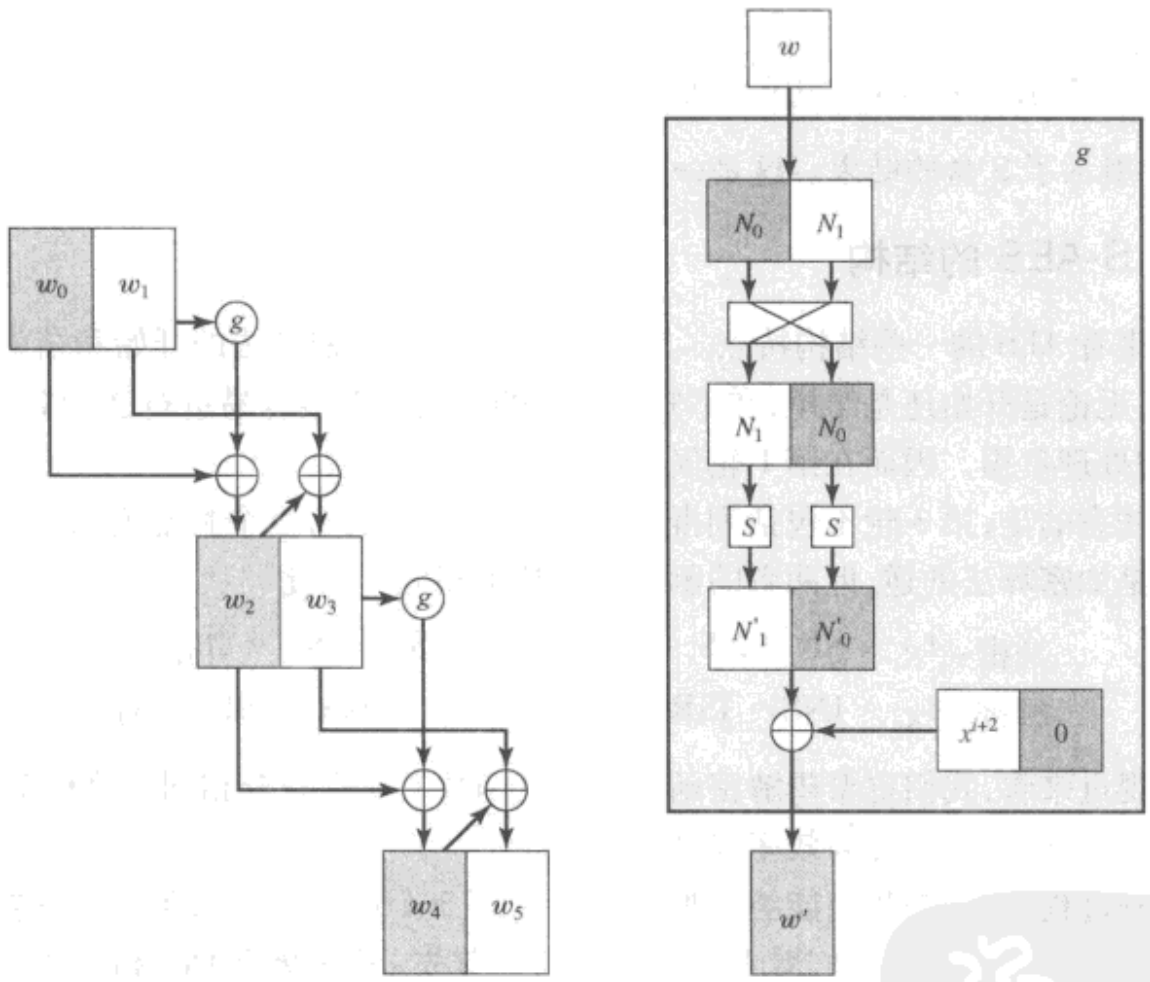
列混淆函数很难用图去描述。因此我们在附录 I 中给出了列混淆函数的其他描述。

密钥扩展

对于密钥扩展，16 位的初始密钥分为两个 8 位的字。如图 5.15 所示，初始的两个字通过计算得出 4 个新的字，扩展为 6 个字。算法如下：

$$w_2 = w_0 \oplus g(w_1) = w_0 \oplus \text{Rcon}(1) \oplus \text{SubNib}(\text{RotNib}(w_1))$$
$$w_3 = w_2 \oplus w_1$$
$$w_4 = w_2 \oplus g(w_3) = w_2 \oplus \text{Rcon}(2) \oplus \text{SubNib}(\text{RotNib}(w_3))$$
$$w_5 = w_4 \oplus w_3$$

Rcon 是轮常数，定义如下：$\text{RC}[i] = x^{i+2}$，因此 $\text{RC}[1] = x^3 = 1000$，$\text{RC}[2] = x^4 \bmod (x^4 + x + 1) = x + 1 = 0011$。RC[$i$]构成一个字节的左半字节，而右半字节全置 0。因此 Rcon(1) = 10000000，Rcon(2) = 00110000。



(a) 整个算法　　(b) 函数g

图 5.15　S-AES 密钥扩展

例如，假设密钥为 2D55 = 0010 1101 0101 0101 = $w_0 w_1$，则

$$w_2 = 00101101 \oplus 10000000 \oplus \text{SubNib}(01010101)$$
$$= 00101101 \oplus 10000000 \oplus 00010001 = 10111100$$
$$w_3 = 10111100 \oplus 01010101 = 11101001$$
$$w_4 = 10111100 \oplus 00110000 \oplus \text{SubNib}(10011110)$$
$$= 10111100 \oplus 00110000 \oplus 00101111 = 10100011$$
$$w_5 = 10100011 \oplus 11101001 = 01001010$$

附录 5B.3 S 盒

S 盒的构造如下:

(1) 以升序逐行使用半字节值初始化 S 盒。第一行包括十六进制值 0,1,2,3;第二行包括 4,5,6,7;以此类推。因此第 i 行、第 j 列的半字节的值为 $4i+j$。

(2) 将每个半字节视为有限域 GF(2^4)上模 x^4+x+1 的元素。每个半字节 $a_0a_1a_2a_3$ 表示一个三次多项式。

(3) 将 S 盒中的每一字节映射到其在有限域 GF(2^4)上模 x^4+x+1 的乘法逆;0 映射到自身。

(4) 将 S 盒中每个字节标记为 4 位(b_0,b_1,b_2,b_3)的组合。对于 S 盒中每个字节的每一位做变换,标准 AES 用如下的矩阵形式描述该变换:

$$\begin{bmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

(5) 符号(′)表示变量是由右边的值更新得出的。注意加法和乘法都是模 2 运算。

表 5.7(a)展示了 S 盒的结果。这是一个非线性可逆矩阵。逆 S 盒如表 5.7(b)所示。

附录 5B.4 S-AES 的结构

现在我们来看 AES 的一些结构特点。首先,注意加密和解密算法的开始和结束都是密钥加函数。其余函数,无论是开始还是结尾,在不知道密钥的情况下都很容易进行逆运算,所以不会增加任何安全,徒增处理负担。因而在第 1 轮仅有密钥加函数。

第二点要注意的是,第 3 轮不包括列混淆函数。其原因跟第三个特点有关,如图 5.11 所示,尽管解密算法是加密算法的逆,但两者的函数次序却不相同。因此

$$\text{加密}: A_{K_2} \circ \text{SR} \circ \text{NS} \circ A_{K_1} \circ \text{MC} \circ \text{SR} \circ \text{NS} \circ A_{K_0}$$

$$\text{解密}: A_{K_0} \circ \text{INS} \circ \text{ISR} \circ \text{IMC} \circ A_{K_1} \circ \text{INS} \circ \text{ISR} \circ A_{K_2}$$

从实现的观点来看,我们更希望解密函数和加密函数的流程完全相同。这样解密算法的实现就和加密算法的实现一致,为提高效率提供了机会。

注意在解密过程中,如果我们能够交换第 2 个和第 3 个函数,……,第 7 个函数,那么解密算法的结构就和加密算法一样。我们看看这是否可行。首先,考虑交换 INS 和 ISR。给定一个状态 N,包括半字节(N_0,N_1,N_2,N_3),INS(ISR(N))转换进行如下操作:

$$\begin{bmatrix} N_0 & N_2 \\ N_1 & N_3 \end{bmatrix} \rightarrow \begin{bmatrix} N_0 & N_2 \\ N_3 & N_1 \end{bmatrix} \rightarrow \begin{bmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_3] & \text{IS}[N_1] \end{bmatrix}$$

这里 IS 指逆 S 盒。将操作顺序颠倒过来,ISR(INS(N))转换进行如下操作:

$$\begin{bmatrix} N_0 & N_2 \\ N_1 & N_3 \end{bmatrix} \rightarrow \begin{bmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_1] & \text{IS}[N_3] \end{bmatrix} \rightarrow \begin{bmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_3] & \text{IS}[N_1] \end{bmatrix}$$

结果完全一样,因此 INS(ISR(N)) = ISR(INS(N))。

现在考虑列混淆求逆操作及随后的密钥加操作:IMC(A_{K_i}(N)),这里轮密钥 K_i 由半字节($k_{0,0},k_{1,0},k_{0,1},k_{1,1}$)组成。于是有

$$\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\left[\begin{bmatrix}k_{0,0} & k_{0,1}\\ k_{1,0} & k_{1,1}\end{bmatrix}\oplus\begin{bmatrix}N_0 & N_2\\ N_1 & N_3\end{bmatrix}\right]=\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\begin{bmatrix}k_{0,0}\oplus N_0 & k_{0,1}\oplus N_2\\ k_{1,0}\oplus N_1 & k_{1,1}\oplus N_3\end{bmatrix}$$

$$=\begin{bmatrix}9(k_{0,0}\oplus N_0)\oplus 2(K_{1,0}\oplus N_1) & 9(k_{0,1}\oplus N_2)\oplus 2(K_{1,1}\oplus N_3)\\ 2(k_{0,0}\oplus N_0)\oplus 9(K_{1,0}\oplus N_1) & 2(k_{0,1}\oplus N_2)\oplus 9(K_{1,1}\oplus N_3)\end{bmatrix}$$

$$=\begin{bmatrix}(9k_{0,0}\oplus 2k_{1,0})\oplus(9N_0\oplus 2N_1) & (9k_{0,1}\oplus 2k_{1,1})\oplus(9N_2\oplus 2N_3)\\ (2k_{0,0}\oplus 9k_{1,0})\oplus(2N_0\oplus 9N_1) & (2k_{0,1}\oplus 9k_{1,1})\oplus(2N_2\oplus 9N_3)\end{bmatrix}$$

$$=\begin{bmatrix}(9k_{0,0}\oplus 2k_{1,0}) & (9k_{0,1}\oplus 2k_{1,1})\\ (2k_{0,0}\oplus 9k_{1,0}) & (2k_{0,1}\oplus 9k_{1,1})\end{bmatrix}\oplus\begin{bmatrix}(9N_0\oplus 2N_1) & (9N_2\oplus 2N_3)\\ (2N_0\oplus 9N_1) & (2N_2\oplus 9N_3)\end{bmatrix}$$

$$=\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\begin{bmatrix}k_{0,0} & k_{0,1}\\ k_{1,0} & k_{1,1}\end{bmatrix}\oplus\begin{bmatrix}9 & 2\\ 2 & 9\end{bmatrix}\begin{bmatrix}N_0 & N_2\\ N_1 & N_3\end{bmatrix}$$

上述所有步骤都用到了有限域的运算特点。结论是 $\mathrm{IMC}(A_{K_1}(N))=\mathrm{IMC}(K_1)\oplus\mathrm{IMC}(N)$。现在定义第二轮的逆轮密钥为 $\mathrm{IMC}(K_1)$，并且逆轮密钥加操作 IA_{K_1} 是 state 向量与逆轮密钥的按位异或。那么就有 $\mathrm{IMC}(A_{K_1}(N))=\mathrm{IA}_{K_1}(\mathrm{IMC}(N))$。因此可以得出

加密：$A_{K_2}\circ\mathrm{SR}\circ\mathrm{NS}\circ A_{K_1}\circ\mathrm{MC}\circ\mathrm{SR}\circ\mathrm{NS}\circ A_{K_0}$

解密：$A_{K_0}\circ\mathrm{INS}\circ\mathrm{ISR}\circ\mathrm{IMC}\circ A_{K_1}\circ\mathrm{INS}\circ\mathrm{ISR}\circ A_{K_2}$

解密：$A_{K_0}\circ\mathrm{ISR}\circ\mathrm{INS}\circ A_{IMC(K_1)}\circ\mathrm{IMC}\circ\mathrm{ISR}\circ\mathrm{INS}\circ A_{K_2}$

现在加密和解密的流程完全相同。注意如果第二轮加密算法包括 MC 函数的话，该推导就无效了。这时有

加密：$A_{K_2}\circ\mathrm{MC}\circ\mathrm{SR}\circ\mathrm{NS}\circ A_{K_1}\circ\mathrm{MC}\circ\mathrm{SR}\circ\mathrm{NS}\circ A_{K_0}$

解密：$A_{K_0}\circ\mathrm{INS}\circ\mathrm{ISR}\circ\mathrm{IMC}\circ A_{K_1}\circ\mathrm{INS}\circ\mathrm{ISR}\circ\mathrm{IMC}\circ A_{K_2}$

现在就没有办法对解密算法中的操作进行互换，使之同加密算法结构相同。

第6章　分组密码的工作模式

6.1　多重加密和三重 DES 算法
　6.1.1　双重 DES
　6.1.2　使用两个密钥的三重 DES
　6.1.3　使用三个密钥的三重 DES
6.2　电码本模式
6.3　密文分组链接模式
6.4　密文反馈模式
6.5　输出反馈模式
6.6　计数器模式
6.7　面向分组存储设备的 XTS-AES
　6.7.1　存储加密要求
　6.7.2　单分组上的运算
　6.7.3　扇区上的运算
6.8　推荐读物和网站
6.9　关键术语、思考题和习题

Many savages at the present day regard their names as vital parts of themselves, and therefore take great pains to conceal their real names, lest these should give to evil-disposed persons a handle by which to injure their owners.

—*The Golden Bough*, Sir James George Frazer

要　点

- 多重加密是将一个加密算法多次使用的技术。在第一次使用中，明文通过加密算法转化为密文。然后将该密文作为输入重新执行加密算法。该过程可以重复任意多次。
- 三重 DES 在三个阶段使用 DES 算法，共用到两组或者三组密钥。
- 工作模式是一项增强密码算法或者使算法适应具体应用的技术，例如将分组密码应用于数据块组成的序列或者数据流。
- 对称密码，如 DES 和 AES 有 5 种标准的工作模式：电码本模式、密文分组链接模式、密文反馈模式、输出反馈模式、计数器模式。
- XTS-AES 是另一种重要的模式，已经由 IEEE 的安全存储工作组定义为标准(P1619)。该标准描述了存储在扇区设备上的数据的加密方法，这种存储方法面临的威胁包括敌手对存储数据的访问。

本章继续讨论对称密码。首先我们讨论多重加密，特别是目前广泛使用的一种多重加密方案：三重 DES。

接下来本章介绍分组密码的工作模式。我们发现使用分组密码加密明文有许多种不同的方法，每一种方法都有各自的优点和特定应用环境。

6.1　多重加密与三重 DES 算法

DES 在穷举攻击之下相对比较脆弱，因此很多人在想办法用某种算法替代它。一种方案是设计全新的算法，例如 AES。还有一种方案，能够保护已有软件和硬件的投资，那就是用 DES 进行多次加密，且使用多个密钥。我们以这种替代方案的一个简单例子开始，再讨论已被广泛接受的三重 DES(3DES)算法。

6.1.1　双重 DES

多次加密的最简单形式是进行两次加密，使用两个密钥[参见图 6.1(a)]。给定明文 P 及密钥 K_1 和 K_2，密文 C 按下述方式生成：

$$C = \mathrm{E}(K_2, \mathrm{E}(K_1, P))$$

解密时逆序使用这两个密钥：

$$P = \mathrm{D}(K_1, \mathrm{D}(K_2, C))$$

对于 DES，这种方法的密钥长度为 $56 \times 2 = 112$ 位，密码强度增加了。不过我们还是要对它进行仔细分析。

约化为单次加密

考虑下述说法：对所有的 56 位密钥，给定 K_1 和 K_2，可能存在密钥 K_3 使得

$$\mathrm{E}(K_2, \mathrm{E}(K_1, P)) = \mathrm{E}(K_3, P) \tag{6.1}$$

若该说法成立，则两层加密，实际上不管用 DES 进行了多少次加密运算，都是没有用的，因为它的效果等同于用一个密钥进行一次 DES 加密的效果。

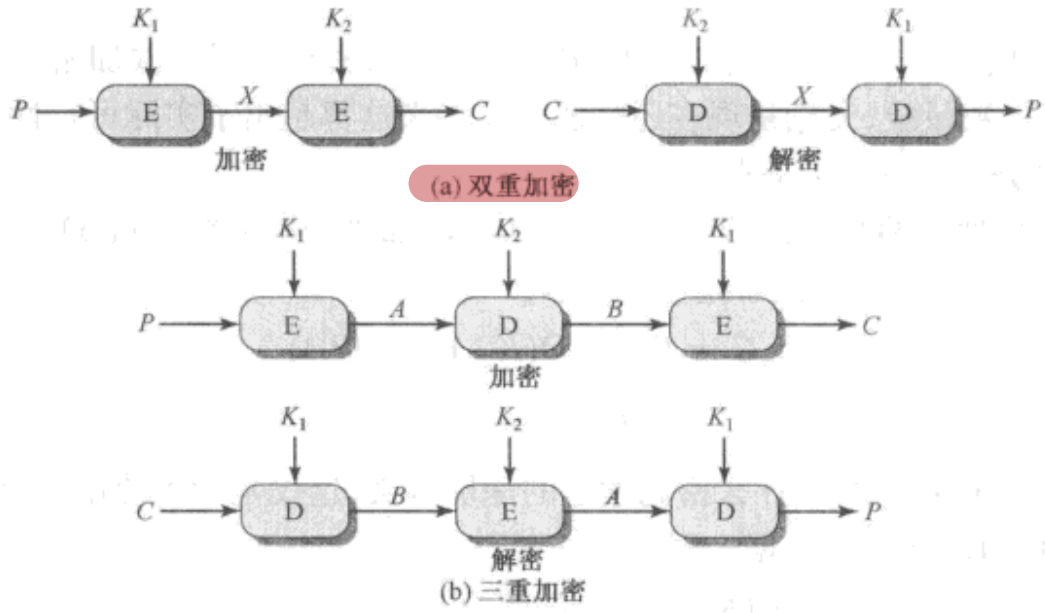


图 6.1　多重加密

表面看来，式(6.1)不太可能成立。DES 加密是 64 位分组之间的映射，实际上映射可以视为一个置换。这就是说，对 2^{64} 个可能的明文分组，DES 用某特定密钥加密后都唯一对应一个 64 位的密文分组。否则，如果两个输入块映射为一个输出块，那么解密以恢复原始的明文也是不可能的。那么，2^{64} 个可能的输入，有多少个一对一映射呢？这个数量是

$$(2^{64})! = 10^{34738000000000000000} > (10^{10^{20}})$$

另一方面，DES 为每个密钥定义了一个映射，映射总数为

$$2^{56} < 10^{17}$$

所以，完全有理由认为，双重 DES 所对应的映射不能为单 DES 所定义。尽管有很多支持该假设的证据，但直到 1992 年该假设才被证明，参见参考文献[CAMP92]。

中间相遇攻击

虽然双重 DES 对应的映射与单 DES 对应的映射不同，但是还有另外一种攻击方法，这种方法不依赖于 DES 的任何特殊性质，对所有分组密码都有效。

该算法称为中间相遇攻击,参考文献[DIFF77]首次对它做过描述。它基于如下观察:假设

$$C = E(K_2, E(K_1, P))$$

则有[参见图 6.1(a)]

$$X = E(K_1, P) = D(K_2, C)$$

给定明密文对(P, C),攻击如下展开:首先,将 P 按所有可能的密钥 K_1 加密,得到的 2^{56} 个结果按 X 的值排序放在一个表内。然后将 C 用所有可能的密钥 K_2 解密,每解密一次就将解密结果与表中的值比较,如果有相等的,就用刚才测试的两个密钥对一个新的明密文对进行验证。如果两个密钥产生了正确的密文,就认定这两个密钥是正确的密钥。

对任意给定的明文 P,采用双重 DES 加密后可能得到 2^{64} 个密文中的一个,双重 DES 使用了 112 位密钥,所以密钥空间为 2^{112}。因此对明文 P,可产生密文 C 的密钥个数平均为 $2^{112}/2^{64} = 2^{48}$。故上述攻击过程对第一个(P, C)对将产生 2^{48} 个错误的结果。而对第二个 64 位的明密文对(P, C),错误结果的概率就降为 $2^{48-64} = 2^{-16}$。换句话说,中间相遇攻击使用两组已知明密文对就可以猜出正确密钥的概率为 $1 - 2^{-16}$。结论是:已知明文攻击可以成功对付密钥长度为 112 位的双重 DES,其付出是 2^{56} 数量级的,比攻击单 DES 所需的 2^{55} 数量级多不了多少。

6.1.2 使用两个密钥的三重 DES

对付中间相遇攻击的一个明显办法是使用三个不同的密钥进行三次加密。这样,已知明文攻击的代价将升为 2^{112} 数量级,无论是现在还是遥远的将来这都超出了实际可行性。然而,它需要长为 $56 \times 3 = 168$ 位的密钥,这多少有些笨拙。

Tuchman 建议仅使用两个密钥[TOCH79]进行三次加密[参见图 6.1(b)]。具体运算过程是加密 - 解密 - 加密(EDE),写成式子是

$$C = E(K_1, D(K_2, E(K_1, P)))$$

$$P = D(K_1, E(K_2, D(K_1, C)))$$

第二步采用解密运算并没有什么密码学上的深层含义,这仅是为了使用三重 DES 的用户可以利用该算法解密单 DES 加密的数据。因为

$$C = E(K_1, D(K_1, E(K_1, P))) = E(K_1, P)$$

$$P = D(K_1, E(K_1, D(K_1, C))) = D(K_1, C)$$

使用两个密钥的三重 DES 已经广泛地替代了 DES,并已用于密钥管理标准 ANS X9.17 和 ISO 8732[①]。

当前,还没有对 3DES 的可行攻击方法。Copper Smith(参见[COPP94])分析后认为对 3DES 的穷举攻击的代价是 $2^{112} \approx (5 \times 10^{33})$ 数量级的,且估计用差分密码分析的代价是按指数增长的,与单 DES 比较超过 10^{52}。

我们不妨看看几种对 3DES 的攻击方法,虽然都不实际,也许对以后产生好的攻击方法有所裨益。

第一个好的攻击建议来自 Merkle 和 Hellman[MERK81]。即挨个查看可能的明文,看哪一个明文的第一个中间值[参见图 6.1(b)]$A = 0$,然后再使用中间相遇攻击来得到两个密钥。这种攻击方法的代价是 2^{56} 数量级的,但是它需要 2^{56} 个选择明密文对,这是不可能的。

① 美国国家标准(ANS):金融机构密钥管理(批发)。从名字看,X9.17 显得并不出名,然而这个标准定义的大量技术已经被其他标准和应用所采用,本书中我们将陆续看到。

差分密码分析=密码分析方法之一
比较2个不同输入之间的差异,然后观察输出之间的关联系。
large sum 大量明文,密文

参考文献[VANO90]中概述了一种已知明文攻击，相对于选择明文攻击，这种方法有些进步，但付出的代价也增多了。攻击是基于观察到：若已知 A 和 C[参见图 6.1(b)]，那问题相当于对双重 DES 的攻击。当然，只要不知道密钥，即使知道了 P 和 C，攻击者还是得不到 A。然而，攻击者可选择 A 的一个可能值，再试着找出一个可产生 A 的已知(P,C)对。具体的攻击过程是这样的：

(1) 获取 n 个(P,C)对，将这些已知(P,C)对按 P 排序放在表(参见表 1)中[参见图 6.2(b)]。

(2) 为 A 随意选择值 a，创建第 2 张表[参见图 6.2(c)]，表的项按下列方式定义。对 2^{56} 中的每个可能的密钥 $K_1 = i$，计算可产生 a 的明文值 P_i

$$P_i = \mathrm{D}(i, a)$$

对所有能与图 6.2(b)中的表 1 项匹配的 P_i，在图 6.2(c)中表 2 中创建一个项，其中有 K_1 的值，以及由(P,C)对产生的 B 的值，即

$$B = \mathrm{D}(i, C)$$

最后将图 6.2(c)中表 2 按 B 的值排序。

(3) 现在表 2 中有了一批 K_1 的候选值，现在我们需要搜索 K_2。对 2^{56} 中的任意可能的 $K_2 = j$，对选定的 a 计算第 2 个中间值：

$$B_j = \mathrm{D}(j, a)$$

每做一步都在表 2 中查找 B_j，若有相等者，则表 2 中的相应密钥 i 和 j 的值就是未知密钥(K_1，K_2)的可能值。因为我们找到一个密钥对(i,j)，可以生成已知的(P,C)对[参见图 6.2(a)]。

(4) 对这样的密钥对(i,j)再进行几个其他的(P,C)对的测试，要是测试出来用某些 P 按密钥(i,j)不能产生 C，则返回步骤 1 选择一个新的 a。否则，便大功告成：(i,j)就是密钥。

对给定的(P,C)，选择 a 成功的可能性为 2^{-64}。所以，给定 n 个(P,C)对，单个选择的 a 成功的可能性为 $n/2^{64}$。概率论中有这样的结果：从一个装有 n 个红球和 $N-n$ 个绿球的箱子里，平均要摸$(N+1)/(n+1)$次才可能摸一个红球，其中球不能重新放回。所以在 n 非常大的时候，尝试的 a 的个数为

$$\frac{2^{64}+1}{n+1} \approx \frac{2^{64}}{n}$$

则攻击的期望运算时间是以下数量级的：

$$\left(2^{56}\right)\frac{2^{64}}{n} = 2^{120-\log_2 n}$$

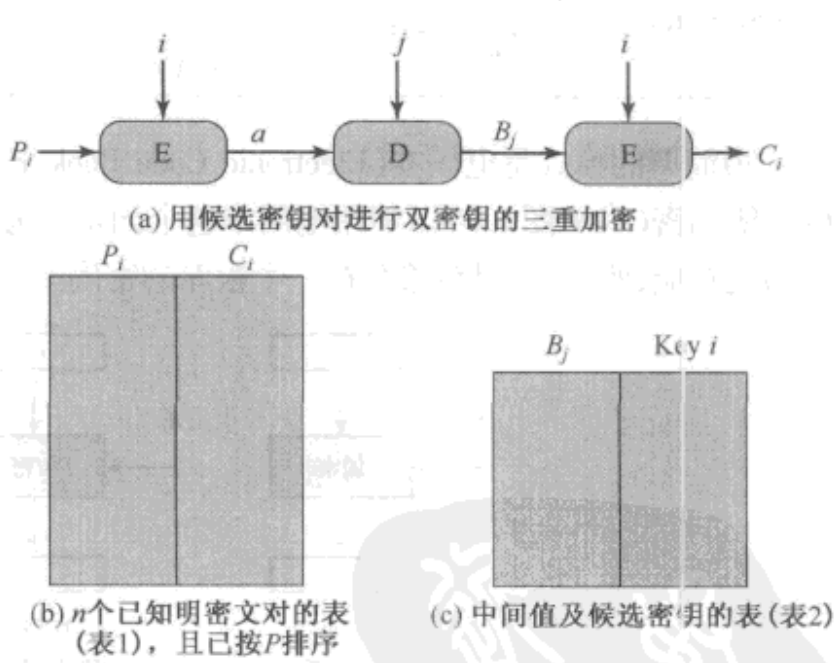


图 6.2　对 3DES 的已知明文攻击

6.1.3　使用三个密钥的三重 DES

尽管上述攻击方法并不实用，但是所有使用双密钥的三重 DES 算法的人还是感觉有点悬。因此，很多人觉得采用三个密钥的三重 DES 算法才是最好方案(例如参考文献[KALI96a])。三个密钥的三重 DES 的密钥长度为 168 位，定义成

$$C = \mathrm{E}(K_3, \mathrm{D}(K_2, \mathrm{E}(K_1, P)))$$

要想和单 DES 兼容，只需设 $K_3 = K_2$ 或 $K_1 = K_2$ 就可以了。

有些基于 Internet 的应用已经采纳了这种三密钥的 3DES,例如第 18 章将要讨论的 PGP 和S/MIME。

6.2 电码本模式

分组密钥的输入为具有 b 位固定长度的明文分组和密钥,输出为 b 位的密文。明文长度若大于 b 位,则可简单将其分成 b 位一组的块。每次使用相同的密钥对多个分组加密,则会引发许多安全问题。为了将分组密码应用于各种各样的实际应用,NIST(SP800-38A)定义了 5 种“工作模式”。从本质上讲,工作模式是一项增强密码算法或者使算法适应具体应用的技术,例如将分组密码应用于数据块组成的序列或者数据流。这 5 种模式实际上覆盖了大量使用分组密码的应用。这些模式可用于包括三重 DES 和高级加密标准(AES)在内的任何分组密码。表 6.1 对这些模式做了一个总结,在余下的章节中将会对它们做简单的描述。

表 6.1 分组密码的工作模式

模　式	描　述	典型应用
电码本(ECB)	用相同的密钥分别对明文分组独立加密	● 单个数据的安全传输(如一个加密密钥)
密文分组链接(CBC)	加密算法的输入是上一个密文组和下一个明文组的异或	● 面向分组的通用传输 ● 认证
密文反馈(CFB)	一次处理 s 位,上一块密文作为加密算法的输入,产生的伪随机数输出与明文异或作为下一单元的密文	● 面向数据流的通用传输 ● 认证
输出反馈(OFB)	与 CFB 类似,只是加密算法的输入是上一次加密的输出,且使用整个分组	● 噪声信道上的数据流的传输(如卫星通信)
计数器(CTR)	每个明文分组都与一个经过加密的计数器相异或。对每个后续分组计数器递增	● 面向分组的通用传输 ● 用于高速需求

最简单的模式是电码本(Electronic Code Book,ECB)模式,它一次处理一组明文分块,每次使用相同的密钥加密(参见图 6.3)。使用电码本这个词是因为对于给定的密钥,任何 b 位的明文组只有唯一的密文与之对应,所以,可以想象存在一个很厚的密码本,根据任意 b 位明文都可以查到相应的密文。

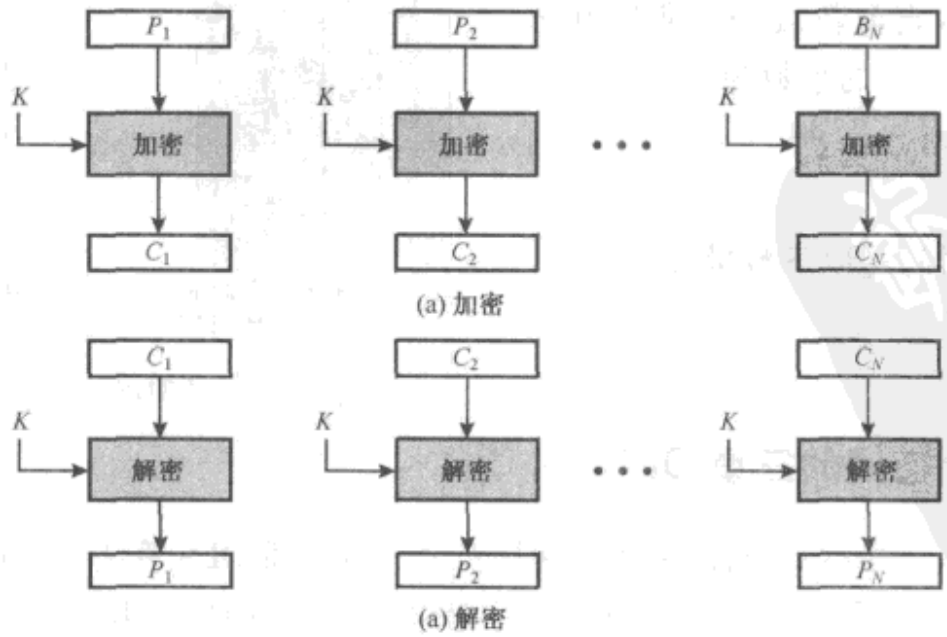


图 6.3 电码本模式

明文若长于 b 位,则可简单将其分成 b 位一组的块,有必要则可对最后一块进行填充。解密也是一次执行一块,且使用相同的密钥。图 6.3 中的明文(必要的时候进行填充)由一串 b 位的块

组成，记为 $P_1, P_2, \cdots, P_N$，相应的密文分组依次是 $C_1, C_2, \cdots, C_N$。我们可以如下定义 ECB 模式。

ECB	$C_j = \mathrm{E}(K, P_j) \quad j = 1, \cdots, N$	$P_j = D(K, C_j) \quad j = 1, \cdots, N$

ECB 模式特别适合于数据较少的情况，比如加密密钥。因此，若想安全传输一个 DES 或 AES 密钥，选择这种模式是合适的。

ECB 最重要的特征是一段消息中若有几个相同的明文组，那么密文也将出现几个相同的密文分组。

对于很长的消息，ECB 模型可能不安全。如果消息是非结构化的，密码分析者可能利用这些规律性特征来破译。例如，若已知这段消息总是以某些固定的字符开头，密码分析者就可以拥有大量已知明密文对以开展攻击。若消息有重复的成分，且重复的周期正好是 b 位的倍数，分析者就能辨认出这些成分，然后可以用代换或重排这些块的方法进行攻击。

6.3　密文分组链接模式

为了克服 ECB 的这些弱点，我们需要将重复的明文分组加密成不同的密文分组。满足这一要求的简单方法是用密文分组链接(Cipher Block Chaining，CBC)模式(参见图 6.4)。这种模式下加密算法的输入是当前的明文组和上一个密文组的异或，而使用的密钥是相同的。这就相当于将所有的明文组链接起来了。加密算法的每次输入与本明文组没有固定的关系。因此，若有重复的明文组，加密后就看不出来了。和 ECB 方式类似，CBC 方式也要求如果最后的分组不是完整的分组，则需要填充至 b 位的满分组。

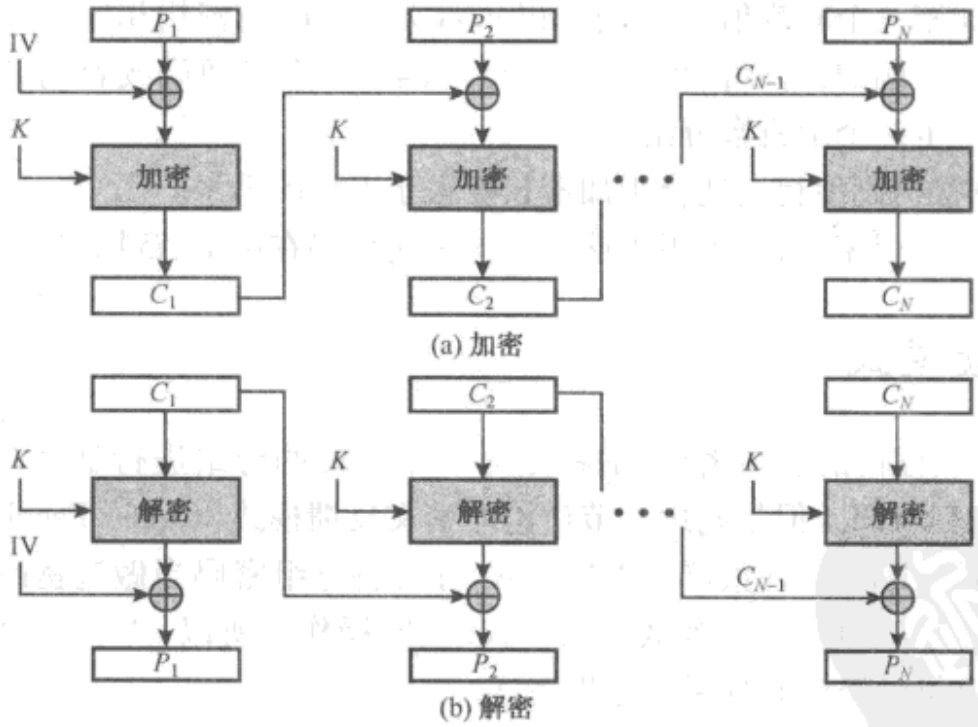


图 6.4　密文分组链接模式

解密时，每个密文分组分别进行解密，再与上一块密文异或就可恢复出明文。下面对这个过程的正确性给出证明：

$$C_j = \mathrm{E}(K, [C_{j-1} \oplus P_j])$$

$$\mathrm{D}(K, C_j) = \mathrm{D}(K, E(K, [C_{j-1} \oplus P_j]))$$

则

$$\mathrm{D}(K, C_j) = C_{j-1} \oplus P_j$$

$$C_{j-1} \oplus \mathrm{D}(K, C_j) = C_{j-1} \oplus C_{j-1} \oplus P_j = P_j$$

第一块明文可以和一个初始向量(IV)异或后再加密,以产生第一个明文分组。解密时将第一块密文解密的结果与 IV 异或而恢复出第一块明文。IV 是和密文具有相同的长度的数据分组。CBC 模式可以如下定义。

CBC	$C_1 = \mathrm{E}(K, [P_1 \oplus \mathrm{IV}])$ $C_j = \mathrm{E}(K, [P_j \oplus C_{j-1}])\ \ j = 2, \cdots, N$	$P_1 = \mathrm{D}(K, C_1) \oplus \mathrm{IV}$ $P_j = \mathrm{D}(K, C_j) \oplus C_{j-1}\ \ j = 2, \cdots, N$

IV 必须为收发双方共享,但第三方不能预测。特别地,对于任意的给定明文,在 IV 产生之前,不能预测和本明文关联的 IV。为了最大程度地安全,IV 不能不经授权而修改。对 IV 先用 ECB 加密后再发送的方式可以实现这一要求。要保护 IV 的一个原因是:攻击者可以欺骗接收者,让他使用不同的 IV,接着攻击者就能够把明文的第一个分组的某些位取反。为了弄明白这一点,请看

$$C_1 = \mathrm{E}(K, [\mathrm{IV} \oplus P_1])$$
$$P_1 = \mathrm{IV} \oplus \mathrm{D}(K, C_1)$$

用 $X[i]$ 表示 b 位 X 的第 i 位,则

$$P_1[i] = \mathrm{IV}[i] \oplus \mathrm{D}(K, C_1)[i]$$

使用 XOR 的性质,我们将上式重写为

$$P_1[i]' = \mathrm{IV}[i]' \oplus \mathrm{D}(K, C_1)[i]$$

撇号表示取反。这意味着攻击者可以预先改变 IV 中的某些位,则接收者收到的 P_1 相应也就改变了。

还有其他利用 IV 的一些攻击方法,见参考文献[VOYD83]。

只要 IV 是不可预测的,那么具体选什么 IV 是不重要的。SP800-38A 推荐了两种方法:第一种方法是用加密函数加密一个时变值①,所用密钥和明文加密所用密钥相同。这个时变值对每次加密运算来说必须唯一。例如,时变值可以是一个计数器、一个时间戳或者消息数目。第二种方法是用随机数发生器产生一个随机数分组。

总之,CBC 的链接机制使得它适合于加密长度大于 b 位的消息。

CBC 除了用来获得保密性,亦可用于认证。这种使用将在第 12 章描述。

PPT

6.4 密文反馈模式

对于 AES、DES 或任何的分组密码,加密是对一个 b 位的分组进行加密。对于 DES 的例子,$b = 64$,而在 AES 时,$b = 128$。但是利用本节讨论的密文反馈模式(Cipher Feedback,CFB)和下面要讨论的输出反馈模式(OFB)及计数器(CTR)模式,亦可将分组密码当做流密码使用。流密码不需要将明文填充到长度是分组长度的整数倍,且可以实时操作。所以,待发送的字符流中任何一个字符都可以用面向字符的流密码加密后立即发送。

流密码有一个让人心动的性质,即密文与明文等长。所以要发送的是 8 位的字符,加密时也是用 8 位。如果多于 8 位,传输能力就浪费了。

图 6.5 描述了 CFB 模式。假设传输单元是 s 位,s 通常为 8。同使用 CBC 模式一样,明文的各个单元要链接起来,所以任意个明文单元的密文都是前面所有明文的函数。在这种情况下,明文被分成 s 位的片段而不是 b 位的单元。

① NIST SP800-90(使用决定性随机位发生器的随机数发生器建议)如下定义时变值:一个随时间变化的值,具有可忽略的重复概率,如每次使用都重新生成的随机值、时间戳、序列号或它们的某种组合。

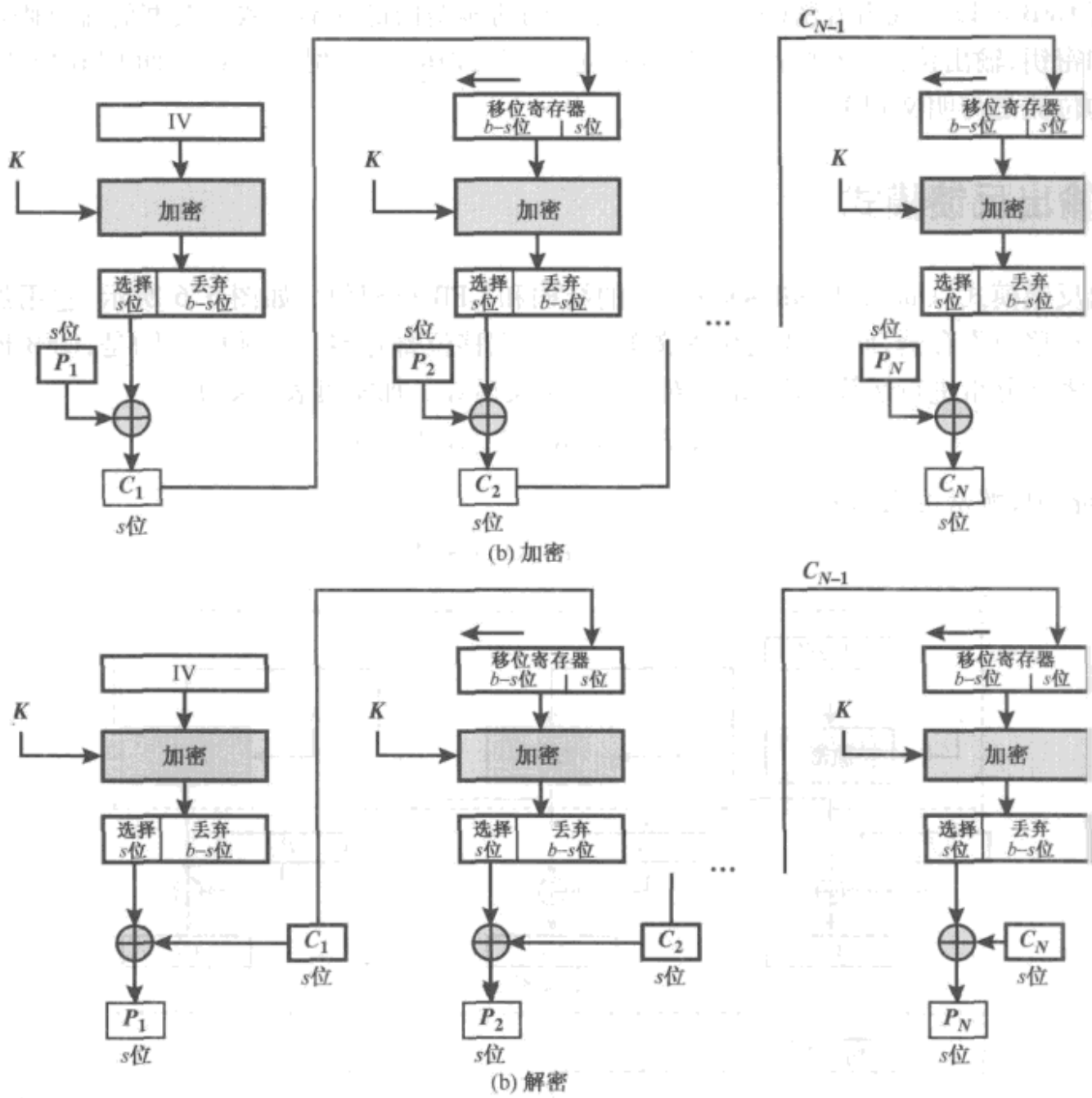


图 6.5　s 位密文反馈(CFB)模式

首先来考虑加密。加密函数的输入是 b 位的移位寄存器,它的值为初始向量 IV。加密函数输出最左边的 s 位与明文的第一个分段 P_1 异或得到密文的第一个单元 C_1,然后将 C_1 发送出去,接着,移位寄存器左移 s 位,C_1 填入移位寄存器的最右边 s 位。就这样,直到所有明文单元被加密完。

解密使用相同的办法,只是有一点不同:将收到的密文单元与加密函数的输出异或得到明文单元。注意,这里使用的是加密函数而非解密函数,这一点很容易解释。设 $\mathrm{MSB}_s(X)$ 表示 X 的最左边 s 位。则

$$C_1 = P_1 \oplus \mathrm{MSB}_s[\mathrm{E}(K, \mathrm{IV})]$$

从而有

$$P_1 = C_1 \oplus \mathrm{MSB}_s[\mathrm{E}(K, \mathrm{IV})]$$

对后续单元亦同理可得。

CFB 模式可以如下定义:

CFB	$I_1 = \mathrm{IV}$ $I_j = \mathrm{LSB}_{b-s}(I_{j-1}) \| C_{j-1}$　$j = 2, \cdots, N$ $O_j = \mathrm{E}(K, I_j)$　$j = 1, \cdots, N$ $C_j = P_j \oplus \mathrm{MSB}_s(O_j)$　$j = 1, \cdots, N$	$I_1 = \mathrm{IV}$ $I_j = \mathrm{LSB}_{b-s}(I_{j-1}) \| C_{j-1}$　$j = 2, \cdots, N$ $O_j = \mathrm{E}(K, I_j)$　$j = 1, \cdots, N$ $P_j = C_j \oplus \mathrm{MSB}_s(O_j)$　$j = 1, \cdots, N$

尽管CFB可以被视为流密码,但是它和流密码的典型构造并不一致。典型的流密码输入某个初始值和密钥,输出位流,这个位流再和明文位进行异或运算(参见图3.1)。而CFB模式里,与明文异或的位流是与明文相关的。

6.5 输出反馈模式

输出反馈模式(Output FeedBack,OFB)的结构和CFB很相似。如图6.6所示,它用加密函数的输出填充移位寄存器,而CFB是用密文单元来填充移位寄存器。其他的不同是,OFB模式对整个明文和密文分组进行运算,而不是仅对 s 位的子集运算。加密可表示如下:

$$C_j = P_j \oplus \mathrm{E}(K, [C_{j-i} \oplus P_{j-1}])$$

通过重排各项,解密可表示如下:

$$P_j = C_j \oplus \mathrm{E}(K, [C_{j-1} \oplus P_{j-1}])$$

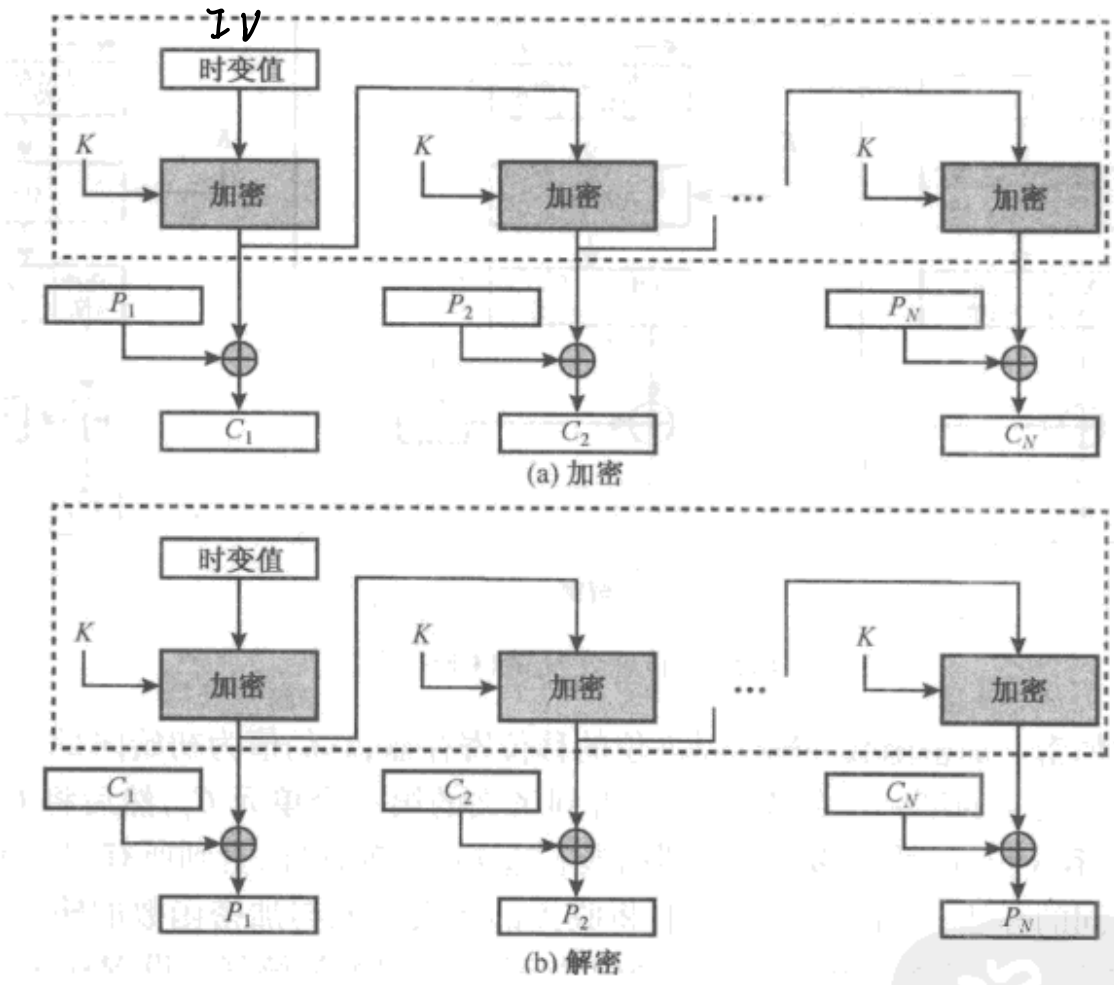


图6.6 输出反馈(OFB)模式

OFB模式可以如下定义:

OFB	I_1 = Nonce $I_j = O_{j-1}$ $j = 2, \cdots, N$ $O_j = \mathrm{E}(K, I_j)$ $j = 1, \cdots, N$ $C_j = P_j \oplus O_j$ $j = 1, \cdots, N-1$ $C_N^* = P_N^* \oplus \mathrm{MSB}_u(O_N)$	I_1 = Nonce $I_j = \mathrm{LSB}_{b-s}(I_{j-1}) \| C_{j-1}$ $j = 2, \cdots, N$ $O_j = \mathrm{E}(K, I_j)$ $j = 1, \cdots, N$ $P_j = C_j \oplus O_j$ $j = 1, \cdots, N-1$ $P_N^* = C_N^* \oplus \mathrm{MSB}_u(O_N)$

令分组的长度为 b。如果明文的最后一个分组包含 u 位(用 * 指示),$u < b$,那么最后的输出分组 O_N 的最左边的 u 位用来做异或运算。最后的输出分组的其余 $b-u$ 位丢弃不用。

如同CBC和CFB一样,OFB模式需要一个初始化向量。在OFB模式时,IV必须是一个时变

值，即 IV 对每次加密运算都是唯一的。原因是加密输出的分组构成的序列 O_i 仅仅依赖于密钥和 IV，而不依赖于明文。因此，对于给定的密钥和 IV，用于和明文流进行异或运算的输出位流是固定的。如果两个不同的消息在相同的地方有一个相同的明文分组，那么攻击者就能够判断出那部分的 O_i 输出流。

OFB 的一个优点是传输过程中在某位上发生的错误不会影响其他位。比如，C_1 中有 1 位发生了错误，只会影响到 P_1 的恢复，后续的明文单元不受影响。而 CFB 中 C_1 还作为了移位寄存器的输入，所以影响了后续的所有消息。

OFB 的缺点是，抗消息流篡改攻击的能力不如 CFB。即密文中的某位取反，恢复出的明文相应位也取反。所以攻击者有办法控制对恢复明文的改变。这样，攻击者可以根据消息的改动而改动校验和，以使改动不被纠错码发现。读者若想做深入了解，请参阅参考文献[VOYD83]。

OFB 具有典型流密码的结构，因为密码产生的位流是初始值和密钥的函数，且产生出的位流和明文进行了异或(参见图 3.1)。产生的位流(即和明文进行异或运算的位流)本身是和明文独立的；图 6.6 用虚线圈起的盒子突出显示了这一点。与第 7 章将要讨论的流密码相比，差别是 OFB 一次加密一个明文分组，分组的典型长度是 64 位或 128 位。许多流密码一次加密一个字节。

6.6　计数器模式

随着计数器模式(Counter，CTR)在 ATM(异步传输模式)网络安全与 IPsec 中的应用，使得人们最近才对它产生了浓厚的兴趣，但是实际上这种模式在很早以前就已经提出来了(如[DIFF79])。

图 6.7 描述了 CTR 模式。计数器使用与明文分组规模相同的长度。SP 800-38A 对此的唯一要求是加密不同的明文组计数器对应的值必须是不同的。典型地，计数器首先被初始化为某一值，然后随着消息块的增加计数器的值加 1(模 2^b，b 为分组长度)。加密时，计数器加密后与明文分组异或得到密文分组；没有链接。解密使用具有相同值的计数器序列，用加密后的计数器的值与密文分组异或来恢复明文分组。因此，解密时必须知道初始计数器的值。给定计数器序列 T_1，T_2，…，T_N，我们可以如下定义 CTR 模式。

CTR	$C_j = P_j \oplus \mathrm{E}(K, T_j) \quad j = 1, \cdots, N-1$ $C_N^* = P_N^* \oplus \mathrm{MSB}_u[\mathrm{E}(K, T_N)]$	$P_j = C_j \oplus \mathrm{E}(K, T_j) \quad j = 1, \cdots, N-1$ $P_N^* = C_N^* \oplus \mathrm{MSB}_u[\mathrm{E}(K, T_N)]$

最后的明文分组可能不是完整的分组，只是长为 u 位的部分分组，最后输出分组的最左边的 u 位用来做异或运算；其余的 $b-u$ 位丢弃。与 ECB、CBC 及 CFB 方式不同的是，由于 CTR 的结构关系，我们不需要对明文进行填充。

如同 OFB 模式一样，初始计数器的值必须为时变值；也就是说，使用相同密钥加密的所有消息必须各自具有不同的 T_1。而且，所有消息的所有 T_i 值也必须唯一。如果违反了这个要求，一个计数器的值使用了多次，那么这个计数器值对应的明文分组的保密性就会泄露。特别地，如果已知用那个给定的计数器值加密的任何明文，那么加密函数的输出很容易从关联的密文推导出来。对于所有使用该计数器值加密的明文来说，从这个输出很容易由密文推导出对应的明文。

保证计数器值唯一的一个办法是每一个消息计数器都增加 1。即每个消息的第一个计数器的值都要比前一条消息的最后一个计数器的值多 1。

参考文献[LIPM00]列出了计数器模式的如下优点：

- **硬件效率**:与三种链接模式不同,CTR 模型能够并行处理多块明文(密文)的加密(解密)。链接模式在处理下一块数据之前必须完成当前数据块的计算。算法的最大吞吐量受限于执行一次加密或解密所需要的交互时间。在 CTR 模型中,吞吐量仅受可并行度的限制。
- **软件效率**:类似地,因为 CTR 模式能够进行并行计算,因此可以充分利用能够支持如下并行特征的各类处理器,如提供流水线、每个时钟周期的多指令分派、大量的寄存器和 SIMD 指令等并行特征。
- **预处理**:基本加密算法的执行并不依靠明文或密文的输入。因此,如果有充足的存储器可用且能提供安全,可以预处理加密盒的输出,这个输出又进一步作为 XOR 函数的输入,如图 6.7 所示。当给出明文或者密文时,所需的计算仅是进行一系列的异或。这样的策略能够极大地提高吞吐量。
- **随机访问**:明文或密文的第 i 个分组能够用一种随机访问的方式处理。在链接模式下知道前面的 $i-1$ 块密文计算出来后才能计算密文 C_i。有很多应用情况是全部密文已储存好,只需要破解其中的某一块密文。对于这种情形,随机访问的方式很有吸引力。
- **可证明安全性**:能够证明 CTR 模式至少和本节讨论的其他模式一样安全。
- **简单性**:与 ECB 和 CBC 不同,CTR 模式只要求实现加密算法,而并不要求实现解密算法。当加密算法与解密算法有很大的不同时,这种模式就显得很重要,而高级加密标准就是这样。另外,也不用实现解密密钥扩展。

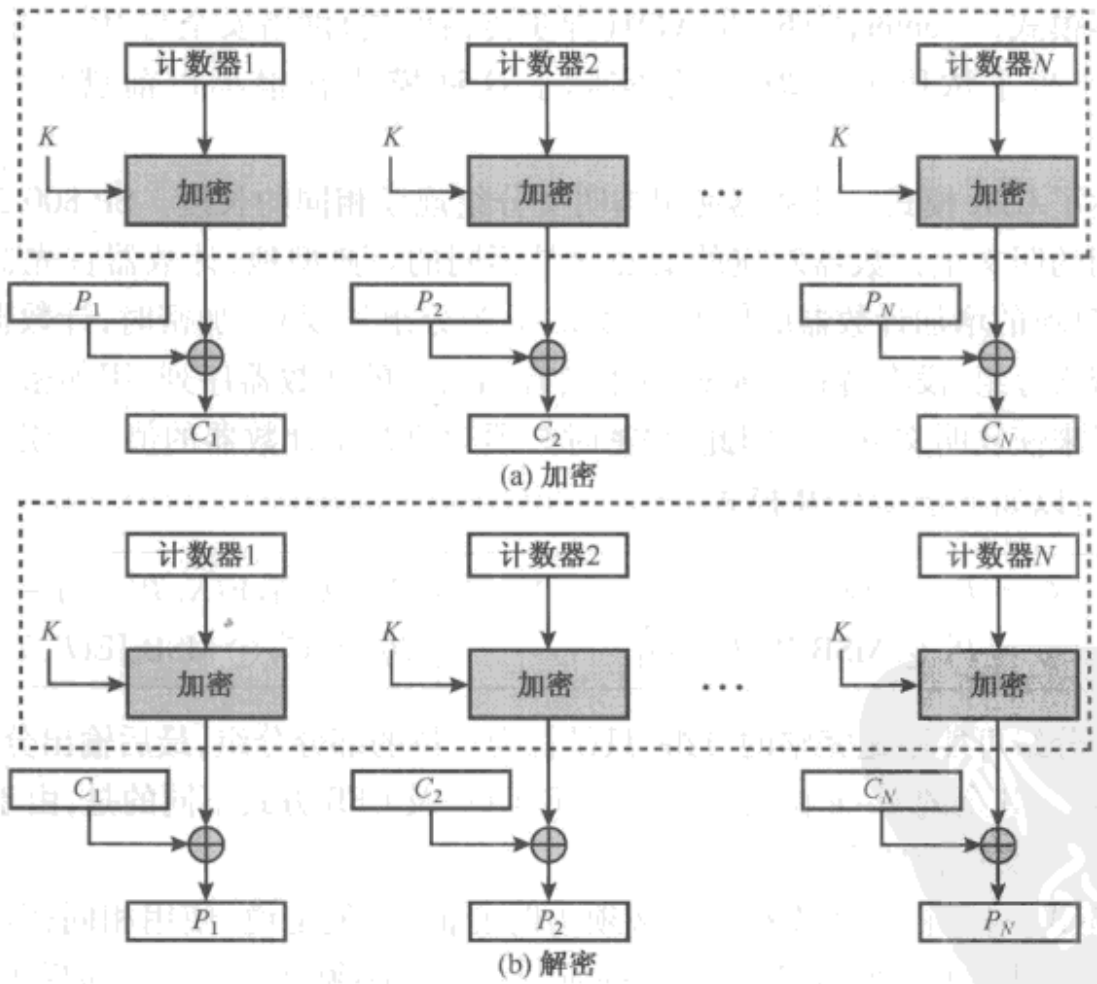


图 6.7 计数器(CTR)模式

请注意,除了 ECB 模式外,NIST 批准的所有分组密码工作模式都含有反馈。这在图 6.8 中很明显。为了突出显示这个反馈机制,可将加密函数想象为从输入寄存器取输入,而把输出存在输出寄存器中,其中输入寄存器的长度等于加密分组长度。这个想法很有用。输入寄存器由反馈机制每次更新一个分组。每次更新后,加密算法执行一次,结果放在输出寄存器中。同时访问明文

分组。注意 OFB 和 CTR 模式的输出独立于明文和密文。因此，它们是流密码的自然候选算法，通过 XOR 运算每次加密一个明文分组。

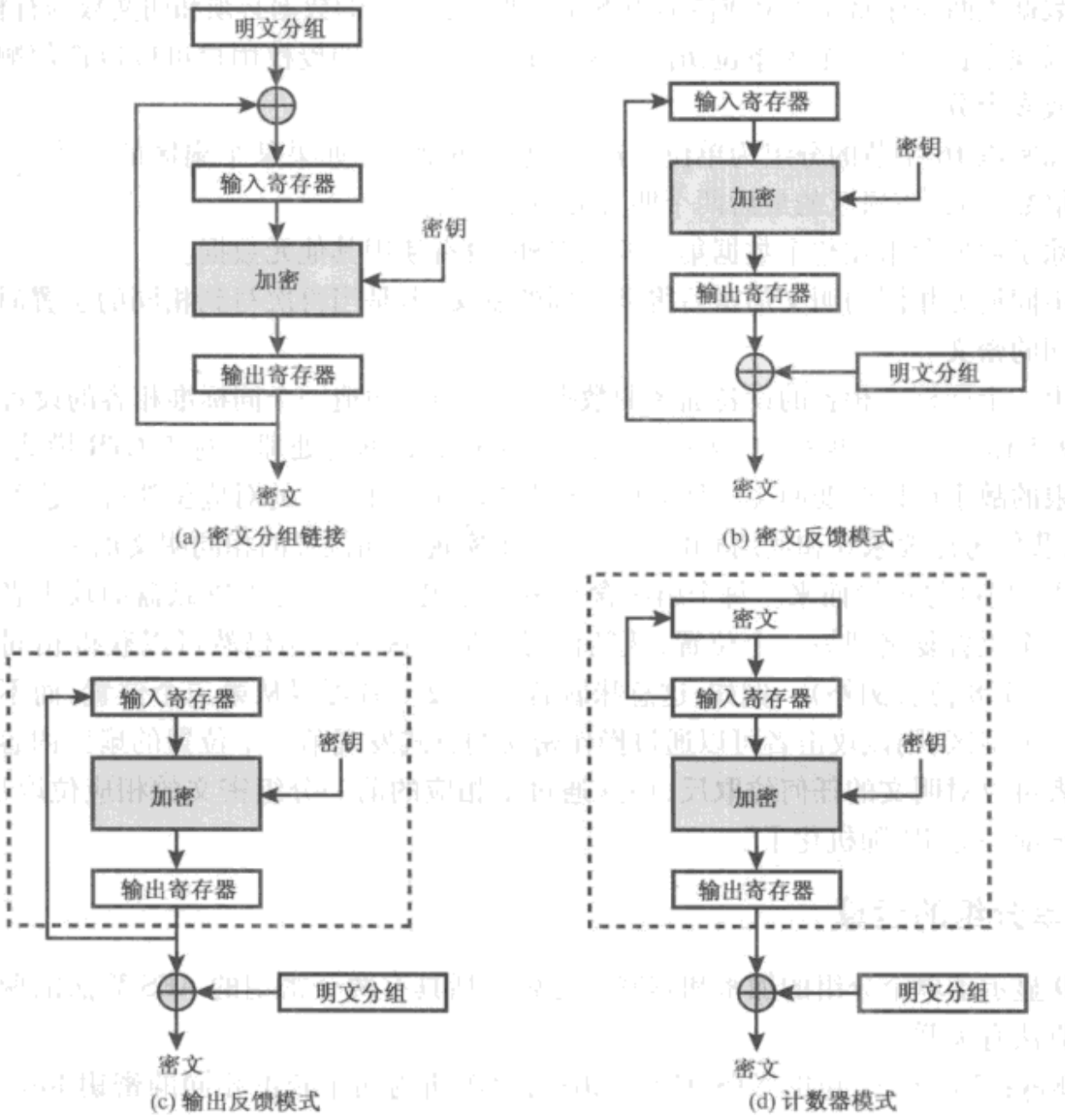


图 6.8　各工作模式的反馈特征

6.7　用于面向分组的存储设备的 XTS-AES 模式

NIST 最近正在忙着批准另外一个分组密码工作模式，即 XTS-AES。这个模式也是 IEEE 的标准，即 IEEE Std 1619-2007，是由 IEEE 存储安全工作组开发的标准。该标准描述了一种对基于扇区的设备上的数据进行加密的方法，此时的威胁模型包括敌手对存储数据的访问。

XTS-AES 模式基于参考文献[LISK02]引入的可微调分组密码的概念。XTS-AES 使用这个概念的形式首先在参考文献[ROGA04]中给出。该标准已经受到工业界的广泛支持。

6.7.1　存储加密的要求

加密存储的数据（也称为静止数据）的要求和加密传输数据的要求有些不同。P1619 标准在设计上有如下特征：

（1）攻击者可以随意获取密文。如下环境可以导致这种局面：

（a）授权一组用户可以访问数据库。数据库内的一些记录经过加密，于是只有特定的用户可以成功读写它们。其他用户可以检索加密记录，但没有密钥的话无法阅读。

(b) 一个非授权用户设法能访问加密记录。

(c) 数据磁盘或者便携式计算机被偷了,让攻击者可以访问加密数据。

(2) 数据版面在存储介质上或传输中不能改变。加密后的数据必须和明文数据有相同的大小。

(3) 以规定长度的分组为单位访问数据,相互间独立。即授权用户可以以任何顺序访问一个或多个分组。

(4) 加密以 16 字节的分组为单位,分组相互之间独立(如果某个扇区的大小不是 16 字节的倍数,那么该扇区的最后两个明文分组是例外)。

(5) 除了数据分组在整个数据集里的位置外,没有使用其他元数据。

(6) 不同地方相同的明文加密后得到不同的密文,但是当再次写到相同的位置时总是写为相同的密文。

(7) 由一个同标准相容的设备加密的数据,可以通过构造一个同标准相容的设备来解密。

P1619 工作组考虑了现存的一些工作模式用于存储数据的处理。对于 CTR 模式,具有对加密媒体写权限的敌手可以改变明文的任何位,这仅仅通过修改密文的对应位就可以达到。

接着,我们考虑要求 6 和使用 CBC 模式。为了实现不同位置的相同明文加密为不同密文的要求,IV 可以从扇区号推导而来。每个扇区含有多个分组。可读/写加密磁盘的攻击者可将一个密文扇区从一个位置复制到另一个位置,从新位置读取扇区的应用仍然可以获得相同的明文扇区(也许第一个 128 位会例外)。例如,这意味着若一个攻击者可以从第二个位置,而不是从第一个位置读取一个扇区,则该攻击者可以通过操作密文的方式发现第一个位置的扇区内容。另一个弱点是攻击者可以对明文的任何位取反,而这通过把相应的前一分组密文的相应位取反即可,当然副作用是把前一分组"随机化了"。

6.7.2 单分组的运算

图 6.9 显示了单个分组的加密和解密。运算包括具有两个密钥的 AES 算法的两个实例。如下参数和算法有关联。

密钥 Key:256 或 512 位的 XTS-AES 密钥;可以解析为两个长度相同的密钥 Key_1 和 Key_2 连接的结果,即 $Key = Key_1 \parallel Key_2$。

P_j:明文的第 j 个分组。除了最后一个分组外,所有分组的长度为 128 位。明文数据单元,通常是一个磁盘扇区,由明文分组序列 $P_1, P_2, \cdots, P_m$ 构成。

C_j:密文的第 j 个分组。除了最后一个分组外,所有分组的长度为 128 位。

j:数据单元内 128 位分组的序列号。

i:128 位微调参数的值。每个数据单位(扇区)被赋予一个具有非负整数值的微调值。这个微调值按顺序赋值,可以从任意非负整数开始。

α: $GF(2^{128})$ 的一个本原元,对应于多项式 x(即 $0000\cdots010_2$)。

α^j: $GF(2^{128})$ 内 α 自乘 j 次。

$\oplus$:位对位异或。

$\otimes$:两个二进制系数的多项式模 $x^{128} + x^7 + x^2 + x + 1$ 乘法。因此,这是 $GF(2^{128})$ 内的乘法。

本质上,参数 j 的功能类似 CTR 模式里的计数器,以保证一个数据单元内不同位置的相同明文分组加密为不同的密文分组。参数 i 从数据单元的层面上说相当于时变值,以保证两个不同的数据单元内相同位置相同明文分组加密为不同的密文分组。更一般地,它可以保证位于不同位置的相同数据单元加密为不同的密文数据单元。

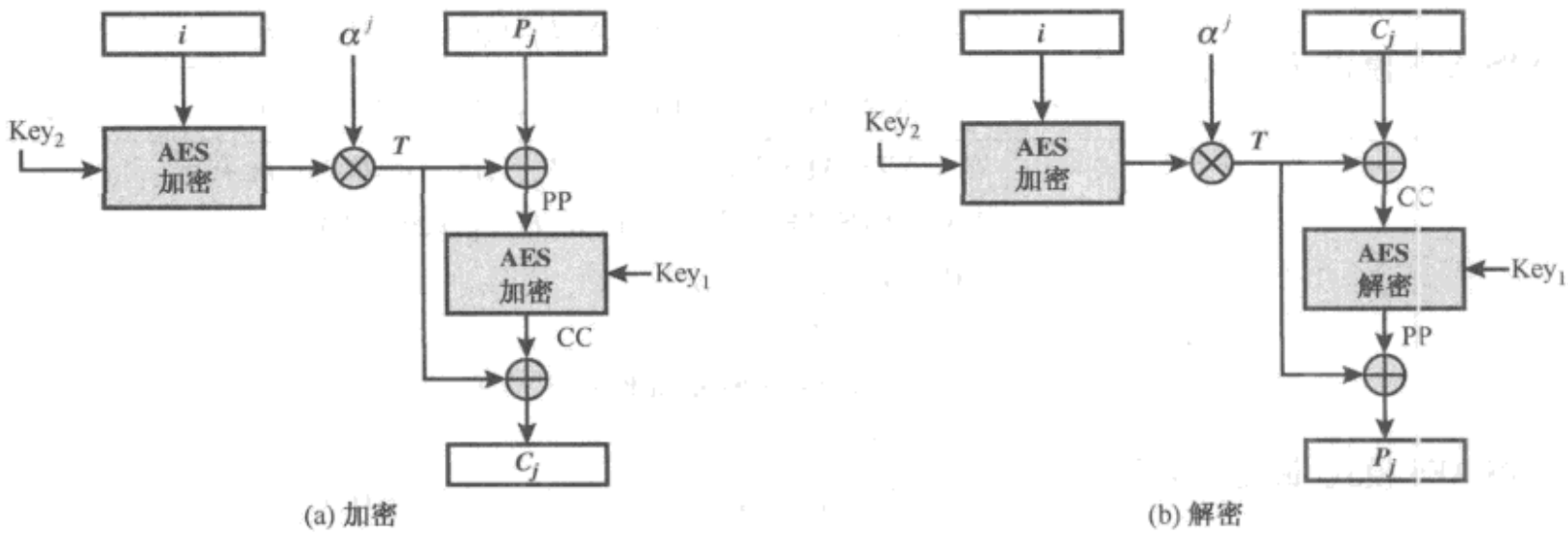


图 6.9　单个分组的 XTS-AES 运算

单分组的加密和解密可以描述为：

XTS-AES 分组运算	$T = \mathrm{E}(K_2, i) \otimes \alpha^j$ $\mathrm{PP} = P \oplus T$ $\mathrm{CC} = \mathrm{E}(K_1, \mathrm{PP})$ $C = \mathrm{CC} \oplus T$	$T = \mathrm{E}(K_2, i) \otimes \alpha^j$ $\mathrm{CC} = C \oplus T$ $\mathrm{PP} = \mathrm{D}(K_1, \mathrm{CC})$ $P = \mathrm{PP} \oplus T$

为了明白解密确实可以恢复明文，让我们把加密和解密的最后一行进行扩展。对于加密，有

$$C = \mathrm{CC} \oplus T = \mathrm{E}(K_1, \mathrm{PP}) \oplus T = \mathrm{E}(K_1, P \oplus T) \oplus T$$

对于解密有

$$P = \mathrm{PP} \oplus T = \mathrm{D}(K_1, \mathrm{CC}) \oplus T = \mathrm{D}(K_1, C \oplus T) \oplus T$$

现在，我们代替 C：

$$\begin{aligned} P &= \mathrm{D}(K_1, C \oplus T) \oplus T \\ &= \mathrm{D}(K_1, [E(K_1, P \oplus T) \oplus T] \oplus T) \oplus T \\ &= \mathrm{D}(K_1, E(K_1, P \oplus T)) \oplus T \\ &= (P \oplus T) \oplus T = P \end{aligned}$$

6.7.3　在扇区上的运算

扇区或者数据单元的明文组织为 128 位的分组，分组标记为 $P_0, P_1, \cdots, P_m$。最后的分组也许是空的，也许含有 1 ~ 127 个位。换句话说，XTS-AES 算法的输入是 m 个 128 位的分组，最后一个分组可能为部分分组。

对于加密和解密，每一个分组都独立处理，过程如图 6.9 所示。当最后一个分组不足 128 位时，唯一的例外就会发生。此时，最后的两个分组要使用密文窃取技术而不使用填充技术来加密/解密。图 6.10 展示了这一方案。P_{m-1}是最后一个满的明文分组，而 P_m是最后的明文分组，其含有 s 位，其中 $1 \leqslant s \leqslant 127$。$C_{m-1}$是最后一个满的密文分组，而 C_m是最后一个密文分组，其含有 s 位。

我们把图 6.9 的加密和解密算法标记为

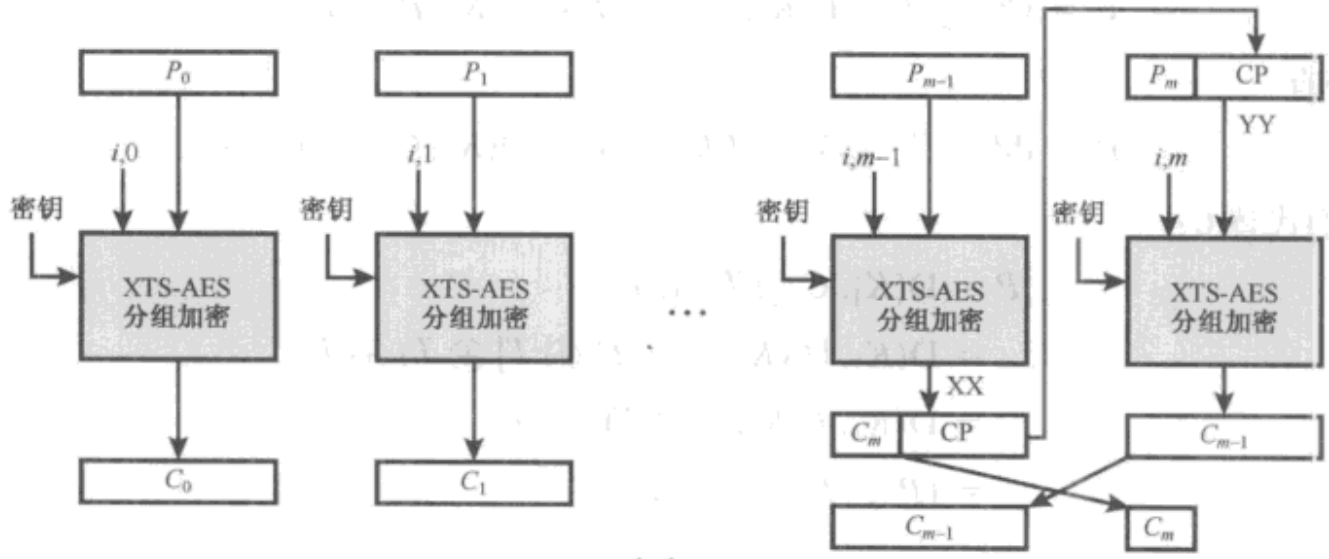
分组加密：XTS-AES-blockEnc(K, P_j, i, j)

分组解密：XTS-AES-blockDec(K, C_j, i, j)

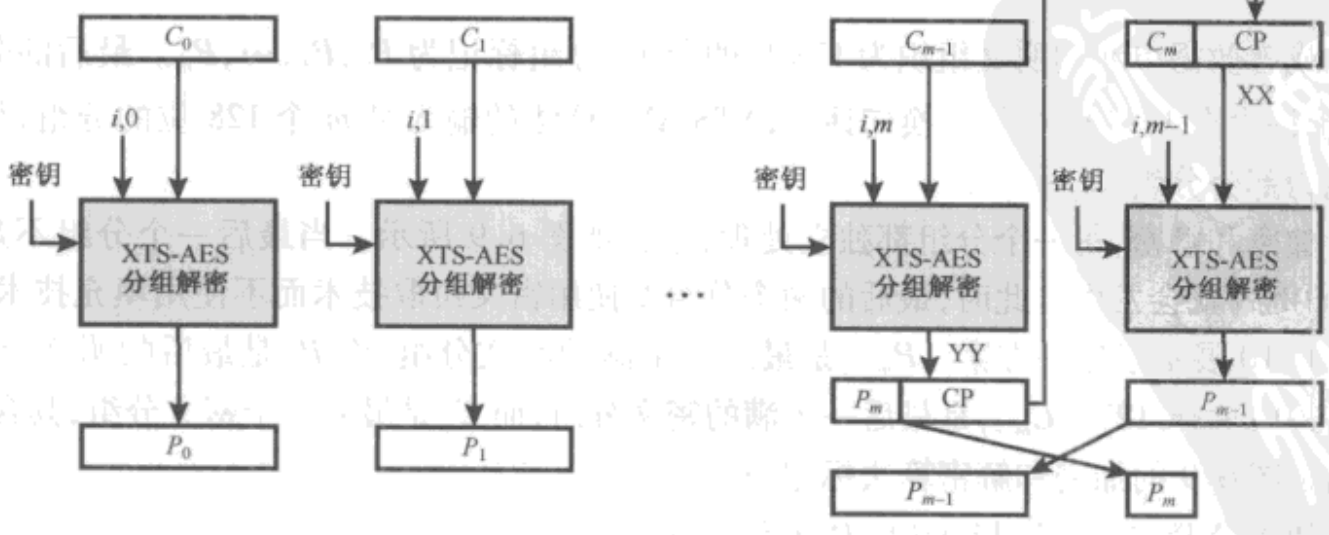
那么，如果最后的分组是空的，XTS-AES 模式如下定义：

XTS-AES 模式,最后分组为空	$C_j = \text{XTS-AES-blockEnc}(K, P_j, i, j) \quad j = 0, \cdots, m-1$
	$P_j = \text{XTS-AES-blockEnc}(K, C_j, i, j) \quad j = 0, \cdots, m-1$
XTS-AES 模式,最后分组含 s 位	$C_j = \text{XTS-AES-blockEnc}(K, P_j, i, j) \quad j = 0, \cdots, m-2$ $\text{XX} = \text{XTS-AES-blockEnc}(K, P_{m-1}, i, m-1)$ $\text{CP} = \text{LSB}_{128-s}(\text{XX})$ $\text{YY} = P_m \parallel \text{CP}$ $\text{C}_{m-1} = \text{XTS-AES-blockEnc}(K, \text{YY}, i, m)$ $C_m = \text{MSB}_s(XX)$
	$P_j = \text{XTS-AES-blockDec}(K, C_j, i, j) \quad j = 0, \cdots, m-2$ $\text{YY} = \text{XTS-AES-blockDec}(K, C_{m-1}, i, m-1)$ $\text{CP} = \text{LSB}_{128-s}(\text{YY})$ $\text{XX} = C_m \parallel \text{CP}$ $P_{m-1} = \text{XTS-AES-blockDec}(K, \text{XX}, i, m)$ $P_m = \text{MSB}_s(\text{YY})$

可以发现,如同 CTR 模式一样,XTS-AES 模式适合并行执行。因为没有链接,多个分组可以同时加密或解密。与 CTR 不同的是,XTS-AES 模式包含一个时变值(参数 i)以及一个计数器(参数 j)。



(a) 加密



(b) 解密

图 6.10　XTS-AES 模式

6.8　推荐读物和网站

推荐网站

- **分组密码的工作模式(Block cipher modes of operation)**:NIST 网页给出了 NIST 推荐的工作模式的全部信息。

6.9　关键术语、思考题和习题

关键术语

分组密码工作模式	密文分组链接模式(CBC)	密文反馈模式(CFB)
密文窃取	计数器模式(CTR)	电码本模式(ECB)
中间相遇攻击	时变值	输出反馈模式(OFB)
三重 DES(3DES)	XTS-AES 模式	

思考题

6.1　什么是三重加密?

6.2　什么是中间相遇攻击?

6.3　在三重加密中用到多少个密钥?

6.4　为什么 3DES 的中间部分采用了解密而不是加密?

6.5　为什么某些分组密码的操作模式仅使用加密算法而其他的模式既使用加密算法又使用解密算法?

习题

6.1　假设你想做一个用 CBC 模式进行分组加密的硬件设备,要求算法强度比 DES 强。3DES 是一个很好的候选算法,但它只在 ECB 模式中定义。现在还没有标准将 3DES 用于 CBC 模式,图 6.11 给出了两种可能,都是遵循 CBC 模式的定义。你将选择哪一个?

(a) 从安全性角度考虑。

(b) 从性能上考虑。

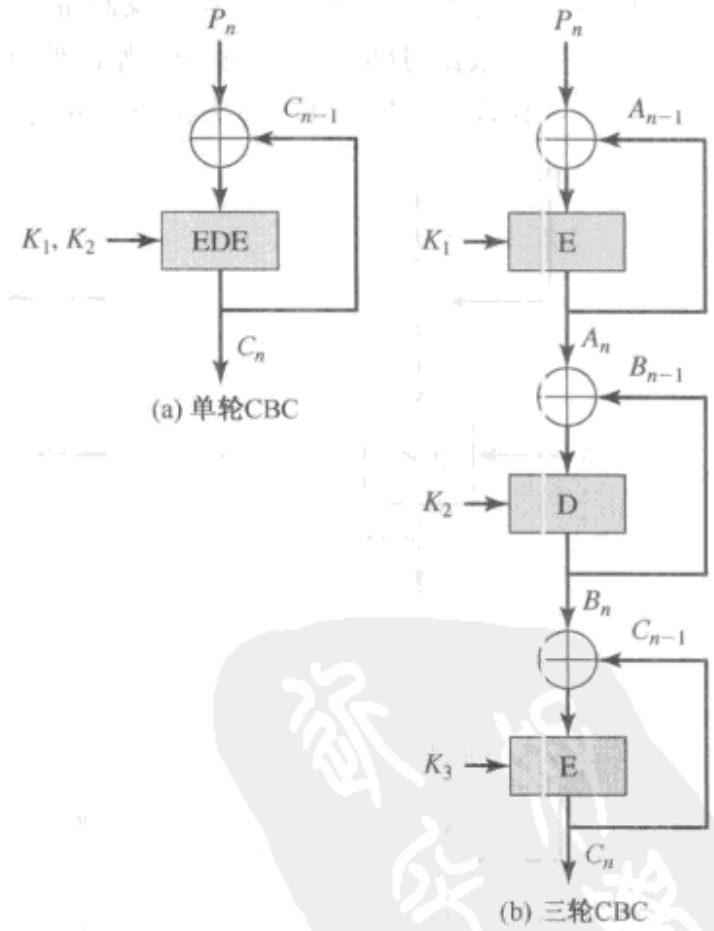


图 6.11　将 3DES 用于 CBC 模式

6.2　仅能够使用 3DES 芯片和一些 XOR 函数,你能对图 6.11 中给出的两种方案进行安全性改进吗?假设仍旧使用两个密钥。

6.3　对 3DES 的 Merkle-Hellman 攻击是这样的:首先令 $A=0$[参见图 6.1(b)],然后,对 K_1 的所有 2^{56} 个取值,找出令 $A=0$ 的明文。请描述算法的剩下部分。

6.4　在 DES 的 ECB 模式中,若在密文的传输过程中,某一块发生了错误,则只有相应的明文分组会有影响。然而,在 CBC 模式中,这种错误具有扩散性。比如,图 6.4 中传输 C_1 时发生的错误将会影响明文分组 P_1 和 P_2。

(a) P_2以后的所有块是否会受到影响?

(b) 假设P_1本来就有一位发生了错误。则这个错误要扩散至多少个密文分组?对接收者解密后的结果有什么影响?

6.5 在CBC模式里有可能对多个明文分组进行并行加密吗?解密呢?

6.6 CBC-Pad是用于RC5密码的一种分组密码工作模式,其可以用于任何分组密码。CBC-Pad能够处理任意长度的明文。密文至多比明文长一个分组的长度。填充的作用是保证输入明文是分组长度的整倍数。假设原始明文是整数个字节。明文在尾部填充1到*bb*字节,这里*bb*为分组的字节长度。填充字节值全都相同并将其值设为填充的字节数。例如,如果有8字节填充,则每个字节为00001000。为什么不允许0字节填充?即如果原始明文是分组大小的整数倍,为什么不省去填充?

6.7 对于ECB、CBC和CFB来说,明文必须是一个或多个完整数据分组(或者对于CFB模式来说,数据段)组成的序列。即对于这三种模式,明文的总位数必须是分组(或段)长度的正整数倍。在需要的时候,一种常用的填充方法是先填1,然后后面跟着填0,也可能没有,直到填满最后一个分组。对于发送者来说,填充每一个消息是好习惯,包括那些最后一个分组已经填满的消息。不需要填充却还要包含一个填充分组的动机是什么?

6.8 在8位的CFB模式中,若传输中一个密文字符发生了一位错,这个错误将传播多远?

6.9 在讨论OFB时,提到如果两个消息在相同的位置有相同的明文分组,有可能恢复出对应的O_i。请给出计算过程。

6.10 在讨论CTR模式时,提到如果用给定的计数器值加密的明文分组内容已知,那么从关联的密文分组很容易求出加密函数的输出。请给出计算过程。

6.11 填充不总是合适的。例如我们希望使用相同的内存缓冲区(明文起始时存于此处)来存储加过密的数据,这时密文必须和原始的明文长度相同。密文窃取模式(CTS)就是满足这样要求的一种工作模式。图6.12(a)为该模式的实现过程。

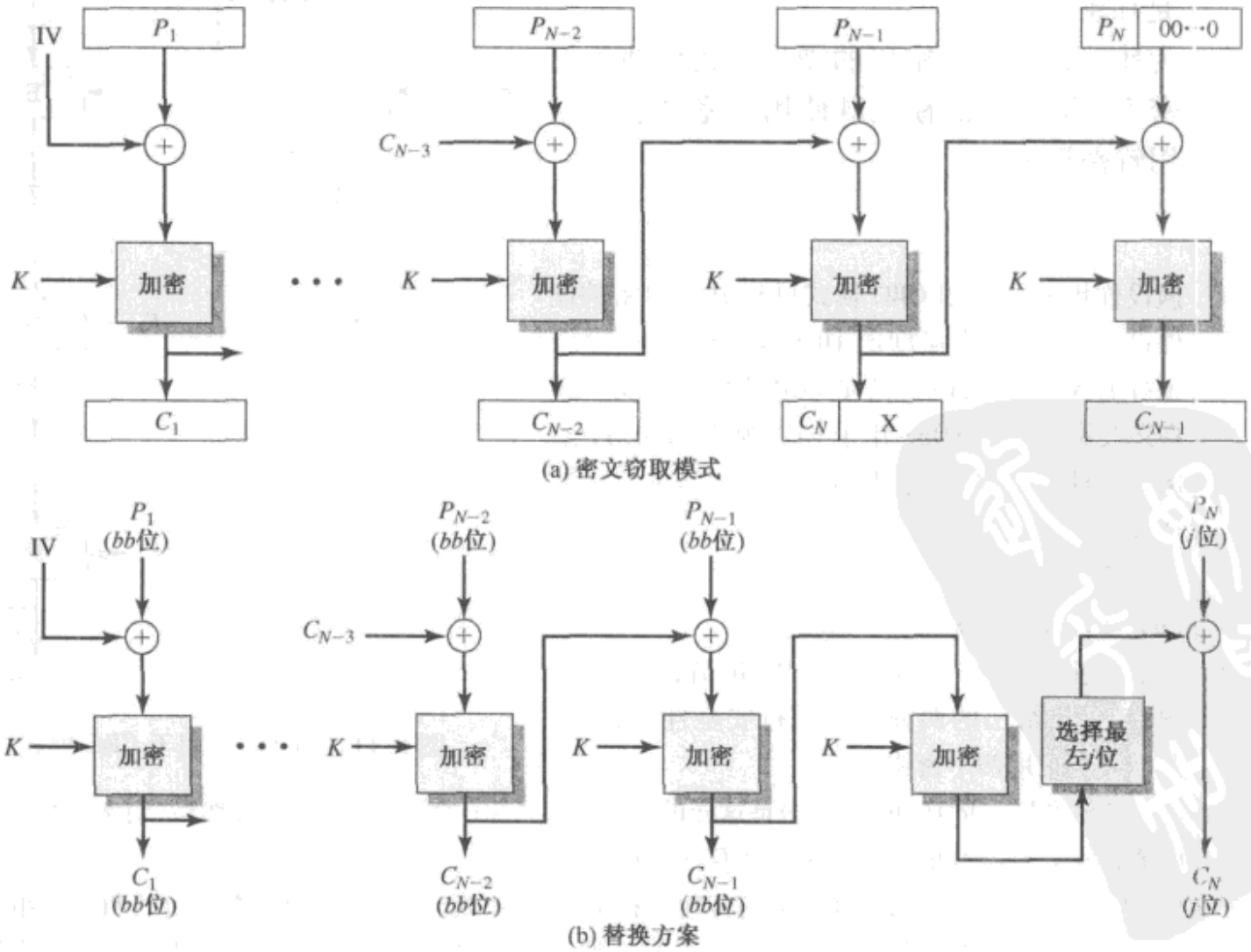


图6.12 明文不是分组长度整数倍的分组密码工作模式

(a) 解释 CTS 是如何工作的。

(b) 描述如何解密 C_{n-1} 和 C_n。

6.12　图 6.12(b)给出了 CTS 的一个替换方案,使得当明文不是分组长度的整数倍时产生的密文长度与明文长度相等。

(a) 解释该算法。

(b) 解释为何 CTS 比图 6.12(b)中的方法更可取。

编程题

6.13　使用如下的一种密码来编写密码分组链接模式的加密和解密软件:模 256 仿射,Hill 模 256,S-DES,DES。S-DES 的测试数据:使用二进制初始向量 1010 1010,使用二进制密钥 01111 11101 加密二进制明文 0000 0001 0010 0011,得出的结果应该是 1111 0100 0000 1011。能够相应地解密。

6.14　使用如下的一种密码来编写 4 位密文反馈模式的加密和解密软件:模 256 加,模 256 仿射,S-DES;或者编写 8 位密文反馈模式的加密和解密软件,使用如下密码:2 × 2 Hill 模 256。S-DES 的测试数据:使用二进制初始向量 1010 1011,使用二进制密钥 01111 11101 加密二进制明文 0001 0010 0011 0100,得出的结果应该是 1110 1100 1111 1010。能够相应地解密。

6.15　使用如下的一种密码来编写计数器模式的加密和解密软件:模 256 仿射,Hill 模 256,S-DES。S-DES 的测试数据:使用计数器从 0000 0000 开始,使用二进制密钥 01111 11101 加密二进制明文 0000 0001 0000 0010 0000 0100,得出的结果应该是 0011 1000 0100 1111 0011 0010。能够相应地解密。

6.16　实现对于 3 轮 S-DES 的差分攻击。

第7章　伪随机数的产生和流密码

7.1　伪随机数发生器的原则
- 7.1.1　随机数的使用
- 7.1.2　TRNG、PRNG 和 PRF
- 7.1.3　PRNG 的要求
- 7.1.4　算法设计

7.2　伪随机数发生器
- 7.2.1　线性同余发生器
- 7.2.2　BBS 伪随机数发生器

7.3　使用分组密码的伪随机数发生器
- 7.3.1　使用分组密码模式运算的 PRNG
- 7.3.2　ANSI X9.17 PRNG

7.4　流密码

7.5　RC4
- 7.5.1　S 的初始化
- 7.5.2　流的产生
- 7.5.3　RC4 的强度

7.6　真随机数发生器
- 7.6.1　熵源
- 7.6.2　偏离

7.7　推荐读物和网站

7.8　关键术语、思考题和习题

The comparatively late rise of the theory of probability shows how hard it is to grasp, and the many paradoxes show clearly that we, as humans, lack a well grounded intuition in this matter.

In probability theory there is a great deal of art in setting up the model, in solving the problem, and in applying the results back to the real world actions that will follow.

—The Art of Probability, Richard Hamming

要　点

- 能够应用到大量密码函数的一种功能是随机或伪随机数的产生。对这个功能的要求是产生的数据流必须不能预测。
- 流密码是对称密码算法，从明文输入流逐位或逐字节产生密文输出。使用最为广泛的此类密码是 RC4。

一个重要的密码函数是具有强密码学意义的伪随机数发生器。伪随机数发生器(PRNG)在许多密码和安全应用中有使用。本章我们先看一下 PRNG 的一些基本原则，并把这些原则和真随机数发生器[①](TRNG)进行比较。接下来我们研究一些常见的 PRNG，包括基于对称分组密码的 PRNG。

本章还将讨论基于 PRNG 的对称流密码。接着本章研究最为重要的流密码 RC4。最后，我们研究真随机数发生器。

7.1　随机数产生的原则

在网络安全的各种应用里，随机数在加密算法中扮演重要的角色。在本节中，我们简短回顾一下随机数在密码和网络安全中的使用情况，重点是伪随机数的产生原则。

① 术语的注解。一些标准，特别的是 NIST 以及 ANSI，认为 TRNG 是一种非确定性的随机数发生器(NRNG)，而 PRNG 是一种确定性的随机数发生器(DRNG)。

7.1.1 随机数的使用

大量的基于密码学的网络安全算法和协议都使用了二进制随机数。例如：

- 密钥分发和相互认证方案，例如第 14 章和第 15 章讨论的那些方案。在那些方案中，通信的双方通过交换信息共同合作分发密钥和/或互相认证对方。许多时候需要使用时变值进行握手以防止重放攻击。时变值使用随机数可以防止攻击者判断或者猜测出时变值。
- 会话密钥的产生。本书内我们将看到大量的协议，需要为对称加密产生一个短时间内使用的秘密密钥。这个密钥通常称为会话密钥。
- RSA 公钥加密算法中密钥的产生（详见第 9 章）。
- 用于对称流密码加密的位流的产生。

这些应用对随机数序列产生提出了两个不同且未必兼容的要求：随机性和不可预测性。

随机性

一般认为随机序列应有良好的统计特性。下面是两个评价标准：

- **分布均匀性**：序列中的位分布应是均匀的，即 0 和 1 出现的频率大约相等。
- **独立性**：序列中任何子序列不能由其他子序列推导出。

尽管有一些成熟的测试来判断位序列是否符合某个特定的分布，比如均匀性分布，然而还没有某种方法可以表明一个序列的独立性好。相反，却有很多测试可以证明一个序列不具有独立性。通常的策略是多进行一些测试，直至可认为它的独立性是足够强的。

密码学中算法的设计经常使用这种“似乎随机”的数序列。例如对第 9 章将要讨论的 RSA 公开密钥加密方案的一个基本要求就是产生素数的能力。一般来说，判断一个给定的大数是否是素数是很难的。采用穷举测试的方法需要把 N 除以 $N^{1/2}$ 以内的奇数。如果 N 是 10^{150} 这种数量级的，这种大数在公钥密码中并非罕见，用穷举测试它是否为素数则超出了人和计算机的实际能力。然而，有许多有效的算法，可以随机地选择一些随机数来进行相对简单的运算，只要随机数足够多［但是远比 $(10^{150})^{1/2}$ 小］，就几乎可判定这个大数是否为素数。这类方法称为不确定性方法，在算法设计中经常用到。本质上说，若某问题的精确求解很难或很耗时，可以采用简单、简短、不确定性方法来取得某种可信程度的解。

不可预测性

在相互认证、会话密钥生成以及流密码之类的应用中，对随机数的要求不能仅仅停留在序列是统计随机的，而要求随机数序列是不可预测的。所谓的“真随机数序列”是每个数都统计独立于其他数，因而不可预测。不过，马上就可以看到，真正的随机数序列很少用；一般看上去随机的随机数序列是由算法产生的。此时，必须要注意敌手不能够从先前的随机数推导出后面的随机数。

7.1.2 TRNG、PRNG 和 PRF

密码应用大多使用算法来生成随机数。这些算法是确定的，所以产生的序列并非统计随机的。不过，要是算法好的话，产生的序列可以经受住随机性检测。这样的数一般称为**伪随机数**。

你也许对这种把确定性算法生成的序列用做随机数的观念感到有点不安。但是，只要我们不追求所谓哲学上的完美性，这种方法的确有效。有位概率论专家这样说［HAMM91］：

为了实用的目的,我们被迫接受"相对随机"这一令人尴尬的概念,这意味着就提到的使用而言,我们没有理由认为它们不是随机的(如理论经常要求的那样)。这对于纯粹主义者来说有点主观而且不令人愉快,但却是统计学家所喜欢的。当他们做随机采样时,他们希望所使用的任何结果和全部样本空间具有相同的性质,而这也是理论上的性质。

图 7.1 把真随机数发生器和两种伪随机数发生器进行了对比。TRNG 把一个很随机的源作为输入,这个源常称为熵源,我们将会在 7.6 节中讨论这种随机源。本质上,熵源是从计算机的物理环境抽取的,可能包括键盘敲击时间模式、磁盘的电活动、鼠标移动、系统时间的瞬时值等。源或诸源的组合作为算法的输入,产生随机的二元输出。TRNG 也许仅仅是把模拟信号源转化为二元输出。TRNG 也可能会做额外的处理以消除源里的不平衡,这在 7.6 节会讨论。

相反,PRNG 取一个固定值,称为种子,作为输入,用一个确定性的算法产生位输出序列。通常,如图 7.1 所示有反馈路径,由此算法的部分结果反馈作为输入,而其他部分作为输出位。需要注意的是,输出位流仅由输入值决定,所以知道算法和种子的敌手可以重现整个位流。

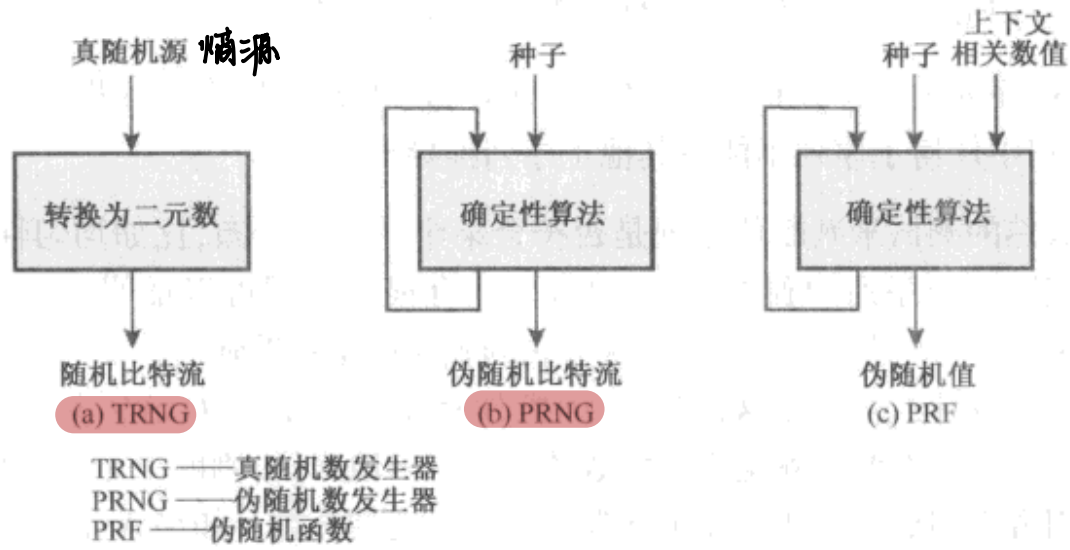


图 7.1 随机和伪随机数发生器

图 7.1 显示了两款基于应用的不同形式的 PRNG。

- **伪随机数发生器**:用于产生不限长位流的算法称为 PRNG。不限长位流的通常应用是作为对称流密码的输入,如同 7.4 节所讨论的那样,也可参见图 3.1(a)。
- **伪随机函数(PRF)**:PRF 用于产生固定长度的伪随机位串。对称加密密钥和时变值就是例子。通常 PRF 的输入为种子加上一些上下文相关的特定值,如用户 ID 或应用 ID。本书中将看到许多 PRF 的例子,尤其是第 16 章和第 17 章。

除了产生位的数量外,PRNG 和 PRF 之间没有差别。这两个应用可以使用相同的算法。两者都需要种子,都必须具有随机性和不可预测性。而且,PRNG 应用可能也需要使用上下文相关的输入。接下来,我们对这两个应用不做区别。

随机数源:真随机数很难获得

7.1.3 对 PRNG 的要求

当 PRNG 或 PRF 用于密码学应用时,基本的要求是不知道种子的敌手不能决定伪随机串。例如,如果伪随机位流用于流密码,那么知道伪随机位流将导致敌手从密文恢复明文。类似地,我们希望保护 PRF 的输出值。对于后者,考虑如下场景。128 位的种子加上一些上下文相关的值用于产生 128 位的密钥,该值接着又用于对称加密。在通常的环境下,128 位的密钥是能抗暴力攻击

的。然而,如果 PRF 不能产生足够随机的 128 位输出值,敌手有可能降低穷举空间并成功地进行暴力攻击。

对 PRNG 和 PRF 输出保密性的这个通用要求导致对随机性、不可预测性以及种子特性的特定要求。我们依次来看一下。

随机性

就随机性而言,对 PRNG 的要求是生成的位流尽管是确定性的,但要显示随机。没有单个的测试可以判定一个 PRNG 生成的数据具有随机的特性。所能做的就是对 PRNG 进行一系列的测试。如果 PRNG 在多次测试的基础上展现了随机性,那么可以认为它满足随机性的要求。NIST SP800-22(密码学应用中的随机数和伪随机数发生器统计测试工具集)规定这些测试应该建立如下三个特征。

- **均匀性**:在产生随机或伪随机位序列的任何点,0 或 1 出现的量大约相等,即每个出现的概率恰好为 1/2。0(或 1)出现的次数为 $n/2$,其中 n 为序列的长度。
- **可伸缩性**:用于序列测试的任何测试也可以用于从序列中抽取的子序列的测试。如果序列是随机的,那么任何从中抽取的子序列也应该是随机的。因此,任何抽取的子序列都应该通过随机性测试。
- **一致性**:对于所有初始值(种子),发生器的行为必须具有一致性。基于单个种子产生的输出测试 PRNG,或者基于单个物理源的输出测试 TRNG,都是不充分的。

SP800-22 列出了 15 种单独的随机性测试。对这些测试的理解需要统计分析的知识,所以此处我们并不准备做技术上的描述。相反,为了使测试不太枯燥,我们在下面列出三个测试以及每个测试的目的。

- **频率测试**:这是最基本的测试,任何测试工具集都必须包含。这个测试的目的是判断序列中 0 和 1 的个数是否和真随机序列的期望值大约相同。
- **游程测试**:本测试的焦点是序列里游程的总量。所谓游程,是指相同位构成的不间断序列,其前后的位取与游程位不同的值。游程测试的目的是判断各种长度的 0 和 1 游程的数量是否符合随机序列的期望值。
- **Maurer 通用统计测试**:该测试的重点是匹配模式(与压缩序列长度相关的一种度量)间的位数。测试的目的是检测序列是否能大幅度压缩而不损失信息。一个能大幅度压缩的序列被认为非随机的。

不可预测性

随机数流应该具有两种形式的不可预测性。

- **前向不可预测性**:如果不知道种子,那么不管知道序列中前面的多少位,都无法预测序列中的下一位。
- **后向不可预测性**:从产生的任何值都不能推断出种子值。很明显,种子和该种子产生的任意值之间应该没有相关性;序列的每个元素应该显得像独立随机事件的结果(其概率为 1/2)。

用于随机性测试的测试集也提供了不可预测性测试。如果产生的位流看上去随机,那么不可能从前面的任何位预测某一位或位序列。类似地,如果位序列看上去随机,那么也不可能从位序列推导出种子,即随机序列和固定值(种子)没有相关性。

种子的要求

在密码学应用时,作为 PRNG 输入的种子必须安全。因为 PRNG 是一个确定性算法,如果敌手能推导出种子,那么就可以决定输出。因此,种子必须不可预测。事实上,种子本身必须是随机数或伪随机数。

典型地,如图 7.2 所示,种子是由 TRNG 产生的。这是 SP800-90 推荐的方案。读者也许会问如果可以用 TRNG,为何还需要用 PRNG 呢? 如果应用是流密码,那么 TRNG 是不实际的。发送者需要产生和明文一样长的位密钥流,并把密钥流和密文安全地传送给接收者。如果使用 PRNG,发送者仅需设法给接收方安全传送流密码的密钥,通常是 54 位或 128 位。

即使是 PRF 应用,即只需要产生有限数量的位,也希望用 TRNG 为 PRF 提供种子,并使用 PRF 的输出而不是直接使用 TRNG。如同 7.6 节解释的,TRNG 可能会产生不平衡的二元序列。PRF 具有使 TRNG 的输出更随机化的效果,从而消除不平衡性。

最后,用于产生真随机数的机制在速率方面也许不能跟得上需要随机位的应用。

7.1.4 算法设计

PRNG 多年来一直是密码学上的研究主题,为此产生了大量的算法。这些算法可以大体分为两类。

- **特意构造的算法**:这些算法是为了产生伪随机位流而特意或专门设计的。许多算法在许多 PRNG 应用中使用。其他算法则特意为了流密码而设计。后者最重要的例子是 7.5 节描述的 RC4。
- **基于现存密码算法的算法**:密码学算法具有随机化输入的效果。的确,这是此类算法的要求。例如,如果对称分组密码产生的密文具有某些规则性,这就有助于密码分析。因此,密码学算法在 PRNG 中起核心作用。三大类密码学算法常用来产生 PRNG:
  - **对称分组密码**:这个方法将在 7.3 节讨论。
  - **非对称密码**:用于非对称密码的数论概念也可以用于 PRNG;这种方法将在第 10 章讨论。
  - **Hash 函数和消息认证码**:这种方法将在第 12 章讨论。

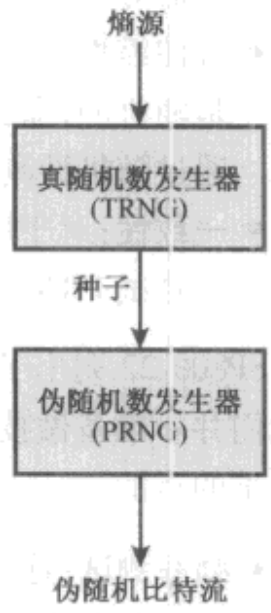


图 7.2 输入到 PRNG 的种子的产生

这些方法的任意一种都可以产生密码学意义上强的 PRNG。特意构造的算法可以由操作系统提供用于通用目的。对于那些已经使用密码学算法来加密或认证的应用来说,把这些代码重用于 PRNG 也是有意义的。因此,所有这些方法都是通用的。

7.2 伪随机数发生器

本节我们要研究用于 PRNG 的两种算法。

7.2.1 线性同余发生器

一个广泛使用的产生伪随机数的方法是由 Lehmer 首先提出的算法[LEHM51],即线性同余方法。算法有以下 4 个参数:

m	模	$m>0$
a	乘数	$0<a<m$
c	增量	$0\leqslant c<m$
X_0	初始值或种子	$0\leqslant X_0<m$

随机数序列$\{X_n\}$按下面的迭代式获得：

$$X_{n+1}=(aX_n+c)\bmod m$$

若 m,a,c 和 X_0都是整数,那么这种方法将产生一个整数序列,且每个整数都满足 $0\leqslant X_n<m$。

要想设计一个好的随机数发生器,对 a,c 和 m 的选择至关重要。例如,假设 $a=c=1$。产生的序列明显不行。假设 $a=7$,$c=0$,$m=32$ 且 $X_0=1$,产生序列是$\{7,17,23,1,7,\cdots\}$,这也明显不行,就是说 32 个可能的值,它只用了 4 个,即随机数序列的周期为 4。如果把 a 改成 5,序列就成了$\{5,25,29,17,21,9,13,1,5,\cdots\}$,周期为 8。

m 一般都很大,因此可产生长连串的不同随机数,一个常见的评价标准是 m 与给定计算机可表示的最大非负整数的值接近相等,那么,对于 32 位机,m 可以选择接近或等于 2^{31} 的值。

参考文献[PARK88a]中提出了评价随机数发生器的三个标准:

T_1:生成函数应是全周期的,即函数在重复之前应该产生 0 ~ m 之间的所有数。

T_2:产生的序列应显得随机。

T_3:生成函数可以用 32 位运算器方便地实现。 $m=2^{31}-1$

选择合适的 a、c 和 m,可以同时满足这三点。对于条件 T_1,可以证明,若 p 是素数且 $c=0$,则 a 的某些取值可以使生成函数的周期为 $m-1$,只是不能得到 0 这个数。对于 32 位算术运算,$2^{31}-1$ 就是一个常用的素数,这时产生的函数为 即满足 { 当a是以自己一个原根，即满足条件(1)

$$X_{n+1}=(aX_n)\bmod(2^{31}-1)$$

a 的可能取值超过 20 亿个,但满足上述条件的只有其中很小的一部分。a 取值为 $7^5=16\,807$ 时可以满足上述条件,这个数最开始是在 IBM 360 系列计算机中使用的[LEWI69],这个发生器使用得最广泛,比起其他 PRNG,其经过了更为全面的测试,尤其适合于统计和仿真方面[JAIN91]。

若乘数和模选择恰当,用线性同余算法产生的随机数序列的统计特性几乎与从集合 1,2,…,$m-1$ 里随机抽取的序列相当(无放回抽取)。但是除了初始值 X_0之外,算法没有任何其余的东西是随机的,一旦 X_0选定了,后续产生的随机数也就确定了,这一点对密码分析有帮助。

如果敌手知道了上述算法及参数(例如 $a=7^5$、$c=0$、$m=2^{31}-1$),只要他知道一个随机数,就可获得后续的所有序列,即使他只知道是采用了线性同余算法,那么只根据随机数序列中的一小部分就可以找到这些参数。设敌手可确定 X_0,X_1,X_2和 X_3,则有

$$\begin{cases}X_1=(aX_0+c)\bmod m\\ X_2=(aX_1+c)\bmod m\\ X_3=(aX_2+c)\bmod m\end{cases}$$

由这三个等式可求解出 a、c 和 m。

因此,尽管这种办法用做伪随机数发生器有很多优点,但是最理想的还是要使产生的序列不可重新产生,这样的话敌手才不能由部分序列求得以后的序列。有几种办法可以达到这个目标。例如,参考文献[BRIG79]建议使用内部系统时钟来修正随机数流,一种方法是每隔 N 个数就以时钟值对 m 取模作为新的种子来产生新的序列。还有一种方法是直接将随机数加上时钟值再对 m 取模。 更新X_0

7.2.2 BBS 发生器

BBS 发生器是产生安全伪随机数的普遍方法,BBS 是三位设计者名字的首字母:Blum、Blum 和 Shub[BLUM86]。它也许是特意构造算法中密码强度有最强公开证明的一个了。产生过程如下:首先,选择两个大素数 p 和 q,且要求

$$p \equiv q \equiv 3 (\text{mod } 4)$$

该符号在第 4 章中有完整解释,表示($p \bmod 4$) = ($q \bmod 4$) = 3。例如,素数 7 和 11 满足 $7 \equiv 11 \equiv 3 \pmod 4$。令 $n = p \times q$。接着,选择一个随机数 s,且要求 s 与 n 互素,即 p 或 q 都不是 s 的因子。然后 BBS 按下列算法产生位 B_i 序列:

$$X_0 = s^2 \bmod n$$
$$\textbf{for}\ i = 1\ \textbf{to}\ \infty$$
$$X_i = (X_{i-1})^2 \bmod n$$
$$B_i = X_i \bmod 2$$

因此每个循环都取最低有效位。表 7.1 给出了 BBS 运算的一个例子。这里,$n = 192\ 649 = 383 \times 503$,种子 $s = 101\ 355$。基于大整数分解的困难性

BBS 被称为密码安全伪随机位发生器(CSPRBG)。它能经受住续位测试,在参考文献[MENE97]中,续位测试定义如下:"称某伪随机位发生器可通过续位测试,若不存在多项式时间复杂度的算法①,对于某输出序列的最初 k 位输入,可以以超过 1/2 的概率预测出第($k+1$)位"。换句话说,给定序列的最开始 k 位,没有有效算法可以让你以超过 1/2 的概率确定下一位是 1 还是 0。所以对于实际应用,这个序列是不可预测的。BBS 的安全性是基于对 n 的因子分解的困难性上的,即给定 n,我们不能确定它的素因子 p 和 q。

表 7.1 BBS 伪随机数发生器的一个例子

i	X_i	B_i	i	X_i	B_i
0	20749		11	137922	0
1	143135	1	12	123175	1
2	177671	1	13	8630	0
3	97048	0	14	114386	0
4	89992	0	15	14863	1
5	174051	1	16	133015	1
6	80649	1	17	106065	1
7	45663	1	18	45870	0
8	69442	0	19	137171	1
9	186894	0	20	48060	0
10	177046	0			

7.3 使用分组密码的伪随机数产生

构造 PRNG 的常用方法是使用对称分组密码作为 PRNG 机制的核心。对于任意的明文分组,对称分组密码产生一个明显随机的分组输出,即密文里没有规则性或模式可用于推导明文。因此,对称分组密码很适合用来构造伪随机数发生器。

如果使用一个成熟的标准分组密码,如 DES 或 AES,那么 PRNG 的安全特性就能得到保证。而且,许多应用已经在使用 DES 或 AES,所以包含分组密码作为 PRNG 的一部分是简单直接的。

① 阶为 k 的多项式时间算法,是指算法的运行时间由阶为 k 的多项式限定。

7.3.1　使用分组密码运算模式的 PRNG

两种使用分组密码构建 PRNG 的方法获得了广泛的接受:CTR 模式和 OFB 模式。SP800-90、ANSI 标准 X9.82 以及 RFC 4086 都推荐了 CTR 模式。X9.82 和 RFC 4086 推荐了 OFB 模式。

图 7.3 展示了两种方法。在每一种情况下,种子由两部分组成:加密密钥值以及每产生一个随机数分组后都要更新的 V 值。因此,对于 AES,种子由 128 位的密钥和 128 位的 V 值构成。当为 CTR 模式时,V 的值每加密一次后增加 1。当为 OFB 模式时,V 的值更新为前一个 PRNG 分组的值。两种情况下,一次产生一个伪随机位分组(例如,对于 AES,一次产生 128 位的 PRNG 位)。

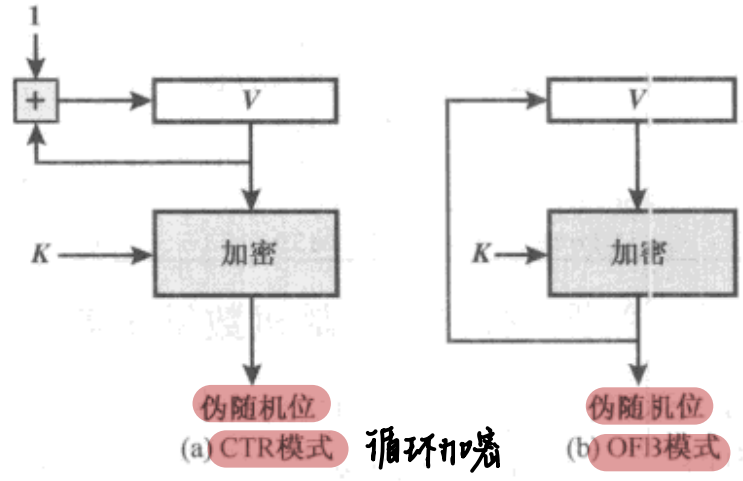


图 7.3　基于分组密码的 PRNG 机制

用于 PRNG 的 CTR 算法总结如下:

```
while (len (temp) < requested_number_of_bits) do
        V = (V + 1) mod 2^128.
        output_block = E(Key, V)
        temp = temp || ouput_block
```

OFB 算法可以总结如下:

```
while (len (temp) < requested_number_of_bits) do
        V = E(Key, V)
        temp = temp || V
```

为了了解这两个 PRNG 的性能,考虑如下的小实验。256 位的随机位序列是从 random. org 处获得的,通过使用三个相互之间可以调节的无线电来获取空气噪声。这 256 位形成种子,如下分配:

密钥	cfb0ef3108d49cc4562d5810b0a9af60
V	4c89af496176b728ed1e2ea8ba27f5a4

256 位的种子里 1 的总数为 124,或者是 48%,很接近理想情况下的 50%。

对于 OFB 的 PRNG,表 7.2 显示了前 8 个输出分组(1024 位),带有两个粗略的安全度量指标。第二列显示了每个 128 位分组里 1 的分数。这对应于 NIST 的一个测试。结果表明输出为 0 和 1 的数量大约相等。第三列显示的是相邻分组匹配位的分数。如果这个数值偏离 0.5 比较大,这就表明分组间有相关,这是一个安全弱点,其结果表明没有相关性。

表 7.2　使用 OFB 的 PRNG 的例子结果

输出分组	位 1 所占分数	和前一分组匹配位所占分数
1786f4c7ff6e291dbdfdd90ec3453176	0.57	—
5e17b22b14677a4d66890f87565eae64	0.51	0.52
fd18284ac82251dfb3aa62c326cd46cc	0.47	0.54
c8e545198a758ef5dd86b41946389bd5	0.50	0.44
fe7bae0e23019542962e2c52d215a2e3	0.47	0.48
14fdf5ec99469598ae0379472803accd	0.49	0.52
6aeca972e5a3ef17bd1a1b775fc8b929	0.57	0.48
f7e97badf359d128f00d9b4ae323db64	0.55	0.45

表7.3显示了CTR模式下使用相同的密钥和V值的结果。结果再一次让人满意。

表7.3 使用CTR的PRNG的例子结果

输出分组	位1所占分数	和前一分组匹配位所占分数
1786f4c7ff6e291dbdfdd90ec3453176	0.57	—
60809669a3e092a01b463472fdcae420	0.41	0.41
d4e6e170b46b0573eedf88ee39bff33d	0.59	0.45
5f8fcfc5deca18ea246785d7fadc76f8	0.59	0.52
90e63ed27bb07868c753545bdd57ee28	0.53	0.52
0125856fdf4a17f747c7833695c52235	0.50	0.47
f4be2d179b0f2548fd748c8fc7c81990	0.51	0.48
1151fc48f90eebac658a3911515c3c66	0.47	0.45

7.3.2 ANSI X9.17 伪随机数发生器

ANSI X9.17中所给出的伪随机数发生器是密码学意义上最强的伪随机数发生器之一。许多应用使用了这种方法,包括金融安全应用和PGP(详见第18章)。

图7.4说明了算法流程,它使用了3DES来加密。构成成分如下所述。

- **输入**:用两个伪随机数输入来驱动发生器。一个是64位的数,代表当前的日期和时间,每产生一个伪随机数它均要更新。另一个是64位的种子,可以被初始化为任意值,并在生成随机数的过程中被更新。
- **密钥**:发生器使用3个3DES加密模块。3个加密模块都使用一个相同的56位密钥对,这个密钥对必须保密,且只在产生随机数时才使用。
- **输出**:输出包括64位的伪随机数和64位的种子。

定义以下变量:

DT_i:算法第i轮开始时的日期/时间值。

V_i:算法第i轮开始时的种子值。

R_i:算法第i轮所产生的伪随机数。

K_1, K_2:各阶段算法所用的DES密钥。

则有

$$R_i = \mathrm{EDE}([K_1, K_2], [V_i \oplus \mathrm{EDE}([K_1, K_2], DT_i)])$$
$$V_{i+1} = \mathrm{EDE}([K_1, K_2], [R_i \oplus \mathrm{EDE}([K_1, K_2], DT_i)])$$

其中$\mathrm{EDE}([K_1, K_2], X)$代表加密-解密-加密序列加密$X$,使用的算法为两个密钥的3DES。

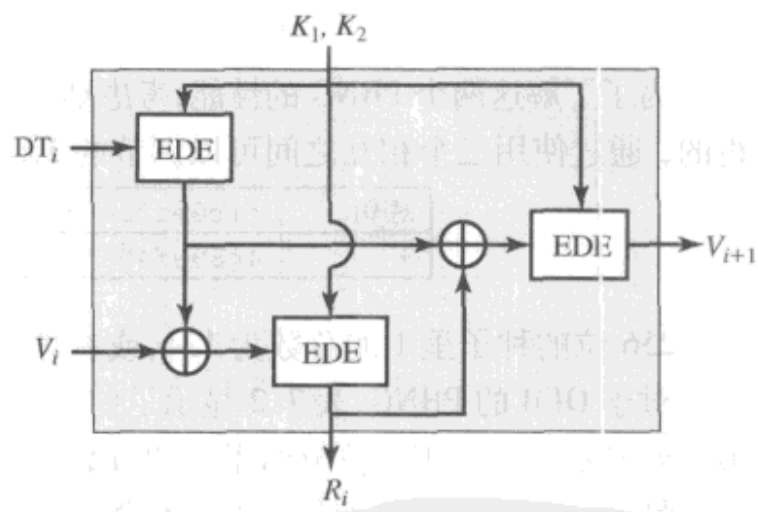


图7.4 ANSI X9.17伪随机数发生器

该方法的密码强度来自几个方面,包括112位密钥和3个EDE共计9次DES加密。整个方案的输入是两个伪随机数,即日期和时间值与一个由发生器产生的种子,且其区别于发生器产生的伪随机数。所以敌手要分析的东西太多了,即使他知道了一个R_i,也不能由R_i推导出V_{i+1},因为产生V_{i+1}时又用了一次EDE运算。

7.4 流密码

一个典型的流密码每次加密一个字节的明文,当然流密码也可被设计为每次操作1位或者大于一个字节的单元。图7.5给出了一个典型的流密码的结构图。在该结构中密钥输入到一个伪随机数(位)发生器,该伪随机数发生器产生一串随机的8位数。发生器的输出密钥流和明文流的

每一个字节进行对位异或运算，得到一个字节。例如，如果发生器产生的下一字节为 01101100，而下一明文字节为 11001100，则得出的密文字节为

```
  11001100   明文
⊕ 01101100   密钥流
  10100000   密文
```

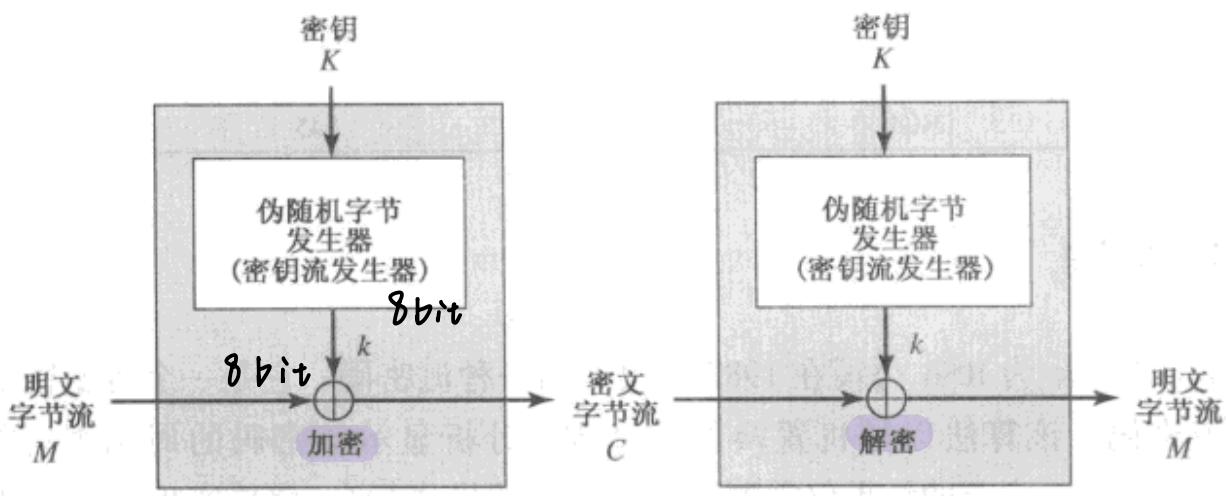


图 7.5　流密码结构图

解密需要使用相同的伪随机序列：

```
  10100000   明文
⊕ 01101100   密钥流
  11001100   密文
```

流密码类似于第 2 章讨论的“一次一密”，不同的是“一次一密”使用的是真正的随机数流，而流密码使用的是伪随机数流。

[KUMA97]列出了设计流密码需要考虑的主要因素。

（1）加密序列的周期要长。伪随机数发生器使用的函数产生确定性的位流，该位流最终将出现重复。重复的周期越长，密码分析的难度就越大。这与讨论 Vigenère 密码的考虑在本质上是一致的，即密钥越长密码分析越困难。

（2）密钥流应该尽可能地接近于真正随机数流的特征。例如，1 和 0 的个数应近似相等。若密钥流为字节流，则所有 256 种可能的字节的值出现的频率应近似相等。密钥流的随机特性越好，则密文越随机，密码分析就越困难。

（3）注意图 7.5 中伪随机数发生器的输出受输入密钥 K 的调节。为了防止穷举攻击，密钥应该足够长，即用于分组密码的考虑在此同样适用。因此，从目前的软硬件技术发展来看，至少应当保证密钥长度不小于 128 位。

通过设计合适的伪随机数发生器，当密钥长度相当时，流密码可以提供和分组密码一样的安全性。例如本章介绍的 RC4，仅仅数行代码就可实现。根据[RESC01]给出的数据，表 7.4 列出了 RC4 同三种常见的对称分组密码执行速度的对比。分组密码的优点是可以重复使用密钥，然而如果用流密码对两个明文加密且使用相同的密钥，则密码分析就会相当容易[DAWS96]。如果对两个密文流进行异或，那么得出的结果就是两个原始明文的异或。如果明文仅仅是文本串、信用卡号，或者其他已知特征的字节流，则密码分析极易成功。

对于需要对数据流进行加密/解密的应用，比如在数据通信信道或者网页浏览连接上，流密码就是很好的解决方案。而对于处理成块的数据，比如文件传输，E-mail 和数据库，分组密码则更为适用。当然，在实际中两种类型的密码都可使用于各种应用。

流密码可以用任何具有强密码学意义的 PRNG 来构造,比如 7.2 节和 7.3 节讨论的例子。下一节里,我们将研究使用 PRNG 的流密码,该 PRNG 是专门设计用于流密码的。

表 7.4 在 Pentium II 上对称密码的速度对比

密 码	密钥长度	速度(Mbps)
DES	56	9
3DES	168	3
RC2	可变	0.9
RC4	可变	45

7.5 RC4 算法

RC4 是 Ron Rivest 为 RSA 公司在 1987 年设计的一种流密码。它是一个可变密钥长度、面向字节操作的流密码。该算法以随机置换作为基础。分析显示该密码的周期很可能大于 10^{100} [ROBS95a]。每输出一字节的结果仅需要 8 ~ 16 条机器操作指令,软件实现的该密码运行很快。RC4 应用很广,例如,它被用于为网络浏览器和服务器间通信而制定的 SSL/TLS(安全套接字协议/传输层安全协议)标准中,它也应用于作为 IEEE 802.11 无线局域网标准一部分的 WEP(Wired Equivalent Privacy)协议和新的 WiFi 受保护访问协议(WPA)中。在当时,RC4 作为 RSA 公司的商业机密并没有公开。直到 1994 年 9 月,RC4 算法才通过 Cypherpunks 匿名邮件列表匿名地公布于 Internet 上。

RC4 算法非常简单,易于描述:用 1 ~ 256 个字节(8 ~ 2048 位)的可变长度密钥初始化一个 256 个字节的状态向量 S,S 的元素记为 $S[0], S[1], \cdots, S[255]$,从始至终置换后的 S 包含从 0 到 255 的所有 8 位数。对于加密和解密,字节 k(参见图 7.5)是从 S 的 255 个元素中按一种系统化的方式选出的一个元素生成的。每生成一个 k 的值,S 中的元素个体就被重新置换一次。

7.5.1 初始化 S

开始时,S 中元素的值按升序被置为 0 ~ 255,即 $S[0]=0, S[1]=1, \cdots, S[255]=255$。同时建立一个临时向量 T。如果密钥 K 的长度为 256 字节,则将 K 赋给 T。否则若密钥长度为 keylen 个字节(keylen < 256),则将 K 的值赋给 T 的前 keylen 个元素,并循环重复用 K 的值赋给 T 剩下的元素,直到 T 的所有元素都被赋值。这些预操作可被概括为

```
/* Initialization */
for i = 0 to 255 do
S[i] = i;
T[i] = K[i mod keylen];
```

然后用 T 产生 S 的初始置换,从 $S[0]$ 到 $S[255]$,对每个 $S[i]$,根据由 $T[i]$ 确定的方案,将 $S[i]$ 置换为 S 中的另一字节:

```
/* Initial Permutation of S */
j = 0;
for i = 0 to 255 do
    j = (j + S[i] + T[i]) mod 256;
Swap (S[i], S[j]);
```

因为对 S 的操作仅是交换,所以唯一的改变就是置换。S 仍然包含所有值为 0 ~ 255 的元素。

7.5.2　密钥流的生成

向量 S 一旦完成初始化，输入密钥就不再被使用。密钥流的生成过程是，从 $S[0]$ 到 $S[255]$，对每个 $S[i]$，根据 S 的当前配置，将 $S[i]$ 与 S 中的另一字节置换。当 $S[255]$ 完成置换后，操作继续重复从 $S[0]$ 开始：

```
/* Stream Generation */
i, j = 0;
while (true)
  i = (i + 1) mod 256;
  j = (j + S[i]) mod 256;
Swap (S[i], S[j]);
t = (S[i] + S[j]) mod 256;
k = S[t];
```

加密中，将 k 的值与明文的下一字节异或；解密中，将 k 的值与密文的下一字节异或。

图 7.6 总结了 RC4 的逻辑结构。

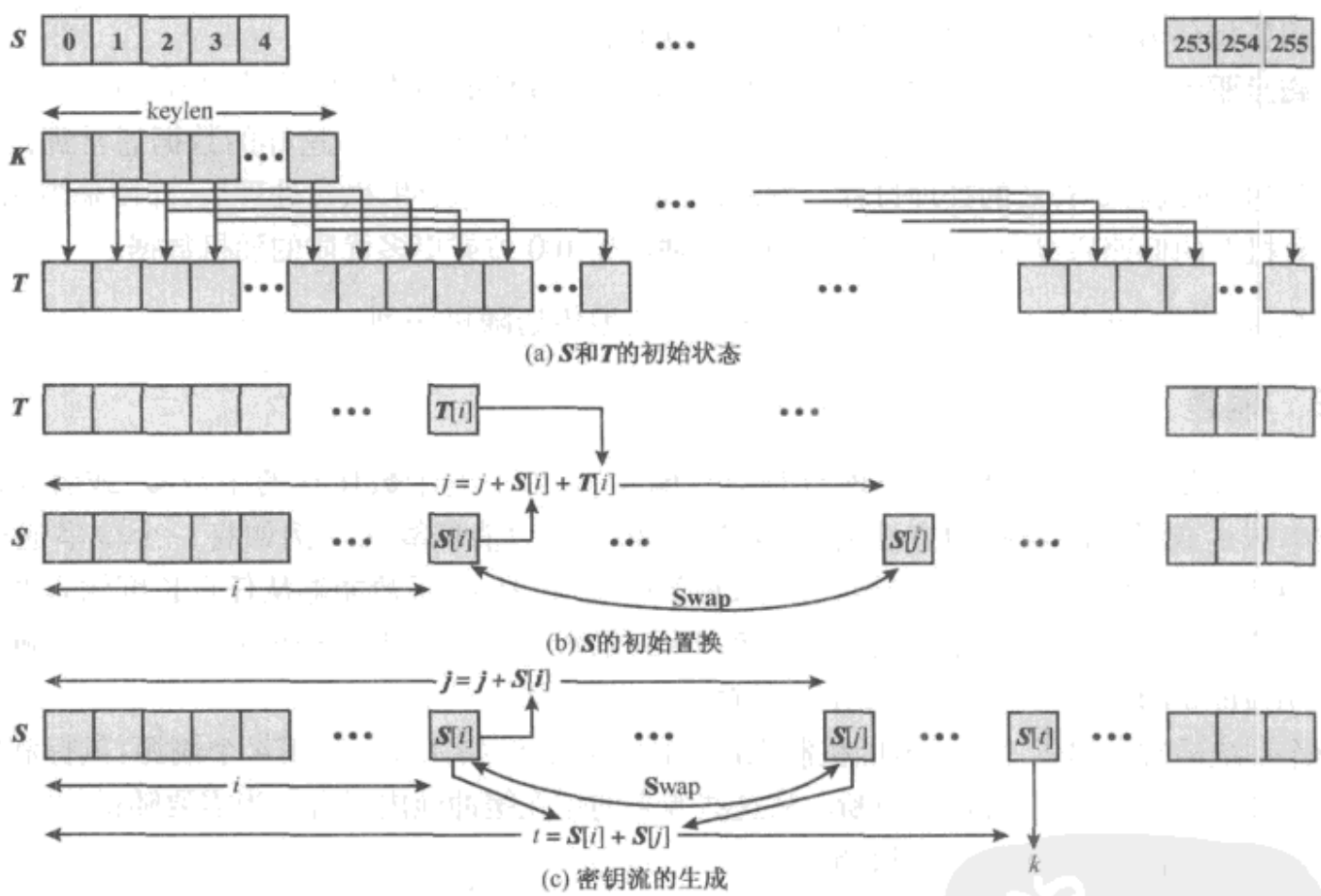


图 7.6　RC4

7.5.3　RC4 的强度

关于分析 RC4 的攻击方法有许多公开发表的文献（如[KNUD98]、[MIST98]、[FLUH00]、[MANT01]）。但是当密钥长度很大时，比如 128 位，没有哪种攻击方法有效。值得注意的是[FLUH01]中的报告，作者指出用于为 802.11 无线局域网络提供机密性的 WEP 协议，易于受到一种特殊的攻击方法的攻击。从本质上讲，这个问题并不在于 RC4 本身，而是作为 RC4 输入的密钥的产生途径有漏洞。这种特殊的攻击方法不适用于其他使用 RC4 的应用，通过修改 WEP 中密钥产生的途径也可以修复这种攻击。这个问题恰恰说明设计一个安全系统的困难性不仅包括密码函数，还包括协议如何正确地使用这些密码函数。

7.6 真随机数发生器

7.6.1 熵源

真随机数发生器(TRNG)使用不可预测源来产生随机数。大多数是通过测量不可预测的自然过程来实现的,例如电离辐射的脉冲检测、气体放电管、漏电电容等。Intel 开发出一种芯片通过放大无驱动电阻的电压对热噪声进行采样[JUN99]。LavaRnd 是一个开放的项目,目标是通过廉价的相机、开源的代码和廉价的硬件来产生真随机数。该系统使用密封的饱和式 CCD 作为混沌源来产生种子,然后用软件将种子加工成各种形式的真随机数。

RFC 4086 列出了如下可能的随机源,这些都很容易用于计算机来产生真随机序列。

- **声音/图像输入**:许多计算机具有对现实世界模拟信号进行数据化的输入设备,这些信号有来自麦克风的声音或来自照相机的声像输入。如果声音数字化设备没有音源插入或者照相机没打开盖,那么来自这些设备的输入本质上是热噪声。如果系统具有足够的增益来检测任何信号,这种输入可以提供高质量的随机位。
- **磁盘驱动**:由于紊乱的空气波动,磁盘驱动在转动时有很小的随机波动[JAKO98]。加上一个底层磁盘时间寻找仪器,就能产生一系列包含随机性的度量值。这样的数据通常都是高度相关的,所以需要有效的处理过程。然而,十年前的实验就表明,做过处理后,即使是当时低速计算机上的低速磁盘驱动,也能容易地每分钟产生 100 位或更多优质的随机数据。

在线服务网(random. org)可以通过互联网安全地传送随机序列。

7.6.2 偏差

一个真随机数发生器的输出应该有时会有偏差,比如 1 的个数比 0 的个数多,或者反之。将一个位串减少或者消除偏差的方法有很多,这些方法称为消偏算法。例如将一个位串通过 MD5 或者 SHA-1 等 Hash 函数进行处理(详见第 11 章)。这些 Hash 函数能够从任意长度输入产生 n 位输出。为了消偏,可以将 m 位的诸分组输入到 Hash 函数,$m \geqslant n$。RFC 4086 推荐收集多个硬件源来的输入,用 Hash 函数进行修正以产生随机的输出。

操作系统通常会提供一个内嵌的机制来产生随机数。例如,Linux 利用 4 个熵源:鼠标和键盘的活动,磁盘的 I/O 操作以及特定的中断。从这些源来的位在缓冲池内组合。当需要随机位时,从中读取适量的位,传给 SHA-1 函数[GUTT06]。

7.7 推荐读物和网站

至于伪随机数产生,最好的读物应该是[KNUT98]。参考文献[BRIG79]则详细解释了线性递归算法,而线性递归算法有些时候可以替代标准的线性同余算法。参考文献[ZENG91]则将不同类型的伪随机数产生方法用在产生 Vernam 密码的变长密钥上,并对其优劣进行了评估。

[RITT91]对 PRNG 做了全面总结,并给出了详尽的参考文献。参考文献[MENE97]讨论了安全伪随机数发生器。RFC 4086 也很不错,不过它的重点在于实际实现问题[EAST05],该 RFC 也讨论了大量消偏的技术。[KELS98]探讨了安全 PRNG 技术及攻击 PRNG 的密码学分析。SP800-90[BARK07]讨论了 NIST 推荐的诸多 PRNG。SP800-22[RUKH08]定义和讨论了 NIST 推荐的 15 个随机性的统计测试。

[KUMA97]就流密码的设计原则进行了很好很长的讨论。[RUEP92]也很不错,很数学化。[ROBS95a]研究了有关流密码设计的诸多问题,有趣且值得研究。

BARK07 Barker, E. , and Kelsey, J. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*. NIST SP 800-90, March 2007.

BRIG79 Bright, H. , and Enison, R. "Quasi-Random Number Sequences from Long-Period TLP Generator with Remarks on Application to Cryptography." *Computing Surveys*, December 1979.

EAST05 Eastlake, D. jSchiller, J. jand Crocker, S. *Randomness Requirements for Security*. RFC 4086, June 2005.

KELS98 Kelsey, J. ; Schneier, B. ; and Hall, C. "Cryptanalytic Attacks on Pseudorandom Number Generators." *Proceedings*, *Fast Software Encryption*, 1998. http://www. schneier. com/paper-prngs. html

KNUT98 Knuth, D. *The Art of Computer Programming*, *Volume 2: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1998.

KUMA97 Kumar, I. *Cryptology*. Laguna Hills, CA: Aegean Park Press, 1997.

MENE97 Menezes, A. ; Oorshcot, P. ; and Vanstone, S. *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1997.

ROBS95a Robshaw, M. *Stream Ciphers*. RSA Laboratories Technical Report TR-701, July 1995.

RITT91 Ritter, T. "The Efficient Generation of Cryptographic Confusion Sequences." *Cryptologia*, vol. 15 no. 2, 1991. www. ciphersbyritter. com/ARTS/CRNG2ART. HTM

RUEP92 Rueppel, T. "Stream Ciphers." In[SIMM92].

RUKH08 Rukhin, A. , et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. NIST SP 800-22, August 2008.

SIMM92 Simmons, G. , ed. *Contemporary Cryptology: The Science of Information Integrity*. Piscataway, NJ: IEEE Press, 1992.

ZENG91 Zeng. K. ; Yang, C, ; Wei, D. ; and Rao, T. "Pseudorandom Bit Generators in Stream-Cipher Cryptography." *Computer*, February 1991.

推荐网站

- **NIST Random Number Generation Technical Working Group**:包括 NIST 关于密码应用的 PRNG 的文档和试验,并且还有许多有用的链接。
- **NIST Random Number Generation Cryptographic Toolkit**:NIST 的另外一个包含文档和链接的站点。
- **LavaRnd**:LavaRnd 是使用混沌源来产生真随机数的开源项目。该网站还提供随机数的基本背景知识。
- **Quantum Random Numbers**:可以抽空访问一下量子随机数。
- **RandomNumber. org**:也是一个随机数资源。
- **A Million Random Digits**:RAND 公司整理。

7.8 关键术语、思考题和习题

关键术语

后向不可预测性
Blum, Blum, Shub 发生器
消偏
熵源
前向不可预测性
密钥流
伪随机函数(PRF)
伪随机数发生器(PRNG)
随机性
RC4
种子
线性同余发生器
流密码
偏差
真随机数发生器(TRNG)
不可预测性

思考题

7.1 统计随机性和不可预测性的区别是什么?
7.2 请列出设计流密码的重要考虑。
7.3 为什么流密码重用密钥不好?
7.4 RC4 用到了什么原语操作?

习题

7.1 如果将产生伪随机数的线性同余的加法项设为 0,即

$$X_{n+1} = (aX_n) \bmod m$$

则可证明若 m 是素数,且给定的 a 使得算法获得最大周期 $m-1$,那么只要 k 小于 m 且 $m-1$ 和 k 互素,则 a^k 也使得算法获得最大周期 $m-1$。以参数 $X_0 = 1, m = 31, a^k = 3, 3^2, 3^3, 3^4$ 为例产生序列来说明上述论断。

7.2 (a) 下述伪随机数发生器可获得的最大周期是多少?

$$X_{n+1} = (aX_n) \bmod 2^4$$

(b) 这时 a 为多少?

(c) 对种子有什么要求?

7.3 你也许对线性同余算法中选用的模数是 $m = 2^{31} - 1$ 而不是 2^{31} 感到奇怪,因为后者表示起来简单一些,且执行模运算也要简单一些,不过一般认为 $2^k - 1$ 比 2^k 要好,这是为什么?

7.4 在线性同余算法中,选择的参数即使可提供全周期,也不一定能提供很好的随机性。例如,有下面两个伪随机数发生器:

$$X_{n+1} = (6X_n) \bmod 13$$

$$X_{n+1} = (7X_n) \bmod 13$$

写出随机数序列以证明它们都是全周期的。哪一个的随机性更好一些呢?

7.5 对伪随机数的任何使用,比如加密、仿真或是统计设计,盲目地相信计算机系统中的伪随机数发生器是危险的。参考文献[PARK88]发现了当代许多教材和软件包中使用的伪随机数发生器有缺陷。下面这道题可使你对你的系统进行测试。

测试基于 Ernesto Cesaro 的一个定理([KNUT98]有证明),他证明了两个随机选择的整数的最大公因子是 1 的可能性是 $6/\pi^2$。根据这个理论及统计学方法可以用程序来计算 π 的值。主程序要调用三个子程序:一个是从系统中找到一些伪随机数发生器,并用它们产生一些伪随机数;一个是计算两个整数的最大公因子的 Euclid 算法;一个是计算平方根。如果后面两个子程序没有现成的,你

需要自己写。主程序应该有一个循环且循环次数很大,以给出上述概率估计。这样,你就可以用这种简单的方法来估计 π 的值了。

若结果接近于 3.14,那么祝贺你。如果不是,结果可能偏小,通常是在 2.7 附近。为什么得到的结果与 π 的值大相径庭呢?

7.6　假设有真随机数发生器,产生位串中的每一位为 0 或 1 的概率相同,并且各位之间互不相关。也就是说,这些位来自相同的独立分布。然而,该位串是有偏的。取 1 的概率为 $0.5+\partial$,取 0 的概率为 $0.5-\partial$,这里 $0<\partial<0.5$。一种简单的消偏算法如下:将序列当做非重叠对进行检查,丢弃所有的 00 和 11 组合,并将每个 01 替换为 0,每个 10 替换为 1。

(a) 在原序列中每对组合发生的概率是多少?

(b) 修改后的序列中 0 和 1 的概率各是多少?

(c) 为了产生 x 位的输出,需要多少位的输入?

(d) 假设算法使用重叠对而不是非重叠对,即第 1 位的输出取决于第 1、2 位的输入,第 2 位的输出取决于第 2、3 位的输入,以此类推。输出位串会怎样?

7.7　另一种消偏算法是将位串视为连续非重叠的 n 位分组,然后输出每个分组的奇偶性,即如果一组包括奇数个 1,则输出为 1;否则输出为 0。

(a) 将该操作表示为布尔函数的形式。

(b) 假设同上题取 1 的概率为 $0.5+\partial$,如果每组包括 2 位,那么输出为 1 的概率是多少?

(c) 如果每组包括 4 位,那么输出为 1 的概率是多少?

(d) 总结出如果每组包括 n 位时输出为 1 的概率。

7.8　哪个 RC4 的密钥可以使 S 在初始化后不变?即在 S 的初始化置换以后,S 的项依次等于 0~255。

7.9　RC4 有保密的内部状态,即 S 的所有值的一个置换以及两个索引 i 和 j。

(a) 用直接的方案来存储内部状态,需要多少位?

(b) 假设从状态表示了多少信息的角度考虑问题。此时,需要判决有多少种不同的状态,然后取以 2 为底的对数求出这个表示了多少位的信息。运用这个方法,需要多少位来表示状态?

7.10　Alice 和 Bob 同意用基于 RC4 加密的电邮进行私有通信,但他们不想每次传输都需要一个新的密钥。Alice 和 Bob 秘密协商了一个 128 位的密钥 k。为了加密由位构成的消息 m,遵循如下程序:

(1) 选择随机的 80 位值 v。

(2) 产生密文 $c=\mathrm{RC4}(v\|k)\oplus m$。

(3) 发送位串 $(v\|c)$。

(a) 假设 Alice 用上述程序给 Bob 发送消息 m。描述 Bob 如何用 k 从 $(v\|c)$ 恢复出消息 m。

(b) 如果一个敌手能够观察到 Alice 和 Bob 间传输的若干个 $(v_1\|c_1),(v_2\|c_2),\cdots$ 值,他/她如何判定两条消息使用了相同的密钥流?

(c) 在一个密钥使用两次之前,Alice 可以大约发送多少消息?使用附录 11A 给出的生日悖论的结果[参见式(11.7)]。

(d) 关于密钥 k 的生命周期,这意味着什么(即用 k 可加密消息的数量)?

PDG

第二部分　公钥密码

第 8 章　数论入门
第 9 章　公钥密码学与 RSA
第 10 章　密钥管理和其他公钥密码体制

第8章　数论入门

8.1 素数

8.2 费马定理和欧拉定理

8.2.1 费马定理

8.2.2 欧拉函数

8.2.3 欧拉定理

8.3 素性测试

8.3.1 Miller-Rabin 算法

8.3.2 一个确定的素数判定算法

8.3.3 素数的分布

8.4 中国剩余定理

8.5 离散对数

8.5.1 模 n 的整数幂

8.5.2 模算术的对数

8.5.3 离散对数的计算

8.6 推荐读物和网站

8.7 关键术语、思考题和习题

The Devil said to Daniel Webster: "Set me a task I can't carry out, and I'll give you anything in the world you ask for."

Daniel Webster: "Fair enough. Prove that for n greater than 2, the equation $a^n + b^n = c^n$ *has no non-trivial solution in the integers."*

They agreed on a three – day period for the labor, and the Devil disappeared.

At the end of three days, the Devil presented himself, haggard, jumpy, biting his lip. Daniel Webster said to him, "Well, how did you do at my task? Did you prove the theorem?"

"Eh? No . . . no, I haven't proved it."

"Then I can have whatever I ask for? Money? The Presidency?"

"What? Oh, that—of course. But listen! If we could just prove the following two lemmas--"

—The Mathematical Magpie, Clifton Fadiman

要　点

- 素数是一种整数，在整除意义下，它只能被自身（正负）和 1 整除。素数在数论和密码学里都扮演重要的角色。
- 在公钥密码里起重要作用的两个定理是费马定理和欧拉定理。
- 许多密码算法的一个重要要求是能够选择一个大的素数。开发一个有效的算法来判定一个随机选择的整数是否为素数，是一个正在研究的课题。
- 离散对数是许多公钥算法的基础。离散对数和普通的对数类似，但是在模算术上进行运算。

数论中的许多概念在设计公钥密码算法时是必不可少的。本章讨论其他章节中所用到的一些概念，熟悉这些内容的读者可略过本章。读者在学习这一章之前，应该先复习一下 4.1 节至 4.3 节。

如第 4 章，本章包含了大量的例子，每一个都用带阴影的方框加以突出显示。

8.1　素数[①]

数论主要关心的是素数，实际上所有关于数论的书都是围绕这一主题来写的（如[CRAN01]，[RIBE96]）。本节简要介绍本书中所要用到的有关数论知识。

整数 $p > 1$ 是素数当且仅当它只有因子[②] ± 1 和 $\pm p$。素数在数论和本章所讨论的各方法中起着重要作用。表 8.1 列出了 2000 以内的素数。请注意素数的分布，特别要注意每 100 个数里素数的个数。

表 8.1　2000 以内的素数

2	101	211	307	401	503	601	701	809	907	1009	1103	1201	1301	1409	1511	1601	1709	1801	1901
3	103	223	311	409	509	607	709	811	911	1013	1109	1213	1303	1423	1523	1607	1721	1811	1907
5	107	227	313	419	521	613	719	821	919	1019	1117	1217	1307	1427	1531	1609	1723	1823	1913
7	109	229	317	421	523	617	727	823	929	1021	1123	1223	1319	1429	1543	1613	1733	1831	1931
11	113	233	331	431	541	619	733	827	937	1031	1129	1229	1321	1433	1549	1619	1741	1847	1933
13	127	239	337	433	547	631	739	829	941	1033	1151	1231	1327	1439	1553	1621	1747	1861	1949
17	131	241	347	439	557	641	743	839	947	1039	1153	1237	1361	1447	1559	1627	1753	1867	1951
19	137	251	349	443	563	643	751	853	953	1049	1163	1249	1367	1451	1567	1637	1759	1871	1973
23	139	257	353	449	569	647	757	857	967	1051	1171	1259	1373	1453	1571	1657	1777	1873	1979
29	149	263	359	457	571	653	761	859	971	1061	1181	1277	1381	1459	1579	1663	1783	1877	1987
31	151	269	367	461	577	659	769	863	977	1063	1187	1279	1399	1471	1583	1667	1787	1879	1993
37	157	271	373	463	587	661	773	877	983	1069	1193	1283		1481	1597	1669	1789	1889	1997
41	163	277	379	467	593	673	787	881	991	1087		1289		1483		1693			1999
43	167	281	383	479	599	677	797	883	997	1091		1291		1487		1697			
47	173	283	389	487		683		887		1093		1297		1489		1699			
53	179	293	397	491		691				1097				1493					
59	181			499										1499					
61	191																		
67	193																		
71	197																		
73	199																		
79																			
83																			
89																			
97																			

素数的表示方法：

任意整数 $a > 1$ 都可以唯一地因子分解为

$$a = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_t^{a_t} \tag{8.1}$$

其中 $p_1, p_2, \cdots, p_t$ 均是素数，$p_1 < p_2 < \cdots < p_t$，且所有的 a_i 都是正整数。这就是算术基本定理。任何有关数论的教材都会含有该定理的证明。

$$91 = 7 \times 13$$
$$3600 = 2^4 \times 3^2 \times 5^2$$
$$11011 = 7 \times 11^2 \times 13$$

下面介绍另一种有用的表示方法。设 P 是所有素数的集合，则任意正整数 a 可唯一地表示为

① 若无特别说明，则本节只讨论非负整数，使用负整数不会有本质上的不同。

② 第 4 章里说 a 是 b 的一个因子如果 a 除 b 没有余数。

$$a = \prod_{p \in P} p^{a_p} \quad \text{其中每一个 } a_p \geq 0$$

上式右边是所有素数之积。对某一整数 a,其大多数指数 a_p 为 0。

我们可以通过列出上述公式中所有非零指数来唯一地表示正整数。

整数 12 可用 $\{a_2 = 2, a_3 = 1\}$ 来表示
整数 18 可用 $\{a_2 = 1, a_3 = 2\}$ 来表示
整数 91 可用 $\{a_7 = 1, a_{13} = 1\}$ 来表示

2 素数乘积=

两数相乘即是指数对应相加。设 $a = \prod_{p \in P} p^{a_p}$, $b = \prod_{p \in P} p^{b_p}$,定义 $k = ab$。我们知道整数 k 可以表示为素数方幂的乘积:$k = \prod_{p \in P} p^{k_p}$。可以推出对于所有的 $p \in P$,有 $k_p = a_p + b_p$ 成立。

$k = 12 \times 18 = (2^2 \times 3) \times (2 \times 3^2) = 216$
$k_2 = 2 + 1 = 3;\ k_3 = 1 + 2 = 3$
$216 = 2^3 \times 3^3 = 8 \times 27$

从素因子的角度看,a 整除 b,即 $(a \mid b)$ 意味着什么呢?由于任意形为 p^n 的整数只能被 p^j 整除(其中 $j \leq k$)。所以有:

$$a = \prod_{p \in P} p^{a_p}, b = \prod_{p \in P} p^{b_p}$$

如果 $a \mid b$,那么对任意的 $p \in P$ 有 $a_p \leq b_p$。

$a = 12; b = 36; 12 \mid 36$
$12 = 2^2 \times 3; 36 = 2^2 \times 3^2$
$a_2 = 2 = b_2$
$a_3 = 1 \leq 2 = b_3$
因此不等式 $a_p \leq b_p$ 对所有素数成立

若将整数表示为素数之积,则很容易确定两个正整数的最大公因子①。

$300 = 2^2 \times 3^1 \times 5^2$
$18 = 2^1 \times 3^2$
$\gcd(18, 300) = 2^1 \times 3^1 \times 5^0 = 6$

下列关系式总是成立:

如果 $k = \gcd(a, b)$,那么 $k_p = \min(a_p, b_p)$,对任意的 $p \in P$

确定一个大数的素因子不是一件容易的事,因此利用上述关系式并不能直接得出计算最大公因子的实用方法。

① 若 c 能整除 a 和 b,且 a 和 b 的任何公因子均是 c 的因子,则称 c 为整数 a 和 b 的最大公因子,记为 $\gcd(a, b)$。参见第 4 章。

8.2　费马定理和欧拉定理

费马定理和欧拉定理在公钥密码学中占有重要地位。

8.2.1　费马定理[①]

费马定理可如下描述：若 p 是素数，a 是正整数且不能被 p 整除，则

$$a^{p-1} \equiv 1 \pmod{p} \tag{8.2}$$

证明　考虑小于 p 的正整数集合 $\{1,2,\cdots,p-1\}$，用 a 乘以集合中所有元素并对 p 取模，则得到集合 $X = \{a \bmod p, 2a \bmod p, \cdots, (p-1)a \bmod p\}$。因为 p 不能整除 a，所以 X 的元素都不等于 0，而且各元素互不相等。为了说明这一点，假设 $ja \equiv ka \pmod{p}$，其中 $1 \leqslant j < k \leqslant p-1$，因为 a 和 p 互素[②]，所以根据式(4.3)可以将等式两边的 a 消去[参见式(4.3)]，则推出 $j \equiv k \pmod{p}$。而这最后的等式是不可能的，因为 j 和 k 都是小于 p 的正整数。因此 X 的 $p-1$ 个元素都是正整数且互不相等。所以说 X 和 $\{1,2,\cdots,p-1\}$ 构成相同，只是元素顺序不同。将两个集合的所有元素分别相乘，并对结果模 p，有

$$a \times 2a \times \cdots \times (p-1)a \equiv [(1 \times 2 \times \cdots \times (p-1)] \pmod{p}$$

$$a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$$

因为 $(p-1)!$ 和 p 互素，根据式(4.5)，我们可以消去 $(p-1)!$ 这一项，得到式(8.2)。证毕。

$$\begin{aligned}
&a = 7, p = 19\\
&7^2 = 49 \equiv 11 \pmod{19}\\
&7^4 \equiv 121 \equiv 7 \pmod{19}\\
&7^8 \equiv 49 \equiv 11 \pmod{19}\\
&7^{16} \equiv 121 \equiv 7 \pmod{19}\\
&a^{p-1} = 7^{18} \equiv 7^{16} \times 7^2 \equiv 7 \times 11 \equiv 1 \pmod{19}
\end{aligned}$$

费马定理的另一种有用的表示形式是：若 p 是素数且 a 是任意正整数，则

$$a^p \equiv a \pmod{p} \tag{8.3}$$

因为没有前提为a

请注意定理的第一种形式[参见式(8.2)]要求 a 与 p 互素，而后一种没有这个要求。

$$\begin{aligned}
&p = 5, a = 3, a^p = 3^5 = 243 \equiv 3 \pmod{5} = a \pmod{p}\\
&p = 5, a = 10, a^p = 10^5 = 100\,000 \equiv 10 \pmod{5} \equiv 0 \pmod{5} = a \pmod{p}
\end{aligned}$$

8.2.2　欧拉函数

在给出欧拉定理之前，我们需要引入数论中一个非常重要的概念，即欧拉函数 $\phi(n)$，它是指

① 该定理有时称为费马小定理。

② 参见第 4 章。两个数若无公共素因子，即它们的唯一公因子为 1，则称这两个数是互素的，也就是说，若两个数的最大公因子为 1，则它们是互素的。

小于 n 且与 n 互素的正整数的个数。习惯上,$\phi(1)=1$。

确定 $\phi(37)$ 和 $\phi(35)$ 的值。
因为 37 是素数,所以从 1 ~ 36 的所有正整数均与 37 互素,因此 $\phi(37)=36$。
要计算 $\phi(35)$,我们列出所有小于 35 且与 35 互素的正整数如下:
1,2,3,4,6,8,9,11,12,13,16,17,18,19,22,23,24,26,27,29,31,32,33,34
由上可知共有 24 个数,因此 $\phi(35)=24$。

表 8.2 列出了 $\phi(n)$ 前 30 项的值,虽然 $\phi(1)$ 的值无意义,但我们仍将它定义为 1。
显然,对素数 p

$$\phi(p) = p - 1$$

假设有两个素数 p 和 q,$p \neq q$,那么对 $n=pq$

$$\phi(n) = \phi(pq) = \phi(p) \times \phi(q) = (p-1) \times (q-1)$$

为了证明 $\phi(n)=\phi(p)\times\phi(q)$,考虑集合 $\{1,\cdots,(pq-1)\}$,不与 n 互素的集合是 $\{p,2p,\cdots,(q-1)p\}$ 和 $\{q,2q,\cdots,(p-1)q\}$,所以

证明: $\phi(n) = (pq-1) - [(q-1)+(p-1)]$
$= pq - (p+q) + 1$
$= (p-1) \times (q-1)$
$= \phi(p) \times \phi(q)$

$\phi(21)=\phi(3)\times\phi(7)=(3-1)\times(7-1)=2\times6=12$
其中这 12 个整数是 $\{1,2,4,5,8,10,11,13,16,17,19,20\}$

表 8.2 欧拉函数 $\phi(n)$ 的前 30 项的值

n	$\phi(n)$	n	$\phi(n)$	n	$\phi(n)$
1	1	11	10	21	12
2	1	12	4	22	10
3	2	13	12	23	22
4	2	14	6	24	8
5	4	15	8	25	20
6	2	16	8	26	12
7	6	17	16	27	18
8	4	18	6	28	12
9	6	19	18	29	28
10	4	20	8	30	8

8.2.3 欧拉定理

欧拉定理说明,对任意互素的 a 和 n,有

$$a^{\phi(n)} \equiv 1 (\mathrm{mod}\, n) \tag{8.4}$$

证明 若 n 是素数,根据 $\phi(n)=(n-1)$ 和费马定理,则式(8.4)成立,但实际上式(8.4)对任意整数 n 都是成立的。请注意 $\phi(n)$ 是指小于 n 且与 n 互素的正整数的个数,我们考虑这些整数所组成的集合:

$$R = \{x_1, x_2, \cdots, x_{\phi(n)}\}$$

即每一个元素都有 $\gcd(x_i, n) = 1$。用 a 与 R 中的每个元素模 n 相乘：

$$S = \{(ax_1 \bmod n), (ax_2 \bmod n), \cdots, (ax_{\phi(n)} \bmod n)\}$$

S 是 R 的一个排列①，因为

（1）a 与 n 互素，且 x_i 与 n 互素，所以 ax_i 必与 n 互素，这样 S 中所有元素均小于 n 且与 n 互素。

（2）S 中没有重复元素，参见式(4.5)。若 $ax_i \bmod n = ax_j \bmod n$，则 $x_i = x_j$。

所以集合 S 是 R 的一个置换。因此

$$\prod_{i=1}^{\phi(n)} (ax_i \bmod n) = \prod_{i=1}^{\phi(n)} x_i$$

$$\prod_{i=1}^{\phi(n)} ax_i \equiv \prod_{i=1}^{\phi(n)} x_i \pmod n$$

$$a^{\phi(n)} \times \left[\prod_{i=1}^{\phi(n)} x_i\right] \equiv \prod_{i=1}^{\phi(n)} x_i \pmod n$$

$$a^{\phi(n)} \equiv 1 \pmod n$$

证毕。

$a = 3;\ n = 10;\ \phi(10) = 4;\ a^{\phi(n)} = 3^4 = 81 \equiv 1 \bmod 10 = 1 \pmod n$

$a = 2;\ n = 11;\ \phi(11) = 10;\ a^{\phi(n)} = 2^{10} = 1024 \equiv 1 \pmod{11} = 1 \pmod n$

这里使用的推理和费马定理的证明一样。类似费马定理，欧拉定理的另一种表述也很有用：

$$a^{\phi(n)+1} \equiv a \pmod n \qquad a, n \text{ 可以不互素} \tag{8.5}$$

同样和费马定理类似，欧拉定理的第一种形式[参见式(8.4)]要求 a 与 n 互素，而后一种没有这个要求。

8.3　素性测试　确定性的问题

许多密码算法都需要随机选择一个或多个非常大的素数，因此我们必须确定一个给定的大数是否是素数。目前还没有简单有效的方法解决该问题。

本节介绍一种较好的常用算法，该算法产生的数不一定是素数，你也许为此会感到惊讶，然而，该算法产生的数几乎可以肯定是素数。我们将很快解释这一点。我们也会讨论决定性算法寻找素数。本节的最后讨论一下素数的分布。

8.3.1　Miller-Rabin 算法②

Miller 和 Rabin 提出的这一算法[MILL75, RABI80]是典型的大数素性测试算法。在阐述算法前，先给出一些背景。首先，对奇整数 $n \geq 3$ 可表示如下：

$$n - 1 = 2^k q \qquad \text{其中 } k > 0, q \text{ 是奇数}$$

① 在第 2 章中提到，有限集 S 的一个排列，是指 S 中的元素的某一个给定的顺序，在该顺序中每个元素只出现一次。

② 某些文献里也称为 Rabin-Miller 算法，Rabin-Miller 测试或 Miller-Rabin 测试。

为了证明这一点,请注意$(n-1)$是一个偶数。接着用2去除$(n-1)$,直至所得结果为奇数q,此处共做了k次除法。如果n是二进制数表示,则将该数向右移,直到最右边的位为1为止就得到了结果,此处移动了k次。现在给出我们要用到的素数的两个性质。

素数的两个性质

第一个性质叙述如下:如果p是素数,a是小于p的正整数,则$a^2 \bmod p = 1$当且仅当$a \bmod p = 1$或$a \bmod p = -1 \bmod p = p-1$。运用模算术运算规则,$(a \bmod p)(a \bmod p) = a^2 \bmod p$。因此,如果$a \bmod p = 1$或$a \bmod p = -1$,则有$a^2 \bmod p = 1$。反过来,如果$a^2 \bmod p = 1$,则$(a \bmod p)^2 = 1$成立的条件是$a \bmod p = 1$或$a \bmod p = -1$。

第二个性质叙述如下:设p是大于2的素数,我们有$p-1 = 2^k q, k>0, q$奇数。设a是整数且$1 < a < p-1$,则下面两个条件之一成立:

(1) a^q模p和1同余,即$a^q \bmod p = 1$,或等价的,$a^q \equiv 1 \pmod p$。

(2) 在整数$a^q, a^{2q}, \cdots, a^{2^{k-1}q}$里存在一个数,模$p$时和$-1$同余。即存在一个$j(1 \leqslant j \leqslant k)$满足$a^{2^{j-1}q} \bmod p = (-1) \bmod p = p-1$,或$a^{2^{j-1}q} \equiv -1 \pmod p$。

证明 若n是素数,则由费马定理[参见式(8.2)]可知,$a^{n-1} \equiv -1 \pmod n$。由于$p-1 = 2^k q$,则$a^{p-1} \bmod p = a^{2^k q} \bmod p = 1$。因此,如果我们看下述的数列:

$$a^q \bmod p,\ a^{2q} \bmod p,\ a^{4q} \bmod p,\ \cdots,\ a^{2^{k-1}q} \bmod p,\ a^{2^k q} \bmod p \tag{8.6}$$

我们知道最后的数为1。而且,每一个数都是前一个数的平方。因此,下面的两条必有一条是正确的:

(1) 数列的第一个数,以及其后的所有数都为1。

(2) 数列里有些数不为1,但它们的平方模p后为1。根据刚才给出的素数第一性质,我们知道满足这个条件的唯一整数为$p-1$。因此,此时数列里必有一个数为$p-1$。证毕。

详细的算法

由上述的讨论可以推知,如果n为素数,则剩余类数列$(a^q, a^{2q}, \cdots, a^{2^{k-1}q}, a^{2^k q})$里,要么第一个数模$n$为1,要么数列里的某一个数模$n$为$(n-1)$;否则$n$为合数。另一方面,如果条件满足,也不一定能推出$n$为素数。例如,设$n = 2047 = 23 \times 89$,则$n-1 = 2 \times 1023$。计算$2^{1023} \bmod 2047 = 1$,所以2047满足条件,但不是素数。

我们可以利用上述的性质做素性测试。过程TEST输入整数n,如果n不是素数,则返回为合数的结果,如果n可能是素数,也可能不是时,返回不确定。

```
TEST (n)
1. Find integers k, q, with k > 0, q odd, so that
   (n - 1 = 2^k q);
2. Select a random integer a, 1 < a < n - 1;
3. if a^q mod n = 1 then return("inconclusive");
4. for j = 0 to k - 1 do
5. if a^(2^j q) mod n = n - 1 then return("inconclusive");
6. return("composite");
```

Eg=

对素数$n = 29$应用上述测试算法。我们有$(n-1) = 28 = 2^2(7) = 2^k q$。首先选取$a = 10$,计算$10^7 \bmod 29 = 17$,它既不为1也不为28,所以我们继续测试。然后计算$(10^7)^2 \bmod 29 = 28$,因此测试算法返回"不确定"(即29可能为素数)。再选取$a = 2$。由计算可知$2^7 \bmod 29 = 12$,$2^{14} \bmod 29 = 28$;因此仍返回"不确定"。如果对1到28之间的所有整数a执行测试算法,则得到的同样是"不确定"。这一点与n为素数是相一致的。

① n是否为素数

② $(n-1) = 2^k q$ 得到奇数q

③ 选取 $1 < a < n-1$

④ $(a^q) \cdots a^{2^k q} (a^{n-1})$ $2^k q$ mod n

1 ... n-1

现在对合数 $n=13\times17=221$ 应用上述测试，则 $(n-1)=220=2^2(55)=2^kq$。选取 $a=5$，则 $5^{55} \bmod 221=112$，它既不是 1 也不是 220；$(5^{55})^2 \bmod 221=168$，因为我们已对 TEST 算法第 4 行中所有的 $j(0,1)$ 进行了测试，所以算法返回“合数”，这表明 221 肯定是合数。但是假设我们选取 $a=21$，则 $21^{55} \bmod 221=200$，$(21^{55})^2 \bmod 221=220$，则测试算法返回“不确定”，表明 221 可能是素数。事实上，在 2 到 219 之间的这 218 个整数中，有 4 个整数会返回“不确定”，它们是 21，47，174，200。

重复使用 Miller-Rabin 算法

如何使用 Miller-Rabin 算法以更高的可信度来判定一个整数是否为素数？据参考文献[KNUT98]，给定一个非素奇数 n 和一个随机选择的整数 a，$1<a<n-1$，程序 TEST 返回不确定的概率小于 1/4（即不能确定 n 不是素数）。因此，如果选择 t 个不同的 a，则它们都能通过测试（返回不确定）的概率小于 $(1/4)^t$。例如，取 $t=10$，则一个非素整数通过 10 次测试的概率小于 10^{-6}。因此，取足够大的 t，如果 Miller 测试总是返回不确定，则我们以很大的把握说 n 是素数。

这为我们决定一个奇整数是素数且具有合理的可信度奠定了基础。其过程如下：对随机选取的 a，重复调用 TEST(n)，如果某时刻 TEST 返回“合数”，则 n 一定不是素数；若 TEST 连续 t 次返回“不确定”，这样当 t 足够大时，我们可以相信 n 是素数。

8.3.2　一个确定的素性判定算法

在 2002 年以前，没有高效的方法证明一个大数的素性。包括最为常用的 Miller-Rabin 算法在内，所有在用的算法给出的都是概率性结果。2002 年，Agrawal、Kayal 和 Saxena[AGRA02]给出了一个相对简单的确定性算法可以有效判定一个大数是否为素数。这个称为“AKS”的算法看上去没有 Miller-Rabin 算法快。因此它没有代替古老的概率算法[BORN03]。

8.3.3　素数的分布

值得注意的是，在利用 Miller-Rabin 测试或任何其他的素数测试算法发现一个素数之前已经测试了多少个整数？由数论中的素数定理可知，n 附近的素数分布情况为：平均每 $(\ln n)$ 个整数中有一个素数。这样，平均而言，在找到一个素数之前，必须测试约 $\ln(n)$ 个整数。因为所有偶数肯定不是素数，因此需测试的整数个数为 $0.5\ln(n)$。例如，若要找 2^{200} 左右的素数，则约需 $0.5\ln(2^{200})=69$ 次测试。然而，这只是平均值。在数轴上的某些位置，素数非常密集，而在其他有些位置，素数非常稀疏。

Edq:

两个相邻的奇数 1 000 000 000 061 和 1 000 000 000 063 都是素数；而另一方面，1001！+2，1001！+3，…，1001！+1000 1001！+1001 这 1000 个连续的整数均是合数。

8.4　中国剩余定理

中国剩余定理(CRT)[①]是数论中最有用的定理之一。CRT 说明某一范围内的整数可通过它的一组剩余类数来重构，这组剩余类数是对该整数用一组两两互素的整数取模得到的。

① 因为人们认为这个定理是公元 100 年由中国数学家孙子(Sun-Tsu)发现的，所以称为中国剩余定理。

Eg:

$Z_{10}(0,1,\cdots,9)$中的10个整数可通过它们对2和5(10的两个素因子)取模所得的两个余数来重构。假设已知十进制数x的余数$r_2=0$且$r_5=3$,即$x \bmod 2=0$且$x \bmod 5=3$,则x是Z_{10}中的偶数且被5除余3,故唯一解为$x=8$。

CRT可有几种不同的表示形式,这里我们给出其中一种最有用的表示形式,习题8.17给出了另一种表示形式。令

$$M = \prod_{i=1}^{k} m_i$$

其中m_i是两两互素的,即对$1 \leqslant i,j \leqslant k, i \neq j$有$\gcd(m_i, m_j) = 1$。我们可将$Z_M$中的任一整数对应一个$k$元组,该$k$元组的元素均在$Z_{m_i}$中,这种对应关系即为

$$A \leftrightarrow (a_1, a_2, \cdots, a_k) \tag{8.7}$$

其中$A \in Z_M$,对$1 \leqslant i \leqslant k$,$a_i \in Z_{m_i}$,且$a_i = A \bmod m_i$。CRT说明下列两个断言成立:

(1)式(8.7)中的映射是Z_M到笛卡儿积$Z_{m_1} \times Z_{m_2} \times \cdots \times Z_{m_k}$的一一对应(称为双射),也就是说,对任何$A$,$0 \leqslant A \leqslant M$,有唯一的$k$元组$(a_1, a_2, \cdots, a_k)$与之对应,其中$0 \leqslant a_i < m_i$,并且对任何这样的$k$元组$(a_1, a_2, \cdots, a_k)$,$Z_M$中有唯一的$A$与之对应。

2. Z_M中元素上的运算可等价于对应的k元组上的运算,即在笛卡儿积的每一个分量上独立地执行运算。

下面证明第一个断言。由A到$(a_1, a_2, \cdots, a_k)$的转换显然是唯一确定的。即只需取$a_i = A \bmod m_i$。对给定的$(a_1, a_2, \cdots, a_k)$,可如下计算A。对$1 \leqslant i \leqslant k$,令$M_i = M/m_i$,因为$M_i = m_1 \times m_2 \times \cdots \times m_{i-1} \times m_{i+1} \times \cdots \times m_k$,所以对所有的$j \neq i$,有$M_i \equiv 0 \pmod{m_j}$。令

$$c_i = M_i \times \left(M_i^{-1} \bmod m_i\right), \qquad 1 \leqslant i \leqslant k \tag{8.8}$$

根据M_i的定义有M_i与m_i互素,所以M_i有唯一的模m_i的乘法逆元,因此根据式(8.8),可得到唯一的c_i。计算

$$A \equiv \left(\sum_{i-1}^{k} a_i c_i\right) \pmod{M} \tag{8.9}$$

要证明式(8.9)产生的A是正确的,必须证明对$1 \leqslant i \leqslant k$有$a_i = A \bmod m_i$。由于$j \neq i$时,$c_j \equiv M_j \equiv 0 \pmod{m_i}$,而且$c_i \equiv 1 \pmod{m_i}$,故$a_i = A \bmod m_i$。

第二个断言,与算术运算有关,可从模算术规则推出,即上述第二个断言也可如下描述:若

$$A \leftrightarrow (a_1, a_2, \cdots, a_k)$$
$$B \leftrightarrow (b_1, b_2, \cdots, b_k)$$

则

$$(A + B) \bmod M \leftrightarrow ((a_1 + b_1) \bmod m_1, \cdots, (a_k + b_k) \bmod m_k)$$
$$(A - B) \bmod M \leftrightarrow ((a_1 - b_1) \bmod m_1, \cdots, (a_k - b_k) \bmod m_k)$$
$$(A \times B) \bmod M \leftrightarrow ((a_1 \times b_1) \bmod m_1, \cdots, (a_k \times b_k) \bmod m_k)$$

中国剩余定理的用途之一是，它给出了一种方法，使得模 M 的大数运算转化到更小的数上来进行运算，当 M 为 150 位或 150 位以上时，这种方法非常有效。然而我们需要事先分解 M。

下面将 973 mod 1813 表示为模 37 和 49 的两个数。我们定义①

$$m_1 = 37$$
$$m_2 = 49$$
$$M = 1813 = m_1 \cdot m_2$$
$$A = 973$$

则 $M_1 = 49$ 且 $M_2 = 37$。利用扩展的 Euclid 算法有 $M_1^{-1} = 34 \bmod m_1$ 且 $M_2^{-1} = 4 \bmod m_2$（注意每个 M_i 和 M_i^{-1} 只需计算一次）。对 37 和 49 取模，因为 973 mod 37 = 11，973 mod 49 = 42，所以 973 可表示为 (11,42)。

假定要计算 678 加 973。如何处理 (11,42) 呢？首先计算 (678) ↔ (678 mod 37, 678 mod 49) = (12,41)，然后将二元组的元素相加并化简 (11 + 12 mod 37, 42 + 41 mod 49) = (23,34)。这个结果是正确的，因为

$$(23,34) \leftrightarrow a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} \bmod M$$
$$= [(23)(49)(34) + (34)(37)(4)] \bmod 1813$$
$$= 43350 \bmod 1813$$
$$= 1651$$

而 (973 + 678) mod 1813 = 1651。请记住上述推导里，M_i^{-1} 是 M_1 模 m_1 的乘法逆，而 M_2^{-1} 是 M_2 模 m_2 的乘法逆。

假定要计算 73 乘 1651 (mod 1813)。我们首先用 73 乘 (23,24) 并化简得 (23 × 73 mod 37, 34 × 73 mod 49) = (14,32)。很容易验证：

$$(14,32) \leftrightarrow [(14)(49)(34) + (32)(37)(4)] \bmod 1813$$
$$= 856$$
$$= 1651 \times 73 \bmod 1813$$

8.5 离散对数

离散对数是包括 Diffie-Hellman 密钥交换和数字签名算法（DSA）在内的许多公钥算法的基础。本节简要介绍离散对数的一般知识，有兴趣者可参阅[ORE76]和[LEVE90]，它们更详细地介绍了这方面的知识。

8.5.1 模 n 的整数幂

欧拉定理[参见式(8.4)]告诉我们，对任何互素的 a 和 n，有

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

其中欧拉函数 $\phi(n)$ 是指小于 n 且与 n 互素的正整数的个数。下面我们考虑欧拉定理更一般的表示形式：

① 本例由佐治亚工学院的 Ken Calvert 教授提供。

$$a^m \equiv 1 \pmod{n} \tag{8.10}$$

若 a 与 n 互素,则至少有一个整数 m 满足式(8.10),即 $M = \phi(n)$,我们称使式(8.10)成立的最小正幂 m 为下列之一:

- a(模 n)的阶
- a 所属的模 n 的指数
- a 所产生的周期长

为说明最后一个概念,考虑下面 7 模 19 的各次幂:

$$7^1 \equiv 7 \pmod{19}$$
$$7^2 = 49 = 2 \times 19 + 11 \equiv 11 \pmod{19}$$
$$7^3 = 343 = 18 \times 19 + 1 \equiv 1 \pmod{19}$$
$$7^4 = 2401 = 126 \times 19 + 7 \equiv 7 \pmod{19}$$
$$7^5 = 16807 = 884 \times 19 + 11 \equiv 11 \pmod{19}$$

由于上述计算中出现了重复,所以不必再往下计算,因为由 $7^3 \equiv 1 \pmod{19}$,可得 $7^{3+j} \equiv 7^3 7^j \equiv 7^j \pmod{19}$,这说明若 7 的两个指数相差 3(或 3 的倍数),则以它们为指数的 7 的(模 19)幂是相同的,换句话说,该序列是周期性的,且其周期长是使 $7^m \equiv 1 \pmod{19}$ 成立的最小正幂 m。

表 8.3 给出了所有小于 19 的正整数 a 模 19 的各次幂,阴影部分指明了每个底 a 的幂序列长。

请注意:

(1) 所有序列均以 1 结束。这与前面的推导是一致的。

(2) 每个序列的长度都可以整除 $\phi(19) = 18$。也就是说,表中每一行都有整数个子序列。

(3) 有些序列长为 18。这时我们说底 a(通过幂运算)生成模 19 的非零整数集,并称这样的整数为模数 19 的本原根。

更一般地,我们说 $\phi(n)$ 是一个数所属的模 n 的可能的最高指数。如果一个数的阶为 $\phi(n)$,则称之为 n 的本原根。本原根的重要之处在于,若 a 是 n 的本原根,则其幂

$$a, a^2, \cdots, a^{\phi(n)}$$

是(模 n)各不相同的,且均与 n 互素。特别地,对素数 p,若 a 是 p 的本原根,则

$$a, a^2, \cdots, a^{p-1}$$

是(模 p)各不相同的。素数 19 的本原根为 2,3,10,13,14 和 15。

并不是所有的整数都有本原根。事实上,只有形为 2,4,p^α 和 $2p^\alpha$ 的整数才有本原根,这里 p 是任何奇素数,α 是正整数。证明并不简单,但在许多数论书里都有,如[ORE76]。

8.5.2 模算术对数

对于普通正实数,对数函数是指数函数的逆函数。对模算术也有类似的函数。

我们先简要讨论普通对数的性质。给定一个数,它的对数是满足下列条件的幂:以某个正数(除 1 外)为底的该次幂恰好等于给定的这个整数,即对底 x 和数 y:

$$y = x^{\log_x(y)}$$

表 8.3　模 19 的整数幂

a	a^2	a^3	a^4	a^5	a^6	a^7	a^8	a^9	a^{10}	a^{11}	a^{12}	a^{13}	a^{14}	a^{15}	a^{16}	a^{17}	a^{18}
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	4	8	16	13	7	14	9	18	17	15	11	3	6	12	5	10	1
3	9	8	5	15	7	2	6	18	16	10	11	14	4	12	17	13	1
4	16	7	9	17	11	6	5	1	4	16	7	9	17	11	6	5	1
5	6	11	17	9	7	16	4	1	5	6	11	17	9	7	16	4	1
6	17	7	4	5	11	9	16	1	6	17	7	4	5	11	9	16	1
7	11	1	7	11	1	7	11	1	7	11	1	7	11	1	7	11	1
8	7	18	11	12	1	8	7	18	11	12	1	8	7	18	11	12	1
9	5	7	6	16	11	4	17	1	9	5	7	6	16	11	4	17	1
10	5	12	6	3	11	15	17	18	9	14	7	13	16	8	4	2	1
11	7	1	11	7	1	11	7	1	11	7	1	11	7	1	11	7	1
12	11	18	7	8	1	12	11	18	7	8	1	12	11	18	7	8	1
13	17	12	4	14	11	10	16	18	6	2	7	15	5	8	9	3	1
14	6	8	17	10	7	3	4	18	5	13	11	2	9	12	16	15	1
15	16	12	9	2	11	13	5	18	4	3	7	10	17	8	6	14	1
16	9	11	5	4	7	17	6	1	16	9	11	5	4	7	17	6	1
17	4	11	16	6	7	5	9	1	17	4	11	16	6	7	5	9	1
18	1	18	1	18	1	18	1	18	1	18	1	18	1	18	1	18	1

对数具有下列性质：

$$\log_x(1) = 0$$

$$\log_x(x) = 1$$

$$\log_x(yz) = \log_x(y) + \log_x(z) \tag{8.11}$$

$$\log_x(y^r) = r \times \log_x(y) \tag{8.12}$$

对某素数 p（对非素数亦可）的本原根 a，a 的 1 到 $(p-1)$ 的各次幂恰可产生 1 到 $(p-1)$ 的每个整数一次且仅一次。而对任何整数 b，根据模算术的定义，b 满足：

$$b \equiv r \pmod p, \text{ 对于某些 } r, \text{ 其中 } 0 \leqslant r \leqslant (p-1)$$

因此，对任何整数 b 和素数 p 的本原根 a，有唯一的幂 i 使得

$$b \equiv a^i \pmod p, \quad \text{其中 } 0 \leqslant i \leqslant (p-1)$$

该指数 i 称为以 a 为底（模 p）b 的离散对数，记为 $\mathrm{dlog}_{a,p}(b)$[①]。

请注意下列各式：

$$\mathrm{dlog}_{a,p}(1) = 0, \quad \text{这是因为 } a^0 \bmod p = 1 \bmod p = 1 \tag{8.13}$$

$$\mathrm{dlog}_{a,p}(a) = 1, \quad \text{这是因为 } a^1 \bmod p = a \tag{8.14}$$

下面的例子使用的是非素数模 $n=9$，这里 $\phi(n)=6$，$a=2$ 是一个本原根，计算 a 的各次幂得

① 许多教科书把离散对数称为指标（index），这个概念没有通用的记号，更没有统一的名字。

$2^0 = 1$ $2^4 \equiv 7 \pmod 9$ 周期长度 $\phi(n)$

$2^1 = 2$ $2^5 \equiv 5 \pmod 9$

$2^2 = 4$ $2^6 \equiv 1 \pmod 9$

$2^3 = 8$

因此,对给定的本原根 $a = 2$(模 9)的离散对数,有下列数表

对数	0	1	2	4	5	7
数	1	2	4	8	7	5

给定一个数,要得到其离散对数,我们可以重排上表

数	1	2	4	5	7	8
对数	0	1	2	5	4	3

考虑

$$x = a^{\text{dlog}_{a,p}(x)} \bmod p \qquad y = a^{\text{dlog}_{a,p}(y)} \bmod p$$

$$xy = a^{\text{dlog}_{a,p}(xy)} \bmod p$$

由模乘法的性质有

$$xy \bmod p = [(x \bmod p)(y \bmod p)] \bmod p$$

$$a^{\text{dlog}_{a,p}(xy)} \bmod p = \left[\left(a^{\text{dlog}_{a,p}(x)} \bmod p\right)\left(a^{\text{dlog}_{a,p}(y)} \bmod p\right)\right] \bmod p$$

$$= \left(a^{\text{dlog}_{a,p}(x)+\text{dlog}_{a,p}(y)}\right) \bmod p$$

根据欧拉定理,对任何互素的 a 和 n,有

$$a^{\phi(n)} \equiv 1 \pmod n$$

又任何正整数 z 可表示为 $z = q + k\phi(n)$,其中 $0 \leqslant q < \phi(n)$,所以由欧拉定理有

$$a^z \equiv a^q \pmod n \qquad z \equiv q \bmod \phi(n)$$

将其代入前面的等式,则有

$$\text{dlog}_{a,p}(xy) \equiv [\text{dlog}_{a,p}(x) + \text{dlog}_{a,p}(y)] (\text{mod}\,\phi(p))$$

由此可得 $\log a^r = r \times \log a$

$$\text{dlog}_{a,p}(y^r) \equiv [r \times \text{dlog}_{a,p}(y)] (\text{mod}\,\phi(p))$$

这说明了普通对数和离散对数之间的相似性。

注意,仅当 a 是 m 的本原根时,才存在有唯一的以 a 为底模 m 的离散对数。

表 8.4 可直接由表 8.3 推出。它列出了模 19 的离散对数集。

表 8.4 模 19 的离散对数表 $a = z^{\hat{z}} \pmod{19}$

(a) 以 2 为底,模为 19 的离散对数

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{2,19}(a)$	18	1	13	2	16	14	6	3	8	17	12	15	5	7	11	4	10	9

(b) 以 3 为底,模为 19 的离散对数 $a = 3^{\hat{z}} \pmod{19}$

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{3,19}(a)$	18	7	1	14	4	8	6	3	2	11	12	15	17	13	5	10	16	9

（续表）

（c）以 10 为底，模为 19 的离散对数

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{10,19}(a)$	18	17	5	16	2	4	12	15	10	1	6	3	13	11	7	14	8	9

（d）以 13 为底，模为 19 的离散对数

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{13,19}(a)$	18	11	17	4	14	10	12	15	16	7	6	3	1	5	13	8	2	9

（e）以 14 为底，模为 19 的离散对数

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{14,19}(a)$	18	13	7	8	10	2	6	3	14	5	12	15	11	1	17	16	4	9

（f）以 15 为底，模为 19 的离散对数

a	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
$\log_{15,19}(a)$	18	5	11	10	8	16	12	15	4	13	6	3	7	17	1	2	14	9

8.5.3　离散对数的计算

考虑方程

$$y = g^x \bmod p$$

对给定的 g，x 和 p，可直接计算出 y，在最坏情况下需执行 x 次乘法，而且存在计算 y 的有效算法（参见第 9 章）。

但是，给定 y，g 和 p，计算 x 一般非常困难（即求离散对数），其难度与 RSA 中因子分解素数之积的难度具有相同的数量级。在本书写作时，已知的最快求模数为素数的离散对数的算法的难度阶为[BETH91]

$$e^{((\ln p)^{1/3}(\ln(\ln p))^{2/3})}$$

对大素数而言，该算法是不可行的。

8.6　推荐读物和网站

有许多教材都非常详细地讨论了数论，其详细程度足以满足本书大部分读者的需要。[ORE67]是一本篇幅不大，但却是有用的初等数论的入门书籍；若读者要更深入地学习数论，可参阅[KUMA98]和[ROSE05]这两本优秀教材；另外，[LEVE90]也详细介绍了数论有关的知识，可读性好。以上这些教材都附有习题及其解答，特别适合于自学。

[BURN97]包含许多习题并附有解答，有时间的读者可通过学习[BURN97]并完成这些习题逐步掌握数论中的基本概念，并牢固掌握数论的基本知识。完成这些习题相当于完成了本科程度的数论课程。

BURN97 Burn, R. *A Pathway to Number Theory*. Cambridge, England: Cambridge University Press, 1997.

KUMA98 Kumanduri, R., and Romero, C. *Number Theory with Computer Applications*. Upper Saddle River, NJ: Prentice Hall, 1998.

LEVE90 Leveque, W. *Elementary Theory of Numbers*. New York: Dover, 1990.

ORE67 Ore, O. *Invitation to Number Theory*. Washington, DC.: The Mathematical Association of America, 1967.

ROSE05 Rosen, K. *Elementary Number Theory and its Applications*. Reading, MA: Addison-Wesley, 2000.

推荐网站

- The Prime Pages:有关素数的研究,记录及相关资料

8.7 关键术语、思考题和习题

关键术语

双射	合数	中国剩余定理
离散对数	欧拉定理	欧拉函数
费马定理	指标	阶
素数	本原根	

思考题

8.1 什么是素数?

8.2 表达式 a 整除 b 的意义是什么?

8.3 什么是欧拉函数?

8.4 Miller-Rabin 测试可确定一个数不是素数,但不能确定一个数是素数。该算法如何用来进行素性测试的?

8.5 一个数的本原根是什么?

8.6 指标和离散对数的区别是什么?

习题

8.1 本题的目的是要判定有多少素数。假定总共有 n 个素数,依次为:$p_1=2<p_2=3<p_3=5<\cdots<p_n$

(a) 定义 $X=1+p_1p_2\cdots p_n$,即所有素数的积加上 1,能够找到一个素数 P_m 整除 X 吗?

(b) m 有何特性?

(c) 请推出素数的个数不可能是有限的。

(d) 证明 $P_{n+1}\leqslant 1+p_1p_2\cdots p_n$。

8.2 本题的目的是要证明,两个随机数互素的概率约为 0.6。

(a) 令 $P=\Pr[\gcd(a,b)=1]$。证明 $P=\Pr[\gcd(a,b)=d]=P/d^2$。提示:请考虑 $\gcd(a/d,b/d)$。

(b) 对所有可能的 d,(a)中结果之和为 1,即 $\sum_{d\geqslant 1}\Pr[\gcd(a,b)=d]=1$,利用上述等式确定 P 的值。提示:请利用恒等式 $\sum_{i=1}^{\infty}\frac{1}{i^2}=\frac{\pi^2}{6}$。

8.3 对两个连续整数 n 和 $n+1$,为什么 $\gcd(n,n+1)=1$?

8.4 利用费马定理计算 $3^{201}\bmod 11$。

8.5 利用费马定理,找一个位于 0 到 72 之间的数 a,使得 a 模 73 与 9794 同余。

8.6 利用费马定理,找一个位于 0 到 28 之间的数 x,使得 x^{85} 模 29 与 6 同余(你不能也不必用穷举搜索方法)。

8.7 利用欧拉定理,找一个位于 0 到 9 之间的整数 a,使得 7^{1000} 模 10 与 a 同余(注意这等于是 7^{1000} 的十进制数展开的最后一位)。

8.8 利用欧拉定理,找一个位于 0 到 28 之间的整数 x,使得 x^{85} 模 35 与 6 同余(你不能也不必用穷举搜索方法)。

8.9 在表 8.2 中,$n>2$ 时 $\phi(n)$ 是偶数。这一点对所有 $n>2$ 都成立。请证明之。

8.10 证明,若 p 是素数,则 $\phi(p^i) = p^i - p^{i-1}$。提示:请考虑哪些数与 p^i 有公因子。

8.11 可以证明(参见任何有关数论的书),若 $\gcd(m,n) = 1$,则 $\phi(mn) = \phi(m)\phi(n)$。利用该性质和上一习题中的有关性质以及对素数 p,$\phi(p) = p - 1$ 这些性质,对任何 n 可直接取得 $\phi(n)$ 的值。请计算

(a) $\phi(41)$ (b) $\phi(27)$ (c) $\phi(231)$ (d) $\phi(440)$

8.12 证明,对于任意的正整数 a,有

$$\phi(a) = \prod_{i=1}^{t} [p_i^{a_i - 1}(p_i - 1)]$$

成立,其中 a 由式(8.1)给出,即 $a = P_1^{a1} P_2^{a2} \cdots P_t^{at}$。

8.13 考虑函数 $f(n)$ = 集合 $\{a: 0 \leq a < n$,且 $\gcd(a,n) = 1\}$ 里元素的个数。这个函数是什么函数?

8.14 虽然中国古代数学家做了很好的工作,提出了中国剩余定理,但是他们并不总是正确的。他们提出的素性测试方法断言:n 是素数当且仅当 n 能整除 $(2^n - 2)$。

(a) 给出一个满足上述条件的奇素数;

(b) $n = 2$ 时条件显然为真。证明 n 是奇素数时条件也为真(即证明充分条件);

(c) 给出一个奇数但不是素数,它不满足上述条件。该非素数可能非常大,这也是导致□国数学家误以为条件为真时,则 n 是素数的原因。

(d) 遗憾的是,中国古代数学家并没有测试 $n = 341$ 的情形。341 不是素数($341 = 11 \times 31$)但能整除 $2^{341} - 2$。证明 $2^{341} \equiv 2 \pmod{341}$(即必要条件不成立)。提示:可利用同余定理证明之,而不必计算 2^{341}。

8.15 证明:若 n 是奇合数,则对 $a = 1$ 和 $a = (n - 1)$,Miller-Rabin 测试将返回"不确定"。

8.16 若 n 是合数,且对底 a 通过了 Miller-Rabin 测试,则 n 称为对底 a 的强伪素数。证明 2047 是对底 2 的强伪素数。

8.17 中国剩余定理的常用表示形式为:令 $m_1, \cdots, m_k$ 是两两互素的整数,且 $1 \leq i, j \leq k, i \neq j$,定义 M 为所有 m_i 的乘积。设 $a_1, \cdots, a_k$ 是整数,则同余方程组

$$\begin{aligned} x &\equiv a_1 \pmod{m_1} \\ x &\equiv a_2 \pmod{m_2} \\ &\vdots \\ x &\equiv a_k \pmod{m_k} \end{aligned}$$

模 M 有唯一解。证明定理的这种表示形式是正确的。

8.18 下面是孙子用来说明 CRT 的一个例子

$$x \equiv 2 \pmod 3;\ x \equiv 3 \pmod 5;\ x \equiv 2 \pmod 7$$

试求解 x。

8.19 六位教授分别在周一至周六开始授课,且分别每隔 2,3,4,1,6 和 5 天授一次课,该大学禁止周日上课(所以周日的课必须停止)。什么时候所有六位教授首次发现必须停课?提示:利用 CRT。

8.20 找出 25 的所有本原根。

8.21 给定 29 的本原根 2,构造离散对数表,并利用该表解下列同余方程:

(a) $17x^2 \equiv 10 \pmod{29}$

(b) $x^2 - 4x - 16 \equiv 0 \pmod{29}$

(c) $x^7 \equiv 17 \pmod{29}$

编程题

8.22 编写一个计算机程序实现模 n 的快速指数运算(连续平方)。

8.23 编写一个计算机程序实现对用户指定的 n 进行 Miller-Rabin 测试。该程序有两种选择:(1)指定一个可能的证词(witness)a,用 Witness 算法进行测试,或者(2)指定一个随机证词 s,用 Miller-Rabin 测试算法进行检验。

第9章　公钥密码学与RSA

9.1　公钥密码体制的基本原理
　9.1.1　公钥密码体制
　9.1.2　公钥密码体制的应用
　9.1.3　对公钥密码的要求
　9.1.4　公钥密码分析
9.2　RSA 算法
　9.2.1　算法描述
　9.2.2　计算方面的问题
　9.2.3　RSA 的安全性
9.3　推荐读物和网站
9.4　关键术语、思考题和习题
附录9A　RSA 算法的证明
附录9B　算法复杂性

Every Egyptian received two names, which were known respectively as the true name and the good name, or the great name and the little name; and while the good or little name was made public, the true or great name appears to have been carefully concealed.

—*The Golden Bough*, Sir James George Frazer

要　点

- 非对称密码是一种密码体制，其加密算法和解密算法使用不同的密钥：一个是公钥，另一个是私钥。非对称密码也称为公钥密码。
- 非对称密码用两个密钥中的一个以及加密算法将明文转换为密文。用另一个密钥以及解密算法从密文恢复出明文。
- 非对称密码可以用来保密，认证或者上述两者功能。
- 应用最广泛的公钥密码体制是 RSA。攻击 RSA 的困难性基于寻找大合数的素因子的困难性。

公钥密码学的发展是整个密码学发展历史中最伟大的一次革命，也许可以说是唯一的一次革命。从密码学产生至今，几乎所有的密码体制都是基于代替和置换这些初等方法的。几千年来，对算法的研究主要是通过手工计算来完成的。随着转轮加密/解密机器的出现，传统密码学有了很大进展，利用电子机械转轮可以开发出极其复杂的加密系统，利用计算机甚至可以设计出更加复杂的系统，最著名的例子是 Lucifer，它是在 IBM 实现数据加密标准（DES）时所设计的密码系统。转轮机和 DES 是密码学发展的重要标志，但是它们都基于代替和置换这些初等方法之上。

公钥密码学与其前的密码学完全不同。首先，公钥算法是基于数学函数而不是基于代替和置换，更重要的是，与只使用一个密钥的对称密码不同，公钥密码是非对称的，它使用两个独立的密钥。我们将会看到，使用两个密钥在消息的保密性、密钥分配和认证领域有着重要意义。

下面我们先讨论有关公钥密码所常见的几种误解。一种误解是，从密码分析的角度看，公钥密码比传统密码更安全。事实上，任何加密方法的安全性依赖于密钥的长度和破译密文所需要的计算量。从抗密码分析的角度看，原则上不能说传统密码优于公钥密码，也不能说公钥密码优于传统密码。

第二种误解是，公钥密码是一种通用的方法，所以传统密码已经过时。其实正相反，由于现有的公钥密码方法所需的计算量大，所以取缔传统密码似乎不太可能。就像公钥密码的发明者之一所说的[DIFF88]，“公钥密码学仅限于用在密钥管理和签名这类应用中，这几乎是已被广泛接受的事实。”

最后一种误解是，传统密码中与密钥分配中心的会话是一件异常麻烦的事情，与之相比，用公

钥密码实现密钥分配则非常简单。事实上，使用公钥密码也需要某种形式的协议，该协议通常包含一个中心代理，并且它所包含的处理过程既不比传统密码中的那些过程更简单，也不比之更有效（参见[NEED78]的分析）。

本章和下一章简要介绍公钥密码。我们先介绍其中的基本概念。有趣的是，这些概念在用于公钥密码中之前就已经提出；然后我们讨论 RSA 算法，它是公钥密码中最重要的一种切实可行的加密/解密算法；第 10 章包含一些其他重要的公钥密码算法。

很多公钥密码体制的理论都基于数论。如果你接受本章中给出的结论，那么就可以不必严格地去理解数论的有关知识。但是要完全理解公钥算法，则需要理解这些数论知识。第 8 章中概要介绍了与公钥密码有关的数论知识。

表 9.1 定义了一些关键术语。

表 9.1　公钥密码术语

非对称密钥
两个密钥：公钥和私钥，用来实现互逆运算，即加密和解密，或者生成签名与验证签名
公钥证书
公钥证书是一种由签证机构用自己的私钥签名的数字文件。它将证书拥有者的姓名和其公钥绑定在一起。证书中所标志的证书拥有者单独控制和使用自己的私钥
公钥密码（非对称密码）算法
含有两个密钥：公钥和私钥。从公钥推出私钥在计算上是不可行的
公钥基础设施（PKI）
一个由政策、处理、服务平台、软件和工作站组成的集合，用于管理证书和公钥 - 私钥对，并且负责产生、维护和废除公钥证书

9.1　公钥密码体制的基本原理

公钥密码学的概念是为了解决传统密码中最困难的两个问题而提出的。第一个问题将在第 14 章中详细讨论的密钥分配问题。

如第 14 章所讨论的，用对称密码进行密钥分配要求：(1)通信双方已经共享一个密钥，而该密钥已通过某种方法分配给通信双方；或者(2)利用密钥分配中心。公钥密码的发明人之一 Whitfield Diffie（和 Martin Hellman，当时同在斯坦福大学工作）认为，第二个要求有悖于密码学的精髓，即应在通信过程中完全保持保密性。正如 Diffie 所说[DIFF88]，“如果必须要求用户与 KDC 共享他们的密钥，这些密钥可能因为盗窃或索取而被泄密，那么设计不可破的密码体制究竟还有什么意义呢?”

Diffie 考虑的第二个问题是“数字签名”问题，该问题显然与第一个问题无关。如果密码学不是仅仅用于军事上，而是广泛用于商业或个人目的，那么像手写签名一样，电子消息和文件也需要签名，也就是说，能否设计一种方法，它能确保数字签名是出自某特定的人，并且各方对此均无异议？对数字签名的要求比对认证的要求更为广泛，在第 13 章中我们将讨论数字签名的特性及其相关的问题。

1976 年 Diffie 和 Hellman 针对上述两个问题提出了一种方法，这种方法与以前 4000 多年来密码学中的所有方法有着根本区别①，它是密码学中一个惊人的成就[DIFF76a，b]。

① Diffie 和 Hellman 于 1976 年首次公开提出了公钥密码学的概念，但是公钥密码学的概念并不是 1976 年才出现。美国国家安全局（NSA）的负责人 Admiral Bobby Inman 声称 NSA 于 20 世纪 60 年代中期就已提出了公钥密码学[SIMM93]。这些概念的最早的文字记录是在 James Ellis 于 1970 年所撰写的一份机密报告中[ELLI70]，这些概念出自与 NSA 类似的英国机构，即通信电子安全研究组（Communications-Electronics Security Group），Ellis 称这种技术为非秘密的加密方法，并在[ELLI99]中描述了这一发现。

在下一小节里,我们先讨论公钥密码学的基本框架,然后讨论加密/解密算法应满足的一些条件,这些条件是公钥体制的核心内容。

9.1.1 公钥密码体制

公钥算法依赖于一个加密密钥和一个与之相关的不同的解密密钥,这些算法都具有下述重要特点:

- 仅根据密码算法和加密密钥来确定解密密钥在计算上是不可行的。

另外有些算法(如 RSA),还有以下特点:

- 两个密钥中的任何一个都可用来加密,另一个用来解密。

公钥密码体制有 6 个组成部分[参见图 9.1(a);请对照图 2.1]:

- **明文**:算法的输入。它们是可读信息或数据。
- **加密算法**:加密算法对明文进行各种变换。
- **公钥和私钥**:算法的输入。这对密钥中一个用于加密,另一个用于解密。加密算法执行的变换依赖于公钥或者私钥。
- **密文**:算法的输出。它依赖于明文和密钥,对给定的消息,不同的密钥产生的密文不同。
- **解密算法**:该算法接收密文和相应的密钥,并产生原始的明文。

其主要步骤如下:

(1) 每一用户产生一对密钥,用来加密和解密消息。

(2) 每一用户将其中一个密钥存于公开的寄存器或其他可访问的文件中,该密钥称为公钥,而另一密钥则保持私密。如图 9.1(a)所示,每一用户可以拥有若干其他用户的公钥。

(3) 若 Bob 要发消息给 Alice,则 Bob 用 Alice 的公钥对消息加密。

(4) Alice 收到消息后,用其私钥对消息解密。由于只有 Alice 知道其自身的私钥,所以其他的接收者均不能解密出消息。

利用这种方法,通信各方均可访问公钥,而私钥是各通信方在本地产生的,所以不必进行分配。只要用户的私钥受到保护,保持秘密性,那么通信就是安全的。在任何时刻,系统可以改变其私钥,并公布相应的公钥以替代原来的公钥。

表 9.2 总结了对称密码和公钥密码的一些重要特征。为了区分二者,我们一般将对称密码中使用的密钥称为秘密钥,将公钥密码中使用的两个密钥分别称为公钥和私钥①。私钥总是保密的,但是为了避免与对称密码中的密钥混淆,我们将之称为私钥而不称为秘密钥。

下面我们利用图 9.2(比较图 2.2)进一步分析公钥密码体制的基本要素。消息源 A 产生明文消息 $X=[X_1,X_2,\cdots,X_M]$,其中 X 的 M 个元素是某有穷字母表上的字符,A 欲将消息 X 发送给 B。B 产生公钥 PU_b 和私钥 PR_b,其中只有 B 知道 PR_b,而 PU_b 则是公开可访问的,所以 A 也可访问 PU_b。

对作为输入的消息 X 和加密密钥 PU_b,A 生成密文 $Y=[Y_1,Y_2,\cdots,Y_N]$:

$$Y=E(PU_b,X)$$

① 我们将使用下述记号。秘密钥用 K_m 表示,其中 m 是修饰符,例如,K_a 表示 A 的秘密密钥;用户 A 的公钥用 PU_a 表示,其相应的私钥用 PR_a 表示。分别用 $E[K_a,X]$、$E[PU_a,X]$ 和 $E[PR_a,X]$ 表示用秘密钥、公钥和私钥对明文 P 加密,同样,分别用 $D[K_a,X]$、$D[PU_a,X]$ 和 $D[PR_a,X]$ 表示用秘密钥、公钥和私钥对密文 C 解密。

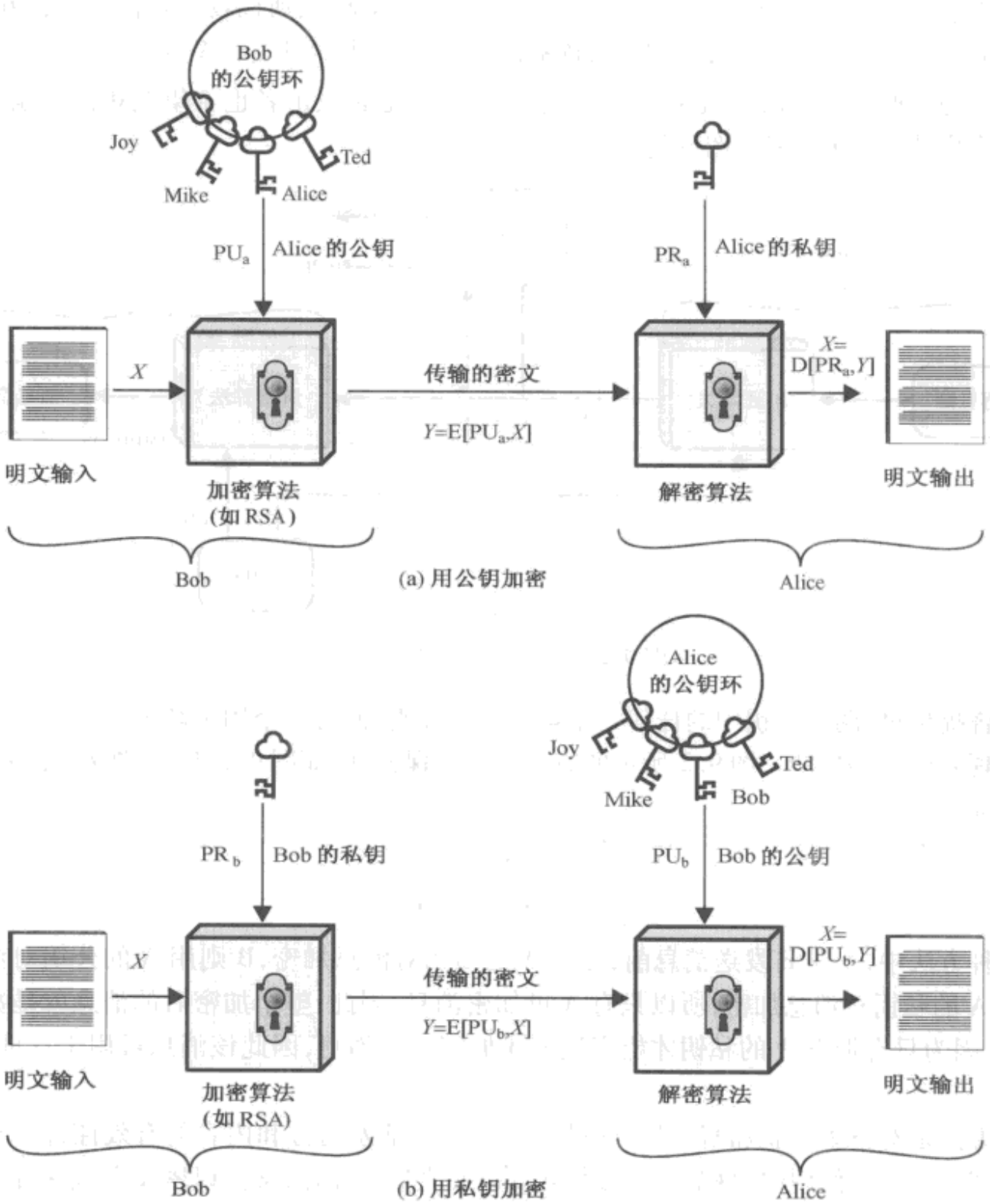


图 9.1　公钥密码体制

表 9.2　传统密码和公钥密码

传统密码	公钥密码
一般要求	**一般要求**
(1) 加密和解密使用相同的密钥和相同的算法	(1) 同一算法用于加密和解密，但加密和解密使用不同密钥
(2) 收发双方必须共享密钥	(2) 发送方拥有加密或解密密钥，而接收方拥有另一密钥
安全性要求	**安全性要求**
(1) 密钥必须是保密的	(1) 两个密钥之一必须是保密的
(2) 若没有其他信息，则解密消息是不可能或至少是不可行的	(2) 若没有其他信息，则解密消息是不可能或至少是不可行的
(3) 知道算法和若干密文不足以确定密钥	(3) 知道算法和其中一个密钥以及若干密文不足以确定另一密钥

期望的接收方因为拥有相应的私钥，所以可进行逆变换：

$$X = D(PR_b, Y)$$

攻击者可观察到 Y 并且可访问 PU_b,但是他不能访问 PR_b 或者 X,所以攻击者肯定会想方设法恢复 X 和/或 PR_b。假定攻击者知道加密(E)和解密(D)算法,如果他只关心 X 这一条消息,那么他会集中精力试图通过产生明文估计值 $\hat{X}$ 来恢复消息 X。但是通常攻击者也希望能获得其他消息,所以他会通过产生估计值 PR_b 来试图恢复 $\hat{PR}_b$。

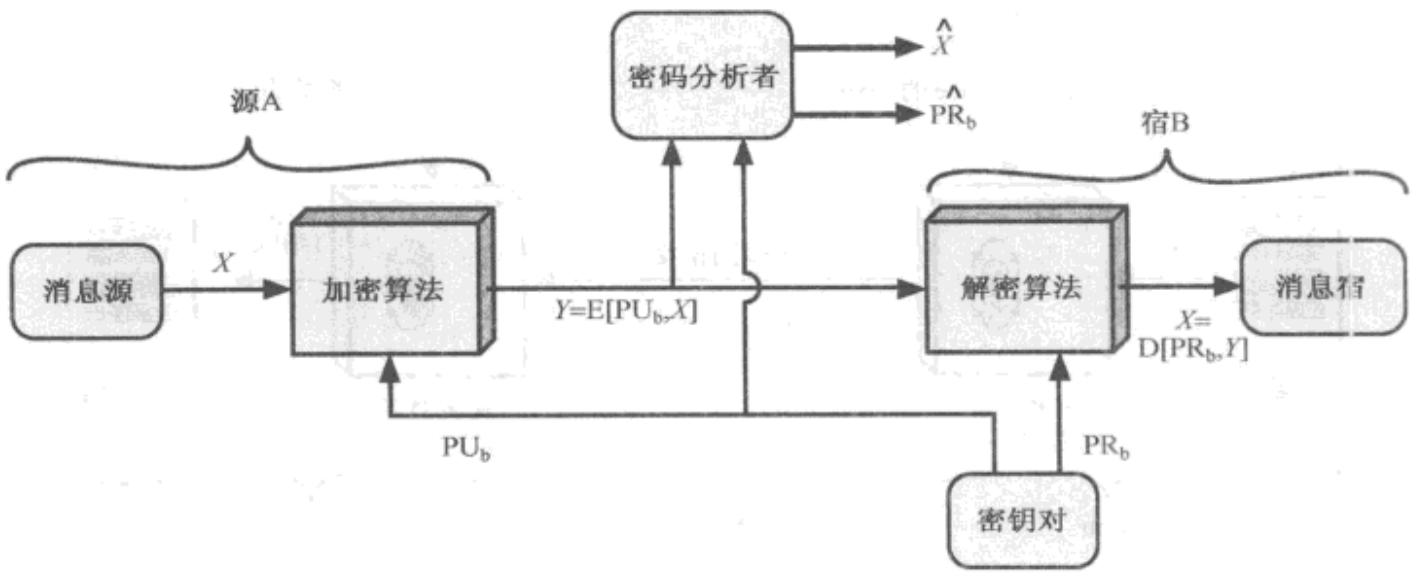


图 9.2 公钥密码体制:保密性

前面曾提到过,两个密钥中的任何一个都可用来加密,而另一个用来解密,这样我们可利用公钥密码实现其他一些功能。图 9.2 所示的方法可提供保密性,而图 9.1(b)和图 9.3 说明公钥密码可用于认证:

$$Y = E(PR_a, X)$$

$$X = D(PU_a, Y)$$

在这种方法中,A 向 B 发送消息前,先用 A 的私钥对消息加密,B 则用 A 的公钥对消息解密。由于是用 A 的私钥对消息加密,所以只有 A 可加密消息,因此,整个加密后的消息就是数字签名。除此之外,因为只有拥有 A 的私钥才能产生上述加密后的消息,因此该消息可用于认证源和数据完整性。

上述方法是对整条消息加密,尽管这种方法可以验证发送方和内容的有效性,但却需要大量的存储空间。在实际使用中,每个文件必须既要以明文形式保存,又要以密文形式保存,以便在发生争执时可以验证消息源及其发送的消息。解决这个问题的更有效途径是,只对一个称为认证符的小数据块加密,它是该消息的函数,对该消息的任何修改必然会引起认证符的变化。如果用发送方的私钥对认证符加密,那么加密的结果就可作为数字签名,它能验证消息源、消息和通信序列的有效性。我们将在第 13 章中详细讨论这种方法。

需要强调指出的是,在图 9.1(b)和图 9.3 中描述的加密过程不能保证消息的保密性,也就是说,它可以防止发送的消息被修改,但不能防止搭线窃听。在基于对部分消息签名的方法中,因为消息的其余部分是以明文形式传输的,所以这种方法显然不能保证保密性。由于任何人都可以用发送方的公钥对消息解密,所以即使用图 9.3 所示的方法对整条消息加密,也不能保证被发送消息的保密性。

但是,如果两次使用公钥方法,则既可提供认证功能,又可保证被发送消息的保密性(参见图 9.4):

$$Z = E(PU_b, E(PR_a, X))$$

$$X = D(PU_a, D(PR_b, Z))$$

在这种方法中，发送方首先用其私钥对消息加密，得到数字签名，然后再用接收方的公钥加密，所得的密文只能被拥有相应私钥的接收方解密，这样可保证消息的保密性，但这种方法的缺点是，在每次通信中要执行四次复杂的公钥算法而不是两次。

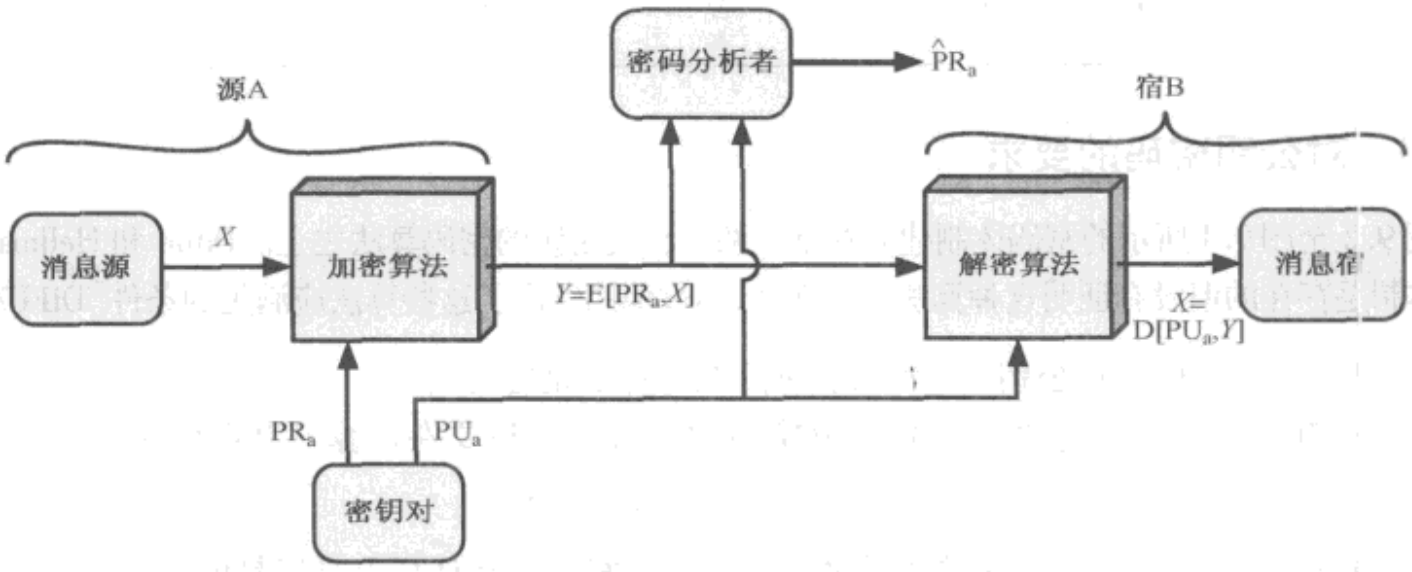


图 9.3　公钥密码体制：认证

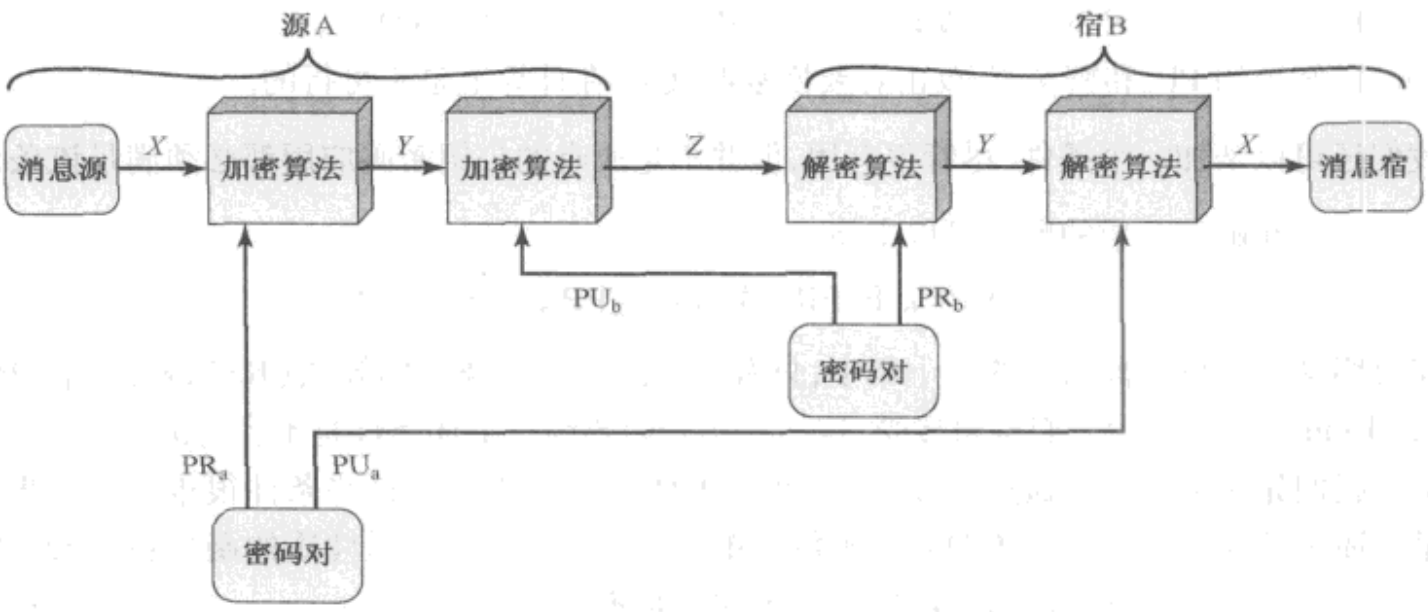


图 9.4　公钥密码体制：保密性和认证

9.1.2　公钥密码体制的应用

我们首先必须澄清公钥密码体制中容易引起混淆的一个问题。公钥密码体制的特点是使用具有两个密钥的密码算法，其中一个密钥是私有的，另一个是公开的。根据不同的应用，发送方可使用其私钥或者接收方的公钥或同时使用二者来执行密码功能。一般，公钥密码体制的应用可分为三类：

9.1.1 有explain

- **加密/解密**：发送方用接收方的公钥对消息加密。
- **数字签名**：发送方用其私钥对消息“签名”。签名可以通过对整条消息加密或者对消息的一个小的数据块加密来产生，其中该小数据块是整条消息的函数。
- **密钥交换**：通信双方交换会话密钥。有几种不同的方法可用于密钥交换，这些方法都使用了通信一方或双方的私钥。

有些算法可用于上述三种应用，而其他一些算法则只适用其中一种或两种应用，表 9.3 列出了本书中所讨论的算法及其所支持的应用。

表 9.3 公钥密码体制的应用

算 法	加密/解密	数字签名	密钥交换
RSA	是	是	是
椭圆曲线	是	是	是
Diffie-Hellman	否	否	是
DSS	否	是	否

9.1.3 对公钥密码的要求

图 9.2 至图 9.4 所示的密码体制建立在基于两个相关密钥的密码算法之上。Diffie 和 Hellman 假定这一体制是存在的但没有证明这种算法的存在性,不过他们给出了这些算法应满足的条件[DIFF76b]:

(1) B 产生一对密钥(公钥 PU_b,私钥 PR_b)在计算上是容易的。

(2) 已知公钥和要加密的消息 M,发送方 A 产生相应的密文在计算上是容易的:

$$C = E(PU_b, M)$$

(3) 接收方 B 使用其私钥对接收的密文解密以恢复明文在计算上是容易的:

$$M = D(PR_b, C) = D[PR_b, E(PU_b, M)]$$

(4) 已知公钥 PU_b 时,攻击者要确定私钥 PR_b 在计算上是不可行的。

(5) 已知公钥 PU_b 和密文 C,攻击者要恢复明文 M 在计算上是不可行的。

我们还可以增加一个条件,尽管很有用,但并不是所有的公钥密码应用都必须满足该条件。

(6) 加密和解密函数的顺序可以交换:

$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)]$$

在公钥密码学概念提出后的几十年中,只有几个满足这些条件的算法(RSA,椭圆曲线密码体制,Diffe-Hellman,DSS)为人们普遍接受,这一事实表明要满足上述条件是不容易的。

下面我们先对上述条件做进一步分析,然后再详细说明为什么这些条件很难满足。事实上,要满足上述条件即是要找一个单向陷门函数。单向函数①是满足下列性质的函数:每个函数值都存在唯一的逆,并且计算函数值是容易的,但求逆却是不可行的:

$$Y = f(X) \quad \text{容易}$$

$$X = f^{-1}(Y) \quad \text{不可行}$$

通常,“容易”是指一个问题可以在输入长度的多项式时间内得到解决,即若输入长度为 n 位,则计算函数值的时间与 n^a 成正比,其中 a 是一个固定的常数,这样的算法称为是 P 类算法。“不可行”的定义比较模糊,一般而言,若解决一个问题所需的时间比输入规模的多项式时间增长更快,则称该问题是不可行的。例如,若输入长度是 n 位,计算函数的时间与 2^n 成正比,则认为是不可行的。遗憾的是,我们很难确定算法是否具有这种复杂性。另外,传统的计算复杂性注重于算法的最坏情况或平均情况复杂性,但是最坏情况复杂性和平均情况复杂性这种方法不适用于密码学,因为密码学要求对任何输入都不能求出函数的逆,而不是在最坏情况或平均情况下不能求出函数的逆。附录 9B 简要介绍了上述有关概念。

下面给出单向陷门函数的定义。一个函数,若计算函数值很容易,并且在缺少一些附加信息

① 请不要将单向函数与单向 Hash 函数混淆。单向 Hash 函数的输入可以是任意长的数据,其输出是定长的,这些函数用于消息认证中(参见第 11 章)。

时计算函数的逆是不可行的，但是已知这些附加信息时，可在多项式时间内计算出函数的逆，那么我们称这样的函数为单向陷门函数，即单向陷门函数是满足下列条件的一类不可逆函数 f_k：

若 k 和 X 已知，则容易计算 $Y=f_k(X)$。
若 k 和 Y 已知，则容易计算 $X=f_k^{-1}(Y)$。
若 Y 已知但 k 未知，则计算出 $X=f_k^{-1}(Y)$ 是不可行的。

由此可见，寻找合适的单向陷门函数是公钥密码体制应用的关键。

9.1.4　公钥密码分析

和对称密码一样，公钥密码也易受穷举攻击，其解决方法也是使用长密钥。但同时也应考虑使用长密钥的利弊，公钥体制使用的是某种可逆的数学函数，计算函数值的复杂性可能不是密钥长度的线性函数，而是比线性函数增长更快的函数，因此，为了抗穷举攻击，密钥必须足够长；同时为了便于实现加密和解密，密钥又必须足够短。在实际中，现已提出的密钥长度确实可以抗穷举攻击，但是它也使加/解密速度太慢，所以公钥密码目前仅限于密钥管理和签名中。

对公钥密码的另一种攻击方法是，找出一种从给定的公钥计算出私钥的方法。到目前为止还未在数学上证明对一特定公钥算法这种攻击是不可行的，所以包括已被广泛使用的 RSA 在内的任何算法都是值得怀疑的。密码分析的历史表明，同一个问题从一个角度看是不可解的，但从另一个不同的角度来看则可能是可解的。

最后，还有一种攻击形式是公钥体制中所特有的，这种攻击本质上就是穷举消息攻击。例如，假定要发送的消息是 56 位的 DES 密钥，那么攻击者可以用公钥对所有可能的密钥加密，并与传送的密文匹配，从而可解密任何消息。因此，无论公钥体制的密钥有多长，这种攻击都可以转化为对 56 位密钥的穷举攻击。抗这种攻击的方法是，在要发送的消息后附加上一个随机数。

9.2　RSA 算法

Diffie 和 Hellman 在其早期的著名论文[DIFF76b]中提出了一种新的密码学方法，事实上，它对密码学家提出了一种挑战，即要去寻找满足公钥体制要求的密码算法。之后很多算法被提出，其中有一些刚提出时似乎很有前途，但后来都被攻破①。

MIT 的 Ron Rivest，Adi Shamir 和 Len Adleman 于 1977 年提出并于 1978 年首次发表的算法[RIVE78]②，可以说是最早提出的成功的公钥算法之一。Rivest-Shamir-Adleman（RSA）算法自其诞生之日起就成为被广泛接受且被实现的通用的公钥加密方法。

RSA 体制是一种分组密码，其明文和密文均是 0 至某 $n-1$ 之间的整数，通常 n 的大小为 1024 位二进制数或 309 位十进制数，也就是说 n 小于 2^{1024}。本节我们详细讨论 RSA 算法。首先我们给出该算法的描述，然后讨论 RSA 算法的计算问题和密码分析问题。

9.2.1　算法描述

RSA 算法使用乘方运算，明文以分组为单位进行加密，每个分组的二进制值均小于 n，也就是说，分组的大小必须小于或等于 $\log_2(n)+1$ 位，在实际应用中，分组的大小是 i 位，其中 $2^i < n \leqslant 2^{i+1}$。对明文分组 M 和密文分组 C，加密和解密过程如下：

① 最著名的算法是由 Ralph Merkle 提出的背包公钥密码，这部分内容可参阅附录 J。
② 第一个可执行的公钥加密/解密系统是由英国 CESG 的 Clifford Cocks 于 1973 年提出的[COCK73]；Cock 的方法本质上与 RSA 是相同的。

$$C = M^e \bmod n$$

$$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$$

其中收发双方均已知 n,发送方已知 e,只有接收方已知 d,因此公钥加密算法的公钥为 $PU = \{e, n\}$,私钥为 $PR = \{d, n\}$。该算法要能用做公钥加密,必须满足下列条件:

(1) 可以找到 e,d 和 n,使得对所有 $M < n$,有 $M^{ed} \bmod n = M$。

(2) 对所有 $M < n$,计算 M^e 和 C^d 是比较容易的。

(3) 由 e 和 n 确定 d 是不可行的。

我们先讨论第一个问题,其他问题将在以后讨论。我们需要找出下列关系式:

$$M^{ed} \bmod n = M$$

当 e 和 d 互为模 $\phi(n)$ 的乘法逆时,上述的关系式成立,其中 $\phi(n)$ 为欧拉函数。在第 8 章中已经证明,对素数 p 和 q,有 $\phi(pq) = (p-1)(q-1)$。e 和 d 的关系如下:

$$ed \bmod \phi(n) = 1 \tag{9.1}$$

上式等价于

$$ed \equiv 1 \bmod \phi(n)$$

$$d \equiv e^{-1} \bmod \phi(n)$$

也就是说,d 和 e 是模 $\phi(n)$ 乘法逆元。根据模算术的性质,仅当 d 与 $\phi(n)$ 互素[因此 e 也与 $\phi(n)$ 互素],即 $\gcd(\phi(n), d) = 1$ 时,d 和 e 是模 $\phi(n)$ 的乘法逆元。式(9.1)满足 RSA 要求的证明参见附录 9A。

下面我们介绍 RSA 算法,该算法用到下列元素:

两个素数 p,q	(保密的,选定的)
$n = pq$	(公开的,计算得出的)
e 满足 $\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$	(公开的,选定的)
$d \equiv e^{-1} \pmod{\phi(n)}$	(保密的,计算得出的)

这里,私钥为 $\{d, n\}$,公钥为 $\{e, n\}$。假定用户 A 已公布了其公钥,用户 B 要发送消息 M 给 A,那么用户 B 计算 $C = M^e \pmod n$,并发送 C;在接收端,用户 A 计算 $M = C^d \pmod n$ 以解密出消息 M。

图 9.5 归纳总结了 RSA 算法。与图 9.1 描述的一样,Alice 产生一对公钥/私钥,Bob 用 Alice 的公钥加密,Alice 用自己的私钥解密。图 9.6 所示的是[SING99]中给出的一个例子。其密钥产生过程如下:

Eg=

(1) 选择两个素数,$p = 17$,$q = 11$。

(2) 计算 $n = pq = 17 \times 11 = 187$。

(3) 计算 $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$。

(4) 选择 e 使其与 $\phi(n) = 160$ 互素且小于 $\phi(n)$,这里选择 $e = 7$。

(5) 确定 d 使得 $de \equiv 1 \bmod 160$ 且 $d < 160$。因为 $23 \times 7 = 161 = 1 \times 160 + 1$,所以 $d = 23$。d 可利用扩展的 Euclid 算法来计算(参见第 4 章)。

所得的公钥 $PU = \{7, 187\}$,私钥 $PR = \{23, 187\}$。该例说明了输入明文 $M = 88$ 时这些密钥的使用情况。加密时,需计算 $C = 88^7 \bmod 187$。利用模算术的性质,我们如下计算:

$$88^7 \bmod 187 = [(88^4 \bmod 187) \times (88^2 \bmod 187) \times (88^1 \bmod 187)] \bmod 187$$

$$88^1 \bmod 187 = 88$$

$88^2 \bmod 187 = 7744 \bmod 187 = 77$

$88^4 \bmod 187 = 59,969,536 \bmod 187 = 132$

$88^7 \bmod 187 = (88 \times 77 \times 132) \bmod 187 = 894,432 \bmod 187 = 11$

密钥产生	
选择 p,q	p 和 q 都是素数，$p \neq q$
计算 $n = p \times q$	
计算 $\phi(n) = (p-1)(q-1)$	
选择整数 e	$\gcd(\phi(n), e) = 1;\ 1 < e < \phi(n)$
计算 d	$d \equiv e^{-1} \pmod{\phi(n)}$
公钥	$PU = \{e, n\}$
私钥	$PR = \{d, n\}$

加　密	
明文	$M < n$
密文	$C = M^e \pmod n$

解　密	
密文	C
明文	$M = C^d \pmod n$

图 9.5　RSA 算法

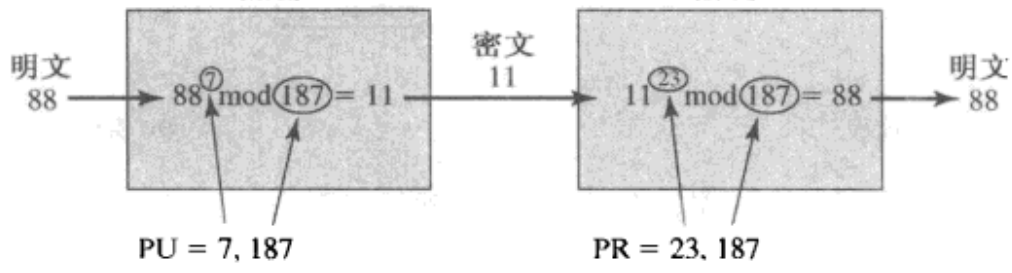


图 9.6　RSA 算法举例

解密时，我们计算 $M = 11^{23} \bmod 187$

$11^{23} \bmod 187 = [(11^1 \bmod 187) \times (11^2 \bmod 187) \times (11^4 \bmod 187) \times (11^8 \bmod 187) \times (11^8 \bmod 187)] \bmod 187$

$11^1 \bmod 187 = 11$

$11^2 \bmod 187 = 121$

$11^4 \bmod 187 = 14,641 \bmod 187 = 55$

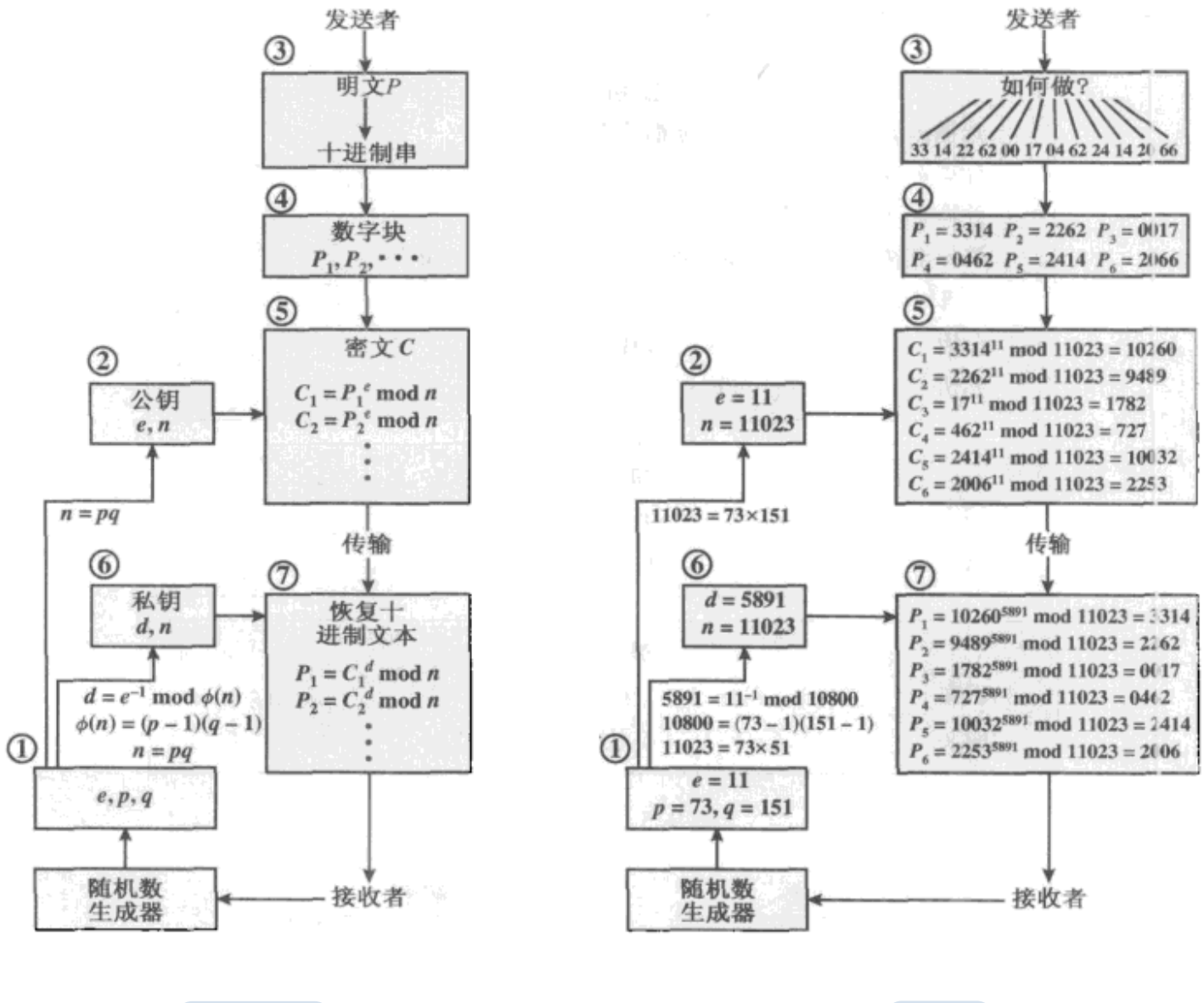
$11^8 \bmod 187 = 214,358,881 \bmod 187 = 33$

$11^{23} \bmod 187 = (11 \times 121 \times 55 \times 33 \times 33) \bmod 187 = 79,720,245 \bmod 187 = 88$

在[HELL79]中给出了一个使用 RSA 对不同的分组块进行加解密的例子，这个例子中，明文是一串字母，每个字母与一个两位的十进制数字对应（如 a = 00，A = 26）①，明文的每一分组块由四

① 完整的字母与十进制数字的对应可在本书提供的网站中下载，文件名是 RSAexample. pdf。

个十进制数字组成,即两个字母。图9.7(a)给出了加密各分组块的过程,图9.7(b)给出了一个实例。图中的数字表示运算的执行顺序。



(a) 一般过程　　(b) 实例

图 9.7　RSA 的加、解密过程

9.2.2 计算方面的问题

下面我们讨论 RSA 的计算复杂性问题,它实际上包括两方面的问题:加密/解密和密钥产生。我们先讨论加密和解密过程,然后再讨论密钥产生问题。

模算术里的求幂运算 快速求幂模

在 RSA 中,加密和解密都需要计算某整数的模 n 整数次幂,如果先求出整数的幂,再对 n 取模,那么中间结果会非常大。幸运的是,正如前面的例子所示,我们可利用模算术的下列性质来计算模幂运算:

$$[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$$

这样我们将中间结果对 n 取模,这使得计算切实可行。

因为 RSA 中所用到的指数很大,所以还应考虑幂运算的效率问题。为说明如何增加效率,以计算 x^{16} 为例。若直接计算则需进行 15 次乘法:

$$x^{16} = x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x$$

如果重复计算每个中间结果的平方,得到 x^2,x^4,x^8 和 x^{16},那么只需 4 次乘法即可计算出 x^{16}。又比如,对于整数 x 和 n,计算 $x^{11} \bmod n$。由于 $x^{11} = x^{1+2+8} = (x)(x^2)(x^8)$,我们先计算 $x \bmod n$,

2的幂次次幂,然后相加

$x^2 \bmod n, x^4 \bmod n, x^8 \bmod n$，再计算$[(x \bmod n) \times (x^2 \bmod n) \times (x^8 \bmod n)] \bmod n$。

更一般地，假定我们要计算 a^b，其中 a 和 b 是正整数。若将 b 表示为二进制数 $b_k b_{k-1} \cdots b_0$，则

$$b = \sum_{b_i \neq 0} 2^i$$

所以

$$\begin{cases} a^b = a^{\left(\sum_{b_i \neq 0} 2^i\right)} = \prod_{b_i \neq 0} a^{(2^i)} \\ a^b \bmod n = \left[\prod_{b_i \neq 0} a^{(2^i)}\right] \bmod n = \left(\prod_{b_i \neq 0} \left[a^{(2^i)} \bmod n\right]\right) \bmod n \end{cases}$$

下面我们讨论计算 $a^b \bmod n$ 的算法①，如图 9.8 所示。表 9.4 举例说明了该算法的执行过程。请注意，这里的变量 c 不是必需的，引入它只是为了便于解释算法，c 的终值即是指数。

```
c ← 0; f ← 1
for i ← k downto 0
    do  c ← 2 × c
        f ← (f × f) mod n
    if  b_i = 1
        then c ← c + 1
             f ← (f × a) mod n
return f
```

注：整数6表示为二进制$b_k b_{k-1} \cdots b_0$。

图 9.8　计算 $a^b \bmod n$ 的算法

表 9.4　计算 $a^b \bmod n$ 的快速模幂算法，其中 $a=7, b=560=1000110000, n=561$

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

用公钥进行有效运算

为了加快 RSA 算法在使用公钥时的运算速度，通常都会选择一个特定的 e，大多数情况下选 e 为 65 537$(2^{16}+1)$，另外两个常用的选择是 3 和 17。这些指数里都只有两个位，所以求幂运算时需要的乘法次数极小化了。

然而，若指数太小，如 $e=3$，RSA 甚至会遭受一些简单的攻击。假设有三个不同的 RSA 用户，他们都使用指数 $e=3$，但他们的模数 n 却各不相同，即 n_1, n_2, n_3。现在有一个用户 A 以加密方式给他们三个发送了相同的消息 M，则三个密文为：$C_1 = M^3 \bmod n_1, C_2 = M^3 \bmod n_2, C_3 = M^3 \bmod n_3$。很有可能 n_1, n_2, n_3 是两两互素的，所以运用中国剩余定理(CRT)可以计算出 $M^3 \bmod (n_1 n_2 n_3)$。根据 RSA 的规则，M 应该比模数 n_i 要小，所以有 $M^3 < n_1 n_2 n_3$，从而攻击者只需要计算 M^3 的三次立方根就可以了。给每一个待加密的消息 M 填充一个唯一的伪随机位串可以阻止这种攻击，随后会讨论这个问题。

① 该算法已经提出了很长时间；这里给出的伪码取自[CORM04]。

读者也许会注意到 RSA 定义(参见图 9.5)里,在密钥产生阶段要求用户选择的 e 要与 $\phi(n)$ 互素。因此,如果用户选择了 $e=65537$,接着生成了素数 p 和 q,很有可能 $\gcd(\phi(n),e)\neq 1$。因此,用户应该去掉那些模 65537 和 1 不同余的 p 和 q。不应该是同余的呀 $(p-1)(q-1)$

用私钥进行有效运算

我们不能为了计算的效率而简单地选择一个小数值的 d。d 的值太小容易遭受穷举攻击和其他形式的密码分析[WIEN90]。然而,运用中国剩余定理可以加快运算速度。我们希望计算 $M=C^d \bmod n$。先定义一些中间结果:

$$V_p = C^d \bmod p \qquad V_q = C^d \bmod q$$

运用 CRT,式(8.8),定义

$$X_p = q \times (q^{-1} \bmod p) \qquad X_q = p \times (p^{-1} \bmod q)$$

用式(8.9),由 CRT 可得

$$M = (V_p X_p + V_q X_q) \bmod n$$

进一步地,我们可以用费马定理来简化 V_p 和 V_q 的计算,即根据:如果 p 和 a 互素,则 $a^{p-1}\equiv 1 \pmod p$。不难看出如下式子是正确的:

$$V_p = C^d \bmod p = C^{d \bmod (p-1)} \bmod p \qquad V_q = C^d \bmod q = C^{d \bmod (q-1)} \bmod q$$

$d \bmod (p-1)$ 和 $d \bmod (q-1)$ 可以预先计算出来。与直接计算 $M=C^d \bmod n$ 相比,上述的计算速度大约快 4 倍[BONE02]。

密钥产生

在应用公钥密码进行通信之前,通信各方都必须产生一对密钥,即需做以下工作:

- 确定两个素数 p 和 q。
- 选择 e 或者 d,并计算 d 或者 e。

我们首先考虑 p 和 q 的选择问题。由于任何攻击者可以知道 $n=pq$,所以为了避免攻击者用穷举法求出 p 和 q,应该从足够大的集合中选取 p 和 q(即 p 和 q 必须是大素数)。另一方面,选择大素数的方法必须是有效的。

目前还没有有效的方法可以产生任意大素数,因此需要使用其他方法来解决这一问题。通常使用的方法是,随机挑选一个期望大小的奇数,然后测试它是否是素数。若不是,则挑选下一个随机数直至检测到素数为止。

各种素性测试方法因此应运而生(如[KNUT98]中介绍了若干素性测试方法),它们几乎全都是概率测试方法,也就是说,这些测试只能确定一个给定的整数可能是素数。尽管存在这种不确定性,但是以某种方式执行这些测试可使得一个整数是素数的概率接近 1.0。例如第 8 章中介绍的 Miller-Rabin 算法就是非常有效且被广泛使用的素数测试方法。Miller-Rabin 算法及其他许多类似算法,测试一个给定的数 n 是否是素数的过程,即执行某种计算,这些计算涉及 n 和一个随机选择的整数 a。若 n"未通过"测试,则 n 不是素数;若 n 通过了测试,则 n 可能是素数,也可能不是素数。若对许多随机选择的不同的 a,n 均能通过测试,则我们几乎可以相信 n 就是素数。

归纳起来,挑选素数的过程如下:

(1) 随机选择一个奇整数 n(如利用伪随机数产生器)。

(2) 随机选择一个整数 $a < n$。

(3) 执行诸如 Miller-Rabin 之类的概率素数测试。若 n 未通过测试,则拒绝 n,并转到步骤(1)。

(4) 若 n 通过测试足够多次,则接受 n;否则转到步骤(2)。

这个过程有些烦琐,但是,一般不会很频繁地执行这个过程,因为只有在需要一对新的密钥(PU,PR)时才会执行它。

值得注意的是,在找到一个素数之前可能有多少个整数会被拒绝。由数论中的素数定理可知,在 N 附近平均每隔 $(\ln N)$ 个整数就有一个素数,这样在找到一个素数之前,平均要测试大约 $\ln(N)$ 个整数。由于每个偶数会被立刻拒绝,所以实际上只需测试大约 $\ln(N)/2$ 个整数。例如,要找一个 2^{200} 左右的素数,则在找到素数之前大约要进行 $\ln(2^{200})/2 = 70$ 次尝试。

若确定了素数 p 和 q,则可通过选择 e 并计算 d 或者选择 d 并计算 e 来产生密钥。假定是前者,那么我们需要选择满足 $\gcd(\phi(n), e) = 1$ 的 e,并计算 $d \equiv e^{-1} \bmod \phi(n)$。幸运的是,存在求两个整数的最大公因子的算法,并且在它们的最大公因子为 1 时,该算法还能同时求出其中一数对另一数取模的逆元,这个算法称为扩展的 Euclid 算法,已在第 4 章中讨论过。由上可知,产生密钥的过程就是要产生若干个随机数,直至找到与 $\phi(n)$ 互素的数为止。我们再次提出这样一个问题:在找到一个可用的数,即与 $\phi(n)$ 互素的数之前,要测试多少个随机数呢?可以很容易地证明,两个随机数互素的概率约为 0.6,因此找到一个恰当的数之前只需进行非常少的测试(参见习题 8.2)。

9.2.3 RSA 的安全性

对 RSA 算法的攻击可能有如下 4 种方式:

- **穷举攻击**:这种方法试图穷举所有可能的私钥。
- **数学攻击**:有多种数学攻击方法,它们的实质都是试图分解两个素数的乘积。
- **计时攻击**:这类方法依赖于解密算法的运行时间。
- **选择密文攻击**:这种攻击利用了 RSA 算法的性质。

如其他的密码体制一样,RSA 抗穷举攻击的方法也是使用大密钥空间,所以 e 和 d 的位数越大越好,但是密钥产生过程和加/解密过程都包含复杂的计算,因此密钥越大,系统运行速度越慢。

在本小节中,我们简要介绍数学攻击和计时攻击。

因子分解问题

用数学方法攻击 RSA 的途径有以下三种:

(1) 分解 n 为两个素因子。这样就可以计算出 $\phi(n) = (p-1)(q-1)$,从而可以确定 $d \equiv e^{-1} (\bmod \phi(n))$。

(2) 直接确定 $\phi(n)$ 而不先确定 p 和 q。这同样也可以确定 $d = e^{-1} (\bmod \phi(n))$。

(3) 直接确定 d,而不先确定 $\phi(n)$。

对 RSA 的密码分析的讨论大都集中于第一种攻击方法,即将 n 分解为两个素数因子。由给定的 n 来确定 $\phi(n)$ 等价于因子分解 n[RIBE96]。现在已知的、从 e 和 n 确定 d 的算法至少和因子分解问题一样费时[KALI95]。因此,我们通过将因子分解的性能作为基准来评价 RSA 的安全性。

尽管因子分解具有大素数因子的数 n 仍然是一个难题,但已不像以前那么困难。下面我们来看一个著名的例子。1977 年 RSA 的三位发明者让杂志 *Scientific American* 的读者对他们发表在 Martin Gardner 上"数学游戏"专栏[LEUT94]中的密文进行解密,解得明文则可获得 100 美元奖金,他们预言需要 4×10^{16} 年才能解得明文。但是,一个研究小组利用 Internet 网只用了 8 个月的时间,于1994 年 4 月解决了这个问题。这里所使用的公钥大小(n 的长度)是 129 位十进制数,即约 428 位二进制数。与对 DES 算法一样,RSA 实验室同时还发布了用位数为 100,110 或 120 等的密钥加密的密文供有兴趣者解密。最近被

解密的是RSA-200,其密钥长度为200的十进制位或663位位。表9.5给出了迄今为止得出的一些结果。我们用MIPS年来描述计算的代价。MIPS年是指一台每秒执行百万条指令的处理器运行一年,即执行约3×10^{13}条指令。一台1 GHz的Pentium机约等于一台250 MIPS机器。

表9.5 因子分解问题的进展情况

十进制位数	二进制位数(近似值)	完成日期	MIPS年	算法
100	332	1991年4月	7	二次筛法
110	365	1992年4月	75	二次筛法
120	398	1993年6月	830	二次筛法
129	428	1994年4月	5000	二次筛法
130	431	1996年4月	1000	一般数域筛法
140	465	1999年2月	2000	一般数域筛法
155	512	1999年8月	8000	一般数域筛法
160	530	2003年4月	——	格筛法
174	576	2003年12月	——	格筛法
200	663	2005年5月	——	格筛法

值得注意的是表9.5所使用的因子分解方法。在20世纪90年代中期以前一直是用二次筛法来进行因子分解,对RSA-130的攻击使用了称为一般数域筛法(GNFS)的新算法,该算法能够因子分解比RSA-129更大的数,但计算代价仅是二次筛法的20%。

计算能力的不断增强和因子分解算法的不断改进,给大密钥的使用造成威胁。我们已经看到,更换一种算法可使速度显著增加。我们期望GNFS还可以进一步改进,有可能设计出更好的算法。事实上,对某种特殊形式的数,用特殊数域筛法(SNFS)算法进行因子分解比用一般数域筛法要快得多,图9.9对这两种算法的性能进行了比较。我们可以期望算法上会有所突破,使一般的因子分解的性能在时间上大约与SNFS一样或者甚至比SNFS更快[ODLY95]。因此我们在选择RSA的密钥大小时应谨慎小心,在最近一段时间里,密钥大小取在1024到2048位是合适的。

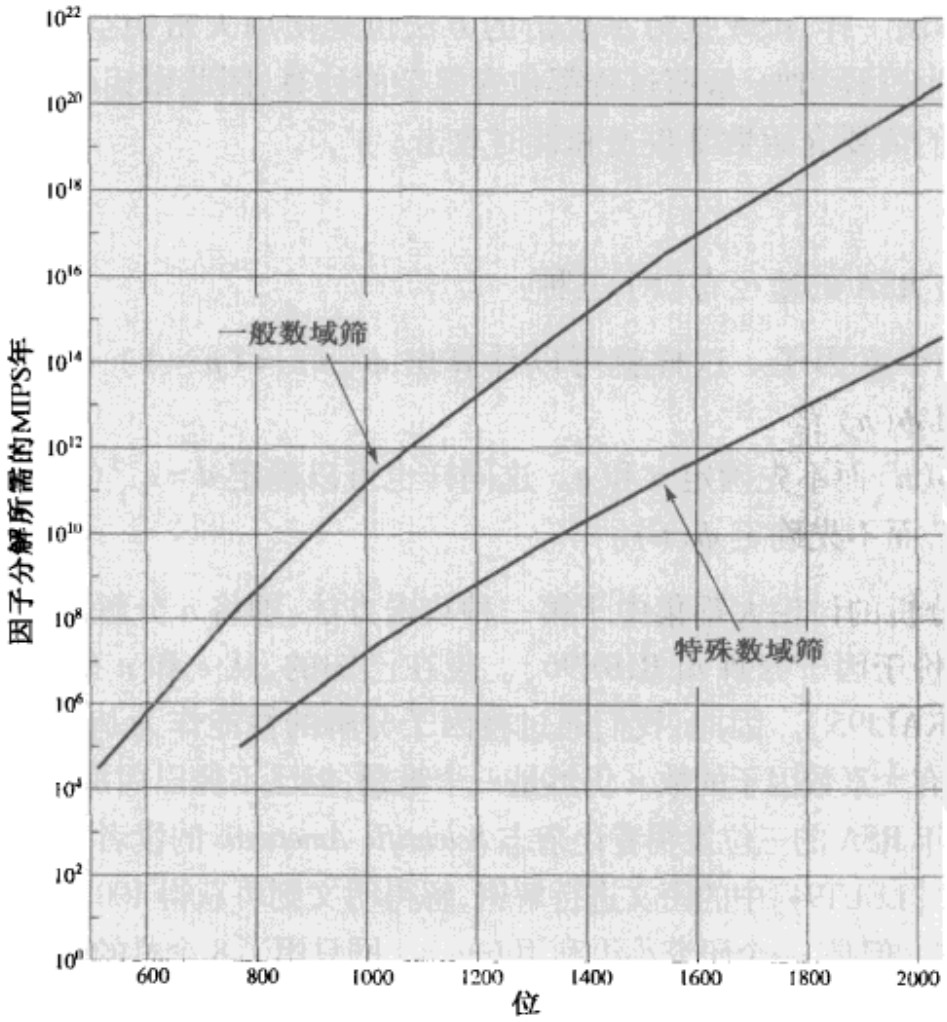


图9.9 因子分解所需的MIPS年

除了要指定 n 的大小外，研究者还提出了其他一些限制条件。为了防止可以很容易地分解 n，RSA 算法的发明者建议 p 和 q 还应满足下列限制条件：

（1）p 和 q 的长度应仅相差几位。这样对 1024 位（309 个十进制位）的密钥而言，p 和 q 都应约在 10^{75} 到 10^{100} 之间。

（2）$(p-1)$ 和 $(q-1)$ 都应有一个大的素因子。

（3）gcd $(p-1, q-1)$ 应该较小。

另外，已经证明，若 $e < n$ 且 $d < n^{1/4}$，则 d 很容易被确定[WIEN90]。

计时攻击

如果你想知道评价密码算法的安全性有多难，那么计时攻击的出现就是最好的例子。密码学顾问 Paul Kocher 已证明，攻击者可以通过记录计算机解密消息所用的时间来确定私钥[KOCH96, KALI96b]。计时攻击不仅可用于攻击 RSA，而且也可以用于攻击其他的公钥密码系统，由于这种攻击的完全不可预知性以及它仅依赖于密文，所以计时攻击具有很大的威胁。

计时攻击类似于窃贼通过观察他人转动保险柜拨号盘的时间长短来猜测密码，我们可以通过图 9.8 中的模幂算法来说明这种攻击，但这种攻击可以攻击任何运行时间不固定的算法。如图 9.8 所示的算法中，模幂运算是通过一位一位来实现的，每次迭代执行一次模乘运算，且若该位为 1，则还需再执行一次模乘运算。

正如 Kocher 在其论文中所指出的，在下述极端情况下，我们很容易理解计时攻击的含义。假定在模幂算法中，模乘函数的执行时间只在几种情形中其执行时间比整个模幂运算的平均执行时间要长得多，但在大多情形下其执行速度相当快。计时攻击是从最左位 b_k 开始，一位一位地进行的。假设攻击者已知前面的 j 位（为了得到整个指数，攻击者可以从 $j=0$ 开始，重复攻击直至已知整个指数为止），则对给定的密文，攻击者可以完成 for 循环的前 j 次迭代，其后的操作依赖于未知的指数位。若该位为 1，则要执行 $d \leftarrow (d \times a) \bmod n$。对有些 a 和 d 的值，模乘运算的执行速度异常慢，假定攻击者知道是哪些值。由于位为 1 时，对这些值执行迭代的速度很慢，若攻击者观察到解密算法的执行总是很慢，则可认为该位为 1；若攻击者多次观察到整个算法的执行都很快，则可认为该位为 0。

在实际中，模幂运算的实现并没有这样大的时间差异，使得一次迭代的执行时间会超过整个算法的平均执行时间，但存在有足够大的差异使得计时攻击切实可行，关于这个问题的详细讨论请参见[KOCH96]。

尽管计时攻击会造成严重的威胁，但是有一些简单可行的解决方法，包括：

- **不变的幂运算时间**：保证所有的幂运算在返回结果前执行的时间都相同。这种方法虽然很简单，但会降低算法的性能。
- **随机延时**：通过在求幂算法中加入随机延时来迷惑计时攻击者可提高性能，Kocher 认为，如果不增加足够的干扰，那么攻击者可以通过收集额外的观察数据来抵消随机延时，仍然可能攻击成功。
- **隐蔽**：在执行幂运算之前先将密文乘上一个随机数，这一过程可使攻击者不知道计算机正在处理的是密文的哪些位，这样可防止攻击者一位一位地进行分析，而这种分析正是计时攻击的本质所在。

RSA 数据安全公司（RSA Data Security）在他们的产品里就使用了隐蔽方法，它用私钥实现操作 $M = C^d$ 的过程如下：

(1) 产生 0 至 $n-1$ 之间的秘密的随机数 r。

(2) 计算 $C' = C(r^e) \bmod n$,其中 e 是公开的指数。

(3) 像通常的 RSA 运算一样,计算 $M' = (C')^d \bmod n$。

(4) 计算 $M = M'r^{-1} \bmod n$,其中 r^{-1} 是 r 的模 n 的乘法逆元,关于乘法逆元的讨论参见第 8 章。根据 $r^{ed} \bmod n = r \bmod n$,可以证明结论是正确的。

RSA 数据安全公司声称由于使用了隐蔽方法,运算性能降低了 2% ~10%。

选择密文攻击和最佳非对称加密填充

基本的 RSA 算法易受选择密文攻击(CCA)。进行 CCA 攻击时,攻击者选择一些密文,并获得相应的明文,这些明文是利用目标对象的私钥解密获得的。因此,攻击者可以选择一个明文,运用目标对象的公钥加密,然后再用目标对象的私钥解密而取回明文。显然,这么做并没有给攻击者任何新的信息。可是攻击者可以利用 RSA 的性质,选择数据块使得当用目标对象的私钥处理时,产生密码分析所需要的信息。

利用 CCA 攻击 RSA 的一个简单例子利用了 RSA 如下的性质:

$$\mathrm{E}(\mathrm{PU}, M_1) \times \mathrm{E}(\mathrm{PU}, M_2) = \mathrm{E}(\mathrm{PU}, [M_1 \times M_2]) \tag{9.2}$$

利用 CCA 攻击,我们可以如下方式解密 $C = M^e \bmod n$。

(1) 计算 $X = (C \times 2^e) \bmod n$。

(2) 将 X 作为选择明文提交,并收到 $Y = X^d \bmod n$。

现在请注意:

$$\begin{aligned} X &= (C \bmod n) \times (2^e \bmod n) \\ &= (M^e \bmod n) \times (2^e \bmod n) \\ &= (2M)^e \bmod n \end{aligned}$$

因此,$Y = (2M) \bmod n$,由此,我们可以得到 M。为了防止这种简单攻击,实用的基于 RSA 的密码体制在加密之前都会对明文进行随机填充。这使得密文随机化了,从而式(9.2)不再成立。然而,复杂一些的 CCA 攻击仍然是可能的,简单的随机填充已经证明是不足以提高期望的安全性。为了防止这种攻击,RSA 安全公司,也是 RSA 的主要厂商,RSA 专利的持有人,推荐使用一种称为最优非对称加密填充(OAEP)的程序对明文进行修改。对安全威胁和 OAEP 的全面讨论超出了本书的范围,[POIN02]有一些介绍,而[BELL94a]则做了彻底的分析。这里,我们对 OAEP 做一些概括性介绍。

图 9.10 显示了 OAEP 加密算法。第一步,待加密的消息 M 被填充。可选参数集 P 作为 Hash 函数 H[①] 的输入,输出用 0 加以填充以获得期望的长度,从而可以放入整体数据块 DB 内。接着,随机选择一个种子,并作为一个 Hash 函数的输入,该 Hash 函数称为掩码生成函数(MGF)。输出 Hash 值和 DB 进行按位异或运算产生掩码 DB(maskedDB)。该 maskedDB 反过来又作为 MGF 的输入产生一个 Hash 值,该 Hash 值和种子进行异或运算产生掩码种子。掩码种子和掩码 DB 连接起来构成编码后的消息 EM。注意,EM 包含填充过的消息,该消息由种子进行掩码,而种子又由 maskedDB 进行掩码。最后用 RSA 对整个 EM 进行加密。

① Hash 函数将变长的数据块或消息映射为定长的值,称为 Hash 码。第 11 章对 Hash 函数有深入的讨论。

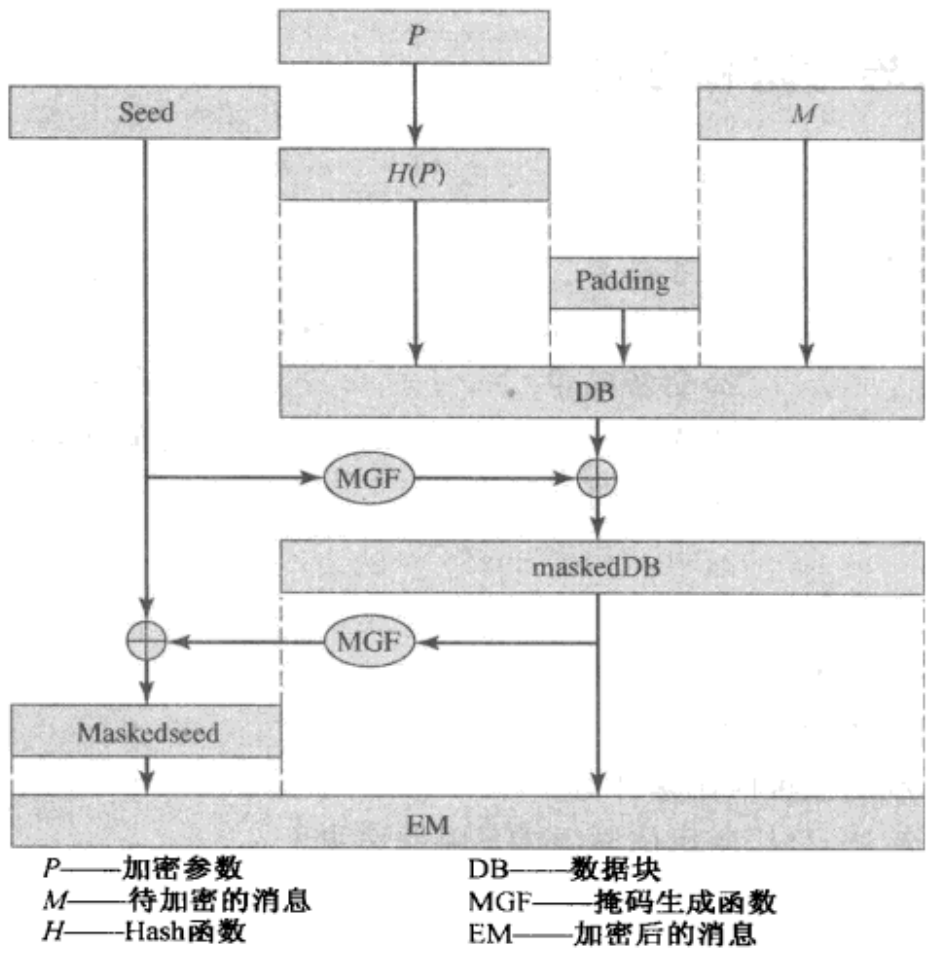


图 9.10　用最优非对称加密填充(OAEP)的加密算法

9.3　推荐读物和网站

第 3 章中给出的有关密码的推荐读物既包含对称密码,也包含公钥密码。

[DIFF88]详细描述了几种安全的双钥密码算法和基于这些算法的各种协议的发展过程。[SALO96]是一本关于公钥密码学的优秀书籍,[CORM04]是一本简明易读的书,它总结了与验证、计算和 RSA 密码分析有关的所有算法。[BONE99]讨论了对 RSA 的各种密码分析攻击。最近关于 RSA 的攻击的进展参见[SHAM03]。

BONE99 Boneh, D. "Twenty Years of Attacks on the RSA Cryptosystem." *Notices of the American Mathematical Society*, February 1999.

CORM04 Cormen, T.; Leiserson, C.; Rivest, R,; and Stein, C. *Introduction to Algorithms*. *Cambridge*, MA: MIT Press, 2001.

DIFF88 Diffie, W. "The First Ten Years of Public - Key Cryptography." *Proceedings of the IEEE*, May 1988.

SHAM03 Shamir, A. ,and Tromer, E. "On the Cost of Factoring RSA - 1024." *CryptoBytes*, Summer 2003. http://www.rsasecurity.com/rsalabs

推荐网站

- **RSA Laboratories**:广泛收集了密码学中有关 RSA 和其他密码学方面的技术资料。

9.4 关键术语、思考题和习题

关键术语

选择密文攻击(CCA)　数字签名　密钥交换
单向函数　最优非对称加密填充(OAEP)　私钥
公钥　公钥密码学　公钥密码体制
公钥加密　RSA　时间复杂性
计时攻击　单向陷门函数

思考题

9.1 公钥密码体制的主要成分是什么?
9.2 公钥和私钥的作用是什么?
9.3 公钥密码体制的三种应用是什么?
9.4 为得到安全算法,公钥密码体制应满足哪些要求?
9.5 什么是单向函数?
9.6 什么是单向陷门函数?
9.7 用一般术语描述一个挑选素数的有效过程。

习题

9.1 在任何诸如 RSA 的公钥体制出现之前,就已经有了关于公钥体制的存在性证明,其目的是为了说明公钥密码在理论上是可行的。考虑函数 $f_1(x_1)=z_1, f_2(x_2,y_2)=z_2, f_3(x_3,y_3)=z_3$,其中的值都是整数,且 $1 \leqslant x_i, y_i, z_i \leqslant N$。函数 f_1 可以用长为 N 的向量 $\boldsymbol{M}_1$ 表示,其中 $\boldsymbol{M}_1$ 的第 k 个分量即是 $f_1(k)$;同样,f_2 和 f_3 可分别用 $N \times N$ 矩阵 $\boldsymbol{M}_2$ 和 $\boldsymbol{M}_3$ 表示。这样表示的目的是希望通过查表实现加/解密过程。N 是一个很大的数,所以这些表也非常大,在实际中不可能实现这些表,但是原则上这些表是可构造的。其构造原理如下:首先,取 $\boldsymbol{M}_1$ 为 1 到 N 上的一个随机置换,即 1 到 N 之间的每个整数在 $\boldsymbol{M}_1$ 中恰好出现一次;$\boldsymbol{M}_2$ 的每一行都是前 N 个整数的随机置换;最后按下述条件生成 $\boldsymbol{M}_3$:

$$f_3(f_2(f_1(k),p),k)=p \text{ 对任意的 } k \text{ 和 } p, \text{且 } 1 \leqslant k,p \leqslant N$$

也就是说:

(1) $\boldsymbol{M}_1$ 的输入为 k,输出为 x。
(2) $\boldsymbol{M}_2$ 的输入为 x 和 p,输出为 z。
(3) $\boldsymbol{M}_3$ 的输入为 z 和 k,输出为 p。

然后,公布已构造好的三张表。

(a) 显然,可以构造出满足这些条件的 $\boldsymbol{M}_3$。例如,在下述简单的情况下可填充表 $\boldsymbol{M}_3$:

$\boldsymbol{M}_1 =$

5
4
2
3
1

$\boldsymbol{M}_2 =$

5	2	3	4	1
4	2	5	1	3
1	3	2	4	5
3	1	4	2	5
2	5	3	4	1

$\boldsymbol{M}_3 =$

约定:$\boldsymbol{M}_1$ 的第 i 分量对应 $k=i$;$\boldsymbol{M}_2$ 的第 i 行对应 $x=i$,$\boldsymbol{M}_2$ 的第 j 列对应 $p=j$;$\boldsymbol{M}_3$ 的第 i 行对应 $z=i$,$\boldsymbol{M}_3$ 的第 j 列对应 $k=j$。

(b) 说明如何使用上述各表在两个用户间实现加密和解密。

(c) 说明这是一种安全的方法。

9.2 用图 9.5 所示的 RSA 算法对下列数据实现加密和解密：

(a) $p=3$; $q=11$, $e=7$; $M=5$

(b) $p=5$; $q=11$, $e=3$; $M=9$

(c) $p=7$; $q=11$, $e=17$; $M=8$

(d) $p=11$; $q=13$, $e=11$; $M=7$

(e) $p=17$; $q=31$, $e=7$; $M=2$

提示：解密并不像你想象的那么难，请使用一些技巧。

9.3 在使用 RSA 的公钥体制中，已截获发给某用户的密文 $C=10$，该用户的公钥 $e=5$, $n=35$，那么明文 M 是多少？

9.4 在 RSA 体制中，某给定用户的公钥 $e=31$, $n=3599$，那么该用户的私钥等于多少？提示：首先用试探法决定 p 和 q，然后用扩展 Euclid 算法寻找 31 模 $\phi(n)$ 的乘法逆。

9.5 使用 RSA 算法时，可以从少量重复的编码中恢复出明文，其可能的原因是什么？

9.6 假定我们已知若干用 RSA 算法编码的分组但不知私钥，假设 $n=pq$, e 是公钥。若某人告诉我们说他知道其中有一个明文分组与 n 有公因子，这对我们有帮助吗？

9.7 在 RSA 公钥密码体制中，每个用户都有一个公钥 e，一个私钥 d。假定 Bob 的私钥已泄密。Bob 决定产生新的公钥和新的私钥，而不是产生新的模数，请问这样安全吗？

9.8 假设 Bob 使用 RSA 密码体制，其中模数 n 非常大以使得因子分解是不可行的。假设 Alice 给 Bob 发消息，其中的字母表示为 0 到 25($A\rightarrow 0, \cdots, Z\rightarrow 25$)。然后对每个字母用 RSA 算法单独加密，参数 e 和 n 都很大。这种方法安全吗？如果不安全，请给出最有效的攻击方法。

9.9 用电子制表软件(如 Excel)或计算器，做如下指定的运算。记录下所有模乘的中间结果。对每一个主要的变换(如加密，解密，素性测试等)判断其中的模乘运算。

(a) 以 2 为底，用 Miller-Rabin 测试算法检测位于 233 到 241 之间的所有奇数的素性。

(b) 用参数为 $e=23$, $n=233\times 241$ 的 RSA 算法加密消息分组 $M=2$。

(c) 根据上述给出的公钥(e,n)，计算相应的私钥(d,p,q)。

(d) 用如下两种不同的方法解密上述得到的密文：

(1) 不用中国剩余定理。

(2) 用中国剩余定理。

9.10 假设你按如下方式生成了一个经过认证且加密的消息：首先用私钥及 RSA 算法进行变换，然后用接受方的公钥及 RSA 算法对消息加密(注意在变换之前你没有使用 Hash 函数)。这个方案能正确工作吗[即，对于发送方的模数 n_S 和接受方的模数 n_R 之间的所有可能关系($n_S > n_R$, $n_S < n_R$, $n_S = n_R$)，给出接受方可以重构原来的消息的可能性]？对你的回答给出解释。如果你的答案是“否”，那么如何修改呢？

9.11 “我想告诉你，福尔摩斯，”华生医生激动地说，“你最近进行的网络安全活动让我对密码学产生了浓厚的兴趣，就在昨天，我发现一次一密的加密方法是可行的。”

“噢？真的吗？”福尔摩斯从睡意蒙胧中醒过来。“这么说，你找到了一种生成强密码序列的确定性方法了？”

“千真万确，福尔摩斯。这个想法倒是挺简单的。对给定的单向函数 F，通过将 F 应用于某标准的变量参数序列，我就产生了一个长伪随机数序列。假使密码分析者知道了 F 和序列的一般性质，这个性质可能很简单，比如 $S, S+1, S+2, \cdots$，而不知道 S，由于 F 的单向性，没人能够对某 i，从 $F(S+i)$ 推出 S，即使他得到了序列的一段，他也不能确定其他部分。”

“华生，我担心你的想法并非无懈可击，至少它要求 F 满足一些附加条件。我们考虑一下。例如，RSA 加密函数 $F(M)=M^K \bmod N$，K 是保密的，这个函数被认为是单向的，但是我不赞成将这种方

法用于类似 $M=2,3,4,5,6,\cdots$ 这样的序列。"

"为什么,福尔摩斯?"华生医生大惑不解,"为什么你认为,如果 K 是保密的,那么像 $2^K \bmod N$, $3^K \bmod N, 4^K \bmod N, \cdots$ 这样的序列不适合于一次一密?"

"因为它至少是部分可预测的,亲爱的华生,即使 K 是保密的,如你刚才所说,假定密码分析者知道了 F 和序列的一般性质,再假设他能截获一小段输出序列,在密码学界这种假设是可行的。对于该输出序列,已知最前面的两个元素,即使他不能预测出所有元素,但他可预测出该序列中后续的许多元素。因此这样的序列在密码学上不能被认为是强序列。利用预测出的较长的序列段,他可预测出序列中更多的元素。瞧,已知序列的一般性质和序列的前面两个元素:$2^K \bmod N, 3^K \bmod N$,就很容易计算出后续元素……"

请说明这是如何做到的?

9.12 说明如何通过习题 9.1 中的矩阵 $\boldsymbol{M}_1, \boldsymbol{M}_2$ 和 $\boldsymbol{M}_3$ 来描述 RSA。

9.13 考虑下列方法:

(1) 挑选一个奇数 E。

(2) 挑选两个素数 P 和 Q,其中 $(P-1)(Q-1)-1$ 是 E 的偶数倍。

(3) P 和 Q 相乘得 N。

(4) 计算 $D=\dfrac{(P-1)(Q-1)(E-1)+1}{E}$。

这种方法是否与 RSA 等价?请说明原因。

9.14 B 用下述方法对发送给 A 的消息加密:

(1) A 选择两个大素数 P 和 Q,它们与 $(P-1)$ 和 $(Q-1)$ 均互素。

(2) A 公布其公钥 $N=PQ$。

(3) A 计算 P' 和 Q',使得 $PP' \equiv 1 \pmod{Q-1}$ 且 $Q\,Q' \equiv 1 \pmod{P-1}$。

(4) 作为对消息 M 的加密,B 计算 $C=M^N \pmod N$。

(5) A 求解 $M \equiv C^{P'} \pmod Q$ 和 $M \equiv C^{Q'} \pmod P$ 得出 M。

(a) 试说明这种方法的工作原理。

(b) 它与 RSA 有什么不同?

(c) 与这种方法相比,RSA 有哪些优点?

(d) 说明如何通过习题 9.1 中的矩阵 $\boldsymbol{M}_1, \boldsymbol{M}_2$ 和 $\boldsymbol{M}_3$ 描述这种方法。

9.15 "这是一个非常有趣的案例,华生。"福尔摩斯说,"这个年轻人爱上了一个女孩,这个女孩也爱他。但是女孩的父亲非常怪,他坚持要求他未来的女婿以公钥密码体制为基础设计一个简单安全的协议,以便他在公司的计算机网络中使用。这个年轻人提出了下列两方通信协议:假设用户 A 要将消息 M 发送给用户 B[交换的消息形为(发送方的姓名,消息正文,接收方的姓名)]"

(1) A 将 $(A, \mathrm{E}(\mathrm{PU_b}, [M, A]), B)$ 发送给 B;

(2) B 发送应答 $(B, \mathrm{E}(\mathrm{PU_a}, [M, B]), A)$ 给 A。

"这个协议确实很简单,但是女孩的父亲还是认为该协议不够简单,因为这种协议中存在一些冗余,可进一步简化为:

(1) A 将 $(A, \mathrm{E}(\mathrm{PU_b}, M), B)$ 发送给 B;

(2) B 发送应答 $(B, \mathrm{E}(\mathrm{PU_a}, M), A)$ 给 A。

由于这个原因,女孩的父亲不许他的女儿与年轻人结婚,这使得他们非常不愉快,因此年轻人来我这里请求我的帮助。"

"嗯,我不知道你会怎样帮助他。"华生想到年轻人要失去他心爱的人,显得有些不快。

"我想我可以帮助他,你知道,华生,冗余有时候对保证协议的安全性是有好处的,因此女孩父亲简化后的协议容易受到一种攻击,而年轻人设计的协议能够抗这种攻击。"福尔摩斯若有所思地说,"有办法了,华生。瞧,攻击者必须是网络用户中的一员,且能够截获 A 和 B 交换的消息。因为是网

络中的用户,所以他自己也有公钥,并且他可以发消息给 A 或 B,也可以接收 A 或 B 发出的消息。如果使用这个简化后的协议,那么他可以按下述过程得出 A 以前发送给 B 的消息 M,……"

请完成上述的过程。

9.16　运用图 9.8 所示的快速求幂算法,计算 $5^{596} \bmod 1234$。给出计算中的步骤。

9.17　下面是快速求幂算法的另一种实现方法。证明它与图 9.8 中的方法是等价的:

(1) `f ← 1; T ← a; E ← b`

(2) `if odd(e) then f ← f × T`

(3) `E ← [ E/2 ]`

(4) `T ← T × T`

(5) `if E > 0 then goto 2`

(6) `output f`

9.18　这个习题说明了选择密文攻击的简单应用。Bob 截获了一份发给 Alice 的密文 C,该密文是用 Alice 的公钥 e 加密的。Bob 想获得原始消息 $M = C^d \bmod n$。Bob 选择一个小于 n 的随机数 r,并计算:

$$Z = r^e \bmod n$$
$$X = ZC \bmod n$$
$$t = r^{-1} \bmod n$$

接着,Bob 让 Alice 用她的私钥对 X 进行认证(如图 9.3 所示),从而对 X 进行解密。Alice 返回 $Y = X^d \bmod n$。请说明 Bob 如何利用获得的信息去求取 M。

9.19　图 9.10 给出了 OAEP 编码算法,请给出对应的 OAEP 解码算法用于解密。

9.20　改进附录 9B 中的算法 P1。

(a) 设计一个需要 $2n$ 次乘法和 $n+1$ 次加法的算法。提示:$x^{i+1} = x^i \times x$。

(b) 设计一个仅需要 $n+1$ 次乘法和 $n+1$ 次加法的算法。提示:$P(x) = a_0 + x \times q(x)$,其中 $q(x)$ 是次数为 $(n-1)$ 的多项式。

关于背包公钥密码算法详见附录 J。

9.21　图 F.1 的背包体制里有哪些项?

9.22　运用背包体制对下列参数做加密和解密运算:

(a) $\boldsymbol{a}' = [1,3,5,10]; w = 7; m = 20; \boldsymbol{x} = 1101$

(b) $\boldsymbol{a}' = [1,3,5,11,23,46,136,263]; w = 203; m = 491; \boldsymbol{x} = 11101000$

(c) $\boldsymbol{a}' = [2,3,6,12,25]; w = 46; m = 53; \boldsymbol{x} = 11101$

(d) $\boldsymbol{a}' = [15,92,108,279,563,1172,2243,4468]; w = 2393; m = 9291; \boldsymbol{x} = 10110001$

9.23　为何要求 $m > \sum_{i=1}^{n} a'_i$。

附录 9A　RSA 算法的证明

RSA 算法的基本要素可以概括如下。给定的两个素数 p 和 q,$n = pq$ 和消息分组 $M < n$,选择两个整数 e 和 d 满足

$$M^{ed} \bmod n = M$$

我们在 9.2 节里说过当 e 和 d 模 $\phi(n)$ 是乘法互逆时,上述关系式成立,其中 $\phi(n)$ 是欧拉函数。第 8 章里已证明了对于素数 p 和 q,$\phi(pq) = (p-1)(q-1)$。e 和 d 的关系可表述为

$$ed \bmod \phi(n) = 1$$

另外一种表述方式是:存在整数 k 满足 $ed = k\phi(n) + 1$。因此,我们必须证明

$$M^{k\phi(n)+1} \bmod n = M^{k(p-1)(q-1)+1} \bmod n = M \tag{9.3}$$

附录 9A.1 基本结果

在证明式(9.3)之前,我们先总结一下一些基本结果。在第 4 章中,我们已证明了模算术的一个性质:

$$[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$$

由此,容易看出,如果 $x \bmod n = 1$,则 $x^2 \bmod n = 1$,且对于任意的整数 y,都有 $x^y \bmod n = 1$ 成立。类似地,如果有 $x \bmod n = 0$,则对任意整数 y,都有 $x^y \bmod n = 0$。

模算术的另一个性质是

$$[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$$

其他我们需要的结果是欧拉定理,已在第 8 章给出了。如果整数 a 和 n 互素,则有 $a^{\phi(n)} \bmod n = 1$。

附录 9A.2 证明

首先我们证明 $M^{k(p-1)(q-1)+1} \bmod p = M \bmod p$。需要考虑两种情况。

情况 1:M 和 p 不是互素的,即 p 整除 M。此时 $M \bmod p = 0$,因此有 $M^{k(p-1)(q-1)+1} \bmod p = 0$,从而 $M^{k(p-1)(q-1)+1} \bmod p = M \bmod p$。

情况 2:如果 M 和 p 互素,由欧拉定理,$M^{\phi(p)} \bmod p = 1$。继续如下步骤:

$$\begin{aligned}
M^{k(p-1)(q-1)+1} \bmod p &= [(M)M^{k(p-1)(q-1)1}] \bmod p \\
&= [(M)(M^{(p-1)})^{k(q-1)}] \bmod p \qquad \text{(由欧拉定理)} \\
&= [(M)(M^{\phi(p)})^{k(q-1)}] \bmod p \\
&= (M \bmod p) \times [(M^{\phi(p)}) \bmod p]^{k(q-1)} \\
&= (M \bmod p) \times (1)^{k(q-1)} \\
&= M \bmod p
\end{aligned}$$

现在我们有

$$[M^{k(p-1)(q-1)+1} - M] \bmod p = [M^{k(p-1)(q-1)+1} \bmod p] - [M \bmod p] = 0$$

因此 p 整除 $[M^{k(p-1)(q-1)+1} - M]$。根据同样的推理,我们可以证明 q 整除 $[M^{k(p-1)(q-1)+1} - M]$。因为 p 和 q 是不同的素数,所以必定存在一个整数 r 满足

$$[M^{k(p-1)(q-1)+1} - M] = (pq)r = nr$$

因此,n 整除 $[M^{k(p-1)(q-1)+1} - M]$,所以 $M^{k\phi(n)+1} \bmod n = M^{k(p-1)(q-1)+1} \bmod n = M$。

附录 9B 算法复杂性

评价密码算法抗密码分析能力的核心问题是这种攻击所需要的时间。通常,我们不能肯定所找到的攻击算法是最有效的方法,而至多只能说对某特定算法,攻击所需的代价具有多大规模。我们通过将该规模与当前或预知的处理器的速度进行比较,来决定算法的安全程度。

算法时间复杂性是衡量算法有效性的常用标准。如果对所有的 n 和所有长度为 n 的输入,算法的执行至多需要 $f(n)$ 步,则我们定义算法的时间复杂性为 $f(n)$,因此,对给定的输入规模和处理器的速度,时间复杂性是算法执行时间的上界。

这里有几个概念的定义不够明确。第一,步的定义不精确。一步可以是图灵机的一次操作、

一条处理器机器指令、一条高级语言机器指令等，不过，步的上述各种定义都可以通过一个简单的乘法常量把它们联系起来。当 n 很大时这些常量并不重要，重要的是相对执行时间增长得有多快。例如，如果我们关心的是使用 50 位（$n=10^{50}$）还是 100 位（$n=10^{100}$）的 RSA 密钥，那么我们不必精确地知道破译这些密钥所需的时间，而是感兴趣于所需时间的近似值，以及破译更长的密钥还需多少额外的代价。

第二，一般地，我们并不能给出 $f(n)$ 的精确公式，而只是给出它的近似公式。但我们主要感兴趣的是，当 n 很大时 $f(n)$ 的变化速度。

算法时间复杂性常用称之为“大 O”的标准的数学符号来描述，其定义为 $f(n)=O(g(n))$，当且仅当存在两个数 a 和 M 使得

$$|f(n)| \leqslant a \times |g(n)|, \quad n \geqslant M \tag{9.4}$$

下面我们通过例子来说明该符号的使用。假定要计算下述形式的多项式

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

[POHL81]中给出了下列简单的算法：

```
algorithm P1;
    n, i, j: integer; x, polyval: real;
    a, S: array [0..100] of real;
    begin
        read(x, n);
        for i := 0 upto n do
        begin
            S[i] := 1; read(a[i]);
            for j := 1 upto i do S[i] := x × S[i];
            S[i] := a[i] × S[i]
        end;
        polyval := 0;
        for i := 0 upto n do polyval := polyval + S[i];
        write ('value at', x, 'is', polyval)
    end.
```

该算法分别计算每个子表达式的值。对每个 $S[i]$]需要 $(i+1)$ 次乘法：计算 $S[i]$需要 i 次乘法，用 $a[i]$乘以 $S[i]$需要一次乘法。计算所有的 n 项需要

$$\sum_{i=0}^{n}(i+1) = \frac{(n+2)(n+1)}{2}$$

次乘法。虽然算法还执行了 $(n+1)$ 次加法，但是由于乘法次数更多，所以相对于乘法次数而言我们可以忽略这些加法运算，因此算法的时间复杂性 $f(n)=(n+2)(n+1)/2$。下面我们证明 $f(n)=O(n^2)$。根据式(9.4)中的定义，我们证明对 $a=1, M=4, g(n)=n^2$ 时关系式成立。施归纳于 n，因为 $(4+2)(4+1)/2=15 < 4^2=16$，所以 $n=4$ 时关系式成立。假设对小于等于 k 的所有的 n，关系式都成立[即 $(k+2)(k+1)/2 < k^2$]，那么 $n=k+1$ 时

$$\begin{aligned}
\frac{(n+2)(n+1)}{2} &= \frac{(k+3)(k+2)}{2} \\
&= \frac{(k+2)(k+1)}{2} + k + 2 \\
&\leqslant k^2 + k + 2 \\
&\leqslant k^2 + 2k + 1 = (k+1)^2 = n^2
\end{aligned}$$

所以 $n=k+1$ 时结论成立。

一般,大 O 符号使用增长速度最快的项,例如

(1) $O[ax^7+3x^3+\sin(x)]=O(ax^7)=O(x^7)$

(2) $O(e^n+an^{10})=O(e^n)$

(3) $O(n!+n^{50})=O(n!)$

大 O 符号还有许多其他意义,[GRAH94]和[KNUT97]给出了很好的叙述,有兴趣的读者可以参阅。设输入规模为 n,称算法是

- **线性的**:若运行时间为 $O(n)$
- **多项式的**:若有某常量 t,使得运行时间为 $O(n^t)$
- **指数的**:若有某常量 t 和多项式 $h(n)$,使得运行时间为 $O(t^{h(n)})$

一般,在多项式时间内可解的问题被认为是可行的,而任何比多项式时间更坏,尤其是指数时间可解的问题,则被认为是不可行的。使用这些术语时必须小心。第一,若输入的规模太小,则即使很复杂的算法也会变成是可行的。例如假定一个系统在单位时间可执行 10^{12} 次操作。表9.6列出了几种不同复杂性的算法在单位时间内能处理的输入规模的大小。对指数或阶乘时间复杂性的算法,只适合于非常小的输入规模。

表9.6 几种不同复杂性的算法的代价

复杂性	规模	操作
$\log_2 n$	$2^{10^{12}}=10^{3\times10^{11}}$	10^{12}
n	10^{12}	10^{12}
n^2	10^6	10^{12}
n^6	10^2	10^{12}
2^n	39	10^{12}
$n!$	15	10^{12}

第二,要注意对输入的描述方式。例如,对加密算法进行密码分析的复杂性可以通过可能的密钥数,同样也可以通过密钥的长度来描述。如对 AES 算法可能的密钥数是 2^{128},密钥长度为 128 位。若我们将一次加密视为一"步",可能的密钥数是 $N=2^n$,则算法的时间复杂性是密钥数的线性函数[$O(N)$],但却是密钥长度的指数函数[$O(2^n)$]。

第 10 章　密钥管理和其他公钥密码体制

key exchange protocol

10.1 Diffie-Hellman 密钥交换
- 10.1.1 算法
- 10.1.2 密钥交换协议
- 10.1.3 中间人攻击

10.2 ElGamal 密码体系

10.3 椭圆曲线算术
- 10.3.1 Abel 群
- 10.3.2 实数域上的椭圆曲线
- 10.3.3 Z_p上的椭圆曲线
- 10.3.4 GF(2^m)上的椭圆曲线

10.4 椭圆曲线密码学
- 10.4.1 用椭圆曲线密码实现 Diffie-Hellman 密钥交换
- 10.4.2 椭圆曲线加/解密
- 10.4.3 椭圆曲线密码的安全性

10.5 基于非对称密码的伪随机数生成器
- 10.5.1 基于 RSA 的 PRNG
- 10.5.2 基于椭圆曲线密码的 PRNG

10.6 推荐读物和网站

10.7 关键术语、思考题和习题

Amongst the tribes of Central Australia every man, woman, and child has a secret or sacred name which is bestowed by the older men upon him or her soon after birth, and which is known to none but the fully initiated members of the group.

This secret name is never mentioned except upon the most solemn occasions; to utter it in the hearing of men of another group would be a most serious breach of tribal custom. When mentioned at all, the name is spoken only in a whisper, and not until the most elaborate precautions have been taken that it shall be heard by no one but members of the group. The native thinks that a stranger knowing his secret name would have special power to work him ill by means of magic.

—*The Golden Bough*, Sir James George Frazer

要　点

- 一个简单的公钥算法是 Diffie-Hellman 密钥交换协议。这个协议使得通信的双方利用基于离散对数问题的公钥算法建立秘密钥。这个协议是安全的，仅当通信双方的真实性能够得到保证。
- 椭圆曲线算术可以用来开发许多椭圆曲线密码方案，包括密钥交换，加密和数字签名。
- 就 ECC 而言，椭圆曲线算术是指使用定义在有限域上的椭圆曲线方程。方程里的系数和变量都是域里的元素。已经开发了很多使用 Z_p和 GF(2^m)的方案。

本章首先描述了一种最早最简单的 PKCS：Diffie-Hellman 密钥交换，同时关注 ElGamal PKCS。接下来还有日益重要的椭圆曲线密码。最后检测了伪随机数生成器的公开密钥算法的应用。

10.1 Diffie-Hellman 密钥交换

Diffie 和 Hellman 在一篇具有独创意义的论文[DIFF76b]中首次提出了公钥算法，给出了公钥密码学的定义，该算法通常称为 Diffie-Hellman 密钥交换①。许多商业产品都使用了这种密钥交换技术。

① 英国 CESG 的 Williamson 几个月前在一份机密文档中提出了相同的方法[WILL76]，并声称是在此几年前设计的，请参阅[ELLI99]的有关讨论。

该算法的目的是使两个用户能安全地交换密钥,以便在后续的通信中用该密钥对消息加密。该算法本身只限于进行密钥交换。

Diffie-Hellman 算法的有效性是建立在计算离散对数是很困难的这一基础上的。简单地说,我们可如下定义离散对数。首先定义素数 p 的本原根。素数 p 的本原根是一个整数,且其幂可以产生 1 到 $p-1$ 之间所有整数,也就是说若 a 是素数 p 的本原根,则

本原根的定义=

$$a \bmod p, a^2 \bmod p, \cdots, a^{p-1} \bmod p$$

各不相同,它是整数 1 到 $p-1$ 的一个置换。

对任意整数 b 和素数 p 的本原根 a,我们可以找到唯一的指数使得

$$b \equiv a^i \pmod p, \qquad 0 \leqslant i \leqslant (p-1)$$

指数 i 称为 b 的以 a 为底的模 p 离散对数,记为 $\mathrm{dlog}_{a,p}(b)$。有关离散对数的进一步讨论请见第 8 章。

10.1.1 算法

图 10.1 概述了 Diffie-Hellman 密钥交换算法。在这种方法中,素数 q 和其本原根 α 是两个公开的整数。假定用户 A 和 B 希望交换密钥,那么用户 A 选择一个随机整数 $X_A < q$,并计算 $Y_A = \alpha^{X_A} \bmod q$。类似地,用户 B 也独立地选择一个随机整数 $X_B < q$,并计算 $Y_B = \alpha^{X_B} \bmod q$。A 和 B 都对其 X 值保密,但对另一方而言,Y 是公开可访问的。用户 A 计算 $K = (Y_B)^{X_A} \bmod q$ 并将其作为密钥,用户 B 计算 $K = (Y_A)^{X_B} \bmod q$ 并将其作为密钥。这两种计算所得的结果是相同的:

$$\begin{aligned}
K &= (Y_B)^{X_A} \bmod q \\
&= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\
&= (\alpha^{X_B})^{X_A} \bmod q \\
&= \alpha^{X_B X_A} \bmod q && \text{根据模算术的运算规律} \\
&= (\alpha^{X_A})^{X_B} \bmod q \\
&= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\
&= (Y_A)^{X_B} \bmod q
\end{aligned}$$

至此 A 和 B 完成了密钥的交换。而且由于 X_A 和 X_B 是私有的,所以攻击者只能通过 q,α,Y_A 和 Y_B 来进行攻击,这样,他就必须求离散对数才能确定密钥。例如,要对用户 B 的密钥进行攻击,攻击者就必须先计算:

$$X_B = \mathrm{dlog}_{\alpha,q}(Y_B)$$

然后他就可以像用户 B 那样计算出密钥 K。

Diffie-Hellman 密钥交换的安全性建立在下述事实之上:计算素数模的幂运算相对容易,而计算离散对数却非常困难;对于大素数,求离散对数被认为是不可行的。

下面给出的例子中,密钥交换中所使用的素数 $q = 353$ 和它的一个本原根 $\alpha = 3$。A 和 B 分别选择密钥 $X_A = 97$ 和 $X_B = 233$,并计算相应的公钥:

A 计算 $Y_A = 3^{97} \bmod 353 = 40$ 40→97 离散对数

B 计算 $Y_B = 3^{233} \bmod 353 = 248$

A 和 B 交换公钥后,双方均可计算出公共的秘密钥:

A 计算 $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$

$(3^{233})^{97} =$

$(3^{97})^{233} =$

B 计算 $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$

我们假定攻击者能够得到下列信息：

$$q = 353; \alpha = 3; Y_A = 40; Y_B = 248$$

全局公开量	
q	素数
α	$\alpha < q$ 且 α 是 q 的本原根

用户 A 的密钥产生	
选择秘密的 X_A	$X_A < q$
计算公开的 Y_A	$Y_A = \alpha^{X_A} \bmod q$

用户 B 的密钥产生	
选择秘密的 X_B	$X_B < q$
计算公开的 Y_B	$Y_B = \alpha^{X_B} \bmod q$

用户 A 计算产生密钥
$K = (Y_B)^{X_A} \bmod q$

用户 B 计算产生密钥钥
$K = (Y_A)^{X_B} \bmod q$

图 10.1　Diffie-Hellman 密钥交换算法

在这个简单的例子中，用穷举攻击确定密钥 160 是可能的。特别地，攻击者可以通过寻找方程 $3^a \bmod 353 = 40$ 或 $3^b \bmod 353 = 248$ 的解来确定该公共密钥。穷举攻击方法即是要计算 3 模 353 的若干幂，当计算结果等于 40 或 248 时则停止。因 $3^{97} \bmod 353 = 40$，所以指数为 97 时可得到期望的结果。

对于较大的数，上述方法实际是不可行的。

10.1.2　密钥交换协议

图 10.2 给出的简单协议使用了 Diffie-Hellman 计算方法。假定 A 希望与 B 建立连接，并使用密钥对该次连接中的消息加密。用户 A 产生一次性私钥 X_A，计算 Y_A，并将 Y_A 发送给 B；用户 B 也产生私钥 X_B，计算 Y_B，并将 Y_B 发送给 A，这样 A 和 B 都可以计算出密钥。当然在通信前 A 和 B 都应已知公开的 q 和 α。方法之一是可由用户 A 选择 q 和 α，并将 q 和 α 放入第一条消息中。

下面是使用 Diffie-Hellman 算法的另一个例子。假定有一组用户（如 LAN 上的所有用户），且每个用户都产生一个在较长时间内有效的秘密值 X_i（用户 i），并计算公开的 Y_i。这些公开值与公开的全局量 q 和 α 一起存于某中心目录中，在任何时刻用户 j 都可以访问用户 i 的公开值，计算出秘密钥，并用它对消息加密后发送给 A。若该中心目录是可信的，则这种形式的通信既可保证保密性，又可保证某种程度的真实性。因为只有 i 和 j 可以确定密钥，所有其他用户均不能读取该消息（保密性）；接收方 i 知道只有用户 j 能用该密钥产生消息（真实性）。但是这种方法不能抗重播攻击。

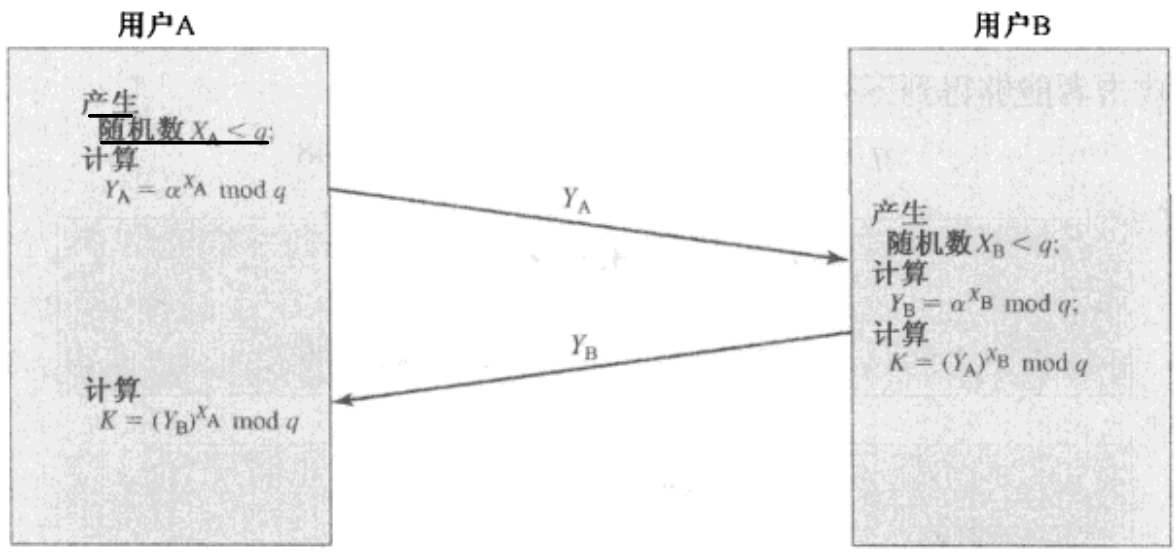


图 10.2 Diffie-Hellman 密钥交换

10.1.3 中间人攻击

图 10.2 所示的协议不能抵抗所谓的中间人攻击。假定 Alice 和 Bob 希望交换密钥,而 Darth 是攻击者。攻击过程如下:

(1) 为了进行攻击,Darth 先生成两个随机的私钥 X_{D1} 和 X_{D2},然后计算相应的公钥 Y_{D1} 和 Y_{D2}。

(2) Alice 将 Y_A 传递给 Bob。

(3) Darth 截获了 Y_A,将 Y_{D1} 传给 Bob。Darth 同时计算 $K_2 = (Y_A)^{X_{D1}} \bmod q$。

(4) Bob 收到 Y_{D1},计算 $K_1 = (Y_{D1})^{X_B} \bmod q$。

(5) Bob 将 Y_B 传给 Alice。

(6) Darth 截获了 Y_B,将 Y_{D2} 传给 Alice。Darth 计算 $K_1 = (Y_B)^{X_{D1}} \bmod q$。

(7) Alice 收到 Y_{D2},计算 $K_2 = (Y_{D2})^{X_A} \bmod q$。

此时,Bob 和 Alice 想,他们已共享了密钥,但实际上,Bob 和 Darth 共享密钥 K_1,而 Alice 和 Darth 共享密钥 K_2。接下来,Bob 和 Alice 之间的通信以下列方式泄密:

(1) Alice 发了一份加了密的消息 M:$E(K_2, M)$。

(2) Darth 截获了该密消息,解密,恢复出 M。

(3) Darth 将 $E(K_1, M)$ 或 $E(K_1, M')$ 发给 Bob,其中 M' 是任意的消息。第一种情况,Darth 只是简单地偷听通信,而不是改变它。第二种情况,Darth 想修改给 Bob 的消息。

密钥交换协议不能抵抗上述的攻击,因为它没有对通信的参与方进行认证。这些缺陷可以通过使用数字签名和公钥证书来克服。这些主题将会在第 13 章和第 14 章讨论。

10.2 ElGamal 密码体系

1984 年,T. Elgamal 提出了一种基于离散对数的公开密钥体制,它与 Diffie-Hellman 密钥分配体制[ELGA84,ELGA85]密切相关。ElGamal 密码体系应用于一些技术标准中,如数字签名标准(DSS)和 S/MIME 电子邮件标准(参见第 18 章),数字签名标准将在第 13 章中论述。

与 Diffie-Hellman 一样,ElGamal 的系统用户也是共同选择一个素数 q,α 是 q 的素根。用户 A 生成的密钥对如下:

（1）产生一个随机整数 X_A，使得 $1 < X_A < q-1$。

（2）计算 $Y_A = \alpha^{X_A} \bmod q$。

（3）A 的私钥为 X_A，公开密钥为 $\{q, \alpha, Y_A\}$。

其他任何用户 B 通过 A 的公开密钥可以加密信息：

（1）将信息表示为一个整数 M，其中 $1 \leqslant M \leqslant q-1$，以分组密码序列的方式来发送信息，其中每个分块的长度不小于整数 q。

（2）选择一个随机整数 k，使得 $1 \leqslant k \leqslant q-1$。

（3）计算一次密钥 $K = (Y_A)^k \bmod q$。

（4）将 M 加密成明文对 (C_1, C_2)，其中

$$C_1 = \alpha^k \bmod q;\ C_2 = KM \bmod q$$

用户 A 恢复明文：

（1）通过计算 $K = (C_1)^{X_A} \bmod q$ 恢复密钥。

（2）计算 $M = (C_2 K^{-1}) \bmod q$。

这些步骤在图 10.3 中有具体的总结。根据图 9.1(a)：Alice 生成公开和保密的密钥对；Bob 用 Alice 的公开密钥加密，然后 Alicce 用自己的私钥解密。

接下来具体阐述 ELGamal 体系的工作原理。首先，我们来看 K 被解密恢复的过程：

$K = (Y_A)^k \bmod q$	K 定义在加密过程中
$K = (\alpha^{X_A} \bmod q)^k \bmod q$	用 $Y_A = \alpha^{X_A} \bmod q$ 来替换
$K = \alpha^{kX_A} \bmod q$	利用同余定理
$K = (C_1)^{X_A} \bmod q$	用 $C_1 = \alpha^k \bmod q$ 来替换

接下来，用 K 来恢复明文

$$C_2 = KM \bmod q$$

$$(C_2 K^{-1}) \bmod q = KMK^{-1} \bmod q = M \bmod q = M$$

我们能够用下面的图 10.3 来重述 ElGamal 过程。

（1）Bob 产生一个随机整数 k。

（2）Bob 用 Alice 的公开密钥 $\{q, k, Y_A\}$ 生成一个一次密钥 K。

（3）Bob 用公开密要参数 α 加密 k，得到 (C_1, C_2)，这给 Alice 足够的信息恢复 K。

（4）Bob 用 K 加密明文 M。

（5）Alice 用自己的私钥从 C_1 恢复出 K。

（6）Alice 用 K^{-1} 通过 C_2 恢复明文信息。

因此，K 函数作为一次性密钥，用于加密和解密消息。

例如，对于素域 GF(19)，即 $q=19$。素根有 $\{2,3,10,13,14,15\}$。如表 8.3 所示，选择 $\alpha = 10$。Alice 生成如下的密钥对：

（1）Alice 选择 $X_A = 5$。

（2）计算 $Y_A = \alpha^{X_A} \bmod q = \alpha^5 \bmod 19 = 3$（参见表 8.3）。

（3）Alice 的私钥为 5；公钥为 $\{q, \alpha, Y_A\} = \{19, 10, 3\}$。

假如 Bob 想将值 $M=17$ 发送,则

(1) Bob 选择 $k=6$。

(2) 计算 $K=(Y_A)^k \bmod q = 3^6 \bmod 19 = 729 \bmod 19 = 7$。

(3) 因此

$$C_1 = \alpha^k \bmod q = \alpha^6 \bmod 19 = 11$$

$$C_2 = KM \bmod q = 7 \times 17 \bmod 19 = 119 \bmod 19 = 5$$

(4) Bob 发送密文(11,5)。

公开全局量	
q	素数
α	$\alpha < q$ 且 α 是 q 的素根

Alice 生成的密钥	
选择私钥 X_A	$X_A < q-1$
计算 Y_A	$Y_A = \alpha^{X_A} \bmod q$
公开密钥	PU $= \{q, \alpha, Y_A\}$
私钥	X_A

Bob 用 Alice 的公钥加密	
明文	$M < q$
随机选择整数 k	$k < q$
计算 K	$K = (Y_A)^k \bmod q$
计算 C_1	$C_1 = \alpha^k \bmod q$
计算 C_2	$C_2 = KM \bmod q$
密文	(C_1, C_2)

用 Alicce 的私钥解密	
密文	(C_1, C_2)
计算 K	$K = (C_1)^{X_A} \bmod q$
明文	$M = (C_2 K^{-1}) \bmod q$

图 10.3 ElGamal 密码体系

解密:

(1) Alice 计算 $K=(C_1)^{X_A} \bmod q = 11^5 \bmod 19 = 161051 \bmod 19 = 7$。

(2) 在 GF(19)中 K^{-1} 为 $7^{-1} \bmod 19 = 11$。

(3) 最终 $M=(C_2K^{-1}) \bmod q = 5 \times 11 \bmod 19 = 55 \bmod 19 = 17$。

如果信息必须分组然后以加密的密钥块序列发送,那么每个分块要有唯一的 k。如果 k 用于多个分块。利用信息的分块 m_1,攻击者会计算出其他块:

$$C_{1,1} = \alpha^k \bmod q; \quad C_{2,1} = KM_1 \bmod q$$

$$C_{1,2} = \alpha^k \bmod q; \quad C_{2,2} = KM_2 \bmod q$$

于是

$$\frac{C_{2,1}}{C_{2,2}} = \frac{KM_1 \bmod q}{KM_2 \bmod q} = \frac{M_1 \bmod q}{M_2 \bmod q}$$

如果 M_1 已知，则很容易计算出 M_2：

$$M_2 = (C_{2,1})^{-1} C_{2,2} M_1 \bmod q$$

ElGamal 的安全性是基于计算离散对数的困难性之上。为了恢复 Alice 的私钥，攻击者将会计算 $X_A = \text{dlog}_{\alpha,q}(Y_A)$。或者，为了恢复一次性密钥 K，攻击者将会选择随机数 k，然后计算离散对数 $k = \text{dlog}_{\alpha,q}(C_1)$。[STIN06]指出，当 $p \geqslant 300$，$q-1$ 至少有一个大的素因子时，这种算法是不可行的。

10.3 椭圆曲线算术

大多数使用公钥密码学进行加密和数字签名的产品和标准都使用 RSA 算法。我们知道，为了保证 RSA 使用的安全性，最近这些年来密钥的位数一直在增加，这对使用 RSA 的应用是很重的负担，对进行大量安全交易的电子商务来说更是如此。近来，出现的一种具有强大竞争力的椭圆曲线密码学(ECC)对 RSA 提出了挑战。在标准化过程中，如关于公钥密码学的 IEEE P1363 标准中，人们也已考虑了 ECC。

与 RSA 相比，ECC 的主要诱人之处在于，它可以使用比 RSA 短得多的密钥得到相同的安全性，因此可以减少处理负荷。另一方面，虽然关于 ECC 的理论已很成熟，但直到最近才出现这方面的产品，而对 ECC 的密码分析的兴趣却持续不断，因此 ECC 的可信度还没有 RSA 高。

ECC 比 RSA 或 Diffie-Hellman 更难阐述，关于 ECC 的完整的数学描述已超出本书范围。本节和下一节中，我们只给出有关椭圆曲线和 ECC 的一些背景知识。我们首先简要讨论 Abel 群，然后讨论定义在实数域上的椭圆曲线，紧接着讨论有限域上的椭圆曲线，最后讨论椭圆曲线密码。

在下述讨论之前，读者可回顾第 4 章中关于有限域的内容。

10.3.1 Abel 群

由第 4 章可知，Abel 群 G 由元素的集合及其上的二元运算 · 组成，有时记为 $\{G, \cdot\}$。将 G 中元素的序偶 (a,b) 与 G 中元素 $(a \cdot b)$ 对应，使得下述公理[①]成立：

(A1)封闭性	若 a 和 b 属于 G，则 $a \cdot b$ 也属于 G。
(A2)结合性	对 G 中任意的 a,b 和 c，$(a \cdot b) \cdot c = a \cdot (b \cdot c)$
(A3)单位元	G 中存在元素 e，使得对 G 中所有的 a，$e \cdot a = a \cdot e = a$。
(A4)逆元	对 G 中任何 a，存在 G 中元素 a'，使得 $a' \cdot a = a \cdot a' = e$。
(A5)交换性	对 G 中任何 a 和 b，有 $a \cdot b = b \cdot a$。

许多公钥密码都使用了 Abel 群。例如 Diffie-Hellman 密钥交换包含若干非零整数对素数 q 的取模运算，通过幂运算来产生密钥，其中幂运算定义为重复相乘。如 $a^k \bmod q = \underbrace{(a \times a \times \cdots \times a)}_{k\text{次}} \bmod q$。要攻击 Diffie-Hellman，攻击者必须对给定 a 和 a^k 确定 k，这即是求离散对数问题。

① 运算符 · 是通用运算，它可以是加法、乘法或其他的数学运算。在本节中，我们用加法运算符 + 表示群的运算。

椭圆曲线密码学使用椭圆曲线上称之为加法的运算,乘法被定义为重复相加。例如

$$a \times k = \underbrace{(a + a + \cdots + a)}_{k\text{次}}$$

其中加法是椭圆曲线上的加法。密码分析者需要从给定的 a 和 $(a \times k)$ 来确定 k。

椭圆曲线是一个具有两个变元及系数的方程。对于密码学而言,变元和系数均限制于有限域上,这导致了有限 Abel 群的定义。在讨论这些之前,我们先讨论变元和系数均是实数的椭圆曲线,这种情况也许更容易想象。

10.3.2 实数域上的椭圆曲线

椭圆曲线并不是椭圆,之所以称为椭圆曲线是因为它们与计算椭圆周长的方程相似,也是用三次方程来表示的。一般,椭圆曲线的三次方程形为

$$y^2 + axy + by = x^3 + cx^2 + dx + e$$

其中 a,b,c,d 和 e 是实数,x 和 y 在实数集上取值[①]。对我们而言将方程限制为下述形式就已足够:

$$y^2 = x^3 + ax + b \tag{10.1}$$

因为方程中的指数最高是3,所以我们称之为三次方程,或者说方程的次数为3。椭圆曲线的定义中还包含一个称为无穷远点或零点的元素,记为 O,我们以后再讨论这个概念。为了画出该曲线,我们需要计算:

$$y = \sqrt{x^3 + ax + b}$$

对给定的 a 和 b,对 x 的每一个值,需画出 y 的正值和负值,这样每一曲线都关于 $y = 0$ 对称。

图 10.4给出了椭圆曲线的两个例子,由图中可知,上述方程有时对应的是一条怪异的曲线。

简单的表示:现在我们考虑满足式(10.1)的所有的点(x,y)和元素 O 所组成的点集 $E(a,b)$。偶对(a,b) $E(a,b)$的值不同,则相应集合 $E(a,b)$ 也不同。使用上述术语,图 10.4 中的两条曲线可分别用集合 $E(-1,0)$和$E(1,1)$表示。

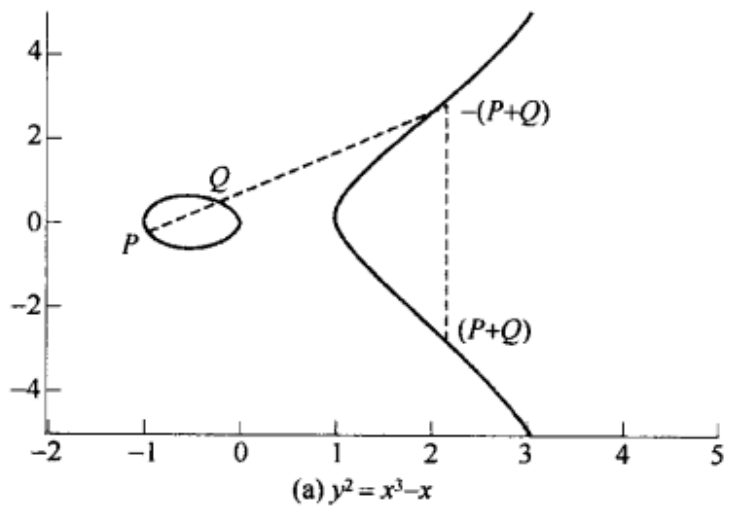


图 10.4 椭圆曲线举例

① 注意:x 和 y 真正的变元,它们具有值。这与第 4 章中讨论的多项式环和域不同,那里 x 被看做是不定元。

O:无穷远点,零点

加法的几何描述

式(10.1)里的参数 a 和 b,如果满足条件

$$4a^3 + 27b^2 \neq 0 \tag{10.2}$$

则可基于集合 $E(a,b)$ 定义一个群。为了在 $E(a,b)$ 上定义一个群,我们必须定义一个运算,称为加法,用 + 表示,其中 a 和 b 满足式(10.2)。用几何术语可如下定义加法的运算规则:若椭圆曲线上的三个点同在一条直线上,则它们的和为 O。从这个定义出发我们可以定义椭圆曲线加法的运算规则:

(1) O 是加法的单位元。这样有 $O = -O$;对椭圆曲线上的任何一点 P,有 $P + O = P$。下面假定 $P \neq Q$ 且 $Q \neq O$。

(2) 点 P 的负元是具有相同 x 坐标和相反的 y 坐标的点,即若 $P = (x,y)$,则 $-P = (x,-y)$。注意这两个点可用一条垂直的线连接起来,并且 $P + (-P) = P - P = O$。

(3) 要计算 x 坐标不相同的两点 P 和 Q 之和,则在 P 和 Q 间作一条直线并找出第三个交点 R,显然存在有唯一的交点 R(除非这条直线在 P 或 Q 处与该椭圆曲线相切,此时我们分别取 $R = P$ 或 $R = Q$)。要形成群,需要定义如下三个点上的加法:$P + Q = -R$。也就是说,定义 $P + Q$ 为第三个交点(相对于 x 轴)的镜像。图 10.4 说明了这一情形。

(4) 上述术语的几何解释也适用于具有相同 x 坐标的两个点 P 和 $-P$ 的情形。用一条垂直的线连接这两点,这也可视为在无穷远点处与曲线相交,因此有 $P + (-P) = O$,与上述步骤(2)相一致。

(5) 为计算点 Q 的两倍,画一条切线并找出另一交点 S,则 $Q + Q = 2Q = -S$。

利用前述的运算规则,可以证明集合 $E(a,b)$ 是 Abel 群。

加法的代数描述

在本小节中,我们给出一些用于椭圆曲线上加法的结论[①]。对不是互为负元的两个不同的点 $P = (x_P, y_P)$ 和 $Q = (x_Q, y_Q)$,连接它们的曲线 l 的斜率 $\Delta = (y_Q - y_P)/(x_Q - x_P)$。$l$ 恰与椭圆曲线相交与另一点,即 P 与 Q 之和的负元。利用某些代数运算,我们可如下表示和 $R = P + Q$:

$$\begin{aligned} x_R &= \Delta^2 - x_P - x_Q \\ y_R &= -y_P + \Delta(x_P - x_R) \end{aligned} \tag{10.3}$$

我们也需要能够计算一个点与它自身相加:$P + P = 2P = R$。当 $y_P \neq 0$ 时,该表达式为

$$\begin{aligned} x_R &= \left(\frac{3x_P^2 + a}{2y_P}\right)^2 - 2x_P \\ y_R &= \left(\frac{3x_P^2 + a}{2y_P}\right)(x_P - x_R) - y_P \end{aligned} \tag{10.4}$$

10.3.3 Z_p 上的椭圆曲线

椭圆曲线密码体制使用的是变元和系数均为有限域中元素的椭圆曲线。密码应用中所使用的两类椭圆曲线是定义在 Z_p 上的素曲线(prime curve)和在 GF(2^m)上构造的二元曲线。对于 Z_p 上的素曲线,我们使用三次方程,其中的变量和系数自集合 $\{0,1,\cdots,p-1\}$ 取值,运算为模 p 运算。

Zp{0,1,...,p-1}

① 关于这些结论的推导请见[KOBL94]或其他关于椭圆曲线的数学资料。

对于 GF(2^m) 上的二元曲线,变量和系数在 GF(2^m)内取值,且运算为 GF(2^m)里的运算。[FERN99]指出,因为不需要二元曲线所要求的位混淆(bit-fiddling)运算,对软件应用最好使用素曲线;而对硬件应用最好使用二元曲线,它可以用异常少的门电路来得到快速且功能强大的密码体制。我们将在本节和下一节中讨论这两类曲线。

对有限域上的椭圆曲线算术没有显而易见的几何解释,用于实数域上椭圆曲线算术的代数解释可以移植过来,我们将采用这种方法。

对 Z_p上的椭圆曲线,如同实数情形一样,我们仅限于讨论形如式(10.1)的方程。此时,变元和系数均在 Z_p里取值

$$y^2 \bmod p = (x^3 + ax + b) \bmod p \tag{10.5}$$

例如,$a=1, b=1, x=9, y=7, \alpha=1, p=23$ 时可满足式(10.5)

$$7^2 \bmod 23 = (9^3 + 9 + 1) \bmod 23$$
$$49 \bmod 23 = 739 \bmod 23$$
$$3 = 3$$

下面考虑由所有满足式(10.5)的整数对(x,y)和无穷远点 O 组成的集合 $E_p(a,b)$。系数 a 和 b,变量 x 和 y 都是 Z_p的元素。

例如,取 $p=23$。考虑椭圆曲线方程 $y^2 = x^3 + x + 1$,这里 $a=b=1$。注意,该方程与图 10.4(b)中的方程是相同的。图中显示了所有满足方程的实点。对 $E_{23}(1,1)$,我们只对如下非负整数感兴趣,它们位于从(0,0)到$(p-1, p-1)$的象限中,满足模 p 的方程。表 10.1 列出了若干点(除 O 外),这些点是 $E_{23}(1,1)$的一部分,图 10.5 给出了 $E_{23}(1,1)$上的点。注意这些点除了一个之外,均关于 $y=11.5$ 对称。

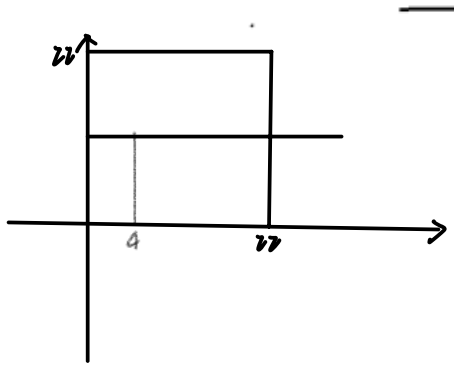


表 10.1 椭圆曲线 $E_{23}(1,1)$上的点

(0,1)	(6,4)	(12,19)
(0,22)	(6,19)	(13,7)
(1,7)	(7,11)	(13,16)
(1,16)	(7,12)	(17,3)
(3,10)	(9,7)	(17,20)
(3,13)	(9,16)	(18,3)
(4,0)	(11,3)	(18,20)
(5,4)	(11,20)	(19,5)
(5,19)	(12,4)	(19,18)

可以证明,若$(x^3 + ax + b) \bmod p$ 无重复因子,则基于集合 $E_p(a,b)$可定义一个有限 Abel 群。这等价于下列条件:

$$(4a^3 + 27b^2) \bmod p \neq 0 \bmod p \tag{10.6}$$

注意式(10.6)和式(10.2)具有相同的形式。

$E_p(a,b)$上的加法运算构造与定义在实数上的椭圆曲线中描述的代数方法是一致的。对任何点 $P, Q \in E_p(a,b)$

(1) $P + O = P$。

(2) 若 $P=(x_P, y_P)$,则 $P + (x_P, -y_P) = O$。点$(x_P, -y_P)$是 P 的负元,记为 $-P$。例如,对 $E_{23}(1,1)$上的点 $P=(13,7)$,有 $-P=(13,-7)$,而 $-7 \bmod 23 = 16$,因此,$-P=(13,16)$,该点也在 $E_{23}(1,1)$上。

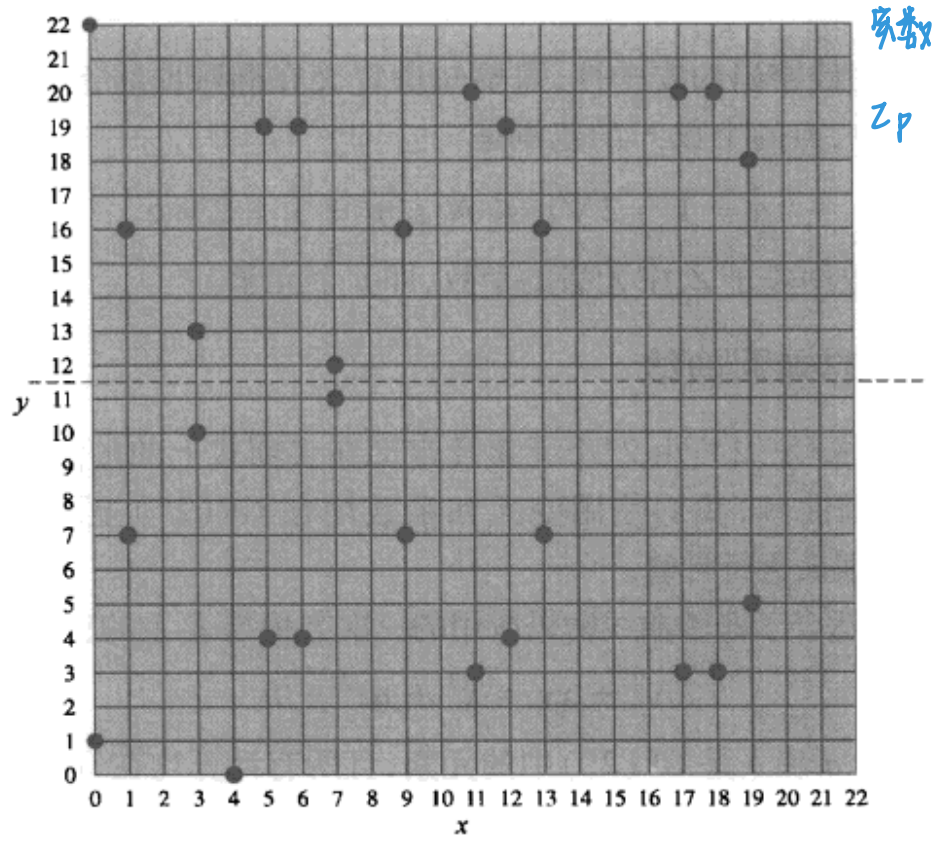


图 10.5　椭圆曲线 $E_{23}(1,1)$

(3) 若 $P=(x_P,y_P)$，$Q=(x_Q,y_Q)$，且 $P\neq -Q$ 则 $R=P+Q=(x_R,y_R)$ 由下列规则确定：

$$x_R=(\lambda^2-x_P-x_Q)\bmod p$$

$$y_R=(\lambda(x_P-x_R)-y_P)\bmod p$$

其中

$$\lambda=\begin{cases}\left(\dfrac{y_Q-y_P}{x_Q-x_P}\right)\bmod p, & P\neq Q\\ \left(\dfrac{3x_P^2+a}{2y_P}\right)\bmod p, & P=Q\end{cases}$$

(4) 乘法定义为重复相加。如 $4P=P+P+P+P$。

例如取 $E_{23}(1,1)$ 上的 $P=(3,10)$，$Q=(9,7)$，那么

$$\lambda=\left(\frac{7-10}{9-3}\right)\bmod 23=\left(\frac{-3}{6}\right)\bmod 23=\left(\frac{-1}{2}\right)\bmod 23=11$$

$$x_R=(11^2-3-9)\bmod 23=109\bmod 23=17$$

$$y_R=(11(3-17)-10)\bmod 23=-164\bmod 23=20$$

所以 $P+Q=(17,20)$。为计算 $2P$，先求

$$\lambda=\left(\frac{3(3^2)+1}{2\times 10}\right)\bmod 23=\left(\frac{5}{20}\right)\bmod 23=\left(\frac{1}{4}\right)\bmod 23=6$$

上述等式的最后一步中需求 4 在 Z_{23} 中的乘法逆元。这可以用 4.4 节定义的扩展 Euclid 算法实现。注意到 $(6\times 4)\bmod 23=24\bmod 23=1$。

$$x_R=(6^2-3-3)\bmod 23=30\bmod 23=7$$

$$y_R=(6(3-7)-10)\bmod 23=(-34)\bmod 23=12$$

可见 $2P=(7,12)$。

为了确定各种椭圆曲线密码的安全性,需要知道定义在椭圆曲线上的有限 Abel 群中点的个数。在有限群 $E_p(a,b)$ 中,点的个数 N 的范围是

$$p+1-2\sqrt{p} \leqslant N \leqslant p+1+2\sqrt{p}$$

所以,$E_p(a,b)$ 上点的个数约等于 Z_p 中元素的个数,即 p 个元素。

10.3.4 GF(2^m)上的椭圆曲线

由第 4 章可知,有限域 GF(2^m)由 2^m 个元素及定义在多项式上的加法和乘法运算组成。给定某 m,对 GF(2^m)上的椭圆曲线,我们使用变元和系数均在 GF(2^m)上取值的三次方程,且利用 GF(2^m)中的算术运算规则来进行计算。

可以证明,GF(2^m)上适合于椭圆曲线密码应用的三次方程与 Z_p 上的三次方程有所不同,其形为

$$y^2+xy=x^3+ax^2+b \tag{10.7}$$

其中变元 x 和 y 以及系数 a 和 b 是 GF(2^m)中的元素,且所有计算均在 GF(2^m)中进行。

考虑由满足式(10.7)的所有整数对(x,y)和无穷远点组成的集合 $E_{2^m}(a,b)$。

例如,我们使用不可约多项式 $f(x)=x^4+x+1$ 定义的有限域 GF(2^4)。其生成元满足 $f(g)=0$,即 $g^4=g+1$,或者二进制 0010。我们可以如下给出 g 的各次方:

$g^0=0001$	$g^4=0011$	$g^8=0101$	$g^{12}=1111$
$g^1=0010$	$g^5=0110$	$g^9=1010$	$g^{13}=1101$
$g^2=0100$	$g^6=1100$	$g^{10}=0111$	$g^{14}=1001$
$g^3=1000$	$g^7=1011$	$g^{11}=1110$	$g^{15}=0001$

例如,$g^5=(g^4)(g)=g^2+g=0110$。

现在考虑椭圆曲线 $y^2+xy=x^3+g^4x^2+1$。此时,$a=g^4$,$b=g^0=1$。满足该方程的一个点为 (g^5,g^3):

$$(g^3)^2+(g^5)(g^3)=(g^5)^3+(g^4)(g^5)^2+1$$
$$g^6+g^8=g^{15}+g^{14}+1$$
$$1100+0101=0001+1001+0001$$
$$1001=1001$$

表 10.2 列出了 $E_{2^4}(g^4,1)$ 的部分点(没有点 O)。图 10.6 画出了这些点。

表 10.2 椭圆曲线 $E_{24}(g^4,1)$ 上的点

$(0,1)$	(g^5,g^3)	(g^9,g^{13})
$(1,g^6)$	(g^5,g^{11})	(g^{10},g)
$(1,g^{13})$	(g^6,g^8)	(g^{10},g^8)
(g^3,g^8)	(g^6,g^{14})	$(g^{12},0)$
(g^3,g^{13})	(g^9,g^{10})	(g^{12},g^{12})

可以证明,只要 $b\neq 0$,则可基于集合 $E_{2^m}(a,b)$ 定义一个有限 Abel 群。加法的运算规则如下所述。对所有点 $P,Q\in E_{2^m}(a,b)$:

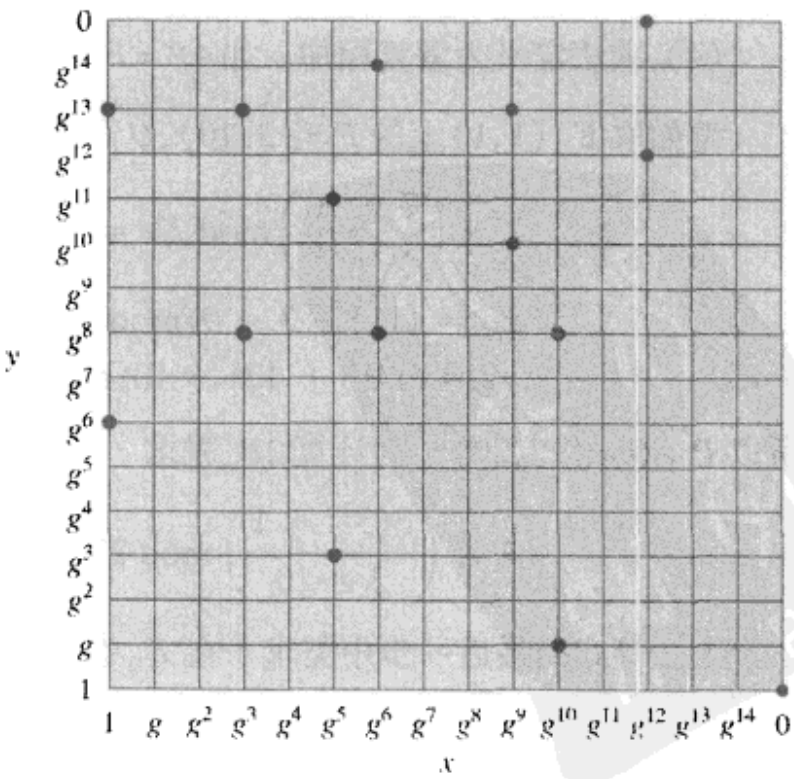


图 10.6 椭圆曲线 $E_{24}(g^4,1)$

(1) $P+O=P$。

(2) 若 $P=(x_P,y_P)$，则 $P+(x_P,x_P+y_P)=O$。点 (x_P,x_P+y_P) 是 P 的负元，记为 $-P$。

(3) 若 $P=(x_P,y_P)$，$Q=(x_Q,y_Q)$，且 $P\neq -Q$，$P\neq Q$，则 $R=P+Q=(x_R,y_R)$ 由下列规则确定：

$$x_R=\lambda^2+\lambda+x_P+x_Q+a$$
$$y_R=\lambda(x_P+x_R)+x_R+y_P$$

其中

$$\lambda=\frac{y_Q+y_P}{x_Q+x_P}$$

(4) 若 $P=(x_P,y_P)$，则 $R=2P=(x_R,y_R)$ 由下列规则确定：

$$x_R=\lambda^2+\lambda+a$$
$$y_R=x_P^2+(\lambda+1x)_R$$

其中

$$\lambda=x_P+\frac{y_P}{x_P}$$

10.4　椭圆曲线密码学

我们将 ECC 中的加法运算与 RSA 中的模乘运算相对应，将 ECC 中的乘法运算与 RSA 中的模幂运算对应，要建立基于椭圆曲线的密码体制，我们需要类似因子分解两个素数之积或求离散对数这样的“难题”。

考虑方程 $Q=kP$，其中 $Q,P\in E_p(a,b)$ 且 $k<p$。对给定的 k 和 P 计算 Q 比较容易，而对给定的 Q 和 P 计算 k 则比较困难。这就是椭圆曲线的离散对数问题。

我们引用 Certicom 网站（www. certicom. com）给出的例子来说明这一问题。考虑由方程 $y^2 \bmod 23=(x^3+9x+17) \bmod 23$ 所定义的群 $E_{23}(9,17)$。以 $P=(16,5)$ 为底的 $Q=(4\ 5)$ 的离散对数 k 为多少？穷举攻击方法通过多次计算 P 的倍数直至找到 Q 为止。这样

$$P=(16,5);\ 2P=(20,20);\ 3P=(14,14);\ 4P=(19,20);\ 5P=(13,10);$$
$$6P=(7,3);\ 7P=(8,7);\ 8P=(12,17);\ 9P=(4,5)$$

因为 $9P=(4,5)=Q$，所以以 $P=(16,5)$ 为底的 $Q=(4,5)$ 的离散对数 $k=9$。在实际应用中，k 的值非常大，从而使穷举攻击方法不可行。

下面，我们介绍 ECC 的两种应用。

10.4.1　用椭圆曲线密码实现 Diffie-Hellman 密钥交换

利用椭圆曲线可如下实现密钥交换。首先，挑选一个大整数 q 以及式(10.5)或式(10.7)中的椭圆曲线参数 a 和 b，这里 q 为素数 p 或是形为 2^m 的整数。由此可以定义出点的椭圆群 $E_q(a,b)$；其次，在 $E_p(a,b)$ 中挑选基点 $G=(x_1,y_1)$，G 的阶为一个非常大的数 n。椭圆曲线上的点 G 的阶 n 是使得 $nG=0$ 成立的最小正整数。$E_q(a,b)$ 和 G 是该密码体制中通信各方均已知的参数。

用户 A 和用户 B 之间完成密钥交换过程如下（参见图 10.7）：

(1) A 选择一个小于 n 的整数 n_A 作为其私钥，然后产生其公钥 $P_A=n_A\times G$；该公钥是 $E_q(a,b)$ 中的一个点。

(2) B 可类似地选择私钥 n_B 并计算公钥 P_B。

(3) A 产生秘密钥 $k = n_A \times P_B$，B 产生秘密钥 $k = n_B \times P_A$。

步骤(3)中 k 的两种计算结果是相同的，因为

$$n_A \times P_B = n_A \times (n_B \times G) = n_B \times (n_A \times G) = n_B \times P_A$$

要破译这种体制，攻击者必须由 G 和 kG 计算 k，这被认为是非常难的。

例如①，取 $p = 211$，$E_p(0, -4)$，$G = (2,2)$，这里 $E_p(0, -4)$ 即是曲线 $y^2 = x^3 - 4$，则计算可得 $240G = O$。A 的私钥 $n_A = 121$，所以 A 的公钥 $P_A = 121(2,2) = (115,48)$。B 的私钥 $n_B = 203$，所以 B 的公钥为 $203(2,3) = (130,203)$，它们共享的秘密钥为 $121(130,203) = 203(115,48) = (161,69)$。

请注意，这里的密钥是一对数字。若将它用做传统密码中的会话密钥，则必须是一个数字，我们可以简单地选取 x 坐标或者 x 坐标的某简单函数。

全局公开量	
$E_q(a,b)$	参数为 a,b 和 q 的椭圆曲线，其中 q 是素数或形为 2^m 的整数
G	阶为 n 的椭圆曲线上的点，其中 n 是大整数

用户 A 的密钥产生	
选择私有的 n_A	$n_A < n$
计算公开的 P_A	$P_A = n_A \times G$

用户 B 的密钥产生	
选择私有的 n_B	$n_B < n$
计算公开的 P_B	$P_B = n_B \times G$

用户 A 产生秘密钥
$K = n_A \times P_B$

用户 B 产生秘密钥
$K = n_B \times P_B$

图 10.7 ECC Diffie-Hellman 密钥交换

10.4.2 椭圆曲线加/解密

一些文献中分析了几种用椭圆曲线实现加/解密的方法，本节介绍也许是其中最简单的一种方法。首先，我们必须将要发送的消息明文 m 编码为形为 $x - y$ 的点 P_m，并对点 P_m 进行加密和其后的解密。注意，不能简单地将消息编码为点的 x 坐标或 y 坐标，因为并不是所有的坐标都在 $E_q(a,b)$ 中，请参见表 10.1。将消息 m 编码为点 P_m 的方法有多种，我们不打算在这里讨论这些方法，但需要说明的是，存在有比较直接的编码方法。

像密钥交换系统一样，加/解密系统也需要点 G 和椭圆群 $E_q(a,b)$ 这些参数。每个用户 A 选择一个私钥 n_A，并产生公钥 $P_A = n_A \times G$。

① 本例由 Santa Clara 大学的 ED Schaefer 提供。

若 A 要将消息 P_m 加密后发送给 B，则 A 随机选择一个正整数 k，并产生密文 C_m，该密文是一个点对：

$$C_m = \{kG, P_m + kP_B\}$$

注意，此处 A 使用了 B 的公钥 P_B。B 要对密文解密，则需用第二个点减去第一个点与 B 的私钥之积：

$$P_m + kP_B - n_B(kG) = P_m + k(n_BG) - n_B(kG) = P_m$$

A 通过将 kP_B 与 P_m 相加来伪装消息 P_m，因为只有 A 知道 k，所以即使 P_B 是公钥，除 A 外任何人均不能除去伪装 kP_B；但是，A 也在伪装后的消息中包含了有关“线索”，使得已知私钥 n_B 时可以除去伪装。攻击者要想恢复消息明文，则他必须从 G 和 kG 求出 k，但这被认为是很困难的。

下面举例说明椭圆曲线加密过程，该例摘自[KOBL94]。取 $p = 751$；$E_p(-1, 188)$，即其椭圆曲线方程为 $y^2 = x^3 - x + 188$，$G = (0, 376)$。假定 A 要将编码为椭圆曲线上点 $P_m = (562, 201)$ 的消息发送给 B，且 A 挑选的随机数 $k = 386$，B 的公钥为 $P_B = (201, 5)$，那么有 $386(0, 376) = (676, 558)$ 和 $(562, 201) + 386(201, 5) = (385, 328)$。于是 A 发送的密文是 $\{(676, 558), (385, 328)\}$。

10.4.3 椭圆曲线密码的安全性

ECC 的安全性是建立在由 kP 和 P 确定 k 的困难程度之上的，这个问题称为椭圆曲线对数问题。Pollard rho 方法是已知的求椭圆曲线对数的最快方法，表 10.3 从密码分析所需计算量的角度，通过给出可比较的密钥大小，比较了各种算法。由表可知，ECC 使用的密钥比 RSA 中使用的密钥要短得多，而且在密钥长度相同时，ECC 与 RSA 所执行的计算量也差不多[JURI97]。因此，与具有同等安全性的 RSA 相比，由于 ECC 使用的密钥更短，所以 ECC 所需的计算量比 RSA 少。

表 10.3　可比较密钥大小（密码分析所需计算量的角度）

对称方案（密钥位位）	基于 ECC 方案（n 的位位）	RSA/DSA（模数位位）
56	112	512
80	160	1024
112	224	2048
128	256	3072
192	384	7680
256	512	15360

来源：Certicom。

10.5 基于非对称密码的伪随机数生成器

在第 7 章中我们提到，因为对称分组密钥流产生的接近随机的输出，它可以作为伪随机数生成器（PRNG）的一组基。同样地，一个非对称的加密算法也能得到接近随机的输出以此来创建一个 PRNG。由于非对称算法的速度比对称算法的速度要明显慢一些，所以非对称算法不用于生成可扩展的 PRNG 位流。但是，对于生成短的伪随机序列，创建伪随机函数（PRF）非对称的方法还是很有用的。

在这一节中，我们将检测两种基于伪随机函数的 PRNG 的设计。

10.5.1 基于 RSA 的 PRNG

对于足够长的密钥，RSA 算法被认为是安全的也是具有很好的条件来形成 PRNG 的一组基。

这种 PRNG,像已知的 Micali-Schnorr PRNG[MICA91],是作为 ANSI 标准 X9.82(随机数生成)和 ISO 标准 18031(随机位生成)。

在图 10.8 中给出了 PRNG 的模式,从中可以看出,PRNG 在结构上与用于 PRNG 的反馈输出模式有很多相同之处[参见图 7.3(b)和图 6.6(a)]。在这种情况下加密算法采用 RSA 要比对称分组密码要好。与此同时,输出的一部分又反馈到下一个加密算法的循环中,剩下的输出则作为伪随机位。将输出分为这样的两种不同的部分的动机是因为,伪随机位从一个状态到另一个状态时不会泄露输入。这种分法一定会对其发展起到不可预测的贡献。

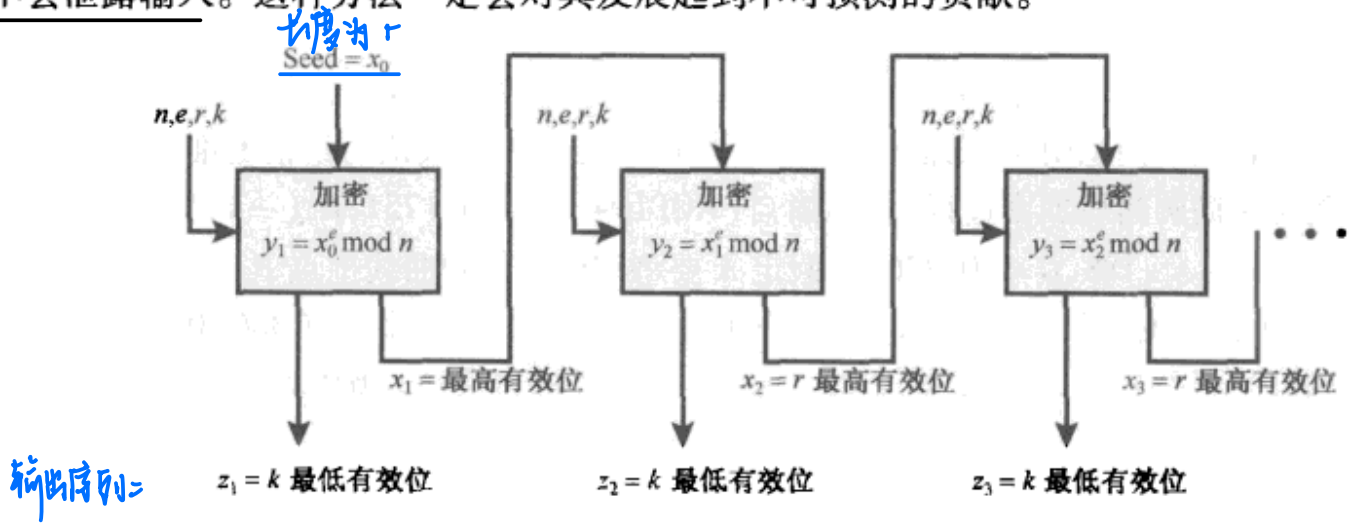


图 10.8 Micali-Schnorr 伪随机位生成器

PRNG 的定义如下:

创建 选择素数 p,q;$n = pq$;$\phi(n) = (p-1)(q-1)$,选择 e 使得 $\gcd(e,\phi(n)) = 1$。这就是标准的 RSA 的参数选择(参见图 9.5)。此外,$N = \lfloor \log_2 n \rfloor + 1$($n$ 的位长度)。选择 r,k,使得 $r + k = N$。

种子 选择位长度为 r 的随机种子 x_0。

生成 生成一个长为 $k \times n$ 的伪随机序列,将 i 从 1 到 m,计算

$$y_i = x_{i-1}^e \bmod n$$
$$x_i = r, y_i \text{ 的最高有效位}$$
$$z_i = k, y_i \text{ 的最低有效位}$$

输出 输出序列为 $z_1 \| z_2 \| \cdots \| z_m$。

参数 n,r,e,k 的选择要遵循下面 6 个要求:

(1) $n = pq$	n 是两个素数的乘积,在 RSA 中则是密码长度。
(2) $1 < e < \phi(n)$;$\gcd(e,\phi(n)) = 1$	确保映射 $s \rightarrow s^e \bmod n$ 是一对一的。
(3) $re \geqslant 2N$	确保求幂运算是有取模过程。
(4) $r \geqslant 2$	防止密码攻击。
(5) k,r 是 8 的倍数	应用方便。
(6) $k \geqslant 8$;$r + k = N$	所有位都有效。

在要求(4)中对于有用长度在 NIST SP 800-90 中定义为在攻破一个密码算法或者系统的工作相关量(即生成器的数量);一个安全的长度是规定好的。同时也是从规定的集合(112,128,192,256)中的一个特定值。所需的总工作量是 2^{strength}。

明显可以看到 r 和 k 之间的平衡。因为与分组密码相比而言,RSA 的计算能力更强,我们希

望每个循环尽可能生成更多的伪随机位，同时又想 k 取较大值。尽管如此，对于密码的长度，我们还是希望 r 尽可能大一些。

例如：如果 $e=3, N=1024$。则有不等式 $3r>1024$，于是 r 只能取 683 个位。当 r 去这个大小时，$k=341$，即每个求幂运算生成的位（任何 RSA 编码）。在这种情况下，每个求幂运算只需一个模去 683 个位数的平方和一个模的倍数。即 $(x_i \times (x_i^2 \bmod n)) \bmod n$。

10.5.2 基于椭圆曲线密码的 PRNG

在这个小节中，主要归纳由美国国家安全局（NSA）公布的对偶椭圆曲线 PRNG（DEC PRNG）技术。这种技术在 NIST SP 800-90，ANST 标准 X9.82 和 ISO 标准 18031 中介绍过。与其他变量（例如像[SCHO06]，[BROW07]）相比，其安全性和有效性要更好一些。

[SCHO06]总结的算法如下：让 P 和 Q 是给定椭圆曲线的两个已知点。这个 DEC PRNG 的种子为一个随机整数 $s_0 \in \{0,1,\cdots,\#E(\mathrm{GF}(p)) - 1\}$，其中 $\#E(\mathrm{GF}(p))$ 表示椭圆曲线的解点数。让 x 表示曲线上的 x 轴上得到点的函数，$lsb_i(s)$ 表示整数 s 的最小有意义位。DEC PRNG 将种子转变为长度为 $240k, k>0$ 的伪随机序列的过程为

```
for i = 1 to k do
    Set s_i ← x(s_{i-1} P)
    Set r_i ← lsb_240 (x(s_i Q))
end for
    Return r_1, . . . , r_k
```

如果这个 PRNG 受到安全性的影响。那么它唯一的动机就是用于与 ECC 互补，但是不可用于建构 PRNG 中用到的对称的、非对称的，或者 Hash 密码算法的系统中。

10.6　推荐读物和网站

[ROSI99]介绍了椭圆曲线密码学，该书简单易懂。其重点在于软件实现。另一本好读但很严谨的书是[HANK04]。[BLAK99]和[ENGE99]讨论了某些非常难的数学问题。[KUMA98]，[STIN06]和[KOBL94]也是非常好的书，其文字更为简洁。[FERN99]和[JURI97]综述了椭圆曲线密码学。

BLAK99 Blake, I.; Seroussi, G.; and Smart, N. *Elliptic Curves in Cryptography*. Cambridge: Cambridge University Press, 1999.

ENGE99 Enge, A. *Elliptic Curves and their Applications to Cryptography*. Norwell, MA: Kluwer Academic Publishers, 1999.

FERN99 Fernandes, A. "Elliptic Curve Cryptography." *Dr. Dobb's Journal*, December 1999.

HANK04 Hankerson, D.; Menezes, A.; and Vanstone, S. *Guide to Elliptic Curve Cryptography*. New York: Springer, 2004.

JURI97 Jurisic, A., and Menezes, A. "Elliptic Curves and Cryptography." *Dr. Dobb's Journal*, April 1997.

KOBL94 Koblitz, N. *A Course in Number Theory and Cryptography*. New York: Springer-Verlag, 1994.

KUMA98 Kumanduri, R., and Romero, C. *Number Theory with Computer Applications*. Upper Saddle River, NJ: Prentice Hall, 1998.

ROSI99 Rosing, M. *Implementing Elliptic Curve Cryptography*. *Greenwich*, CT: Manning Publications, 1999.

STIN06 Stinson, D. *Cryptography: Theory and Practice*. Boca Raton, FL: CRC Press, 2006.

推荐网站

- Certicom：广泛收集了椭圆曲线密码学和密码学其他领域的技术资料。

10.7 关键术语、思考题和习题

关键术语

Abel 群	二元曲线	三次方程
Diffie-Hellman 密钥交换	离散对数	椭圆曲线
椭圆曲线算术	椭圆曲线密码学	有限域
密钥分配	中间人攻击	素曲线
本原根	零点	

思考题

10.1 简要说明 Diffie-Hellman 密钥交换。

10.2 什么是椭圆曲线?

10.3 什么是椭圆曲线的零点?

10.4 椭圆曲线上同在一条直线上的三个点的和是什么?

习题

10.1 用户 A 和 B 使用 Diffie-Hellman 密钥交换技术来交换密钥,设公用素数 $q=71$,本原根 $\alpha=7$。

(a) 若用户 A 的私钥 $X_A=5$,则 A 的公钥 Y_A 为多少?

(b) 若用户 B 的私钥 $X_B=12$,则 B 的公钥 Y_B 为多少?

(c) 共享的密钥为多少?

10.2 设 Diffie-Hellman 方法中,公用素数 $q=11$,本原根 $\alpha=2$。

(a) 证明 2 是 11 的本原根。

(b) 若用户 A 的公钥 $Y_A=9$,则 A 的私钥 X_A 为多少?

(c) 若用户 B 的公钥 $Y_B=3$,则共享的密钥 K 为多少?

10.3 在 Diffie-Hellman 协议里,每一方都选择一个秘密参数 x,给对方发送 $\alpha^x \bmod q$,其中 α 是一个公开的数。如果通信参与方给对方发送的是 x^α,α 是一个公开的数,将会怎样?请给出至少一种方法使得 Alice 和 Bob 可以协商密钥。Eve 在不知道秘密参数的情况下能攻击你的系统吗?Eve 能找到秘密参数吗?

10.4 这个习题表明如果没有取模的步骤,则 Diffie-Hellman 协议是不安全的,即非离散对数不是难题!假设你是 Eve,你已捕捉了 Alice 和 Bob,并监控了他们。你偷听到如下对话:

Bob:哦,我们不要再为 Diffie-Hellman 协议里的素数费心了,那样将会更简单。

Alice:好吧,但我们仍需要一个基 α,那么 $g=3$ 如何?

Bob:好啊,如此,我的结果是 27。

Alice:我的是 243。

Bob 的秘密参数 X_B,Alice 的秘密参数 X_A 是多少? 他们的共享密钥是多少(别忘了写出你的过程)?

10.5　10.2 节介绍了针对 Diffie-Hellman 密钥交换协议的中间人攻击。攻击者生成了两组公 - 私钥对,请解释如果只生成一组公 - 私钥对可以完成攻击吗?

10.6　设 ElGamal 体制的公用素数 $q=71$,其本原根 $\alpha=7$。

(a) 若 B 的公钥 $Y_B=3$,A 选择的随机整数 $k=2$,则 $M=30$ 的密文是什么?

(b) 若 A 选择的 k 值使得 $M=30$ 的密文为 $C=(59,C_2)$,则整数 C_2 是多少?

10.7　根据实数域上椭圆曲线算术中的规则(5),要计算点 Q 的两倍,则画一条切线并找出另一交点 S。那么 $Q+Q=2Q=-S$。若该切线不是垂直的,则恰好只有一个交点。但是若切线是垂直的,那么在这种情况下,$2Q$ 的值是多少? $3Q$ 的值是多少?

10.8　证明图 10.4 中两条实数域上的椭圆曲线均满足群的条件。

10.9　点(4,7)在椭圆曲线 $y^2=x^3-5x+5$(定义在实数域上)上吗?

10.10　设实数域上的椭圆曲线为 $y^2=x^3-36x$,令 $P=(-3.5,9.5)$,$Q=(-2.5,8.5)$。计算 $P+Q$ 和 $2P$。

10.11　椭圆曲线方程 $y^2=x^3+10x+5$ 在 Z_{17} 上能定义一个群吗?

10.12　考虑椭圆曲线 $E_{11}(1,6)$,即由 $y^2=x^3+x+6$ 定义的曲线,其模数 $p=11$。确定 $E_{11}(1,6)$ 上所有的点。提示:对 x 的所有可能值计算方程右边的值。

10.13　下列 Z_{17} 上椭圆曲线的点的负数为几? $P=(5,8)$;$Q=(3,0)$;$R=(0,6)$。

10.14　对 $E_{11}(1,6)$ 上的点 $G=(2,7)$,计算 $2G$ 到 $13G$ 的值。

10.15　利用 10.4 节中所给出的方法实现椭圆曲线的加/解密。该密码体制的参数是 $E_{11}(1,6)$ 和 $G=(2,7)$,B 的私钥 $n_B=7$。

(a) 找出 B 的公钥 P_B。

(b) A 要加密消息 $P_m=(10,9)$,其选择的随机值 $k=3$,试确定密文 C_m。

(c) 试给出 B 由 C_m 恢复 P_m 的计算过程。

10.16　下面是椭圆曲线签名方案的一种尝试。我们有一个全局的椭圆曲线,素数 p,和"生成元" G。Alice 挑一个秘密的签名密钥 X_A,并产生公开的验证密钥 $Y_A=X_AG$。为了对消息 M 签名:

- Alice 挑一个值 k。
- Alice 将 M,k,以及签名 $S=M-kX_AG$ 发送给 Bob。
- Bob 验证 $M=S+kY_A$

(a) 请说明该方案的工作原理。即说明验证过程可以说明签名是有效的。

(b) 通过描述对任意消息的伪造签名来说明该方案是不可接受的。

10.17　下面是上述习题方案的一种改进。如前,我们有一个全局的椭圆曲线,素数 p,和"生成元" G。Alice 挑一个秘密的签名密钥 X_A 并产生公开的验证密钥 $Y_A=X_AG$。为了对消息 M 签名:

- Bob 挑一个值 k。
- Bob 将 $C_1=kG$ 发给 Alice。
- Alice 将 M,以及签名 $S=M-X_AC_1$ 发送给 Bob。
- Bob 验证 $M=S+kY_A$

(a) 请说明该方案的工作原理。即说明验证过程可以说明签名是有效的。

(b) 说明在这个方案里伪造签名的困难性和攻击(ElGamal)椭圆曲线密码一样难(或者找出一种简单的办法去伪造消息)。

(c) 与我们看过的其他密码体制和签名方案比较,本方案有一个额外的"传递"。这有何缺点?

第三部分　密码学数据完整性算法

第 11 章　密码学 Hash 函数
第 12 章　消息认证码
第 13 章　数 字 签 名

第 11 章　密码学 Hash 函数

11.1　密码学 Hash 函数的应用
- 11.1.1　消息认证
- 11.1.2　数字签名
- 11.1.3　其他应用

11.2　两个简单的 Hash 函数

11.3　需求和安全性
- 11.3.1　密码学 Hash 函数的安全性需求
- 11.3.2　穷举攻击
- 11.3.3　密码分析

11.4　基于分组密码链接的 Hash 函数

11.5　安全 Hash 算法(SHA)
- 11.5.1　SHA-512 逻辑
- 11.5.2　SHA-512 轮函数
- 11.5.3　示例

11.6　SHA-3

11.7　推荐读物和网站

11.8　关键术语、思考题和习题

附录 11A　生日攻击的数学基础

Each of the messages, like each one he had ever read of Stern's commands, began with a number and ended with a number or row of numbers. No efforts on the part of Mungo or any of his experts had been able to break Stern's code, nor was there any clue as to what the preliminary number and those ultimate numbers signified.

—*Talking to Strange Men*, Ruth Rendell

The Douglas Squirrel has a distinctive eating habit. It usually eats pine cones from the bottom end up. Partially eaten cones can indicate the presence of these squirrels if they have been attacked from the bottom first. If, instead, the cone has been eaten from the top end down, it is more likely to have been a crossbill finch that has been doing the dining.

—*Squirrels: A Wildlife Handbook*, Kim Long

- Hash 函数将可变长度的消息映射为固定长度的 Hash 值或消息摘要。
- 事实上，所有的密码学 Hash 函数都使用了压缩函数的迭代结构。
- 安全 Hash 函数中用到的压缩函数可以分为两类：专为 Hash 函数设计的函数或对称分组密码。SHA 和 Whirlpool 分别是这两种方法的例子。

Hash 函数 H 将可变长度的数据块 M 作为输入，产生固定长度的 Hash 值 $h = H(M)$。一个“好”的 Hash 函数具有这样的特点：对于大的输入集合使用该函数，产生的输出结果均匀地分布且看起来随机。概括地说，Hash 函数的首要目标是保证数据完整性，对于 M 任何一位或几位的改变，都将极大可能改变其 Hash 码。

在安全应用中使用的 Hash 函数称为密码学 Hash 函数。密码学 Hash 函数要求如下两种情况在计算上不可行(即没有攻击方法比穷举攻击更有效)：(a)对预先指定的 Hash 值找到对应的数据块(单向性)；(b)找到两个不同的数据块对应相同的 Hash 值(抗碰撞性)。由于 Hash 函数具有上述特性，所以常被用于判断数据是否被篡改过。

图 11.1 描述了密码学 Hash 函数的操作过程。通常输入数据的长度首先被填充为某固定长

度(如 1024 位)分组的整数倍,填充的内容包括原始消息的位长度信息。填充长度信息能够提高攻击者修改消息而保持 Hash 值不变的难度。

本章首先介绍密码学 Hash 函数的各种广泛用途;然后讨论 Hash 函数的安全性需求;接下来介绍如何通过分组密码链构造密码学 Hash 函数;本章剩余的部分介绍最重要的也是应用最广泛的一类密码学 Hash 函数:安全 Hash 算法(SHA)系列算法。

附录 N 介绍另外一种广泛使用的密码学 Hash 函数,即 Whirlpool 算法。

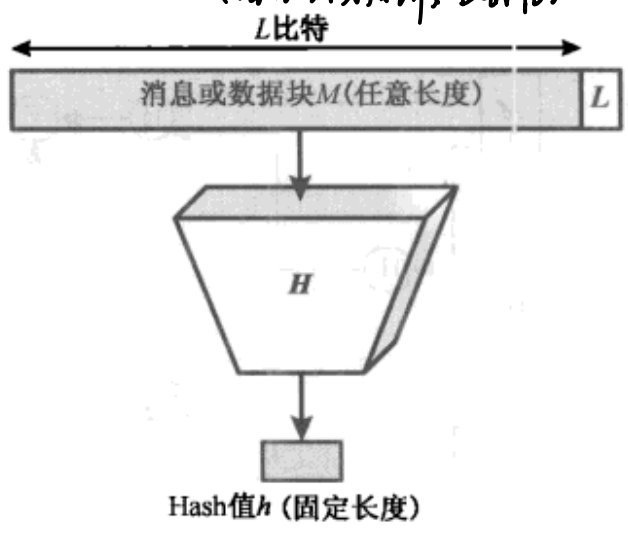


图 11.1　密码学 Hash 函数框图;$h = H(M)$

11.1　密码学 Hash 函数的应用

密码学 Hash 函数或许是用途最多的密码算法,它被广泛应用于各种不同的安全应用和网络协议中。为了更好地理解密码学 Hash 函数的安全性需求和含义,我们有必要首先考虑密码学 Hash 函数的应用范围。

11.1.1　消息认证

消息认证是用来验证消息完整性的一种机制或服务。消息认证确保收到的数据确实和发送时的一样(即没有修改、插入、删除或重放)。此外,通常还要求消息认证机制确保发送方声称的身份是真实有效的。当 Hash 函数用于提供消息认证功能时,Hash 函数值通常称为消息摘要。

图 11.2 展示了 Hash 码能够通过如下各种不同的方法用于提供消息认证。

(a) 使用对称密码算法 E 加密消息和 Hash 码。因为只有 A 和 B 共享密钥 K,所以消息必然是发自 A 处,并且未被更改过。这里附加的 Hash 码提供了实现认证功能的结构。因为对于整个消息以及 Hash 码都使用了加密,保密性也被提供。

(b) 使用对称密码算法 E 只对 Hash 码进行加密。对于无须保密性的应用,这种方案减少了加解密操作的负担。

(c) 不使用加密算法,仅使用 Hash 函数也能够实现消息认证。该方案假设通信双方共享相同的秘密值 S。发送方 A 将消息 M 和秘密值 S 串联后计算其 Hash 值,并将得到的 Hash 值附在消息 M 后发送。因为接收方 B 同时掌握 S,所以能够重新计算该 Hash 值进行验证。由于秘密值 S 本身并没有在信道传送,攻击者不能够对在信道上拦截的消息进行修改,进而也不能制作假消息。

(d) 通过将整个消息和 Hash 值加密,能够在方案(c)的基础上提供保密性。

由于方案(b)中所需的计算较少,而方案(a)和方案(d)需要加密整个消息,所以如果不要求提供保密性,方案(b)比方案(a)和(d)更有优势。而且人们越来越对那些不含加密函数的方法感兴趣[参见图 11.2(c)],所以(b)和(c)比那些对整条消息加密的方法要好一些。[TSUD92]中给出了几种不希望使用加密函数的理由:

- 加密软件速度慢。即使每条消息需要加密的数据量不大,但是总有消息串需要通过加密系统输入或输出。
- 加密硬件成本不容忽视。尽管已有实现 DES 的低成本芯片,但是如果网络中所有结点都必须有该硬件,则总成本可能很大。

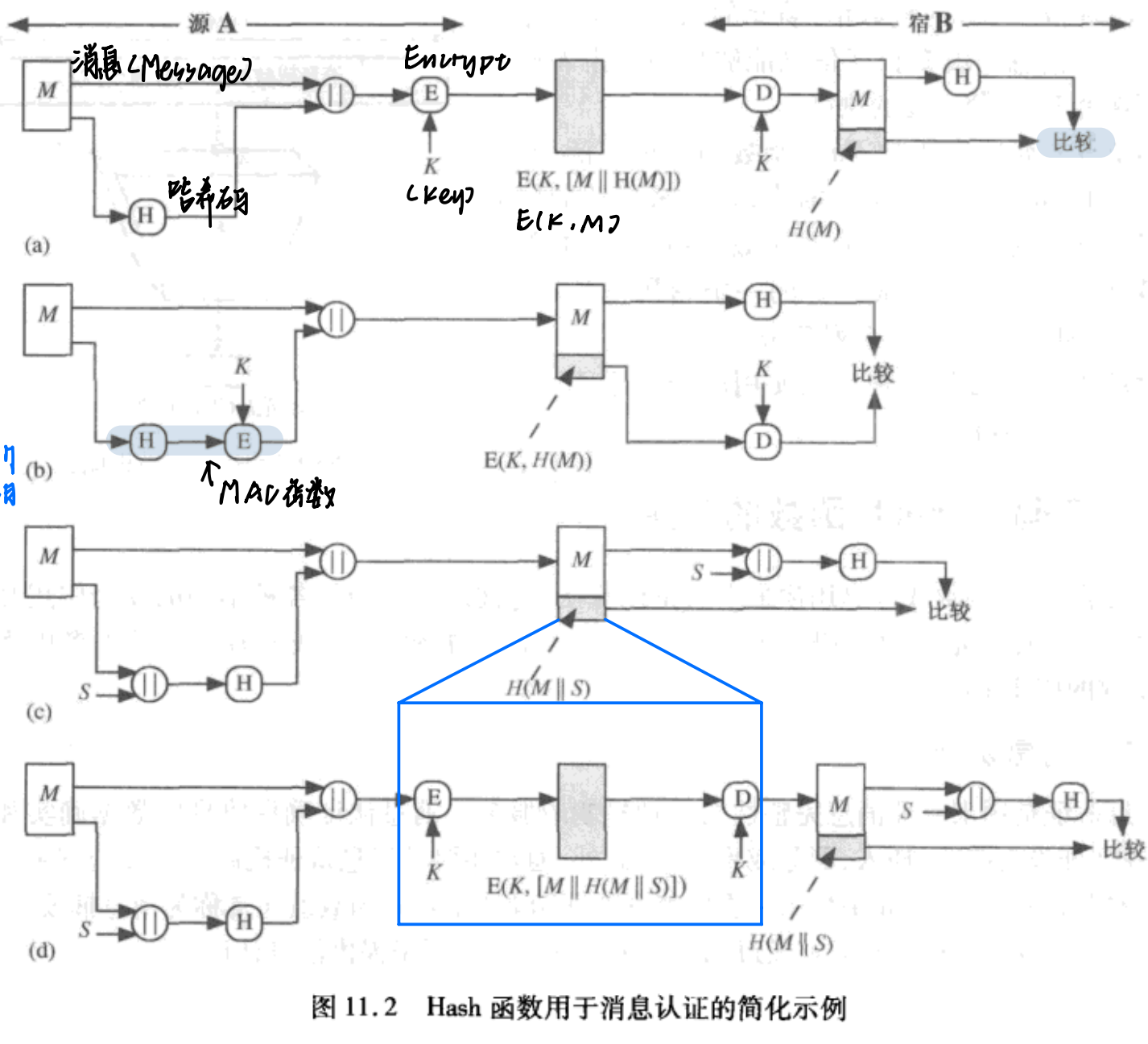


图 11.2 Hash 函数用于消息认证的简化示例

- 加密硬件的优化通常是针对大数据块的。对于小数据块,大比例的时间开销在初始化/调用之上。
- 加密算法可能受专利保护,这也会增加成本。

更一般地,消息认证是通过使用消息认证码(MAC)实现的,即带密钥的 Hash 函数。通常情况下,通信双方基于共享的同一密钥来认证彼此之间交互的信息时,就会使用 MAC。MAC 函数将密钥和数据块作为输入,产生 Hash 值作为 MAC 码,然后将 MAC 码和受保护的消息一起传递或存储。需要检查消息的完整性时,使用 MAC 函数对消息重新计算,并将计算结果与存储的 MAC 码对比。攻击者能对消息进行篡改,但在不知道密钥的情况下不能够计算出与篡改后的消息相匹配的 MAC 值。注意,这里验证方也知道发送方是谁,因为除了通信双发之外其他人不知道密钥。

总体来看,MAC[参见图 11.2(b)]是 Hash 函数和加密函数操作的结合,即对于函数 $E(K,H(M))$,长度可变的消息 M 和密钥 K 是函数的输入,输出是固定长度的值。MAC 提供安全保护,用于抵抗不知道密钥的攻击者的攻击。在实现中,往往使用比加密算法效率更高的特殊设计的 MAC 算法。

我们将在第 12 章详细讨论消息认证码。

11.1.2 数字签名

与消息认证应用类似,对于 Hash 函数的另外一个重要应用就是数字签名。数字签名的操作与 MAC 相似,在进行数字签名过程中使用用户的私钥加密消息的 Hash 值,其他任何知道该用户公钥的人都能够通过数字签名来验证消息的完整性。在这种情况下,攻击者要想篡改消息,则需

要知道用户的私钥。在第 14 章中我们将看到，数字签名的应用比消息认证更为广泛。

图 11.3 简要描述了 Hash 码用于提供数字签名的方案。

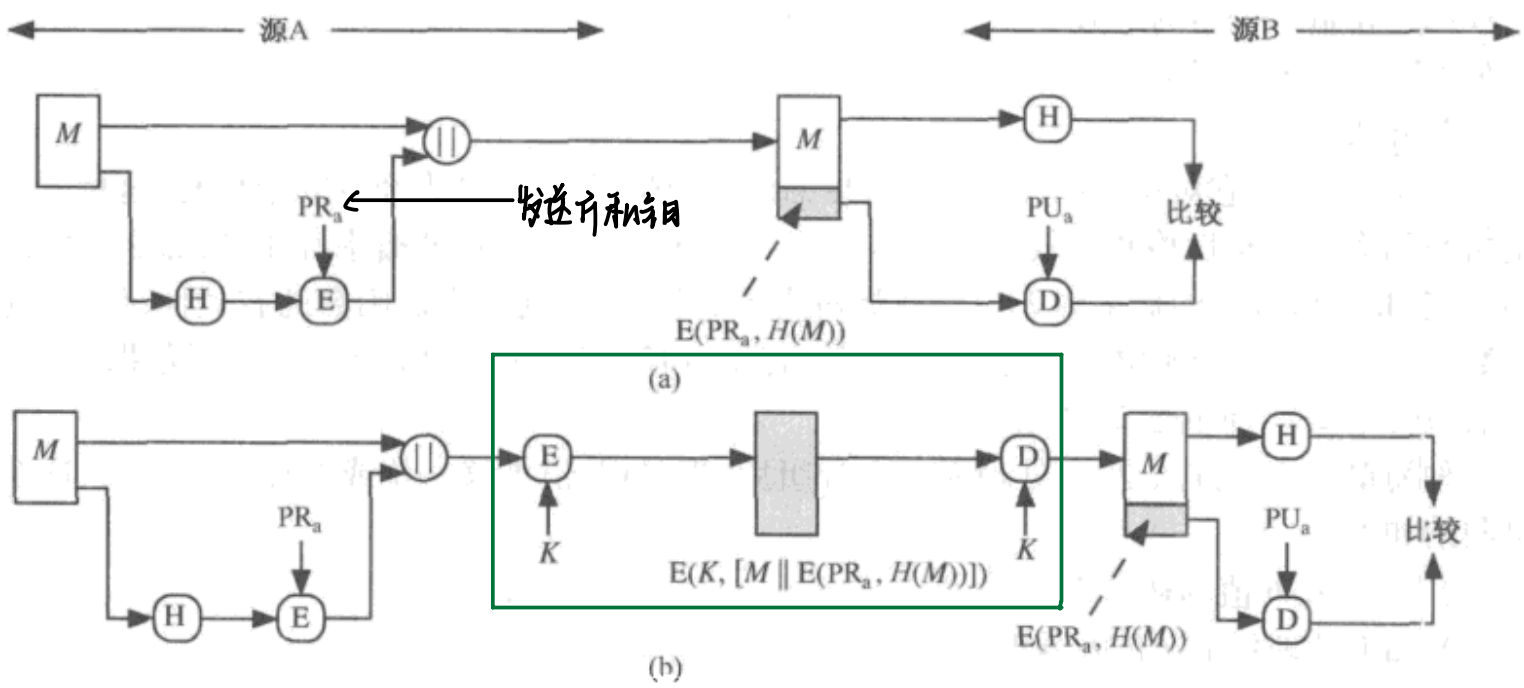


图 11.3　数字签名简要示例

(a)使用发送方的私钥利用公钥密码算法对 Hash 码进行加密。与图 11.2(b)一样，这种方法可提供认证；由于只有发送方可以产生加密后的 Hash 码，所以这种方法也提供了数字签名，事实上，这就是数字签名技术的本质所在。

(b)若既希望保证保密性又希望有数字签名，则先用发送方的私钥对 Hash 码加密，再用对称密码中的密钥对消息和公钥算法加密结果进行加密，这种技术比较常用。

11.1.3　其他应用

① 对于 Hash 函数，通常还被用于产生单向口令文件。第 20 章解释了操作系统为何存储口令的 Hash 值而不是口令本身的原因，即这样一来黑客即使能够访问口令文件，也不能够获取真正的口令。当用户输入口令时，操作系统将比对输入口令的 Hash 值和存储在口令文件中的 Hash 值。在大多数操作系统中都使用了这样的口令保护机制。

② Hash 函数能用于入侵检测和病毒检测。将每个文件的 Hash 值 $H(F)$ 存储在安全系统中(如 CD-R 中)，随后就能够通过重新计算 $H(F)$ 来判断文件是否被修改过。入侵者只能够改变 F，而不能够改变 $H(F)$。

③ 密码学 Hash 函数能够用于构建随机函数(PRF)或用做伪随机数发生器(PRNG)。基于 Hash 函数的 PRF 可用于对称密码中的密钥产生，我们将在第 12 章详细讨论。

11.2　两个简单的 Hash 函数

为了展示密码学 Hash 函数应考虑到的安全性，本节我们给出两个简单的、不够安全的 Hash 函数例子。所有的 Hash 函数都按下面的一般原理进行运算：输入(消息、文件等)都可视为一个 n 位分组的序列，其输出是 n 位的 Hash 值。Hash 函数每次处理一个分组，重复该过程直至处理完所有的输入分组。

最简单的 Hash 函数之一，是将每个分组相应位异或(XOR)，这个函数可如下描述为

$$C_i = b_{i1} \oplus b_{i2} \oplus \ldots \oplus b_{im}$$

其中

C_i为 Hash 码的第 i 位,$1 \leqslant i \leqslant n$

m 为 n 位输入分组的个数

b_{ij}为第 j 个分组的第 i 位

⊕为异或运算

上述运算过程,它对每一位产生一个简单的奇偶校验,我们称之为经度冗余校验,这种方法用于随机数的数据完整性检查比较有效。如果每个 n 位 Hash 值出现的概率都相同,那么数据出错而不引起 Hash 值改变的概率为 2^{-n};若数据格式不是随机的,则会降低函数的有效性,例如,通常大多数文本文件中每个 8 位字节的高位总为 0,若使用 128 位的 Hash 值,则对于这类数据,Hash 函数的有效性是 2^{-112} 而不是 2^{-128}。

一种简单的改进方法是,每处理完一个分组后,将 Hash 值平移一位或循环移位一次,这个过程可归纳如下:

(1) n 位 Hash 值的初值为 0。

(2) 如下处理每个 n 位的分组:

(a) 将当前的 Hash 值循环左移一位。

(b) 将该分组与 Hash 值异或。

这样可使输入更加完全地"随机",从而消除输入数据的规则性。图 11.4 给出了两种产生16 位 Hash 值的 Hash 函数。

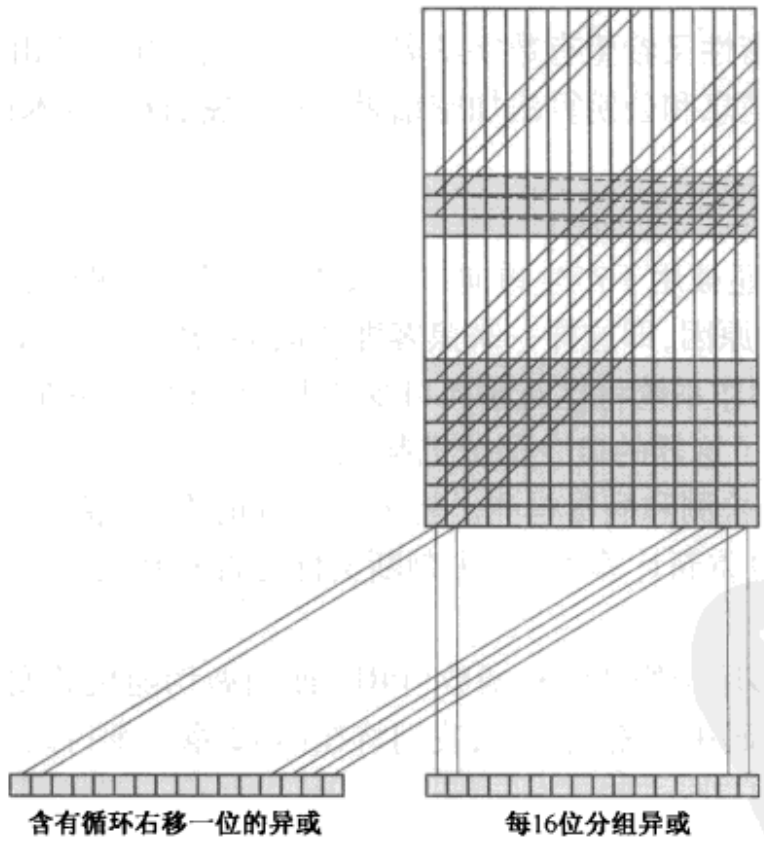


图 11.4 两种简单的 Hash 函数

虽然第二种方法可以很好地保证数据完整性,但是如果使用图 11.2(b)和图 11.3(a)中所示的方法,即将加密后的 Hash 码附加在明文之后,那么该方法不能保证数据安全性。因为很容易产生一条新消息,使它与给定的消息具有相同的 Hash 码:先选定某消息,然后在其后附加一个 n 位分组,使它们与给定的消息具有相同的 Hash 码。

如果只对 Hash 码加密,那么上述简单异或或循环异或(RXOR)方法不能保证数据安全性,但是如果对消息和 Hash 码均加密,也许你会觉得那样一个简单函数还是有用的[参见图 11.2(a)]。

但必须要小心。美国国家标准局最初提出了一种方法，这种方法对 64 位的分组执行简单异或操作，使用密文分组链接（CBC）方式对整个消息加密。给定消息 $X_1, X_2, \cdots, X_N$，其中 X_i 是 64 位的分组，其 Hash 码 $h = H(M)$ 为所有分组的异或，并且将该 Hash 码作为最后一个分组：

$$h = X_{N+1} = X_1 \oplus X_2 \oplus \cdots \oplus X_N$$

然后使用 CBC 方式对消息和 Hash 码加密得到 $Y_1, Y_2, \cdots, Y_{N+1}$。[JUEN85] 中给出了几种改变消息密文而 Hash 码无法检测的攻击。例如，根据 CBC 的定义（参见图 6.4），我们有

$$X_1 = IV \oplus \mathrm{D}(K, Y_1)$$
$$X_i = Y_{i-1} \oplus \mathrm{D}(K, Y_i)$$
$$X_{N+1} = Y_N \oplus \mathrm{D}(K, Y_{N+1})$$

且 Hash 码 X_{N+1} 为

$$\begin{aligned} X_{N+1} &= X_1 \oplus X_2 \oplus \cdots \oplus X_N \\ &= [\mathrm{IV} \oplus \mathrm{D}(K, Y_1)] \oplus [Y_1 \oplus \mathrm{D}(K, Y_2)] \oplus \cdots \oplus [Y_{N-1} \oplus \mathrm{D}(K, Y_N)] \end{aligned}$$

由于上述等式中异或可以按任意顺序计算，所以改变密文分组的顺序，Hash 码仍然不变。

11.3　需求和安全性

首先我们需要定义两个术语：对于 Hash 函数 $h = H(x)$，称 x 是 h 的原像。即把数据块 x 作为输入，使用 Hash 函数 H 得到 h。因为 H 是多对一映射，所以对于任意给定的 Hash 值 h，对应有多个原像。如果满足 $x \neq y$ 且 $H(x) = H(y)$，则称出现碰撞。因为我们使用 Hash 函数的目的是保证数据完整性，碰撞的出现显然是我们不希望的。

我们首先考虑对于给定的 Hash 值，有多少个对应的原像，这也是对于给定的 Hash 值的潜在碰撞个数的衡量。假设 Hash 码的长度是 n 位，函数 H 的输入消息或数据块长度是 b 位，且 $b > n$。那么消息的全部可能取值的个数为 2^b，Hash 值的全部可能取值的个数为 2^n，平均每个 Hash 值对应 $2^{b/n}$ 个原像。如果 H 的 Hash 值趋于均匀分布，事实上每个 Hash 值将对应近似 $2^{b/n}$ 个原像。如果我们允许输入是任意长度而不是固定长度，那么每个 Hash 值对应的原像个数将更大。然而，实际使用 Hash 函数的安全性风险并非如上面的分析所表现的一样。为了更好地理解密码学 Hash 函数的安全含义，需要精确地定义 Hash 函数的安全性需求。

11.3.1　密码学 Hash 函数的安全性需求

表 11.1 列出了被广泛认同的密码学 Hash 函数的安全性需求。前三个特性是 Hash 函数实际应用的需求。

第 4 个条件单向性是指，由消息很容易计算出 Hash 码，但是由 Hash 码却不能计算出相应的消息。对使用一个秘密值的认证方法[参见图 11.2(c)]，这个性质非常重要。虽然该秘密值本身并不传送，但若 Hash 函数不是单向的，则攻击者可以按如下方式很容易地找出这个秘密值：若攻击者能够观察或截获到传送的消息，则他可以得到消息 M 和 Hash 码 $h = H(S_{AB} \| M)$，然后求出 Hash 函数的逆，从而得出 $S \| M = H^{-1}(\mathrm{MD}_M)$。由于攻击者已知 M 和 $S_{AB} \| M$，所以可得出 S_{AB}。

第 5 个性质抗弱碰撞性可以保证，不能找到与给定消息具有相同 Hash 值的另一消息，因此可以在使用对 Hash 码加密的方法中防止伪造[参见图 11.2(b)和图 11.3(a)]。如果第 5 条不成立，那么攻击者可以先观察或截获一条消息及其加密的 Hash 码，然后由消息产生一个未加密的 Hash 码，最后产生另一个具有相同 Hash 码的替代消息。

表 11.1 密码学 Hash 函数 H 的安全性需求

需　求	描　述
输入长度可变	H 可应用于任意大小的数据块
输出长度固定	H 产生定长的输出
效率	对任意给定的 x,计算 $H(x)$ 比较容易,用硬件和软件均可实现
抗原像攻击(单向性)	对任意给定的 Hash 码 h,找到满足 $H(y)=h$ 的 y 在计算上是不可行的
抗第二原像攻击(抗弱碰撞性)	对任何给定的分块 x,找到满足 $y\neq x$ 且 $H(x)=H(y)$ 的 y 在计算上是不可行的
抗碰撞攻击(抗强碰撞性)	找到任何满足 $H(x)=H(y)$ 的偶对 (x,y) 在计算上是不可行的
伪随机性	H 的输出满足伪随机性测试标准

如果一个 Hash 函数满足表 11.1 中的前 5 个要求,就称其为弱 Hash 函数。如果第 6 个性质抗强碰撞性也满足,就称其为强 Hash 函数。强 Hash 函数能够保证免受如下的攻击:通信双方中的一方生成消息,而另一方对消息进行签名。例如,假设 Bob 写一条 IOU(借据)消息并发送给 Alice,Alice 在借据上签名认可。Bob 如果能够找到两个消息具有同样的 Hash 值,其中一个借据消息要求 Alice 归还金额较小,另一个金额很大,那么让 Alice 签下第一个小额借据后,Bob 就能够声称第二个(大额)借据是真实的。

图 11.5 展示了三个安全特性(抗原像攻击、抗弱碰撞攻击、抗强碰撞攻击)之间的联系。一个函数如果是抗强碰撞的,那么也同时是抗弱碰撞的,但反之则不一定成立。一个函数可以是抗强碰撞的,但不一定是抗原像攻击的,反之亦然。一个函数可以是抗弱碰撞的,但不一定是抗原像攻击的,反之亦然。详细讨论请参见参考文献[MENE97]。

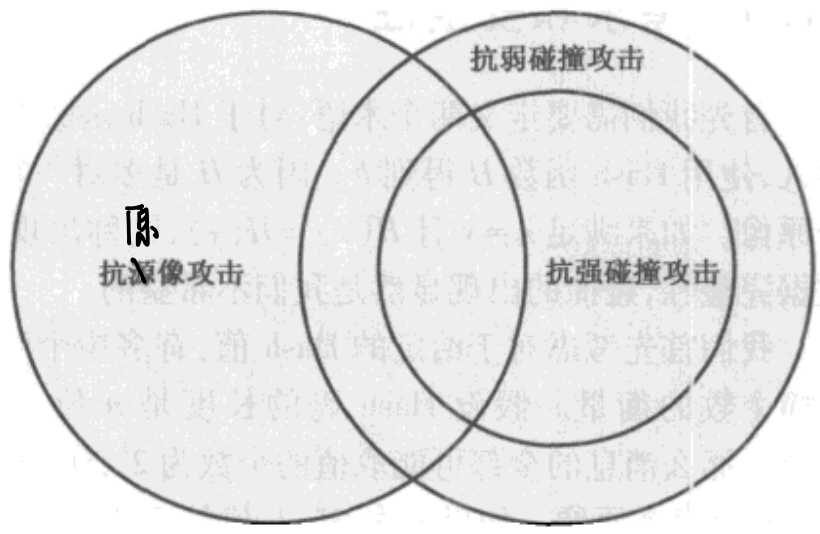


图 11.5　Hash 函数安全特性之间的联系

表 11.2 展示了在不同应用环境下 Hash 函数的安全性需求。

表 11.1 中的最后一个性质是伪随机性,在传统观念中并没有作为密码学 Hash 函数的安全性需求,但却在实际使用中或多或少有所要求。参考文献[JOHN05]指出密码学 Hash 函数通常用于密钥产生、伪随机数发生器以及消息完整性应用,上述三个安全特性都要求 Hash 函数的输出是随机的。因此需要 Hash 函数的输出具有伪随机性。

表 11.2 各种数据完整性应用中的 Hash 函数安全性需求

	抗原像攻击	抗弱碰撞攻击	抗强碰撞攻击
Hash + 数字签名	是	是	是*
入侵检测和病毒检测		是	
Hash + 对称加密			
单向口令文件	是		
MAC	是	是	是*

注:标 * 处要求攻击者能够实现选择消息攻击。

11.3.2　穷举攻击

同加密算法一样,对于 Hash 函数的攻击也分为两类:穷举攻击和密码分析。穷举攻击不依赖于任何算法的细节,仅与相应的长度相关。对于 Hash 函数,穷举攻击仅与算法所产生的 Hash 值

的长度相关。与之相反，密码分析依赖于具体算法的设计缺点。我们首先介绍穷举攻击。

原像攻击和第二原像攻击

对于原像攻击和第二原像攻击，攻击者对给定的 Hash 值 h，试图找到满足 $H(y)=h$ 的 y。穷举攻击的方法是随机选择 y，尝试计算其 Hash 值直到碰撞出现。对于 m 位的 Hash 值，穷举规模大约是 2^m。对于攻击者平均尝试次数为 2^{m-1}，才能找到一个满足 $H(y)=h$ 的 y。该结论请参考本章的附录 11A 中的式(11.1)。

碰撞攻击

对于碰撞攻击，攻击者试图找到两个消息或数据块 x 和 y，满足 $H(x)=H(y)$。可想而知，与原像攻击和第二原像攻击相比，其穷举规模相对更小一些，这也通过数学上的生日悖论得到印证。本质上，如果我们在均匀分布的 0 到 $N-1$ 的范围内选择随机整数变量，那么在 $\sqrt{N}$ 次选择后发生重复的概率就会超过 0.5。因此，对于 m 位的 Hash 值，如果我们随机选择数据块，预计在 $\sqrt{2^m}=2^{m/2}$ 次尝试后就将找到两个具有相同 Hash 值的数据块。具体证明请见本章的附录 11A。

Yuval 提出了以下策略通过生日悖论进行碰撞攻击[YUVA79]：

(1) 发送方 A 准备对文本消息 x 进行签名，其使用的方法是：用 A 的私钥对 m 位的 Hash 码加密并将加密后的 Hash 码附于消息之后[参见图 11.3(a)]。

(2) 攻击者产生该(合法)消息 x 的 $2^{m/2}$ 种变式 x'，且每一种变式表达相同的意义，将这些消息以及对应的 Hash 值存储起来。

(3) 攻击者准备伪造一条消息 y，并想获取 A 的签名。

(4) 攻击者再产生该伪造(非法)消息 y 的消息变式 y'，每个变式 y' 与 y 表达相同的意义。对于每个 y'，攻击者计算 $H(y')$，并与任意的 $H(x')$ 进行比对，重复这一过程直到碰撞出现。即这一过程直到找到一个 y' 与某个 x' 具有相同的 Hash 值。

(5) 攻击者将该合法消息变式 x' 提供给 A 签名，将该签名附于伪造消息的有效变式 y' 后并发送给预期的接收方。因为上述两个变式的 Hash 码相同，所以它们产生的签名也相同，因此攻击者即使不知道加密密钥也能攻击成功。

这样，如果使用 64 位的 Hash 码，那么所需代价的数量级仅为 2^{32}[参见本章的附录 11A 中的式(11.7)]。

产生多个具有相同意义的变式并不困难。例如，攻击者可以在文件的词与词之间插入若干“空格 - 空格 - 退格”字符对，然后在实例中用“空格 - 退格 - 空格”替代这些字符，从而产生各种变式。攻击者也可以简单地改变消息中的某些词但不改变消息的意义，图 11.6[DAVI89]举例说明了这种方法。

总之，对长度为 m 的 Hash 码，对于穷举攻击所需的代价分别与下表中的相应量成正比。

抗原像攻击	2^m
抗弱碰撞攻击	2^m
抗强碰撞攻击	$2^{m/2}$

如果要求抗强碰撞能力(通用的安全 Hash 码通常期望具有该性质)，那么值 $2^{m/2}$ 决定了该 Hash 码抗穷举攻击的强度。Van Oorschot 和 Wiener[VANO94]耗资 1000 万美元，为攻击 MD5 设计了一台碰撞搜寻机器，它能在 24 天内找到一个碰撞。MD5 使用的是 128 位 Hash 码，因此一般认为 128 位 Hash 码是不够的。如果将 Hash 码视为一串 32 位的序列，那么以后将要使用 160 位的 Hash

码。对于160位Hash码,用相同的搜寻机器则需要4000年才能找到一个碰撞。然而,随着软硬件技术的进步,搜索的时间会大大缩短,所以对于穷举攻击现在看来160位也被认为比较弱。

Dear Anthony,

{This letter is / I am writing} to introduce {you to / to you} {Mr. / --} Alfred {P. / --}
Barton, the {new / newly appointed} {chief / senior} jewellery buyer for {our / the}
Northern {European / Europe} {area / division}. He {will take / has taken} over {the / --}
responsibility for {all / the whole of} our interests in {watches and jewellery / jewellery and watches}
in the {area / region}. Please {afford / give} him {every / all the} help he {may need / needs}
to {seek out / find} the most {modern / up to date} lines for the {top / high} end of the
market. He is {empowered / authorized} to receive on our behalf {samples / specimens} of the
{latest / newest} {watch and jewellery / jewellery and watch} products, {up / subject} to a {limit / maximum}
of ten thousand dollars. He will {carry / hold} a signed copy of this {letter / document}
as proof of identity. An order with his signature, which is {appended / attached}
{authorizes / allows} you to charge the cost to this company at the {above / head office}
address. We {fully / --} expect that our {level / volume} of orders will increase in
the {following / next} year and {trust / hope} that the new appointment will {be / prove}
{advantageous / an advantage} to both our companies.

图11.6 具有2^{37}种变式的信[DAVI89]

11.3.3 密码分析

与对密码算法的攻击一样,对Hash函数的密码分析攻击,也是利用算法的某种性质而不是通过穷举来进行攻击的。评价Hash算法抗密码分析能力的方法是,将其与穷举攻击所需的代价相比,也就是说,理想的Hash函数算法要求密码分析攻击所需的代价大于或等于穷举攻击所需的代价。

最近这些年,人们在研究对Hash函数的密码分析攻击方面做了大量的工作,其中有些攻击是成功的。在讨论这些问题之前,我们需要先讨论一下典型的安全Hash函数的总体结构,如图11.7所示。这种结构称为迭代Hash函数,它是由Merkle[MERK79,MERK89]提出的。包括本章后面将会介绍的SHA在内的目前所使用的大多数Hash函数都是这种结构。Hash函数将输入消息分为L个固定长度的分组,每一分组长为b位,最后一个分组不足b位时需要将其填充为b位,最后一个分组包含输入的总长度。由于输入中包含长度,所以攻击者必须找出具有相同Hash值且长度相等的两条消息,或者找出两条长度不等但加入消息长度后Hash值相同的消息,从而增加了攻击的难度。

Hash函数中重复使用了压缩函数f,它的输入包括两部分:前一步中得出的n位结果(称为链

接变量)和一个 b 位分组,输出为一个 n 位分组。链接变量的初值由算法在开始时指定,其终值即为 Hash 值,通常 $b > n$,因此称为压缩。Hash 函数可归纳如下:

$$CV_0 = IV = \text{初始}n\text{位值}$$

$$CV_i = f(CV_{i-1}, Y_{i-1}) \quad 1 \leqslant i \leqslant L$$

$$H(M) = CV_L$$

其中 Hash 函数的输入为消息 M,它由分组 $Y_0, Y_1, Y_2, \cdots, Y_{L-1}$ 组成。

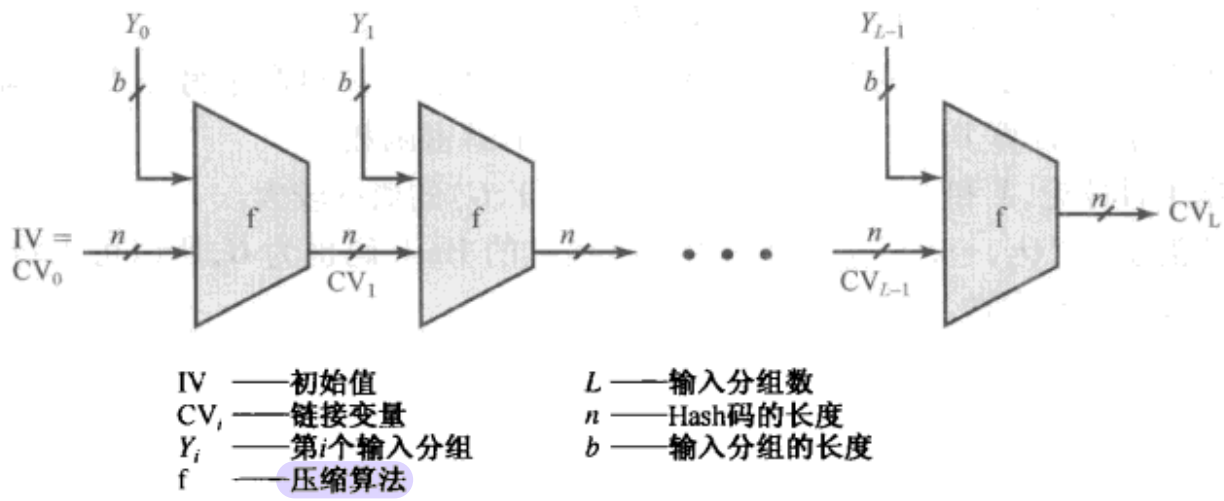


图 11.7　安全 Hash 码的一般结构

Merkle[MERK89]和 Damgard[DAMG89]发现,如果压缩函数具有抗碰撞能力,那么迭代 Hash 函数也具有抗碰撞能力①,因此 Hash 函数常使用上述迭代结构,这种结构可用于对任意长度的消息构造安全 Hash 函数。由此可见,设计安全 Hash 函数可简化为设计具有抗碰撞能力的压缩函数问题,并且该压缩函数的输入是定长的。

对 Hash 函数的密码分析主要集中在对 f 的内部结构进行分析,并试图找到能由 f 的单步执行产生碰撞的有效方法,如果能找到这种方法,那么它必定要考虑固定值 IV。对 f 的攻击要依靠对其内部结构的研究。像对称分组密码一样,f 由若干轮组成,因此要攻击 f,攻击者就必须分析轮与轮之间位变化的模式。

值得注意的是,由于添加了一个长度域,我们将长度至少为两倍于分组大小 b 的消息映射为长度为 n 的 Hash 码,其中 $b \geqslant n$,所以任何 Hash 函数都存在有碰撞。因此能要求的仅是:找出碰撞在计算上是不可行的。

对 Hash 函数的攻击相当复杂,这些内容已超出了我们的讨论范围,有兴趣的读者可参阅[DOBB96]和[BELL97]。

11.4　基于分组密码链接的 Hash 函数

人们已经提出的 Hash 函数中,有许多基于分组密码链接方法,但没有使用密钥。例如 Rabin 所提出方法[RABI78],将消息 M 分成固定大小的分组 $M_1, M_2, \cdots, M_N$,并使用对称加密体制,如 DES,计算其 Hash 码 G:

Hash
$$\begin{cases} H_0 = \text{初始值} \\ H_i = E(M_i, H_{i-1}) \\ G = H_N \end{cases}$$

① 其逆不一定为真。

这种方法类似于 CBC 方法,但不使用密钥。像任何 Hash 码一样,这种方法也易受到生日攻击,如果使用 DES 加密算法而且仅产生 64 位 Hash 码,那么系统非常脆弱。

即使攻击者只能截获一条消息及其签名,他仍然可以进行另一种形式的生日攻击。例如,假定攻击者截获了一条已签名的消息,该签名是加密后的 Hash 码,加密前 Hash 码长度为 n 位。

(1) 用本小节开始部分定义的算法计算该未加密的 Hash 码 G。
(2) 构造任何形为 $Q_1, Q_2, \cdots, Q_{N-2}$ 的(想要伪造的)消息。
(3) 对 $1 \leqslant i \leqslant (N-2)$,计算 $H_i = \mathrm{E}(Q_i, H_{i-1})$。
(4) 产生 $2^{m/2}$ 个随机分组,对每一分组 X,计算 $\mathrm{E}(X, H_{N-2})$。再产生另外 $2^{m/2}$ 个随机分组,对每一分组 Y,计算 $\mathrm{D}(Y, G)$,其中 D 是对应 E 的解密函数。E(X,HN-2)=D(Y,G)=HN-1
(5) 根据生日悖论,X 和 Y 满足 $\mathrm{E}(X, H_{N-2}) = \mathrm{D}(Y, G)$ 的概率较大。
(6) 构造消息 $Q_1, Q_2, \cdots, Q_{N-2}, X, Y$。因为该消息的 Hash 码也为 G,所以可以将该消息与截获的签名一起发送。

这种形式的攻击称为中间相遇攻击,为了加强密文分组链接方法的抗攻击能力,许多研究者提出了修改建议。例如,Davies 和 Price[DAVI89]提出了下列修改方法:

$$H_i = \mathrm{E}(M_i, H_{i-1}) \oplus H_{i-1}$$

[MEYE88]中提出的修改方法是

$$H_i = \mathrm{E}(H_{i-1}, M_i) \oplus M_i$$

已经证明上述两种方法都易受到各种攻击[MIYA90]。更一般地,可以证明只要 Hash 码较短(如不超过 64 位)或者 Hash 码很长但可分解为独立的子码,就存在有生日攻击,它能成功地攻击任何使用密文分组链接但不使用密钥的 Hash 方法[JUEN87]。

因此我们需要其他的 Hash 方法,但是已经证明,许多这种方法都存在有一些弱点[MITC92]。

11.5 安全 Hash 算法(SHA)

近年来,安全 Hash 算法(SHA)是使用最广泛的 Hash 函数。事实上,由于其余的被广泛应用的 Hash 函数被发现存在安全性缺陷,从 2005 年以来 SHA 或许是这几年中仅存的 Hash 算法标准。SHA 由美国标准与技术研究所(NIST)设计,并于 1993 年作为联邦信息处理标准(FIPS 130)发布。随后该版本的 SHA(即 SHA-0)被发现存在缺陷,修订版于 1995 年发布(FIPS 180-1),通常称之为 SHA-1。实际的标准文件称为安全 Hash 标准。SHA 算法建立在 MD4 算法之上,其基本框架与 MD4 类似。RF C3174 也给出了 SHA-1,它基本上是复制 FIPS 180-1 中的内容,但增加了 C 代码实现。

SHA-1 产生 160 位的 Hash 值。2002 年,NIST 发布了修订版 FIPS 180-2,其中给出了三种新的 SHA 版本,Hash 值长度依次为 256、384 和 512 位,分别称为 SHA-256、SHA-384 和 SHA-512。这些算法被统称为 SHA-2。SHA-2 同 SHA-1 类似,都使用了同样的迭代结构和同样的模算术运算与二元逻辑操作。在 2008 年发布的修订版 FIP PUB 180-3 中,增加了 224 位版本(参见表 11.3)。SHA-2 在 RF C4634 中也有描述,基本上也是复制 FIPS 180-3 中的内容,但增加了 C 代码实现。

2005 年,NIST 宣布了逐步废除 SHA-1 的意图,计划到 2010 年逐步转而依赖 SHA-2 的其他版本。此后不久,一个研究小组[WANG05]给出了一种攻击,用 2^{69} 次操作可以找到两个独立的消息使它们有相同的 SHA-1 值,而以前认为要找到一个 SHA-1 碰撞需要 2^{80} 次操作,所需操作大为减少。这一结果应该加快向 SHA-2 版本的过渡。

本节,我们介绍 SHA-512,其他版本类似。

表 11.3　SHA 参数比较

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
消息摘要长度	160	224	256	384	512
消息长度	$<2^{64}$	$<2^{64}$	$<2^{64}$	$<2^{128}$	$<2^{128}$
分组长度	512	512	512	1024	1024
字长度	32	32	32	64	64
步骤数	80	64	64	80	80

注：所有的长度以位为单位。

11.5.1　SHA-512 逻辑

算法的输入是最大长度小于 2^{128} 位的消息，输出是 512 位的消息摘要，输入消息以 1024 位的分组为单位进行处理。图 11.8 显示了处理消息、输出摘要的总体过程，其迭代结构遵循图 11.7 所示的一般结构。这个过程包含下列步骤：

步骤 1：附加填充位。填充消息使其长度模 1024 与 896 同余[即长度≡896（mod 1024）]，即使消息已经满足上述长度要求，仍然需要进行填充，因此填充位数在 1 ~ 1024 之间。填充由一个 1 和后续的 0 组成。

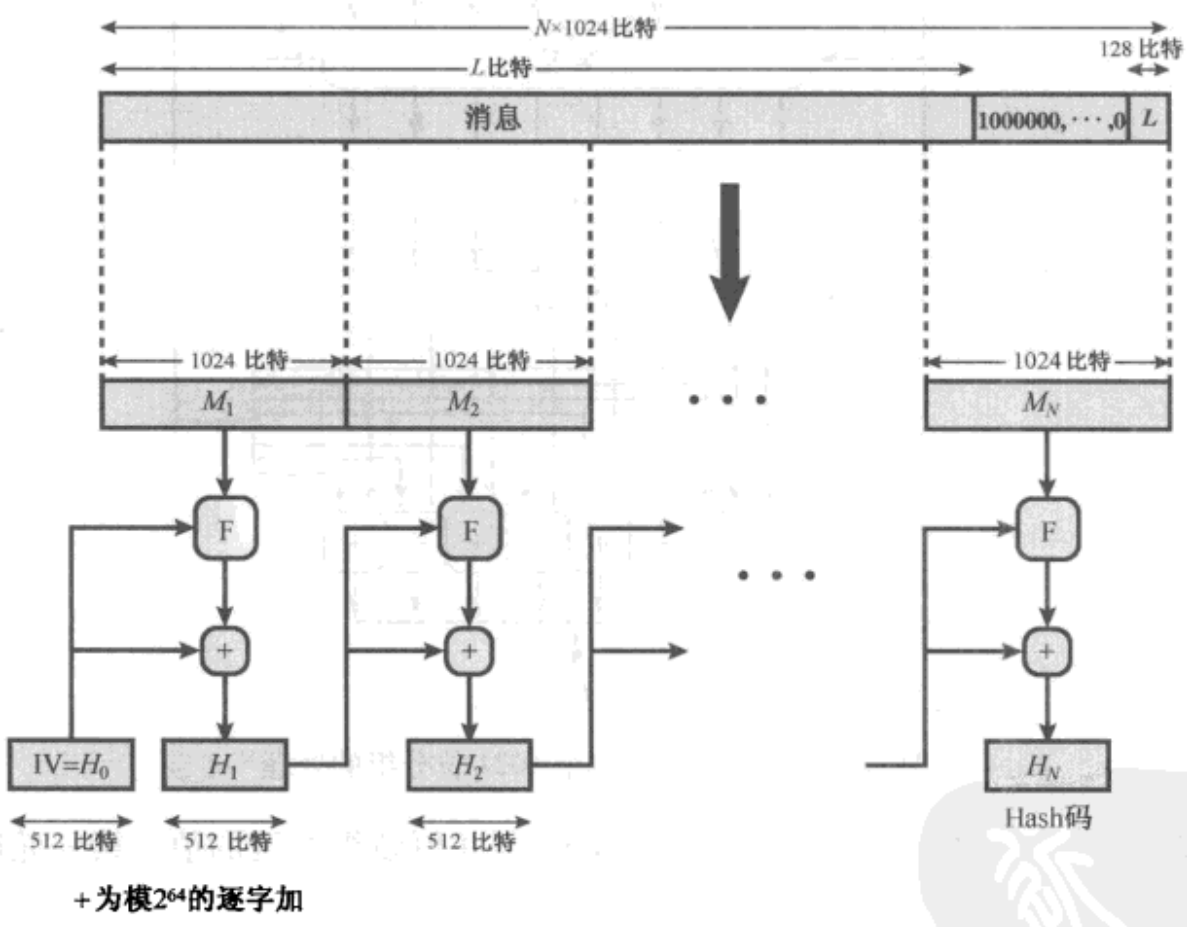


图 11.8　SHA-512 生成消息摘要

步骤 2：附加长度。在消息后附加一个 128 位的块，将其视为 128 位的无符号整数（最高有效字节在前），它包含填充前消息的长度。

前两步的结果是产生了一个长度为 1024 整数倍的消息。图 11.8 中，扩展的消息被表示为一串长度为 1024 位的消息分组 $M_1, M_2, \cdots, M_N$，因此扩展消息的长度为 $N \times 1024$ 位。

步骤 3：初始化 Hash 缓冲区。Hash 函数的中间结果和最终结果保存于 512 位的缓冲区中，缓冲区用 8 个 64 位的寄存器（a、b、c、d、e、f、g、h）表示，并将这些寄存器初始化为下列 64 位的整数（十六进制值）：

```
a = 6A09E667F3BCC908   e = 510E527FADE682D1
b = BB67AE8584CAA73B   f = 9B05688C2B3E6C1F
c = 3C6EF372FE94F82B   g = 1F83D9ABFB41BD6B
d = A54FF53A5F1D36F1   h = 5BE0CD19137E2179
```

这些值以高位在前格式存储,也就是说,字的最高有效字节存于低地址字节位置(最左边)。这些字的获取方式如下:前 8 个素数取平方根,取小数部分的前 64 位。

步骤 4:以 1024 位的分组(128 个字节)为单位处理消息。算法的核心是具有 80 轮运算的模块,图 11.8 中,该模块标记为 F。图 11.9 给出了它的逻辑原理。

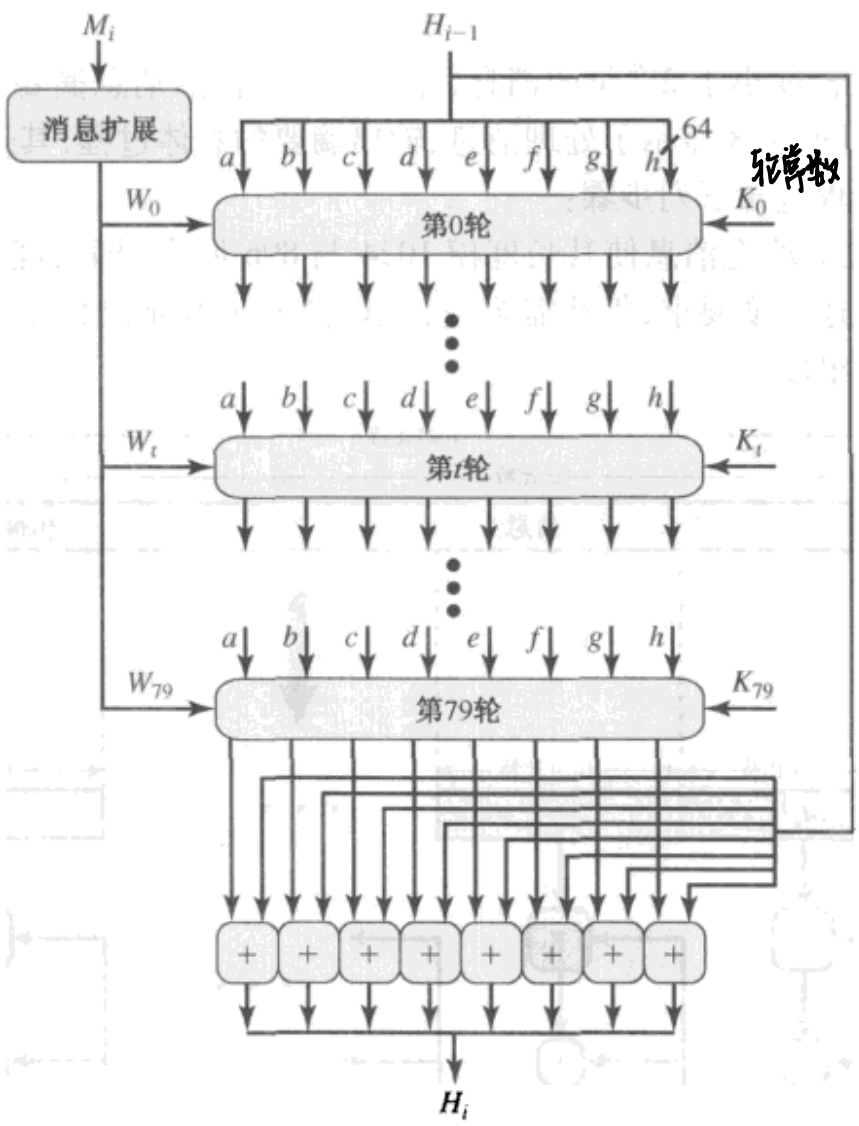


图 11.9 SHA-512 对单个 1024 位分组的处理

每一轮都把 512 位缓冲区的值 $abcdefgh$ 作为输入,并更新缓冲区的值。第一轮时,缓冲区里的值是中间值 H_{i-1}。每一轮,如第 t 轮,使用一个 64 位的值 W_t,该值由当前被处理的 1024 位消息分组 M_i 导出,导出算法是下面将要讨论的消息扩展算法。每一轮还将使用附加的常数 K_t,其中 $0 \leqslant t \leqslant 79$,用来使每轮的运算不同。这些常数的获得方法如下:对前 80 个素数开立方根,取小数部分的前 64 位。这些常数提供了 64 位随机串集合,可以初步消除输入数据中的统计规律。表 11.4 给出了轮常数的十六进制值(从左到右)。

第 80 轮的输出和第一轮的输入 H_{i-1} 相加产生 H_i。缓冲区中的 8 个字和 H_{i-1} 中对应的字分别进行模 2^{64} 的加法运算。

步骤 5:输出。所有的 N 个 1024 位分组都处理完以后,从第 N 阶段输出的是 512 位的消息摘要。

表 11.4　SHA-512 轮常数

428a2f98d728ae22	7137449123ef65cd	b5c0fbcfec4d3b2f	e9b5dba58189dbbc
3956c25bf348b538	59f111f1b605d019	923f82a4af194f9b	ab1c5ed5da6d8118
d807aa98a3030242	12835b0145706fbe	243185be4ee4b28c	550c7dc3d5ffb4e2
72be5d74f27b896f	80deb1fe3b1696b1	9bdc06a725c71235	c19bf174cf692694
e49b69c19ef14ad2	efbe4786384f25e3	0fc19dc68b8cd5b5	240ca1cc77ac9c65
2de92c6f592b0275	4a7484aa6ea6e483	5cb0a9dcbd41fbd4	76f988da831153b5
983e5152ee66dfab	a831c66d2db43210	b00327c898fb213f	bf597fc7beef0ee4
c6e00bf33da88fc2	d5a79147930aa725	06ca6351e003826f	142929670a0e6e70
27b70a8546d22ffc	2e1b21385c26c926	4d2c6dfc5ac42aed	53380d139d95b3df
650a73548baf63de	766a0abb3c77b2a8	81c2c92e47edaee6	92722c851482353b
a2bfe8a14cf10364	a81a664bbc423001	c24b8b70d0f89791	c76c51a30654be30
d192e819d6ef5218	d69906245565a910	f40e35855771202a	106aa07032bbd1b8
19a4c116b8d2d0c8	1e376c085141ab53	2748774cdf8eeb99	34b0bcb5e19b48a8
391c0cb3c5c95a63	4ed8aa4ae3418acb	5b9cca4f7763e373	682e6ff3d6b2b8a3
748f82ee5defb2fc	78a5636f43172f60	84c87814a1f0ab72	8cc702081a6439ec
90befffa23631e28	a4506cebde82bde9	bef9a3f7b2c67915	c67178f2e372532b
ca273eceea26619c	d186b8c721c0c207	eada7dd6cde0eb1e	f57d4f7fee6ed178
06f067aa72176fba	0a637dc5a2c898a6	113f9804bef90dae	1b710b35131c471b
28db77f523047d84	32caab7b40c72493	3c9ebe0a15c9bebc	431d67c49c100d4c
4cc5d4becb3e42b6	597f299cfc657e2a	5fcb6fab3ad6faec	6c44198c4a475817

总结 SHA-512 的运算如下：

$$H_0 = \text{IV}$$

$$H_i = \text{SUM}_{64}(H_{i-1}, abcdefgh_i)$$

$$\text{MD} = H_N$$

其中，IV 是第三步中定义的 *abcdefgh* 缓冲区的初始值；$abcdefgh_i$ 是第 i 个消息分组处理的最后一轮的输出；N 为消息（包括填充和长度域）中的分组数；SUM_{64} 表示对输入对中的每个字进行独立的模 2^{64} 加；MD 表示最后的消息摘要值。

11.5.2　SHA-512 轮函数

下面我们仔细讨论 80 轮的每一轮的内部逻辑，每一轮处理 512 位的单分组（参见图 11.10）。每一轮由如下的方程定义：

$$T_1 = h + \text{Ch}(e, f, g) + \left(\sum_1^{512} e\right) + W_t + K_t$$

$$T_2 = \left(\sum_0^{512} a\right) + \text{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

$$e = d + T_1$$

$$d = c$$

$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$

其中:

t 为步骤数,$0 \leqslant t \leqslant 79$。

$\mathrm{Ch}(e,f,g) = (e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$ 条件函数:如果 e,则 f,否则 g。

$\mathrm{Maj}(a,b,c) = (a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$ 函数为真当且仅当变量的多数(2 个或 3 个)为真。

$$\left(\sum\nolimits_0^{512} a\right) = \mathrm{ROTR}^{28}(a) \oplus \mathrm{ROTR}^{34}(a) \oplus \mathrm{ROTR}^{39}(a)$$

$$\left(\sum\nolimits_1^{512} e\right) = \mathrm{ROTR}^{14}(e) \oplus \mathrm{ROTR}^{18}(e) \oplus \mathrm{ROTR}^{41}(e)$$

$\mathrm{ROTR}^n(x)$ = 对 64 位的变量 x 循环右移 n 位

W_t 为一个 64 位字,从当前的 512 位输入分组导出

K_t 为一个 64 位附加常数

+ 为模 2^{64} 加

轮函数有如下两个特点:

(1) 轮函数输出的 8 个字中的 6 个(b、c、d、f、g、h)是通过简单的轮转置换实现的。如图 11.10 中的阴影部分。(下面)

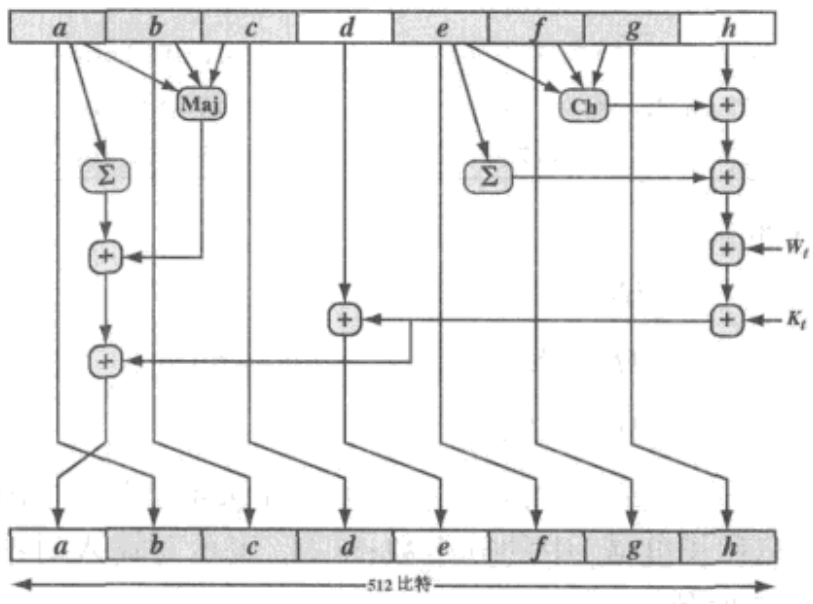


图 11.10 基本的 SHA-512 运算(单轮)

(2) 输出中只有 2 个字(a,e)通过替代操作产生。字 e 是将输入变量(d、e、f、g、h)以及轮消息 W_t 和轮常数 K_t 作为输入的函数。字 a 是将除 d 之外的输入变量以及轮消息 W_t 和轮常数 K_t 作为输入的函数。

还需要指出的是 64 位的字 W_t 是如何从 1024 位消息里导出的。图 11.11 说明了该映射的结构。前 16 个 W_t 直接取自当前分组的 16 个字。余下的值按如下方式导出:

$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

其中:

$$\sigma_0^{512}(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x)$$

$$\sigma_1^{512}(x) = \mathrm{ROTR}^{19}(x) \oplus \mathrm{ROTR}^{61}(x) \oplus \mathrm{SHR}^6(x)$$

$\mathrm{ROTR}^n(x)$ 为对 64 位的变量 x 循环右移 n 位

$\mathrm{SHR}^n(x)$ 为对 64 位变量 x 向左移动 n 位,右边填充 0

+ 为模 2^{64} 加

因此,前 16 步处理过程中,W_t 的值等于消息分组中的相应字。对于余下的 64 步,W_t 的值由其

前面的 4 个值的异或①形成的值构成，4 个值中的两个要进行移位和循环移位操作。这增加了被压缩消息分组的冗余性和相互依赖性，所以对给定的消息，找出具有相同压缩结果的消息会非常复杂。

图 11.2 总结了 SHA-512 的整个结构。

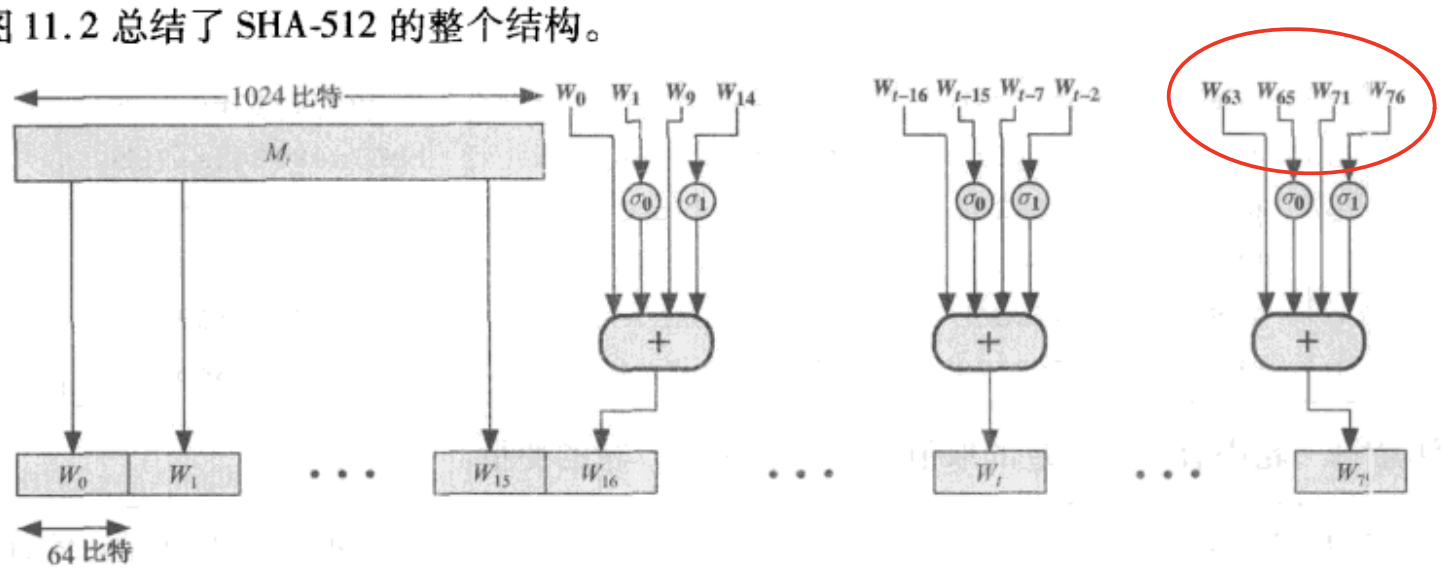


图 11.11　为 SHA-512 处理单个分组而产生的 80 字输入序列

SHA-512 算法具有如下特性：Hash 码的每一个位都是全部输入位的函数。基本函数 F 多次复杂重复运算使得结果充分混淆，从而使得随机选择两个消息，甚至于这两个消息有相似特征，都不太可能产生相同的 Hash 码。除非 SHA-512 中存在目前未公开的隐藏缺陷，找到两个具有相同摘要的消息的复杂度需要 2^{256} 次操作，而给定摘要寻找消息的复杂度需要 2^{512} 次操作。

11.5.3　示例

我们这里给出 FIPS 180 中提供的一个示例。假设对由三个 ASCII 字符“abc”构成的消息块进行 Hash 处理，该消息块可写成如下 24 位的串：

```
01100001 01100010 01100011
```

从 SHA 算法的第一步开始，首先消息被填充后使其长度模 1024 与 896 同余。对于上面的消息块，填充长度为 896 − 24 = 872 位，填充内容包括 1 个“1”和后面紧跟的 871 个“0”。然后 128 位的长度信息附在填充消息的后面，保存的是消息的（填充前的）原始长度。本例中原始长度是 24 位，即十六进制值 18。将上述内容连接在一起构成的完整 1024 位消息分块的十六进制表示是

```
6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

在消息扩展过程中，该数据块的各字被分别赋值给 $W_0, \cdots, W_{15}$，如下所示：

$W_0 = 6162638000000000$　　$W_8 = 0000000000000000$
$W_1 = 0000000000000000$　　$W_9 = 0000000000000000$
$W_2 = 0000000000000000$　　$W_{10} = 0000000000000000$
$W_3 = 0000000000000000$　　$W_{11} = 0000000000000000$
$W_4 = 0000000000000000$　　$W_{12} = 0000000000000000$
$W_5 = 0000000000000000$　　$W_{13} = 0000000000000000$
$W_6 = 0000000000000000$　　$W_{14} = 0000000000000000$
$W_7 = 0000000000000000$　　$W_{15} = 0000000000000018$

① 原文有“再循环左移一位”，但和公式不对应——译者注。

如图11.12所示,从a到h的8个64位变量,首先被赋值为从$H_{0,0}$到$H_{0,7}$。下表给出了这些变量的初始值和前两轮的值。

a	6a09e667f3bcc908	f6afceb8bcfcddf5	1320f8c9fb872cc0
b	bb67ae8584caa73b	6a09e667f3bcc908	f6afceb8bcfcddf5
c	3c6ef372fe94f82b	bb67ae8584caa73b	6a09e667f3bcc908
d	a54ff53a5f1d36f1	3c6ef372fe94f82b	bb67ae8584caa73b
e	510e527fade682d1	58cb02347ab51f91	c3d4ebfd48650ffa
f	9b05688c2b3e6c1f	510e527fade682d1	58cb02347ab51f91
g	1f83d9abfb41bd6b	9b05688c2b3e6c1f	510e527fade682d1
h	5be0cd19137e2179	1f83d9abfb41bd6b	9b05688c2b3e6c1f

注意每一轮中有6个变量的取值是直接复制上一轮的变量。

扩展后的消息包括分组$M_1, M_2, \cdots, M_N$,每个消息分组M_i由16个64位字$M_{i,0}, M_{i,1}, \cdots, M_{i,15}$构成。所有的加法运算都是指模$2^{64}$加法。

$H_{0,0} = 6a09e667f3bcc908$　　$H_{0,4} = 510e527fade682d1$

$H_{0,1} = bb67ae8584caa73b$　　$H_{0,5} = 9b05688c2b3e6c1f$

$H_{0,2} = 3c6ef372fe94f82b$　　$H_{0,6} = 1f83d9abfb41bd6b$

$H_{0,3} = a54ff53a5f1d36f1$　　$H_{0,7} = 5be0cd19137e2179$

for $i = 1$ **to** N

1. 预处理消息扩展

　for $t = 0$ **to** 15

　　$W_t = M_{i,t}$

　for $t = 16$ **to** 79

　　$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$

2. 初始化工作变量

$a = H_{i-1,0}$　　$e = H_{i-1,4}$

$b = H_{i-1,1}$　　$f = H_{i-1,5}$

$c = H_{i-1,2}$　　$g = H_{i-1,6}$

$d = H_{i-1,3}$　　$h = H_{i-1,7}$

3. 执行主Hash计算

　for $t = 0$ **to** 79

$T_1 = h + \mathrm{Ch}(e,f,g) + (\sum_1^{512} e) + W_t + K_t$

$T_2 = (\sum_0^{512} a) + \mathrm{Maj}(a,b,c)$

$h = g$

$g = f$

$f = e$

$e = d + T_1$

$d = c$

$c = b$

$b = a$

$a = T_1 + T_2$

4. 计算Hash中间值

$H_{i,0} = a + H_{i-1,0}$　　$H_{i,4} = e + H_{i-1,4}$

$H_{i,1} = b + H_{i-1,1}$　　$H_{i,5} = f + H_{i-1,5}$

$H_{i,2} = c + H_{i-1,2}$　　$H_{i,6} = g + H_{i-1,6}$

$H_{i,3} = d + H_{i-1,3}$　　$H_{i,7} = h + H_{i-1,7}$

返回 $\{ H_{N,0} \| H_{N,1} \| H_{N,2} \| H_{N,3} \| H_{N,4} \| H_{N,5} \| H_{N,6} \| H_{N,7} \}$

图11.12　SHA-512逻辑

该过程重复 80 轮,最后一轮的输出是

```
73a54f399fa4b1b2 10d9c4c4295599f6 d67806db8b148677 654ef9abec389ca9
d08446aa79693ed7 9bb4d39778c07f9e 25c96a7768fb2aa3 ceb9fc3691ce8326
```

其中的 Hash 值通过如下计算产生:

$$H_{1,0} = \texttt{6a09e667f3bcc908} + \texttt{73a54f399fa4b1b2} = \texttt{ddaf35a193617aba}$$

$$H_{1,1} = \texttt{bb67ae8584caa73b} + \texttt{10d9c4c4295599f6} = \texttt{cc417349ae204131}$$

$$H_{1,2} = \texttt{3c6ef372fe94f82b} + \texttt{d67806db8b148677} = \texttt{12e6fa4e89a97ea2}$$

$$H_{1,3} = \texttt{a54ff53a5f1d36f1} + \texttt{654ef9abec389ca9} = \texttt{0a9eeee64b55d39a}$$

$$H_{1,4} = \texttt{510e527fade682d1} + \texttt{d08446aa79693ed7} = \texttt{2192992a274fc1a8}$$

$$H_{1,5} = \texttt{9b05688c2b3e6c1f} + \texttt{9bb4d39778c07f9e} = \texttt{36ba3c23a3feebbd}$$

$$H_{1,6} = \texttt{1f83d9abfb41bd6b} + \texttt{25c96a7768fb2aa3} = \texttt{454d4423643ce80e}$$

$$H_{1,7} = \texttt{5be0cd19137e2179} + \texttt{ceb9fc3691ce8326} = \texttt{2a9ac94fa54ca49f}$$

最终得到的 512 位消息摘要是

```
ddaf35a193617aba cc417349ae204131 12e6fa4e89a97ea2 0a9eeee64b55d39a
2192992a274fc1a8 36ba3c23a3feebbd 454d4423643ce80e 2a9ac94fa54ca49f
```

假设我们将输入消息改变 1 个位,从“abc”变成“cbc”,那么 1024 位消息块为

```
6362638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

对应的 512 位消息摘要是

```
531668966ee79b70 0b8e593261101354 4273f7ef7b31f279 2a7ef68d53f93264
319c165ad96d9187 55e6a204c2607e27 6e05cdf993a64c85 ef9e1e125c0f925f
```

前后两个消息的 Hash 值有 253 位不同,几乎一半发生了改变,这说明 SHA-512 有良好的雪崩效应。

11.6　SHA-3

在本书编写过程中,SHA-1 暂时还未被“攻破”,即还没有出现比穷举更好的碰撞攻击方法。但是 SHA-1 在设计上与 MD5 和 SHA-0 具有相似的结构和基本数学运算,而后两者已经被攻破。因此 SHA-1 被认为是不安全的,应逐步被 SHA-2 取代。

对于 SHA-2,特别是 512 版本的安全强度是毋庸置疑的。但是 SHA-2 同样和它的前任一样在设计上有类似的结构和基本数学运算,这值得我们关注。或许在若干年后 SHA-2 的缺陷也会被发现而被取代,因此 NIST 决定开展新的 Hash 标准的制定工作。

NIST 在 2007 年宣布公开征集新一代 NIST 的 Hash 函数标准,称其为 SHA-3。NIST 计划在 2012 年末推出这一新的标准,但同时强调该时间表并不固定,日程可能会被调整。任何设计者提交的 SHA-3 候选算法需要满足如下基本要求:

(1) SHA-3 算法对于任意应用都能够直接替代 SHA-2 算法。这要求 SHA-3 必须也支持产生 224、256、384、512 位的 Hash 值。

(2) SHA-3 必须保持 SHA-2 的在线处理能力。这要求 SHA-3 必须能处理小的数据块(如 512 或 1024 位),而不需要在处理之前将整个大块消息缓存到存储器中。

除了这些基本需求,NIST 还制定了一系列评估准则。这些准则也是根据 SHA-2 所支持的主要应用需求而设计的,包括数字签名、消息认证码、随机数发生器等。对于新 Hash 函数的评估准则,重要性由高到低列举如下。

- **安全性**:对于抵抗原像和碰撞攻击的能力,SHA-3 的安全强度必须接近与不同的 Hash 长度对应的最大理论强度。SHA-3 算法的设计必须能够抵抗已有或潜在的对于 SHA-2 函数的攻击方法。实际上这意味着 SHA-3 的设计或者在结构上,或者在数学函数上,或者两者兼有,要与 SHA-1、SHA-2 和 MD5 算法有本质的不同。
- **效率**:SHA-3 在各种硬件实现平台上的时间和存储效率都要高。
- **算法和实现特性**:在设计过程中要考虑到灵活性(例如设置可选参数来提供安全性/效率折中选择,便于并行计算等)和简单性等因素。后者可以使分析算法的安全性更加直观。

11.7 推荐读物和网站

[PREN99]对密码学 Hash 函数做了较全面的综述。[GILB03]讨论了从 SHA-256 到 SHA-512 的安全性。

GILB03 Gilbert, H. and Handschuh, H. "Security Analysis of SHA-256 and Sisters." *Proceedings, CRYPTO'03*, 2003; *published by Springer-Verlag.*

PREN99 Preneel, B. "The State of Cryptographic Hash Functions." *Proceedings, EUROCRYPT'96* 1996; published by Springer-Verlag.

推荐网站

- NIST Secure Hashing Pages:SHA FIPS 和有关文档。
- Cryptographic Hash Algorithm Competition:NIST 征集新一代 Hash 函数标准即 SHA-3 的官方网站。

11.8 关键术语、思考题和习题

关键术语

高位在前格式	抗碰撞攻击	密码学 Hash 函数
生日攻击	MD4	SHA-224
Hash 码	MD5	SHA-256
Hash 函数	消息摘要	SHA-384
带密钥 Hash 函数	单向 Hash 函数	SHA-512
低位在前格式	抗原像攻击	抗强碰撞攻击
消息认证码(MAC)	抗第二原像攻击	抗弱碰撞攻击
生日悖论	压缩函数	

思考题

11.1　安全 Hash 函数需要具有哪些特性?

11.2　抗弱碰撞和抗强碰撞之间的区别是什么?

11.3　Hash 函数中的压缩函数的作用是什么?

11.4　高位在前格式和低位在前格式的区别是什么?

11.5　SHA 中使用的基本算术和逻辑函数是什么?

习题

11.1　高速传输协议 XTP(Xpress Transfer Protocol)使用 32 位校验和函数,它是两个 16 位函数的连接,两个函数为 XOR 和 RXOR,即 11.4 节图 11.4 中所示的"两种简单的 Hash 函数"。

(a) 该校验和能否检测出由奇数位错所引起的所有错误? 请说明原因。

(b) 该校验和能否检测出由偶数位错所引起的所有错误? 若不能,请说明该校验和所不能检测出的错误类型的特征。

(c) 若将该函数作为消息认证中的 Hash 函数,试分析其有效性。

11.2　(a) 考虑 11.4 节中给出的 Davies 和 Price 提出的 Hash 码方案,假定使用 DES 作为加密函数:

$$H_i = H_{i-1} \oplus \mathrm{E}(M_i, H_{i-1})$$

前面讲过 DES 的互补性(参见习题 3.14):若 $Y = \mathrm{E}(K, X)$,则 $Y' = \mathrm{E}(K', X')$。利用该性质说明如何修改分组为 $M_1, M_2, \cdots, M_N$ 的消息而不改变其 Hash 码。

(b) 证明存在有类似的攻击可成功地攻击[MEYE88]中提出的方法:

$$H_i = M_i \oplus \mathrm{E}(H_{i-1}, M_i)$$

11.3　(a) 考虑下列 Hash 函数。消息是 Z_n 中的整数序列,$M = (a_1, a_2, \cdots, a_t)$。对于某个预定义好的 n,计算 Hash 值 $h = \left(\sum_{i=1}^{t} a_i\right)$。这个 Hash 函数满足表 11.1 所列出的 Hash 函数的要求吗? 给出解释。

(b) 对于 Hash 函数 $h = \left(\sum_{i=1}^{t} (a_i)^2\right) \bmod n$,按(a)的要求做题。

(c) 当 $M = (189, 632, 900, 722, 349)$,$n = 989$ 时,计算(b)中的 Hash 函数值。

11.4　可以利用 Hash 函数构造类似 DES 结构的分组密码。但 Hash 是单向的,而分组密码是可逆的(为了能够解密),那么如何用 Hash 码构造分组密码呢?

11.5　考虑相反的问题:利用加密算法构造单向 Hash 函数。考虑使用有一个已知密钥的 RSA 算法。如下处理含有若干分组的消息:加密第一分组,将加密结果与第二分组异或并加密之,等等。通过解决下面的问题,说明该方法是不安全的。给定两个分组消息 B_1 和 B_2,其 Hash 码为

$$\mathrm{RSAH}(B_1, B_2) = \mathrm{RSA}(\mathrm{RSA}(B_1) \oplus B_2)$$

给定任一分组 C_1,选择 C_2 使得 $\mathrm{RSAH}(C_1, C_2) = \mathrm{RSAH}(B_1, B_2)$。因此该 Hash 函数不满足抗弱碰撞性。

11.6　设 $H(m)$ 是一个抗碰撞 Hash 函数,将任意长消息映射为定长的 n 位 Hash 值。对于所有的消息 x 和 x',$x \neq x'$,都有 $H(x) \neq H(x')$。上述说法是否正确? 给出解释。

11.7　图 11.11 中,假设有一个大小为 80 个字的数组(字长为 64 位)来存储 W_t 的值,所以在开始处理一个分组时可以预计算它们的值。现在假设空间是短缺资源。作为替代方案,考虑使用 16 个字大小的循环缓冲区,用值 W_0 到 W_{15} 进行初始化。设计一个算法,对于每一步 t,计算需要的输入 W_t。

11.8　给出 SHA-512 中 W_{16}、W_{17}、W_{18}、W_{19} 的值。

11.9　对于 SHA-512,如果消息分别为如下长度,给出填充区域的值。

(a) 1919 位

(b) 1920 位

(c) 1921 位

11.10 对于 SHA-512,如果消息分别为如下长度,给出长度区域的值。

(a) 1919 位

(b) 1920 位

(c) 1921 位

11.11 假定 $a_1a_2a_3a_4$ 是长为 32 位的字中的 4 个字节,将每个 a_i 视为 0 ~ 255 之间的二进制整数,在高位在前格式中,该字表示整数

$$a_1 2^{24} + a_2 2^{16} + a_3 2^8 + a_4$$

在低位在前格式中,该字表示整数

$$a_4 2^{24} + a_3 2^{16} + a_2 2^8 + a_1$$

(a) 一些 Hash 函数如 MD5 使用的是低位在前格式。注意,消息摘要不依赖于低层结构这一点很重要,因此要在高位在前格式平台上执行 MD5 和 RIPEMD-160 的模 2 加法运算,必须进行调整。假定 $X = x_1x_2x_3x_4$ 且 $Y = y_1y_2y_3y_4$,说明 MD5 的加法运算 $(X + Y)$ 如何在高位在前格式的机器上执行。

(b) SHA 使用高位在前格式,说明 SHA 的运算 $(X + Y)$ 如何在低位在前格式的机器上执行。

11.12 本习题介绍了一个本质上和 SHA 类似的 Hash 函数,它对字母操作,而不是二元数据,称为趣味四字母 Hash(tth)①。给定一个由字母序列构成的消息,tth 产生一个四字母的 Hash 值。首先,tth 将消息划分为 16 个字母的分组,忽略空格、标点符号和大写。如果消息不能被 16 整除,用空白来填充。维持一个四个数的运行和,其初始值为(0,0,0,0)。将其输入压缩函数用于处理第一个分组。压缩函数由两轮组成。

第一轮:取下一文本分组,按行序排列成 4 × 4 的文本分组,并将文本转换为数字(A = 0,B = 1 等)。例如,对于分组 ABCDEFGHIJKLMNOP,我们有

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

接着,将各列分别模 26 加,并将结果和运行和模 26 加。本例中,运行和为(24,2,6,10)。

第二轮:运用第一轮的矩阵,将第一行循环左移 1 位,第二轮 2 位,第三轮 3 位,而第四轮则是将元素反序。本例中,有

B	C	D	A
G	H	E	F
L	I	J	K
P	O	N	M

1	2	3	0
6	7	4	5
11	8	9	10
15	14	13	12

现在,将各列分别模 26 相加,并将结果和运行和相加。新的运行和为(5,7,9,11)。该运行和作为压缩函数的输入,处理下一文本分组。最后一个分组处理完后,将其转换为字母。例如,如果消息是 ABCDEFGHIJKLMNOP,则 Hash 值为 FHJL。

(a) 画图表示 tth 的总体逻辑和压缩函数的逻辑,该图和图 11.8 及图 11.9 相仿。

(b) 计算 48 个字母的消息"I leave twenty million dollars to my friendly cousin Bill."的 Hash 值。

(c) 为了说明 tth 的弱点,请找出 48 个字母的分组也产生相同的 Hash 值。提示:使用多个 A。

下面的习题是关于 Hash 函数 Whirlpool 算法的,在附录 N 中介绍 Whirlpool 算法。

11.13 类似于表 4.9,给出域 $GF(2^8)$ 的表,其中 $m(x) = x^8 + x^4 + x^3 + x^2 + 1$。

11.14 将表 N.2 中的 E 和 E^{-1} 小盒表示为传统方阵形式的 S 盒,譬如表 5.4 那样。

---

① 感谢 *The Cryptogram* 杂志的 William K. Mason 提供了这个例子。

11.15　验证图 N.5 是表 N.2(a)所示的 S 盒的正确实现。通过给出三个输入值 00,55,1E 所涉及的计算过程来验证。

11.16　给出图 N.5 所示 S 盒的布尔表达式。

11.17　Whirlpool 使用了如下的构造结构：$H_i = E(H_{i-1}, M_i) \oplus H_{i-1} \oplus M_i$。Preneel 证明了另一种安全的 Hash 结构，其构造结构为 $H_i = E(H_{i-1}, M_i) \oplus M_i$。现在请注意 Whirlpool 的密钥扩展方法和用轮常数定义的伪密钥对密文密钥的加密相仿，所以 Hash 处理过程的核心部分可以形式化地视为两个交互的加密过程。考虑加密算法 $E(H_{i-1}, M_i)$，该分组的最后一个轮密钥可以写为 $K_{10} = E(\mathrm{RC}, H_{i-1})$。请证明由于密钥扩展的定义方法，上述的两个 Hash 构造结构本质上等价。

附录 11A　生日攻击的数学基础

在本附录中，我们给出生日攻击的数学证明。首先介绍一个与 Hash 函数相关的问题，然后讨论生日攻击问题，并说明生日攻击这个名称的来历。

11A.1　相关问题

下面给出与 Hash 函数有关的一个常见问题。给定 Hash 函数 H 和某 Hash 值 $H(x)$，假定 H 有 n 种可能的输出，如果 H 作用于 k 个随机的输入，那么至少有一个 y 使得 $H(y) = H(x)$ 的概率为 0.5 的 k 的值是多少？

对某 y 值，$H(y) = H(x)$ 的概率恰为 $1/n$，反过来，$H(x) \neq H(y)$ 的概率为 $[1 - (1/n)]$。如果产生 k 个随机的 y 值，那么它们均不能与 x 匹配的概率等于每个值不与 x 匹配的概率之积，即 $[1 - (1/n)]^k$，因此至少有一个匹配的概率是 $1 - [1 - (1/n)]^k$。

二项式定理可描述如下：

$$(1 - a)^k = 1 - ka + \frac{k(k-1)}{2!}a^2 - \frac{k(k-1)(k-2)}{3!}a^3 \cdots$$

a 很小时上式约为 $(1 - ka)$。因此至少有一个匹配的概率约为 $1 - [1 - (1/n)]^k \approx 1 - [1 - (k/n)] = k/n$。要使概率为 0.5，则 $k = n/2$。

特别地，若 Hash 码为 m 位，则可能有 2^m 个 Hash 码，使上述概率为 1/2 的 k 为

$$k = 2^{(m-1)} \tag{11.1}$$

11A.2　生日悖论

在初等概率课程中常用生日悖论来说明一些违背直觉的结果。我们可以如下描述这类问题：使 k 个人中至少有两个人生日相同的概率大于 0.5 的最小 k 值是多少？我们不考虑二月闰月 29 日并且假定每个生日出现的概率相同。

我们可以通过如下推导得出答案：任意两个人生日不同的概率是 364/365(因为对于任何一个人来说，在 365 天中选择其生日，只有 1 天使得此人的生日恰好与另一个人的生日相同)。第三个人与前两个人生日不同的概率是 363/365；第四个人是 362/365；以此类推，第 24 个人是 342/365。我们把得到的 23 个因数进行连乘，按照假设乘积应超过全部 24 个人的生日都不相同的概率。实际上乘积约等于 0.507，比 1/2 稍大。即 23 个人中有两人生日相同的概率大于 0.5。

为了从理论上推导上述问题，可定义：

$P(n,k) = \Pr[k$ 个元素中至少有一个元素重复出现，其中每个元素出现的概率均为 $1/n]$

所以，我们就是要找使得 $P(365,k) \geq 0.5$ 的最小的 k。用 $Q(365,k)$ 表示没有重复的概率，若 $k > 365$，则所有值都不等是不可能的。所以假定 $k \leq 365$。设 k 个元素均不重复的次数为 N，那

么,第一个元素有365种取值,第二个元素有364种取值,等等。因此,不同的方式数为

$$N = 365 \times 364 \times \cdots (365 - k + 1) = \frac{365!}{(365 - k)!} \tag{11.2}$$

如果允许重复,那么每一元素有365种取法,共有365^k种取法,所以不重复的概率为无重复数除以总数:

$$Q(365, k) = \frac{365!/(365 - k)!}{(365)^k} = \frac{365!}{(365 - k)!(365)^k}$$

则

$$P(365, k) = 1 - Q(365, k) = 1 - \frac{365!}{(365 - k)!(365)^k} \tag{11.3}$$

该函数如图11.13所示,对那些以前没有考虑过该问题的人来说,该概率大得有些出奇。许多人以为要使至少有一个重复的概率大于0.5,那么大约应有100人。事实上,因为$P(365,23) = 0.5037$,所以人数为23。$k = 100$时至少有一个重复的概率为0.9999997。

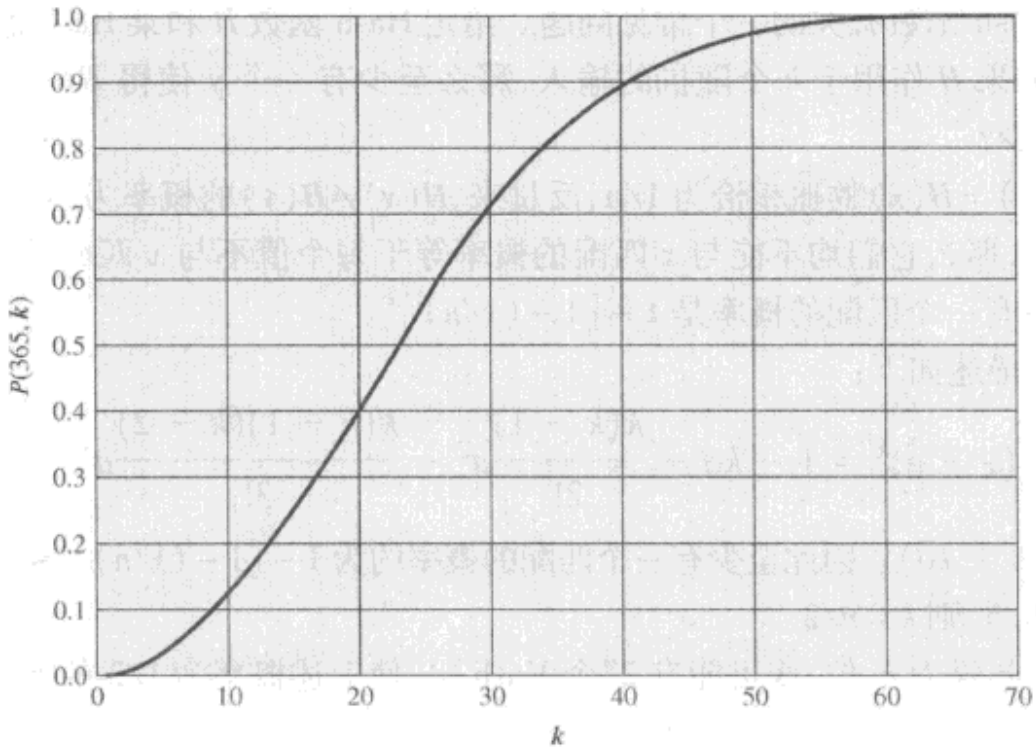


图11.13 生日悖论

结果显得出乎意料的原因也许如下:如果你考虑组里某个特定的人,那么组里有其他人和这个人有相同生日的概率是很小的,但是这里我们考虑的是任意两个人生日相同的概率。在23个人中,有$23(23-1)/2 = 253$种不同的双人组合,因此生日相同的概率较大。

11A.3 常用不等式

在讨论生日悖论的一般问题之前,我们先推导一个有用的不等式:

$$(1 - x) \leqslant e^{-x}, \qquad x \geqslant 0 \tag{11.4}$$

图11.14给出了该不等式的图像。为了说明不等式成立,注意到下面的直线在$x = 0$处与e^{-x}相切,其斜率为e^{-x}在$x = 0$处的导数:

$$f(x) = e^{-x}$$
$$f'(x) = \frac{d}{dx} e^{-x} = -e^{-x}$$
$$f'(0) = -1$$

这条切线是形为 $ax+b$ 的直线，其中 $a=-1$，它在 $x=0$ 处的值等于 $e^{-0}=1$，因此这条切线即是函数 $(1-x)$，这说明不等式(11.4)成立，并且对较小的 x，有 $(1-x)\approx e^{-x}$。

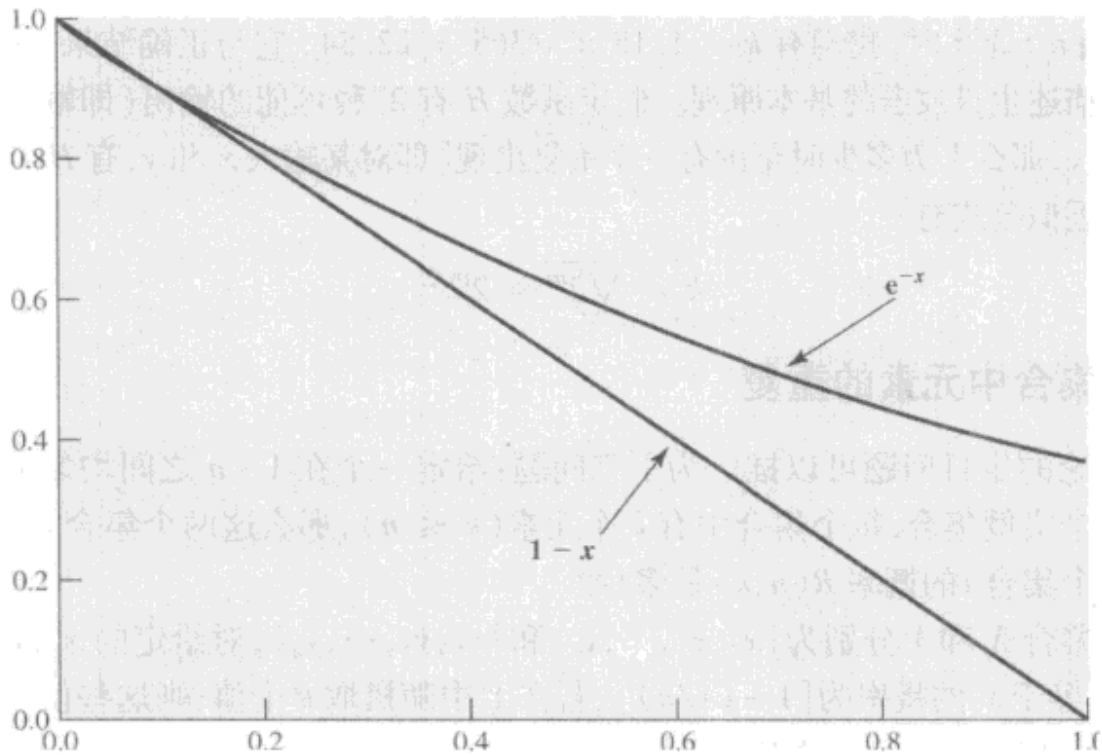


图 11.14　常用不等式

11A.4　元素重复的一般情形

生日悖论可推广为下述情形：给定一个在 $1\sim n$ 之间均匀分布的随机整数变量以及它的 k 个实例($k\leqslant n$)，那么至少有一个重复的概率 $P(n,k)$ 是多少？生日悖论是其在 $n=365$ 时的特例。和前面的推导过程一样，我们可推广等式(11.3)为

$$P(n,k)=1-\frac{n!}{(n-k)!n^k} \tag{11.5}$$

即

$$\begin{aligned}P(n,k)&=1-\frac{n\times(n-1)\times\cdots\times(n-k+1)}{n^k}\\&=1-\left[\frac{n-1}{n}\times\frac{n-2}{n}\times\cdots\times\frac{n-k+1}{n}\right]\\&=1-\left[\left(1-\frac{1}{n}\right)\times\left(1-\frac{2}{n}\right)\times\cdots\times\left(1-\frac{k-1}{n}\right)\right]\end{aligned}$$

根据不等式(11.4)有

$$\begin{aligned}P(n,k)&>1-[(e^{-1/n})\times(e^{-2/n})\times\cdots\times(e^{-(k-1)/n})]\\&>1-e^{-[(1/n)+(2/n)+\cdots+\{(k-1)/n\}]}\\&>1-e^{-(k\times(k-1))/2n}\end{aligned}$$

下面我们来看一下 k 为多少时，$P(n,k)>0.5$。要使 $P(n,k)>0.5$，则

$$\begin{aligned}1/2&=1-e^{-(k\times(k-1))/2n}\\2&=e^{(k\times(k-1))/2n}\\\ln 2&=\frac{k\times(k-1)}{2n}\end{aligned}$$

k 较大时我们可用 k^2 取代 $k \times (k-1)$,则有

$$k = \sqrt{2(\ln 2)n} = 1.18\sqrt{n} \approx \sqrt{n} \tag{11.6}$$

作为验证,当 $n=365$ 时,我们有 $k = 1.18 \times \sqrt{365} = 22.54$,它与正确结果 23 非常接近。

下面我们来描述生日攻击的基本原理。假定函数 H 有 2^m 种可能的输出(即输出为 m 位),H 作用于 k 个随机输入,那么 k 为多少时至少有一个重复出现[即对某输入 x 和 y,有 $H(x)=H(y)$]? 利用式(11.6)中的近似公式有

$$k = \sqrt{2^m} = 2^{m/2} \tag{11.7}$$

11A.5 两个集合中元素的重复

我们上面讨论的生日问题可以推广为下列问题:给定一个在 $1 \sim n$ 之间均匀分布的随机整数变量以及它的两个实例集合,每个集合中有 k 个元素($k \leqslant n$),那么这两个集合相交(即至少有一个元素同属于两个集合)的概率 $R(n,k)$ 是多少?

假设这两个集合 X 和 Y 分别为 $\{x_1, x_2, \cdots, x_k\}$ 和 $\{y_1, y_2, \cdots, y_k\}$,对给定的 x_1,$y_1 = x_1$ 的概率恰为 $1/n$,所以 y_1 不等于 x_1 的概率为 $[1-(1/n)]$,若在 Y 中随机取 k 个值,则这些值均不等于 x_1 的概率为 $[1-(1/n)]^k$,因此至少有一个值等于 x_1 的概率为 $1-[1-(1/n)]^k$。

如果 n 和 k 均较大(如 k 约为 $\sqrt{n}$),那么 k 个值中只有少数重复,大多数值都不相同。假定 X 中的元素均不相同,则有

$$\Pr[Y \text{中没有元素与} x_1 \text{匹配}] = \left(1 - \frac{1}{n}\right)^k$$

$$\Pr[Y \text{中没有元素与} X \text{中元素匹配}] = \left(\left(1 - \frac{1}{n}\right)^k\right)^k = \left(1 - \frac{1}{n}\right)k^2$$

$$R(n,k) = \Pr[Y \text{中至少有一元素与} X \text{中元素匹配}] = 1 - \left(1 - \frac{1}{n}\right)k^2$$

由不等式(11.4),则有

$$R(n,k) > 1 - \left(e^{-1/n}\right)^{k^2}$$

$$R(n,k) > 1 - \left(e^{-k^2/n}\right)$$

下面我们来看这样一个问题:k 为多少时,$R(n,k) > 0.5$? 要使 $R(n,k) > 0.5$,则

$$\begin{aligned} 1/2 &= 1 - (e^{-k^2/n}) \\ 2 &= e^{k^2/n} \\ \ln(2) &= \frac{k^2}{n} \\ k &= \sqrt{(\ln(2))n} = 0.83\sqrt{n} \approx \sqrt{n} \end{aligned} \tag{11.8}$$

我们可以将上述问题用与生日攻击有关的术语描述如下:假定函数 H 有 2^m 种可能的输出(即输出为 m 位),H 作用于 k 个随机输入得到集合 X,H 作用于另外 k 个随机输入得到集合 Y,那么 k 为多少时,这两个集合中至少有一个匹配[即对某输入 $x \in X$ 和 $y \in Y$,有 $H(x)=H(y)$]? 由式(11.8)中的近似公式有

$$k = \sqrt{2^m} = 2^{m/2}$$

第12章　消息认证码

12.1　对消息认证的要求
12.2　消息认证函数
　12.2.1　消息加密
　12.2.2　消息认证码
12.3　对消息认证码的要求
12.4　MAC 的安全性
　12.4.1　穷举攻击
　12.4.2　密码分析
12.5　基于 Hash 函数的 MAC：HMAC
　12.5.1　HMAC 设计目标
　12.5.2　HMAC 算法
　12.5.3　HMAC 的安全性
12.6　基于分组密码的 MAC：DAA 和 CMAC
　12.6.1　数据认证算法
　12.6.2　基于密码的消息认证码(CMAC)
~~12.7　认证加密：CCM 和 GCM~~
　12.7.1　分组密码链接－消息认证码
　12.7.2　Galois/计数器模式
~~12.8　使用 Hash 函数和 MAC 产生伪随机数~~
　12.8.1　基于 Hash 函数的 PRNG
　12.8.2　基于 MAC 的 PRNG
12.9　推荐读物和网站
12.10　关键术语、思考题和习题

At cats' green on the Sunday he took the message from the inside of the pillar and added Peter Moran's name to the two names already printed there in the "Brontosaur" code. The message now read: "Leviathan to Dragon: Martin Hillman, Trevor Allan, Peter Moran: observe and tail." What was the good of it John hardly knew. He felt better, he felt that at last he had made an attack on Peter Moran instead of waiting passively and effecting no retaliation. Besides, what was the use of being in possession of the key to the codes if he never took advantage of it?

—Talking to Strange Men, Ruth Rendell

要　点

- 消息认证是用来验证消息完整性的一种机制或服务。消息认证确保收到的数据确实和发送时的一样(即没有修改、插入、删除或重放),且发送方声称的身份是真实有效的。
- 对称密码在那些相互共享密钥的用户间提供认证。
- 消息认证码(MAC)是一种需要使用密钥的算法,以可变长度的消息和密钥作为输入,产生一个认证码。拥有密钥的接收方能够计算认证码来验证消息的完整性。
- 构造 MAC 的方法之一,是将密码学 Hash 函数以某种方式和密钥捆绑起来。
- 构造 MAC 的另外一种方法是使用对称分组密码,将可变长度的输入转为固定长度的输出。

消息认证和数字签名是密码学中最吸引人也最复杂的研究领域之一。只用少量的篇幅来介绍所有已提出或实现的与消息认证与数字签名的密码函数和协议是不可能的。本章和下一章的目的是概要介绍上述内容,并给出对各种方法的系统描述。

本章首先介绍对认证和数字签名的要求以及可能遇到的攻击类型,然后归纳总结一些基本方法。本章其余部分讨论实现消息认证的基本方法,即消息认证码;在对MAC 进行概述和安全性需求分析后,随后将讨论两类 MAC:使用密码学 Hash 函数的 MAC 和使用分组密码的 MAC。然后我们将讨论较新的认证加密。最后将介绍使用 Hash 函数和 MAC 的伪随机数发生器。

An attempt to assign a value larger than N to the distance field results in the assignment of N.

12.1 对消息认证的要求

在网络通信环境中,可能有下述攻击:

(1) **泄密**:将消息透露给没有合法密钥的任何人或程序。
(2) **传输分析**:分析通信双方的通信模式。在面向连接的应用中,确定连接的频率和持续时间;在面向连接或无连接的环境中,确定双方的消息数量和长度。
(3) **伪装**:欺诈源向网络中插入一条消息。如攻击者产生一条消息并声称这条消息来自某合法实体,或者非消息接收方发送的关于收到或未收到消息的欺诈应答。
(4) **内容修改**:对消息内容的修改,包括插入、删除、转换和修改。
(5) **顺序修改**:对通信双方消息顺序的修改,包括插入、删除和重新排序。
(6) **计时修改**:对消息的延时和重播。在面向连接的应用中,整个消息序列可能是前面某合法消息序列的重播,也可能是消息序列中的一条消息被延时或重播;在面向无连接的应用中,可能是一条消息(如数据报)被延时或重播。
(7) **发送方否认**:发送方否认发送过某消息。
(8) **接收方否认**:接收方否认接收到某消息。

对付前两种攻击的方法属于消息保密性范畴,我们已在第一部分讨论了消息保密性;对付第3种至第6种攻击的方法一般称为消息认证;对付第7种攻击的方法属于数字签名。一般而言,数字签名方法也能够抗第3种至第6种攻击中的某些或全部攻击;对付第8种攻击需要使用数字签名和为抗此种攻击而设计的协议。

归纳起来,消息认证就是验证所收到的消息确实是来自真正的发送方,且是未被修改的消息,它也可验证消息的顺序和及时性。数字签名是一种认证技术,其中的一些方法可用来抗发送方否认攻击。

12.2 消息认证函数

任何消息认证或数字签名机制在功能上基本都有上下两层。下层中一定有某种产生认证符的函数,认证符是一个用来认证消息的值;上层协议中将该函数作为原语使接收方可以验证消息的真实性。

本节讨论可以用来产生认证符的函数类型,这些函数可以分为如下三类。

- **Hash 函数**:它是将任意长的消息映射为定长的 Hash 值的函数,以该 Hash 值作为认证符。
- **消息加密**:把整个消息加密后的密文作为认证符。
- **消息认证码(MAC)**:它是消息和密钥的函数,它产生定长的值,以该值作为认证符。

我们已经在第11章讨论了 Hash 函数及其如何用于消息认证。本节将简要讨论剩余两类问题。本章剩余部分将重点讨论关于 MAC 的话题。

12.2.1 消息加密

消息加密本身提供了一种认证手段。对称密码和公钥密码体制中,对消息加密的分析是不相同的。

对称加密

考虑一个使用传统加密的简单例子[参见图 12.1(a)]。发送方 A 用 A 和 B 共享的密钥 K 对

发送到接收方 B 的消息 M 加密，如果没有其他方知道该密钥，那么可提供保密性，因为任何其他方均不能恢复出消息明文。

此外，B 可确信该消息是由 A 产生的。为什么呢？因为除 B 外只有 A 拥有 K，A 能产生出可用 K 解密的密文，所以该消息一定来自于 A。由于攻击者不知道密钥，他也就不知道如何改变密文中的信息位才能在明文中产生预期的改变，因此若 B 可以恢复出明文，则 B 可以认为 M 中的每一位都未被改变。

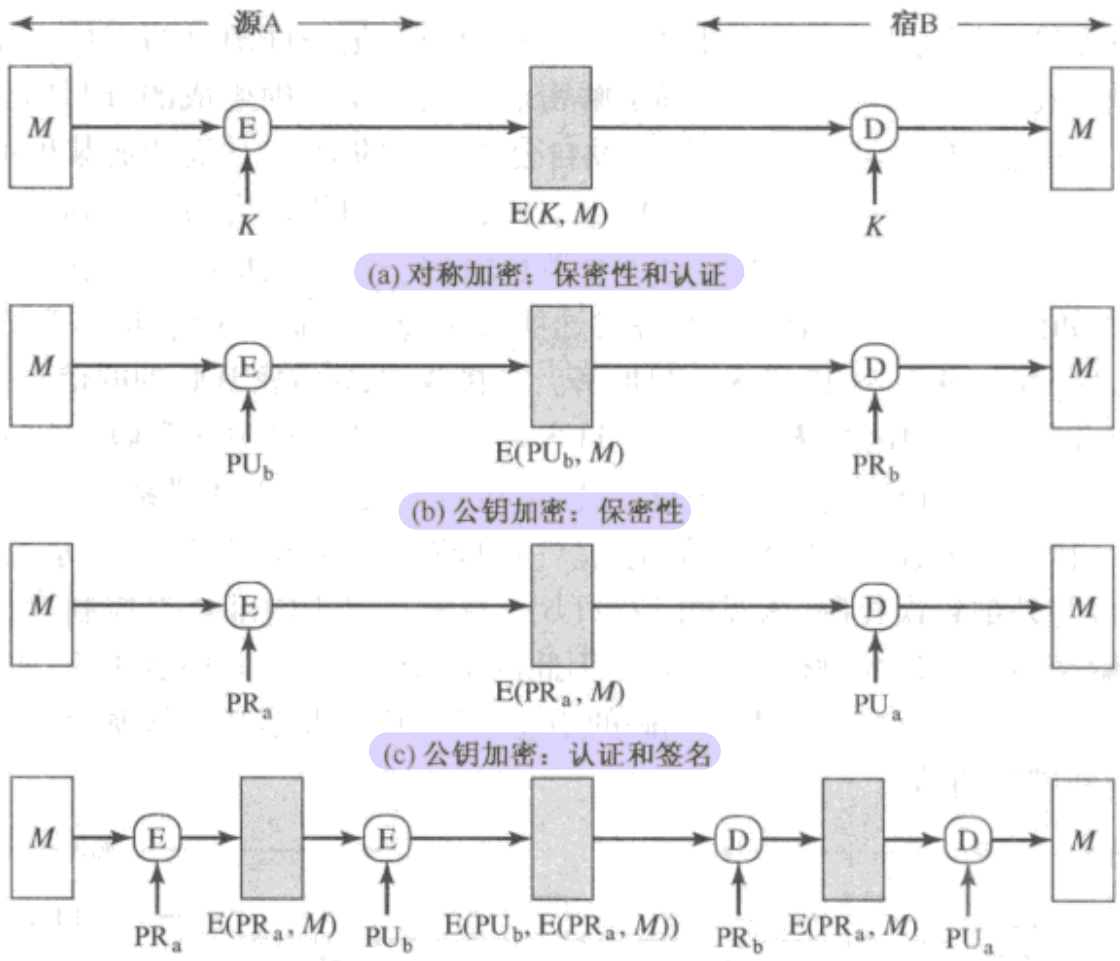


图 12.1　消息加密的基本使用

所以我们可以说，对称密码既可提供认证又可提供保密性，但这不是绝对的。考虑在 B 方所发生的事件，给定解密函数 D 和密钥 K，接收方可接收任何输入 X，并产生输出 $Y = D(K, X)$。若 X 是用相应的加密函数对合法消息 M 加密生成的密文，则 Y 就是明文消息 M，否则 Y 可能是无意义的位串，因此在 B 端需要有某种方法能确定 Y 是合法的明文以及消息确实发自于 A。

从认证的角度来看，上述推理存在这样一个问题：如果消息 M 可以是任何的位模式，那么接收方无法确定接收到的消息是合法明文的密文。显然，若 M 可以是任意的位模式，那么不管 X 的值是什么，$Y = D(K, X)$ 都会作为真实的密文被接受。

一般地，我们要求合法明文只是所有可能位模式的一个小子集，这样，由任何伪造的密文都不太可能得出合法的明文。例如，假定 10^6 种位模式中只有一种是合法明文的位模式，那么随机选择一个位模式作为密文，它产生合法明文消息的概率就只有 10^{-6}。

许多应用和加密方法，都满足上述条件。例如，假定我们利用具有一次移动（$K = 1$）的 Caesar 密码来传递英文消息，A 发送下列合法的消息：

```
nbsftfbupbutboeepftfbupbutboemjuumfmbnctfbujwz
```

B 解密并产生下列明文：

```
mareseatoatsanddoeseatoatsandlittlelambseativy
```

通过简单的频率分析,可以发现这个消息具有普通英语的特点。若攻击者产生下列随机的字符序列:

zuvrsoevgqxlzwigamdvnmhpmccxiuureosfbcebtqxsxq

则它被解密为

ytuqrndufpwkyvhfzlcumlgolbbwhttqdnreabdaspwrwp

这个序列不具有普通英语的特点。

对接收到的密文解密所得明文的可读性进行自动判别,是一件困难的事情。比如,若明文是二进制文件或数字化的 X 射线,那么很难确定解密后的消息是正确生成的,即是真实的明文。因此,攻击者可以简单地发布任何消息并伪称是发自合法用户的消息,从而造成某种程度的破坏。

解决这个问题的方法之一是,要求明文具有某种易于识别的结构,并且不通过加密函数是不能重复这种结构的。例如,可以在加密前对每个消息附加一个错误检测码,也称之为帧校验序列(FCS)或校验和,如图 12.2(a)所示。A 准备发送明文消息 M,那么 A 将 M 作为 F 的输入,产生 FCS,将 FCS 附加在 M 后并对 M 和 FCS 一起加密。在接收端,B 解密其收到的信息,并将其视为消息和附加的 FCS,B 用相同的函数 F 重新计算 FCS。若计算得到的 FCS 和收到的 FCS 相等,则 B 认为消息是真实的。任何随机的位串不可能产生 M 和 FCS 之间的上述联系。

请注意,FCS 和加密函数执行的顺序很重要。[DIFF79]中将图 12.2(a)所示的这种序列称为**内部错误控制**,以与外部错误控制[参见图 12.2(b)]对应。对于内部错误控制,由于攻击者很难产生密文,使得解密后其错误控制位是正确的,因此内部错误控制可以提供认证;如果 FCS 是外部码,那么攻击者可以构造具有正确错误控制码的消息,虽然攻击者不知道解密后的明文是什么,但他可以造成混淆并破坏通信。

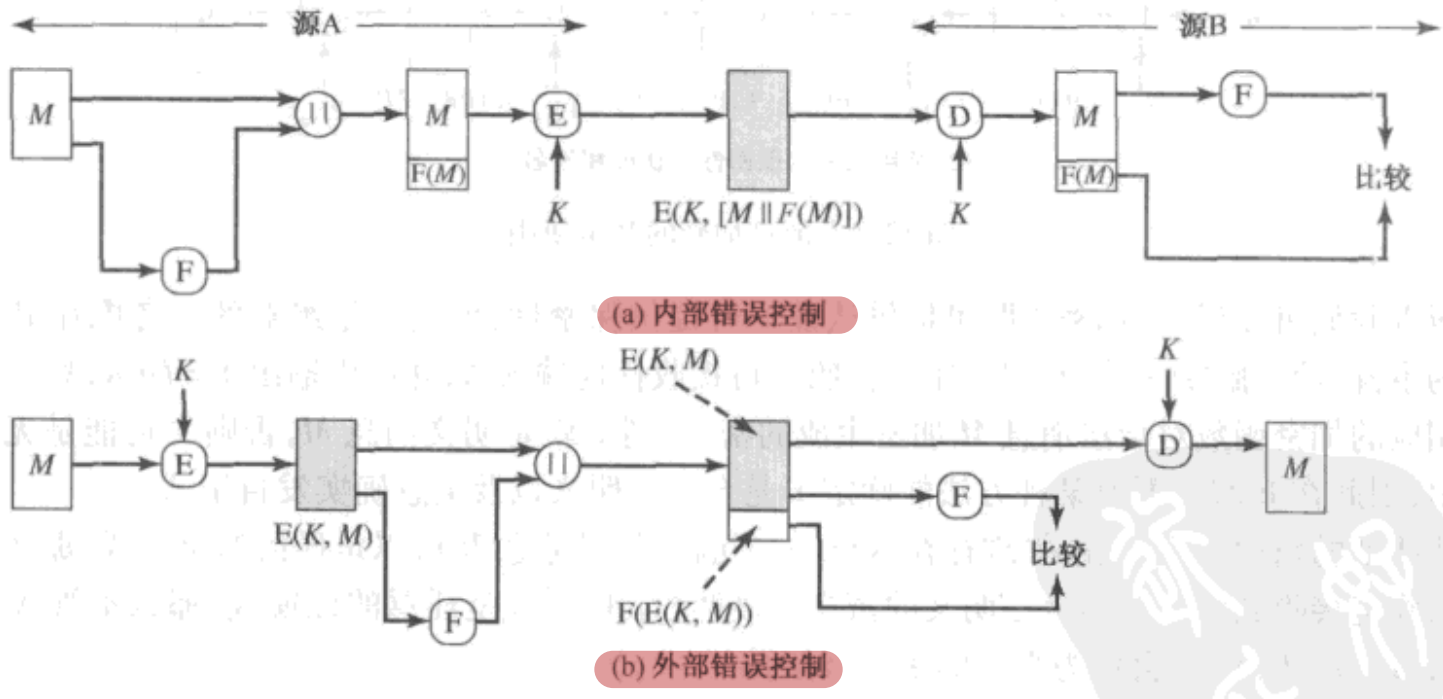


图 12.2 内部和外部错误控制

错误控制码仅是具有上述结构的一个例子。事实上,在要发送的消息中加入任何类型的结构信息都会增强认证能力。分层协议通信体系可以提供这种结构,例如,考虑使用 TCP/IP 协议传输的消息的结构。图 12.3 给出了 TCP 段的格式并说明了 TCP 头的结构。假定每对主机共享一个密钥,并且无论是何种应用,每对主机间都使用相同的密钥进行信息交换,那么我们可以对除 IP 头外的所有数据报加密,如果攻击者用一条消息替代加密后的 TCP 段,那么解密后所得出的明文将不会包含有意义的头信息。在这种方法中,头不仅包含校验和(校验整个头部),而且还含有其他

一些有用的信息，如序列号。因为对于给定连接，连续的 TCP 段是按顺序编号的，所以加密使得攻击者不能延时、删除任何段或改变段的顺序。

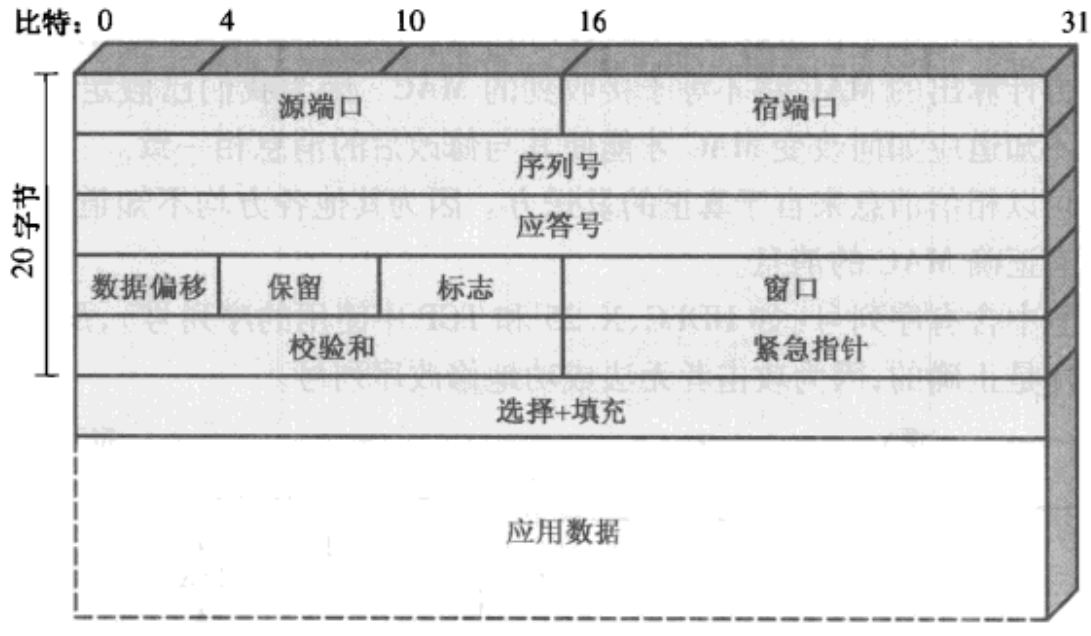


图 12.3　TCP 段

公钥加密 12.1 删除3行

直接使用公钥加密[参见图 12.1(b)]可提供保密性，但不能提供认证。发送方(A)使用接收方(B)的公钥 PU_b 对 M 加密，由于只有 B 拥有相应的私钥 PR_b，所以只有 B 能对消息解密。但是任何攻击者可以假冒 A 用 B 的公钥对消息加密，所以这种方法不能保证真实性。

若要提供认证，则 A 用自己的私钥对消息加密，而 B 用 A 的公钥对接收的消息解密[参见图 12.1(c)]。和对称密码情形的推理一样，这提供了认证功能：因为只有 A 拥有 PR_a，能产生用 PU_a 可解密的密文，所以该消息一定来自于 A。同样，对明文也必须有某种内部结构以使接收方能区分真实的明文和随机的位串。

假定明文具有这种结构，那么图 12.1(c)的方法既可提供认证，又可提供数字签名功能[①]。由于只有 A 拥有 PR_a，所以只有 A 能够产生密文，甚至接收方 B 也不能产生密文，因此若 B 接收到密文消息，则 B 可以确认该消息来自于 A。事实上，A 通过用其私钥对消息加密来对该消息“签名”。注意，这种方法不能提供保密性，因为任何拥有 A 的公钥的人都可将密文解密。

如果既要提供保密性又要提供认证，那么 A 可先用其私钥对 M 加密，这就是数字签名；然后 A 用 B 的公钥对上述结果加密，这可保证保密性[参见图 12.1(d)]。但这种方法的缺点是，一次通信中要执行四次而不是两次复杂的公钥算法。

12.2.2　消息认证码

消息认证码，又称**密码校验和**或者 **MAC**，也是一种认证技术，它利用密钥来生成一个固定长度的短数据块，并将该数据块附加在消息之后。在这种方法中，假定通信双方比如 A 和 B，共享密钥 K。若 A 向 B 发送消息时，则 A 计算 MAC，它是消息和密钥的函数，即

$$\text{MAC} = C(K, M)$$

其中，M 是输入消息，C 是 MAC 函数，K 是共享的密钥，MAC 是消息认证码。

消息和 MAC 一起被发送给接收方。接收方对收到的消息用相同的密钥 K 进行相同的计算，

① 这不是构造数字签名的一般方法，但是我们将会看到，它们的基本原理是相同的。

得出新的 MAC,并将接收到的 MAC 与其计算出的 MAC 进行比较[参见图 12.4(a)],如果我们假定只有收发双方知道该密钥,那么若接收到的 MAC 与计算得出的 MAC 相等,则

(1) 接收方可以相信消息未被修改。如果攻击者改变了消息,但他无法改变相应的 MAC,所以接收方计算出的 MAC 将不等于接收到的 MAC。因为我们已假定攻击者不知道密钥,所以他不知道应如何改变 MAC 才能使其与修改后的消息相一致。

(2) 接收方可以相信消息来自于真正的发送方。因为其他各方均不知道密钥,因此他们不能产生具有正确 MAC 的消息。

(3) 如果消息中含有序列号(如 HDLC、X.25 和 TCP 中使用的序列号),那么接收方可以相信消息顺序是正确的,因为攻击者无法成功地修改序列号。

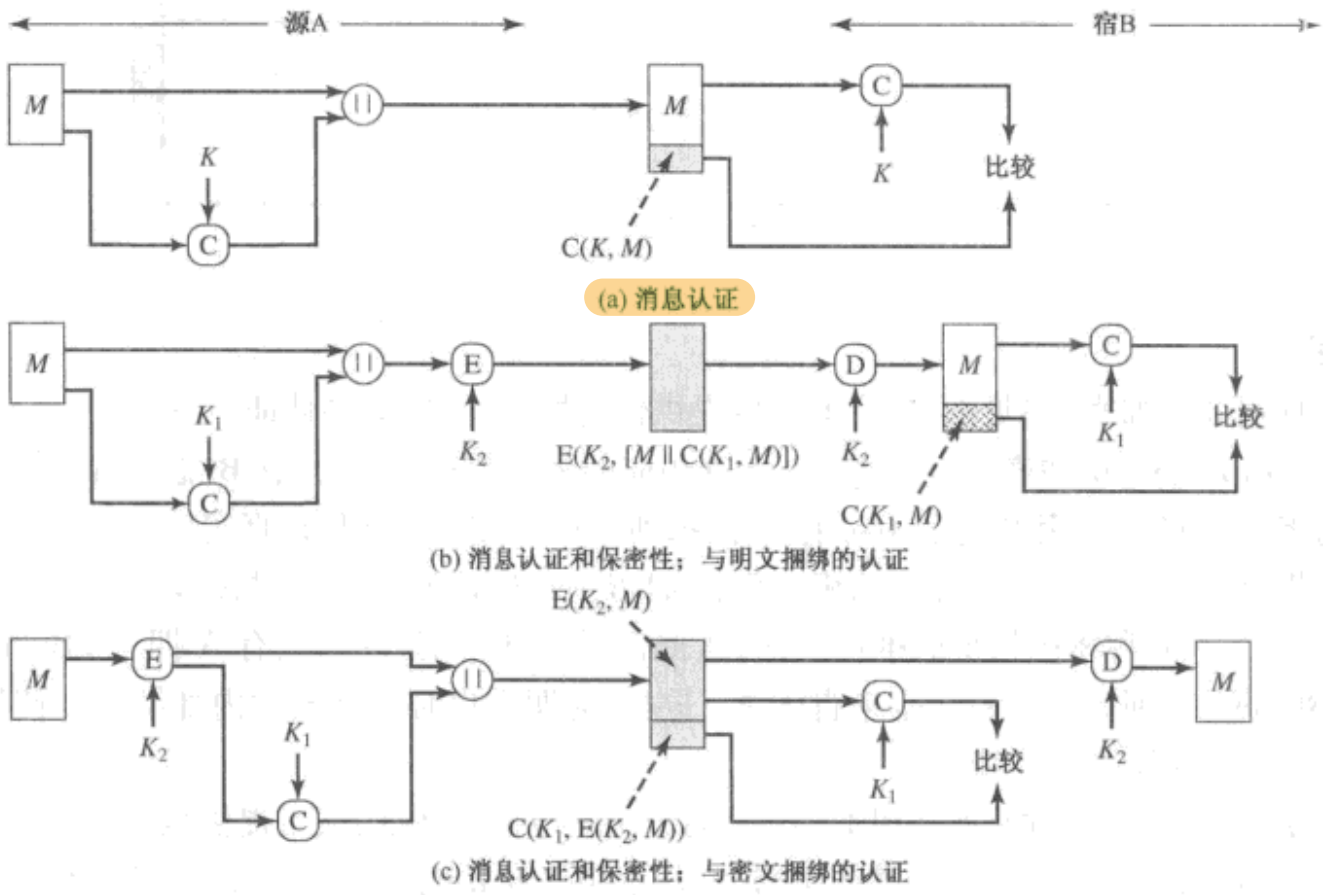


图 12.4 消息认证码(MAC)的基本用途

MAC 函数与加密类似。其区别之一是,MAC 算法不要求可逆性,而加密算法必须是可逆的。一般而言,MAC 函数是多对一函数,其定义域由任意长的消息组成,而值域由所有可能的 MAC 和密钥组成。若使用 n 位长的 MAC,则有 2^n 个可能的 MAC,而有 N 条可能的消息,其中 $N \gg 2^n$。而且若密钥长为 k,则有 2^k 种可能的密钥。

例如,假定使用 100 位的消息和 10 位的 MAC,那么总共有 2^{100} 不同的消息,但仅有 2^{10} 种不同的 MAC。所以平均而言,同一 MAC 可以由 $2^{100}/2^{10} = 2^{90}$ 条不同的消息产生。若使用的密钥长为 5 位,则从消息集合到 MAC 值的集合有 $2^5 = 32$ 种不同的映射。

可以证明,由于认证函数的数学性质,与加密相比,认证函数更不易被攻破。

图 12.4(a)所示的过程可以提供认证,但不能提供保密性,因为整个消息是以明文形式传送的。若在 MAC 算法之后[参见图 12.4(b)]或之前[参见图 12.4(c)]对消息加密,则可以获得保密性。这两种情形都需要两个独立的密钥,并且收发双方共享这两个密钥。在第一种情形中,先将消息作为输入,计算 MAC,并将 MAC 附加在消息后,然后对整个信息块加密;在第二种情形中,先将消息加密,然后将此密文作为输入,计算 MAC,并将 MAC 附加在上述密文之后形成待发送的信息块。一般而言,将 MAC 直接附加于明文之后要更好一些,所以通常使用图 12.4(b)中的方法。

对称加密可以提供认证，且它已被广泛用于现有产品之中，那么为什么不直接使用这种方法而要使用重新设计的消息认证码呢？[DAVI89]中提出了三种必须使用消息认证码的情形。

(1) 有许多应用是将同一消息广播给很多接收者。例如需要通知各用户网络暂时不可使用，或一个军事控制中心要发一条警报，这种情况下，一种经济可靠的方法就是只要一个接收者负责验证消息的真实性，所以消息必须以明文加上消息认证码的形式进行广播。上述负责验证的接收者拥有密钥并执行认证过程，若 MAC 错误，则他发警报通知其他各接收者。

(2) 在信息交换中，可能有这样一种情况，即通信某一方的处理负荷很大，没有时间解密收到的所有消息，他应能随机选择消息并对其进行认证。

(3) 对明文形式的计算机程序进行认证是一种很有意义的服务。运行一个计算机程序而不必每次对其解密，因为每次对其解密会浪费处理器资源。若将消息认证码附于该程序之后，则可在需要保证程序完整性的时候才检验消息认证码。

除此以外，还有下述三种情形。

(4) 一些应用并不关心消息的保密性，而关心消息认证。例如简单网络管理协议版本 3 (SNMPv3)就是如此，它将提供保密性和提供认证分离开来。对这些应用，管理系统应对其收到的 SNMP 消息进行认证，这一点非常重要，尤其是当消息中包含修改系统参数的命令时更是如此，但对这些应用不必加密 SNMP 传输。

(5) 将认证和保密性分离开来，可使层次结构更加灵活。例如，在应用层我们可能希望对消息进行认证，而在更低层上，如传输层，我们可能希望提供保密性。

(6) 仅在接收消息期间对消息实施保护是不够的，用户可能希望延长对消息的保护时间。就消息加密而言，消息被解密后就不再受任何保护，这样只是在传输中可以使消息不被修改，而不是在接收方系统中保护消息不被修改。

最后，要注意的是，由于收发双方共享密钥，因此 MAC 不能提供数字签名。

12.3　对消息认证码的要求

MAC 也称为密码校验和，它由如下形式的函数产生：

$$T = \mathrm{MAC}(K, M)$$

其中 M 是一个变长消息，K 是收发双方共享的密钥，$\mathrm{MAC}(K, M)$ 是定长的认证符，有时也称为标记(tag)。在假定或已知消息正确时，将 MAC 附于发送方的消息之后；接收方可通过计算 MAC 来认证该消息。

为了获得保密性，可用对称或非对称密码对整个消息加密，这种方法的安全性一般依赖于密钥的位长。除非算法中本身存在某些弱点，否则攻击者必须对所有可能的密钥进行穷举攻击，一般对于 k 位的密钥，穷举攻击需要 $2^{(k-1)}$ 步。特别地，对于唯密文攻击，若给定密文 C，攻击者要对所有可能的 K_i 计算 $P_i = D(K_i, C)$，直到产生的某 P_i 具有适当的明文结构为止。

对 MAC 情况则完全不同。一般 MAC 函数是多对一函数。攻击者如何用穷举方法找到密钥呢？如果没有提供保密性，那么攻击者可访问明文形式的消息及其 MAC，假定 $k > n$，即假定密钥位数比 MAC 长，那么对满足 $T_1 = \mathrm{MAC}(K_1, M_1)$ 的 M_1 和 T_1，密码分析者要对所有可能的密钥值 K_i 计算 $T_i = \mathrm{MAC}(K_i, M_1)$，那么至少需要有一个密钥会使得 $T_i = T_1$。注意，这里总共会产生 2^k 个 MAC，但只有 2^n

$< 2^k$个不同的 MAC 值,所以许多密钥都会产生正确的 MAC,而攻击者却不知哪一个是正确的密钥。平均来说,有$2^k/2^n = 2^{(k-n)}$个密钥会产生正确的 MAC,因此攻击者必须重复下述攻击:

- **循环 1**
  给定 M_1及 $T_1 = \text{MAC}(K, M_1)$
  对所有 2^k个密钥,计算 $T_i = \text{MAC}(K_i, M_1)$
  匹配数$\approx 2^{(k-n)}$
- **循环 2**
  给定 M_2及 $T_2 = \text{MAC}(K, M_2)$
  对余下的 $2^{(k-n)}$个密钥,计算 $T_i = \text{MAC}(K_i, M_2)$
  匹配数$\approx 2^{(k-2\times n)}$

以此类推。平均来讲,若 $k = \alpha \times n$,则需 α 次循环。例如,如果使用 80 位的密钥和长为 32 位的 MAC,那么第一次循环会得到约 2^{48}个可能的密钥,第二次循环会得到约 2^{16}个可能的密钥,第三次循环则得到唯一一个密钥,这个密钥就是发送方所使用的密钥。

如果密钥的长度小于或等于 MAC 的长度,则很可能第一次循环中就得到一个密钥,当然也可能得到多个密钥,这时攻击者还需要对新的(消息,MAC)对执行上述测试。

由此可见,用穷举方法来确定认证密钥不是一件容易的事,而且确定认证密钥比确定同样长度的加密密钥更困难。不过,可能存在有无须去寻找密钥的其他攻击。

考虑下面的 MAC 算法。令消息 $M = (X_1 \| X_2 \| \cdots \| X_m)$是由 64 位分组 X_i连接而成的。定义

$$\Delta(M) = X_1 \oplus X_2 \oplus \cdots \oplus X_m$$

$$\text{MAC}(K, M) = \text{E}(K, \Delta(M))$$

其中⊕是异或(XOR)运算,加密算法是电子密码本方式 DES,那么密钥长为 56 位,MAC 长为 64 位。若攻击者知道$\{M \| \text{MAC}(K, M)\}$,则确定 K 的穷举攻击需要执行至少 2^{56}次加密,但是攻击者可以用任何期望的 Y_1至 Y_{m-1}替代 X_1至 X_{m-1},用 Y_m替代 X_m来进行攻击,其中 Y_m是按如下方式计算的:

$$Y_m = Y_1 \oplus Y_2 \oplus \cdots \oplus Y_{m-1} \oplus \Delta(M)$$

攻击者可以将 Y_1至 Y_m与原来的 MAC 连接成一个新的消息,而接收方却会认为该消息是真实的。用这种办法,攻击者可以随意插入任意的长为 $64 \times (m-1)$位的消息。

因此,评价 MAC 函数的安全性时,我们应考虑对该函数的各种类型的攻击。下面将介绍 MAC 函数应满足的要求。假定攻击者知道 MAC 函数,但不知道 K,那么 MAC 函数应具有下列性质:

(1) 若攻击者已知 M 和 $\text{MAC}(K, M)$,则他构造满足 $\text{MAC}(K, M') = \text{MAC}(K, M)$的消息 M' 在计算上是不可行的。
(2) $\text{MAC}(K, M)$应是均匀分布的,即对任何随机选择的消息 M 和 M',$\text{MAC}(K, M') = \text{MAC}(K, M')$的概率是 2^{-n},其中 n 是 MAC 的位数。
(3) 设 M' 是 M 的某个已知的变换,即 $M' = f(M)$,例如 f 可能是将 M 的一位或多位取反,要求 $\Pr[\text{MAC}(K, M) = \text{MAC}(K, M')] = 2^{-n}$。

前面已讲过,攻击者即使不知道密钥,他也可以构造出与给定的 MAC 匹配的新消息,第一个要求就是针对这种情况提出的。第二个要求是为了阻止基于选择明文的穷举攻击,也就是说,假定攻击者不知道 K,但是他可以访问 MAC 函数,能对消息产生 MAC,那么攻击者可以对各种消息计算 MAC,直至找到与给定 MAC 相同的消息为止。如果 MAC 函数具有均匀分布的特征,那么穷举方法平均需要 $2^{(n-1)}$步才能找到具有给定 MAC 的消息。

最后一条要求，认证算法对消息的某部分或位不应比其他部分或位更弱，否则，已知 M 和 MAC(K,M)的攻击者可以对 M 的已知的“弱点”处进行修改，然后再计算 MAC，这样有可能更早得出具有给定 MAC 的新消息。

12.4　MAC 的安全性

同加密算法和 Hash 函数一样，我们也可将对 MAC 的攻击分为两类：穷举攻击和密码分析。

12.4.1　穷举攻击

对 MAC 的穷举攻击由于需要知道 <消息- MAC> 对，所以这种攻击会比对 Hash 函数的攻击更加困难，下面来分析其原因。攻击者可以按下述方式对 Hash 码进行攻击：对给定的消息 x 及其 n 位 Hash 码 $h=H(x)$，寻找碰撞的穷举攻击方法可以随机挑选一个位串 y，检查是否有 $H(y)=H(x)$。攻击者可以以离线方式重复上述操作，但对 MAC 算法是否能使用离线攻击则依赖于密钥和 MAC 的长度。

在进一步讨论之前，我们先讨论 MAC 算法所应具有的安全性质，该性质可描述如下：

- **抗计算性**：给定一个或多个 <消息 - MAC> 对[x_i,MAC(K,x_i)]，对任何新的输入 $x \neq x_i$，计算 <消息- MAC> 对[x,$C(K,x)$]在计算上是不可行的。

换句话说，对给定的消息 x，攻击者可通过攻击密钥空间和攻击 MAC 值这两种方法来找出其 MAC。下面我们依次来讨论这些攻击。

如果攻击者能够确定 MAC 密钥，那么他就可以对任何输入 x 产生有效的 MAC。假定密钥长为 k 位，并且攻击者已知一个 <消息- MAC> 对，那么攻击者可用所有可能的密钥对该消息计算其 n 位的 MAC，这样至少有一个密钥会产生正确的 MAC，该密钥就是原来用来产生该 <消息- MAC> 对的密钥，此处所需的代价约为 2^k（即对 2^k 个可能的密钥中的每一个执行一次操作）。但是，如前所说，因为 MAC 是多对一映射的，所以可能有其他一些密钥也会产生正确的 MAC 值，因此，如果不止一个密钥产生正确值，那么必须检查其他一些 <消息- MAC> 对。可以证明，检查这些 <消息- MAC> 对所需的代价会迅速减小，并且总的代价约为 2^k[MENE97]。

攻击者也可以攻击 MAC 而不试图去找出密钥，这种攻击的目的是，对给定的消息产生其有效的 MAC，或者对给定的 MAC 产生相应的消息。这两种情形中，其代价为 2^n，与攻击具有单向性或抗弱碰撞能力的 Hash 码所需的代价相同。对 MAC 攻击时，因为攻击者需要有已选择的 <消息- MAC> 对或者密钥信息，所以这种攻击不能离线进行。

总之，对 MAC 算法的穷举攻击所需的代价为 $\min(2^k,2^n)$。其强度的评价与对称密码算法中的讨论类似。密钥长度和 MAC 的长度应满足关系式 $\min(k,n) \geqslant N$，其中 N 可以为 128 位。

至少128位安全

12.4.2　密码分析

与对加密算法和 Hash 函数的攻击一样，对 MAC 算法的密码分析攻击，也是利用算法的某种性质而不是通过穷举来进行的。评价 MAC 算法抗密码分析能力的方法是，将其与穷举攻击所需的代价相比，也就是说，理想的 MAC 算法要求密码分析攻击所需的代价大于或等于穷举攻击所需的代价。

与 Hash 函数相比，MAC 的结构种类更多，而且对 MAC 的密码分析攻击的研究很少，所以很难归纳总结对 MAC 的密码分析。[PREN96]中概述了对某些特定 MAC 的密码分析攻击。

12.5 基于 Hash 函数的 MAC:HMAC

在本章稍后部分,我们将讨论使用对称分组密码的消息认证码(MAC),这一直是构造 MAC 的最常用方法。近年来,人们越来越感兴趣于利用密码学 Hash 函数来设计 MAC,因为

(1) 一般像 MD5 和 SHA 这样的密码学 Hash 函数,其软件执行速度比诸如 DES 这样的对称分组密码要快。

(2) 有许多共享的密码学 Hash 函数代码库。

随着 AES 的开发以及密码算法代码的可用性日益广泛,上述考虑的意义将会削弱,但是基于 Hash 函数的 MAC 仍将持续广泛使用。

诸如 SHA 这样的 Hash 函数并不是专为 MAC 而设计的,由于 Hash 函数不依赖于密钥,所以它不能直接用于 MAC。目前,已经提出了许多方案将密钥加到现有的 Hash 函数中。HMAC(RFC 2104)是最受支持的方案[BELL96a,BELL96b],它是 IP 安全里必须实现的 MAC 方案,并且其他 Internet 协议中(如 SSL)也使用了 HMAC。HMAC 也已作为 NIST 的标准发布(FIPS 198)。

12.5.1 HMAC 设计目标

RFC 2104 给出了 HMAC 的设计目标。

- 不必修改而直接使用现有的 Hash 函数。特别地,很容易免费得到软件上执行速度较快的 Hash 函数及其代码。
- 如果找到或者需要更快或更安全的 Hash 函数,应能很容易地替代原来嵌入的 Hash 函数。
- 应保持 Hash 函数的原有性能,不能过分降低其性能。
- 对密钥的使用和处理应较简单。
- 如果已知嵌入的 Hash 函数的强度,则完全可以知道认证机制抗密码分析的强度。

前两个目标是 HMAC 为人们所接受的重要原因,HMAC 将 Hash 函数视为"黑盒"有两个好处。第一,实现 HMAC 时可将现有 Hash 函数作为一个模块,这样可以对许多 HMAC 代码预先封装,并在需要时直接使用;第二,若希望替代 HMAC 中的 Hash 函数,则只需删去现有的 Hash 函数模块并加入新的模块,例如需要更快的 Hash 函数时就可如此处理。更重要的是,如果嵌入的 Hash 函数的安全受到威胁,那么只需用更安全的 Hash 函数替换嵌入的 Hash 函数(如用 SHA-3 替代 SHA-2),仍然可保持 HMAC 的安全性。

上述最后一个设计目标实际上是 HMAC 优于其他基于 Hash 的一些方法的主要方面。只要嵌入的 Hash 函数有合理的密码分析强度,则可以证明 HMAC 是安全的。本节后面再来讨论这个问题,下面我们先讨论 HMAC 的结构。

12.5.2 HMAC 算法

图 12.5 给出了 HMAC 的总体结构。定义下列符号:

H 为嵌入的 Hash 函数(如 MD5、SHA-1、RIPEMD-160)

IV 为 Hash 函数输入的初始值

M 为 HMAC 的消息输入(包括由嵌入 Hash 函数定义的填充位)

Y_i 为 M 的第 i 个分组,$0 \leqslant i \leqslant (L-1)$

L 为 M 中的分组数

b 为每一分组所含的位数

n 为嵌入的 Hash 函数所产生的 Hash 码长

K 为密钥；建议密钥长度 $\geq n$。若密钥长度大于 b，则将密钥作为 Hash 函数的输入，来产生一个 n 位的密钥

K^+ 为使 K 为 b 位长而在 K 左边填充 0 后所得的结果

ipad = 00110110（十六进制数 36）重复 $b/8$ 次的结果

opad = 01011100（十六进制数 5C）重复 $b/8$ 次的结果

HMAC 可描述如下：

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \| H[(K^+ \oplus \text{ipad}) \| M]]$$

该算法描述如下。

（1）在 K 左边填充 0，得到 b 位的 K^+（例如，若 K 是 160 位，$b = 512$，则在 K 中加入 44 个字节的 0）。

（2）K^+ 与 ipad 执行异或运算（位异或）产生 b 位的分组 S_i。

（3）将 M 附于 S_i 后。

（4）将 H 作用于步骤 3 所得出的结果。

（5）K^+ 与 opad 执行异或运算（位异或）产生 b 位的分组 S_o。

（6）将步骤 4 中的 Hash 码附于 S_o 后。

（7）将 H 作用于步骤（6）所得出的结果，并输出该函数值。

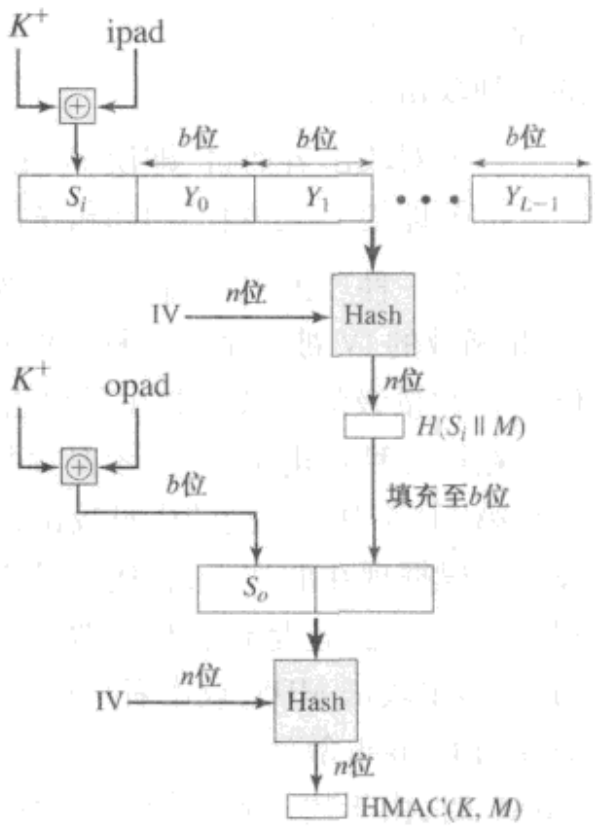


图 12.5　HMAC 的结构

注意，K 与 ipad 异或后，其信息位有一半发生了变化；同样，K 与 opad 异或后，其信息位的另一半也发生了变化，这样通过将 S_i 与 S_o 传给 Hash 算法中的压缩函数，我们可以从 K 伪随机地产生出两个密钥。

HMAC 多执行了三次 Hash 压缩函数（对 S_i、S_o 和内部的 Hash 产生的分组），但是对于长消息，HMAC 和嵌入的 Hash 函数的执行时间应该大致相同。

实现 HMAC 有更为有效的方法，如图 12.6 所示。我们可以预计算两个值：

$$f(\text{IV}, (K^+ \oplus \text{ipad}))$$
$$f(\text{IV}, (K^+ \oplus \text{opad}))$$

其中 $f(\text{cv}, \text{block})$ 是 Hash 函数的压缩函数，其输入是 n 位的链接变量和 b 位的分组，输出是 n 位的链接变量。上述这些值只在初始化或密钥改变时才需计算，实际上这些预先计算的值取代了 Hash 函数中的初值（IV）。这样，只多执行了一次压缩函数，在产生 MAC 的消息大多数都较短的情况下，这种实现特别有意义。

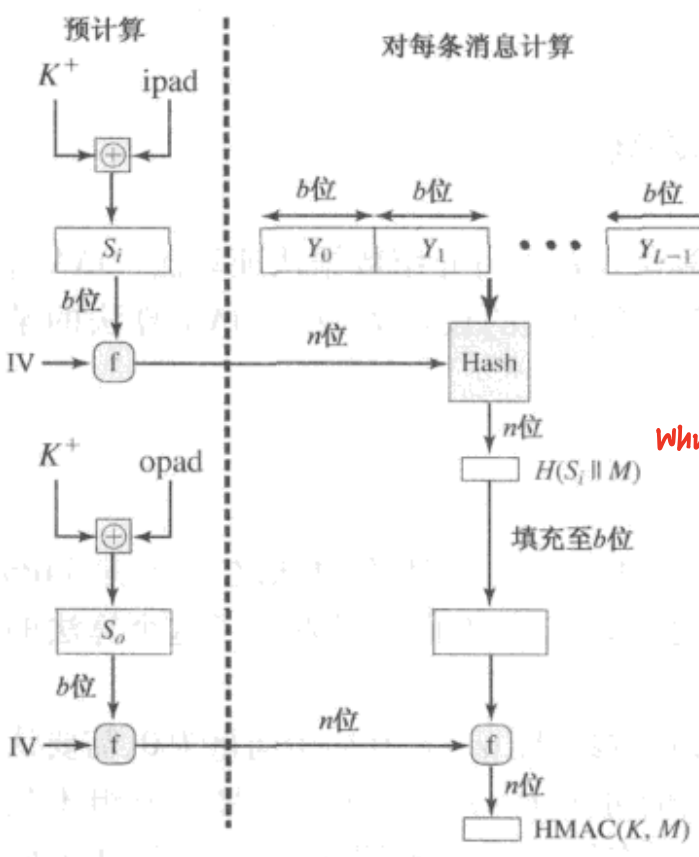


图 12.6　HMAC 的有效实现方案

12.5.3　HMAC 的安全性

任何建立在嵌入 Hash 函数基础上的 MAC，其安全性在某种程度上依赖于该 Hash 函数的强度。HMAC 的好处在于，其设计者可以证明嵌入 Hash 函数的强度与 HMAC 的强度之间的联系。

在给定时间内，给定的一定数量的 < 消息 - MAC > 对（用相同密钥产生），伪造者伪造成功的

概率可以用来描述 MAC 函数的安全性。参考文献[BELL96a]中已经证明,如果攻击者已知若干由合法用户生成的(时间,<消息-MAC>)对,则成功攻击 HMAC 的概率等价于对嵌入 Hash 函数的下列攻击之一。

(1) 对攻击者而言,即使 IV 是随机的、秘密的和未知的,攻击者也能计算压缩函数的输出。

(2) 即使 IV 是随机的和秘密的,攻击者也能找到 Hash 函数中的碰撞。

在第一种攻击中,我们可将压缩函数视为将 Hash 函数应用于只含有一个 b 位分组的消息,Hash 函数的 IV 被一个 n 位秘密的随机值代替。攻击该 Hash 函数或者是对密钥的穷举攻击(其代价是 2^n 数量级的),或者是生日攻击(这是第二种攻击的特例,请见下面的讨论)。

在第二种攻击中,攻击者要找两条消息 M 和 M',它们产生相同的 Hash 码:$H(M) = H(M')$,这就是第 11 章中所讨论的生日攻击,我们已经证明当 Hash 码长为 n 位时其所需的代价是 $2^{n/2}$ 数量级的。根据现在的技术,若代价是 2^{64} 数量级的,则被认为是可计算的,所以 MD5 的安全性不能得到保证。但是,这是否意味着像 MD5 这样的 128 位的 Hash 函数不能用于 HMAC 呢?回答是否定的,因为要攻击 MD5,攻击者可以选择任何消息集,并用专用计算机离线计算来寻找碰撞,由于攻击者知道 Hash 算法和默认的 IV,因此攻击者可以对其产生的任何消息计算 Hash 码。但是,攻击 HMAC 时,由于攻击者不知道 K,所以他不能离线产生消息/Hash 码对,他必须观察 HMAC 用相同的密钥产生的消息序列,并对这些消息进行攻击。Hash 码长为 128 位时,攻击者必须观察 2^{64} 个由同一密钥产生的分组(2^{72} 位),对 1 Gb/ps 连接,要想攻击成功,攻击者约需 150 000 年来观察用同一密钥产生的连续的消息流。因此,当注重执行速度时,用 MD5 而不是 SHA-1 作为 HMAC 的嵌入 Hash 函数,是完全可以接受的。

12.6 基于分组密码的 MAC:DAA 和 CMAC

本节我们讨论两个基于分组密码工作模式的 MAC 算法。首先将介绍数据认证算法(DAA),该算法较陈旧,目前已经被废止。然后介绍 CMAC 算法,该算法的设计克服了 DAA 算法的某些缺陷。

12.6.1 数据认证算法

数据认证算法(DAA)建立在 DES 之上,是多年以来使用最广泛的 MAC 算法之一。它是 FIPS 标准(FIPS PUB 113)和 ANSI 标准(X9.17)。然而我们随后会介绍,人们已经发现了这个算法的安全弱点,并正使用一个更新、更强的算法来代替它。

数据认证算法采用 DES 运算的密文块链接(CBC)方式(参见图 6.4),其初始向量为 0,需要认证的数据(如消息、记录、文件或程序)分成连续的 64 位的分组 $D_1, D_2, \cdots, D_N$,若最后分组不足 64 位,则在其后填 0 直至成为 64 位的分组。利用 DES 加密算法 E 和密钥 K,计算数据认证码(DAC)的过程如图 12.7 所示。

$$
\begin{aligned}
O_1 &= \mathrm{E}(K, D_1) \\
O_2 &= \mathrm{E}(K, [D_2 \oplus O_1]) \\
O_3 &= \mathrm{E}(K, [D_3 \oplus O_2]) \\
&\vdots \\
O_N &= \mathrm{E}(K, [D_N \oplus O_{N-1}])
\end{aligned}
$$

其中,DAC 可以是整个块 O_N,也可以是其最左边的 M 位,其中 $16 \leqslant M \leqslant 64$。

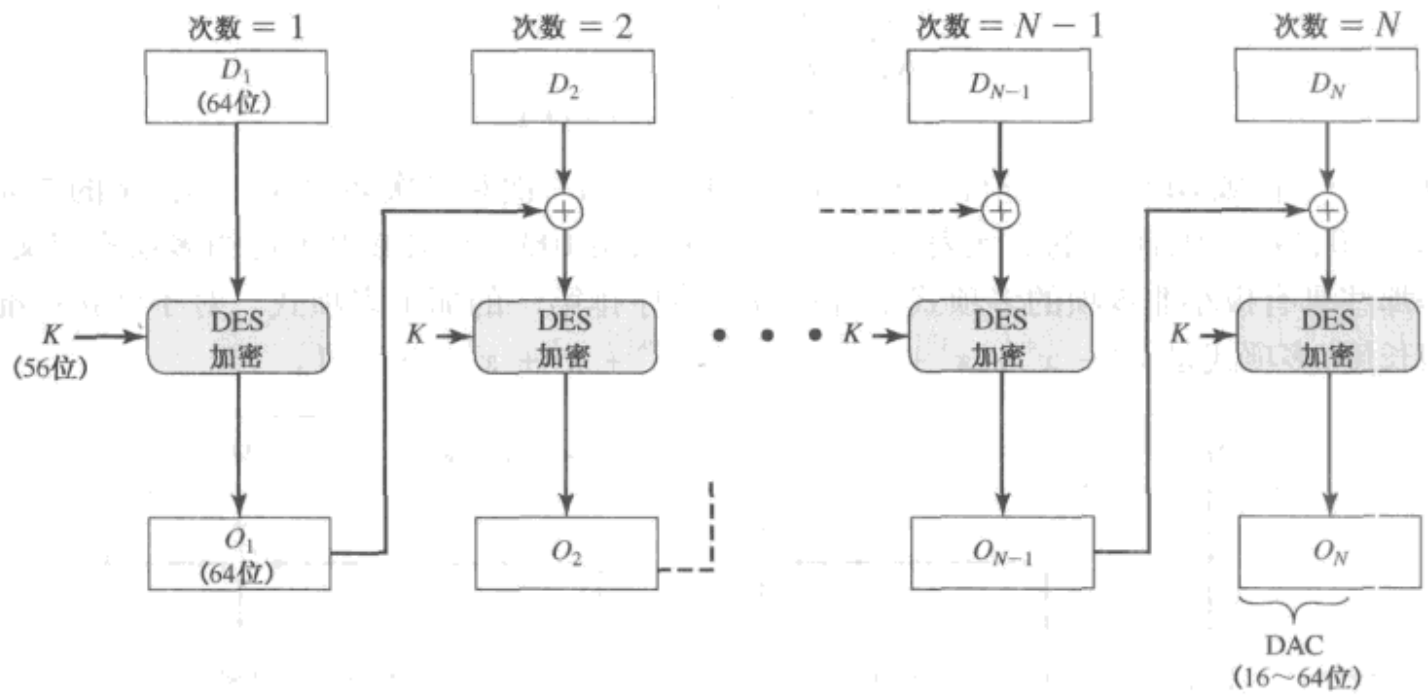


图 12.7　数据认证算法(FIPS PUB 113)

12.6.2　基于密码的消息认证码(CMAC)

我们已经提过 DAA 算法在政府和工业界广泛采用。[BELL00]证明了在合理的安全准则下这种 MAC 是安全的,但有如下的限制:仅能处理固定长度为 mn 的消息,其中 n 是密文分组的长度,m 是一个固定的正整数。例如,给定一个消息分组 X 的 CBC MAC 码,如 $T = \text{MAC}(K, X)$,则攻击者马上就知道两个消息分组 $X \| (X \oplus T)$ 的 CBC MAC 码,因为这还是 T。

Black 和 Rogaway[BLAC00]证明了这种限制可以使用三个密钥来克服:一个密钥长为 K,用在密文分组链接的每一步,两个长度为 n 的密钥,其中 k 是密钥长度,n 为密文分组长度。Iwata 和 Kurosawa 又优化了该构造,使得两个 n 位的密钥可以从加密密钥导出,而不是单独提供[IWAT03]。这种优化已经被 NIST 采用作为基于密码的消息认证码(CMAC)的运算模式,对于 AES、3DES 适用。NIST 的专门出版物 800-38B 有该规范。

① 首先,当消息长度是分组长度 b 的 n 倍时,我们考虑 CMAC 的运算情况。对 AES,$b = 128$,对于 3DES,$b = 64$。这个消息被划分为 n 组$(M_1, M_2, \cdots, M_n)$。算法使用了 k 位的加密密钥 K 和 n 位的常数 K_1。对于 AES,密钥长度 k 为 128、192 或 256 位,对于 3DES,密钥长度为 112 或168 位。CMAC 按如下方式计算(参见图 12.8):

$$
\begin{aligned}
C_1 &= \text{E}(K, M_1) \\
C_2 &= \text{E}(K, [M_2 \oplus C_1]) \\
C_3 &= \text{E}(K, [M_3 \oplus C_2]) \\
&\vdots \\
C_n &= \text{E}(K, [M_n \oplus C_{n-1} \oplus K_1]) \\
T &= \text{MSB}_{\text{Tlen}}(C_n)
\end{aligned}
$$

其中,T 为消息认证码,也称为 tag;Tlen 是 T 的位长度,$\text{MSB}_s(X)$是位串的 X 最左边的 s 位。

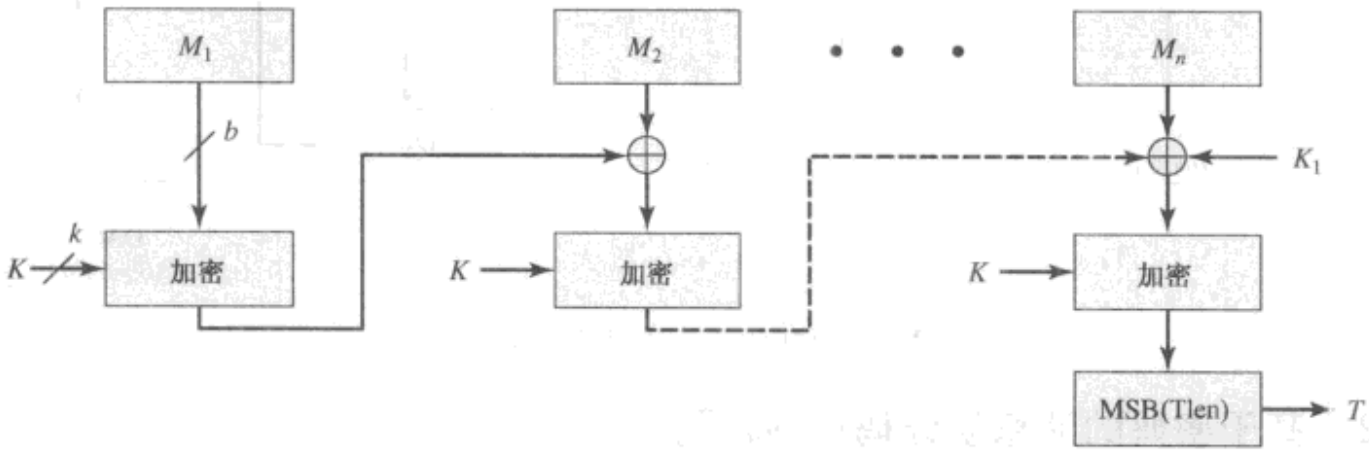
② 如果消息不是密文分组长度的整数倍,则最后分组的右边(低有效位)填充一个 1 和若干 0 使得最后的分组长度为 b。除了使用一个不同的 n 位密钥 K_2 代替 K_1 外,和前面所述的一样进行 CMAC 运算。

两个 n 位的密钥由 k 位的加密密钥按如下方式导出:

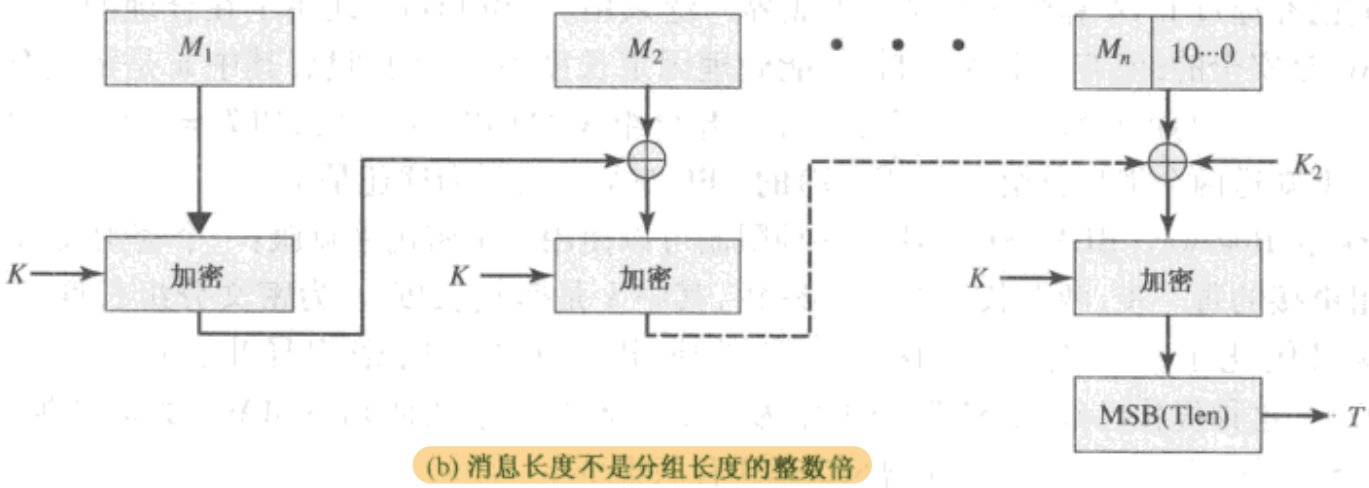
$$L = \mathrm{E}(K, 0^n)$$
$$K_1 = L \cdot x$$
$$K_2 = L \cdot x^2 = (L \cdot x) \cdot x$$

看不懂?

其中乘法(·)在域 GF(2^n)内进行,x 和 x^2 是域 GF(2^n)的一次和二次多项式。因此 x 的二元表示为 $n-2$ 个 0,后跟 10,而 x^2 的二元表示是 $n-3$ 个 0,后跟 100。有限域由不可约多项式定义,该多项式是那些具有极小非零项的多项式集合里按字典序排第一的那个多项式。对于已获批准的两个分组长度,多项式是 $x^{64} + x^4 + x^3 + x + 1$ 以及 $x^{128} + x^7 + x^2 + x + 1$。



(a) 消息长度是分组长度的整数倍



(b) 消息长度不是分组长度的整数倍

图 12.8 基于密码的消息认证码(CMAC)

为了生成 K_1 和 K_2,分组密码应用到一个全 0 分组上。第一个子密钥从所得密文导出,即先左移一位,并且根据条件和一个常数进行异或运算得到,其中常数依赖于分组的大小。第二个子密钥采用相同的方式从第 1 个子密钥导出。形式为 GF(2^n)的有限域的这种性质在第 5 章讨论列混淆时已经解释过。

12.7 认证加密:CCM 和 GCM

认证加密(AE)是指在通信中同时提供保密性和认证(完整性)的加密系统。许多应用和协议中都同时需要这两种形式的安全性保证,但这两类安全系统一直分离设计,直到近几年才合并在一起考虑。

[BLAC05]讨论了 4 种同时提供认证和加密的通用方案:

- **HtE:先 Hash 再加密**。对于 M 首先使用密码学 Hash 函数计算 $h = H(M)$,然后将消息和 Hash 值一起加密:$\mathrm{E}(K, (M \| h))$。
- **MtE:先 MAC 再加密**。使用两个密钥。首先通过计算 MAC 值 $T = \mathrm{MAC}(K_1, M)$ 对明文进

行认证，然后将消息和 MAC 一起加密：$E(K_2,(M \| T))$。SSL/TLS 协议（参见第 16 章）就使用该方案。

- **EtM：先加密再 MAC**。使用两个密钥。首先加密消息得到密文 $C = E(K_2,M)$，然后通过计算 MAC 值 $T = MAC(K_1,C)$ 并考查 (C,T) 对明文进行认证。IPsec 协议（参见第 19 章）就使用该方案。
- **E&M：加密并且 MAC**。使用两个密钥。加密消息得到密文 $C = E(K_2,M)$；通过计算 MAC 值 $T = MAC(K_1,M)$ 并考查 (C,T) 对明文进行认证。这两个步骤的先后顺序可以交换。SSH 协议（参见第 16 章）就使用该方案。

对于每种方案，可直接进行解密和验证。对于 HtE、MtE 和 E&M，要先解密，后验证。对于 EtM，要先验证，后解密。这些方案都存在有安全缺陷。HtE 方案在无线加密协议（WEP）中用于保护 WiFi 网络，该方案存在根本性的缺陷以至于 WEP 协议被取代。[BLAC05] 和 [BELL00] 指出了这三个加密/MAC 方案的安全性问题。当然，设计恰当的话，这些方案都能够提供高强度的安全性。这也是本节接下来将要介绍的两个方案的目标，这两个方案都是 NIST 的标准。

12.7.1　分组密码链接 - 消息认证码

CCM 工作模式由 NIST 作为标准提出，用于保护 IEEE 802.11 WiFi 无线局域网（参见第 17 章）的安全，当然也可用于任何需要认证和加密的网络应用。CCM 是 E&M 方案的改进，可提供认证加密。NIST SP 800-38C 有该规范。

组成 CCM 的关键算法是 AES 加密算法（参见第 5 章）、CTR 工作模式（参见第 6 章）和 CMAC 认证算法（参见 12.6 节），在加密和 MAC 算法中共用一个密钥 K。CCM 加密过程的输入包括三部分：

（1）将要被认证和加密的数据，即明文消息 P 数据块。

（2）将要被认证但不需加密的相关数据 A，例如在协议进行时，协议头必须以明文传递，但需要认证保护。

（3）临时量 N 作为负载和相关数据的补充，对于每条消息在协议生命期内，N 的取值唯一，这可以防止重放攻击或其他相应的攻击。

图 12.9 展示了 CCM 的工作过程。对于认证，输入包括临时量、相关数据和明文。输入部分被格式化为从 B_0 到 B_r 的分组，第一块包括临时量以及存储 N、A 和 P 长度信息的位，后面跟着 0 个或者多个包含 A 的分组，然后是 0 个或者多个包含 P 的分组。整个分组作为 CMAC 算法的输入，产生长度为 Tlen 的 MAC 值，其中 Tlen 小于或等于分组长度[参见图 12.9(a)]。

对于加密，计数器 CTR 产生与临时量无关的序列。认证码使用计数器 Ctr_0 以 CTR 模式加密，输出的高 Tlen 位与 MAC 异或后产生加密后的 MAC。剩余的计数器用于以 CTR 模式加密明文（参见图 6.7）。加密后的明文和加密后的 MAC 一起形成密文输出[参见图 12.9(b)]。

SP 800-38C 定义了如下认证/加密过程。

（1）使用格式函数将 (N,A,P) 格式化为分组 $B_0,B_1,\cdots,B_r$。

（2）令 $Y_0 = E(K,B_0)$。

（3）For $i=1$ to r, do $Y_i = E(K,(B_i \oplus Y_{i-1}))$。

（4）令 $T = MSB_{Tlen}(Y_r)$。

（5）执行计数器生成算法，产生计数器分组 $Ctr_0,Ctr_1,\cdots,Ctr_m$，其中 $m = \lceil Plen/128 \rceil$。

（6）For $j=0$ to m, do $S_j = E(K,Ctr_j)$。

(7) 令 $S = S_1 \| S_2 \| \cdots \| S_m$。

(8) 返回 $C = (P \oplus \text{MSB}_{\text{Plen}}(S)) \| (T \oplus \text{MSB}_{\text{Tlen}}(S_0))$。

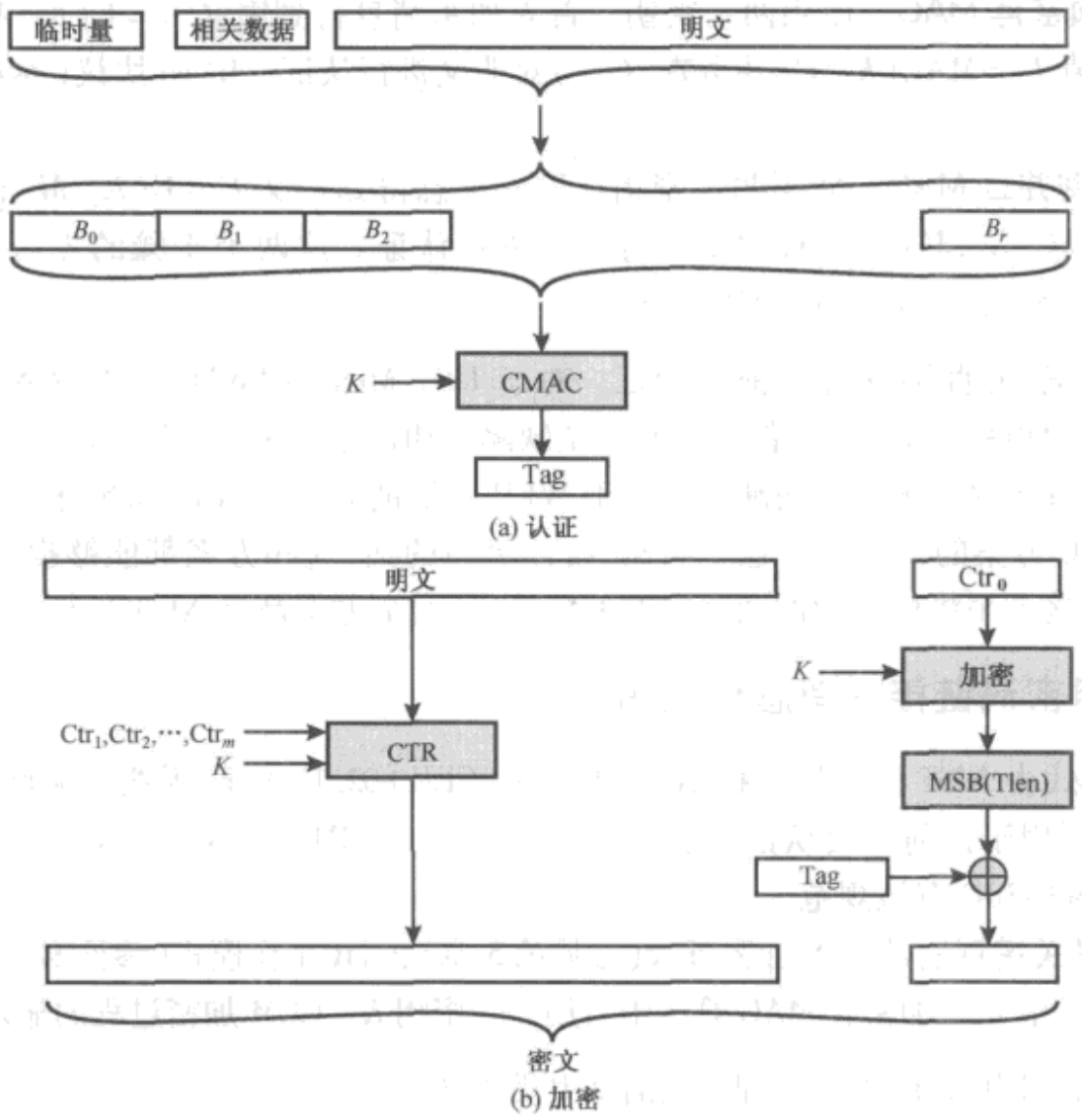


图 12.9 分组密码链-消息认证码(CCM)

对于解密和验证,接收方需要如下输入:密文 C,临时量 N,相关数据 A,密钥 K,以及初始计数器 Ctr_0。处理步骤如下:

(1) 如果 Clen≤ Tlen,则返回"无效"。

(2) 执行计数器生成算法,产生计数器分组 $\text{Ctr}_0, \text{Ctr}_1, \cdots, \text{Ctr}_m$,这里 $m = \lceil \text{Clen}/128 \rceil$。

(3) For $j = 0$ to m, do $S_j = \text{E}(K, \text{Ctr}_j)$。

(4) 令 $S = S_1 \| S_2 \| \cdots \| S_m$。

(5) 令 $P = \text{MSB}_{\text{Clen}-\text{Tlen}}(C) \oplus \text{MSB}_{\text{Clen}-\text{Tlen}}(S)$。

(6) 令 $T = \text{LSB}_{\text{Tlen}}(C) \oplus \text{MSB}_{\text{Tlen}}(S_0)$。

(7) 使用格式函数将(N, A, P)格式化为分组 $B_0, B_1, \cdots, B_r$。

(8) 令 $Y_0 = \text{E}(K, B_0)$。

(9) For $i = 1$ to r, do $Y_i = \text{E}(K, (B_i \oplus Y_{i-1}))$。

(10) 如果 $T \neq \text{MSB}_{\text{Tlen}}(Y_r)$,则返回"无效",否则返回 P。

CCM 是一个相对复杂的算法。注意对于明文需要两次完全的处理:一次用来生成 MAC 值,一次用来加密。另外,该标准还要求临时量和 MAC 的长度进行折中选择,这项限制并不必要。同时还要注意,加密密钥在 CTR 模式下使用了两遍:一次用来生成 MAC 值,一次用来加密明文和 MAC。这些增加的复杂操作是否增强了安全性还无定论。无论如何,关于算法的两个分析结果([JONS02]和[ROGA03])表明 CCM 提供了高强度的安全性。

12.7.2　Galois/计数器模式

Galois/计数器(GCM)工作模式由 NIST 作为 NIST SP 800-38D 标准提出,基于并行化设计,因此可以提供高效的吞吐率和低成本、低延迟。其本质是消息在变型的 CTR 模式下加密,密文结果与密钥以及消息长度信息在 GF(2^{128})域上相乘。该标准同时还制定了仅支持 MAC 的工作模式,即 GMAC。

GCM 模式使用两个函数:带密钥的 Hash 函数 GHASH,以及计数器每次增 1 的 CTR 模式的 GCTR。

$\text{GHASH}_H(X)$将 Hash 密钥 H 和位串 X 作为输入,其中 $\text{len}(X) = 128m$ 位,m 是正整数,输出是 128 位的 MAC 值。函数描述如下[参见图 12.10(a)]:

(1) 令 $X_1, X_2, \cdots, X_{m-1}, X_m$表示输入序列,其中输入串 $X = X_1 \| X_2 \| \cdots \| X_{m-1} \| X_m$。

(2) 令 Y_0为 128 位的 0,表示为 0^{128}。

(3) For $i = 1, \cdots, m$,令 $Y_i = (Y_{i-1} \oplus X_i) \cdot H$。其中 · 表示 GF($2^{128}$)域上的乘法。

(4) 返回 Y_m。

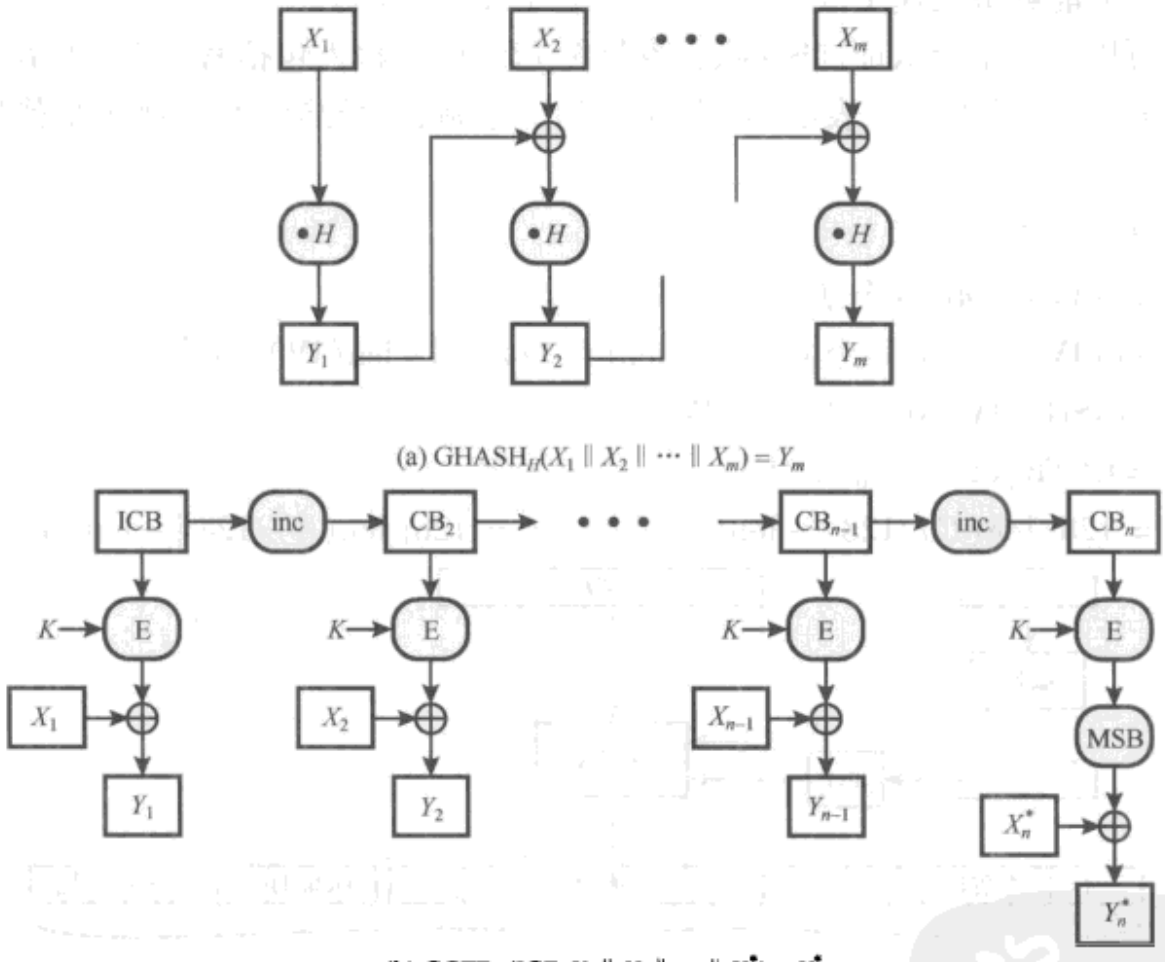


图 12.10　GCM 认证和加密函数

$\text{GHASH}_H(X)$函数可表示为

$$(X_1 \cdot H^m) \oplus (X_2 \cdot H^{m-1}) \oplus \cdots \oplus (X_{m-1} \cdot H^2) \oplus (X_m \cdot H)$$

上式非常适合快速实现。如果使用相同的 Hash 密钥认证多个消息,那么 $H^2, H^3, \cdots$能够通过一次预计算来对所有消息进行认证操作,并且待认证的数据分组($X_1, X_2, \cdots, X_m$)能够并行处理,因为每组运算都相互独立。

$\text{GCTR}_K(\text{ICB}, X)$将密钥 K 和任意长度的位串 X 作为输入,输出是长度与 X 相同的密文 Y。函数描述如下[参见图 12.10(b)]:

(1) 如果 X 是空串,则返回空串 Y。

(2) 令 $n = \lceil (\text{len}(X)/128) \rceil$,即 n 是大于等于 $\text{len}(X)/128$ 的最小整数。

(3) 令 $X_1, X_2, \cdots, X_{n-1}, X_n^*$ 表示输入序列,其中输入串

$$X = X_1 \| X_2 \| \cdots \| X_{n-1} \| X_n^*$$

其中 $X_1, X_2, \cdots, X_{n-1}$ 是 128 位完整分组。

(4) 令 $\text{CB}_1 = \text{ICB}$。

(5) For $i = 2$ to n,令 $\text{CB}_i = \text{inc}_{32}(\text{CB}_{i-1})$。其中 $\text{inc}_{32}(S)$ 函数对于 S 的最右 32 位增 1 并取模 2^{32},剩余位不变。

(6) For $i = 1$ to $n-1$, do $Y_i = X_i \oplus \text{E}(K, \text{CB}_i)$。

(7) 令 $Y_n^* = X_n^* \oplus \text{MSB}_{\text{len}(X_n^*)}(\text{E}(K, \text{CB}_n))$。

(8) 令 $X = X_1 \| X_2 \| \cdots \| X_{n-1} \| Y_n^*$。

(9) 返回 Y。

注意计数器值能够被快速生成,并且加密操作可以并行执行。

现在我们可以定义整个认证加密函数(参见图 12.11)。输入包括密钥 K、初始向量 IV、明文 P 以及附加认证信息 A。符号 $[x]_s$ 表示非负整数 x 的二进制表示的第 s 位。函数步骤如下:

(1) 令 $H = \text{E}(K, 0^{128})$。

(2) 定义分组 J_0 如下:

如果 $\text{len}(\text{IV}) = 96$,则令 $J_0 = \text{IV} \| 0^{31} \| 1$。

如果 $\text{len}(\text{IV}) \neq 96$,则令 $s = 128\lceil \text{len}(\text{IV})/128 \rceil - \text{len}(\text{IV})$,并令

$J_0 = \text{GHASH}_H(\text{IV} \| 0^{s+64} \| [\text{len}(\text{IV})]_{64})$。

(3) 令 $C = \text{GCTR}_K(\text{inc}_{32}(J_0), P)$。

(4) 令 $u = 128\lceil \text{len}(C)/128 \rceil - \text{len}(C)$,并令 $v = 128\lceil \text{len}(A)/128 \rceil - \text{len}(A)$。

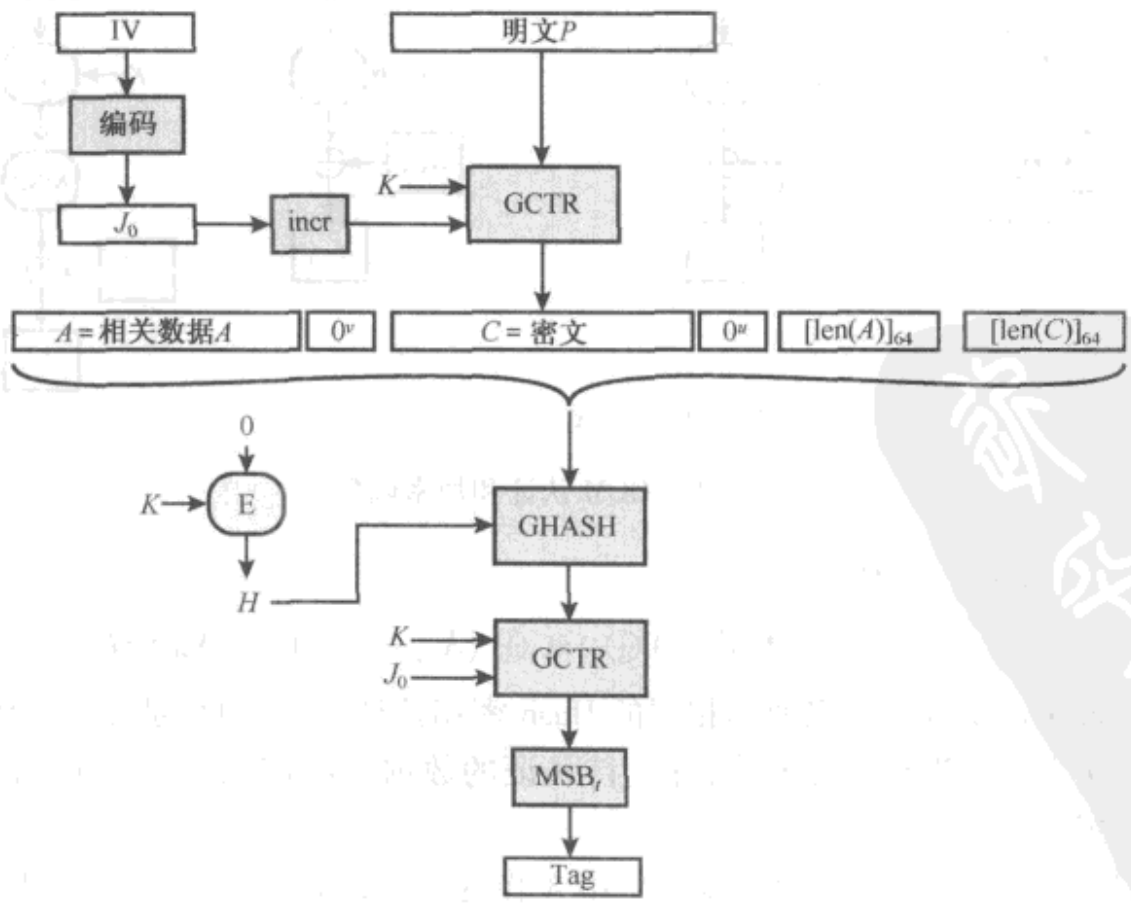


图 12.11 Galois/计数器 - 消息认证码(GCM)

(5) 定义分组 S 如下：

$$S = \text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64})。$$

(6) 令 $T = \text{MSB}_t(\text{GCTR}_K(J_0, S))$，这里 t 是支持的 MAC 长度。

(7) 返回 (C, T)。

在第 1 步中，通过使用密钥 K 加密 128 位的全 0 产生 Hash 密钥。在第 2 步中，由 IV 预生成计数器分组(J_0)，如果 IV 的长度是 96 位，则把 $0^{31} \parallel 1$ 作为填充串附在 IV 后面生成 J_0，否则 IV 被最少的 0 填充，直到长度被填充为 128 位(分组)的整数倍，结果再填充 64 个 0，再附加 64 位的 IV 长度信息，GHASH 对整个填充后的串计算得出 J_0。

然后 GCM 基于 CTR 工作模式，仅需认证时的附加认证信息和消息加入 MAC。用于计算 Hash 的函数仅是 Galois 域上的乘法。之所以这样设计，是因为执行 Galois 域上的乘法运算很容易，便于硬件实现[MCGR05]。

参考文献[MCGR04]讨论了分组密码工作模式，结果显示基于 CTR 的认证加密方案在高速网络中是最高效的工作模式。该文还进一步说明了 GCM 满足高的安全性要求。

12.8　使用 Hash 函数和 MAC 产生伪随机数

所有伪随机数发生器(PRNG)的关键组成包括随机种子值和生成伪随机位流的确定性算法。如果算法使用伪随机函数(PRF)来产生诸如会话密钥之类的值，那么要求仅有 PRF 的使用者知道种子。如果该算法用于产生流密码的加密密钥流，那么种子的作用是仅为发送方和接收方知道的密钥。

我们在第 7 章和第 10 章中讨论过这个话题，是因为一个加密算法产生的输出是随机的，可以作为构造 PRNG 的基础。类似地，Hash 函数和 MAC 的输出也是随机的，也能够用于构造 PRNG。ISO 标准 18031(随机位生成)和 NIST SP 800-90(使用确定性随机位生成器做随机数发生器的建议)都定义了使用密码学 Hash 函数进行随机数产生的方案。SP 800-90 还定义了基于 MAC 的随机数发生器。我们接下来分别介绍这两种方案。

12.8.1　基于 Hash 函数的 PRNG

图 12.12(a)展示了 SP 800-90 和 ISO 18031 中基于 Hash 的 PRNG 的基本方案。算法步骤如下：

V 为种子

seedlen 为 V 的位长度，要求 $V \geqslant k + 64$，这里 k 是需要的安全强度的长度

n 为需要的输出位数

算法使用产生 outlen 位长度 Hash 值的密码学 Hash 函数 H。算法基本操作如下：

$m = \lceil n/\text{outlen} \rceil$
data = V
W = the null string
For i = 1 to m
　　$w_i = H(\text{data})$
　　$W = W \parallel w_i$
　　$\text{data} = (\text{data} + 1) \bmod 2^{\text{seedlen}}$
Return leftmost n bits of W

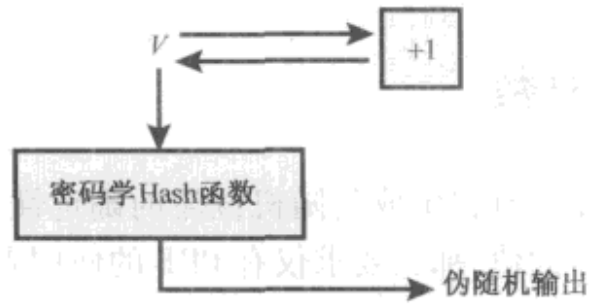
伪随机位流是 $w_1 \parallel w_2 \parallel \cdots \parallel w_m$，如果需要可将最后的块截断。

为了增加安全性,SP 800-90 标准还对 V 提供了周期性的更新。该标准还表示对于诸如 SHA-2 等强的密码学 Hash 算法,目前没有发现基于 Hash 方案的 PRNG 的已知或可疑缺陷。

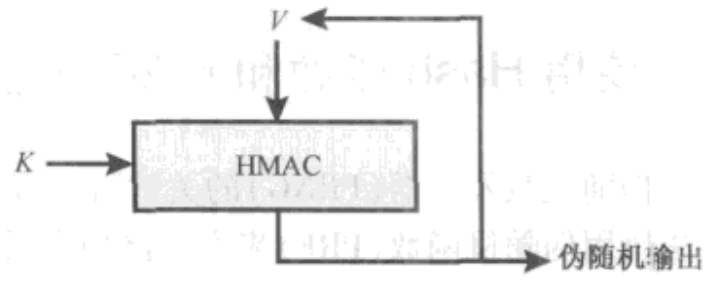
12.8.2 基于 MAC 的 PRNG

对于图 12.12(a)中描述的 PRNG,尽管没有发现基于密码学 Hash 算法的 PRNG 存在已知或可疑缺陷,但使用 MAC 可令我们更加放心。基于 MAC 的 PRNG 几乎都是使用 HMAC 构造的,这是因为 HMAC 是应用最广泛的标准 MAC 算法,在许多协议和应用中都已被实现。如 SP 800-90 中所述,与基于 Hash 的方案相比,其劣势是执行时间会增大一倍,这是因为 HMAC 对于每个输出块都要执行两次 Hash 函数运算。与基于 Hash 的方案相比,基于 HMAC 方案的优势是可以提供更高的安全生。

对于基于 MAC 的方案,需要两个输入:密钥 K 和种子 V。事实上,在 SP 800-90 中将 K 和 V 统称为种子。图 12.12(b)展示了 PRNG 机制的基本结构,图 12.13 的最左列说明了具体逻辑关系。注意,对于每个输出分组的密钥都是相同的,而每个分组的输入数据都等于前一分组的 MAC 值。为了增加安全性,SP 800-90 标准也提供了对 K 和 V 的周期性更新。



(a) 使用密码学Hash函数的PRNG



(b) 使用HMAC的PRNG

图 12.12 基于 Hash 的 PRNG 的基本结构(SP 800-90)

$m = \lceil n/\text{outlen} \rceil$ $w_0 = V$ W 为空字符串 For $i = 1$ to m $w_i = \text{MAC}(K, w_{i-1})$ $W = W \parallel w_i$ 返回 W 的左边 n 位	$m = \lceil n/\text{outlen} \rceil$ W 为空字符串 For $i = 1$ to m $w_i = \text{MAC}(K, (V \parallel i))$ $W = W \parallel w_i$ 返回 W 的左边 n 位	$m = \lceil n/\text{outlen} \rceil$ $A(0) = V$ W 为空字符串 For $i = 1$ to m $A(i) = \text{MAC}(K, A(i-1))$ $w_i = \text{MAC}(K, (A(i) \parallel V))$ $W = W \parallel w_i$ 返回 W 的左边 n 位
NIST SP 800-90	IEEE 802.11i	TLS/WTLS

图 12.13 基于 HMAC 的三个 PRNG

将一些应用中使用 HMAC 作为 PRNG 的方法与 SP 800-90 标准相比较是有益的,如图 12.13 所示。对于 IEEE 802.11i 无线局域网安全标准(参见第 17 章),输入数据包括种子和计数器的连接,对于每个输出分组 w_i 计数器依次递增。该方案与 SP 800-90 标准相比,似乎安全性更高。对于 SP 800-90 标准,对于输出分组 w_i 的输入数据恰好是前一个 w_{i-1} 执行 HMAC 的输出。因此攻击者能够通过观察伪随机输出,就知道 HMAC 的输入和输出。尽管如此,假设 HMAC 是安全的,知道输入和输出并不能恢复出 K,也就不能猜测出接下来的随机位。

传输层安全协议(参见第 16 章)和无线传输层安全协议(参见第 17 章)使用的方案包括每个输出分组 w_i 调用两次 HMAC。同 IEEE 802.11 一样,这样使得输出并不直接与输入相关。但双倍调用 HMAC 意味着操作负担也变为双倍,看起来有些过度了。

12.9　推荐读物和网站

[JUEN85]和[JUEN87]给出了消息认证的背景知识,集中讨论了密码学 MAC 和 Hash 函数。[BELL96a]和[BELL96b]给出了对 HMAC 的总体论述。

BELL96a Bellare, M.; Canetti, R.; and Krawczyk, H. "Keying Hash Functions for Message Authentication." *Proceedings, CRYPTO'96*, August 1996; published by Springer-Verlag. An expanded version is available at http://www-cse.ucds.edu/users/mihir.

BELL96b Bellare, M.; Canetti, R.; and Krawczyk, H. "The HMAC Construction." *CryptoBytes*, Spring 1996.

JUEN85 Jueneman, R.; Matyas, S.; and Meyer, C. "Message Authentication." *IEEE Communications Magazine*, September 1988.

JUEN87 Jueneman, R. "Electronic Document Authentication." *IEEE Network Magazine*, April 1987.

推荐网站

- **分组密码工作模式**:NIST 页面,有大量关于 CMAC、CCM 和 GCM 的信息。

12.10　关键术语、思考题和习题

关键术语

认证符
基于密码的消息认证码(CMAC)
数据认证算法(DAA)
Galois/计数器模式(GCM)
密码校验和
密码学 Hash 函数
消息认证码(MAC)
HMAC
消息认证
分组密码链—消息认证码(CCM)

思考题

12.1　消息认证是为了对付哪些类型的攻击?
12.2　消息认证或数字签名方法有哪两层功能?
12.3　产生消息认证有哪些方法?
12.4　对称加密和错误控制码一起用于消息认证时,这两个函数必须以何种顺序执行?
12.5　什么是消息认证码?
12.6　消息认证码和单向 Hash 函数之间的区别是什么?
12.7　为提供消息认证,应以何种方式保证 Hash 值的安全?
12.8　为了攻击 MAC 算法,必须要恢复密钥吗?
12.9　为了用一个 Hash 函数替代另一个 Hash 函数,HMAC 中需要进行哪些改变?

习题

12.1　如果 F 是错误检测函数,那么无论是用做内部函数还是用做外部函数(参见图 12.2),它都具有错误检测能力。如果被传输消息的任何位被改变,那么不管是在加密函数内或外执行 FCS,接收到

的 FCS 和计算出的 FCS 都将会不一致。有些编码还具有错误纠正能力,如果在传输中有一位或少数位被改变,则纠错码应含有足够的冗余信息以确定错误位并纠正之。显然,纠错码用在加密函数外时,它具有纠错能力。如果纠错码用在加密函数内,那么它具有纠错能力吗?

12.2 可以将 12.6 节中给出的数据认证算法定义为使用密文分组链接(CBC)方式的 DES 运算,其初始向量为 0(参见图 12.7)。证明通过密文反馈方式可以得出同样的结果。

12.3 在 12.6 节的开头,当给定一个单分组消息 X 的 CBC MAC 值 $T = \text{MAC}(K, X)$ 时,攻击者立即就可以知道两分组消息 $X \| (X \oplus T)$ 的 CBC MAC 值,因为该值仍然是 T。请证明上述结论。

12.4 本习题我们证明对于 CMAC,当最后一轮加密完成后再和第二个密钥异或的变异方法没有效果。我们就消息长度是分组长度的整数倍的情况考虑这个问题。变异方法可以表示为 $\text{VMAC}(K, M) = \text{CBC}(K, M) \oplus K_1$。假设攻击者可以获得三条消息的 MAC:消息 $\mathbf{0} = 0^n$,其中 n 是分组的长度;消息 $\mathbf{1} = 1^n$,以及消息 $\mathbf{1} \| \mathbf{0}$。结果,攻击者可以得到:$T_0 = \text{CBC}(K, \mathbf{0}) \oplus K_1$、$T_1 = \text{CBC}(K, \mathbf{1}) \oplus K_1$ 和 $T_2 = \text{CBC}(K, [\text{CBC}(K, \mathbf{1})]) \oplus K_1$。请证明攻击者可以计算出他没有提问过的消息 $\mathbf{0} \| (T_0 \oplus T_1)$ 的正确 MAC 值。

12.5 在讨论 CMAC 子密钥生成时,首先将分组密码应用到一个全 0 分组上。第一个子密钥从所得密文导出,即先左移一位,并且根据条件和一个常数进行异或运算得到,其中常数依赖于分组的大小。第 2 个子密钥是采用相同的方式从第 1 个子密钥导出的。

(a) 对于长度为 64 或 128 位的分组,需要何种常数?

(b) 说明如何进行左移和异或运算以得到结果。

12.6 12.6 节给出了三种实现认证加密的基本途径:MtE、EtM 和 E&M。

(a) CCM 使用的是哪种途径?

(b) GCM 使用的是哪种途径?

12.7 给出 GHASH 函数的计算过程:

$$(X_1 \cdot H^m) \oplus (X_2 \cdot H^{m-1}) \oplus \cdots \oplus (X_{m-1} \cdot H^2) \oplus (X_m \cdot H)$$

12.8 对于认证解密过程给出类似图 12.11 的流程图。

12.9 Alice 想通过长度为 2 的字发送单位信息(是或否)给 Bob。Alice 和 Bob 有 4 种可能的密钥来进行认证。下表中给出了基于每个密钥所发送的 2 位字:

	消 息	
密钥	0	1
1	00	01
2	10	00
3	01	11
4	11	10

(a) 上表对于 Alice 的操作是必需的,请为 Bob 构造类似的表格用于认证。

(b) 攻击者冒充 Alice 的成功概率是多少?

(c) 攻击者对截获的消息进行替代的成功概率是多少?

第13章 数字签名

13.1 数字签名
- 13.1.1 特征
- 13.1.2 攻击和伪造
- 13.1.3 数字签名需求
- 13.1.4 直接数字签名

13.2 ElGamal 数字签名方案

13.3 Schnorr 数字签名方案

13.4 数字签名标准
- 13.4.1 DSS 方法
- 13.4.2 数字签名算法

13.5 推荐读物和网站

13.6 关键术语、思考题和习题

To guard against the baneful influence exerted by strangers is therefore an elementary dictate of savage prudence. Hence before strangers are allowed to enter a district, or at least before they are permitted to mingle freely with the inhabitants, certain ceremonies are often performed by the natives of the country for the purpose of disarming the strangers of their magical powers, or of disinfecting, so to speak, the tainted atmosphere by which they are supposed to be surrounded.

—*The Golden Bough*, Sir James George Frazer

要 点

- 数字签名是一种认证机制，它使得消息的产生者可以添加一个起签名作用的码字。通过计算消息的 Hash 值并用产生者的私钥加密 Hash 值来生成签名。签名保证了消息的来源和完整性。
- 数字签名标准(DSS)是 NIST 标准，它使用安全 Hash 算法(SHA)。

数字签名是公钥密码学发展过程中最重要的概念之一，它可以提供其他方法难以实现的安全性。

图 13.1 是产生和使用数字签名过程的一般模型。Bob 可以使用数字签名生成算法对消息签名，生成算法的输入是消息和 Bob 的私钥。其他任何用户，如 Alice 能够使用验证算法来验证签名，验证算法输入时的消息、签名和 Bob 的公钥。图 13.2 说明了数字签名机制的本质。图 13.3 再次展示了这些关系。在本书的网站上有使用 RSA 实现的数字签名实例。

本章先简要介绍数字签名的有关内容，然后介绍数字签名标准(DSS)。

13.1 数字签名

13.1.1 特征

消息认证的局限性：

消息认证可以保护信息交换双方不受第三方的攻击，但是它不能处理通信双方自身发生的攻击，这种攻击可以有多种形式。

↓ explanation

例如，假定 John 使用图 12.1 中的某种方法给 Mary 发送一条认证消息。考虑下面两种情形：

(1) Mary 可以伪造一条消息并称该消息发自 John。Mary 只需产生一条消息，用 John 和 Mary 共享的密钥产生认证码，并将认证码附于消息之后。

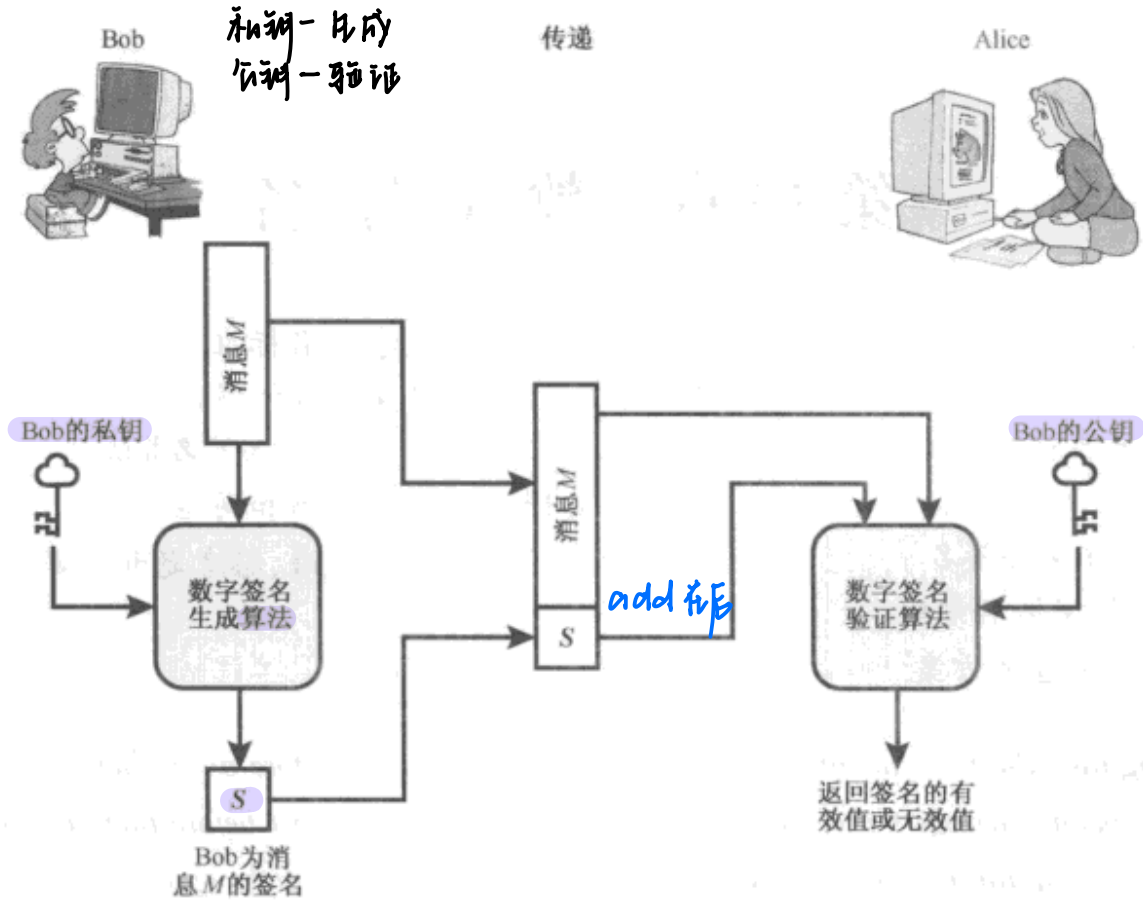


图 13.1 数字签名过程的一般模型

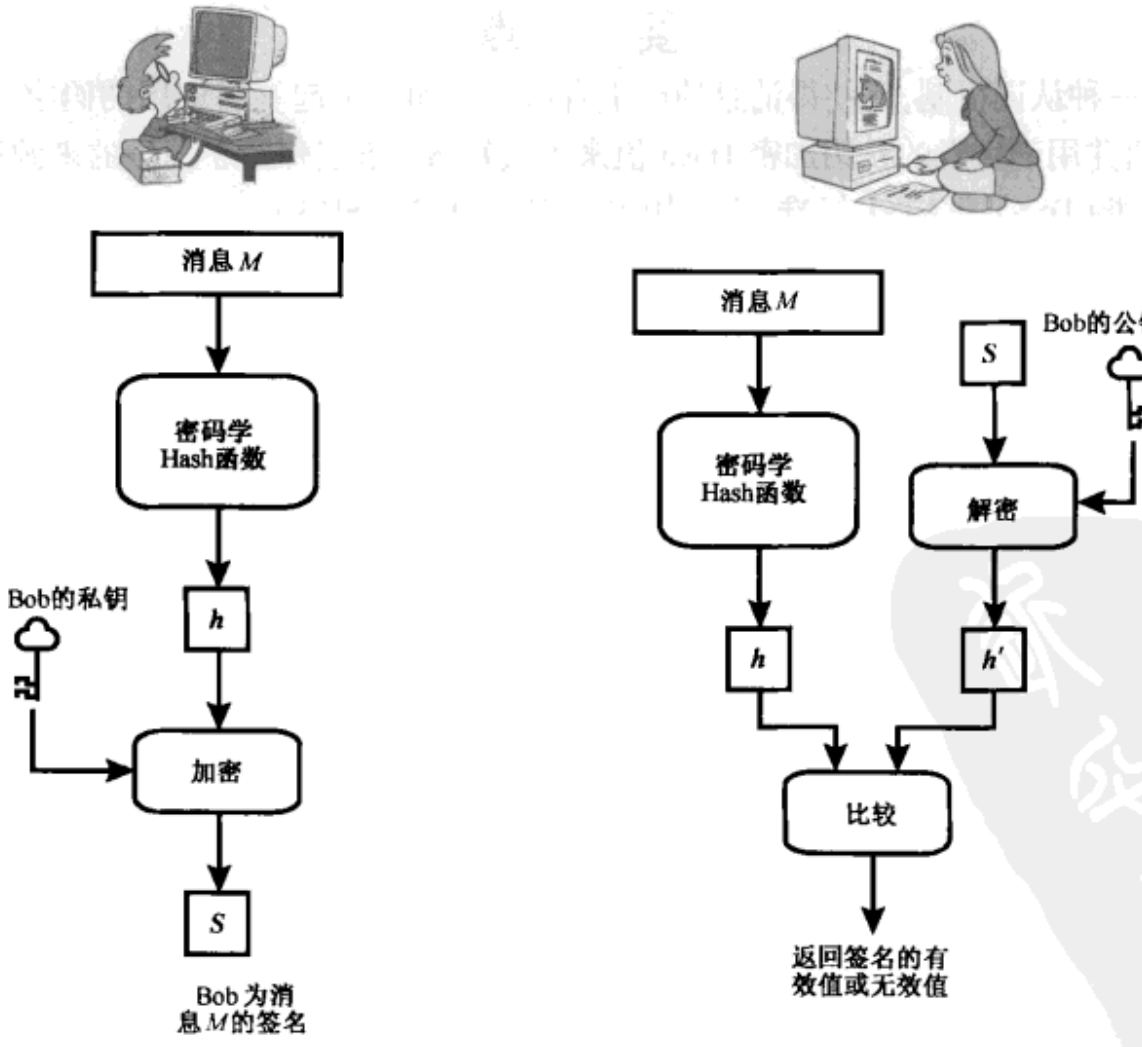


图 13.2 数字签名过程机制的简单描述

(2) John 可以否认曾发送过某条消息。因为 Mary 可以伪造消息，所以无法证明 John 确实发送过该消息。

这两种情形都是法律关注的。例如，对于第一种情形，在进行电子资金转账时，接收方可以增加转账资金，并声称这是来自发送方的转账资金额；对于第二种情形，股票经纪人收到有关电子邮件消息，要他进行一笔交易，而这笔交易后来赔钱了，但是发送方可以伪称从未发送过这条消息。

在收发双方不能完全信任的情况下，就需要除认证之外的其他方法来解决这些问题。数字签名是解决这个问题的最好方法。数字签名必须具有下列特征：

- 它必须能验证签名者、签名日期和时间。
- 它必须能认证被签的消息内容。
- 签名应能由第三方仲裁，以解决争执。

因此，数字签名具有认证功能。

13.1.2　攻击和伪造

[GOLD88]给出了危害程度从高到低排列的下列攻击类型。这里 A 代表其签名方案遭受攻击的用户，C 代表攻击者。

- **唯密钥攻击：**C 仅知道 A 的公钥。
- **已知消息攻击：**C 掌握一些消息以及对应的合法签名。
- **一般选择消息攻击：**在攻击 A 的签名方案之前，C 首先选择一些消息，而无须知道 A 的公钥。然后 C 对于这些选择的消息从 A 处获得合法签名。该攻击是一般性的，因为与 A 的公钥无关，同样的攻击可用于其他用户。
- **定向选择消息攻击：**同一般选择消息攻击类似，不同的是攻击者选择消息的时间是在 C 掌握 A 的公钥之后，在签名生成之前。
- **适应性选择消息攻击：**允许 C 将 A 作为“oracle”进行轮询。这意味着 A 可以被要求对于特定的消息签名，而这些消息与 C 之前已经获得的 < 消息-签名 > 对相关。

[GOLD88]还定义了何谓成功的攻击签名方案，即 C 能够以一定的概率进行下列攻击：

- **完全破译：**C 判断出 A 的私钥。
- **通用伪造：**C 掌握一个有效的签名算法，使得对于任意消息都能够等价地构造出合法签名。
- **选择伪造：**C 对于所选特定消息能够伪造出合法签名。
- **存在性伪造：**C 至少可以伪造出一个消息的合法签名，但 C 不能控制该消息的选择。这种伪造对于 A 的危害最低。

13.1.3　数字签名需求

根据刚才讨论的基本特征和攻击，数字签名应满足下列条件：

- 签名必须是与消息相关的二进制位串。
- 签名必须使用发送方某些独有的信息，以防伪造和否认。
- 产生数字签名比较容易。
- 识别和验证签名比较容易。
- 伪造数字签名在计算上是不可行的。无论是从给定的数字签名伪造消息，还是从给定的消息伪造数字签名，在计算上都是不可行的。

- 保存数字签名的副本是可行的。

安全 Hash 函数在基于图 13.2 所示的方案中被使用,提供了满足上述条件的基础。然而具体的方案细节要仔细设计。

13.1.4 直接数字签名

直接数字签名指只涉及通信双方(发送方和接收方)的数字签名方案。假定接收方已知发送方的公钥。

用共享的密钥(对称密码)对整个消息和签名加密,则可以获得保密性。注意这里是先进行签名,然后才执行外层的加密,这样在发生争执时,第三方可以查看消息及其签名。若先对消息加密,然后才对消息的密文签名,那么第三方必须知道解密密钥才能读取原始消息。但是签名若在内层进行,那么接收方可以存储明文形式的消息及其签名,以备将来解决争执时使用。

上述直接签名方法都有这样一个弱点,即这些方法的有效性依赖于发送方私钥的安全性。如果发送方想否认以前曾发送过某条消息,那么他可以称其私钥已丢失或被盗用,其他人伪造了他的签名。虽然在某种程度上这种威胁可能存在,但我们可以通过在私钥的安全性方面进行管理和控制来阻止或至少减少这种情况的发生。例如,可以要求每条要签名的消息都包含一个时间戳(日期和时间),以及在密钥被泄密后应立即向管理中心报告。

另一种可能的威胁是,X 的私钥可能在时刻 T 被盗用,但攻击者可用 X 的签名签发一条消息并加盖一个在 T 或 T 之前的时间戳。

被广泛接受的克服以上问题的方法是,使用数字证书的证书管理中心(CA)。我们将在第 14 章讨论这些话题,本章主要关注数字签名算法。

13.2 ElGamal 数字签名方案

在介绍 NIST 数字签名标准之前,首先介绍 ElGamal 和 Schnorr 签名方案。第 10 章中介绍的 ElGamal 加密方案能够使用用户的公钥进行加密,使用用户的私钥进行解密。ElGamal 数字签名方案则是使用私钥进行加密,使用公钥进行解密[ELGA84,ELGA85]。

在具体介绍之前,我们先复习数论的一个结论。回顾第 8 章中,对于素数 q,如果 α 是 q 的原根,则有

$$\alpha, \alpha^2, \cdots, \alpha^{q-1}$$

取模(mod q)后各不相同。如果 α 是 q 的原根,则进一步有

(1) 对于任意整数 m, $\alpha^m \equiv 1 \pmod{q}$ 当且仅当 $m \equiv 0 \pmod{q-1}$。

(2) 对于任意整数 i,j, $\alpha^i \equiv \alpha^j \pmod{q}$ 当且仅当 $i \equiv j \pmod{q-1}$。

同 ElGamal 加密方案一样,ElGamal 数字签名方案的基本元素是素数 q 和 α,其中 α 是 q 的原根。用户 A 通过如下步骤产生公钥/私钥对:

(1) 生成随机整数 X_A,使得 $1 < X_A < q-1$。

(2) 计算 $Y_A = \alpha^{X_A} \bmod q$。

(3) A 的私钥是 X_A;A 的公钥是 $\{q,\alpha,Y_A\}$。

为了对消息 M 进行签名,用户 A 首先计算 Hash 值 $m = H(M)$,这里 m 是满足 $0 \leqslant m \leqslant q-1$ 的整数。然后 A 通过如下步骤产生数字签名:

(1) 选择随机整数 K,使得满足 $1 \leqslant K \leqslant q-1$ 以及 $\gcd(K,q-1) = 1$,即 K 与 $q-1$ 互素。

(2) 计算 $S_1 = \alpha^K \bmod q$。注意这同 ElGamal 加密方案中 C_1 的计算相同。

(3) 计算 $K^{-1} \bmod (q-1)$，即计算 K 模 $q-1$ 的逆。

(4) 计算 $S_2 = K^{-1}(m - X_A S_1) \bmod (q-1)$。

(5) 签名包括 (S_1, S_2) 对。

任意用户 B 都能通过如下步骤验证签名：

(1) 计算 $V_1 = \alpha^m \bmod q$。

(2) 计算 $V_2 = (Y_A)^{S_1} (S_1)^{S_2} \bmod q$。

如果 $V_1 = V_2$，则签名合法。下面我们给出证明。假设等式成立，那么有

$\alpha^m \bmod q = (Y_A)^{S_1} (S_1)^{S_2} \bmod q$	假设 $V_1 = V_2$
$\alpha^m \bmod q = \alpha^{X_A S_1} \alpha^{KS_2} \bmod q$	代入 Y_A 和 S_1
$\alpha^{m - X_A S_1} \bmod q = \alpha^{KS_2} \bmod q$	等式左右移项
$m - X_A S_1 \equiv KS_2 \bmod (q-1)$	原根的性质
$m - X_A S_1 \equiv KK^{-1}(m - X_A S_1) \bmod (q-1)$	代入 S_2

例如，对于素数域 GF(19)，即 $q = 19$，如表 8.3 所示，其原根是 $\{2,3,10,13,14,15\}$。我们选择 $\alpha = 10$。

Alice 通过如下步骤产生密钥对：

(1) Alice 选择 $X_A = 16$。

(2) 则 $Y_A = \alpha^{X_A} \bmod q = \alpha^{16} \bmod 19 = 4$。

(3) Alice 的私钥是 16；Alice 的公钥是 $\{q, \alpha, Y_A\} = \{19, 10, 4\}$。

假设 Alice 要对 Hash 值 $m = 14$ 的消息进行签名：

(1) Alice 选择 $K = 5$，其与 $q - 1 = 18$ 互素。

(2) $S_1 = \alpha^K \bmod q = 10^5 \bmod 19 = 3$（参见表 8.3）。

(3) $K^{-1} \bmod (q-1) = 5^{-1} \bmod 18 = 11$。

(4) $S_2 = K^{-1}(m - X_A S_1) \bmod (q-1) = 11(14 - (16)(3)) \bmod 18 = -374 \bmod 18 = 4$。

Bob 能够通过如下步骤验证签名：

(1) $V_1 = \alpha^m \bmod q = 10^{14} \bmod 19 = 16$。

(2) $V_2 = (Y_A)^{S_1} (S_1)^{S_2} \bmod q = (4^3)(3^4) \bmod 19 = 5184 \bmod 19 = 16$。

于是签名是合法的。

13.3 Schnorr 数字签名方案

同 ElGamal 数字签名方案相同，Schnorr 数字签名方案也基于离散对数[SCHN89, SCHN91]。Schnorr 方案将生成签名所需的消息计算量最小化。生成签名的主要工作不依赖于消息，可以在处理器空闲时执行。生成签名过程与消息相关的部分需要进行 $2n$ 位长度的整数与 n 位长度的整数相乘。

该方案基于素数模 p，且 $p-1$ 包含大素数因子 q，即 $p - 1 \equiv 0 \pmod q$。一般取 $p \approx 2^{1024}$ 和 $q \approx 2^{160}$，即 p 是 1024 位整数，q 是 160 位整数，也正好等于 SHA-1 中 Hash 值的长度。

该方案的第一部分是生成公钥/私钥对，包括以下步骤：

(1) 选择素数 p 和 q，使得 q 是 $p-1$ 的素因子。

(2) 选择整数 α,使得 $\alpha^q = 1 \bmod p$。值 α、p 和 q 构成全局公钥参数,在用户组内的每个用户都可以取此值。

(3) 选择随机整数 s,$0 < s < q$,作为用户的私钥。

(4) 计算 $v = \alpha^{-s} \bmod p$,作为用户的公钥。

⑦ 对于私钥为 s、公钥为 v 的用户,通过如下步骤产生签名:

(1) 选择随机整数 r,$0 < r < q$,并计算 $x = \alpha^r \bmod p$。该过程与待签名消息 M 无关,可以在预处理过程计算。

(2) 将 x 附在消息后面一起计算 Hash 值 e:

$$e = \mathrm{H}(M \| x)$$

(3) 计算 $y = (r + se) \bmod q$。签名包括 (e, y) 对。

⑧ 其他用户能够通过如下步骤验证签名:

(1) 计算 $x' = \alpha^y v^e \bmod p$。

(2) 验证是否 $e = \mathrm{H}(M \| x')$。

对于该验证过程,有

$$x' \equiv \alpha^y v^e \equiv \alpha^y \alpha^{-se} \equiv \alpha^{y-se} \equiv \alpha^r \equiv x \pmod{p}$$

于是 $\mathrm{H}(M \| x') = \mathrm{H}(M \| x)$。

13.4 数字签名标准

美国国家标准与技术研究所(NIST)发布的联邦信息处理标准 FIPS 186,称为数字签名标准(DSS)。DSS 使用第 12 章中讨论的安全 Hash 算法(SHA),给出了一种新的数字签名方法,即数字签名算法(DSA)。DSS 最初提出于 1991 年,1993 年根据公众对于其安全性的反馈意见进行了一些修改。1996 年又稍做修改。2000 年发布了该标准的扩充版,即 FIPS 186-2,随后在 2009 年更新为 FIPS 186-3。这个最新版本还包括基于 RSA 和椭圆曲线密码的数字签名算法。本节我们讨论最初的 DSS 算法。

13.4.1 DSS 方法

DSS 使用的是只提供数字签名功能的算法。与 RSA 不同,DSS 虽然是一种公钥密码方案,但不能用于加密或密钥交换。

图 13.3 对用 DSS 产生数字签名和用 RSA 产生数字签名这两种方法进行了对比。在 RSA 方法中,Hash 函数的输入是要签名的消息,输出是定长的 Hash 码,用发送方的私钥将该 Hash 码加密形成签名,然后发送消息及其签名。接收方收到消息,计算 Hash 码。接收方用发送方的公钥对签名解密,如果计算出的 Hash 码与解密出的结果相同,则认为签名是有效的。因为只有发送方拥有私钥,所以只有发送方能够产生有效的签名。

DSS 方法也使用 Hash 函数,它产生的 Hash 码和为此次签名而产生的随机数 k 作为签名函数的输入,签名函数依赖于发送方的私钥($\mathrm{PR_a}$)和一组参数,这些参数为一组通信伙伴所共有,我们可以认为这组参数构成全局公钥($\mathrm{PU_G}$)①。签名由两部分组成,标记为 s 和 r。

接收方对接收到的消息产生 Hash 码,这个 Hash 码和签名一起作为验证函数的输入,验证函

① 对不同的用户,这些附加参数可以不同,以便它们是用户公钥的一部分。在实际应用中,更可能的情形是将全局公钥与每个用户的公钥分开使用。

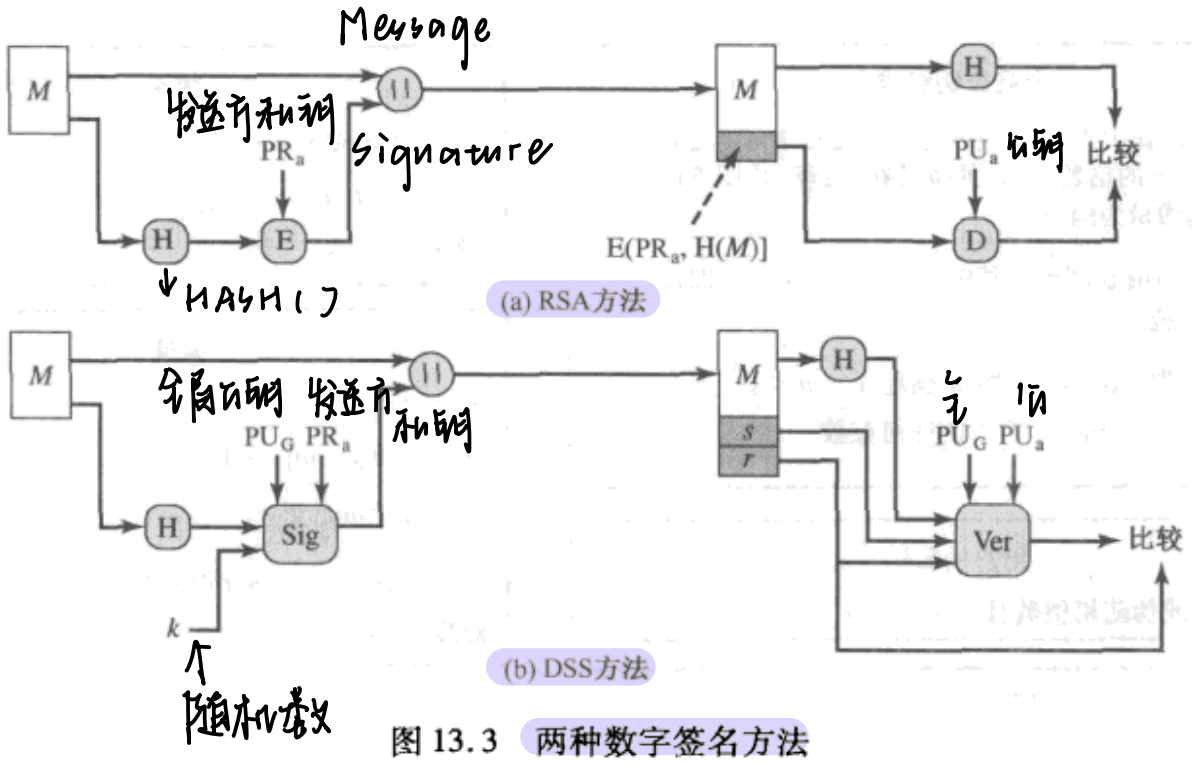


图 13.3　两种数字签名方法

数依赖于全局公钥和发送方公钥(PU_a),该发送方公钥同发送方的私钥组成密钥对。若验证函数的输出等于签名中的 r 成分,则签名是有效的。签名函数保证只有拥有私钥的发送方才能产生有效签名。

下面我们详细讨论数字签名算法。

13.4.2　数字签名算法

DSA 是建立在求离散对数之困难性(参见第 8 章)以及 ElGamal [ELGA85] 和 Schnorr [SCHN91] 最初提出的方法之上的。

图 13.4 归纳总结了 DSA 算法,其中有三个公开参数为一组用户所共有。选择一个 160 位的素数 q;然后选择一个长度在 512 ~ 1024 之间且满足 q 能整除 $(p-1)$ 的素数 p;最后选择形为 $h^{(p-1)/q} \bmod p$ 的 g,其中 h 是 1 到 $p-1$ 之间的整数,使得 g 大于①。DSA 的公开参数的选择与 Schnorr 签名方案完全一样。

选定这些参数后,每个用户选择私钥并产生公钥。私钥 x 必须是随机或伪随机选择的、位于 1 到 $q-1$ 之间的数,由 $y = g^x \bmod p$ 计算出公钥。由给定的 x 计算 y 比较简单,而由给定的 y 确定 x 则在计算上是不可行的,因为这就是求 y 的以 g 为底的模 p 的离散对数(参见第 8 章)。

要进行签名,用户需计算两个量 r 和 s。r 和 s 是公钥(p,q,g)、用户私钥(x)、消息的 Hash 码 $H(M)$ 和附加整数 k 的函数,其中 k 是随机或伪随机产生的,且 k 对每次签名是唯一的。

接收端用图 13.4 所示的公式进行验证。接收方计算值 v,它是公钥(p,q,g)、发送方公钥、接收到的消息的 Hash 码的函数。若 v 与签名中的 r 相同,则签名是有效的。

图 13.5 描述了 DSS 的签名和验证函数。

图 13.5 所示的算法有这样一个特点,接收端的验证依赖于 r,但是 r 却根本不依赖于消息,它是 k 和全局公钥的函数。k 模 p 的乘法逆元传给函数 f_1,f_1 的输入还包含消息的 Hash 码和用户私钥,函数的这种结构使接收方可利用其收到的消息和签名、它的公钥以及全局公钥来恢复 r。从图 13.4和图 13.5 不容易看出这种方法的正确性,本书附录 K 中给出了证明。

由于求离散对数的困难性,攻击者从 r 恢复出 k 或从 s 恢复出 x 都是不可行的。

① 在数论术语中,g 是 q 模 p 的阶(参见第 8 章)。

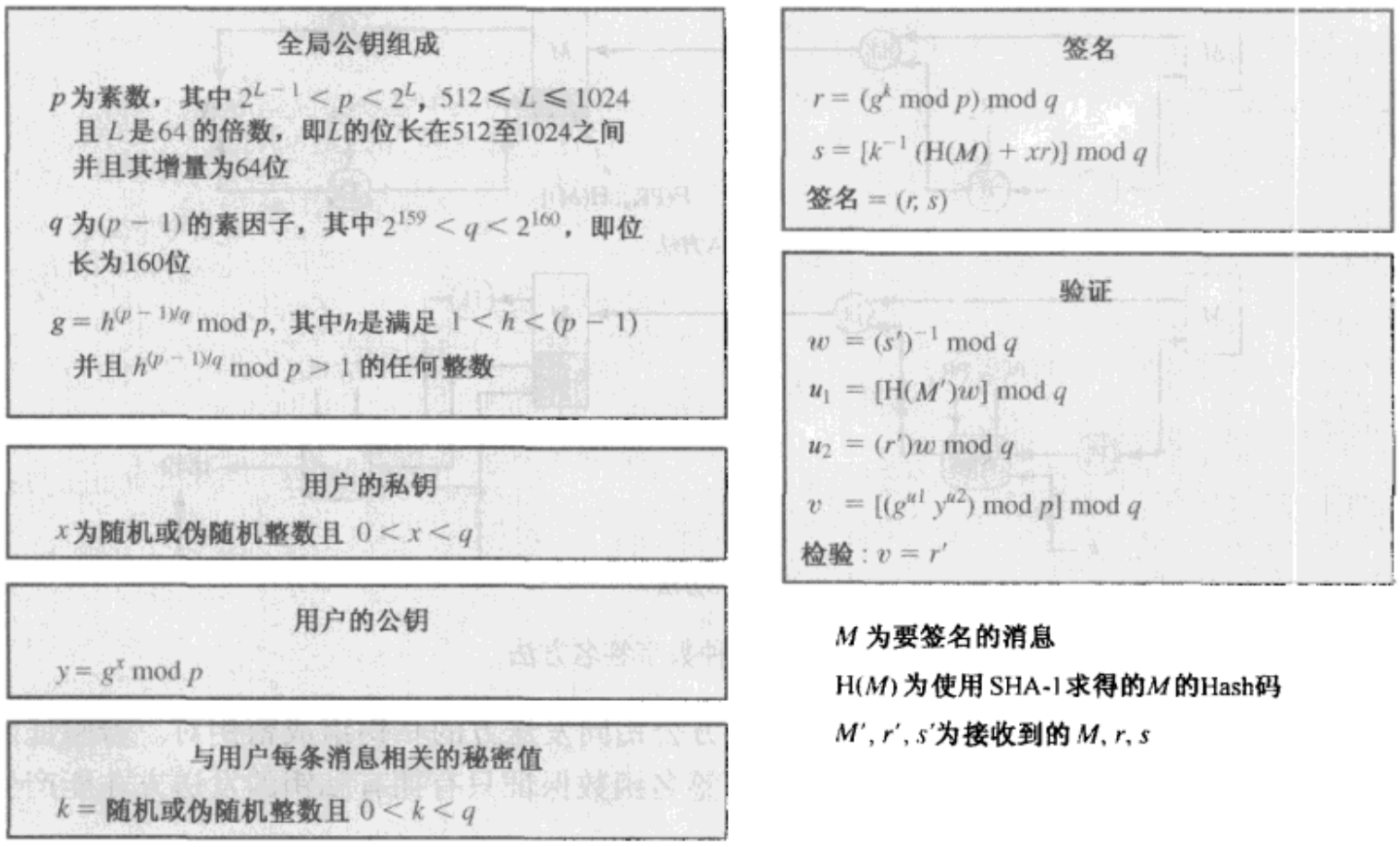


图 13.4 数字签名算法(DSA)

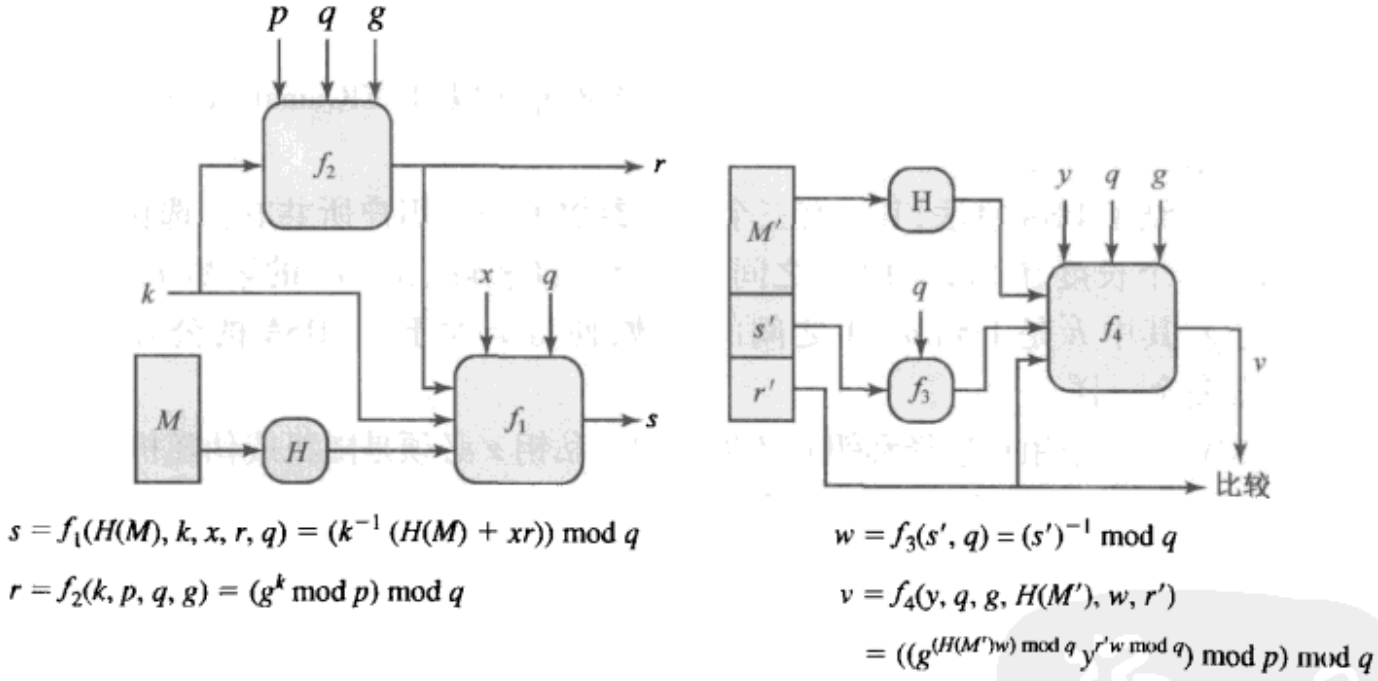


图 13.5 DSS 签名和验证

另一点需要注意的是，产生签名对指数运算 $g^x \bmod p$ 的计算要求很高，但由于它不依赖于被签名的消息，因此可以预先计算。实际上，用户甚至可以根据需要预先计算许多个用于签名的 r。其他要求比较高的任务是确定乘法逆元 k^{-1}。需再次提请注意的是，这些值中有许多是可以预先计算的。

13.5 推荐读物和网站

[AKL83]是关于数字签名的经典论文，仍有很高的影响力。[MITC92]是关于数字签名的优秀的综述文章。

AKL83 Akl, S. "Digital Signatures: A Tutorial Survey." *Computer*, February, 1983.
MITC92 Mitchell, C.; Piper, F.; and Wild, P. "Digital Signatures." In [SIMM92a].

推荐网站

Digital Signatures：NIST 网页，提供了已批准的数字签名选项的信息。

13.6　关键术语、思考题和习题

关键术语

直接数字签名
数字签名
数字签名算法(DSA)
数字签名标准(DSS)
ElGamal 数字签名
Schnorr 数字签名
时间戳

思考题

13.1　列出消息认证中出现的两种争议。
13.2　数字签名应该具有哪些性质？
13.3　数字签名应满足哪些要求？
13.4　直接数字签名和仲裁数字签名的区别是什么？
13.5　签名函数和保密函数应以何种顺序作用于消息？为什么？
13.6　直接数字签名方法中会遇到哪些威胁？

习题

13.1　华生医生耐心地等待福尔摩斯，直至福尔摩斯退出系统后才问："有什么有趣的问题需要解决吗？"
"噢，没有，我只是看一看邮件，并做了几个网络实验，而不是通常的化学实验。我现在只有一个客户，并且我已经解决了他的问题。我清楚地记得，你曾说过你也喜欢进行密码研究，所以你也许会感兴趣的。"
"我只是业余的密码学家，福尔摩斯，但我想我会感兴趣的。你说的是什么问题呢？"
"我的客户是 Hosgrave 先生，他是一家小银行的董事，他的银行已完全计算机化，当然也广泛地使用网络通信，他们使用 RSA 来保护数据，并对传输的文件进行数字签名。现在，他们打算修改一些过程，特别是，对某些文件需要两个签名者来签名，使得
(1) 第一签名者形成文件，对文件签名，并传送给第二签名者。
(2) 第二签名者首先验证该文件确实已由第一签名者签名，然后他将其签名加到文件中，任何接收者都可验证该文件确实是由两个签名者签过的文件，但只有第二签名者可验证步骤(1)中的签名。即接收方(或公众)仅可以验证具有两个签名的完整文件，而不是只有一个签名的中间文件。而且，银行希望利用现有的支持 RSA 数字签名的模块。"
"嗯，我知道如何将 RSA 用于只有一个签名者的数字签名中，福尔摩斯。我猜你已经适当推广了 RSA 数字签名并解决了 Hosgrave 先生的问题。"
"是的，华生。"福尔摩斯点头道，"RSA 数字签名是用签名者的私钥(d)加密产生的，任何人都可用公钥(e)解密来验证签名，验证签名 S 是由已知 d 的唯一的签名者产生的。对这个过程稍加推广，就可解决 Hosgrave 先生的问题，也就是说……"
请完成上述叙述。

13.2 DSA 中指出,如果签名产生过程中出现 $s=0$,则必须产生新的 k 并重新计算签名。为什么?

13.3 如果用于产生 DSA 签名的 k 已被泄密,则会出现什么问题?

13.4 DSS 包括一个推荐的素性测试算法,该算法如下:

(1) [**选择 w**] 令 w 是随机的奇数,则 $(w-1)$ 是偶数且可表示为 $2^a m$,其中 m 是奇数,也就是说,2^a 是整除 $(w-1)$ 的 2 的最大幂。

(2) [**产生 b**] 令 b 是随机整数,$1 < b < w$。

(3) [**求幂**] 置 $j=0$ 且 $z = b^m \bmod w$。

(4) [**完成?**] 若 $j=0$ 且 $z=1$ 或者 $z=w-1$,则 w 通过测试,可能是素数,转步骤(8)。

(5) [**终止?**] 若 $j>0$ 且 $z=1$,则 w 不是素数,对该 w 算法终止。

(6) [**增值 j**] 置 $j=j+1$。若 $j<a$,则置 $z=z^2 \bmod w$ 并转步骤(4)。

(7) [**终止**] w 不是素数,对该 w 算法终止。

(8) [**继续测试?**] 若已测试足够多的 b,则认为该 w 是素数并终止算法,否则转步骤(2)。

(a)说明该算法的工作原理。

(b)证明该算法与第 8 章中给出的 Miller-Rabin 测试是等价的。

13.5 因为 DSS 对每个签名产生一个 k,所以即使对同一消息签名,在不同的情况下签名也不相同,但 RSA 签名则不能做到这一点。这种区别有什么实际意义?

13.6 考虑 DSA 参数定义域生成问题。假设我们已经找到了参数 p 和 q,使得 $q|(p-1)$。现在我们需要寻找参数 $g \in Z_p$,使得 g 模 p 的阶为 q。考虑如下两种算法:

算法 1	算法 2
重复	**重复**
选择 $g \in Z_p$	**选择** $h \in Z_p$
$h \leftarrow g^q \bmod p$	$g \leftarrow h^{(p-1)/p} \bmod p$
直至($h=1$ 且 $g \neq 1$)	**直至**($g \neq 1$)
返回 g	**返回** g

(a) 证明算法 1 返回的参数值的阶为 q。

(b) 证明算法 2 返回的参数值的阶为 q。

(c) 假设 $p=40\ 193$,$q=157$。算法 1 需要经过多少轮循环才可以找到一个生成元?

(d) 如果 p 为 1024 位长,q 为 160 位长,你愿意推荐使用算法 1 来寻找 g 吗?为什么?

(e) 假设 $p=40\ 193$,$q=157$。算法 2 首次循环找到生成元的概率有多大?(如果有用,你可以使用 $\sum_{d|n} \varphi(d) = n$ 这一事实来回答该问题)。

13.7 在 Diffie-Hellman 算法的基础上,设计可用于数字签名的方法是很有意义的。下面的方法比 DSA 更简单,它需要私钥但不需要秘密的随机数。

公开量: q 素数

$\alpha\alpha < q$ 且α是 q 的本原根

私钥: $XX < q$

公钥: $Y = \alpha^X \bmod q$

要对消息 M 签名,则先计算该消息的 Hash 码 $h = H(M)$。我们要求 $\gcd(h, q-1) = 1$. 若 $\gcd(h, q-1)$不为 1,则将该 Hash 码附于消息后再计算 Hash 码,继续该过程直至产生的 Hash 码与 $(q-1)$ 互素;然后计算满足 $Z \times h \equiv X \bmod (q-1)$ 的 Z,并将α^Z作为对该消息的签名。验证签名即是验证 $Y = (\alpha^Z)^h = \alpha^X \bmod q$。

(a) 证明该体制能正确运行,即证明如果签名是有效的,那么在验证过程中将有上述等式成立。

(b) 给出一种简单的方法可以对任意消息伪造用户签名,以证明这种体制是不可接受的。

13.8 运用对称密码实现数字签名的早期方案基于如下叙述的内容:为了对 n 位的消息签名,发送者事

先随机产生 $2n$ 个 56 位的密钥。

$$k_1, K_1, k_1, K_2, \cdots, k_n, K_n$$

这些参数是保密的。发送者再相应地事先准备好两套 64 位的验证参数：

$$u1, U_1, u_2, U_2, \cdots, u_n, U_n \text{ 和 } v_1, V_1, v_2, V_2, \cdots, v_n, V_n$$

这两套参数公开，其中 $v_i = \mathrm{E}(k_i, u_i)$，$V_i = \mathrm{E}(k_i, U_i)$。

消息 M 按如下方式签名。对于消息的第 i 位，依赖该消息位的值为 0 或 1，将 k_i 或 K_i 附在消息上。例如，如果消息的前三位为 011，则签名的前三个密钥为 k_1, K_2, K_3。

(a) 接收方如何验证消息？

(b) 该方法安全吗？

(c) 一套保密密钥可以安全地为不同的消息签名多少次？

(d) 该方案会带来何种实际问题(如果存在的话)？

第四部分　相互信任

第 14 章　密钥管理和分发
第 15 章　用户认证

第 14 章　密钥管理和分发

14.1　对称加密的对称密钥分发
- 14.1.1　密钥分发方案
- 14.1.2　层次密钥控制
- 14.1.3　会话密钥生命周期
- 14.1.4　透明的密钥控制方案
- 14.1.5　分布式密钥控制
- 14.1.6　控制密钥的使用

14.2　非对称加密的对称密钥分发
- 14.2.1　简单密钥分发方案
- 14.2.2　确保保密性和身份认证的密钥分发方案
- 14.2.3　混合方案

14.3　公钥分发
- 14.3.1　公钥的公开发布
- 14.3.2　公开可访问的目录
- 14.3.3　公钥授权
- 14.3.4　公钥证书

14.4　X.509 认证服务
- 14.4.1　证书
- 14.4.2　X.509 版本 3

14.5　公钥基础设施
- 14.5.1　PKIX 管理任务
- 14.5.2　PKIX 管理协议

14.6　推荐读物和网站

14.7　关键术语、思考题和习题

No Singhalese, whether man or woman, would venture out of the house without a bunch of keys in his hand, for without such a talisman he would fear that some devil might take advantage of his weak state to slip into his body.

—*The Golden Bough*, Sir James George Frazer

John wrote the letters of the alphabet under the letters in its first lines and tried it against the message. Immediately he knew that once more he had broken the code. It was extraordinary the feeling of triumph he had. He felt on top of the world. For not only had he done it, had he broken the July code, but he now had the key to every future coded message, since instructions as to the source of the next one must of necessity appear in the current one at the end of each month.

—*Talking to Strange Men*, Ruth Rendall

要　点

- 密钥分发的功能是给想要交换安全加密数据的双方分发密钥，并提供密钥安全分发所需要的一些方法或者协议。
- 密钥分发常包括双方之间的频繁使用且长期存在的主密钥以及临时使用的会话密钥。
- 只有在公钥的可靠性可以保证时，公钥加密方案才是安全的。公钥证书保证公钥的安全性。
- X.509 为公钥证书定义了格式，该格式广泛用于各种各样的应用中。
- 公钥基础设施(PKI)被定义为由硬件、软件、人、策略和程序构成的一整套体系。这些程序用来创建、管理、存储、分发和撤销建立在非对称密码算法之上的数字证书。
- 典型地，PKI 的执行应用了 X.509 认证服务。

密钥管理和密钥分发很复杂，从对密码的、协议的和管理的角度进行了考虑。本章的目的是让读者意识到这些问题，并对密钥管理和分发的各个方面有一个大致的认识。若想了解更多的信息，请参照 NIST SP 800-57 第 3 卷以及本章结束时推荐的阅读列表。

14.1 对称加密的对称密钥分发

就对称加密来说，通信双方必须使用相同的密钥并且该密钥要对其他人保密，进而，如果攻击者在攻击密钥，为了减少攻击者攻陷密钥所危害的数据量，希望频繁地更换密钥。因此，任何密码系统的强度取决于密钥分发技术，即在想要交换数据的两者之间传递密钥且不被其他人知道的方法。对 A 和 B 来说，密钥的分发能以以下不同的方式得到：

（1）A 选择一个密钥后以物理的方式传递给 B。

（2）第三方选择密钥后物理地传递给 A 和 B。

（3）如果 A 和 B 先前或者最近使用过一个密钥，则一方可以将新密钥用旧密钥加密后发送给另一方。

（4）如果 A 和 B 到第三方 C 有加密连接，则 C 可以在加密连接上传送密钥给 A、B。

方式 1 和方式 2 需要人工交付一个密钥，对于链路加密来说，这是必需的，因为每个链路加密设备都只能与链路另一端的伙伴交换数据。然而，人工交付对于网络中的端到端加密是不实用的。在分布的系统中，任何给定的主机或终端都可能需要同时与很多其他主机以及终端交换数据，因此每个设备都需要动态提供大量的密钥。这个问题在大范围的分布系统中更加明显。

这个问题的规模依赖于需要支持的通信对的数目。如果端到端加密在网络中或者 IP 层执行，那么网络中每一对想要通信的主机都需要一个密钥，即若有 N 台主机，则需要的密钥数目为 $[N(N-1)]/2$。如果加密在应用层执行的话，每一对需要通信的用户或者进程都需要一个密钥，而一个网络可能有上百台主机，但却有上千个用户和进程。图 14.1 中举例说明了端到端加密的密钥分发任务的量级①。一个基于节点的网络有 1000 个节点，就需要分发大概 50 万个密钥，如果相同的网络支持 10 000 个应用，则在应用层加密就需要 5000 万个密钥。

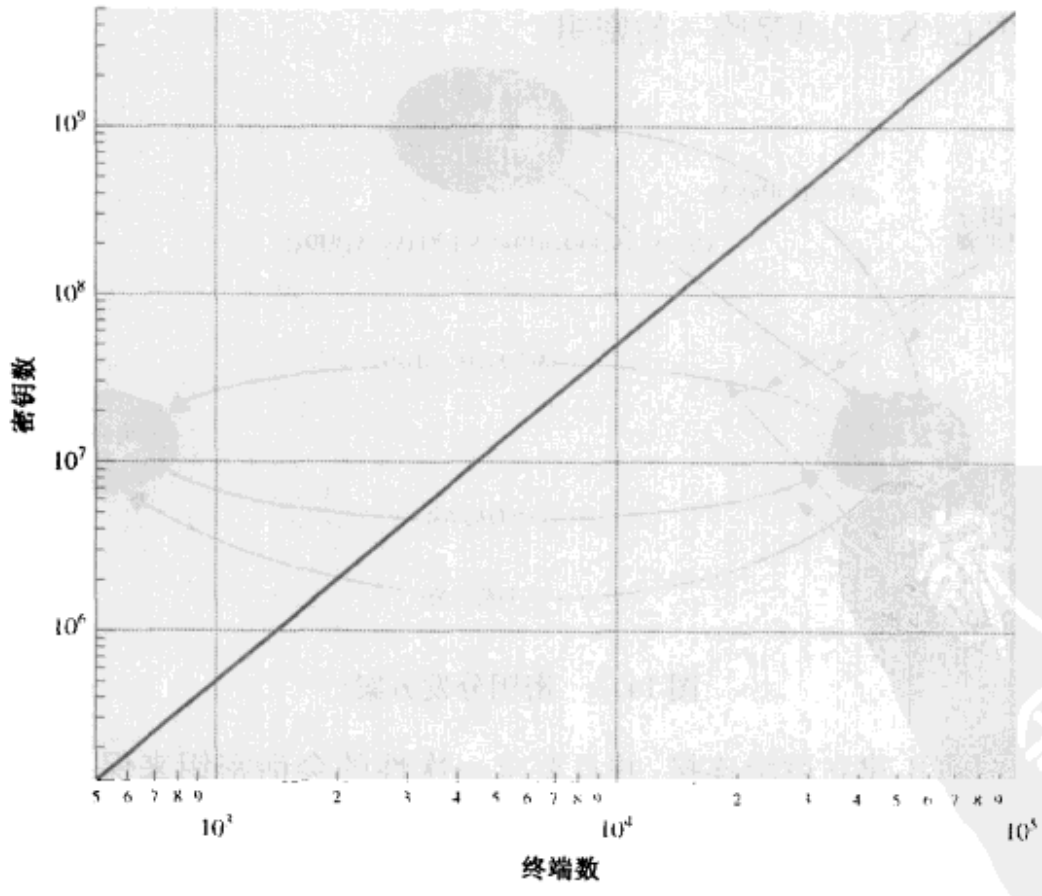


图 14.1　支持终端之间的任意连接所需要的密钥数

① 注意，该图使用对数刻度，于是线性图标代表指数增长。对数刻度的基本概述在计算机科学学生资源网站 William-Stallings. com/StudentSupport. html 上的数学复习资料中。

上面列出的密钥分发方式3可用于链接加密或者端到端加密。但是,如果攻击者成功地获得了一个密钥,则随后的密钥都会泄露,那么潜在的数百万密钥就必须进行重新分发。

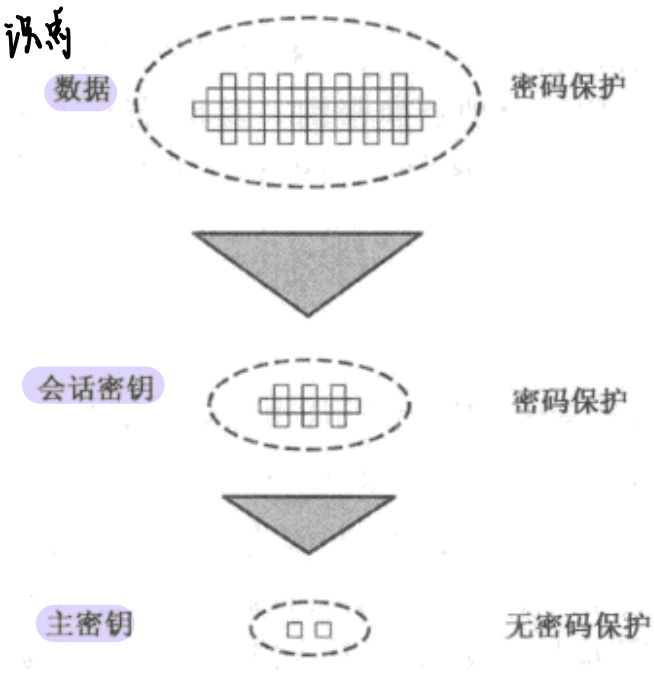


图 14.2 密钥层次的使用

对于端到端的加密,关于密钥分发方式4的很多变体已被广泛采用。在这种方案中,负责为用户(主机、进程或者应用)分发密钥的密钥分发中心是必需的,且为了密钥的分发,每个用户都需要和密钥分发中心共享唯一的一个密钥。

密钥分发中心是基于密钥层次体系的,最少需要两个密钥层(参见图14.2)。两个终端系统之间的通信使用临时密钥加密,这个临时密钥通常称为会话密钥。会话密钥往往被用于逻辑连接中,如帧的转发或传输连接,然后随着连接的断开而丢弃。终端用户通信所使用的会话密钥从密钥分发中心得到,因此,会话密钥可以用密钥分发中心与终端用户或者系统共有的主密钥加密后进行传送。

每一个终端系统或者用户和密钥分发中心共用唯一的主密钥。如果有 N 个实体想要逐对地通信,那么每次通信需要大概 $[N(N-1)]/2$ 个会话密钥,然而,这里只需要每个实体一个主密钥,即共 N 个。因此,主密钥的分发可以通过一些不加密的方式完成,如物理传递。

14.1.1 密钥分发方案

密钥分发可以用不同的方式实现,一种典型的方案如图14.3所示[POPE79]。该方案假设每个用户和密钥分发中心(KDC)共享唯一的密钥。

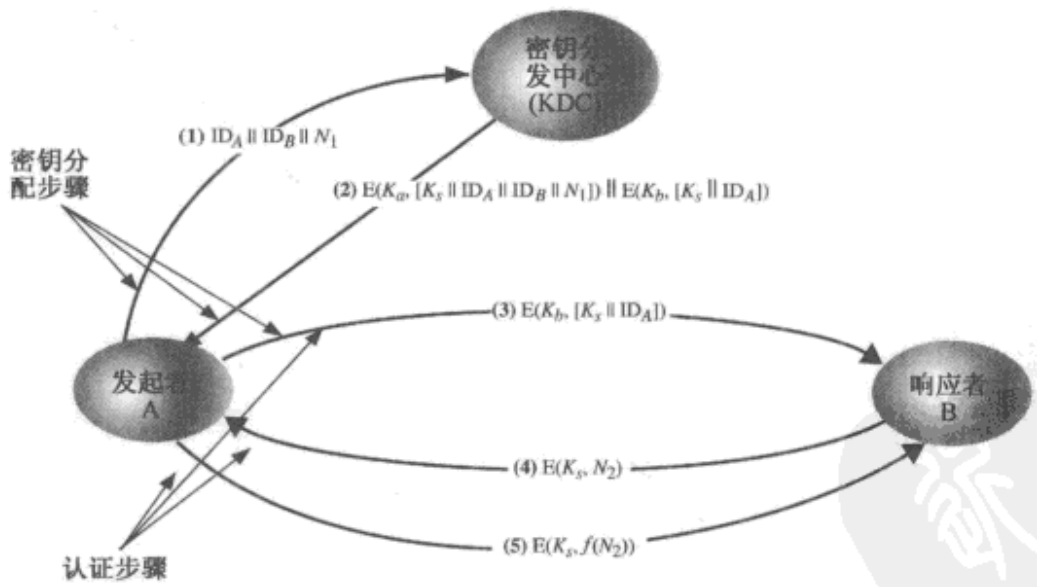


图 14.3 密钥分发方案

假设用户A想要和B建立逻辑连接,并且要求一次性的会话密钥来保护连接上传输的数据。A有主密钥 K_a(只有它自己和KDC知道),B有主密钥 K_b(和KDC共享),步骤如下:

(1) A向KDC发送包请求一个会话密钥来保护到B的逻辑连接的安全,其中包括A、B的身份以及该次传输的唯一标志 N_1,并称其为临时交互号(nonce)。这个临时交互号可以是一个时间戳、计数器或者一个随机数。最低要求是每次请求的临时交互号是不同的,而且为了防止假冒,还要求对手猜出该临时交互号是困难的,于是,用随机数作为临时交互号是一个好的选择。

(2) KDC 返回的信息是用 K_s 加密的，因此只有 A 能成功地读取该信息并且 A 知道是由 KDC 发来的。该信息中包含 A 想获取的两个部分：

- 用于会话的一次性会话密钥 K_s。
- 之前的请求信息，包括临时交互号，该信息使得 A 能够将这些返回信息和之前的请求相比较。

因此，A 可以知道它的原始信息在 KDC 收到之前是否被更改过，并且因为临时交互号的原因，可以知道先前的请求是否被重放。

另外，该信息中也包含 B 想获取的两个部分：

- 用于会话的一次性会话密钥 K_s。
- A 的标志（如 A 的网络地址）ID_A。

后面这两个部分由 K_b 加密，发送给 B，以达到建立连接并检验 A 的身份的目的。

(3) A 存储将要使用的会话密钥，把来源于 KDC 的信息发送给 B，即 $E(K_b, [K_s \| ID_A])$。因为这个消息是用 K_b 加密的，所以可以防止窃听。B 现在知道会话密钥为 K_s、想建立连接的另一方是 A（ID_A），以及该信息是由 KDC 加密的（因为是使用 K_b 加密的）。

此时，会话密钥已经安全地在 A 和 B 之间建立并且 A、B 可以使用其来保护通信。然而，还有另外两步要满足：

(4) B 使用新的会话密钥 K_s 加密临时交互号 N_2 并将结果发送给 A。

(5) 同样，A 使用 K_s 加密 $f(N_2)$ 后发给 B，其中 f 是一个对 N_2 进行变换的函数（如加 1）。

这一步能保证 B 在步骤(3)中收到的信息没有受到重放攻击。

注意，实际的密钥分发方案只包括步骤(1)至步骤(3)，步骤(3)、(4)、(5)执行认证功能。

14.1.2　层次密钥控制

对于单个的 KDC 没有必要限制其密钥分发功能，但是，对大型网络来说这样做是不实际的。于是，可以建立 KDC 的层次体系。例如有本地 KDC，每个本地 KDC 负责网络中的一个小区域，如一个局域网（LAN）或者一栋建筑物。由于同一个本地域中的各个实体要相互通信，故由本地 KDC 负责其密钥分发。如果两个实体在不同的域，则需要一个共享密钥，此共享密钥由两个本地 KDC 通过全局 KDC 协商产生。这种情况下，这三个 KDC 中的任意一个都可以选择密钥。层次的概念可以依据用户的规模以及内部网络的地理范围扩展到三层或者更多的层。

层次方案使得主密钥分发的开销最小化，因为大部分的主密钥是由 KDC 及本地域实体所共享的，而且，该方案将有缺陷的或者被破坏的 KDC 的危害限制在本地域中。

14.1.3　会话密钥生命周期

会话密钥更换得越频繁就越安全，因为在这种情况下，对于任一给定的会话密钥，攻击者拥有的密文会比较少。另一方面，会话密钥分发会延迟交换的开始时间，增加网络负担。安全管理者在决定特定会话密钥的生命周期时，必须平衡这些竞争代价。

对于面向对象的协议，在会话的整个生命周期中使用同一个会话密钥，为每一次新的会话使用新的会话密钥。如果一个逻辑连接的生命周期很长，则它需要周期性地改变会话密钥，协议数据单元（PDU）的序列号也随之重置。

对一个无连接协议，如面向事务（transaction）的协议，没有明确的连接初始和终止，因此不知道需要多长时间来更换一次密钥，最安全的方法是每次都使用新的会话密钥。然而，这也否定了

无连接协议最主要的一个优点,即最小化每次执行的开销和延时。一个比较好的策略是为特定时期或者特定数量的事务分配不同的会话密钥。

14.1.4 透明的密钥控制方案

图 14.3 提出的方法有很多变体,这里将讲述其中的一种(参见图 14.4)。这种方案以一种对终端用户透明的方式,对网络或者传输层提供点到点的加密。假定通信使用面向连接的端到端的协议如 TCP,该方法设定会话安全单元(SSM),SSM 包含协议层执行端到端加密的功能以及代表主机、终端获取会话密钥的功能。

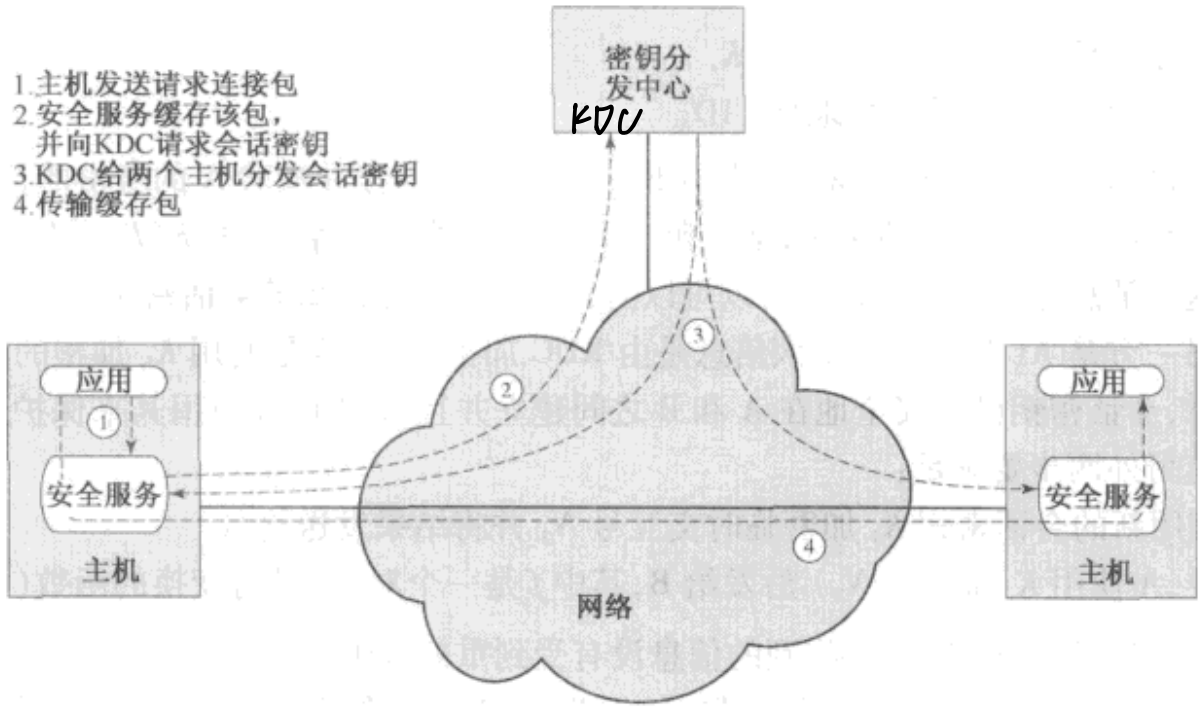


图 14.4 面向连接的密钥自动分发协议

建立连接的步骤如图 14.4 所示,当一台主机想要与另一台主机建立连接时,会发送一个连接建立请求包(步骤 1);SSM 保存该请求包,并向 KDC 申请建立连接(步骤 2),其中 SSM 和 KDC 之间的通信是由两者共享的主密钥加密的;如果 KDC 同意建立连接,则产生会话密钥并分发给双方的 SSM,分发的过程中使用 SSM 的主密钥加密(步骤 3);SSM 释放请求连接包,并在两个终端系统之间建立连接(步骤 4)。两个终端系统间所有用户数据的交互都需要各自的 SSM 使用会话密钥加密。

密钥自动分发方案使得终端用户访问主机或者在主机之间交换数据时更加灵活。

14.1.5 分布式密钥控制

密钥分发中心必须是可信的,而且是防破坏的,但如果密钥是完全分布式的,则无此要求。虽然对只使用对称加密的大型网络,完全分布式是不实用的,但是在局部环境下还是很有用的。

为了进行会话密钥的分发,分布式方法要求每个终端系统都能够以安全的方式与所有潜在的伙伴或系统进行通信。因此,配置 n 个终端的分布式密钥分发中心需要大概 $[n(n-1)]/2$ 个主密钥。

设置一个会话密钥大致需要以下几步(参见图 14.5):

(1) A 发送会话密钥请求给 B,其中包括一个临时交互号 N_1。

(2) B 用共享主密钥加密信息并回复给 A,该信息中包含 B 选择的会话密钥、B 的标志符、值 $f(N_1)$、临时交互号 N_2。

(3) 使用新的会话密钥,A 返回信息 $f(N_2)$ 给 B。

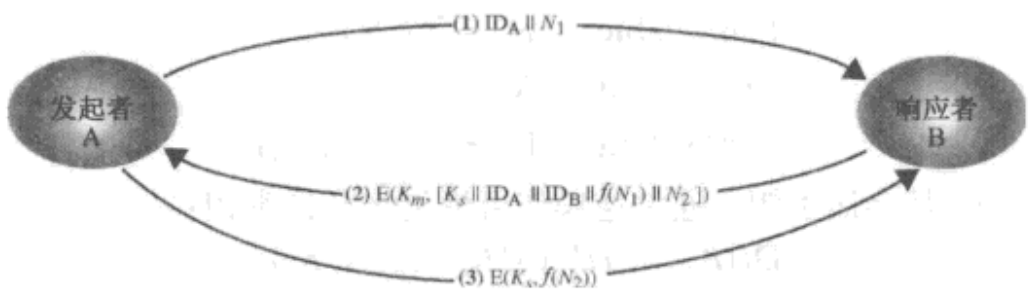


图 14.5　分布式密钥分发

每个节点至多拥有$(n-1)$个主密钥，与所需要的会话密钥一样多。信息传递使用主密钥的时间很短，因此密钥分析很困难。与之前一样，会话密钥用于在有限的时间内保护消息。

14.1.6　控制密钥的使用

密钥层次体系的概念和自动密钥分发技术的使用，大大减少了必须人工管理和分发的密钥数量。但是，仍然希望对自动分发密钥的应用进行控制。例如，除了划分主密钥和会话密钥外，还希望根据不同用途定义不同类型的会话密钥，如

- 数据加密密钥，用于网络中的通用通信
- PIN 加密密钥，用于电子资金转账和销售点应用的个人识别码(PIN)
- 文件加密密钥，用于可公开访问的加密文件

另外还需要考虑一个主密钥被作为数据加密密钥的风险。通常，主密钥在 KDC 和终端系统的硬件中是物理安全的，所以该主密钥加密的会话密钥是安全的。然而，如果主密钥曾作为会话密钥使用，则非授权的应用可能会获得以后通信中用该主密钥加密的会话密钥。

基于密钥关联的特征，在限制密钥使用方式的系统中添加控制是可行的。一个简单的设计是为每一个密钥关联一个标签([JONE82]，也见[DAVI89])，这里提出的技术用于 DES，每个 64 位的 DES 密钥中有 8 个非密钥(non-key)位，通常这是预留给奇偶校验位的，以构成密钥的标签。这些位位有以下解释：

- 1 位位表示该密钥是会话密钥还是主密钥
- 1 位位表示该密钥能否用于加密
- 1 位位表示该密钥能够用于解密
- 剩下的位位预留为将来使用

因为标签嵌入在密钥中，密钥分发时标签也一起被加密。这种方案的缺陷是：

(1) 标签的长度被限制为 8 位，限制了它的灵活性。

(2) 标签不能以清晰的格式传播，所以只能解密后使用，限制了对密钥使用方式的控制。

一个更加灵活的方案是控制向量，在[MATY91a and b]中有描述。在这种策略中，每个会话密钥有一个关联控制向量，该向量包含很多域，详细说明了会话密钥的用法和限制。控制向量的长度各不相同。

会话密钥在 KDC 中产生时，通过密码运算使得控制向量和密钥紧密地关联在一起。耦关联和去掉关联的过程如图 14.6 所示。第一步，控制向量通过 Hash 函数处理，产生一个长度和加密密钥长度相同的值。Hash 函数已在第 11 章详细讨论过。本质上，Hash 函数用一种合理的方式将数据从一个大的范围均匀地映射到一个小的范围中。例如，范围是 1 ~ 100 的数字被映射到 1 ~ 10 之间的数字，大约 10% 的原值被映射到一个目标值。

然后，将 Hash 值和主密钥异或得到一个输出，该输出作为输入密钥来加密会话密钥，即

$$\text{Hash value} = H = h(\text{CV})$$

$$\text{Key input} = K_m \oplus H$$

$$\text{Ciphertext} = \text{E}([K_m \oplus H], K_s)$$

其中，K_m 是主密钥，K_s 是会话密钥。通过逆操作可恢复出会话密钥：

$$\text{D}([K_m \oplus H], \text{E}([K_m \oplus H], K_s))$$

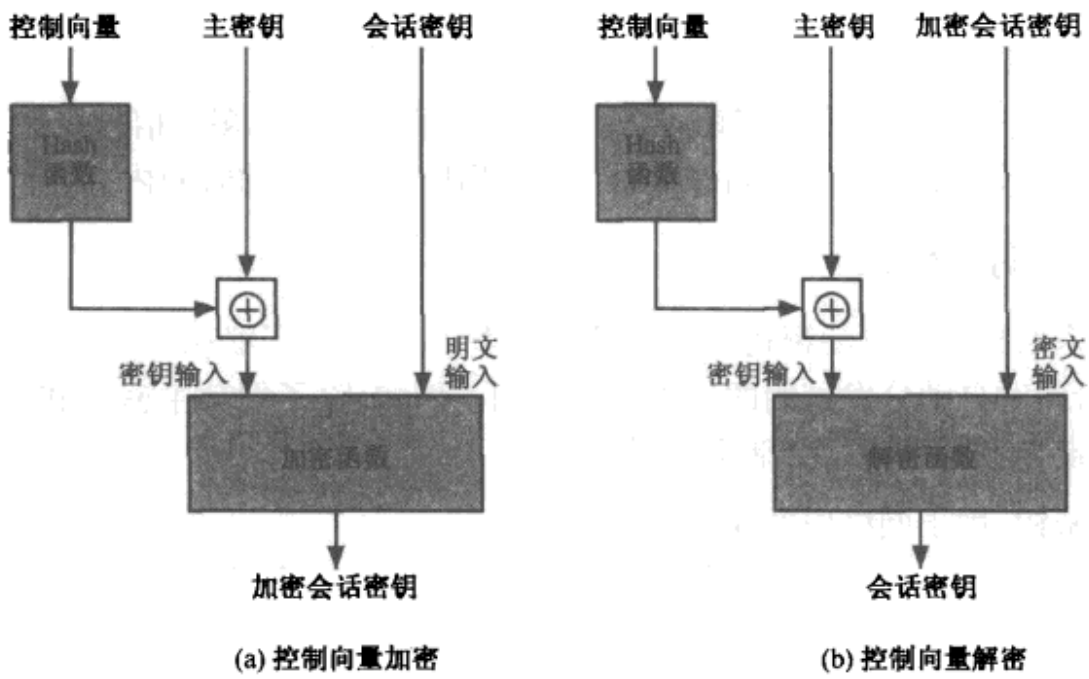


图 14.6 控制向量加密和解密

KDC 将会话密钥传递给用户时，伴随着一个明文形式的控制向量。恢复会话密钥必须同时使用用户与 KDC 共享的主密钥以及控制向量，即会话密钥与其控制向量之间的关联是被保持的。

与使用 8 位标签相比，使用控制向量有以下两个优点：第一，控制向量的长度没有限制，可以给密钥的使用添加任意复杂的控制；第二，在所有的操作处理中，都可以以明文形式使用控制向量。密钥控制向量可以在很多地方得到应用。

14.2 非对称加密的对称密钥分发

因为公钥加密系统的效率比较低，所以几乎不会用于大数据块的直接加密，而是经常用于小块数据的加密。公钥密码系统最重要的应用之一是用于密钥的加密分发，在本书第五部分将会介绍很多例子。这里，我们讨论一些通用的原则和典型的方法。

14.2.1 简单密钥分发方案

Merkle 在[MERK79]中提出了一种非常简单的方案，参见图 14.7。如果 A 想要和 B 通信，则需要以下几步：

(1) A 产生一个公私钥对$\{\text{PU}_a, \text{PR}_a\}$，然后发送包含 PU_a 和 A 的标志符 ID_A 的消息给 B。

(2) B 产生密钥 K_s，用 A 的公钥加密后发送给 A。

(3) A 计算 $\text{D}(\text{PR}_a, \text{E}(\text{PU}_a, K_s))$从而恢复密钥，因为只有 A 能解密该信息，故只有 A 和 B 知道 K_s。

(4) A 丢弃 PR_a、PU_a，B 丢弃 PU_a

A 和 B 可以使用会话密钥 K_s 加密以进行通信，在通信完成以后，A、B 都丢弃 K_s。尽管很简单，但也是一个很引人注目的协议。在通信开始之前和结束之后都不存在密钥，即密钥被攻破的风险也是最小的，同时通信是防窃听的。

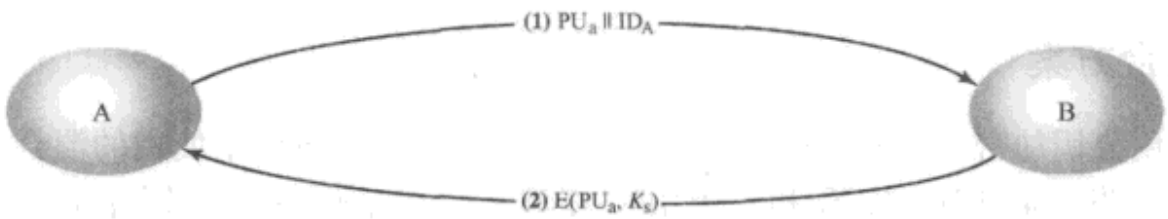


图 14.7　利用公钥加密建立会话密钥

当有敌手截获消息然后再转发消息或者替换为其他消息[参见图 1.3(c)]时，图 14.7 描述的协议是不安全的，这样的攻击称为中间人攻击[RIVE84]。这种情况下，如果敌手 E 已经控制了中间的通信信道，则可以通过以下方式在不被觉察的情况下控制通信：

（1）A 产生一个公私钥对$\{PU_a, PR_a\}$，后发送包含 PU_a 和 A 的标志符 ID_A 的消息给 B。
（2）E 截获信息后，创建自己的公私钥对$\{PU_e, PR_e\}$，发送 $PU_e \| ID_A$ 给 B。
（3）B 产生一个密钥 K_s，发送 $E(PU_e, K_s)$。
（4）E 截获信息后计算 $D(PR_e, E(PU_e, K_s))$，获取 K_s。
（5）E 发送 $E(PU_a, K_s)$给 A。

结果就是 A、B 知道了密钥 K_s 但没有意识到 E 也已经获得了密钥 K_s。此时，若 A 和 B 使用 K_s 交换信息，E 不再需要主动的干扰通信信道而只需要简单地窃听，因为知道 K_s，所以 E 可以解密所有的信息。因此，该协议存在中间人攻击的威胁。

14.2.2　确保保密性和身份认证的密钥分发方案

如图 14.8 所示，基于在[NEED78]中提出的方法，可以防止主动的和被动的攻击。我们首先假定 A 和 B 将通过本章随后要介绍的方案中的一种来交换公钥，以下是该方案的步骤：

（1）A 用 B 的公钥去加密包含 A 的标志符 ID_A 和一个临时交互号 N_1 的消息并发给 B，其中临时交互号被用来唯一地标志该次消息传递。
（2）B 用 PU_a 加密包含 A 的临时交互号 N_1 及 B 产生的新的临时交互号 N_2 的消息发送给 A。因为只有 B 可以解密消息(1)，故 N_1 在消息(2)中出现可以使 A 确信该消息来自于 B。
（3）A 使用的 B 的公钥加密后返回 N_2，使 B 可以确信消息来自于 A。
（4）A 选择密钥 K_s 后发送 $M = E(PU_b, E(PR_a, K_s))$ 给 B。用 B 的公钥加密确保只有 B 可以读取该消息，用 A 的私钥加密保证只有 A 可以发送该消息。
（5）B 计算 $D(PU_a, D(PR_b, M))$从而恢复密钥 K_s。

该方案可以保证交换密钥过程中的保密性和身份认证。

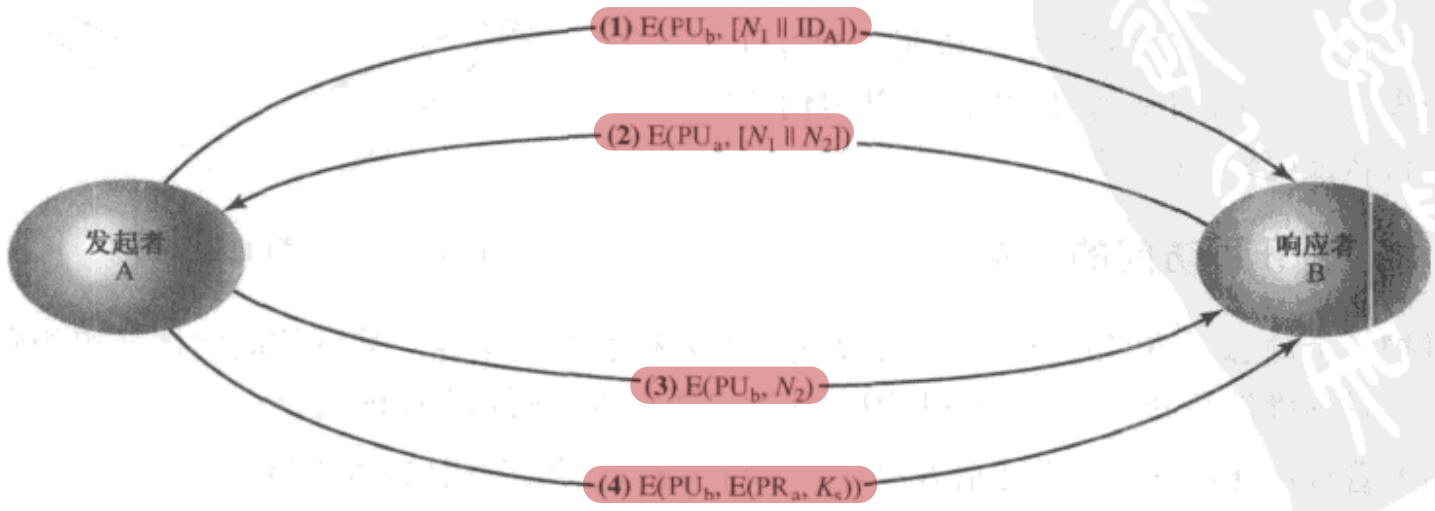


图 14.8　公钥加密的密钥分发

14.2.3 混合方案

另一种使用公钥加密来分发密钥的方法是在 IBM 大型机[LE93]中使用的混合方法。该方案仍会用到密钥分发中心 KDC,KDC 和每个用户共享一个主密钥,用主密钥加密要分发的会话密钥,公钥方案被用于分发主密钥。以下是这个三层方案的基本原理。

- **性能**:应用广泛,尤其是在会话密钥变动频繁的面向事务的应用中。用公钥加密的会话密钥分发会降低整个系统的性能,因为公钥加密和解密需要相对较高的计算负荷。对一个三层体系,公钥加密只用于偶尔更新用户和 KDC 之间的主密钥。
- **后向兼容性**:混合方案容易覆盖存在很小的破坏或者软件更改的 KDC 方案。

增加一个公钥层可以提供一个安全有效的分发主密钥的方式。这在一个单个的 KDC 要为一个庞大的用户集分发密钥的机构中是一个优势。

14.3 公钥分发

人们已经提出了集中公钥分配方法,所有这些方法本质上都可以归结为以下几种方法:

- 公开发布
- 公开可访问的目录
- 公钥授权
- 公钥证书

14.3.1 公钥的公开发布

表面上看,公钥密码的特点是公钥可以公开,因此如果有像 RSA 这样的为人们广泛接受的公钥算法,那么任一通信方可以将他的公钥发送给另一个通信方或广播给通信各方(参见图 14.9)。例如,越来越被人们广泛使用的 PGP(Pretty Good Privacy,将在第 18 章讨论)中使用了 RSA 算法,所以很多 PGP 用户在给诸如 USENET 新闻组和 Internet 邮件列表这样的一些公开论坛发送消息时,都将其公钥附加在要发送的消息之后。

虽然这种发送比较简便,但是有一个较大的缺点,即任何人都可以伪造这种公钥并公开发布,也就是说,某个用户可以假冒用户 A 并将一个公钥发送给通信的另一方或广播该公钥。在用户 A 发现这种假冒并通知其他用户之前,该假冒者可以读取本应发送给 A 的加密消息,并且可以用伪造的密钥进行认证(参见图 9.3)。

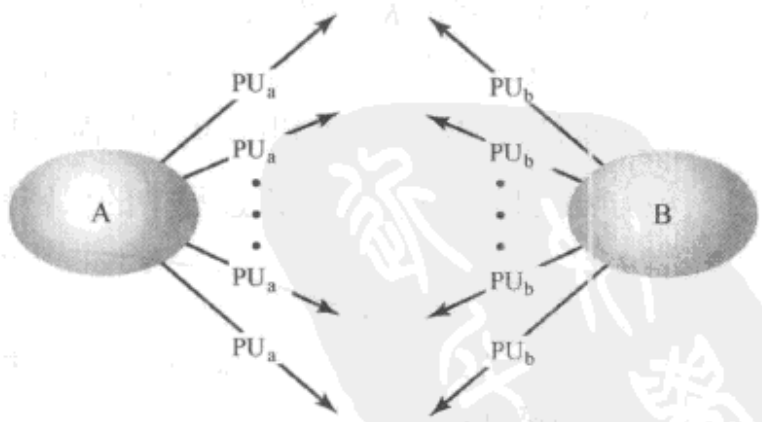


图 14.9 无控制的公钥分发

14.3.2 公开可访问的目录

维护一个动态可访问的公钥目录可以获得更大程度的安全性。某可信的实体或组织负责这个公开目录的维护和分配(参见图 14.10),这种方法包含下面几方面的内容:

(1) 管理员通过为每一个通信方建立一个目录项{姓名,公钥}来维护该目录。

(2) 每个通信方通过目录管理员来注册一个公钥。注册必须亲自或通过安全的认证通信进行。

(3) 通信方在任何时刻可以用新的密钥替代当前密钥。用户希望更换公钥,可能是因为公钥已用于大量的数据加密,也可能是因为相应的私钥已经泄露。

(4) 通信方也可以访问该目录,为实现这一目标,必须有从管理员到通信方的安全的认证通信。

这种方法显然比由个人公开发布公钥要安全,但是它也存在缺点。一旦攻击者获得或计算出目录管理员的私钥,则他可以假冒任何通信方,以窃取发送给该通信方的消息。另外,攻击者也可以通过修改目录管理员保存的记录来达到这一目的。

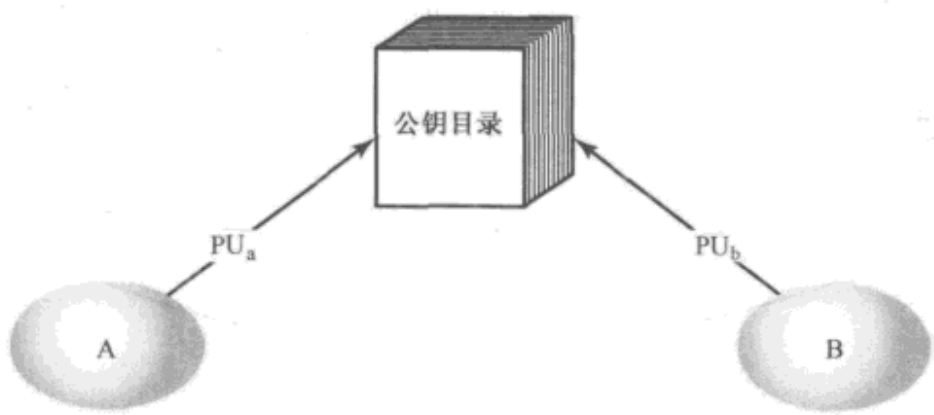


图 14.10　公开的公钥发布

14.3.3　公钥授权 (有一个公钥管理机构)

通过更加严格地控制目录中的公钥分配,可使公钥分配更加安全。图 14.11 举例说明了一个典型的公钥分配方案,它基于[POPE79]中给出的图形。像之前一样,该方案中假定中心管理员负责维护通信各方公钥的动态目录,除此之外,每一通信方可靠地知道该目录管理员的公钥,并且只有管理员知道相应的私钥。这种方案包含以下步骤(与图 14.11 中的序号相对应):

(1) A 发送一条带有时间戳的消息给公钥管理员,以请求 B 的当前公钥。

(2) 管理员给 A 一条用其私钥 PR_{auth} 加密的消息,这样 A 就可用管理员的公钥对接收到的消息解密,因此 A 可以确信该消息来自管理员。这条消息包含以下内容:

- B 的公钥 PU_b,A 用来加密发给 B 的消息。
- 原始请求,这样 A 可以将该请求与其最初发出的请求进行比较,以确保在管理员收到请求之前,其原始请求未被修改。
- 原先的时间戳,A 可以确定它收到的不是来自管理员的旧消息。

(3) 存储 B 的公钥,并用它去加密包含 A 的身份标志符 ID_A 和临时交互号 N_1 的消息发送给 B,其中 N_1 唯一标志该次交互。

(4,5) 与 A 检索 B 的公钥一样,B 使用同样的方法从管理员处得到 A 的公钥。

此时,公钥已被安全地传递给 A 和 B,A、B 之间的信息交换将会受到保护。尽管如此,最好还包含以下两步:

(6) B 用 A 的公钥 PU_a 加密包含 A 的临时交互号 N_1 和 B 新产生的临时交互号 N_2 的消息,并发送给 A。因为只有 B 能解密消息(3),所以消息(6)中 N_1 可以使 A 确信该消息来自于 B。

(7) A 用 B 的公钥加密包含 N_2 的消息给 B,这样 B 就可以知道该消息来自于 A。

这样,总共需要 7 条消息,然而,前面的 4 条消息不会被频繁使用,因为 A、B 可以存储彼此的公钥以备将来之用(这种方法称为暂存)。用户需要周期性地请求当前公钥信息,以保证通信中使用的是当前的公钥。

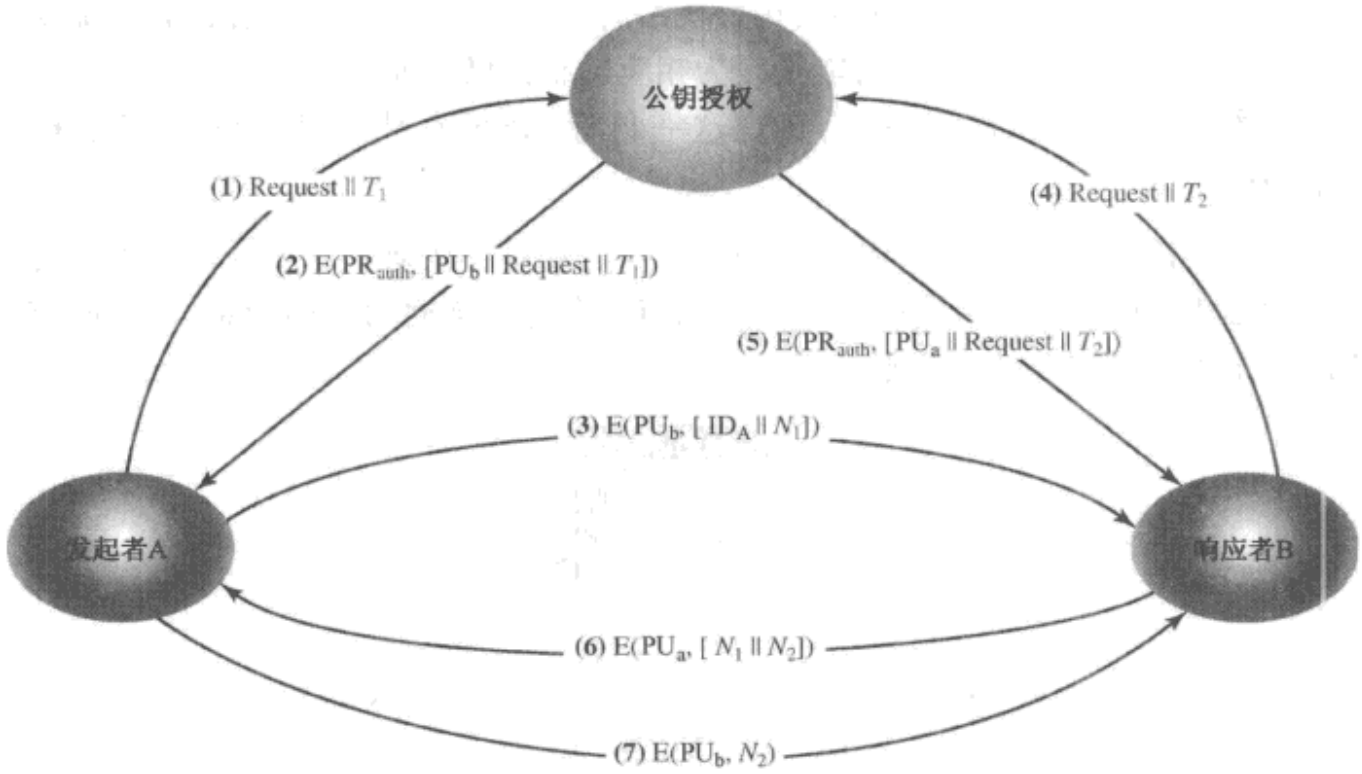


图 14.11 公钥发布方案

14.3.4 公钥证书

图 14.11 的方案虽然不错,但是它还有缺陷。因为用户要与其他用户通信,他就必须向目录管理员申请对方的公钥,因此公钥管理员就会成为系统的瓶颈。像前面一样,管理员所维护的含有姓名和公钥的目录也容易被篡改。

最早由 Kohnfelder 提出了使用证书的方法可参阅[KOHN78]。通信双方使用证书来交换密钥而不是通过公钥管理员。在某种意义上,这种方案与直接从公钥管理员处获得密钥的可靠性相同。证书包含公钥和公钥拥有者的标志,整个数据块由可信的第三方进行签名。通常,第三方是证书管理员,如政府机构或者金融机构,为用户群所信任。一个用户以一种安全的方式将他的公钥交给管理员,从而获得一个证书,接着用户就可以公开证书。任何需要该用户公钥的人都可以获得该证书,并通过查看附带的可信签名来验证证书的有效性。通信的一方也可以通过传递证书的方式将他的密钥信息传达给另一方。其他通信各方可以验证该证书确实是由证书管理员生成的。这种方法应满足下列要求:

(1) 任何通信方可以读取证书并确定证书拥有者的姓名和公钥。

(2) 任何通信方可以验证该证书出自证书管理员,而不是伪造的。

(3) 只有证书管理员可以产生并更新证书。

[KOHN78]中最初提出的方法能满足上述条件,后来的 Denning[DENN83]增加了下列要求:

(4) 任何通信方可以验证证书的时效性。

图 14.12 举例说明了证书的使用方法。通信方向证书管理员提供一个公钥并请求证书。申请必须是当事人亲自或通过某种安全的认证通信提出的,对于申请者 A,管理员提供如下形式的证书:

$$C_A = E(PR_{auth}, [T \| ID_A \| PU_a])$$

其中,PR_{auth}是证书管理员私钥,T 为时间戳。A 可以把该证书传递给任何其他参与者,后者通过以下方式读取和验证证书:

$$D(PU_{auth}, C_A) = D(PU_{auth}, E(PR_{auth}, [T \| ID_A \| PU_a])) = (T \| ID_A \| PU_a)$$

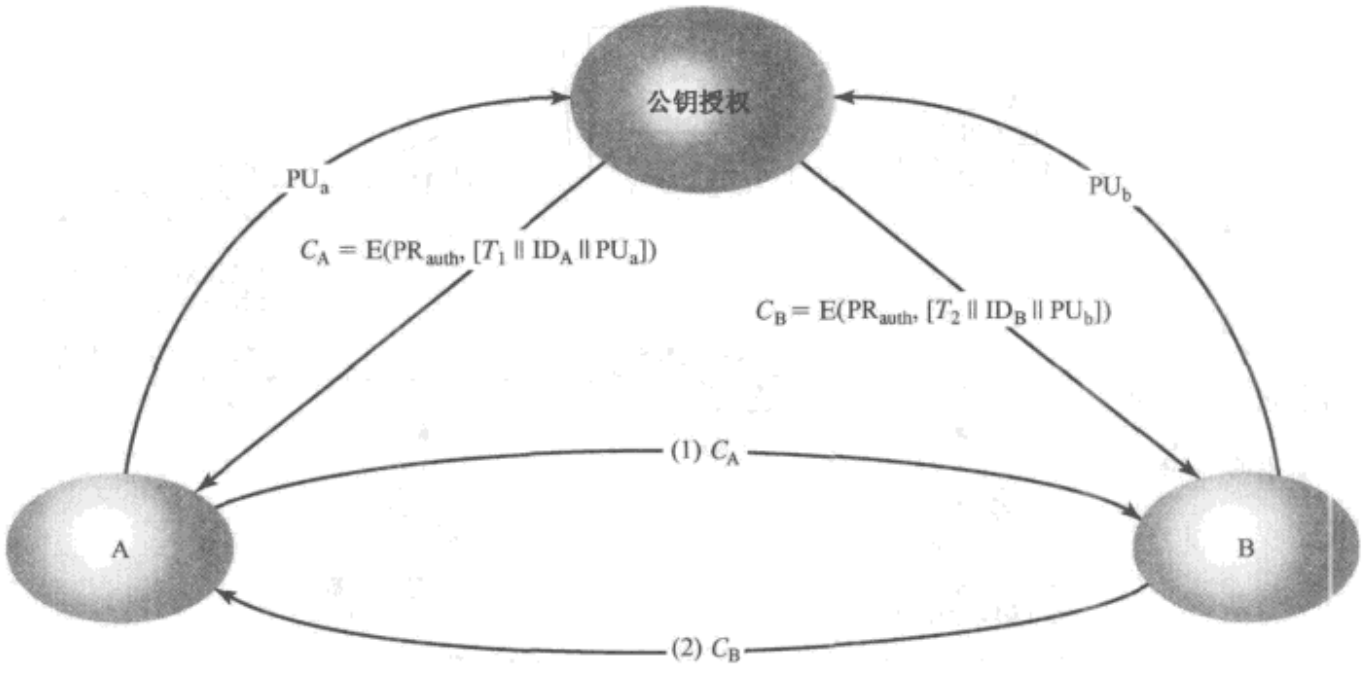


图 14.12　公钥证书交换

接收方使用证书管理员的公钥 PU_{auth} 对证书解密，因为证书只有使用管理员的公钥才能读取证书，因此接收方可验证证书确实来自于证书管理员。ID_A 和 PU_a 向接收方提供证书持有者的名字和公钥，时间戳 T 验证证书的时效性。时间戳可以在攻击者已知 A 的私钥的情况下抗攻击，假设 A 产生新的公私钥对并向公钥管理员申请新的证书，同时，攻击者重放 A 的旧证书给 B，若 B 用旧公钥加密数据，则攻击者就可以读取信息。

在这种情况下，私钥的泄密就如同信用卡的丢失一样，卡的持有者会注销信用卡号，但只有在所有可能的通信方均已知旧信用卡过时的时候，才能保证卡的持有者的安全，因此，时间戳有一些像截止日期。若一个证书超时，会被认为证书已失效。

X.509 标准是一个广为接受的方案，用来规范公钥证书的格式。X.509 证书在大部分网络安全应用中都有使用，包括 IP 安全、传输层安全（TLS）和 S/MIME，所有这些都在本书第五部分讨论。X.509将在下一节进行详细的介绍。

14.4　X.509 认证服务

ITU-T 建议书中的 X.509 是 X.500 系列中定义目录服务的一部分。实际上，目录是一个服务器或管理用户信息的分布式的服务器集合。这些信息包括用户信息、用户信息属性以及用户名到网络地址的映射等。

X.509 定义了 X.500 用户目录的一个认证服务框架，这个目录可以作为在 14.3 节中讨论的公钥证书类型的存取库。每个证书包含该用户的公钥并由一个可信的签证机构用私钥签名。另外，X.509 还定义了基于公钥证书的一个认证协议。

X.509 是关于证书结构和认证协议的一种重要标准，并被广泛使用。例如，X.509 证书格式用于 S/MIME（参见第 18 章）、IP 安全性（参见第 19 章）和 SSL/TLS 与 SET（参见第 16 章）。

X.509 首次于 1988 年发布，随后参考文献[IANS90]和[MITC90]修订了其安全性，修订稿于 1993 年发布，1995 年发布第三版，2000 年被再次修订。

X.509 是基于公钥密码体制和数字签名的服务。其标准中并未规定使用某个特定的算法，但推荐使用 RSA；其数字签名需要用到 Hash 函数，但并没有规定具体的 Hash 算法。1988 年的建议书中推荐的 Hash 算法被证明不安全后，在 1993 年的建议书中被删除。图 14.13 描述了一个公钥证书的产生过程。

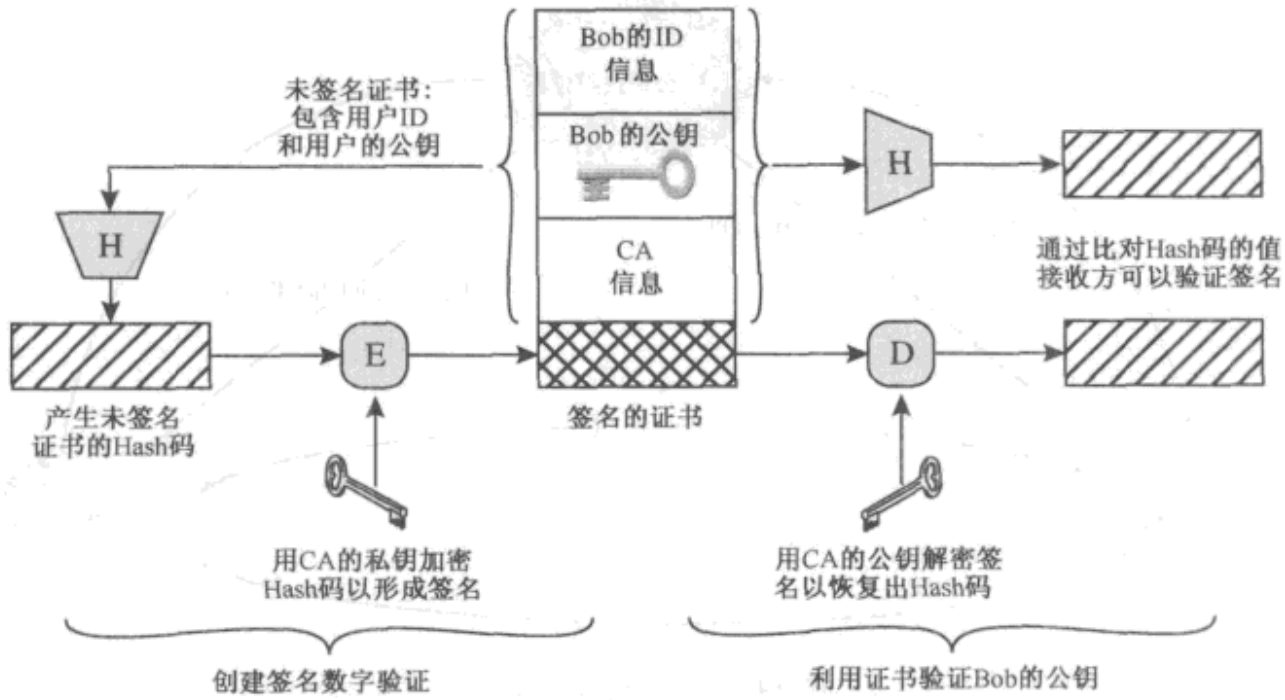


图 14.13 公钥证书应用

14.4.1 证书

X.509 的核心是与每个用户相关的公钥证书。这些用户证书由一些可信的签证机构(CA)创建并被 CA 或用户放入目录服务器中。目录服务器本身不创建公钥和证书,仅为用户获得证书提供一种简单的存取方式。

图 14.14(a)表明证书的常用的格式包含以下要素:

- **版本号(Version)**:区分证书的不同版本,默认设置为 1。如果存在发行商唯一标志或主体唯一标志,则版本号为 2,如果存在一个或多个扩展,则版本号为 3。
- **序列号(Serial number)**:一个整数,在 CA 中唯一标志证书。
- **签名算法标志(Signature algorithm identifier)**:带参数的、用于给证书签名的算法,由于此信息在证书尾部的域 Signature 中还会出现,这里很少包含信息。
- **发行者名称(Issuer name)**:X.500 中创建、签名证书的签证机构 CA 的名字。
- **有效期(Period of validity)**:包含两个日期,即证书的生效日期和终止日期。
- **证书主体名(Subject name)**:获得证书的用户名,此证书证明主体的公钥。
- **证书主体公钥信息(Subject's public-key information)**:主体的公钥以及将被使用的密钥的算法标志,带有相关的参数。
- **发行商唯一标志(Issuer unique identifier)**:一个可选位串,由于 X.500 的名字被许多不同实体引用,因此用此可选位串唯一地标志签证机构。
- **证书主体唯一标志(Subject unique identifier)**:一个可选位串,由于 X.500 的名字被许多不同实体引用,因此用此可选位域唯一地标志证书主体。
- **扩展(Extensions)**:一个或多个扩展域集,扩展域是在版本 3 中增加的,将在以后讨论。
- **签名(Signature)**:用 CA 私钥对证书的所有域及对这些域的 Hash 值一起加密。

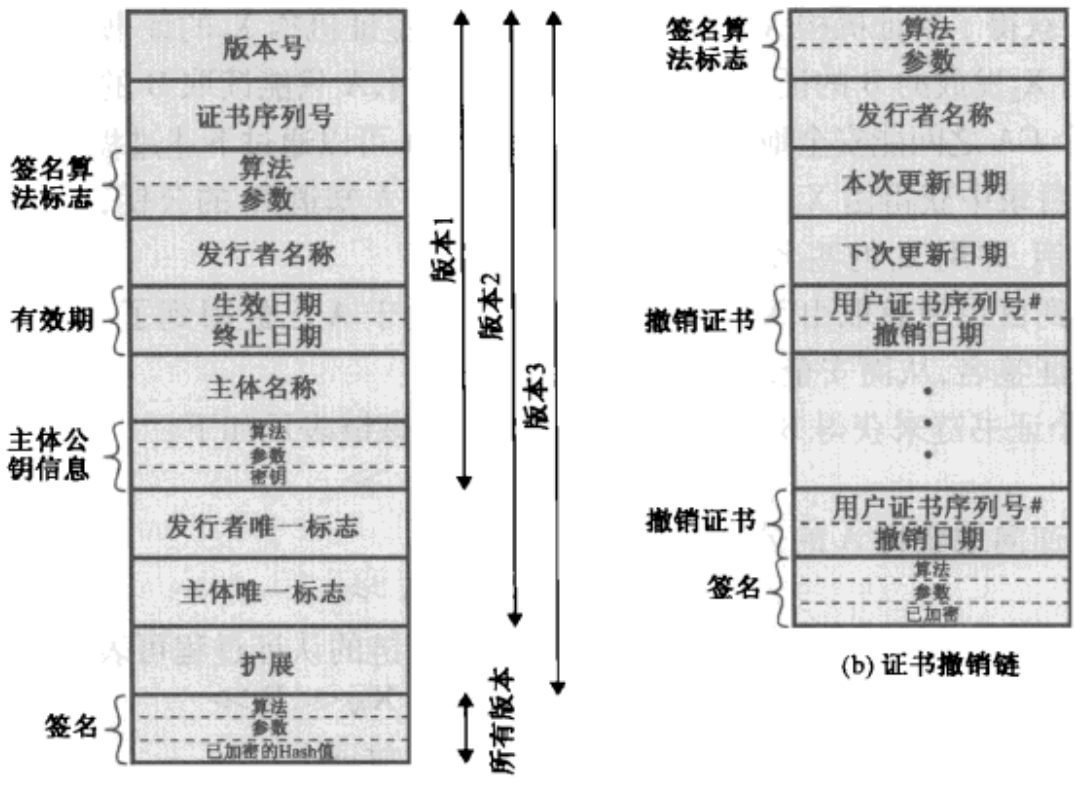
唯一标志域是在版本 2 中加入的,在证书主体或发行者名字出现重名时使用,一般很少使用。

该标准使用如下标注定义证书:

$$CA\ll A\gg = CA\{V, SN, AI, CA, UCA, A, UA, Ap, T^A\}$$

其中:

$Y\ll X\gg$ 表示用户 X 的证书是签证机构 Y 发放的



(a) X.509证书　(b) 证书撤销链

图 14.14　X.509 格式

Y{I}表示 Y 签名 I,包含 I 和 I 被加密后的 Hash 值

V 表示证书的版本

SN 表示证书的序列号

AI 表示用于给证书签名的算法的标志

CA 表示签署证书机构的名称

UCA 表示 CA 的可选择的唯一标志

A 表示用户 A 的名称

UA 表示 A 的可选择的唯一标志

Ap 表示用户 A 的公钥

T^A 表示证书的有效周期

CA 用自己的私钥签署证书,如果用户知道相应的公钥,则用户就可以验证证书是 CA 签署的。在图 13.2 中举例说明了这种典型的数字签名方法。

以下为证书管理:

获得一个用户证书

CA 生成的用户证书具有以下特点:

- 任何可以访问 CA 公钥的用户均可获得证书中的用户公钥。
- 只有 CA 可以修改证书。

由于证书不可伪造,因此证书可以存放在目录中,而不需要对目录进行特别保护。

如果所有用户都属于同一个 CA,则说明用户普遍信任 CA,所有用户的证书均被存放于同一个目录中,所有用户都可以进行存取。另外,用户也可以直接将其证书传给其他用户。一旦 B 拥有了 A 的证书,B 即可确信用 A 的公钥加密的消息是安全的,不可能被窃取,同时,用 A 的私钥签名的消息也不可能仿造。

如果用户数量很多,不可能期望所有用户从同一个 CA 获得证书,由于证书是由 CA 签发的,每一个用户都需要拥有一个 CA 的公钥来验证签名,该公钥必须用一种绝对安全的方式提供给每个用户,使用户可以信任该证书。

现在,假设 A 获得了签证机构 X_1 的证书,而 B 获得了签证机构 X_2 的证书,如果 A 无法安全地获得 X_2 的公钥,则由 X_2 发放的 B 的证书对 A 而言就无法使用,A 只能读取 B 的证书,但无法验证其签名,然而,如果两个 CA 之间能安全地交换它们的公钥,则 A 可以通过下述过程获得 B 的公钥:

步骤 1:A 从目录中获得由 X_1 签名的 X_2 的证书,由于 A 知道 X_1 的公钥,A 可从证书中获得 X_2 的公钥,并用 X_1 的签名来验证证书。

步骤 2:A 再到目录中获取由 X_2 颁发的 B 的证书,由于 A 已经得到了 X_2 的公钥,A 即可利用它验证签名,从而安全地获得 B 的公钥。

A 使用了一个证书链来获得 B 的公钥,在 X.509 中,该链表示如下:

$$X_1 \ll X_2 \gg X_2 \ll B \gg$$

同样,B 可以逆向地获得 A 的公钥:

$$X_2 \ll X_1 \gg X_1 \ll A \gg$$

上述模式并不仅仅限于两个证书,对长度为 N 的 CA 链的认证过程可表示如下:

$$X_1 \ll X_2 \gg X_2 \ll X_3 \gg \cdots X_N \ll B \gg$$

在这种情况下,链中的每对 CA(X_i, X_{i+1})必须互相发放证书。

所有由 CA 发放给 CA 的证书必须放在一个目录中,用户必须知道如何找到一条路径来获得其他用户的公钥证书,在 X.509 中,推荐采用层次结构放置 CA 证书,以利于建立强大的导航机制。

图 14.15 描述了 X.509 中推荐的层次结构,相连的圆圈表示 CA 间的层次结构,相连的方框表示每个 CA 发放的证书所在的目录,每个 CA 目录入口中包含两种证书:

- **前向证书**:由其他 CA 发给 X 的证书。
- **后向证书**:X 发给其他 CA 的证书。

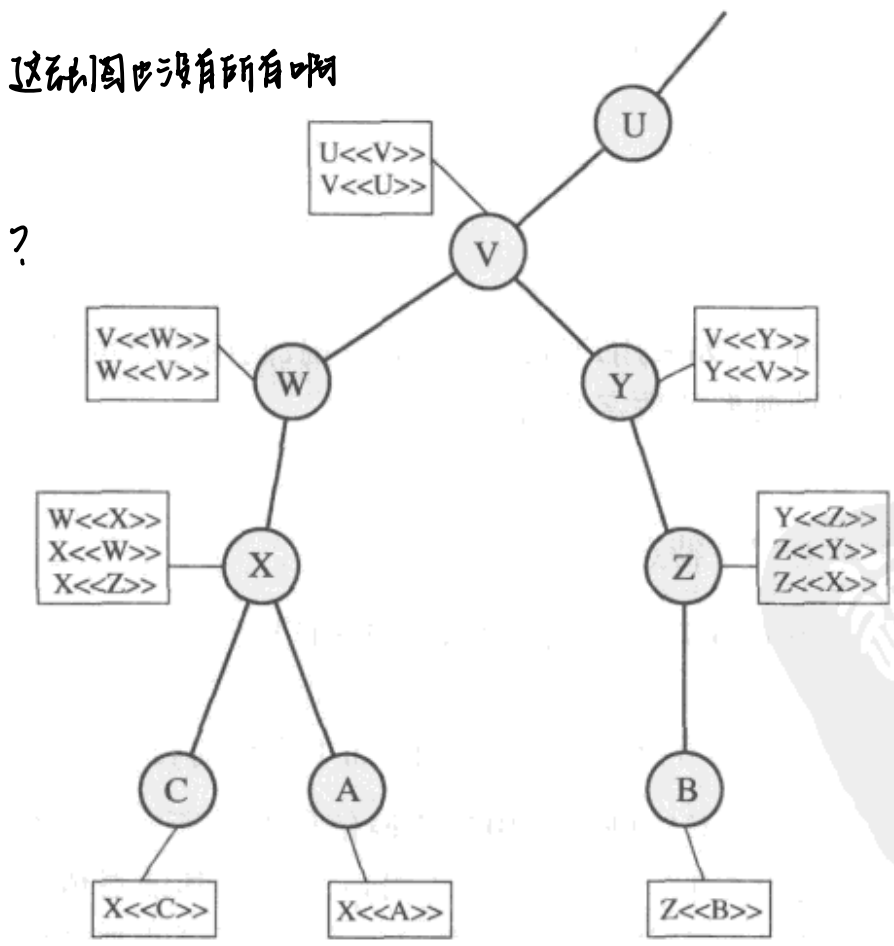


图 14.15 X.509 的层次结构:层次结构实例

例如,用户 A 可以通过创建一条到 B 的路径获得相关证书:

$$X \ll W \gg W \ll V \gg V \ll Y \gg Y \ll Z \gg Z \ll B \gg$$

当 A 获得相关证书后,可以通过顺序展开证书路径来获得 B 的公钥,用该公钥,A 可将加密消

息送往 B，如果 A 想得到 B 返回的加密消息或对发往 B 的消息签名，则 B 需要按照下述证书路径来获得 A 的公钥：

$$Z \ll Y \gg Y \ll V \gg V \ll W \gg W \ll X \gg X \ll A \gg$$

B 可以获得目录中的证书集，或 A 可在它发给 B 的初始消息中将其包含进去。

证书撤销

回想图 14.14，每一个证书都有一个有效期，这与信用卡相似。通常，新的证书会在旧证书失效前发放。另外，还可能由于以下原因提前撤销证书：

（1）用户私钥被认为不安全。

（2）用户不再信任该 CA。原因包括主体名已改变、证书已被废弃或者证书没有按照 CA 的规则发行。

（3）CA 证书被认为不安全。

每个 CA 必须保留一张表，其中包含所有被 CA 撤销且还未到期的证书，包括发给用户和其他 CA 的证书，这张表也应被放在目录中。

每个放在目录中的证书撤销表（CRL）均被其发行者签名［参见图 14.14（b）］，并包含发行者的名字、表创建的时间、下一张 CRL 表发放的时间以及每个撤销证书的入口。每个入口中包含该证书的序列号和撤销时间。由于一个 CA 中的序列号是唯一的，因此序列号足以表示一张证书。

当一个用户在一个消息中接收了一个证书时，用户必须确定该证书是否已被撤销。用户可以在接到证书时检查目录，为了避免目录搜索时的延迟，用户可以将证书和 CRL 缓存。

14.4.2　X.509 版本 3

X.509 的版本 2 中未将设计和实践当中所需要的所用信息均包含进去。［FORD95］列出了版本 2 中未满足的需求：

（1）证书主体域不足以将密钥所有者转换为公钥用户。X.509 中的名字相对较短，不能满足用户需要知道标志细节的要求。

（2）证书主体域在许多应用中不足以满足需要，通常需要互联网邮件地址、URL 和其他与互联网相关的标志。

（3）需要标明安全策略信息。这可以将安全应用或安全功能，如 IPsec 等，与 X.509 联系，将策略应用于证书。

（4）需要对证书的使用范围进行限定，以缩小 CA 失误或恶意破坏造成的影响。

（5）能区分同一个用户在不同时刻使用不同密钥是非常重要的。这一特性支持密钥的生命期管理，特别是在一般或特殊情况下更新用户和 CA 密钥对的能力。

与向固定格式中继续增加字段相比，标准的开发者认为需要一种更方便的方法。于是，版本 3 的格式增加了一些可选的扩展项。每一个扩展项有一个扩展标志、一个危险指示和一个扩展值。危险指示用于指出该扩展项是否能安全地被忽略，如果值为 TRUE 且实现时未处理它，则其证书将会被当做非法证书。

证书扩展项有三类：密钥和策略信息、证书主体和发行者属性以及证书路径约束。

密钥和策略信息

此类扩展项传递的是与证书主体和发行者密钥相关的附加信息，以及证书策略的指示信息。一个证书策略是一个已命名的规则集，在普通安全级别上描述特定团体或应用类型证书的使用范围。例如，某个策略可用于电子数据交换（EDI）在一定价格范围内的贸易认证。

这个范围包括:

- **授权密钥标志符**:标志用于验证证书或 CRL 上的签名的公钥。同一个 CA 的不同密钥得以区分,该字段的一个用法是用于更新 CA 密钥对。
- **主体密钥标志符**:标志被证实了的公钥,用于更新主体的密钥对。同样,一个主体对不同目的的不同证书可以拥有许多密钥对(例如,数字签名和加密密钥协议)。
- **密钥使用**:说明被证实的公钥的使用范围和使用策略。可以包含以下内容:数字签名、非抵赖、密钥加密、数据加密、密钥一致性、CA 证书的签名验证和 CA 的 CRL 签名验证。
- **私钥使用期**:表明与公钥相匹配的私钥的使用期。通常,私钥的使用期与公钥不同,例如,在数字签名密钥中,私钥的使用期一般比公钥短。
- **证书策略**:证书可以在应用多种策略的各种环境中使用。该扩展项中列出了证书所支持的策略集,包括可选的限定信息。
- **策略映射**:仅用于其他 CA 发给 CA 的证书中。策略映射允许发行 CA 将其一个或多个策略等同于主体 CA 域中的某个策略。

证书主体和发行者属性

该扩展支持证书主体或发行者以可变的形式拥有可变的名字,并可传递证书主体的附加信息,使得证书所有者更加确信证书主体是一个特定的人或实体。例如,一些信息如邮局地址、公司位置或一些图片。

扩展域包括:

- **主体可选名字**:包括使用任何格式的一至两个可选名字。该字段对特定应用,如电子邮件、EDI、IPsec 等使用自己的名字形式非常重要。
- **发行者可选名字**:包括使用任何格式的一至两个可选名字。
- **主体目录属性**:将 X.500 目录的属性值转换为证书的主体所需要的属性值。

证书路径约束

该扩展项允许在 CA 或其他 CA 发行的证书中包含限制说明。这些限制信息可以限制主体 CA 所能发放的证书种类或证书链中的种类。

扩展域包括:

- **基本限制**:标志该主体是否可作为 CA,如果可以,证书路径长度将被限制。
- **名字限制**:表示证书路径中的所有后续证书的主体名的名字空间必须确定。
- **策略限制**:说明对确定的证书策略标志的限制,或证书路径中继承的策略映射的限制。

14.5 公钥基础设施

RFC 2822(互联网安全术语)定义了 PKI 系统是由硬件、软件、人、策略和程序构成的一整套体系。这些程序用来创建、管理、存储、分发和撤销建立在非对称密码算法之上的数字证书。创建 PKI 的主要目的就是用来安全、便捷、高效地获得公钥。IETF(Internet Engineering Task Force)的 PKIX(Public Key Infrastructure X.509)工作组在 X.509 的基础上,建立了一个可以用来构建网络认证体系的基本模型。本节简单介绍一下 PKIX 模型。

图 14.16 描述了在 PKIX 模型中各个关键元素之间的相互关系。这些元素是:

- **端实体**:是必备的元素,它可以是一个终端用户、设备(比如应用服务器、路由器等),或是其他可以在一个公钥数字证书作用范围中被认证的实体。终端实体支持 PKI 相关的设备。
- **签证机构(CA)**:是证书和证书撤销列表(CRL)的发行人,常常在其上运行着一个或多个注册机构(RA),同时它还承担一些管理任务。
- **注册机构(RA)**:是可选的元素,可以承担一些签证机构(CA)的管理任务。一般来说都是和端实体的注册进程相关的任务,也可以支持一些其他的管理任务。
- **证书撤销列表(CRL)发布机构**:是可选的元素,签证机构可以通过它来发布证书撤销列表(CRL)。
- **证书存取库**:是必备的元素,它提供了存取数字证书和证书撤销列表(CRL)的方法,可以被终端用户检索。

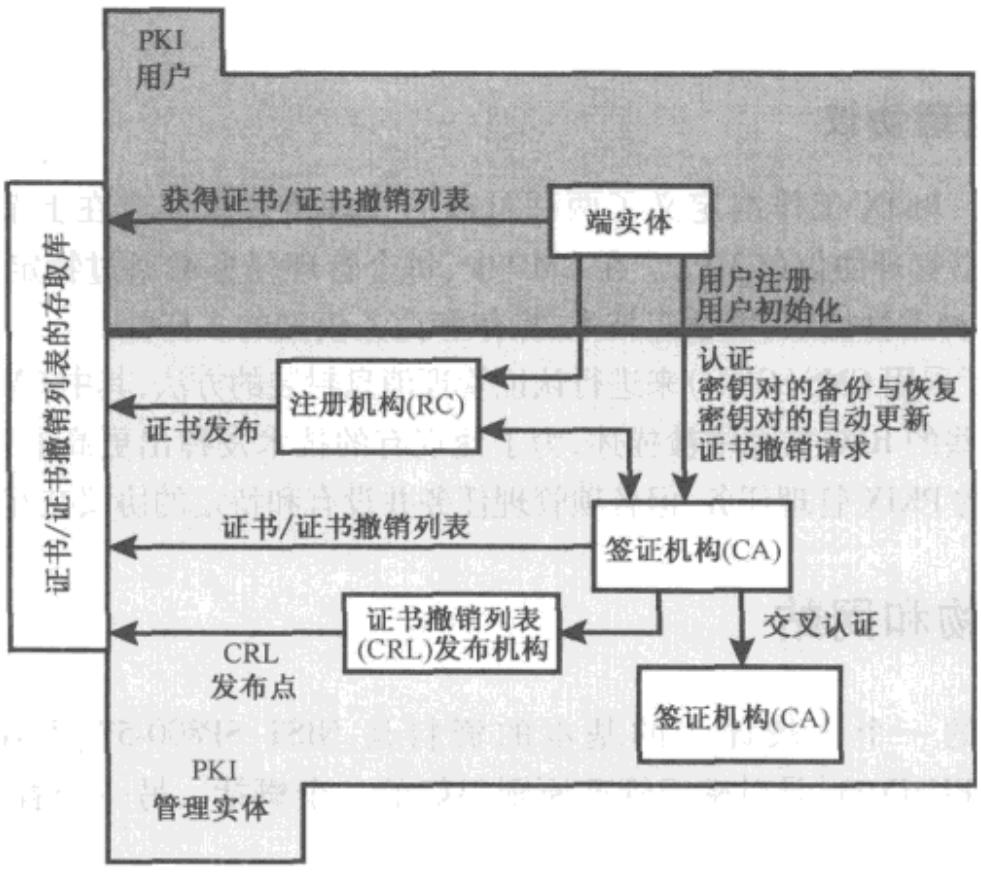


图 14.16　PKIX 结构模型

14.5.1　PKIX 管理任务

PKIX 体系中定义了一系列的管理任务,它们是在管理协议的支持下进行工作的。图 14.16 中描述了这些管理任务:

- **用户注册**:这是用户第一次进行认证之前进行的活动,它先于 CA 为用户颁布一个或多个证书。在 PKI 中注册从登记进程开始,注册通常包括一系列的在线和离线的交互认证。通常,终端实体会在后续认证中发布一个或者多个共享密钥。
- **初始化**:在一个客户系统安全操作之前,有必要将和密钥存储相关的关键材料安装在基础设施的别处,例如,客户需要用公钥和其他一些来自可信 CA 的确认信息安全的初始化,用于确认证书路径。
- **认证**:这个进程中,签证机构通过用户的公钥向用户提供一个数字证书,返回证书给用户客户系统,并在证书存取库中进行保存。
- **密钥对的恢复**:密钥对可以用于数字签名的创建、验证和数据加解密。而对于数据加解密

来说,当解密密钥丢失时,必须提供机制来恢复解密密钥,这对恢复加密数据来说非常重要。密钥的丢失通常是由密钥遗忘、磁盘驱动崩溃、硬件令牌毁坏等原因造成的。密钥对的恢复使得终端实体重新存储源于授权的密钥反馈机制(如发布终端实体证书的 CA)的加解密密钥。

- **密钥对更新**:所有的密钥都需要定期地更新(即替换为新的密钥对),及发布相应的证书。当证书的生命周期终止或者证书撤销时,需要更新。
- **证书撤销请求**:一个授权用户可以向签证机构提出要求撤销证书。当发生密钥泄露、从属关系变更或更名等时,需要提交这种请求。
- **交叉认证**:如果两个签证机构之间要交换数据,则可以通过交叉认证来建立信任关系。一个交叉认证证书是一个签证机构发布给另一个签证机构的,包括用来发布证书的数字签名。

14.5.2 PKIX 管理协议

在 PKIX 实体间,PKIX 工作组定义了两种可选的管理协议来完成在上节描述的管理任务。RFC 2510 定义了证书管理协议(CMP)。在 CMP 中,每个管理任务都通过特定协议交换来明确识别。另外,CMP 是一种柔性协议,能适应技术、操作和商务模型的多样性。

RFC 2797 定义了采用 CMS(CMC)来进行认证管理消息封装的方法,其中 CMS 在 RFC 2630 中被描述,CMC 则在更早些的 RFC 文档中被描述,为了使现有的技术发挥出更高的效能才设计了它们。另外,虽然支持所有的 PKIX 管理任务,但各项管理任务并没有和特定的协议交互一一对应。

14.6 推荐读物和网站

关于本章主题的一个比较详尽的基本的资料是 NIST SP800-57 [BARK07b, BARK07c, BARK08]第 3 卷。[FUMY93]是对密钥管理原则很好的一个概括。另一个着眼于密钥管理技术的调查是[HEGL06]。

[PERL99]回顾了能被用于 PKI 的不同的可信模型,[CUTM02]突出了 PKI 使用中的难点和使 PKI 发挥出效用的建议。

BARK07b Barker, E., et al. *Recommendation for Key Management-Part 1: General*. NIST SP800-57, March 2007.

BARK07c Barker, E., et al. *Recommendation for Key Management-Part 2: Best Practices for Key Management Organization*. NIST SP800-57, March 2007.

BARK08 Barker, E., et al. *Recommendation for Key Management-Part 3: Specific Key Management Guidance*. NIST SP800-57, March 2008.

FUMY93 Fumy, S., and Landrock, P. "Principles Key Management." *IEEE Journal on Selected Areas Communications*, June 1993.

GUTM02 Gutmann, P. "PKI: It's Not Dead, Just Resting." *Computer*, August 2002.

HEGL06 Hegland, A., et al. "A Survey of Key Management in Ad Hoc Network." *IEEE Communications Surveys & Tutorials*. 3rd Quarter 2006.

PERL99 Perlman, R. "An Overview of PKI Trust Models." *IEEE Network*, November/ December 1999.

推荐网站

- Public-Key Infrastructure Working Grop：开发基于 X.509v3 的标准的 IETF 组。
- Verisign：X.509 相关产品的开发商，该站点上有白皮书和其他有价值的资料。
- NIST PKI Program：不错的信息来源。

14.7　关键术语、思考题和习题

关键术语

端到端加密	中间人攻击	公钥证书
密钥分发	主密钥	公钥目录
密钥分发中心(KDC)	临时交互号	X.509 证书
密钥管理		

思考题

14.1　列出在两个通信方之间分发密钥的方法。
14.2　会话密钥和主密钥之间有什么不同？
14.3　什么是临时交互号？
14.4　什么是密钥分发中心？
14.5　基于公钥加密的两种不同的密钥分发方法是什么？
14.6　列出四种公钥分发模式的类型。
14.7　公钥目录的必要要素是什么？
14.8　什么是公钥证书？
14.9　公钥证书的使用有哪些要求？
14.10　X.509 标准的用途是什么？
14.11　什么是证书链？
14.12　X.509 证书如何撤销？

习题

14.1　本地网络向量提供一个密钥分发方案，如图 14.17 所示。
（a）描述该方案。
（b）相比图 14.3 的方案，有哪些优点和缺点。

14.2　“Holmes，我们有很大的压力。”侦探 Lestrade 看起来很紧张。“我们已经知道，一些敏感的政府文件复印件出现在伦敦一个外国大使馆的计算机上，通常，这些文件一般以电子形式存放在少数几台政府部门的计算机上，满足最严格的安全要求。有时要通过网络连接发给所有的政府计算机，但是这些消息是由我们最好的加密专家鉴定的绝密算法加密的，NSA 和 KGB 都无法破解。但是现在这些文件却出现在一个小的甚至是无关紧要的国家的外交官手里，我想不明白为什么会发生这种事情。”
“但是你已经有了嫌疑人，不是吗？” Holmes 问。
“是的，我们做了一些调查，有一个男人合法地访问过一台政府部门的计算机，并且和那个外交官联系很频繁。但是他访问的不是那些文件通常存放的计算机，他是嫌疑犯，但我们不知道他是如何获得文件复印件的。即便他得到加密的文件，他也不能解密。”

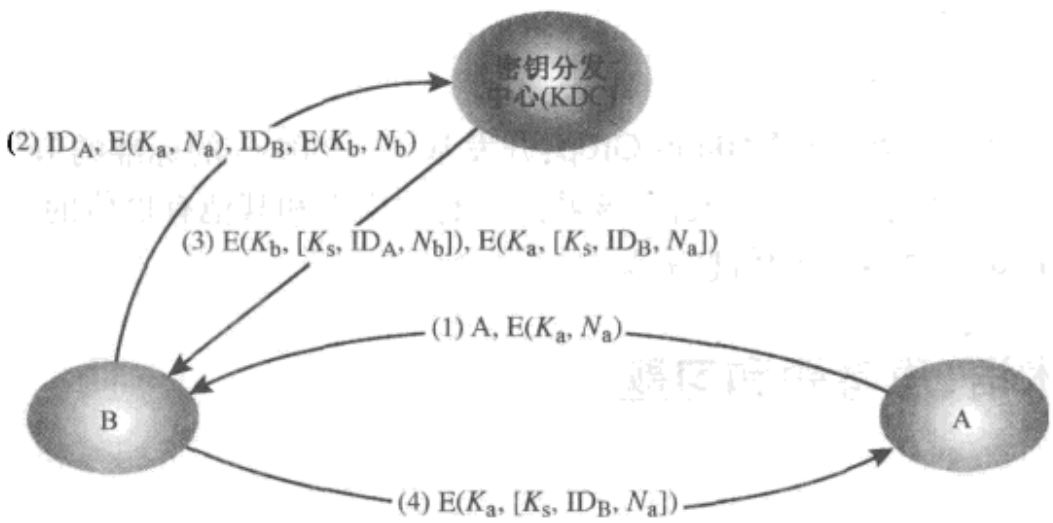


图 14.17 习题 14.1 图

"请描述一下,该网络使用的通信协议。" Holmes 睁开眼睛,以告诉 Lestrade 他虽然一脸睡意,但是他还是在认真地听着。

"协议如下,网络中的每个节点 N 都有唯一的密钥 K_n ,用于节点和可信服务器之间的安全通信,即所有的密钥也在服务器中存放。用户 A 想要发送秘密消息 M 给 B 时,使用以下协议:

(1) A 产生临时交互号 R,发送自己的名字 A、目的地 B 和 $E(K_a, R)$ 给服务器。

(2) 服务器回复消息 $E(K_b, R)$ 给 A。

(3) A 发送 $E(R, M)$ 和 $E(K_b, R)$ 给 B。

(4) B 使用 K_b 解密 $E(K_b, R)$ 得到 R ,随后应用 R 解密 $E(R, M)$ 得到消息 M 。

每次有消息发送时产生一个随机密钥,我承认他可能在几个节点之间发送时截获消息,但是,他不可能解密消息。"

"我相信你有你的道理,Lestrade。这个协议是不安全的,因为服务器不能鉴定是谁发的请求。明显地,协议设计者相信发送 $E(K_x, R)$ 就可以鉴定发送者为用户 X,因为只有 X 知道 K_x ,但是你也知道 $E(K_x, R)$ 可能被截获然后重放。只要知道漏洞在哪里,通过监控该男子对访问计算机的使用,可以得到更多的证据。他最有可能是如下做的:截获 $E(K_a, R)$ 和 $E(R, M)$ 后,我们称该男子为 Z,会假装 A 然后……"

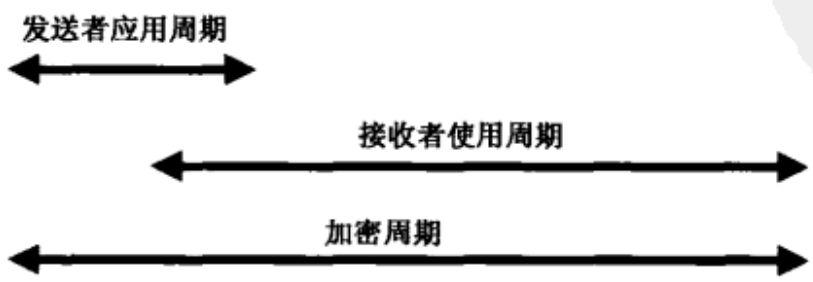
完成 Holmes 的话。

14.3 X.509 的 1988 年版中列出了这样的内容:在现有的知识层面上,分解大整数是很困难的,故 RSA 一定是安全的。对公开的指数和模数 n 有如下限制:

必须确保 $e > \log_2(n)$,才能阻止通过用第 e 个根来模 n 以得到明文的攻击。尽管这个限制是对的,但是给出的原因却是错误的。错在哪里?正确的原因是什么?

14.4 在你的计算机上,找出至少一个中间的证书管理员的证书和一个根证书管理员的证书(举例来说,在浏览器)。显示每个证书大致的和详细的表的屏幕截图。

14.5 NIST 定义了加密周期(cryptoperiod)作为时间跨度,期间一个密钥被授权使用,或者,给定系统或应用的密钥仍旧有效。一个关于密钥管理的文件中,为共享密钥使用了如下的时间图表。



通过给出一个应用实例来解释重叠部分,在该实例中要求发送者对会话密钥的使用周期开始得比接收者应用周期早,且结束得也比较早。

14.6 考虑以下协议,为了 A 和 B 得到一个新的会话密钥 K'_{AB} ,假定它们已经共享了一个长期密钥 K_{AB} 。

(1) $A \rightarrow B: A, N_A$

(2) $B \rightarrow A: E(K_{AB}, [N_A, K'_{AB}])$

(3) $A \rightarrow B: E(K'_{AB}, N_A)$

(a) 首先尝试去理解设计者的思路：

—在协议运行之后，A 和 B 为什么会相信共享 K'_{AB} 的会话密钥会来自于彼此？

—它们为什么会相信这个会话密钥是最新的？

在这种情况下，你要解释 A 和 B 相信的原因，所以你的回答应完成下列句子：

A 相信和 B 共享 K'_{AB} 是因为_____。

B 相信和 A 共享 K'_{AB} 是因为_____。

A 相信 K'_{AB} 是最新的，因为_____。

B 相信 K'_{AB} 是最新的，因为_____。

(b) 假定现在 A 和 B 开始运行该协议，然而，连接被 C 截获。展示 C 如何利用映像重新运行协议，使得 A 认为已经和 B 就新的会话密钥达成一致(尽管事实上是和 C 进行了通信)。从而，(a)中是错误的。

(c) 为了阻止该攻击，提出一种修改方案。

14.7　PKI 的关键组成部分有哪些？简单地描述各个部分。

14.8　解释密钥管理的问题以及它是如何影响对称加密的。

注意：剩下的习题涉及 IBM 开发的一个加密产品，在本书的网址上的一个文件(IBMCrypto. pdf)做了简单的描述。在阅读完文件后，尝试以下习题。

14.9　添加指令 EMK_i：

$$EMK_i: X \rightarrow E(KMH_i, X)\ i = 0, 1$$

14.10　假设 N 个不同的 IBM 加密子系统中有主机主密钥 $KMH_i (i = 1, 2, \cdots, N)$，设计一种系统之间的通信方法，而各个系统既不用共享一个主机主密钥又不会泄露自己的主机主密钥。提示：每个系统都需要有自己主密钥的三个不同的变体。

14.11　IBM 密码子系统的主要目的是保护终端和处理系统之间的通信。设计一个程序，可能会添加指令，允许处理器产生一个会话密钥 K_S，并分发给终端 i 和终端 j，而不需要在主机中存储与密钥相关的变量。

第15章 用户认证

15.1 远程用户认证原理
- 15.1.1 双向认证
- 15.1.2 单向认证

15.2 基于对称加密的远程用户认证
- 15.2.1 双向认证
- 15.2.2 单向认证

15.3 Kerberos
- 15.3.1 设计动机
- 15.3.2 Kerberos 版本 4
- 15.3.3 Kerberos 版本 5

15.4 基于非对称加密的远程用户认证
- 15.4.1 双向认证
- 15.4.2 单向认证

15.5 联合身份管理
- 15.5.1 身份管理
- 15.5.2 身份联合

15.6 推荐读物和网站

15.7 关键术语、思考题和习题

We cannot enter into alliance with neighboring princes until we are acquainted with their designs.

—*The Art of War*, Sun Tzu

要 点

- 双向认证协议能够使通信双方互相认证彼此身份并交换会话密钥。
- Kerberos 是一种用在分布式环境中的认证服务。
- Kerberos 提供可信的第三方认证服务，使客户端和服务器可以建立认证的通信。
- 身份管理是一种集中式的、自动的方法，可以提供雇员或者其他授权的个人在企业范围内对资源的访问控制。
- 事实上，身份联合是对身份管理在多个安全域的一个扩展。

本章举例说明了一些能支持网络应用认证的认证功能。15.1 节介绍一些在网络或者互联网上实现用户认证的概念和要素，15.2 节介绍基于对称加密的用户认证协议，15.3 节介绍最早、应用最广泛的用户认证服务之一：Kerberos，15.4 节介绍基于非对称加密的用户认证协议，之后是对 X.509 用户认证协议的讨论，最后介绍联合身份（federated identity）的概念。

15.1 远程用户认证原理

在大多数的计算机安全环境中，用户认证是最基本也是最重要的一道防线，用户认证是访问控制以及用户承担责任的基础。RFC 2828 对用户认证的定义如下所述。

核实身份的方法是由或是对一个系统实体提出的。认证方法包括以下两步：

- **鉴定阶段**：给安全系统提供身份标志（身份标志要认真分配，因为已认证的身份是其他安全服务的基础，如访问控制服务）。
- **核实阶段**：提供或者产生可以证实实体和标志之间对应关系的认证信息。

例如，用户 Alice Toklas 有用户标志 ABTOKLAS，该信息存储在 Alice 想要访问的服务器或者计算机系统中，并且要使系统管理员及其他用户知道。一个典型的和用户 ID 相连的认证信息项

是口令(只有 Alice 和系统知道)。如果没有人得到或者猜到 Alice 的口令,则 ID 和口令的结合使得管理员可以建立 Alice 的访问许可并且可以审查她的活动,因为 ID 是不保密的,故其他用户可以给她发送 E-mail,但由于口令是保密的,所以不能冒充 Alice。

事实上,身份鉴定是指用户提供一个声明的身份给系统,用户认证是使得这一声明成为有效的一种方法。注意用户认证和消息认证是不一样的,就像第 12 章定义的那样,消息认证允许通信双方验证接收到的消息是否被更改以及资源是否可信。本章只关注用户认证。

认证一个用户的身份大致有四个常用工具,可以单独使用,也可以联合使用:

- **知道什么**:如口令、个人身份号(PIN)或者之前准备的问题的答案。
- **拥有什么**:如加密密钥、电子密钥卡、智能卡和物理密钥,这种类型的认证信息称为令牌。
- **静态生物特征**:如指纹、视网膜和脸。
- **动态生物特征**:如声音模式、手写特征和打字节奏。

所有这些方法通过适当的执行和应用都可以提供安全的用户认证。然而,每种方法都有缺陷,简单地说,攻击者可能伪造或者窃取令牌,用户可能会忘记密码或者丢失令牌。此外,管理系统的密钥和令牌信息并确保这些信息的安全将会增加不少的系统开销。关于生物计量的认证信息,存在各种各样的问题,如处理假阳性和假阴性、用户满意度、成本和便利性等。对于基于网络的用户认证,最重要的认证方法包括加密密钥和用户个人口令等。

15.1.1　双向认证

双向认证的一个重要应用领域是双向认证协议,双向认证协议能够使通信双方互相认证彼此身份并交换会话密钥。该内容已在第 14 章介绍过,当时主要考虑的是密钥分发,在这里我们讨论认证更广泛的含义。

已认证的密钥交换主要关注两个问题:保密性和时效性。为了阻止伪装和会话密钥泄露,必要的身份鉴别和会话密钥信息必须以加密的形式进行通信,这也就要求有已存在的密钥或者公钥来做加密。第二个问题,因为消息重放的威胁使得保证时效性很重要。重放在最坏情况下,会使敌手获得会话密钥或者扮演会话的另一方;最好的情况下,成功的重放能中断通信。

[GONG93]列出了以下重放攻击的例子:

- **简单重放**:敌手只是复制消息,然后重放给消息接收者。
- **记录重复**:敌手在有效的时间窗口中重放一个时间戳消息给消息接收者。
- **截获重放**:原始消息被截获不能到达,只有重放消息可以到达消息接收者。
- **无更改的反向重放**:这种重放是给消息发送者的。如果使用对称加密,或者发送者很难识别发送的消息与所接受的消息在内容上的不同,则这种攻击就可能发生。

① 防止重放攻击的方法是对每一个用于认证交互的消息附上一个序列号,新的消息只有其序列号满足适当的顺序时才会被接收。这种方法的困难是,要求每一方都跟踪记录与其交互的通信方的最新序列号。考虑到开销问题,序列号基本上不会用于认证和密钥交换,反而是以下两种方法更常用:

② - **时间戳**:只有当消息中包含一个时间戳时,A 才接收该消息。该时间戳由 A 来判断,要接近于 A 所知的当前时间。该方法要求不同参与者之间的时钟是同步的。

③ - **挑战/应答**:A 想要一个来自 B 的新消息,首先发给 B 一个临时交互号(询问),并要求后面从 B 收到的消息(回复)中包含正确的临时交互号。挑战-应答身份鉴别协议

④局限: 一些学者(如[LAM92a])认为,由于技术本身的困难,时间戳方法不能用于面向连接的应用。

首先,为了维持不同处理器的时钟同步,需要协议必须容错(处理网络故障)和安全(应对敌手攻击)。其次,由于任何一方的时钟机制出现错误都会造成暂时的同步消失,使得攻击成功率增大。最后,网络延迟的多变性及不可预测性,使得分布环境下的时钟不能维持精确同步。因此,任何基于时间戳的进程,都要求有一个足够大的时间窗口来适应网络延迟,或者足够小的时间窗口来最小化攻击机会。

另一方面,挑战/应答方法不适用于无连接类型的应用,因为在无连接传输之前的握手开销,实际上是否定了无连接传输的主要优点。对于这些应用,依靠安全的时间服务器和各方的一致性要求来保持时钟同步可能是最好的方法([IAM92b])。

15.1.2 单向认证

加密越来越受欢迎的一个应用领域是电子邮件服务。电子邮件的本质及主要优势就是不需要发送者和接收者同时在线,电子邮件信息会直接发到接收者的电子邮箱,直到接收者有空闲去读。

"信封"即邮件的头部必须清楚,存储转发邮件协议才能处理该消息,如简单邮件转发协议(SMTP)或者 X.400。通常我们希望邮件处理协议不要求访问消息的明文,否则就必须要有可信的邮件处理协议。因此,邮件消息需要加密,并且邮件处理系统不拥有解密密钥。

第二个要求是认证,通常接收者需要确保消息确实来自于所谓的发送者。

15.2 基于对称加密的远程用户认证

15.2.1 双向认证

第 14 章讨论过,在分布式环境中,一个两层的对称加密密钥可用于为通信提供保密性。该方案需要可信的密钥分发中心(KDC)参与,每一方和 KDC 之间都共享一个密钥(称为主密钥)。KDC 负责产生两者之间的会话密钥,并使用主密钥来保证会话密钥分发的安全。例如,15.3 节主要讨论的 Kerberos 机制。

图 14.3 说明了最初由 Needham 提出的认证协议[NEED78]。协议总结如下[①]:

(1) $A \rightarrow KDC$: $ID_A \| ID_B \| N_1$

(2) $KDC \rightarrow A$: $E(K_a, [K_s \| ID_B \| N_1 \| E(K_b, [K_s \| ID_A])])$

(3) $A \rightarrow B$: $E(K_b, [K_s \| ID_A])$

(4) $B \rightarrow A$: $E(K_s, N_2)$

(5) $A \rightarrow B$: $E(K_s, f(N_2))$

密钥 K_a、K_b 分别是 A、B 与 KDC 所共享的主密钥,协议的目的是安全地将会话密钥 K_s 分发给 A 和 B。第 2 步 A 安全地收到会话密钥,第 3 步的消息只能由 B 解密,第 4 步反映了 B 收到的 K_s,第 5 步使 B 明确了自己与 A 拥有相同的会话密钥,且临时交互号 N_2 保证 B 得到的消息是最新的。回顾第 14 章,第 4、5 步的目的是阻止特定类型的重放攻击。需要指出的是,敌手捕获第 3 步的消息并重放它,将会在某些方式上打乱 B 的操作。

尽管有第 4 步和第 5 步,该协议还是很容易受到一种形式的重放攻击。假设敌手 X 已知之前的会话密钥,虽然这比敌手简单地观察和记录步骤 3 更难发生,但这是一个安全隐患。除非 B 无限期地记得所有之前和 A 会话使用过的会话密钥,否则 B 就不能确定这是一个重放攻击。如果 X

① 冒号左边的部分表示发送者和接收者;冒号右边的内容表示消息的内容;符号 ‖ 表示连接。

能截获第 4 步的握手消息，他就能伪造第 5 步 A 的回复并将其发送给 B，而 B 却认为该消息来自于 A 且用已认证的会话密钥的加密。]

② 改进！ Denning[DENN81，DENN82]提议通过在第 2 步和第 3 步添加时间戳来修改 Needham/Schroeder 协议，克服以上弱点。假定主密钥 K_a 和 K_b 是安全的，包含以下步骤：

Denning 协议

(1) $A \rightarrow KDC$: $ID_A \| ID_B$

(2) $KDC \rightarrow A$: $E(K_a, [K_s \| ID_B \| T \| E(K_b, [K_s \| ID_A \| T])])$

(3) $A \rightarrow B$: $E(K_b, [K_s \| ID_A \| T])$

(4) $B \rightarrow A$: $E(K_s, N_1)$

(5) $A \rightarrow B$: $E(K_s, f(N_1))$

T 是时间戳，为 A、B 的会话密钥产生的时间，A 和 B 的时效性验证需要满足

$$|Clock - T| < \Delta t_1 + \Delta t_2$$

其中，Δt_1 是估算的 KDC 时钟和本地时钟（A 或 B）的正常时间差，Δt_2 是网络时延期望值。每个节点都可以按照相关标准来设置自己的时钟，因为时间戳是用安全的主密钥加密的，攻击者即便知道旧的会话密钥也不能成功，因为重放第 3 步会被 B 察觉。

第 4 步和第 5 步在[DENN81]中没有，但在[DENN82]中添加了，这两步能确保 B 收到了会话密钥。

与 Needham/Schroeder 的协议相比，Denning 协议似乎提供了较高的安全度。然而，一个新的问题出现了：新的方案要求在整个网络中时钟是同步的。[GONG92]指出威胁依然存在，主要是基于这样的事实：时钟或者同步机制的破坏或者错误，都可能造成分布的时钟不同步①。当发送者的时钟快于接收者的时钟时，攻击者从发送者处截获消息之后重放，消息中的时间戳刚好能满足接收者的时间。这个重放的后果是不可预料的，Gong 将该攻击称为压制重放攻击（Suppress Replay Attack）。

一种防止压制重放攻击的方法是，强制要求各方定期检查自己的时钟是否和 KDC 同步。另一种方法不需要时钟同步，依赖使用临时交互号的握手协议，该方法不容易遭受压制重放攻击，因为接收者选择的临时交互号对发送者来说是不可预测的。Needham/Schroeder 协议只依赖于临时交互号，但是就像我们看到的那样，有其他的缺陷。

KEN92 [KEHN92]中想要找出一种方式，能防止压制重放攻击，同时处理 Needham/Schroeder 协议的问题。但是在下列协议中的不一致引起了大家的注意，[NEUM93a]②给出了改进方法。协议如下：

(1) $A \rightarrow B$: $ID_A \| N_a$

(2) $B \rightarrow KDC$: $ID_B \| N_b \| E(K_b, [ID_A \| N_a \| T_b])$

(3) $KDC \rightarrow A$: $E(K_a, [ID_B \| N_a \| K_s \| T_b]) \| E(K_b, [ID_A \| K_s \| T_b]) \| N_b$

(4) $A \rightarrow B$: $E(K_b, [ID_A \| K_s \| T_b]) \| E(K_s, N_b)$

协议的步骤如下：

(1) A 产生临时交互号 N_a 和自己的标志 ID_A 一起以明文的形式发给 B，该临时交互号会和会话密钥共同在加密消息中返回给 A，确保 A 的时效性。

(2) B 向 KDC 请求一个会话密钥，向 KDC 发送的消息包含自己的标志和临时交互号 N_b，其中临时交互号和会话密钥共同在加密消息中返回给 B，确保 B 的时效性。B 发给 KDC 的消息还包含一个用 B 和 KDC 的共享主密钥加密的加密块，用于指示 KDC 需要发布证书给 A；内容包括证书的接收人、证书的截止时间和来自于 A 的临时交互号。

---

① 这样的事情确实可能发生。近年来，有缺陷的芯片被用于许多计算机和其他电子系统中，以记录时间和日期。这些芯片的时间有时会超前一天[NEUM90]。

② 构造正确的协议的确很困难。

(3) KDC 发送 B 的临时交互号以及用 KDC 与 B 共享的主密钥加密的数据给 A,该块作为"票据"用于 A 之后的认证。KDC 也发送给 A 加密块,该块是用 KDC 与 A 共享的主密钥加密的,该块可用于明确 B 收到了 A 的原始消息 ID_B 且是一个没有重放(N_a)且及时的消息,也提供给 A 一个会话密钥 K_s 及其使用时间限制(T_b)。

(4) A 把 $E(K_b, [ID_A \| K_s \| T])$ 及 B 的临时交互号一起传递给 B,其中临时交互号使用会话密钥加密。$E(K_b, [ID_A \| K_s \| T])$ 提供给 B 解密 $E(K_s, N_b)$ 的密钥以恢复临时交互号。事实上,B 的临时交互号用会话密钥加密证明了该消息不是重放。

该协议为 A、B 获得会话密钥提供了一个有效且安全的方法。此外,协议使 A 拥有认证 B 的密钥,且避免了重复的联系认证服务器。假设 A 和 B 利用协议建立和结束了一次会话,那么协议设置的限制时间内,A 想要和 B 再一次建立新的会话,便可以按以下协议执行:

(1) $A \rightarrow B$: $E(K_b, [ID_A \| K_s \| T_b]) \| N'_a$

(2) $B \rightarrow A$: $N'_b \| E(K_s, N'_a)$

(3) $A \rightarrow B$: $E(K_s, N'_b)$

当 B 接收到第 1 步的消息时,验证时间戳是否过期。新产生的临时交互号 N'_a 和 N'_b 确证双方都不会受到重放攻击。

T_b 所指定的时间只和 B 的时钟相关,因为只有 B 检查该时间戳,因此不要求时钟同步。

15.2.2 单向认证

在如图 14.5 所示的分布式密钥分发环境中,使用对称加密是不切实际的。该方法要求发送者首先向接收者发起会话请求,等收到包含会话密钥的回复之后才能发送消息。

对加密的电子邮件来说,图 14.3 中描述的 KDC 策略是个不错的选择,但为了避免要求接收者(B)和发送者(A)同时在线,第 4 步和第 5 步需要删除。对内容为 M 的消息,步骤如下:

(1) $A \rightarrow KDC$: $ID_A \| ID_B \| N_1$

(2) $KDC \rightarrow A$: $E(K_a, [K_s \| ID_B \| N_1 \| E(K_b, [K_s \| ID_A])])$

(3) $A \rightarrow B$: $E(K_b, [K_s \| ID_A]) \| E(K_s, M)$

该方法可以保证只有消息指定的接收者才能读取消息,同时提供了对发送者 A 的认证。但该协议不能抵抗重放攻击,已提供的防御策略中包括在消息中添加时间戳,但是由于电子邮件发送存在延误,时间戳可能会限制邮件的发送。

15.3 Kerberos

Kerberos[①] 是作为 MIT 大学的 Athena 计划的认证服务而开发的。Kerberos 阐述了这样一个问题:假设有一个开放的分布环境,工作站用户想通过网络对分布在网络中的各种服务提出请求,那么,希望服务器能够只对授权用户提供服务,并能鉴别服务请求的种类。但在这种环境下,一个工作站无法准确判定它的终端用户以及请求的服务是否合法。特别是存在以下三种威胁:

(1) 用户可能通过某种途径进入工作站并假装成其他用户操作工作站。

(2) 用户可以通过变更工作站的网络地址,从该机上发送伪造的请求。

(3) 用户可以监听信息或使用重放攻击,以获得服务或破坏正常操作。

---

① "在希腊神话中,Kerberos 是地狱入口的守卫者,长着许多头,通常是三个头,带有毒蛇似的尾巴。"摘自 *Dictionary of Subjects and Symbols in Art*, James Hall, Harper & Row, 1979。像希腊神话中的 Kerberos 有三个头一样,现代的 Kerberos 也要有三个"头"来守卫网络的大门,那就是认证、清算和审计。现在,后面的两个"头"还未实现。

在上述任何一种情况下，一个非授权用户均可能获得未授权的服务或数据。针对上述情况，Kerberos 通过提供一个集中的授权服务器来负责用户对服务器的认证和服务器对用户的认证，而不是为每个服务器提供详细的认证协议。与本书中其他认证方案不同的是，Kerberos 仅仅依赖于对称加密体制而未使用公钥加密体制。

目前常用的 Kerberos 有两个版本。版本 4[MILL88，STEI88]被广泛使用，而版本 5 [KOHL94]改进了版本 4 中的安全性，并成为 Internet 标准草案(RFC 4120)①。

本节首先简要介绍 Kerberos 的设计动机。由于 Kerberos 的复杂性，通过版本 4 中的认证协议来理解 Kerberos 策略的本质，而不用考虑掌握小的安全威胁细节。最后，阐述版本 5。

15.3.1　设计动机

当一组用户使用相互独立的计算机而不用网络相连时，每个用户的资源和文件都可以利用物理手段保护；当一组用户使用一个集中式分时系统时，则必须由分时操作系统来提供安全保护，操作系统可以使用基于用户身份的操作控制策略，并通过登录过程来认证用户。

今天，更为普遍的情况是由许多用户工作站和分布(或集中)的服务器所组成的分布式体系结构。在这种环境下，有三种安全方案可以采用：

(1) 由客户端工作站确保用户或用户组的身份，由服务器提供基于用户标志 ID 的安全策略。

(2) 要求客户端系统向服务器提供身份认证，相信客户端系统提供的用户身份。

(3) 要求客户向服务器提供身份认证，同时需要服务器向客户提供身份认证。

在一个小型、封闭的环境中，所有的系统属于同一个组织且被其访问，则应选择第一方案或第二方案②。但对更为开放的网络互联环境而言，则应选择第三种方案保护用户信息和服务器资源。Kerberos 支持第三方案。Kerberos 假设其体系结构为分布的客户/服务器体系结构，并拥有一个或多个 Kerberos 服务器提供认证服务。

Kerberos 发布的第一个报告[STEI88]中列出了以下 Kerberos 的需求：

- **安全性**：网络监听者不可能通过冒充其他用户获取有用信息。通常，Kerberos 应有足够的坚固性使得潜在攻击者无法找到其中的薄弱环节。
- **可靠性**：对依赖于 Kerberos 访问控制的所有服务而言，Kerberos 服务缺乏可用性意味着其所支持的服务缺乏可用性。因此，Kerberos 必须具有高可靠性，使用分布式服务器结构，即一个系统可以备份其他系统。
- **透明性**：理想状况下，用户除了要输入口令外，不需要知道认证发生的地点。
- **可伸缩性**：系统应能支持大量的客户端和服务器，这需要具有模块化、分布式的体系结构。

为了支撑这些要求，Kerberos 的整体设计方案基于协议的可信第三方认证服务(在第 7 章中讨论过)[NEED78]，客户与服务器均信任 Kerberos 的认证(参见 15.2 节)。假设 Kerberos 协议设计充分，则认证服务的安全性取决于 Kerberos 服务器的安全性③。

---

① 版本 1 到版本 3 是内部开发版本，版本 4 才是“原始的”Kerberos 版本。

② 然而，即使是封闭环境也面临怀有不满情绪的雇员的攻击。

③ 要记住，Kerberos 服务器并不能自动具有安全性，而是要采取措施确保其安全(如安装到上锁的房间里)。还要记住希腊神话中 Kerberos 的命运。欧律斯透斯命令大力神捕捉 Kerberos，以作为他的第 12 个劳奴：“大力神发现那只戴着链子的大狗，就卡住它的脖子。那时 Kerberos 的三个头企图反攻，还用它那强有力的尾巴来回鞭打。大力神冷酷地坚持不懈，最终 Kerberos 失去知觉，被制伏。欧律斯透斯看见 Kerberos 的三个头流着口水，惊讶地发现大力神还活着，由于惧怕他的智慧，就跳回到他那安全的铜缸里。”摘自 Michael Stapleton，Hamlyn，1982 年版的 *The Hamlyn Concise Dictionary of Greek and Roman Mythology* 一书。

15.3.2 Kerberos 版本 4

Kerberos 的版本 4 在协议中使用了 DES 提供认证服务,但从整体上很难看出使用 DES 的必要性。因此,使用 Athena 计划中被 Bill Bryant[BRYA88]用过的策略,通过建立一系列假设会话来帮助理解该协议,后一个会话在前一个会话的基础上增加了一些安全性措施,以解决暴露的安全漏洞。

一个简单的认证会话

在一个无保护的网络中,任何客户端可以向任意服务器提出服务请求。显然,安全风险在于可能假冒,一个攻击者可以假装成其他客户获得未授权的服务。因此,服务器必须确定申请服务的客户身份。服务器必须在每次客户/服务器交互中进行认证,但在一个开放式的环境中,将给服务器带来许多额外的负担。

一种变通的方法是使用一个认证服务器(AS)将所有用户的口令存放于一个集中式数据库中,并且 AS 与每一个服务器共享一个唯一的密钥,密钥按物理方法或其他安全方法分发。考虑以下包含 3 个消息的会话:

(1) $C \rightarrow AS: \quad ID_C \| P_C \| ID_V$

(2) $AS \rightarrow C: \quad Ticket$

(3) $C \rightarrow V: \quad ID_C \| Ticket$

$$Ticket = E(K_v, [ID_C \| AD_C \| ID_V])$$

其中:C 表示客户端;AS 表示认证服务器;V 表示应用服务器;ID_C 表示 C 上的用户标志;ID_V 表示 V 的标志;P_C 表示 C 上的用户口令;AD_C 表示 C 的网络地址;K_V 表示 AS 与 V 共享的加密密钥。

(1) 此时,用户登录到工作站,并请求访问服务器 V,用户工作站上的客户端模块 C 要求用户输入口令,并将包含用户标志 ID、服务器标志 ID 和用户口令的消息送往 AS。(2) AS 查询数据库,检查用户口令是否与用户标志相符,并判断此用户是否有访问服务器 V 的权限。如果两个检测均通过,则 AS 认为此用户合法,并通知服务器该用户是合法的。为了达到这一目的,AS 创建一个包含该用户 ID、用户网络地址和服务器 ID 的票据(Ticket),用 AS 和此服务器共享的密钥加密,并将加密后的票据返回 C。由于票据被加密,因此不可能被 C 或其他攻击者修改。

(3) 使用该票据,C 可以向 V 提出服务请求。C 向 V 发出包含 C 的 ID 和票据的消息,由 V 解密票据,并验证票据里的用户 ID 是否与消息中未加密的用户 ID 一致,如果验证通过,则服务器认为该用户真实,并为其提供服务。

消息(3)的每一个成分都非常重要。票据加密后可防止更改或伪造。将服务器标志(ID_V)包含在票据中,使得服务器可以验证它是否能正确地解密该票据;包含在票据中的标志 ID_C 可以说明该票据是从 C 发布的;最后,可以用 AD_C 来对抗下述威胁。如果票据中没有网络地址,攻击者可能在消息(2)的传输过程中捕获该票据,然后使用标志 ID_C 以消息(3)的格式从另一个工作站上发送消息,这样,服务器就可以得到一个与该用户 ID 相匹配的合法票据,将相应的权利授给其他工作站。为了防止这种攻击,AS 可以在票据中加入消息来源的网络地址,则从生成票据的工作站发出的票据才是合法的。

一个更加安全的认证会话

虽然上述会话解决了开放网络环境中认证的一些问题,但还存在两个突出问题。首先,我们可能希望能使用户输入口令的次数最小化。假设每个票据仅能使用一次,那么,如果用户 C 早晨在一个工作站上登录,希望查看他在邮件服务器上的邮件,C 为了与邮件服务器通信就必须提供口令得到票据。但如果 C 想在一天中多次查询邮件,则每一次都需要他重新输入口令。我们可以

通过重用票据来解决这一问题。对一个简单的登录会话，工作站可以将它收到的邮件服务器的票据存储，用户可以在多次访问邮件服务器时使用相同的票据。

但在这种模式下，用户必须为每种不同的服务创建一个新的票据。如果一个用户想同时访问打印服务器、邮件服务器、文件服务器等，则每次访问必须输入用户口令，以获得新的票据。

第二个问题是上述会话中包含对口令（消息 1）的明文传输，网络窃听者捕获口令后可以使用受害者的任何服务。

为了解决这些问题，我们引进了一个避免口令明文传输的模式和一个票据授权服务器（TGS）。新的会话如下所述。

每个用户仅一次的登录会话：

(1) $C \rightarrow AS$: $ID_C \| ID_{tgs}$

(2) $AS \rightarrow C$: $E(K_c, Ticket_{tgs})$

每种服务类型仅一次：

(3) $C \rightarrow TGS$: $ID_C \| ID_V \| Ticket_{tgs}$

(4) $TGS \rightarrow C$: $Ticket_v$

每次服务会话仅一次：

(5) $C \rightarrow V$: $ID_C \| Ticket_v$

$$Ticket_{tgs} = E(K_{tgs}, [ID_C \| AD_C \| ID_{tgs} \| TS_1 \| Lifetime_1])$$

$$Ticket_v = E(K_v, [ID_C \| AD_C \| ID_v \| TS_2 \| Lifetime_2])$$

TGS 将票据发给通过 AS 认证的用户，于是，用户首先应向 AS 申请得到一个票据授权票据 $Ticket_{tgs}$，由用户工作站的客户端模块保存。每当用户申请一项新的服务，客户端则用该票据证明自己的身份，向 TGS 发出申请，由 TGS 向特定的服务授予一个票据，客户将每个服务授权票据保存后，在每次请求特定服务时使用该票据证实自己的身份：

(1) 客户端将用户标志、TGS 标志一起送往 AS（表明申请一个的 TGS 服务），申请得到票据授权票据。

(2) AS 利用对应用户口令 K_c 的密钥加密票据，并将其返回给客户端，客户端提示用户输入口令，生成密钥，解密传来的消息，如果口令正确，则可以成功地恢复票据。

由于只有正确的用户知道口令 K_c，只有该用户能恢复票据。因此，可以在 Kerberos 中使用口令来获得可信度，避免口令的明文传输。票据本身由用户标志、用户网络地址和 TGS 标志组成，使得票据可以在客户端申请服务授权票据时多次使用，因此票据授权票据是可以重用的。考虑以下情况：一个攻击者捕获票据后等待，直至原用户退出网络。此时，攻击者可以通过使用原用户工作站或将其自己的工作站配置为同样的网络地址来使用该票据欺骗 TGS。为防止这种情况发生，颁发票据中应包含带有日期和时间的时间戳，并包含票据合法使用时间长度（如 8 小时）的生命期（Lifetime），这样，客户端既可以重用票据，又可以不用为每个新的服务保存一个口令。最后，票据授权票据是被一个仅有 AS 和 TGS 知道的密钥加密的，防止了对票据授权票据的篡改，且该票据被用户口令的密钥再加密，确保了只有正确的用户才能恢复该票据，起到了认证的作用。

现在，客户端得到了票据授权票据，通过步骤(3)、(4)来对各种服务器进行访问：

(3) 客户端根据用户请求申请一个服务授权票据。为此，客户端将包含由用户标志 ID、服务标志 ID 和票据授权票据组成的消息送往 TGS。

(4) TGS 对得到的票据授权票据解密，通过其标志验证该票据的正确性。确定该票据的生命期未超时，并比较用户 ID 和网络地址是否与消息来源一致。如果用户被允许访问服务器 V，则 TGS 发布相应服务的服务授权票据。

服务授权票据与票据授权票据具有相同的结构。实际上,由于 TGS 是服务器,我们可以认为 TGS 认证客户端与认证应用服务器所需要的元素是相同的。其票据也应包含时间戳和生命期。如果用户想在后来再次使用同一个服务,该客户端可以简单地使用原来获得的服务授权票据,而不需要用户重新输入口令。注意,加密票据授权票据的密钥 K_V 只有 TGS 和特定服务器知道,可防止篡改。

最后,使用特定的服务授权票据,客户端即可通过步骤(5)请求相应服务:

(5) 端请求服务。为此,客户端将包含用户标志和服务授权票据的消息送往服务器。服务器通过该票据的内容认证用户的合法性。

这种新的模式可以满足在每次用户会话时仅询问一次口令和保护用户口令的需求。

版本 4 的认证会话

虽然上述会话与第一种方式相比在安全性方面有所增强,但仍然存在两个问题。第一个问题与票据授权票据的生命期有关。如果生命期太短(例如几分钟),则用户将被要求重复输入口令;而如果生命期太长(如几小时),则为攻击者提供了大量攻击机会。攻击者在网络上监听、捕获票据授权票据后等待合法用户退出网络。此时,攻击者可以伪造合法用户的网络地址,并按照步骤(3)发送消息给 TGS,因此将会使攻击者得到合法使用该用户资源和文件的机会。

同样,如果攻击者获取了服务授权票据并在其过期之前使用,则可获得相应的服务。

因此,我们提出了一个额外的需求。一个网络服务(TGS 或应用服务)必须能够证实票据的使用者与票据的所有者的一致性。

第二个问题是服务器有向用户证实自己身份的需求。没有这样的认证,攻击者即可伪造配置使得送往服务器的消息被定向到其他节点。假冒的服务器即可伪装成真正的服务器捕获用户信息而对用户提供虚假服务。

我们将逐个讨论这些问题,表 15.1 描述了实际的 Kerberos 协议。

表 15.1 Kerberos 版本 4 的消息交换

(a)服务认证交换:获得票据授权票据	
(1) C → AS	$ID_C \| ID_{tgs} \| TS_1$
(2) AS → C	$E(K_c, [K_{c,tgs} \| ID_{tgs} \| TS_2 \| Lifetime_2 \| Ticket_{tgs}])$
	$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \| ID_C \| AD_C \| ID_{tgs} \| TS_2 \| Lifetime_2])$
(b)服务授权票据交换:获取服务授权票据	
(3) C → TGS	$ID_V \| Ticket_{tgs} \| Authenticator_c$
(4) TGS → C	$E(K_{c,tgs}, [K_{c,v} \| ID_V \| TS_4 \| Ticket_v])$
	$Ticket_{tgs} = E(K_{tgs}, [K_{c,tgs} \| ID_C \| AD_C \| ID_{tgs} \| TS_2 \| Lifetime_2])$
	$Ticket_v = E(K_v, [K_{c,v} \| ID_C \| AD_C \| ID_V \| TS_4 \| Lifetime_4])$
	$Authenticator_c = E(K_{c,tgs}, [ID_C \| AD_C \| TS_3])$
(c)客户/服务器认证交换:获取服务	
(5) C → K	$Ticket_v \| Authenticator_c$
(6) K → C	$E(K_{c,v}, [TS_5 + 1])$(对双向认证)
	$Ticket_v = E(K_v, [K_{c,v} \| ID_C \| AD_C \| ID_V \| TS_4 \| Lifetime_4])$
	$Authenticator_c = E(K_{c,v}, [ID_C \| AD_C \| TS_5])$

首先考虑票据授权票据的捕获问题和确定票据使用者与票据所有者一致的需求。其威胁是攻击者窃取票据并在其有效期间内使用。为了解决这一问题,我们让 AS 为客户端和 TGS 同时用安全方式提供一条秘密信息,然后客户端用该秘密信息以同样的安全方式向 TGS 证实自己的身份。Kerberos 的会话密钥使用的是加密密钥。

表 15.1(a)描述了分发会话密钥的技术。如前所述,客户端向 AS 发送消息请求访问 TGS,由 AS 回送应答消息,并用从用户口令派生的密钥(K_c)将其加密。加密的消息中还含有一份会话密钥 $K_{c,tgs}$,其下标表示该密钥是属于 C 和 TGS 共享的会话密钥。由于会话密钥包含在用 K_c 加密的消息中,只有该用户的客户端可以解密。此会话密钥同样也存在于票据中,而只能被 TGS 解密。因此,此会话密钥可以安全地在 C 和 TGS 间传递。

在继续进行会话前,还有一些附带的信息要加到会话的第一阶段中。由于消息(1)包含时间戳,使得 AS 知道该消息是新鲜的;消息(2)包含 C 可以访问票据中的若干元素,这些元素使得 C 能确定此票据是送往 TGS 的,并知道它的有效期。

有了票据和会话密钥,C 就可以与 TGS 通话。同样,由 C 向 TGS 发送消息,消息中包含票据和所申请服务的标志 ID,如表 15.1(b)中的消息(3)。另外,C 还发送一条认证消息,包括 C 的用户标志 ID、网络地址和时间戳,与票据的可重用性不同,此认证消息仅能使用一次且生命期极短。当 TGS 接到消息后,用与 AS 共享的密钥 $K_{c,tgs}$ 解密该票据,票据中包含的信息说明用户 C 已得到会话密钥 $K_{c,tgs}$,即相当于宣布"任何使用 $K_{c,tgs}$ 的用户必为 C"。接着,TGS 使用该会话密钥解密认证消息,用得到的信息检查消息来源的网络地址,如匹配,则 TGS 确认该票据的发送者与票据的所有者是一致的。实际上,认证消息说明"在时间 TS_3,AD_C 使用 $K_{c,tgs}$"。注意,票据并不能证明任何人的身份,而只是安全分发密钥的一种形式;而认证消息则用于证明用户的身份。由于认证消息仅能使用一次且生命期极短,因此攻击者同时窃取票据和认证消息的威胁将在后面论述。

TGS 的应答消息为消息(4),与消息(2)的格式相同。该消息用 TGS 和 C 的共享会话密钥加密,且包含 C 与服务器 V 的共享密钥、服务器 V 的标志 ID 以及票据的时间戳。票据自身包含相同的会话密钥。

现在,C 拥有了服务器 V 的一个可重用的服务授权票据。当 C 按消息(5)的方式使用票据时,同样需要一个认证消息。由服务器解密票据,恢复会话密钥,并解密认证消息。

如果需要双向认证,则服务器应按表 15.1 中的消息(6)发送一个应答消息。服务器返回的消息中时间戳的值为认证消息时间戳的值加 1,并用会话密钥加密。C 解密消息后可得到增加后的时间戳,由于消息是被会话密钥加密的,C 可以确信此消息只可能由服务器 V 生成。消息中的内容确保该应答不是一个对以前消息的应答。

最后,客户端与服务器共享一个密钥,该密钥可以用于加密在它们之间传递的消息或交换新的随机会话密钥。

表 15.2 总结了 Kerberos 协议各元素的合理性,图 15.1 简要概括了整个会话的过程。

表 15.2　Kerberos 版本 4 协议中所用到元素的作用

(a)认证服务交换

消息(1)	客户端申请票据授权票据
ID_C	告知 AS 客户端的用户标志
ID_{tgs}	告知 AS 用户请求访问 TGS
TS_1	使 AS 能验证客户端时钟是否与 AS 时钟同步
消息(2)	AS 返回票据授权票据
K_c	基于用户口令的密钥使得 AS 与客户端能验证口令,保护消息(2)的内容
$K_{c,tgs}$	客户端可访问的会话密钥,由 AS 创建,使得客户端和 TGS 在不需要共享永久密钥的前提下安全交换信息
ID_{tgs}	标志该票据是为 TGS 生成的
TS_2	通知客户端票据发放的时间戳
$Lifetime_2$	通知客户端票据的生命期
$Ticket_{tgs}$	客户端用于访问 TGS 的票据

(续表)

(b)服务授权票据交换

消息(3)	客户端申请服务授权票据
ID_V	告知 TGS 用户希望访问的服务器 V
$Ticket_{tgs}$	告知 TGS 该用户已被 AS 认证
$Authenticator_c$	客户端生成的合法票据
消息(4)	TGS 返回服务授权票据
$K_{c,tgs}$	用 C 与 TGS 共享的密钥保护消息(4)的内容
$K_{c,v}$	客户端可访问的会话密钥,由 TGS 创建,使得客户端和服务器在不需要共享永久密钥的前提下安全交换信息
ID_V	标志该票据是为服务器 V 生成的
TS_4	通知客户端票据发放的时间戳
$Ticket_V$	客户端用于访问服务器 V 的票据
$Ticket_{tgs}$	重用,以免用户重新输入口令
K_{tgs}	由 AS 和 TGS 共享的密钥加密的票据,防止伪造
$K_{c,tgs}$	TGS 可访问的会话密钥,用于解密认证消息即认证票据
ID_C	标志票据的合法所有者
AD_C	防止票据在与申请票据时的不同工作站上使用
ID_{tgs}	向服务器确保票据解密正确
TS_2	通知 TGS 票据发放的时间
$Lifetime_2$	防止票据过期后继续使用
$Authenticator_c$	向 TGS 确保此票据所有者与票据发放时的所有者相同,用短生命期防止重用
$K_{c,tgs}$	用客户端与 TGS 共享的密钥加密认证消息,防止伪造
ID_C	票据中必须与认证消息匹配的标志 ID
AD_C	票据中必须与认证消息匹配的网络地址
TS_3	通知 TGS 认证消息的生成时间

(c)客户/服务器认证交换

消息(5)	客户端申请服务
$Ticket_V$	向服务器证明该用户通过了 AS 的认证
$Authenticator_c$	客户端生成的合法票据
消息(6)	可选:客户端认证服务器
$K_{c,v}$	向客户端 C 证明该消息来源于服务器 V
$TS_5 + 1$	向客户端 C 证明该应答不是对原来消息的应答
$Ticket_v$	可重用,使得用户在多次使用同一服务器时不需要向 TGS 申请新票据
K_v	用 TGS 与服务器共享的密钥加密的票据,防止仿造
$K_{c,v}$	客户端可访问的会话密钥,用于解密认证消息
ID_C	标志票据的合法所有者
AD_C	防止票据在与申请票据时的不同工作站上使用
ID_V	确保服务器能正确解密票据
TS_4	通知服务器票据发放的时间
$Lifetime_4$	防止票据超时使用
$Authenticator_c$	向服务器确保此票据所有者与票据发放时的所有者相同,用短生命期防止重用
$K_{c,v}$	用客户端与服务器共享的密钥加密的认证消息,防止假冒
ID_C	票据中必须与认证消息匹配的标志 ID
AD_C	票据中必须与认证消息匹配的网络地址
TS_5	通知服务器认证消息的生成时间

Kerberos 域和多重 Kerberi

Kerberos 环境包括 Kerberos 服务器、若干客户端和若干应用服务器:

(1) Kerberos 服务器必须有存放用户标志(UID)和用户口令的数据库。所有用户必须在 Kerberos 服务器注册。

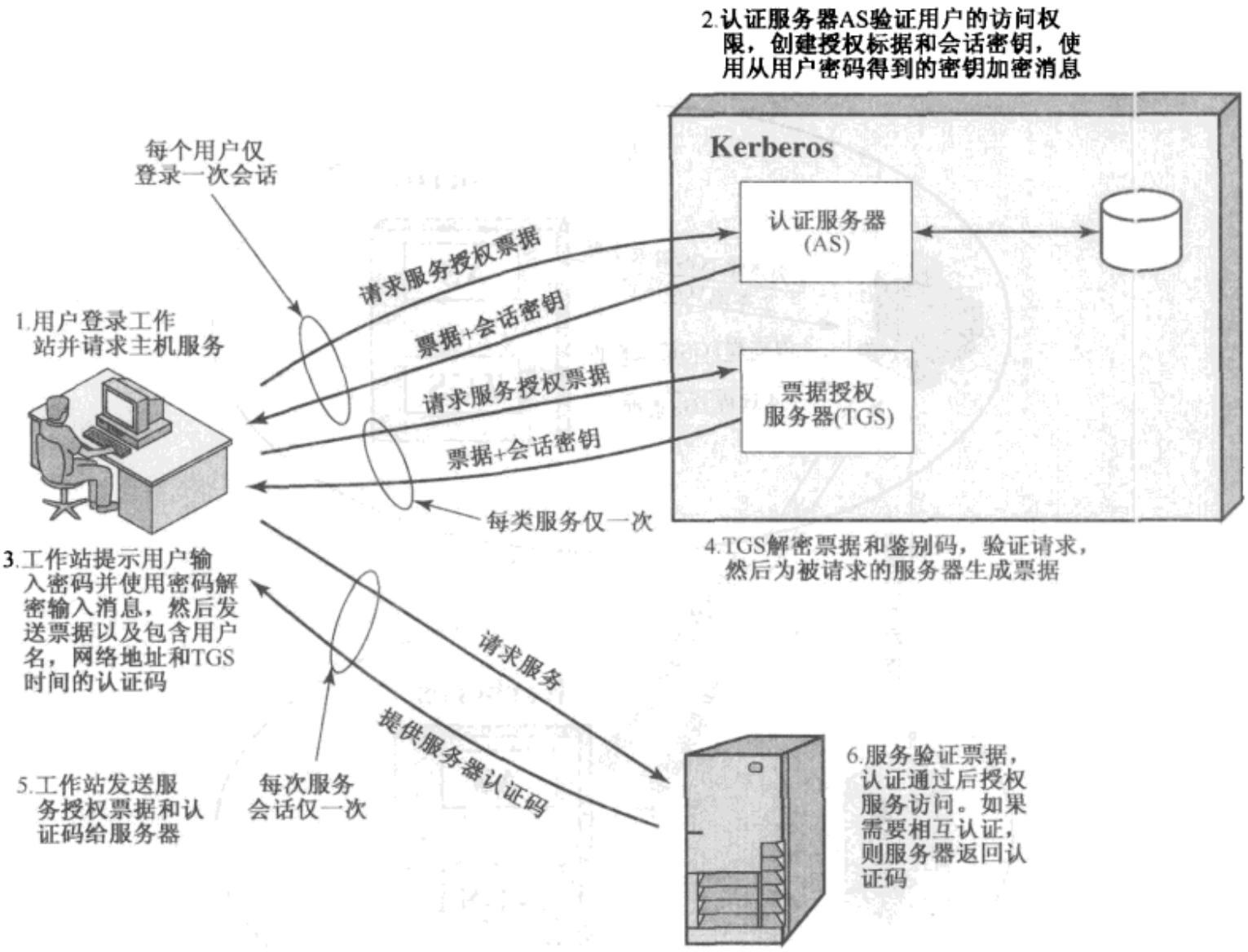


图 15.1　Kerberos 概述

（2）Kerberos 服务器必须与每个应用服务器共享一个特定密钥，所有应用服务器必须在 Kerberos 服务器注册。

这种环境被称为一个 Kerberos 域。Kerberos 域是一组受管节点，它们共享同一 Kerberos 数据库。Kerberos 数据库驻留在 Kerberos 主控计算机系统上，该计算机系统应位于物理上安全的房间内。Kerberos 数据库的只读副本也可以驻留在其他 Kerberos 计算机系统上。但是，对数据库的所有更改都必须在主控计算机系统进行。更改或访问 Kerberos 数据库要求有 Kerberos 主控密码。这里还有一个概念就是 Kerberos 主体。Kerberos 主体是 Kerberos 系统知道的服务或用户。每个 Kerberos 主体通过主体名称进行标志。主体名称由三部分组成：服务或用户名称、实例名称以及域名。

隶属于不同行政机构的客户/服务器网络通常构成了不同域，在一个 Kerberos 服务器中注册的客户与服务器属于同一个行政区域，但由于一个域中的用户可能需要访问另一个域中的服务器，而某些服务器也希望能给其他域的用户提供服务，所以也应该为这些用户提供认证。

Kerberos 提供了一种支持这种域间认证的机制。为支持域间认证，应满足第三个需求：

（3）每个互操作域的 Kerberos 服务器应和其他域的服务器共享一个密钥，两个域的 Kerberos 服务器应相互注册。

这种模式要求一个域的 Kerberos 服务器必须信任其他域的 Kerberos 服务器对其用户的认证。另外，其他域的应用服务器也必须信任第一域中的 Kerberos 服务器。

有了以上规则，就可以用图 15.2 来描述该机制：当用户访其他域的服务时，须获得其他域中

该服务的服务授权票据,用户按照通常的程序与本地 TGS 交互,并申请获得远程 TGS(另一个域的 TGS)的票据授权票据。客户端可以向远程 TGS 申请远程 TGS 域中服务器的服务授权票据。

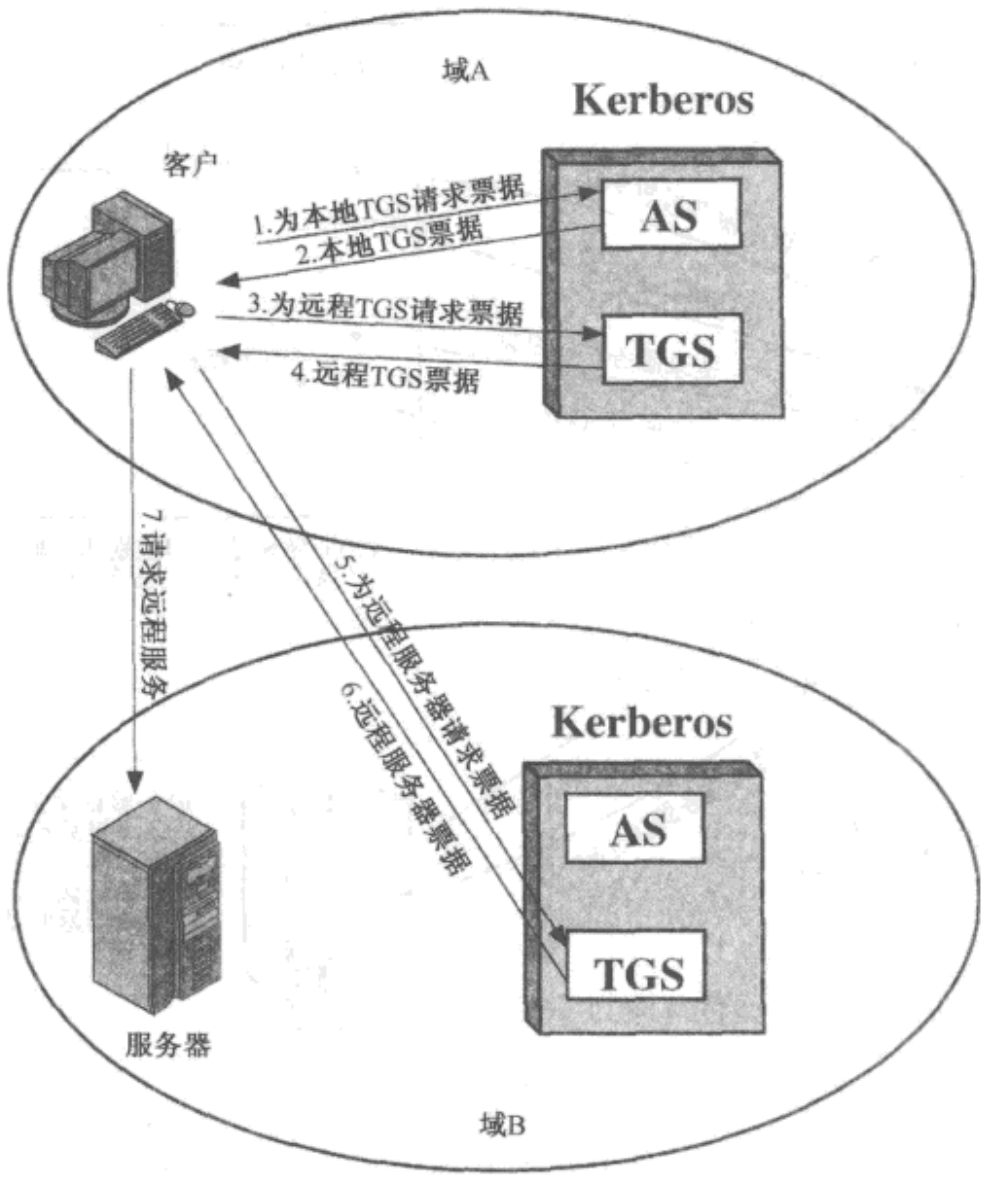


图 15.2 请求其他域的服务

图 15.2 中交换的消息细节如下(与表 15.1 比较):

(1) $C \rightarrow AS$: $ID_C \| ID_{tgs} \| TS_1$

(2) $AS \rightarrow C$: $E(K_c, [K_{c,tgs} \| ID_{tgs} \| TS_2 \| Lifetime_2 \| Ticket_{tgs}])$

(3) $C \rightarrow TGS$: $ID_{tgsrem} \| Ticket_{tgs} \| Authenticator_c$

(4) $TGS \rightarrow C$: $E(K_{c,tgs}, [K_{c,tgsrem} \| ID_{tgsrem} \| TS_4 \| Ticket_{tgsrem}])$

(5) $C \rightarrow TGS_{rem}$: $ID_{vrem} \| Ticket_{tgsrem} \| Authenticator_c$

(6) $TGS \rightarrow C$: $E(K_{c,tgsrem}, [K_{c,vrem} \| ID_{vrem} \| TS_6 \| Ticket_{vrem}])$

(7) $C \rightarrow V_{rem}$: $Ticket_{vrem} \| Authenticator_c$

送往远程服务器(V_{rem})的票据表明了用户原认证所在的域,服务器可以决定是否接受远程请求。

上述方式带来的一个问题是,对多域的可伸缩性不大。如果有 N 个域,则必须有 $N(N-1)/2$ 次安全密钥交换,每个 Kerberos 域可以与其他 Kerberos 域交互。

15.3.3 Kerberos 版本 5

Kerberos 版本 5 在 RFC 4120 中有详细描述,介绍了版本 4 的一些改进[KOHL94]。在此,我们首先介绍版本 4 与版本 5 的不同,接着讨论第 5 版协议。

版本 4 与版本 5 的区别

版本 5 从两个方面阐述了版本 4 的局限性:环境缺陷和技术不足。我们先简单介绍这两个方面的改进①。

Kerberos 版本 4 是为在 Athena 项目环境中使用而开发的,而没有过多地考虑一般情况,这就导致了以下一些环境缺陷:

(1) **加密系统依赖性**:版本 4 使用 DES,因此,它依赖于 DES 的强度和输出限制。版本 5 用加密类型标记密文,使得可以使用任何加密技术。用类型和长度标记的密钥允许同一个密钥在不同算法中使用,并允许在给定算法中具有不同的描述。

(2) **Internet 协议依赖性**:版本 4 需要使用 IP 地址,不支持其他地址类型,如 ISO 网络地址。版本 5 用类型和长度标记网络地址,允许使用任何类型的网络地址。

(3) **消息字节顺序**:版本 4 中,由消息的发送者用标记说明规定消息的字节顺序,而不遵循已有的惯例。在版本 5 中,所有消息结构按照抽象语法表示(ASN. 1)和基本编码规则(BER)规定,确定消息字节顺序。

(4) **票据的生命期**:版本 4 的生命期用一个 8 位表示,每单位代表 5 分钟,因此,最大生命期为 $2^8 \times 5 = 1280$ 分钟,约为 21 小时,但这对某些应用不够长。在版本 5 中,票据中包含了精确的起始时间和终止时间,允许票据拥有任意长度的生命期。

(5) **向前认证**:版本 4 中,发给客户端的证书不能转发给其他用户进行其他相关操作。此操作是指服务器为了完成客户端请求的服务而请求其他服务器协作的能力,例如,客户端申请打印服务器的服务,而打印服务器需要利用客户端证书访问文件服务器得到客户文件。版本 5 提供这项功能。

(6) **域间认证**:版本 4 中,N 个域的互操作需要 $O(N^2)$ 个 Kerberos-to-Kerberos 关系。版本 5 中支持一种需要较少连接的方法。

除了这些环境限制,版本 4 中还存在一些技术不足。大多数不足在[BELL90]中有详细叙述,版本 5 试图予以解决。其不足如下:

(1) **两次加密**:在表 15.1 中,提供给客户端的票据需要经过两次加密[消息(2)和消息(4)],第一次使用的是目标服务器的密钥,第二次使用的是客户端密钥,而第二次加密并不是必需的。

(2) **PCBC 加密**:版本 4 加密所使用 DES 的非标准模式 PCBC(Propagating Cipher Block chaining)②,已被证明易受交换密码块攻击[KOHL89]。PCBC 试图在加密操作中提供完整性检查。版本 5 提供了精确的完整性检查机制,并能够用标准的 CBC 模式加密。特别地,一份消息的校验码或 Hash 特征码优先用 CBC 加密。

(3) **会话密钥**:每个票据包含一个会话密钥,用于客户端加密认证信息,并与票据一起送往服务器。另外,会话密钥还可用于保护客户端与服务器间会话的消息。然而,多次使用同一个票据获得特定服务器的服务,将会增大攻击者从客户端或服务器获得以前的会话消息的可能性。在版本 5 中,客户端与服务器可以协商得到子会话密钥,使得每个密钥仅被使用一次。这种新的客户端访问方式将会导致一种新的子密钥生成。

(4) **口令攻击**:这两种版本均易受到口令攻击。从 AS 发往客户端的消息是用基于客户端用户口令的密钥加密的③,攻击者可以捕获消息,并通过尝试各种口令解密。如果某一次解

---

① 下面的讨论参照[KOHL94]。

② 这将在附录 15A 中介绍。

③ 附录 15A 将讲述口令与加密密钥的映射。

密成功,则攻击者即可得到客户端口令,进而使用该口令从 Kerberos 获取认证证书。这与第 20 章中描述的口令攻击类型相同,用相同的对策处理。版本 5 提供了一种称为预认证的机制使得口令攻击更为困难,但无法杜绝它。

版本 5 认证会话

表 15.3 描述了版本 5 的基本会话,可以与版本 4 的表 15.1 进行对比。

表 15.3 Kerberos 版本 5 消息交换

(a)认证服务交换:获取票据授权票据	
(1) C → AS	$\text{Options} \parallel \text{ID}_C \parallel \text{Realm}_c \parallel \text{ID}_{tgs} \parallel \text{Times} \parallel \text{Nonce}_1$
(2) AS → C	$\text{Realm}_c \parallel \text{ID}_C \parallel \text{Ticket}_{tgs} \parallel K_c[K_{c,tgs} \parallel \text{Times} \parallel \text{Nonce}_1 \parallel \text{Realm}_{tgs} \parallel \text{ID}_{tgs}]$
	$\text{Ticket}_{tgs} = E(K_{tgs}, [\text{Flags} \parallel K_{c,tgs} \parallel \text{Realm}_c \parallel \text{ID}_C \parallel \text{AD}_C \parallel \text{Times}])$
(b)服务授权票据交换:获取服务授权票据	
(3) C → TGS	$\text{Options} \parallel \text{ID}_V \parallel \text{Times} \parallel \text{Nonce}_2 \parallel \text{Ticket}_{tgs} \parallel \text{Authenticator}_c$
(4) TGS → C	$\text{Realm}_c \parallel \text{ID}_C \parallel \text{Ticket}_V \parallel E(K_{c,tgs}, [K_{c,v} \parallel \text{Times} \parallel \text{Nonce}_2 \parallel \text{Realm}_V \parallel \text{ID}_V])$
	$\text{Ticket}_{tgs} = E(K_{tgs}, [\text{Flags} \parallel K_{c,tgs} \parallel \text{Realm}_c \parallel \text{ID}_C \parallel \text{AD}_C \parallel \text{Times}])$
	$\text{Ticket}_V = E(K_v, [\text{Flags} \parallel K_{c,v} \parallel \text{Realm}_c \parallel \text{ID}_C \parallel \text{AD}_C \parallel \text{Times}])$
	$\text{Authenticator}_c = E(K_{c,tgs}, [\text{ID}_C \parallel \text{Realm}_c \parallel \text{TS}_1])$
(b)服务授权票据交换:获取服务授权票据	
(5) C → V	$\text{Options} \parallel \text{Ticket}_V \parallel \text{Authenticator}_c$
(6) V → C	$E_{K_{c,v}}[\text{TS}_2 \parallel \text{Subkey} \parallel \text{Seq\#}]$
	$\text{Ticket}_V = E(K_v, [\text{Flags} \parallel K_{c,v} \parallel \text{Realm}_c \parallel \text{ID}_C \parallel \text{AD}_C \parallel \text{Times}])$
	$\text{Authenticator}_c = E(K_{c,v}, [\text{ID}_C \parallel \text{Realm}_c \parallel \text{TS}_2 \parallel \text{Subkey} \parallel \text{Seq\#}])$

首先考虑认证服务交换,消息(1)是客户端请求票据授权票据的过程。如前所述,它包括用户和 TGS 的标志,新增的元素包括:

- **Realm**:标志用户所属的域。
- **Options**:用于请求在返回的票据中设置指定的标志位,如下所述。
- **Times**:用于客户端请求在票据中设置时间:
  - —from:请求票据的起始时间
  - —till:请求票据的过期时间
  - —rtime:请求 till 更新时间
- **Nonce**:在消息(2)中重复使用的临时交互号,用于确保应答是刷新的,且未被攻击者使用。

消息(2)返回票据授权票据、标志客户端信息和一个用用户口令形成的密钥加密的数据块。该数据块包含在客户端和 TGS 间使用的会话密钥、消息(1)中设定的时间和临时交互号以及 TGS 的标志信息。票据本身包含会话密钥、客户端的标志信息、需要的时间值、影响票据状态的标志和选项。这些标志为版本 5 带来的一些新功能将在以后讨论。

比较版本 4 和版本 5 的服务授权票据交换。两者的消息(3)均包含认证码、票据和请求服务的名字。在版本 5 中,还包括与消息(1)类似的票据请求的时间、选项和一个临时交互号;认证码的作用与版本 4 中的相同。

消息(4)与消息(2)结构相同,返回票据和一些客户端需要的信息,后者被客户端和 TGS 共享的会话密钥加密。

最后，版本 5 对客户/服务器认证交换进行了一些改进，如在消息(5)中，客户端可以请求选择双向认证选项。认证也增加了以下新域：

- **子密钥**：客户端选择一个子密钥保护某一特定应用会话，如果此域被忽略，则使用票据中的会话密钥 $K_{c,v}$。
- **序号**：可选域，用于说明在此次会话中服务器向客户端发送消息的序列号，将消息排序可以防止重播攻击。

如果请求双向认证，则服务器按消息(6)应答。该消息中包含从认证者处得到的时间戳。在版本 4 中该时间戳被加 1，而在版本 5 中，由于攻击者不可能在不知道正确密钥的情况下创建消息(6)，因此不需要对时间戳进行上述处理。如果有子密钥域存在，则覆盖消息(5)中相应的子密钥域。而选项序列号则说明了客户端使用的起始序列号。

票据标志

版本 5 票据中的标志域支持许多版本 4 中没有的功能。表 15.4 总结了票据中可能包含的标志。

表 15.4　Kerberos 版本 5 标志

INITIAL	按照 AS 协议发布的服务授权票据而不是基于票据授权票据发布的
PRE-AUTHENT	在初始认证中，客户在授予票据前即被 KDC 认证
HW-AUTHENT	初始认证协议要求使用带名客户端独占的硬件资源
RENEWABLE	告知 TGS 此票据可用于获得最近超时票据的新票据
MAY-POSTDATE	告知 TGS 事后通知的票据可能是基于票据授权票据
POSTDATED	表示该票据是事后通知的，终端服务器可以检查 authtime 域，查看认证发生的时间
INVALID	不合法的票据在使用前必须通过 KDC 使之合法化
PROXIABLE	告知 TGS 根据当前票据可以发放给不同网络地址新的服务授权票据
PROXY	表示该票据是一个代理
FORWARDABLE	告知 TGS 根据此票据授权票据可以发放给不同网络地址新的票据授权票据
FORWARDED	表示该票据或是经过转发的票据或是基于转发的票据授权票据认证后发放的票据

标志 INITIAL 用于表示票据是由 AS 发放的，而不是由 TGS 发放的。当客户端向 TGS 申请服务授权票据时，必须拥有 AS 发放的票据授权票据。在版本 4 中，这是唯一获得服务授权票据的方法。版本 5 提供了一种可以直接从 AS 获得服务授权票据的手段，其机制是：一个服务器(如口令变更服务器)希望知道客户端口令近来已被验证。

标志 PRE-AUTHENT 如果被设置，则表示当 AS 接收初始请求[消息(1)]时，在发放票据前应先对客户端进行认证，其预认证的确切格式在此未做详细说明。例如，MIT 实现版本 5 时，加密的时间戳默认设置为预认证。当用户想得到一个票据时，他将一个带有临时交互号的预认证块、版本号和时间戳用基于客户口令的密钥加密后送往 AS。AS 解密后，如果预认证块中的时间戳不在允许的时间范围之内(时间间隔取决于时钟迁移和网络延迟)，则 AS 不返回票据授权票据。另一种可能性是使用智能卡(smart card)生成不断变化的口令，将其包含在预认证消息中。卡所生成的口令基于用户口令，但经过了一定的变换，使得生成的口令具有随机性，防止了简单猜测口令的攻击。如果使用了智能卡或其他相似设备，则设置 HW-AUTHENT 标志。

当票据的生命期较长时，就在相当一段时间内存在票据被攻击者窃取并使用的威胁。而缩短票据的生命期可降低这种威胁，主要开销将在于获取新票据，如对票据授权票据而言，客户端可以存储用户密钥(危险性较大)或重复向用户询问口令来解决。一种解决方案是使用可重新生成的票据。一个具有标志 RENEWABLE 的票据中包含两个有效期：一个是此特定票据的有效期，另一个是最大许可值的有效期。客户端可以通过将票据提交给 TGS 申请得到新的有效期的方法获得

新票据。如果这个新的有效期在最大有效期的范围之内,TGS 即发放一个具有新的会话时间和有效期的新票据。这种机制的好处在于 TGS 可以拒绝更新已报告为被盗用的票据。

客户端可请求 AS 提供一个具有标志 MAY-POSTDATE 的票据授权票据。客户端可以使用此票据从 TGS 申请一个具有标志 POSTDATED 或 INVALID 的票据,然后,客户端提交合法的超时票据。这种机制在服务器上运行批处理任务和经常需要票据时特别有用。客户端可以通过一次会话得到一组具有扩展性时间值的票据。但第一个票据被初始化为非法标志,当执行进行到某一阶段需要某一特定票据时,客户端即将相应的票据合法化。用这种方法,客户端就不再需要重复使用票据授权票据去获取服务授权票据。

在版本 5 中,服务器可以作为客户端的代理,获取客户端的信任和权利,并向其他服务器申请服务。如果客户端想使用这种机制,需要申请获得一个带有 PROXIABLE 标志的票据授权票据。当此票据传给 TGS 时,TGS 发布一个具有不同网络地址的服务授权票据。该票据的标志 PROXY 被设置,接收到这种票据的应用可以接收它或请求进一步认证,以提供审计跟踪①。

代理在转发时有一些限制。如果票据被设置了 FORWARDABLE 标志,TGS 给申请者发放一个具有不同网址和 FORWARDED 标志的票据授权票据,于是此票据可以被送往远程 TGS。这使得用户端在不需要每个 Kerberos 都包含与其他域中的 Kerberos 服务器共享密钥的前提下,可以访问其他域的服务器。例如,各个域有层次结构时,客户端可以向上遍历到一个公共节点后再向下到达目标域。每一步都是转发票据授权票据到图中的下一个 TGS。

15.4 基于非对称加密的远程用户认证

15.4.1 双向认证

在第 14 章中,介绍了将公钥加密应用于会话密钥分发的方案(参见图 14.8),该协议假定双方都拥有对方最新的公钥,但这个假定恐怕不能被满足。

[DENN81]提出了使用时间戳的协议:

(1) $A \rightarrow AS: \quad ID_A \| ID_B$

(2) $AS \rightarrow A: \quad E(PR_{as}, [ID_A \| PU_a \| T]) \| E(PR_{as}, [ID_B \| PU_b \| T])$

(3) $A \rightarrow B: \quad E(PR_{as}, [ID_A \| PU_a \| T]) \| E(PR_{as}, [ID_B \| PU_b \| T]) \| E(PU_b, E(PR_a, [K_s \| T]))$

这种情况下,涉及的中心系统作为一个认证服务器(AS),因为事实上它并不负责密钥分发,AS 提供公钥证书。会话密钥由 A 选择并加密,因此,AS 不会知道,时间戳能防止窃取密钥的重放攻击。

该协议很简洁,但要求时钟同步。另一个是 Woo 和 Lam[WOO92a]提出的使用临时交互号的协议,包括以下几步:

(1) $A \rightarrow KDC: \quad ID_A \| ID_B$

(2) $KDC \rightarrow A: \quad E(PR_{auth}, [ID_B \| PU_b])$

(3) $A \rightarrow B: \quad E(PU_b, [N_a \| ID_A])$

(4) $B \rightarrow KDC: \quad ID_A \| ID_B \| E(PU_{auth}, N_a)$

(5) $KDC \rightarrow B: \quad E(PR_{auth}, [ID_A \| PU_a]) \| E(PU_b, E(PR_{auth}, [N_a \| K_s \| ID_B]))$

① 对代理能力应用的讨论参见参考文献[NEUM93b]。

(6) B→A:　$E(PU_a, [E(PR_{auth}, [N_a \| K_s \| ID_B]) \| N_b])$

(7) A→B:　$E(K_s, N_b)$

第 1 步,A 通知 KDC 它想要与 B 建立安全的连接。第 2 步,KDC 返回 B 的公钥证书给 A。第 3 步,使用 B 的公钥,A 发送临时交互号 N_a 给 B,表示想要和 B 建立通信。第 4 步,B 向 KDC 请求 A 的公钥证书以及会话密钥,因为 B 将 A 的临时交互号也发给了 KDC(该临时交互号用 KDC 的公钥加密),所以 KDC 可以用该临时交互号来标记会话密钥。第 5 步,KDC 将 A 的公钥证书和信息 $\{N_a, K_s, ID_B\}$ 返回给 B。该信息指出 K_s 由 KDC 产生,能代表 B 并与 N_a 相关联,K_s 和 N_a 的绑定能使 A 确定 K_s 是最新的,用 KDC 的私钥加密允许 B 验证信息来自 KDC,同时使用 B 的公钥加密是为了防止其他想和 A 建立不正当连接的实体使用。第 6 步,将用 KDC 的私钥加密的 $\{N_a, K_s, ID_B\}$ 和 B 的临时交互号 N_b 一起转发给 A,转发给 A 的这些信息都是用 A 的公钥加密过的。A 得到会话密钥 K_s 后用其加密 N_b 返回给 B,最后的消息能确保 B 知道 A 已经获知会话密钥。

这似乎是一个考虑了各种攻击的安全协议,然而,作者自己找出了缺陷,并在[WOO92b]中做了修正:

(1) A→KDC:　$ID_A \| ID_B$

(2) KDC→A:　$E(PR_{auth}, [ID_B \| PU_b])$

(3) A→B:　$E(PU_b, [N_a \| ID_A])$

(4) B→KDC:　$ID_A \| ID_B \| E(PU_{auth}, N_a)$

(5) KDC→B:　$E(PR_{auth}, [ID_A \| PU_a]) \| E(PU_b, E(PR_{auth}, [N_a \| K_s \| ID_A \| ID_B]))$

(6) B→A:　$E(PU_a, [E(PR_{auth}, [N_a \| K_s \| ID_A \| ID_B]) \| N_b])$

(7) A→B:　$E(K_s, N_b)$

在第 5 步和第 6 步中,A 的标志 ID_A 被加入了用 KDC 加密的项目集,这将会话密钥 K_s 和双方标志绑定在一起。ID_A 的加入说明了临时交互号 N_a 在 A 产生的所有临时交互号中是唯一的,但并非在双方产生的临时交互号中都是唯一的,即 $\{ID_A, N_a\}$ 才能唯一地标志连接请求来自于 A。

该例和之前描述的协议可以说明,看起来安全的协议也需要不断地分析修正。这些例子强调了在认证领域中想要完美是很困难的。

15.4.2 单向认证

我们已经介绍了适合电子邮件的公钥加密方法,对全部信息的直接加密包括为了保密[参见图 12.1(b)]、认证[参见图 12.1(c)]或者两者都有[参见图 12.1(d)]。这些方法要求或者发送者已知接收者的公钥(保密),或者接收者知道发送者的公钥(认证),或者两者皆有(保密和认证)。另外,对可能很长的消息来说,公钥算法必须执行一次或者两次。

如果保密性是首先要考虑的,以下方法可能更有效:

$$A \rightarrow B: \quad E(PU_b, K_s) \| E(K_s, M)$$

此时,消息用会话密钥加密,A 用 B 的公钥加密会话密钥,只有 B 能解密得到会话密钥,然后使用该密钥解密消息,该方法比只用 B 的公钥简单的加密全部消息更有效。

如果主要考虑认证,则在图 13.2 中描述的数字签名就可以满足:

$$A \rightarrow B: \quad M \| E(PR_a, H(M))$$

该方法可以保证 A 事后不能否认发送过消息,但该技术对其他敌手也是公开的。Bod 写了一封包含一个能节省公司钱财的办法的邮件给他的老板 Alice,附上自己的签名后发送进电子邮件系统,最后该消息会被传送到 Alice 的邮箱。但是,假定 Max 听说了 Bob 的想法,在邮件传送之前

访问了邮件列表,找到 Bob 的邮件,去掉其签名并换成自己的,后将邮件传送给 Alice,最终 Max 会得到该想法的所有权。

为了防止这种情况,消息和签名都要用接收者的公钥加密:

$$A \rightarrow B: \quad E(PU_b, [M \| E(PR_a, H(M))])$$

后两种方案要求 B 知道 A 的公钥并且是新鲜的,用第 14 章介绍的数字证书来保证。则有

$$A \rightarrow B: \ M \| E(PR_a, H(M)) \| E(PR_{as}, [T \| ID_A \| PU_a])$$

A 发送给 B 的信息包括:消息 M、A 对消息的签名和用认证服务器私钥加密的 A 的证书。消息接收者首先使用证书得到 A 的公钥并验证其真实性,然后用该公钥来验证消息 M。如果同时要求保密,则全部的消息都要使用 B 的公钥加密,或者,用会话密钥加密全部消息,该会话密钥用 B 的公钥加密后传送给 A。这种方法将在第 18 章介绍。

15.5 联合身份管理

在多个企业以及大量应用的身份管理方案中,联合身份管理是一个相对较新的概念,它是一种能够处理多个单位、大量应用并支持数千甚至数百万用户的公共身份管理方案。首先介绍身份管理的概念,然后介绍联合身份管理。

15.5.1 身份管理

身份管理是一个集中式的、自动的方法,提供雇员或者其他授权的个人对资源拥有企业范围的访问。身份管理的重点是为用户(人或者进程)定义一个身份,为该身份关联属性,并完成用户的身份认证。身份管理系统的主要概念是 SSO(Single Sign-On),SSO 能使用户在一次认证之后访问所有网络资源。

[PELT07]中列出了身份管理系统的要素:

- **认证**(authentication):证明用户和提供的用户名相匹配。
- **授权**(authorization):在认证的基础上,授权访问特定服务。
- **审计**(accounting):一个程序,用来记录访问和认证过程。
- **物质供应**(provisioning):用户在系统中的登录。
- **工作流自动化**(workflow automation):商业进程中的数据移动。
- **管理**(delegated administration):一种基于角色的访问控制管理。
- **口令同步**(password synchronization):为单点登录(SSO)或者简化登录(RSO)创建进程,SSO 能使用户在一次认证登录之后访问所有网络资源,RSO 可能需要多次认证登录,但比每个资源和服务都要认证时需要的用户开销要少。
- **自助口令重置**(self-service password reset):支持用户修改自己的密码。
- **联合**(federation):可以使得认证和许可从一个系统传递到另一个,通常是在多个企业之间,因此可以减少用户的认证次数。

注意,Kerberos 包含许多身份管理系统的要素。

如图 15.3 所示,[LINN06]说明了通用身份管理结构中的实体和数据流。一个当事人(principal)就是一个身份持有者,也就是一个想要访问网络资源和服务的人(用户),用户设施、代理程序和服务器系统都可能作为当事人。当事人向身份提供者证明自己,身份提供者为当事人关联认证信息,如属性、一个或者多个身份标志符。

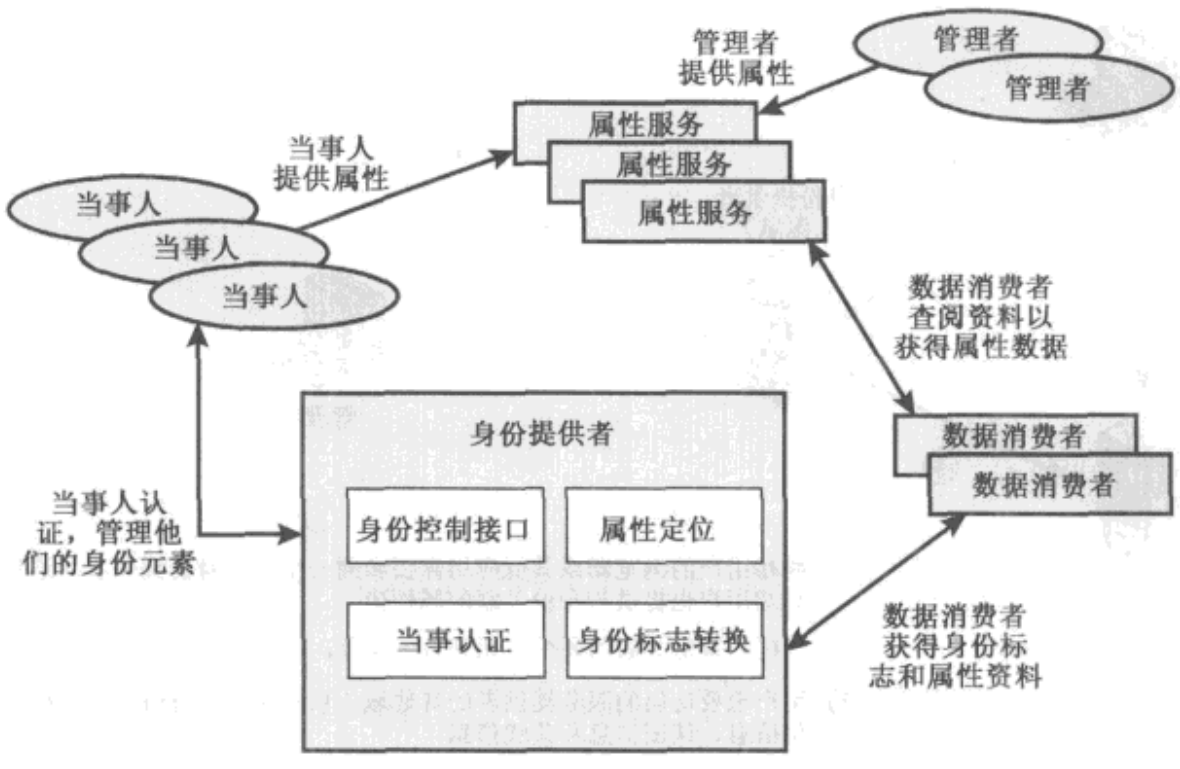


图 15.3　通用的身份管理架构

如今,数字身份包含的属性已不只是简单的标志符和认证信息(如密码和生物计量信息)了。属性服务管理这些属性的创建和维护,如网购用户每次下订单的时候都要提供一个运送地址给网店店主,若用户迁居时该信息需要修改。身份管理使得用户一次性提供这些信息并将其保存,在满足授权和隐私策略时发布给数据消费者。用户可以自己创建一些属性关联到其身份,如地址。管理者也可为用户指派属性,如角色、访问权限和雇员信息。

数据消费者是一些实体,能够得到并使用由身份和属性提供者提供的数据,数据消费者常被用于支持授权决定和收集审计信息,如数据库服务器或者文件服务器,需要客户的证书来决定为该客户提供什么样的访问权限。

15.5.2　身份联合

在本质上,身份联合是身份管理在多个安全域上的一个扩展,这些域包括自治的内部商业单元、外部的商业伙伴以及其他第三方的应用和服务。目的是提供数字身份共享,使得用户只需要一次认证就可以访问多个域的应用和资源。这些域是相对自治或者独立的,因此可以采用非集中化控制,当然,合作的组织之间必须形成一个基于协商和相互信任的标准来安全地共享数字身份。

联合身份管理涉及协议、标准和技术,支持数万用户的身份、身份属性和访问权限在多个企业以及大量的应用间移植。当多个组织执行联合身份管理时,一个组织的雇员能够使用 SSO 访问联盟中其他组织的服务,如一个雇员登录本公司的内部网,经过认证可以在内部网执行授权的功能、访问授权的服务,之后可以从外部网的卫生保健提供者处访问自己的服务,而不需要重新认证。

除了 SSO,联合身份管理也提供了其他的功能,如属性的标准化方式。渐渐地,数字身份包含的属性已经不只是简单的标志符和认证信息(如密码和生物计量信息),属性的例子包括账号、组织的角色、物理定位和文件所有权。一个用户可能有多个身份标志符,每个标志符可能关联唯一的角色,拥有自己的访问权限。

联合身份管理的另一个主要功能是身份映射,不同的安全域可能表示的身份和属性不同,而且,一个人在一个域中的信息总量可能多于其在另一个域的必需的信息总量。联合身份管理协议将一个域中用户的身份和属性映射到另一个域。

图 15.4 给出了通用联合身份管理结构中的实体和数据流。

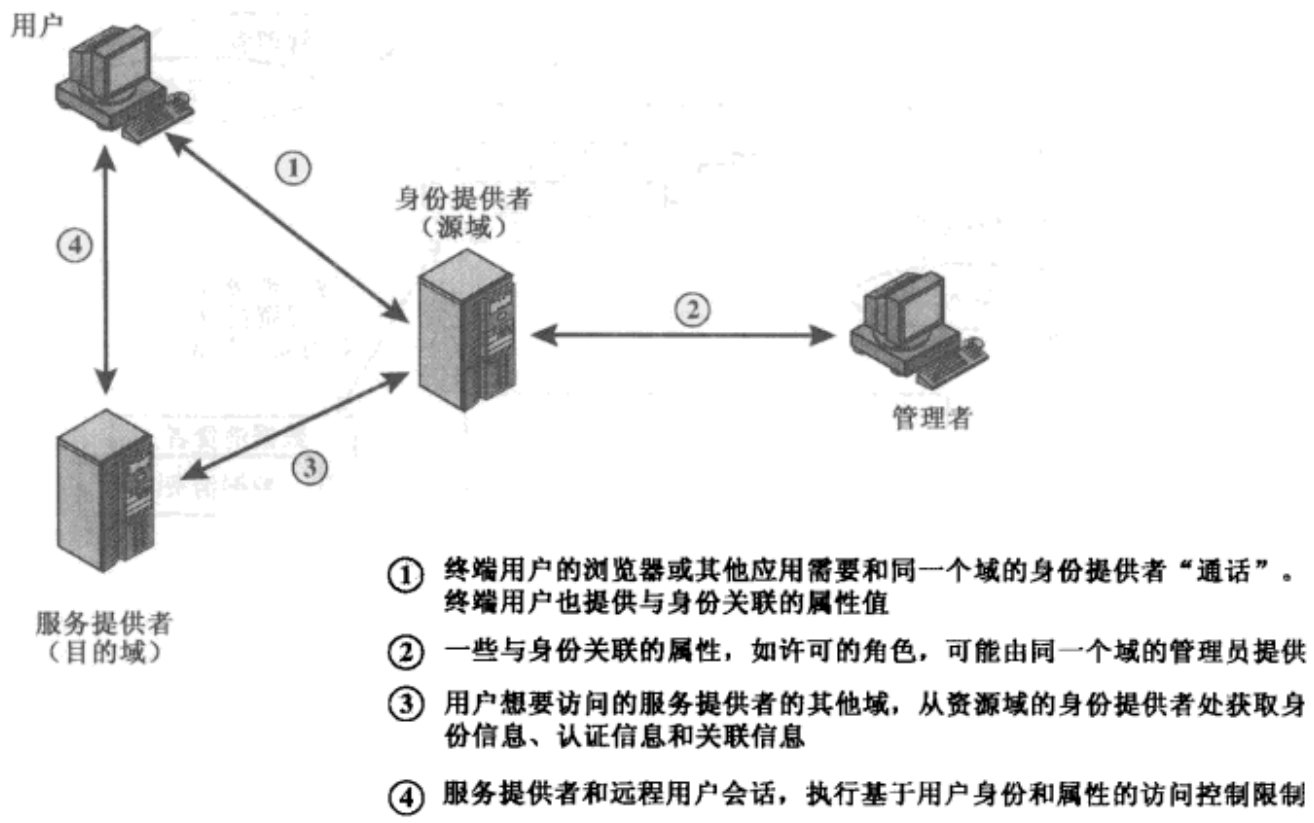


图 15.4 联合身份操作

身份提供者通过和用户以及管理者会话、协议交换来获得属性信息,如网购用户每次下订单的时候都要提供一个运送地址给网店店主,若用户迁居时该信息需要修改。身份管理使得用户一次性提供这些信息并将其保存,在满足授权和隐私策略时发布给数据消费者。

服务提供者是一些实体,能够得到和使用由身份和属性提供者维持和提供的数据,数据消费者常被用于支持授权决定和收集审计信息,如数据库服务器或者文件服务器,需要客户的证书来决定为该客户提供什么样访问权限。一个服务提供者可以和用户以及身份提供者在同一个域,但联合身份管理中服务提供者和用户在不同的域。

标准

为了在不同域或多个系统之间进行安全的身份交换,联合身份管理使用很多标准来构建模块。事实上,组织可以为用户发布一些安全且可被合作伙伴处理的票据。身份联合标准关心的是如何定义这些票据包括内容和格式,如何为交换票据和执行管理任务提供协议。这些任务包括构建系统来执行属性传递和身份映射,以及执行登录和审计功能。

联合身份最根本的标准是安全声明标记语言(SAML),SAML 定义了在线商业伙伴之间的安全信息交换。SAML 以主题声明的形式传输认证信息,而声明是对认证实体发布的主题的描述。

SAML 是由 OASIS(结构化信息标准进步组织)为联合身份管理发布的标准集的一部分,如 WS-Federation 完成基于浏览器的联合,依据安全令牌服务来协调网站服务之间的身份、属性和认证的相互信任。

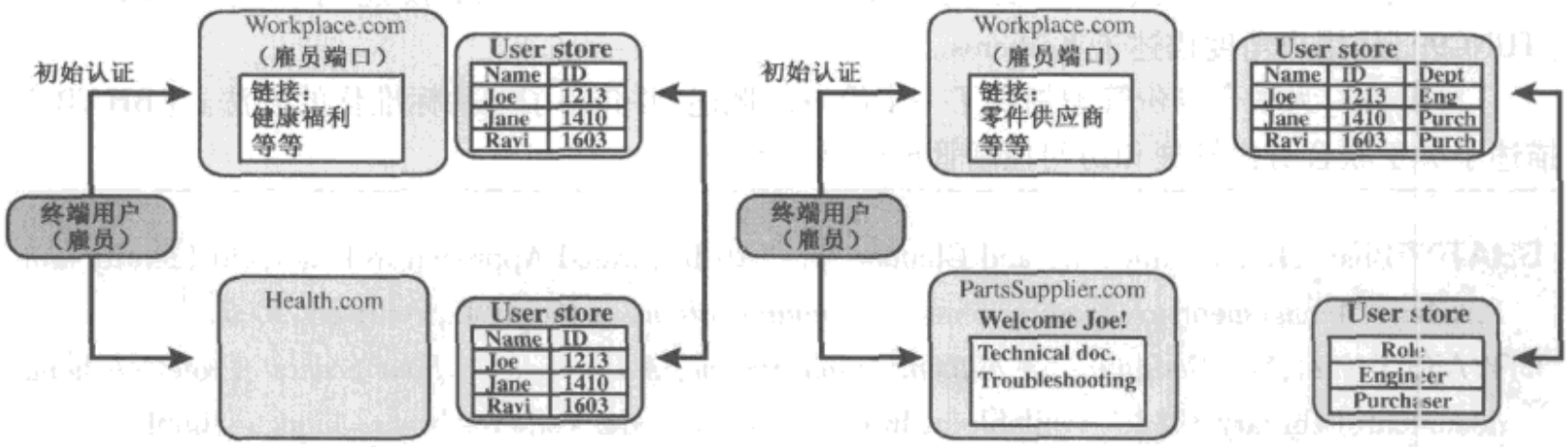
联合身份管理的挑战是整合多种技术、标准和服务来提供一个安全的对用户友好的终端,关键是依靠一些被工业界广泛接受的成熟标准。联合身份管理似乎已经达到了成熟水平。

例子

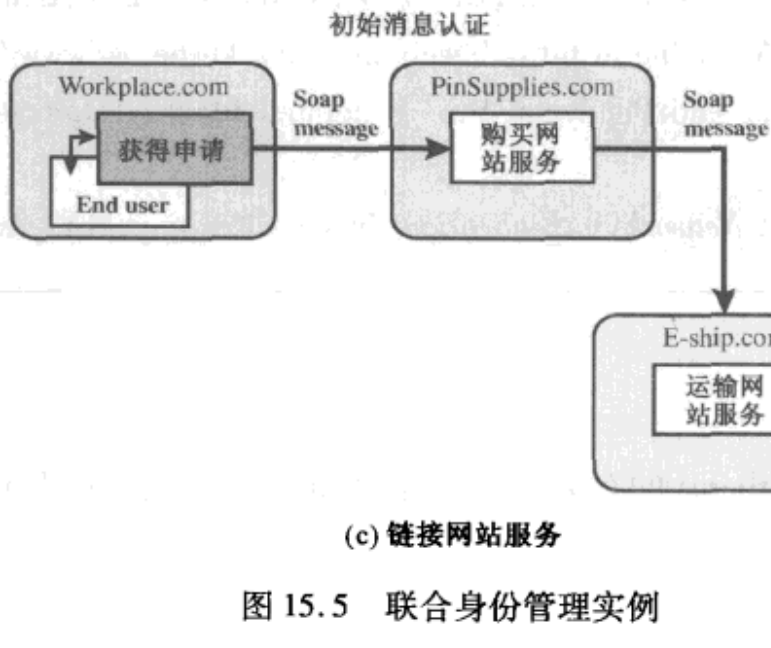
为了对联合身份有更好的认识,我们来看三种应用场景[COMP06]。

第一种方案[参见图 15.5(a)],Workplace. com 联系 Health. com 为雇员提供健康福利,一个雇员使用网站接口打开 Workplace. com,通过一个认证程序访问 Workplace. com 中授权的服务和资源,当雇员点击链接查看健康福利时,她的浏览器将会用安全的方式改变方向到 Health. com。这

两个组织之间是合作交换用户的联合标志符，Health. com 维持 Workplace. com 中每个雇员的用户身份，并为其关联健康福利信息以及访问权限。在这个例子中，两个公司之间的链接是基于账户信息的，用户参与是基于浏览器的。



(a) 基于账户链的联合　　(b) 基于角色的联合



(c) 链接网站服务

图 15.5　联合身份管理实例

图 15.5(b)展示了基于浏览器的第二种方案。PartsSupplier. com 是 Workplace. com 的“零件”供应商，这种情况下，对信息的访问使用基于角色的访问控制(RBAC)。在 Workplace. com 的员工入口处，有一个工程师认证且点击链接可以访问 PartsSupplier. com 的信息，因为已认证用户是一个工程师，所以直接被带去 PartsSupplier. com 网页的技术文件和故障处理部分，而不需要再次认证；同样，一个负责采购的雇员经认证登录 Workplace. com 后到 PartsSupplier. com 购买东西，也不再需要得到 PartsSupplier. com 的认证。这种情况下，PartsSupplier. com 没有 Workplace. com 单个雇员的身份信息，当然，联合双方的链接是基于角色的。

图 15.5(c)所示的情景是基于文件的，而不是基于浏览器的。在第三个例子中，Workplace. com 和 PartsSupplier. com 有购买协议，且后者和 E-Ship. com 有商业关系。一个 Workplace. com 的雇员登录并认证购买后，雇员可得到供应商名单和可订购的部分，用户单击 PinSupplies 按钮会出现一个订购网页(HTML 页面)，然后填写表单并提交。采购程序会产生一个 XML/SOAP 文件，并将该文件插入一个基于 XML 的消息的信封体中，同时将用户的信誉以及 Workplace. com 组织的身份插入消息的信封头部，然后将该消息投递给 PinSupplies. com 的网站购买服务。该服务认证消息并处理请求，然后发送 SOAP 消息给他的运送伙伴来满足此次订购，该消息包括信封头部 PinSupplies. com 的安全令牌以及信封体中的发送项目列表和用户配送信息。运送网站认证请求并处理运送任务。

15.6 推荐读物和网站

为掌握 Kerberos 概念,可参考[BRYA88];Kerberos 的详细叙述可参考[KOHL94]。[TUNG99]从用户角度描述了 Kerberos。

[SHIM05]为联合身份管理提供了一个简单的概述,并介绍了一种标准化的方法。[BHAT07]描述了关于联合身份管理和访问控制管理链接的方法。

BHAT07 Bhatti, R.; Bertino, E.; and Ghafoor, A. "An Integrated Approach to Federated Identity and Privilege Management in Open Systems." *Communications of the ACM*, February 2007.

BRYA88 Bryant, W. *Designing an Authentication System: A Dialogue in Four Scenes.* Project Athena document, February 1988. Available at http://web.mit.edu/kerberos/www/dialogue.html。

KOHL94 Kohl, J.; Neuman, B.; and Ts'o, T. "The Evolution of the Kerberos Authentication Service." in Brazier, F., and Johansen, D. *Distributed Open Systems.* Los Alamitos, CA: IEEE Computer Society Press, 1994. Available at http://web.mit.edu/kerberos/www/papers.html。

SHIM05 Shim, S.; Bhalla, G.; and Pendyala, V. "Federated Identity Management." *Computer*, December 2005.

TUNG99 Tung, B. *Kerberos: A Network Authentication System.* Reading, MA: AddisonWesley, 1999.

推荐网站

- **MIT Kerberos Site**:Kerberos 的有关信息,包括 FAQ、论文、文档以及相关商业产品。
- **MIT Kerberos Consortium**:建立 Kerberos,为世界计算机网络创建通用认证平台。
- **USC/ISI Kerberos Page**:Kerberos 素材。
- **Kerberos Working Group**:开发基于 Kerberos 的标准的 IETF 组。

15.7 关键术语、思考题和习题

关键术语

认证	双向认证	重放攻击
认证服务器	临时交互号	压制重放攻击
联合身份管理	单向认证	票据
身份管理	传播密码块链模式	票据授权服务器(TGS)
Kerberos	域	时间戳
Kerberos 域		

思考题

15.1 给出一个重放攻击的例子。

15.2 列出三个常用的防止重放攻击的方法。

15.3 什么是压制重放攻击?

15.4　Kerberos 主要处理什么问题?

15.5　在网络或者 Internet 上,和用户认证相关联的三个威胁是什么?

15.6　列出三种确保分布式环境中用户认证安全的方法。

15.7　Kerberos 定义的四个要求是什么?

15.8　组成一个完全服务的 Kerberos 环境需要哪些实体?

15.9　在 Kerberos 中,什么是域?

15.10　Kerberos 的版本 4 和版本 5 之间的主要不同有哪些?

习题

15.1　在 15.4 节中,我们介绍了[WOO92a]中提出的针对密钥分发的公钥方案,其修正版中在第 5 步及第 6 步包含 ID_A。该版本主要是为了防止什么攻击?

15.2　上题中的协议可以从 7 步减少到 5 步,有以下的序号:

(1) A→B

(2) A→KDC

(3) KDC→B

(4) B→A

(5) A→B

写出每一步传递的信息。提示:这个协议中最后消息和原协议中最后的消息是一样的。

15.3　参考 15.2 节中对压制重放攻击的描述,回答以下问题:

(a) 给出一个攻击的例子,其中一方的时钟快于 KDC 的时钟。

(b) 给出一个攻击的例子,其中一方的时钟快于另一方的时钟。

15.4　使用临时交互号作为询问的典型方法有三种,假定 N_a 是 A 产生的临时交互号,A 和 B 共享密钥 K,$f()$ 是一个函数(如一个增量),三种用法如下:

Usage 1	Usage 2	Usage 3
(1) A → B: N_a	(1) A → B: $E(K, N_a)$	(1) A → B: $E(K, N_a)$
(2) B → A: $E(K, N_a)$	(2) B → A: N_a	(2) B → A: $E(K, f(N_a))$

15.5　在 PCBC 模型(参见附录 15A 中的图 15.7)中,说明为什么密文块中的一个随机错误会传播给随后的所有明文块?

15.6　假设在 PCBC 模型中,密文块 C_i 和 C_{i+1} 在传输中互换,说明为什么受影响的是明文块 P_i 和 P_{i+1},而不是随后的所有块。

15.7　除了为公钥证书格式提供标准以外,X.509 也指定了一个认证协议,X.509 的之前版本中包含一个安全缺陷,该协议内容如下:

$$\begin{aligned} &A \rightarrow B: \quad A\{t_A, r_A, ID_B\} \\ &B \rightarrow A: \quad B\{t_B, r_B, ID_A, r_A\} \\ &A \rightarrow B: \quad A\{r_B\} \end{aligned}$$

其中,t_A 和 t_B 为时间戳,r_A 和 r_B 为临时交互号,符号 $X\{Y\}$ 标志消息 Y 被 X 传输、加密及签名。X.509 中的叙述表明检验时间戳 t_A 和 t_B 对认证是可选的,但考虑以下例子:假定 A 和 B 在之前的某些情况下使用了该协议,攻击者 C 截获以上三条消息,另外,假定时间戳没有用且全部被设为 0。最后,C 想要伪装为 A 与 B 通信,C 发送截获到的第一条消息给 B:

$$C \rightarrow B: \quad A\{0, r_A, ID_B\}$$

B 以为自己在和 A 通信,而事实上是和 C 在通信,B 回复:

$$B \rightarrow C: \quad B\{0, r'_B, ID_A, r_A\}$$

C 同时通过一些方法引起 A 与其建立初始通信。结果,A 发送给 C 消息如下:

$$A \rightarrow C: \quad A\{0, r'_A, ID_C\}$$

C 用 B 提供临时交互号回复 A:

$$C \rightarrow A: \quad C\{0, r'_B, ID_A, r'_A\}$$

A 回复

$$A \rightarrow C: \quad A\{r'_B\}$$

当然 C 要向 B 证明 B 正在和 A 通信,因此 C 将接收到的如下消息给 B:

$$C \rightarrow B: \quad A\{r'_B\}$$

此时 B 会相信自己正在和 A 通信,而事实是在和 C 通信。找出一种方法可以解决以上问题,且不使用时间戳。

15.8 考虑以下基于非对称加密的单向认证技术:

$$\begin{aligned} A &\rightarrow B: \quad ID_A \\ B &\rightarrow A: \quad R_1 \\ A &\rightarrow B: \quad E(PR_a, R_1) \end{aligned}$$

(a) 解释该协议。

(b) 该协议易受哪种类型的攻击?

15.9 考虑以下基于非对称加密的单向认证技术:

$$\begin{aligned} A &\rightarrow B: \quad ID_A \\ B &\rightarrow A: \quad E(PU_a, R_2) \\ A &\rightarrow B: \quad R_2 \end{aligned}$$

(a) 解释该协议。

(b) 该协议易受哪种类型的攻击?

15.10 在 Kerberos 中,当 Bob 收到一个来自于 Alice 的票据时,如何得知其是否真实?

15.11 在 Kerberos 中,当 Bob 收到一个来自于 Alice 的票据时,如何得知其确实来自于 Alice?

15.12 在 Kerberos 中,若 Alice 收到一个回复,她如何得知该消息来自于 Bob(且是 Bob 最新的回复)?

15.13 在 Kerberos 中,票据包含哪些信息允许 Alice 和 Bob 安全地通信?

附录 15A Kerberos 加密技术

Kerberos 包含一个加密库,它支持各种与加密相关的操作。在 Kerberos 版本 5 的说明书里有介绍,通常应用于商业中。2005 年 2 月,IETF 发布 RFC 3961 和 RFC 3962,扩展了该加密技术。在这个附录中,我们描述之前的技术。

密码到密钥的转换

在 Kerberos 中,密码被限制使用字符,这些字符能用 7 位的 ASCII 码形式表示。这个密码有固定的长度,转换后的密钥存储在 Kerberos 的数据库中,如图 15.6 所示。

首先,字符串存储在位串 b 中,使得第一个字符能存储在第一个 7 位位中,第二个字符存在第二个 7 位中,以此类推,表示如下:

$$\begin{aligned} b[0] &= \text{bit } 0 \text{ of } s[0] \\ &\cdots \\ b[6] &= \text{bit } 6 \text{ of } s[0] \\ b[7] &= \text{bit } 0 \text{ of } s[1] \\ &\cdots \\ b[7i + m] &= \text{bit } m \text{ of } s[i] \qquad 0 \leqslant m \leqslant 6 \end{aligned}$$

然后,将位串压缩到 56 位,通过以扇状折叠的方式排列位位后,再执行位范围内的异或来完成压缩。例如,若位串的长度是 59,则

$$
\begin{aligned}
b[55] &= b[55] \oplus b[56] \\
b[54] &= b[54] \oplus b[57] \\
b[53] &= b[53] \oplus b[58]
\end{aligned}
$$

将会产生一个 56 位的 DES 密钥。为了符合 64 位密钥格式，该串被视为 8 个 7 位块的序列，为了形成输入密钥 K_{pw} 而被映射为 8 个 8 位块。

最后，原始密码使用 DES 的密文分组链接（CBC）模型的密钥 K_{pw} 加密，最后的 64 位块从该进程返回，被称为 CBC 校验和，是和该密码关联的输出密钥。

整个算法可以被视为一个 Hash 函数，将一个任意的密码映射为一个 64 位的 Hash 编码。

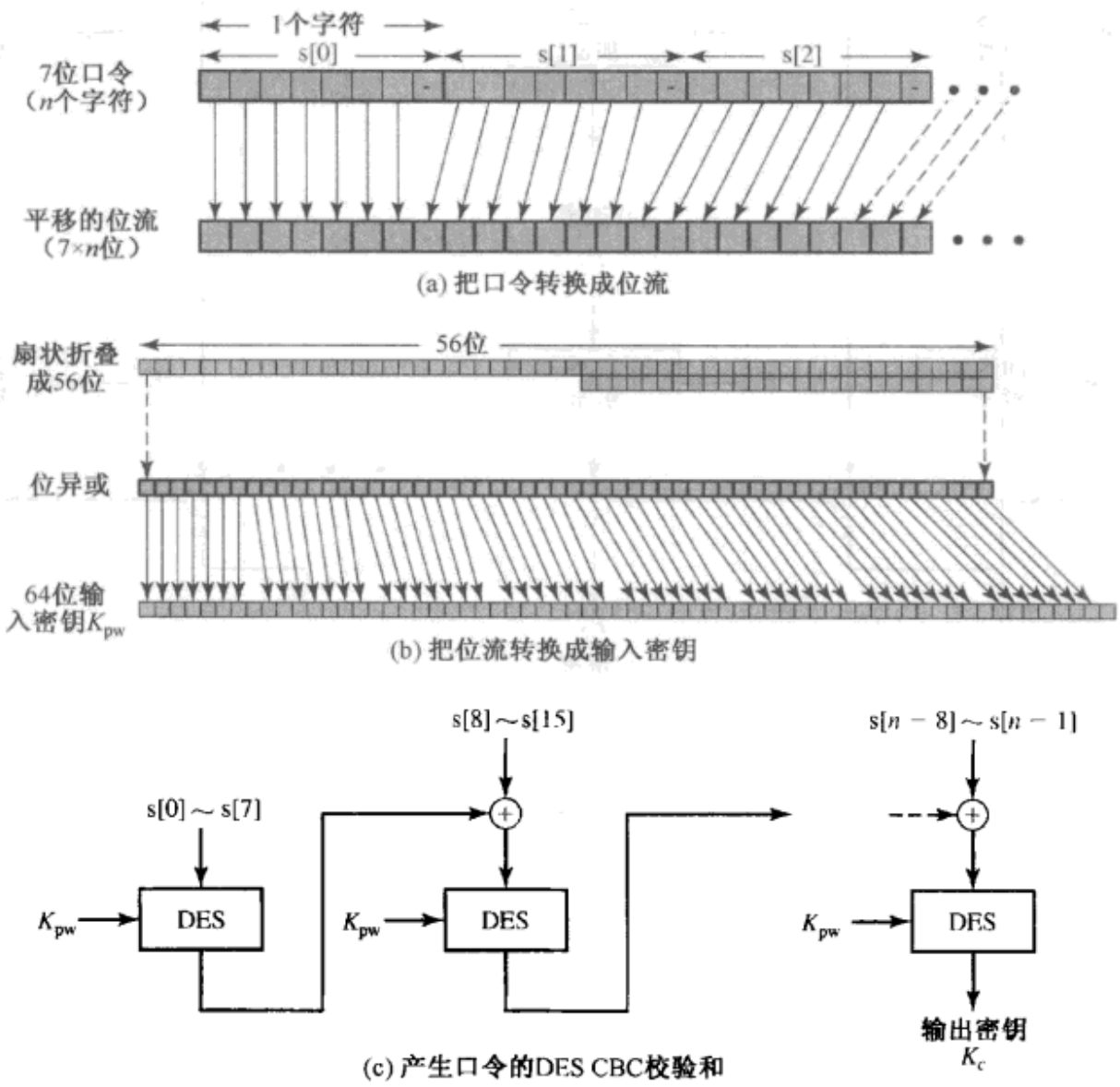


图 15.6　由口令产生加密密钥

传播的 CBC 模型

在第 6 章 DES 的 CBC 模型中，每个阶段输入的 DES 算法中包含当前明文块与之前密文块的异或，每块都使用相同的密钥（参见图 6.4）。与电码本相比，该模型的优点是：明文块之间独立地加密，也就是说，若用 CBC 重复加密，则相同的明文块会得到不同的密文块。

CBC 有这样的性质，即若密文块 C_i 在传输中产生一个错误，则该错误会传递给恢复出的明文块 P_I 和 P_{I+1}。

Kerberos 版本 4 中使用了 CBC 的一个扩展，称为传播的密钥分组链接（PCBC）模型［MEYE82］，该模型有如下性质：密文块的一个错误会传播给随后所有解密得到的明文块，致使所有块失效。因此，数据加密和整合被融合为一个操作（有一个例外，参见习题 15.6）。

PCBC 在图 15.7 中有说明,这种方法中,加密算法的输入是当前的明文块、先前的密文块以及先前的明文块的异或:

$$C_n = E(K, [C_{n-1} \oplus P_{n-1} \oplus P_n])$$

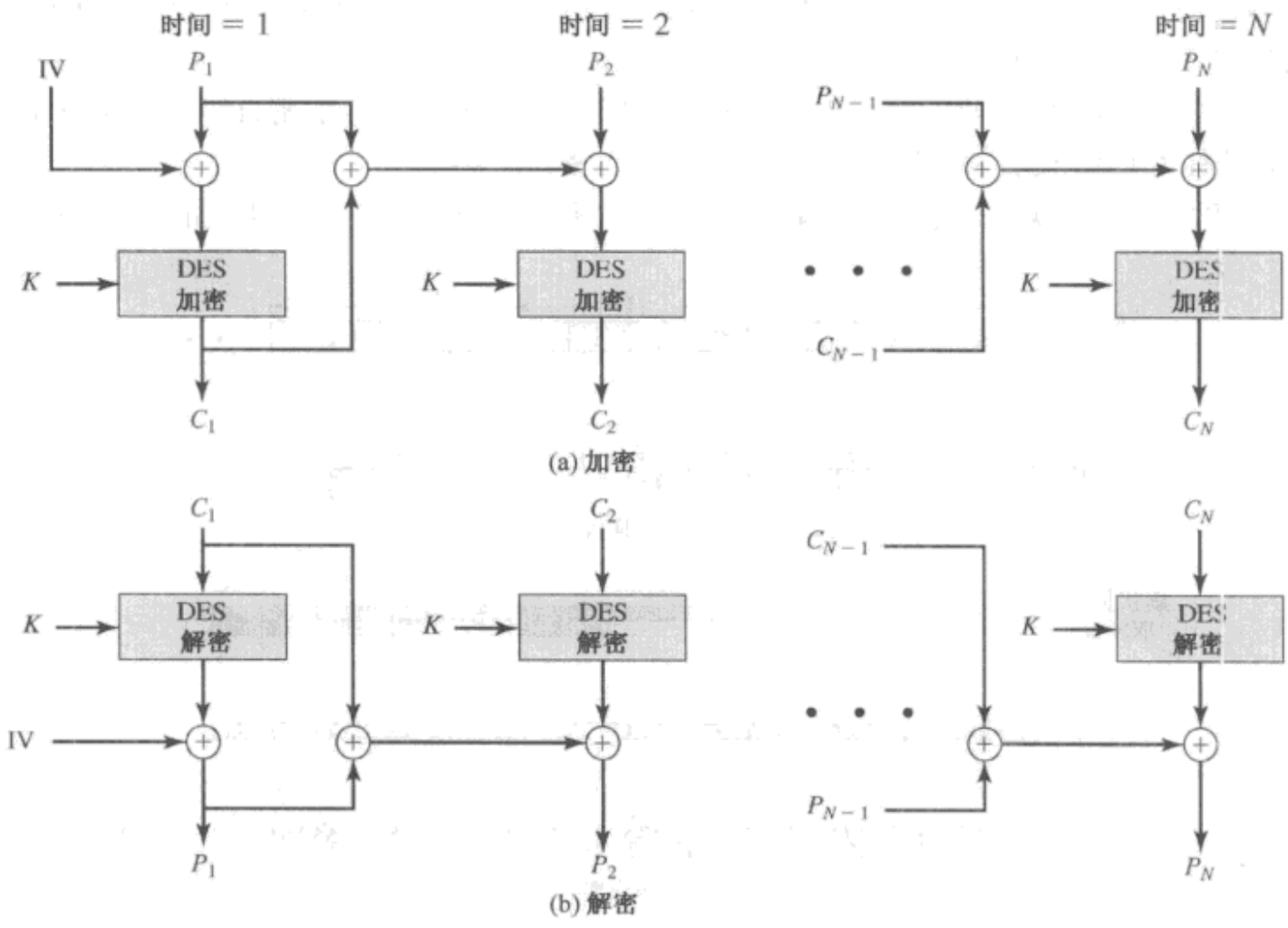


图 15.7 传播的密码分组链接(PCBC)模型

解密时,每个密文块依次通过解密算法,然后输出为与之前的密文块以及之前的明文块的异或。我们可以将该方法的工作过程描述如下:

$$D(K, C_n) = D(K, E(K, [C_{n-1} \oplus P_{n-1} \oplus P_n]))$$
$$D(K, C_n) = C_{n-1} \oplus P_{n-1} \oplus P_n$$
$$C_{n-1} \oplus P_{n-1} \oplus D(K, C_n) = P_n$$

第五部分　网络与因特网安全

第 16 章　传输层安全
第 17 章　无线网络安全
第 18 章　电子邮件安全
第 19 章　IP 安全性

第 16 章　传输层安全

16.1　Web 安全性思考
- 16.1.1　Web 安全威胁
- 16.1.2　Web 流量安全方法

16.2　安全套接层和传输层安全
- 16.2.1　SSL 体系结构
- 16.2.2　SSL 记录协议
- 16.2.3　SSL 修改密码规范协议
- 16.2.4　SSL 警报协议
- 16.2.5　SSL 握手协议
- 16.2.6　密码计算

16.3　传输层安全
- 16.3.1　版本号
- 16.3.2　消息认证码
- 16.3.3　伪随机函数
- 16.3.4　警报码
- 16.3.5　密码套件
- 16.3.6　客户端证书类型
- 16.3.7　Certificate_verify 和 Finished 消息
- 16.3.8　密码计算
- 16.3.9　填充

16.4　HTTPS
- 16.4.1　连接初始化
- 16.4.2　连接关闭

16.5　SSH
- 16.5.1　传输层协议
- 16.5.2　用户认证协议
- 16.5.3　连接协议

16.6　推荐读物和网站

16.7　关键术语、思考题和习题

Use your mentality
Wake up to reality

——*From the song*, "I've Got You Under My Skin" by Cole Porter

要　点

- 安全套接层(SSL)在 TCP 和使用 TCP 的应用程序之间提供安全服务。互联网标准版本称为传输层安全协议(TLS)。
- SSL/TLS 使用对称加密提供保密性,使用消息认证码提供消息的完整性。
- SSL/TLS 机制使得两个 TCP 用户可以决定他们将使用的安全服务。
- HTTPS(SSL 基础上的 HTTP)是指结合 HTTP 和 SSL 实现 Web 浏览器和 Web 服务器之间的安全通信。
- SSH 提供了安全的远程登录和其他安全的客户端/服务器工具。

实际上,几乎所有的商业、大多数政府机构和许多个人都有 Web 网站。访问互联网的个人和公司数量的增长速度非常快,且他们均使用图形界面的 Web 浏览器。因此,许多公司都热衷于在 Web 上进行电子商务。但是,现实情况是互联网和 Web 容易受到攻击,大家越来越意识到这种现实,因此安全 Web 服务应运而生。

Web 安全性的话题非常广泛,本章首先讨论 Web 安全性的普遍需求,然后集中讨论三种在 Web 商务中越来越重要并且主要关注于传输层安全的标准模式:SSL/TLS,HTTPS 和 SSH。

16.1　Web 安全性思考

WWW 本质上是一种运行于互联网和 TCP/IP 上的客户/服务器应用。同样地，到目前为止，本书讨论的安全工具和方法也适用于 Web 安全性。但与其中指出的一样[GARF02]，Web 带来了与一般计算机和网络安全不太相同的挑战。

- 互联网是双向的。与传统的发布环境不同，电子发布系统将涉及电子文本、电视广播、语音应答或传真反馈等，使得 Web 服务器容易受到来自于互联网的攻击。
- Web 越来越多作为商业合作和产品信息的出口以及商务交易的平台，如果 Web 服务器被破坏，就可能发生信誉受损和金钱失窃等问题。
- 虽然 Web 浏览器非常易于使用，Web 服务器相对而言易于配置和管理，Web 内容也易于开发，但其底层的软件却非常复杂。复杂的软件可能隐藏潜在的安全漏洞。在 Web 使用的短短历史中，各种新的和升级的系统容易受到各种各样的安全性攻击。
- Web 服务器可以作为公司或机构整个计算机系统的核心。一旦 Web 服务器被攻陷，攻击者不仅可以访问 Web 服务，也可获得与之相连的整个本地站点服务器的数据和系统访问权限。
- 通常使用 Web 服务的用户是一些突发的、未受训练的用户，这些用户不需要知道隐藏在服务背后的安全隐患，因此也没有有效防范的工具和知识。

16.1.1　Web 安全威胁

表 16.1 总结了在使用 Web 时将要面临的一些安全威胁的类别。一种归类的方式是将它们区分为被动和主动攻击：被动攻击包括在浏览器和服务器通信时窃听，获得原本被限制使用的权限；主动攻击包括伪装成其他用户、篡改客户和服务器之间的消息或篡改 Web 站点的信息。

另一种分类方法是按威胁的位置分类：Web 服务器、Web 浏览器和服务器与浏览器之间的网络通信。服务器与浏览器的安全问题是计算机系统自身的安全性问题，本书第四部分论述的系统安全性问题也适用于 Web 系统的安全性，通信的安全性则是本章将要论述的重点。

表 16.1　Web 安全性威胁对照表

	威　胁	后　果	对　策
完整性	● 修改用户数据 ● 特洛伊木马浏览器 ● 内存修改 ● 修改传送中的消息	● 消息丢失 ● 机器损害 ● 易受所有其他威胁	密码校验和
保密性	● 网上窃听 ● 窃取服务器数据 ● 窃取客户端数据 ● 窃取网络配置信息 ● 窃取客户端与服务器通话信息	● 信息失窃 ● 秘密失窃	加密、Web 代理
拒绝服务	● 破坏用户线程 ● 用假消息使机器溢出 ● 填满硬盘或内存 ● 使用 DNS 攻击来孤立机器	● 中断 ● 干扰 ● 阻止正常工作	难于防止
认证	● 伪装成合法用户 ● 伪造数据	● 用户错误 ● 相信虚假信息	加密技术

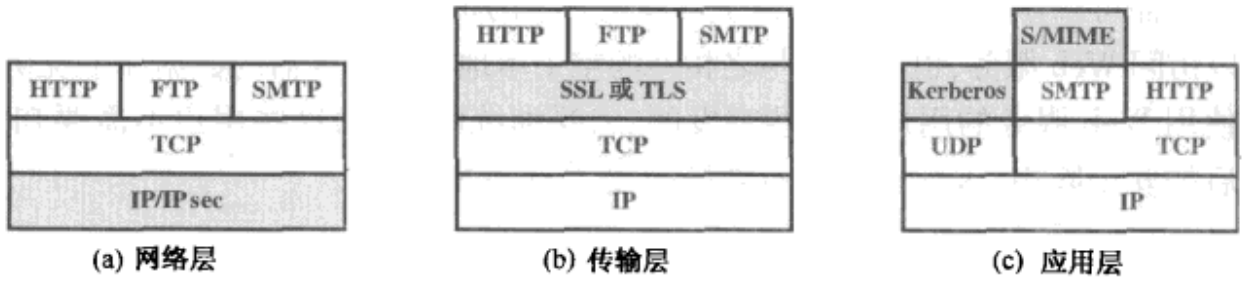
16.1.2 Web 流量安全方法

现在已有许多提供 Web 安全性的方法。这些方法的使用机理是相似的,只是各自的应用范围及在 TCP/IP 协议栈中的相对位置不同。

图 16.1 说明了这种区别。提供 Web 安全性的一种方法是使用 IP 安全性(IPsec)[如图 16.1(a)所示]。使用 IPsec 的优点在于,它对终端用户和应用均是透明的,并且提供通用的解决方案。另外,IPsec 还具有过滤功能,以便仅用 IPsec 处理所选的流量。

另一种解决方案就是在 TCP 之上实现安全性[如图 16.1(b)所示]。这种方法最先的例子是安全套接层(SSL,Secure Sockets Layer)和被称为第二代互联网标准的传输层安全协议(TLS,Transport Layer Security)。对应的有两种实现方法,一般来说,SSL(或 TLS)可以作为潜在的协议对应用透明,也可以在特定包中使用,如 Netscape 和 IE 浏览器均提供 SSL,大多数 Web 服务器都实现了此协议。

特定安全服务在特定应用中得以体现。图 16.1(c)是一个示意图。这种方法的好处在于它是为给定应用定制的。



(a) 网络层　(b) 传输层　(c) 应用层

图 16.1　在 TCP/IP 协议栈中安全功能的相对位置

16.2　安全套接层和传输层安全

SSL(安全套接层协议)源于 Netscape。协议第 3 版在通过公开评论和工业界使用后成为互联网草案,接着在达成共识之后由 IETF 的 TLS(传输层的安全协议)工作组将其开发为一般标准。目前 TLS 工作的目标是开发互联网标准,TLS 开发的第一个版本 SSLv3.1 将非常接近 SSLv.3,且对 SSLv.3 兼容。

本节主要讨论 SSLv3。在下一节,将介绍 SSLv3 和 TLS 的主要区别。

16.2.1　SSL 体系结构

SSL 为 TCP 提供可靠的端到端安全服务。SSL 不是单个协议而是两层协议,如图 16.2 所示。

SSL 记录协议(SSL Record Protocol)为高层协议提供基本的安全服务。特别是,为 Web 客户端/服务器交互提供传送服务的 HTTP 协议可以在上层访问 SSL。SSL 协议上定义了三个高层协议:握手协议、修改密码规范协议和警报协议。这些 SSL 上层协议用于对 SSL 交换进行管理。

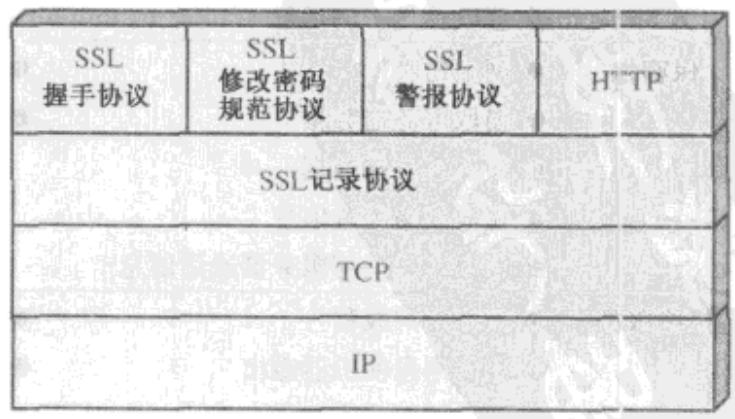


图 16.2　SSL 协议栈

SSL 中包含两个重要概念:SSL 会话和 SSL 连接。在规范中定义如下:

- **连接**:连接是提供合适服务类型的一种传输(OSI 层次模型定义)。对 SSL 来说,连接表示的是对等网络关系,且连接是短暂的,每个连接与一个会话相关。

- **会话**:SSL 会话是一个客户端和服务器间的关联,会话是通过握手协议创建的,定义了一组多个连接共享的密码安全参数。会话可用于减少为每次连接建立安全参数的昂贵协商费用。

在多方会谈中(如客户端和服务器间 HTTP 应用),需要多个安全连接。从理论上说,可以在多方之间同时发生会话,但还没有在实际中使用。

每个会话实际上与多种状态相关,一旦会话建立,则进入针对读和写(如接收和发送)的当前操作状态。另外,在握手协议中,会创建读挂起状态和写挂起状态。在握手协议成功完成后,挂起状态成为当前状态。

一个会话状态由以下参数定义(参见 SSL 规范):

- **会话标志**:服务器用于标志活动或恢复的会话状态所选的一个随机字节序列。
- **对等实体证书**:对等实体的 X509.v3 证书,此状态元素可以为空。
- **压缩方法**:在加密前使用的压缩数据的算法。
- **密码规范**:描述主要数据加密算法(如 null、AES 等)和计算 MAC 的散列算法(如 MD5 或 SHA-1),同时也定义如散列值的大小等加密属性。
- **主密钥**:客户和服务器间 48 字节的共享密码。
- **可恢复性**:表明会话是否可被用于初始化新连接的标志。

连接状态可用以下参数定义:

- **服务器和客户端随机数**:服务器和客户端为每个连接选择字节序列。
- **服务器写 MAC 密码**:服务器发送数据时在 MAC 操作中使用的密码。
- **客户端写 MAC 密码**:客户端发送数据时在 MAC 操作中使用的密码。
- **服务器写密钥**:服务器加密和客户端解密数据时使用的传统加密密钥。
- **客户端写密钥**:客户端加密和服务器解密数据时使用的传统加密密钥。
- **初始化向量**:使用 CBC 时,需要为每个密钥维护一个初始化向量(IV)。该域首先被 SSL 握手协议初始化,其后,每个记录的最后一个密码块被保存,以作为后续记录的 IV。
- **序列号**:会话的各方为每个连接传送和接收消息维护一个单独的序列号。当接收或发送一个修改密码规范协议报文时,序列号被设为 0。序列号不能超过 $2^{64}-1$。

16.2.2　SSL 记录协议

SSL 记录协议为 SSL 连接提供两种服务:

- **保密性**:握手协议定义了加密 SSL 载荷的传统加密共享密钥。
- **消息完整性**:握手协议也定义了生成消息认证代码(MAC)的共享密钥。

图 16.3 例示了 SSL 记录协议的整个操作过程。记录协议接收一个要传送的应用消息,将其段分为块、压缩(可选)、加上 MAC、加密,再加上一个 SSL 头,将得到的最终数据单元放入一个 TCP 段中。接收的数据被解密、验证、解压、重组后,再传递给高层用户。

第一步是分段,每个上层消息被分成若干小于或等于 2^{14} 字节(16 384 字节)的段;接着进行可选择压缩,压缩必须采用无损压缩方法,并且增加长度不能超过 1024 个字节①。在 SSLv3(和当前的 TLS)中,没有指定压缩算法,所以默认的压缩算法为空。

---

① 当然,都希望压缩的结果是数据量变少而不是变多。但是对非常小的数据块有可能由于格式的转换使压缩算法的输出比输入更长。

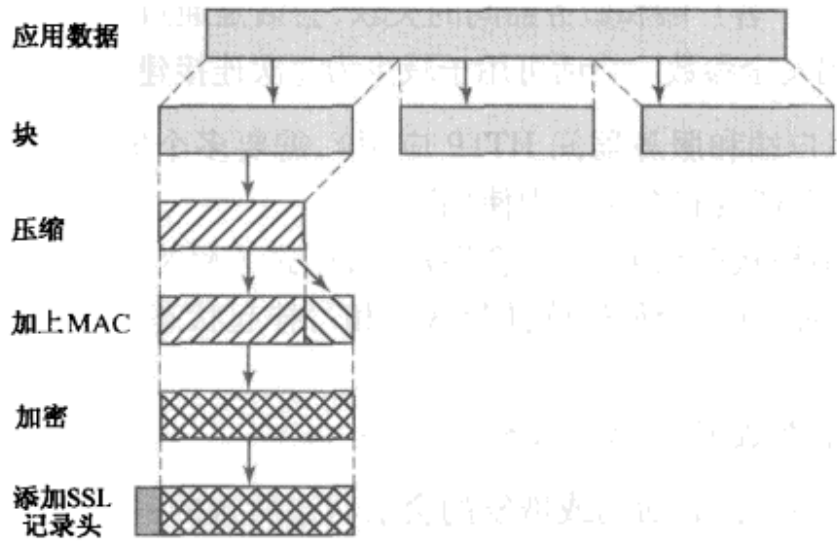


图 16.3 SSL 记录协议操作

接着对压缩数据计算其消息认证码 MAC。为此,需要使用共享密钥,其计算方式如下:

```
hash(MAC_write_secret ‖ pad_2‖
     hash(MAC_write_secret ‖ pad_1‖seq_num ‖
     SSLCompressed.type ‖ SSLCompressed.length ‖
     SSLCompressed.fragment))
```

其中

‖	=连接
MAC_write_secret	=共享密钥
Hash	=Hash 算法,MD5 或 SHA-1
pad_1	=字节 0x36(0011 0110),对 MD5 重复 48 次(384 位);对 SHA-1,重复 40 次(320 位)
pad_2	=字节 0x5C(0101 1100),对 MD5 重复 48 次;对 SHA-1,重复 40 次
seq_num	=此消息的序列号
SSLCompressed.type	=处理此分段的上层协议
SSLCompressed.length	=压缩后的分段长度
SSLCompressed.fragment	=压缩后的分段(如果没有压缩,则为明文段)

注意,这与第 12 章定义的 HMAC 算法非常相似,其区别在于在 SSLv3 中两个填充域是连接关系,而在 HMAC 中是异或关系。SSLv3 的 MAC 算法是基于 HMAC 的原始互联网草案的,且使用连接关系。而在 HMAC 的最后版本 RFC 2104 中,使用异或关系。

接下来,将压缩消息和 MAC 用对称加密方法加密。加密对内容增加长度不能超过 1024 个字节,以便整个长度不能超过 $2^{14}+2048$。以下加密算法是允许的。

块密码		流密码	
算法	密钥大小	算法	密钥大小
AES	128 256	RC4-40	40
IDEA	128	RC4-128	128
RC2-40	40		
DES-40	40		
DES	56		
3DES	168		
Fortezza	80		

Fortezza 可被用于智能卡加密模式。

对流加密而言，压缩消息和 MAC 一起被加密。注意，MAC 在加密之前计算，然后将 MAC 和明文或压缩后的明文一起加密。

对分组加密而言，填充应在 MAC 之后、加密之前进行。填充的格式是一定长度的填充字节后跟一个字节的填充长度。整个填充域的长度是使得总长度（明文 + MAC + 填充域的长度）为规定的加密分组长度整数倍的最小长度。例如，明文（或如果使用了压缩则为压缩文本）长度为 58 个字节，MAC 长度为 20 个字节（使用 SHA-1），使用分组长度为 8 个字节的加密算法（如 DES），则加上填充长度域的 1 个字节总共 79 个字节，为了达到 8 的整数倍，则需要增加一个字节的填充。

SSL 记录协议的最后一步是加上一个由如下域组成的 SSL 头：

- **内容类型（8 位）**：封装段使用的高层协议。
- **主版本号（8 位）**：表明 SSL 使用的主版本号，如 SSLv3 的值为 3。
- **从版本号（8 位）**：表明 SSL 使用的从版本号，如 SSLv3 的值为 0。
- **压缩长度（16 位）**：明文段（如果使用了压缩，则为压缩段）的字节长度，最大值为 $2^{14}+2048$。

已经定义的内容类型包括修改密码规范、警报、握手和应用数据。接下来讨论前三个类型。注意，在各种应用中使用 SSL 并没有什么限制，它们提供的数据内容对 SSL 来说是不透明的。

图 16.4 示例了 SSL 记录格式。

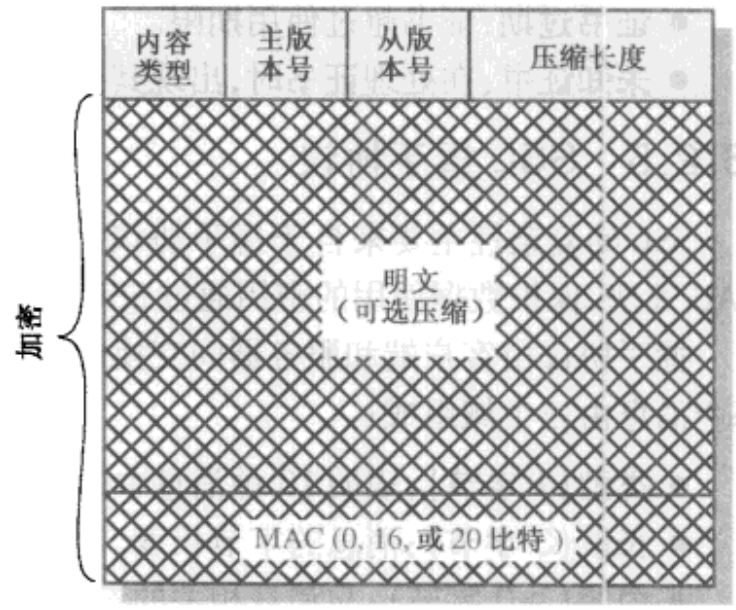


图 16.4　SSL 记录格式

16.2.3　SSL 修改密码规范协议

修改密码规范协议是 SSL 三个特定协议之一，也是最简单的一个。协议由一个仅包含一个字节的值为 1 的消息组成［如图 16.5（a）所示］，此消息使得挂起状态被复制到当前状态中，用于更新此连接使用的密码套件。

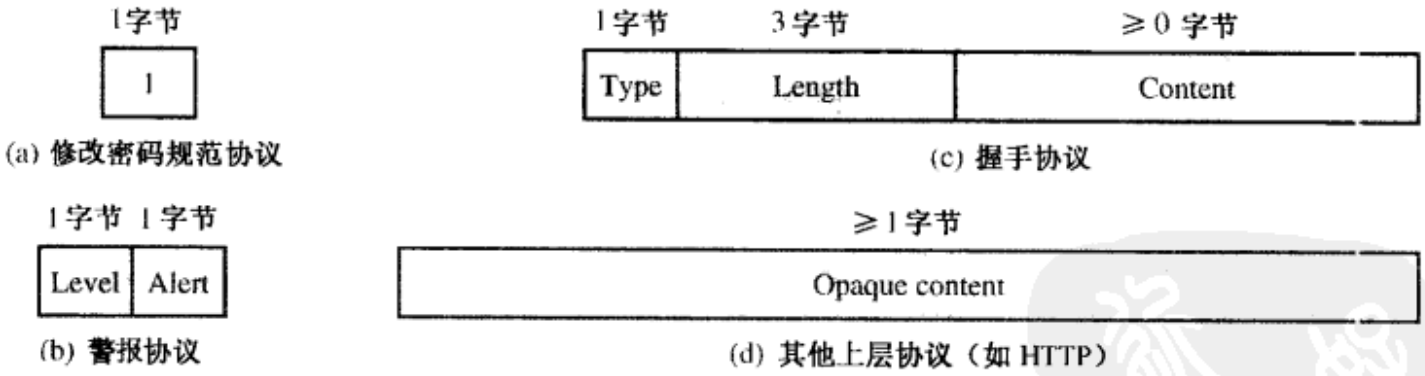


图 16.5　SSL 记录协议有效载荷

16.2.4　SSL 警报协议

报警协议用于向对等实体传递 SSL 相关的警报。和使用 SSL 的其他应用一样，警报消息按照当前状态压缩和加密。

此协议的每个消息由两个字节组成［如图 16.5（b）所示］。第一个字节，值 1 表示警报，值 2 表示致命错误，传递消息出错的严重程度。如果级别为致命，则 SSL 将立即终止连接，而会话中的其他连接将继续进行，但不会在此会话中建立新连接。第二个字节包含描述特定警报信息的代码。首先，我们将通常导致致命错误的警报列举如下（参见 SSL 规范定义）：

- **意外消息**:接收到不正确的消息。
- **MAC 记录出错**:接收到不正确的 MAC。
- **解压失败**:解压函数接收到不正确的输入(如不能解压或解压长度大于允许值的数据长度)。
- **握手失败**:发送者无法在给定选项中协商出一个可以接受的安全参数集。
- **非法参数**:握手消息中的某个域超出范围或与其他域不一致。

其余的警报如下:

- **结束通知**:通知接收者,发送者将不再用此连接发送任何消息。各方在关闭连接的写端时均需发送结束通知。
- **无证书**:如果无适当的证书可用,用可能作为证书请求的响应来发送。
- **证书出错**:接受的证书被破坏(如签名无法通过验证)。
- **不支持的证书**:不支持接收的证书类型。
- **证书撤销**:证书被其签名者撤销。
- **证书过期**:证书超过使用期限。
- **未知证书**:在处理证书时,出现其他错误,使得证书不被接受。

16.2.5 SSL 握手协议

SSL 的复杂性主要来自于握手协议。此协议允许客户端和服务器端相互认证、协商加密和 MAC 算法,保护数据使用的密钥通过 SSL 记录传送。握手协议在传递应用数据之前使用。

握手协议由客户端和服务器间交换的一系列消息组成,这些消息的格式如图 16.5(c)所示。每个消息由三个域组成:

- **类型(1 字节)**:表明 10 种消息中的一种,表 16.2 列举了所定义的消息类型。
- **长度(3 字节)**:消息的字节长度。
- **内容(≥0 字节)**:与消息相关的参数,如表 16.2 所示。

表 16.2 握手协议消息类型

消息类型	参 数
hello_request	空
client_hello	版本号、随机数、会话标志、密码套件、压缩方法
server_hello	版本号、随机数、会话标志、密码套件、压缩方法
certificate	X.509v3 证书链
server_key_exchange	参数、签名
certificate_ request	类型、认证机构
server_done	空
certificate_verify	签名
client_key_exchange	参数、签名
finished	Hash 值

图 16.6 表明了在客户端与服务器之间建立逻辑连接的初始交换。此交换由四个阶段组成。

阶段 1. 建立安全功能

此阶段用于建立初始的逻辑连接,并建立与之相关联的安全功能,客户端发起这个交换,发送具有如下参数 `client_hello message` 消息:

- **版本**:客户端所支持的最高 SSL 版本。
- **随机数**:由客户端生成的随机数结构,用 32 位时间戳和一个安全随机数生成器生成的 28 字节随机数组成。这些值作为随机数,在密钥交换时防止重放攻击。

- **会话标志**:一个变长的会话标志。非 0 值意味着客户端想更新现存连接的参数或为此会话创建一个新的连接;0 值意味着客户端想在新会话上创建一个新连接。
- **密码套件**:按优先级的降序排列的、客户端支持的密码算法列表。表的每个元素定义了一个密钥交换算法和一个密码说明。
- **压缩方法**:一个客户端支持的压缩方法列表。

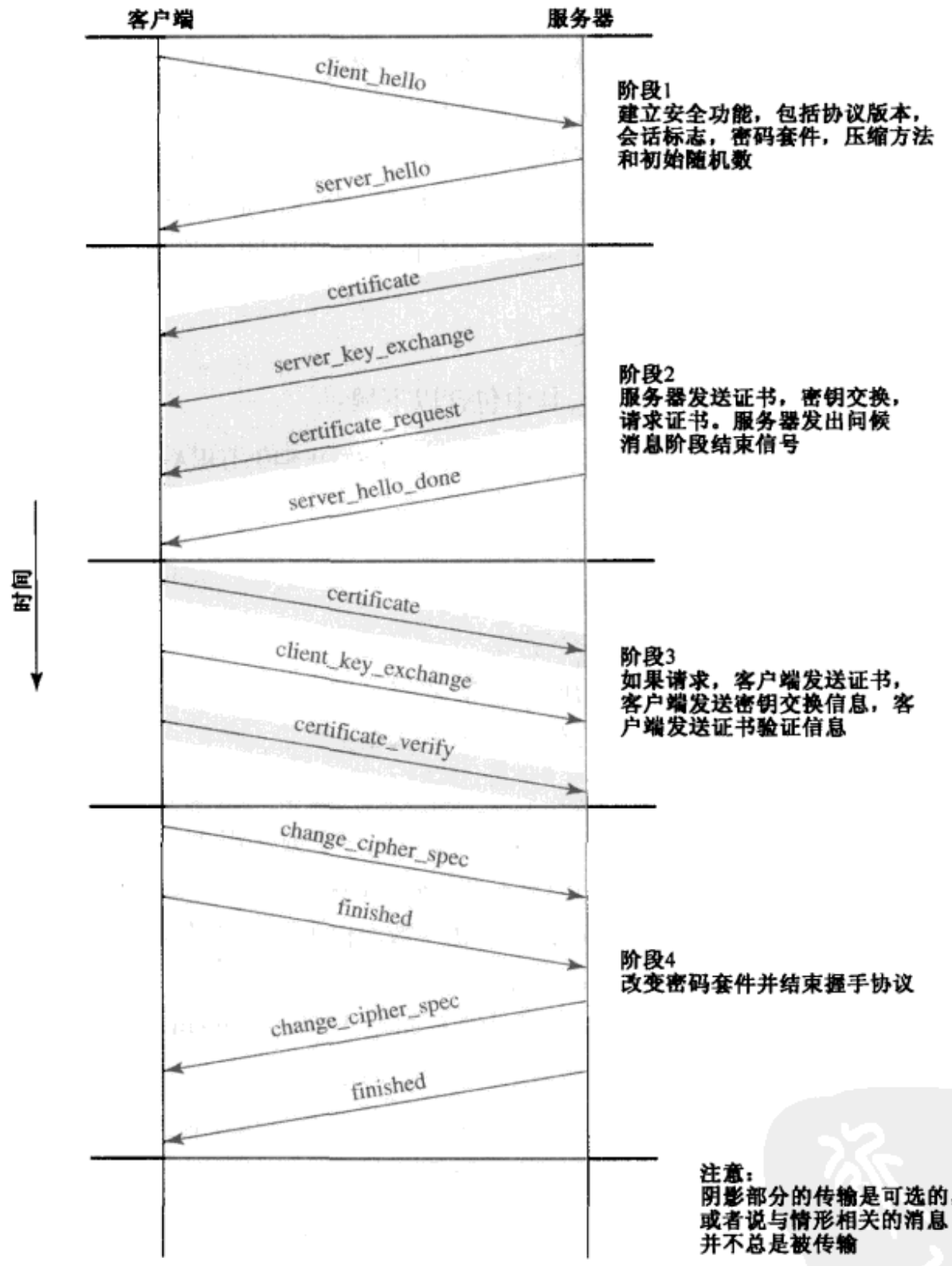


图 16.6　握手协议的处理过程

客户端发出消息 client_hello 后,会等待包含与消息 client_hello 参数相同的 server_hello message 消息的到来。对 server_hello 消息而言,应用了如下惯例:版本域中包含的是客户端支持的最低版本号和服务器支持的最高版本号。随机数域是由服务器生成的,与客户端的随机数域相互独立。如果客户端会话标志非 0,则服务器使用与之相同的值,否则,服务器的会话标志域包含新对话的值。密码套件域包含着服务器从客户端所给的密码套件中选出的密码套件。压缩域包含的是服务器从客户端所给的压缩方法中选出的压缩方法。

密码套件参数的第一个元素是密钥交换方法(如传统加密密钥和 MAC 交换的方法)。支持下述密钥交换方法:

- **RSA**:用接收者的 RSA 公钥加密的密钥,必须拥有接收者公钥的公钥证书。
- **固定 Diffie-Hellman**:Diffie-Hellman 密钥交换,其中包含签证机构签发的 Diffie-Hellman 公钥参数的服务器证书,也就是说,公钥证书包含 Diffie-Hellman 公钥参数。客户端在证书中提供它的 Diffie-Hellman 公钥参数,或需要进行客户端认证时,在密钥交换消息中提供证书。
- **瞬时 Diffie-Hellman**:此技术用于创建瞬时(临时、一次性)的密钥。在这种情况下,Diffie-Hellman 公钥在交换时使用发送者的 RSA 或 DSS 私钥签名。接收者使用相应的公钥验证签名。由于它使用的是临时的认证密钥,因此在三种 Diffie-Hellman 选项中最安全。
- **匿名 Diffie-Hellman**:使用基本的 Diffie-Hellman 算法,没有认证。即,在向对方发送其 Diffie-Hellman 公钥参数时,不进行认证。这种方法容易受到中间人攻击,攻击者可以使用匿名 Diffie-Hellman 与双方进行通话。
- **Fortezza**:为 Fortezza 方案定义的技术。

密钥交换方法定义之后的是 CipherSpec,其中包含以下域:

- **密码算法**:任何前面提及的算法:RC4、RC2、DES、3DES、DES40、IDEA、Fortezza
- **MAC 算法**:MD5 或 SHA-1
- **密码类型**:流或块
- **可否出口**:真或假
- **散列长度**:0,16(MD5)或 20(SHA-1)个字节
- **密钥材料**:字节序列,包含生成写密钥所使用的数据
- **IV 大小**:密码分组链 CBC 加密使用的初始矢量的大小

阶段 2. 服务器认证和密钥交换

如果需要进行认证,则服务器开始于发送其证书:消息包含一个或一组 X.509 证书。除匿名 Diffie-Hellman 方法外,其他密钥交换方法均需要证书消息 certificate message。注意,如果使用固定 Diffie-Hellman,此证书消息将由于包含了服务器 Diffie-Hellman 公钥参数而作为服务器的密钥交换消息。

接着,如果需要,可以发送服务器密钥交换消息(`server_key_exchange message`)。在以下两种情况下,不需要此消息:(1)服务器发送了带有固定 Diffie-Hellman 参数的证书;(2)使用 RSA 密钥交换。以下情况需要 `server_key_exchange` 消息:

- **匿名 Diffie-Hellman**:消息内容包含两个全局 Diffie-Hellman 值[一个素数和它的原根(primitive root)]和服务器 Diffie-Hellman 公钥(如图 10.1 所示)。
- **瞬时 Diffie-Hellman**:消息内容包含三个 Diffie-Hellman 参数,包括匿名 Diffie-Hellman 中的两个参数和它们的参数签名。
- **RSA 密钥交换,服务器在使用 RSA 时仅用了 RSA 签名密钥**:因此,客户端不能简单地通过服务器公钥加密其密钥后传送,而服务器必须创建一个临时 RSA 公钥/私钥对,并使用服务器密钥交换消息发送公钥。消息内容包含两个临时的 RSA 公钥参数(指数和模,如图 9.5 所示)和参数签名。
- **Fortezza**

关于签名的其他细节均能得到保证。通常情况下,通过对消息使用散列函数并使用发送者私钥加密获得签名。在此,散列函数定义如下:

```
hash(ClientHello.random ‖ ServerHello.random ‖
ServerParams)
```

散列不仅包含 Diffie-Hellman 或 RSA 参数，还包含初始 hello 消息中的两个随机数，可以防止重放攻击和伪装。对 DSS 签名而言，散列函数使用 SHA-1 算法；对 RSA 签名而言，将要计算 MD5 和 SHA-1，再将两个散列结果串接(36 字节)后，用服务器私钥加密。

接下来，一个非匿名服务器(服务器不使用匿名 Diffie-Hellman)需要向客户端申请证书。证书请求消息 certificate_request 包含两个参数：证书类型和签证机构。证书类型表明了公钥算法和它的用途：

- RSA：仅用于签名
- DSS：仅用于签名
- 固定 Diffie-Hellman 的 RSA：此时，发送 RSA 签名证书，其签名仅用于认证
- 固定 Diffie-Hellman 的 DSS：仅用于认证
- 瞬时 Diffie-Hellman 的 RSA
- 瞬时 Diffie-Hellman 的 DSS
- Fortezza

证书请求消息中的第二个参数是一个可接受的签证机构名字表。

阶段 2 中的最后一个消息——服务器完成消息 server_done 通常是必要的。此消息由服务器发送，表明服务器的 hello 和相关消息结束。在此消息发送之后，服务器将等待客户端应答，此消息不带参数。

阶段 3. 客户端认证和密钥交换

在接收到服务器完成消息之后，如果请求了证书，客户端需要验证服务器是否提供了合法证书，并且检查 server_hello 参数是否可接受。如果所有的条件均满足，则客户端向服务器发回一个或多个消息。

如果服务器请求了证书，则在此阶段客户端开始发送一条证书消息 **certificate message**。如果未提供合适的证书，则客户端将发送一个“无证书警报”。

接下来是此阶段必须要发的客户端密钥交换消息 client_key_exchange，消息的内容依赖于密钥交换的类型：

- **RSA**：客户端生成 48 字节的次密钥，并使用服务器证书中的公钥或服务器密钥交换消息中的临时 RSA 密钥加密。它被用于生成稍后介绍的主密钥计算。
- **瞬时或匿名 Diffie-Hellman**：发送的客户端 Diffie-Hellman 公钥参数。
- **固定 Diffie-Hellman**：由于证书消息中包括 Diffie-Hellman 公钥参数，因此，此消息内容为空。
- **Fortezza**：发送客户端的 Fortezza 参数。

在此阶段的最后，客户端可以发送一个证书验证消息 certificate_verify 来提供对客户端证书的精确认证。此消息只有在客户端证书具有签名能力时发送(如除带有固定 Diffie-Hellman 参数外的所有证书)。此消息对一个基于前述消息的散列编码的签名，其定义如下：

```
CertificateVerify.signature.md5_hash=
     MD5(master_secret ‖ pad_2 ‖ MD5(handshake_messages ‖
         master_secret ‖ pad_1));
CertificateVerify.signature.sha_hash=
     SHA(master_secret ‖ pad_2 ‖ SHA(handshake_messages ‖
         master_secret ‖ pad_1));
```

其中,pad_1 和 pad_2 是前面 MAC 定义的值,握手消息指的是所有发送的握手协议消息或接收到的从 client_hello 消息开始不包括此消息的所有消息。主密钥的计算方法将在以后介绍。如果用户私钥是 DSS,则被用于加密 SHA-1 散列;如果用户私钥是 RSA 的密钥,则被用于加密 MD5 和 SHA-1 散列连接。不管在哪种情况下,其目的都是为了使用私钥验证客户证书的客户所有权。即使有人误用了客户证书,它也无法发送消息。

阶段 4. 完成

此阶段完成安全连接的设置。客户端发送改变密码规范消息 change_cipher_spec 并向当前 CipherSpec 中复制挂起 CipherSpec。注意,此消息不是握手协议的一部分,而是使用的修改密码规范协议发送的。于是,客户端立即使用新的算法、密钥和密码发送新的完成消息 **finished message**。完成消息对密钥交换和认证过程的正确性进行验证,完成消息的内容包含两个散列值的连接:

```
MD5(master_secret || pad2 || MD5(handshake_messages ||
    Sender || master_secret || pad1))
SHA(master_secret || pad2 || SHA(handshake_messages ||
    Sender || master_secret || pad1))
```

其中发送者 Sender 表示发送者为客户端,握手消息 handshake_messages 包括除此消息之外的所有握手消息数据。

在应答这两个消息时,服务器发送自己的修改密码规范消息 change_cipher_spec,并向当前 CipherSpec 中复制挂起 CipherSpec,发送完成消息。此时,握手完成,客户端和服务器即可开始交换应用层数据。

16.2.6 密码计算

下面介绍通过密钥交换创建共享主密钥和使用主密钥生成密码参数。

主密钥的创建

共享主密钥是利用安全密钥交换为此会话建立的一个一次性 48 字节的值(384 位)。生成此密钥共分为两个阶段:首先,交换次密钥 pre_master_secret;其次,双方共同计算主密钥 master_secret,对于次密钥有两种可能:

- **RSA**:由客户端生成 48 字节的次密钥,用服务器的 RSA 公钥加密后,发往服务器。服务器用其私钥解密密文,得到次密钥。
- **Diffie-Hellman**:客户端和服务器同时生成 Diffie-Hellman 公钥。密钥交换后,各方执行 Diffie-Hellman 计算,创建共享次密钥。

然后,双方按如下方法计算主密钥:

```
master_secret = MD5(pre_master_secret || SHA('A' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
                MD5(pre_master_secret || SHA('BB' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random)) ||
                MD5(pre_master_secret || SHA('CCC' ||
                    pre_master_secret || ClientHello.random ||
                    ServerHello.random))
```

其中,ClientHello.random 和 ServerHello.random 是在初始 hello 消息中交换的两个随机数。

生成密码参数

CipherSpecs 需要的客户端写 MAC 密钥、服务器写 MAC 密钥、客户端写密钥、服务器写密钥、客户端写初始向量和服务器写初始向量，均是通过主密钥生成的。主密钥通过散列函数把所有参数映射为足够长的安全字节序列。

从主密钥生成各主要参数的方法与从次密钥中生成主密钥的方法相同：

```
key_block = MD5(master_secret || SHA('A' || master_secret ||
                ServerHello.random || ClientHello.random)) ||
            MD5(master_secret || SHA('BB' || master_secret ||
                ServerHello.random || ClientHello.random)) ||
            MD5(master_secret || SHA('CCC' || master_secret ||
                ServerHello.random || ClientHello.random)) || . . .
```

直到生成足够的输出。此算法结构的结果是一个伪随机函数，可以将主密钥看做该函数的伪随机种子值(seed value)；客户端和服务器随机数则可被视为是复杂密码分析方法的敏感值(salt value，参见第 20 章)。

16.3　传输层安全

TLS 是 IETF 标准的初衷。他们的目标是编写 SSL 的互联网标准。当前 TLS 的草案 RFC5246 与 SSLv3 非常相似。在本节中我们将把主要精力集中在它们的区别上。

16.3.1　版本号

TLS 记录格式与 SSL 记录格式相同(如图 16.4 所示)，头中各域的含义也相同。其区别在于版本值。目前，在 TLS 版本中，其主版本号为 3，从版本号为 1。

16.3.2　消息认证码

SSLv3 和 TLS 的 MAC 模式有两点不同：实际算法和 MAC 计算的范围。TLS 使用 RFC 2104 中定义的 HMAC 算法。回想第 12 章中的定义：

$$\mathrm{HMAC}_K(M) = \mathrm{H}[(K^+ \oplus \mathrm{opad}) \| \mathrm{H}[(K^+ \oplus \mathrm{ipad}) \| M]]$$

其中

H　　=表示嵌入散列函数(对 TLS 而言，MD5 或 SHA-1)

M　　=输入给 HMAC 的消息

K^+　　=左边添 0 的密钥，使得其长度等于散列代码的块长度(对 MD5 和 SHA-1 而言，块长度为 512 位)。

ipad =00110110(十六进制的 36)重复 64 次(512 位)

opad =01011100(十六进制的 5C)重复 64 次(512 位)

SSLv3 除了填充块与密钥串联外，它们使用相同算法。两者的安全级别相同。

对 TLS 而言，MAC 计算包括以下表达式的各域：

```
MAC(MAC_write_secret, seq_num || TLSCompressed.type ||
    TLSCompressed.version || TLSCompressed.length ||
    TLSCompressed.fragment)
```

MAC 计算不仅覆盖了 SSLv3 中 MAC 计算的各域，还增加了一个体现协议版本号的域 TLSCompressed.version。

16.3.3 伪随机函数

TLS 使用称为 PRF 的伪随机函数,将密码扩展成为生成密钥的数据分组。其目的是使用相对较小的共享密码值,生成较长的数据分组,防止对散列函数和 MAC 的攻击。伪随机函数基于下述数据扩展函数(参见图 16.7):

$$\begin{aligned}\texttt{P_hash(secret, seed)} = {} & \texttt{HMAC_hash(secret, A(1) } \| \texttt{ seed)} \| \\ & \texttt{HMAC_hash(secret, A(2) } \| \texttt{ seed)} \| \\ & \texttt{HMAC_hash(secret, A(3) } \| \texttt{ seed)} \| \ \ldots\end{aligned}$$

其中,A( )定义为:

$$\texttt{A(0)} = \text{seed}$$

$$\texttt{A(i)} = \texttt{HMAC_hash(secret, A(}i-1\texttt{))}$$

数据扩展函数使用以 MD5 或 SHA-1 为基本散列函数的 HMAC 算法。P_hash 可以迭代任意次,产生所需的数据。例如,如果使用 P_SHA-1 生成 64 个字节的数据,就需要迭代 4 次,产生 80 个字节的数据,略去最后的 16 个字节。如果使用 P_MD5 则也需要迭代 4 次,恰好产生 64 个字节的数据。注意,每次迭代执行两次 HMAC,每次 HMAC 执行两次基本散列函数。

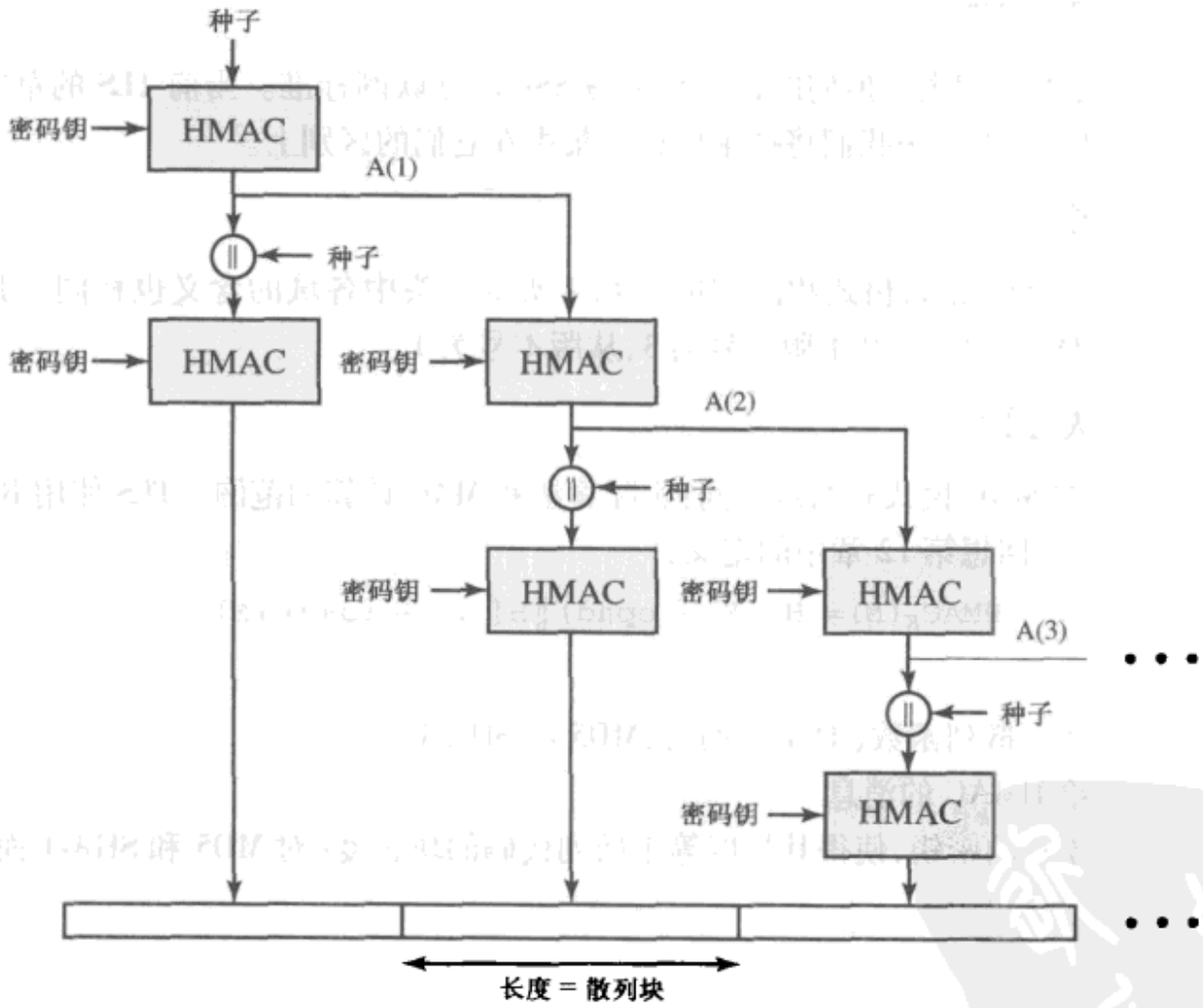


图 16.7 TLS 函数 P_hash(secret,seed)

为了使 PRF 足够安全,PRF 同时使用两种散列函数。只要有一种算法是安全的,则 PRF 是安全的。PRF 定义如下:

$$\texttt{PRF(secret, label, seed)} = \texttt{P_hash(S1, label } \| \texttt{ seed)}$$

PRF 以密码值 secret、标志 label 和种子值 seed 为输入,产生任意长度的输出。

16.3.4　警报码

TLS 支持除无证书以外的 SSLv3 中定义的所有警报代码，并且还定义了许多附加的代码，以下列举出了其中的主要部分：

- **记录溢出**：接收的 TLS 记录中的载荷（密文）长度超过 $2^{14}+2048$ 个字节或者密文解密后长度超过 $2^{14}+1024$。
- **未知的签证机构**：接收到正确的证书链或部分链，但由于证书不能定位或不能与可信任的签证机构匹配而不接收证书。
- **拒绝访问**：接收到合法证书，但发送者拒绝进行协商访问。
- **解码出错**：由于域超出了指定范围或消息长度不对使得消息不能解码。
- **输出限制**：密钥长度的输出限制不能达成一致。
- **协议版本**：客户端试图协商的协议版本可以识别但不能支持。
- **安全不足**：服务器需要的安全级别客户端无法支持时协商失败的返回值，代替握手失败。
- **中间出错**：与对方或协议正确性无关的中间环节出错，使得无法继续操作。
- **解密出错**：握手密码访问失败，包括无法验证签名、解密密钥交换或校验完成消息。

新的警报代码如下：

- **用户取消**：握手由于某些与协议错误的原因而被取消。
- **不再重新协商**：由客户端响应 hello 请求或服务器在初始握手后响应客户端的 hello，通常应答消息都会导致重新协商，但此警报表明发送者不能重新协商。此消息通常是警报消息。

16.3.5　密码套件

SSLv3 和 TLS 提供的密码套件有一些小的差别：

- **密钥交换**：TLS 支持除 Fortezza 外的所有 SSLv3 的密钥交换技术。
- **对称加密算法**：TLS 包括除 Fortezza 外的所有 SSLv3 的对称加密算法。

16.3.6　客户端证书类型

TLS 定义了以下证书请求消息中需要的证书类型：`rsa_sign`、`dss_sign`、`rsa_fixed_dh` 和 `dss_fixed_dh`，这些都是 SSLv3 中定义了的类型。另外，SSLv3 中还包括 `rsa_ephemeral_dh`、`dss_ephemeral_dh` 和 `fortezza_kea`。瞬时 Diffie-Hellman 使用 RSA 或 DSS 对 Diffie-Hellman 参数加密。对 TLS 而言，不包括 Fortezza 模式，仅使用 `rsa_sign` 和 `dss_sign` 类型对 Diffie-Hellman 参数加密。

16.3.7　Certificate_verify 和 Finished 消息

在 TLS 证书验证消息中，MD5 和 SHA-1 的散列值只根据握手消息计算。SSLv3 中散列值的计算还包括主密钥和填充域，但这些额外的域不能增加它的安全性。

与 SSLv3 中的完成消息相比，TLS 中的完成消息是基于共享主密钥、先前的握手消息和标志服务器或客户端的标签的 Has。某些计算不同。在 TLS 中，有

```
PRF(master_secret,finished_label,MD5(handshake_messages)‖
    SHA-1(handshake_messages))
```

其中,finished_label 或为客户端的字符串“client finished”,或为服务器端的字符串“server finished”。

16.3.8 密码计算

TLS 中次密钥的计算方法与 SSLv3 中的相同。与 SSLv3 一样,TLS 的主密钥是次密钥和两个随机 hello 值的散列,但计算方法如下:

```
master_secret= PRF(pre_master_secret,"master secret",
                   ClientHello.random‖ServerHello.random)
```

算法执行直到产生 48 个字节的伪随机数。密钥分组元素(MAC 密钥、会话加密密钥和初始向量)的计算方法如下:

```
key_block = PRF(master_secret, "key expansion",
                SecurityParameters.server_random‖
                SecurityParameters.client_random)
```

直到产生足够多的输出。在 SSLv3 中,key_block 是一个关于 master_secret、客户端和服务器随机数的函数,而与 TLS 中实际使用的算法不同。

16.3.9 填充

在 SSL 加密之前,使用填充域使得用户数据为加密所需的分组长度的最小整数倍。而在 TLS 中,填充域长度不超过 255 个字节,使用户数据为加密所需的分组长度的任意整数倍的值。例如,如果明文(或压缩明文)加上 MAC 和填充长度域共为 79 个字节,则填充域可以是 1、9、17…249 位。可变的填充可以防止基于交换消息长度分析的攻击。

16.4 HTTPS

HTTPS(SSL 之上的 HTTP)是指结合 HTTP 和 SSL 来实现 Web 浏览器和 Web 服务器之间的安全通信。HTTPS 已经融合在当今的 Web 浏览器中了,而其使用与否还取决于 Web 服务器是否支持 HTTPS 通信。例如搜索引擎就不支持 HTTPS。

Web 浏览器用户能看到的最基本的区别是在 URL(统一资源定位器)地址中是以 https://开头而不是 http://。一个普通的 HTTP 连接使用 80 端口,而当指定使用 HTTPS 时,用的却是 443 端口,这是用来唤醒 SSL 的。

当使用 HTTPS 时,如下的通信元素是加密处理的:

- 请求文档的 URL
- 文档的内容
- 浏览器格式的内容(格式是由浏览器用户决定的)
- 在浏览器和服务器之间传输的 Cookies
- HTTP 报头的内容

HTTPS 的规范文档可参阅 RFC 2818(HTTP Over TLS),在使用 SSL 之上的 HTTP 和 TLS 之上的 HTTP 是没有根本性区别的,这两种方法的实现都成为 HTTPS。

16.4.1 连接初始化

对于 HTTPS 而言,用做 HTTP 客户端的代理和用做 TLS 客户端的代理是一致的。客户端在一个适当的端口初始化一个与服务器的连接,然后发送 TLS ClientHello 来开始 TLS 握手。当 TLS 握

手结束后,客户端会初始化第一个 HTTP 请求。所有的 HTTP 数据都将当做 TLS 应用数据发送。然后是包括保持连接在内的传统 HTTP 操作。

我们应该理解 HTTPS 中有三层不同的含义。在 HTTP 层,一个 HTTP 客户端通过向下一层发送一个连接请求来请求与 HTTP 服务器建立连接。通常,下一层就是 TCP 层或者也可能是TLS/SSL。在 TLS 层,TLS 客户端和 TLS 服务器之间会建立会话。该会话可以在任何时候支持多个连接。正如我们所看到的,一个 TLS 请求的建立是从建立客户端 TCP 实体和服务器端 TCP 实体的 TCP 连接开始的。

16.4.2　连接关闭

一个 HTTP 客户端或者服务器端可以通过在 HTTP 记录中加入如下行:`Connection:close` 来指示关闭连接。当该记录发送后,就意味着连接关闭。

HTTPS 连接的关闭要求 TLS 关闭与远程的对等 TLS 实体之间的连接,该过程要求关闭下一层的 TCP 连接。在 TLS 层,正确关闭连接的方法是通信双方使用 TLS 警报协议发送 `close_notify` 警报。TLS 的实例必须在关闭连接之前初始化一个关闭警报的交换操作。一个 TLS 实例可能允许在一方发送了关闭警报而并未等待对方发送关闭警报而关闭连接,称之为“不完全关闭”。值得注意的是,如果一个用户这么做了,可能是为了之后再次使用该会话。而这也只能在应用层(通常是通过检查 HTTP 消息边界)知道了所有其关心的数据之后才行。

HTTP 客户端还必须处理这种情形,即下一层的 TCP 连接是在没有 `close_notify` 警报和 `Connection: close` 指示的情况下关闭的。这种情形可能是因为服务器端程序出错或者是通信出错导致的 TCP 连接断开。而这种未声称而关闭的 TCP 可能会成为攻击者利用的对象。因此当这些情况发生时,HTTPS 客户端应该有一些安全的提醒。

16.5　SSH

SSH(Secure Shell)是一种保障网络通信安全的协议,并且简单易实现。最初的版本,SSH1 主要是提供一个安全的远程登录工具以取代 TELNET 和其他不安全的远程登录模式。SSH 也能提供 C/S 服务以及类似于文件传输和电子邮件的网络功能。新版本 SSH2 修补了在原版本中的漏洞,并在 RFC 4250 到 RFC 4256 之间的文档中定义。

SSH 客户端和服务器应用程序对大多数操作系统都可用。它可以作为远程登录或者 X 隧道技术的备选方法同时也用于嵌入式系统之外的加密技术的应用程序中。

SSH 是按照三种在 TCP 之上的协议组织的,如图 16.8 所示。

- **传输层协议**:提供服务器认证、数据保密和带前向安全性的数据完整(例如,如果某个密钥在一次会话中泄密了,而该密钥不会影响到之前的会话安全性)。在传输层也提供可选的数据压缩。
- **用户认证协议**:为服务器认证用户。
- **连接协议**:在单一的低层 SSH 连接上提供多逻辑的通信信道。

16.5.1　传输层协议

主机密钥

服务器认证在传输层进行,是根据服务器持有一个公私密钥对的原理。一台服务器可能会有

多个主机密钥用于多个不同的非对称加密算法。而多个主机又可能共享同一个主机密钥。在任何情况下,服务器的主机密钥都是在密钥交换阶段用来认证主机身份的。为了其可行性,客户端必须有服务器主机公钥的先验信息。RFC 4251 给出了两个可供选择的可信模型:

(1) 在客户端拥有一个本地的数据库将每一个主机名(由用户输入的)与对应的主机公钥联系起来。该方法不需要集中的管理设施或第三方协助。底侧是用户名和密钥对的数据库维护负担比较大。

(2) 主机的用户名和密钥对的认证过程由可信认证中心(CA)进行。客户端只知道 CA 的根密钥并只能验证由 CA 认证的所有的主机密钥的合法性。这种可选择性降低了维护的难度,因为理论上只要在客户端安全地存储一个 CA 密钥。另一方面是,在认证成为可能之前每一个主机密钥必须有认证中心进行验证。

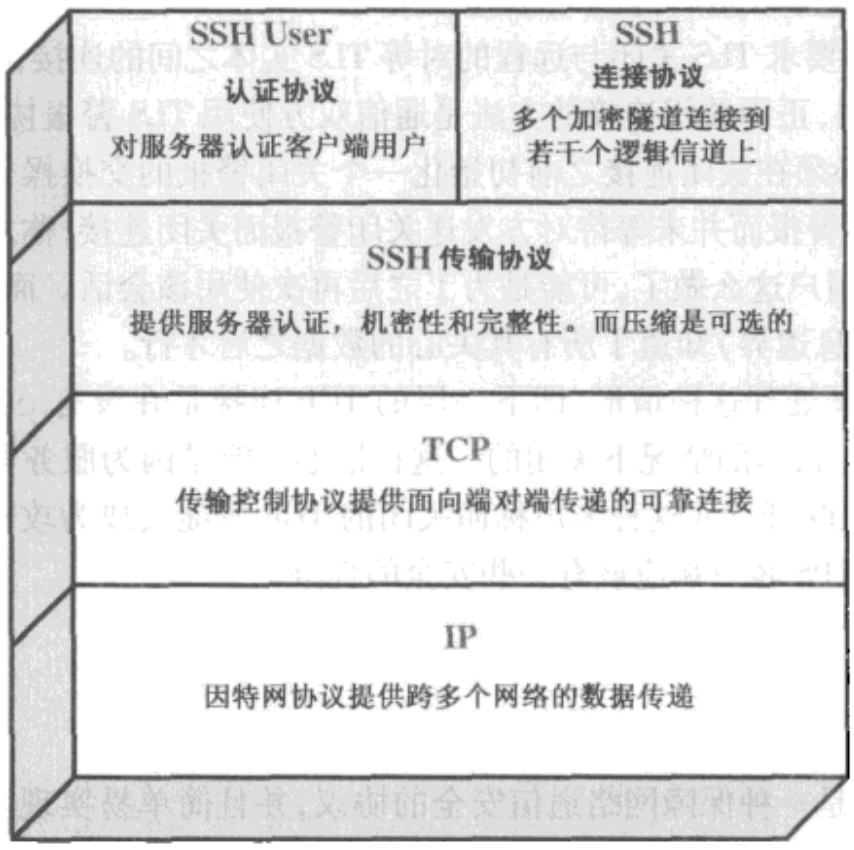


图 16.8 SSH 协议栈

数据包交换

图 16.9 描述了 SSH 传输层协议中的事件序列。首先,客户端建立一个到服务器的 TCP 连接。这是通过 TCP 协议完成的并且是不属于传输层协议的部分。当连接建立以后,客户端和服务器端就开始交换数据(也成为数据包报文,在 TCP 层的数据域)。每一个数据包都有如下的格式(如图 16.10 所示)。

- **数据包长度**:以字节为单位的数据包长度,不包含包长度域和 MAC 域。
- **填充长度**:随机填充域的长度。
- **有效载荷**:数据包的有用内容。在算法协商之前,该域是未压缩的。当压缩算法协商以后,下一个数据包的有效载荷域就会被压缩。
- **随机填充**:当协商成功一个加密算法,则该域就会附上。它包含了用于填充的随机字节,以使整个数据包(不包含 MAC 域)的长度是密码块大小的整数倍或者是 8 个字节的流密码。
- **消息认证码(MAC)**:当协商好消息认证,则该域就包含了 MAC 的值。MAC 是在这个数据包(不包含 MAC 域)和一个序列码上计算的。该序列码是一个 32 位的指示数据包序列的

编码,并且将第一个数据包初始化为 0 而后续包的序列码则随之递增。序列码是不包含在数据包中在 TCP 连接链路上传输的。

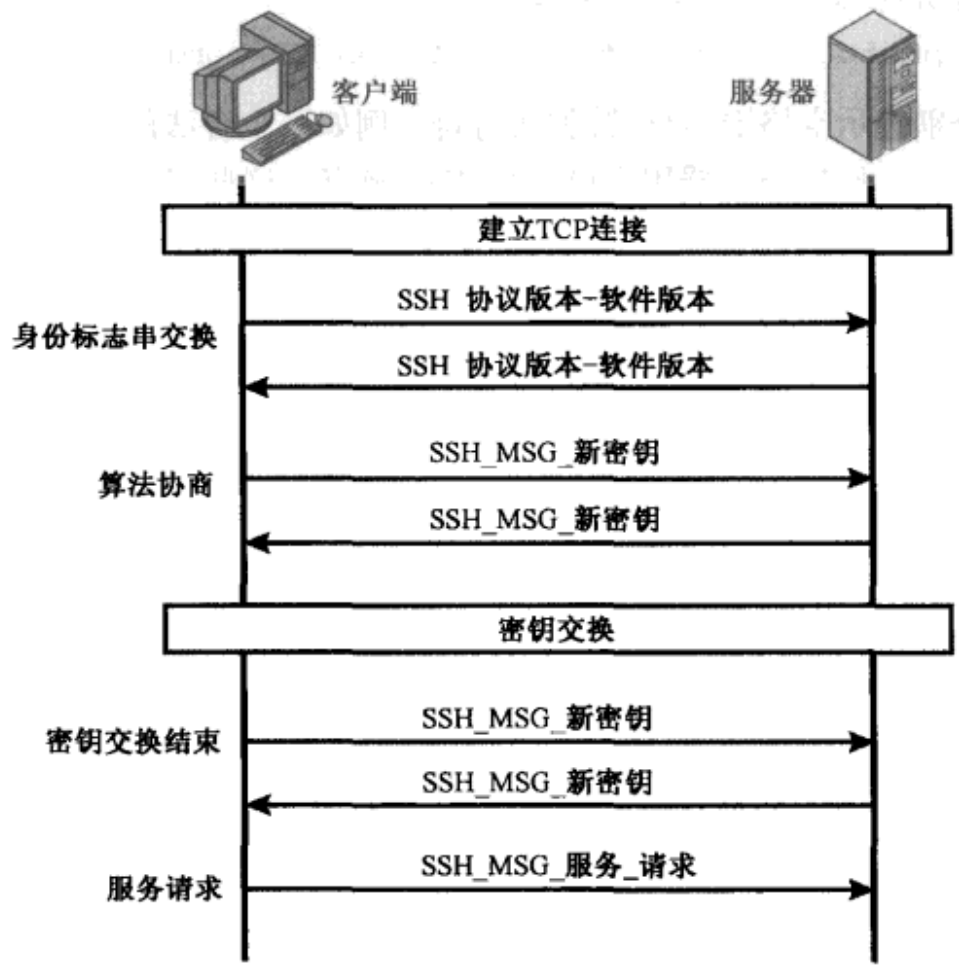


图 16.9　SSH 传输层协议包交换

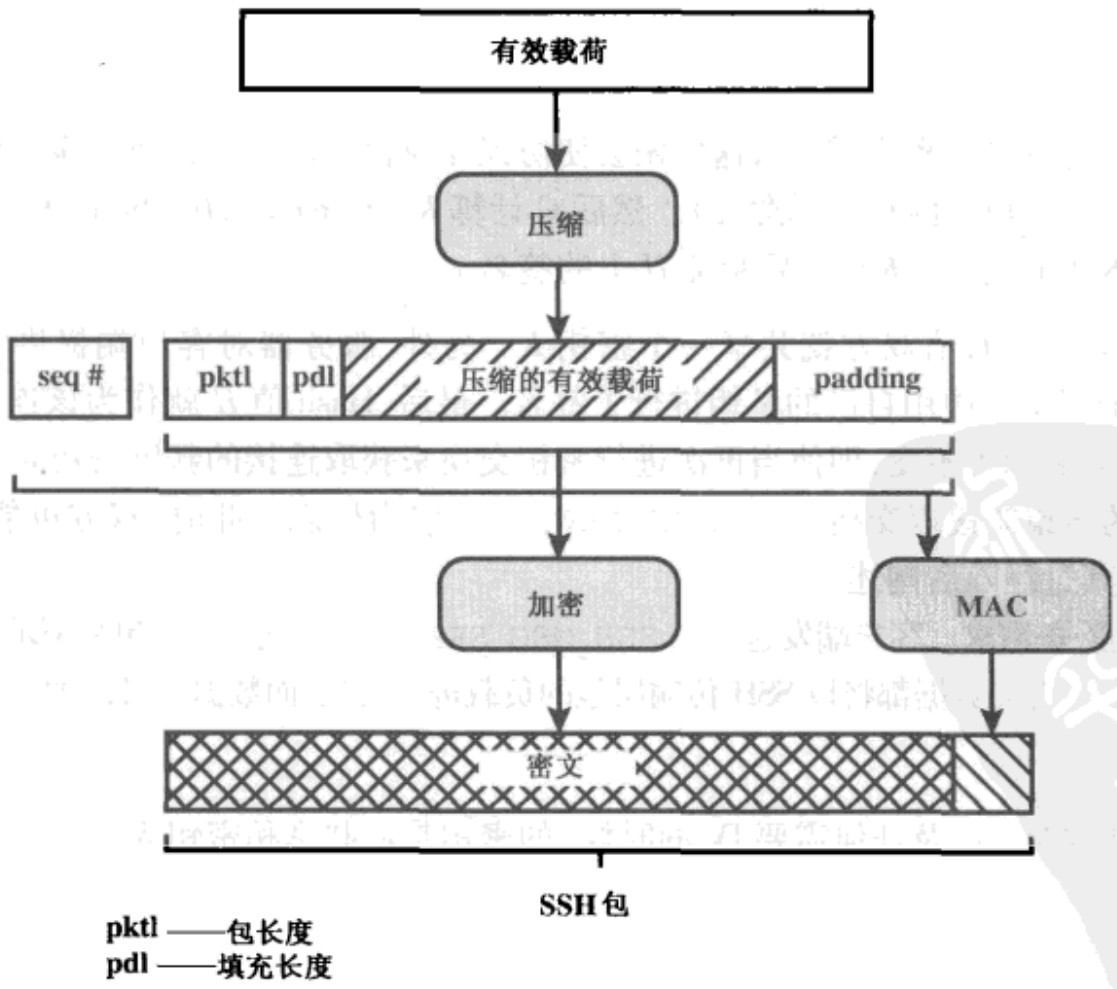


图 16.10　SSH 传输层协议包格式

当加密算法协商好并计算好 MAC 值之后,整个数据包(除 MAC 域之外)会被加密。

SSH 传输层数据包交换包含了一系列步骤(如图 16.9 所示)。第一步,**身份标志串交换**,客户端发送一个包含了身份标志串(形式如下)的数据包:

```
SSH-protoversion-softwareversion SP comments CR LF
```

其中,SP,CR 和 LF 分别表示空格键,回车符和换行符。例如一个合法的串是 SSH-2.0-billsSSH_3.6.3q3 <CR> <LF>。然后服务器用自己的标志串响应。这些身份标志串用在 Diffie-Hellman 密钥加密换中。

随后的是**算法协商**。通信双发各自发送一个 SSH_MSG_KEXINIT,其中包含了支持算法的列表并以自己倾向程度进行排序。对每一种密码算法都有一个列表。这些算法包含密钥交换、加密、MAC 算法和压缩算法。对每一类,所选择的算法是在客户端列表中的第一个算法并且服务器端也支持该算法。

下一步是**密钥交换**。规范文档允许有多种可选方法用来密钥交换,但如今,只有 Diffie-Hellman 密钥交换的版本可以使用。RFC 2409 定义了这两种版本并且对每个方向只需要一个包。在密钥交换中会涉及如下步骤。这里,定义 C 为客户端,S 为服务器端,p 是一个安全的大整数,g 是有限域 GF(p)的循环子群的生成元,q 是该循环子群的阶;V_S 是 S 的标志串,V_C 是 C 的标志串,K_S 是 S 的公开主机密钥,I_C 是 C 的 SSH_MSG_KEXINIT 消息,I_S 是 S 的 SSH_MSG_KEXINIT 消息(该消息在之前的部分已经交换了)。算法选择协商之后客户端和服务器端双方都知道 p,g 和 q 的值。Hash 函数 hash()也在算法协商阶段被决定了。

(1) C 生成一个随机整数 $x(1<x<q)$并计算 $e=g^x \bmod p$。C 将 e 发送给 S。

(2) S 生成一个随机整数 $y(0<y<p)$并计算 $f=g^y \bmod p$。S 接收到 e,然后计算 $K=e^y \bmod p$,$H=\text{hash}(V_C \| V_S \| I_C \| I_S \| K_ \| e \| f \| K)$,然后用自己的主机私钥签名 H 形成 s。S 将(K_S $\|$ $f \| s$)发送给 C。签名操作可能会涉及第二次散列运算。

(3) C 验证 K_S 确实是 S 的主机密钥(例如使用证书或者本地数据库)。C 也允许在未进行验证情况下接受密钥,但是,这样做会使协议在应付主动攻击方面有脆弱性(但是可能在许多环境中短期内有实际意义)。然后 C 计算 $K=f^x \bmod p$,$H=\text{hash}(V_C \| V_S \| I_C \| I_S \| K_S \| e \| f \| K)$,然后验证 H 上的签名 s。

这些步骤结束后,通信双方就共享一个密钥 K。另外,服务器对客户端提供了认证,因为在 Diffie-Hellman 密钥交换中用自己的私钥进行了签名。最后,Hash 值 H 就作为该连接的会话标志。计算完成之后,会话标志不变,即使当再次进行密钥交换来获取连接的新鲜密钥时。

密钥交换的结束阶段以交换 SSH_MSG_NEWKEYS 包为标志。此时,双方可能开始使用从 K 中生成的密钥,其过程随后阐述。

最后一步是**服务请求**。客户端发送一个 SSH_MSG_SERVICE_REQUEST 包来请求用户认证或者连接协议。这之后,所有的数据都将以 SSH 传输层包的负载进行交换,而数据包又由加密和 MAC 保护。

密钥生成

用于加密和 MAC(以及任何需要 IV 的时候)的密钥是从共享私密钥 K、从密钥交换 H 函数中获得的 Hash 值、会话标志生成的。其中会话标志和 H 是一致的,除非在初始密钥交换之后又有随后的密钥交换进行。其值的计算如下:

- 客户端到服务器的初始化值:HASH($K \| H \| $ "A" $\|$ session_id)
- 服务器到客户端的初始化值:HASH($K \| H \| $ "B" $\|$ session_id)

- 客户端到服务器的加密密钥:HASH(*K* ‖ *H* ‖ "C" ‖ session_id)
- 服务器到客户端的加密密钥:HASH(*K* ‖ *H* ‖ "D" ‖ session_id)
- 客户端到服务器的完整性密钥:HASH(*K* ‖ *H* ‖ "E" ‖ session_id)
- 服务器到客户但的完整性密钥:HASH(*K* ‖ *H* ‖ "F" ‖ session_id)

其中 HASH()是在密钥协商阶段决定的散列函数。

表 16.3　SSH 传输层密码算法

密　文	
3des-cbc*	CBC 模式下的三密钥 3DES
blowfish-cbc	CBC 模式下的 blowfish
twofish256-cbc	CBC 模式下的 twofish,有 256 位密钥
twofish192-cbc	192 位密钥的 twofish
twofish128-cbc	129 位密钥的 twofish
aes256-cbc	CBC 模式下的 256 位密钥的 AES
aes192-cbc	192 位密钥的 AES
aes128-cbc**	128 位密钥的 AES
Serpent256-cbc	CBC 模式下 256 位密钥的 Serpent
Serpent192-cbc	192 位密钥的 Serpent
Serpent128-cbc	128 位密钥的 Serpent
arcfour	128 位密钥的 RC4
Cast128-cbc	CBC 模式下的 CAST-128

MAC 算法	
hmac-sha1*	HAMC-SHA1;摘要长度 = 密钥长度 = 20
hmac-sha1-96**	HAMC-SHA1 前 96 位;摘要长度 = 12;密钥长度 = 20
hmac-md5	HAMC-SHA1;摘要长度 = 密钥长度 = 16
hmac-md5-96	HAMC-SHA1 的前 96 位;摘要长度 = 12;密钥长度 = 16

压缩算法	
none*	无压缩
zlib	在 RFC 1950 和 RFC 1951 中定义

* ——必需的　　** ——建议的

16.5.2　用户认证协议

用户认证协议提供了用户向服务器证明自己的方法。

消息类型和格式

在用户认证协议中常使用三种类型的消息。从客户端发出的认证请求的格式:

byte	SSH_MSG_USERAUTH_REQUEST(50)
string	user name
string	service name
string	method name
…	method specific fields

其中 user name 是指用户名客户端声称的认证标志,service name 是指客户端请求访问的设备(通常是 SSH 连接协议),method name 是请求中用来认证的方法。第一个 byte 的值为十进制数的 50,表示该消息是 SSH_MSG_USERAUTH_REQUEST。

当服务器拒绝认证请求或者接受请求但是还需要更多的认证方法时,服务器会发出如下格式的消息:

byte	SSH_MSG_USERSUTH_FAILURE(51)
name-list	authentications that can continue
boolean	partial success

其中 name-list 是可能会继续对话的方法。当服务器接受认证,就发送一个 byte 消息:SSH_MSG_USERAUTH_SUCCESS(52)。

消息交换

消息交换涉及如下步骤:

(1) 客户端发送一个 SSH_MSG_USERAUTH_REQUEST。
(2) 服务器查看并确认 user name 是否合法。若不合法,服务器返回 SSH_MSG_USERAUTH_FAILURE 和请求失败的部分成功值;若合法,服务器执行步骤(3)。
(3) 服务器返回 SSH_MSG_USERAUTH_FAILURE 和将会使用的认证方法列表。
(4) 客户端选择一种可以接受的认证方法并发送 SSH_MSG_USERAUTH_REQUEST 以及该方法名和需要的方法指定域。这里会有一系列的信息交换来执行方法。
(5) 若认证成功并需要更多的认证方法,则服务器转向执行步骤(3)并使用一个真实的部分成功值。若认证失败,则服务器执行步骤(3)并使用一个为假的部分成功值。
(6) 当所有的认证方法都成功时,服务器发送一个 SSH_MSG_USERAUTH_SUCCESS 消息,并且认证协议结束。

认证方法

服务器可能会要求使用若干如下的认证方法:

- **公开密钥**:该方法的细节取决于采用的公钥密码算法。基本上是客户端给服务器发送一个消息,其中包含了客户端的公钥,并用客户端的私钥签名该消息。当服务器接收到该消息就验证消息中提供的密钥是否能进行认证,如果可以就验证签名是否正确。
- **口令密码**:客户端发送一个消息,其中包含了明文形式的口令密码,而其又在传输层协议中被加密。
- **基于主机**:认证在客户端的主机上进行而不是客户端本身。因此,一个支持多客户端的主机可以提供该主机上所有客户端的认证。该方法中客户端发送一个由其所在的主机的私钥签名的消息。因此,SSH 服务器并不是直接验证用户的身份,而是认证了客户端主机的身份并且当主机声称在客户机一方以及验证了该用户时,服务器就认为已经验证了用户。

16.5.3 连接协议

SSH 连接协议在 SSH 传输层协议的顶层运行,并假设使用了安全的认证连接[①]。安全的认证连接是指连接协议用一个通道虚拟出多条逻辑信道。

信道机制

使用 SSH 的所有类型的通信,例如终端会话,都支持使用独立信道。通信双方的任意一方都可以开启信道。对每一条信道,每一端都与一个唯一的信道序列码对应,即在同一条信道的两端不必使用同一个序列号。使用窗口机制对信道进行流量控制。只有当接收到表示窗口空间可用的消息时才能在信道中发送数据。

信道的生命周期有三个阶段:开启信道、数据传输和关闭信道。

当任何一端希望开启新的信道时,它就为信道动态分配一个本地序列号并发送如下格式的消息:

byte	SSH_MSG_CHANNEL_OPEN
string	channel type

① RFC 4254, *The Secure Shell (SSH) Connection Protocol*, 定义了连接协议运行在传输层协议和用于认证协议之上。RFC 4251, *SSH Protocol Architecture*, 定义了连接协议运行在用于认证协议之上。事实上,连接协议是运行在传输层协议之上的,但是前提是假设用户认证协议已经在之前被唤醒了。

uint32	sender channel
uint32	initial window size
uint32	maximum packet size
…	channel type specific data follows

其中 uint32 是指无符号 32 位整型。channel type 标志了该信道使用的应用程序(随后会介绍)。sender channel 是本地的信道序列号。initial window size 表示发送者在不调整窗口的情况下可以发送多少字节的数据。maximum packet size 指发送者发送的独立的数据包的最大尺寸。例如,一个用户可能希望使用小的数据包进行交互式的连接来获取在低速链路上的更好的交互回应。

当远程端可以开启信道时,它返回一个 SSH_MSG_CHANNEL_OPEN_CONFIRMATION 消息,其中包含了发送者的信道序列号、接收者的信道序列号、窗口数和数据包的大小。否则,远程端返回一个 SSH_MSG_CHANNEL_OPEN_FAILURE 消息和一个表示失败原因的报错码。

当一个信道开启时,就发送 SSH_MSG_CHANNEL_DATA 消息并执行数据传输。消息中包含接收者信道序列号和数据块。只要信道开启,这些消息可以任意方向传输。

当任意一方希望关闭信道时,就发送 SSH_MSG_CHANNEL_CLOSE 消息,其中包含了接收方信道序列号。

图 16.11 给出了连接协议消息交换的例子。

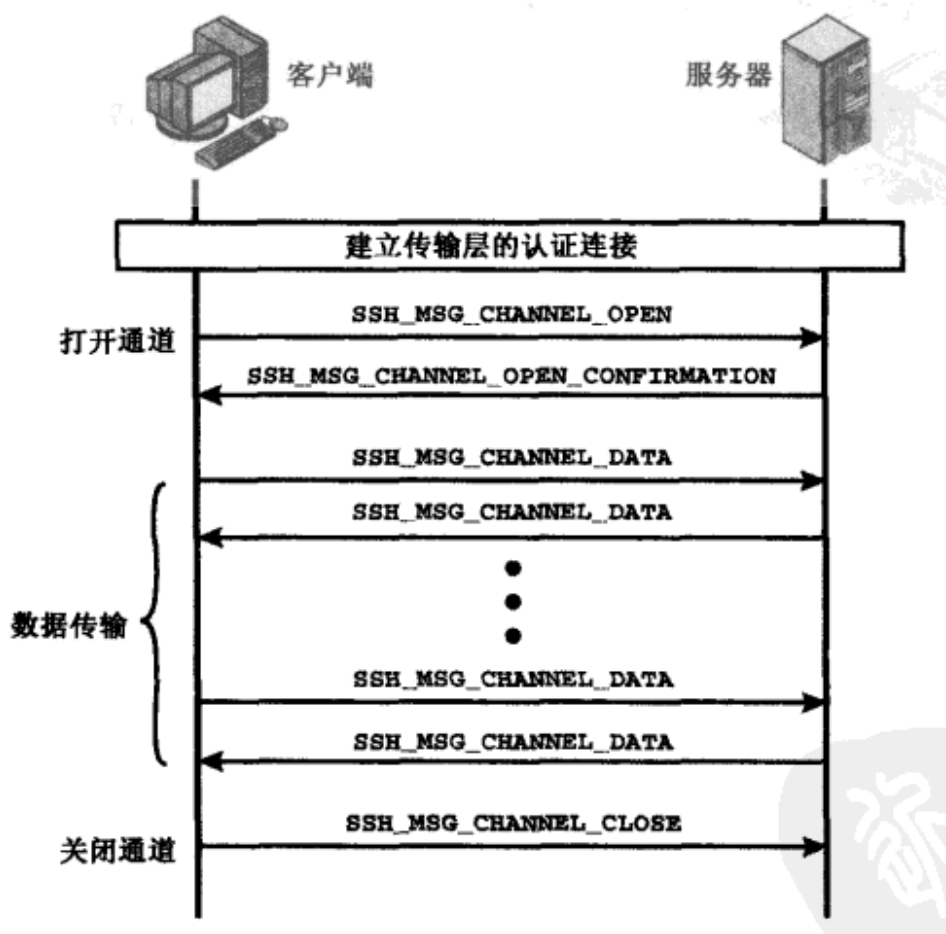


图 16.11　SSH 连接协议消息举例

信道类型

在 SSH 连接协议的规范文件中有四种信道类型:

- **会话**:程序的远程执行。这些程序可以是 shell、诸如文件传输或者是电子邮件的应用程序、系统命令和内置的子系统。当一个会话信道开启时,接下来的一些请求就用来开始运行远程程序。

- **x11**:这是指 X Window 系统,它是为网络计算机提供图形化用户界面(GUI)的一个计算机系统和网络协议。X 允许程序在网络服务器上运行并且在桌面机上显示。
- **前向 tcpip**:这是远程端口转发的,在下一节中会介绍。
- **直接 tcpip**:这是本地端口转发的,也在下一节中介绍。

端口转发

SSH 最有用的一个特征是端口转发。基本上,端口转发可以将任何的非安全的 TCP 连接转换成安全的 SSH 连接。这也被称为 SSH 的隧道技术。我们必须理解在该语境下的端口。端口是指一个 TCP 用户的标志。因此,任何在 TCP 顶层运行的应用程序都有一个端口号。TCP 链路就是在端口号的基础上向合适的应用程序发送数据的。一个应用程序可能会使用多个端口号。例如,简单邮件传输协议(SMTP),在服务器端是用 25 端口号监听的,于是一个 SMTP 请求使用 TCP 并发送数据到目的端 25 端口。TCP 识别这是 SMTP 服务器地址,于是就将数据路由至 SMTP 服务器应用程序。

图 16.12 描述了端口转发相关的基本概念。我们有一个用端口号 x 标志的客户端应用程序,并且服务器端的应用程序标志是端口号 y。在某时刻,客户端应用程序唤醒本地 TCP 实体并请求一个到端口号为 y 的远程服务器的连接。本地的 TCP 实体就与远程的 TCP 实体协商一个 TCP 连接,这样本地端口 x 就和远程端口 y 连接起来了。

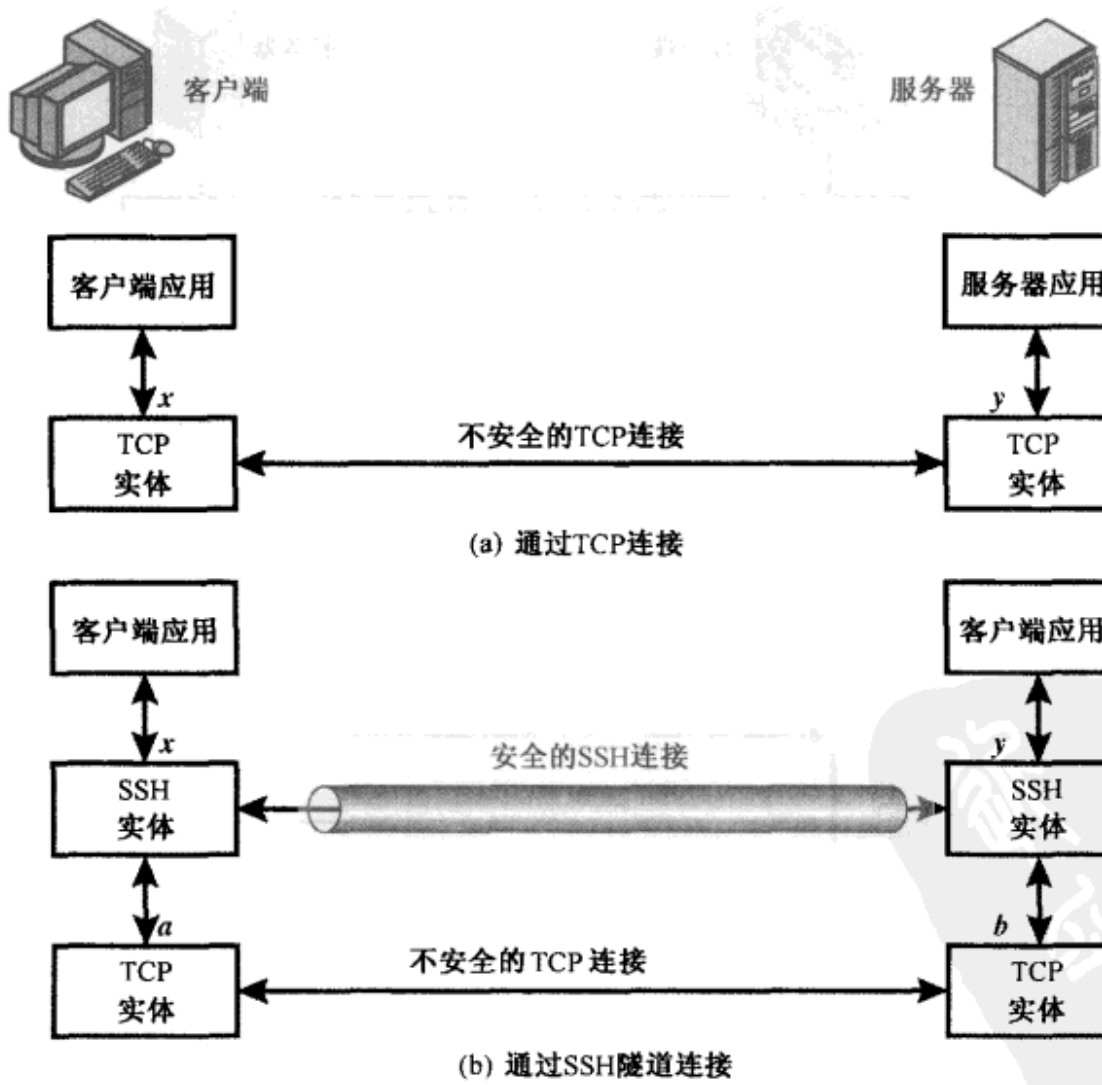


图 16.12 SSH 传输层包交换

为了使连接安全,需要配置 SSH 以使 SSH 传输层协议建立一个 SSH 客户端和服务器实体(分别用 TCP 端口号 a 和 b 表示)间的 TCP 连接。一个安全的 SSH 隧道就在 TCP 连接的基础上建立了。数据包从客户端 x 端口发出转到本地 SSH 实体,并通过隧道转发至远程 SSH 实体,再有远程 SSH 实体将数据转发到端口为 y 的服务器应用程序。相反方向的传输也类似。

SSH 支持两种类型的端口转发：本地转发和远程转发。本地转发允许客户端建立“hijacker”进程。该进程可以截获特定的应用层数据包并将其从一个非安全的 TCP 连接转发到安全的 SSH 隧道。可以设置 SSH 监听特定的端口。SSH 可以抓取特点端口的所有数据包并通过 SSH 隧道将其发送。在另一端，SSH 服务器发送接收到的数据包到客户端应用程序指示的目的端口。

下面的例子可以帮助我们透彻理解本地转发。假若你在本地桌面机上有一个电子邮件的客户端程序，并使用它从你的邮件服务器通过邮政协议（POP）发送一个电子邮件。POP3 指定的端口号是 110。我们可以通过如下方法来使数据包安全的传输：

（1）SSH 客户端建立一个到远程服务器的连接。
（2）选定一个未使用的本地端口号，不妨设为 9999，配置 SSH 使其接收从该端口发出并且目标是服务器的 110 端口的数据包。
（3）SSH 客户端通知 SSH 服务器端创建一个到目标地址的连接，在这个例子中邮件服务器的端口号是 110。
（4）客户端发送任意位位的数据到本地端口 9999 并让其将数据用 SSH 会话加密发送到服务器端。SSH 服务器解密收到的数据并将明文发送给端口 110。
（5）另一个方向，SSH 服务器获取任意位位从端口 110 接收到的数据，并将其在 SSH 会话中发送回客户端。客户端解密数据并将明文发送到与端口 9999 相连的进程。

使用远程转发，用户的 SSH 客户端代替服务器行动。客户端在一个给定的目标端口接收到数据包，将数据包放至正确的端口并将其发送到用户选择的目的地址。如下是一个典型的远程转发的例子。你希望在家里访问一台办公室的服务器。因为办公室的服务器在防火墙之后，它将不接受你家里计算机发出的 SSH 请求。但是，你可以使用远程转发从办公室建立一个 SSH 隧道。如下是具体步骤：

（1）在办公室计算机上建立一个到家里计算机的 SSH 连接。防火墙是允许这样做的，因为这是向外的连接。
（2）配置 SSH 服务器使其监听本地端口，不妨设为 22，并通过 SSH 连接分发数据到远程的端口，设为 2222。
（3）你现在可以使用家中的计算机将 SSH 配置为可以接收端口 2222 的数据包。
（4）这样就有了一个 SSH 隧道可用于远程登录办公室服务器。

16.6　推荐读物和网站

[RESC01]对 SSL 和 TLS 细节描述得比较全面。[BARR05]给出了 SSH 的整体介绍。[YLON96]介绍了 SSH 的原始版本（SSH-1）。

BARR05 Barrett, D.; Silverman, R.; and Byrnes, R. *SSH The Secure Shell: The Definitive Guide*. Sebastopol, CA: O'Reilly, 2005.

RESC01 Rescorla, E. *SSL and TLS: Designing and Building Secure Systems*. Reading, MA: Addison-Wesley, 2001.

YLON96 Ylonen, T. "SSH-Secure Login Connection over the Internet." *Proceedings, Sixth USENIX Security Symposium*, July 1996.

推荐网站

- Transport Layer Security Charter:最新的 RFC 和 TLS 的互联网草案。
- OpenSSL Project:开源的 SSL 和 TLS 软件的开发工作组。该站点包含相关文档和连接。

16.7 关键术语、思考题和习题

关键术语

警报协议
密码变换协议
握手协议
HTTPS(SSL 上的 HTTP)
主密钥
安全外壳(SSH)
安全套接子层(SSL)
传输层安全(TLS)

思考题

16.1 指出图 16.1 中每种方法的优点。
16.2 SSL 包含哪些协议?
16.3 SSL 连接和 SSL 会话的区别是什么?
16.4 列举并简单定义 SSL 会话状态的参数。
16.5 列举并简单定义 SSL 会话状态的连接。
16.6 SSL 记录协议提供哪些服务?
16.7 SSL 记录协议传输有哪些步骤?
16.8 HTTPS 的目的是什么?
16.9 哪些应用程序可以使用 SSH?
16.10 列举并简单定义 SSH 协议。

习题

16.1 在 SSL 和 TLS 中,为什么有分离的修改密码规范协议,而不是在握手协议中包含修改密码规范消息?
16.2 在 SSL 修改密码规范协议中的 MAC 服务的作用是什么?
16.3 考虑以下 Web 安全性威胁,并描述 SSL 是如何防止这些威胁的。
(a) 穷举密码分析攻击:穷举传统加密算法的密钥空间。
(b) 已知明文字典攻击:许多消息中包含的是可以预测的明文,如 HTTP 中的 GET 命令。攻击者构造一个包含各种可能的已知明文加密字典。截获加密消息,攻击者可以将包含已知明文的加密部分和字典中的密文进行比较。如果多次匹配成功,可以得到正确的密码。此攻击对于小尺寸的密钥空间非常有效(如 40 位的密钥)。
(c) 重放攻击:重放先前的 SSL 握手消息。
(d) 中间人攻击:在密钥交换时,攻击者向服务器假扮客户端,向客户端假扮服务器。
(e) 密码窃听:HTTP 或其他应用流量的密码被窃听。
(f) IP 欺诈:使用伪造的 IP 地址使主机接收伪造的数据。
(g) IP 劫持:中断两个主机间活动的、经过认证的连接,攻击者代替一方的主机进行通信。
(h) SYN 洪:攻击者发送 TCP SYN 消息请求连接,但不回答建立连接的最后一条消息。被攻击的 TCP 模块通常为此预留几分钟的"半开连接",重复 SYN 消息可以阻塞 TCP 模块。
16.4 利用本章中学到的知识,回答:在 SSL 中接收者在接收到顺序紊乱的 SSL 记录块时能为它们排序吗?如果可以,解释它是如何做的;如果不可以,为什么?
16.5 对于 SSH,包含 MAC 之内的数据包加密方法有哪些优势?

第 17 章　无线网络安全

17.1　IEEE 802.11 无线网络概述
- 17.1.1　Wi-Fi 联盟
- 17.1.2　IEEE 802 协议体系结构
- 17.1.3　IEEE 802.11 网络构成和体系结构模型
- 17.1.4　IEEE 802.11 服务

17.2　IEEE 802.11i 无线局域网安全
- 17.2.1　IEEE 802.11i 服务
- 17.2.2　IEEE 802.11 i 操作阶段
- 17.2.3　发现阶段
- 17.2.4　认证阶段
- 17.2.5　密钥管理阶段
- 17.2.6　安全数据传输阶段
- 17.2.7　IEEE 802.11i 伪随机函数

17.3　无线应用通信协议概述
- 17.3.1　操作概述
- 17.3.2　无线标记语言
- 17.3.3　WAP 体系结构
- 17.3.4　无线通信应用环境
- 17.3.5　WAP 协议体系结构

17.4　无线传输层安全
- 17.4.1　WTLS 会话和连接
- 17.4.2　WTLS 协议体系结构
- 17.4.3　密码算法

17.5　WAP 端到端安全

17.6　推荐读物和网站

17.7　关键术语、思考题和习题

Investigators have published numerous reports of birds taking turns vocalizing; the bird spoken to gave its full attention to the speaker and never vocalized at the same time, as if the two were holding a conversation. Researchers and scholars who have studied the data on avian communication carefully write: (a) the communication code of birds, such as crows, has not been broken by any means; (b) probably all birds have wider vocabularies than anyone realizes; and (c) greater complexity and depth are recognized in avian communication as research progresses.

—*The Human Nature of Birds*, Theodore Barber

要　点

- IEEE 802.11 是一个无线局域网标准。互操作的兼容标准的实现是 Wi-Fi。
- IEEE 802.11i 规定了 IEEE 802.11 局域网的安全标准，包含了认证、数据完整性、数据机密性和密钥管理。互操作的实现是 Wi-Fi 保护访问。
- 无线应用通信协议(WAP)是提供手机用户和其他无线终端访问包括互联网和 Web 在内的通信和信息服务的标准。
- WAP 的安全性能主要由无线传输层安全(WTLS)提供，WTLS 主要提供了移动设备和 WAP 网关与互联网之间的安全服务。
- 保障 WAP 端到端安全有很多方法。其中一个著名的方法是使移动设备实现 TCP/IP 上的 TLS 以使无线网络支持传输 IP 数据包。

本章主要介绍两种重要的无线网络安全方案。首先，我们阐述无线局域网安全的 IEEE 802.11i 标准，该标准是 IEEE 802.11 标准的一部分，也被称为 Wi-Fi。我们从讨论 IEEE 802.11 开始，然后介绍 IEEE 802.11i 的一些细节。

本章的其他内容是介绍诸如手机这类的无线移动设备对 Web 访问的安全标准。开始部分我们先纵览一下无线应用通信协议(WAP),它是在蜂窝网络的移动设备和 Web 服务器之间的一系列通信标准的集合。然后,我们探讨无线传输层安全(WTLS)协议,它提供移动设备与运行在蜂窝网络和互联网之间的网关这两者之间的安全性。最后,我们阐述 WAP 和 Web 服务器之间的端到端安全通信。

17.1 IEEE 802.11 无线网络概述

IEEE 802 是一个开发了大量局域网(LAN)标准的委员会。在 1990 年,IEEE 802 委员会成立了新的工作组——IEEE 802.11,负责开发无线局域网(WLAN)的协议和传输规范。从那时起,开发了不同频段和不同数据传输速率的无线局域网需求。与需求同步,IEEE 802.11 工作组提出了一系列持续扩展的标准。表 17.1 简要给出了 IEEE 802.11 标准中使用的关键术语。

表 17.1 IEEE 802.11 术语表

接入点(AP)	任何具有基站功能并且提供通过无线媒体与已连接的基站访问分布系统的实体
基本服务集(BSS)	由一个单一的协调功能控制的基站的集合
协调功能	逻辑功能,决定基本服务集中运行的基站何时允许传输和可以接收 PDU
分布式系统(DS)	用来互连 BSS 并整合 LAN 以生成 ESS 的系统
扩展服务集(ESS)	若干个互连的 BSS 的集合,整合了 LAN 使任何与此 BSS 其中之一相连的基站都觉得是单一的 BSS 到 LLC 层的连接
MAC 协议数据单元(MPDU)	使用物理层服务的对等 MAC 实体之间交换的数据单元
MAC 服务数据单元(MSDU)	在 MAC 用户之间传输的信息单元
基站(Station)	任何符合 IEEE 802.11 MAC 和物理层标准的设备

17.1.1 Wi-Fi 联盟

第一个受到工业界广泛支持的 802.11 标准是 802.11b。尽管 802.11b 的产品都是基于相同的标准生产的,但是仍然对不同生产商生产的产品的兼容性有质疑。为了解决该问题,无线以太网兼容性联盟(WECA)以一个工业团体的形式在 1999 年形成。该组织随后更名为无线保真(Wi-Fi)联盟并创建了一套用于验证 802.11b 产品兼容性的工具。验证 802.11b 产品的专业数据就是 Wi-Fi,Wi-Fi 的验证也成了 802.11b 产品的外延。Wi-Fi 联盟还开发了 802.11a 产品的验证过程,称之为 Wi-Fi5。Wi-Fi 联盟关注与无线局域网相关的大部分领域,包括企业、家庭和热点。

最近,Wi-Fi 开发了 IEEE 802.11 安全标准的验证过程,称之为 Wi-Fi 受保护访问(WPA)。最新的 WPA 版本是 WPA2,整合了所有 IEEE 802.11i 无线局域网安全规范的特性。

17.1.2 IEEE 802 协议体系结构

在开始之前先简要介绍一下 IEEE 802 协议的体系结构。IEEE 802.11 标准是在层次化协议集的框架下定义的。用于 IEEE 802 标准的该框架如图 17.1 所示。

物理层

IEEE 802 参考模型的最底层就是**物理层**,它包含了信号的编码/解码、位传输/接收等功能。另外,物理层还包括了传输介质的规范。在 IEEE 802.11 中还定义了频率带宽和天线的特性。

媒体访问控制层

所有的局域网都包含共享网络传输能力的设备集合。为了有序和高效地使用传输能力,对传

输媒体的访问控制是必要的。这就是媒体访问控制(MAC)层的功能。MAC 层从上层协议[通常是逻辑链路控制(LLC)层]中接收数据块[也被称为 MAC 服务数据单元(MSDU)]。总之,MAC 层执行了如下的功能:

- 在传输方面,封装数据成帧[通常称为 **MAC 协议数据单元(MPDU)**]并带有地址和校验域。
- 在接收方面,拆封数据帧,执行地址辨别和错误检查。
- 管理对 LAN 传输媒介的访问。

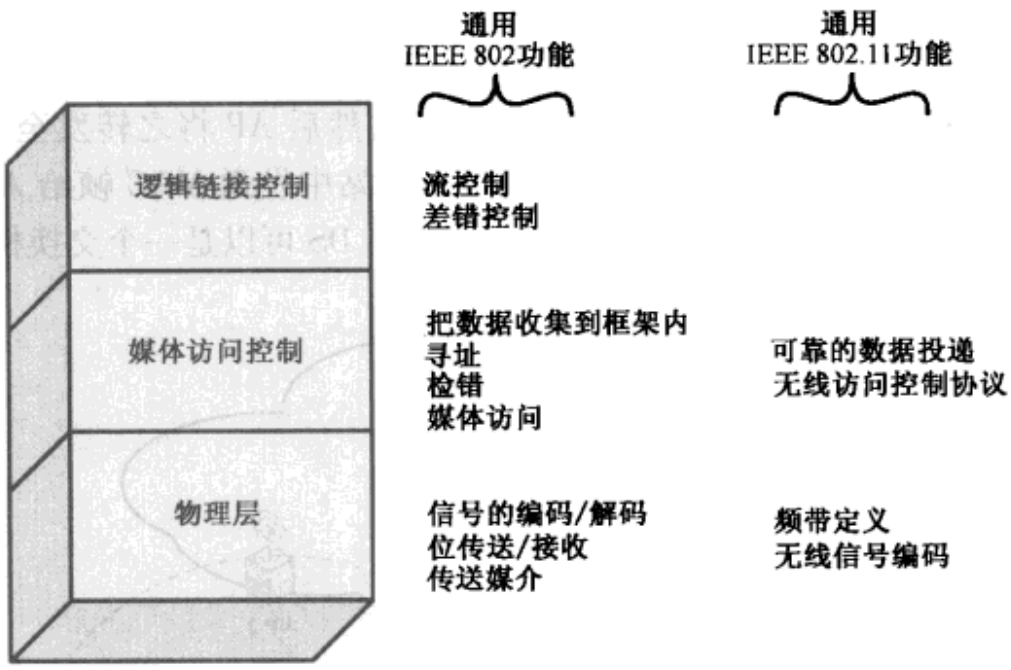


图 17.1　IEEE 802.11 协议栈

MPDU 的具体格式与所使用的 MAC 协议有关。但是基本上,所有的 MPDU 都有与图 17.2 相似的格式。这个框架有如下的域:

- **MAC 控制**:该域包含 MAC 协议功能需要的任何协议控制信息。例如,在这里会指示一个优先层。
- **目的 MAC 地址**:局域网中 MPDU 的目的物理地址。
- **源 MAC 地址**:局域网中 MPDU 的源物理地址。
- **MAC 服务数据单元**:上一层的数据。
- **CRC**:循环冗余校验域,也被称为帧校验序列(FCS)域。和用在其他数据链路层控制协议中一样,这是一种检错码。CRC 是在整个 MPDU 的位上计算的。发送者计算了 CRC 并将其附加在数据帧上。接收者在接收到的 MPDU 上执行相同的计算并对比该计算结果与接收到的 MPDU 中 CRC 域的值。若两个值不匹配,则传输中必然在后若干位发生了改变。

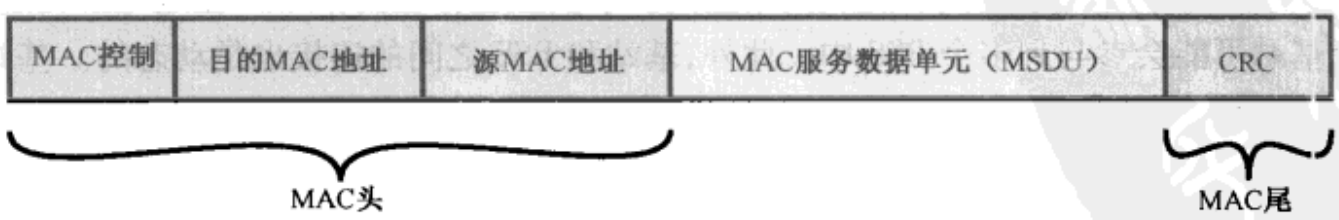


图 17.2　通用 IEEE 802 MPDU 格式

MSDU 前面的域被称为 **MAC 头部**,MSDU 之后的域称为 **MAC 尾部**。MAC 头部和 MAC 尾部包含控制信息,这些控制信息和数据域一起在 MAC 协议中发挥作用。

逻辑链路控制

在大多数数据链路控制协议中,数据链路协议实体不仅负责使用 CRC 检错还要通过对失效

帧的重发来纠正传输中的错误。在局域网协议框架中,这两个功能被分解到 MAC 层和 LLC 层。MAC 层负责检错并将出错的数据帧丢弃。LLC 层跟踪任意接收成功的帧并重发失效的帧。

17.1.3 IEEE 802.11 网络构成和体系结构模型

图 17.3 描述了有 802.11 工作小组开发的模型。无线局域网最小的组成单元是基本服务集(BSS),它包含了执行相同 MAC 协议和竞争访问相同的共享无线媒介的无线基站。一个 BSS 可以是独立的也可以是通过接入点(AP)与主干分布式系统(DS)相连接。接入点的功能相当于网桥或中继点。在 BSS 中,客户基站之间并不是直接相互通信的。假如 BSS 中的一个基站要和同一个 BSS 中的另一个基站通信,则源基站向 AP 发送 MAC 帧,然后 AP 将之转发至目的地基站。相似地,假如 BSS 中一个基站要与远程的基站通信,则从源基站中发送 MAC 帧给 AP,并通过 DS 中的 AP 中继,直到送至目的基站为止。BSS 相当于一个蜂窝。DS 可以是一个交换机、一个有线网络或者是无线网络。

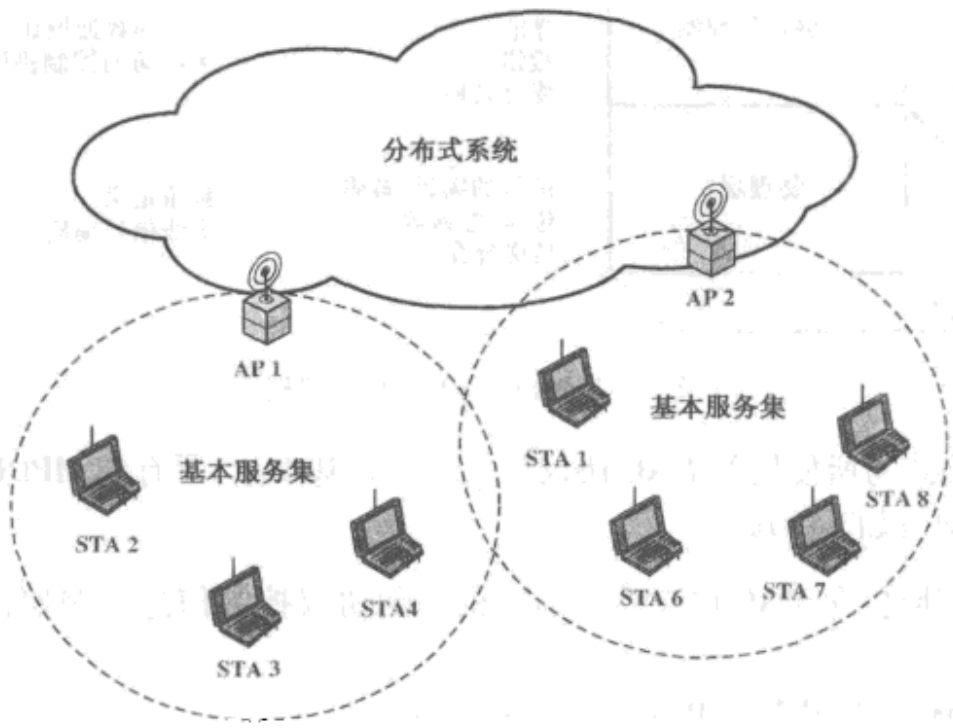


图 17.3 IEEE 802.11 扩展服务集

当 BSS 中所有的基站都是移动基站并且不使用 AP 而相互之间可以直接通信,则该 BSS 就称为**独立 BSS(IBSS)**。IBSS 的典型例子是 ad hoc 网络。在一个 IBSS 中,所有基站之间都是直接通信,不使用 AP。

如图 17.3 只是给出了一种最简单的情形。图中每一个基站都属于一个 BSS,即每一个基站都只和同一个 BSS 中的基站享用同一无线网络空间。事实上也存在两个 BSS 在地理上交迭的情形,此时一个基站可能会参与到多个 BSS 中。此外,基站和 BSS 之间的连接也是动态的。基站可能关闭,可能进入或退出某个范围。

扩展服务集(ESS)包含两个或两个以上通过 DS 互联的基本服务集。在逻辑链路控制层,扩展服务集可以表示为一个逻辑上独立的局域网。

17.1.4 IEEE 802.11 服务

IEEE 802.11 定义了 9 种无线局域网提供的服务,这些服务用来达成与有线局域网相等的功能。表 17.2 列出了这些服务并指出了两种分类方法。

(1) 服务可以分为是由基站提供的或是由 DS 提供的两种。基站服务在每一个 802.11 基站

中实现，包括 AP 基站。DS 的服务是在 BSS 之间提供的，这些服务可能是在一个 AP 中实现，也可能是在 DS 中因特定目的加入的设备中实现。

（2）其中三种服务是用来控制 IEEE 802.11 局域网的访问和机密性的。另外 6 个服务是用来支持 MSDU 在基站间转发的。假如一个 MSDU 太大而不能在一个 MPDU 中传输，则可以将其分割，然后在一系列 MPDU 中传输。

表 17.2　IEEE 802.11 服务

服　务	提 供 者	目　的
关联	分布式系统	MSDU 传输
认证	基站	LAN 的访问和安全
解除认证	基站	LAN 的访问和安全
关闭关联	分布式系统	MSDU 传输
分发	分布式系统	MSDU 传输
整合	分布式系统	MSDU 传输
MSDU 传输	基站	MSDU 传输
隐私	基站	LAN 的访问和安全
重新连接	分布式系统	MSDU 传输

根据 IEEE 802.11 文档，我们按序介绍这些服务以弄清 IEEE 802.11 ESS 网络的操作。作为基本的服务 MSDU 传输已经在之前提到了。与安全相关的服务参见 17.2 小节。

DS 中的消息分发

涉及 DS 中的消息分发的两种服务是分发和整合。分发服务是指当 MPDU 必须跨越从一个 BSS 中的基站到另一个 BSS 中的基站的 DS 时，用于基站间交互 MPDU 的最主要的服务方式。例如，假设有一个数据帧要从基站 2（STA 2）发送到基站 7（STA 7）（如图 17.3 所示）。数据帧从基站 2（STA 2）发送到在同一个 BSS 中的接入点（AP 1），然后 AP 1 将数据帧交给 DS，而 DS 的功能是将帧发送到与目标 BSS 中的 STA 7 相连的接入点（AP 2）上。AP 2 接收到数据帧并将其转发到 STA 7。消息是怎么在 DS 中传输的就不是 IEEE 802.11 标准管辖的范围了。

假如通信的两个基站在同一个 BSS 中，那么逻辑分发服务就只要通过该 BSS 中的 AP 就可以了。

整合服务使得数据能在 IEEE 802.11 局域网的基站与整合后的 IEEE 802.x 局域网中基站间传输。术语整合的（integrated）表示一个与 DS 物理连接的有线局域网，并且其基站通过整合服务可以与 IEEE 802.11 局域网在逻辑上连接。整合服务主要是进行数据交换中的地址翻译和媒体转换逻辑。

相互关联服务

MAC 层的主要功能是在 MAC 实体之间传输 MSDU，而该功能是由分发服务完成的。为了完成该功能，这需要相互关联服务提供 ESS 之内的基站信息。在分发服务能够向基站发送数据或从基站接收数据之前，基站必须被关联。在讲述关联概念之前，我们先看一下移动性的概念。标准定义了三种基于移动性的转换类型：

- **无转换**：一个基站要么是固定的，要么是移动的，但是这种移动仅仅是在一个孤立的 BSS 基站的直接通信范围内的移动。
- **BSS 转换**：定义了基站可以从一个 BSS 移动到同一 ESS 中的另一个 BSS。这种情况下，基站的数据传输要求地址功能以使其能识别基站的新位置。
- **ESS 转换**：定义了基站可以从一个 BSS 移动到不同 ESS 中的 BSS。这种情况下，要求基站是能够移动的。802.11 支持的上层连接并不能得到保证。事实上，服务的中断会时常发生。

为了在 DS 中发送消息,分发服务必须知道目的基站的位置。另外,为了消息能到达目的基站,DS 还需要知道消息应该先发到哪一个 AP 上。为了满足这些要求,一个基站必须与其当前所在 BSS 中的 AP 保持连接。关于这些要求的服务有三种:

- **关联**:在基站和 AP 之间建立初始关联。在无线局域网中,一个基站传输或接收数据帧之前,必须发布自己的标志和地址。为此,基站必须与同一 BSS 中的 AP 建立关联。然后 AP 可以将这些信息告知 ESS 中的其他 AP,以方便路由和传输地址帧。
- **重新关联**:使已建立的关联可以从一个 AP 向另一个 AP 转移,这样可以使移动基站从一个 BSS 向另一个 BSS 移动。
- **关闭关联**:从一个基站或者 AP 上发出公告表明当前的某个已存在的关联终止。一个基站必须在离开某个 ESS 或关闭之前公告该消息。当然,MAC 管理工具也能保护协议在基站未公告关闭关联消息就退出的情况下仍然正常运行。

17.2 IEEE 802.11i 无线局域网安全

在有线局域网中有两个特征与是无线局域网是不同的。

(1) 为了能在有线局域网中传输信息,基站必须与该局域网物理连接。但是在无线局域网中,任何基站只要处在局域网中其他设备传输无线电波可达的范围之内即可。因此在有线局域网中就顺带有某种程度的认证,因为它需要一些实际可见的行为来使基站与有线局域网相连。

(2) 相类似的是,为了接收一个从有线局域网中基站发出的消息,接收方基站也必须与该有线局域网相连。但是,在无线局域网中,任何在无线电波覆盖范围内的任何基站都可以接收。因此,有线局域网提供了一定程度的隐私性,限制数据的接收基站要与局域网相连。

无线局域网和有线局域网的差异性就要求无线局域网有足够的安全服务和机制。在 802.11 规范文档中有一些关于隐私性和认证性的特征描述,但是这些特征都非常薄弱。在隐私性方面,802.11 定义了 WEP(Wired Equivalent Privacy)算法。802.11 标准中隐私性部分有致命的缺陷。在 WEP 文档之后,802.11i 工作组开发了一系列有关无线局域网(WLAN)安全问题的性能。为了推进 WLAN 的强安全性的引入,Wi-Fi 联盟发布了 WPA(Wi-Fi Protected Access)并已经作为 Wi-Fi 标准。WPA 是一系列可以减少大多数 802.11 安全性问题的安全机制,并且也是基于当前的 802.11i 标准。802.11i 标准的最终形式是强安全网络(RSN, Robust Security Network)。Wi-Fi 联盟在 WPA2 项目下确认生产商是否符合完整的 802.11i 标准。

17.2.1 IEEE 802.11i 服务

802.11i 强安全网络安全性规范定义了如下服务:

- **认证性**:一种协议,用来定义用户和认证服务器(AS)之间的交互,以提供相互认证,并生成用于客户端与 AP 之间通过无线连接进行通信的短期密钥。
- **访问控制**[①]:该功能增强了认证功能的使用,为消息提供路由,并为密钥交换提供支持。它能与多种认证协议协同工作。

① 在此,我们将访问控制作为一个安全功能来讨论。如 17.1 节所述,它是一个与媒体访问控制(MAC)不同的功能。而在某些文献和标准中都使用访问控制这个术语。

- **带消息完整性的机密性**：MAC 层的数据（例如 一个 LLC PDU）与消息完整性码一起加密以保证数据不被改变。

图 17.4(a)给出了用于支持这些服务的安全协议，而图 17.4(b)列出了用于这些服务的密码学算法。

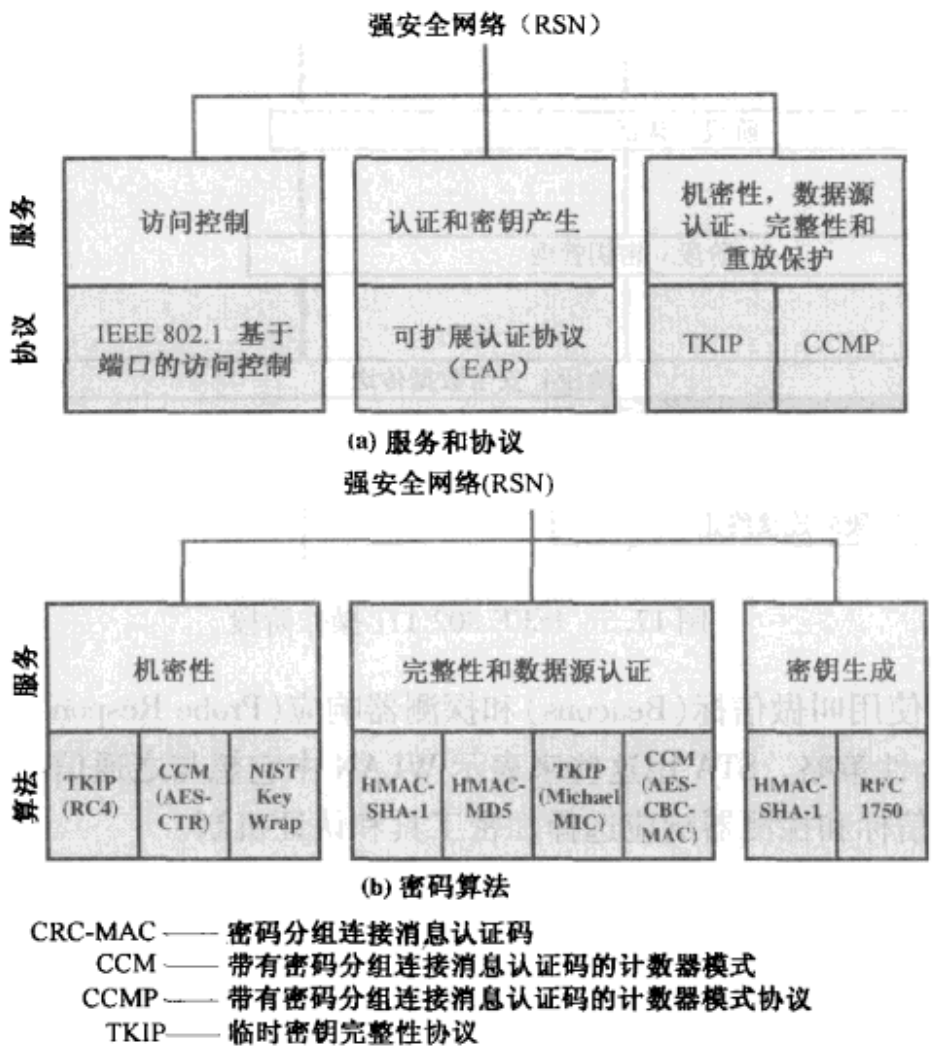


图 17.4　IEEE 802.11i 的元素

17.2.2　IEEE 802.11 i 操作阶段

IEEE 802.11i 强安全网络操作可以划分为 5 个相对独立的操作阶段。这些阶段的特性将依赖于配置和通信终端如下所述（参见图 17.3）：

（1）在同一个 BSS 中的两个无线基站通过 BSS 中的 AP 进行通信。
（2）在同一个 ad hoc IBSS 中的两个无线基站之间直接相互通信。
（3）不同 BSS 中的两个无线基站通过各自的 AP 利用分布式系统进行通信。
（4）一个无线基站与一个有线网络的终端基站通过 AP 和分布式系统进行通信。

IEEE 802.11i 的安全性仅关注基站（STA）和 AP 之间的安全通信。在上述列出的第 1 种情形下，安全通信通过每一个 STA 与 AP 之间建立安全通信来保证。第 2 种情形类似，AP 在功能上与 STA 在同一个域中。第 3 种情形，因为不在同一个 BSS 中，IEEE802.11 层的分布式系统并没有提供安全性。如果需要的话，端到端的安全性必须由更高的层来提供。类似的是，第 4 种情形也没有提供 STA 与 AP 之间的安全性。

考虑到这些因素，图 17.5 描述了一个强安全网络的 5 个操作阶段并将它们映射到对应的网络构成中。其中一个新的构件是认证服务器（AS）。矩形表示一系列 MPDU 的交换。这 5 个阶段如下定义：

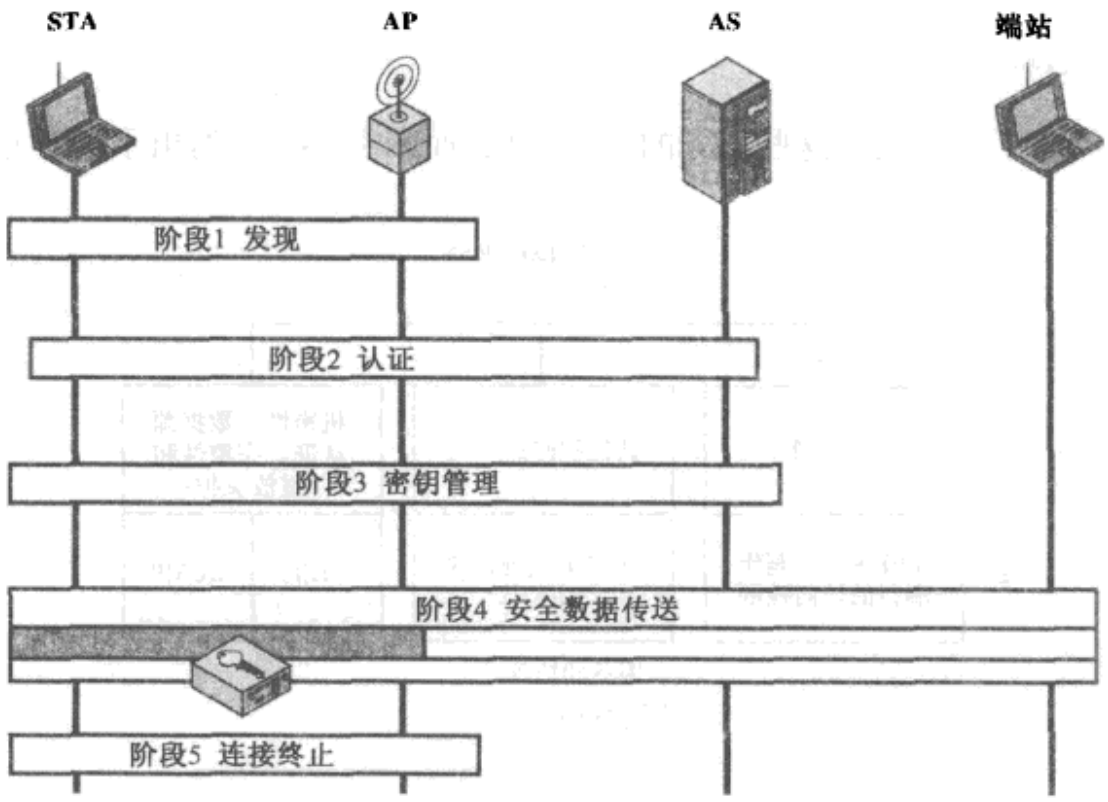


图 17.5 IEEE 802 11i 操作阶段

- **发现**:一个 AP 使用叫做信标(Beacons)和探测器响应(Probe Responses)的对象来描述 IEEE 802.11i 的安全性策略。STA 用这些来表示 WLAN 中它想与之通信的 AP。STA 和 AP 建立连接时,AP 用信标和探测器响应选择加密工具和认证机制。
- **认证**:在该阶段,STA 和 AS 相互向对方证明自己的标志。认证交互成功之前,AP 会阻塞 STA 和 AS 之间非认证的流量。除了发送 STA 和 AS 之间的通信,AP 并不参与认证交互过程。
- **密钥管理**:AP 和 STA 执行一系列操作,以生成加密密钥并置于 AP 和 STA。而帧交换只在 AP 和 STA 之间进行。
- **安全数据传输**:帧通过 AP 在 STA 和端站(end station)之间交换。如图 17.5 中阴影和加密模块图标所示,安全的数据传输只在 STA 和 AP 之间进行;并未提供端到端的安全性。
- **连接终止**:AP 和 STA 交换帧。在这个阶段,安全连接被拆除,连接回复到初始状态。

17.2.3 发现阶段

我们现在来看看强安全网络操作阶段的更多细节,最开始是发现阶段,在图 17.6 的上半部分有说明。这个阶段的目的是让 STA 和 AP 相互辨认,协商安全功能配置,并为将来使用这些安全功能建立关联。

安全功能

在此阶段,STA 和 AP 决定以下方面的具体技术:

- 用于保护单播流量(仅在该 STA 和 AP 之间)的机密性和 MPDU 完整性协议
- 认证方法
- 加密密钥管理方法

由于一个多播组中所有的 STA 必须使用相同的协议和加密,所以用于保护多播/广播流量的机密性和完整性协议由 AP 掌控。具体的协议和选定的密钥长度(如果是可变的)被称为密码套件(cipher suite)。机密性和完整性密码套件的选项有:

- WEP,有一个 40 位或 104 位的密钥,允许与过去的 IEEE 802.11 产品向后兼容
- TKIP
- CCMP
- 供应商的具体方法

另一个可协商套件是认证和密钥管理(AKM)套件,套件定义了:(1)AP 和 STA 进行相互认证的办法;(2)生成根密钥的办法,其他密钥可以从根密钥派成。可能的 AKM 套件是:

- IEEE 802.1X
- 预共享密钥(没有进行明确的认证且只有当 STA 和 AP 共享一个独特的秘有密钥时才会施行相互认证)
- 供应商的具体方法

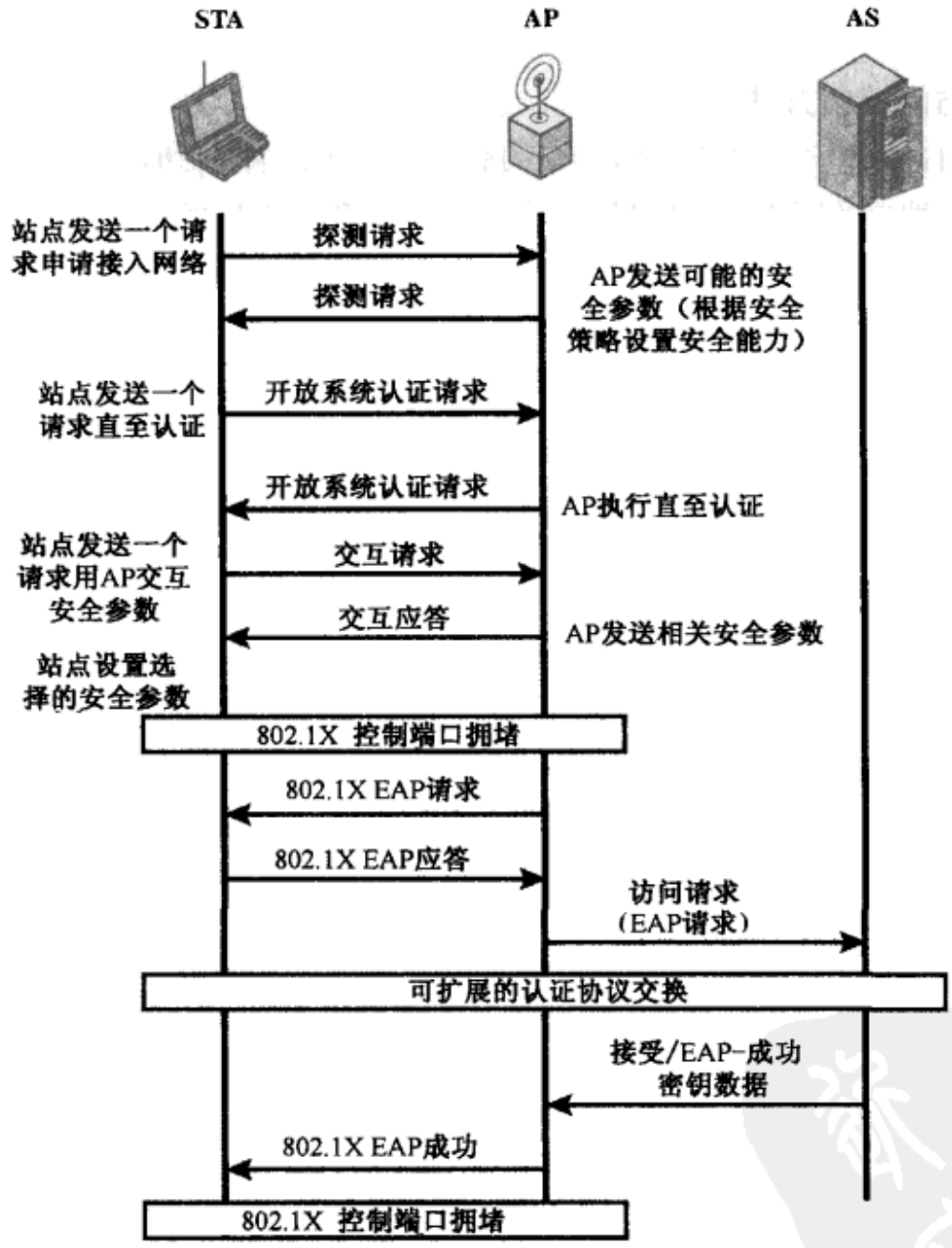


图 17.6　IEEE 802.11i 操作阶段:功能发现,认证和交互

MPDU 交换发现阶段由三个交换组成:

- **网络和安全功能发现**:在该交换中,STA 发现与之通信的网络的存在。AP 要么定时广播它的安全功能(图中没有显示),广播按照强安全网络信息元素(RSN IE)的指示,通过信标帧在一个特定的频道进行;要么通过探测响应帧对一个基站的探测请求做出回应。一个无线基站可以通过被动的监测信标帧,或者主动探测每个频道来发现可用的接入点以及相应的安全功能。

- **开放系统认证**:该帧序列的目的并不是提供安全性,而仅仅是为了与 IEEE 802.11 状态的机器保持向后兼容性。基本上,这两个设备(STA 和 AP)仅仅是交换了身份标志。
- **协商**:这一步的目的是协商一套将要使用的安全功能配置。STA 向 AP 发送一个协商请求帧。在这个帧里面,STA 从 AP 告知的能力中指定一套相匹配的能力(一个认证和密钥管理套件,一对密码套件和一组密钥套件)。如果 AP 和 STA 之间没有匹配功能,AP 就拒绝协商请求。STA 也将其阻塞,以防止关联到一个恶意 AP 或者有人在它的频道非法插入帧。如图 17.6 所示,IEEE 802.1X 控制端口被阻塞,没有用户流量通过 AP。阻塞端口的概念随后解释。

17.2.4 认证阶段

正如前面提到的,认证阶段使得一个 STA 与分布式系统(DS)中的一个认证服务器(AS)能够相互认证。设计认证是为了只允许授权基站使用网络,并且向 STA 保证它连接的是一个合法的网络。

IEEE 802.1X 访问控制方法

IEEE 802.11i 利用了被设计用来为局域网提供访问控制功能的另一个标准,这个标准是 802.1X,它是基于端口的网络访问控制。所使用的协议称之为扩展认证协议(EAP),在 IEEE 802.1X 标准中定义。IEEE 802.1X 使用了术语:请求者,认证者和认证服务器(AS)。在 802.11 无线局域网的上下文中,前两个术语对应无线基站和 AP。AS 通常是网络中有线连接的独立设备(例如,通过 DS 可访问的),但是也可以直接处于认证者一边。

在一个请求者被 AS 使用一个认证协议认证前,认证者只在请求者和 AS 之间传递控制和认证消息;802.1X 控制频道是非阻塞的,但是 802.11 数据频道是阻塞的并受预定义的请求者对网络的访问控制限制,一旦请求者被认证且授予其密钥,认证者可以发送来自请求者的数据,处于这些情况下,数据频道是非阻塞的。

如图 17.7 所示,802.1X 使用控制和非控制的概念。端口在认证者定义中是逻辑实体且指向物理网络连接。对于一个无线局域网,认证者(AP)可能只有两个物理端口:一个连接到 DS,另一个和它的 BSS 进行无线通信。每个逻辑端口都被映射到两个物理端口的其中之一。不管请求者的认证状态如何,一个非控制的端口允许请求者和其他 AS 之间的 PDU 交换。只有当请求者的当前状态授权这样一个交换时,一个控制端口才允许请求者和局域网内其他系统之间进行 PDU 交换。

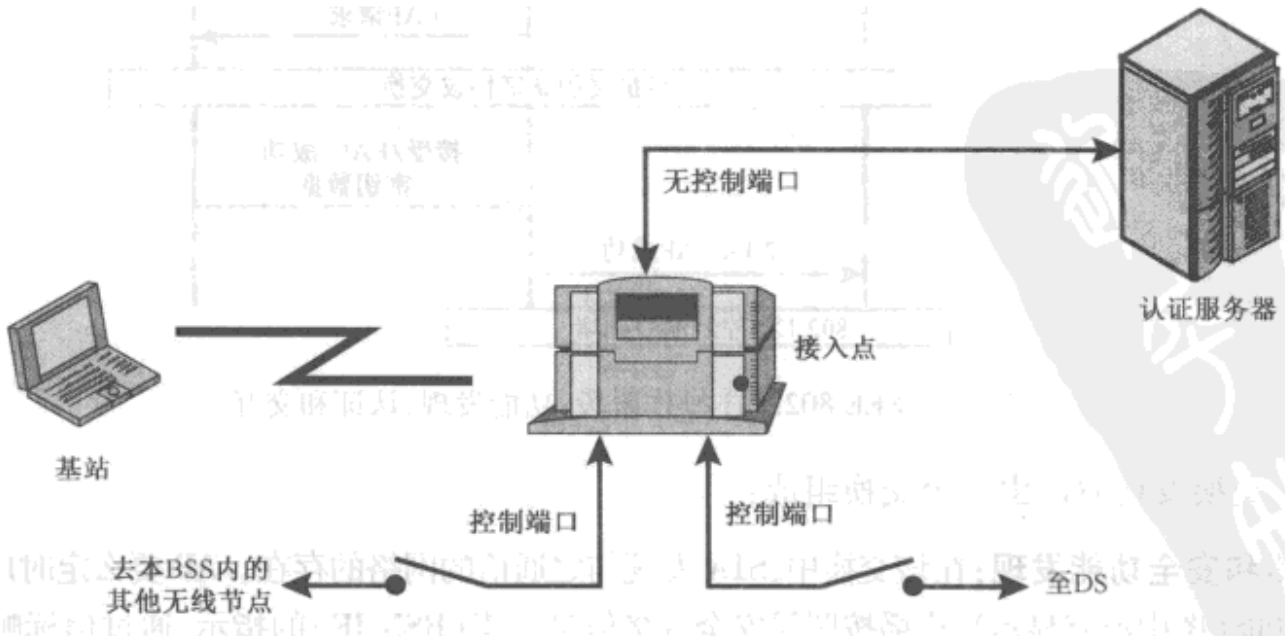


图 17.7 802.1X 访问控制

带有上层认证协议的 802.1X 框架与一个包含许多无线基站和一个 AP 的 BSS 架构非常适应。尽管如此,对于一个 IBSS 是没有 AP 的。对于一个 IBSS,802.11i 提供了一个更加复杂的解决方法,从本质上讲,涉及 IBSS 上基站间的成对认证。

MPDU 交换

图 17.6 的下半部分显示,对于认证阶段 IEEE 802.11 控制了 MPDU 交换。我们可以认为认证阶段由以下三个阶段组成。

- **连接到 AS:**STA 向它的 AP(与该 STA 相关联)发送一个请求以连接到 AS。AP 识别这个请求并给 AS 发送一个访问请求。
- **EAP 交换:**这个交换让 STA 和 AS 相互授权。正如随后要解释的,许多可选的交换是可能的。
- **安全密钥分发:**一旦认证建立,AS 产生一个主会话密钥(MSK)并发送给 STA,此密钥也被称为认证(Authentication),授权(Authorization)和审计(Accounting)(合称 AAA)密钥。如随后解释的,STA 和它的 AP 进行安全通信所需的所有加密密钥都从这个 MSK 产生。IEEE 802.11i 没有规定 MSK 安全分发的方法而是说它依赖于 EAP。不管用的是什么方法,都涉及从 AS 发出,经过 AP,到达 AS 的一个包含加密过的 MSK 的 MPDU。

EAP 交换

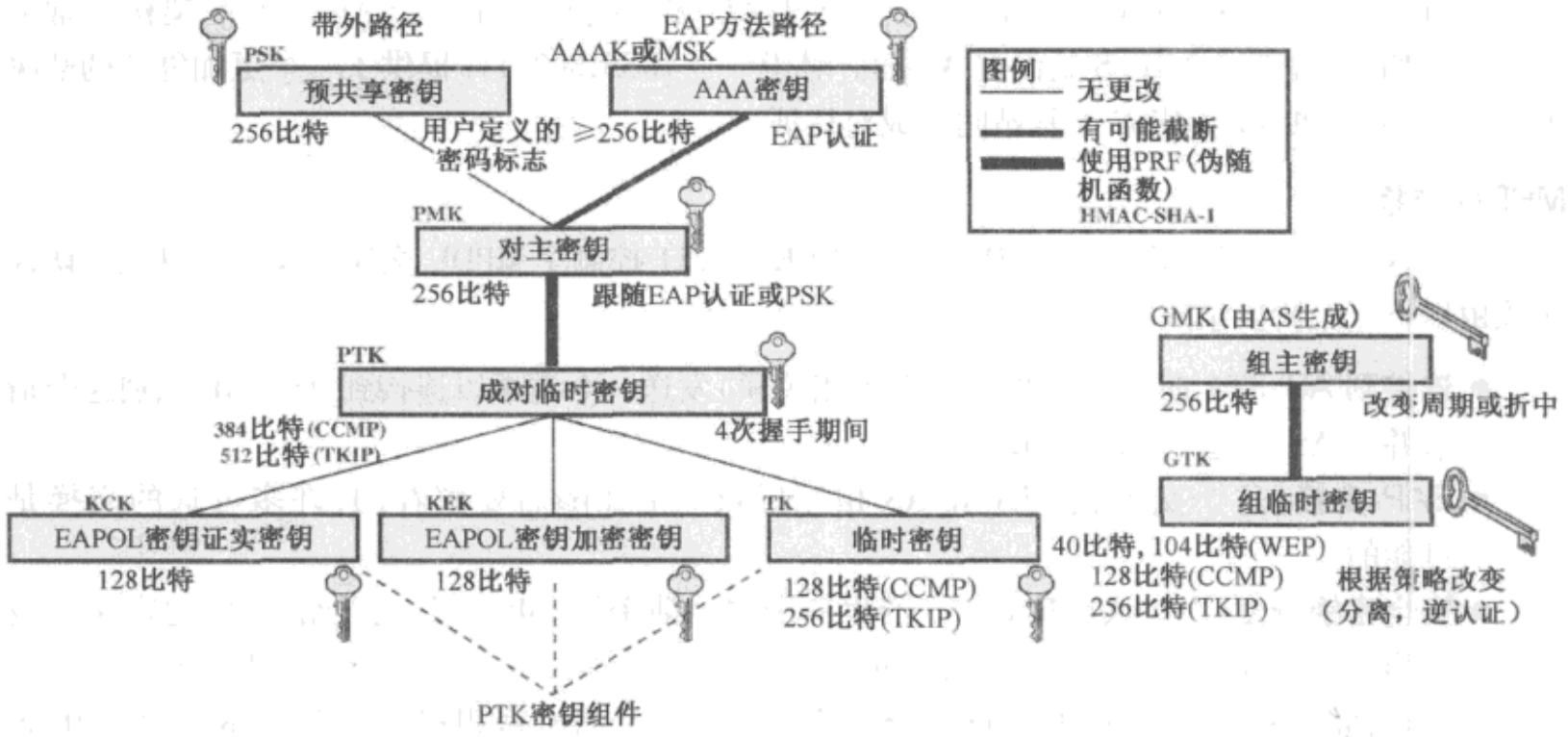
如上所述,在认证期间有许多可能的 EAP 交换可以使用。通常地,STA 和 AP 之间流通的消息采用局域网上的 EAP(EAPOL)协议,而 AP 和 AS 之间流通的消息则使用用户服务中的远程认证拨号(RADIUS)协议,而且对于 STA 到 AP 和 AP 到 AS 都还有其他可选协议。[FRAN07]对使用 EAPOL 和 RADIUS 的认证交换提供了以下总结。

(1) EAP 交换从 AP 发送一个 EAP-Request/Identity 帧给 STA 开始。
(2) STA 回应一个 EAP 应答/身份帧,AP 通过非控制端口接收。接着使用 RADIUS 封装数据包作为 RADIUS 访问请求数据包,通过 EAP 传送给 RADIUS 服务器。
(3) AAA 服务器回应一个 RADIUS-Access-Challenge 包,数据包作为 EAP - 请求被送往 STA。该请求有适当的认证类型且包含有关挑战信息。
(4) STA 制定一个 EAP-Response 消息并发送给 AS。该应答被 AP 转换成 RADIUS-Access-Request,以将挑战的应答作为数据域。步骤(3)和步骤(4)根据使用的 EAP 方法可能重复多次。对于 TLS 隧道方法,验证通常要经过 10 ~ 20 轮的尝试。
(5) AAA 服务器利用 Radius-Access-Accept 包授权访问。AP 发送一个 EAP-Success 帧(为了验证真伪,一些协议在 TLS 隧道内部要求对 EAP 成功进行确认)。控制端口被授权,用户可以开始访问网络。

从图 17.6 可以注意到 AP 控制端口对于一般用户流量仍然是阻塞的。虽然验证成功,端口保持阻塞直到临时密钥安装到 STA 和 AP 上,密钥安装发生于 4 次(4-way)握手期间。

17.2.5　密钥管理阶段

在密钥管理阶段期间,各种加密密钥被生成并分发给各个 STA。有两种类型的密钥:用于 STA 和 AP 间通信的对密钥及用于多播通信的组密钥。基于[FRAN07],图 17.8 显示了两个密钥层次结构,同时表 17.3 定义了独立密钥。



(a) 对密钥层次结构　　(b) 组密钥层次结构

图 17.8　IEEE 802.11i 密钥层次结构

表 17.3　数据保密性和完整性协议的 IEEE 802.11i 密钥

缩　写	名　称	描述/目的	大小(位)	类　型
AAA 密钥	认证,审计和授权密钥	用来派生 PMK。和 IEEE 802.1X 认证及密钥管理方法一起使用。和 MMSK 一样	≥256	密钥生成密钥,根密钥
PSK	预共享密钥	在预共享密钥环境成为 PMK	256	密钥生成密钥,根密钥
PMK	对主密钥	和其他输入一起使用派生 PTK	256	密钥生成密钥
GMK	组主密钥	和其他输入一起使用派生 GTK	128	密钥生成密钥
PTK	成对临时密钥	由 PMK 派生,包括 EAPOL-KCK,EAPOL-KEK 和 TK 及(对于 TKIP)MIC 密钥	512(TKIP) 384(CCMP)	组合密钥
TK	临时密钥	和 TKIP 或 CCMP 一起使用,为单播用户流量提供保密性和完整性保护	256(TKIP) 128(CCMP)	流量密钥
GTK	组临时密钥	从 GMK 派生,用来为多播/广播用户流量提供保密性和完整性保护	256(TKIP) 128(CCMP) 40,104(WEP)	流量密钥
MIC 密钥	消息完整性码密钥	TKIP 的 Michael 算法使用该密钥提供消息的完整性保护	64	消息完整性密钥
EAPOL-KCK	EAPOL 密钥确认密钥	为 4 次握手期间的密钥材料分发提供完整性保护	128	消息完整性密钥
EAPOL-KEK	EAPOL 密钥加密密钥	用来确保 GTK 和其他 4 次握手的密钥材料的保密性	128	流量密钥/密钥加密密钥
WEP 密钥	有线等价保密密钥	WEP 使用	40,104	流量密钥

对密钥

对密钥用于一对设备之间的通信,通常是 STA 和 AP。这些密钥形成一个层次结构,开始于一个动态派生其他密钥的主密钥,并且在有限时间内使用。

层次结构的最上层有两种可能。预共享密钥(PSK)是一个被 AP 和 STA 共享的秘密信息,以及 IEEE 802.11i 之外的一些方式安装。另一个选择是主会话密钥(MSK),也被称为 AAAK,如上所述,它在认证期间使用 IEEE 802.1X 协议生成。密钥生成的实际方法取决于所使用的认证协议

细节。不管是哪一种情况(PSK 或 MSK),都会有一个被 AP 及与 AP 通信的每一个 STA 共享的独一无二的密钥。由这个主密钥派生的所有其他密钥,在一个 AP 和一个 STA 之间都是独一无二的。因此,如图 17.8(a)所示的层次结构描绘的那样,每一个 STA 在任何时候都有一套密钥,同时 AP 相对它的每个 STA 都有一套这样的密钥。

对主密钥(PMK)派生自主密钥。如果使用 PSK,那 PSK 被作为 PMK;如果使用 MSK,那 PMK 通过截断(truncation)(如果有必要)MSK 派生。在认证阶段的最后,根据 802.1X EAP 成功信息(如图 17.6 所示)所记,AP 和 STA 都有一份共享 PMK。

PMK 被用来生成成对临时密钥(PTK,Pairwise Transient Key),PTK 实际由三个密钥组成,在 STA 和 AP 相互认证后用于两者之间的通信。为了派生 PTK,需要对 PMK 应用 HMAC-SHA-1 函数,需要 STA 和 AP 的 MAC 地址,还有必要时生成的随机数。在 PTK 生成中使用 STA 和 AP 地址以对抗会话劫持和会话模拟;使用随机数提供了额外的随机密钥材料。

PTK 的三部分密钥如下所示:

- **局域网上的 EAP(EAPOL)密钥确认密钥(EAPOL-KCK)**:支持 RSN 操作设置期间 STA 到 AP 控制帧的完整性和数据来源真实性。同时执行访问控制功能:拥有 PMK 的证明。拥有 PMK 的实体可以被授权使用链接。
- **EAPOL 密钥加密密钥(EAPOL-KEK)**:在一些强安全网络协商步骤期间保护密钥和其他数据的机密性。
- **临时密钥(TK)**:为用户流量提供实际保护。

组密钥

组密钥用于多播通信,一个 STA 可以发送 MPDU 给多个 STA。在组密钥层次结构的最上层是组主密钥(GMK)。GMK 是生成密钥的密钥,和其他输入一起使用派生组临时密钥(GTK)。不像 PTK 使用 AP 和 STA 的素材来生成,GTK 由 AP 生成并发送给其相关的 STA。究竟 GTK 是如何生成的还没有定义。不过,IEEE 802.11i 要求 GTK 的值和随机值在计算上是可区分的。利用已经建立的对密钥,GTK 被安全分发。每次有设备离开网络 GTK 都要改变。

对密钥分发

图 17.9 的上部显示了用于对密钥分发的 MPDU 交换。此交换被称为 4 次握手。STA 和 SP 使用这个握手来确认 PMK 的存在性,验证加密套件的选择,为接下来的数据会话派生一个新的 PTK。交换的 4 个部分如下所示。

- **AP→STA**:消息包含 AP 的 MAC 地址和一个随机数(Anonce)。
- **STA→AP**:STA 生成自己的随机数(Snonce),使用随机数和 MAC 地址,加上 PMK,来生成一个 PTK。STA 接着发送一个包含 MAC 地址和 Snonce 的消息,使 AP 能够生成一个同样的 PTK。此消息包括使用 HMAC-MD5 或 HMAC-SHA-1-128 的消息完整性码(MIC)[①]。和 MIC 一起使用的密钥是 KCK。
- **AP→STA**:AP 现在能够生成 PTK。AP 接着发送一个消息到 STA,包含与第一个消息同样的信息,但是多了 MIC。
- **STA→AP**:这仅仅是一个确认消息,也受 MIC 保护。

---

① 尽管在密码学中通常用 MAC 来表示消息验证码,但是在 802.11i 中却使用 MIC,因为 MAC 在网络中有另外的标准意义,即媒体访问控制。

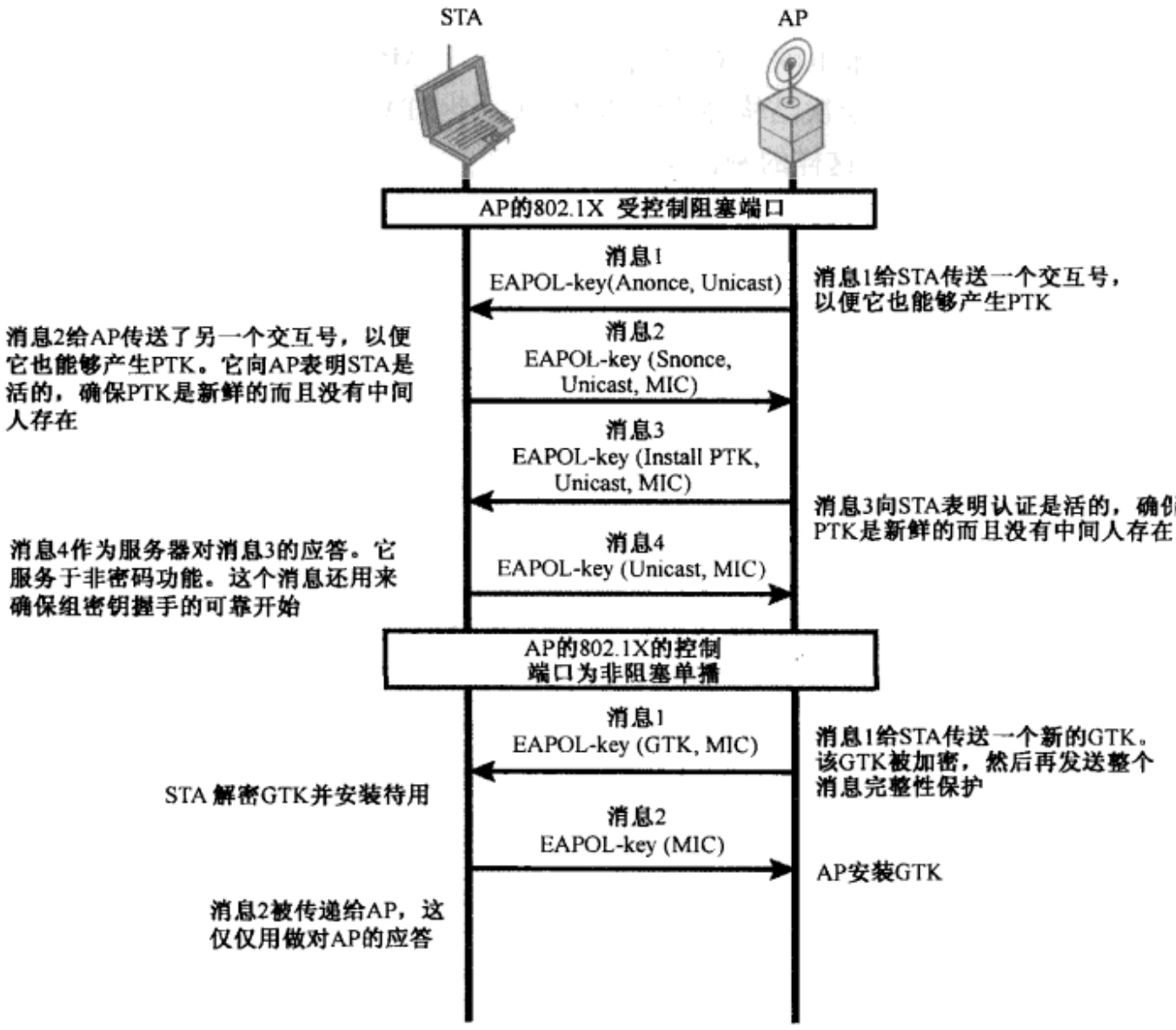


图 17.9 IEEE 802.11i 操作阶段:4 次握手和组密钥握手

组密钥分发

对于组密钥分发,AP 生成一个 GTK 并分发给多播组中的每个 STA。和每个 STA 交换的两个消息如下:

- **AP→STA**:这个消息包括 GTK,使用 RC4 或 AES 加密。用于加密的密钥是 KEK。附加有一个 MIC 值。
- **STA→AP**:STA 确认对 GTK 的接收。消息包括一个 MIC 值。

17.2.6 安全数据传输阶段

IEEE 802.11i 定义了两个方案以保护 802.11MPDU 中的数据传输:临时密钥完整性协议(TKIP)和计数器模式 CBC-MAC(加密-阻塞-链接消息验证码)协议(CCMP)。

TKIP

TKIP 的设计只要求对实施称为有线对等保密(WEP)的旧的无线 LAN 安全方法的设备进行软件改变。TKIP 提供两种服务:

- **消息完整性**:TKIP 在 802.11 MAC 帧的数据域后面加了一个消息完整性码(MIC)。MIC 由叫做 Michael 的算法生成,算法使用源和目的 MAC 地址值和数据域再加上密钥作为输入,计算出一个 64 位值。
- **数据保密**:通过加密 MPDU 加上使用 RC4 的 MIC 值提供数据保密。

256 位 TK(参见图 17.8)使用如下。两个 64 位密钥通过使用 Michael 消息摘要算法产生一个消息完整性码。一个密钥用来保护 STA 到 AP 的消息,另一个密钥用来保护 AP 到 STA 的消息。剩下的 128 位截断生成 RC4 密钥用来加密传输数据。

对于额外保护,一个单调递增 TKIP 序列计数器(TSC)会被分配给每个帧。TSC 服务于两个目的。第一,TSC 包含于每个 MPDU,受 MIC 保护,以防止重放攻击。第二,TSC 和会话 TK 结合产生一个随每个传输 MPDU 改变的动态加密密钥,这样就使密码分析更加困难。

CCMP

CCMP 适用于更新的 IEEE 802.11 设备,设备配置有支持这一方案的硬件。正如 TKIP,CCMP 提供两种服务:

- **消息完整性**:CCMP 使用加密-阻塞-链接消息验证码(CBC-MAC),在第 12 章有所描述。
- **数据保密**:CCMP 使用 CTR 阻塞加密模块操作,用 AES 加密。CTR 在第 6 章有所描述。

同样的 128 位 AES 密钥用于完整性和保密性。方案使用一个 48 位的数据包序列号来构建一个随机数以防止重放攻击。

17.2.7　IEEE 802.11i 伪随机函数

在 IEEE 802.11i 方案的许多地方都会使用一个伪随机函数(PRF)。例如,用它来生成一个随机数来扩展对密钥,以及生成 GTK。最好的安全实践指出对于这些不同的目的要使用不同的伪随机数流。尽管如此,为了执行效率,我们会依靠单一的伪随机数生成函数。

PRF 是建立在使用 HMAC-SHA-1 来生成一个伪随机流之上。回调那个 HMAC-SHA-1 需要一个消息(数据阻塞)和一个长度至少 160 位密钥,生成一个 160 位的 Hash 值。SHA-1 有一个特性,输入中哪怕一位的改变产生的新 Hash 值跟先前的 Hash 值也没有明显的联系。这个特性是伪随机数生成的基础。

IEEE 802.11i 中 PRF 需要 4 个输入参数,产生要求位数的随机位。函数形式 PRF(*K*,*A*,*B*,Len),这里

K 为一个秘密密钥

A 为一个对应特定应用的文本字符串(例如,随机数生成或对密钥扩展)

B 为对应每个情况的一些数据

Len 为要求随机位的位数

例如,对于 CCMP 的成对临时密钥:

```
PTK = PRF(PMK, "Pairwise key expansion", min(AP-
      Addr, STA-Addr) || max(AP-Addr, STA-Addr) || min
      (Anonce, Snonce) || max(Anonce, Snonce), 384)
```

所以,在这个例子中,参数就是

K 为 PMK

A 为文本字符串"Pairwise key expansion"

B 为两个 MAC 地址和两个随机数串联的字节序列

Len 为 384 位

相类似地,一个随机数这样产生

```
Nonce = PRF(Random Number, "Init Counter", MAC || Time, 256)
```

这里时间是随机数产生者所知的网络时间的度量。组临时密钥这样产生

GTK = PRF(GMK, "Group key expansion", MAC ‖ Gnonce, 256)

图 17.10 说明了函数 PRF(K,A,B,Len)。参数 K 作为密钥输入 HMAC。消息由 4 个项目串联组成:参数 A;一个 0 字节;参数 B 和一个计数器 i,计数器被初始化为 0。HMAC 算法执行一次,产生一个 160 位的 Hash 值。如果要求产生更多位,HMAC 使用相同的输入再运行一次,除了 i 每次都会增加直到产生需要数量的位。我们可以把逻辑表达为

```
PRF(K, A, B, Len)
  R ← null string
  for i ← 0 to ((Len + 159)/160 - 1) do
  R ← R ‖ HMAC-SHA-1(K, A ‖0 ‖B ‖i)
  Return Truncate-to-Len(R, Len)
```

17.3 无线应用通信协议概述

无线应用通信协议(WAP)是由 WAP 论坛开发的统一开放标准,以使无线手机用户和其他诸如 PDA 等无线设备能够访问包括互联网在内的技术与信息服务。WAP 开发之初就设计成与所有的无线网络技术(如 GSM、CDMA 和 TDMA)兼容。WAP 尽量以现有的互联网标准作为基础,例如 IP、XML、HTML 和 HTTP。同时它也包含安全特性。在写本书的时候,WAP 规范的最新版本是 2.0 版。

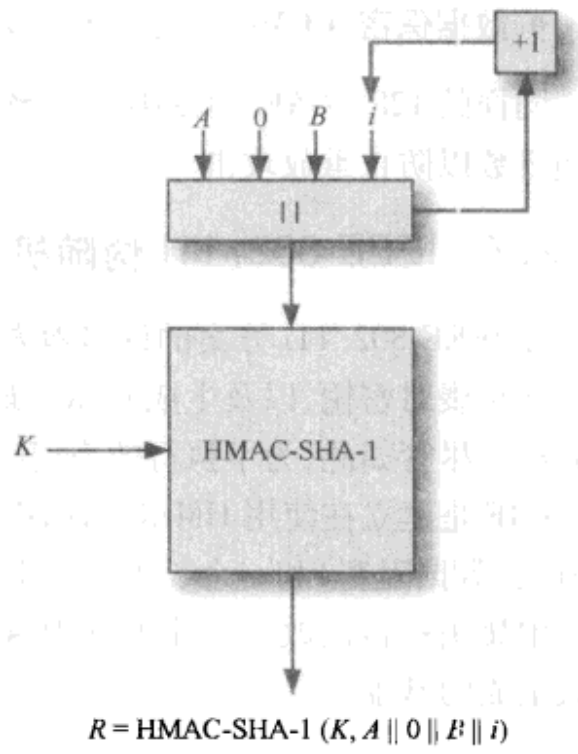


图 17.10 IEEE 802.11i 伪随机功能

对设备和与之相连的网络的限制很大程度地影响了手机和其他移动终端用户的数据服务能力。这些设备通常在处理器、内存和电池寿命方面都有限制。用户界面也很有限,因为显示器会很小。与有线网络相比,无线网络通常有带宽相对较低,延迟较长并且网络可用性和稳定性也较不确定等特性。并且,无线网络由于设备和网络的不同,其上述性能也有较大差别。最后,与其他信息系统用户相比,无线移动用户有各种不同的使用需求。例如,移动终端应该非常易用,比工作站和 PC 好用得多。WPA 就是设计来解决如上问题的。WAP 的规范包括:

- 基于 WWW 的程序设计模型
- 无线网络的标记语言,继承自 XML
- 适用于移动无线终端的小型浏览器的规范
- 轻量的通信协议栈
- 无线电话应用程序(WTA)的框架

17.3.1 操作概述

WAP 的程序设计模型基于三个元素:客户端、网关和源服务器(如图 17.11 所示)。HTTP 是用在网关和源服务器之间转换内容的。网关的功能相当于无线域内服务器的代理,它可以将手持、移动和无线终端的有限能力透明化。例如,网关提供 DNS 服务,WAP 协议栈和 WWW 栈(HTTP 和 TCP/IP)之间的转换,将 Web 上的信息编码成更合理的形式以降低无线通信量,还可以解码无线网中的数据使之成为标准的用于 Web 通信的数据。同时,网关还缓存了经常被访问的信息。

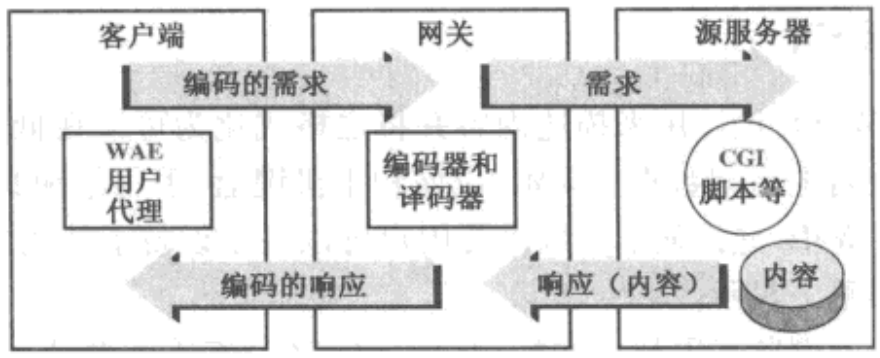


图 17.11　WAP 编程模型

图 17.12 描述了 WAP 环境中的关键构成。使用 WAP，移动用户能够浏览普通服务器上的 Web 内容。Web 服务器提供用 HTML 格式编码的网页内容并使用标准的 Web 协议栈（HTTP/TCP/IP）传输信息。HTML 内容必须经过 HTML 过滤器，过滤器可能在 WAP 代理上也可能在一个单独的物理模块上，它将 HTML 内容转换成 WML 内容。假如过滤器与代理相分离，则用 HTTP/TCP/IP 将 WML 发送至代理上。代理就将 WML 转换成更紧凑的二进制 WML 并使用 WAP 协议栈通过无线网络转发到移动用户。

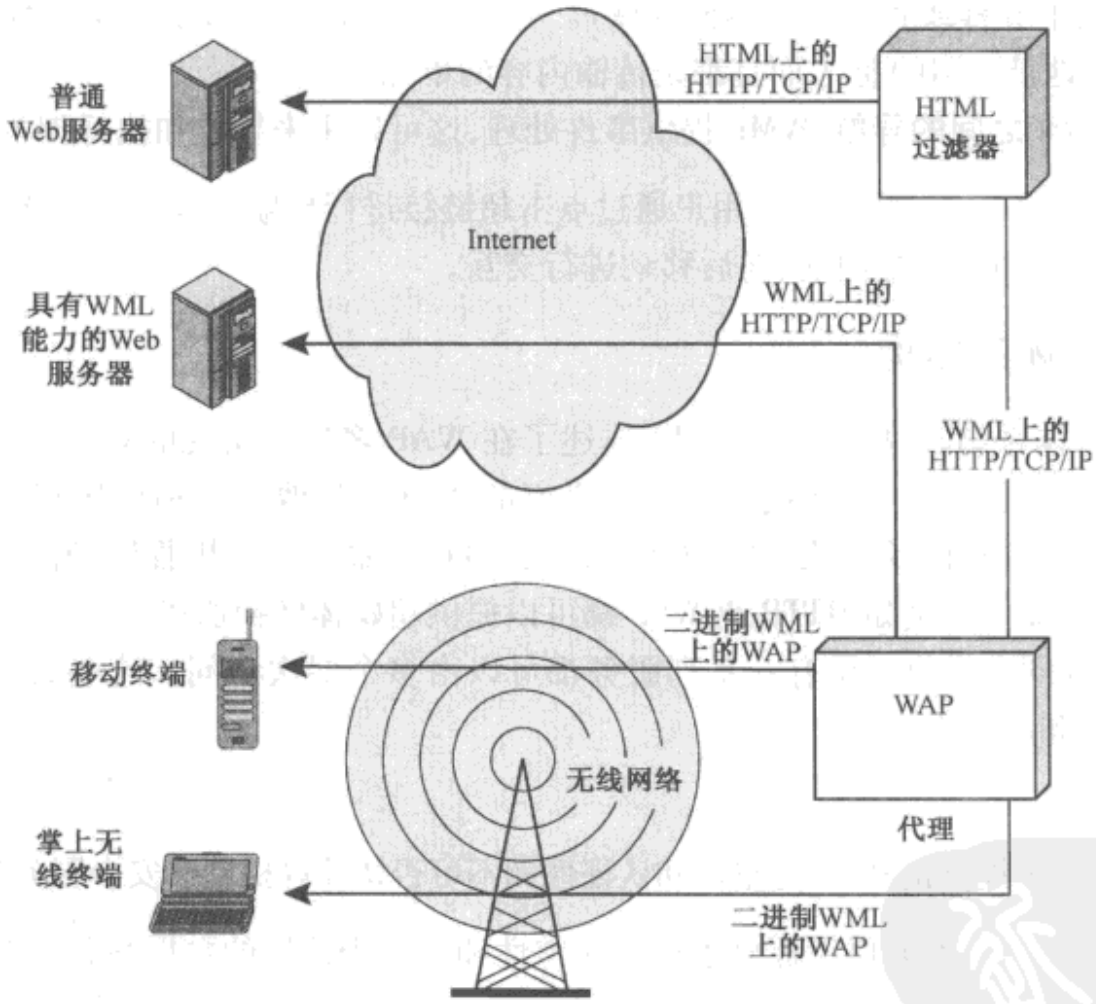


图 17.12　WAP 基础设施

假如 Web 服务器可以直接生成 WML 内容，直接使用 HTTP/TCP/IP 将 WML 发送至代理上，然后由代理将其转换为二进制 WML 并使用 WAP 协议将其转发到移动节点。

WAP 体系结构是用来解决无线网络访问 Web 的两个基本限制的：移动节点的局限性（屏幕尺寸小和有限的输入能力）和无线数据网络的低数据率。即使引入了提供宽带数据率的 3G 无线网络，手持式移动节点输入能力和显示能力仍然有限。于是，未来将需要更高性能的 WAP 或者一个类似的功能。

17.3.2 无线标记语言

无线标记语言(WML)是设计用来描述内容并将之格式化为可以在低带宽、屏幕尺寸有限和用户输入能力有限的设备上呈现数据。WML 应该和手机键盘、手写笔和其他在移动无线通信中常用的输入设备兼容。WML 允许显示缩放以便用户可以在小设备上使用"两行"屏幕显示,也可以在智能手机的较大屏幕上显示。

对于普通的 PC,Web 浏览器用超文本标记语言(HTML)编码的 Web 页面格式展现内容。为了将 HTML 编码的 Web 页面转换成适用于无线设备的 WML 格式,很多信息(特别是图像和动画)应该被去除。WML 主要呈现基于文本的信息以抓住 Web 页面的本质,这样的组织形式可以方便移动设备的用户访问。

WML 的重要特性包括:

- **支持文本和图像**:提供文本和有限的图像能力的格式化和布局命令。
- **deck/card 型的组织形式**:WML 文件划分成更小、更便于用户交互并称之为 card 的单元。用户通过将 card 向前、向后移动来导航。一个 card 规定了一个或多个交互单元(一个菜单、一个文本屏幕或者一个文本输入域)。一个 WML deck 类似于一个 HTML 页面,它们都是用 Web 地址(URL)标志而且都是传输内容的单元。
- **card 和 deck 之间的导航**:WML 提供事件处理,这可以用来导航和执行脚本。

在基于 HTML 的 Web 浏览器中,用户通过点击超链接进行导航。在一个有 WML 能力的移动设备上,用户用 card 通过将之向前、向后移动进行交互。

17.3.3 WAP 体系结构

来自于 WAP 体系结构文档的图 17.13 描述了在 WAP 客户端实现的整个协议栈结构。大体上,这是一个 5 层模型。每一层通过一系列定义好的接口为其他服务和应用程序提供一系列功能。结构中的每一层都可以被该层之上的任意一层和其他服务与应用程序访问。栈中的许多服务可能由多个协议提供。例如 HTTP 或 WSP 都可以提供超媒体转换服务。

所有 5 层的共同点是每一层有一系列服务都可以由多个层次访问。这些共同的服务可以分为两类:安全服务和服务发现。

安全服务

WAP 规范包含了提供机密性、完整性和认证性和不可否认性等机制。安全服务包含以下内容:

- **密码库**:这个应用框架层库提供了用于完整性和不可否认性的数据签名服务。
- **认证**:WAP 提供了各种用于客户端和服务器之间的认证机制。在会话服务层,HTTP 客户端认证(RFC 2617)可以用来为客户端向代理和应用服务器提供认证。在传输服务层,WTLS 和 TLS 握手可以用来认证客户端和服务器。
- **身份认证**:WAP 的身份认证模块(WIM)提供了存储和处理用户身份和认证所需信息的功能。
- **PKI**:一系列使用和管理公钥密码和证书的服务。
- **安全传输**:传输服务层协议用来保证基于数据报和基于连接传输的安全性。WTLS 用来保证基于数据报传输的安全,TLS 用来保证基于连接传输(如 TCP)的安全。
- **安全发送者**:一些发送网络提供发送层安全。例如,IP 网络(特别是在 IPv6 的环境中)使用 IPsec 来提供发送层安全。

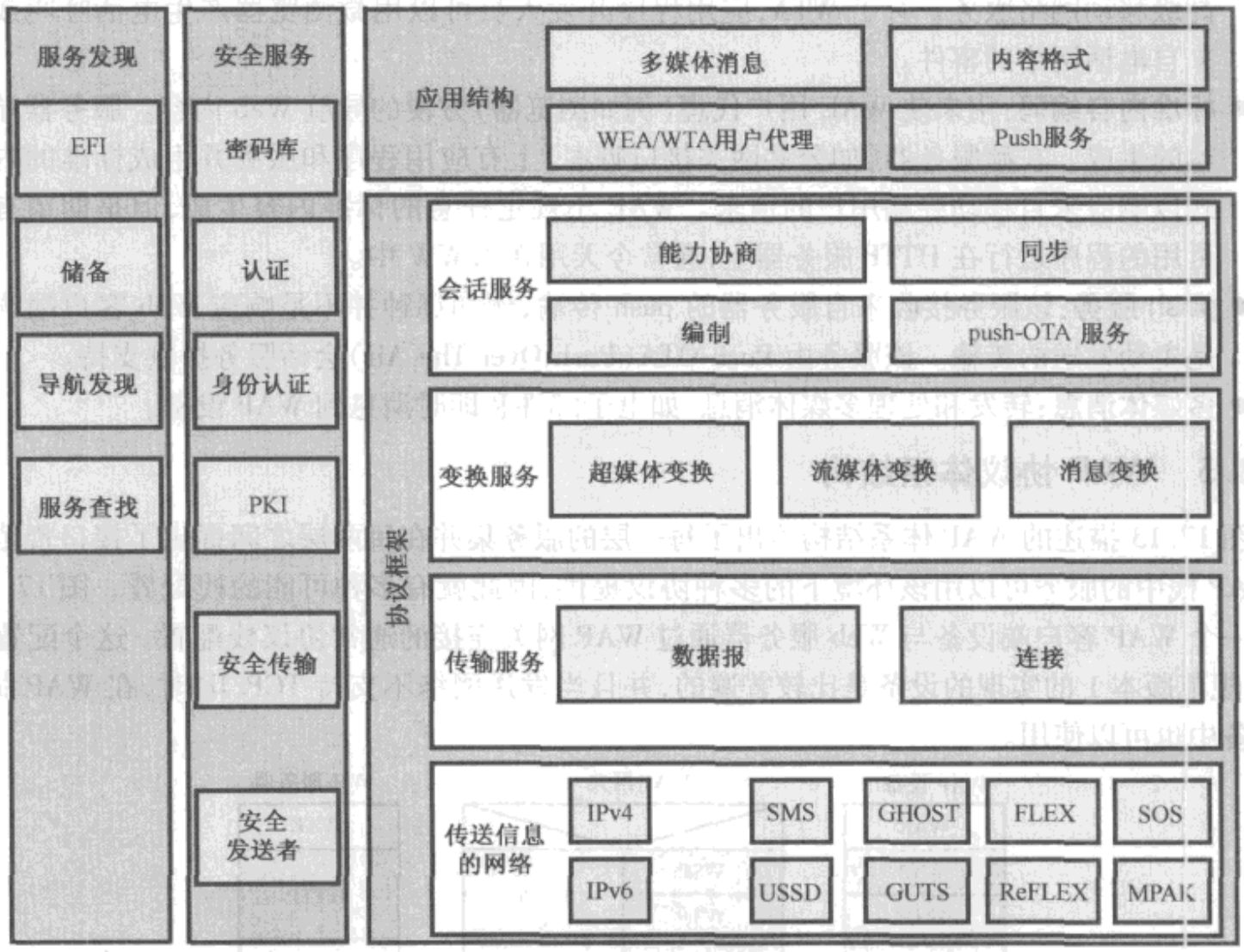


图 17.13　WAP 结构

服务发现

有一系列的服务发现的服务使 WAP 客户端和 Web 服务器决定它们的功能和服务。服务发现服务的例子包含如下：

- **外部功能性接口(EFI)**：外部功能性接口允许应用程序发现设备上有什么外部功能和服务可以使用。
- **储备**：该服务允许设备以必要的参数来为访问网络服务提供准备。
- **导航发现**：该服务使设备能在诸如从超媒体服务器下载资源等导航过程中发现新的网络服务(例如安全的 pull 代理)。在 17.5 节提到的 WAP 传输层端到端安全规范定义了一个导航发现协议。
- **服务查找**：该服务提供了通过名字在字典中查找以发现服务参数。一个典型的例子就是域名服务器(DNS)。

17.3.4　无线通信应用环境

无线通信应用环境(WAE)规定了诸如移动电话、电子书和 PDA 等无线设备的应用程序框架。基本上，WAE 包含了开发 WAP 支持的应用和设备的工具和格式。如图 17.13 所示，WAE 模型的主要元素如下：

- **WAE 用户代理**：在用户无线设备中执行的软件，它为终端用户提供了指定的功能(例如显示内容)。
- **无线电话应用程序(WTA)**：一系列电话扩展的呼叫和控制机制。这些机制为用户提供了

高级移动网络服务。有了 WTA,应用程序开发人员可以用微浏览器产生电话呼叫或回应来自电话网络的事件。

- **标准内容编码**:用来使 WAE 用户代理(例如浏览器)方便的导航 Web 内容。服务器端是内容的生成。在源服务器(如公共网关接口脚本)上有应用程序和服务并生成标准的内容格式以响应来自移动终端用户的请求。WAE 不规定任意的标准内容生成,但是期望有多个可用的程序运行在 HTTP 服务器上,通常今天用在 WWW 中。
- **push 服务**:该服务接收来自服务器的 push 传输,例如那种并不是响应 Web 客户端请求而是主动发送的传输。该服务由 Push-OTA(Push Over The Air)会话服务提供支持。
- **多媒体消息**:转发和处理多媒体消息,如电子邮件和即时消息到 WAP 设备。

17.3.5 WAP 协议体系结构

图 17.13 描述的 WAP 体系结构给出了每一层的服务集并在每两层之间提供了接口规范。由于 WAP 栈中的服务可以用该环境下的多种协议提供,因此就有多种可能的栈配置。图 17.14 给出了一个 WAP 客户端设备与 Web 服务器通过 WAP 网关连接的通常协议栈配置。这个配置对于 WAP 规范版本 1 的实现的设备是比较普遍的,并且当发送网络不支持 TCP/IP 时,在 WAP 版本 2 的设备中也可以使用。

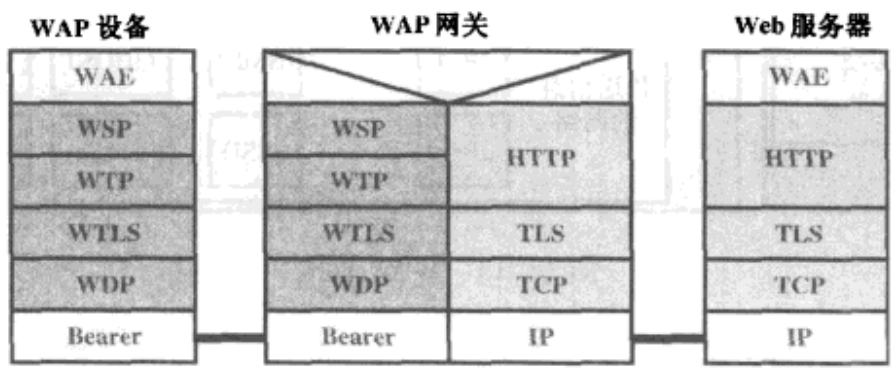


图 17.14 WTP 1.x 网关

在本小节的其他部分中我们将主要介绍 WAP 协议(除了将在 17.4 节中介绍的 WTLS)。

无线会话协议(WSP)

WSP 提供了两个会话服务的接口应用。面向连接的会话服务在 WTP 上运行,而无连接的会话服务在不可靠的传输协议 WDP 上运行。基本上,WSP 是基于 HTTP 并进行了部分添加和修改以使其能在无线信道上的传输更方便。无线网络的最大限制是数据传输率比较低,并且更易由于覆盖率差或者是节点超载导致连接失败。

WSP 是一个基于请求和响应的面向传输的协议。每一个 WSP 协议数据单元(PDU)都包含一个正文和一个消息头。其中正文包含了 WML、WML 脚本或图像,而消息头则包含了关于正文和传输的信息。WSP 也定义了服务器 push 操作,使用该操作服务器可以给客户端设备发送未被请求的内容。这可以用于广播消息或者是诸如新闻标题或者是股票报价等服务,而这些服务都是为客户端设备量身定做的。

无线传输协议(WTP)

WTP 通过在用户代理(例如 WAP 浏览器)和应用程序(例如浏览和电子商务)服务器之间传递请求和响应来管理传输过程。WTP 提供可靠的传输服务,但是 TCP 的分发代价比较大,于是就产生了轻量的协议以适应“瘦”客户端(例如移动节点)的实现,并更适合于低带宽的无线连接。WTP 包含如下特征:

- 有三类处理服务。
- 任意端到端可靠性:WTP 用户触发对每个接收消息的验证。
- 认可任意频带外的数据。
- PDU 级联和延迟确认来减少消息发送的数目。
- 异步处理。

WTP 是面向处理而不是面向连接的。在 WTP 中,没有明确的连接建立和关闭,而是可靠的无连接服务。

WTP 提供三种可能由 WSP 或其他高层协议唤醒的处理类型:

- **第 0 类**:不可靠唤醒消息并且无结果消息。
- **第 1 类**:可靠的唤醒消息并且无结果消息。
- **第 2 类**:不可靠的唤醒消息并且有一个结果消息。

第 0 类提供不可靠的数据报服务,这可以用在一个不可靠的 push 操作。来自 WTP 用户的数据由 WTP(动作发出者或客户端)封装在一个唤醒 PDU 中并将其发送到目标 WTP(响应者或服务器)而不需确认。响应方 WTP 将数据转发到目标 WTP 用户。

第 1 类提供可靠的数据报服务,这可以用于可靠的 push 操作。来自发送方的数据封装到一个唤醒 PDU 中并且发送给响应者。响应者将数据发送到目标 WTP 用户,并通过回发一个 ACK PDU 给发送者 WTP 实体以表明确认接收到数据。响应方 WTP 保持一个状态信息以便以后 ACK 发送之后,如果 ACK 丢失了或者发送者重发唤醒 PDU 时需要重发 ACK。

第 2 类提供请求/响应处理服务并支持在一个 WSP 会话中执行多个处理过程。来自发送者的数据封装在一个环形 PDU 中并将其发送给响应方,然后响应方将其转发给目标 WTP 用户。目标 WTP 用户准备响应数据并将之传给本地 WTP 实体。响应方 WTP 实体回发这些数据为结果 PDU。假如生成响应数据的延迟超过了一个时间阈值,则响应方就会在发送结果 PDU 之前发送一个 ACK PDU。这可以防止发送方不必要地重新发送唤醒信息。

无线数据报协议(WDP)

WDP 是用来调整高层 WAP 协议使之适应用于移动节点和 WAP 网关的通信机制(通常称为发送者)。调整过程包含将数据分割成适合发送方尺寸的报文并与发送方网络交互。WDP 使各种发送方网络的细节对 WAP 的其他层透明。在一些实例中,WAP 是在 IP 层的顶部实现的。

17.4　无线传输层安全

无线传输层安全(WTLS)提供了移动设备(客户端)和 WAP 网关之间的安全服务。WTLS 是基于工业标准的传输层安全协议(TLS)①,该协议是安全套接层(SSL)协议的升级。TLS 是用于 Web 浏览器和 Web 服务器之间的标准安全协议。WTLS 比 TLS 更有效,因为它需要更少的信息交换。为了提供端到端安全,WTLS 用于在客户端和网关之间,TLS 用于网关和目标服务器之间(如图 17.14 所示)。WAP 系统在 WAP 网关内进行 WTLS 和 TLS 之间的转换。因此,网关成为一个攻击的软肋,于是需要有更高层次的安全性保证来应对外部攻击。

WTLS 提供了如下的特征:

---

① 关于 SSL/TLS 的讨论详见第 16 章。但是,本节中的讨论是独立的,读者并不需要事先阅读 TLS 的描述。

- **数据完整性**:使用消息认证来保证在客户端和网关之间传输的数据没有被修改。
- **机密性**:使用加密方法保证数据不能被第三方读到。
- **认证性**:使用数字证书来认证通信双方。
- **拒绝服务攻击防御**:检测并拒绝恶意重发或是未成功验证的消息。

17.4.1 WTLS 会话和连接

WTLS 中两个重要的概念是安全会话和安全连接,这两个概念在规范文档中定义如下:

- **安全连接**:一个连接是一个提供合适服务类型的传输器(如 OSI 层次模型中定义的)。对于 SSL,每一个连接是 P2P 的关系。每一个连接都是瞬间的并与一个会话关联。
- **安全会话**:一个 SSL 会话是一个客户端和服务器之间的关联,并由握手协议生成。会话定义了一系列密码学安全参数,这些参数可以在多方连接中共享。会话是用来避免每个连接之间协商新的安全参数带来的昂贵代价。

在任意一对通信双方(例如在客户端和服务器之间的 HTTP 应用)中都存在多个安全连接。理论上,通信双方会同时存在多个会话,但是实际应用中却很少这么使用。

每一个会话会与多个状态关联。当建立一个会话时,对读和写会有一个当前操作状态(例如接收和发送)。另外,在握手协议过程中,会创建读挂起和写挂起状态。当握手协议成功结束时,挂起就成为当前状态。

一个会话状态会用如下参数定义:

- **会话标志符**:服务器选取的一个任意字节序列来表示一个活动或一个可恢复的会话状态。
- **协议版本**:WTLS 协议版本号。
- **同等证书**:通信对方的证书。该状态元素应该是 null。
- **压缩方法**:在加密之前用于压缩的算法。
- **密码规范**:规定了块数据加密算法(例如 null,RC5,DES 等)和一个用于计算消息验证码(MAC)Hash 算法(例如 MD5 或者是 SHA-1)。同时还定义了其他密码学参数,例如 hash_size。
- **主密钥**:客户端和服务器之间共享的一个 20 字节的秘密信息。
- **序列号**:在该安全连接中使用哪种序列号方案(off,implicit 或 explicit)。
- **密钥更新**:定义一些连接状态值[加密密钥,MAC 密钥和初始向量(IV)]更新的周期。
- **是否可恢复**:一个指示该会话能否初始化一个新连接的标志。

连接状态是记录协议的操作环境。它包含所有密码操作(加/解密和 MAC 的计算/验证)所需的所有参数。每一个安全连接都有一个用如下参数定义的连接状态:

- **连接终端**:指示在该安全会话中该实体是客户端还是服务器。
- **块加密算法**:包含了该算法中密钥长度、密钥的隐私性、是块加密还是流加密以及加密的块的大小(以及是否合适)。
- **MAC 算法**:包含了用于计算 MAC 的密钥长度和 MAC 算法返回的 Hash 值的长度。
- **压缩算法**:包含所有用于压缩算法的信息。
- **主密钥**:客户端和服务器间共享的 20 字节的秘密信息。
- **客户端随机数**:客户端提供的 16 字节的随机数。
- **服务器随机数**:服务器端提供的 16 字节的随机数。
- **序列号模式**:在该安全连接中使用哪种序列号方案。

- **密钥更新**:定义一些连接状态参数[加密密钥,MAC 密钥和初始向量(IV)]更新的周期。新的密钥在每 $n=2^{key_refresh}$ 消息计算,即序列号为 $0,2^n,3^n$ 等。

17.4.2　WTLS 协议体系结构

WTLS 不是单个协议而是两个协议层,如图 17.15 所示。WTLS 的记录协议为各种高层协议提供基本的安全服务。特别是,为 Web 客户端和服务器交互提供传输服务的超文本传输协议(HTTP)可以在 WTLS 的协议顶层运行。WTLS 定义了三部分高层协议:握手协议、修改密码规范协议和警报协议。这些 WTLS 规定的协议是用来管理 WTLS 交换过程的,我们将在随后讲解。

WTLS 记录协议

WTLS 记录协议从上一层(WTP,WTLS 握手协议、WTLS 警报协议和 WTLS 修改密码规范协议)获取用户数据并将其封装在一个 PDU 中。其过程如下(参见图 17.16):

第 1 步:使用无损压缩算法压缩有效载荷。

第 2 步:使用 HMAC 算法在压缩后的数据上计算消息验证码(MAC)。可以将某种 Hash 算法(如 MD-5 或 SHA-1)与 HMAC 配合使用。Hash 码的长度为 0,5 或 10 个字节。MAC 计算的值附加在压缩后的数据之后。

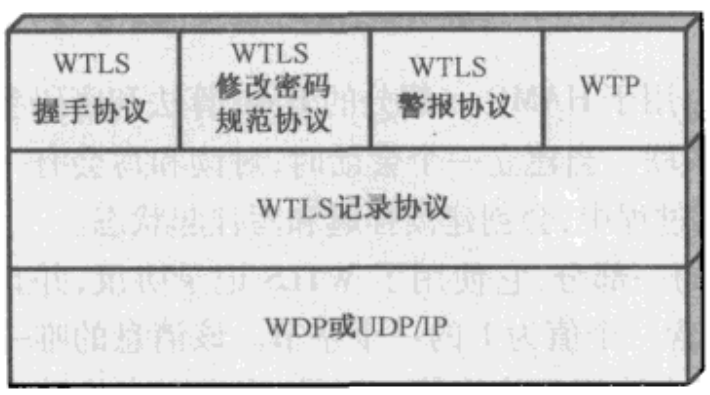


图 17.15　WTLS 协议栈

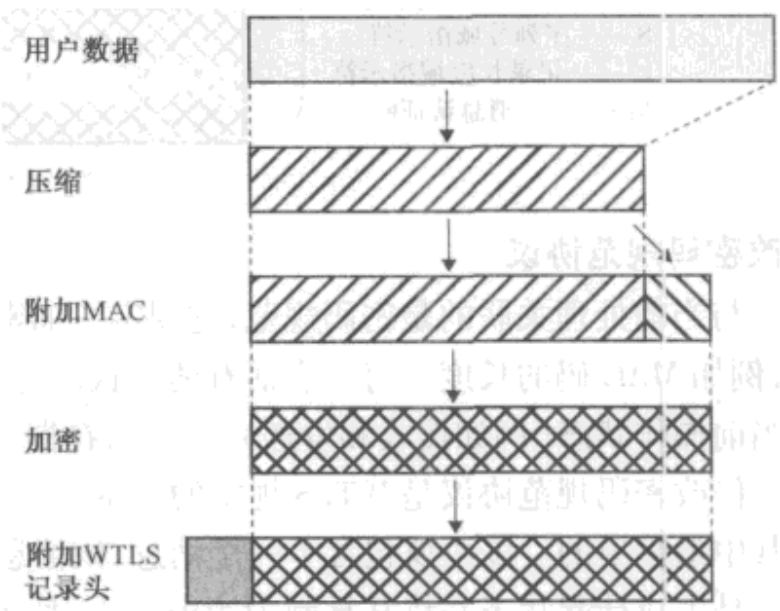


图 17.16　WTLS 记录协议操作

第 3 步:使用对称加密算法对压缩后的数据加 MAC 码加密。使用的加密算法有 DES、三重 DES、RC5 和 IDEA。

第 4 步:记录协议为加密后的有效载荷预先计算一个报头。

记录协议报头包含如下数据域(如图 17.17 所示):

- 记录类型(8 位):包含如下子域:
  —记录长度域指示符(1 位):指示记录长度域是否有效。
  —序列号域指示符(1 位):指示序列号域是否有效。
  —加密规范指示符(1 位):假如该位为 0,表示没有使用压缩算法、MAC 保护或加密算法。
  —内容类型(4 位):WTLS 记录协议之上的高层协议。
- 序列号(16 位):与该记录关联的某序列号。提供了在不可靠传输服务上的可靠性。
- 记录长度(16 位):明文(或当使用压缩后的压缩数据)的长度,以字节为单位。

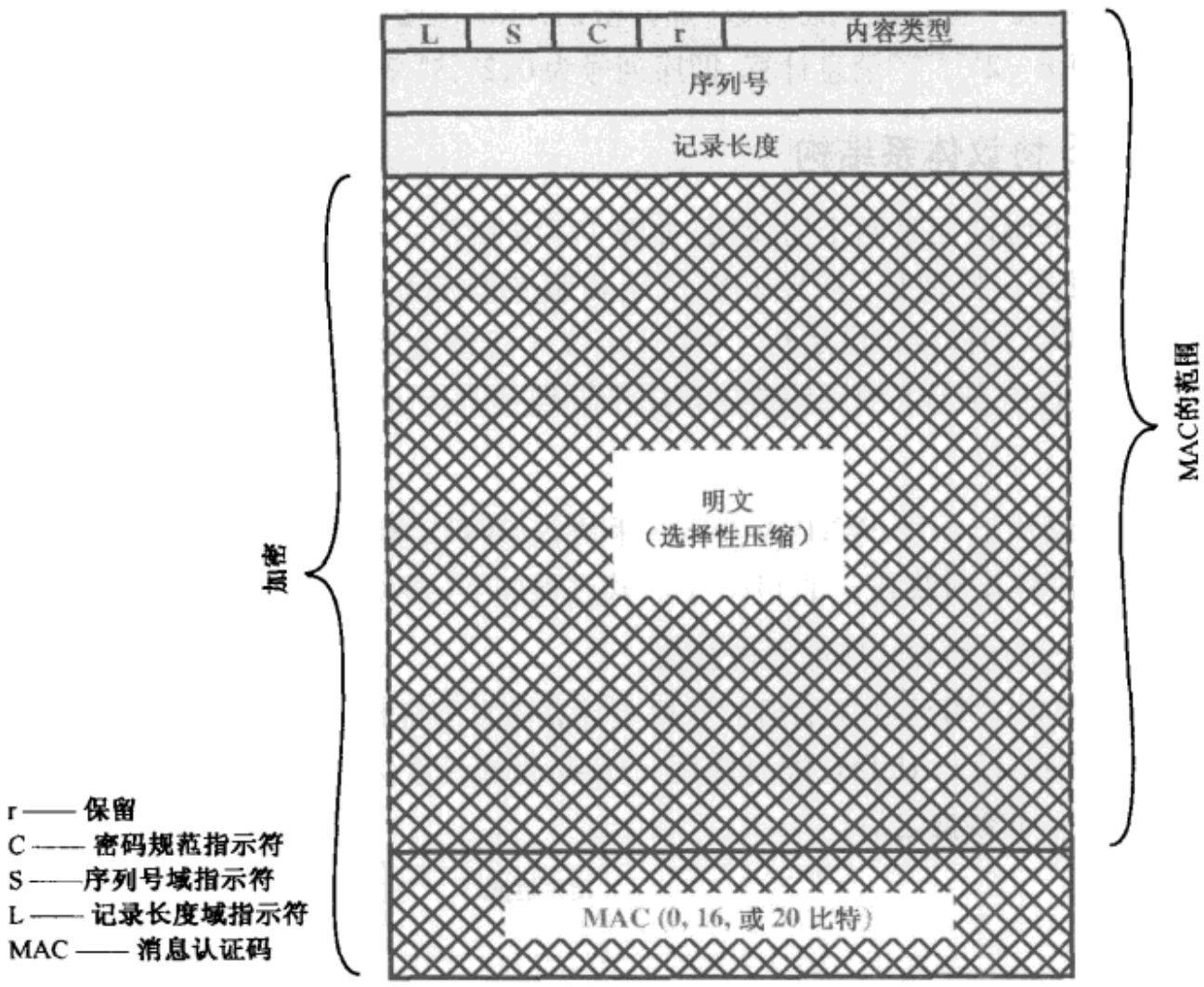


图 17.17 WTLS 记录格式

修改密码规范协议

与当前处理关联的是密码规范,它规定了加密算法、用于 HAMC 一部分的 Hash 算法和密码参数,例如 MAC 码的长度。每个会话有两个状态与之相关联。当建立一个会话时,对读和写会有一个当前操作状态(例如接收和发送)。另外,在握手协议过程中,会创建读挂起和写挂起状态。

修改密码规范协议是 WTLS 规定的三部分协议中的一部分,它使用了 WTLS 记录协议,并且是其中最简单的。该协议包含了一个消息,该消息值包含一个值为 1 的一个字节。该消息的唯一目的是生成挂起状态并将其复制为当前状态,使该密码方法用于该连接。于是,当密码变换消息到达时,消息的发送者就将当前的写状态置为挂起,而接收者则将当前的读状态置为挂起。

警报协议

警报协议是用来向对等实体发送 WTLS 相关的警报信息。和其他使用 WTLS 的应用一样,警报信息如当前状态所指示的那样进行压缩和加密。

协议中每个消息都包含两个字节。第一个字节的值为警报(1),关键(2),致命(3)来表示该消息的严重程度。第二个字节包含一个码字来指示规定的警报。假如是致命级的,WTLS 立即终止该连接。其他在该会话上的连接可能会继续,但是不会再在该连接上建立新的连接。在安全会话上的其他连接可以继续并且该安全表示也可以用来创建其他安全连接。

使用告警信息可以关闭连接。通信中的任意一方都可以初始化交换为关闭消息。当接收到关闭消息时,任何在该消息之后的数据都将被拒绝。同时也需要通过响应关闭消息来通知验证会话结束。

WTLS 中的错误处理也是基于警报消息的。当检测到一个错误时,检测方就发送一个包含当前错误的警报消息。进一步的处理取决于发生的错误的严重程度。

致命警报的例子如下：

- session_close_notify：通知接收方、发送方将不再使用该连接或该安全会话发送任何消息。
- unexpected_message：接收到一个不合适的消息。
- bad_record_mac：接收到一个不正确的消息验证码(MAC)。
- decompression_failure：接收到的解压缩函数不正确(例如,无法解压缩或解压缩后尺寸超过允许的上限值)。
- handshake_failure：当前条件下发送者未能协商成一系列可接受的安全参数。
- illegal_parameter：在握手消息中值超出了限定范围或与其他域不相符。

非致命警报的例子如下：

- connection_close_notify：通知接收方、发送方将不再使用该连接发送任何消息。
- bad_certificate：接收到的证书不合法(例如包含一个无法验证的签名)。
- unsupported_certificate：接收到的证书类型不支持。
- certificate_revoked：证书已被签名者撤销。
- certificate_expired：证书已经过期。
- certificate_unknown：在处理证书过程中遇到的其他未知事项。

握手协议

WTLS 中最复杂的是握手协议。该协议允许服务器和客户端相互认证并协商加密和 MAC 算法,以及用于保护在 WTLS 记录中发送的数据的密钥。握手协议是在任何应用程序传输数据之前进行的。握手协议的一个重要功能是生成一个前主密钥,这在随后是用来生成主密钥的,然后该主密钥用来生成各种密钥。

握手协议包含一系列客户端与服务器之间的消息交换。图 17.18 给出了在建立客户端与服务器之间逻辑连接之前的初始交换。该交换过程可以看成有 4 个阶段。

第一阶段初始化一个逻辑连接并建立一个与之相关联的安全性能。交换过程由客户端进行初始化。客户端发送一个 client_hello 消息,消息中包含了一个会话标志、客户端支持的密码算法和压缩算法的列表(按性能降序将每种类型的算法排序)。发送 client_hello 消息之后,客户端就等待 server_hello 消息。该消息指示在交换中将使用哪种密码算法和哪种压缩算法。

第二阶段进行服务器认证和密钥交换。当需要被认证时,服务器从发送一个公钥证书开始。然后,当需要时将发送一个 server_key_exchange 消息。在使用对称密钥交换的公钥算法中可能需要该消息。然后,服务器可以发送 certificate_request 来要求客户端提供公钥证书。第二阶段中最后的消息是 server_hello_done 消息,该消息由服务器发出并表明服务器 hello 结束。在发送该消息之后,服务器将等待客户端响应,该消息没有任何参数。

第三阶段进行客户端认证和密钥交换。当接到 server_hello_done 消息后,客户端应该验证服务器提供的证书并确认 sever_hello 参数是否有效。假如所有条件都满足,则客户端发送一个或多个消息给服务器。如果服务器发出了证书请求,客户端就发送一个证书消息。然后是客户端发送 client_key_exchange 消息,该消息必须在本阶段发送,消息的内容取决于密钥交换的类型。最后,客户端发送一个 certificate_verity 消息来提供客户端证书的明确验证。

第四阶段完成安全连接的建立。客户端发送一个 change_cipher_spec 消息并将挂起 Cipher-

Spec 复制到当前的 CipherSpec 中。值得注意的是,该消息不是握手协议的部分而是用修改密码规范协议发送的。然后客户端在新算法、密钥和秘密信息下立即发送一个结束消息。该结束消息确认了密钥交换和认证过程顺利完成。为了回应这两个消息,服务器发送自己的 change_cipher_spec 消息,将 CipherSpec 置为挂起,然后发送结束消息。至此,握手结束,客户端和服务器开始交换应用层数据。

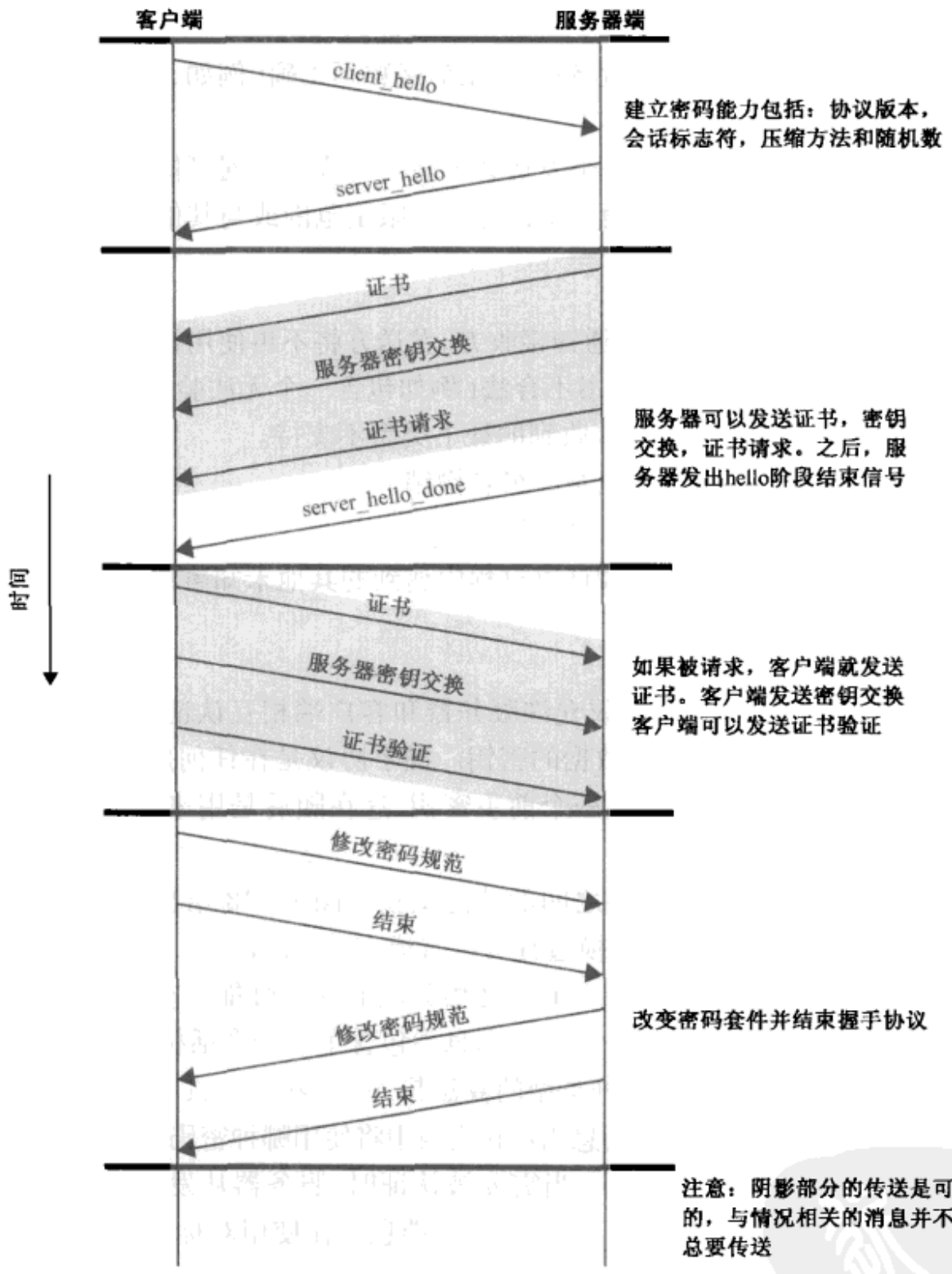


图 17.18 WTLS 握手协议操作过程

17.4.3 密码算法

认证

在 WTLS 中认证是通过证书来完成的。认证可以在客户端和服务器之间进行也可以仅仅是客户端认证服务器。后者的过程仅当服务器允许时才发生。服务器也可以要求客户端向自己认证自身。因此,WTLS 规范定义的认证是一个可选择的过程。目前支持 X.509v3,X9.68 和 WTLS 证书。WTLS 证书在尺寸方面做了优化,其组成元素如下(与图 14.14 对比):

- Certificate_version:证书的版本。

- Signature_algorithm:证书使用签名算法。
- Issuer:定义了签名证书的一方,通常是证书中心(CA)。
- Valid_not_before:证书合法的启用时间。
- Valid_not_after:该时刻之后证书将失去合法性。
- Subject:密钥所有者,与验证的公钥相关联。
- Public_key_type:公钥(算法)的类型。
- Parameter_specifier:规定与公钥相关的参数。
- Public_key:验证的公钥。
- Signature:用 CA 的私钥签名。

密钥交换

WTLS 协议的目的是生成一个客户端和服务器可以共享的前主密钥。然后使用该密钥生成主密钥。WTLS 支持许多密钥交换协议。这些协议可以划分为包含 server_key_exchange 消息作为握手协议一部分和不可以作为握手协议另一部分这两类(参见图 11.18)。

仅当服务器证书消息(如果已经发生了)未包含足够的数据允许客户端交换前主密钥时,服务器才发送 server_key_exchange 消息。如下三种方法需要使用 server_key_exchange 消息。

- DH_anon:匿名(未认证)的执行传统的 Diffie-Hellman 计算。协商的密钥(Z)就作为前主密钥(pre_master_secret)。
- ECDH_anon:执行椭圆曲线上的 Diffie-Hellman 计算。协商的密钥(Z)就作为前主密钥(pre_master_secret)。
- RSA_anon:这是无认证的 RSA 密钥交换。服务发送它的 RSA 公开密钥。在该方法中,客户端生成一个 20 字节的秘密信息,然后用服务器的公钥加密,并将之发送给服务器。然后服务器就用自己的私钥解密该秘密信息。此时前主密钥(pre_master_secret)就是该秘密信息和服务器的公钥。

在如下的密钥交换方法中服务器并不发送密钥交换消息。

- ECDH_ECDSA:使用基于证书的 ECDSA 的椭圆曲线上的 Diffie-Hellman 进行密钥交换。服务器发送一个包含其 ECDH 公钥的证书。服务器的证书由客户端可信的第三方使用 ECDSA 签名。根据客户端是否被认证,服务器发送包含其由自己信任的可信第三方用 ECDSA 签名的 ECDH 公钥的证书或者仅仅是自己(暂时)的 ECDH 公钥。通信双方用自己的密钥和接收到的或是证书中包含的对方的公钥来计算前主密钥。
- RSA:RSA 密钥交换使用基于 RSA 的证书。服务器发送一个包含自己 RSA 公钥的证书。该证书由客户端信任的第三方使用 RSA 签名。客户端从接收到的证书中获取服务器的公钥,然后生成秘密信息并用服务器的公钥加密,然后将其发送给服务器。前主密钥就是该秘密信息和服务器的公钥。假如客户端是被认证过的,它可以用自己的 RSA 私钥签名一些数据(在握手中交互的消息),然后发送自己的证书和签名数据。

伪随机函数(PRF)

PRF 是用于在 WTLS 中生成特殊用途的自然数。PRF 将秘密信息值、标志和种子输入然后输出任意长度的位流。在 TLS 标准中,使用了两个 Hash 算法类使 PRF 尽可能安全。为了节约资源,WTLS 可以使用一个 Hash 算法来实现 PRF。作为密钥变换协议的一部分,确定使用哪个 Hash 算法可以在握手阶段协商。

PRF 基于数据扩展函数

```
P_hash(secret, seed) =  HMAC_hash(secret, A(1) ‖ seed) ‖
                        HMAC_hash(secret, A(2) ‖ seed) ‖
                        HMAC_hash(secret, A(3) ‖ seed) ‖ ...
```

其中 ‖ 表示串联,A( ) 定义如下:

```
A(0) = seed
A(i) = HMAC_hash(secret, A(i - 1))
```

于是

```
PRF(secret, label, seed) = P_hash(secret, label ‖ seed)
```

主密钥生成

通过密钥交换方式生成的共享主密钥是一次性的 20 字节的值(160 位)。首先是交换前主密钥(pre_master_secret),然后通信双方计算主密钥(master_secret)。使用的函数如下:

```
master_secret = PRF(pre_master_secret, "master secret",
                    ClientHello.random ‖ ServerHello.random)
```

其中 ClientHello.random 和 ServerHello.random 是在握手协议第一阶段交换的随机数。

于是可以用主密钥生成 MAC 密钥和加密密钥。MAC 计算使用 HMAC 算法(参见第 12 章)并使用了如下的数据域:

```
HMAC_hash (MAC_secret, seq_number ‖ WTLSCompressed.
  record_type ‖ WTLSCompressed.length ‖ WTLS
  Compressed.fragment)
```

其中 WTLSCompressed.fragment 表示(可选的)压缩的明文数据。

在 HMAC Hash 函数中可能会使用 MD5 或 SHA-1。

会对除报头外的所有 WTLS 记录进行加密,如下是允许使用的加密算法。

算　法	密钥长度(位)
RC5	40 56 64 128
DES	192
3DES	40
IDEA	40 56

17.5 WAP 端到端安全

如图 17.19 表明,基本的 WAP 传输模型涉及一个 WAP 客户端,一个 WAP 网关和一个 Web 服务器,导致了一个安全缺口。该图对应图 17.14 显示的协议架构。移动设备和 WAP 网关建立一个安全 WTLS 会话。WAP 网关接着和 Web 服务器建立一个安全 SSL 或 TLS 会话。在网关内,数据在传输过程中没有加密。所以网关也是数据可能被攻陷的一个点。

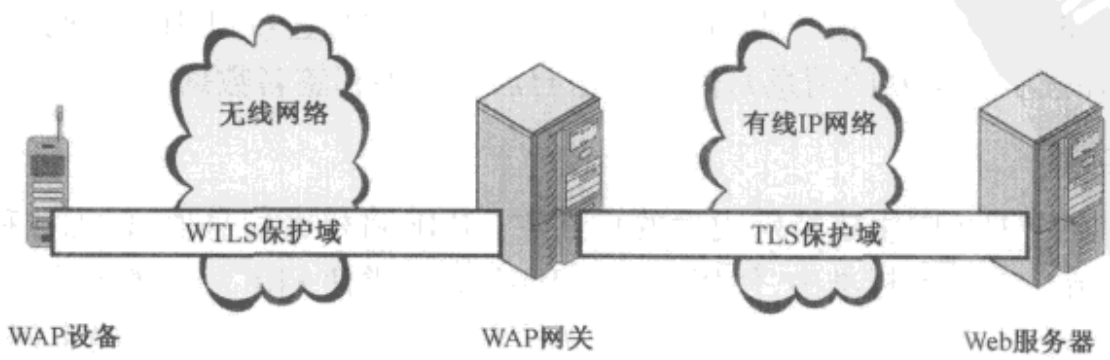


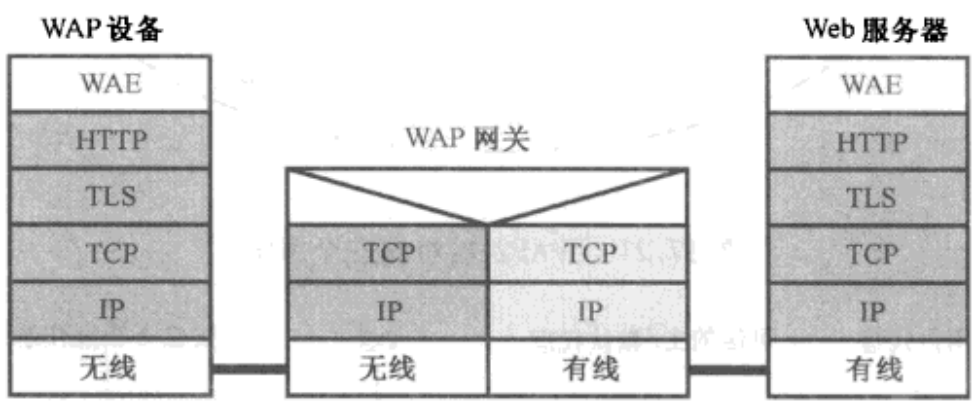
图 17.19　采用标准安全服务的安全区域

有许多方法可以在移动客户端和 Web 服务器之间提供端到端安全。在 WAP 版本 2(被称为 WAP2)架构文档中,WAP 论坛定义了几个协议部署可以允许端到端安全。

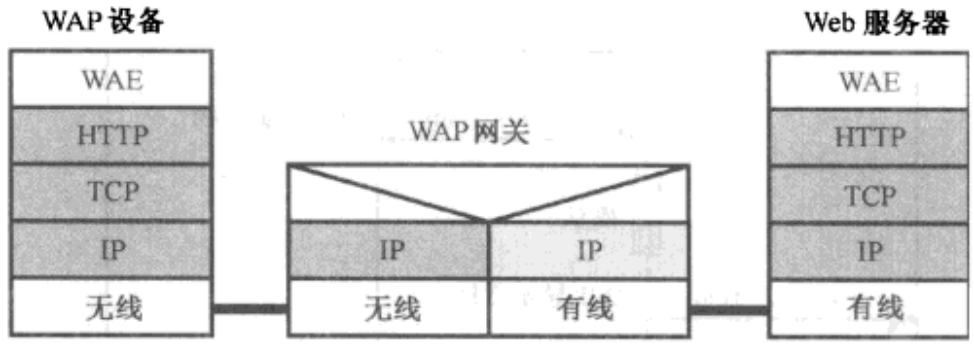
WAP 的版本 1 假设无线网络上的一组简化协议,并假设无线网路不支持 IP。WAP2 提供了选项,可以让移动设备实施全部基于 TCP/IP 的协议,以及在有 IP 功能的无线网络上操作。图 17.20 表明有两种方法可以利用 IP 功能提供端到端安全。在这两种方法中,移动客户端都实施 TCP/IP 和 HTTP。

第一种方法[参见图 17.20(a)]利用客户端和服务器之间的 TLS。一个安全的 TLS 会话在端点间建立。WAP 网关如 TCP 层的网关一样工作,把两个 TCP 连接拼接起来,以承载端点之间的流量。尽管如此,TCP 用户数据域(TLS 记录)在经过网关时仍然是加密的,所以保持了端到端的安全。

另一个可能的方法显示在图 17.20(b)。这里我们假设 WAP 网关只是作为一个简单的互联网路由器。在这种情况下,可以通过在 IP 层使用 IPsec(将在第 19 章讨论)提供端到端安全。



(a) 基于TLS的安全



(b) 基于IPsec的安全

图 17.20　WAP2 端对端安全方法

然而,某种程度上更复杂的另一种方法已经被 WAP 论坛在更具体的条款中定义了,命名为“WAP 传输层端到端安全。”这个方法在图 17.21 中说明(基于[ASHI01]中的一个图)。在这个情景中,WAP 客户端连接到它常用的 WAP 网关,并尝试通过网关发送一个请求给一个安全域。安全内容服务决定了出于安全性的考虑,要求移动客户端连接到它的本地 WAP 网关而不是它的默认网关。Web 服务器用一个 HTTP 重定向消息回应初始客户端请求,消息重定向客户端到作为企业网络一部分的 WAP 网关。这个消息通过默认网关传回,网关会验证重定向并把消息传给客户端。客户端捕获重定向消息,利用 WTLS 和企业 WAP 网关建立一个安全会话。在连接终止后,默认网关被重新选择并用于跟其他 Web 服务器的后续通信。注意这种方法要求企业在客户使用的无线网络上维持一个 WAP 网关。

图 17.22 从 WAP 条款中说明了该会话。

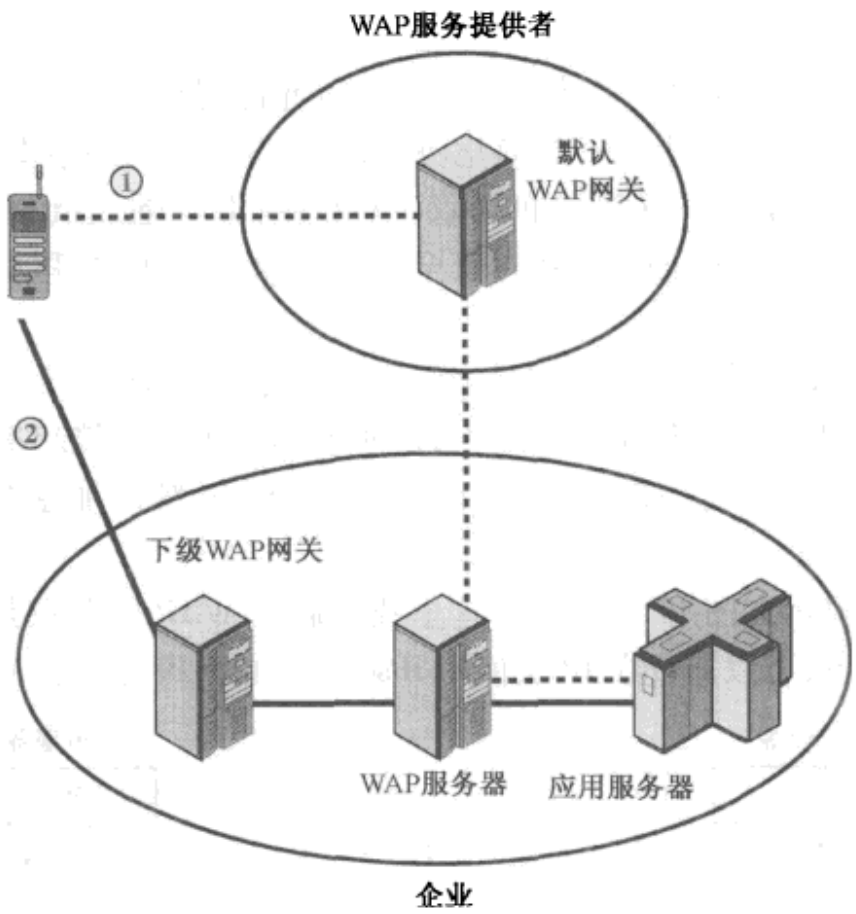


图 17.21 WAP2 端对端安全方案

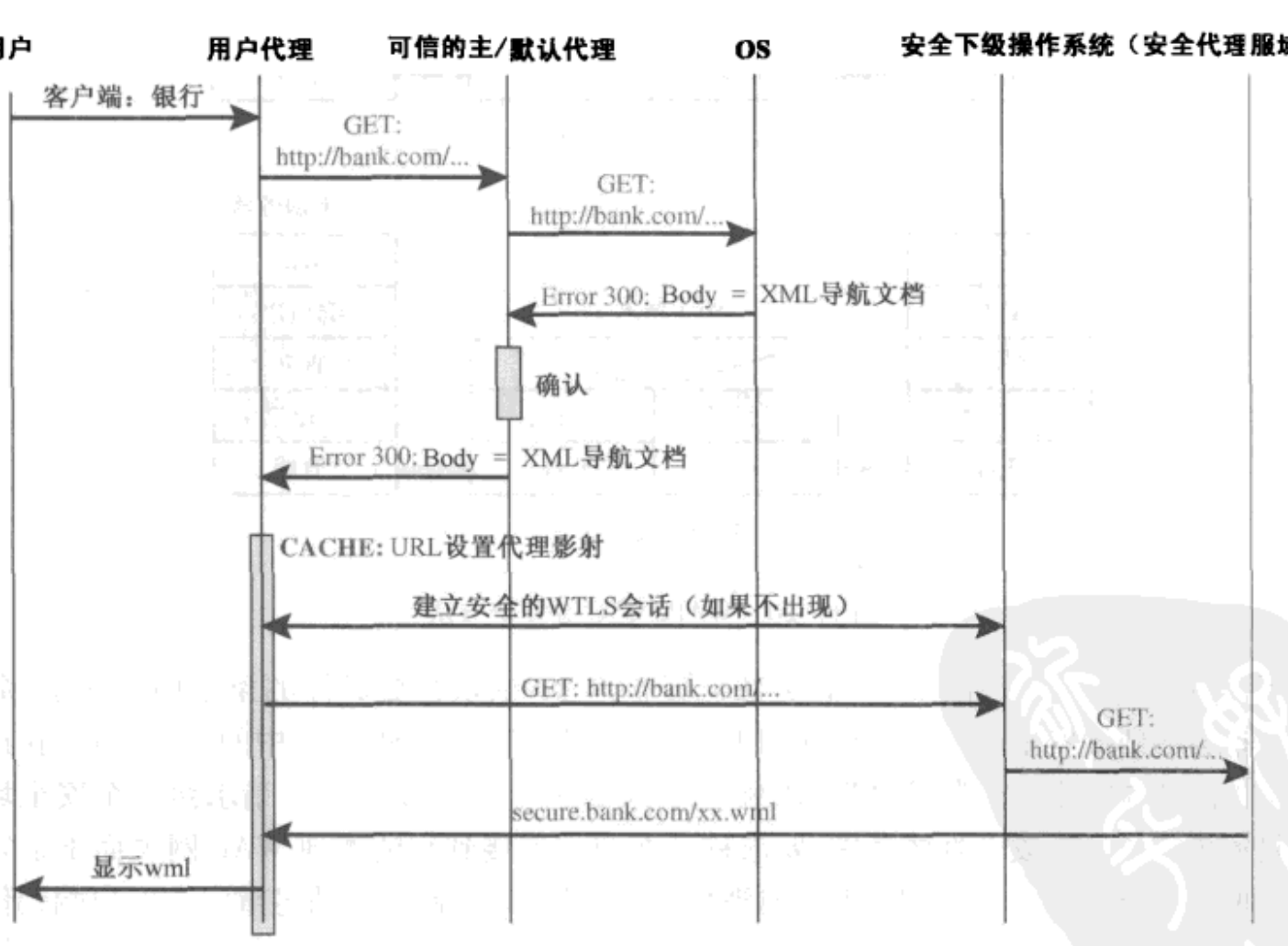


图 17.22 WAP 传输层端对端安全实例

17.6 推荐读物和网站

IEEE 802.11 和 Wi-Fi 规定在[STAL07]中有更多细节,[ROSH04]是长度合适的好书,[FRAN07]对 IEEE 802.11i 有很好的消息介绍。[CHEN05]提供了 IEEE 802.11i 的概述。

CHEN05 Chen, J.; Jiang, M.; and Liu, Y. "Wireless LAN Security and IEEE 802.11i" *IEEE Wireless Communications*, February 2005.

FRAN07 Frankel, S.; Eydt, B.; L.; and Scarfone, K. *Establishing Wireless Robust Security Networks: A Guide to IEEE 802.11i.* NIST Special Publication SP 800-97, February 2007.

ROSH04 Roshan, P., and Leary, J. 802.11 *Wireless LAN Fundamentals.* Indianapolis: Cisco Press, 2004.

STAL07 Stallings, W. *Data and Computer Communications, Eighth Edition.* Upper Saddle River, NJ: Prentice Hall, 2007.

推荐网站

- **The IEEE 802.11 Wireless LAN Working Group**：包含工作组文档和讨论存档。
- **Wi-Fi Alliance**：促进 802.11 产品彼此之间及和以太网的互操作性的一个工业组。
- **Wireless LAN Association**：给出技术介绍，包括执行审议的讨论和来自用户的案例研究。相关网站的链接。
- **扩展认证协议(EAP)工作组**：IETF 工作组负责 EAP 和相关议题。网站包括 RFC 文档和互联网草案。
- **Open Mobile Alliance**：WAP 论坛和开放移动架构论坛的合并。提供 WAP 技术规定和工业链接。

17.7　关键术语、思考题和习题

关键术语

4 次握手
接入点(AP)
警报协议
基本服务集(BSS)
修改密码规范协议
计数器模式 CBC MAC 协议(CCMP)
分布式系统(DS)
扩展服务集(ESS)
组密钥
握手协议
IEEE 802.11X
IEEE 802.11
IEEE 802.11i
独立 BSS(IBSS)
逻辑链路控制层(LLC)
媒体访问控制层(MAC)
MAC 协议数据单元(MPDU)
MAC 服务数据单元(MSDU)
消息完整性码(MIC)
Michael 算法
对密钥
伪随机函数
RSN
临时密钥完整性协议(TKIP)
有线等效保密(WEP)
无线应用环境(WAE)
无线应用协议(WAP)
无线报文协议(WDP)
无线局域网(WLAN)
无线标记语言(WML)
无线会话协议(WSP)
无线交互协议(WTP)
无线传输层安全(WTLS)
Wi-Fi
Wi-Fi 受保护访问(WPA)
WTLS 记录协议

思考题

17.1　什么是一个 802.11WLAN 的基本构建块?

17.2　定义一个扩展服务集。

17.3 列出并简要定义 IEEE 802.11 服务。
17.4 分布式系统是一个无线网络吗?
17.5 互连性和流动性的观点是如何相联系的?
17.6 哪些安全区域是 IEEE 802.11i 提出的?
17.7 简要描述 IEEE 802.11i 操作的 4 个阶段。
17.8 TKIP 和 CCMP 的区别?
17.9 HTML 过滤器和 WAP 代理的区别?
17.10 WSP 提供什么服务?
17.11 三个 WTP 交互类何时会被用到?
17.12 列举并简要定义 WTLS 提供的安全性服务。
17.13 简单描述 WTLS 的 4 个协议元素。
17.14 列举并简要定义 WTLS 中的所有密钥。
17.15 描述提供 WAP 端到端安全性的可选方法。

习题

17.1 在 IEEE 802.11 中,开放系统认证简单地由两个通信组成。一个认证是客户端的请求,包含基站 ID(通常是 MAC 地址)。随后的一个认证是来自 AP/路由器的回应,包含一个成功或失败消息。失败可能发生的一个例子是,客户端 MAC 地址明显不包含于 AP/路由器配置中。
(a) 这个认证方案的好处是什么?
(b) 这个认证方案的安全性弱点是什么?

17.2 在介绍 IEEE 802.11i 之前,IEEE 802.11 的安全性方案是有线对等保密(WEP)。WEP 假设网络中的所有设备共享一个秘有密钥。认证的目的是让 STA 证明它拥有该秘有密钥。认证步骤如图 17.23 所示。STA 发送一个消息给 AP,请求认证。AP 发出一个挑战,这是一个以明文发送的 128 字节的随机序列。STA 用共享密钥加密挑战并回送给 AP。AP 解密接收的值并和自己发送的挑战比较。如果符合,AP 就确认认证已经成功。
(a) 该认证方案的好处?
(b) 该认证方案是不完整的。缺了什么以及为什么它是重要的?提示:增加一个或两个消息就能解决问题。
(c) 该方案的加密弱点是什么?

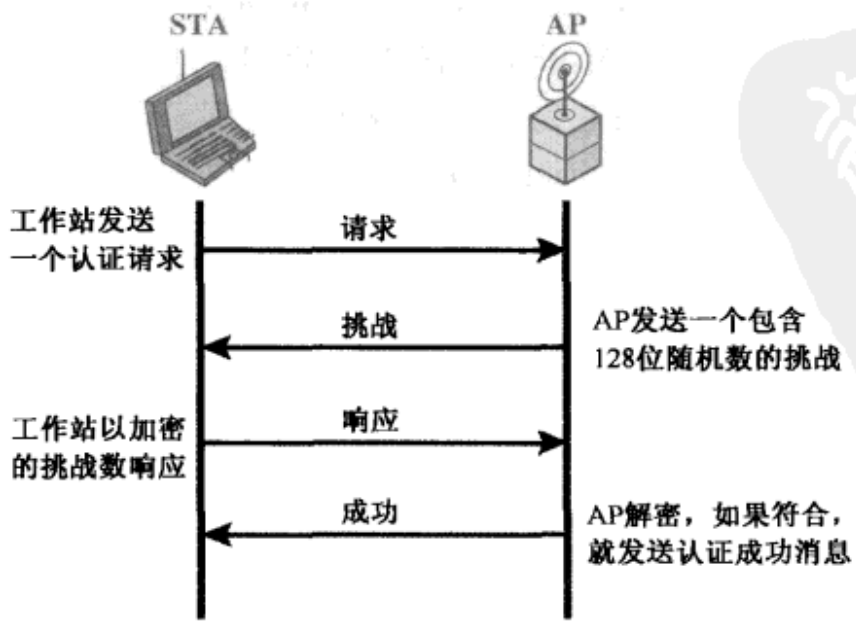


图 17.23 WEP 认证

17.3　对于 WEP,数据完整性和数据保密性使用 RC4 流加密算法实现。MPDU 发送者执行如下步骤,称为封装:

(1) 发送者选择一个初始向量(IV)值。

(2) IV 值和被发送者与接收者共享的 WEP 密钥串联形成一个种子或密钥输入给 RC4。

(3) 对 MAC 数据域计算一个 32 位循环冗余检验码(CRC)并附加到数据域。CRC 是常见的用于数据链路控制协议的错误检验码。在这种情况下,CRC 作为一个完整性检验值(ICV)。

(4) 第 3 步的结果使用 RC4 加密形成密文块。

(5) 明文 IV 被前置于加密块形成要传输的封装 MPDU。

(a) 画出框图说明封装过程。

(b) 描述接收端发现明文和执行完整性检验的步骤。

(c) 画出框图说明(b)部分。

17.4　以 CRC 做完整性检验的潜在弱点是一个线性函数。这意味着如果消息中的单位改变,你可以预测 CRC 的哪个位会发生改变。进一步说,可以确定消息中可以被改变的位组合,以使 CRC 中的网络结果不会改变。因此,有许多明文消息的位翻转组合可以使 CRC 不改变,所以消息完整性就被攻破了。尽管如此,在 WEP 中,如果攻击者不知道加密密钥,攻击者就不能访问明文,只能访问密文块。这是否意味着 ICV 就可以不受位翻转攻击了? 请解释。

17.5　WTLS 中的一个潜在弱点是使用 CBC 模式密码加密。CBC 模式块密码的标准状态,每个记录的 IV(初始向量)按一下方法计算:record_IV = IV $\oplus S$,这里 IV 是原始 IV,而 S 是通过串联记录中 2 字节的序列号获得的,串联的次数取决于 IV 的字节数。也就是说,如果 IV 是 8 字节长度,记录的序列号要和本身串联 4 次。现在在 CBC 模式下,对于序列号为 i 的记录,第一个明文块会被加密为(参见图 6.4)

$$C_1 = \mathrm{E}(K, [\mathrm{IV} \oplus S \oplus P_{s,1}])$$

这里 $P_{s,1}$ 是序列号为 s 的记录的第一个明文块,S 是 s 的串联。考虑一个终端应用(比如 telnet),这里每个按键作为独立的记录发送。Alice 向这个应用输入密码,Eve 捕获这些加密记录。注意序列号对 Eve 是已知的,因为这部分记录是没有加密的(参见图 17.17)。现在不知何故 Eve 掌握了 Alice 的频道,可能是通过一些应用的回波特征。这意味着 Eve 可以向频道发送未加密记录并观察加密结果。提出一个 Eve 可以猜到 Alice 密码字符的暴力攻击方法。提示:利用异或的这些特性:$x \oplus x = 1$;$x \oplus 1 = x$。

17.6　WTLS 的一个早期版本支持一个 40 位异或 MAC 也支持 RC4 流加密。异或 MAC 工作机制是通过用 0 填充消息,并将其划分成 5 个字节的块,再将这些块异或。说明该方案没有提供消息完整性保护。

第 18 章　电子邮件安全

18.1　PGP
- 18.1.1　符号约定
- 18.1.2　操作描述
- 18.1.3　加密密钥和密钥环
- 18.1.4　公钥管理

18.2　S/MIME
- 18.2.1　RFC 5322
- 18.2.2　MIME
- 18.2.3　S/MIME 功能
- 18.2.4　S/MIME 消息
- 18.2.5　S/MIME 证书处理过程
- 18.2.6　增强安全性服务

18.3　DKIM
- 18.3.1　因特网邮件结构
- 18.3.2　电子邮件的威胁
- 18.3.3　DKIM 策略
- 18.3.4　DKIM 功能流

18.4　推荐读物和网站

18.5　关键术语、思考题和习题

附录 18A　基数 64 转换

Despite the refusal of VADM Poindexter and LtCol North to appear, the Board's access to other sources of information filled much of this gap. The FBI provided documents taken from the files of the National Security Advisor and relevant NSC staff members, including messages from the PROF system betueen VADM Poindexter and LtCol North. The PROF messages were conversations by computer, written at the time events occurred and presumed by the writers to be protected from disclosure. In this sense, they provide a first-hand, contemporaneous account of events.

—The Tower Commission Report to President Reagan on the Iran-Contra Affair, 1987

要　点

- PGP 是保障电子邮件安全性的开源软件包。它通过使用数字签名提供认证，通过使用对称分组加密提供保密性，用 ZIP 算法做压缩，并使用基数 64 编码模式提供电子邮件兼容性。
- PGP 融合了公钥信任模型开发和公钥证书管理的工具。
- S/MIME 是保障电子邮件安全的因特网标准方法，它集成了与 PGP 相类似的功能。
- DKIM 是电子邮件服务提供方用来在源域上加密和签名电子邮件消息的规范。

事实上，在所有的分布式环境中，电子邮件是最繁重的网络应用。无论双方使用何种操作系统和通信软件，用户都希望能够给直接或间接与因特网连接着的对方发送电子邮件。随着对电子邮件依赖性的爆炸式增长，认证性和保密性服务需求也在日益增长。两种被广泛应用的方法：PGP 和 S/MIME 脱颖而出。本章将阐述这两种方法，并在最后谈论 DKIM。

18.1　PGP

主要由 Phil Zimmermann 提出的 PGP 提供了可用于电子邮件和文件存储应用的保密性和认证性。基本上，Zimmermann 做了如下工作：

(1) 在构建数据块时,选用了最合适、可用的密码算法。
(2) 将这些算法整合到通用应用程序中,这些应用程序独立于不同操作系统和处理器,并且是基于一个小规模的易用指令集。
(3) 将其制作成软件包并形成文档,包括源代码。这些资源都可以通过因特网,广告牌和诸如 AOL(America On Line)的商用网络免费获取。
(4) 与 Viacrypt(现在的 Network Associates)公司达成协议,提供完全兼容、低成本的商用版 PGP。

PGP 的应用迅速普及,呈爆炸式增长。其原因大致可归纳如下:

(1) 提供了在世界范围内的各种免费可用的版本,可运行于各种平台,包括 Windows,UNIX,Macintosh 等。另外,其商用版满足了用户的需求并获得了商家的支持。
(2) 使用的算法是经过广泛的使用检验并被认为是非常安全的算法。值得指出的是,这个软件包中包含了用于公钥加密的 RSA,DSS 和 Diffie-Hellman 算法,用于对称加密的 CAST-128,IDEA和 3DES 算法以及用 SHA-1 做 Hash 编码。
(3) 应用范围广泛,既可用于公司选择和增强加密文件与消息的标准化模式,也可用于个人通过因特网或其他网络进行安全通信。
(4) 不是由政府或者是标准化组织开发或控制。对那些本能地对上述"组织"有不信任的人们来说,PGP 非常有吸引力。
(5) PGP 现已成为因特网标准文档(RFC 3156)。同时,PGP 还有着些许通过个人努力并获得成功的色彩。

我们从整体上介绍 PGP 的操作开始,接着阐述如何生成和存储加密密钥,最后说明公钥管理中最重要的问题。

18.1.1　符号约定

本章中所使用的符号大多数在之前的章节中也已经被使用过,但是也出现了一些新的术语。因此我们在本章的开始就总结这些将用到的符号可能更加合适。本章中使用了如下的符号:

K_s 为会话密钥,用于对称加密
PR_a 为用户 A 的私钥,用于公钥加密
PU_a 为用户 A 的公钥,用于公钥加密
EP 为公钥加密
DP 为公钥解密
EC 为对称加密
DC 为对称解密
H 为 Hash 函数
‖ 为串联
Z 为用 ZIP 算法压缩
R64 为转换成基数 64 的 ASCII 码格式①

PGP 文档中常使用术语"密钥"(secret key)来表示在公钥加密模式下与公开密钥相对的私有密钥。正如前文所提到的,这种提法可能会和对称加密中的"密钥"的概念相混淆。因此,在这里我们使用私钥(private key)代替。

① 美国信息交换标准码(ASCII)的描述可参阅附录 Q。

18.1.2 操作描述

与密钥管理相对应,PGP 的实际操作由 4 类服务组成:认证(数字签名)、保密(消息加密)、压缩和电子邮件兼容性(如表 18.1 所示)。下面我们一一介绍。

表 18.1 PGP 服务概述

功 能	使用的算法	描 述
数字签名	DSS/SHA 或 RSA/SHA	用 SHA-1 生成消息的散列(Hash)码,采用 DSS 或者 RSA 算法,用发送者的私钥加密该消息摘要并将其加入消息中
消息加密	CAST 或 IDEA 或 3DES, Diffie-Hellman 或 RSA	采用 CAST-128 或 IDEA 或 3DES 算法,使用由发送者生成的一次性会话密钥加密消息。使用 Diffie-Hellman 算法或者是 RSA 算法中接收者的公开密钥加密该会话密钥并将其加入到消息中
压缩	ZIP	使用 ZIP 算法来实现消息压缩以便存储或传输
电子邮件兼容性	基数 64 转换	为电子邮件应用程序提供透明性,一个加密的消息可能会用基数 64 转换成一个 ASCII 码字符串

认证

图 18.1(a)描述了 PGP 提供的数字签名服务。该数字签名方案在第 13 章和图 13.2 讨论过。过程如下:

(1) 发送方生成消息。

(2) 用 SHA-1 算法生成消息的 160 位散列(Hash)码。

(3) 用发送方的私钥按 RSA 算法加密该散列码并将结果预先加入到消息中。

(4) 接收方用发送方的公钥按 RSA 算法解密消息并恢复散列码。

(5) 接收方对该消息用相同的散列函数生成新的散列码,并比较该散列码与解密得到的散列码,如果两者吻合则该消息为可信的。

SHA-1 和 RSA 的结合提供了一个高效的数字签名方案。鉴于 RSA 的安全强度,接收者可以确信只有对应私钥的拥有者才能生成该签名。鉴于 SHA-1 的安全强度,接收者可以确信如果是不同的消息必定无法生成与该散列码匹配散列值。于是保证了原消息签名有效。

DSS/SHA-1 也是生成数字签名的可选方案。

尽管通常情况下签名都会附在消息或者是文件中,但也不尽然:签名也可以被分离出来。一个分离式签名可能会被与对应的消息分开存储或传输。这在一些情形下是有用的,例如某用户可能希望能为所有发送或接收到的消息的各分离式签名建立并保持一个日志。一个可执行程序的分离式签名可以用来甄别该程序是否被病毒感染。另外,分离式签名还可以用于多个成员对一份诸如合法契约等文件进行签名,每一个成员的签名都是独立的并且只用于该文件,否则签名可能会被嵌套,例如第二个成员会对文档和第一个成员生成的签名进行签名等。

保密

PGP 提供的另一个服务是保密,这是通过对传输的消息或者是本地存储的文件进行加密来实现的。在如上两种情形下对称加密算法 CAST-128 是可用的。同时,IDEA 和 3DES 也是可用的。使用 64 位密文反馈模式(CFB)。

通常,涉及加密时必须讨论密钥分配的问题。在 PGP 中,每一个对称密钥都只使用一次,这意味着对每一个消息都会随机生成一个 128 位的自然数用做新密钥。因此,尽管这在标准文档中是指会话密

钥,其实是一次性密钥。因为它只使用一次,所以会话密钥会和消息绑定并随之一起传输。用接收者的公开密钥加密以保护该会话密钥。图 18.1(b)描述了这个流程,用文字描述如下:

(1) 发送者生成消息和仅用于加密该消息的会话密钥,该会话密钥为 128 位随机自然数。
(2) 采用 CAST-128(或 IDEA 或 3DES)算法,使用该会话密钥加密消息。
(3) 采用 RSA 算法,用接收者的公开密钥加密该会话密钥并预先发送给对方。
(4) 接收者采用 RSA 算法,用自己的私钥解密并恢复会话密钥。
(5) 用会话密钥解密消息。

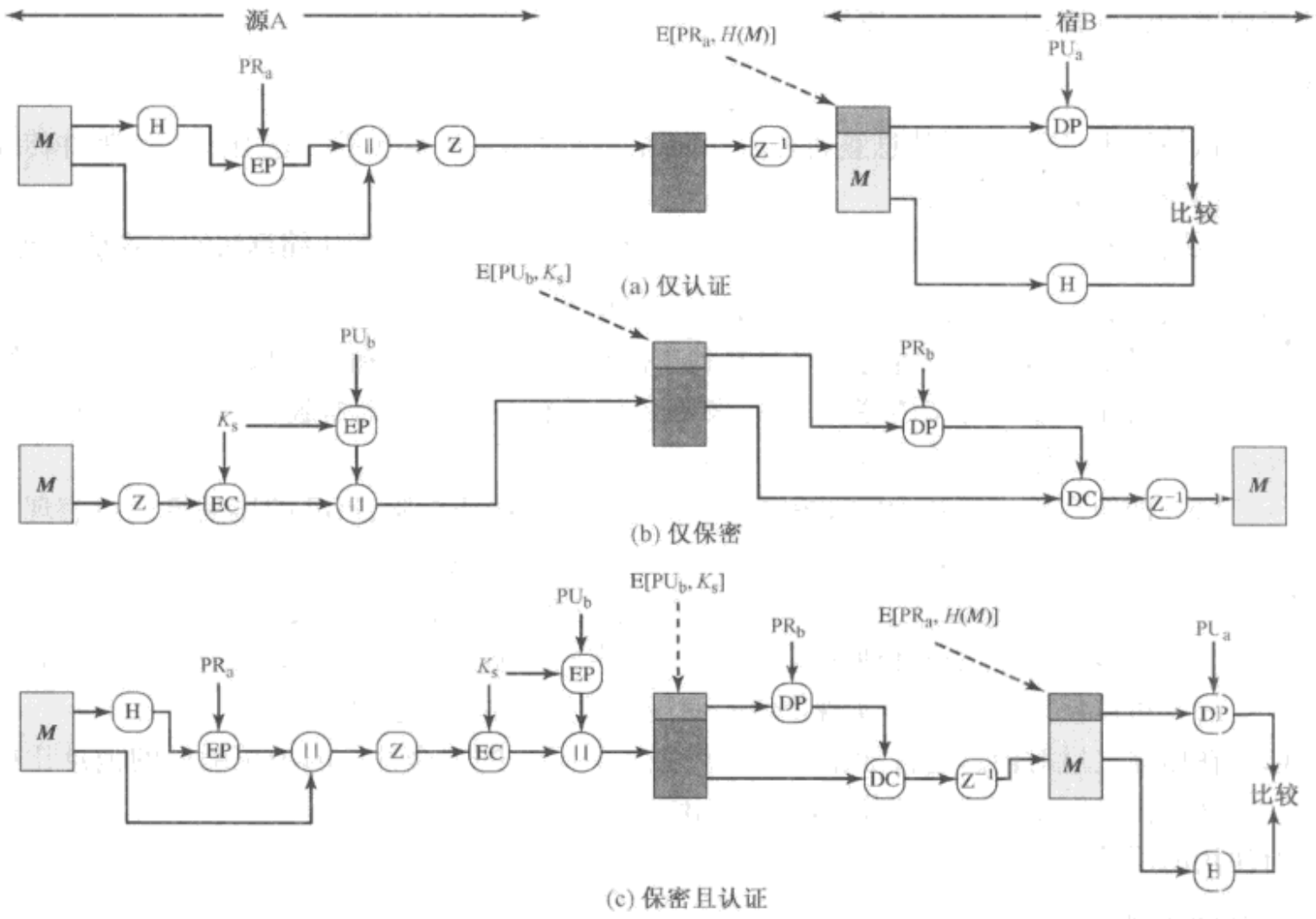


图 18.1　PGP 的密码功能

作为用于对密钥进行加密的 RSA 算法的替换,PGP 还提供了 Diffie-Hellman 算法作为选择。正如在第 10 章中所介绍的那样,Diffie-Hellman 是一个密钥交换算法。事实上,PGP 使用了大量的 Diffie-Hellman 来提供加/解密,例如第 10 章中介绍的 ElGamal 算法。

上述过程中我们会发现一些问题。首先,为了减少加密时间,对称加密和公钥加密的结合在效率上优于仅仅使用 RSA 或者 ElGamal 直接对消息进行加密:CAST-128 和其他对称加密算法远远快于 RSA 或 ElGamal。其次,公钥算法的使用解决了会话密钥分配的问题,因为只有接收者才能还原与消息绑定的会话密钥。值得注意的是,在这里我们不需要使用像在第 14 章中讨论的会话密钥交换协议,因为我们不会从正在进行的会话中开始,而是对每一个消息都使用独立的一次性密钥。此外,鉴于电子邮件存储转发的特性,使用握手协议来确保通信双方拥有相同的会话密钥也是不切实际的。最后,使用一次性对称密钥也使原本安全强度较高的对称加密方法更安全。每一个密钥都只加密少量的明文,并且密钥与密钥之间没有任何关联。因此,在某种意义上说只要公钥算法是安全的,那么整个方案就是安全的。同时,PGP 为用户提供了密钥长度从 768 位到 3072 位范围之间的选择(用于数字签名的 DSS 密钥限制在 1024 位)。

保密性和认证性

如图 18.1(c)所描述,保密性和认证性这两种服务可能会对同一个消息使用。首先,生成一个明文的签名,然后用 CAST-128(或 IDEA 或 3DES)对明文消息和签名进行加密,然后用 RSA(或 ElGamal)对会话密钥加密。这个流程比先加密消息然后生成密文的签名的过程要好。这样更便于保存明文消息的签名。此外,在需要第三方验证时,假如签名在一开始就进行,这样第三方在不需要考虑对称密钥的情况下就可以进行签名的验证。

总之,当两种服务都使用时,发送者先用自己的私钥签名消息,然后用会话密钥加密消息,最后用接收者的公开密钥加密会话密钥。

压缩

在默认情况下,PGP 在对消息签名之后和加密之前对消息进行压缩。这对于电子邮件传输和文件存储的空间节省大有好处。

用于压缩的 Z 和用于解压的 Z^{-1}(如图 18.1 所示)所表示的压缩/解压缩算法的放置是关键的。

(1) 生成签名在压缩之前进行是由于如下两个原因:

(a) 对未压缩的消息进行签名更有利于仅存储未压缩消息和签名以便未来的签名验证。假如压缩后对文档签名,则将来要么存储消息的一个压缩版,要么在需要时对消息重新压缩以便验证签名。

(b) 即使有用户愿意为验证签名动态的生成消息的压缩版,PGP 的压缩算法也表现出不合适。因为该算法是不确定的,在运行速度和压缩比之间权衡时会产生该算法各种不同形式的实现。但是,这些不同的算法实现是可以同时使用的,因为该算法的任何版本都可以正确地解压任何其他版本的压缩文件。在 Hash 函数和签名之后使用可以使所有的 PGP 实现都统一使用一种版本的压缩算法。

(2) 在压缩之后对消息进行加密是为了增强密码运算的安全性。因为压缩后的消息比原明文的信息冗余少,这让密码分析更加困难。

所使用的压缩算法 ZIP 在附录 O 中有介绍。

电子邮件兼容性

当使用 PGP 时,数据块中至少被传输的那部分会被加密。假如使用了数字签名服务,那么消息的摘要会用发送者的私钥加密。假如使用了保密服务,那么消息和签名(如果有签名的话)会用一次性对称密钥加密。因此,数据块的整体或部分会由任意的字节流组成。但是大多数的电子邮件系统都只允许使用由 ASCII 文本组成的数据块。为了适应该限制,PGP 提供了将原 8 字节流转换成可打印的 ASCII 字符流的服务。

基数 64 就是用于该目的的解决方案。每三个字节组成的二进制数据可以映射成 4 个 ASCII 字符。这种格式也附加了 CRC 校验码用来检查传输错误。详见附录 18A。

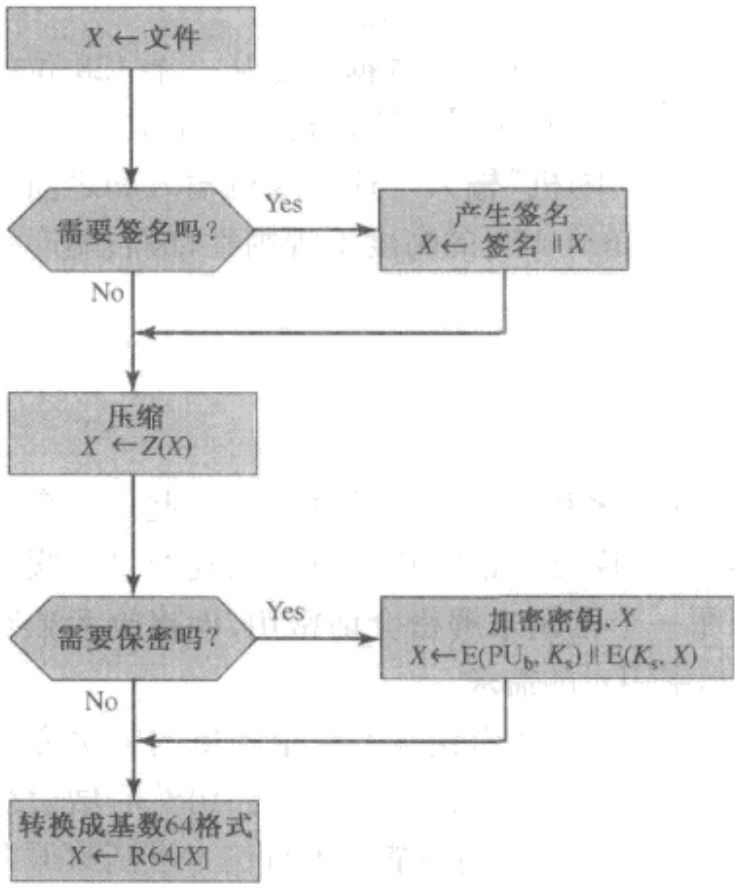
基数 64 的使用使消息长度增加了 33%。幸运的是,会话密钥和消息的签名部分相当紧凑,并且明文消息也已被压缩。事实上压缩的过程在弥补基数 64 带来的扩张之外还能使消息长度有所减短。例如,[HELD96]报告了 ZIP 的平均压缩比为 2.0。假设我们忽略相当短的签名和密钥部分,一个长为 X 的文件的典型的压缩和扩张效果可以表示成 $1.33 \times 0.5 \times X = 0.665 \times X$。因此在整体上仍然有三分之一的压缩效果。

基数 64 算法有个值得一提的特性是,它能将任意的输入流转换成基数 64 的格式,不管其内容是什么,即使输入流恰巧也是 ASCII 文本。因此,假如对整个数据块的压缩在对消息签名之后

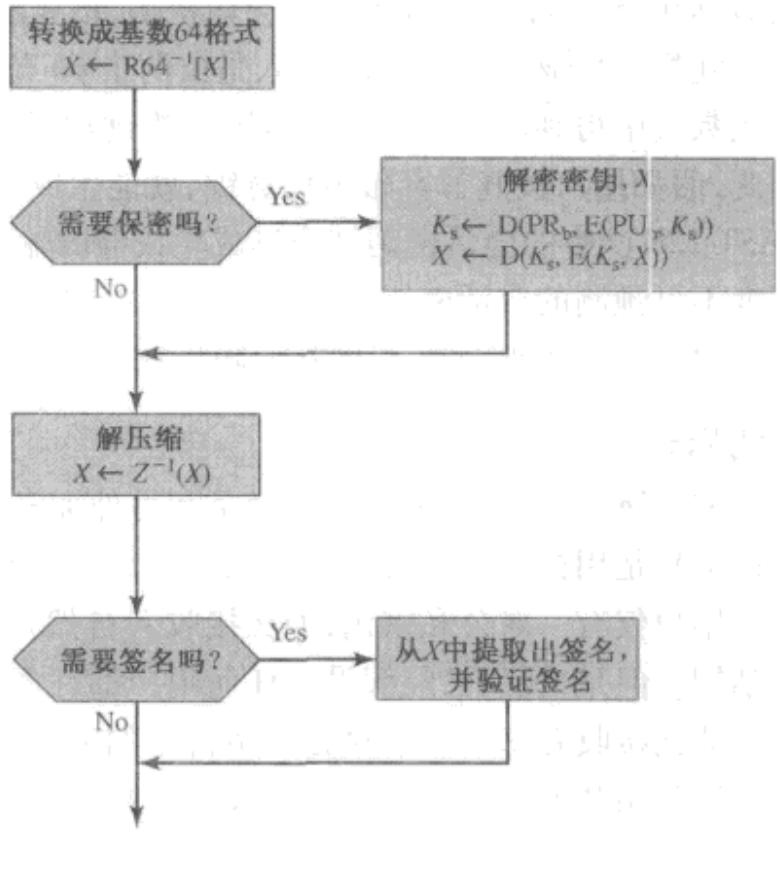
并且是加密之前进行，那么对于一些偶然的侦听者而言输出流将是不可读的，这在一定程度上也提供了保密性。还有一种观点，PGP 可以配置成只对签名的明文部分做基数 64 转换，这样接收者就可以在不使用 PGP 的情况下读取消息的内容，并且 PGP 仍然可以提供签名的验证。

图 18.2 显示了到目前为止讨论的 4 种服务之间的关系。在传输方，假如需要可以生成一个明文的散列码用做签名。然后压缩明文（如果有签名就加上签名）。接着，如果有保密性要求，这个数据块（压缩了的明文或者是压缩了的签名和明文的集合）就用对称密钥加密，而该对称密钥则是预先用公钥加密的。最后，将整个数据块转换成基数 64 格式。

在接收方，首先把接到的数据块由基数 64 格式转换成二进制。然后，假如消息是加密的，那么接收者就用自己的私钥恢复出会话密钥并用会话密钥解密消息。然后把得到的结果解压。假如消息是签名的，那么接收者就恢复出传输过来的散列码并与自己计算的散列码进行比较。



(a) 一般发送处理（从A）



(b) 一般接收处理（到B）

图 18.2　PGP 的消息传送与接收处理

18.1.3　加密密钥和密钥环

PGP 使用了 4 种类型的密钥：一次性会话对称加密密钥、公开密钥、私有密钥和基于口令的对称密钥（passphrase-based symmetric keys）（稍后介绍）。对于这些密钥定义了三种不同的需求：

（1）需要有生成不可预测的会话密钥的方法。

（2）应该允许用户有多个公私密钥对。其中一个原因是用户可能需要时常更换其密钥对。一旦更换了密钥，那么之前传输在双方的任何消息都是由陈旧的密钥构建的。而且只有当接收者收到了密钥的更新信息才会知道对方之前的公开密钥已过时。用户不仅仅有经常更换密钥的需求，同时，用户需要多个公私密钥对是因为在同一时刻某用户可能会和不同群中的对方进行交互或者仅仅是为了限制一个密钥加密信息的数量来提升安全性。以上的分析结果表明用户和公钥之间不应该是一对一的关系。由此，也需要有鉴别不同的密钥对的方法。

(3) 每一个 PGP 用户都必须建立并维持一份自己的公私密钥对文件以及别人的公钥集合的文件。

下面我们依次仔细研究上述的每个需求。

生成会话密钥

每一个会话密钥都和一个消息相关联并且仅用于该消息的加解密。如之前所述,消息的加解密是用对称密码算法进行的。CAST-128 和 IDEA 用 128 位密钥;3DES 用 168 位密钥。为下面方便讨论,我们假设以 CAST-128 为例。

随机的 128 位自然数是由 CAST-128 本身生成的。随机自然数生成器的输入是 128 位的密钥和两个 64 位,即将被加密的明文数据块。使用密文反馈模式,CAST-128 加密器产生两个 64 位密文数据块,将之串联形成 128 位会话密钥,该算法基于 ANSI X12.17 文档规范。

随机数生成器的"明文"输入是由两个 64 位数据块组成的,这两个数据块是从一个 128 位的随机数流中得到的。这些数是由用户按键输入的,用按键时间和实际的按键行为来生成随机流。因此,假如用户按其节奏敲击任意键,就能生成一个合理的"随机"输入。这个随机输入和之前用 CAST-128 产生的会话密钥结合形成生成器的输入。鉴于 CAST-128 的高度不规则性,最后能产生高度不可预测的会话密钥序列。

附录 P 详细地介绍了 PGP 的随机数生成技术。

密钥标志

如上所述,一个加密了的消息是和另外加密了的用来加密该消息的会话密钥在一起的。会话密钥本身是用接收者的公钥加密的。因此只有接收者才能恢复会话密钥并以此恢复消息。假如每个用户仅有一对公私密钥,那么接收者自然知道用哪一个密钥来解密会话密钥,即接收者独有的私钥。但是,我们已经假设了任何用户都有多个公私密钥对的需求。

那么接收者怎么知道对方是用自己的哪一个公钥加密的会话密钥呢?一个简单的解决方案是将该公钥和消息一起传输给接收者。接收者验证这个公钥确实是自己的一个公钥然后做后续处理。这个方案是可行的,但是却额外地浪费了空间。一个 RSA 公钥可能有上百位十进制数那么长。另一个解决方案是给每一个公钥都建立一个相应的标志,该标志至少对该用户而言是唯一的。于是用户标志和密钥标志结合就可以有效地唯一标志一个密钥。因此,只需传输长度短了很多的密钥标志。但是这个方案也带来了管理和开销的问题:密钥标志必须被分配并存储以便发送者和接收者能从密钥标志映射出公开密钥。这看起来也是无谓的负担。

PGP 采用的解决方案是给每一个公钥都分配一个密钥标志,并且该密钥标志在一个用户标志[①]内很大程度是唯一的。与公钥相关联的密钥标志至少由 64 位组成,即公开密钥 PU_a 的密钥标志是($PU_a \bmod 2^{64}$)。为了使出现相同密钥标志的概率很小,这个长度已经足够了。

PGP 的数字签名也需要一个密钥标志。因为发送者可能会使用多个私钥中的一个来加密(签名)消息,于是接收者必须知道使用哪一个对应的公钥来验证。因此消息的数字签名的成分中也包含了所需公钥的 64 位密钥标志。当接收到消息时,接收者确认这个密钥标志对应的发送者的公钥,然后进行签名验证。

如上我们介绍了密钥标志的概念,接着将更仔细地描述消息传输的格式(如图 18.3 所示)。

---

① 之前在 8.3 节中我们也介绍过类似的概率概念,即确定一个自然数是否为素数。在设计算法时也会类似用概率方法来设计一个时间消耗较小或计算复杂度较低的算法。

一个消息由三部分组成:消息成分、签名成分(可选)和会话密钥成分(可选)。

消息成分包含实际用于存储或传输的数据、文件名和标志创建的时间戳。

签名成分包含如下内容:

- **时间戳**:签名生成的时刻。
- **消息摘要**:用发送者签名私钥加密的 160 位 SHA-1 摘要。这个摘要是在时间戳和消息的数据部分的基础上计算的。在签名中引入时间戳是为了抵抗重放类攻击。不加入消息的文件名和消息创建的时间戳是为了保证分离后的签名和分离前保持一致。因为分离式的签名是在独立的文件中计算的,该文件没有任何原消息成分中的头文件域。
- **消息摘要的前两个字节**:通过比较明文副本中前两个字节和解密后摘要的前两个字节,接收者可以确定是否使用了正确的公钥来解密消息摘要以实现认证功能。这两个字节也就作为消息的 16 位验证序列。
- **发送者公钥的密钥标志**:标志了用来解密消息摘要的公钥,同时也标志了用来加密消息生成摘要的私钥。

消息成分和可选的签名成分可能用 ZIP 压缩并用会话密钥加密。

会话密钥成分包含会话密钥和加密会话密钥的公钥的标志,该公钥为接收者的某一公钥并被发送者用来加密会话密钥。

整个数据块通常用基数 64 方案编码。

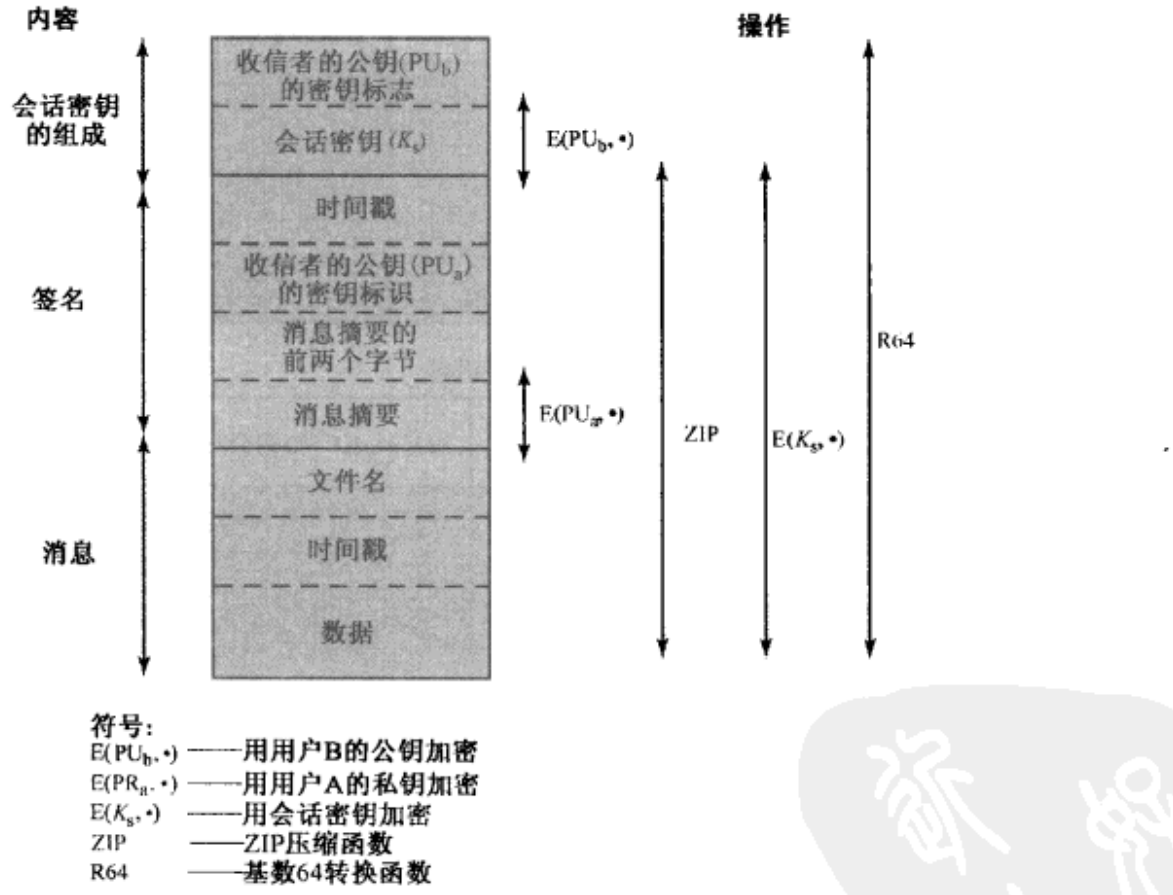


图 18.3　PGP 消息的一般格式(从 A 到 B)

密钥环

我们已经看到了 PGP 操作中密钥标志的重要性以及在任何一个 PGP 消息中都包含着用来提供保密性和认证性的两个密钥标志。这些密钥应该用系统的方法组织和存储以便各方高效使用。PGP 采用的方案是为每一个节点提供一对数据结构,一个用于存储该节点自身的公私密钥对,另一个用于存储该节点所知道的其他用户的公钥。这些数据结构分别是私钥环和公钥环。

图 18.4 显示了私钥环的大体结构。我们可以把这个环看成一张表,表中的每一行表示用户拥有的一对公私密钥对。并且每一行都包含如下实体:

- **时间戳**:密钥对生成的日期/时间。
- **密钥标志**:该实体的公钥至少需要 64 位的标志。
- **公开密钥**:密钥对中公钥部分。
- **私有密钥**:密钥对中私钥部分,该部分是加密的。
- **用户标志**:通常这是指用户的电子邮箱地址(如 stallings@ acm. org)。但是,用户可能会选择使用不同的名字来关联每一个密钥对(如 Stallings, WStallings, WilliamStallings 等)或者多次使用同样的用户标志。

可以通过用户标志或者密钥标志来检索私钥环,随后我们将看到这两种检索方法所需的条件。

尽管理论上认为私钥环是仅存放在创建并拥有该密钥对的用户机器上并且只有该用户可以访问,但是让私有密钥尽可能的保密安全的考虑仍然是值得的。如上所述,私有密钥本身并没有被直接存放在密钥环中,而是用 CAST-128(或 IDEA 或 3DES)加密后存放。其过程如下:

(1) 用户选择一个口令用来加密私有密钥。

(2) 当系统采用 RSA 生成一个新的公私密钥对时就会询问用户口令。采用 SHA-1 将口令生成一个 160 位的散列码并丢弃口令本身。

(3) 系统采用 CAST-128 以散列码中的 128 位作为密钥加密私钥,丢弃(2)中生成的散列码并将加密后的私钥存放在私钥环中。

私钥环

时间戳	密钥标志*	公钥	加密的私钥	用户标志*
⋮	⋮	⋮	⋮	⋮
T_i	$PU_i \bmod 2^{64}$	PU_i	$E(H(P_i), PR_i)$	用户 i
⋮	⋮	⋮	⋮	⋮

公钥环

时间戳	密钥标志*	公钥	所有者的信任度	用户标志*	密钥合理性	签名	签名信任度
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
T_i	$PU_i \bmod 2^{64}$	PU_i	$trust_flag_i$	用户 i	$trust_flag_i$		
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

* ——用于索引表的域

图 18.4 私钥环和公钥环的一般结构

随后,当用户需要访问私钥环并且恢复私有密钥时,他必须提供口令,PGP 首先生成口令的散列码,采用 CAST-128 算法,以该散列码为密钥解密所需私有密钥的密文即可。

这是一个简洁而高效的方案。和其他任何基于口令的系统一样,这个系统的安全性也依赖于口令的安全性。为了避免将口令写下来,用户应该选择一个既不容易被猜到又易于自己记忆的口令。

图 18.4 同时给出了公钥环的大体结构。该数据结构是用来存储本用户所知道的其他用户的公开密钥。现在我们先重点讨论其中的一些数据域。

- **时间戳**:该实体生成的日期/时间。
- **密钥标志**:该实体的公钥至少需要 64 位的标志。

- **公开密钥**:该实体的公钥。
- **用户标志**:标志该密钥的所有者。多个用户标志可能会对应一个公钥。

可以通过用户标志或者密钥标志来检索公钥环,随后我们将看到这两种检索方法所需的条件。

现在我们来讨论密钥环是如何在消息传输和接收中使用的。为方便讨论,先省去压缩和基数 64 转换的过程。首先考虑消息传输(如图 18.5 所示)并假设消息是要签名和加密的。PGP 的发送实体需要执行如下步骤。

(1) 消息签名:

(a) PGP 以“your_uerid”作为索引将发送者的私钥从私钥环中恢复出来。若未提供“your_userid”则恢复密钥环中的第一个私钥。

(b) PGP 提醒用户输入口令以恢复出私钥。

(c) 执行消息的签名组件。

(2) 消息加密:

(a) PGP 生成会话密钥并加密消息。

(b) PGP 以“her_userid”为检索将接收者公钥从公钥环中取出。

(c) 执行消息的会话密钥组件。

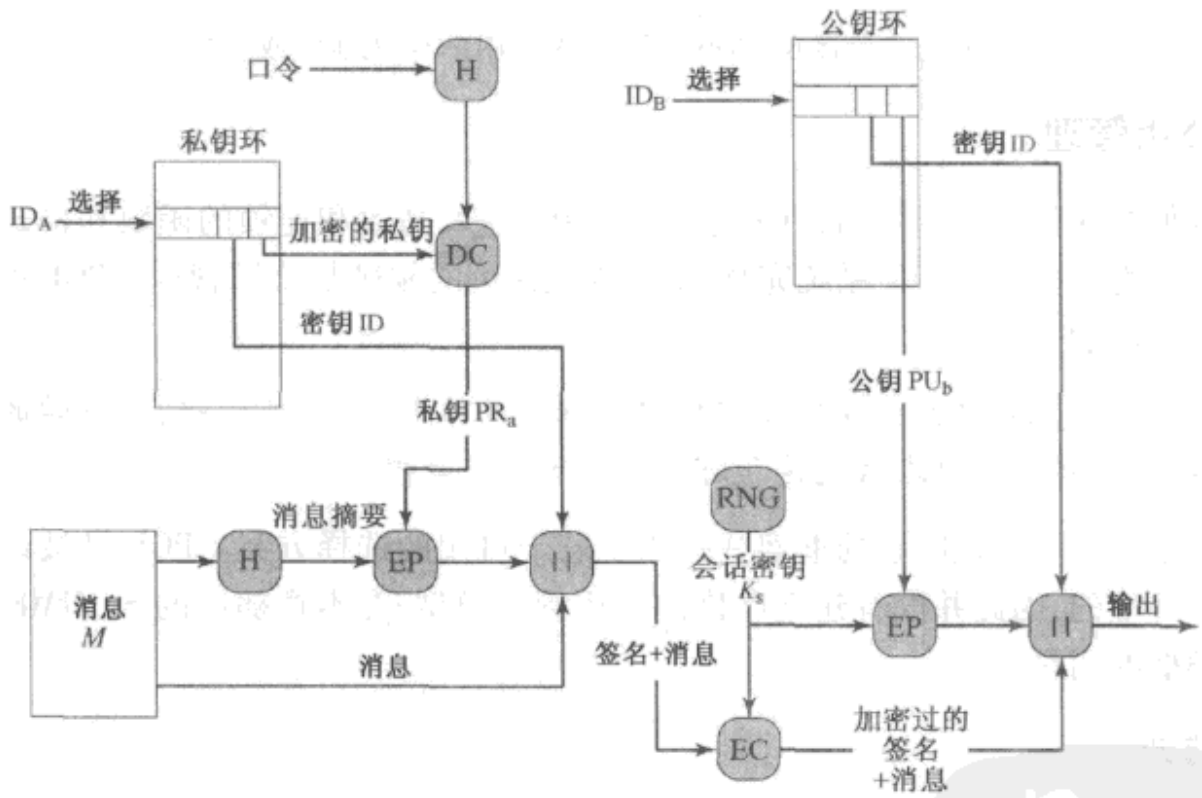


图 18.5　PGP 的消息产生(从用户 A 到用户 B:无压缩或基数 64 转换)

PGP 的接收实体需要执行如下步骤(如图 18.6 所示)。

(1) 消息解密:

(a) PGP 以会话密钥组件中的密钥标志域为索引将接收者的私钥从私钥环中取出。

(b) PGP 提醒用户输入口令以恢复出私钥。

(c) PGP 恢复出会话密钥并解密消息。

(2) 消息认证:

(a) PGP 以签名密钥组件中密钥标志域为索引将发送者的公钥从公钥环中取出。

(b) PGP 恢复出传输过来的消息的摘要。

(c) PGP 对接收到的消息计算摘要并将之与(b)中恢复的摘要进行比较以认证消息。

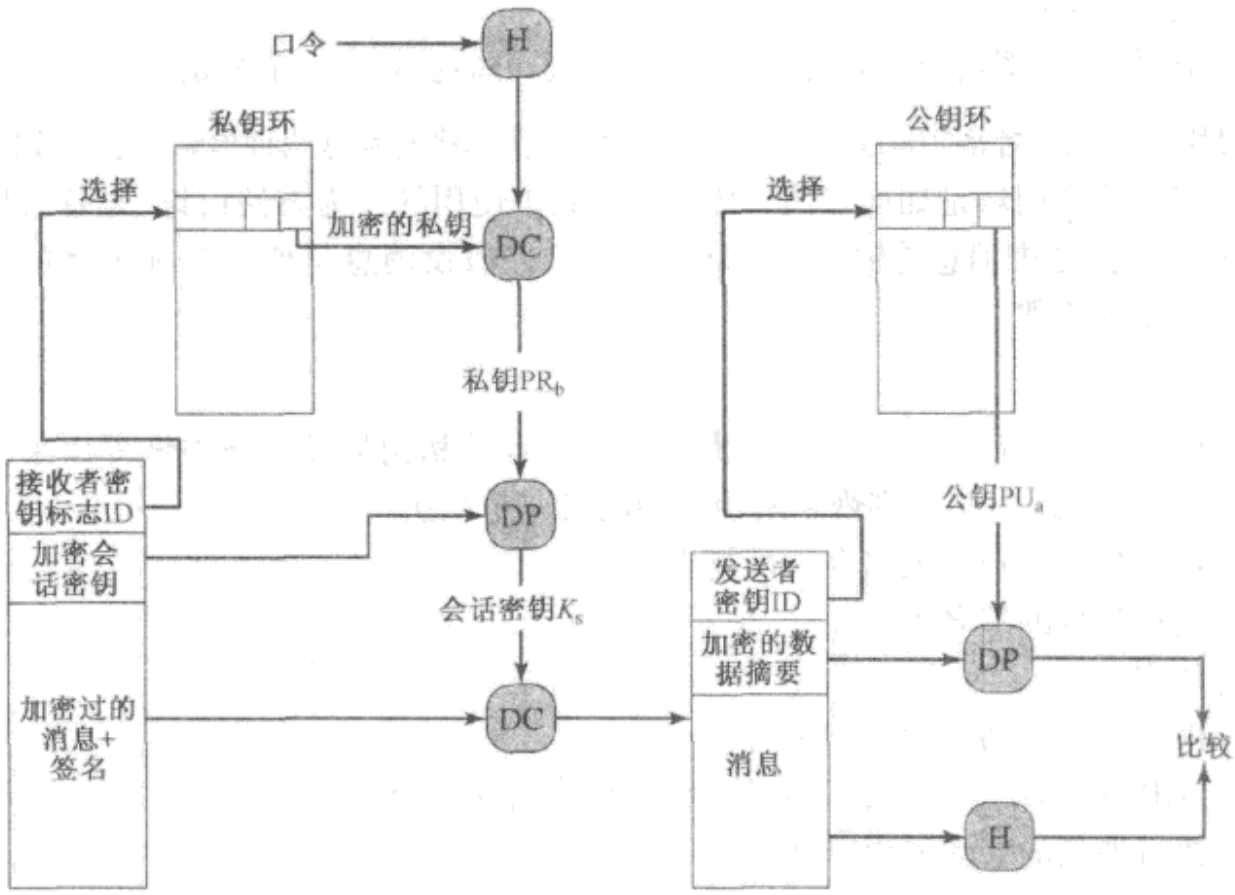


图 18.6 PGP 的消息接收处理(从用户 A 到用户 B:无压缩或基数 64 转换)

18.1.4 公钥管理

从以上我们的讨论中可以看出,PGP 包含了一组巧妙、高效和连锁的函数和格式来保证有效的保密性和认证性服务。为了使系统完整,最后需要讨论的领域是公钥管理。PGP 的标准文档已经抓住了这个领域的重要性:

> 保护公开密钥防止被篡改是实际公钥应用中唯一的难题。这也是公钥密码学的关键和要害所在,许多的软件复杂性也是和解决这个问题密切相关的。

PGP 提供了一个解决该问题的框架并提供了若干可用的选择方案。PGP 是被设计用在各种正式和非正式环境的,因此并没有建立严格的公钥管理模型,在本章随后的 S/MIME 的讨论中我们也能看到类似的问题。

公钥管理的方法

公钥管理的实质是:用户 A 为了使用 PGP 和其他用户进行交互操作,必须建立一个包含其他用户的公钥的公钥环。假设用户 A 的密钥环中含有属于 B 的公钥,但是,事实上该公钥却是用户 C 的。这种情况如 A 从 B 用来发布自己公钥的 BBS 上获得了这个密钥,但是这个 BBS 已经被 C 入侵了。这样的结果存在两种威胁:首先,C 可以仿造 B 的签名并把消息发送给 A,A 接收到消息并以为该消息来自 B;其次,任何从 A 加密发给 B 的消息 C 都可以读到。

有许多方法可以降低用户公钥环中包含有假公钥的风险。假设 A 希望获得一个可靠的 B 的公钥,可以采用如下的一些方法:

(1) 物理上从 B 处获得密钥。B 可以将自己公钥(PU_b)存放在软盘上并将软盘交给 A。然后 A 把密钥从软盘中加载到自己的系统上。这是一种非常安全的方法,但是很明显有实用性局限。

(2) 用电话验证密钥。假如 A 可以在电话上和 B 取得联系,那么 A 就可以通过电话向 B 询问基数 64 格式的密钥。一种更实际的选择是,B 把自己的密钥以电子邮件的方式发送

给 A。A 用 PGP 生成密钥的 160 位 SHA-1 摘要并将其以十六进制的形式显示，这就类似于密钥的“指纹”。然后 A 通过电话询问 B 该“指纹”。假如这两个“指纹”匹配，则该密钥得到验证。

(3) 从相互都信任的第三方用户 D 处获得 B 的密钥。为此，用户 D 就需要生成一个签名证书。这个证书包含 B 的公钥以及该密钥生成的时刻和该密钥的有效期。D 生成该证书的 SHA-1 摘要，并用自己的私钥将其加密，然后把签名附于证书上。因为只有用户 D 才能生成签名，于是就没有其他用户可以生成假公钥并冒充 D 的签名。这个签名证书可以由 B 或 D 直接发送给 A 或者也可以在 BBS 上公布。

(4) 从可信的认证机构获取 B 的公钥。同样，由该中心生成公钥证书并签名。然后 A 可以访问该公钥证书，提供自己的用户名并获取签名证书。

如第 3 种和第 4 种方案，用户 A 已经可以获取第三方提供的公钥副本并相信该公钥是合法的。本质上，这就取决于 A 对于第三方的信任程度。

使用信任机制

尽管 PGP 并没有建立认证机构或建立信任机制，但它却提供了一个使用信任机制的便利手段：将可信度与公钥联系并开发可信度信息。其基本结构如下：正如上文所述的那样，公钥环中的每一个记录都是一个公钥证书。和每条这样的记录项关联的是**密钥合法性域**，该域表示 PGP 认为该公钥合法的可信程度，可信程度越高，则用户标志和该密钥的绑定越紧密，该域是由 PGP 计算的。与每条记录相关联的还有拥有该密钥环的用户搜集到的签名，该签名是针对证书的。同时，每一个签名又关联了其**签名可信域**，该域表示了 PGP 用户认为该证书公钥签名的可信程度。密钥合法性域是从签名可信域的集合中计算获得的。最后，每一个记录定义了一个公钥与某特定用户的关联，并包含了**所有者可信域**，该域表示使用该公钥对其他公钥证书进行签名的可信度，而该可信度是由用户指定的。我们可以将签名可信域看做其他记录中所有者可信域的副本集合。

上文中提到的三个数据域都包含在称为信任标志字节的数据结构中。信任标志字节中的每个域的内容如表 18.2 所示。假设以用户 A 的公钥环为例，我们可以如下描述信任机制运行过程的操作。

(1) 当用户 A 将一个新的公钥插入到自己的公钥环中时，PGP 为信任标志指定一个值，该值与此公钥的所有者关联。假如所有者是用户 A，那么此公钥也将放入到私钥环中，并且自动将可信域赋值为 ultimate trust。否则，PGP 询问用户 A 对于此公钥所有者的信任度，然后 A 输入一个信任度值。A 可以指定该所有者为 unknown，untrusted，marginally trusted，或者 completely trusted。

(2) 当新的公钥输入时，会有一个或多个签名附在其上，并且更多的签名会在随后加入。当一个签名被插入到记录中时，PGP 会查询公钥环确认该签名的作者是否为已知的公钥所有者，若然，则将该所有者的签名可信域赋值为 ownertrust，否则将之赋值为 unknownuser。

(3) 密钥合法性域的值是依据记录中的签名可信域来计算的。假如至少有一个签名的签名可信域为 ultimate，则密钥合法性域的值即为 complete，否则，PGP 计算加权可信值。A 将 always trusted 的签名权值置为 $1/X$，给 usually trusted 的签名权值置为 $1/Y$，X 和 Y 是可以用户设置的参数。当某个对应密钥或用户标志的第三方的权值和为 1 时，认为该绑定为可信的，并且将密钥合法性域设置为 complete。因此，当值不为 complete 时，至少需要 X 个签名是 always trusted，Y 个签名是 usually trusted 或者需要其他的组合。

PGP 周期性的处理公钥环以使其维持一致性。本质上,这是一个自上往下的过程。对每一个 OWNERTRUST 域,PGP 都扫描密钥环中该所有者签写的所有签名并更新 SIGTRUST 域,使之与 OWNERTRUST 域相等。该过程从值为 ultimate trust 的密钥开始,然后所有的 KEYLEGIT 域都附在其后的签名计算中。

表 18.2 可信标志字节的内容

(a)公钥所有者的可信度(在密钥包后出现,由用户定义)	(b)公钥/用户标志对的可信度(在用户标志包后出现,由 PGP 计算)	(c)签名的可信度(在签名包后出现,该签名者的 OWNERTRUST 的备份)
OWNERTRUST 域 —未定义信任 —未知用户 —通常不信任对其他密钥的签名 —通常信任对其他密钥的签名 —一直信任对其他密钥的签名 —该密钥出现在秘密密钥环中(绝对信任)	KEYLEGIT 域 —未知或未定义信任 —密钥所有者身份不被信任 —密钥所有者身份绝对信任	SIGTRUST 域 —未定义信任 —未知用户 —通常不信任对其他密钥的签名 —通常信任对其他密钥的签名 —一直信任对其他密钥的签名 —密钥出现在秘密密钥环中(绝对信任)
BUCKSTOP 位 —若该密钥在私钥环中出现则置位	WARNONLY 位 —若仅当用户使用不合法的密钥加密时才被警报则置位	CONTIG 位 —若签名在信任证书链中被证实则置位

图 18.7 提供了一个签名可信度和密钥合法性关联的例子①。该图显示了公钥环的结构,用户获取了大量的公钥,其中有些是直接从所有者处获取,有些是从类似密钥服务器的第三方获取。

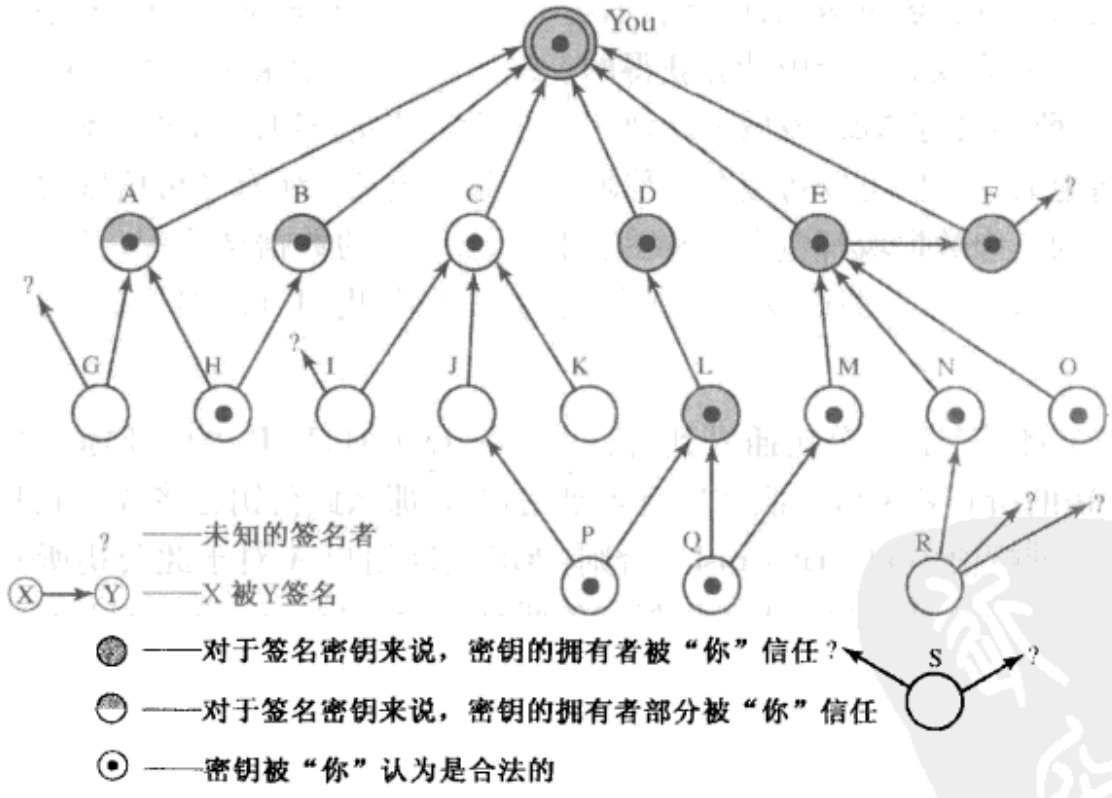


图 18.7 PGP 的信任模型举例

节点“You”表示公钥环中对应该用户的记录。该密钥是合法的,并且其 OWNERTRUST 域的值为 ultimate trust。密钥环中其他节点的 OWNERTRUST 域除用户指定外都置为 undefined。在这个例子中,该用户认为用户 D,E,F,L 对其他密钥的签名总是可信的,并且部分信任用户 A和用户 B对其他密钥的签名。

① 该图是由 Phil Zimmermann 提供。

如图 18.7 给出了该用户指定的可信等级。树结构也表明了哪些密钥被哪些其他用户签名过。假如一个密钥被一个其密钥也在该密钥环中的用户签名，则箭头就表示了从被签名的密钥到签名者。假如密钥是由其密钥不在该公钥环中的用户签名，则箭头表示从被签名的密钥到问号，表明签名者对该用户是未知的。

图 18.7 说明了以下几点：

(1) 除了节点 L 外，该用户对所有他认为完全或部分信任的用户的所有密钥都进行了签名。这样的用户签名并非总是必需的，节点 L 的存在就表明了这一点。但是在实际中，大多数用户倾向于为几乎所有他们信任的用户签名其密钥。因此，正如例子中所示，即使可信第三方 F 已经对 E 的密钥签名了，用户还是选择直接对 E 进行签名。

(2) 假设两个部分可信的签名可以足够证明一个密钥。于是，PGP 认为用户 H 的密钥是合法的，因为用户 A 和 B 对该密钥签名了，而 A 和 B 是部分可信的。

(3) 用户 A 的密钥会因为有一个完全信任或两个部分信任的签名者签名而被认为是合法的，但是，用户 A 对其他用户的签名却并不能被直接认为是可信的。例如，N 的密钥是合法的，因为用户 E 对其签名了，而该用户对 E 是信任的，但是 N 对其他密钥的签名却不可信，因为该用户并未给 N 赋可信度值。因此，尽管 R 的密钥是有 N 签名的，PGP 却并不认为 R 的密钥是合法的。该情形很有说服力。假如希望发送一个隐私消息，不需要对对方有任何程度的信任，只需确认你有对方正确的公钥即可。

(4) 图 18.7 还说明了拥有两个 unknown 签名的节点 S。这种密钥是从密钥服务器获取的。PGP 不会因其来自一个有信誉的服务器而认为该密钥是合法的。用户必须通过对其进行签名以声称该密钥合法或通过告诉 PGP 愿意完全信任该密钥的某个签名者。

最后要说明的是，正如前文中提到的，公钥环中的每一个公钥都可能与多个用户标志关联。这是因为用户修改了名字或者是使用多个名字签名，如一个人使用多个不同的电子邮件地址。因此可以将公钥看做是树的根节点。一个公钥有多个用户标志与之关联，而每个用户标志又有多个签名。一个特定的用户标志与密钥之间的绑定关系取决于用户标志和其密钥与签名的关联。反之，用来给其他密钥签名的密钥的可信程度则取决于所有相关签名。

公钥撤销

因为受到安全威胁或者是避免同一个密钥超期服役，用户可能希望撤销他当前的公钥。安全受到威胁是指攻击者获取了用户未加密的私钥副本或者同时获取了用户的私钥环和口令。

公钥撤销的通常惯例是密钥所有者发布一个由自己签名的密钥撤销证书。该证书和通常的签名证书形式相同，不同之处在于其包含了一个表明该证书的目的为撤销此公钥的使用。应该注意的是，对应的私钥应该用来签名撤销此公钥证书。此公钥的所有者应该尽快在大范围内发布其撤销公钥以使潜在的通信方能及时更新他们的公钥环。

有意思的是，一个获取了密钥所有者私钥的攻击者也可以发布这样的证书。但是这样做会让攻击者和合法所有者都无法使用公钥，因此这样看起来比攻击者恶意使用所窃取的私钥威胁性小一些。

18.2　S/MIME

S/MIME(Secure/Multipurpose Internet Mail Extension)在 RSA 数据安全性的基础上，加强了互联网电子邮件格式标准 MIME 的安全性。尽管 PGP 和 S/MIME 都是 IETF 工作组推出的标准，但是 S/MIME 倾向于推广为商业或组织用户的工业标准，而 PGP 则用来为个人电子邮件安全提供服务选

择。S/MIME 在很多的文档中被定义了,其中最重要的是 RFC 3370,RFC 3850,RFC 3851 和 RFC 3852。

为了理解 S/MIME,首先应对其使用的电子邮件格式(即 MIME)有一个大概的了解。但是为了理解 MIME 的重要性,又应回到传统的电子邮件格式标准(RFC 822),而该标准仍然广泛使用。最新版本的格式规范是 RFC 5322(互联网消息格式)。因此,本节我们将首先介绍那两个较早的标准,然后再开始讨论 S/MIME。

18.2.1 RFC 5322

RFC 5322 定义了用电子邮件发送的文本消息的格式。这已经是基于因特网的文本邮件消息的标准并且仍然在广泛使用。在 RFC 5322 中,消息被看做是一个信封和内容的组合。信封包含了任何用来完成传输和发送功能的信息。内容则构成了发送给接收者的对象。RFC 5322 标准关注的是内容部分。但是,内容标准包含了一系列报头域,这些报头域是由邮件系统用来生成信封的,而该标准也致力于使程序获取报头域信息更方便。

符合 RFC 5322 的消息的大体结构非常简单。一个消息包含了若干行的报头和无限制的正文。报头和正文中间用一空行隔开。另外,消息是 ASCII 文本,第一个空行之上的所有行都是邮件系统中用户代理部分使用的报头。

报头通常包含关键字、冒号和关键字的参数,该格式允许将一个长行分割成若干短行。使用最频繁的关键字是 From,To,Subject 和 Date。例如:

```
Date: October 8, 2009 2:15:49 PM EDT
From: "William Stallings" <ws@shore.net>
Subject: The Syntax in RFC 5322
To: Smith@Other-host.com
Cc: Jones@Yet-Another-Host.com

Hello. This section begins the actual
message body, which is delimited from the
message heading by a blank line.
```

在 RFC 5322 中常见的另一个域是消息标志(Message-ID),该域包含一个与该消息关联的唯一标志符。

18.2.2 MIME

MIME 是 RFC 5322 的扩展,目的是解决一些因使用简单邮件传输协议(SMTP,在 RFC 821 中定义)或其他一些邮件传输协议而产生的限制。[PARZ06]列出了 SMTP/5322 方案的限制。

(1) SMTP 不能传输可执行文件和其他二进制对象。SMTP 邮件系统使用了很多用来将二进制文件转换为文本格式的方案,如流行的 UNIX UUencode/UUdecode 方案。但是,没有一个成为标准或者事实上的标准。

(2) SMTP 不能传输含有国际语言字符的文本数据,因为这些字符是由 8 位码表示的,其值可能是十进制数 128 或者更大的数,而 SMTP 限于 7 位 ASCII 码。

(3) SMTP 服务器会拒绝超过一定大小的消息。

(4) ASCII 码和 EBCDIC 码之间转换的 SMTP 网关并未使用一致的映射集,因此常会出现转换错误。

(5) X.400 电子邮件网络的 SMTP 网关不能处理 X.400 消息中的非文本数据。

(6) 一些 SMTP 的实现并没有严格遵守 RFC 821 文档中定义的 SMTP 标准,因此会导致以下常见的问题:

- 回车和换行的删除、添加和重排
- 截断或者是限制超过 76 个字符长度的行
- 去除白字符(制表符和空格符)
- 将消息中的行填充为等长
- 将制表符转换成多个空格符

MIME 期望能解决上述这些问题并与现存的 RFC 5322 实现兼容。详细的文档规范参见 RFC 2045 到 RFC 2049

概述

MIME 规范包含以下要素:

(1) 定义了 5 个新的报头域,这些域提供了消息正文的相关信息(在 RFC 5322 报头中可能有定义)。
(2) 定义了一些新的内容格式,标准化的表示支持了多媒体的电子邮件。
(3) 定义了编码转换方式,以使邮件系统能将任何形式的内容统一(转换为同一种形式)。

在本小节中,我们将介绍这 5 个报头域,在随后的两节中将讨论内容格式和编码转换方式。MIME 中定义的 5 个报头域是:

- MIME-Version(MIME 版本):参数的值必须为 1.0,该域表明消息遵循 RFC 2045 和 RFC 2046 的格式。
- Content-Type(内容类型):详细描述了正文中的数据类型,以使接收方的代理可以采用合适的代理或机制来为用户展示数据或以一个合适方法来处理数据。
- Content-Transfer-Encoding(内容转换编码):描述消息正文转换为可以传输的类型转换方式。
- Content-ID(内容标志):在多重上下文中唯一标志 MIME 实体。
- Content-Description(内容描述):正文对象的文本描述,在该对象不可读时使用(如音频数据)。

通常在一个 RFC 5322 报头中会出现一个或多个上述域。一个合适的实现必须支持 MIME-Version,Content-Transfer-Encoding 域,接收方的实现可以选择使用或忽略 Content-ID 和 Content-Description 域。

MIME 内容类型

MIME 规范文档中有很大的篇幅是关于各种内容类型的定义。这也反映了在多媒体环境下提供处理各种信息展现形式的需求。

表 18.3 列出了 RFC 2046 中规定的内容类型。主要有 7 个不同的大类和总共 15 个子类。一般而言,用内容类型表示数据的通用类型,用子类型来表示该数据类型的特定格式。

若正文为**文本类型(text type)**,则除了要支持制定字符集外不需要特殊的软件来获取文字的全部含义,因此主要的子类型为纯文本(plain text),由简单的 ASCII 码或 ISO 8859 字符组成的字符串。子类型 enriched 则允许拥有更多的格式信息。

类型为**多部分类型(multipart type)**时,表示正文中包含多个相互独立的部分。域 Content-Type 中包含一个称为分界符的参数,该分界符定义了正文中各部分的分隔符。此分隔符不能随意出现,而必须从一个新行开始,并在两个连字符后跟分界符值,最后一个分界符表示最后一个部分的结尾,并有两个连字符做后缀。在每个部分,可以有一个通常的 MIME 报头。

表 18.3 MIME 内容类型

类 型	子 类 型	描 述
Text	Plain	无格式文本,为 ASSCII 码或 ISO 8859 码
	Enriched	提供丰富的文档信息
Multipart	Mixed	各部分相互独立但被一起传送。接收方看到的形式与邮件消息中形式相同
	Parallel	除了传送时各部分无序外,其余与 Mixed 相同
	Alternative	不同部分是同一消息的不同表现形式,按增序排列,接收方系统将按照最佳方式显示
	Digest	与 Mixed 相似,但每部分默认的类型/子类型为 message/rfc 822
Message	rfc822	封装消息的正文为 RFC 822 一致
	Partial	允许按接收方透明的方式对大邮件分段
	External-body	包含一个指针,该指针是对存在于别处对象的引用
Image	jpeg	JFIF 编码的 JPEG 格式图像
	gif	图像为 GIF 格式
Video	mpeg	MPEG 格式
Audio	Basic	单通道 8 位 ISDN,8 kHz 编码
Application	PostScript	Adobe Postscript 格式
	octet-stream	一般的 8 位二进制数据

以下是一个多正文消息的简单例子,包含由简单文本组成的两个部分(参见 RFC 2046):

```
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: Sample message
MIME-Version: 1.0
Content-type: multipart/mixed; boundary="simple
boundary"

This is the preamble. It is to be ignored, though it
is a handy place for mail composers to include an
explanatory note to non-MIME conformant readers.
—simple boundary

This is implicitly typed plain ASCII text. It does NOT
end with a linebreak.
—simple boundary
Content-type: text/plain; charset=us-ascii

This is explicitly typed plain ASCII text. It DOES end
with a linebreak.

—simple boundary—
This is the epilogue. It is also to be ignored.
```

Multipart 类型有 4 种子类型,其语法相同。**子类型 multipart/mixed** 用于将相互独立的各部分按一定的顺序绑定。**子类型 multipart/parallel** 表示各部分的顺序无关。如果接收系统合适,则各部分可并行接收。例如,在图片或文本显示时,可同时播放音频。

子类型 multipart/alternative 表示这若干个部分是同一个信息的不同表示。例如:

```
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: Formatted text mail
MIME-Version: 1.0
Content-Type: multipart/alternative;
boundary=boundary42
```

```
    –boundary42
Content-Type: text/plain; charset=us-ascii

    ...plain text version of message goes here....
–boundary42
Content-Type: text/enriched

    .... RFC 1896 text/enriched version of same message
goes here ...

–boundary42–
```

在此子类型中,各部分按优先级递增的方式排序。例如,如果接收方可以处理 text/enriched 格式的信息,则按此格式处理,否则就使用纯文本方式。

子类型 multipart/digest 用于将正文的每个部分作为一个带报头的 RFC 5322 消息,从而使得一个消息中可以包含多个独立的消息。例如,可以用一个组缓冲器搜集组中各成员的电子邮件消息并将其封装到一个 MIME 消息中发送。

类型 message 提供了许多 MIME 的重要特征。子类型 message/rfc822 表明该正文是一个完整的消息,包含了报头和正文。除了子类型的名字外,封装的消息既可以是 RFC 5322 消息,也可以是任何 MIME 消息。

子类型 message/partial 使得可以将一个大消息分段为若干部分,并可在目的地重新组装。对这个子类型而言,Content-Type:Message/Partial 域需要说明三个参数:一个对同一个消息所有分片一致的标志 id,一个每段唯一的序列号 sequence number 和总的段数 total。

子类型 message/external-body 表明消息所要表达的数据并未包含在该正文中。而是在正文中包含了对该数据的访问。和其他消息类型一样,子类型 message/external-body 也有一个外报头和包含其报头的封装。外报头中唯一必需的域为内容类型(Content-Type)域,它指示了子类型 message/external-body。内报头是封装消息的报头。报头中的内容类型(Content-Type)域必须包含一个访问类型(access-type)参数,它指示了访问方法,例如 FTP(文件传输协议)。

类型 application 是指其他类型的数据,如不能解释的二进制数据或由邮件应用程序处理的信息。

MIME 转换编码

MIME 规范中另一个主要部分是定义消息正文的转换编码。其目的是在庞大的网络环境中提供可靠的传输方式。

MIME 标准定义了两种编码方式,但 Content-Transfer-Encoding 域可以取 6 个值,如表 18.4 所示。其中当值为 7 位,8 位和 binary 时,并不进行编码,而是提供一些与数据属性相关的信息。对 SMTP 传输而言,用 7 位比较安全,而在其他邮件传输协议中可以使用 8 位和 binary。Content-Transfer-Encoding 域也可以取值 x-token,用来表示其他编码方式,并提供该方式的名字。这种方式可以是生产上规定的或应用程序规定的方式。目前,有两种方式:quoted-printable 和 base64,它们都是为了提供一种将所有数据类型的紧凑表示转换为便于人们阅读的形式而设计的。

表 18.4　MIME 转换编码

7 位	所有数据都是用短行的 ASCII 码字符表示
8 位	短行,但可以有非 ASCII 码字符(高位为 1 的 8 位)
binary	不仅可以有非 ASCII 码字符,而且可以存在 SMTP 无法传输的长行
quoted-printable	如果被编码的数据大多是 ASCII 码字符,则数据编码后的形式大部分仍能被人所识别的一种数据编码方式
base64	将 6 位输入转为 8 位输出,转换后的数据均为可打印的 ASCII 码字符
x-token	非标准编码方式的名称

quoted-printable 转换编码使用数据由大量可打印的 ASCII 字符组成。其本质上是一种不安全的十六进制字符表示方法,并引入软回车来解决每行不得超过 76 个字符的限制。

base64 转换编码是一种基数 64 的编码方式,是一种对二进制数据进行编码的通用方法,在邮件传输程序中无懈可击,也用于 PGP,详见附录 18A。

一个多部分的例子

图 18.8(详见 RFC 2045)描述了一个复杂的多部分消息。该消息有依次出现的 5 个部分:两个介绍性的纯文本,一个嵌入的多部分消息,一个 richtext 部分和一个封装了非 ASCII 字符集的文本消息。其中嵌入消息有两个需要并行播放的部分:图片和声音片段。

```
MIME-Version: 1.0
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: A multipart example
Content-Type: multipart/mixed;
    boundary=unique-boundary-1

This is the preamble area of a multipart message. Mail readers that understand multipart format should ignore
this preamble. If you are reading this text, you might want to consider changing to a mail reader that understands
how to properly display multipart messages.

--unique-boundary-1

   ...Some text appears here...
[Note that the preceding blank line means no header fields were given and this is text, with charset US ASCII.
It could have been done with explicit typing as in the next part.]

--unique-boundary-1
Content-type: text/plain; charset=US-ASCII

This could have been part of the previous part, but illustrates explicit versus implicit typing of body parts.

--unique-boundary-1
Content-Type: multipart/parallel;  boundary=unique-boundary-2

--unique-boundary-2
Content-Type: audio/basic
Content-Transfer-Encoding: base64

   ... base64-encoded 8000 Hz single-channel mu-law-format audio data goes here....

--unique-boundary-2
Content-Type: image/jpeg
Content-Transfer-Encoding: base64

   ... base64-encoded image data goes here....

--unique-boundary-2--

--unique-boundary-1
Content-type: text/enriched

This is <bold><italic>richtext.</italic></bold> <smaller>as defined in RFC 1896</smaller>

Isn't it <bigger><bigger>cool?</bigger></bigger>

--unique-boundary-1
Content-Type: message/rfc822

From: (mailbox in US-ASCII)
To: (address in US-ASCII)
Subject: (subject in US-ASCII)
Content-Type: Text/plain; charset=ISO-8859-1
Content-Transfer-Encoding: Quoted-printable

   ... Additional text in ISO-8859-1 goes here ...

--unique-boundary-1--
```

图 18.8　MIME 消息格式举例

规范格式

规范格式是 MIME 和 S/MIME 的一个重要概念。规范格式指的是一种与内容类型相对应的格式，可以在系统中作为标准使用。表 18.5（详见 RFC 2049）可说明此问题。

表 18.5　本地格式和规范格式

本地格式	被传输的正文部分是按系统的本地格式创建的，使用本地字符集和行结果标记。正文可以是 UNIX 风格的文本文件，Sun 光栅图片或 VMS 索引文件或依赖于系统的仅存储在内存中的声音文件等任何信息。从根本上说，数据是依据媒体类型按本地服务创建的
规范格式	整个正文部分包括外部信息，如记录长度和可能的文件属性信息均转化为一致的规范格式。正文中特定媒体类型和其属性将按本地格式使用，将其转换为合适的本地格式并将涉及字符集的转换、声音数据的转换、压缩或其他与媒体类型相关的操作。如果涉及字符集的转换则应注意理解与字符集密切相关的媒体类型的语义（如 text 子类型中的 plain 和含义丰富的字符集）

18.2.3　S/MIME 功能

就一般功能而言，S/MIME 与 PGP 非常相似。两者都提供了签名和加密消息的能力。在本节中，我们将简要介绍 S/MIME 的功能，然后再从消息格式和消息准备方面详细叙述这个功能。

功能

S/MIME 有如下功能：

- **封装数据**：由任何类型的加密内容和加密该内容所用的加密密钥组成，密钥可以是与一个或多个接收方的密钥。
- **签名数据**：数字签名通过提取待签名内容的数字摘要，并用签名的私钥加密得到。然后，用 base64 编码方法重新对内容和签名编码。因此，一个签名了的数据消息只能被具有 S/MIME能力的接收方处理。
- **透明签名数据**：签名的数据形成了内容的数字签名。但在这种情况下，只有数字签名采用了 base64 编码，因此没有 S/MIME 能力的接收方虽然无法验证签名但却可以看到消息内容。
- **签名并封装数据**：仅签名实体和仅封装实体可以嵌套，能对加密后的数据进行签名和对签名后的数据或透明签名数据进行加密。

密码算法

表 18.6 总结了在 S/MIME 中使用的密码算法。S/MIME 中使用了如下术语（详见 RFC 2119 规范说明需求等级）：

- MUST：在规范说明书中表示一定要满足的需求，其实现必须与规范说明中的功能一致。
- SHOULD：如果在特定条件下有合理的理由也可以忽略，但推荐其实现包含该功能。

S/MIME 整合了三种公钥算法。第 13 章的 DSS（Digital Signature Standard）是其推荐的数字签名算法。Diffie-Hellman 是其推荐的密钥交换算法，实际上，在 S/MIME 中使用的是其能加密解密的变体 ElGamal（详见第 10 章）。第 9 章讲述的 RSA 既可以用做签名，也可以用来加密会话密钥。这些算法与 PGP 中使用的算法相同，提供了高层安全性。文档规范推荐使用 160 位的 SHA-1 算法作为数字签名的散列函数，但是要求就收方能支持 128 位的 MD5 算法。第 11 章对 MD5 安全性的讨论指出 SHA-1 的安全性要更好一些，但由于 MD5 已经有了广泛的应用，因此必须提供相关支持。

表 18.6 S/MIME 中使用的密码算法

功 能	要 求
创建用于数字签名的数字摘要	必须支持 SHA-1;接收方应该支持 MD5,以便向后兼容
加密数字摘要形成数字签名	发送代理和接收代理必须支持 DSS;发送代理应该支持 RSA 加密;接收代理应该支持验证密钥大小在 512 位至 1024 位的 RSA 签名
为传输消息加密会话密钥	发送代理和接收代理应该支持 Diffie-Hellman;发送代理必须支持密钥大小在 512 位至 1024 位 RSA 加密
用一次性会话密钥加密消息	发送代理和接收代理必须支持 3DES;发送代理必须支持 AES 加密,发送代理应该支持 RC2/40 加密
创建一个消息鉴定代码	接收代理必须支持 SHA-1 HMAC;发送代理应该支持 SHA-1 HMAC

对消息加密而言,推荐使用 3DES。但实现时也必须支持 40 位的 RC2,后者是一种弱加密算法,美国允许出口该算法。

S/MIME 文档规范包含如何决定采用何种加密算法。从本质上说,一个发送方代理需要做如下两个选择:一是,发送方代理必须确定接收方代理是否能够使用该加密算法进行解密。二是,如果接收方代理只能接收弱加密的内容,发送方代理必须确定弱加密方式是否是可以接受的。为能达到上述要求,发送方代理可以在他发送消息之前宣布他的解密能力,由接收方代理将该消息存储,留给将来使用。

发送方代理必须按照如下顺序进行:

(1) 如果发送方代理有一个接收方解密性能表,则他应该选择表中的第一个(即优先级最高)。
(2) 如果发送方代理没有接收方代理的解密性能表,但曾经接收到一个或多个来自该接收方的消息,则应该使用与最近接收到的消息一样的加密算法,加密将要发送给接收方的消息。
(3) 如果发送方代理没有接收方的任何解密性能方面的知识,并且想冒险一试(接收方可能无法解密消息),则应该选择 3DES。
(4) 如果发送方代理没有接收方任何解密性能方面的知识,并且不想冒险,则发送方代理必须使用 RC2/40。

如果消息需要发给多个接收方,并且他们没有一个可以共同接受的加密算法,则发送方代理需要发送两条消息,此时,应该特别注意该消息的安全性将由于弱安全性的一份副本而受到威胁。

18.2.4 S/MIME 消息

S/MIME 使用了一系列新的 MIME 内容类型,如表 18.7 所示。所有的新类型都使用 PKCS (Public-Key Cryptography Specification)设计,PKCS 是纪念为 S/MIME 做出贡献的 RSA 实验室及他们发布的一组公钥密码规范说明。

表 18.7 S/MIME 内容类型

类 型	子 类 型	smime 参数	描 述
Multipart	Signed		两部分的透明签名消息:一部分是消息,另一部分是签名
Application	pkcs7-mime	签名数据	签名的 S/MIME 实体
	pkcs7-mime	封装数据	加密的 S/MIME 实体
	pkcs7-mime	退化的签名数据	仅包含公钥证书的实体
	pkcs7-mime	压缩数据	一个压缩的 S/MIME 实体
	pkcs7-signature	签名数据	签名子部分的内容类型为 multipart/signed 的消息

下面逐一介绍 S/MIME 消息处理过程。

保护 MIME 实体

S/MIME 用签名和/或加密保护 MIME 实体。一个 MIME 实体可以是一个整体消息(除 RFC 5322 报头之外),或当 MIME 内容类型为 multipart 时,MIME 实体是一个或多个消息的子部分。MIME 实体将按照 MIME 消息准备规则进行准备,然后将 MIME 实体和一些与安全相关的数据(如算法标志符、证书)一起用 S/MIME 处理,得到所谓的 PKCS 对象,最后,将 PKCS 对象作为消息内容封装到 MIME 中(提供合适的 MIME 报头)。后面我们将举例说明。

总之,将要发送的消息转换为规范形式。特别地,对给定的类型和子类型而言,相应的规范形式将作为消息内容。对一个多部分的消息而言,合适的规范形式被用于每个子部分。

使用编码转换时应注意,在大多数情况下,使用安全算法会产生部分或全部二进制数据表示的对象,将该对象放入外部 MIME 消息后,一般用 base64 转换编码对其进行转换。而对一个多部分签名的消息,安全处理过程并不改变子部分的消息内容,除了内容用 7 位表示的以外,其他代码转换应使用 base64 或 quoted printable,使得应用签名的内容不会被改变。

下面逐个讨论 S/MIME 内容类型。

封装数据

子类型 `application/pkcs7-mime` 拥有一个 smime-type 参数。其结果(对象)采用 ITU-T 推荐的 X.209 中定义的 BER(Basic Encoding Rules)表示法。BER 格式由 8 位字符串组成,即为二进制数据,因此,此对象可在外部 MIME 消息中使用 base64 转换算法编码。首先,看看封装的数据。

MIME 实体准备封装数据的步骤如下:

(1) 为特定的对称加密算法(RC2/40 或 3DES)生成伪随机的会话密钥。

(2) 用每个接收方的 RSA 公钥分别加密会话密钥。

(3) 为每个接收方准备一个接收方信息块(`RecipientInfo`),其中包含接收方的公钥证书[①]标志,加密会话密钥的算法标志和加密后的会话密钥。

(4) 用会话密钥加密消息内容。

接收方信息块(`RecipientInfo`)后面紧跟着构成封装数据的加密内容,然后用 base64 编码,例如(不包含 RFC 5322 头):

```
Content-Type:  application/pkcs7-mime;  smime-type=enveloped-
       data; name=smime.p7m
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename=smime.p7m

rfvbnj756tbBghyHhHUujhJhjH77n8HHGT9HG4VQpfyF467GhIGfHfYT6
7n8HHGghyHhHUujhJh4VQpfyF467GhIGfHfYGTrfvbnjT6jH7756tbB9H
f8HHGTrfvhJhjH776tbB9HG4VQbnj7567GhIGfHfYT6ghyHhHUujpfyF4
0GhIGfHfQbnj756YT64V
```

为了恢复加密的消息,接收方首先去掉 base64 编码,然后用其私钥恢复会话密钥,最后,用会话密钥解密得到消息内容。

签名数据

smime-type 的签名数据可以被一个或多个签名者使用。为了简单起见,我们将讨论范围限定在单个数字签名内。MIME 实体准备签名数据的步骤如下:

---

① 这是一个 X.509 证书,在随后的小结中将会讨论。

(1)选择消息摘要算法(SHA 或 MD5)。

(2)计算带签名内容的消息摘要或散列函数。

(3)用签名者的私钥加密数字摘要。

(4)准备一个签名者信息块(SignerInfo),其中包含签名者的公钥证书,消息摘要算法的标志符,加密消息摘要的算法标志符和加密的消息摘要。

签名数据实体包含一系列块,包含一个消息摘要算法标志符,被签名的消息和签名者信息块(signerInfo),签名数据实体可以包含一组公钥证书,该证书可以构成从上级认证机构或更高级的认证机构到该签名者的一条链路。最后将这些数据用 base64 转换编码。例如(不包含 RFC 5322 头):

```
Content-Type:  application/pkcs7-mime;  smime-type=signed-
       data; name=smime.p7m
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename=smime.p7m

567GhIGfHfYT6ghyHhHUujpfyF4f8HHGTrfvhJhjH776tbB9HG4VQbnj7
77n8HHGT9HG4VQpfyF467GhIGfHfYT6rfvbnj756tbBghyHhHUujhJhjH
HUujhJh4VQpfyF467GhIGfHfYGTrfvbnjT6jH7756tbB9H7n8HHGghyHh
6YT64V0GhIGfHfQbnj75
```

为了恢复签名消息并验证签名,接收方首先去掉 base64 编码,然后用签名者的公钥解密消息摘要,接收方独立计算消息摘要,并将其与解密得到的消息摘要进行比较,从而验证签名。

透明签名

透明签名(Clear signing)在对多部分内容类型的子类型签名时使用。签名过程并不涉及对签名消息的转换,因此该消息发送时是明文。因此,具有 MIME 能力而不具备 S/MIME 能力的接收者也能阅读输入的消息。

一个 multipart/signed 消息由两部分组成。第一部分可以是任何 MIME 类型但必须做好准备,使之在从源端到目的端的传输过程中不被改变,这意味着第一部分不能为 7 位,需要用 base64 或 quoted-printable 编码。而后其处理过程与签名数据相同,但签名数据格式的对象中消息内容域为空,该对象与签名相分离,再将它用 base64 编码,作为 multipart/signed 消息的第二部分。第二部分 MIME 内容类型为 application,子类型为 pkcs7-signature。例如:

```
Content-Type: multipart/signed;
   protocol="application/pkcs7-signature";
   micalg=sha1; boundary=boundary42

--boundary42
Content-Type: text/plain

This is a clear-signed message.

--boundary42
Content-Type: application/pkcs7-signature; name=smime.p7s
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename=smime.p7s

ghyHhHUujhJhjH77n8HHGTrfvbnj756tbB9HG4VQpfyF467GhIGfHfYT6
4VQpfyF467GhIGfHfYT6jH77n8HHGghyHhHUujhJh756tbB9HGTrfvbnj
n8HHGTrfvhJhjH776tbB9HG4VQbnj7567GhIGfHfYT6ghyHhHUujpfyF4
7GhIGfHfYT64VQbnj756
--boundary42--
```

协议的参数表明它是一个由两部分组成的透明签名实体。参数 micalg 表明使用的消息摘

要类型。接收方可以从第一部分获得消息摘要,并同第二部分中恢复得到的消息摘要进行比较来进行认证。

注册请求

典型地,一个应用或用户向认证机构申请公钥证书。S/MIME 的实体 application/pkcs10 用于传递证书请求。证书请求包括证书的请求信息块,公钥加密算法标志符,用发送方私钥对证书请求信息块的签名。证书请求信息块包含证书主题的名字(拥有待证实的公钥实体)和该用户公钥的标志位串。

仅含证书消息

仅包含证书或证书撤销表(CRL)的消息在应答注册请求时发送。该消息的类型/子类型为 application/pkcs7-mime,并带一个退化的 smime-type 参数。其步骤除没有消息内容及签名者信息块为空以外,其他过程与创建签名数据信息相同。

18.2.5　S/MIME 证书处理过程

S/MIME 使用公钥证书的方式与 X.509 的版本 3 一致(详见第 14 章)。S/MIME 使用的密钥管理模式是严格的 X.509 证书层次和 PGP 的 Web 信任的一种混合方式。按照 PGP 模式,S/MIME 的管理者和(或)用户必须配置每个客户端的信任密钥表和证书撤销表。也就是说,验证接收到的签名和输出消息的签名工作都是通过本地维护证书实现的。另一方面,证书由认证机构颁发。

用户代理角色

一个 S/MIME 用户需要执行若干密钥管理职能:

- **密钥生成**:与一些行政管理机构相关的用户(如与局域网管理相关)必须能生成单独的 Diffie-Hellman 和 DSS 密钥对,并且应该能生成 RSA 密钥对。每个密钥对必须是利用非确定的随机输入生成,并用安全的方式保护。用户代理应该能生成长度为 768 位到 1024 位的 RSA 密钥对,且禁止生成长度大于 512 位的 RSA 密钥对。
- **注册**:为了获得 X.509 公钥证书,用户的公钥必须到认证机构注册。
- **证书存储和检索**:为了验证接收到的签名和加密输出消息,用户需要存取本地的证书列表。该列表必须由本地维护。

VeriSign 证书

有不少公司都可以提供证书认证服务。例如,Nortel 设计了一种企业认证解决方案,能够在组织内部提供 S/MIME 支持。还有一些基于互联网的认证机构,包括 VeriSign,GTE 和 U.S. Postal Service。其中使用最广泛的是 VeriSign 的认证服务。

VeriSign 提供一个与 S/MIME 和其他一系列应用兼容的一种认证服务,颁发 VeriSign 数字证书(VeriSign Digital ID)的 X.509 证书。到 1998 年初,已有 35 000 家商业 Web 站点使用 VeriSign 数字证书,100 万使用 Netscape 和 Microsoft 浏览器的用户拥有 VeriSign 数字证书。

VeriSign 数字证书包含的内容依赖于数字证书的类型和它的用途。但每个数字证书至少包含如下内容:

- 用户公钥
- 用户名或别名
- 数字证书的有效期

- 数字证书的序列号
- 颁发数字证书的认证机构名
- 颁发数字证书的认证机构的数字签名

数字证书还可以包含其他用户提供的信息:

- 地址
- 电子邮件地址
- 基本注册信息(国家、邮政编码、年龄和性别)

VeriSign 提供了公钥证书的三种安全级别,如表 18.8 所示。用户以在线的方式向 VeriSign 的 Web 站点或其他相关站点申请证书。第一类和第二类请求可以在线处理,而且大多数只需要几秒钟就可以完成。以下将简要叙述处理过程:

- 对第 1 类数字证书而言,VeriSign 通过发送一个 PIN 命令和从应用中提取电子邮件地址确认用户的电子邮件地址。
- 对第 2 类数字证书而言,VeriSign 除了进行与第一类数字证书相同的检查外,还使用一个用户数据库自动比较应用中提供的信息。最后,向给定的邮件地址发送一个确认信息,告诉该用户已经按其名字颁发了一个数字证书。
- 对第 3 类数字证书而言,VeriSign 需要更高级的身份证实。个人必须提供有效证明来证实其身份。

表 18.8 VeriSign 公钥证书类型

	第 1 类	第 2 类	第 3 类
证书确认	姓名和电子邮件地址的自动搜索	第 1 类检查登记信息检查和自动地址检查	第 1 类检查本人出席、ID 文档、第 2 类自动身份检查组织、团体的业务记录
IA 私钥保护	PCA:信任的硬件,CA:信任的硬件和软件	PCA 和 CA:信任的硬件	PCA 和 CA:信任的硬件
证书申请者的私钥保护	推荐使用加密软件(PIN 保护)	要求使用加密软件(PIN 保护)	要求使用加密软件(PIN 保护)
用户实现或期望的应用	Web 浏览和使用电子邮件	个人、公司内或公司间电子邮件、在线订购、更换密码和软件确认	电子银行、公司数据存取、个人银行、基于会员的在线服务、内容集成服务、电子贸易服务、软件确认,以及 LRAA 的认证、对服务的强加密

IA:Issuing Authority(签发认证)

CA:Certification Authority(认证机构)

PCA:VeriSign Public Primary Certification Authority(VeriSign 公钥认证证书)

PIN:Personal Identification Number(个人身份号码)

LRAA:Local Registration Authority Administrator(本地认证管理机构)

18.2.6 增强安全性服务

因特网草案还提出了三种增强安全性服务,其细节可能会发生变动,也可能增加一些新的服务。这三种服务分别是:

- **签收**:对签名数据对象要求进行签收。返回一条签收消息可以告诉消息的发送方,已经收到消息,并通知第三方自己已经收到消息。本质上说,就是接收方将对整个原始消息和发送方的原始签名进行签名,并将此签名与消息一起形成一个新的 S/MIME 消息。

- **安全标签**:在签名数据对象的认证属性中可以包括安全标签。安全标签是一个描述被 S/MIME 封装的信息的敏感度的安全信息集合。该标签既可用于访问控制,描述该对象能被哪些用户存取,还可描述优先级(秘密,机密和受限等)或角色(哪种人可以查看信息,如患者的病历等)。
- **安全邮寄列表**:当用户向多个接收方发消息时,需要进行一些与每个接收方相关的处理,包括使用各接收方的公钥。用户可以通过使用 S/MIME 提供的邮件列表代理(Mail List Agent)来完成这一工作。邮件列表代理可以对一个输入消息为各个接收方进行相应的加密处理,而后自动发送消息。消息的发送方只需将用 MLA 的公钥加密过的消息发给 MLA 即可。

18.3　DKIM

DKIM(DomainKeys Identified Mail)域名密钥识别邮件是一个用密码学方法签名电子邮件的规范,它支持一个签名域对邮件流中的某个邮件签名。消息接收方(或其代理)可以通过直接询问签名者域以获取合适公钥来验证消息是否是由拥有该签名域私钥的一方签名。DKIM 已成为因特网标准[RFC 4871:DomainKeys Identified Mail(DKIM) Signatures]。DKIM 已经被广泛采纳,包括公司,政府机构,gmail,yahoo 和多许多因特网服务提供商(ISP)。

本节将概要介绍 DKIM。在开始讨论 DKIM 之前,我们首先介绍标准的因特网邮件结构。然后分析 DKIM 的威胁,最后简要阐述 DKIM 的操作过程。

18.3.1　因特网邮件结构

为了理解 DKIM 的操作过程,首先对因特网邮件结构有一个基本掌握是很有用的,该结构已在[CROC09]中定义。本小节简要给出其基本概念。

最基础的是,因特网邮件结构包含了以消息用户代理(MUA)形式的用户群,以消息处理服务(MHS)形式的传输群,其中 MHS 是由消息传输代理(MTA)组成的。MHS 从一个用户接收消息,然后将之分发给其他用户,形成一个虚拟的 MUA 到 MUA 的交换环境。该结构涉及三种类型的互用性。一个是直接处于用户之间的:消息必须由代表消息拥有者的 MUA 格式化以使目的端 MUA 可以将之向消息接收方展示。在 MUA 和 MHS 之间也有互用性要求,首先消息从 MUA 向 MHS 发出,然后经 MHS 向目的端 MUA 传输。在通过各个 MHS 的传输路径上的 MTA 成员之间也有互用性要求。

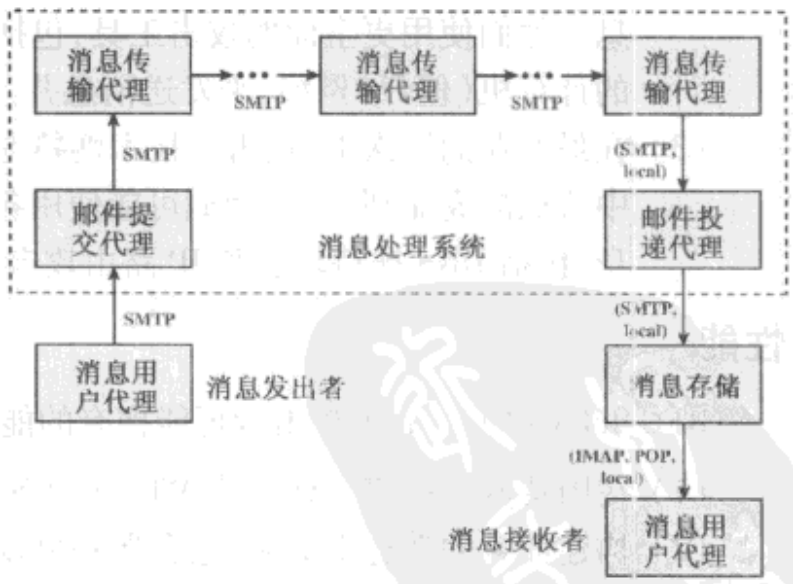


图 18.9　因特网邮件系统的功能模块和标准协议

图 18.9 描述了因特网邮件结构的密钥成员包含以下内容:

- **消息用户代理(MUA)**:代表用户行为和用户应用程序工作。这是他们在邮件服务中的代理。典型地,该功能是部署在用户机器上并被认为是客户端电子邮件程序或者是本地网络电子邮件服务器。MUA 格式化消息并通过 MSA 向 MHS 初始提交。接收端 MUA 处理接收邮件用来存储或给接收方用户呈现。
- **邮件提交代理(MSA)**:接收由 MUA 提交的消息并增强源域的安全策略和因特网标准的要

求。该功能可能部署在 MUA 或作为一个独立的功能模块。在后者情况下,简单邮件传输协议(SMTP)就运行在 MUA 与 MSA 之间。

- **消息传输代理(MTA)**:在应用层中继邮件。它就像包交换或 IP 路由一样使消息向接收方移动。由一系列的 MTA 执行中继直到消息到达目标地 MDA。MTA 还在消息头中加入路径信息。SMTP 就用在 MTA 之间和 MTA 与 MSA 或 MDA 之间。
- **邮件投递代理(MDA)**:负责将消息从 MHS 传输到 MS。
- **消息存储(MS)**:一个 MUA 可以使用一个长期的 MS。MS 可以部署在远程服务器上也可以与 MUA 部署在同一台机器上。典型的是,MUA 从一个使用邮局协议(POP)或交互式邮件存取协议(IMAP)的远程服务器上提取消息。

还有两个概念需要定义。**行政管理域(ADMD)**是一个因特网邮件提供者。例子中包含一个执行一个本地邮件中继的部门(MTA),一个执行企业邮件中继的 IT 部门和提供公共电子邮件服务的 ISP。每一个 ADMD 会有不同的执行策略和基于可信度的决策。一个明显的例子是在组织内邮件交换和组织之间邮件交换的区别。处理这两种通信的法则是完全不同的。

域名系统(DNS)提供路径查找服务,该服务能在因特网主机名和其 IP 地址之间映射。

18.3.2 电子邮件的威胁

RFC 4684(DKIM 的威胁分析)描述了 DKIM 在特征,性能和本地潜在攻击方面带来的威胁。

特征

RFC 刻画了攻击者在三个层次上的攻击范围。

(1) 在较低层次上,攻击者仅仅是给那些不希望收到邮件的用户发送电子邮件,攻击者可以使用众多可用的商用工具以使发送者可以伪造消息的源地址。这使得接收者很难根据源地址或源域来过滤垃圾邮件。

(2) 在中等层次上,大量垃圾邮件的专业发送者以商业组织的模式运转并代表第三方发送消息。他们使用更全面的攻击工具,包括邮件传输代理(MTA)和已注册的域名以及控制了的计算机(僵尸)网络,来发送消息并(在大多数情况下)获取更多发送地址。

(3) 在最专业的层次上,攻击者技术娴熟并有充实的财力支持(例如从基于电子邮件的诈骗中获取的商业利益)。他们可能使用各种上述提到的机制并可能攻击因特网基础设施本身,包括 DNS 挟持攻击和 IP 路由攻击。

性能

RFC 4686 列出了如下攻击者可能拥有的能力:

(1) 在因特网上用多个地址给 MTA 和 MSA 提交消息。

(2) 构建任意的消息报头域,包含那些可以表示目标地址清单、中继者和其他邮件代理。

(3) 在他们的控制下代表域对消息签名。

(4) 生成大量貌似签名或根本未签名的消息用来进行拒绝服务(DoS)攻击。

(5) 重发以前被合法域签名的消息。

(6) 传输任何封装了必要信息的消息。

(7) 假装成某个被控制的计算机并发送消息。

(8) 操纵 IP 路由。目的是从某指定的 IP 地址或难以追踪的地址发送消息或者转发消息到某指定的目标域。

(9) 使用例如缓存挟持等方法影响部分域名服务器(DNS)。这样可以影响消息的路由或者伪造基于 DNS 的广告者的密钥和签名。

(10) 控制大量的计算机资源,例如,通过"征召"感染蠕虫的"僵尸"计算机。这样可以让攻击者能进行各种蛮力攻击。

(11) 监听一个正在进行的通信信道,例如对象可能是无线网络。

定位

DKIM 主要关注那些管理单元之外的攻击者,而这些管理单元往往是声称了源发送者和接收者的。这些管理单元通常对应那些临近源发送者和接收者的受保护的网络隔离设备。这些部分正需要信任关系以使认证消息的提交不存在并且事实上也不适用于大规模的情形。相反,有了这些管理单元,就有其他更容易配置的方法(例如提交认证消息),并且这些方法比 DKIM 更可用。外部的攻击者通常试图利用电子邮件的任意方到任意方的特性来使接收方 MTA 能接收来自任何地址的消息并将其发送至本地域。他们可能会生成不带签名的消息,或者是不正确的签名或者是无法追踪的域上的合法签名。他们也可能仿造邮件列表和其他代理来为其他用户发送或转发邮件。

18.3.3　DKIM 策略

DKIM 是提供一个对终端用户透明的电子邮件认证技术的服务。基本上,用户的电子邮件都是由其所在的管理域的私钥签名的。这个签名包含消息的所有内容和 RFC 5322 消息头的一部分。在接收方终端,MDA 可以通过 DNS 访问相应的公钥并验证签名,以认证从声称拥有该消息的域认证该消息。于是,一些从某个地址发过来的消息声称是从其他域发出的就将无法通过认证测试并被拒绝接收。该方法和 S/MIME 或 PGP 都不同,后者是使用发送方的私钥签名消息的内容。

DKIM 的产生源于如下因素[①]:

(1) S/MIME 要求接收方和发送方都使用 S/MIME。但是,对几乎所有的用户而言,大量的软件并不适用 S/MIME,并且大量给接收方发送的邮件也并不使用 S/MIME。

(2) S/MIME 仅仅对消息内容签名,因此,RFC 5322 报头中关于发送源的部分可能会受到安全性威胁。

(3) DKIM 并不是在客户端(MUA)实现的,因此对用户是透明的,用户不需要采取任何行动。

(4) DKIM 可应用于所有从协作域发出的邮件。

(5) DKIM 可以使合法的发送者证明其发送了某邮件并阻止仿冒者假冒合法用户。

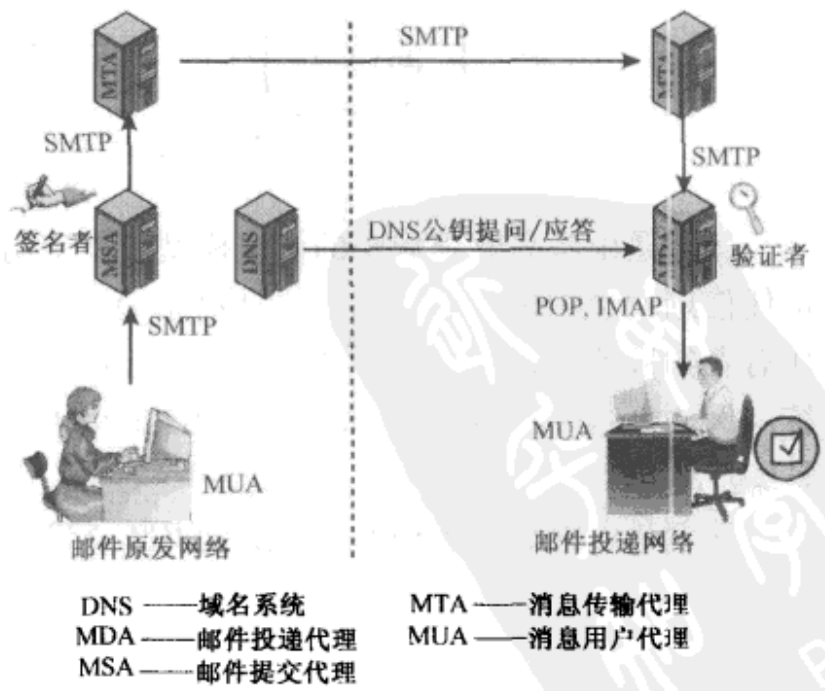


图 18.10　DKIM 部署的简单示例

图 18.10 是 DKIM 操作的一个简单例子。我们从一个用户生成消息并发送到 MHS 中让其发送给与该用户在同一个管理域中 MSA 的情形开始。一个电子邮件客户端生成一份电子邮件。电子邮件的提供者用自己的私钥签名了消息的内容和选择的 RFC 5322 头域。签

① 在 S/MIME 中使用了因素,同样在 PGP 中也会应用到。

名者是同某个域相关联的,而该域可能是一个合法的本地网络或者因特网服务提供商(ISP)或者类似于 E-mail 的公共电子邮件工具。然后,签名消息就通过一系列的 MTA 向前传输。到目的地时,MDA 就索取来件签名所对应的公钥并在将该消息转发到目的地电子邮件客户端之前验证签名。默认的签名算法是结合了 SHA-256 的 RSA 算法,有时也用 SHA-1 与 RSA 的组合。

18.3.4 DKIM 功能流

图 18.11 提供了一个 DKIM 操作元素的详细示例。基本的消息处理过程是区分签名行政管理域(ADMD)和验证行政管理域。在最简单的情形下就是区分源 ADMD 和转发 ADMD,但是通常情况会涉及传输路径中的其他 ADMD。

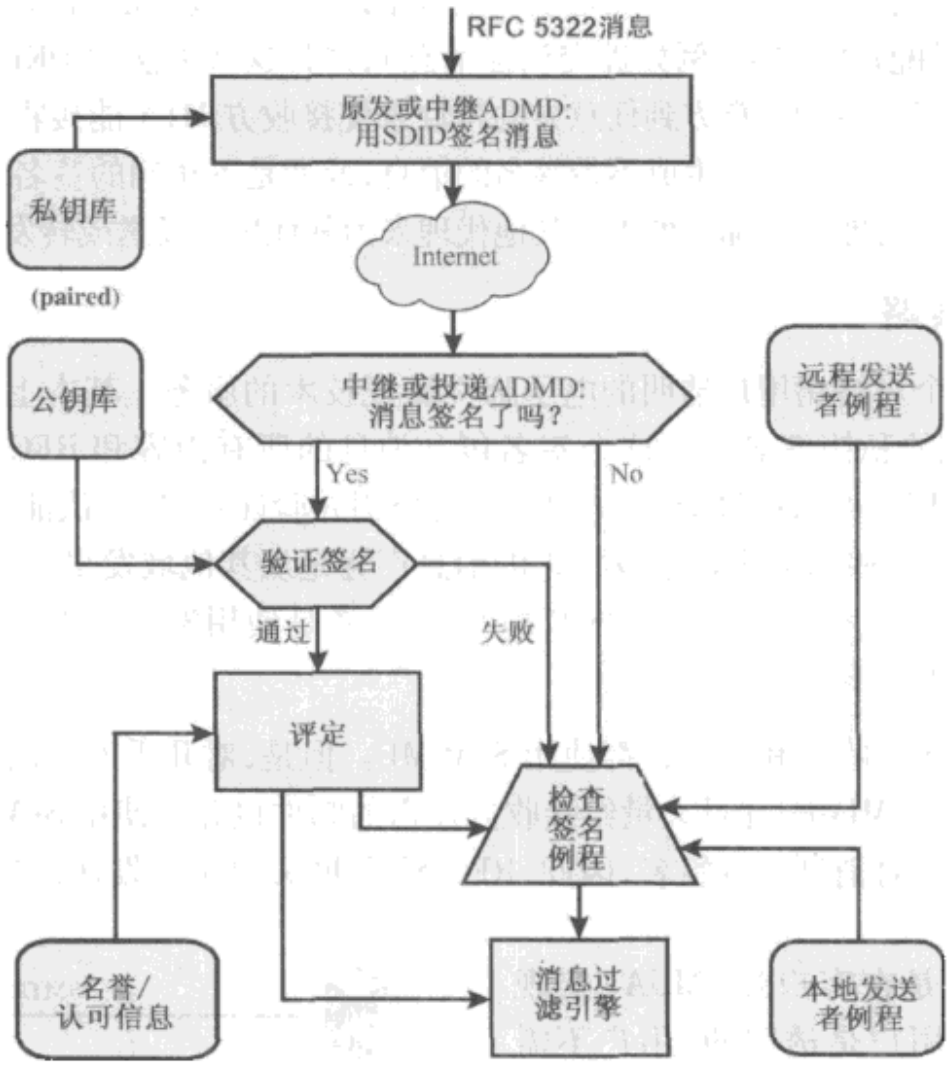


图 18.11 DKIM 功能流

签名是由签名管理域中的认证模型使用密钥库中的私钥信息进行的。假如签名通过了,那么信任信息就会用来评估签名者并且消息会通过邮件过滤系统。但若签名验证失败或没有用消息所有者的域签名,则关于签名的信息会与远程或本地的所有者联系,该信息也会通过邮件过滤系统。例如,假如发送者(如 gmail)使用了 DKIM,但是没有给出 DKIM 签名,那么邮件可能会被认为是编造的。

签名是以附加报头的形式添加到 RFC 5322 消息中的,并以关键字 Dkim-Signature 开头。读者可以使用 View Long Headers(或者类似的字符)选项查看来件。如下是一个例子:

```
Dkim-Signature:      v=1;  a=rsa-sha256;  c=relaxed/relaxed;
                     d=gmail.com; s=gamma; h=domainkey-signa-
                     ture:mime-version:received:date:message-
                     id:subject   :from:to:content-type:con-
                     tent-transfer-encoding;
```

```
bh=5mZvQDyCRuyLb1Y28K4zgS2MPOemFToDBgvbJ
7GO90s=;
b=PcUvPSDygb4ya5Dyj1rbZGp/VyRiScuaz7TTG
J5qW5slM+klzv6kcfYdGDHzEVJW+Z
FetuPfF1ETOVhELtwH0zjSccOyPkEiblOf6gILO
bm3DDRm3Ys1/FVrbhVOlA+/jH9Aei
uIIw/5iFnRbSH6qPDVv/beDQqAWQfA/wF7O5k=
```

在对消息签名之前，会对 RFC 5322 消息的报头域和正文都进行标准化。标准化是用来处理消息中的微小变化，包括字符编码、消息行中的空格处理，以及头域中“可折叠”和“不可折叠”的行。标准化的目的是尽最大可能减小消息的传输（为了签名，消息本身没有改变，于是验证会重复进行一次标准化）以使接收端能最大可能地产生规范的值。DKIM 定义了两个报头标准化算法（“simple”和“relaxed”）和两个正文标准化算法（如前者）。simple 算法能容忍几乎没有任何修饰，而 relaxed 能容忍普通的修饰。

签名包含了一些域。每个域都包含标志码并后接等号且以分号结尾。这些域包含如下内容：

- v 为 DKIM 版本。
- a 为用来声称签名的算法，必须是 rsa-sha1 或者是 rsa-sha256。
- c 为作用在报头和正文上的标准化方法。
- d 为一个用做身份认证的域名，表示一个用户或一个组织的标志。在 DKIM 中，该标志称为签名域 ID（SDID）。在我们的例子中，该域表示发送者使用的是 gmail 地址。
- s 为由于在同一个签名域的不同环境下可能需要使用不同的密钥（允许密钥到期和分离式签名之类），DKIM 定义了一个选择器（与密钥相关），是验证方在签名验证阶段用来获取合适密钥的。
- h 为签名报头域。用冒号隔开的头域名称列表，这些报头域名标志了签名算法的报头域。在如上的例子中，签名给出了域密钥签名域。这与一个现在仍在使用的旧的算法（尽管已经被 DKIM 取代）有关。
- bh 为消息正文部分标准化后的 Hash 值。这提供了消息验证失败时的附加信息。
- b 为基数 64 形式的签名数据，为加密后的 Hash 码。

18.4　推荐读物和网站

[LEIB07]给出了 DKIM 的概貌。

LEIB07 Leiba, B., and Fenton, J. “DomianKeys Identified Mail (DKIM): Using Digital Signatures for Domain Verification.” *Proceedings of Fourth Conference on E-mail and Anti-Spam (CEAS07)*, 2007.

推荐网站

- **PGP Home Page**：Network Associates 提供的 PGP Web 站点，PGP 主流销售商。
- **国际 PGP Home Page**：用来提高 PGP 在世界范围内的应用。包括文档和兴趣链接。
- **PGP Charter**：开放规格 PGP 的最新 RFC 和 Internet 草案。
- **S/MIME Charter**：最新的 RFC 和 S/MIME 的 Internet 草案。
- **DKIM**：Mutual Internet Practices Association 主办的网站，该站点包含于 DKIM 相关的大量文档和信息。
- **DKIM Charter**：DKIM 的最新 RFC 文档和 Internet 草案。

18.5 关键术语、思考题和习题

关键术语

分离式签名	MIME	S/MIME
DKIM	PGP	信任
电子邮件	R64	
ZIP	会话密钥	

思考题

18.1 PGP 提供的 5 个基本服务是什么?
18.2 分离签名的用途是什么?
18.3 PGP 为什么在压缩前生成签名?
18.4 什么是 R64 转换?
18.5 E-mail 应用为什么使用 R64 转换?
18.6 PGP 如何使用信任关系?
18.7 RFC 5322 是什么?
18.8 MIME 是什么?
18.9 S/MIME 是什么?
18.10 DKIM 是什么?

习题

18.1 PGP 使用了 CAST-128 的密文反馈模式(CFB),而传统的加密应用(不是指密钥加密)使用密文块链接模式有:

$$\text{CBC: } C_i = \text{E}(K, [C_{i-1} \oplus \text{P}_i]); \quad \text{P}_i = C_{i-1} \oplus \text{D}(K, C_i)$$

$$\text{CFB: } C_i = \text{P}_i \oplus \text{E}(K, C_{i-1}); \quad \text{P}_i = C_i \oplus \text{E}(K, C_{i-1})$$

这两种模式看起来提供了相当的安全性,问为什么 PGP 选择了 CFB?

18.2 在 PGP 体制中,在前一次会话密钥生成后,希望生成多少个会话密钥?

18.3 在 PGP 中,一个拥有 N 个公钥的用户有多大的可能性有至少一个重复密钥 ID?

18.4 PGP 签名的 128 位消息摘要中的前 16 位是以明文方式解释的。

(a) 这对 Hash 算法的安全性有多大威胁?

(b) 这对其设计功能有多少帮助? 也就是说,帮助判断解密摘要的 RSA 密钥是否正确。

18.5 图 18.4 公钥环的每一项都有一个所有者信任域可以指明其所有者的信任度。它为什么不够使用呢? 所有者是被信任的,而且这也是该所有者的公钥,为什么还不能被 PGP 足够信任使用这个公钥?

18.6 在密钥层次和密钥信任方面,X.509 和 PGP 的基本区别是什么?

18.7 Phil Zimmermann 选择了 IDEA、3 密钥 3DES 和 CAST-128 作为 PGP 的对称钥加密算法。试列举本书所提及的其他加密算法在 PGP 中合适或不合适的原因。例如:DES、2 密钥 3 DES 和 AES。

18.8 基数 64 转换是一种加密形式,它没有密钥。但假设对手知道某种加密文本的替代算法,这种算法对抗密码分析的能力如何?

18.9 用如下技术为明文编码。假设给定的字符是存在 8 位 ASCII 码中并使用偶校验。

(a) Radix-64

(b) Quoted-printable

附录 18A　基数 64 转换

PGP 和 S/MIME 都用基数 64 转换的编码技术。这种技术可以将任意二进制输入转换成为可打印字符。这种转换有如下相关特性：

（1）函数输出的字符即使在所有场合都可以被表示出来，而不是某个字符集的二进制编码。那么，这些字符就可以被特定的系统编码为任何它们需要的形式。例如，字母“E”在 ASCII 系统中被编码为十六进制 45，在 EBCDIC 系统中则是十六进制 C5。

（2）这个字符集有 65 个可打印字符。其中有一个用来填充，其他 $2^6 = 64$ 个字符可以组成任意的 6 位输入。

（3）字符集中不包含任何控制字符。这样的基数 64 编码就可以通过那些在报文中搜索控制符的邮件系统。

（4）不包含“-”符号。这个符号对 RFC 5322 有特殊含义，应该被回避。

表 18.9 列举了从 6 位输入到字符的映射关系。字符集包括数字、字符和“+”、“/”。“=”作为填充符。

表 18.9　基数 64 编码

6 位数值	编码字符	6 位数值	编码字符	6 位数值	编码字符	6 位数值	编码字符
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/
						（pad）	=

图 18.12 演示了映射的过程。输入以每 3 个 8 位组或 24 位为一块，每 6 位一组，映射为一个字符。图中，这些字符用 8 位表示。通常 24 位输入会被扩展为 32 位输出。

例如，24 位串 00100011 01011100 10010001，十六进制表示为 235C91。按 6 位一组排列：

001000　110101　110010　010001

这些 6 位数用十进制表示为 8，53，50，17。查表 18.9 得到如下字符：I1yR，如果用 8 位 ASCII 码表示（奇偶校验位置零）有

01001001　00110001　01111001　01010010

有十六进制数：49317952。现总结如下：

输入数据	
二进制表示	00100011 01011100 10010001
十六进制表示	235C91
输入数据的基数 64 编码	
字符表示	I1yR
ASCII 码(8 位,偶校验)	01001001 00110001 01111001 01010010
十六进制表示	49317952

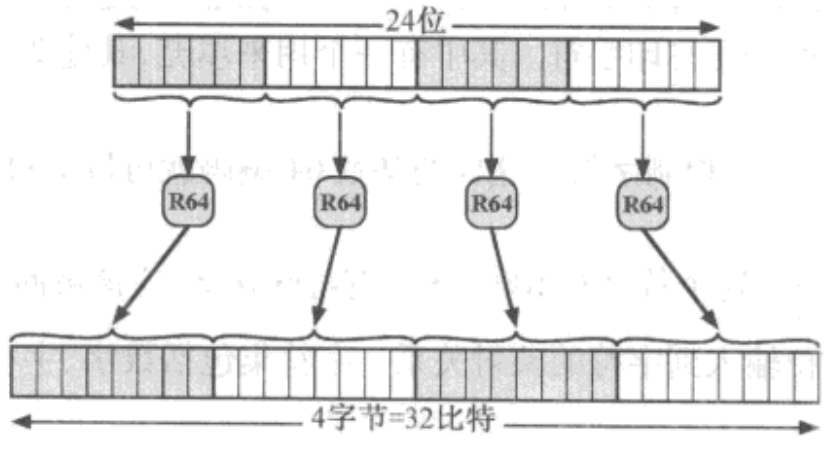


图 18.12 可打印编码或二进制数据转成基数 64 格式

第19章 IP安全性

19.1 IP安全性概述
- 19.1.1 IPsec的应用
- 19.1.2 IPsec的优点
- 19.1.3 路由应用
- 19.1.4 IPsec文档
- 19.1.5 IPsec服务
- 19.1.6 传输和隧道模式

19.2 IP安全性策略
- 19.2.1 安全性关联
- 19.2.2 安全性关联数据库
- 19.2.3 安全性策略数据库
- 19.2.4 IP通信过程

19.3 封装安全性有效载荷
- 19.3.1 ESP格式
- 19.3.2 加密和认证算法
- 19.3.3 填充
- 19.3.4 防止重放服务
- 19.3.5 传输和隧道模式

19.4 结合安全性关联
- 19.4.1 认证性和机密性
- 19.4.2 安全关联的基本结合

19.5 因特网密钥交换
- 19.5.1 密钥确定协议
- 19.5.2 报头和有效载荷格式

19.6 密码学套件

19.7 推荐读物和网站

19.8 关键术语、思考题和习题

If a secret piece of news is divulged by a spy before the time is ripe, he must be putto death, together with the man to whom the secret was told.

—The Art of War, Sun Tzu

要 点

- IP安全性(IPsec)可以通过将额外的头部添加到当前版本的互联网协议(IPv4或IPv6)来实现。
- IPsec包含三个方面的功能:认证、保密及密钥管理。
- 认证使用HMAC信息认证码。认证既可以应用于整个初始化IP包(隧道模式)也可以应用于除去IP头部(传输模式)后的IP包。
- 保密是通过已知的安全加密方式保证的。适用于隧道模式和传输模式。
- IKE定义了若干密钥管理技术。

互联网社团开发了各种应用的安全机制,如电子邮件(S/MIME、PGP),客户端/服务器(Kerberos),Web访问(SSL)等。然而,用户还有与协议层相关的安全需求。例如,企业为了确保内部IP网络的安全性和隐私性,它可能会不允许链接不信任站点,并对向外发出的数据包加密以及对进来的数据包认证。通过实现IP级安全性,企业不仅可以保护各种带有安全机制的应用,而且可以保护许多无安全机制的应用。

IP级安全性包括三个方面的内容:认证、保密和密钥管理。认证机制不仅要确保收到的包是由包头部所标志的源端发出,还要确保该包在传输过程中未被篡改。保密性是将消息加密后传送,防止第三方窃听。密钥管理机制与密钥的安全交换有关。

本章介绍IP安全性(IPsec)的概要和IPsec的体系结构,然后再依次详细介绍其三类功能。附录L介绍了互联网的一些协议。

19.1 IP 安全性概述

1994 年因特网结构委员会(IAB)在报告中提出了“因特网结构中的安全性”(RFC 1636)问题。该报告定义了安全机制中的关键领域。在这些领域中包含保障网络基础设施使网络通信免于在未认证情况下被监控,以及保证终端用户到终端用户之间使用认证和密码机制通信。

为了提供安全性,IAB 将认证和加密作为下一代 IP(即 IPv6)中必要的安全特性。幸运的是,这些安全特性可以同时在 IPv4 和 IPv6 中使用。这意味着生产商可以现在就提供这样的安全性能,而事实上很多厂商也已将 IPsec 的性能加入到产品中。IPsec 规范现在已经有了一系列的互联网标准。

19.1.1 IPsec 的应用

IPsec 提供了在 LAN、WAN 和互联网中安全通信的能力。其用途包括如下方面:

- **分支机构通过互联网安全互连:**一个公司可以在互联网和公用 WAN 上建立安全的虚拟专用网,使得依赖于互联网的交易成为可能,并减少了对专用网络的需求,节省了开销和网络管理费用。
- **远程安全访问互联网:**使用了 IP 安全协议的终端用户可以通过本地访问互联网服务提供商(ISP),以获得对公司网络的安全访问,减少了远程通信的费用。
- **与合作者建立外联网和内联网联系:**使用 IPsec 可以与其他组织进行安全通信,确保认证、保密并提供密钥交换机制。
- **加强电子商务的安全性:**虽然一些 Web 和电子商务应用程序是建立在安全协议之上的,但使用 IPsec 能加强其安全性。IPsec 通过在应用层附加一个安全层来保证任何由网络管理员指定的通信都是经过加密和认证的。

IPsec 能支持各种应用的关键在于它可以在 IP 层加密和(或)认证所有流量,这样就可以保护所有的分布应用,包括远程登录、客户/服务器,E-mail、文件传输、Web 访问等。

图 19.1 是 IPsec 的一个典型工作示意图。一个使用 LAN 的组织可以分布在各地。非安全的 IP 流量在各 LAN 内部使用,通过专用或公用 WAN 的外部流量则使用 IPsec 协议。这些协议在网络设备,如路由器或防火墙中运行,它们将各 LAN 与外部世界相连。IPsec 网络设备将对所有进入 WAN 的流量加密、压缩,并解密和解压来自 WAN 的流量,这些操作对 LAN 上的工作站和服务器是透明的。当然,对于使用拨号上网的个人用户也可以进行安全传输。这些用户的工作站就必须使用 IPsec 协议提供安全保障。

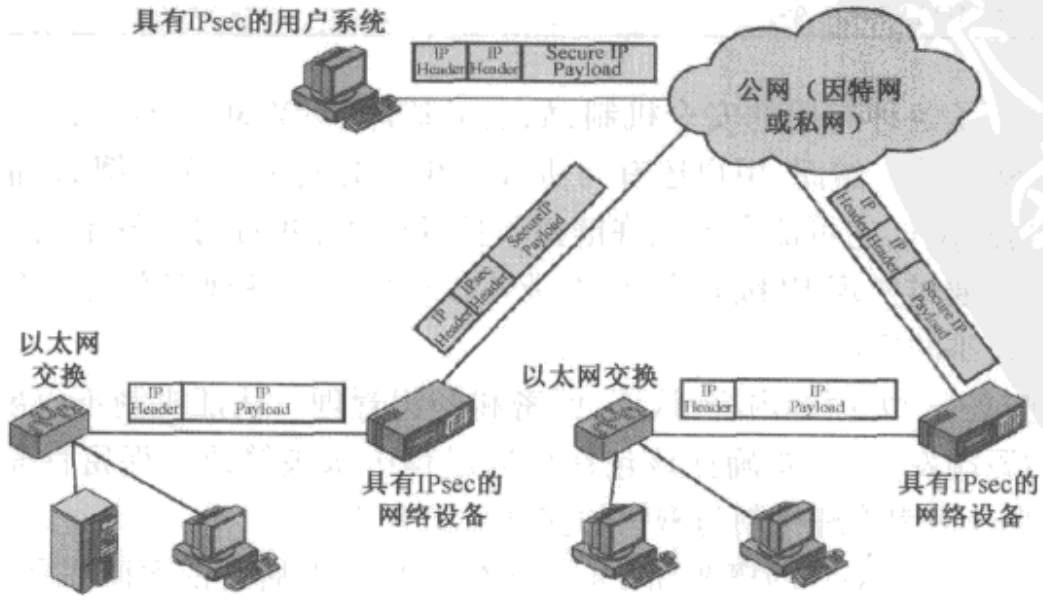


图 19.1 一种 IP 安全方案

19.1.2　IPsec 的优点

IPsec 的优点如下：

- 当防火墙或路由器使用 IPsec 时，它能对通过其边界的所有通信提供强安全保障。而公司或工作组内部的通信不会导致与安全处理相关的开销。
- 防火墙内的 IPsec 能阻止所有外部流量的旁路，因为防火墙是从互联网进入组织内部的唯一通道。
- 位于传输层（TCP、UDP）之下的 IPsec 对所有应用都是透明的。因此，当防火墙或路由器使用 IPsec 时，不需要对用户系统或服务系统做任何改变，即使在终端系统中使用 IPsec，也不需要改变上层的软件和应用。
- IPsec 可以对终端用户透明，不需要对用户进行安全机制培训，如对每个用户分配一个密钥，在用户离开组织时回收密钥等。
- 如果需要，IPsec 可以为个体用户提供安全性。这对上网上班族非常有用，它可以在进行敏感应用程序时为用户和企业之间建立一个虚拟子网。

19.1.3　路由应用

除了支持终端用户和保护上述系统和网络外，IPsec 在互联网的路由结构中扮演了一个非常重要的角色。[HUIT98]列举了使用 IPsec 的例子，IPsec 确保：

- 路由广播（一个新的路由器公告它的存在）来自于授权路由器。
- 邻居广播（路由器寻找建立或维护与其他路由区域中路由器的相邻关系）来源于授权路由器。
- 重定向报文来源于发送包的初始路由器。
- 路由更新无法伪造。

没有以上安全措施，攻击者可以中断通信或转发某些流量。路由协议如开放式最短路径优先（OSPF）必须在用 IPsec 安全协议的路由器上运行。

19.1.4　IPsec 文档

IPsec 包含三个功能领域：认证、保密和密钥管理。这个 IPsec 规范是由数十个 RFC 文档和 IETF 草案文档组成的，这使得掌握所有的 IETF 规范显得非常复杂和困难。掌握 IPsec 的最好方法是查阅最新版本的 IPsec 文档目录，如[FRAN09]。所有的 IPsec 文档可以划分为如下几类：

- **体系结构**：包括 IPsec 的一般概念、安全需求、定义和机制。当前的规定是 RFC4301，Security Architecture for the Internet Protocol。
- **认证头（AH）**：AH 是一个用于提供消息认证的扩展头。当前的规范是 RFC4302，IP Authentication Header。由于消息认证是由 ESP 提供的，所以 AH 的使用是透明的。这是在 IPsecv3 中用来保证后向兼容性的并且无法在新的应用中使用。本章将不具体介绍 AH 的内容。
- **封装安全性有效载荷（ESP）**：ESP 包含了一个封装的头和尾用来提供加密或机密和认证的结合。当前的规范是 RFC 4303，IP Encapsulating Security Payload（ESP）。
- **因特网密钥交换（IKE）**：这是描述用于 IPsec 中的密钥交换方案的文档集合。主要的规范是 RFC 4306，Internet Key Exchange（IKEv2）Protocol，而且还有大量相关的 RFC 文档。

- **密码算法**:这一类中是大量定义和描述用于加密、消息认证、伪随机函数(PRF)和密钥交换的密码算法文档。
- **其他**:是其他与 IPsec 相关的 RFC 文档,包含那些处理安全策略和管理信息库(MIB)内容。

19.1.5 IPsec 服务

IPsec 通过允许系统选择所需的安全协议、决定服务所使用的算法和提供任何服务需要的密钥来提供 IP 级的安全服务。两种协议都提供安全性:一个是使用 AH 协议报头的认证协议,另一个是为数据包设计的加密/认证协议 ESP。RFC 4301 列出了如下服务:

- 访问控制
- 无连接完整性
- 数据源认证
- 拒绝重放包(部分顺序完整性格式)
- 保密性(加密)
- 限制流量保密性

19.1.6 传输和隧道模式

AH 和 ESP 均支持两种模式:传输模式和隧道模式,详细描述可以参阅 19.3 节,在此做简要介绍。

传输模式

传输模式主要是为上层协议提供保护,同时增加了 IP 包[①]载荷的保护。例如,TCP 段或 UDP 段、ICMP 包均是在主机协议栈的 IP 层进行操作。典型地,传输模式用于在两台主机(如服务器与工作站之间、两个工作站之间)进行的端到端通信。当一个主机在 IPv4 上运行 AH 或 ESP 时,其载荷是跟在 IP 报头后面的数据,对 IPv6 而言,其载荷是跟在 IP 报头后面的数据和 IPv6 的任何扩展头。

传输模式的 ESP 可以加密和认证(可选)IP 载荷,但不包括 IP 头。传输模式的 AH 可以认证 IP 载荷和 IP 头的选中部分。

隧道模式

隧道模式对整个 IP 包提供保护。为了达到这个目的,当 IP 包加 AH 或 ESP 域之后,整个数据包加安全域被当做一个新 IP 包的载荷,并拥有一个新的外部 IP 包头。原来或内部的整个包利用隧道在网络之间传输,沿途路由器不能检查内部 IP 包头。由于原来的包被封装,新的、更大的包可以拥有完全不同的源地址与目的地址,以增强安全性。当 SA 的一端或两端为安全网关时使用隧道模式,如使用 IPsec 的防火墙或路由器。防火墙外的主机在没有 IPsec 时也可以实现安全通信。而当主机生成的未保护包通过本地网络边缘的防火墙或安全路由器时,IPsec 提供隧道模式的安全性。

IPsec 如何操作隧道模式的例子如下。一个网络中的主机 A 生成与另一个网络中主机 B 作为目的地址的一个 IP 包,该包选择的路由是从源主机到 A 网络边界的防火墙或安全路由器;再由防火墙过滤所有的向外发送的包,来决定是否需要 IPsec 的处理。如果从 A 到 B 的包需要 IPsec 处

① 在本章中,术语 IP 包是指 IPv4 的报文或者是 IPv6 的数据包。

理,则防火墙执行 IPsec 处理并在外部 IP 头中封装包,外部 IP 包中的源 IP 地址为此防火墙的 IP 地址,目的地址可能为 B 本地网络边界的防火墙的地址。这样,包被传送到 B 的防火墙,而其间经过的中间路由器仅检查外部 IP 头;在 B 的防火墙处,除去外部 IP 头,内部的包被送往主机 B。

ESP 在隧道模式中加密和认证(可选)整个内部 IP 包,包括内部 IP 包头。AH 在隧道模式中认证整个内部 IP 包和外部 IP 头中的选中部分。

表 19.1 总结了传输模式和隧道模式的功能。

表 19.1　传输模式和功能

	传输模式 SA	隧道模式 SA
AH	认证 IP 载荷和 IP 头中的选中部分、IPv6 扩展头	认证整个内部 IP 包(内部包头和 IP 载荷)和部分选中的外部 IP 头、外部 IPv6 扩展头
ESP	加密 IP 载荷和跟在 ESP 头后的所有 IPv6 扩展头	加密内部 IP 包
带认证的 ESP	加密 IP 载荷和跟在 ESP 头后的所有 IPv6 扩展头;认证 IP 载荷,但没有 IP 头	加密内部 IP 包认证内部 IP 包

19.2　IP 安全性策略

IPsec 操作的基本概念是应用于每一个从源端到目的端传输的 IP 包的安全性策略。IPsec 策略主要由两个数据库的交互决定,这两个数据库是安全关联数据库(SAD)和安全性策略数据库(SPD)。本节将概要介绍这两种数据库并总结其在 IPsec 操作中的使用。图 19.2 描述其相互关系。

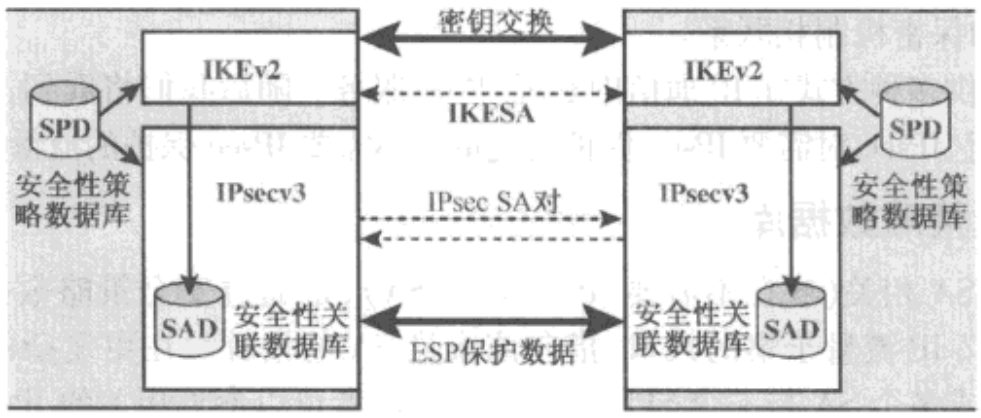


图 19.2　IPsec 结构

19.2.1　安全性关联

在 IP 的认证和保密机制中出现的一个核心概念是安全性关联(SA)。一个关联是一个发送方和接收方之间的单向关系,该关联为双方的通信提供安全服务。如果需要双向安全交换,则需要建立两个安全关联。安全服务可以由 AH 或 ESP 提供,但不能两者都提供。

一个安全关联由三个参数唯一确定:

- **安全参数索引(SPI)**:一个与 SA 相关的位串,仅在本地有意义。SPI 由 AH 和 ESP 携带,使得接收系统能选择合适的 SA 处理接收包。
- **IP 目的地址**:这是 SA 的目的地址,可以是用户终端系统、防火墙或路由器。
- **安全协议标志**:来自外部 IP 报头并标志该关联是一个 AH 安全关联或 ESP 安全关联。

然而,在任何 IP 包中,安全关联由 IPv4 或 IPv6 头中的目的地址唯一标志,SPI 被封装于 AH 或 ESP 扩展头中。

19.2.2 安全性关联数据库

在任何 IPSec 实现中,一个名义上[①]称之为安全关联数据库(SAD)的对象,该对象定义了每个 SA 的相关参数。一个 SAD 中的安全关联通常用以下参数定义:

- **安全参数索引**:由一个 SA 的接收终端选择一个 32 位的值来唯一标志该 SA。在一个通向外部 SA 的 SAD 入口中,SPI 用来创建 AH 包和 ESP 头部。在一个通向内部 SA 的 SAD 入口中 SPI 用来将通信映射到一个合适的 SA 上。
- **序列号计数器**:生成 AH 或 ESP 报头中序列号的 32 位值,详见 19.3 节(必须实现)。
- **序列号溢出标志**:标志序列号计数器是否溢出,生成审核事件。溢出时,阻止在此 SA 上继续传输包(必须实现)。
- **防止重放窗口**:用于判定一个内部 AH 或 ESP 数据包是否是重放,详见 19.3 节(必须实现)。
- **AH 信息**:认证算法、密钥、密钥生存期和 AH 的相关参数(AH 必须实现)。
- **ESP 信息**:加密和认证算法、密钥、初始值、密钥生存期和 ESP 的相关参数(ESP 必须实现)。
- **安全关联的生存期**:一个特定的时间间隔或字节计数。超过后,必须终止或由一个新的 SA(和新 SPI)替代,并加上相应的操作指示(必须实现)。
- **IPsec 协议模式**:隧道、传送或通配符。
- **Path MTU**:最大传输单元(不需要分段传输的最大包长度)路径和迟滞变量(必须实现)。

认证和保密机制与密钥管理机制相互独立。分布密钥使用的密钥管理机制只通过安全参数索引(SPI)与认证和保密机制相联系。

IPsec 为用户提供多种方式在 IP 通信中实现 IPsec 服务。随后我们将看到,SA 可以根据用户的意愿进行配置。并且,IPsec 对需要 IPsec 保护的流量和不需要 IPsec 保护的流量进行大粒度区分。

19.2.3 安全性策略数据库

IP 流量与特定 SA 相关(或在不需要 IPsec 时无 SA),是通过安全策略数据库(SPD)实现的。SPD 至少应包括定义 IP 流量子集的入口、指向该流量 SA 的指针。在更复杂的情况下,多个入口可与一个 SA 相连,或多个 SA 与一个 SPD 入口相关。读者可以参见相关的 IPsec 文档。

每个 SPD 入口由一个 IP 集合和上层协议定义,称为选择子。实际上,这些选择子过滤输出流量,并将它们映射到某个特定的 SA。每个 IP 包的输出过程遵守如下过程:

(1) 在 SPD 中,比较包中相应域的值(选择子域),寻找匹配的 SPD 入口,可能指向零个或多个 SA。

(2) 如果该包存在 SA,则选定 SA 和其关联 SPI。

(3) 执行所需的 IPsec 处理(如 AH 或 ESP 处理)。

SPD 入口由以下选择子决定:

- **远程 IP 地址**:可以是单一 IP 地址、一组或一定范围的地址和地址掩码。后两者要求多个目的系统共享相同的 SA(例如,位于防火墙之后)。
- **本地 IP 地址**:可以是单一 IP 地址、一组或一定范围的地址和地址掩码。后两者要求多个源系统共享相同的 SA(例如,位于防火墙之后)。
- **下一层协议**:IP 协议的报头(IPv4、IPv6 或者是扩展 IPv6)包含一个指定在 IP 之上运行的协议的数据域(IPv4 的协议,IPv6 或扩展 IPv6 的下一个报头)。这是一个独立的协议码。假

① 名义上在这里表示安全性关联数据库提供的功能必须在任何的 IPsec 实现中有所体现,但是该功能实现的具体形式却由生产商决定。

如使用 AH 或者是 ESP,IP 协议头将立即生成包中的 AH 和 ESP 头。

- **名称**:来自操作系统的用户标志。用户操作系统提供该标志,而不是 IP 或上层报头域提供。
- **本地和远程端口**:可以是单个 TCP 或 UDP 端口、一组端口和端口通配符。

表 19.2 提供了在主机系统(与防火墙或路由器等网络系统相对)上的 SPD 的例子。上述表反映了如下配置:本地网络配置包含两个网络。基本协作网络配置的 IP 是 1.2.3.0/24。本地配置中也包含一个安全的 LAN,通常称为 DMZ,地址为 1.2.4.0/24。DMZ 通过防火墙可以免于外部和协作 LAN 中的其他成员的攻击。例子中主机的 IP 地址是 1.2.3.10,并且接受了认证可以连接到 DMZ 中的 1.2.4.10 服务器。

表 19.2　主机 SPD 的例子

协　议	本地 IP	端　口	远程 IP	端　口	动　作	备　注
UDP	1.2.3.101	500	*	500	BYPASS	IKE
ICMP	1.2.3.101	*	*	*	BYPASS	错误消息
*	1.2.3.101	*	1.2.3.0/24	*	PROTECT:ESP 内传输模式	加密内联通信流
TCP	1.2.3.101	*	1.2.4.10	80	PROTECT:ESP 内传输模式	加密到服务器
TCP	1.2.3.101	*	1.2.4.10	443	BYPASS	TLS:避免双向加密
*	1.2.3.101	*	1.2.4.0/24	*	DISCARD	其他 DMZ
*	1.2.3.101	*	*	*	BYPASS	Internet

SPD 中的入口应该带自解释功能。例如,UDP 端口 500 是为 IKE 指定的端口号,任何为了 IKE 密钥交换目的的从本地主机到远程主机传输的数据都会经过 IPsec 处理。

19.2.4　IP 通信过程

IPsec 是在逐个包上执行的。当实现了 IPsec 时,每一个发往外部的 IP 包都会在发送之前由 IPsec 逻辑进行处理,而每一个发往内部的 IP 包也会在接收到之后并且在传递给上一层(如 TCP 或 UDP)之前由 IPsec 处理。我们依次来看一下这两种情形的过程。

向外发包

图 19.3 给出了向外通信的 IPsec 处理过程的主要元素。例如将 TCP 层的上一层的数据块传递到 IP 层并形成 IP 包,IP 包是由 IP 头和 IP 正文组成的。然后执行如下步骤:

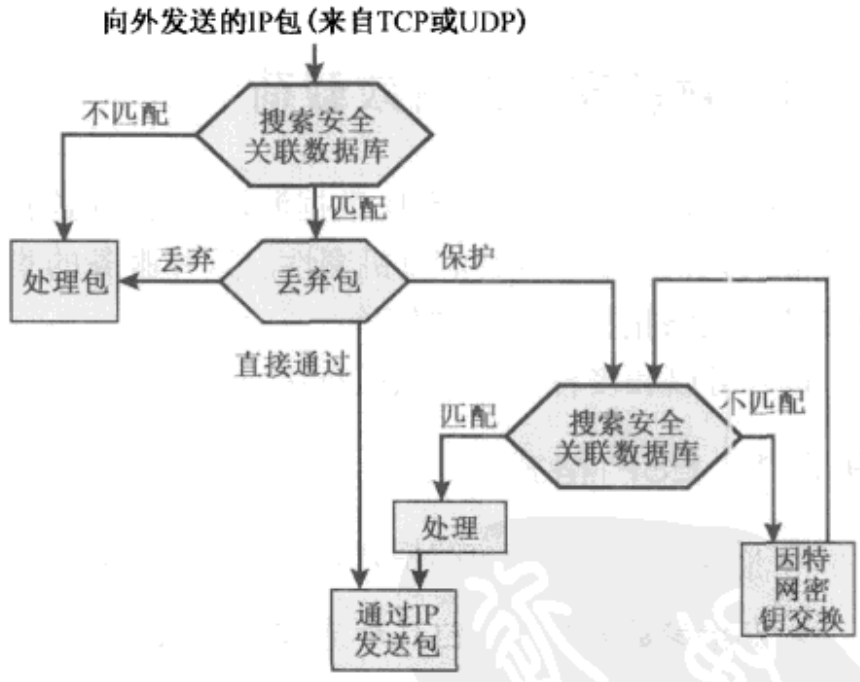


图 19.3　向外发包处理模型

(1) IPsec 搜索与该包匹配的 SPD。

(2) 假如未找到匹配的 SPD,则丢弃该包并生成错误信息。

(3) 假如找到匹配的 SPD,则由找到的第一个 SPD 入口决定往后的过程。假如对该包的策略是丢弃,则丢弃该包;假如对该包的策略是通过,则 IPsec 过程结束,IP 包就用于网络传输。

(4) 假如对该包的策略是保护,则搜索匹配的 SAD。如果没有找到 SAD,则唤醒 IKE 用合适的私钥生成 SA 以及 SA 的入口。

(5) 假如找到匹配的 SAD,则进一步的处理就由 SAD 决定。加密、认证或两者都执行,使用传输或隧道模式。然后 IP 包用于网络传输。

向内发包

图 19.4 给出了向内通信的 IPsec 处理过程的主要元素。一个到来的 IP 包触发 IPsec 处理过程,然后执行如下步骤:

(1) IPsec 通过检查 IP 协议域(IPv4)或下一个报头域(IPv6)来判断该包是一个非安全的 IP 包还是有 ESP 或 AH 头/尾的 IP 包。

(2) 假如是非安全的包,IPsec 搜索匹配的 SPD。假如第一个匹配的入口的策略是通过,则处理 IP 报头然后将包的正文传递给上一层,如 TCP 层。假如第一个匹配的入口的策略是保护或丢弃,或没有找到匹配的 SPD,则丢弃该包。

(3) 如果是安全的包,IPsec 就搜索 SAD。假如没有找到匹配的 SAD 入口,则丢弃该包;否则 IPsec 执行合适的 ESP 或 AH 过程。然后处理 IP 报头并将包正文发送给上一层,例如 TCP 层。

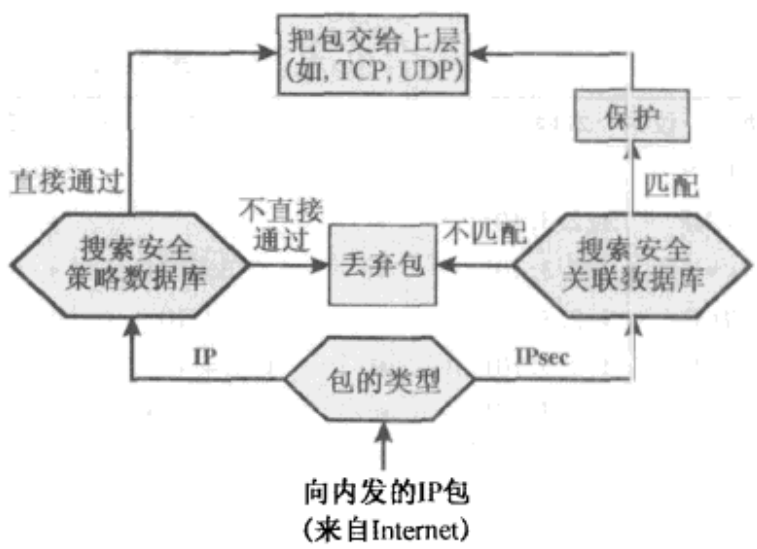


图 19.4 向内发包处理模型

19.3 封装安全性有效载荷

ESP 可以用来提供机密性、数据源认证性、无连接完整性和防止重放服务(一种部分序列完整性的形式)和(有限的)通信流机密性。这些服务的集合取决于建立安全性关联(SA)时选择的选项和其在网络拓扑中所处的位置。

ESP 可以和多种加密和认证算法协同工作,包括带认证的加密算法,如 GCM。

19.3.1 ESP 格式

图 19.5(a)是 ESP 包的格式,包含如下各域:

- **安全参数索引(32 位)**:标志安全关联。
- **序列号(32 位)**:单调递增计数值,提供防止重放功能。
- **载荷数据(可变)**:被加密保护的传输层分段(传输模式)或 IP 包(隧道模式)。
- **填充域(0 ~255 字节)**:此域的目的稍后讨论。
- **填充长度(8 位)**:填充数据的长度。
- **邻接头(8 位)**:标志载荷中第一个报头的数据类型(如 IPv6 中的扩展头或上层协议 TCP 等)。
- **完整性验证值(可变)**:变长域(必须为 32 位字长的整数倍),包含根据除认证数据域外的 ESP 包计算的完整性校验值。

当使用任何的结合型算法时,算法本身会被期望返回一个解密后的明文和指示完整性验证成功或失败的值。对于结合型算法,当选择检验完整性时,通常出现在 ESP 包尾部的 ICV 可以省略。

当 ICV 省略并且要求检验完整性时，结合型算法就负责将一个与 ICV 相当的验证性的包加入到载荷数据中。

当 ESP 中有加密或认证的需要时，在载荷中会出现两个额外的域[如图 19.5(b)所示]，一个是初始值(IV)或随机数。假如使用隧道模式，IPsec 可能会将通信流机密性(TFC)填充到载荷数据和填充域之间，随后将讲述其过程。

19.3.2　加密和认证算法

载荷数据、填充数据、填充长度和邻接头域在 ESP 中均被加密。如果加密载荷的算法需要初始向量 IV 这样的同步数据，则必须从载荷数据域报头取值，IV 通常作为密文的开头不被加密。

ICV 域是可选的，只有当选用了完整性服务时才存在，并且其要么是由单独的完整性算法提供，要么是由使用了 ICV 的结合型算法给出。ICV 的计算在对数据加密之后执行。按此顺序进行可以方便接收者在解密包之前快速检测并拒绝重放和伪造的包，因此在一定程度上降低了拒绝服务(DoS)攻击的影响。这也允许接收方可以对包并行处理，例如解密和完整性检测可以并行执行。值得注意的是，由于 ICV 没有用密码保护，因此必须使用一个基于密钥的完整性算法来计算 ICV。

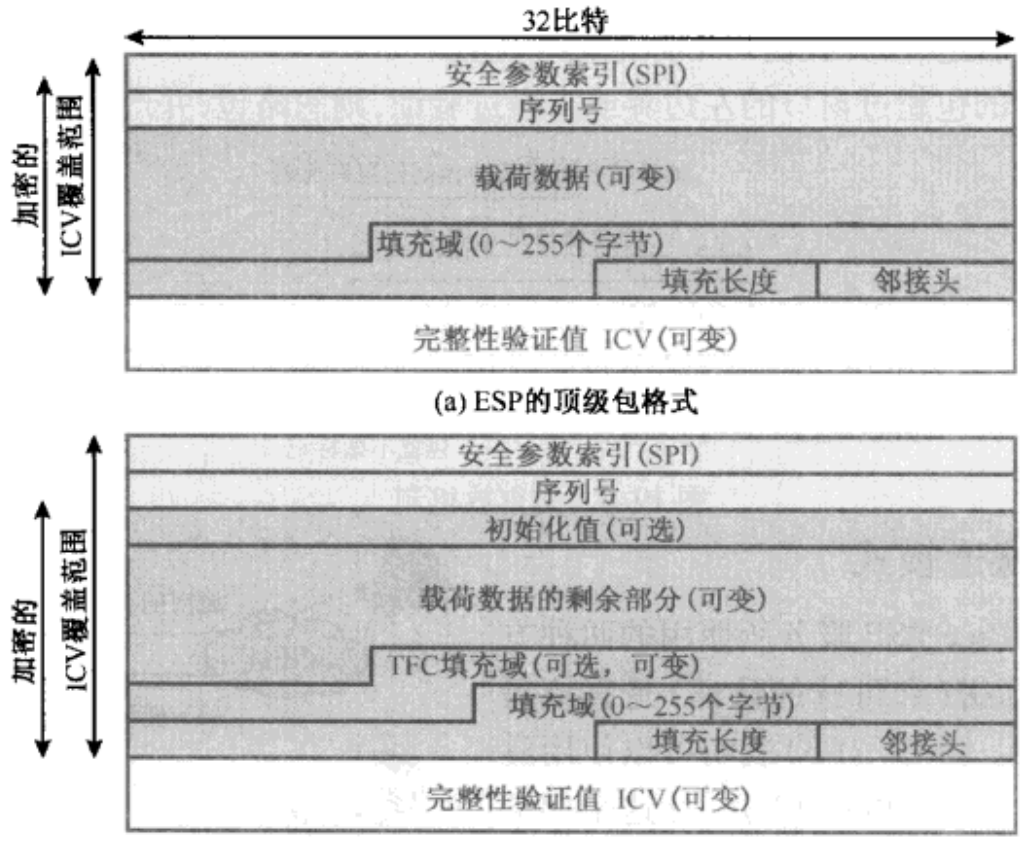


图 19.5　ESP 包格式

19.3.3　填充

填充域功能如下：

- 如果加密算法需要原文满足一定的长度要求，则填充域可用于扩展原文长度(包括载荷数据、填充数据、填充长度和邻接头域)到所需长度。
- ESP 格式需要填充长度和邻接头域为右对齐的 32 位字，以及密文长度需要 32 位的整数倍，不足位用填充域来确保。
- 增加额外的填充域可以隐藏载荷的实际长度，并提供部分流量保护。

19.3.4 防止重放服务

重放攻击是指攻击者在得到一个经过认证的包后,在以后的某个时刻将其传送到目的站点的行为。而重复接收经过认证的 IP 包可能会以某种方式中断服务或产生不可预料的后果。序列号域就可以防止上述攻击。首先,了解发送方如何生成序列号,再讨论接收方如何处理它。

当建立了一个新的 SA 时,发送方将序列号初值设为 0,每次在 SA 上发送一个包,则计数器加 1,将值写入序列号域,这样,使用的第一个值即为 1。如果要求反重放(默认设置),则发送方不允许循环计数(到达 $2^{32}-1$ 后返回 0),否则,同一个序列号可以有多个合法的包。如果该序列号达到 $2^{32}-1$,则 SA 必须终止,用新的密钥协商生成新的 SA。

由于 IP 是无连接、不可靠的服务,协议不能保证包按顺序传输,也不能保证所有的包均被传输。因此,IPsec 认证文档声明,接收方应实现一个大小为 W 的窗口(W 默认为 64)。窗口的右边界代表最大的序列号 N,记录目前收到的合法包的最大序列号,任何序列号在 $N-W+1$ 到 N 之间的包均可以被正确接收(如被验证),标记窗口的正确位置(如图 19.6 所示)。接收到包后的处理过程如下:

(1) 如果所接收的包在窗口中且是新包,则验证 MAC,如果验证通过,则在窗口中标记相应位置。

(2) 如果所接收的包超过窗口的右边界且是新包,则验证 MAC,如果验证通过,则以这个序列号为窗口的右边界,并在窗口中标记相应位置。

(3) 如果所接收的包超过窗口的左边界或未通过验证,则忽略包,并产生审核事件。

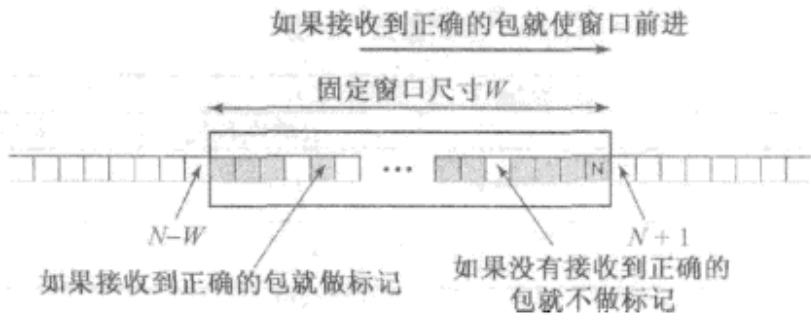


图 19.6 防重放机制

19.3.5 传输和隧道模式

图 19.7 给出了 IPsec ESP 服务所使用的两种方法。在图 19.7(a),加密(和可选的认证)是直接提供在两个主机之间的。图 19.7(b)表明可以使用隧道模式操作来建立虚拟专用网(VPN)。在这个例子中,该组织有 4 个通过因特网互联的虚拟专用网。在内部网络的主机使用因特网来传输数据,但是并不与其他基于因特网的主机交互。通过在安全网关处终止到内部网络的隧道可以使主机避免执行安全性。前者的技术是通过传输模式的 SA 实现,后者的技术则是使用隧道模式的 SA 实现。

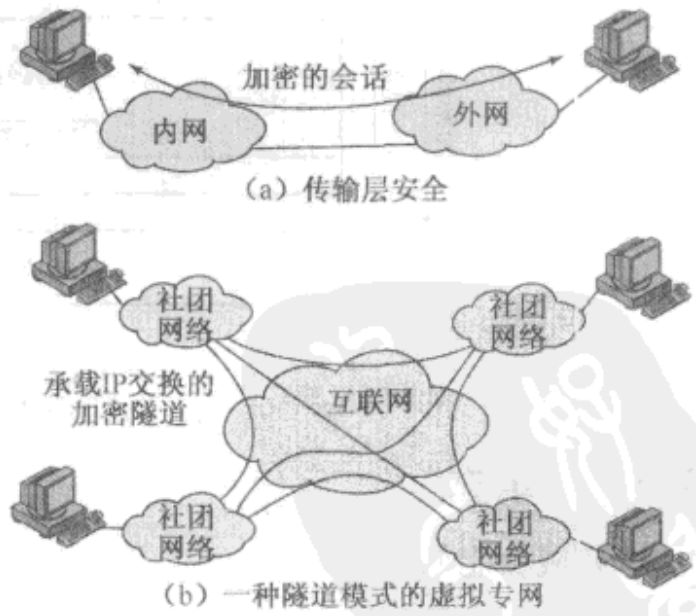


图 19.7 传输模式与加密隧道模式

在本小节中,我们大致了解 ESP 的两种模式。IPv4 和 IPv6 会有一些不同。开始我们使用图 19.8(a)的包格式。

传输模式的 ESP

传输模式的 ESP 使用加密和可选择的认证对由 IP(例如 TCP 报文)携带的数据进行处理,如图 19.8(b)所示。

在传输模式下使用 IPv4，ESP 头被直接插入到传输层头（如 TCP、UDP、ICMP）之前，并将 ESP 尾（填充、填充长度和下个头域）放在 IP 包之后。假如选择了认证，ESP 认证数据域就添加在 ESP 尾之后。对整个传输层报文和 ESP 尾部进行加密。认证则覆盖了整个密文和 ESP 头部。

在 IPv6 的环境下，ESP 被看做是端到端的载荷。即它不会被中间路由检查或处理。因此，ESP 头部在 IPv6 基本头部和多跳、目的地、路由和扩展报文头部之后。根据语义要求目的选项扩展头可以在 ESP 头之前也可以在其之后。同样，认证覆盖了密文和 ESP 头。

传输模式操作可以归纳如下：

（1）在源端，由 ESP 尾部和整个传输层报文组成的数据块是被加密的，并且用密文代替明文形成 IP 包来传输。如果有认证则将其加上。

（2）然后将包路由到目的地。每一个中间路由都会检查和处理 IP 报头和明文 IP 扩展头，但是不检查密文。

（3）目的节点检查和处理 IP 头和任何明文 IP 扩展头。然后，根据 ESP 中的 SPI 解密包中的剩余部分恢复成明文传输层报文。

传输模式操作为任何使用它的应用程序提供机密性，这样可以避免在每个主机上的应用程序实现机密性。该模式的一个缺陷是可以根据传输的数据包进行通信量分析。

隧道模式的 ESP

隧道模式的 ESP 对整个 IP 包加密［如图 19.8（c）所示］。对该模式，ESP 头预先放入了包中并加密了整个包和 ESP 尾部。该方法可以用来抵抗通信量分析。

因为 IP 头包含目的地址和源路由指示和跳转信息，所以用 ESP 头传输加密的 IP 包比较困难。中间路由器无法处理该数据包。因此需要用新的 IP 头来封装整个数据块（ESP 头加密文加认证数据），该 IP 头中包含有足够的路由信息但不足以用于通信量分析。

传输模式适合于保护主机间的连接来支持 ESP 特性，而隧道模式则对于配置包含路由器和其他类型的安全网关来保护可信网络免于外部网络的侵袭是很有用的。在后者的情形下，加密仅仅在外部主机和安全网关之间以及两个安全网关之间进行。这减缓了内部网络主机加密的负担，同时也通过减少需要密钥的实体来降低密钥分发的任务量。另外，这还可以防止基于最终终端的通信量分析。

考虑到外部的主机可能希望和在防火墙之后的内部网络通信，在这种情况下就将 ESP 实现在外部主机和防火墙上。如下给出了从外部主机向内部主机发送传输层报文的步骤：

（1）源端准备了一个内部 IP 包，其目的地址是目标内部主机，该报文事先加入了 ESP 头。然后该包和 ESP 会被加密并且可能会加上认证数据。处理后的数据块再用新的 IP 头（基本头和可选扩展头，如 IPv6 中的路由和跳转选项）封装，新的 IP 报头的目标地址是防火墙。这就形成了外部 IP 包。

（2）外部包又被路由到目的地防火墙。每一个中间路由都会检查和处理外部 IP 头和任何外部 IP 扩展头，但是不会检查密文。

（3）目的防火墙检查和处理外部 IP 头和任何外部 IP 扩展头。然后根据 ESP 头中的 SPI，解密包中剩余的部分来恢复明文形成内部 IP 包。然后将该包发送给内部网路。

（4）内部包在经过若干路由传递到目的地主机。

图 19.9 给出了这两种方法的协议结构。

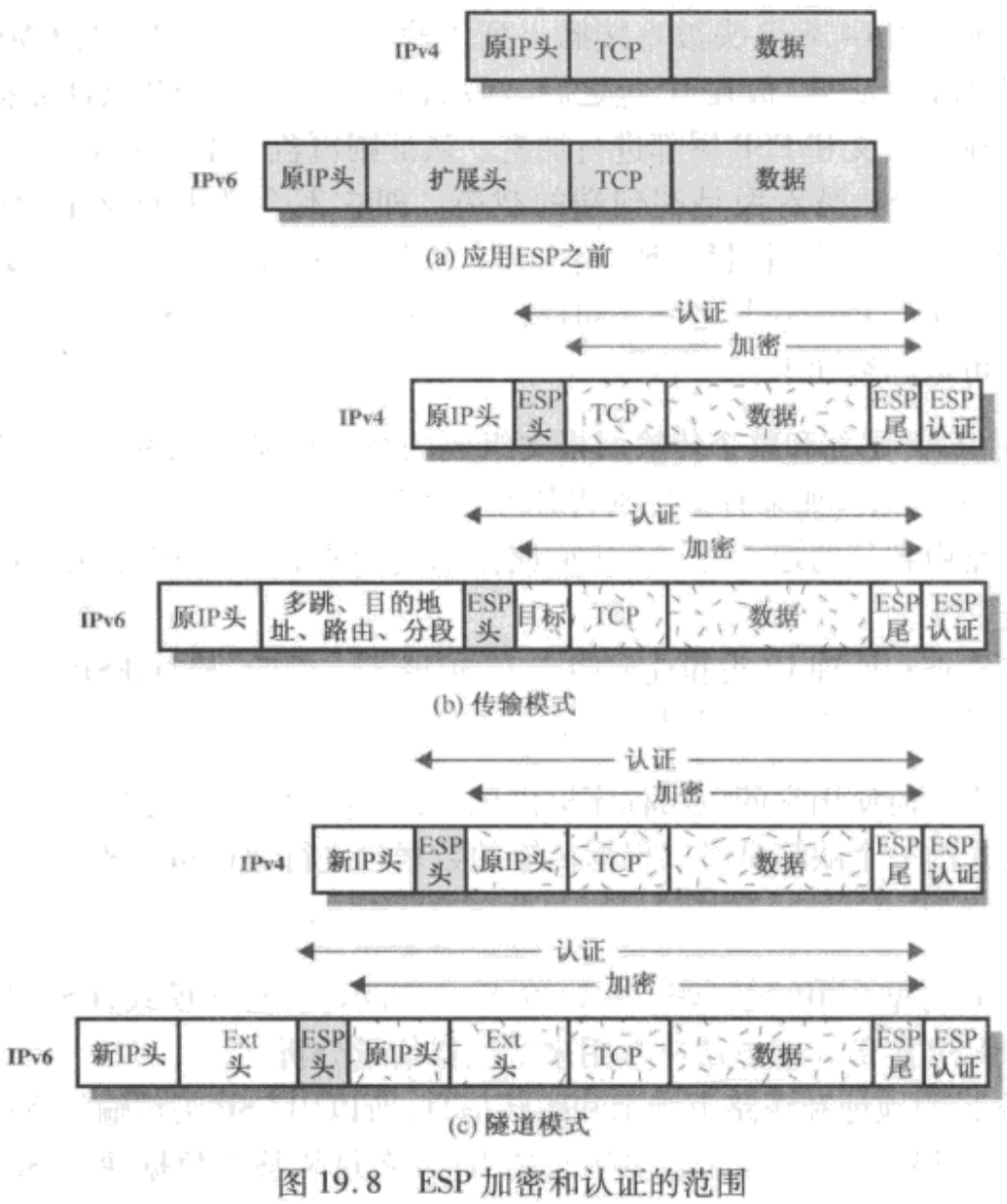


图 19.8 ESP 加密和认证的范围

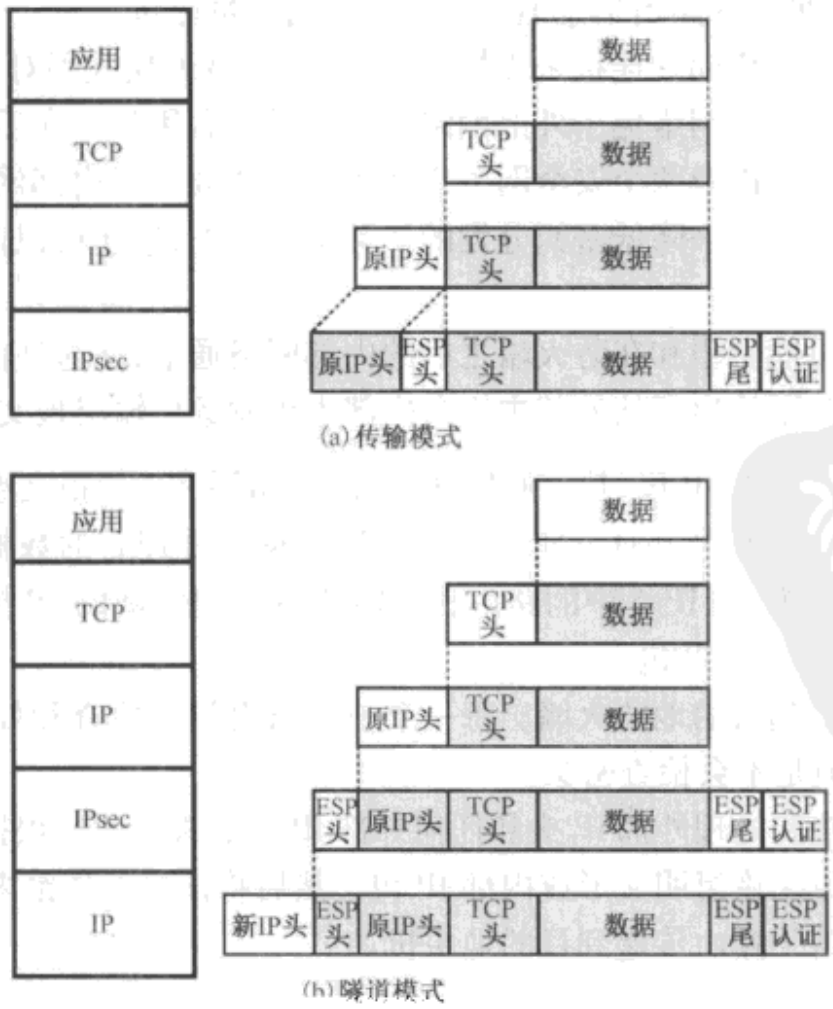


图 19.9 ESP 的协议操作

19.4　组合安全性关联

单个的 SA 可以实现 AH 或 ESP 协议，但不能两者都实现。有时，特定的流量需要在主机间提供 IPsec 服务，并在安全网关间（如防火墙间）为相同流量提供分离服务。此时，为了达到理想的 IPsec 服务，需要为相同流量提供多个 SA。安全关联束是指为提供特定的 IPsec 服务集所需的一个 SA 序列。安全关联束中的 SA 可以在不同节点终止，也可以在同一个节点终止。

安全关联可以通过两种方式来组合成束：

- **传输邻接**：指在不调用隧道的情况下，对一个 IP 包使用多个安全协议。组合 AH 和 ESP 的方法仅允许一级组合，因为对一个 IPsec 实例进行多次嵌套没有任何好处。
- **隧道迭代**：指通过 IP 隧道应用多层安全协议。由于每个隧道可以在路径上的不同 IPsec 节点起始和终止，因此，该方法允许多层嵌套。

两种方法可以组合使用（例如，在主机间使用传输 SA 而在安全网关之间的路径上使用隧道 SA）。

当考虑 SA 束时一个值得一提的问题是认证和加密的顺序和方法，我们将在后面详细讨论，下面了解至少涉及一个隧道的 SA 组合。

19.4.1　认证性和机密性

在主机间要求传送经过加密和认证的 IP 包时加密和认证是可以组合的，以下是一些组合途径。

带认证的 ESP

该方法如图 19.8 所示。用户首先对要保护的数据应用 ESP，然后加上认证数据，这又分为两种情况：

- **传输模式 ESP**：认证和加密仅作用于传送到主机的 IP 载荷，不保护 IP 头。
- **隧道模式 ESP**：认证作用于整个要发往外部 IP 目的地址（如防火墙）整个 IP 包，并在目的地进行验证。整个内部 IP 包在传往内部 IP 目的地时按专用机制保护。

两种情况，认证都只对密文使用而不是应用于明文。

传输邻接

另一种加密后认证的方法是使用两个绑定在一起传输 SA，内部是 ESP SA，外部是 AH SA。此时，ESP 没有认证。由于内部 SA 是一个传输 SA，仅对 IP 载荷加密。得到的包包含一个 IP 头（可能有 IPv6 扩展头）和 ESP。然后，使用传输模式的 AH，使得认证覆盖 ESP 和除可变域外的原始 IP 头。与简单地使用 ESP 带认证的 ESP SA 相比，此方法的好处在于认证覆盖了更多域，包括源 IP 地址和目的 IP 地址，缺点在于两次 SA 的开销大于一次 SA 的开销。

传输-隧道束

在加密之前认证有如下优点。首先，由于认证数据被加密保护，因此任何人都不可能从报文中截取和改变认证数据而不被发现；其次，可能希望在报文的目的地接收并保留认证信息以备后用。此时，在加密之前进行报文验证将更方便，否则，需要再次加密报文来保留认证的签名信息。

在两个主机间先认证后加密可以使用包含内部 AH 的传输 SA 和外部 ESP 的隧道 SA 的安全关联束。此时，认证作用于 IP 载荷和除可变域外的 IP 头（包括扩展），然后将隧道模式的 ESP 作用于得到的 IP 包，该数据包含经过认证的整个内部的密文和新的外部 IP 头。

19.4.2 安全关联的基本组合

IPsec 结构文档列举了 4 种 SA 组合,IPsec 的主机(如工作站、服务器)或安全网关(如防火墙、路由器)必须支持这些组合,如图 19.10 所示。图的下部表示元素的物理连接,上部表示一个或多个嵌套的 SA 逻辑连接。每个 SA 可以是 AH 或 ESP。对主机到主机的 SA 而言,模式可以是传输的或隧道的,否则,必须是隧道模式。

情况 1 为实施 IPsec 的终端系统间提供所需的安全机制,通过 SA 通信的任何两个终端系统,必须共享特定的密钥,可能的连接如下:

(a) 传输模式的 AH
(b) 传输模式的 ESP
(c) 传输模式 AH 后紧跟 ESP(AH SA 内置于 ESP SA)
(d) 在 AH 或 ESP 隧道模式中拥有(a)、(b)或(c)

我们已经讨论了如何用各种关联支持认证、加密、认证前加密和认证后加密。

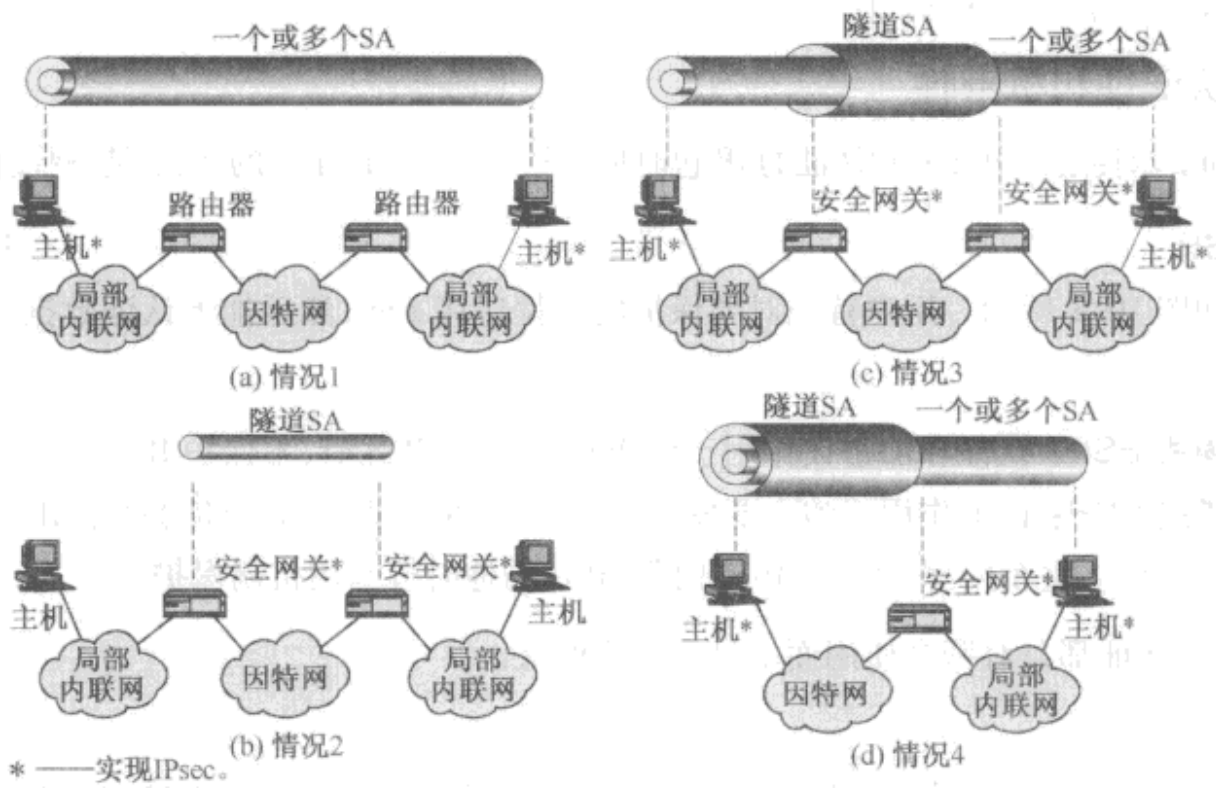


图 19.10 安全关联的基本组合

情况 2 仅在网关间提供安全保护,主机不实现 IPsec。此时,支持简单的虚拟专用网。隧道可支持 AH、ESP 或带认证的 ESP。由于 IPsec 将作用于整个内部包,故而不需要隧道嵌套。

情况 3 在情况 2 的基础上增加了端到端保护。情况 1 和情况 2 讨论的组合在此都允许。网关到网关的隧道对终端系统间的所有通信提供认证和/或保密。网关到网关的隧道 ESP 可对流量提供一定的保密性。单个主机可以根据特定应用实现任何额外的 IPsec 服务或为用户提供端到端的 SA。

情况 4 支持远程主机使用互联网到达企业的防火墙,访问防火墙后的某些服务器或工作站。在防火墙和远程用户之间仅需要隧道模式。如情况 1 一样,可在远程主机和本地主机间使用一到两个 SA。

19.5　因特网密钥交换

IPsec 的密钥管理包括密钥的确定和分发。两个应用程序需要 4 个密钥：发送和接收对的 AH 和 ESP。IPsec 体系结构文档要求支持两种密钥管理类型：

- **手动**：系统管理员手工地为每个系统配置自己的密钥和其他通信系统密钥，应用于小规模、相对静止的环境。
- **自动**：自动系统可以在大型分布系统中使用可变配置为 SA 动态按需创建密钥。

默认的 IPsec 自动密钥管理协议指的是 ISAKMP/Oakley，由以下元素组成：

- **Oakley 密钥确定协议**：Oakley 提供增值安全性，是基于 Diffie-Hellman 算法的密钥交换协议。Oakley 是通用的，没有任何特别格式。
- **互联网安全关联和密钥管理协议 ISAKMP**：ISAKMP 提供互联网密钥管理框架和特定协议支持，包括格式和安全属性协商。

ISAKMP 自身不包含特定的交换密钥算法而是由一系列使用各种交换密钥算法的报文类型集合组成。Oakley 是 ISAKMP 的第一个版本规定使用的交换密钥算法。

尽管在基本功能上两者是相似的，但在 IKEv2 中就不再使用 Oakley 和 ISAKMP 了，因为这个在 IKEv1 中使用的 Oakley 和 ISAKMP 有显著差别。在这一节中，我们将介绍 IKEv2 规范。

19.5.1　密钥确定协议

IKE 密钥确定协议是 Diffie-Hellman 交换密钥算法的细化。Diffie-Hellman 涉及用户 A 和用户 B 间的交互。在两个全局参数上首先达成一致：大素数 q 和 q 的生成元α。A 选择一个随机整数 X_A 作为它的私钥，将其公钥 $\gamma_A = \alpha^{X_A}$ 模 q 传给 B。同样，B 也选择一个随机整数 X_B 作为它的私钥，将其公钥 $\gamma_B = \alpha^{X_B}$ 模 q 传给 A。这样，双方即可计算它们的会话密钥：

$$K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = \alpha^{X_A X_B} \bmod q$$

Diffie-Hellman 算法有两个很好的特性：

- 仅在需要时创建密钥，不需要长时间的存放密钥，减少了泄露的可能性。
- 除了利用全局参数达成协议外，不需要额外的交换开销。

然而[HUIT98]指出该算法也有一些缺点：

- 不提供任何标志各方身份的信息。
- 易受中间人的攻击：第三方 C 可以在与 A 通信时冒充 B，而在与 B 通信时冒充 A。中间人攻击过程如下：

  (1) B 给 A 发送公钥 Y_B（如图 10.2 所示）。

  (2) 敌方 E 窃听到该消息，将 B 的公钥保存下来，并向 A 以 B 的用户标志发送带有 E 的公钥 Y_E 的报文。该报文伪装成从 B 的主机发送出来的形式，于是 A 接收了 E 的报文，将 E 的公钥和 B 的用户标志一起存储；同样地，E 伪装成 A 向 B 发送一个带有 E 公钥的报文。

  (3) B 在 B 的私钥和 Y_E 的基础上计算会话密钥 K_1，A 在 A 的私钥和 Y_E 的基础上计算会话密钥 K_2，E 使用 E 的私钥 X_E 和 Y_B 计算 K_1，使用 X_E 和 Y_A 计算 K_2。

  (4) 此后，E 就可以通过转接从 A 到 B 和从 B 到 A 的消息来获得 A 和 B 的通信内容，而 A 和 B 却无法知道他们在与 E 共享通信。

- 具有很强的可计算性。当攻击者申请许多密钥时,受害者很容易受阻塞攻击(clogging attack)。受害者需要花费相当多的资源做无意义的乘方、取模运算。

IKE 密钥确定是一种保持了 Diffie-Hellman 优点而去掉了其缺点的一种算法。

IKE 密钥确定特性

IKE 密钥确定算法有如下 5 个重要特性:

(1) 可以防止阻塞攻击。
(2) 双方可以协商得到一个组,本质上与 Diffie-Hellman 密钥交换的全局参数一样。
(3) 使用临时交互号防止重放攻击。
(4) 可以交换 Diffie-Hellman 的公钥值。
(5) 对 Diffie-Hellman 交换进行认证,防止中间人攻击。

我们已经讨论过 Diffie-Hellman,再来看看一些其他属性。首先,在阻塞攻击中,攻击者伪装成合法用户的源地址,发送一个 Diffie-Hellman 密钥给受害者。受害者于是执行乘方取模运算计算密钥,重复这类消息可以阻塞受害者的机器,使其系统无法正常工作。Cookie 交换要求各方在发给对方确认的初始报文中发送一个伪随机数 Cookie,此确认必须在 Diffie-Hellman 密钥交换的第一个报文中重复。如果源地址是伪造的,则攻击者不能得到该 Cookie。因此,攻击者只能要求用户生成确认报文但不可能要求用户执行 Diffie-Hellman 计算。

IKE 要求 Cookie 的生成应满足三个基本需求:

(1) Cookie 必须与特定的通话方相关。这样可以防止攻击者使用合法的 IP 地址和 UDP 端口获得 Cookie 后,将它用于其他 IP 地址和端口。
(2) 被某个实体承认的 Cookie 只能由其发行实体生成,不可能由其他实体生成,使得发行实体使用本地秘密信息生成 Cookie,继而验证它。并且,该秘密信息不可能从其他 Cookie 中推导出来。其本质在于发行实体不需要保存它所发行的 Cookie,当在需要时能验证收到的 Cookie,从而降低了被发现的可能性。
(3) Cookie 的生成和验证方法必须足够快,以防范企图占用处理器资源的攻击。

推荐创建 Cookie 的方法是根据 IP 的源地址、目的地址和 UDP 的源端口、目的端口和一个本地生成的秘密值能快速生成 Hash 值(如 MD5)。

IKE 密钥确定支持在 Diffie-Hellman 密钥交换时使用不同的组,每组包含两个全局参数和算法标志,目前,规范中包括如下组:

- 768 位模数的乘方取模

$$q = 2^{768} - 2^{704} - 1 + 2^{64} \times (\lfloor 2^{638} \times \pi \rfloor + 149\,686)$$
$$\alpha = 2$$

- 1024 位模数的乘方取模

$$q = 2^{1024} - 2^{960} - 1 + 2^{64} \times (\lfloor 2^{894} \times \pi \rfloor + 129\,093)$$
$$\alpha = 2$$

- 1536 位模数的乘方取模
  - 参数待定
- 2^{155} 的椭圆曲线
  - 生成器(十六进制):$X = 7B, Y = 1C8$

  - 椭圆曲线参数(十六进制):$A = 0, Y = 7338F$
- 2^{185}的椭圆曲线
  - 生成器(十六进制):$X = 18, Y = D$
  - 椭圆曲线参数(十六进制):$A = 0, Y = 1EE9$

前三种是使用乘方取模的传统 Diffie-Hellman 算法,后两种是使用椭圆曲线模拟 Diffie-Hellman,参见第 10 章。

IKE 密钥确定使用随机数来防止重放攻击。每个随机数是一个本地生成的伪随机数,它在应答中出现,并在交换的特定部分加密以对它进行保护。

在 IKE 密钥确定中使用了三种认证方法:

- **数字签名**:对双方均可取到的 Hash 进行签名来验证交换,各方用自己的私钥加密 Hash。Hash 在生成时使用重要参数,如用户 ID、随机数等。
- **公钥加密**:使用发送方的私钥对参数,如 ID、随机数加密来验证交换。
- **对称密钥加密**:使用其他方法传送密钥,再使用该密钥和对称加密算法对交换信息加密来验证交换。

IKEv2 交换

IKEv2 涉及消息的相互交换。交换的第一对称为初始交换[如图 19.11(a)所示]。在第一次交换中,对等实体间交换关于密码算法和其他安全参数的信息。这些安全参数是通信双方准备和随机数与 Diffie-Hellman(DH)共同使用的值。第一次交换的结果是建立一个称之为 IKE SA(如图 19.2所示)的特殊 SA。该 SA 定义了对等实体间用于进行后续交换的安全信道的参数。因此,所有随后的 IKE 信息交换都有加密和消息认证保护。在第二次交换中,双方进行相互认证并建立第一个 IPsec SA 放入 SADB 中用于保护两者间的通信数据(如非 IKE)。因此为了建立用于通信的第一个 SA 需要 4 个消息。

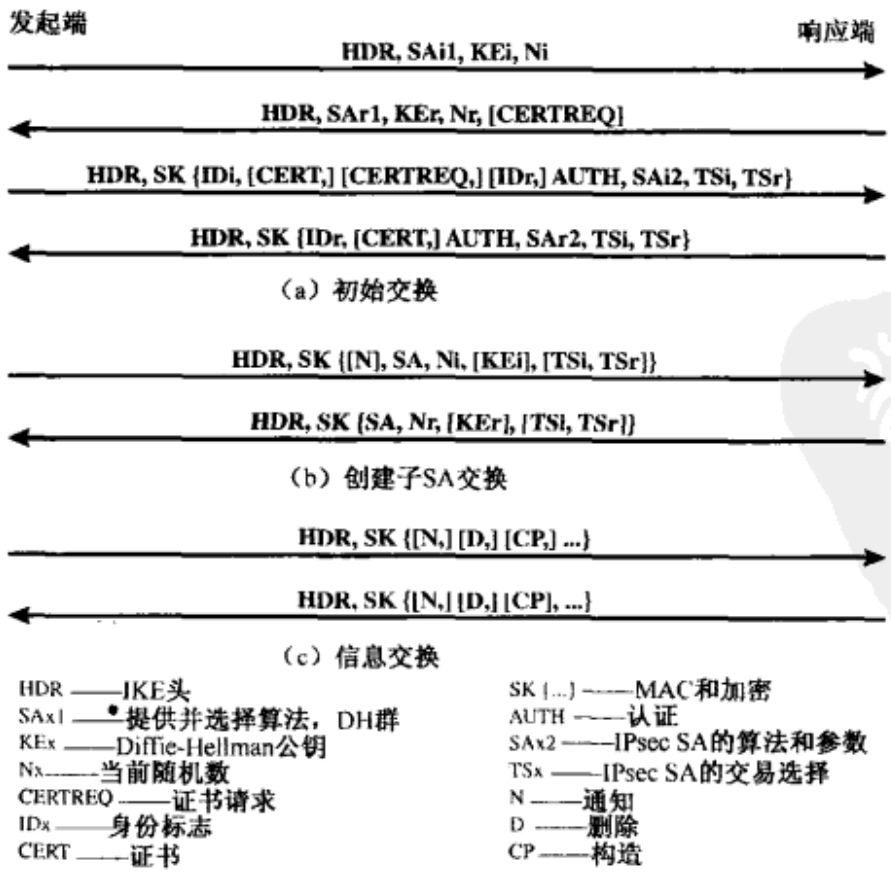


图 19.11　IKEv2 交换

CREATE_CHILD_SA 交换用于进一步建立为保护通信服务的 SA。信息交换用户交换管理信息、IKEv2 的错误信息和其他指示信息。

19.5.2 报头和有效载荷格式

IKE 定义了建立、协商、修改和删除安全关联的过程和包格式。IKE 定义了生成交换密钥的载荷和认证数据。载荷的格式提供了与特定的密钥交换协议、加密算法和认证机制无关的一致性框架。

IKE 报头格式

IKE 报文由 ISAKMP 头和一个或多个载荷组成,并包含在传输协议之中。规范指明了在实现时必须在传输协议中支持 UDP。

图 19.12(a)指明了 IKE 报文的头格式,它由以下域组成:

- **发起者 SPI(64 位)**:由发起者选择一个值,以确定一个独立的安全性关联(SA)。
- **响应者 SPI(64 位)**:由应答者选择一个值,以确定独立的 IKE SA。
- **邻接载荷(8 位)**:表明报文中第一个载荷的类型,载荷将在下一小节中讨论。
- **主版本号(4 位)**:使用的 IKE 的主版本号。
- **从版本号(4 位)**:使用的从版本号。
- **交换类型(8 位)**:表明交换类型,在本节中稍后讨论。
- **标志(8 位)**:IKE 交换的可选项集合,目前定义了两位:当跟在 IKE 报头后面的所有载荷都使用了此 SA 的加密算法加密,则设置加密位;在 SA 创建完成之前没有接收到任何加密消息时设置提交位。
- **报文标志(32 位)**:报文的唯一标志。
- **长度(32 位)**:报文(头 + 所有载荷)总的字节长度。

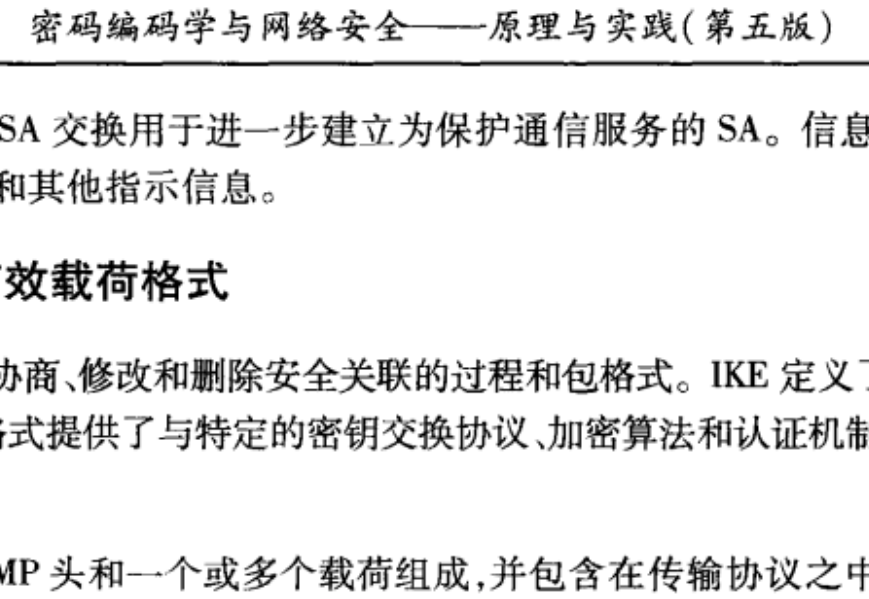


图 19.12 IKE 格式

IKE 载荷类型

所有 IKE 载荷开始于如图 19.12(b)所示的载荷头,而报文中最后一个载荷的邻接载荷域的值为 0。载荷长度域标明载荷头的该载荷字节长度。

当接收者不理解前一个载荷中下一个载荷域的载荷类型码,并且发送者希望其跳过该载荷时,critical 位就置为 0。而当接收者不理解载荷类型,并且发送者希望其拒绝整个消息时,将critical 置为 1。

表 19.3 总结了在 IKE 中定义的载荷类型，列举了每种载荷的部分域和参数。SA 载荷用于开始创建一个 SA。载荷是一个复杂的层次化结构，并包含有多个建议，每一个建议又包含多个协议，每个协议又包含多个转换，每个转换包含多个属性。载荷中的元素可以形式化为：

- **建议**：该结构包含一个建议号，协议标志（AH、ESP 或 IKE），转换号的表示和一个转换子结构。当一个建议中包含多个协议时，在一个建议号下就会有一个建议的子结构。
- **转换**：不同的协议支持不同的转换类型。转换通常是用来定义使用在特定协议中的密码算法。
- **属性**：每一个转换会包含多个用于修改和完成转换规范的属性，例如密钥长度。

密钥交换载荷可以被各种密钥交换技术使用，包括 Oakley、Diffie-Hellman 和 PGP 使用的基于 RSA 的密钥交换。密钥交换数据域包括生成会话密钥所需的数据，与所使用的密钥交换算法相关。

表 19.3　IKE 载荷类型

类　型	参　数
安全性关联	建议
密钥交换	DH 群#，密钥交换数据
标志	标志类型，标志数据
证书	证书编码，证书数据
证书请求	证书编码，证书中心
认证	认证方法，认证数据
随机数	随机数值
公告	协议标志，SPI 大小，公告消息类型，SPI，公告数据
删除	协议标志，SPI 大小，SPI 的#，SPI
生产商标志	生产商标志
通信选择器	通信选择器（TS）的值，通信选择器
加密	初始向量，加密的 IKE 载荷，填充，填充长度，ICV
配置	配置（CFG）类型，配置属性
扩展认证协议	扩展认证协议（EAP）消息

标志载荷用于确定通信方的标志和使用的认证信息。一般地，标志数据域包含 IPv4 或 IPv6 地址。证书载荷用于传送公钥证书。证书编码域标明证书类型或与证书相关的信息如下：

- PKCS#7 限制的 X.509 证书
- PGP 证书
- DNS 签名密钥
- X.509 签名证书
- X.509 密钥交换证书
- Kerberos 令牌
- 证书撤销表（CRL）
- 认证撤销表（ARL）
- SPKI 证书

在任何 IKE 交换中，发送方可以使用证书请求载荷去请求其他通信实体的证书。载荷必须列举可接受的多种证书类型和可接受的多个签证机构 CA。

在认证载荷中包含了用于消息认证目的的数据。当前定义了的认证方法类型有 RSA 数字签名、共享密钥消息完整性码和 DSS 数字签名。

随机数载荷包含用于交互的随机数据，以防止重放攻击。

通知载荷包含与 SA 或 SA 协商相关的出错或状态信息，下表列出了 IKE 公告信息。

错误消息	状态消息
Unsupported Critical Payload	Initial Contact
Invalid IKE SPI	Set Window Size
Invalid Major Version	Additional TS Possible
Invalid Syntax	IPCOMP Supported
Invalid Payload Type	NAT Detection Source IP
Invalid Message ID	NAT Detection Destination IP
Invalid SPI	Cookie
No ProposalChosen	Use Transport Mode
Invalid KE Payload	HTTP Cert Lookup Supported
Authentication Failed	Rekey SA
Single Pair Required	ESP TFC Padding Not Supported
No Additional SAS	Non First Fragments Also
Internal Address Failure	
Failed CP Required	
TS Unacceptable	
Invalid Selectors	

删除载荷表明发送方将一个或多个 SA 从它的数据库中删除,从而不再合法。

生产商标志载荷包含一个生产商定义的常量。生产商用该常量来标志并识别他们的设备。该机制可以使生产商在满足后向兼容性的同时尝试一些新的特性。

通信选择器载荷允许对等实体通过 IPsec 服务标志数据包的流。

加密的载荷包含其他载荷的加密形式。加密载荷的格式与 ESP 相似。当加密算法要求时会有初始向量(IV)并且当要求有认证时会有 ICV。

配置载荷用于 IKE 对等实体间交换配置信息。

扩展认证协议(EAP)载荷允许 IKE SA 用 EAP 进行认证,详见第 17 章。

19.6 密码学套件

IPsecv3 和 IKEv3 协议依赖于多种密码算法。如本书中所描述的,对于每种应用有多个密码算法,每种算法又有多个参数,例如密钥长度。为了提升互操作性,有两个 RFC 文档定义了各种应用的推荐密码算法和参数。

RFC 4308 定义了两个密码套件用于建立虚拟专用网。在 2005 年提出 IKEv2 时,VPN-A 与在 IKEv1 中使用的 VPN 是匹配的。VPN-B 提供了更强的安全性并且推荐在 IPsecv3 和 IKEv2 等新的 VPN 中使用。

表 19.4(a)列出了这两个套件的算法和参数。关于这两个套件还有一些说明。值得注意的是,对于对称加密算法,VPN-A 使用了 3DES 和 HMAC,而 VPN-B 使用了 AES。使用了三种密钥算法:

- **加密**:对于加密使用了加密块链(CBC)模式。
- **消息认证**:对于消息认证,VPN-A 使用了 HMAC 和 SHA-1 并且输出为 96 位,而 VPN-B 使用了 CMAC 并且输出也为 96 位。
- **伪随机函数**:IKEv2 通过重复使用用于消息认证的 MAC 来生成伪随机数。

RFC 4869 定义了四类符合美国国家安全局(NSA)B 套件规范的可选密码算法套件。在 2005 年,NSA 在密码现代化工程[LATT09]中提出了 B 套件,其中定义了保护敏感信息的算法和强度。在 RFC 4869 中定义的四种套件为 ESP 和 IKE 提供了选择。这四个套件根据密码算法强度和 ESP 中是同时提供机密性和完整性还是仅提供完整性来区分的。上述四种套件比 RFC 4308 中定义的两种 VPN 的安全强度更高一些。

表 19.4　IPsec 的密码套件

(a) 虚拟专用网(RFC 4308)

	VPN-A	VPN-B
ESP 加密	3DES-CBC	AES-CBC(128 位密钥)
ESP 完整性	HMAC-SHA1-96	AES-XCBC-MAC-96
IKE 加密	3DES-CBC	AES-CBC(128 位密钥)
IKE PRF	HMAC-SHA1	AES-XCBC-PRF-128
IKE 完整性	HMAC-SHA1-96	AES-XCBC-MAC-96
IKE DH 群	1024 位 MODP	2048 位 MODP

(b) NSA B 套件(RFC 4869)

	GCM-128	GCM-256	GMAC-128	GMAC-256
ESP 加密/完整性	AES-GCM(128 位密钥)	AES-GCM(256 位密钥)	Null	Null
ESP 完整性	Null	Null	AES-GMAC(128 位长度)	AES-GMAC(256 位长度)
IKE 加密	AES-CBC(128 位密钥)	AES-CBC(256 位长度)	AES-CBC(128 位长度)	AES-CBC(256 位长度)
IKE PRF	HMAC-SHA-256	HMAC-SHA-384	HMAC-SHA-256	HMAC-SHA-384
IKE 完整性	HAMC-SHA-256-128	HMAC-SHA-384-192	HMAC-SHA-256-128	HMAC-SHA-384-192
IKE DH 群	256 位随机 ECP	384 位随机 ECP	256 位随机 ECP	384 位随机 ECP
IKE 认证	ECDSA-256	ECDSA-384	ECDSA-256	ECDSA-384

表 19.4(b)列出了这两个套件的算法和参数。正如 RFC 4308 所述,有如下三类密钥算法:

- **加密**:对于 ESP,通过使用 128 位或 256 位 AES 密钥的 GCM 模式提供认证的加密。IKE 加密使用了 CBC,如 VPN 套件。
- **消息认证**:对于 ESP,假如只要求认证,则使用 GMAC。如第 12 章所述,GMAC 是 GMC 的认证部分。对于 IKE,消息认证通过使用 HMAC 和 SHA-3 中的某个 Hash 函数来提供。
- **位随机函数**:如 VPN 套件,IKEv2 通过重复使用用于消息认证的 MAC 来生成伪随机数。

对于 Diffie-Hellman 算法,规定使用椭圆曲线群上对素数求模。对于认证性,也有使用椭圆曲线上的数字签名。早期的 IKEv2 文档使用了基于 RSA 的数字签名。当使用 ECC 时可以使用更短的密钥来实现相当或更好的强度。

19.7　推荐读物和网站

[STAL07]详细叙述了 IPv6 和 IPv4。[CHEN98]很好地讨论了 IPsec 设计。[FRAN05]是 IPsec 的综合读物。[PATE06]是 IPsecv3 和 IKEv2 在密码方面的较好的综述性文章。

CHEN98　Cheng, P., et al. "A Security Architecture for the Internet Protocol." *IBM Systems Journal*, Number 1, 1998.

FRAN05　Frankel, S., et al. *Guide to IPsec VPNs.* NIST SP 800-77, 2005.

PATE06　Paterson, K. "A Cryptographic Tour of the IPsec Standards." "*Cryptology ePrint Archive: Report 2006/097*, April 2006."

STAL07　Stallings, W. *Data and Computer Communications, Eighth Edition.* Upper Saddle River, NJ: Prentice Hall, 2007.

推荐网站

- NIST IPsec Project :包括论文、介绍及相关引用。
- IPsec Maintenance and Extensions Charter: IPsec 方面最新的 RFC 和互联网草案。

19.8 关键术语、思考题和习题

关键术语

防止重放服务
因特网密钥交换(IKE)
重放攻击
认证头(AH)
IP 安全性(IPsec)
安全性关联(AS)
封装安全性有效载荷(ESP)
IPv4
传输模式
因特网安全性关联和密钥管理(ISAKMP)
IPv6
隧道模式
Oakley 密钥确定协议

思考题

19.1 给出 IPsec 的一个应用实例。
19.2 IPsec 提供哪些服务?
19.3 哪些参数标志 SA,哪些参数刻画一个特定 SA 的本质?
19.4 指出传输模式和隧道模式的区别?
19.5 什么是重放攻击?
19.6 为什么 ESP 包含填充域?
19.7 形成 SA 束有哪些基本方法?
19.8 IPsec 中 Oakley 密钥确定协议和 ISAKMP 的作用是什么?

习题

19.1 描述并解释表 19.2 中的每个实体。
19.2 为 AH 画一个与图 19.8 类似的示意图。
19.3 分别列出 AH 和 ESP 提供的主要安全性服务。
19.4 在讨论 AH 的处理中,曾经提到,并非 IP 头中的所有字段都参与 MAC 的计算。
(a) 对于 IPv4 头的每个域,指明哪些是不变的、哪些会变但可以预测、哪些是随意的(0 优先级 ICV 计算)。
(b) 对 IPv6 头的每一个域,问题同(a)。
(c) 对 IPv6 扩展头的每一个域,问题同(a)。
在各种情况下,判断你对各个域的理解。
19.5 假设当前的重放窗口的序号从 120 到 530。
(a) 假如下一个到来的认证包的序列号为 105,接收方如何处理该包,并且处理之后窗口的参数如何变化。
(b) 假如下一个到来的认证包的序列号为 440,接收方如何处理该包,并且处理之后窗口的参数如何变化。

(c) 假如下一个到来的认证包的序列号为 540,接收方如何处理该包,并且处理之后窗口的参数如何变化。

19.6 当使用隧道模式时,构造一个新的外部 IP 头。IPv4 和 IPv6 都指明了外部包的外部 IP 头的每个域和每个扩展头与内部 IP 包对应域或扩展头的关系。那么,请指出哪些外部数据是从内部数据继承的,哪些是由独立于内部数据重新构造的?

19.7 端对端的认证和加密应该在两台主机之间。参照图 16.8 中的表示:

(a) 传输邻接,认证前加密。

(b) 一个隧道 SA 中有一个传输 SA,认证前加密。

(c) 一个隧道 SA 中有一个传输 SA,加密前认证。

19.8 IPsec 构架文档中说,当两个传输模式 SA 被绑定,在同一个端对端流中允许 AH 和 ESP 两种协议,看起来只有一种安全协议比较合适:先实施 ESP 协议再实施 AH 协议。为什么不推荐先认证后加密?

19.9 对于 IKE 的密钥交换,在每个消息中哪些参数是哪种 ISAKMP 载荷类型。

19.10 IPsec 在协议栈中处于什么位置?

附录 A　用于密码学和网络安全教学的项目

A.1　Sage 计算机代数项目
A.2　黑客项目
A.3　分组密码项目
A.4　实验室练习
A.5　研究项目
A.6　编程项目
A.7　安全评估实践
A.8　书面作业
A.9　阅读/报告作业

Analysis and observation, theory and experience must never disdain or exclude each other; on the contrary, they support each other.

—*On War*, Carl Von Clausewitz

很多教师都相信做项目对于加深对密码学和网络安全的理解是非常重要的。没有项目，学生就很难掌握一些基本的概念和弄清楚各个构件之间的相互关系。项目可以加深学生对书本上概念的理解，使学生对加密算法和协议的工作原理有更深入的理解，这可以激励学生并让他们相信自己不仅能够掌握而且能够实现安全有关的技术细节。

本书将尽可能清楚地介绍密码学和信息安全方面的概念，并且提供很多的家庭作业来强化这些概念。然而，很多教师仍希望用做项目的方法来深入这些知识。本附录在这方面提供一些指导，并对于教师资源中心(IRC)中的可用资源进行介绍，这些教学资源可以在 Prentice Hall 网站上获取。这些资料包括 9 个方面的项目：

- Sage 计算机代数项目
- 黑客项目
- 分组密码项目
- 实验室练习
- 研究项目
- 编程项目
- 安全评估实践
- 书面作业
- 阅读/报告作业

A.1　Sage 计算机代数项目

本书第五版新增的一项最重要的内容就是使用 Sage 作为密码算法示例和作业。Sage 是开源、跨平台支持的免费软件，能够在数学和计算机代数系统的学习过程中提供强大、灵活和易学的软件包。计算机代数系统(CAS)是能够不仅进行数值运算，还能够进行符号运算的软件。CAS 用于教学已经有数十年的时间，目前已有大量相关文献。在开展密码学相关课程中，使用 CAS 作为实践的工具是再合适不过了。

与 Mathematica, Maple 和 MATLAB 等系统不同，Sage 没有专利保护和使用费的限制。因此 Sage 可以在学校的计算机和网络上使用，学生也可以分别将其下载到自己的个人计算机上在家里

使用。使用 Sage 的另外一个好处是学生可以掌握一个非常强大、灵活的工具来帮助计算几乎所有的数学问题，而不仅限于密码学。Sage 网站（http://www.sagemath.org）提供了大量文档，介绍如何在各种类型计算机平台上安装、设置和使用 Sage，以及如何通过 Web 在线使用 Sage。

在对密码算法数学基础的教学过程中，使用 Sage 能够显著增强教学效果。附录 B 提供了涵盖各种密码学概念的大量示例。附录 C 按照密码学概念的分类给出了习题集，使学生能够通过练习手把手地理解密码算法。附录 B 和附录 C 都在网上提供了电子文档，因此学生不需要再把附录中的代码逐行重复键入。

教师资源中心 IRC 提供了附录 C 中所有问题的答案。

附录 B 和附录 C 中的所有示例和作业都是由微软和 Washington 大学的 Dan Shumow 完成的。

A.2　黑客项目

本项目的目标是以黑客的身份通过一系列步骤侵入一个“公司”的网络。该公司的名字是 Extreme In Security，从名字可以看出该公司具有安全突破口，聪明的黑客能够通过侵入其网络来对其评估。IRC 需要对网站进行设置，学生的目标是获取关于公司下周询价的秘密信息，从而得到政府项目的合同。

学生首先从网站开始，找到进入网络的路径。在每一步中，如果学生成功完成了，就会有提示如何进行下一步骤的信息。

项目可以通过三种方法进行：

（1）不提供任何形式的帮助。

（2）提供一些提示。

（3）提供全部的指导。

本项目需要的文件在 IRC 中提供，包括：

（1）网站安全项目。

（2）网站黑客练习（XSS 和脚本攻击），包括客户端和服务器端漏洞扫描。

（3）上述功能的安装和使用文档。

（4）网站侵入的教学幻灯片（.ppt）。该文件通过抓屏，清晰演示了练习的过程。

该项目由达科他州立（Dakota State）大学的 Sreekanth Malladi 教授设计和实现。

A.3　分组密码项目

本实验对 AES 加密算法的操作过程进行跟踪，手工进行一轮计算，并使用不同的分组密码工作模式。实验也包括 DES 算法。每种情况下都由在线（或离线下载）Java 小程序来实现 AES 或 DES 的运算。

对于 AES 和 DES，实验都分为下面三部分：

- **分组密码内部**：该部分对明文加密，并考察每一轮的中间结果。对于 AES 和 DES 都有在线计算器提供中间结果和最终密文的计算结果。
- **分组密码轮函数**：该部分对单轮运算进行手工计算，正确性可与计算器提供的结果对比检查。
- **分组密码工作模式**：使学生能够对 CBC 模式和 CFB 模式进行比较。

IRC 提供了本实验的. html 和. jar 文件,这些文件在教师建立自己的实验网站中是必需的。这些材料也在本书网站的学生资源中提供,请点击相关链接。

本项目由澳大利亚国防研究院的 Lawrie Brown 提供。

A.4 实验室练习

作为教师资源中心的一部分,Purdue 大学的 Sanjay Rao 教授和 Ruben Torres 教授准备了一套实验室练习。这些实施项目设定在 Linux 系统上编程实现,但也适应于任何 UNIX 环境。这些实验室练习在实现安全功能和应用时为学生提供了实际经验。

A.5 研究项目

加深对课程中介绍的基本概念的理解和教给学生研究技巧的最有效的方法就是实施一个研究项目。这种项目可以是文献研究也可以是对供应商产品、研究室活动和标准化成果等进行网上搜索。项目可以分配给一个组,比较小的项目也可以分配给个人。无论如何,最好在一个学期早期做好各种项目的建议,给教师足够时间去评价相应的主题和其所取得的相应成绩。学生的科研报告应该包括以下几个部分:

- 提议的格式
- 最终报告的格式
- 中期和最终期限的进度安排表
- 可能项目的主题列表

学生们可以从 IRC 提供的主题列表中选择一个或者自己设计一个相当的项目。IRC 提供了一份包括提议和最终报告的推荐格式和一份具有 15 个相应研究性项目的主题列表。

A.6 编程项目

编程项目是一个有用的教学工具。像独立编制新的安全工具这种编程项目具有好几个引人入胜的特点:

(1) 教师能够从多个密码学和网络安全概念中选择布置项目。

(2) 学生可以在任何一台机器用任何语言进行编程;他们可以随意选择平台和语言。

(3)教师不需要为独立的项目下载、安装和配置特定的基础环境。

项目的大小具有灵活性。比较大的项目可以给学生们更多的成就感,但是能力不够或者组织性不强的学生可能落在后面。大项目通常会让好学生抽出更多的时间和精力进行练习。比较小的项目具有更高的概念 - 代码效率,由于大多数项目都可以分配下去,学生所涉及的领域更广。

再次,和研究项目一样,学生们应该首先提交报告。学生们的实验报告同样应该包括和上节所提到的那些要素。教师的工作就是准备 12 个相应的编程项目。

下面这些人提供了教师资源中心 IRC 的研究和编程项目,他们是:哥伦比亚大学的 Henning Schulzrinne,俄勒冈州立大学的 Cetin Kaya Koc,可信信息系统公司和乔治·华盛顿大学的 David M. Balenson。

A.7　安全评估实践

对于当前机构的现有架构和实现进行检查，是提高安全评估能力的最好办法。IRC 包含了一系列这样的活动。学生可以独立或分组来选择合适的小型或中型机构，然后访问这些机构的主管，结合其 IT 架构和实现得出对安全风险评估和检查的合适选择。结果他们能够提供一些合理的改进建议，来增强机构的安全性。这些活动帮助学生更好地理解现有的安全实现机制，增强对安全性进行评估和改进的能力。

本项目由澳大利亚国防研究院的 Lawrie Brown 提供。

A.8　书面作业

学习密码学和网络安全这样的技术类课程，书面作业可以起到事半功倍的效果。写编课程（WAC）活动（http://wac.colostate.edu）的忠实执行者报告了在促进学习方面书面作业的众多好处。书面作业可以使学生就某个专题做深入细致全面的思考。另外，书面作业可以克服学生的一种不好倾向：总是希望最快地学完一门课，只了解事实和解决问题的技巧，而不是深入理解课程的问题。

教师资源中心里按章推荐了很多书面作业。教师最后也许会发现这是讲授本课方法的最重要部分。如果你就这个问题有所反馈，并且推荐一些其他书面作业的话，我将非常感谢。

A.9　阅读/报告作业

另外一个可以加深学生对课堂知识理解和教给学生研究经验的有效方法是布置一些论文让学生们阅读和分析，要求学生对于指定的论文给出简要的读书报告。教师资源中心提供了建议论文列表，每章给出一到两篇。教师资源中心提供了每篇论文的 PDF 文件，教师资源中心还提供了一份推荐作业的关键词。

附录 B　Sage 示例

Dan Shumow
华盛顿大学(University of Washington)

B.1　线性代数和矩阵函数
B.2　第 2 章:古典密码
B.3　第 3 章:分组密码和数据加密标准 DES
B.4　第 4 章:数论和有限域基本概念
B.5　第 5 章:高级加密标准 AES
B.6　第 6 章:伪随机数发生器和流密码
B.7　第 8 章:数论
B.8　第 9 章:公钥密码和 RSA
B.9　第 10 章:其他公钥密码算法
B.10　第 11 章:密码学 Hash 函数
B.11　第 13 章:数字签名

本附录包括大量的密码实例,并按照本书章节的组织分类。所有的例子都是在 Sage 上实现的①。关于如何使用 Sage 以及 Sage 语法和操作的简要介绍请参见附录 C。我们首先简要介绍一些基本的 Sage 矩阵和线性代数运算。

本附录的例子应该很容易理解,如果读者对于 Sage 代码的理解有困难,可以参考附录 C.2。

B.1　线性代数和矩阵函数

Sage 包含线性代数的矩阵函数的计算功能。下面展示可用于密码学的一些基本函数。

在 Sage 中将矩阵表示为数字组成的列表,传递给 matrix 函数。例如可以做如下的数字列表的赋值:

```
sage: M = matrix([[1, 3],[7,9]]); M
[1 3]
[7 9]
```

如果矩阵元素是有理数,赋值方式如下:

```
sage: M = matrix([[1/2, 2/3, 3/4],[5, 7, 8]]); M
[1/2 2/3 3/4]
[  5   7   8]
```

还可以使用 IntegerModRing 函数(随后将介绍该函数)指定输入被取模:

```
Sage: R = IntegerModRing(100)
sage: M = matrix(R, [[1],[102],[1003]]); M
[1]
[2]
[3]
```

也可以指定输入是在有限域上(随后也将介绍)

```
sage: F = GF(2);
sage: M = matrix(F, [[1, 2, 0, 3]]); M
[1 0 0 1]
```

Sage 还支持矩阵的乘法、加法和逆矩阵,如下所示:

---

① 在本书的网站上有所有的 Sage 源代码,读者可以下载和执行任何想要的程序,访问信息请参见前言。

```
sage: M1 = matrix([[1, 2],[3,4]]);
sage: M2 = matrix([[1,-1],[1, 1]]);
sage: M1*M2
[3 1]
[7 1]

sage: M1 + M2
[2 1]
[4 5]

sage: M2^-1
[ 1/2 1/2]
[-1/2 1/2]
```

B.2　第 2 章:古典密码

下面是古典密码的例子和习题:

```
en_alphabet = "abcdefghijklmnopqrstuvwxyz"

#
# This function returns true if and only if the character
c is an
# alphabetic character
#
def is_alphabetic_char(c):
    return (c.lower() in en_alphabet)

#
# This function converts a single character into its
numeric value
#
def char_to_num(c):
    return en_alphabet.index(c.lower())

#
# This function returns the character corresponding to x
mod 26
# in the English alphabet
#
def num_to_char(x):
    return en_alphabet[x % 26]
```

例 1:对于密钥(整数 0,1,2,… ,25)和字符串,用 Sage 实现加/解密函数。加/解密函数只用对于字符'a','b',… ,'z'(都是大写或都是小写)操作,其他字符不做改变。

解:

```
def CaesarEncrypt(k, plaintext):
    ciphertext = ""
    for j in xrange(len(plaintext)):
        p = plaintext[j]
        if is_alphabetic_char(p):
```

```
            x = (k + char_to_num(p)) % 26
            c = num_to_char(x)
        else:
            c = p
        ciphertext += c
    return ciphertext

def CaesarDecrypt(k, ciphertext):
    plaintext = ""
    for j in xrange(len(ciphertext)):
        c = ciphertext[j]
        if is_alphabetic_char(c):
            x = (char_to_num(c) - k) % 26
            p = num_to_char(x)
        else:
            p = c
        plaintext += p
    return plaintext
```

例2:实现对于密文进行穷举攻击的程序,要求能够将密钥和对应的解密结果打印出来。还要求能够通过可选参数控制,根据选择子字符串作为参数,只打印潜在解密结果中的明文。

解:

```
def BruteForceAttack(ciphertext, keyword=None):
    for k in xrange(26):
        plaintext = CaesarDecrypt(k, ciphertext)
        if (None==keyword) or (keyword in plaintext):
            print "key", k, "decryption", plaintext
    return
```

例3:根据如下输入(密钥,明文)对,给出例1中加密函数的输出。

- k =16 明文 = "Get me a vanilla ice cream, make it a double. "
- k = 15 明文 = "I don't much care for Leonard Cohen. "
- k = 16 明文 = "I like root beer floats. "

解:

```
sage: k = 6; plaintext = 'Get me a vanilla ice cream,
make it a double.'
sage: CaesarEncrypt(k, plaintext)
'mkz sk g bgtorrg oik ixkgs, sgqk oz g juahrk.'

sage: k = 15; plaintext = "I don't much care for
Leonard Cohen."
sage: CaesarEncrypt(k, plaintext)
"x sdc'i bjrw rpgt udg atdcpgs rdwtc."

sage: k = 16; plaintext = "I like root beer floats."
sage: CaesarEncrypt(k, plaintext)
'y byau heej ruuh vbeqji.'
```

例 4:根据如下输入(密钥,密文)对,给出例 1 中解密函数的输出。

- k = 12 密文 = "nduzs ftq buzq oazqe. "
- k = 3 密文 = "fdhvdu qhhgv wr orvh zhljkw. "
- k = 20 密文 = "ufgihxm uly numnys. "

解:

```
sage: k = 12; ciphertext = "nduzs ftq buzq oazqe."
sage: CaesarDecrypt(k, ciphertext)
'bring the pine cones.'

sage: k = 3; ciphertext = "fdhvdu qhhgv wr orvh
zhljkw."
sage: CaesarDecrypt(k, ciphertext)
'caesar needs to lose weight.'

sage: k = 20; ciphertext = "ufgihxm uly numnys."
sage: CaesarDecrypt(k, ciphertext)
'almonds are tastey.'
```

例 5:对于如下密文,给出例 2 中穷举攻击程序的输出。如果指定了可选参数中的密钥字,传递给攻击程序。

- 密文 = 'gryy guru gob tab gb nzoebfr puncry. ' 密钥字 = 'chapel'
- 密文 = 'wziv kyv jyfk nyve kyv tpdsrcj tirjy. '密钥字 = 'cymbal'
- 密文 = 'baeeq klwosjl osk s esf ozg cfwo lgg emuz. ' 无密钥字

解:

```
sage: ciphertext = 'gryy gurz gb tb gb nzoebfr puncry.'
sage: BruteForceAttack(ciphertext, 'chapel')
key 13 decryption tell them to go to ambrose chapel.

sage: ciphertext = 'wziv kyv jyfk nyve kyv tpdsrcj tirjy.'
sage: BruteForceAttack(ciphertext, 'cymbal')
key 17 decryption fire the shot when the cymbals crash.

sage: ciphertext = 'baeeq klwosjl osk s esf ozg cfwo lgg emuz.'
sage: BruteForceAttack(ciphertext)
key 0 decryption baeeq klwosjl osk s esf ozg cfwo lgg emuz.
key 1 decryption azddp jkvnrik nrj r dre nyf bevn kff dlty.
key 2 decryption zycco ijumqhj mqi q cqd mxe adum jee cksx.
key 3 decryption yxbbn hitlpgi lph p bpc lwd zctl idd bjrw.
key 4 decryption xwaam ghskofh kog o aob kvc ybsk hcc aiqv.
key 5 decryption wvzzl fgrjneg jnf n zna jub xarj gbb zhpu.
key 6 decryption vuyyk efqimdf ime m ymz ita wzqi faa ygot.
key 7 decryption utxxj dephlce hld l xly hsz vyph ezz xfns.
key 8 decryption tswwi cdogkbd gkc k wkx gry uxog dyy wemr.
key 9 decryption srvvh bcnfjac fjb j vjw fqx twnf cxx vdlq.
key 10 decryption rquug abmeizb eia i uiv epw svme bww uckp.
key 11 decryption qpttf zaldhya dhz h thu dov ruld avv tbjo.
key 12 decryption posse yzkcgxz cgy g sgt cnu qtkc zuu sain.
key 13 decryption onrrd xyjbfwy bfx f rfs bmt psjb ytt rzhm.
key 14 decryption nmqqc wxiaevx aew e qer als oria xss qygl.
```

```
key 15 decryption mlppb vwhzduw zdv d pdq zkr nqhz wrr pxfk.
key 16 decryption lkooa uvgyctv ycu c ocp yjq mpgy vqq owej.
key 17 decryption kjnnz tufxbsu xbt b nbo xip lofx upp nvdi.
key 18 decryption jimmy stewart was a man who knew too much.
key 19 decryption ihllx rsdvzqs vzr z lzm vgn jmdv snn ltbg.
key 20 decryption hgkkw qrcuypr uyq y kyl ufm ilcu rmm ksaf.
key 21 decryption gfjjv pqbtxoq txp x jxk tel hkbt qll jrze.
key 22 decryption feiiu opaswnp swo w iwj sdk gjas pkk iqyd.
key 23 decryption edhht nozrvmo rvn v hvi rcj fizr ojj hpxc.
key 24 decryption dcggs mnyquln qum u guh qbi ehyq nii gowb.
key 25 decryption cbffr lmxptkm ptl t ftg pah dgxp mhh fnva.
```

B.3 第3章:分组密码和数据加密标准 DES

例1:实现 DES 的简化版本 SDES 的例子,SDES 在附录 G 中介绍。

```
#
# The Expansions/Permutations are stored as lists of
bit positions
#
P10_data = [3, 5, 2, 7, 4, 10, 1, 9, 8, 6];

P8_data = [6, 3, 7, 4, 8, 5, 10, 9];

LS1_data = [2, 3, 4, 5, 1];

LS2_data = [3, 4, 5, 1, 2];

IP_data = [2, 6, 3, 1, 4, 8, 5, 7];

IPinv_data = [4, 1, 3, 5, 7, 2, 8, 6];

EP_data = [4, 1, 2, 3, 2, 3, 4, 1];

P4_data = [2, 4, 3, 1];

SW_data = [5, 6, 7, 8, 1, 2, 3, 4];

#
# SDES lookup tables
#

S0_data = [[1, 0, 3, 2],
           [3, 2, 1, 0],
           [0, 2, 1, 3],
           [3, 1, 3, 2]];

S1_data = [[0, 1, 2, 3],
           [2, 0, 1, 3],
           [3, 0, 1, 0],
           [2, 1, 0, 3]];

def ApplyPermutation(X, permutation):
    r"""
    This function takes a permutation list (list of
    bit positions.)
    And outputs a bit list with the bits taken from X.
```

```
    """
    # permute the list X
    l = len(permutation);
    return [X[permutation[j]-1] for j in xrange(l)];

def ApplySBox(X, SBox):
    r"""
    This function Applies the SDES SBox (by table
    look up
    """
    r = 2*X[0] + X[3];
    c = 2*X[1] + X[2];
    o = SBox[r][c];
    return [o & 2, o & 1];

#
# Each of these functions uses ApplyPermutation
# and a permutation list to perform an SDES
# Expansion/Permutation
#

def P10(X):
    return ApplyPermutation(X, P10_data);

def P8(X):
    return ApplyPermutation(X, P8_data);

def IP(X):
    return ApplyPermutation(X, IP_data);

def IPinv(X):
    return ApplyPermutation(X, IPinv_data);

def EP(X):
    return ApplyPermutation(X, EP_data);

def P4(X):
    return ApplyPermutation(X, P4_data);

def SW(X):
    return ApplyPermutation(X, SW_data);

def LS1(X):
    return ApplyPermutation(X, LS1_data);

def LS2(X):
    return ApplyPermutation(X, LS2_data);

#
# These two functions perform the SBox substitution:
#
def S0(X):
    return ApplySBox(X, S0_data);

def S1(X):
    return ApplySBox(X, S1_data);

def concatenate(left, right):
```

```
    r"""
    Joins to bit lists together.
    """
    ret = [left[j] for j in xrange(len(left))];
    ret.extend(right);
    return ret;

def LeftHalfBits(block):
    r"""
    Returns the left half bits from block.
    """
    l = len(block);
    return [block[j] for j in xrange(l/2)];

def RightHalfBits(block):
    r"""
    Returns the right half bits from block.
    """
    l = len(block);
    return [block[j] for j in xrange(l/2, l)];

def XorBlock(block1, block2):
    r"""
    Xors two blocks together.
    """
    l = len(block1);
    if (l != len(block2)):
        raise ValueError, "XorBlock arguments must be
        same length"
    return [(block1[j]+block2[j]) % 2 for j in
    xrange(l)];

def SDESKeySchedule(K):
    r"""
    Expands an SDES Key (bit list) into the two round
    keys.
    """
    temp_K = P10(K);

    left_temp_K = LeftHalfBits(temp_K);
    right_temp_K = RightHalfBits(temp_K);

    K1left = LS1(left_temp_K);
    K1right = LS1(right_temp_K);

    K1temp = concatenate(K1left, K1right);
    K1 = P8(K1temp);

    K2left = LS2(K1left);
    K2right = LS2(K1right);

    K2temp = concatenate(K2left, K2right);

    K2 = P8(K2temp);

    return (K1, K2);
```

```
def f_K(block, K):
    r"""
    Performs the f_K function supplied block and K.
    """
    left_block = LeftHalfBits(block);
    right_block = RightHalfBits(block);
    temp_block1 = EP(right_block);
    temp_block2 = XorBlock(temp_block1, K);
    left_temp_block2 = LeftHalfBits(temp_block2);
    right_temp_block2 = RightHalfBits(temp_block2);
    S0_out = S0(left_temp_block2);
    S1_out = S1(right_temp_block2);
    temp_block3 = concatenate(S0_out, S1_out);
    temp_block4 = P4(temp_block3)
    temp_block5 = XorBlock(temp_block4, left_block);
    output_block = concatenate(temp_block5,
    right_block)
    return output_block;
def SDESEncrypt(plaintext_block, K):
    r"""
    Performs a single SDES plaintext block encryption.
    (Given plaintext and key as bit lists.)
    """
    (K1, K2) = SDESKeySchedule(K);
    temp_block1 = IP(plaintext_block);
    temp_block2 = f_K(temp_block1, K1);
    temp_block3 = SW(temp_block2);
    temp_block4 = f_K(temp_block3, K2);
    output_block = IPinv(temp_block4);
    return output_block;
```

B.4　第 4 章:数论和有限域基本概念

例 1:求最大公约数的欧几里得(Euclid)算法。

```
def EUCLID(a,b):
    r"""
    The Euclidean algorithm for finding the gcd of a and b.
    This algorithm assumes that a > b => 0
    INPUT:
```

```
        a - positive integer

        b - nonnegative integer less than a

    OUTPUT:

        g - greatest common divisor of a and b
        """

        if (b < 0) or ( a <= b):
            raise ValueError, "Expected 0 < a < b"

        (A, B) = (a,b);

        while (True):

            if (0 == B):
                return A;

            R = A % B;
            A = B;
            B = R;
```

例2:求最大公约数的扩展欧几里得算法。

```
def EXTENDED_EUCLID(m,b):
    r"""
    The extended Euclidean algorithm to find gcd(m,b).
    The input is expected to be such that 0 <= b < m.

    INPUT:

        m - positive integer

        b - nonnegative integer less than m

OUTPUT:

        (g, b_inv) - g is the gcd of m and b, b_inv is
        the multiplicative inverse of b mod m.

"""

if (m < b) or (b < 0):

        raise ValueError, "Expected input (0 < b < m)"

(A1,A2,A3) = (1,0,m);
(B1,B2,B3) = (0,1,b);

 while (True):

        if (0 == B3):
            return (A3, None)

        if (1 == B3):
            return (B3, B2)

        Q = floor(A3/B3)

        (T1,T2,T3) = (A1-Q*B1, A2-Q*B2, A3-Q*B3)
        (A1, A2, A3) = (B1, B2, B3)
        (B1, B2, B3) = (T1, T2, T3)
```

例 3:求多项式(系数为域上元素)gcd 的欧几里得算法。

```
def POLYNOMIAL_EUCLID(A, B):
    r"""
    Euclidian algorithm for polynomial GCD:
    Given two polynomials over the same base field,
    Assuming degree(A) => degree(B) => 0.

    INPUT:

        A - polynomial over a field.

        B - polynomial over the same field as A, and 0 <=
            degree(B) <= degree(A).

    OUTPUT:

        G - greatest common divisor of A and B.

    """
    degA = A.degree();
    degB = B.degree();

    if ((degB < 0) or (degA < degB)):
        raise ValueError, "Expected 0 <= degree(B) <= de-
        gree(A)"

    while(True):

        if (0 == B):
            return A;

        R = A % B;

        A = B;
        B = R;
```

例 4:求两个多项式(系数为同一域上的元素)gcd 的扩展欧几里得算法。

```
def POLYNOMIAL_EXTENDED_EUCLID(m, b):
    r"""
    Extended Euclidian algorithm for polynomial GCD:
    Given two polynomials over the same base field,
    Assuming degree(m) => degree(b) => 0

    INPUT:

        m - polynomial over a field.

    b - polynomial over the same field as A, and 0 <=
        degree(B) <= degree(M).

    OUTPUT:

        (g,b_inv) - the pair where:

        g - greatest common divisor of m and b.

        m_inv - is None if G is not of degree 0,
                and otherwise it is the polynomial such
                that b(X)*b_inv(X) = 1 mod m(X)
```

```
    """

    degm = m.degree();
    degb = b.degree();

    if(degb < 0) or (degm < degb):
        raise ValueError, "expected 0 <= degree(b) <=
        degree(m)"

    (A1, A2, A3) = (1, 0, m);
    (B1, B2, B3) = (0, 1, b);

    while (True):

        if (0 == B3):
            return (A3, None);

        if (0 == B3.degree()):
            return (B3/B3, B2/B3);

        Q = A3.quo_rem(B3)[0];

        (T1, T2, T3) = (A1 - Q*B1, A2 - Q*B2, A3 - Q*B3);
        (A1, A2, A3) = (B1, B2, B3);
        (B1, B2, B3) = (T1, T2, T3);
```

例 5:Sage 已经将 gcd 建成了函数库。通常的求最大公约数函数可直接调用:

```
sage: gcd(15,100)
5
sage: gcd(90,65311)
1
```

对于整数对象还可以用对象方法的方式调用:

```
sage: x = 10456890
sage: x.gcd(100)
10
```

Sage 的函数库还包括求最大公约数的扩展欧几里得算法。调用函数 xgcd(a,b)将返回三元组,第一个元素是 gcd,第二个和第三个元素是满足 gcd(a,b) = $u*a + v*b$ 的系数 u,v。可如下所示进行调用:

```
sage: xgcd(17,31)
(1, 11, -6)
sage: xgcd(10, 115)
(5, -11, 1)
```

对于整数对象还可以用对象方法的方式调用:

```
sage: x = 300
sage: x.xgcd(36)
(12, 1, -8)
```

例 6:Sage 提供对于有限域和有限域算术运算的鲁棒支持。如果要初始化一个素数阶的有限域,可以使用 GF 命令把阶作为参数传递。

```
sage: F = GF(2)
sage: F
Finite Field of size 2

sage: F = GF(37)
```

```
sage: F
Finite Field of size 37

sage: p = 95131
sage: K = GF(p)
sage: K
Finite Field of size 95131
```

如果要初始化一个阶为素数的幂的有限域,可以使用 GF 命令及如下符号定义(扩域中单位元的迹)。

```
sage: F.<a> = GF(128)
sage: F
Finite Field in a of size 2^7
```

有限域中的运算通过如下符号定义:

```
sage: K = GF(37)
sage: a = K(3)
sage: b = K(18)
sage: a - b
22
sage: a + b
21
sage: a * b
17
sage: a/b
31
sage: a^-1
25
sage: 1/a
25
```

在素数幂扩域中进行运算,可通过如下定义和操作有限域的元素:

```
sage: F.<a> = GF(128)
sage: b = a^2 + 1
sage: c = a^5 + a^3 + 1
sage: b - c
a^5 + a^3 + a^2
sage: b + c
a^5 + a^3 + a^2
sage: b*c
a^3 + a^2 + a
sage: b/c
a^5 + a^3 + a^2 + a
sage: b^-1
a^5 + a^3 + a
sage: 1/b
a^5 + a^3 + a
```

例 7:在 Sage 中可以直接创建有限域上的多项式环。可通过如下方法创建有限域上的多项式环:

```
sage: R.<x> = GF(2)[]
sage: R

Univariate Polynomial Ring in x over Finite Field of
size 2 (using NTL)
```

```
sage: R.<x> = GF(101)[]

sage: R
sage: R.<x> = F[]
sage: R
Univariate Polynomial Ring in x over Finite Field in
a of size 2^7
```

初始化多项式环后,可按如下示例执行各种运算:

```
sage: R.<x> = GF(2)[]
sage: f = x^3 + x + 1
sage: g = x^5 + x
sage: f + g
x^5 + x^3 + 1
sage: f*g
x^8 + x^6 + x^5 + x^4 + x^2 + x
```

函数 quo_rem 执行除法运算:

```
sage: g.quo_rem(f)
(x^2 + 1, x^2 + 1)
```

同样可以计算最大公因式:

```
sage: f.gcd(g)
1

sage: g.gcd(g^2)
x^5 + x
```

```
sage: R.<x> = GF(17)[]
sage: f = 3*x^3 + 2*x^2 + x
sage: g = x^2 + 5
sage: f - g
3*x^3 + x^2 + x + 12
sage: f * g
3*x^5 + 2*x^4 + 16*x^3 + 10*x^2 + 5*x
sage: f.quo_rem(g)
(3*x + 2, 3*x + 7)
```

在该多项式环上计算 gcd:

```
sage: f.gcd(g)
1

sage: f.gcd(x^2 + x)
x
```

当创建一个素数幂扩域,Sage 将找出一个不可约多项式。例如:

```
sage: F.<a> = GF(32)
a^5 + a^2 + 1
```

然而 GF(2) 上的 5 次不可约多项式不止一个,例如 x^5 + x^3 + 1。假设想要任选不可约多项式来建立二元域的 5 次扩域,可通过如下进行:

```
sage: R.<x> = GF(2)[]
sage: F = GF(2).extension(x^5 + x^3 + 1, 'a')
sage: a = F.gen()
```

这时需要在最后一步选择生成元。因为使用 GF 函数创建扩域时会自动选定生成元,但指定扩域时生成元不会自动选定。

B.5　第 5 章:高级加密标准 AES

例 1:简化版本 AES。

```
#
# These structures are the underlying
# Galois Field and corresponding Vector Space
# of the field used in the SAES algorithm
# These structures allow us to easily compute with these fields.
#
F = GF(2);
L.<a> = GF(2^4);
V = L.vector_space();
VF8 = VectorSpace(F, 8);
#
# The MixColumns and its Inverse matrices are stored
# as 2x2 matrices with elements in GF(2^4) (as are state matrices.)
# The MixColumns operation (and its inverse) are performed by
# matrix multiplication.
#
MixColumns_matrix = Matrix(L, [[1,a^2],[a^2,1]]);
InverseMixColumns_matrix = MixColumns_matrix.inverse();
SBox_matrix = Matrix(L,
  [
  [       1 + a^3,            a^2,            a + a^3, 1 + a + a^3],
  [ 1 + a^2 + a^3,              1,                a^3,     1 + a^2],
  [       a + a^2,              0,                  a,       1 + a],
  [     a^2 + a^3, a + a^2 + a^3, 1 + a + a^2 + a^3, 1 + a + a^2]
  ]);
InverseSBox_matrix = Matrix(L,
  [
  [   a + a^3,     1 + a^2,       1 + a^3,       1 + a + a^3],
  [         1, 1 + a + a^2,           a^3, 1 + a + a^2 + a^3],
  [   a + a^2,           0,             a,               1 + a],
  [ a^2 + a^3,         a^2, 1 + a^2 + a^3,     a + a^2 + a^3]
  ]);
RCON = [
  VF8([F(0), F(0), F(0), F(0), F(0), F(0), F(0), F(1)]),
  VF8([F(0), F(0), F(0), F(0), F(1), F(1), F(0), F(0)])
    ];
def SAES_ToStateMatrix(block):
    r"""
    Converts a bit list into an SAES state matrix.
    """
    B = block;
```

```
    # form the plaintext block into a matrix of GF(2^n)
    elements
    S00 = L(V([B[0], B[1], B[2], B[3]]));
    S01 = L(V([B[4], B[5], B[6], B[7]]));
    S10 = L(V([B[8], B[9], B[10], B[11]]));
    S11 = L(V([B[12], B[13], B[14], B[15]]));

    state_matrix = Matrix(L, [[S00,S01],[S10,S11]]);

    return state_matrix;

def SAES_FromStateMatrix(State Matrix):
    r"""
    Converts an SAES State Matrix to a bit list.
    """

    output = [];

    # convert State Matrix back into bit list
    for r in xrange(2):
        for c in xrange(2):
            v = V(State Matrix[r,c]);
            for j in xrange(4):
                output.append(Integer(v[j]));

    return output;

def SAES_AddRoundKey(state_matrix, K):
    r"""
    Adds a round key to an SAES state matrix.
    """

    K_matrix = SAES_ToStateMatrix(K);

    next_state_matrix = K_matrix + state_matrix;

    return next_state_matrix;
def SAES_MixColumns(state_matrix):
    r"""
    Performs the Mix Columns operation.
    """
    next_state_matrix = MixColumns_matrix*state_matrix;
    return next_state_matrix;
def SAES_InverseMixColumns(state_matrix):
    r"""
    Performs the Inverse Mix Columns operation.
    """
    next_state_matrix = InverseMixColumns_matrix*
    state_matrix;
    return next_state_matrix;
def SAES_ShiftRow(state_matrix):
    r"""
    Performs the Shift Row operation.
    """
    M = state_matrix;
```

```
    next_state_matrix = Matrix(L, [
                                   [M[0,0], M[0,1]],
                                   [M[1,1], M[1,0]]
                                   ]);
    return next_state_matrix;

def SAES_SBox(nibble):
    r"""
    Performs the SAES SBox look up in the SBox matrix
    (lookup table.)
    """
    v = nibble._vector_();
    c = Integer(v[0]) + 2*Integer(v[1]);
    r = Integer(v[2]) + 2*Integer(v[3]);
    return SBox_matrix[r,c];

def SAES_NibbleSubstitution(state_matrix):
    r"""
    Performs the SAES SBox on each element of an SAES state
    matrix.
    """
    M = state_matrix;
    next_state_matrix = Matrix(L,
          [ [ SAES_SBox(M[0,0]), SAES_SBox(M[0,1])],
            [ SAES_SBox(M[1,0]), SAES_SBox(M[1,1])] ]);
    return next_state_matrix;

def SAES_InvSBox(nibble):
    r"""
    Performs the SAES Inverse SBox look up in the SBox
    matrix (lookup table.)
    """
    v = nibble._vector_();
    c = Integer(v[0]) + 2*Integer(v[1]);
    r = Integer(v[2]) + 2*Integer(v[3]);
    return InverseSBox_matrix[r,c];

def SAES_InvNibbleSub(state_matrix):
    r"""
    Performs the SAES Inverse SBox on each element of an
    SAES state matrix.
    """
    M = state_matrix;
    next_state_matrix = Matrix(L,
    [ [ SAES_InvSBox(M[0,0]), SAES_InvSBox(M[0,1])],
       [ SAES_InvSBox(M[1,0]), SAES_InvSBox(M[1,1])] ]);
    return next_state_matrix;

def RotNib(w):
    r"""
    Splits an 8 bit list into two elements of GF(2^4)
    """
    N_0 = L(V([w[j] for j in xrange(4)]));
    N_1 = L(V([w[j] for j in xrange(4,8)]));
```

```
    return (N_1, N_0);

def SAES_g(w, i):
    r"""
    Performs the SAES g function on the 8 bit list w.
    """
    (N0, N1) = RotNib(w);
    N0 = V(SAES_SBox(N0));
    N1 = V(SAES_SBox(N1));
    temp1 = VF8( [ N0[0], N0[1], N0[2], N0[3],
                   N1[0], N1[1], N1[2], N1[3] ] );
    output = temp1 + RCON[i];
    return output;

def SAES_KeyExpansion(K):
    r"""
    Expands an SAES key into two round keys.
    """

    w0 = VF8([K[j] for j in xrange(8)]);
    w1 = VF8([K[j] for j in xrange(8,16)]);

    w2 = w0 + SAES_g(w1, 0);
    w3 = w1 + w2;

    w4 = w2 + SAES_g(w3, 1);
    w5 = w3 + w4;

    K0 = [w0[j] for j in xrange(8)];
    K0.extend([w1[j] for j in xrange(8)]);

    K1 = [w2[j] for j in xrange(8)];
    K1.extend([w3[j] for j in xrange(8)]);
    K2 = [w4[j] for j in xrange(8)];
    K2.extend([w4[j] for j in xrange(8)]);

    return (K0, K1, K2);

#
# Encrypts one plaintext block with key K
#
def SAES_Encrypt(plaintext, K):
    r"""
    Performs a SAES encryption on a single plaintext
    block.
    (Both block and key passed as bit lists.)
    """

    # get the key schedule
    (K0, K1, K2) = SAES_KeyExpansion(K);

    state_matrix0 = SAES_ToStateMatrix(plaintext);

    state_matrix1 = SAES_AddRoundKey(state_matrix0, K0);

    state_matrix2 = SAES_NibbleSubstitution
    (state_matrix1);
```

```
    state_matrix3 = SAES_ShiftRow(state_matrix2);
    state_matrix4 = SAES_MixColumns(state_matrix3);
    state_matrix5 = SAES_AddRoundKey(state_matrix4, K1);
    state_matrix6 = SAES_NibbleSubstitution
    (state_matrix5);
    state_matrix7 = SAES_ShiftRow(state_matrix6);
    state_matrix8 = SAES_AddRoundKey(state_matrix7, K2);
    output = SAES_FromStateMatrix(state_matrix8);
    return output;

#
# Decrypts one ciphertext block with key K
#
def SAES_Decrypt(ciphertext, K):
    r"""
    Performs a single SAES decryption operation on a
    ciphertext block.
    (Both block and key passed as bit lists.)
    """

    # perform key expansion
    (K0, K1, K2) = SAES_KeyExpansion(K);
    # form the ciphertext block into a matrix of GF(2^n)
    elements
    state_matrix0 = SAES_ToStateMatrix(ciphertext);
    state_matrix1 = SAES_AddRoundKey(state_matrix0, K2);
    state_matrix2 = SAES_ShiftRow(state_matrix1);
    state_matrix3 = SAES_InvNibbleSub(state_matrix2);
    state_matrix4 = SAES_AddRoundKey(state_matrix3, K1);
    state_matrix5 = SAES_InverseMixColumns
    (state_matrix4);
    state_matrix6 = SAES_ShiftRow(state_matrix5);
    state_matrix7 = SAES_InvNibbleSub(state_matrix6);
    state_matrix8 = SAES_AddRoundKey(state_matrix7, K0);
    output = SAES_FromStateMatrix(state_matrix8);
    return output;
```

B.6 第 6 章:伪随机数发生器和流密码

例 1:BBS(Blum Blum Shub)伪随机数发生器。

```
def BlumBlumShub_Initialize(bitlen, seed):
    r"""
    Initializes a Blum-Blum-Shub RNG State.

    A BBS-RNG State is a list with two elements:
    [N, X]
    N is a 2*bitlen modulus (product of two primes)
    X is the current state of the PRNG.

    INPUT:

        bitlen - the bit length of each of the prime
        factors of n

        seed - a large random integer to start out the prng

    OUTPUT:

        state - a BBS-RNG internal state

    """

    # note that this is not the most cryptographically
      secure
    # way to generate primes, because we do not know how the
    # internal sage random_prime function works.

    p = 3;
    while (p < 2^(bitlen-1)) or (3 != (p % 4)):
        p = random_prime(2^bitlen);

    q = 3;
    while (q < 2^(bitlen-1)) or (3 != (q % 4)):
        q = random_prime(2^bitlen);

    N = p*q;

    X = (seed^2 % N)

    state = [N, X]

    return state;

def BlumBlumShub_Generate(num_bits, state):
    r"""
     Blum-Blum-Shum random number generation function.

     INPUT:

        num_bits - the number of bits (iterations) to
        generate with this RNG.

        state - an internal state of the BBS-RNG (a
        list [N, X].)

     OUTPUT:

        random_bits - a num_bits length list of random
        bits.

        """
```

```
    random_bits = [];

    N = state[0]
    X = state[1]

    for j in xrange(num_bits):

       X = X^2 % N
       random_bits.append(X % 2)

    # update the internal state
    state[1] = X;

    return random_bits;
```

例 2:线性同余伪随机数发生器。

```
def LinearCongruential_Initialize(a, c, m, X0):
    r"""
    This functional initializes a linear congruential
    RNG state.

    This state is a list of four integers: [a, c, m, X]

    a,c,m are the parameters of the linear congruential
    instantiation  X is the current state of the PRNG.

    INPUT:

        a - The coefficient
        c - The offset
        m - The modulus
        X0 - The initial state

    OUTPUT:

         state - The initial internal state of the RNG

    """

    return [a,c,m,X0]

def LinearCongruential_Generate(state):
    r"""
    Generates a single linear congruential RNG output
    and updates the state.

    INPUT:

         state - an internal RNG state.

    OUTPUT:

         X - a single output of the linear congruential
        RNG.

    """

    a = state[0]
    c = state[1]
    m = state[2]
```

```
X = state[3]
X_next = (a*X + c) % m
state[3] = X_next
return X_next
```

B.7 第8章:数论

例1:中国剩余定理。

```
def chinese_remainder_theorem(moduli, residues):
    r"""
    Function that implements the chinese remainder
    theorem.

    INPUT:

      moduli - list or positive integers.

      residues - list of remainders such that remainder
at position j results when divided by the corresponding
modulus at position j in moduli.

    OUTPUT:

      x - integer such that division by moduli[j] gives
remainder residue[j].

    """
    if (len(moduli) != len(residues)):
     raise   ValueError,   "expected   len(moduli)   ==
len(residues)"
    M = prod(moduli);
    x = 0;
    for j in xrange(len(moduli)):
       Mj = moduli[j]
       Mpr = M/Mj
       (Mj_Mpr_gcd, Mpr_inv, Mj_inv) = xgcd(Mpr, Mj)
       Mpr_inv = Mpr_inv
       if (Mj_Mpr_gcd != 1):
        raise  ValueError,  "Expected  all  moduli  are
        coprime."
       x += residues[j]*Mpr*Mpr_inv;
    return x;
```

例2:Miller-Rabin 素性测试。

```
r"""
EXAMPLES:

    sage: MILLER_RABIN_TEST(101)
    False
```

```
        sage: MILLER_RABIN_TEST(592701729979)
        True

    """
def MILLER_RABIN_TEST(n):
        r"""

        This function implements the Miller-Rabin Test.
        It either returns "inconclusive" or "composite."

        INPUT:

           n - positive integer to probabilistically deter-
           mine the primality of.

        OUTPUT:

         If the function returns False, then the test was
        inconclusive.

           If the function returns True, then the test was
conclusive and n is composite.

        """

        R = IntegerModRing(n); # object for integers mod n
        # (1) Find integers k, q w/ k > 0 and q odd so that
        (n-1) == 2^k * q
        q = n-1
        k = 0
        while (1 == (q % 2)):
           k += 1
           q = q.quo_rem(2)[0] # q/2 but with result of type
            Integer

        # (2) select random a in 1 < a < n-1

        a = randint(1,n-1)

        a = R(a) # makes it so modular exponentiation is
        done fast


        # if a^q mod n == 1 then return inconclusive
        if (1 == a^q):
           return False

        # (3) for j = 0 to k-1 do: if a^(2^j * q) mod n =
n-1 return inconclusive
        e = q

        for j in xrange(k):
           if (n-1) == (a^e):
            return False
           e = 2*e

        # (4) if you've made it here return composite.
        return True
```

例3:模幂运算(平方和乘法)。

```
def ModExp(x,e,N):
    r"""
    Calculates x^e mod N using square and multiply.

    INPUT:

     x - an integer.
     e - a nonnegative integer.
     N - a positive integer modulus.

    OUTPUT:

     y - x^e mod N

    """
     e_bits = e.bits()
     e_bitlen = len(e_bits)

    y = 1

    for j in xrange(e_bitlen):

       y = y^2 % N

       if (1 == e_bits[e_bitlen-1-j]):
           y = x*y % N
    return y
```

例4:使用 Sage 内建函数计算中国剩余定理 CRT。

Sage 已经内建了执行中国剩余定理 CRT 的计算函数。对于各种类型中国剩余定理 CRT 的计算有多个函数可供选择。最简单的函数执行两个模的 CRT,其中 CRT(或小写 crt)调用方式如下:

```
crt(a,b,m,n)
```

结果会返回同时满足与 a 模 m 以及 b 模 n 同余的整数。其中所有参数都应是整数,并且参数 m 和 n 互素。该函数的示例如下:

```
sage: CRT(8, 16, 17, 49)
-3120
sage: CRT(1,2,5,7)
16

sage: CRT(50,64,101,127)
-62166
```

如果需要对更多的余数和模执行中国剩余定理,Sage 提供了 CRT_list 函数。

```
CRT_list(v, modulii)
```

该函数需要 v 和 modulii 都是整数列表且长度相同,并且 modulii 中的元素必须互素。输出是与所有 $v[i]$模 modulii$[i]$[这里 i 取遍 len(v)]都同余的整数。例如,上面最后一个对 CRT 函数的调用可以通过如下另一种方法实现:

```
sage: CRT_list([50,64],[101,127])
1969
```

注意答案同上例不同,当然你可以对两个答案检查是否满足中国剩余定理。更大规模的例子如下:

```
sage: CRT_list([8, 20, 13], [49, 101, 127])
608343

sage: CRT_list([10,11,12,13,14],[29,31,37,41,43])
36657170
```

CRT basis 函数可以用来预计算一组给定模的取值。如果 modulii 是一组互素的模，则 CRT_basis (modulii)返回一组 a。在列表 a 中，如果 x 是模 modulii 的剩余，则 CRT 的输出可通过如下求和得出：

```
a[0]*x[0] + a[1]*x[1] + ... + a[len(a)-1]*x[len(a)-1]
```

当使用 CRT_list 函数计算上面最后一个例子时，结果如下：

```
sage: CRT_basis([29,31,37,41,43])
[32354576, 20808689, 23774055, 17163708, 23184311]
```

Sage 提供的最后一个中国剩余定理函数是 CRT_vectors。该函数对数个不同的列表(每个列表的模相同)执行 CRT_list，并同时返回答案。这使得在使用相同的 CRT_basis 时更加高效，因为不需要对每个列表重复预计算。例如：

```
sage:
CRT_vectors([[1,10],[2,11],[3,12],[4,13],[5,14]],
[29,31,37,41,43])

[36657161, 36657170]
```

例 5：使用 Sage 内建函数进行模幂运算。

Sage 能够使用快速算法执行模幂运算(如平方和乘法)，且无须增大中间运算量。该功能通过 IntegerModRing 对象实现。特别地，创建 IntegerModRing 对象意味着运算是模算术，然后对环上的整数，其运算都将是取模运算。随即环上的所有元素的幂运算都将被快速执行。例如：

```
sage: R = IntegerModRing(101)
sage: x = R(10)
sage: x^99
91

sage: R = IntegerModRing(1024)
sage: x = R(111)
sage: x^345
751

sage: x = R(100)
sage: x^200
0

sage: N = 127*101
sage: R = IntegerModRing(N)
sage: x = R(54)
sage: x^95
9177
```

创建 IntegerModRing 对象同创建有限域 GF(...)类似，唯一区别是模数可以是普通的合数。

例 6：使用 Sage 内建函数进行欧拉函数运算。

Sage 已经内建了执行欧拉函数的运算。因为通常使用希腊字母 phi 来表示这个函数，所以该函数叫 euler_phi 函数。

```
sage: euler_phi(101)
100

sage: euler_phi(1024)
512

sage: euler_phi(333)
216

sage: euler_phi(125)
100

sage: euler_phi(423)
276
```

B.8 第9章:公钥密码和RSA

例1:使用Sage我们能够模仿执行RSA的加密和解密过程。

```
sage: # randomly select some prime numbers
sage: p = random_prime(1000); p
191
sage: q = random_prime(1000); q
601
sage: # compute the modulus
sage: N = p*q
sage: R = IntegerModRing(N)
sage: phi_N = (p-1)*(q-1)
sage: # we can choose the encrypt key to be anything
sage: # relatively prime to phi_N
sage: e = 17
sage: gcd(d, phi_N)
1
sage: # the decrypt key is the multiplicative inverse
sage: # of d mod phi_N
sage: d = xgcd(d, phi_N)[1] % phi_N
sage: d
60353
sage: # Now we will encrypt/decrypt some random 7
digit numbers

sage: P = randint(1,127); P
97

sage: # encrypt
sage: C = R(P)^e; C
46685
sage: # decrypt
sage: R(C)^d
97

sage: P = randint(1,127); P
46

sage: # encrypt
```

```
sage: C = R(P)^e; C
75843
sage: # decrypt
sage: R(C)^d
46

sage: P = randint(1,127); P
3
sage: # encrypt
sage: C = R(P)^e; C
288
sage: # decrypt
sage: R(C)^d
3
```

Sage 还能够容易地对更大的数进行运算：

```
sage: p = random_prime(1000000000); p
114750751
sage: q = random_prime(1000000000); q
8916569
sage: N = p*q
sage: R = IntegerModRing(N)
sage: phi_N = (p-1)*(q-1)
sage: e = 2^16 + 1
sage: d = xgcd(e, phi_N)[1] % phi_N
sage: d
2371507350934 73

sage: P = randint(1,1000000); P
955802
sage: C = R(P)^e
sage: R(C)^d
955802
```

例 2：我们还可以使用 Sage 进行 RSA 签名/验证。

```
sage: p = random_prime(10000); p
1601
sage: q = random_prime(10000); q
4073
sage: N = p*q
sage: R = IntegerModRing(N)
sage: phi_N = (p-1)*(q-1)
sage: e = 47
sage: gcd(e, phi_N)
1
sage: d = xgcd(e,phi_N)[1] % phi_N
sage: # Now by exponentiating with the private key
sage: # we are effectively signing the data
sage: # a few examples of this

sage: to_sign = randint(2,2^10); to_sign
650
```

```
sage: # the signature is checked by exponentiating
sage: # and checking vs the to_sign value
sage: signed = R(to_sign)^d; signed
2910116
sage: to_sign == signed^e
True
sage: to_sign = randint(2,2^10); to_sign
362
sage: signed = R(to_sign)^d; signed
546132
sage: to_sign == signed^e
True
sage: # we can also see what happens if we try to
verify a bad signature
sage: to_sign = randint(2,2^10); to_sign
605
sage: signed = R(to_sign)^d; signed
1967793
sage: bad_signature = signed - randint(2,100)
sage: to_sign == bad_signature^e
False
```

B.9 第 10 章:其他公钥密码算法

例 1:下面是 Sage 中 Alice 和 Bob 执行 Diffie-Hellman 密钥交换的例子。

```
sage: # Alice and Bob agree on the domain parameters:
sage: p = 619
sage: F = GF(p)
sage: g = F(2)
sage: # Alice picks a random value x in 1...618
sage: x = randint(1,618); x
571
sage: # Alice computes X = g^x and sends this to Bob
sage: X = g^571; X
591
sage: # Bob picks a random value y in 1...618
sage: y = randint(1,618);y
356
sage: # Bob computes Y = g^y and sends this to Alice
sage: Y = g^y; Y
199
sage: # Alice computes Y^x
sage: Y^x
563
sage: # Bob computes X^y
sage: X^y
563
sage: # Alice and Bob now share a secret value
```

例 2:为了防止小子群攻击,素数 p 的选择应满足 $p = 2q + 1$,这里 q 也是素数。

```
sage: q = 761
sage: p = 2*q + 1
sage: is_prime(q)
True
sage: is_prime(p)
True
sage: F = GF(p)
sage: g = F(3)
sage: g^q
1
sage: # note that g^q = 1 implies g is of order q
sage: # Alice picks a random value x in 2...q-1
sage: x = randint(2,q-1); x
312
sage: # Alice computes X = g^x and sends it to Bob
sage: X = g^x; X
26
sage: # Bob computes a random value y in 2...q-1
sage: y = randint(2,q-1); y
24
sage: # Bob computes Y = g^y and sends it to Alice
sage: Y = g^y; Y
1304
sage: # Alice computes Y^x
sage: Y^x
541
sage: # Bob computes X^y
sage: X^y
541
sage: # Alice and Bob now share the secret value 541
```

例 3:Sage 对于椭圆曲线的相关运算提供了大量的支持。因为这些功能可以使得计算过程简化，对于学习过程非常有用;如果直接手工计算这些例子则非常烦琐。首先通过指定所在的域,以及 Weierstrass 方程的系数,来选定椭圆曲线。为此我们把 Weierstrass 方程表示为:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

Sage 函数 EllipticCurve(R,[a_1,a_2,a_3,a_4,a_6])在环 R 上建立椭圆曲线。

```
sage: E = EllipticCurve(GF(17), [1,2,3,4,5])
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 +
2*x^2 + 4*x + 5 over Finite Field of size 17

sage: E = EllipticCurve(GF(29), [0,0,0,1,1])
sage: E
Elliptic Curve defined by y^2 = x^3 + x + 1 over
Finite Field of size 29

sage: E = EllipticCurve(GF(127), [0,0,0,2,17])
sage: E
Elliptic Curve defined by y^2 = x^3 + 2*x + 17 over
Finite Field of size 127

sage: F.<theta> = GF(2^10)
```

```
sage: E = EllipticCurve(F, [1,0,0,1,0])
sage: E
Elliptic Curve defined by y^2 + x*y = x^3 + x over
Finite Field in theta of size 2^10
```

例4: Koblitz 曲线。Koblitz 曲线是在二元域上由下式定义的椭圆曲线。

$$y^2 + xy = x^3 + ax^2 + 1$$

这里 $a = 0$ 或 1。FIPS 186-3 推荐了一系列 Koblitz 曲线用于数字签名标准(DSS)。这里我们给出 Koblitz 曲线的例子:

```
sage: F.<theta> = GF(2^17)
sage: E = EllipticCurve(F,[1,0,0,theta,1])
sage: E
Elliptic Curve defined by y^2 + y = x^3 + theta* x^2 = 1
over Finite Field in theta of size 2^17
```

例5: Sage 还能够指定曲线的密码学安全规模,如 FIPS 186-3 曲线之一 K163。

```
sage: F.<theta> = GF(2^163)
sage: E = EllipticCurve(F, [1,0,0,1,1])
sage: E
Elliptic Curve defined by y^2 + x*y = x^3 + x^2 + 1
over Finite Field in theta of size 2^163
```

然而指定曲线的密码学安全规模时必须小心,因为实际运行时曲线对象的某些函数需要指数时间运行而不能正常工作。如果需要进行这些计算的实验时,最好在小规模的曲线上进行。

还能够计算曲线的某些值,例如点的个数:

```
sage: E = EllipticCurve(GF(107), [0,0,0,1,0])
sage: E.order()
108
```

还可以判断曲线的生成元:

```
sage: E = EllipticCurve(GF(101), [0,0,0,1,0])
sage: E.gens()
((7 : 42 : 1), (36 : 38 : 1))
```

注意输出结果表示为$(x : y : z)$。这是出于快速实现的考虑,因为 Sage 使用"投影坐标"来存储曲线上的点。而精简表示的意义不大,因为对于非无穷远点,其值 z 的取值总是 1,而前两个值同预期一样取 x 坐标和 y 坐标。该表示方法的优点是可以将无穷远点表示为 z 坐标为 0 的点。

```
sage: E(0)
(0 : 1 : 0)
```

这样就可以识别一个点是否是无穷远点。如果想得到曲线上点的 x 坐标和 y 坐标,只需如下操作:

```
sage: P = E.random_point(); P
(62 : 38 : 1)
sage: (x,y) = P.xy(); (x,y)
(62, 38)
```

通过将排好的对投射到曲线上,可得到一个点:

```
sage: P = E((62,-38)); P
(62 : 63 : 1)
```

现在已经可以找到曲线的生成元并指定点,以及进行算术运算。继续使用前面例子中的曲线 E,可以设置 G1 和 G2 作为生成元:

```
sage: (G1, G2) = E.gens()
sage: P = E.random_point(); P
(49 : 29 : 1)
```

对于两个点的加法运算示例如下:

```
sage: G1 + G2 + P
(69 : 96 : 1)
sage: G1 + P
(40 : 62 : 1)
sage: P + P + G2
(84 : 25 : 1)
```

使用单目运算符负号(-) 可以计算点的逆:

```
sage: -P
(49 : 72 : 1)
sage: -G1
(7 : 59 : 1)
```

使用 * 运算符可以进行点的自加运算(点重复与自身相加):

```
sage: 13*G1
(72 : 23 : 1)
sage: 2*G2
(9 : 58 : 1)
sage: 88*P
(87 : 75 : 1)
```

对于较小规模有限域上的曲线可以计算阶(对于无穷远点的离散对数)。

```
sage: G1.order()
10

sage: G2.order()
10

sage: P.order()
10
```

例 6:使用 Sage 椭圆曲线函数执行椭圆曲线 Diffie-Hellman(ECDH)密钥交换。

```
sage: # calculate domain parameters
sage: F = GF(127)
sage: E = EllipticCurve(F, [0, 0, 0, 3, 4])
sage: G = E.gen(0); G
(94 : 6 : 1)
sage: q = E.order(); q
122

sage:  # Alice computes a secret value x in 2...
q-1
sage: x = randint(2,q-1); x
33
```

```
sage: # Alice computes a public value X = x*G
sage: X = x*G; X
(55 : 89 : 1)

sage: # Bob computes a secret value y in 2...q-1
sage: y = randint(2,q-1); y
55
sage: # Bob computes a public value Y = y*G
sage: Y = y*G; Y
(84 : 39 : 1)

sage: # Alice computes the shared value
sage: x*Y
(91 : 105 : 1)
sage: # Bob computes the shared value
sage: y*X
(91 : 105 : 1)
```

在很多实践中使用阶为素数的曲线:

```
sage: # Calculate the domain parameters
sage: F = GF(101)
sage: E = EllipticCurve(F, [0, 0, 0, 25, 7])
sage: G = E((97,34))
sage: q = E.order()
sage: # Alice computes a secret values x in 2...q-1
sage: x = randint(2,q-1)
sage: # Alice computes a public value X = x*G
sage: X = x*G
sage: # Bob computes a secret value y in 2...q-1
sage: y = randint(2,q-1)
sage: # Bob computes a public value Y = y*G
sage: Y = y*G
sage: # Alice computes the shared secret value
sage: x*Y
(23 : 15 : 1)
sage: # Bob computes the shared secret value
sage: y*X
(23 : 15 : 1)
```

B.10 第 11 章:密码学 Hash 函数

例 1:接下来是 Sage 中 Hash 函数 MASH 的例子。MASH 是基于模算术的算法,使用了类似 RSA 的模 M,其中 M 的长度决定安全强度。M 必须难于被分解因子,并且对于 M 的某些因子,安全性基于模运算下方根的困难性。M 同时还决定了处理消息分组的长度。本质上 MASH 的定义是:

$$H_i = ((x_i \oplus H_{i-1})^2 \mathrm{OR}\, H_{i-1}) \pmod{M}$$

这里

A 为 0xFF00…00

H_{i-1}为小于 M 的最大素数

x_i为对于输入 n 按 M 的幂方展开,所得结果的第 i 位。即我们将 n 表示为 M 进制的形式:

$$n = x_0 + x_1M + x_2M^2 + \cdots$$

接下来是 Sage 中 Hash 函数 MASH 的例子。

```
#
# This function generates a mash modulus
# takes a bit length, and returns a Mash
# modulus l or l-1 bits long (if n is odd)
# returns p, q, and the product N
#
def generate_mash_modulus(l):

    m = l.quo_rem(2)[0]

    p = 1
    while (p < 2^(m-1)):
    p = random_prime(2^m)

    q = 1
    while (q < 2^(m-1)):
    q = random_prime(2^m)

    N = p*q

    return (N, p, q)

#
# Mash Hash
# the value n is the data to be hashed.
# the value N is the modulus
# Returns the hash value.
#
def MASH(n, N):

     H = previous_prime(N)

     q = n

     while (0 != q):
      (q, a) = q.quo_rem(N)
      H = ((H+a)^2 + H) % N


     return H
```

该函数的输出如下：

```
sage: data = ZZ(randint(1,2^1000))
sage: (N, p, q) = generate_mash_modulus(20)
sage: MASH(data, N)
220874
sage: (N, p, q) = generate_mash_modulus(50)
sage: MASH(data, N)
455794413217080
sage: (N, p, q) = generate_mash_modulus(100)
sage: MASH(data, N)
```

```
268864504538508517754648285037
sage: data = ZZ(randint(1,2^1000))
sage: MASH(data, N)
236862581074736881919296071248
sage: data = ZZ(randint(1,2^1000))
sage: MASH(data, N)
395463068716770866931052945515
```

B.11 第13章:数字签名

例1:使用Sage能够执行DSA签名和验证:

```
sage: # First we generate the domain parameters
sage: # Generate a 16 bit prime q
sage: q = 1;
sage: while (q < 2^15): q = random_prime(2^16)
....:
sage: q
42697
sage: # Generate a 64 bit p, such that q divides (p-1)
sage: p = 1
sage: while (not is_prime(p)):
....: p = (2^48 + randint(1,2^46)*2)*q + 1
....:
sage: p
12797003281321319017
sage: # Generate h and g
sage: h = randint(2,p-2)
sage: h
5751574539220326847
sage: F = GF(p)
sage: g = F(h)^((p-1)/q)
sage: g
96705626822589458555

sage: # Generate a user public / private key
sage: # private key
sage: x = randint(2,q-1)
sage: x
20499
sage: # public key
sage: y = F(g)^x
sage: y
79550528281976107511

sage: # Sign and verify a random value
sage: H = randint(2,p-1)

sage: # Signing
sage: # random blinding value
sage: k = randint(2,q-1)
sage: r = F(g)^k % q
```

```
sage: r = F(g)^k
sage: r = r.lift() % q
sage: r
6805
sage: kinv = xgcd(k,q)[1] % q
sage: s = kinv*(H + x*r) % q
sage: s
26026

sage: # Verifying
sage: w = xgcd(s,q)[1]; w
12250
sage: u1 = H*w % q; u1
6694
sage: u2 = r*w % q; u2
16706
sage: v = F(g)^u1 * F(y)^u2
sage: v = v.lift() % q
sage: v
6805
sage: v == r
True

sage: # Sign and verify another random value
sage: H = randint(2,p-1)

sage: k = randint(2,q-1)
sage: r = F(g)^k
sage: r = r.lift() % q
sage: r
3284
sage: kinv = xgcd(k,q)[1] % q
sage: s = kinv*(H + x*r) % q
sage: s
2330

sage: # Verifying
sage: w = xgcd(s,q)[1]; w
4343
sage: u1 = H*w % q; u1
32191
sage: u2 = r*w % q; u2
1614
sage: v = F(g)^u1 * F(y)^u2
sage: v = v.lift() % q
sage: v
3284
sage: v == r
True
```

例 2:下面的函数实现 DSA 的参数域生成,密钥生成以及 DSA 签名。

```
#
# Generates a 16 bit q and 64 bit p, both prime
```

```
# such that q divides p-1
#
def DSA_generate_domain_parameters():

   g = 1

   while (1 == g):

        # first find a q
        q = 1
        while (q < 2^15): q = random_prime(2^16)
        # next find a p
        p = 1
        while (not is_prime(p)):
             p = (2^47 + randint(1,2^45)*2)*q + 1

         F = GF(p)

         h = randint(2,p-1)

         g = (F(h)^((p-1)/q)).lift()

  return (p, q, g)

#
# Generates a users private and public key
# given domain parameters p, q, and g
#
def DSA_generate_keypair(p, q, g):
    x = randint(2,q-1)

    F = GF(p)

    y = F(g)^x
    y = y.lift()

    return (x,y)

#
# Given domain parameters p, q and g
# as well as a secret key x
# and a hash value H
# this performs the DSA signing algorithm
#
def DSA_sign(p, q, g, x, H):

    k = randint(2,q-1)

    F = GF(p)
    r = F(g)^k
    r = r.lift() % q
    kinv = xgcd(k,q)[1] % q
    s = kinv*(H + x*r) % q
    return (r, s)
```

参考文献

In matters of this kind everyone feels he is justified in writing and publishing the first thing that comes into his head when he picks up a pen, and thinks his own idea as axiomatic as the fact that two and two make four. If critics would go to the trouble of thinking about the subject for years on end and testing each conclusion against the actual history of war, as I have done, they would undoubtedly be more careful of what they wrote.

—On War *Carl von Clausewitz*

缩　　写

ACM	Association for Computing Machinery(计算机协会)
IBM	International Business Machines Corporation(国际商用机器公司)
IEEE	Institute of Electrical and Electronics Engineers(电气电子工程师学会)

ACM04 The Association for Computing Machinery. *USACM Policy Brief: Digital Millennium Copyright Act (DMCA).* February 6, 2004. acm.org/usacm/Issues/DMCA.htm

ADAM90 Adams, C., and Tavares, S. "Generating and Counting Binary Bent Sequences." *IEEE Transactions on Information Theory*, 1990.

AGRA02 Agrawal, M.; Keyal, N.; and Saxena, N. "PRIMES is in P." *IIT Kanpur, Preprint*, August 2002. http://www.cse.iitk.ac.in/news/primality.pdf.

ANDR04 Andrews, M., and Whittaker, J. "Computer Security." *IEEE Security and Privacy*, September/October 2004.

AKL83 Akl, S. "Digital Signatures: A Tutorial Survey." *Computer*, February 1983.

ALVA90 Alvare, A. "How Crackers Crack Passwords or What Passwords to Avoid." *Proceedings, UNIX Security Workshop II*, August 1990.

ANDE80 Anderson, J. *Computer Security Threat Monitoring and Surveillance.* Fort Washington, PA: James P. Anderson Co., April 1980.

ANDE93 Anderson, R., et al. "Using the New ACM Code of Ethics in Decision Making." *Communications of the ACM*, February 1993.

ANTE06 Ante, S., and Grow, B. "Meet the Hackers." *Business Week*, May 29, 2006.

ASHL01 Ashley, P.; Hinton, H.; and Vandenwauver, M. "Wired versus Wireless Security: The Internet, WAP and iMode for E-Commerce." *Proceedings, Annual Computer Security Applications Conference*, 2001.

AUDI04 Audin, G. "Next-Gen Firewalls: What to Expect." *Business Communications Review*, June 2004.

AXEL00 Axelsson, S. "The Base-Rate Fallacy and the Difficulty of Intrusion Detection." *ACM Transactions and Information and System Security*, August 2000.

AYCO06 Aycock, J. *Computer Viruses and Malware.* New York: Springer, 2006.

BACE00 Bace, R. *Intrusion Detection.* Indianapolis, IN: Macmillan Technical Publishing, 2000.

BARK91 Barker, W. *Introduction to the Analysis of the Data Encryption Standard (DES),.* Laguna Hills, CA: Aegean Park Press, 1991.

BARK07a Barker, E., and Kelsey, J. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators.* NIST SP 800-90, March 2007.

BARK07b Barker, E., et al. *Recommendation for Key Management — Part 1: General.* NIST SP800-57, March 2007.

BARK07c Barker, E., et al. *Recommendation for Key Management — Part 2: Best Practices for Key Management Organization.* NIST SP800-57, March 2007.

BARK08 Barker, E., et al. *Recommendation for Key Management — Part 3: Specific Key Management Guidance.* NIST SP800-57, August 2008.

BARR05 Barrett, D.; Silverman, R.; and Byrnes, R. *SSH The Secure Shell: The Definitive Guide.* Sebastopol, CA: O'Reilly, 2005.

BAUE88 Bauer, D., and Koblentz, M. "NIDX—An Expert System for Real-Time Network Intrusion Detection." *Proceedings, Computer Networking Symposium*, April 1988.

BELL90 Bellovin, S., and Merritt, M. "Limitations of the Kerberos Authentication System." *Computer Communications Review*, October 1990.

BELL94a Bellare, M, and Rogaway, P. "Optimal Asymmetric Encryption — How to Encrypt with RSA." *Proceedings, Eurocrypt '94*, 1994.

BELL94b Bellovin, S., and Cheswick, W. "Network Firewalls." *IEEE Communications Magazine*, September 1994.

BELL96a Bellare, M.; Canetti, R.; and Krawczyk, H. "Keying Hash Functions for Message Authentication." *Proceedings, CRYPTO '96*, August 1996; published by Springer-Verlag. An expanded version is available at http://www-cse.ucsd.edu/users/mihir.

BELL96b Bellare, M.; Canetti, R.; and Krawczyk, H. "The HMAC Construction." *CryptoBytes*, Spring 1996.

BELL97 Bellare, M., and Rogaway, P. "Collision-Resistant Hashing: Towards Making UOWHF's Practical." *Proceedings, CRYPTO '97*, 1997; published by Springer-Verlag.

BELL00 Bellare, M.; Kilian, J.; and Rogaway, P. "The Security of the Cipher Block Chaining Message Authentication Code." *Journal of Computer and System Sciences*, December 2000.

BERL84 Berlekamp, E. *Algebraic Coding Theory*. Laguna Hills, CA: Aegean Park Press, 1984.

BETH91 Beth, T.; Frisch, M.; and Simmons, G. eds. *Public-Key Cryptography: State of the Art and Future Directions*. New York: Springer-Verlag, 1991.

BHAT03 Bhatkar, S.; DuVarney, D.; and Sekar, R. "Address Obfuscation: An Efficient Approach to Combat a Broad Range of Memory Error Exploits." *Proceedings, 12th Unix Security Symposium*, 2003.

BHAT07 Bhatti, R.; Bertino, E.; and Ghafoor, A. "An Integrated Approach to Federated Identity and Privilege Management in Open Systems." *Communications of the ACM*, February 2007.

BIHA93 Biham, E., and Shamir, A. *Differential Cryptanalysis of the Data Encryption Standard*. New York: Springer-Verlag, 1993.

BLAC00 Black, J., and Rogaway, P.; and Shrimpton, T. " CBC MACs for Arbitrary-Length Messages: The Three-Key Constructions." *Advances in Cryptology – CRYPTO '00* 2000.

BLAC05 Black, J. "Authenticated Encryption." *Encyclopedia of Cryptography and Security*. Springer, 2005.

BLAK99 Blake, I.; Seroussi, G.; and Smart, N. *Elliptic Curves in Cryptography*. Cambridge: Cambridge University Press, 1999.

BLOO70 Bloom, B. "Space/time Trade-offs in Hash Coding with Allowable Errors." *Communications of the ACM*, July 1970.

BLUM86 Blum, L.; Blum, M.; and Shub, M. "A Simple Unpredictable Pseudo-Random Number Generator." *SIAM Journal on Computing*, No. 2, 1986.

BONE99 Boneh, D. "Twenty Years of Attacks on the RSA Cryptosystem." *Notices of the American Mathematical Society*, February 1999.

BONE02 Boneh, D., and Shacham, H. "Fast Variants of RSA." *CryptoBytes*, Winter/Spring 2002. http://www.rsasecurity.com/rsalabs.

BORN03 Bornemann, F. "PRIMES is in P: A Breakthrough for Everyman." *Notices of the American Mathematical Society*, May 2003.

BRAU01 Braunfeld, R., and Wells, T. "Protecting Your Most Valuable Asset: Intellectual Property." *IT Pro*, March/April 2000.

BRIG79 Bright, H., and Enison, R. "Quasi-Random Number Sequences from Long-Period TLP Generator with Remarks on Application to Cryptography." *Computing Surveys*, December 1979.

BROW07 Brown, D., an d Gjosteen, K. " A Security Analysis of the NIST SP 800-90 Elliptic Curve Random Number Generator." *Proceedings, Crypto 07*, 2007.

BRYA88 Bryant, W. *Designing an Authentication System: A Dialogue in Four Scenes*. Project Athena document, February 1988. Available at http://web.mit.edu/kerberos/www/dialogue.html.

BURN97 Burn, R. *A Pathway to Number Theory*. Cambridge: Cambridge University Press, 1997.

BURR08 Burr, W. "A New Hash Competition." *IEEE Security & Privacy*, May/June 2008.

BROW72 Browne, P. "Computer Security — A Survey." *ACM SIGMIS Database*, Fall 1972.

CAMP92 Campbell, K., and Wiener, M. "Proof that DES is not a Group." *Proceedings, Crypto '92*, 1992; published by Springer-Verlag.

CAMP03 Camp, L. " First Principles of Copyright for DRM Design." *IEEE Internet Computing*, May/June 2003.

CASS01 Cass, S. "Anatomy of Malice." *IEEE Spectrum*, November 2001.

CERT01 CERT Coordination Center. "Denial of Service Attacks." June 2001. http://www.cert.org/tech_tips/denial_of_service.html

CHAN02 Chang, R. "Defending Against Flooding-Based Distributed Denial-of-Service Attacks: A Tutorial." *IEEE Communications Magazine*, October 2002.

CHAP00 Chapman, D., and Zwicky, E. *Building Internet Firewalls.* Sebastopol, CA: O'Reilly, 2000.

CHAP06 Chapman, C. "Fundamental Ethics in Information Systems." *Proceedings of the 39th Hawaii International Conference on System Sciences*, 2006.

CHEN98 Cheng, P., et al. "A Security Architecture for the Internet Protocol." *IBM Systems Journal*, Number 1, 1998.

CHEN04 Chen, S., and Tang, T. "Slowing Down Internet Worms," *Proceedings of the 24th International Conference on Distributed Computing Systems*, 2004.

CHEN05 Chen, J.; Jiang, M.; and Liu, Y. "Wireless LAN Security and IEEE 802.11i." *IEEE Wireless Communications*, February 2005.

CHES97 Chess, D. "The Future of Viruses on the Internet." *Proceedings, Virus Bulletin International Conference*, October 1997.

CHES03 Cheswick, W., and Bellovin, S. *Firewalls and Internet Security: Repelling the Wily Hacker.* Reading, MA: Addison-Wesley, 2003.

CHIN05 Chinchani, R., and Berg, E. "A Fast Static Analysis Approach to Detect Exploit Code Inside Network Flows." *Recent Advances in Intrusion Detection, 8th International Symposium*, 2005.

COCK73 Cocks, C. *A Note on Non-Secret Encryption.* CESG Report, November 1973.

COHE94 Cohen, F. *A Short Course on Computer Viruses.* New York: Wiley, 1994.

COMP06 Computer Associates International. *The Business Value of Identity Federation.* White Paper, January 2006.

CONR02 Conry-Murray, A. "Behavior-Blocking Stops Unknown Malicious Code." *Network Magazine*, June 2002.

COPP94 Coppersmith, D. "The Data Encryption Standard (DES) and Its Strength Against Attacks." *IBM Journal of Research and Development*, May 1994.

CORM04 Cormen, T.; Leiserson, C.; Rivest, R.; and Stein, C. *Introduction to Algorithms.* Cambridge, MA: MIT Press, 2004.

COST05 Costa, M., et al. "Vigilante: End-to-end Containment of Internet Worms." *ACM Symposium on Operating Systems Principles*, 2005.

CRAN01 Crandall, R., and Pomerance, C. *Prime Numbers: A Computational Perspective.* New York: Springer-Verlag, 2001.

CROC09 Crocker, D. *Internet Mail Architecture.* Internet draft draft-crocker-email-arch-13, May 15, 2009.

CYMR06 Team Cymru, "Cybercrime: An Epidemic." *ACM Queue*, November 2006.

DAEM99 Daemen, J., and Rijmen, V. *AES Proposal: Rijndael, Version 2.* Submission to NIST, March 1999. http://csrc.nist.gov/encryption/aes.

DAEM01 Daemen, J., and Rijmen, V. "Rijndael: The Advanced Encryption Standard." *Dr. Dobb's Journal*, March 2001.

DAEM02 Daemen, J., and Rijmen, V. *The Design of Rijndael: The Wide Trail Strategy Explained.* New York: Springer-Verlag, 2002.

DAMG89 Damgard, I. "A Design Principle for Hash Functions." *Proceedings, CRYPTO '89*, 1989; published by Springer-Verlag.

DAVI89 Davies, D., and Price, W. *Security for Computer Networks.* New York: Wiley, 1989.

DAVI93 Davies, C., and Ganesan, R. "BApasswd: A New Proactive Password Checker." *Proceedings, 16th National Computer Security Conference*, September 1993.

DAWS96 Dawson, E., and Nielsen, L. "Automated Cryptoanalysis of XOR Plaintext Strings." *Cryptologia*, April 1996.

DENN81 Denning, D. "Timestamps in Key Distribution Protocols." *Communications of the ACM*, August 1981.

DENN82 Denning, D. *Cryptography and Data Security.* Reading, MA: Addison-Wesley, 1982.

DENN87 Denning, D. "An Intrusion-Detection Model." *IEEE Transactions on Software Engineering*, February 1987.

DESK92 Deskins, W. *Abstract Algebra.* New York: Dover, 1992.

DIFF76a Diffie, W., and Hellman, M. "New Directions in Cryptography." *Proceedings of the AFIPS National Computer Conference*, June 1976.

DIFF76b Diffie, W., and Hellman, M. "Multiuser Cryptographic Techniques." *IEEE Transactions on Information Theory*, November 1976.

DIFF77 Diffie, W., and Hellman, M. "Exhaustive Cryptanalysis of the NBS Data Encryption Standard." *Computer*, June 1977.

DIFF79 Diffie, W., and Hellman, M. "Privacy and Authentication: An Introduction to Cryptography." *Proceedings of the IEEE*, March 1979.

DIFF88 Diffie, W. "The First Ten Years of Public-Key Cryptography." *Proceedings of the IEEE*, May 1988.

DOBB96 Dobbertin, H. "The Status of MD5 After a Recent Attack." *CryptoBytes*, Summer 1996.

DOJ00 U.S. Department of Justice. *The Electronic Frontier: The Challenge of Unlawful Conduct Involving the Use of the Internet.* March 2000. usdoj.gov/criminal/cybercrime/unlawful.htm

EAST05 Eastlake, D.' Schiller, J.; and Crocker, S. *Randomness Requirements for Security.* RFC 4086, June 2005.

EFF98 Electronic Frontier Foundation. *Cracking DES: Secrets of Encryption Research, Wiretap Politics, and Chip Design.* Sebastopol, CA: O'Reilly, 1998.

ELGA84 Elgamal, T. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms." Proceedings, Crypto 84, 1984.

ELGA85 Elgamal, T. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms." *IEEE Transactions on Information Theory*, July 1985.

ELLI70 Ellis, J. *The Possibility of Secure Non-Secret Digital Encryption.* CESG Report, January 1970.

ELLI99 Ellis, J. "The History of Non-Secret Encryption." *Cryptologia*, July 1999.

ENGE80 Enger, N., and Howerton, P. *Computer Security.* New York: Amacom, 1980.

ENGE99 Enge, A. *Elliptic Curves and Their Applications to Cryptography.* Norwell, MA: Kluwer Academic Publishers, 1999.

FEIS73 Feistel, H. "Cryptography and Computer Privacy." *Scientific American*, May 1973.

FEIS75 Feistel, H.; Notz, W.; and Smith, J. "Some Cryptographic Techniques for Machine-to-Machine Data Communications." *Proceedings of the IEEE*, November 1975.

FERN99 Fernandes, A. "Elliptic Curve Cryptography." *Dr. Dobb's Journal*, December 1999.

FLUH00 Fluhrer, S., and McGrew, D. "Statistical Analysis of the Alleged RC4 Key Stream Generator." *Proceedings, Fast Software Encryption 2000*, 2000.

FLUH01 Fluhrer, S.; Mantin, I.; and Shamir, A. "Weakness in the Key Scheduling Algorithm of RC4." *Proceedings, Workshop in Selected Areas of Cryptography*,2001.

FORR97 Forrest, S.; Hofmeyr, S.; and Somayaji, A. "Computer Immunology." *Communications of the ACM*, October 1997.

FRAN05 Frankel, S., et al. *Guide to IPsec VPNs.* NIST SP 800-77, 2005.

FRAN07 Frankel, S.; Eydt, B.; Owens, L.; and Scarfone, K. *Establishing Wireless Robust Security Networks: A Guide to IEEE 802.11i.* NIST Special Publication SP 800-97, February 2007.

FRAN09 Frankel, S., and Krishnan, S. *IP Security (IPsec) and Internet Key Exchange (IKE) Document Roadmap.* draft-ietf-ipsecme-roadmap-01.txt, March 6, 2009.

FRAS97 Fraser, B. *Site Security Handbook.* RFC 2196, September 1997.

FUMY93 Fumy, S., and Landrock, P. "Principles of Key Management." *IEEE Journal on Selected Areas in Communications*, June 1993.

GARD72 Gardner, M. *Codes, Ciphers, and Secret Writing.* New York: Dover, 1972.

GARD77 Gardner, M. "A New Kind of Cipher That Would Take Millions of Years to Break." *Scientific American*, August 1977.

GARF02 Garfinkel, S., and Spafford, G. *Web Security, Privacy & Commerce.* Sebastapol, CA: O'Reilly, 2002.

GARR01 Garrett, P. *Making, Breaking Codes: An Introduction to Cryptology.* Upper Saddle River, NJ: Prentice Hall, 2001.

GAUD00 Gaudin, S. "The Omega Files." *Network World*, June 26, 2000.

GIBB00 Gibbs, J. "The Digital Millennium Copyright Act." *ACM Ubiquity*, August 2000.

GILB03 Gilbert, H. and Handschuh, H. "Security Analysis of SHA-256 and Sisters." "*Proceedings, CRYPTO '03*, 2003; published by Springer-Verlag.

GOLD88 Goldwasser, S.; Micali, S.; and Rivest, R. "A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks." *SIAM Journal on Computing*, April 1988.

GONG92 Gong, L. "A Security Risk of Depending on Synchronized Clocks." *Operating Systems Review*, January 1992.

GONG93 Gong, L. "Variations on the Themes of Message Freshness and Replay." *Proceedings, IEEE Computer Security Foundations Workshop*, June 1993.

GOTT99 Gotterbarn, D. " How the New Software Engineering Code of Ethics Affects You." *IEEE Software*, November/ December 1999.

GRAH94 Graham, R.; Knuth, D.; and Patashnik, O. *Concrete Mathematics: A Foundation for Computer Science.* Reading, MA: Addison-Wesley, 1994.

GRAN04 Grance, T.; Kent, K.; and Kim, B. *Computer Security Incident Handling Guide.* NIST Special Publication SP 800-61, January 2004.

GUTM02 Gutmann, P. "PKI: It's Not Dead, Just Resting." *Computer*, August 2002.

GUTT06 Gutterman, Z.; Pinkas, B.; and Reinman, T. " Analysis of the Linux Random Number Generator." *Proceedings, 2006 IEEE Symposium on Security and Privacy*, 2006.

HAMM91 Hamming, R. *The Art of Probability for Scientists and Engineers.* Reading, MA: Addison-Wesley, 1991.

HANK04 Hankerson, D.; Menezes, A.; and Vanstone, S. *Guide to Elliptic Curve Cryptography.* New York: Springer, 2004.

HARR90 Harrington, S., and McCollum, R. "Lessons from Corporate America Applied to Training in Computer Ethics." *Proceedings of the ACM Conference on Computers and the Quality of Life (SIGCAS and SIGCAPH)*, September 1990.

HEBE92 Heberlein, L.; Mukherjee, B.; and Levitt, K. "Internetwork Security Monitor: An Intrusion-Detection System for Large-Scale Networks." *Proceedings, 15th National Computer Security Conference.* October 1992.

HEGL06 Hegland, A., et al. "A Survey of Key Management in Ad Hoc Networks." *IEEE Communications Surveys & Tutorials.* 3rd Quarter 2006.

HELD96 Held, G. *Data and Image Compression: Tools and Techniques.* New York: Wiley, 1996.

HELL79 Hellman, M. "The Mathematics of Public-Key Cryptography." *Scientific American*, August 1970.

HEVI99 Hevia, A., and Kiwi, M. "Strength of Two Data Encryption Standard Implementations Under Timing Attacks." *ACM Transactions on Information and System Security*, November 1999.

HERS75 Herstein, I. *Topics in Algebra.* New York: Wiley, 1975.

HEYS95 Heys, H., and Tavares, S. "Avalanche Characteristics of Substitution-Permutation Encryption Networks." *IEEE Transactions on Computers*, September 1995.

HEYS02 Heys, H. "A Tutorial on Linear and Differential Cryptanalysis." *Cryptologia*, July 2002.

HONE01 The Honeynet Project. *Know Your Enemy: Revealing the Security Tools, Tactics, and Motives of the Blackhat Community.* Reading, MA: Addison-Wesley, 2001.

HORO71 Horowitz, E. "Modular Arithmetic and Finite Field Theory: A Tutorial." *Proceedings of the Second ACM Symposium and Symbolic and Algebraic Manipulation*, March 1971.

HUIT98 Huitema, C. *IPv6: The New Internet Protocol.* Upper Saddle River, NJ: Prentice Hall, 1998.

HYPP06 Hypponen, M. "Malware Goes Mobile." *Scientific American*, November 2006.

IANN06 Iannella, R. "Digital Rights Management." In Bidgoli, H., editor. *Handbook of Information Security.* New York: Wiley, 2006.

IANS90 I'Anson, C., and Mitchell, C. "Security Defects in CCITT Recommendation X.509–The Directory Authentication Framework" *Computer Communications Review*, April 1990.

ILGU93 Ilgun, K. "USTAT: A Real-Time Intrusion Detection System for UNIX." *Proceedings, 1993 IEEE Computer Society Symposium on Research in Security and Privacy*, May 1993.

ISAT02 Information Science and Technology Study Group. "Security with Privacy," *DARPA Briefing on Security and Privacy*, Dec. 2002. www.cs.berkeley.edu/~tygar/papers/ISAT-final-briefing.pdf

IWAT03 Iwata, T., and Kurosawa, K. "OMAC: One-Key CBC MAC." *Proceedings, Fast Software Encryption*, FSE '03, 2003.

JAIN91 Jain, R. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling.* New York: Wiley, 1991.

JAKO98 Jakobsson, M.; Shriver, E.; Hillyer, B.; and Juels, A. "A practical secure physical random bit generator." *Proceedings of The Fifth ACM Conference on Computer and Communications Security*, November 1998.

JANS01 Jansen, W. *Guidelines on Active Content and Mobile Code.* NIST Special Publication SP 800-28, October 2001.

JAVI91 Javitz, H., and Valdes, A. "The SRI IDES Statistical Anomaly Detector." *Proceedings, 1991 IEEE Computer Society Symposium on Research in Security and Privacy*, May 1991.

JHI07 Jhi, Y., and Liu, P. "PWC: A Proactive Worm Containment Solution for Enterprise Networks." *Third International Conference on Security and Privacy in Communications Networks*, 2007.

JOHN05 Johnson, D. "Hash Functions and Pseudorandomness." *Proceedings, First NIST Cryptographic Hash Workshop*, 2005.

JONS02 Jonsson, J. "On the Security of CTR + CBC-MAC." *Proceedings of Selected Areas in Cryptography – SAC 2002*. 2002.

JUDY09 Judy, H., et al. "Privacy in Cyberspace: U.S. and European Perspectives." In Bosworth, S. et al., eds. *Computer Security Handbook*. New York: Wiley, 2009.

JUEN85 Jueneman, R.; Matyas, S.; and Meyer, C. "Message Authentication." *IEEE Communications Magazine*, September 1988.

JUEN87 Jueneman, R. "Electronic Document Authentication." *IEEE Network Magazine*, April 1987.

JUN99 Jun, B., and Kocher, P. *The Intel Random Number Generator*. Intel White Paper, April 22, 1999.

JUNG04 Jung, J.; et al. "Fast Portscan Detection Using Sequential Hypothesis Testing," *Proceedings, IEEE Symposium on Security and Privacy*, 2004.

JURI97 Jurisic, A., and Menezes, A. "Elliptic Curves and Cryptography." *Dr. Dobb's Journal*, April 1997.

KAHN96 Kahn, D. *The Codebreakers: The Story of Secret Writing*. New York: Scribner, 1996.

KALI95 Kaliski, B., and Robshaw, M. "The Secure Use of RSA." *CryptoBytes*, Autumn 1995.

KALI96a Kaliski, B., and Robshaw, M. "Multiple Encryption: Weighing Security and Performance." *Dr. Dobb's Journal*, January 1996.

KALI96b Kaliski, B. "Timing Attacks on Cryptosystems." *RSA Laboratories Bulletin*, January 1996. http://www.rsasecurity.com/rsalabs.

KATZ00 Katzenbeisser, S., ed. *Information Hiding Techniques for Steganography and Digital Watermarking*. Boston: Artech House, 2000.

KEHN92 Kehne, A.; Schonwalder, J.; and Langendorfer, H. "A Nonce-Based Protocol for Multiple Authentications." *Operating Systems Review*, October 1992.

KELS98 Kelsey, J.; Schneier, B.; and Hall, C. "Cryptanalytic Attacks on Pseudorandom Number Generators." *Proceedings, Fast Software Encryption*, 1998. http://www.schneier.com/paper-prngs.html.

KENT00 Kent, S. "On the Trail of Intrusions into Information Systems." *IEEE Spectrum*, December 2000.

KEPH97a Kephart, J.; Sorkin, G.; Chess, D.; and White, S. "Fighting Computer Viruses." *Scientific American*, November 1997.

KEPH97b Kephart, J.; Sorkin, G.; Swimmer, B.; and White, S. "Blueprint for a Computer Immune System." *Proceedings, Virus Bulletin International Conference*, October 1997.

KIRK06 Kirk, J. "Tricky New Malware Challenges Vendors." *Network World*, October 30, 2006.

KISS06 Kissel, R., ed. *Glossary of Key Information Security Terms*. NIST IR 7298, 25 April 2006.

KLEI90 Klein, D. "Foiling the Cracker: A Survey of, and Improvements to, Password Security." *Proceedings, UNIX Security Workshop II*, August 1990.

KNUD98 Knudsen, L., et al. "Analysis Method for Alleged RC4." *Proceedings, ASIACRYPT '98*, 1998.

KNUT97 Knuth, D. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Reading, MA: Addison-Wesley, 1997.

KNUT98 Knuth, D. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1998.

KOBL92 Koblas, D., and Koblas, M. "SOCKS." *Proceedings, UNIX Security Symposium III*, September 1992.

KOBL94 Koblitz, N. *A Course in Number Theory and Cryptography*. New York: Springer-Verlag, 1994.

KOCH96 Kocher, P. "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems." *Proceedings, Crypto '96*, August 1996.

KOHL89 Kohl, J. "The Use of Encryption in Kerberos for Network Authentication." *Proceedings, Crypto '89*, 1989; published by Springer-Verlag.

KOHL94 Kohl, J.; Neuman, B.; and Ts'o, T. "The Evolution of the Kerberos Authentication Service." in Brazier, F., and Johansen, D. *Distributed Open Systems*. Los Alamitos, CA: IEEE Computer Society Press, 1994. Available at http://web.mit.edu/kerberos/www/papers.html.

KONH81 Konheim, A. *Cryptography: A Primer*. New York: Wiley, 1981.

KORN96 Korner, T. *The Pleasures of Counting*. Cambridge: Cambridge University Press 1996.

KSHE06 Kshetri, N. "The Simple Economics of Cybercrimes." *IEEE Security and Privacy*, January/February 2006.

KUMA97 Kumar, I. *Cryptology*. Laguna Hills, CA: Aegean Park Press, 1997.

KUMA98 Kumanduri, R., and Romero, C. *Number Theory with Computer Applications.* Upper Saddle River, NJ: Prentice Hall, 1998.

LAM92a Lam, K., and Gollmann, D. "Freshness Assurance of Authentication Protocols." *Proceedings, ESORICS 92*, 1992; published by Springer-Verlag.

LAM92b Lam, K., and Beth, T. "Timely Authentication in Distributed Systems." *Proceedings, ESORICS 92*, 1992; published by Springer-Verlag.

LAMP04 Lampson, B. "Computer Security in the Real World." *Computer*, June 2004.

LAND04 Landau, S. "Polynomials in the Nation's Service: Using Algebra to Design the Advanced Encryption Standard." *American Mathematical Monthly*, February 2004.

LATT09 Lattin, B. "Upgrade to Suite B Security Algorithms." *Network World*, June 1, 2009.

LEHM51 Lehmer, D. "Mathematical Methods in Large-Scale Computing." *Proceedings, 2nd Symposium on Large-Scale Digital Calculating Machinery*, Cambridge: Harvard University Press, 1951.

LEIB07 Leiba, B., and Fenton, J. "DomainKeys Identified Mail (DKIM): Using Digital Signatures for Domain Verification." *Proceedings of Fourth Conference on Email and Anti-Spam (CEAS 07)*, 2007.

LEUT94 Leutwyler, K. "Superhack." *Scientific American*, July 1994.

LEVE90 Leveque, W. *Elementary Theory of Numbers.* New York: Dover, 1990.

LEWA00 Lewand, R. *Cryptological Mathematics.* Washington, DC: Mathematical Association of America, 2000.

LEWI69 Lewis, P.; Goodman, A.; and Miller, J. "A Pseudo-Random Number Generator for the System/360." *IBM Systems Journal*, No. 2, 1969.

LIDL94 Lidl, R., and Niederreiter, H. *Introduction to Finite Fields and Their Applications.* Cambridge: Cambridge University Press, 1994.

LINN06 Linn, J. "Identity Management." In Bidgoli, H., editor. *Handbook of Information Security.* New York: Wiley, 2006.

LIPM00 Lipmaa, H.; Rogaway, P.; and Wagner, D. "CTR Mode Encryption." *NIST First Modes of Operation Workshop*, October 2000. http://csrc.nist.gov/encryption/modes.

LISK02 Liskov, M.; Rivest, R.; and Wagner, D. "Tweakable Block Ciphers." *Advances in Cryptology—CRYPTO '02. Lecture Notes in Computer Science*, Vol. 2442, pp. 31–46. Springer-Verlag, 2002.

LIU03 Liu, Q.; Safavi-Naini, R.; and Sheppard, N. "Digital Rights Management for Content Distribution." *Proceedings, Australasian Information Security Workshop 2003 (AISW2003)*, 2003.

LODI98 Lodin, S., and Schuba, C. "Firewalls Fend Off Invasions from the Net." *IEEE Spectrum*, February 1998.

LUNT88 Lunt, T., and Jagannathan, R. "A Prototype Real-Time Intrusion-Detection Expert System." *Proceedings, 1988 IEEE Computer Society Symposium on Research in Security and Privacy*, April 1988.

MADS93 Madsen, J. "World Record in Password Checking." *Usenet, comp.security.misc newsgroup*, August 18, 1993.

MANT01 Mantin, I., Shamir, A. "A Practical Attack on Broadcast RC4." *Proceedings, Fast Software Encryption*, 2001.

MATS93 Matsui, M. "Linear Cryptanalysis Method for DES Cipher." *Proceedings, EUROCRYPT '93*, 1993; published by Springer-Verlag.

MCGR04 McGrew, D., and Viega, J. "The Security and Performance of the Galois/Counter Mode (GCM) of Operation." *Proceedings, Indocrypt 2004.*

MCGR05 McGrew, D., and Viega, J. " Flexible and Efficient Message Authentication in Hardware and Software." 2005. http://www.cryptobarn.com/gcm/gcm-paper.pdf.

MCHU00 McHugh, J.; Christie, A.; and Allen, J. "The Role of Intrusion Detection Systems." *IEEE Software*, September/October 2000.

MEIN01 Meinel, C. "Code Red for the Web." *Scientific American*, October 2001.

MENE97 Menezes, A.; van Oorschot, P.; and Vanstone, S. *Handbook of Applied Cryptography.* Boca Raton, FL: CRC Press, 1997.

MERK78a Merkle, Ra. "Secure Communication Over an Insecure Channel." *Communications of the ACM*, March 1978.

MERK78b Merkle, R., and Hellman, M. "Hiding Information and Signatures in Trap Door Knapsacks." *IEEE Transactions on Information Theory*, September 1978.

MERK79 Merkle, R. *Secrecy, Authentication, and Public Key Systems.* Ph.D. Thesis, Stanford University, June 1979.

MERK81 Merkle, R., and Hellman, M. "On the Security of Multiple Encryption." *Communications of the ACM*, July 1981.

MERK89 Merkle, R. "One Way Hash Functions and DES." *Proceedings, CRYPTO '89*, 1989; published by Springer-Verlag.

MEYE82 Meyer, C., and Matyas, S. *Cryptography: A New Dimension in Computer Data Security.* New York: Wiley, 1982.

MEYE88 Meyer, C., and Schilling, M. "Secure Program Load with Modification Detection Code." *Proceedings, SECURICOM 88*, 1988.

MICA91 Micali, S., and Schnorr, C. "Efficient, Perfect Polynomial Random Number Generators." *Journal of Cryptology*, January 1991.

MILL75 Miller, G. "Riemann's Hypothesis and Tests for Primality." *Proceedings of the Seventh Annual ACM Symposium on the Theory of Computing*, May 1975.

MILL88 Miller, S.; Neuman, B.; Schiller, J.; and Saltzer, J. "Kerberos Authentication and Authorization System." *Section E.2.1, Project Athena Technical Plan*, M.I.T. Project Athena, Cambridge, MA, 27 October 1988.

MIRK04 Mirkovic, J., and Relher, P. "A Taxonomy of DDoS Attack and DDoS Defense Mechanisms." *ACM SIGCOMM Computer Communications Review*, April 2004.

MIST96 Mister, S., and Adams, C. "Practical S-Box Design." *Proceedings, Workshop in Selected Areas of Cryptography, SAC' 96*, 1996.

MITC92 Mitchell, C.; Piper, F.; and Wild, P. "Digital Signatures." in [SIMM92].

MIYA90 Miyaguchi, S.; Ohta, K.; and Iwata, M. "Confirmation that Some Hash Functions Are Not Collision Free." *Proceedings, EUROCRYPT '90*, 1990; published by Springer-Verlag.

MOOR01 Moore, M. " Inferring Internet Denial-of-Service Activity." *Proceedings of the 10th USENIX Security Symposium*, 2001.

MURP90 Murphy, S. "The Cryptanalysis of FEAL-4 with 20 Chosen Plaintexts." *Journal of Cryptology*, No. 3, 1990.

MURP00 Murphy, T. *Finite Fields.* University of Dublin, Trinity College, School of Mathematics, 2000. Document available at this book's Web site.

MUSA03 Musa, M.; Schaefer, E.; and Wedig, S. "A Simplified AES Algorithm and Its Linear and Differential Cryptanalyses." *Cryptologia*, April 2003.

MYER91 Myers, L. *Spycomm: Covert Communication Techniques of the Underground.* Boulder, CO: Paladin Press, 1991.

NACH97 Nachenberg, C. "Computer Virus-Antivirus Coevolution." *Communications of the ACM*, January 1997.

NACH02 Nachenberg, C. "Behavior Blocking: The Next Step in Anti-Virus Protection." *White Paper*, SecurityFocus.com, March 2002.

NEED78 Needham, R., and Schroeder, M. "Using Encryption for Authentication in Large Networks of Computers." *Communications of the ACM*, December 1978.

NEUM93a Neuman, B., and Stubblebine, S. "A Note on the Use of Timestamps as Nonces." *Operating Systems Review*, April 1993.

NEUM93b Neuman, B. "Proxy-Based Authorization and Accounting for Distributed Systems." *Proceedings of the 13th International Conference on Distributed Computing Systems*, May 1993.

NEWS05 Newsome, J.; Karp, B.; and Song, D. "Polygraph: Automatically Generating Signatures for Polymorphic Worms." *IEEE Symposium on Security and Privacy*, 2005.

NICH96 Nichols, R. *Classical Cryptography Course.* Laguna Hills, CA: Aegean Park Press, 1996.

NICH99 Nichols, R. ed., *ICSA Guide to Cryptography.* New York: McGraw-Hill, 1999.

NING04 Ning, P., et al. "Techniques and Tools for Analyzing Intrusion Alerts." *ACM Transactions on Information and System Security*, May 2004.

NIST95 National Institute of Standards and Technology. *An Introduction to Computer Security: The NIST Handbook.* Special Publication 800-12. October 1995.

NRC91 National Research Council. *Computers at Risk: Safe Computing in the Information Age.* Washington, D.C.: National Academy Press, 1991.

ODLY95 Odlyzko, A. "The Future of Integer Factorization." *CryptoBytes*, Summer 1995.

OPPL97 Oppliger, R. "Internet Security: Firewalls and Beyond." *Communications of the ACM*, May 1997.

ORE67 Ore, O. *Invitation to Number Theory.* Washington, D.C.: The Mathematical Association of America, 1967.

ORMA03 Orman, H. "The Morris Worm: A Fifteen-Year Perspective." *IEEE Security and Privacy*, September/October 2003.

PARK88a Park, S., and Miller, K. "Random Number Generators: Good Ones are Hard to Find." *Communications of the ACM*, October 1988.

PARK88b Parker, D.; Swope, S.; and Baker, B. *Ethical Conflicts in Information and Computer Science, Technology and Business.* Final Report, SRI Project 2609, SRI International1988.

PARZ06 Parziale, L., et al. *TCP/IP Tutorial and Technical Overview.* ibm.com/redbooks, 2006.

PATE06 Paterson, K. "A Cryptographic Tour of the IPsec Standards." *Cryptology ePrint Archive: Report 2006/097*, April 2006.

PATR04 Patrikakis, C.; Masikos, M.; and Zouraraki, O. "Distributed Denial of Service Attacks." *The Internet Protocol Journal*, December 2004.

PELT07 Peltier, J. "Identity Management." *SC Magazine*, February 2007.

PERL99 Perlman, R. "An Overview of PKI Trust Models." *IEEE Network*, November/December 1999.

PIAT91 Piattelli-Palmarini, M. "Probability: Neither Rational nor Capricious." *Bostonia*, March 1991.

POHL81 Pohl, I., and Shaw, A. *The Nature of Computation: An Introduction to Computer Science.* Rockville, MD: Computer Science Press, 1981.

POIN02 Pointcheval, D. "How to Encrypt Properly with RSA." *CryptoBytes*, Winter/Spring 2002. http://www.rsasecurity.com/rsalabs.

POPP06 Popp, R., and Poindexter, J. "Countering Terrorism through Information and Privacy Protection Technologies." *IEEE Security and Privacy*, November/December 2006.

PORR92 Porras, P. *STAT: A State Transition Analysis Tool for Intrusion Detection.* Master's Thesis, University of California at Santa Barbara, July 1992.

PREN96 Preneel, B., and Oorschot, P. "On the Security of Two MAC Algorithms." *Lecture Notes in Computer Science 1561; Lectures on Data Security*, 1999; published by Springer-Verlag.

PREN99 Preneel, B. "The State of Cryptographic Hash Functions." *Proceedings, EUROCRYPT '96*, 1996; published by Springer-Verlag.

PROC01 Proctor, P., *The Practical Intrusion Detection Handbook.* Upper Saddle River, NJ: Prentice Hall, 2001.

RABI78 Rabin, M. "Digitalized Signatures." *Foundations of Secure Computation*, DeMillo, R.; Dobkin, D.; Jones, A.; and Lipton, R., eds. New York: Academic Press, 1978.

RABI80 Rabin, M. "Probabilistic Algorithms for Primality Testing." *Journal of Number Theory*, December 1980.

RADC04 Radcliff, D. "What Are They Thinking?" *Network World*, March 1, 2004.

RESC01 Rescorla, E. *SSL and TLS: Designing and Building Secure Systems.* Reading, MA: Addison-Wesley, 2001.

RIBE96 Ribenboim, P. *The New Book of Prime Number Records.* New York: Springer-Verlag, 1996.

RIVE78 Rivest, R.; Shamir, A.; and Adleman, L. "A Method for Obtaining Digital Signatures and Public Key Cryptosystems." *Communications of the ACM*, February 1978.

ROBS95a Robshaw, M. *Stream Ciphers.* RSA Laboratories Technical Report TR-701, July 1995. http://www.rsasecurity.com/rsalabs.

ROBS95b Robshaw, M. *Block Ciphers.* RSA Laboratories Technical Report TR-601, August 1995. http://www.rsasecurity.com/rsalabs.

ROBS95c Robshaw, M. *MD2, MD4, MD5, SHA and Other Hash Functions.* RSA Laboratories Technical Report TR-101, July 1995. http://www.rsasecurity.com/rsalabs.

ROGA03 Rogaway, P., and Wagner, A. "A Critique of CCM." *Cryptology ePrint Archive: Report 2003/070*, April 2003.

ROGA04 Rogaway, P. "Efficient Instantiations of Tweakable Blockciphers and Refinements to Modes OCB and PMAC." *Advances in Cryptology—Asiacrypt 2004. Lecture Notes in Computer Science*, Vol. 3329. Springer-Verlag, 2004.

ROSE05 Rosen, K. *Elementary Number Theory and its Applications.* Reading, MA: Addison-Wesley, 2000.

ROSH04 Roshan, P., and Leary, J. *802.11 Wireless LAN Fundamentals.* Indianapolis: Cisco Press, 2004.

ROSI99 Rosing, M. *Implementing Elliptic Curve Cryptography.* Greenwich, CT: Manning Publications, 1999.

RITT91 Ritter, T. "The Efficient Generation of Cryptographic Confusion Sequences." *Cryptologia*, Vol. 15, No. 2, 1991. www.ciphersbyritter.com/ARTS/CRNG2ART.HTM.

RUEP92 Rueppel, T. "Stream Ciphers." In [SIMM92].

RUKH08 Rukhin, A., et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications.* NIST SP 800-22, August 2008.

SALT75 Saltzer, J., and Schroeder, M. "The Protection of Information in Computer Systems." *Proceedings of the IEEE*, September 1975.

SCAR07 Scarfone, K., and Mell, P. *Guide to Intrusion Detection and Prevention Systems.* NIST Special Publication SP 800-94, February 2007.

SCHN89 Schnorr, C. "Efficient Identification and Signatures for Smart Cards." *EUROCRYPT*, 1988.

SCHN91 Schnorr, C. "Efficient Signature Generation by Smart Cards." *Journal of Cryptology*, No. 3, 1991.

SCHN96 Schneier, B. *Applied Cryptography.* New York: Wiley, 1996.

SCHN00 Schneier, B. *Secrets and Lies: Digital Security in a Networked World.* New York: Wiley 2000.

SCHO06 Schoenmakers, B., and Sidorenki, A. "Cryptanalysis of the Dual Elliptic Curve Pseudorandom Generator." *Cryptology ePrint Archive*, Report 2006/190, 2006. eprint.iacr.org.

SHAM03 Shamir, A., and Tromer, E. "On the Cost of Factoring RSA-1024." *CryptoBytes*, Summer 2003. http://www.rsasecurity.com/rsalabs.

SHAN49 Shannon, C. "Communication Theory of Secrecy Systems." *Bell Systems Technical Journal*, No. 4, 1949.

SHAN77 Shanker, K. "The Total Computer Security Problem: An Overview." *Computer*, June 1977.

SHIM05 Shim, S.; Bhalla, G.; and Pendyala, V. "Federated Identity Management." *Computer*, December 2005.

SIDI05 Sidiroglou, S., and Keromytis, A. "Countering Network Worms Through Automatic Patch Generation." *IEEE Security and Privacy*, November-December 2005.

SILV06 Silverman, J. *A Friendly Introduction to Number Theory.* Upper Saddle River, NJ: Prentice Hall, 2006.

SIMM92 Simmons, G., ed. *Contemporary Cryptology: The Science of Information Integrity.* Piscataway, NJ: IEEE Press, 1992.

SIMM93 Simmons, G. "Cryptology." *Encyclopaedia Britannica, Fifteenth Edition*, 1993.

SIMO95 Simovits, M. *The DES: An Extensive Documentation and Evaluation.* Laguna Hills, CA: Aegean Park Press, 1995.

SING99 Singh, S. *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography.* New York: Anchor Books, 1999.

SINK66 Sinkov, A. *Elementary Cryptanalysis: A Mathematical Approach.* Washington, D.C.: The Mathematical Association of America, 1966.

SMIT97 Smith, R. *Internet Cryptography.* Reading, MA: Addison-Wesley, 1997.

SNAP91 Snapp, S., et al. "A System for Distributed Intrusion Detection." *Proceedings, COMPCON Spring '91*, 1991.

SPAF92a Spafford, E. "Observing Reusable Password Choices." *Proceedings, UNIX Security Symposium III*, September 1992.

SPAF92b Spafford, E. "OPUS: Preventing Weak Password Choices." *Computers and Security*, No. 3, 1992.

STAL07 Stallings, W. *Data and Computer Communications, Eighth Edition.* Upper Saddle River, NJ: Prentice Hall, 2007.

STAL08 Stallings, W., and Brown, L. *Computer Security.* Englewood Cliffs, NJ, 2008.

STEI88 Steiner, J.; Neuman, C.; and Schiller, J. "Kerberos: An Authentication Service for Open Networked Systems." *Proceedings of the Winter 1988 USENIX Conference*, February 1988.

STEP93 Stephenson, P. "Preventive Medicine." *LAN Magazine*, November 1993.

STIN06 Stinson, D. *Cryptography: Theory and Practice.* Boca Raton, FL: Chapman & Hall, 2006.

SUMM84 Summers, R. "An Overview of Computer Security." *IBM Systems Journal*, Vol. 23, No. 4, 1984.

SYMA01 Symantec Corp. *The Digitial Immune System*, Symantec Technical Brief, 2001.

SYMA07 Symantec. "Security Implications of Microsoft Windows Vista." *Symantec Research Paper*, 2007. symantec.com

SZOR05 Szor, P., *The Art of Computer Virus Research and Defense.* Reading, MA: Addison-Wesley, 2005.

TAVA00 Tavani, H. " Defining the Boundaries of Computer Crime: Piracy, Break-Ins, and Sabotage in Cyberspace." *Computers and Society*, September 2000.

THOM84 Thompson, K. "Reflections on Trusting Trust (Deliberate Software Bugs)." *Communications of the ACM*, August 1984.

TIME90 Time, Inc. *Computer Security, Understanding Computers Series.* Alexandria, VA: Time-Life Books, 1990.

TSUD92 Tsudik, G. "Message Authentication with One-Way Hash Functions." *Proceedings, INFOCOM '92*, May 1992.

TUCH79 Tuchman, W. "Hellman Presents No Shortcut Solutions to DES." *IEEE Spectrum*, July 1979.

TUNG99 Tung, B. *Kerberos: A Network Authentication System.* Reading, MA: Addison-Wesley, 1999.

VACC89 Vaccaro, H., and Liepins, G. "Detection of Anomalous Computer Session Activity." *Proceedings of the IEEE Symposium on Research in Security and Privacy,* May 1989.

VANO94 van Oorschot, P., and Wiener, M. "Parallel Collision Search with Application to Hash Functions and Discrete Logarithms." *Proceedings, Second ACM Conference on Computer and Communications Security*, 1994.

VANO90 van Oorschot, P., and Wiener, M. "A Known-Plaintext Attack on Two-Key Triple Encryption." *Proceedings, EUROCRYPT '90*, 1990; published by Springer-Verlag.

VERN26 Vernam, G "Cipher Printing Telegraph Systems for Secret Wire and Radio Telegraphic Communications." *Journal AIEE*, 1926.

VIJA02 Vijayan, J. "Denial-of-Service Attacks Still a Threat." *ComputerWorld*, April 8, 2002.

VOYD83 Voydock, V., and Kent., S. "Security Mechanisms in High-Level Network Protocols." *Computing Surveys*, June 1983.

WACK02 Wack, J.; Cutler, K.; and Pole, J. *Guidelines on Firewalls and Firewall Policy*. NIST Special Publication SP 800-41, January 2002.

WAGN00 Wagner, D., and Goldberg, I. "Proofs of Security for the UNIX Password Hashing Algorithm." *Proceedings, ASIACRYPT '00*, 2000.

WANG05 Wang, X.; Yin, Y.; and Yu, H. "Finding Collisions in the Full SHA-1." *Proceedings, Crypto '05*, 2005; published by Springer-Verlag.

WARE79 Ware, W., ed. *Security Controls for Computer Systems.* RAND Report 609-1. October 1979. http://www.rand.org/pubs/reports/R609-1/R609.1.html.

WAYN96 Wayner, P. *Disappearing Cryptography.* Boston: AP Professional Books, 1996.

WEAV03 Weaver, N., et al. "A Taxonomy of Computer Worms." *The First ACM Workshop on Rapid Malcode (WORM)*, 2003

WEBS86 Webster, A., and Tavares, S. "On the Design of S-Boxes." *Proceedings, Crypto 85*, 1985; published by Springer-Verlag.

WHIT99 White, S. *Anatomy of a Commercial-Grade Immune System.* IBM Research White Paper, 1999.

WIEN90 Wiener, M. "Cryptanalysis of Short RSA Secret Exponents." *IEEE Transactions on Information Theory,* Vol. IT-36, 1990.

WILS05 Wilson, J. "The Future of the Firewall." *Business Communications Review*, May 2005.

WOO92a Woo, T., and Lam, S. "Authentication for Distributed Systems." *Computer*, January 1992.

WOO92b Woo, T., and Lam, S. " 'Authentication' Revisited." *Computer*, April 1992.

YLON96 Ylonen, T. "SSH - Secure Login Connections over the Internet." *Proceedings, Sixth USENIX Security Symposium*, July 1996.

YUVA79 Yuval, G. "How to Swindle Rabin." *Cryptologia*, July 1979.

ZENG91 Zeng. K.; Yang, C.; Wei, D.; and Rao, T. "Pseudorandom Bit Generators in Stream-Cipher Cryptography." *Computer*, February 1991.

ZOU05 Zou, C., et al. "The Monitoring and Early Detection of Internet Worms." *IEEE/ACM Transactions on Networking*, October 2005.

索　引

A

Abelian Group　阿贝尔群,又称交换群　4.4 节,10.3 节
Access Control　访问控制　1.4 节
Access point (AP)　接入点(AP)　17.1 节
Active Attack　主动攻击　1.3 节
Add key (A_K) function　加密钥(A_K)函数　附录 5B
Administrator　管理员　15.5 节
Advanced Encryption Standard(AES)　高级加密标准　第 5 章
  AddRoundKey transformation　轮密钥加变换　5.2 节
  Equivalent inverse cipher　等价逆密码　5.6 节
  Finite field　有限域　4.7 节,5.1 节
  Irreducible polynomial　既约多项式　5.1 节
  Key expansion algorithm　密钥扩展算法　5.4 节
  MixColumns Transformation　列混淆变换　5.1 节,5.2 节,5.3 节,附录 5A
  ShiftRows Transformation　行移位变换　5.1 节,5.2 节,5.3 节
  S-box　S 盒　5.3 节,5.1 节,附录 5B
  State array　状态阵列　5.1 节,5.2 节
  SubByte(Substitute Byte) Transformation　字节代换变换　5.1 节,5.2 节,5.3 节
AKS (deterministic primality) algorithm　AKS(确定性素性判定)算法　8.3 节
Alert codes　警报代码　16.3 节
Alert Protocol　警报协议　16.2 节,17.4 节
Algorithm　算法　0.1 节,0.2 节,2.1 节,3.4 节,3.6 节,4.1 节,4.3 节,4.6 节,5.4 节,附录 5B,7.1 节,7.2 节,8.3 节,9.2 节,附录 9A,10.1 节,第 11 ~ 12 章,17.4 节,18.2 节,19.3 节
  AES key expansion　AES 密钥扩展　5.4 节
  AKS (deterministic primality) algorithm　AKS(确定性素性判定)算法　8.3 节
  Blowfish　Blowfish 密码　3.6 节
  Blum Blum Shub (BBS) generator　BBS 发生器　7.2 节
  Data authentication　数据认证　12.6 节
  Data encryption　数据加密　3.4 节
  Data integrity　数据完整性　第 11,12,13 章
  Decryption　解密　2.1 节
  Diffie-Hellman key exchange　Diffie-Hellman 密钥交换　10.1 节,10.4 节,16.2 节
Digital signature　数字签名　13.4 节
  Encryption　加密　2.1 节
  Euclidian　欧几里得　4.2 节,4.3 节,4.7 节
  Linear congruential number generator　线性同余数产生器　7.2 节
  Pseudorandom number generation (PRNG)　伪随机数产生　7.1 节,7.2 节
American National Standard　美国国家标准　6.1 节
ANSI X.9.17 PRNG　ANSI X9.17 伪随机数发生器　7.3 节
Anti-replay service　反重放攻击服务　19.3 节
Asymmetric cipher　非对称密码　第 1 章,第 9 章,附录 9B,第 10 章,14.2 节,15.4 节
  Diffie-Hellman key exchange　Diffie-Hellman 密钥交换　10.1 节,10.4 节,16.2 节
  ElGamal cryptographic system　ElGamal 密码体制　10.2 节
  Elliptic Curve Cryptography(ECC)　椭圆曲线密码　10.3 节,10.4 节,10.5 节
  Public key cryptography　公钥密码　第 9 章,第 10 章
  RSA algorithm　RSA 算法　9.2 节
Asymmetric key　非对称密钥　9.1 节
Attack　攻击　第 1 章,2.1 节,3.3 节,6.1 节,9.2 节,11.3 节,12.4 节,13.1 节,15.1 节,15.2 节,15.3 节
  Active attack　主动攻击　1.4 节
  Brute-force attack　强力攻击　2.1 节,9.2 节,11.3 节,12.4 节
  Chosen plaintext attack　选择明文攻击　2.1 节
  Differential cryptanalysis　差分分析　3.5 节
  Linear cryptanalysis　线性分析　3.5 节
  Meet-in-the-middle attack　中间相遇攻击　6.1 节,10.2 节,11.4 节
  Passive attack　被动攻击　1.4 节
  Replay attack　重放攻击　1.3 节,15.1 节,15.2 节
  Timing attack　定时攻击　3.4 节,9.2 节
  Traffic analysis　流量分析　1.4 节
Attribute service　属性服务　15.5 节
Authentication　认证　第 1 章,11.1 节,12.1 节,12.2 节,第 15 章,16.2 节,17.2 节,17.4 节,18.1 节,19.4 节,19.5 节
  Data-origin authentication　数据源认证　1.4 节
  Federated identity management　联合身份管理　15.5 节
  Message Authentication Codes(MAC)　消息认证码　11.1 节,11.3 节,第 12 章
  Mutual authentication　相互认证　15.1 节,15.4 节
  One-way authentication　单向认证　15.1 节,15.2 节,15.4 节

Peer entity authentication　对等实体认证　1.4 节
Timestamp authentication　时间戳认证　15.1 节,18.1 节
Authenticated encryption (AE)　认证加密　12.7 节
Cipher block chaining-message (CBCM) authentication code　密码块链接消息认证码　12.7 节
Counter (CRT) mode　计数器模式　12.7 节
Galois counter-message (GCM) authentication code　Galois 计数器消息认证码　12.7 节
Authentication service (AS)　认证　15.3 节
Autokey system　密钥自动生成系统　2.2 节
Availability of service　服务的可用性　1.1 节
Avalanche effect　雪崩效应　5.5 节

B

Base-64 (radix-64) transter encoding　基 64 转换编码　18.2 节
Basic service set (BSS)　基本服务集　17.1 节
Big-O notation　大 O 符号　附录 9B
Birthday attack　生日攻击　11.3 节,11.4 节,附录 11A
Cipher Block Chaining(CBC) mode　密文分组链接模式　6.3 节
Overlap between two set　两个集合中元素的重复　附录 11A
Paradox　悖论　11.3 节,附录 11A
Bit independence criterion (BIC)　位独立标准　3.6 节
Bit-by-bit exclusive-OR (XOR)　按位异或　11.2 节
Block cipher　分组密码　2.1 节,第 3 章,第 6 章,7.3 节,12.6 节
Cipher-based message authentication code (CMAC)　基于密码的消息认证码　12.6 节
Cipher Block Chaining(CBC) mode　密文分组链接模式　6.3 节
Cipher Feedback(CFB) mode　密码反馈模式　6.4 节
Conversion to stream cipher (modes)　转换成序列密码模式　6.4 节,6.5 节,6.6 节
Counter(CTR) mode　计数器模式　6.4 节,6.6 节,7.3 节
Electronic code book (ECB) mode　电码本模式　6.2 节
Output Feedback (OFB) mode　输出反馈模式　6.4 节,6.5 节,7.3 节
Blowfish　Blowfish 密码　3.6 节
Blum Blum Shub (BBS) generator　BBS 发生器　7.2 节
Brute-force attack　强力攻击　2.1 节,2.2 节,9.2 节,11.3 节,12.4 节
Birthday paradox　生日悖论　11.3 节
Collision resistant　抗碰撞　11.3 节
Message Authentication Codes(MAC)　消息认证码　11.1 节,11.3 节,第 12 章

C

Caesar cipher　恺撒密码　2.2 节
Canonical form　规范格式　18.2 节
Certificate　证书　第 9 章,14.3 节,14.4 节,14.5 节,16.2 节,16.3 节,18.2 节
Authority (CA)　签证中心　14.4 节,14.5 节,18.2 节
Registration authority (RA)　注册中心　14.5 节
Revocation list (CRL)　证书撤销列表　14.4 节,14.5 节,18.2 节
Certificates-only message　仅含证书消息　18.2 节
Change Cipher Spec Protocol　修改密码规范协议　16.2 节,17.4 节
Channel　通道　16.5 节
Chinese remainder theorem　中国剩余定理　8.4 节
Chosen ciphertext attack　选择密文攻击(CCA)　2.1 节,9.2 节
Chosen plaintext attack　选择明文攻击(CPA)　2.1 节
Cipher-Based Message Authentication Code(CMAC)　基于密码的消息认证码 12.6 节
Cipher Block Chaining-Message (CBCM) Authentication code　密码块链接消息认证码　12.7 节
Cipher Block Chaining(CBC) mode　密码块链接模式　6.3 节,11.3 节,11.4 节
Cipher feedback (CFB) mode　密码反馈模式　6.4 节
Cipher Suit　密码套件　16.3 节
Cipher　密码　2.1 节,2.2 节,2.3 节,第 3 章,5.6 节,第 6 章,第 7 章
Caesar　恺撒密码　2.2 节
Hill　Hill 密码　2.2 节
Monoalphabetic　单表代替密码　2.2 节
Playfair　Playfair 密码　2.2 节
Rail fence　栅栏密码　2.3 节
Vernam　Vernam 密码　2.2 节
Vigenère　vigenère 密码　2.2 节
Ciphertext　密文　2.1 节,2.2 节,9.1 节
Ciphertext-stealing technique　密文挪用技术　6.7 节
Clear signing　透明签名　18.2 节
Client/Server authentication　客户机/服务器认证　15.3 节
Collision　碰撞　11.3 节
Commutative ring　交换环　4.4 节
Compression　压缩　11.4 节,16.2 节,18.1 节
Computation resistance　抗计算性　9.1 节
Confidentiality　机密性　1.1 节,1.4 节,18.1 节,19.3 节,19.4 节
Connection　连接　16.2 节
Connection confidentiality and integrity　连接机密性和完整性　1.4 节
Connection Protocol　连接协议　16.5 节
Connectionless confidentiality and integrity　无连接机

密性和完整性 1.4 节
Constant Polynomial 常数多项式 4.6 节
Cookie exchange Cookie 交换 19.5 节
Counter (CTR) mode 计数器模式 6.4 节,6.6 节,7.3 节,12.7 节
Cipher block chaining-message (GCM) authentication code 密码块链接消息认证码 12.7 节
Galois counter-message (CCM) authentication code Galois 计数器消息认证码 12.7 节
Cryptanalysis 密码分析学 2.1 节,3.5 节,9.2 节,11.4 节,
Differential 差分分析 3.5 节
Linear 线性分析 3.5 节
Cryptographic algorithm 密码算法 0.2 节
Cryptographic checksum 密码校验和 9.1 节
Cryptographic computation 密码计算 16.2 节
Cryptographically secure pseudorandom bit generator (CSPRBG) 密码安全伪随机位产生器 7.2 节
Cryptography 密码编码学 第2章,第9章,第10章
Cryptology 密码学 第2章
Cyclic group 循环群 4.4 节

D

Data authentication algorithm (DAA) 数据认证算法 12.6 节
Data authentication code (DAC) 数据认证码 12.6 节
Data consumer 数据消费者 15.5 节
Data Encryption Algorithm (DEA) 数据加密算法 3.2 节,3.4 节
Data Encryption Standard (DES) 数据加密标准 第3章
Double DES (2DES) 2 重 DES 6.1 节
F function *F* 函数 3.6 节
Initial permutation 初始置换 3.2 节
S-box S 盒 3.2 节,3.6 节
Triple DES (3DES) 3 重 DES 6.1 节
Data integrity algorithm (DIA) 数据完整性算法 第1章,第11章,第13章
Data-origin authentication 数据源认证 1.4 节
Decentralized key control 分散式密钥控制 14.1 节
Decryption 解密 2.1 节,3.2 节,5.2 节,5.5 节,附录5B,9.1 节
Denial of service 拒绝服务 1.3 节
Deskewing Algorithm 消偏算法 7.6 节
Determinant 行列式 2.2 节
Deterministic primality (AKS) Algorithm 确定性素性判定算法(AKS) 8.3 节
Differential cryptanalysis 差分分析 3.5 节
Diffie-Hellman key exchange Diffie-Hellman 密钥交换 10.1 节,10.4 节,16.2 节
Elliptic curve cryptography (ECC) analog 用椭圆曲线密码实现 Diffie-Hellman 密钥交换 10.4 节
Man-in-the-middle attack 中间人攻击 10.2 节
SSL Handshake protocol SSL 握手协议 16.2 节
Digital Signature 数字签名 第13章,16.6 节
Digital Signatrue Algorithm (DSA) 数字签名算法 13.4 节
Digital Signatrue Standard (DSS) 数字签名标准 13.4 节
Schnorr scheme Schnorr 签名方案 13.3 节
Digram 双字母对 2.2 节
Direct Digital Signature 直接数字签名 13.1 节
Discrete Logarithm 离散对数 8.5 节
Distribution system 分布式系统 17.1 节
Divisibility 可除性 4.1 节
Division algorithm 除法算法 4.1 节
Divisor 除数 4.1 节,4.6 节
DomainKeys Identified Mail (DKIM) 域名密钥识别邮件 18.3 节
Double DES (2DES) 2 重 DES 6.1 节

E

Electronic Code Book (ECB) mode 电码本模式 6.2 节
Electronic Mail Security 电子邮件安全 第18、19章
DomainKeys identified Mail (DKIM) 域名密钥识别邮件 18.3 节
Pretty Good Privacy (PGP) PGP 邮件系统 18.1 节,18.2 节
Secure/Multipurpose Internet Mail Extension (S/MIME) 安全/多用途 Internet 邮件扩展 18.2 节
ElGamal encryption ElGamal 加密 10.2 节,13.2 节
Elliptic Curve Cryptography (ECC) 椭圆曲线密码 10.3 节,10.6 节
Binary curve over GF(2^m) GF(2^m)上的二元椭圆曲线 10.3 节,10.7 节
Prime curve over Z_p Z_p 上的椭圆曲线 10.3 节
Encapsulating Security Payload (ESP) 载荷安全封装 19.3 节
Encryption 加密 第2章,第3章,5.3 节,附录5B,6.7 节,9.1 节,12.2 节,13.2 节,14.1 节
Encryption/Decryption system 加/解密系统 10.4 节
End Entity 端实体 14.5 节
End to End Encryption 端对端加密 14.1 节
Entropy source 熵源 7.1 节,7.6 节
Enveloped Data 封装数据 18.2 节
Equivalent inverse cipher 等价逆密码 5.6 节
Error control 差错控制 12.2 节
Euclidean Algorithm 欧几里得算法 4.2 节,4.3 节,4.7 节
Extended Euclidean Algorithm 扩展欧几里得算法 4.3 节,4.7 节
Euler's Theorem 欧拉定理 8.3 节

Euler's Totient Function　欧拉函数　8.2 节
Extended Basic Service Set (EBSS)　基本服务集　17.1 节
External Functionality Interface (EFI)　外部功能性接口　17.3 节

F
F function　*F* 函数　3.6 节
Federal Information Processing Standard (FIPS)　联邦信息处理标准　0.4 节
Federated identity management　联合身份管理　15.5 节
Feistel Cipher　Feistel 密码　3.1 节
Fermat's Theorem　费马定理　8.2 节
Finite Field　有限域　第 4 章
Forgery　伪造　13.1 节
Fortezza key exchange　Fortezza 密钥交换　16.2 节
Fragmentation　拆分　16.2 节
Frame Check Sequence (FCS)　帧校验序列　12.2 节
Frequency　频率　7.1 节

G
Galois Counter-Message (GCM) authentication code　伽罗瓦计数器消息认证码　12.7 节
Generator　产生元　4.4 节,4.7 节
Greatest common divisor (gcd)　最大公因子　4.2 节,4.6 节
Group Master Key (GMK)　群主密钥　17.2 节
Group(G)　群　4.4 节
Guaranteed Avalanche(GA)　保证雪崩　3.6 节

H
Handshake Protocol　握手协议　16.2 节,17.4 节
Hash Function　Hash 函数　第 11 章,12.8 节
  Birthday attack　生日攻击　11.3 节,附录 11A
  Brute-force attack　强力攻击　11.3 节
  Compression function　压缩函数　11.3 节
  Hash value　Hash 值　第 11 章
  Preimage　原像　11.3 节
  Secure Hash Algorithm (SHA)　安全 Hash 算法　15.5 节
Hill Cipher　希尔密码　2.2 节
HMAC　基于 Hash 函数的消息认证码　12.5 节
Host key　主机密钥　16.5 节
HTTP　HTTP 协议　16.4 节

I
Identity federation　身份联合　15.5 节
Identity management　身份管理　15.5 节
IEEE 802.11 LAN　IEEE 802.11 局域网　17.1 节
  Association-related service　相互关联服务　17.1 节
  Protocol architecture　协议结构　17.1 节
IEEE 802.11i LAN　IEEE 802.11i 局域网　17.2 节,17.3 节
  Authentication phase　认证阶段　17.2 节
  Discovery phase　发现阶段　17.2 节
  Key management phase　密钥管理阶段　17.2 节
  Phases of operation　操作阶段　17.2 节
  Proteted data transfer phase　保护数据传输阶段　17.2 节
  Robust Security Network (RSN)　强安全网络　17.2 节
Independent basic service　独立基本服务　17.2 节
Indeterminate Variable　不确定性变量　4.6 节
Information Access Threat　信息访问威胁　1.7 节
Information Exchange　信息交换　16.6 节
Initial Permutation (IP)初始置换　3.2 节
Integrity　完整性　1.1 节,1.4 节
International Organization for Standardization (ISO)　国际标准化组织　0.4 节
International-Telecommunication Union(ITU)　国际电信联盟　0.4 节
Internet Architecture Board(IAB)　互联网框架委员会　0.4 节,19.1 节
Internet Engineering Task Force(IETF)　互联网工程任务组　0.4 节
Internet Key Exchange(IKE)　互联网密钥交换　19.5 节
  Header and payload format　头部和有效载荷格式　19.5 节
  Key determination protocol　密钥确定协议　19.5 节
Internet Protocol(IP)　互联网协议　16.1 节,第 19 章
  Combining security association (SA)　组合安全关联　19.4 节
  Cryptographic suites　密码套件　19.6 节
  Encapsulating security payload (ESP)　封装安全有效载荷　19.3 节
  Security associations database (SAD)　安全关联数据库　19.2 节
  Security policy database (SPD)　安全策略数据库　19.2 节
Internet Protocol security(IPsec)　互联网协议安全　19.1 节,19.2 节
  Packet　包　19.2 节
  Routing　路由　19.1 节
  Transport mode　传输模式　19.1 节
  Tunnel mode　隧道模式　19.2 节
Internet Security　互联网安全　第 0 章,附录 15A,第 18 章,第 19 章
  Electronic mail　电子邮件　17.4 节,第 18 章,第 19 章
  Transport Layer Security (TLS)　传输层安全　16.1 节
Internet Security Association and Key Management Protocol(ISAKMP)　互联网安全关联和密钥管理协议　19.5 节
Internet Society (ISOC)　互联网协会　0.4 节
Intrusion Detection　入侵检测　12.2 节,20.2 节

Irreducible polynomial 既约多项式 4.6 节,5.1 节
ITU Telecommunication Standardization Sector (ITU-T) 国际电信联盟电信标准化部 0.4 节,1.3 节

K

Kerberos 一种应用于分布式环境下的认证服务 15.3 节,附录 15A
Authentication dialogue 认证会话 15.3 节
Authentication server (AS) 认证服务 15.3 节
Client/server authentication 客户机/服务器认证 15.3 节
Environmental shortcoming 环境缺陷 15.3 节
Password-to-key transformation 口令到密钥的转换 附录 15A
Propagating Cipher Block Chaning (PCBC) 传播密码块链接 15.3 节,附录 15A
Realm 域 附录 15A
Ticket flag 票据标志 15.3 节
Ticket-Granting Server (TGS) 票据授权服务器 15.3 节
Key distribution 密钥分配 第 14 章,17.2 节,18.1 节,18.2 节
Decentralized Key Control 分散式密钥控制 14.1 节
Hierarchical Key Control 分层密钥控制 14.1 节,17.2 节
Key identifier 密钥标志符 18.1 节
Key ring 密钥环 18.1 节
Key usage control 密钥应用控制 14.1 节
Private key 私钥 18.1 节
Public key 公钥 14.3 节,18.1 节
Public-Key Infrastructure (PKI) 公钥基础设施 14.5 节
Secret key 秘密钥 14.2 节
Session key 会话密钥 14.1 节,18.1 节
Transparent key control 透明密钥控制 14.1 节
X.509 certificate X.509 证书 14.4 节
Key Exchange(KE) 密钥交换 9.1 节,10.1 节,10.4 节,16.2 节,16.5 节,17.4 节,19.5 节
Diffie-Hellman Key Exchange Diffie-Hellman 密钥交换 10.1 节,10.4 节,16.2 节
Fortezza key exchange Fortezza 密钥交换 16.2 节
Internet (IKE) key determination protocol 互联网密钥确定协议 19.5 节
Key Expansion algorithm 密钥扩展算法 5.4 节,附录 5B
Key Generation 密钥产生 3.2 节,9.2 节,16.5 节,17.4 节,18.1 节
Key Identifier 密钥标志 18.1 节
Key Management 密钥管理 参见 Key distribution
Key ring 密钥环 18.1 节
Key schedule algorithm 密钥扩展算法 3.6 节

L

Linear algorithm 线性算法 附录 9B
Linear congruential number generator 线性同余数产生器 7.2 节
Linear cryptanalysis 线性密码分析 3.5 节
Logic Link Control (LLC) layer 逻辑链路控制层 17.1 节

M

MAC Protocol Data Unit (MPDU) MAC 协议数据单元 17.1 节
MAC Service Data Unit (MSDU) MAC 服务数据单元 17.1 节
Masquerade attack 伪装攻击 1.3 节
Master Key 主密钥 14.1 节,17.4 节
Master secret creation 主密钥创建 16.2 节
Master Session Key (MSK) 主会话密钥 17.2 节
Mathematical attack 基于数学的攻击 9.2 节
Maurer's universal statistical test Maurer 通用统计测试 7.1 节
Media access control (MAC) layer 媒体访问控制层 17.1 节
Meet-in-the-middle attack 中间相遇攻击 6.1 节,10.2 节,11.4 节
Message 消息 1.3 节,2.3 节,12.2 节,16.5 节,18.1 节,18.2 节,19.5 节
Frame check sequence (FCS) 帧校验序列 12.2 节
Internal and external error control 内外部差错控制 12.2 节
Message authentication 消息认证 11.1 节,12.1 节,12.2 节
Message Authentication Codes(MAC) 消息认证码 11.1 节,11.3 节,第 12 章
Authenticated encryption (AE) 认证加密 9.2 节
Brute-force Attack 强力攻击 12.4 节
Cryptographic checksum method 密码校验和方法 9.1 节
Data authentication algorithm (DAA) 数据认证算法 12.6 节
Data authentication code (DAC) 数据认证码 12.6 节
Message Digest 消息摘要 11.1 节
Miller-Rabin algorithm Miller-Rabin 算法 3.3 节
Mix column (MC) function 混合列函数 附录 5B
MixColumn transformation 列混合变换 5.2 节,5.3 节,附录 5B
Modular arithmetic 模算术 4.3 节,4.7 节,8.5 节,9.2 节
Congruence (mod n) 模 n 同余 4.3 节,8.5 节
Discrete logarithm 离散对数 8.5 节
Euclidean Algorithm 欧几里得算法 4.2 节,

4.3 节,4.7 节
Exponentiation　幂运算　9.2 节
Residue class　剩余类　4.3 节
Monic polynomial　首 1 多项式　4.6 节
Monoalphabetic cipher　单表代换密码　2.2 节
Multiplicative inverse　乘法逆　4.4 节,4.5 节,4.7 节
Multipurpose Internet Mail Extension(MIME)　多用途互联网邮件扩充　18.2 节
Mutual authentication　相互认证　15.1 节,15.4 节
Mutual trust　相互信任　第 14 章

N

National Institute of Standards and Technology(NIST)　美国国家标准和技术研究所　0.4 节,3.2 节,6.7 节,11.5 节
National Security Agency (NSA)　美国国家安全局　9.1 节
Network access security model　网络访问安全模型　1.6 节
Secure Socket Layer (SSL)　安全套接字层　第 16 章
Secure Shell (SSH)　安全外壳协议　16.5 节
Next-bit test　相邻位测试　7.2 节
Nibble substitution (NS) function　半字节代替函数　附录 5B
Nonce　临时交互号　6.3 节,19.5 节
Nonrepudiation　不可否认性　1.4 节
Number Theory　数论　第 4 章
Chinese remainder theorem　中国剩余定理　8.4 节
Discrete logarithm　离散对数　8.5 节
Divisibility　可除性　4.1 节
Euclidian Algorithm　欧几里得算法　4.2 节,4.3 节,4.7 节

O

Oakley Key Determination Protocol　Oakley 密钥确定协议　19.4 节
One-bit rotation hash function　一位循环移位 Hash 函数　11.2 节
One-Time Pad　一次一密　2.3 节
One-way authentication　单向认证　15.1 节,15.2 节,15.4 节
One-way encryption function　单向加密函数　9.1 节
One-way password file　单向口令文件　11.1 节
Open System Interconnection (OSI)　开放系统互连　1.2 节,1.3 节,1.4 节,1.5 节
Optimal Asymmetric Encryption Padding(AOEP)　最佳非对称加密填充　9.2 节
Output Feedback (OFB) Mode　输出反馈模式　6.4 节,6.5 节,7.3 节

P

Packet　包　19.2 节
Packet exchange　包交换　16.5 节
Padding　填充　16.3 节,19.3 节
Pairwise Master Key (PMK)　对主密钥　17.2 节
Pairwise Transient Key (P TK)　对临时密钥　17.2 节
Passive attack　被动攻击　1.3 节
Password　口令　15.1 节,15.3 节,15.5 节,附录 15A
Keberos password-to-key transformation　Keberos 系统口令到密钥的转换　附录 15A
Path constraint　路径约束　14.5 节
Peer entity authentication　对等实体认证　1.4 节
Permutation　置换　2.2 节,3.1 节,3.6 节,4.4 节
Physical layer　物理层　17.1 节
Plaintext　明文　2.1 节,2.2 节,2.3 节
Asymmetric encryption　非对称加密　9.1 节
Substitution encryption　代替加密　2.2 节
Symmetric encryption　对称加密　2.1 节,2.2 节,2.3 节
Transposition technique　移位技术　2.3 节
Playfair cipher　Playfair 密码　2.2 节
Polyalphabetic cipher　多表代替加密　2.2 节
Polynomial algorithm　多项式算法　附录 9B
Polynomial Arithmetic　多项式算术　4.6 节,4.7 节,附录 5A
Euclidean Algorithm　欧几里得算法　4.2 节,4.3 节,4.7 节
Finite field　有限域　4.7 节,5.1 节
Greatest common divisor (gcd)　最大公因子　4.2 节,4.6 节
Modular　模　4.7 节
Multiplication by x　用 x 乘　附录 5A
Polynomial ring　多项式环　4.6 节
Port forwarding　端口转发　16.5 节
Preimage　原像　11.3 节
Preoutput　预输出　3.2 节
Pre-Shared Key (PSK)　预共享密钥　17.2 节
Pretty Good Privacy (PGP)　PGP 邮件系统　18.1 节,18.2 节
Prime Number　素数　8.1 节,8.2 节,8.3 节
AKS (deterministic primality) Algorithm　AKS(确定性素性判定)算法　8.3 节
Euler's theorem　欧拉定理　8.2 节
Euler's totient function $\phi(n)$　欧拉函数 $\phi(n)$　8.2 节
Fermat's theorem　费马定理　8.2 节
Fundamental theorem of arithmetic　算术基本定理　8.1 节
Miller-Rabin algorithm　Miller-Rabin 算法　8.3 节
Prime polynomial　素多项式　4.6 节
Private Key　私钥　9.1 节,9.2 节,18.1 节
Propagating Cipher Block Chaning (PCBC)　传播密码块链接　15.3 节,附录 15A
Protected data transfer phase　保护数据传送阶段　17.2 节

Pseudorandom Function (PRF) 伪随机函数 7.1节,11.1节,12.8节
Pseudorandom number generation (PRNG) 伪随机数产生 第7章,10.5节,11.1节,12.8节
Blum Blum Shab (BBS) generator BBS发生器 7.2节
Linear congruential generator 线性同余产生器 7.2节
Randomness criteria 随机性标准 7.1节
Trust random number generator (TRNG) 真随机数产生 7.1节,7.6节
Unpredictability criteria 不可预测性标准 7.1节
Pseudorandomness 伪随机性 11.3节
Public Key 公钥 9.1节,9.2节,14.3节,14.5节,18.1节
Authority 授权 14.3节
Certificate 证书 第9章,14.3节,14.5节
Key Exchange 密钥交换 9.1节
Private key 私钥 18.1节
Public-key infrastructure (PKI) 公钥基础设施 15.5节
Public Key Cryptography 公钥密码学 第9章,第10章
Cryptanalysis 密码分析学 9.2节
Cryptography 密码编码学 第9章,第10章
Cryptosystem 密码体制 9.1节
Digital Signature 数字签名 第13章,16.6节
ElGamal encryption ElGamal加密 10.2节,13.2节
Elliptic Curve Cryptography(ECC) 椭圆曲线密码 10.3节,10.6节
RSA algorithm RSA算法 9.2节
Public-Key Infrastructure (PKI) 公钥基础设施 15.5节

Q

Quoted-printable transfer encoding 引用可打印转换编码 18.2节

R

Radix-64 conversion 基数64转换 附录18A
Rail fence cipher 栅栏密码 2.3节
Random number 随机数 7.1节
Randomness criteria 随机性标准 7.1节
RC4 stream cipher RC4密码 7.5节
Record Protocol 记录协议 16.2节
Rduced sign-on (RSO) 简化登录 15.5节
Registration Authority(RA) 注册机构 14.6节
Relactively prime integer 互素整数 4.2节,4.3节,4.5节
Release of message content 消息内容泄漏 1.3节
Remote user authentication 远程用户认证 15.1节,15.2节,15.4节
Replay attack 重放攻击 1.3节,15.1节,19.3节
Repository 证书存取库 14.6节
Residue 剩余 4.1节,5.6节
Rivest-Shamir-Adleman(RSA) algorithm RSA算法 9.2节,附录9A
Rotor machines 轮转机 2.4节
Rounds 轮 3.2节,3.6节

S

S-AES 简化AES 附录5B
S-box S盒 3.2节,3.5节,5.3节,附录5B
Sage project and example Sage项目和例题 附录A,附录B
Schnorr digital signature Schnorr数字签名 13.3节
Secret Key 秘密钥 2.1节,9.1节,14.2节
Secure Hash Algorithm (SHA) 安全Hash算法 11.5节
SHA-512 algorithm SHA-512算法 11.5节
SHA-3 algorithm SHA-3算法 11.6节
Secure Shell (SSH) 安全外壳协议 16.5节
Channel 通道 16.5节
Connection protocol 连接协议 16.5节
Host key 主机密钥 16.5节
Port forwarding 端口转发 16.5节
Transport Layer Protocol 传输层协议 16.5节
User authentication protocol 用户认证协议 16.5节
Secure Socket Layer(SSL) 安全套接层 16.1节,16.2节
Alert protocol 报警协议 16.2节
Change Cipher Spec Protocol 修改密码规范协议 16.2节
Handshake protocol 握手协议 16.2节
Hypertext Transfer Protocol (HTTP) 超文本协议 16.2节
Record protocol 记录协议 16.2节
Secure/ Multipurpose Internet Mail Extension(S/MIME) 安全/多用途互联网邮件扩充 18.2节
Certificate processing 证书处理 18.2节
Clear signing 透明签名 18.2节
Multipurpose Internet Mail Extension (MIME) 多用途Internet邮件扩展 18.2节
Security 安全性 第0章,第1章,9.2节,12.4节,12.5节
Attack 攻击 1.3节
Availability 可用性 1.1节,1.4节
Confidentiality 秘密性 1.1节,1.4节
Integrity 完整性 1.1节,1.4节
Open System Interconnection (OSI) security arthitecture 开放式互连安全结构 1.2节
Security Assertion Markup Language (SAMI.) 安全声明标记语言 15.5节
Security Association(SA) 安全关联 19.2节,19.4节

Security Association Database(SAD) 安全关联数据库 19.2 节
Security Policy Database(SPD) 安全策略数据库 19.2 节
Seed 种子 7.1 节
Selective-field confidentiality and integrity 选择域秘密性和完整性 1.4 节
Service threat 服务威胁 1.7 节
Session 会话 16.2 节
Session key 会话密钥 14.1 节,15.3 节,18.1 节
Session Security Module (SSM) 会话安全模块 14.1 节
Shift row (SR) function 行移位函数 附录 5B
ShiftRow transformation 行移位变换 5.2 节,5.3 节
Simplified Advanced Encryption Standard(S-AES) 简化 AES 附录 5B
  Add key 密钥加 附录 5B
  Key expansion algorithm 密钥扩展算法 附录 5B
  Mix column(MC)function 列混合函数 附录 5B
  Shift row (SR) function 行移位函数 附录 5B
  S-box construction S 盒构造 附录 5B
Single Sign-On (SSO) 单点登录 15.5 节
Skew 偏差 7.6 节
State Array 状态数组 5.2 节
State vector (S) initialization 状态向量初始化 7.5 节
Steganography 隐写术 2.5 节
Stream Cipher 流密码 2.1 节,3.1 节,6.4 节,6.5 节,6.6 节
  Cipher Feedback(CFB)mode 密码反馈模式 6.4 节
  Counter(CTR)mode 计数器模式 6.4 节,6.6 节,7.3 节
  Output Feedback (OFB) mode 输出反馈模式 6.4 节,6.5 节,7.3 节
Strict Avalanche Criterion(SAC) 严格雪崩效应准则 3.6 节
SubBytes (substitue byte) transformation 字节代换变换 5.2 节,5.3 节
Substitution cipher technique 代换密码技术 2.2 节,3.1 节
  Auto key system 自动密钥系统 2.2 节
  Caesar cipher 恺撒密码 2.2 节
  Feistel cipher Feistel 密码 3.1 节
  Hill cipher Hill 密码 2.2 节
  Monoalphabetic cipher 单表代换密码 2.2 节
  One-time pad 一次以密 2.3 节
  Playfair cipher Playfair 密码 2.2 节
  Polyalphabetic cipher 多表代替加密 2.2 节
Substitution/Permutation Network(SPN) 代替/置换网络 3.1 节
Suppress-replay attack 压制重放攻击 15.2 节
Symmetric Cipher 对称密码 2.1 节,第 3、4、5、6、7 章,12.2 节,14.1 节
  Advanced Encryption Standard(AES) 高级加密标准 第 5 章
  Block cipher 分组密码 2.1 节,第 3 章,第 6 章,7.3 节,12.6 节
  Data Encryption Standard(DES) 数据加密标准 第 3 章
  Stream cipher 流密码 2.1 节,3.1 节,6.4 节,6.5 节,6.6 节
Symmetric key distribution 对称密钥分配 14.1 节,14.2 节
  Key Distribution Center (KDC) 密钥分配中心 14.1 节,14.2 节
  Transparent key control 透明密钥控制 14.1 节

T

Threats 威胁 1.2 节,1.7 节,18.3 节
Ticket 票据 15.3 节
Ticket flag 票据标志 15.3 节
Ticket-Granting Server (TGS) 票据授权服务器 15.3 节
Time complexity of algorithm 算法的时间复杂性 附录 9B
Timestamp authentication 时间戳认证 15.1 节,18.1 节
Timing attack 定时攻击 3.4 节,9.2 节
Traffic Analysis 流量分析 1.3 节
Traffic Flow Confidentiality (TFC) 传输流秘密性 1.4 节,19.3 节
Transformation function 变换函数 5.3 节,附录 5A
  AddRoudKey 轮密钥加 5.2 节,3.3 节
  MixColumn transformation 列混合变换 5.2 节,3.3 节,附录 5A
  ShiftRow transformation 行移位变换 5.2 节,3.3 节
  SubByte transformation 字节代替变换 5.2 节,3.3 节
Transparent Key Control 透明的密钥控制 14.1 节
Transport Layer Protocol 传输层协议 16.5 节
Transport Layer Security (TLS) 传输层安全 16.1 节,16.3 节
  Alert code 报警码 16.3 节
  Certificate type 证书类型 16.3 节
  Cipher suite 密码套件 16.3 节
  Message Authentication Code(MAC) 消息认证码 11.1 节
  PseudoRandom Function (PRF) 位随机函数 16.3 节
Transport-level security 传输层安全 第 16 章
  Secure Shell (SSH) 安全外壳协议 16.5 节
  Transport Layer Security (TLS) 传输层安全 16.1 节,16.3 节
  Web consideration Web 安全性思考 16.1 节
Transport Mode 传输模式 19.1 节,19.3 节,19.4 节

Transposition cipher technique 置换技术 2.3 节
Trap-door one-way function 单向陷门函数 9.2 节
Triple DES 三重 DES 3.2 节,6.1 节
True Random Number Generator(TRNG) 真随机数产生器 7.1 节,7.6 节
Trust 信任 18.1 节
Tunnel mode 隧道模式 14.1 节,19.3 节,19.4 节

U

Uniform distribution 分布均匀性 7.1 节
Unpredictability criteria 不可预测性标准 7.1 节
User authentication protocol 用户认证协议 16.5 节
User authentication 用户认证 第15章
  Federated identity management 联合身份管理 15.5 节
  Replay attack 重放攻击 1.3 节,15.1 节,19.3 节
User certificate 用户证书 14.4 节

V

VeriSign Certificate VeriSign 证书 18.2 节
Vernam cipher Vernam 密码 2.2 节
Version number 版本号 16.3 节
Vigenére cipher Vigenére 密码 2.2 节
Virus detection 病毒检测 11.2 节

W

Web resource 网站资源 0.3 节
Web Security 网站安全 16.1 节,16.4 节
Wi-Fi protected Access (WPA) Wi-Fi 受保护访问 17.1 节,17.2 节
Wireless application environment (WAE) 无线应用环境 17.3 节
Wireless application protocol (WAP) 无线应用协议 17.3 节,17.5 节
  End-to-end security 端到端安全 17.5 节
  Programming model 规划模型 17.3 节
  Wireless Markup Language (WML) 无线网络标记语言 17.3 节
  Wireless Session Protocol (WSP) 无线会话协议 17.3 节
  Wireless Transaction Protocol (WTP) 无线处理协议 17.3 节
Wireless Ethernet Compatibility Alliance (WECA) 无线以太网兼容性联盟 17.1 节
Wireless Markup Language (WML) 无线网络标记语言 17.3 节
Wireless netword security 无线网络安全 第17章
  IEEE 802.11 LAN IEEE 802.11 局域网 17.1 节
  IEEE 802.11i LAN IEEE 802.11i 局域网 17.2 节
  Robust Security Network(RSN) 强安全网络 17.2 节
  Wi-Fi Protected Access (WPA) Wi-Fi 受保护访问 17.1 节,17.2 节
  Wired Equivalent Privacy (WEP) 有线等效隐私 17.2 节
  Wireless Application Protocol (WAP) 无线应用协议 17.3 节,17.5 节
  Wireless Transport Layer Security (WTLS) 无线传输层安全 17.1 节,17.4 节
Wireless Session Protocol (WSP) 无线会话协议 17.3 节
Wireless Transaction Protocol (WTP) 无线处理协议 17.3 节
Wireless Transport Layer Security (WTLS) 无线传输层安全 17.1 节,17.4 节
  Alert Protocol 警报协议 16.2 节,17.4 节
  Authentication 认证 17.4 节
  Change Cipher Spec Protocol 修改密码规范协议 16.2 节,17.4 节
  Handshake Protocol 握手协议 16.2 节,17.4 节
  Key exchange 密钥交换 17.4 节
  PseudoRandom Function (PRF) 伪随机函数 17.4 节
  Record protocol 记录协议 17.4 节

X

X.509 ITU-T recommendation X.509 ITU-T 建议 14.4 节,14.5 节
  Certificate Authority (CA) 签证中心 14.4 节
  Certificate 证书 14.4 节
  Public-Key Infrastructure (PKI) 公钥基础设施 14.5 节
X.800 ITU-T recommendation X.800 ITU-T 建议 1.3 节,1.4 节
  Active and passive attack 主动和被动攻击 1.3 节
  Security mechanism 安全机制 1.5 节
  Security service 安全服务 1.4 节
XTS-AES mode XTS-AES 模式 6.7 节
  Block-oriented storage devices using 面向分组的存储设备应用 6.7 节
  Ciphertext-stealing technique 密文挪用技术 6.7 节
  Sector (data unit) operation 数据块操作 6.7 节
  Storage encryption requirement 存储加密需求 6.7 节